

# SWIG-4.3 Documentation

---

## Table of Contents


**SWIG-4.3 Documentation**

- Sections
  - SWIG Core Documentation
  - Supported Language Modules Documentation
  - Experimental Language Modules Documentation
  - Deprecated Language Modules Documentation
  - Developer Documentation

**1 Preface**

- 1.1 Introduction
- 1.2 SWIG Versions
- 1.3 SWIG License
- 1.4 SWIG resources
- 1.5 Prerequisites
- 1.6 Organization of this manual
- 1.7 How to avoid reading the manual
- 1.8 Backwards compatibility
- 1.9 Release notes
- 1.10 Credits
- 1.11 Bug reports
- 1.12 Installation
  - 1.12.1 Windows installation
  - 1.12.2 Unix installation
  - 1.12.3 Macintosh OS X installation
  - 1.12.4 Testing
  - 1.12.5 Examples

**2 Introduction**

- 2.1 What is SWIG?
- 2.2 Why use SWIG?
- 2.3 Target languages
  - 2.3.1 Supported status
  - 2.3.2 Experimental status
  - 2.3.3 Deprecated status
- 2.4 A SWIG example
  - 2.4.1 SWIG interface file
  - 2.4.2 The swig command
  - 2.4.3 Building a Perl5 module
  - 2.4.4 Building a Python module
  - 2.4.5 Shortcuts
- 2.5 Supported C/C++ language features
- 2.6 Non-intrusive interface building
- 2.7 Incorporating SWIG into a build system
- 2.8 Hands off code generation
- 2.9 SWIG and freedom

**3 Getting started on Windows**

- 3.1 Installation on Windows
  - 3.1.1 Windows Executable
- 3.2 SWIG Windows Examples
  - 3.2.1 Instructions for using the Examples with Visual Studio
    - 3.2.1.1 C#
    - 3.2.1.2 Java
    - 3.2.1.3 Python
    - 3.2.1.4 TCL
  - 3.2.2 Instructions for using the Examples with other compilers
- 3.3 Building swig.exe on Windows
  - 3.3.1 Building swig.exe using CMake
  - 3.3.2 Building swig.exe using MSYS2 and MinGW-w64
  - 3.3.3 Building swig.exe using MinGW and MSYS
  - 3.3.4 Building swig.exe using Cygwin
    - 3.3.4.1 Running the examples on Windows using Cygwin
- 3.4 Microsoft extensions and other Windows quirks
  - 3.4.1 Visual C++ standards compliance
  - 3.4.2 Calling conventions

**4 Scripting Languages**

- 4.1 The two language view of the world
- 4.2 How does a scripting language talk to C?
  - 4.2.1 Wrapper functions
  - 4.2.2 Variable linking
  - 4.2.3 Constants
  - 4.2.4 Structures and classes
  - 4.2.5 Proxy classes
- 4.3 Building scripting language extensions
  - 4.3.1 Shared libraries and dynamic loading
  - 4.3.2 Linking with shared libraries
  - 4.3.3 Static linking

**5 SWIG Basics**

- 5.1 Running SWIG

- 5.1.1 Input format
- 5.1.2 SWIG output
- 5.1.3 Comments
- 5.1.4 C Preprocessor
- 5.1.5 SWIG directives
- 5.1.6 Parser limitations
- 5.1.7 Parse tree

- 5.2 Wrapping simple C declarations
  - 5.2.1 Basic type handling
  - 5.2.2 Global variables
  - 5.2.3 Constants
  - 5.2.4 A brief word about const
  - 5.2.5 A cautionary tale of char *
- 5.3 Pointers and complex objects
  - 5.3.1 Simple pointers
  - 5.3.2 Run time pointer type checking
  - 5.3.3 Derived types, structs, and classes
  - 5.3.4 Undefined datatypes
  - 5.3.5 Typedef
- 5.4 Other Practicalities
  - 5.4.1 Passing structures by value
  - 5.4.2 Return by value
  - 5.4.3 Linking to structure variables
  - 5.4.4 Linking to char *
  - 5.4.5 Arrays
  - 5.4.6 Creating read-only variables
  - 5.4.7 Renaming and ignoring declarations
    - 5.4.7.1 Simple renaming of specific identifiers
    - 5.4.7.2 Ignoring identifiers
    - 5.4.7.3 Advanced renaming support
    - 5.4.7.4 Limiting global renaming rules
    - 5.4.7.5 Ignoring everything then wrapping a few selected symbols
  - 5.4.8 Default/optional arguments
  - 5.4.9 Pointers to functions and callbacks
- 5.5 Structures and unions
  - 5.5.1 Typedef and structures
  - 5.5.2 Character strings and structures
  - 5.5.3 Array members
  - 5.5.4 Structure data members
  - 5.5.5 C constructors and destructors
  - 5.5.6 Adding member functions to C structures
  - 5.5.7 Nested structures
  - 5.5.8 Other things to note about structure wrapping
- 5.6 Code Insertion
  - 5.6.1 The output of SWIG
  - 5.6.2 Code insertion blocks
  - 5.6.3 Inlined code blocks
  - 5.6.4 Initialization blocks
- 5.7 An Interface Building Strategy
  - 5.7.1 Preparing a C program for SWIG
  - 5.7.2 The SWIG interface file
  - 5.7.3 Why use separate interface files?
  - 5.7.4 Getting the right header files
  - 5.7.5 What to do with main()

**6 SWIG and C++**

- 6.1 Comments on C++ Wrapping
- 6.2 Approach
- 6.3 Supported C++ features
- 6.4 Command line options and compilation
- 6.5 Proxy classes
  - 6.5.1 Construction of proxy classes
  - 6.5.2 Resource management in proxies
  - 6.5.3 Language specific details
- 6.6 Simple C++ wrapping
  - 6.6.1 Constructors and destructors
  - 6.6.2 Default constructors, copy constructors and implicit destructors
  - 6.6.3 When constructor wrappers aren't created
  - 6.6.4 Copy constructors
  - 6.6.5 Member functions
  - 6.6.6 Static members
  - 6.6.7 Member data
- 6.7 Protection
- 6.8 Enums and constants
- 6.9 Friends
  - 6.9.1 Friend classes
  - 6.9.2 Friend function definitions
  - 6.9.3 Friend function declarations
  - 6.9.4 Unqualified friend functions
- 6.10 References and pointers
- 6.11 Pass and return by value
- 6.12 Inheritance
- 6.13 A brief discussion of multiple inheritance, pointers, and type checking
- 6.14 Default arguments
- 6.15 Overloaded functions and methods
  - 6.15.1 Dispatch function generation
  - 6.15.2 Ambiguity in overloading
  - 6.15.3 Renaming and ambiguity resolution
  - 6.15.4 Comments on overloading
- 6.16 Overloaded operators
- 6.17 Class extension
  - 6.17.1 Replacing class methods
- 6.18 Templates
  - 6.18.1 The %template directive
  - 6.18.2 Function templates
  - 6.18.3 Default template arguments
  - 6.18.4 Template base classes
  - 6.18.5 Empty template instantiation
  - 6.18.6 Template specialization
  - 6.18.7 Member templates
  - 6.18.8 Scoping and templates

- 6.18.9 More on templates
- 6.19 Namespaces
  - 6.19.1 The nspace feature for namespaces
    - 6.19.1.1 %nspace for mirroring namespace hierarchies
    - 6.19.1.2 %nspacemove for modifying namespace hierarchies
    - 6.19.1.3 More about the nspace feature
- 6.20 Renaming templated types in namespaces
- 6.21 Exception specifications
- 6.22 Exception handling with %catches
- 6.23 Pointers to Members
- 6.24 Smart pointers and operator->()
- 6.25 C++ reference counted objects - ref/unref feature
- 6.26 Using declarations and inheritance
- 6.27 Nested classes
- 6.28 A brief rant about const-correctness
- 6.29 Callbacks to the target language
  - 6.29.1 Introduction to director classes
  - 6.29.2 Using directors and target language callbacks
- 6.30 Where to go for more information

**7 SWIG and C++11**

- 7.1 Introduction
- 7.2 Core language changes
  - 7.2.1 Rvalue reference and move semantics
    - 7.2.1.1 Rvalue reference inputs
    - 7.2.1.2 Rvalue reference outputs
    - 7.2.1.3 Movable and move-only types by value
  - 7.2.2 Generalized constant expressions
  - 7.2.3 Extern template
  - 7.2.4 Initializer lists
  - 7.2.5 Uniform initialization
  - 7.2.6 Type inference
  - 7.2.7 Range-based for-loop
  - 7.2.8 Lambda functions and expressions
  - 7.2.9 Alternate function syntax
  - 7.2.10 Object construction improvement
  - 7.2.11 Explicit overrides and final
  - 7.2.12 Null pointer constant
  - 7.2.13 Strongly typed enumerations
  - 7.2.14 Double angle brackets
  - 7.2.15 Explicit conversion operators
  - 7.2.16 Type alias and alias templates
  - 7.2.17 Unrestricted unions
  - 7.2.18 Variadic templates
  - 7.2.19 New character literals
  - 7.2.20 New string literals
  - 7.2.21 User-defined literals
  - 7.2.22 Thread-local storage
  - 7.2.23 Explicitly defaulted functions and deleted functions
  - 7.2.24 Type long long int
  - 7.2.25 Static assertions
  - 7.2.26 Allow sizeof to work on members of classes without an explicit object
  - 7.2.27 Exception specifications and noexcept
  - 7.2.28 Control and query object alignment
  - 7.2.29 Attributes
  - 7.2.30 Methods with ref-qualifiers
- 7.3 Standard library changes
  - 7.3.1 Threading facilities
  - 7.3.2 Tuple types
  - 7.3.3 Hash tables
  - 7.3.4 Regular expressions
  - 7.3.5 General-purpose smart pointers
  - 7.3.6 Extensible random number facility
  - 7.3.7 Wrapper reference
  - 7.3.8 Polymorphic wrappers for function objects
  - 7.3.9 Type traits for metaprogramming
  - 7.3.10 Uniform method for computing return type of function objects

**8 SWIG and C++14**

- 8.1 Introduction
- 8.2 Core language changes
  - 8.2.1 Binary integer literals
  - 8.2.2 Return type deduction
- 8.3 Standard library changes

**9 SWIG and C++17**

- 9.1 Introduction
- 9.2 Core language changes
  - 9.2.1 Nested namespace definitions
  - 9.2.2 UTF-8 character literals
  - 9.2.3 Hexadecimal floating literals
  - 9.2.4 Fold expressions
- 9.3 Standard library changes

**10 SWIG and C++20**

- 10.1 Introduction
- 10.2 Core language changes
  - 10.2.1 Spaceship operator
  - 10.2.2 Lambda templates
  - 10.2.3 Constexpr destructors
- 10.3 Preprocessor changes
  - 10.3.1 __VA_OPT__()
- 10.4 Standard library changes

**11 Preprocessing**

- 11.1 File inclusion
- 11.2 File imports
- 11.3 Conditional Compilation

- 11.4 Macro Expansion
- 11.5 SWIG Macros
- 11.6 Variadic Macros
- 11.7 Preprocessing and delimiters
  - 11.7.1 Preprocessing and %{ ... %} & " ... " delimiters
  - 11.7.2 Preprocessing and { ... } delimiters
- 11.8 Preprocessor and Typemaps
- 11.9 Viewing preprocessor output
- 11.10 The #error and #warning directives
- 11.11 Trigraphs
- 11.12 Digraphs

**12 SWIG library**

- 12.1 The %include directive and library search path
- 12.2 C arrays and pointers
  - 12.2.1 argcargv.i
  - 12.2.2 cpointer.i
  - 12.2.3 carrays.i
  - 12.2.4 cmalloc.i
  - 12.2.5 cdata.i
- 12.3 C string handling
  - 12.3.1 Default string handling
  - 12.3.2 Passing a string with length
  - 12.3.3 Using %newobject to release memory
  - 12.3.4 cstring.i
- 12.4 C standard library
  - 12.4.1 Complex floating types
- 12.5 STL/C++ library
  - 12.5.1 std::string
  - 12.5.2 std::string_view
  - 12.5.3 std::vector
  - 12.5.4 STL exceptions
  - 12.5.5 shared_ptr smart pointer
    - 12.5.5.1 shared_ptr basics
    - 12.5.5.2 shared_ptr and inheritance
    - 12.5.5.3 shared_ptr and method overloading
    - 12.5.5.4 shared_ptr and templates
    - 12.5.5.5 shared_ptr and directors
  - 12.5.6 unique_ptr smart pointer
    - 12.5.6.1 unique_ptr passed by value
    - 12.5.6.2 unique_ptr passed by reference
  - 12.5.7 auto_ptr smart pointer
- 12.6 Utility Libraries
  - 12.6.1 exception.i
  - 12.6.2 attribute.i
    - 12.6.2.1 %attribute and C++ templates

**13 Argument Handling**

- 13.1 The typemaps.i library
  - 13.1.1 Introduction
  - 13.1.2 Input parameters
  - 13.1.3 Output parameters
  - 13.1.4 Input/Output parameters
  - 13.1.5 Using different names
- 13.2 Applying constraints to input values
  - 13.2.1 Simple constraint example
  - 13.2.2 Constraint methods
  - 13.2.3 Applying constraints to new datatypes

**14 Typemaps**

- 14.1 Introduction
  - 14.1.1 Type conversion
  - 14.1.2 Typemaps
  - 14.1.3 Pattern matching
  - 14.1.4 Reusing typemaps
  - 14.1.5 What can be done with typemaps?
  - 14.1.6 What can't be done with typemaps?
  - 14.1.7 Similarities to Aspect Oriented Programming
  - 14.1.8 The rest of this chapter
- 14.2 Typemap specifications
  - 14.2.1 Defining a typemap
  - 14.2.2 Typemap scope
  - 14.2.3 Copying a typemap
  - 14.2.4 Deleting a typemap
  - 14.2.5 Placement of typemaps
- 14.3 Pattern matching rules
  - 14.3.1 Basic matching rules
  - 14.3.2 Typedef reductions matching
  - 14.3.3 Default typemap matching rules
  - 14.3.4 Multi-arguments typemaps
  - 14.3.5 Matching rules compared to C++ templates
  - 14.3.6 Debugging typemap pattern matching
- 14.4 Code generation rules
  - 14.4.1 Scope
  - 14.4.2 Declaring new local variables
  - 14.4.3 Special variables
    - 14.4.3.1 Type related special variables
    - 14.4.3.2 Non-type related special variables
  - 14.4.4 Special variable macros
    - 14.4.4.1 $descriptor(type)
    - 14.4.4.2 $typemap(method, typepattern)
    - 14.4.4.3 $typemap(method:attribute, typepattern)
  - 14.4.5 Special variables and typemap attributes
  - 14.4.6 Special variables combined with special variable macros
- 14.5 Common typemap methods
  - 14.5.1 "in" typemap
  - 14.5.2 "typecheck" typemap
  - 14.5.3 "out" typemap
  - 14.5.4 "arginit" typemap
  - 14.5.5 "default" typemap

        - 14.5.6 "check" typemap
        - 14.5.7 "argout" typemap
        - 14.5.8 "freearg" typemap
        - 14.5.9 "newfree" typemap
        - 14.5.10 "ret" typemap
        - 14.5.11 "memberin" typemap
        - 14.5.12 "varin" typemap
        - 14.5.13 "varout" typemap
        - 14.5.14 "throws" typemap
    - 14.6 Some typemap examples
        - 14.6.1 Typemaps for arrays
        - 14.6.2 Implementing constraints with typemaps
    - 14.7 Typemaps for multiple target languages
    - 14.8 Optimal code generation when returning by value
    - 14.9 Multi-argument typemaps
    - 14.10 Typemap warnings
    - 14.11 Typemap fragments
        - 14.11.1 Fragment type specialization
        - 14.11.2 Fragments and automatic typemap specialization
    - 14.12 The run-time type checker
        - 14.12.1 Implementation
        - 14.12.2 Usage
    - 14.13 Typemaps and overloading
        - 14.13.1 SWIG_TYPECHECK_POINTER precedence level and the typecheck typemap
    - 14.14 More about %apply and %clear
    - 14.15 Passing data between typemaps
    - 14.16 C++ "this" pointer
    - 14.17 Where to go for more information?

**15 Customization Features**

- 15.1 Exception handling with %exception
    - 15.1.1 Handling exceptions in C code
    - 15.1.2 Exception handling with longjmp()
    - 15.1.3 Handling C++ exceptions
    - 15.1.4 Exception handlers for variables
    - 15.1.5 Defining different exception handlers
    - 15.1.6 Special variables for %exception
    - 15.1.7 Using The SWIG exception library
- 15.2 Object ownership and %newobject
- 15.3 Features and the %feature directive
    - 15.3.1 Feature attributes
    - 15.3.2 Feature flags
    - 15.3.3 Clearing features
    - 15.3.4 Features and default arguments
    - 15.3.5 Feature example

**16 Contracts**

- 16.1 The %contract directive
- 16.2 %contract and classes
- 16.3 Constant aggregation and %aggregate_check
- 16.4 Notes

**17 Variable Length Arguments**

- 17.1 Introduction
- 17.2 The Problem
- 17.3 Default varargs support
- 17.4 Argument replacement using %varargs
- 17.5 Varargs and typemaps
- 17.6 Varargs wrapping with libffi
- 17.7 Wrapping of va_list
- 17.8 C++ Issues
- 17.9 Discussion

**18 SWIG and Doxygen Translation**

- 18.1 Doxygen translation overview
- 18.2 Preparations
    - 18.2.1 Enabling Doxygen translation
    - 18.2.2 Doxygen-specific %feature directives
        - 18.2.2.1 doxygen:notranslate
        - 18.2.2.2 doxygen:alias:<command-name>
        - 18.2.2.3 doxygen:ignore:<command-name>
        - 18.2.2.4 doxygen:nolinktranslate
        - 18.2.2.5 doxygen:nostripparams
    - 18.2.3 Additional command line options
- 18.3 Doxygen to Javadoc
    - 18.3.1 Basic Javadoc example
    - 18.3.2 Javadoc tags
    - 18.3.3 Unsupported tags for Javadoc
- 18.4 Doxygen to Pydoc
    - 18.4.1 Basic Pydoc example
    - 18.4.2 Pydoc translator
    - 18.4.3 Unsupported tags for Pydoc
- 18.5 Doxygen to XML C# documentation
    - 18.5.1 Basic C# example
    - 18.5.2 C# tags
    - 18.5.3 Unsupported tags for C#
- 18.6 Troubleshooting
    - 18.6.1 Problem with conditional compilation
- 18.7 Developer information
    - 18.7.1 Doxygen translator design
    - 18.7.2 Debugging the Doxygen parser and translator
    - 18.7.3 Tests
- 18.8 Extending to other languages

**19 Warning Messages**

- 19.1 Introduction
- 19.2 Warning message suppression
- 19.3 Enabling extra warnings

- 19.4 Issuing a warning message
- 19.5 Symbolic symbols
- 19.6 Commentary
- 19.7 Warnings as errors
- 19.8 Message output format
- 19.9 Warning number reference
  - 19.9.1 Deprecated features (100-199)
  - 19.9.2 Preprocessor (200-299)
  - 19.9.3 C/C++ Parser (300-399)
  - 19.9.4 Types and typemaps (400-499)
  - 19.9.5 Code generation (500-559)
  - 19.9.6 Doxygen comments (560-599)
  - 19.9.7 Language module specific (700-899)
  - 19.9.8 User defined (900-999)
- 19.10 History

**20 Working with Modules**

- 20.1 Modules Introduction
- 20.2 Basics
- 20.3 The SWIG runtime code
- 20.4 External access to the runtime
- 20.5 A word of caution about static libraries
- 20.6 References
- 20.7 Reducing the wrapper file size

**21 Using SWIG with ccache - ccache-swig(1) manpage**

- 21.1 NAME
- 21.2 SYNOPSIS
- 21.3 DESCRIPTION
- 21.4 OPTIONS SUMMARY
- 21.5 OPTIONS
- 21.6 INSTALLATION
- 21.7 EXTRA OPTIONS
- 21.8 ENVIRONMENT VARIABLES
- 21.9 CACHE SIZE MANAGEMENT
- 21.10 CACHE COMPRESSION
- 21.11 HOW IT WORKS
- 21.12 USING CCACHE WITH DISTCC
- 21.13 SHARING A CACHE
- 21.14 HISTORY
- 21.15 DIFFERENCES FROM COMPILERCACHE
- 21.16 CREDITS
- 21.17 AUTHOR

**22 SWIG and Android**

- 22.1 Overview
- 22.2 Android examples
  - 22.2.1 Examples introduction
  - 22.2.2 Simple C example
  - 22.2.3 C++ class example
  - 22.2.4 Other examples
- 22.3 C++ STL

**23 SWIG and C#**

- 23.1 Introduction
  - 23.1.1 SWIG 2 Compatibility
  - 23.1.2 Additional command line options
- 23.2 Differences to the Java module
- 23.3 Type mapping
  - 23.3.1 Primitive types
  - 23.3.2 Other types
  - 23.3.3 Void pointers
- 23.4 C# Arrays
  - 23.4.1 The SWIG C arrays library
  - 23.4.2 Managed arrays using P/Invoke default array marshalling
  - 23.4.3 Managed arrays using pinning
- 23.5 C# Exceptions
  - 23.5.1 C# exception example using "check" typemap
  - 23.5.2 C# exception example using %exception
  - 23.5.3 C# exception example using exception specifications
  - 23.5.4 Custom C# ApplicationException example
- 23.6 C# Directors
  - 23.6.1 Directors example
  - 23.6.2 Directors implementation
  - 23.6.3 Director caveats
- 23.7 Multiple modules
- 23.8 C# named and optional arguments
- 23.9 C# Typemap examples
  - 23.9.1 Memory management when returning references to member variables
  - 23.9.2 Memory management for objects passed to the C++ layer
  - 23.9.3 Date marshalling using the csin typemap and associated attributes
  - 23.9.4 A date example demonstrating marshalling of C# properties
  - 23.9.5 Date example demonstrating the 'pre' and 'post' typemap attributes for directors
  - 23.9.6 Turning proxy classes into partial classes
  - 23.9.7 Turning proxy classes into sealed classes
  - 23.9.8 Extending proxy classes with additional C# code
  - 23.9.9 Underlying type for enums

**24 SWIG and D**

- 24.1 Introduction
- 24.2 Command line invocation
- 24.3 Typemaps
  - 24.3.1 C# <-> D name comparison
  - 24.3.2 ctype, imtype, dtype
  - 24.3.3 in, out, directorin, directorout
  - 24.3.4 din, dout, ddirectorin, ddirectorout
  - 24.3.5 typecheck typemaps
  - 24.3.6 Code injection typemaps

- 24.3.7 Special variable macros
- 24.4 Other D code control features
  - 24.4.1 D begin
  - 24.4.2 D and %feature
- 24.5 Pragmas
- 24.6 D Exceptions
- 24.7 D Directors
- 24.8 Other features
  - 24.8.1 Extended namespace support (nspace)
  - 24.8.2 Native pointer support
  - 24.8.3 Operator overloading
  - 24.8.4 Running the test-suite
- 24.9 D Typemap examples
- 24.10 Work in progress and planned features

**25 SWIG and Go**

- 25.1 Overview
- 25.2 Examples
- 25.3 Running SWIG with Go
  - 25.3.1 Go-specific Commandline Options
  - 25.3.2 Generated Wrapper Files
- 25.4 A tour of basic C/C++ wrapping
  - 25.4.1 Go Package Name
  - 25.4.2 Go Names
  - 25.4.3 Go Constants
  - 25.4.4 Go Enumerations
  - 25.4.5 Go Classes
    - 25.4.5.1 Go Class Memory Management
    - 25.4.5.2 Go Class Inheritance
  - 25.4.6 Go Templates
  - 25.4.7 Go and C/C++ Threads
  - 25.4.8 Go and C++ Exceptions
  - 25.4.9 Go Director Classes
    - 25.4.9.1 Example C++ code
    - 25.4.9.2 Enable director feature
    - 25.4.9.3 Constructor and destructor
    - 25.4.9.4 Override virtual methods
    - 25.4.9.5 Call base methods
    - 25.4.9.6 Subclass via embedding
    - 25.4.9.7 Memory management with runtime.SetFinalizer
    - 25.4.9.8 Complete FooBarGo example class
  - 25.4.10 Default Go primitive type mappings
  - 25.4.11 Output arguments
  - 25.4.12 Adding additional go code
  - 25.4.13 Go typemaps

**26 SWIG and Guile**

- 26.1 Supported Guile Versions
- 26.2 Meaning of "Module"
- 26.3 Old GH Guile API
- 26.4 Linkage
  - 26.4.1 Simple Linkage
  - 26.4.2 Passive Linkage
  - 26.4.3 Native Guile Module Linkage
  - 26.4.4 Old Auto-Loading Guile Module Linkage
  - 26.4.5 Hobbit4D Linkage
- 26.5 Underscore Folding
- 26.6 Typemaps
- 26.7 Representation of pointers as smobs
  - 26.7.1 Smobs
  - 26.7.2 Garbage Collection
- 26.8 Native Guile pointers
- 26.9 Exception Handling
- 26.10 Procedure documentation
- 26.11 Procedures with setters
- 26.12 GOOPS Proxy Classes
  - 26.12.1 Naming Issues
  - 26.12.2 Linking

**27 SWIG and Java**

- 27.1 Overview
- 27.2 Preliminaries
  - 27.2.1 Running SWIG
  - 27.2.2 Additional Commandline Options
  - 27.2.3 Getting the right header files
  - 27.2.4 Compiling a dynamic module
  - 27.2.5 Using your module
  - 27.2.6 Dynamic linking problems
  - 27.2.7 Compilation problems and compiling with C++
  - 27.2.8 Building on Windows
    - 27.2.8.1 Running SWIG from Visual Studio
    - 27.2.8.2 Using NMAKE
- 27.3 A tour of basic C/C++ wrapping
  - 27.3.1 Modules, packages and generated Java classes
  - 27.3.2 Functions
  - 27.3.3 Global variables
  - 27.3.4 Constants
  - 27.3.5 Enumerations
    - 27.3.5.1 Anonymous enums
    - 27.3.5.2 Typesafe enums
    - 27.3.5.3 Proper Java enums
    - 27.3.5.4 Type unsafe enums
    - 27.3.5.5 Simple enums
  - 27.3.6 Pointers
  - 27.3.7 Structures
  - 27.3.8 C++ classes
  - 27.3.9 C++ inheritance
  - 27.3.10 Pointers, references, arrays and pass by value
    - 27.3.10.1 Null pointers
  - 27.3.11 C++ overloaded functions

        - 27.3.12 C++ default arguments
        - 27.3.13 C++ namespaces
        - 27.3.14 C++ templates
        - 27.3.15 C++ Smart Pointers
            - 27.3.15.1 The shared_ptr Smart Pointer
            - 27.3.15.2 Generic Smart Pointers
    - 27.4 Further details on the generated Java classes
        - 27.4.1 The intermediary JNI class
            - 27.4.1.1 The intermediary JNI class pragmas
        - 27.4.2 The Java module class
            - 27.4.2.1 The Java module class pragmas
        - 27.4.3 The Java constants interface
            - 27.4.3.1 The Java constants interface pragmas
        - 27.4.4 Java proxy classes
            - 27.4.4.1 Memory management
            - 27.4.4.2 Inheritance
            - 27.4.4.3 Proxy classes and garbage collection
            - 27.4.4.4 The premature garbage collection prevention parameter for proxy class marshalling
            - 27.4.4.5 Single threaded applications and thread safety
        - 27.4.5 Type wrapper classes
        - 27.4.6 Enum classes
            - 27.4.6.1 Typesafe enum classes
            - 27.4.6.2 Proper Java enum classes
            - 27.4.6.3 Type unsafe enum classes
        - 27.4.7 Interfaces
    - 27.5 Cross language polymorphism using directors
        - 27.5.1 Enabling directors
        - 27.5.2 Director classes
        - 27.5.3 Overhead and code bloat
        - 27.5.4 Simple directors example
        - 27.5.5 Director threading issues
        - 27.5.6 Director performance tuning
        - 27.5.7 Java exceptions from directors
            - 27.5.7.1 Customizing director exceptions
        - 27.5.8 Accessing virtual protected methods
        - 27.5.9 Accessing non-virtual protected members
    - 27.6 Common customization features
        - 27.6.1 C/C++ helper functions
        - 27.6.2 Class extension with %extend
        - 27.6.3 Class extension with %proxycode
        - 27.6.4 Exception handling with %exception and %javaexception
        - 27.6.5 Method access with %javamethodmodifiers
        - 27.6.6 Java begin
    - 27.7 Tips and techniques
        - 27.7.1 Input and output parameters using primitive pointers and references
        - 27.7.2 Simple pointers
        - 27.7.3 Wrapping C arrays with Java arrays
        - 27.7.4 Unbounded C Arrays
        - 27.7.5 Passing a string with length
        - 27.7.6 Overriding new and delete to allocate from Java heap
    - 27.8 Java typemaps
        - 27.8.1 Default primitive type mappings
        - 27.8.2 Default typemaps for non-primitive types
        - 27.8.3 Sixty four bit JVMs
        - 27.8.4 What is a typemap?
        - 27.8.5 Typemaps for mapping C/C++ types to Java types
        - 27.8.6 Java typemap attributes
        - 27.8.7 Java special variables
        - 27.8.8 Typemaps for both C and C++ compilation
        - 27.8.9 Java code typemaps
        - 27.8.10 Director specific typemaps
    - 27.9 Typemap Examples
        - 27.9.1 Simpler Java enums for enums without initializers
        - 27.9.2 Handling C++ exception specifications as Java exceptions
        - 27.9.3 NaN Exception - exception handling for a particular type
        - 27.9.4 Converting Java String arrays to char **
        - 27.9.5 Expanding a Java object to multiple arguments
        - 27.9.6 Using typemaps to return arguments
        - 27.9.7 Adding Java downcasts to polymorphic return types
        - 27.9.8 Adding an equals method to the Java classes
        - 27.9.9 Void pointers and a common Java base class
        - 27.9.10 Struct pointer to pointer
        - 27.9.11 Memory management when returning references to member variables
        - 27.9.12 Memory management for objects passed to the C++ layer
        - 27.9.13 Date marshalling using the javain typemap and associated attributes
    - 27.10 Living with Java Directors
    - 27.11 Odds and ends
        - 27.11.1 JavaDoc comments
        - 27.11.2 Functional interface without proxy classes
        - 27.11.3 Using your own JNI functions
        - 27.11.4 Performance concerns and hints
        - 27.11.5 Debugging
    - 27.12 Java Examples

**28 SWIG and Javascript**

    - 28.1 Overview
    - 28.2 Preliminaries
        - 28.2.1 Running SWIG
        - 28.2.2 Running Tests and Examples
        - 28.2.3 Known Issues
    - 28.3 Integration
        - 28.3.1 Creating node.js Extensions
            - 28.3.1.1 Using yeoman to generate a Node-API skeleton
            - 28.3.1.2 Troubleshooting
        - 28.3.2 Embedded Webkit
            - 28.3.2.1 Mac OS X
            - 28.3.2.2 GTK
        - 28.3.3 Creating Applications with node-webkit
    - 28.4 Examples
        - 28.4.1 Simple
        - 28.4.2 Class

- 28.5 Implementation
  - 28.5.1 Source Code
  - 28.5.2 Code Templates
  - 28.5.3 Emitter
  - 28.5.4 Emitter states
  - 28.5.5 Handling Exceptions in JavascriptCore
  - 28.5.6 Handling Exceptions in Node-API

**29 SWIG and Lua**

- 29.1 Preliminaries
- 29.2 Running SWIG
  - 29.2.1 Additional command line options
  - 29.2.2 Compiling and Linking and Interpreter
  - 29.2.3 Compiling a dynamic module
  - 29.2.4 Using your module
- 29.3 A tour of basic C/C++ wrapping
  - 29.3.1 Modules
  - 29.3.2 Functions
  - 29.3.3 Global variables
  - 29.3.4 Constants and enums
    - 29.3.4.1 Constants/enums and classes/structures
  - 29.3.5 Pointers
  - 29.3.6 Structures
  - 29.3.7 C++ classes
  - 29.3.8 C++ inheritance
  - 29.3.9 Pointers, references, values, and arrays
  - 29.3.10 C++ overloaded functions
  - 29.3.11 C++ operators
  - 29.3.12 Class extension with %extend
  - 29.3.13 Using %newobject to release memory
  - 29.3.14 C++ templates
  - 29.3.15 C++ Smart Pointers
  - 29.3.16 C++ Exceptions
  - 29.3.17 Namespaces
    - 29.3.17.1 Compatibility Note
    - 29.3.17.2 Names
    - 29.3.17.3 Inheritance
- 29.4 Typemaps
  - 29.4.1 What is a typemap?
  - 29.4.2 Using typemaps
  - 29.4.3 Typemaps and arrays
  - 29.4.4 Typemaps and pointer-pointer functions
- 29.5 Writing typemaps
  - 29.5.1 Typemaps you can write
  - 29.5.2 SWIG's Lua-C API
- 29.6 Customization of your Bindings
  - 29.6.1 Writing your own custom wrappers
  - 29.6.2 Adding additional Lua code
- 29.7 Details on the Lua binding
  - 29.7.1 Binding global data into the module.
  - 29.7.2 Userdata and Metatables
  - 29.7.3 Memory management

**30 SWIG and Octave**

- 30.1 Preliminaries
- 30.2 Running SWIG
  - 30.2.1 Command-line options
  - 30.2.2 Compiling a dynamic module
  - 30.2.3 Using your module
- 30.3 A tour of basic C/C++ wrapping
  - 30.3.1 Modules
  - 30.3.2 Functions
  - 30.3.3 Global variables
  - 30.3.4 Constants and enums
  - 30.3.5 Pointers
  - 30.3.6 Structures and C++ classes
  - 30.3.7 C++ inheritance
  - 30.3.8 C++ overloaded functions
  - 30.3.9 C++ operators
  - 30.3.10 Class extension with %extend
  - 30.3.11 C++ templates
  - 30.3.12 C++ Smart Pointers
    - 30.3.12.1 The shared_ptr Smart Pointer
    - 30.3.12.2 Generic Smart Pointers
  - 30.3.13 Directors (calling Octave from C++ code)
  - 30.3.14 Threads
  - 30.3.15 Memory management
  - 30.3.16 STL support
  - 30.3.17 Matrix typemaps

**31 SWIG and Perl5**

- 31.1 Overview
- 31.2 Preliminaries
  - 31.2.1 Getting the right header files
  - 31.2.2 Compiling a dynamic module
  - 31.2.3 Building a dynamic module with MakeMaker
  - 31.2.4 Building a static version of Perl
  - 31.2.5 Using the module
  - 31.2.6 Compilation problems and compiling with C++
  - 31.2.7 Compiling for 64-bit platforms
- 31.3 Building Perl Extensions under Windows
  - 31.3.1 Running SWIG from Developer Studio
  - 31.3.2 Using other compilers
- 31.4 The low-level interface
  - 31.4.1 Functions
  - 31.4.2 Global variables
  - 31.4.3 Constants
  - 31.4.4 Pointers
  - 31.4.5 Structures
  - 31.4.6 C++ classes

- 31.4.7 C++ classes and type-checking
- 31.4.8 C++ overloaded functions
- 31.4.9 Operators
- 31.4.10 Modules and packages
- 31.5 Input and output parameters
- 31.6 Exception handling
- 31.7 Remapping datatypes with typemaps
  - 31.7.1 A simple typemap example
  - 31.7.2 Perl5 typemaps
  - 31.7.3 Typemap variables
  - 31.7.4 Useful functions
- 31.8 Typemap Examples
  - 31.8.1 Converting a Perl5 array to a char **
  - 31.8.2 Return values
  - 31.8.3 Returning values from arguments
  - 31.8.4 Accessing array structure members
  - 31.8.5 Turning Perl references into C pointers
  - 31.8.6 Pointer handling
- 31.9 Proxy classes
  - 31.9.1 Preliminaries
  - 31.9.2 Structure and class wrappers
  - 31.9.3 Object Ownership
  - 31.9.4 Nested Objects
  - 31.9.5 Proxy Functions
  - 31.9.6 Inheritance
  - 31.9.7 Modifying the proxy methods
- 31.10 Adding additional Perl code
- 31.11 Cross language polymorphism
  - 31.11.1 Enabling directors
  - 31.11.2 Director classes
  - 31.11.3 Ownership and object destruction
  - 31.11.4 Exception unrolling
  - 31.11.5 Overhead and code bloat
  - 31.11.6 Typemaps

**32 SWIG and PHP**

- 32.1 Generating PHP Extensions
  - 32.1.1 Building a loadable extension
  - 32.1.2 Using PHP Extensions
- 32.2 Basic PHP interface
  - 32.2.1 Constants
  - 32.2.2 Global Variables
  - 32.2.3 Functions
  - 32.2.4 Overloading
  - 32.2.5 Pointers and References
  - 32.2.6 Structures and C++ classes
    - 32.2.6.1 Using -noproxy
    - 32.2.6.2 Constructors and Destructors
    - 32.2.6.3 Static Member Variables
    - 32.2.6.4 Static Member Functions
    - 32.2.6.5 Specifying Implemented Interfaces
    - 32.2.6.6 Dynamic Properties
  - 32.2.7 PHP Pragmas, Startup and Shutdown code
- 32.3 Cross language polymorphism
  - 32.3.1 Enabling directors
  - 32.3.2 Director classes
  - 32.3.3 Ownership and object destruction
  - 32.3.4 Exception unrolling
  - 32.3.5 Overhead and code bloat
  - 32.3.6 Typemaps
  - 32.3.7 Miscellaneous

**33 SWIG and Python**

- 33.1 Overview
- 33.2 Preliminaries
  - 33.2.1 Running SWIG
  - 33.2.2 Using distutils
  - 33.2.3 Hand compiling a dynamic module
  - 33.2.4 Static linking
  - 33.2.5 Using your module
  - 33.2.6 Compilation of C++ extensions
  - 33.2.7 Compiling for 64-bit platforms
  - 33.2.8 Building Python extensions under Windows
  - 33.2.9 Additional Python commandline options
- 33.3 A tour of basic C/C++ wrapping
  - 33.3.1 Modules
  - 33.3.2 Functions
  - 33.3.3 Global variables
  - 33.3.4 Constants and enums
  - 33.3.5 Pointers
  - 33.3.6 Structures
  - 33.3.7 C++ classes
  - 33.3.8 C++ inheritance
  - 33.3.9 Pointers, references, values, and arrays
  - 33.3.10 C++ overloaded functions
  - 33.3.11 C++ operators
  - 33.3.12 C++ namespaces
  - 33.3.13 C++ templates
  - 33.3.14 C++ Smart Pointers
    - 33.3.14.1 The shared_ptr Smart Pointer
    - 33.3.14.2 Generic Smart Pointers
  - 33.3.15 C++ reference counted objects
- 33.4 Further details on the Python class interface
  - 33.4.1 Proxy classes
  - 33.4.2 Built-in Types
    - 33.4.2.1 Limitations
    - 33.4.2.2 Operator overloads and slots -- use them!
  - 33.4.3 Memory management
- 33.5 Cross language polymorphism
  - 33.5.1 Enabling directors

- 33.5.2 Director classes
- 33.5.3 Ownership and object destruction
- 33.5.4 Exception unrolling
- 33.5.5 Overhead and code bloat
- 33.5.6 Typemaps
- 33.5.7 Thread safety
- 33.5.8 Miscellaneous
- 33.6 Common customization features
  - 33.6.1 C/C++ helper functions
  - 33.6.2 Adding additional Python code
  - 33.6.3 Class extension with %extend
  - 33.6.4 Exception handling with %exception
  - 33.6.5 Optimization options
    - 33.6.5.1 -fastproxy
  - 33.6.6 Stable ABI
- 33.7 Tips and techniques
  - 33.7.1 Input and output parameters
  - 33.7.2 Simple pointers
  - 33.7.3 Unbounded C Arrays
  - 33.7.4 String handling
  - 33.7.5 Default arguments
- 33.8 Typemaps
  - 33.8.1 What is a typemap?
  - 33.8.2 Python typemaps
  - 33.8.3 Typemap variables
  - 33.8.4 Useful Python Functions
- 33.9 Typemap Examples
  - 33.9.1 Converting a Python list to a char **
  - 33.9.2 Expanding a Python object into multiple arguments
  - 33.9.3 Using typemaps to return arguments
  - 33.9.4 Mapping Python tuples into small arrays
  - 33.9.5 Mapping sequences to C arrays
  - 33.9.6 Pointer handling
  - 33.9.7 Memory management when returning references to member variables
- 33.10 Docstring Features
  - 33.10.1 Module docstring
  - 33.10.2 %feature("autodoc")
    - 33.10.2.1 %feature("autodoc", "0")
    - 33.10.2.2 %feature("autodoc", "1")
    - 33.10.2.3 %feature("autodoc", "2")
    - 33.10.2.4 %feature("autodoc", "3")
    - 33.10.2.5 %feature("autodoc", "docstring")
  - 33.10.3 %feature("docstring")
  - 33.10.4 Doxygen comments
- 33.11 Python Packages
  - 33.11.1 Setting the Python package
  - 33.11.2 Absolute and relative imports
  - 33.11.3 Enforcing absolute import semantics
  - 33.11.4 Importing from __init__.py
  - 33.11.5 Implicit namespace packages
  - 33.11.6 Location of modules
    - 33.11.6.1 Both modules in the same package
    - 33.11.6.2 Both modules are global
    - 33.11.6.3 Split modules custom configuration
    - 33.11.6.4 More on customizing the module import code
    - 33.11.6.5 Statically linked C modules
- 33.12 Python 3 Support
  - 33.12.1 Python function annotations and variable annotations
    - 33.12.1.1 C/C++ annotation types
  - 33.12.2 Buffer interface
  - 33.12.3 Abstract base classes
  - 33.12.4 Byte string output conversion
  - 33.12.5 Python 2 Unicode
- 33.13 Support for Multithreaded Applications
  - 33.13.1 UI for Enabling Multithreading Support
  - 33.13.2 Multithread Performance

**34 SWIG and R**

- 34.1 Bugs
- 34.2 Using R and SWIG
- 34.3 Precompiling large R files
- 34.4 General policy
- 34.5 Language conventions
- 34.6 C++ classes
  - 34.6.1 Examples
- 34.7 Enumerations

**35 SWIG and Ruby**

- 35.1 Preliminaries
  - 35.1.1 Running SWIG
  - 35.1.2 Getting the right header files
  - 35.1.3 Compiling a dynamic module
  - 35.1.4 Using your module
  - 35.1.5 Static linking
  - 35.1.6 Compilation of C++ extensions
- 35.2 Building Ruby Extensions under Windows 95/NT
  - 35.2.1 Running SWIG from Developer Studio
- 35.3 The Ruby-to-C/C++ Mapping
  - 35.3.1 Modules
  - 35.3.2 Functions
  - 35.3.3 Variable Linking
  - 35.3.4 Constants
  - 35.3.5 Pointers
  - 35.3.6 Structures
  - 35.3.7 C++ classes
  - 35.3.8 C++ Inheritance
  - 35.3.9 C++ Overloaded Functions
  - 35.3.10 C++ Operators
  - 35.3.11 C++ namespaces
  - 35.3.12 C++ templates

- 35.3.13 C++ Standard Template Library (STL)
- 35.3.14 C++ STL Functors
- 35.3.15 C++ STL Iterators
- 35.3.16 C++ Smart Pointers
  - 35.3.16.1 The shared_ptr Smart Pointer
  - 35.3.16.2 Generic Smart Pointers
- 35.3.17 Cross-Language Polymorphism
  - 35.3.17.1 Exception Unrolling
- 35.4 Naming
  - 35.4.1 Defining Aliases
  - 35.4.2 Predicate Methods
  - 35.4.3 Bang Methods
  - 35.4.4 Getters and Setters
- 35.5 Input and output parameters
- 35.6 Exception handling
  - 35.6.1 Using the %exception directive
  - 35.6.2 Handling Ruby Blocks
  - 35.6.3 Raising exceptions
  - 35.6.4 Exception classes
- 35.7 Typemaps
  - 35.7.1 What is a typemap?
  - 35.7.2 Typemap scope
  - 35.7.3 Copying a typemap
  - 35.7.4 Deleting a typemap
  - 35.7.5 Placement of typemaps
  - 35.7.6 Ruby typemaps
    - 35.7.6.1 "in" typemap
    - 35.7.6.2 "typecheck" typemap
    - 35.7.6.3 "out" typemap
    - 35.7.6.4 "arginit" typemap
    - 35.7.6.5 "default" typemap
    - 35.7.6.6 "check" typemap
    - 35.7.6.7 "argout" typemap
    - 35.7.6.8 "freearg" typemap
    - 35.7.6.9 "newfree" typemap
    - 35.7.6.10 "memberin" typemap
    - 35.7.6.11 "varin" typemap
    - 35.7.6.12 "varout" typemap
    - 35.7.6.13 "throws" typemap
    - 35.7.6.14 directorin typemap
    - 35.7.6.15 directorout typemap
    - 35.7.6.16 directorargout typemap
    - 35.7.6.17 ret typemap
    - 35.7.6.18 globalin typemap
  - 35.7.7 Typemap variables
  - 35.7.8 Useful Functions
    - 35.7.8.1 C Datatypes to Ruby Objects
    - 35.7.8.2 Ruby Objects to C Datatypes
    - 35.7.8.3 Macros for VALUE
    - 35.7.8.4 Exceptions
    - 35.7.8.5 Iterators
  - 35.7.9 Typemap Examples
  - 35.7.10 Converting a Ruby array to a char **
  - 35.7.11 Collecting arguments in a hash
  - 35.7.12 Pointer handling
    - 35.7.12.1 Ruby Datatype Wrapping
  - 35.7.13 Example: STL Vector to Ruby Array
- 35.8 Docstring Features
  - 35.8.1 Module docstring
  - 35.8.2 %feature("autodoc")
    - 35.8.2.1 %feature("autodoc", "0")
    - 35.8.2.2 %feature("autodoc", "1")
    - 35.8.2.3 %feature("autodoc", "2")
    - 35.8.2.4 %feature("autodoc", "3")
    - 35.8.2.5 %feature("autodoc", "docstring")
  - 35.8.3 %feature("docstring")
- 35.9 Advanced Topics
  - 35.9.1 Operator overloading
  - 35.9.2 Creating Multi-Module Packages
  - 35.9.3 Specifying Mixin Modules
- 35.10 Memory Management
  - 35.10.1 Mark and Sweep Garbage Collector
  - 35.10.2 Object Ownership
  - 35.10.3 Object Tracking
  - 35.10.4 Mark Functions
  - 35.10.5 Free Functions
  - 35.10.6 Embedded Ruby and the C++ Stack

**36 SWIG and Scilab**

- 36.1 Preliminaries
- 36.2 Running SWIG
  - 36.2.1 Generating the module
  - 36.2.2 Building the module
  - 36.2.3 Loading the module
  - 36.2.4 Using the module
  - 36.2.5 Scilab command line options
- 36.3 A basic tour of C/C++ wrapping
  - 36.3.1 Overview
  - 36.3.2 Identifiers
  - 36.3.3 Functions
    - 36.3.3.1 Argument passing
    - 36.3.3.2 Multiple output arguments
  - 36.3.4 Global variables
  - 36.3.5 Constants and enumerations
    - 36.3.5.1 Constants
    - 36.3.5.2 Enumerations
  - 36.3.6 Pointers
    - 36.3.6.1 Utility functions
    - 36.3.6.2 Null pointers:
  - 36.3.7 Structures
  - 36.3.8 C++ classes

        - 36.3.9 C++ inheritance
        - 36.3.10 C++ overloading
        - 36.3.11 Pointers, references, values, and arrays
        - 36.3.12 C++ templates
        - 36.3.13 C++ operators
        - 36.3.14 C++ namespaces
        - 36.3.15 C++ exceptions
        - 36.3.16 C++ STL
    - 36.4 Type mappings and libraries
        - 36.4.1 Default primitive type mappings
        - 36.4.2 Arrays
        - 36.4.3 Pointer-to-pointers
        - 36.4.4 Matrices
        - 36.4.5 STL
    - 36.5 Module initialization
    - 36.6 Building modes
        - 36.6.1 No-builder mode
        - 36.6.2 Builder mode
    - 36.7 Generated scripts
        - 36.7.1 Builder script
        - 36.7.2 Loader script
        - 36.7.3 Gateway XML files
    - 36.8 Other resources

**37 SWIG and Tcl**

- 37.1 Preliminaries
    - 37.1.1 Getting the right header files
    - 37.1.2 Compiling a dynamic module
    - 37.1.3 Static linking
    - 37.1.4 Using your module
    - 37.1.5 Compilation of C++ extensions
    - 37.1.6 Compiling for 64-bit platforms
    - 37.1.7 Setting a package prefix
    - 37.1.8 Using namespaces
- 37.2 Building Tcl/Tk Extensions under Windows 95/NT
    - 37.2.1 Running SWIG from Developer Studio
    - 37.2.2 Using NMAKE
- 37.3 A tour of basic C/C++ wrapping
    - 37.3.1 Modules
    - 37.3.2 Functions
    - 37.3.3 Global variables
    - 37.3.4 Constants and enums
    - 37.3.5 Pointers
    - 37.3.6 Structures
    - 37.3.7 C++ classes
    - 37.3.8 C++ inheritance
    - 37.3.9 Pointers, references, values, and arrays
    - 37.3.10 C++ overloaded functions
    - 37.3.11 C++ operators
    - 37.3.12 C++ namespaces
    - 37.3.13 C++ templates
    - 37.3.14 C++ Smart Pointers
- 37.4 Further details on the Tcl class interface
    - 37.4.1 Proxy classes
    - 37.4.2 Memory management
- 37.5 Input and output parameters
- 37.6 Exception handling
- 37.7 Typemaps
    - 37.7.1 What is a typemap?
    - 37.7.2 Tcl typemaps
    - 37.7.3 Typemap variables
    - 37.7.4 Converting a Tcl list to a char **
    - 37.7.5 Returning values in arguments
    - 37.7.6 Useful functions
    - 37.7.7 Standard typemaps
    - 37.7.8 Pointer handling
- 37.8 Turning a SWIG module into a Tcl Package.
- 37.9 Building new kinds of Tcl interfaces (in Tcl)
    - 37.9.1 Proxy classes
- 37.10 Tcl/Tk Stubs

**38 SWIG and C as the target language**

- 38.1 Overview
    - 38.1.1 Known C++ Shortcomings in Generated C API
- 38.2 Preliminaries
    - 38.2.1 Running SWIG
    - 38.2.2 Command line options
    - 38.2.3 Compiling a dynamic module
    - 38.2.4 Using the generated module
- 38.3 Basic C wrapping
    - 38.3.1 Functions
    - 38.3.2 Variables
- 38.4 Basic C++ wrapping
    - 38.4.1 Enums
    - 38.4.2 Classes
- 38.5 Backend Developer Documentation
- 38.6 Typemaps
    - 38.6.1 C Typemaps, a Code Generation Walkthrough
        - 38.6.1.1 The Interface
        - 38.6.1.2 The Wrapper
        - 38.6.1.3 The Proxy
- 38.7 Exception handling
- 38.8 C++ Wrappers
    - 38.8.1 Additional customization possibilities
    - 38.8.2 Exception handling

**39 SWIG and MzScheme/Racket**

- 39.1 Creating native structures
- 39.2 Simple example
- 39.3 External documentation

**40 SWIG and OCaml**

- 40.1 Preliminaries
  - 40.1.1 Running SWIG
  - 40.1.2 Compiling the code
  - 40.1.3 The camlp4 module
  - 40.1.4 Using your module
  - 40.1.5 Compilation problems and compiling with C++
- 40.2 The low-level Ocaml/C interface
  - 40.2.1 The generated module
  - 40.2.2 Enums
    - 40.2.2.1 Enum typing in Ocaml
  - 40.2.3 Arrays
    - 40.2.3.1 Simple types of bounded arrays
    - 40.2.3.2 Complex and unbounded arrays
    - 40.2.3.3 Using an object
    - 40.2.3.4 Example typemap for a function taking float * and int
  - 40.2.4 C++ Classes
    - 40.2.4.1 STL vector and string Example
    - 40.2.4.2 C++ Class Example
    - 40.2.4.3 Compiling the example
    - 40.2.4.4 Sample Session
  - 40.2.5 Director Classes
    - 40.2.5.1 Director Introduction
    - 40.2.5.2 Overriding Methods in Ocaml
    - 40.2.5.3 Director Usage Example
    - 40.2.5.4 Creating director objects
    - 40.2.5.5 Typemaps for directors, directorin, directorout, directorargout
    - 40.2.5.6 directorin typemap
    - 40.2.5.7 directorout typemap
    - 40.2.5.8 directorargout typemap
  - 40.2.6 Exceptions
- 40.3 Documentation Features
  - 40.3.1 Module docstring

**41 Extending SWIG to support new languages**

- 41.1 Introduction
- 41.2 Prerequisites
- 41.3 The Big Picture
- 41.4 Execution Model
  - 41.4.1 Preprocessing
  - 41.4.2 Parsing
  - 41.4.3 Parse Trees
  - 41.4.4 Attribute namespaces
  - 41.4.5 Symbol Tables
  - 41.4.6 The %feature directive
  - 41.4.7 Code Generation
  - 41.4.8 SWIG and XML
- 41.5 Primitive Data Structures
  - 41.5.1 Strings
  - 41.5.2 Hashes
  - 41.5.3 Lists
  - 41.5.4 Common operations
  - 41.5.5 Iterating over Lists and Hashes
  - 41.5.6 I/O
- 41.6 Navigating and manipulating parse trees
- 41.7 Working with attributes
- 41.8 Type system
  - 41.8.1 String encoding of types
  - 41.8.2 Type construction
  - 41.8.3 Type tests
  - 41.8.4 Typedef and inheritance
  - 41.8.5 Lvalues
  - 41.8.6 Output functions
- 41.9 Parameters
- 41.10 Writing a Language Module
  - 41.10.1 Execution model
  - 41.10.2 Starting out
  - 41.10.3 Command line options
  - 41.10.4 Configuration and preprocessing
  - 41.10.5 Entry point to code generation
  - 41.10.6 Module I/O and wrapper skeleton
  - 41.10.7 Low-level code generators
  - 41.10.8 Configuration files
  - 41.10.9 Runtime support
  - 41.10.10 Standard library files
  - 41.10.11 User examples
  - 41.10.12 Test driven development and the test-suite
    - 41.10.12.1 Running the test-suite
  - 41.10.13 Documentation
  - 41.10.14 Coding style guidelines
  - 41.10.15 Target language status
    - 41.10.15.1 Supported status
    - 41.10.15.2 Experimental status
    - 41.10.15.3 Deprecated status
  - 41.10.16 Prerequisites for adding a new language module to the SWIG distribution
- 41.11 Debugging Options
- 41.12 Guide to parse tree nodes
- 41.13 Further Development Information

---

# SWIG-4.3 Documentation

Last update : SWIG-4.3.1 (15 Apr 2025)

## Sections

### SWIG Core Documentation

- Preface
- Introduction
- Getting started on Windows
- Scripting
- SWIG Basics (Read this!)
- SWIG and C++
- SWIG and C++11
- SWIG and C++14
- SWIG and C++17
- SWIG and C++20
- The SWIG preprocessor
- The SWIG library
- Argument handling
- Typemaps
- Customization features
- Contracts
- Variable length arguments
- Doxygen documentation comments
- Warning messages
- Working with Modules
- Using SWIG with ccache

**Supported Language Modules Documentation**

- Android support
- C# support
- D support
- Go support
- Guile support
- Java support
- Javascript support
- Lua support
- Octave support
- Perl5 support
- PHP support
- Python support
- R support
- Ruby support
- Scilab support
- Tcl support

**Experimental Language Modules Documentation**

- C target language support
- OCaml support

**Deprecated Language Modules Documentation**

- MzScheme/Racket support

**Developer Documentation**

- Extending SWIG


# 1 Preface


- Introduction
- SWIG Versions
- SWIG License
- SWIG resources
- Prerequisites
- Organization of this manual
- How to avoid reading the manual
- Backwards compatibility
- Release notes
- Credits
- Bug reports
- Installation
  - Windows installation
  - Unix installation
  - Macintosh OS X installation
  - Testing
  - Examples


## 1.1 Introduction

SWIG (Simplified Wrapper and Interface Generator) is a software development tool for building scripting language interfaces to C and C++ programs. Originally developed in 1995, SWIG was first used by scientists in the Theoretical Physics Division at Los Alamos National Laboratory for building user interfaces to simulation codes running on the Connection Machine 5 supercomputer. In this environment, scientists needed to work with huge amounts of simulation data, complex hardware, and a constantly changing code base. The use of a scripting language interface provided a simple yet highly flexible foundation for solving these types of problems. SWIG simplifies development by largely automating the task of scripting language integration--allowing developers and users to focus on more important problems.

Although SWIG was originally developed for scientific applications, it has since evolved into a general purpose tool that is used in a wide variety of applications--in fact almost anything where C/C++ programming is involved.

## 1.2 SWIG Versions

In the late 1990's, the most stable version of SWIG was release 1.1p5. Versions 1.3.x were officially development versions and these were released over a period of 10 years starting from the year 2000. The final version in the 1.3.x series was 1.3.40, but in truth the 1.3.x series had been stable for many years. An official stable version was released along with the decision to make SWIG license changes and this gave rise to version 2.0.0 in 2010. Version 3.0.0 was released in 2014 focusing on adding C++11 support and C++ nested classes. Version 4.0.0 was released in 2019 to add in Doxygen support. Some target languages were disabled as part of a clean up and others were given a new status of either 'Supported' or 'Experimental'.

## 1.3 SWIG License

The LICENSE file shipped with SWIG in the top level directory contains the SWIG license. For further insight into the license including the license of SWIG's output code, please visit the SWIG legal page - https://www.swig.org/legal.html.

The license was clarified in version 2.0.0 so that the code that SWIG generated could be distributed under license terms of the user's choice/requirements and at the same time the SWIG source was placed under the GNU General Public License version 3.

## 1.4 SWIG resources

The official location of SWIG related material is

```
https://www.swig.org
```

This site contains the latest version of the software, users guide, and information regarding bugs, installation problems, and implementation tricks.

You can also subscribe to the swig-user mailing list by visiting the page

```
https://www.swig.org/mail.html
```

The mailing list often discusses some of the more technical aspects of SWIG along with information about beta releases and future work.

Git and Subversion access to the latest version of SWIG is also available. More information about this can be obtained at:

```
SWIG Bleeding Edge
```

## 1.5 Prerequisites

This manual assumes that you know how to write C/C++ programs and that you have at least heard of scripting languages such as Tcl, Python, and Perl. A detailed knowledge of these scripting languages is not required although some familiarity won't hurt. No prior experience with building C extensions to these languages is required---after all, this is what SWIG does automatically. However, you should be reasonably familiar with the use of compilers, linkers, and makefiles since making scripting language extensions is somewhat more complicated than writing a normal C program.

Over time SWIG releases have become significantly more capable in their C++ handling--especially support for advanced features like namespaces, overloaded operators, and templates. Whenever possible, this manual tries to cover the technicalities of this interface. However, this isn't meant to be a tutorial on C++ programming. For many of the gory details, you will almost certainly want to consult a good C++ reference. If you don't program in C++, you may just want to skip those parts of the manual.

## 1.6 Organization of this manual

The first few chapters of this manual describe SWIG in general and provide an overview of its capabilities. The remaining chapters are devoted to specific SWIG language modules and are self contained. Thus, if you are using SWIG to build Python interfaces, you can probably skip to that chapter and find almost everything you need to know.

## 1.7 How to avoid reading the manual

If you hate reading manuals, glance at the "Introduction" which contains a few simple examples. These examples contain about 95% of everything you need to know to use SWIG. After that, simply use the language-specific chapters as a reference. The SWIG distribution also comes with a large directory of examples that illustrate different topics.

## 1.8 Backwards compatibility

If you are a previous user of SWIG, don't expect SWIG to provide complete backwards compatibility. Although the developers strive to the utmost to keep backwards compatibility, this isn't always possible as the primary goal over time is to make SWIG better---a process that would simply be impossible if the developers are constantly bogged down with backwards compatibility issues. Potential incompatibilities are clearly marked in the detailed release notes.

If you need to work with different versions of SWIG and backwards compatibility is an issue, you can use the SWIG_VERSION preprocessor symbol which holds the version of SWIG being executed. SWIG_VERSION is a hexadecimal integer such as 0x010311 (corresponding to SWIG-1.3.11). This can be used in an interface file to define different typemaps, take advantage of different features etc:

```
#if SWIG_VERSION >= 0x010311
/* Use some fancy new SWIG feature */
#endif
```

Note: The SWIG preprocessor has defined SWIG_VERSION since SWIG-1.3.11.

The SWIG_VERSION macro is also generated into the SWIG wrapper file for use by the C preprocessor in the generated code since SWIG-4.1.0.

## 1.9 Release notes

The CHANGES.current, CHANGES and RELEASENOTES files shipped with SWIG in the top level directory contain, respectively, detailed release notes for the current version, detailed release notes for previous releases and summary release notes from SWIG-1.3.22 onwards.

## 1.10 Credits

SWIG is an unfunded project that would not be possible without the contributions of many people working in their spare time. If you have benefitted from using SWIG, please consider Donating to SWIG to keep development going. There have been a large varied number of people who have made contributions at all levels over time. Contributors are mentioned either in the COPYRIGHT file or CHANGES files shipped with SWIG or in submitted bugs.

## 1.11 Bug reports

Although every attempt has been made to make SWIG bug-free, we are also trying to make feature improvements that may introduce bugs. To report a bug, either send mail to the SWIG developer list at the swig-devel mailing list or report a bug at the SWIG bug tracker. In your report, be as specific as possible, including (if applicable), error messages, tracebacks (if a core dump occurred), corresponding portions of the SWIG interface file used, and any important pieces of the SWIG generated wrapper code. We can only fix bugs if we know about them.

## 1.12 Installation

### 1.12.1 Windows installation

Please see the dedicated Windows chapter for instructions on installing SWIG on Windows and running the examples. The Windows distribution is called swigwin and includes a prebuilt SWIG executable, swig.exe, included in the top level directory. Otherwise it is exactly the same as the main SWIG distribution. There is no need to download anything else.

### 1.12.2 Unix installation

These installation instructions are for using the distributed tarball, for example, `swig-3.0.8.tar.gz`. If you wish to build and install from source on Github, extra steps are required. Please see the Bleeding Edge page on the SWIG website.

You must use GNU make to build and install SWIG.

PCRE2 needs to be installed on your system to build SWIG, in particular pcre2-config must be available. If you have PCRE2 headers and libraries but not pcre2-config itself or, alternatively, wish to override the compiler or linker flags returned by pcre2-config, you may set PCRE2_LIBS and PCRE2_CFLAGS variables to be used instead. And if you don't have PCRE2 at all, the configure script will provide instructions for obtaining it.

To build and install SWIG, simply type the following:

```
$ ./configure
$ make
$ make install
```

By default SWIG installs itself in /usr/local. If you need to install SWIG in a different location or in your home directory, use the `--prefix` option to `./configure`. For example:

```
$ ./configure --prefix=/home/yourname/projects
$ make
$ make install
```

Note: the directory given to `--prefix` must be an absolute pathname. Do **not** use the ~ shell-escape to refer to your home directory. SWIG won't work properly if you do this.

The INSTALL file shipped in the top level directory details more about using configure. Also try

```
$ ./configure --help.
```

The configure script will attempt to locate various packages on your machine including Tcl, Perl5, Python and all the other target languages that SWIG supports. Don't panic if you get 'not found' messages -- SWIG does not need these packages to compile or run. The configure script is actually looking for these packages so that you can try out the SWIG examples contained in the 'Examples' directory without having to hack Makefiles. Note that the `--without-xxx` options, where xxx is a target language, have minimal effect. All they do is reduce the amount of testing done with 'make check'. The SWIG executable and library files installed cannot currently be configured with a subset of target languages.

SWIG used to include a set of runtime libraries for some languages for working with multiple modules. These are no longer built during the installation stage. However, users can build them just like any wrapper module as described in the Modules chapter. The CHANGES file shipped with SWIG in the top level directory also lists some examples which build the runtime library.

Note:

- If you checked the code out via Git, you will have to run `./autogen.sh` before `./configure`. In addition, a full build of SWIG requires a number of packages to be installed. Full instructions at SWIG bleeding edge.

### 1.12.3 Macintosh OS X installation

SWIG is known to work on various flavors of OS X. Follow the Unix installation instructions above. However, as of this writing, there is still great deal of inconsistency with how shared libraries are handled by various scripting languages on OS X.

Users of OS X should be aware that Darwin handles shared libraries and linking in a radically different way than most Unix systems. In order to test SWIG and run the examples, SWIG configures itself to use flat namespaces and to allow undefined symbols (`-flat_namespace -undefined suppress`). This mostly closely follows the Unix model and makes it more likely that the SWIG examples will work with whatever installation of software you might have. However, this is generally not the recommended technique for building larger extension modules. Instead, you should utilize Darwin's two-level namespaces. Some details about this can be found here Understanding Two-Level Namespaces.

Needless to say, you might have to experiment a bit to get things working at first.

### 1.12.4 Testing

If you want to test SWIG after building it, a check can be performed on Unix operating systems. Type the following:

```
$ make -k check
```

This step can be performed either before or after installation. The check requires at least one of the target languages to be installed. If it fails, it may mean that you have an uninstalled language module or that the file 'Examples/Makefile' has been incorrectly configured. It may also fail due to compiler issues such as a broken C++ compiler. Even if the check fails, there is a pretty good chance SWIG still works correctly --- you will just have to mess around with one of the examples and some makefiles to get it to work. Some tests may also fail due to missing dependency packages, eg PCRE or Boost, but this will require careful analysis of the configure output done during configuration.

The test suite executed by the check is designed to stress-test many parts of the implementation including obscure corner cases. If some of these tests fail or generate warning messages, there is no reason for alarm --- the test may be related to some new SWIG feature or a difficult bug that we're trying to resolve. Chances are that SWIG will work just fine for you. Note that if you have more than one CPU/core, then you can use parallel make to speed up the check as it does take quite some time to run, for example:

```
$ make -j2 -k check
```

Also, SWIG's support for C++ is sufficiently advanced that certain tests may fail on older C++ compilers (for instance if your compiler does not support member templates). These errors are harmless if you don't intend to use these features in your own programs.

Note: The test-suite currently contains over 800 tests. If you have many different target languages installed and a slow machine, it might take more than an hour to run the test-suite.

### 1.12.5 Examples

The Examples directory contains a variety of examples of using SWIG and it has some browsable documentation. Simply point your browser to the file "Example/index.html".

The Examples directory also includes Visual C++ project 6 (.dsp) files for building some of the examples on Windows. Later versions of Visual Studio will convert these old style project files into a current solution file.

---

# 2 Introduction


- What is SWIG?
- Why use SWIG?
- Target languages
  - Supported status
  - Experimental status
  - Deprecated status

- A SWIG example
  - SWIG interface file
  - The swig command
  - Building a Perl5 module
  - Building a Python module
  - Shortcuts
- Supported C/C++ language features
- Non-intrusive interface building
- Incorporating SWIG into a build system
- Hands off code generation
- SWIG and freedom


## 2.1 What is SWIG?

SWIG is a software development tool that simplifies the task of interfacing different languages to C and C++ programs. In a nutshell, SWIG is a compiler that takes C/C++ declarations and creates the wrappers needed to access those declarations from other languages including Perl, Python, Tcl, Ruby, Guile, and Java. SWIG normally requires no modifications to existing code and can often be used to build a usable interface in only a few minutes. Possible applications of SWIG include:

- Building interpreted interfaces to existing C programs.
- Rapid prototyping and application development.
- Interactive debugging.
- Reengineering or refactoring of legacy software into scripting language components.
- Making a graphical user interface (using Tk for example).
- Testing of C libraries and programs (using scripts).
- Building high performance C modules for scripting languages.
- Making C programming more enjoyable (or tolerable depending on your point of view).
- Impressing your friends.
- Obtaining vast sums of research funding (although obviously not applicable to the author).

SWIG was originally designed to make it extremely easy for scientists and engineers to build extensible scientific software without having to get a degree in software engineering. Because of this, the use of SWIG tends to be somewhat informal and ad-hoc (e.g., SWIG does not require users to provide formal interface specifications as you would find in a dedicated IDL compiler). Although this style of development isn't appropriate for every project, it is particularly well suited to software development in the small; especially the research and development work that is commonly found in scientific and engineering projects. However, nowadays SWIG is known to be used in many large open source and commercial projects.

## 2.2 Why use SWIG?

As stated in the previous section, the primary purpose of SWIG is to simplify the task of integrating C/C++ with other programming languages. However, why would anyone want to do that? To answer that question, it is useful to list a few strengths of C/C++ programming:

- Excellent support for writing programming libraries.
- High performance (number crunching, data processing, graphics, etc.).
- Systems programming and systems integration.
- Large user community and software base.

Next, let's list a few problems with C/C++ programming

- Writing a user interface is rather painful (i.e., consider programming with MFC, X11, GTK, or any number of other libraries).
- Testing is time consuming (the compile/debug cycle).
- Not easy to reconfigure or customize without recompilation.
- Modularization can be tricky.
- Security concerns (buffer overflows for instance).

To address these limitations, many programmers have arrived at the conclusion that it is much easier to use different programming languages for different tasks. For instance, writing a graphical user interface may be significantly easier in a scripting language like Python or Tcl (consider the reasons why millions of programmers have used languages like Visual Basic if you need more proof). An interactive interpreter might also serve as a useful debugging and testing tool. Other languages like Java might greatly simplify the task of writing distributed computing software. The key point is that different programming languages offer different strengths and weaknesses. Moreover, it is extremely unlikely that any programming is ever going to be perfect. Therefore, by combining languages together, you can utilize the best features of each language and greatly simplify certain aspects of software development.

From the standpoint of C/C++, a lot of people use SWIG because they want to break out of the traditional monolithic C programming model which usually results in programs that resemble this:

- A collection of functions and variables that do something useful.
- A `main()` program that starts everything.
- A horrible collection of hacks that form some kind of user interface (but which no-one really wants to touch).

Instead of going down that route, incorporating C/C++ into a higher level language often results in a more modular design, less code, better flexibility, and increased programmer productivity.

SWIG tries to make the problem of C/C++ integration as painless as possible. This allows you to focus on the underlying C program and using the high-level language interface, but not the tedious and complex chore of making the two languages talk to each other. At the same time, SWIG recognizes that all applications are different. Therefore, it provides a wide variety of customization features that let you change almost every aspect of the language bindings. This is the main reason why SWIG has such a large user manual ;-).

## 2.3 Target languages

SWIG in essence is a tool to generate code for making C/C++ code available to various other programming languages. These higher level programming languages are the target languages for the SWIG code generator and C or C++ are the input languages. A single target language must be specified when SWIG is run. This results in generating code for C/C++ and the specified target language to interface with each other. SWIG can be invoked multiple times, but with a different target language specified on each invocation. This ability to interface C/C++ to many different target languages is one of SWIG's core strengths and features.

SWIG is very broadly composed of two components. A core component creates a parse tree from the input ISO C/C++ and SWIG directives (extensions to the C/C++ standards). The parse tree is then passed to a second component, one of the target language modules for generating code specific to a higher level language. SWIG supports many different target languages. These target languages are given a status of either Supported, Experimental or Deprecated. This status is provided to indicate the level of maturity to expect when using a particular target language as not all target languages are fully developed or being kept up to date.

The second part of the SWIG documentation contains a chapter for each target level language. The target language chapters are under one of the sections indicating the status (Supported, Experimental or Deprecated) for that language.

### 2.3.1 Supported status

A target language is given the 'Supported' status when

- It is in a mature, well functioning state.
- It has its own comprehensive chapter in the documentation.
- It passes all of the main SWIG test-suite and has a range of working examples.
- It supports the vast majority of SWIG features.
- It provides strong backwards compatibility between releases.

The above is a short summary and further details are outlined in the Supported status section in the Extending chapter. The good news is that all the well-known and most popular languages have this status.

### 2.3.2 Experimental status

A target language is given the 'Experimental' status when

- It is of sub-standard quality, failing to meet the above 'Supported' status.
- It is somewhere between the mid to mature stage of development.
- It does not guarantee any backwards compatibility between releases.
- It is in need of help to finish development.

Anyone using an experimental target language is strongly urged to assist with development of the target language module if they wish to use it.

SWIG displays a warning when an experimental target language is used in order to set expectations and emphasize the experimental status of the target language. The usual warning suppression techniques can be used if required.

The above is a short summary and further details are outlined in the Experimental status section in the Extending chapter.

### 2.3.3 Deprecated status

A target language that was once 'Supported' or 'Experimental' is changed to the 'Deprecated' status when it has been neglected over time and become non-functional. It requires interested community member's contributions to restore it to its former status.

Please see the Deprecated status section in the Extending chapter for more details.

SWIG displays a deprecated warning notice whenever a 'Deprecated' language is used. The usual warning suppression techniques can be used if required.

## 2.4 A SWIG example

The best way to illustrate SWIG is with a simple example. Consider the following C code:

```
/* File : example.c */

double My_variable = 3.0;

/* Compute factorial of n */
int fact(int n) {
  if (n <= 1)
    return 1;
  else
    return n*fact(n-1);
}

/* Compute n mod m */
int my_mod(int n, int m) {
  return(n % m);
}
```

Suppose that you wanted to access these functions and the global variable `My_variable` from Tcl. You start by making a SWIG interface file as shown below (by convention, these files carry a .i suffix) :

### 2.4.1 SWIG interface file

```
/* File : example.i */
%module example
%{
/* Put headers and other declarations here */
extern double My_variable;
extern int    fact(int);
extern int    my_mod(int n, int m);
%}

extern double My_variable;
extern int    fact(int);
extern int    my_mod(int n, int m);
```

The interface file contains ISO C function prototypes and variable declarations. The `%module` directive defines the name of the module that will be created by SWIG. The `%{ %}` block provides a location for inserting additional code, such as C header files or additional C declarations, into the generated C wrapper code.

### 2.4.2 The swig command

SWIG is invoked using the `swig` command. We can use this to build a Tcl module (under Linux) as follows :

```
$ swig -tcl example.i
$ gcc -c -fPIC example.c example_wrap.c -I/usr/include/tcl8.7
$ gcc -shared example.o example_wrap.o -o example.so
$ tclsh
% load ./example.so
% fact 4
24
% my_mod 23 7
2
% expr $My_variable + 4.5
7.5
%
```

The `swig` command produced a new file called `example_wrap.c` that should be compiled along with the `example.c` file. Most operating systems and scripting languages now support dynamic loading of modules. In our example, our Tcl module has been compiled into a shared library that can be loaded into Tcl. When loaded, Tcl can now access the functions and variables declared in the SWIG interface. A look at the file `example_wrap.c` reveals a hideous mess. However, you almost never need to worry about it.

### 2.4.3 Building a Perl5 module

Now, let's turn these functions into a Perl5 module. Without making any changes type the following (shown for Solaris):

```
unix > swig -perl5 example.i
unix > gcc -c example.c example_wrap.c \
        -I/usr/local/lib/perl5/sun4-solaris/5.003/CORE
unix > ld -G example.o example_wrap.o -o example.so # This is for Solaris
unix > perl5.003
```

```
use example;
print example::fact(4), "\n";
print example::my_mod(23, 7), "\n";
print $example::My_variable + 4.5, "\n";
<ctrl-d>
24
2
7.5
unix >
```

### 2.4.4 Building a Python module

Finally, let's build a module for Python (shown for Linux).

```
$ swig -python example.i
$ gcc -c -fPIC example.c example_wrap.c -I/usr/include/python3.12
$ gcc -shared example.o example_wrap.o -o _example.so
$ python3
Python 3.12.4 (main, Jun 12 2024, 19:06:53) [GCC 13.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import example
>>> example.fact(4)
24
>>> example.my_mod(23, 7)
2
>>> example.cvar.My_variable + 4.5
7.5
```

### 2.4.5 Shortcuts

To the truly lazy programmer, one may wonder why we needed the extra interface file at all. As it turns out, you can often do without it. For example, you could also build a Perl5 module by just running SWIG on the C header file and specifying a module name as follows

```
unix > swig -perl5 -module example example.h
unix > gcc -c example.c example_wrap.c \
       -I/usr/local/lib/perl5/sun4-solaris/5.003/CORE
unix > ld -G example.o example_wrap.o -o example.so
unix > perl5.003
use example;
print example::fact(4), "\n";
print example::my_mod(23, 7), "\n";
print $example::My_variable + 4.5, "\n";
<ctrl-d>
24
2
7.5
```

## 2.5 Supported C/C++ language features

A primary goal of the SWIG project is to make the language binding process extremely easy. Although a few simple examples have been shown, SWIG is quite capable in supporting most of C++. Some of the major features include:

- Full C99 preprocessing.
- All ISO C and C++ datatypes.
- Functions, variables, and constants.
- Classes.
- Single and multiple inheritance.
- Overloaded functions and methods.
- Overloaded operators.
- C++ templates (including member templates, specialization, and partial specialization).
- Namespaces.
- Variable length arguments.
- C++ smart pointers.

Most of C++11 is also supported. Details are in the C++11 chapter. C++14 support is covered in theC++14 chapter. C++17 support is covered in theC++17 chapter. C++20 support is covered in the C++20 chapter.

It is important to stress that SWIG is not a simplistic C++ lexing tool like several apparently similar wrapper generation tools. SWIG not only parses C++, it implements the full C++ type system and it is able to understand C++ semantics. SWIG generates its wrappers with full knowledge of this information. As a result, you will find SWIG to be just as capable of dealing with nasty corner cases as it is in wrapping simple C++ code. In fact, SWIG is able to handle C++ code that stresses the very limits of many C++ compilers.

## 2.6 Non-intrusive interface building

When used as intended, SWIG requires minimal (if any) modification to existing C or C++ code. This makes SWIG extremely easy to use with existing packages and promotes software reuse and modularity. By making the C/C++ code independent of the high level interface, you can change the interface and reuse the code in other applications. It is also possible to support different types of interfaces depending on the application.

## 2.7 Incorporating SWIG into a build system

SWIG is a command line tool and as such can be incorporated into any build system that supports invoking external tools/compilers. SWIG is most commonly invoked from within a Makefile, but is also known to be invoked from popular IDEs such as Microsoft Visual Studio.

If you are using the GNU Autotools ( Autoconf/ Automake / Libtool) to configure SWIG use in your project, the SWIG Autoconf macros can be used. The primary macro is `ax_pkg_swig`, see http://www.gnu.org/software/autoconf-archive/ax_pkg_swig.html#ax_pkg_swig . The `ax_python_devel` macro is also helpful for generating Python extensions. See the Autoconf Archive for further information on this and other Autoconf macros.

There is growing support for SWIG in some build tools, for example CMake is a cross-platform, open-source build manager with built in support for SWIG. CMake can detect the SWIG executable and many of the target language libraries for linking against. CMake knows how to build shared libraries and loadable modules on many different operating systems. This allows easy cross platform SWIG development. It can also generate the custom commands necessary for driving SWIG from IDEs and makefiles. All of this can be done from a single cross platform input file. The following example is a CMake input file for creating a Python wrapper for the SWIG interface file, example.i:

```
# This is a CMake example for Python
```

```
FIND_PACKAGE(SWIG REQUIRED)
INCLUDE(${SWIG_USE_FILE})

FIND_PACKAGE(PythonLibs)
INCLUDE_DIRECTORIES(${PYTHON_INCLUDE_PATH})

INCLUDE_DIRECTORIES(${CMAKE_CURRENT_SOURCE_DIR})

SET(CMAKE_SWIG_FLAGS "")

SET_SOURCE_FILES_PROPERTIES(example.i PROPERTIES CPLUSPLUS ON)
SET_SOURCE_FILES_PROPERTIES(example.i PROPERTIES SWIG_FLAGS "-includeall")
SWIG_ADD_MODULE(example python example.i example.cxx)
SWIG_LINK_LIBRARIES(example ${PYTHON_LIBRARIES})
```

The above example will generate native build files such as makefiles, nmake files and Visual Studio projects which will invoke SWIG and compile the generated C++ files into _example.so (UNIX) or _example.pyd (Windows). For other target languages on Windows a dll, instead of a .pyd file, is usually generated.

## 2.8 Hands off code generation

SWIG is designed to produce working code that needs no hand-modification (in fact, if you look at the output, you probably won't want to modify it). You should think of your target language interface being defined entirely by the input to SWIG, not the resulting output file. While this approach may limit flexibility for hard-core hackers, it allows others to forget about the low-level implementation details.

## 2.9 SWIG and freedom

No, this isn't a special section on the sorry state of world politics. However, it may be useful to know that SWIG was written with a certain "philosophy" about programming---namely that programmers are smart and that tools should just stay out of their way. Because of that, you will find that SWIG is extremely permissive in what it lets you get away with. In fact, you can use SWIG to go well beyond "shooting yourself in the foot" if dangerous programming is your goal. On the other hand, this kind of freedom may be exactly what is needed to work with complicated and unusual C/C++ applications.

Ironically, the freedom that SWIG provides is countered by an extremely conservative approach to code generation. At its core, SWIG tries to distill even the most advanced C++ code down to a small well-defined set of interface building techniques based on ISO C programming. Because of this, you will find that SWIG interfaces can be easily compiled by virtually every C/C++ compiler and that they can be used on any platform. Again, this is an important part of staying out of the programmer's way----the last thing any developer wants to do is to spend their time debugging the output of a tool that relies on non-portable or unreliable programming features. Dependencies are often a source of incompatibilities and problems and so additional third party libraries are not used in the generated code. SWIG will also generally avoid generating code that introduces a dependency on the C++ Standard Template Library (STL). SWIG will generate code that depends on the C libraries though.

# 3 Getting started on Windows


- Installation on Windows
  - Windows Executable
- SWIG Windows Examples
  - Instructions for using the Examples with Visual Studio
    - C#
    - Java
    - Python
    - TCL
  - Instructions for using the Examples with other compilers
- Building swig.exe on Windows
  - Building swig.exe using CMake
  - Building swig.exe using MSYS2 and MinGW-w64
  - Building swig.exe using MinGW and MSYS
  - Building swig.exe using Cygwin
    - Running the examples on Windows using Cygwin
- Microsoft extensions and other Windows quirks
  - Visual C++ standards compliance
  - Calling conventions


This chapter describes SWIG usage on Microsoft Windows. Installing SWIG and running the examples is covered as well as building the SWIG executable. Usage within the Unix like environments MinGW and Cygwin is also detailed.

## 3.1 Installation on Windows

SWIG does not come with the usual Windows type installation program, however it is quite easy to get started. The main steps are:

- Download the swigwin zip package from theSWIG website and unzip into a directory. This is all that needs downloading for the Windows platform.
- Set environment variables as described in theSWIG Windows Examples section in order to run examples using Visual C++.

### 3.1.1 Windows Executable

The swigwin distribution contains the SWIG Windows 64-bit executable, swig.exe, which will only run on 64-bit versions of Windows. If you want to build your own swig.exe have a look at Building swig.exe on Windows.

## 3.2 SWIG Windows Examples

Microsoft Visual C++ is used for compiling and linking SWIG's output on Windows for some languages. However, MinGW and gcc is often the only supported toolchain for a number of target languages. The Examples directory has a few Visual C++ project files (.vcxproj files) where the target languages are known to work with Visual C++. These were produced by Visual Studio 2019. Newer versions of Visual Studio, such as Visual Studio 2022, are able to open and convert these project files if necessary. Each C# example comes with a Visual Studio 2019 solution in addition to the .vcxproj file the C# (.csproj) project file. The project files have been set up to execute SWIG in a custom build rule for the SWIG interface (.i) file. Alternatively run the examples using Cygwin.

More information on each of the examples is available with the examples distributed with SWIG (Examples/index.html).

### 3.2.1 Instructions for using the Examples with Visual Studio

Ensure the SWIG executable is as supplied in the SWIG root directory in order for the examples to work. Most languages require some environment variables to be set **before** running Visual C++. Note that Visual Studio must be re-started to pick up any changes in environment variables. Open up an example .vcxproj file or .sln file, Visual Studio will prompt you to upgrade the project if necessary. Ensure the Release build is selected then do a Rebuild Solution from the Build menu. The required environment variables are displayed with their current values during the build.

The list of required environment variables for each module language is also listed below. They are usually set from the Control Panel and System properties, but this depends on which flavour

of Windows you are running. If you don't want to use environment variables then change all occurrences of the environment variables in the .dsp files with hard coded values. If you are interested in how the project files are set up there is explanatory information in some of the language module's documentation.

#### 3.2.1.1 C#

The C# examples do not require any environment variables to be set as a C# project file is included. Just open up the .sln solution file in Visual Studio 2019 or later, select Release Build, and do a Rebuild Solution from the Build menu. The accompanying C# and C++ project files are automatically used by the solution file.

#### 3.2.1.2 Java

**`JAVA_INCLUDE`** : Set this to the directory containing jni.h
**`JAVA_BIN`** : Set this to the bin directory containing javac.exe

Example using the openjdk package installed in a Conda environment:
```
JAVA_INCLUDE: C:\miniconda3\envs\java\Library\lib\jvm\include
JAVA_BIN: C:\miniconda3\envs\java\Library\lib\jvm\bin
```

#### 3.2.1.3 Python

**`PYTHON_INCLUDE`** : Set this to the directory that contains Python.h
**`PYTHON_LIB`** : Set this to the Python library including path for linking

Example using Python 3.13 installed in a Conda environment:
```
PYTHON_INCLUDE: C:\miniconda3\envs\python\include
PYTHON_LIB: C:\miniconda3\envs\python\libs\python313.lib
```

#### 3.2.1.4 TCL

**`TCL_INCLUDE`** : Set this to the directory containing tcl.h
**`TCL_LIB`** : Set this to the TCL library including path for linking

Example using ActiveTcl 8.6
```
TCL_INCLUDE: C:\ActiveTcl\include
TCL_LIB: C:\ActiveTcl\lib\tcl86t.lib
```

### 3.2.2 Instructions for using the Examples with other compilers

If you do not have access to Visual C++ you will have to set up project files / Makefiles for your chosen compiler. There is a section in each of the language modules detailing what needs setting up using Visual C++ which may be of some guidance. Alternatively you may want to use Cygwin as described in the following section.

## 3.3 Building swig.exe on Windows

The SWIG distribution provides a pre-built swig.exe and so it is not necessary for users to build the SWIG executable. However, this section is provided for those that want to modify the SWIG source code in a Windows environment. Normally this is not needed, so most people will want to ignore this section.

There are various ways to build the SWIG executable including CMake which is able to generate project files for building withVisual Studio MSVC.
SWIG can also be compiled and run using MSYS2 with MinGW-w64, Cygwin or MinGW, all of which provide a Unix like front end to Windows and comes free with the GCC C/C++ compiler.
SWIG can also be compiled with MSYS2 using MSVC and SWIG MSVC wrapper.

### 3.3.1 Building swig.exe using CMake

SWIG can be built using CMake and Visual Studio rather than autotools. As with the other approaches to building SWIG the dependencies need to be installed. The steps below are one of a number of ways of installing the dependencies without requiring Cygwin or MinGW. For fully working build steps always check the Continuous Integration (CI) setups currently detailed in the GitHub Actions YAML file.

1. Install Nuget from https://www.nuget.org/downloads (v6.0.0 is used in this example, and installed to `C:\Tools`). Nuget is the package manager for .NET, but allows us to easily install CMake and other dependencies required by SWIG.
2. Install CMake-win64 Nuget package using the following command:

   ```
   C:\Tools\nuget install CMake-win64 -Version 3.15.5 -OutputDirectory C:\Tools\CMake
   ```

   Using PowerShell the equivalent syntax is:

   ```
   & "C:\Tools\nuget" install CMake-win64 -Version 3.15.5 -OutputDirectory C:\Tools\CMake
   ```

   Alternatively you can download CMake from https://cmake.org/download/.
3. Install the Bison Nuget package using the following command:

   ```
   C:\Tools\nuget install Bison -Version 3.7.4 -OutputDirectory C:\Tools\bison
   ```

   Alternatively download Bison from https://sourceforge.net/projects/winflexbison/files/ (Bison 3.7.4 is used in this example) and save to a folder e.g. `C:\Tools\Bison`
4. Install the PCRE2 Nuget package using the following command:

   ```
   C:\Tools\nuget install PCRE2 -Version 10.39 -OutputDirectory C:\Tools\pcre2
   ```

   Note this is a x64 build, if this is not suitable PCRE2 can be built from source using https://github.com/PhilipHazel/pcre2/. Alternatively, set `WITH_PCRE=OFF` to disable PCRE2 support if you are sure you do not require it.
5. We will also need the SWIG source code. Either download a zipped archive from GitHub, or if git is installed clone the latest codebase using:

   ```
   git clone https://github.com/swig/swig.git
   ```

   In this example we are assuming the source code is available at `C:\swig`

6. Now we have all the required dependencies we can build SWIG using PowerShell and the commands below. We are assuming Visual Studio 2019 is installed. For other versions of Visual Studio change `"Visual Studio 16 2019 -A x64"` to the relevant Visual Studio Generator and architecture. We add the required build tools to the system PATH, and then build a Release version of SWIG. If all runs successfully a new swig.exe should be generated in the `C:/swig/install2/bin` folder.

```
cd C:\swig

$env:PATH="C:\Tools\CMake\CMake-win64.3.15.5\bin;C:\Tools\bison\Bison.3.7.4\bin;" + $env:PATH
$PCRE_ROOT="C:\Tools\pcre2\PCRE2.10.39.0"

cmake -G "Visual Studio 16 2019" -A "x64" `
-DCMAKE_INSTALL_PREFIX="C:/swig/install2" `
-DCMAKE_C_FLAGS="/DPCRE2_STATIC" `
-DCMAKE_CXX_FLAGS="/DPCRE2_STATIC" `
-DPCRE2_INCLUDE_DIR="$PCRE_ROOT/include" `
-DPCRE2_LIBRARY="$PCRE_ROOT/lib/pcre2-8-static.lib" `
-S . -B build

cmake --build build --config Release
```

```
cmake --install build --config Release

# to test the exe built correctly
cd install2/bin
./swig.exe -version
./swig.exe -help
```

In addition to Release builds you can create a Debug build using:

```
cmake --build build --config Debug
```

A Visual Studio solution file should be generated named swig.sln. This can be opened and debugged by running the swig project and setting `Properties > Debugging > Command Arguments`. For example to debug one of the test-suite .i files included with the SWIG source use the following:

```
-python -c++ -o C:\Temp\doxygen_parsing.cpp C:\swig\Examples\test-suite\doxygen_parsing.i
```

### 3.3.2 Building swig.exe using MSYS2 and MinGW-w64

Download and install MSYS2 from www.msys2.org (tested with version msys2-x86_64-20201109). Launch the MSYS2 shell.

Install the packages needed to build swig:

```
pacman -S git autoconf automake bison gcc make pcre2-devel
```

Clone the repository to /usr/src/:

```
mkdir /usr/src/
cd /usr/src/
git clone https://github.com/swig/swig.git
```

Configure and build:

```
cd /usr/src/swig
./autogen.sh
./configure
make
```

Finally you may also want to install SWIG:

```
make install
```

### 3.3.3 Building swig.exe using MinGW and MSYS

Warning: These instructions were added in 2006 and have barely changed since so are unlikely to work exactly as written.

The short abbreviated instructions follow...

- Install MinGW and MSYS from the MinGW site. This provides a Unix environment on Windows.
- Follow the usual Unix instructions in the README file in the SWIG root directory to build swig.exe from the MinGW command prompt.

The step by step instructions to download and install MinGW and MSYS, then download and build the latest version of SWIG from Github follow... Note that the instructions for obtaining SWIG from Github are also online at SWIG Bleeding Edge.

**Pitfall note:** Execute the steps in the order shown and don't use spaces in path names. In fact it is best to use the default installation directories.

1. Download the following packages from the MinGW download page. Note that at the time of writing, the majority of these are in the Current release list and some are in the Snapshot or Previous release list.
   - MinGW-3.1.0-1.exe
   - MSYS-1.0.11-2004.04.30-1.exe
   - msysDTK-1.0.1.exe
   - bison-2.0-MSYS.tar.gz
   - msys-autoconf-2.59.tar.bz2
   - msys-automake-1.8.2.tar.bz2
2. Install MinGW-3.1.0-1.exe (C:\MinGW is default location.)
3. Install MSYS-1.0.11-2004.04.30-1.exe. Make sure you install it on the same windows drive letter as MinGW (C:\msys\1.0 is default). In the post install script,
   - Answer y to the "do you wish to continue with the post install?"
   - Answer y to the "do you have MinGW installed?"
   - Type in the folder in which you installed MinGW (C:/MinGW is default)
4. Install msysDTK-1.0.1.exe to the same folder that you installed MSYS (C:\msys\1.0 is default).
5. Copy the following to the MSYS install folder (C:\msys\1.0 is default):
   - msys-automake-1.8.2.tar.bz2
   - msys-autoconf-2.59.tar.bz2
   - bison-2.0-MSYS.tar.gz
6. Start the MSYS command prompt and execute:

   ```
   cd /
   tar -jxf msys-automake-1.8.2.tar.bz2
   tar -jxf msys-autoconf-2.59.tar.bz2
   tar -zxf bison-2.0-MSYS.tar.gz
   ```

7. The very latest development version of SWIG is available from SWIG on Github and can be downloaded as a zip file or if you have Git installed, via Git. Either download the latest Zip file snapshot and unzip and rename the top level folder to /usr/src/swig. Otherwise if using Git, type in the following:

   ```
   mkdir /usr/src
   cd /usr/src
   git clone https://github.com/swig/swig.git
   ```

   **Pitfall note:** If you want to place SWIG in a different folder to the proposed /usr/src/swig, do not use MSYS emulated windows drive letters, because the autotools will fail miserably on those.
8. The PCRE2 third party library needs to be built next. Download the latest PCRE2 source tarball, such as `pcre2-10.39.tar.bz2`, from www.pcre.org and place in the `/usr/src/swig` directory. Build PCRE2 as a static library using the Tools/pcre-build.sh script as follows:

```
cd /usr/src/swig
Tools/pcre-build.sh
```

9. You are now ready to build SWIG. Execute the following commands to build swig.exe:

```
cd /usr/src/swig
./autogen.sh
./configure
make
```

### 3.3.4 Building swig.exe using Cygwin

Note that SWIG can also be built using Cygwin. However, SWIG will then require the Cygwin DLL when executing. Follow the Unix instructions in the README file in the SWIG root directory. Note that the Cygwin environment will also allow one to regenerate the autotool generated files which are supplied with the release distribution. These files are generated using the `autogen.sh` script and will only need regenerating in circumstances such as changing the build system.

#### 3.3.4.1 Running the examples on Windows using Cygwin

The examples and test-suite work as successfully on Cygwin as on any other Unix operating system. The modules which are known to work are Python, Tcl, Perl, Ruby, Java and C#. Follow the Unix instructions in the README file in the SWIG root directory to build the examples.

## 3.4 Microsoft extensions and other Windows quirks

### 3.4.1 Visual C++ standards compliance

The Visual C++ compiler (MSVC) has a long history of not being standards compliant, but this has been getting better over the years.

SWIG's approach for C++ is to generate standards compliant C++98 code along with enhancements for later standards if the C++ compiler supports a later standard. Unfortunately by default, in 2024, Visual C++ still does not set the `__cplusplus` macro correctly to pick up these later C++ standard features. MSVC users are urged to ensure this macro is defined correctly by consulting the latest Microsoft Visual C++ documentation, in particular, /std and /Zc:__cplusplus.

### 3.4.2 Calling conventions

A common problem when using SWIG on Windows are the Microsoft function calling conventions which are not in the C++ standard. SWIG parses ISO C/C++ so cannot deal with proprietary conventions such as `__declspec(dllimport)`, `__stdcall` etc. There is a Windows interface file, `windows.i`, to deal with these calling conventions though. The file also contains typemaps for handling commonly used Windows specific types such as `__int64`, `BOOL` , `DWORD` etc. Include it like you would any other interface file, for example:

```
%include <windows.i>

__declspec(dllexport) ULONG __stdcall foo(DWORD, __int32);
```

Note that if you follow Microsoft's recommendation of wrapping the `__declspec` calls in a preprocessor definition, you will need to make sure that the definition is included by SWIG as well, by either defining it manually or via a header. For example, if you have specified the preprocessor definition in a header named `export_lib.h` and include other headers which depend on it, you should use the `%include` directive to include the definition explicitly. For example, if you had a header file, `bar.h`, which depended on `export_lib.h`, your SWIG definition file might look like:

```
// bar.i
%module bar
%include <windows.i>
%include "export_lib.h"
%include "bar.h"
```

where export_lib.h may contain:

```
// export_lib.h
#define BAR_API __declspec(dllexport)
```

and bar.h may look like:

```
// bar.h
#include "export_lib.h"
BAR_API void bar_function(int, double);
```

Using the preprocessor to remove BAR_API is a popular simpler solution:

```
// bar.i
%module bar
#define BAR_API
%include "bar.h"
```

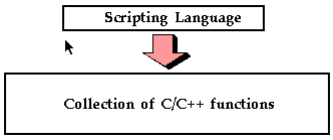
# 4 Scripting Languages


- The two language view of the world
- How does a scripting language talk to C?
  - Wrapper functions
  - Variable linking
  - Constants
  - Structures and classes
  - Proxy classes
- Building scripting language extensions
  - Shared libraries and dynamic loading
  - Linking with shared libraries
  - Static linking


This chapter provides a brief overview of scripting language extension programming and the mechanisms by which scripting language interpreters access C and C++ code.

## 4.1 The two language view of the world

When a scripting language is used to control a C program, the resulting system tends to look as follows:




In this programming model, the scripting language interpreter is used for high level control whereas the underlying functionality of the C/C++ program is accessed through special scripting language "commands." If you have ever tried to write your own simple command interpreter, you might view the scripting language approach to be a highly advanced implementation of that. Likewise, If you have ever used a package such as MATLAB or IDL, it is a very similar model--the interpreter executes user commands and scripts. However, most of the underlying functionality is written in a low-level language like C or Fortran.

The two-language model of computing is extremely powerful because it exploits the strengths of each language. C/C++ can be used for maximal performance and complicated systems programming tasks. Scripting languages can be used for rapid prototyping, interactive debugging, scripting, and access to high-level data structures such associative arrays.

## 4.2 How does a scripting language talk to C?

Scripting languages are built around a parser that knows how to execute commands and scripts. Within this parser, there is a mechanism for executing commands and accessing variables. Normally, this is used to implement the builtin features of the language. However, by extending the interpreter, it is usually possible to add new commands and variables. To do this, most languages define a special API for adding new commands. Furthermore, a special foreign function interface defines how these new commands are supposed to hook into the interpreter.

Typically, when you add a new command to a scripting interpreter you need to do two things; first you need to write a special "wrapper" function that serves as the glue between the interpreter and the underlying C function. Then you need to give the interpreter information about the wrapper by providing details about the name of the function, arguments, and so forth. The next few sections illustrate the process.

### 4.2.1 Wrapper functions

Suppose you have an ordinary C function like this :

```
int fact(int n) {
  if (n <= 1)
    return 1;
  else
    return n*fact(n-1);
}
```

In order to access this function from a scripting language, it is necessary to write a special "wrapper" function that serves as the glue between the scripting language and the underlying C function. A wrapper function must do three things :

- Gather function arguments and make sure they are valid.
- Call the C function.
- Convert the return value into a form recognized by the scripting language.

As an example, the Tcl wrapper function for the `fact()` function above example might look like the following :

```
int wrap_fact(ClientData clientData, Tcl_Interp *interp, int argc, char *argv[]) {
  int result;
  int arg0;
  if (argc != 2) {
    interp->result = "wrong # args";
    return TCL_ERROR;
  }
  arg0 = atoi(argv[1]);
  result = fact(arg0);
  sprintf(interp->result, "%d", result);
  return TCL_OK;
}
```

Once you have created a wrapper function, the final step is to tell the scripting language about the new function. This is usually done in an initialization function called by the language when the module is loaded. For example, adding the above function to the Tcl interpreter requires code like the following :

```
int Wrap_Init(Tcl_Interp *interp) {
  Tcl_CreateCommand(interp, "fact", wrap_fact, (ClientData) NULL,
                    (Tcl_CmdDeleteProc *) NULL);
  return TCL_OK;
}
```

When executed, Tcl will now have a new command called "`fact` " that you can use like any other Tcl command.

Although the process of adding a new function to Tcl has been illustrated, the procedure is almost identical for Perl and Python. Both require special wrappers to be written and both need additional initialization code. Only the specific details are different.

### 4.2.2 Variable linking

Variable linking refers to the problem of mapping a C/C++ global variable to a variable in the scripting language interpreter. For example, suppose you had the following variable:

```
double Foo = 3.5;
```

It might be nice to access it from a script as follows (shown for Perl):

```
$a = $Foo * 2.3;   # Evaluation
$Foo = $a + 2.0;   # Assignment
```

To provide such access, variables are commonly manipulated using a pair of get/set functions. For example, whenever the value of a variable is read, a "get" function is invoked. Similarly, whenever the value of a variable is changed, a "set" function is called.

In many languages, calls to the get/set functions can be attached to evaluation and assignment operators. Therefore, evaluating a variable such as `$Foo` might implicitly call the get function. Similarly, typing `$Foo = 4` would call the underlying set function to change the value.

### 4.2.3 Constants

In many cases, a C program or library may define a large collection of constants. For example:

```
#define RED   0xff0000
#define BLUE  0x0000ff
#define GREEN 0x00ff00
```

To make constants available, their values can be stored in scripting language variables such as `$RED`, `$BLUE`, and `$GREEN`. Virtually all scripting languages provide C functions for creating variables so installing constants is usually a trivial exercise.

### 4.2.4 Structures and classes

Although scripting languages have no trouble accessing simple functions and variables, accessing C/C++ structures and classes present a different problem. This is because the implementation of structures is largely related to the problem of data representation and layout. Furthermore, certain language features are difficult to map to an interpreter. For instance, what does C++ inheritance mean in a Perl interface?

The most straightforward technique for handling structures is to implement a collection of accessor functions that hide the underlying representation of a structure. For example,

```
struct Vector {
  Vector();
  ~Vector();
  double x, y, z;
};
```

can be transformed into the following set of functions :

```
Vector *new_Vector();
void delete_Vector(Vector *v);
double Vector_x_get(Vector *v);
double Vector_y_get(Vector *v);
double Vector_z_get(Vector *v);
void Vector_x_set(Vector *v, double x);
void Vector_y_set(Vector *v, double y);
void Vector_z_set(Vector *v, double z);
```

Now, from an interpreter these function might be used as follows:

```
% set v [new_Vector]
% Vector_x_set $v 3.5
% Vector_y_get $v
% delete_Vector $v
% ...
```

Since accessor functions provide a mechanism for accessing the internals of an object, the interpreter does not need to know anything about the actual representation of a `Vector`.

### 4.2.5 Proxy classes

In certain cases, it is possible to use the low-level accessor functions to create a proxy class, also known as a shadow class. A proxy class is a special kind of object that gets created in a scripting language to access a C/C++ class (or struct) in a way that looks like the original structure (that is, it proxies the real C++ class). For example, if you have the following C++ definition :

```
class Vector {
public:
  Vector();
  ~Vector();
  double x, y, z;
};
```

A proxy classing mechanism would allow you to access the structure in a more natural manner from the interpreter. For example, in Python, you might want to do this:

```
>>> v = Vector()
>>> v.x = 3
>>> v.y = 4
>>> v.z = -13
>>> ...
>>> del v
```

Similarly, in Perl5 you may want the interface to work like this:

```
$v = new Vector;
$v->{x} = 3;
$v->{y} = 4;
$v->{z} = -13;
```

Finally, in Tcl :

```
Vector v
v configure -x 3 -y 4 -z -13
```

When proxy classes are used, two objects are really at work--one in the scripting language, and an underlying C/C++ object. Operations affect both objects equally and for all practical purposes, it appears as if you are simply manipulating a C/C++ object.

## 4.3 Building scripting language extensions

The final step in using a scripting language with your C/C++ application is adding your extensions to the scripting language itself. There are two primary approaches for doing this. The preferred technique is to build a dynamically loadable extension in the form of a shared library. Alternatively, you can recompile the scripting language interpreter with your extensions added to it.

### 4.3.1 Shared libraries and dynamic loading

To create a shared library or DLL, you often need to look at the manual pages for your compiler and linker. However, the procedure for a few common platforms is shown below:

```
# Build a shared library for Solaris
gcc -fPIC -c example.c example_wrap.c -I/usr/local/include
ld -G example.o example_wrap.o -o example.so

# Build a shared library for Linux
gcc -fPIC -c example.c example_wrap.c -I/usr/local/include
gcc -shared example.o example_wrap.o -o example.so
```

To use your shared library, you simply use the corresponding command in the scripting language (load, import, use, etc...). This will import your module and allow you to start using it. For example:

```
% load ./example.so
% fact 4
24
%
```

When working with C++ codes, the process of building shared libraries may be more complicated--primarily due to the fact that C++ modules may need additional code in order to operate correctly. On many machines, you can build a shared C++ module by following the above procedures, but changing the link line to the following :

```
c++ -shared example.o example_wrap.o -o example.so
```

### 4.3.2 Linking with shared libraries

When building extensions as shared libraries, it is not uncommon for your extension to rely upon other shared libraries on your machine. In order for the extension to work, it needs to be able to find all of these libraries at run-time. Otherwise, you may get an error such as the following :

```
>>> import graph
Traceback (innermost last):
  File "<stdin>", line 1, in ?
  File "/home/sci/data1/beazley/graph/graph.py", line 2, in ?
    import graphc
ImportError:  1101:/home/sci/data1/beazley/bin/python: rld: Fatal Error: cannot
successfully map soname 'libgraph.so' under any of the filenames /usr/lib/libgraph.so:/
lib/libgraph.so:/lib/cmplrs/cc/libgraph.so:/usr/lib/cmplrs/cc/libgraph.so:
>>>
```

What this error means is that the extension module created by SWIG depends upon a shared library called "`libgraph.so`" that the system was unable to locate. To fix this problem, there are a few approaches you can take.

- Link your extension and explicitly tell the linker where the required libraries are located. Often times, this can be done with a special linker flag such as `-R`, `-rpath`, etc. This is not implemented in a standard manner so read the man pages for your linker to find out more about how to set the search path for shared libraries.
- Put shared libraries in the same directory as the executable. This technique is sometimes required for correct operation on non-Unix platforms.
- Set the UNIX environment variable `LD_LIBRARY_PATH` to the directory where shared libraries are located before running Python. Although this is an easy solution, it is not recommended. Consider setting the path using linker options instead.

### 4.3.3 Static linking

With static linking, you rebuild the scripting language interpreter with extensions. The process usually involves compiling a short main program that adds your customized commands to the language and starts the interpreter. You then link your program with a library to produce a new scripting language executable.

Although static linking is supported on all platforms, this is not the preferred technique for building scripting language extensions. In fact, there are very few practical reasons for doing this--consider using shared libraries instead.

# 5 SWIG Basics


- Running SWIG
  - Input format
  - SWIG output
  - Comments
  - C Preprocessor
  - SWIG directives
  - Parser limitations
  - Parse tree
- Wrapping simple C declarations
  - Basic type handling
  - Global variables
  - Constants
  - A brief word about const
  - A cautionary tale of char *
- Pointers and complex objects
  - Simple pointers
  - Run time pointer type checking
  - Derived types, structs, and classes
  - Undefined datatypes
  - Typedef
- Other Practicalities
  - Passing structures by value
  - Return by value
  - Linking to structure variables
  - Linking to char *
  - Arrays

- Creating read-only variables
- Renaming and ignoring declarations
  - Simple renaming of specific identifiers
  - Ignoring identifiers
  - Advanced renaming support
  - Limiting global renaming rules
  - Ignoring everything then wrapping a few selected symbols
- Default/optional arguments
- Pointers to functions and callbacks

- Structures and unions
  - Typedef and structures
  - Character strings and structures
  - Array members
  - Structure data members
  - C constructors and destructors
  - Adding member functions to C structures
  - Nested structures
  - Other things to note about structure wrapping
- Code Insertion
  - The output of SWIG
  - Code insertion blocks
  - Inlined code blocks
  - Initialization blocks
- An Interface Building Strategy
  - Preparing a C program for SWIG
  - The SWIG interface file
  - Why use separate interface files?
  - Getting the right header files
  - What to do with main()


This chapter describes the basic operation of SWIG, the structure of its input files, and how it handles standard ISO C declarations. C++ support is described in the next chapter. However, C++ programmers should still read this chapter to understand the basics. Specific details about each target language are described in later chapters.

## 5.1 Running SWIG

To run SWIG, use the `swig` command with options and a filename like this:

```
swig [ options ] filename
```

where `filename` is a SWIG interface file or a C/C++ header file. Full help can be seen by running `swig -help`. Below is the common set of options that can be used. Additional options are also defined for each target language. A full list can be obtained by running `swig -<lang> -help` for language *<lang>* specific options, for example, `swig -ruby -help` for Ruby.

```
Supported Target Language Options
     -csharp         - Generate C# wrappers
     -d              - Generate D wrappers
     -go             - Generate Go wrappers
     -guile          - Generate Guile wrappers
     -java           - Generate Java wrappers
     -javascript     - Generate Javascript wrappers
     -lua            - Generate Lua wrappers
     -octave         - Generate Octave wrappers
     -perl5          - Generate Perl 5 wrappers
     -php7           - Generate PHP 8 or later wrappers
     -python         - Generate Python wrappers
     -r              - Generate R (aka GNU S) wrappers
     -ruby           - Generate Ruby wrappers
     -scilab         - Generate Scilab wrappers
     -tcl8           - Generate Tcl 8 wrappers
     -xml            - Generate XML wrappers

Experimental Target Language Options
     -c              - Generate C wrappers
     -ocaml          - Generate OCaml wrappers

Deprecated Target Language Options
     -mzscheme       - Generate MzScheme/Racket wrappers

General Options
     -addextern      - Add extra extern declarations
     -c++            - Enable C++ processing
     -co <file>      - Check <file> out of the SWIG library
     -copyctor       - Automatically generate copy constructors wherever possible
     -cpperraswarn   - Treat the preprocessor #error statement as #warning (default)
     -cppext <ext>   - Change file extension of generated C++ files to <ext>
                       (default is cxx)
     -copyright      - Display copyright notices
     -debug-classes  - Display information about the classes found in the interface
     -debug-module <n>- Display module parse tree at stages 1-4, <n> is a csv list of stages
     -debug-symtabs  - Display symbol tables information
     -debug-symbols  - Display target language symbols in the symbol tables
     -debug-csymbols - Display C symbols in the symbol tables
     -debug-lsymbols - Display target language layer symbols
     -debug-quiet    - Display less parse tree node debug info when using other -debug options
     -debug-tags     - Display information about the tags found in the interface
     -debug-template - Display information for debugging templates
     -debug-top <n>  - Display entire parse tree at stages 1-4, <n> is a csv list of stages
     -debug-typedef  - Display information about the types and typedefs in the interface
     -debug-typemap  - Display typemap debugging information
     -debug-tmsearch - Display typemap search debugging information
     -debug-tmused   - Display typemaps used debugging information
     -directors      - Turn on director mode for all the classes, mainly for testing
     -dirprot        - Turn on wrapping of protected members for director classes (default)
     -D<symbol>[=<value>] - Define symbol <symbol> (for conditional compilation)
     -E              - Preprocess only, does not generate wrapper code
     -external-runtime [file] - Export the SWIG runtime stack
     -fakeversion <v>- Make SWIG fake the program version number to <v>
     -fcompact       - Compile in compact mode
```

```
     -features <list>- Set global features, where <list> is a comma separated list of
                       features, eg -features directors,autodoc=1
                       If no explicit value is given to the feature, a default of 1 is used
     -fastdispatch   - Enable fast dispatch mode to produce faster overload dispatcher code
     -Fmicrosoft     - Display error/warning messages in Microsoft format
     -Fstandard      - Display error/warning messages in commonly used format
     -fvirtual       - Compile in virtual elimination mode
     -help           - Display help
     -I-             - Don't search the current directory
     -I<dir>         - Look for SWIG files in directory <dir>
     -ignoremissing  - Ignore missing include files
     -importall      - Follow all #include statements as imports
     -includeall     - Follow all #include statements
     -l<ifile>       - Include SWIG library file <ifile>
     -macroerrors    - Report errors inside macros
     -M              - List all dependencies
     -MD             - Is equivalent to `-M -MF <file>', except `-E' is not implied
     -MF <file>      - Generate dependencies into <file> and continue generating wrappers
     -MM             - List dependencies, but omit files in SWIG library
     -MMD            - Like `-MD', but omit files in SWIG library
     -module <name>  - Set module name to <name>
     -MP             - Generate phony targets for all dependencies
     -MT <target>    - Set the target of the rule emitted by dependency generation
     -nocontract     - Turn off contract checking
     -nocpperraswarn - Do not treat the preprocessor #error statement as #warning
     -nodefaultctor  - Do not generate implicit default constructors
     -nodefaultdtor  - Do not generate implicit default destructors
     -nodirprot      - Do not wrap director protected members
     -noexcept       - Do not wrap exception specifiers
     -nofastdispatch - Disable fast dispatch mode (default)
     -nopreprocess   - Skip the preprocessor step
     -notemplatereduce - Disable reduction of the typedefs in templates
     -O              - Enable the optimization options:
                        -fastdispatch -fvirtual
     -o <outfile>    - Set name of C/C++ output file to <outfile>
     -oh <headfile>  - Set name of C++ output header file for directors to <headfile>
     -outcurrentdir  - Set default output dir to current dir instead of input file's path
     -outdir <dir>   - Set language specific files output directory to <dir>
     -pcreversion    - Display PCRE2 version information
     -small          - Compile in virtual elimination and compact mode
     -std=<standard> - Set the C or C++ language <standard> for inputs
     -swiglib        - Report location of SWIG library and exit
     -templatereduce - Reduce all the typedefs in templates
     -U<symbol>      - Undefine symbol <symbol>
     -v              - Run in verbose mode
     -version        - Display SWIG version number
     -Wall           - Remove all warning suppression, also implies -Wextra
     -Wallkw         - Enable keyword warnings for all the supported languages
     -Werror         - Treat warnings as errors
     -Wextra         - Adds the following additional warnings: 309,403,405,512,321,322
     -w<list>        - Suppress/add warning messages, eg -w401,+321 - see Warnings.html
     -xmlout <file>  - Write XML version of the parse tree to <file> after normal processing
```

Arguments may also be passed in a command-line options file (also known as a response file) which is useful if they exceed the system command line length limit. To do this, put the arguments in a file, then provide the file name prefixed with `@` like so:

```
swig @file
```

The options read from the file are inserted in place of the file option. If the file does not exist, or cannot be read, then the option will be treated literally and not removed.

Options in the file are separated by whitespace. A whitespace character may be included in an option by surrounding the entire option in either single or double quotes. Any character (including a backslash) may be included by prefixing the character to be included with a backslash. The file may itself contain additional `@file` options; any such options will be processed recursively.

### 5.1.1 Input format

As input, SWIG expects a file containing ISO C/C++ declarations and special SWIG directives. More often than not, this is a special SWIG interface file which is usually denoted with a special `.i` or `.swg` suffix. In certain cases, SWIG can be used directly on raw header files or source files. However, this is not the most typical case and there are several reasons why you might not want to do this (described later).

The most common format of a SWIG interface is as follows:

```
%module mymodule
%{
#include "myheader.h"
%}
// Now list ISO C/C++ declarations
int foo;
int bar(int x);
...
```

The module name is supplied using the special `%module` directive. Modules are described further in the Modules Introduction section.

Everything in the `%{ ... %}` block is simply copied verbatim to the resulting wrapper file created by SWIG. This section is almost always used to include header files and other declarations that are required to make the generated wrapper code compile. It is important to emphasize that just because you include a declaration in a SWIG input file, that declaration does *not* automatically appear in the generated wrapper code---therefore you need to make sure you include the proper header files in the `%{ ... %}` section. It should be noted that the text enclosed in `%{ ... %}` is not parsed or interpreted by SWIG. The `%{...%}` syntax and semantics in SWIG is analogous to that of the declarations section used in input files to parser generation tools such as yacc or bison.

### 5.1.2 SWIG output

The output of SWIG is a C/C++ file that contains all of the wrapper code needed to build an extension module. SWIG may generate some additional files depending on the target language. By default, an input file with the name `file.i` is transformed into a file `file_wrap.c` or `file_wrap.cxx` (depending on whether or not the `-c++` option has been used). The name of the output C/C++ file can be changed using the `-o` option. In certain cases, file suffixes are used by the compiler to determine the source language (C, C++, etc.). Therefore, you have to use the `-o` option to change the suffix of the SWIG-generated wrapper file if you want something different than the default. For example:

```
$ swig -c++ -python -o example_wrap.cpp example.i
```

The C/C++ output file created by SWIG often contains everything that is needed to construct an extension module for the target scripting language. SWIG is not a stub compiler nor is it usually necessary to edit the output file (and if you look at the output, you probably won't want to). To build the final extension module, the SWIG output file is compiled and linked with the rest of your C/C++ program to create a shared library.

For many target languages SWIG will also generate proxy class files in the target language. The default output directory for these language specific files is the same directory as the generated C/C++ file. This can be modified using the `-outdir` option. For example:

```
$ swig -c++ -python -outdir pyfiles -o cppfiles/example_wrap.cpp example.i
```

If the directories `cppfiles` and `pyfiles` exist, the following will be generated:

```
cppfiles/example_wrap.cpp
pyfiles/example.py
```

If the `-outcurrentdir` option is used (without `-o`) then SWIG behaves like a typical C/C++ compiler and the default output directory is then the current directory. Without this option the default output directory is the path to the input file. If `-o` and `-outcurrentdir` are used together, `-outcurrentdir` is effectively ignored as the output directory for the language files is the same directory as the generated C/C++ file if not overridden with `-outdir`.

### 5.1.3 Comments

C and C++ style comments may appear anywhere in interface files. In previous versions of SWIG, comments were used to generate documentation files. Doxygen comments can now be used to generate target language specific documentation, see the Doxygen chapter.

### 5.1.4 C Preprocessor

Like C, SWIG preprocesses all input files through an enhanced version of the C preprocessor. All standard preprocessor features are supported including file inclusion, conditional compilation and macros. However, `#include` statements are ignored unless the `-includeall` command line option has been supplied. The reason for disabling includes is that SWIG is sometimes used to process raw C header files. In this case, you usually only want the extension module to include functions in the supplied header file rather than everything that might be included by that header file (i.e., system headers, C library functions, etc.).

It should also be noted that the SWIG preprocessor skips all text enclosed inside a `%{...%}` block. In addition, the preprocessor includes a number of macro handling enhancements that make it more powerful than the normal C preprocessor. These extensions are described in the "Preprocessor" chapter.

### 5.1.5 SWIG directives

Most of SWIG's operation is controlled by special directives that are always preceded by a "`%`" to distinguish them from normal C declarations. These directives are used to give SWIG hints or to alter SWIG's parsing behavior in some manner.

Since SWIG directives are not legal C syntax, it is generally not possible to include them in header files. However, SWIG directives can be included in C header files using conditional compilation like this:

```
/* header.h  --- Some header file */

/* SWIG directives -- only seen if SWIG is running */
#ifdef SWIG
%module foo
#endif
```

`SWIG` is a special preprocessing symbol defined by SWIG when it is parsing an input file.

### 5.1.6 Parser limitations

Although SWIG can parse most C/C++ declarations, it does not provide a complete C/C++ parser implementation. Most of these limitations pertain to very complicated type declarations and certain advanced C++ features. Specifically, the following features are not currently supported:

- Non-conventional type declarations. For example, SWIG does not support declarations such as the following (even though this is legal C):

  ```
  /* Non-conventional placement of storage specifier (extern) */
  const int extern Number;

  /* Extra declarator grouping */
  Matrix (foo);    // A global variable

  /* Extra declarator grouping in parameters */
  void bar(Spam (Grok)(Doh));
  ```

  In practice, few (if any) C programmers actually write code like this since this style is never featured in programming books. However, if you're feeling particularly obfuscated, you can certainly break SWIG (although why would you want to?).

- Running SWIG on C++ source files (the code in a .C, .cpp or .cxx file) is not recommended. The usual approach is to feed SWIG header files for parsing C++ definitions and declarations. The main reason is if SWIG parses a scoped definition or declaration (as is normal for C++ source files), it is ignored, unless a declaration for the symbol was parsed earlier. For example

  ```
  /* bar not wrapped unless foo has been defined and
     the declaration of bar within foo has already been parsed */
  int foo::bar(int) {
    ... whatever ...
  }
  ```

- Certain advanced features of C++ such as nested classes are not yet fully supported. Please see the C++ Nested classes section for more information.

In the event of a parsing error, conditional compilation can be used to skip offending code. For example:

```
#ifndef SWIG
... some bad declarations ...
#endif
```

Alternatively, you can just delete the offending code from the interface file.

One of the reasons why SWIG does not provide a full C++ parser implementation is that it has been designed to work with incomplete specifications and to be very permissive in its handling of C/C++ datatypes (e.g., SWIG can generate interfaces even when there are missing class declarations or opaque datatypes). Unfortunately, this approach makes it extremely difficult to implement certain parts of a C/C++ parser as most compilers use type information to assist in the parsing of more complex declarations (for the truly curious, the primary complication in the implementation is that the SWIG parser does not utilize a separate *typedef-name* terminal symbol as described on p. 234 of K&R).

### 5.1.7 Parse tree

SWIG can dump its parse tree in either a simple text or XML format. This can be useful for advanced development or debugging purposes. Details are covered in the developer documentation Extending SWIG chapter.

## 5.2 Wrapping simple C declarations

SWIG wraps simple C declarations by creating an interface that closely matches the way in which the declarations would be used in a C program. For example, consider the following interface file:

```
%module example

%inline %{
extern double sin(double x);
extern int strcmp(const char *, const char *);
extern int Foo;
%}
#define STATUS 50
#define VERSION "1.1"
```

In this file, there are two functions `sin()` and `strcmp()`, a global variable `Foo`, and two constants `STATUS` and `VERSION`. When SWIG creates an extension module, these declarations are accessible as scripting language functions, variables, and constants respectively. For example, in Tcl:

```
% sin 3
5.2335956
% strcmp Dave Mike
-1
% puts $Foo
42
% puts $STATUS
50
% puts $VERSION
1.1
```

Or in Python:

```
>>> example.sin(3)
5.2335956
>>> example.strcmp('Dave', 'Mike')
-1
>>> print example.cvar.Foo
42
>>> print example.STATUS
50
>>> print example.VERSION
1.1
```

Whenever possible, SWIG creates an interface that closely matches the underlying C/C++ code. However, due to subtle differences between languages, run-time environments, and semantics, it is not always possible to do so. The next few sections describe various aspects of this mapping.

### 5.2.1 Basic type handling

In order to build an interface, SWIG has to convert C/C++ datatypes to equivalent types in the target language. Generally, scripting languages provide a more limited set of primitive types than C. Therefore, this conversion process involves a certain amount of type coercion.

Most scripting languages provide a single integer type that is implemented using the `int` or `long` datatype in C. The following list shows all of the C datatypes that SWIG will convert to and from integers in the target language:

```
int
short
long
unsigned
signed
unsigned short
unsigned long
unsigned char
signed char
bool
```

When an integral value is converted from C, a cast is used to convert it to the representation in the target language. Thus, a 16 bit short in C may be promoted to a 32 bit integer. When integers are converted in the other direction, the value is cast back into the original C type. If the value is too large to fit, it is silently truncated.

`unsigned char` and `signed char` are special cases that are handled as small 8-bit integers. Normally, the `char` datatype is mapped as a one-character ASCII string.

The `bool` datatype is cast to and from an integer value of 0 and 1 unless the target language provides a special boolean type.

Some care is required when working with large integer values. Most scripting languages use 32-bit integers so mapping a 64-bit long integer may lead to truncation errors. Similar problems may arise with 32 bit unsigned integers (which may appear as large negative numbers). As a rule of thumb, the `int` datatype and all variations of`char` and `short` datatypes are safe to use. For `unsigned int` and `long` datatypes, you will need to carefully check the correct operation of your program after it has been wrapped with SWIG.

Although the SWIG parser supports the `long long` datatype, not all language modules support it. This is because `long long` usually exceeds the integer precision available in the target language. In certain modules such as Tcl and Perl5, `long long` integers are encoded as strings. This allows the full range of these numbers to be represented. However, it does not allow `long long` values to be used in arithmetic expressions. It should also be noted that although `long long` is part of the ISO C99 standard, it is not universally supported by all C compilers. Make sure you are using a compiler that supports `long long` before trying to use this type with SWIG.

SWIG recognizes the following floating point types :

```
float
```

```
double
```

Floating point numbers are mapped to and from the natural representation of floats in the target language. This is almost always a C `double`. The rarely used datatype of `long double` is not supported by SWIG.

The `char` datatype is mapped into a NULL terminated ASCII string with a single character. When used in a scripting language it shows up as a tiny string containing the character value. When converting the value back into C, SWIG takes a character string from the scripting language and strips off the first character as the char value. Thus if the value "foo" is assigned to a `char` datatype, it gets the value `f'.

The `char *` datatype is handled as a NULL-terminated ASCII string. SWIG maps this into a 8-bit character string in the target scripting language. SWIG converts character strings in the target language to NULL terminated strings before passing them into C/C++. The default handling of these strings does not allow them to have embedded NULL bytes. Therefore, the `char *` datatype is not generally suitable for passing binary data. However, it is possible to change this behavior by defining a SWIG typemap. See the chapter on Typemaps for details about this.

At this time, SWIG provides limited support for Unicode and wide-character strings (the C `wchar_t` type). Some languages provide typemaps for wchar_t, but bear in mind these might not be portable across different operating systems. This is a delicate topic that is poorly understood by many programmers and not implemented in a consistent manner across languages. For those scripting languages that provide Unicode support, Unicode strings are often available in an 8-bit representation such as UTF-8 that can be mapped to the `char *` type (in which case the SWIG interface will probably work). If the program you are wrapping uses Unicode, there is no guarantee that Unicode characters in the target language will use the same internal representation (e.g., UCS-2 vs. UCS-4). You may need to write some special conversion functions.

### 5.2.2 Global variables

Whenever possible, SWIG maps C/C++ global variables into scripting language variables. For example,

```
%module example
double foo;
```

results in a scripting language variable like this:

```
# Tcl
set foo [3.5]                    ;# Set foo to 3.5
puts $foo                        ;# Print the value of foo

# Python
cvar.foo = 3.5                   # Set foo to 3.5
print(cvar.foo)                  # Print value of foo

# Perl
$foo = 3.5;                      # Set foo to 3.5
print $foo, "\n";                # Print value of foo

# Ruby
Module.foo = 3.5                # Set foo to 3.5
print Module.foo, "\n"          # Print value of foo
```

Whenever the scripting language variable is used, the underlying C global variable is accessed. Although SWIG makes every attempt to make global variables work like scripting language variables, it is not always possible to do so. For instance, in Python, all global variables must be accessed through a special variable object known as `cvar` (shown above). In Ruby, variables are accessed as attributes of the module. Other languages may convert variables to a pair of accessor functions. For example, the Java module generates a pair of functions `double get_foo()` and `set_foo(double val)` that are used to manipulate the value.

Finally, if a global variable has been declared as `const`, it only supports read-only access. Note: this behavior is new to SWIG-1.3. Earlier versions of SWIG incorrectly handled `const` and created constants instead.

### 5.2.3 Constants

Constants can be created using `#define`, enumerations, or a special `%constant` directive. The following interface file shows a few valid constant declarations :

```
#define I_CONST       5              // An integer constant
#define PI            3.14159        // A Floating point constant
#define S_CONST       "hello world"  // A string constant
#define NEWLINE       '\n'           // Character constant

enum boolean {NO=0, YES=1};
enum months {JAN, FEB, MAR, APR, MAY, JUN, JUL, AUG,
             SEP, OCT, NOV, DEC};
%constant double BLAH = 42.37;
#define PI_4 PI/4
#define FLAGS 0x04 | 0x08 | 0x40
```

In `#define` declarations, the type of a constant is inferred by syntax. For example, a number with a decimal point is assumed to be floating point. In addition, SWIG must be able to fully resolve all of the symbols used in a `#define` in order for a constant to actually be created. This restriction is necessary because `#define` is also used to define preprocessor macros that are definitely not meant to be part of the scripting language interface. For example:

```
#define EXTERN extern

EXTERN void foo();
```

In this case, you probably don't want to create a constant called `EXTERN` (what would the value be?). In general, SWIG will not create constants for macros unless the value can be completely determined by the preprocessor. For instance, in the above example, the declaration

```
#define PI_4  PI/4
```

defines a constant because `PI` was already defined as a constant and the value is known. However, for the same conservative reasons even a constant with a simple cast will be ignored, such as

```
#define F_CONST (double) 5           // A floating point constant with cast
```

This logic can lead to false attempts at converting `#define` into `%constant` though. For example the following case does not have any undefined symbols within the macro:

```
// For indicating pure virtual functions such as: virtual void f() PURE;
#define PURE = 0
```

A warning is issued:

```
pure.h:1: Warning 305: Bad constant value (ignored).
```

In such cases simply ignore the warning or suppress it using the normal warning suppression techniques.

The use of constant expressions is allowed, but SWIG does not evaluate them. Rather, it passes them through to the output file and lets the C compiler perform the final evaluation (SWIG does perform a limited form of type-checking however).

For enumerations, it is critical that the original enum definition be included somewhere in the interface file (either in a header file or in the `%{ %}` block). SWIG only translates the enumeration into code needed to add the constants to a scripting language. It needs the original enumeration declaration in order to get the correct enum values as assigned by the C compiler.

The `%constant` directive is used to more precisely create constants corresponding to different C datatypes. Although it is not usually needed for simple values, it is more useful when working with pointers and other more complex datatypes. Typically, `%constant` is only used when you want to add constants to the scripting language interface that are not defined in the original header file.

### 5.2.4 A brief word about const

A common confusion with C programming is the semantic meaning of the `const` qualifier in declarations--especially when it is mixed with pointers and other type modifiers. In fact, previous versions of SWIG handled `const` incorrectly--a situation that SWIG-1.3.7 and newer releases have fixed.

Starting with SWIG-1.3, all variable declarations, regardless of any use of `const`, are wrapped as global variables. If a declaration happens to be declared as `const`, it is wrapped as a read-only variable. To tell if a variable is `const` or not, you need to look at the right-most occurrence of the `const` qualifier (that appears before the variable name). If the right-most `const` occurs after all other type modifiers (such as pointers), then the variable is `const`. Otherwise, it is not.

Here are some examples of `const` declarations.

```
const char a;           // A constant character
char const b;           // A constant character (the same)
char *const c;          // A constant pointer to a character
const char *const d;    // A constant pointer to a constant character
```

Here is an example of a declaration that is not `const`:

```
const char *e;          // A pointer to a constant character.  The pointer
                        // may be modified.
```

In this case, the pointer `e` can change---it's only the value being pointed to that is read-only.

Please note that for const parameters or return types used in a function, SWIG pretty much ignores the fact that these are const, see the section on const-correctness for more information.

**Compatibility Note:** One reason for changing SWIG to handle `const` declarations as read-only variables is that there are many situations where the value of a `const` variable might change. For example, a library might export a symbol as `const` in its public API to discourage modification, but still allow the value to change through some other kind of internal mechanism. Furthermore, programmers often overlook the fact that with a constant declaration like `char *const`, the underlying data being pointed to can be modified--it's only the pointer itself that is constant. In an embedded system, a `const` declaration might refer to a read-only memory address such as the location of a memory-mapped I/O device port (where the value changes, but writing to the port is not supported by the hardware). Rather than trying to build a bunch of special cases into the `const` qualifier, the new interpretation of `const` as "read-only" is simple and exactly matches the actual semantics of `const` in C/C++. If you really want to create a constant as in older versions of SWIG, use the `%constant` directive instead. For example:

```
%constant double PI = 3.14159;
```

or

```
#ifdef SWIG
#define const %constant
#endif
const double foo = 3.4;
const double bar = 23.4;
const int    spam = 42;
#ifdef SWIG
#undef const
#endif
...
```

### 5.2.5 A cautionary tale of char *

Before going any further, there is one bit of caution involving `char *` that must now be mentioned. When strings are passed from a scripting language to a C `char *`, the pointer usually points to string data stored inside the interpreter. It is almost always a really bad idea to modify this data. Furthermore, some languages may explicitly disallow it. For instance, in Python, strings are supposed to be immutable. If you violate this, you will probably receive a vast amount of wrath when you unleash your module on the world.

The primary source of problems are functions that might modify string data in place. A classic example would be a function like this:

```
char *strcat(char *s, const char *t)
```

Although SWIG will certainly generate a wrapper for this, its behavior will be undefined. In fact, it will probably cause your application to crash with a segmentation fault or other memory related problem. This is because `s` refers to some internal data in the target language---data that you shouldn't be touching.

The bottom line: don't rely on `char *` for anything other than read-only input values. However, it must be noted that you could change the behavior of SWIG using typemaps.

## 5.3 Pointers and complex objects

Most C programs manipulate arrays, structures, and other types of objects. This section discusses the handling of these datatypes.

### 5.3.1 Simple pointers

Pointers to primitive C datatypes such as

```
int *
double ***
char **
```

are fully supported by SWIG. Rather than trying to convert the data being pointed to into a scripting representation, SWIG simply encodes the pointer itself into a representation that contains the actual value of the pointer and a type-tag. Thus, the SWIG representation of the above pointers (in Tcl), might look like this:

```
_10081012_p_int
_1008e124_ppp_double
_f8ac_pp_char
```

A NULL pointer is represented by the string "NULL" or the value 0 encoded with type information.

All pointers are treated as opaque objects by SWIG. Thus, a pointer may be returned by a function and passed around to other C functions as needed. For all practical purposes, the scripting language interface works in exactly the same way as you would use the pointer in a C program. The only difference is that there is no mechanism for dereferencing the pointer since this would require the target language to understand the memory layout of the underlying object.

The scripting language representation of a pointer value should never be manipulated directly. Even though the values shown look like hexadecimal addresses, the numbers used may differ from the actual machine address (e.g., on little-endian machines, the digits may appear in reverse order). Furthermore, SWIG does not normally map pointers into high-level objects such as associative arrays or lists (for example, converting an `int *` into an list of integers). There are several reasons why SWIG does not do this:

- There is not enough information in a C declaration to properly map pointers into higher level constructs. For example, an `int *` may indeed be an array of integers, but if it contains ten million elements, converting it into a list object is probably a bad idea.
- The underlying semantics associated with a pointer is not known by SWIG. For instance, an `int *` might not be an array at all--perhaps it is an output value!
- By handling all pointers in a consistent manner, the implementation of SWIG is greatly simplified and less prone to error.

### 5.3.2 Run time pointer type checking

By allowing pointers to be manipulated from a scripting language, extension modules effectively bypass compile-time type checking in the C/C++ compiler. To prevent errors, a type signature is encoded into all pointer values and is used to perform run-time type checking. This type-checking process is an integral part of SWIG and can not be disabled or modified without using typemaps (described in later chapters).

Like C, `void *` matches any kind of pointer. Furthermore,`NULL` pointers can be passed to any function that expects to receive a pointer. Although this has the potential to cause a crash, `NULL` pointers are also sometimes used as sentinel values or to denote a missing/empty value. Therefore, SWIG leaves NULL pointer checking up to the application.

### 5.3.3 Derived types, structs, and classes

For everything else (structs, classes, arrays, etc...) SWIG applies a very simple rule :

**Everything else is a pointer**

In other words, SWIG manipulates everything else by reference. This model makes sense because most C/C++ programs make heavy use of pointers and SWIG can use the type-checked pointer mechanism already present for handling pointers to basic datatypes.

Although this probably sounds complicated, it's really quite simple. Suppose you have an interface file like this :

```
%module fileio
FILE *fopen(char *, char *);
int fclose(FILE *);
unsigned fread(void *ptr, unsigned size, unsigned nobj, FILE *);
unsigned fwrite(void *ptr, unsigned size, unsigned nobj, FILE *);
void *malloc(int nbytes);
void free(void *);
```

In this file, SWIG doesn't know what a `FILE` is, but since it's used as a pointer, so it doesn't really matter what it is. If you wrapped this module into Python, you can use the functions just like you expect :

```
# Copy a file
def filecopy(source, target):
    f1 = fopen(source, "r")
    f2 = fopen(target, "w")
    buffer = malloc(8192)
    nbytes = fread(buffer, 8192, 1, f1)
    while (nbytes > 0):
        fwrite(buffer, 8192, 1, f2)
            nbytes = fread(buffer, 8192, 1, f1)
    free(buffer)
```

In this case `f1`, `f2`, and `buffer` are all opaque objects containing C pointers. It doesn't matter what value they contain--our program works just fine without this knowledge.

### 5.3.4 Undefined datatypes

When SWIG encounters an undeclared datatype, it automatically assumes that it is a structure or class. For example, suppose the following function appeared in a SWIG input file:

```
void matrix_multiply(Matrix *a, Matrix *b, Matrix *c);
```

SWIG has no idea what a "`Matrix`" is. However, it is obviously a pointer to something so SWIG generates a wrapper using its generic pointer handling code.

Unlike C or C++, SWIG does not actually care whether `Matrix` has been previously defined in the interface file or not. This allows SWIG to generate interfaces from only partial or limited information. In some cases, you may not care what a `Matrix` really is as long as you can pass an opaque reference to one around in the scripting language interface.

An important detail to mention is that SWIG will gladly generate wrappers for an interface when there are unspecified type names. However, **all unspecified types are internally handled as pointers to structures or classes!** For example, consider the following declaration:

```
void foo(size_t num);
```

If `size_t` is undeclared, SWIG generates wrappers that expect to receive a type of `size_t *` (this mapping is described shortly). As a result, the scripting interface might behave strangely. For example:

```
foo(40);
TypeError: expected a _p_size_t.
```

The only way to fix this problem is to make sure you properly declare type names using `typedef`.

### 5.3.5 Typedef

Like C, `typedef` can be used to define new type names in SWIG. For example:

```
typedef unsigned int size_t;
```

`typedef` definitions appearing in a SWIG interface are not propagated to the generated wrapper code. Therefore, they either need to be defined in an included header file or placed in the declarations section like this:

```
%{
/* Include in the generated wrapper file */
typedef unsigned int size_t;
%}
/* Tell SWIG about it */
typedef unsigned int size_t;
```

or

```
%inline %{
typedef unsigned int size_t;
%}
```

In certain cases, you might be able to include other header files to collect type information. For example:

```
%module example
%import "sys/types.h"
```

In this case, you might run SWIG as follows:

```
$ swig -I/usr/include -includeall example.i
```

It should be noted that your mileage will vary greatly here. System headers are notoriously complicated and may rely upon a variety of non-standard C coding extensions (e.g., such as special directives to GCC). Unless you exactly specify the right include directories and preprocessor symbols, this may not work correctly (you will have to experiment).

SWIG tracks `typedef` declarations and uses this information for run-time type checking. For instance, if you use the above `typedef` and had the following function declaration:

```
void foo(unsigned int *ptr);
```

The corresponding wrapper function will accept arguments of type `unsigned int *` or `size_t *`.

## 5.4 Other Practicalities

So far, this chapter has presented almost everything you need to know to use SWIG for simple interfaces. However, some C programs use idioms that are somewhat more difficult to map to a scripting language interface. This section describes some of these issues.

### 5.4.1 Passing structures by value

Sometimes a C function takes structure parameters that are passed by value. For example, consider the following function:

```
double dot_product(Vector a, Vector b);
```

To deal with this, SWIG transforms the function to use pointers by creating a wrapper equivalent to the following:

```
double wrap_dot_product(Vector *a, Vector *b) {
  Vector x = *a;
  Vector y = *b;
  return dot_product(x, y);
}
```

In the target language, the `dot_product()` function now accepts pointers to Vectors instead of Vectors. For the most part, this transformation is transparent so you might not notice.

### 5.4.2 Return by value

C functions that return structures or classes datatypes by value are more difficult to handle. Consider the following function:

```
Vector cross_product(Vector v1, Vector v2);
```

This function wants to return `Vector`, but SWIG only really supports pointers. As a result, SWIG creates a wrapper like this:

```
Vector *wrap_cross_product(Vector *v1, Vector *v2) {
  Vector x = *v1;
  Vector y = *v2;
  Vector *result;
  result = (Vector *) malloc(sizeof(Vector));
  *(result) = cross_product(x, y);
  return result;
}
```

or if SWIG was run with the `-c++` option:

```
Vector *wrap_cross_product(Vector *v1, Vector *v2) {
  Vector x = *v1;
  Vector y = *v2;
  Vector *result = new Vector(cross_product(x, y)); // Uses default copy or move constructor
  return result;
}
```

In both cases, SWIG allocates a new object and returns a reference to it. It is up to the user to delete the returned object when it is no longer in use. Clearly, this will leak memory if you are unaware of the implicit memory allocation and don't take steps to free the result. That said, it should be noted that some language modules can now automatically track newly created objects and reclaim memory for you. Consult the documentation for each language module for more details.

It should also be noted that the handling of pass/return by value in C++ has some special cases. For example, the above code fragments don't work correctly if `Vector` doesn't define a default constructor. The section on SWIG and C++ has more information about this case.

### 5.4.3 Linking to structure variables

When global variables or class members involving structures are encountered, SWIG handles them as pointers. For example, a global variable like this

```
Vector unit_i;
```

gets mapped to an underlying pair of set/get functions like this :

```
Vector *unit_i_get() {
  return &unit_i;
}
void unit_i_set(Vector *value) {
  unit_i = *value;
}
```

Again some caution is in order. A global variable created in this manner will show up as a pointer in the target scripting language. It would be an extremely bad idea to free or destroy such a pointer. Also, C++ classes must supply a properly defined copy constructor in order for assignment to work correctly.

### 5.4.4 Linking to char *

When a global variable of type `char *` appears, SWIG uses `malloc()` or `new` to allocate memory for the new value. Specifically, if you have a variable like this

```
char *foo;
```

SWIG generates the following code:

```
/* C mode */
void foo_set(char *value) {
  free(foo);
  foo = (char *) malloc(strlen(value)+1);
  strcpy(foo, value);
}

/* C++ mode.  When -c++ option is used */
void foo_set(char *value) {
  delete [] foo;
  foo = new char[strlen(value)+1];
  strcpy(foo, value);
}
```

If this is not the behavior that you want, consider making the variable read-only using the `%immutable` directive. Alternatively, you might write a short assist-function to set the value exactly like you want. For example:

```
%inline %{
  void set_foo(char *value) {
    strncpy(foo, value, 50);
  }
%}
```

Note: If you write an assist function like this, you will have to call it as a function from the target scripting language (it does not work like a variable). For example, in Python you will have to write:

```
>>> set_foo("Hello World")
```

A common mistake with `char *` variables is to link to a variable declared like this:

```
char *VERSION = "1.0";
```

In this case, the variable will be readable, but any attempt to change the value results in a segmentation or general protection fault. This is due to the fact that SWIG is trying to release the old value using `free` or `delete` when the string literal value currently assigned to the variable wasn't allocated using `malloc()` or `new`. To fix this behavior, you can either mark the variable as read-only, write a typemap (as described in Chapter 6), or write a special set function as shown. Another alternative is to declare the variable as an array:

```
char VERSION[64] = "1.0";
```

When variables of type `const char *` are declared, SWIG still generates functions for setting and getting the value. However, the default behavior does *not* release the previous contents (resulting in a possible memory leak). In fact, you may get a warning message such as this when wrapping such a variable:

```
example.i:20. Typemap warning. Setting const char * variable may leak memory
```

The reason for this behavior is that `const char *` variables are often used to point to string literals. For example:

```
const char *foo = "Hello World\n";
```

Therefore, it's a really bad idea to call `free()` on such a pointer. On the other hand, it *is* legal to change the pointer to point to some other value. When setting a variable of this type, SWIG allocates a new string (using malloc or new) and changes the pointer to point to the new value. However, repeated modifications of the value will result in a memory leak since the old value is not released.

### 5.4.5 Arrays

Arrays are fully supported by SWIG, but they are always handled as pointers instead of mapping them to a special array object or list in the target language. Thus, the following declarations :

```
int foobar(int a[40]);
void grok(char *argv[]);
void transpose(double a[20][20]);
```

are processed as if they were really declared like this:

```
int foobar(int *a);
void grok(char **argv);
void transpose(double (*a)[20]);
```

Like C, SWIG does not perform array bounds checking. It is up to the user to make sure the pointer points to a suitably allocated region of memory.

Multi-dimensional arrays are transformed into a pointer to an array of one less dimension. For example:

```
int [10];         // Maps to int *
int [10][20];     // Maps to int (*)[20]
int [10][20][30]; // Maps to int (*)[20][30]
```

It is important to note that in the C type system, a multidimensional array `a[][]` is **NOT** equivalent to a single pointer `*a` or a double pointer such as `**a`. Instead, a pointer to an array is used (as shown above) where the actual value of the pointer is the starting memory location of the array. The reader is strongly advised to dust off their C book and re-read the section on arrays before using them with SWIG.

Array variables are supported, but are read-only by default. For example:

```
int   a[100][200];
```

In this case, reading the variable 'a' returns a pointer of type `int (*)[200]` that points to the first element of the array`&a[0][0]`. Trying to modify 'a' results in an error. This is because SWIG does not know how to copy data from the target language into the array. To work around this limitation, you may want to write a few simple assist functions like this:

```
%inline %{
void a_set(int i, int j, int val) {
  a[i][j] = val;
}
int a_get(int i, int j) {
  return a[i][j];
}
%}
```

To dynamically create arrays of various sizes and shapes, it may be useful to write some helper functions in your interface. For example:

```
// Some array helpers
%inline %{
  /* Create any sort of [size] array */
  int *int_array(int size) {
    return (int *) malloc(size*sizeof(int));
  }
  /* Create a two-dimension array [size][10] */
  int (*int_array_10(int size))[10] {
    return (int (*)[10]) malloc(size*10*sizeof(int));
  }
%}
```

Arrays of `char` are handled as a special case by SWIG. In this case, strings in the target language can be stored in the array. For example, if you have a declaration like this,

```
char pathname[256];
```

SWIG generates functions for both getting and setting the value that are equivalent to the following code:

```
char *pathname_get() {
  return pathname;
}
void pathname_set(char *value) {
  strncpy(pathname, value, 256);
}
```

In the target language, the value can be set like a normal variable.

### 5.4.6 Creating read-only variables

A read-only variable can be created by using the `%immutable` directive as shown :

```
// File : interface.i

int a;       // Can read/write
%immutable;
int b, c, d;   // Read only variables
```

```
%mutable;
double x, y;  // read/write
```

The `%immutable` directive enables read-only mode until it is explicitly disabled using the `%mutable` directive. As an alternative to turning read-only mode off and on like this, individual declarations can also be tagged as immutable. For example:

```
%immutable x;                   // Make x read-only
...
double x;                       // Read-only (from earlier %immutable directive)
double y;                       // Read-write
...
```

The `%mutable` and `%immutable` directives are actually %feature directives defined like this:

```
#define %immutable   %feature("immutable")
#define %mutable     %feature("immutable", "")
```

If you wanted to make all wrapped variables read-only, barring one or two, it might be easier to take this approach:

```
%immutable;                     // Make all variables read-only
%feature("immutable", "0") x;   // except, make x read/write
...
double x;
double y;
double z;
...
```

Read-only variables are also created when declarations are declared as `const`. For example:

```
const int foo;               /* Read only variable */
char * const version="1.0";  /* Read only variable */
```

SWIG will also create read-only variables for non-assignable types, more details in the Member data section in the C++ chapter.

### 5.4.7 Renaming and ignoring declarations

#### 5.4.7.1 Simple renaming of specific identifiers

Normally, the name of a C declaration is used when that declaration is wrapped into the target language. However, this may generate a conflict with a keyword or already existing function in the scripting language. To resolve a name conflict, you can use the `%rename` directive as shown :

```
// interface.i

%rename(my_print) print;
extern void print(const char *);

%rename(foo) a_really_long_and_annoying_name;
extern int a_really_long_and_annoying_name;
```

SWIG still calls the correct C function, but in this case the function `print()` will really be called "`my_print()`" in the target language.

The placement of the `%rename` directive is arbitrary as long as it appears before the declarations to be renamed. A common technique is to write code for wrapping a header file like this:

```
// interface.i

%rename(my_print) print;
%rename(foo) a_really_long_and_annoying_name;

%include "header.h"
```

`%rename` applies a renaming operation to all future occurrences of a name. The renaming applies to functions, variables, class and structure names, member functions, and member data. For example, if you had two-dozen C++ classes, all with a member function named `print' (which is a keyword in Python), you could rename them all to `output' by specifying :

```
%rename(output) print; // Rename all `print' functions to `output'
```

A new `%rename` for the same name will replace the current`%rename` for all uses after it in the file, and setting the new name to "" will remove the rename. So, for instance, if you wanted to rename some things in one file and not in another, you could do:

```
 %rename(print1) print;
 %include "header1.h" //Anything "print" in here will become "print1"
 %rename(print2) print;
 %include "header2.h" //Anything "print" in here will become "print2"
 %rename("") print;
 %include "header3.h" //Anything "print" in here will remain "print"
```

SWIG does not normally perform any checks to see if the functions it wraps are already defined in the target scripting language. However, if you are careful about namespaces and your use of modules, you can usually avoid these problems.

When wrapping C code, simple use of identifiers/symbols with `%rename` usually suffices. When wrapping C++ code, simple use of simple identifiers/symbols with `%rename` might be too limiting when using C++ features such as function overloading, default arguments, namespaces, template specialization etc. If you are using the `%rename` directive and C++, make sure you read the SWIG and C++ chapter and in particular the section on Renaming and ambiguity resolution for method overloading and default arguments.

#### 5.4.7.2 Ignoring identifiers

Closely related to `%rename` is the `%ignore` directive. `%ignore` instructs SWIG to ignore declarations that match a given identifier. For example:

```
%ignore print;         // Ignore all declarations named print
%ignore MYMACRO;       // Ignore a macro
...
#define MYMACRO 123
void print(const char *);
...
```

Any function, variable etc which matches `%ignore` will not be wrapped and therefore will not be available from the target language. A common usage of `%ignore` is to selectively remove certain declarations from a header file without having to add conditional compilation to the header. However, it should be stressed that this only works for simple declarations. If you need to remove a whole section of problematic code, the SWIG preprocessor should be used instead.

#### 5.4.7.3 Advanced renaming support

While writing `%rename` for specific declarations is simple enough, sometimes the same renaming rule needs to be applied to many, maybe all, identifiers in the SWIG input. For example, it may be necessary to apply some transformation to all the names in the target language to better follow its naming conventions, like adding a specific prefix to all wrapped functions. Doing it individually for each function is impractical so SWIG supports applying a renaming rule to all declarations if the name of the identifier to be renamed is not specified:

```
%rename("myprefix_%s") ""; // print -> myprefix_print
```

This also shows that the argument of `%rename` doesn't have to be a literal string but can be a `printf()`-like format string. In the simplest form, `"%s"` is replaced with the name of the original declaration, as shown above. However this is not always enough and SWIG provides extensions to the usual format string syntax to allow applying a (SWIG-defined) function to the argument. For example, to wrap all C functions `do_something_long()` as more Java-like `doSomethingLong()` you can use the `"lowercamelcase"` extended format specifier like this:

```
%rename("%(lowercamelcase)s") ""; // foo_bar -> fooBar; FooBar -> fooBar
```

Some functions can be parametrized, for example the `"strip"` one strips the provided prefix from its argument. The prefix is specified as part of the format string, following a colon after the function name:

```
%rename("%(strip:[wx])s") ""; // wxHello -> Hello; FooBar -> FooBar
```

Below is the table summarizing all currently defined functions with an example of applying each one. Note that some of them have two names, a shorter one and a more descriptive one, but the two functions are otherwise equivalent:

| Function | Returns | Example (in/out) | |
|---|---|---|---|
| `uppercase` or `upper` | Upper case version of the string. | `Print` | `PRINT` |
| `lowercase` or `lower` | Lower case version of the string. | `Print` | `print` |
| `title` | String with first letter capitalized and the rest in lower case. | `print` | `Print` |
| `firstuppercase` | String with the first letter capitalized and the rest unchanged. | `printIt` | `PrintIt` |
| `firstlowercase` | String with the first letter in lower case and the rest unchanged. | `PrintIt` | `printIt` |
| `camelcase` or `ctitle` | String with capitalized first letter and any letter following an underscore (which are removed in the process) and rest in lower case. | `print_it` | `PrintIt` |
| `lowercamelcase` or `lctitle` | String with every letter following an underscore (which is removed in the process) capitalized and rest, including the first letter, in lower case. | `print_it` | `printIt` |
| `undercase` or `utitle` | Lower case string with underscores inserted before every upper case letter in the original string and any number not at the end of string. Logically, this is the reverse of `camelcase`. | `PrintIt` | `print_it` |
| `schemify` | String with all underscores replaced with dashes, resulting in more Lispers/Schemers-pleasing name. | `print_it` | `print-it` |
| `strip:[prefix]` | String without the given prefix or the original string if it doesn't start with this prefix. Note that square brackets should be used literally, e.g. `%rename("strip:[wx]")` | `wxPrint` | `Print` |
| `rstrip:[suffix]` | String without the given suffix or the original string if it doesn't end with this suffix. Note that square brackets should be used literally, e.g. `%rename("rstrip:[Cls]")` | `PrintCls` | `Print` |
| `regex:/pattern/subst/` | String after (Perl-like) regex substitution operation. This function allows applying arbitrary regular expressions to the identifier names. The *pattern* part is a regular expression in Perl syntax (as supported by the Perl Compatible Regular Expressions) (PCRE2 library) and the *subst* string can contain back-references of the form `\N` where `N` is a digit from 0 to 9, or one of the following escape sequences: `\l`, `\L`, `\u`, `\U` or `\E`. The back-references are replaced with the contents of the corresponding capture group while the escape sequences perform the case conversion in the substitution string: `\l` and `\L` convert to the lower case, while `\u` and `\U` convert to the upper case. The difference between the elements of each pair is that `\l` and `\u` change the case of the next character only, while `\L` and `\U` do it for all the remaining characters or until `\E` is encountered. Finally please notice that backslashes need to be escaped in C strings, so in practice `"\\"` must be used in all these escape sequences. For example, to remove any alphabetic prefix before an underscore and capitalize the remaining part you could use the following directive: `%rename("regex:/(\\w+)_(.*)/\\u\\2/")` | `prefix_print` | `Print` |

The most general function of all of the above ones is the `regex` one. Here are some more examples of its use:

```
// Strip the wx prefix from all identifiers except those starting with wxEVT
%rename("%(regex:/wx(?!EVT)(.*)/\\1/)s") ""; // wxSomeWidget -> SomeWidget
                                             // wxEVT_PAINT -> wxEVT_PAINT

// Apply a rule for renaming the enum elements to avoid the common prefixes
// which are redundant in C#/Java
%rename("%(regex:/^([A-Z][a-z]+)+_(.*)/\\2/)s", %$isenumitem) ""; // Colour_Red -> Red

// Remove all "Set/Get" prefixes.
%rename("%(regex:/^(Set|Get)(.*)/\\2/)s") ""; // SetValue -> Value
                                              // GetValue -> Value
```

As before, everything that was said above about `%rename` also applies to `%ignore`. In fact, the latter is just a special case of the former and ignoring an identifier is the same as renaming it to the special `"$ignore"` value. So the following snippets

```
%ignore print;
```

and

```
%rename("$ignore") print;
```

are exactly equivalent and `%rename` can be used to selectively ignore multiple declarations using the previously described matching possibilities.

#### 5.4.7.4 Limiting global renaming rules

As explained in the previous sections, it is possible to either rename individual declarations or apply a rename rule to all of them at once. In practice, the latter is however rarely appropriate as there are always some exceptions to the general rules. To deal with them, the scope of an unnamed `%rename` can be limited using subsequent `match` parameters. They can be applied to any of the attributes associated by SWIG with the declarations appearing in its input. For example:

```
%rename("foo", match$name="bar") "";
```

can be used to achieve the same effect as the simpler

```
%rename("foo") bar;
```

and so is not very interesting on its own. However `match` can also be applied to the declaration type, for example `match="class"` restricts the match to class declarations only (in C++) and `match="enumitem"` restricts it to the enum elements. SWIG also provides convenience macros for such match expressions, for example

```
%rename("%(title)s", %$isenumitem) "";
// same as:
%rename("%(title)s", match="enumitem") "";
```

will capitalize the names of all the enum elements but not change the case of the other declarations. Similarly, `%$isclass`, `%$isfunction`, `%$isconstructor`, `%$isunion`, `%$istemplate`, and `%$isvariable` can be used. Many other checks are possible and this documentation is not exhaustive, see the "%rename predicates" section in `swig.swg` for the full list of supported match expressions.

A logical not is also possible by using `notmatch`. For example, `notmatch="enumitem"` will restrict the match to all items that are not enum elements. There is also a `%$not` macro which simply expands to "not". Be careful using this as some of the other macros in `swig.swg` are complex expressions and so it will only "notmatch" the first part of the expression.

```
%rename("%(title)s", %$not %$isenumitem) "";
// same as:
%rename("%(title)s", notmatch="enumitem") "";
```

For a comprehensive understanding of how the matching works, the internal parse tree needs to be examined using the command line option: `-debug-module 1 -debug-quiet`. A snippet of the resulting output might be:

```
          +++ destructor ---------------------------------------
          | access       - "public"
          | decl         - "f()."
          | ismember    - "1"
          | name         - "~Shape"
          | storage      - "virtual"
          | sym:name     - "~Shape"
```

Here the node type is a "destructor" and in order to match all destructor nodes, use `match="destructor"`. To match one of the listed attributes in the node, such as when the storage is virtual, use `match$storage="virtual"`. This will match all nodes that have a storage attribute set to "virtual". To match only virtual destructors, combine them and use `match="destructor", match$storage="virtual"` .

While the vast majority of these internal parse tree nodes are unlikely to change from one version of SWIG to the next, **use these matching rules at your own risk** as there are no guarantees that they will not change.

In addition to literally matching some string with `match` you can also use `regexmatch` or `notregexmatch` to match a string against a regular expression. For example, to ignore all functions having "Old" as a suffix you could use

```
%rename("$ignore", regexmatch$name="Old$") "";
```

For simple cases like this, specifying the regular expression for the declaration name directly can be preferable and can also be done using `regextarget`:

```
%rename("$ignore", regextarget=1) "Old$";
```

Notice that the check is done only against the name of the declaration itself, if you need to match the full name of a C++ declaration you must use `fullname` attribute:

```
%rename("$ignore", regextarget=1, fullname=1) "NameSpace::ClassName::.*Old$";
```

As for `notregexmatch`, it restricts the match only to the strings not matching the specified regular expression. So to rename all declarations to lower case except those consisting of capital letters only:

```
%rename("$(lower)s", notregexmatch$name="^[A-Z]+$") "";
```

Finally, variants of `%rename` and `%ignore` directives can be used to help wrap C++ overloaded functions and methods or C++ methods which use default arguments. This is described in the Renaming and ambiguity resolution section in the C++ chapter.

#### 5.4.7.5 Ignoring everything then wrapping a few selected symbols

Using the techniques described above it is possible to ignore everything in a header and then selectively wrap a few chosen methods or classes. For example, consider a header, `myheader.h` which has many classes in it and just the one class called `Star` is wanted within this header, the following approach could be taken:

```
%ignore ""; // Ignore everything

// Unignore chosen class 'Star'
%rename("%s") Star;
```

```
// As the ignore everything will include the constructor, destructor, methods etc
// in the class, these have to be explicitly unignored too:
%rename("%s") Star::Star;
%rename("%s") Star::~Star;
%rename("%s") Star::shine; // named method

%include "myheader.h"

%rename("%s") ""; // Undo the %ignore
```

If `Star` was in the `Galaxy` namespace, you would need to unignore the namespace, too, and add the namespace to all the renames:

```
%rename("%s") Galaxy;
%rename("%s") Galaxy::Star;
%rename("%s") Galaxy::Star::Star;
...
```

Another approach which might be more suitable as it does not require naming all the methods in the chosen class is to begin by ignoring just the classes. This does not add an explicit ignore to any members of the class, so when the chosen class is unignored, all of its methods will be wrapped.

```
%rename($ignore, %$isclass) ""; // Only ignore all classes
%rename("%s") Star; // Unignore 'Star'
%include "myheader.h"
%rename("%s", %$isclass) ""; // Stop ignoring all classes
```

### 5.4.8 Default/optional arguments

SWIG supports default arguments in both C and C++ code. For example:

```
int plot(double x, double y, int color=WHITE);
```

In this case, SWIG generates wrapper code where the default arguments are optional in the target language. For example, this function could be used in Tcl as follows :

```
% plot -3.4 7.5    # Use default value
% plot -3.4 7.5 10 # set color to 10 instead
```

Although the ISO C standard does not allow default arguments, default arguments specified in a SWIG interface work with both C and C++.

**Note:** There is a subtle semantic issue concerning the use of default arguments and the SWIG generated wrapper code. When default arguments are used in C code, the default values are emitted into the wrappers and the function is invoked with a full set of arguments. This is different to when wrapping C++ where an overloaded wrapper method is generated for each defaulted argument. Please refer to the section on default arguments in the C++ chapter for further details.

### 5.4.9 Pointers to functions and callbacks

Occasionally, a C library may include functions that expect to receive pointers to functions--possibly to serve as callbacks. SWIG provides full support for function pointers when the callback functions are defined in C and not in the target language. For example, consider a function like this:

```
int binary_op(int a, int b, int (*op)(int, int));
```

When you first wrap something like this into an extension module, you may find the function to be impossible to use. For instance, in Python:

```
>>> def add(x, y):
...     return x+y
...
>>> binary_op(3, 4, add)
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
TypeError: Type error. Expected _p_f_int_int__int
>>>
```

The reason for this error is that SWIG doesn't know how to map a scripting language function into a C callback. However, existing C functions can be used as arguments provided you install them as constants. One way to do this is to use the `%constant` directive like this:

```
/* Function with a callback */
int binary_op(int a, int b, int (*op)(int, int));

/* Some callback functions */
%constant int add(int, int);
%constant int sub(int, int);
%constant int mul(int, int);
```

In this case, `add`, `sub`, and `mul` become function pointer constants in the target scripting language. This allows you to use them as follows:

```
>>> binary_op(3, 4, add)
7
>>> binary_op(3, 4, mul)
12
>>>
```

Unfortunately, by declaring the callback functions as constants, they are no longer accessible as functions. For example:

```
>>> add(3, 4)
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
TypeError: object is not callable: '_ff020efc_p_f_int_int__int'
```

```
>>>
```

If you want to make a function available as both a callback function and a function, you can use the `%callback` and `%nocallback` directives like this:

```
/* Function with a callback */
int binary_op(int a, int b, int (*op)(int, int));

/* Some callback functions */
%callback("%s_cb");
int add(int, int);
int sub(int, int);
int mul(int, int);
%nocallback;
```

The argument to `%callback` is a printf-style format string that specifies the naming convention for the callback constants (`%s` gets replaced by the function name). The callback mode remains in effect until it is explicitly disabled using `%nocallback`. When you do this, the interface now works as follows:

```
>>> binary_op(3, 4, add_cb)
7
>>> binary_op(3, 4, mul_cb)
12
>>> add(3, 4)
7
>>> mul(3, 4)
12
```

Notice that when the function is used as a callback, special names such as `add_cb` are used instead. To call the function normally, just use the original function name such as `add()`.

SWIG provides a number of extensions to standard C printf formatting that may be useful in this context. For instance, the following variation installs the callbacks as all upper case constants such as `ADD`, `SUB`, and `MUL`:

```
/* Some callback functions */
%callback("%(uppercase)s");
int add(int, int);
int sub(int, int);
int mul(int, int);
%nocallback;
```

A format string of `"%(lowercase)s"` converts all characters to lower case. A string of `"%(title)s"` capitalizes the first character and converts the rest to lower case.

And now, a final note about function pointer support. Although SWIG does not normally allow callback functions to be written in the target language, this can be accomplished with the use of typemaps and other advanced SWIG features. See the Typemaps chapter for more about typemaps and individual target language chapters for more on callbacks. The 'director' feature can be used to make callbacks from C/C++ into the target language, see Callbacks to the target language.

## 5.5 Structures and unions

This section describes the behavior of SWIG when processing ISO C structures and union declarations. Extensions to handle C++ are described in the next section.

ISO C has a separate tag name space in which the names of structures, unions and enumerated types are put, which is separate from the name space for ordinary identifiers (function names, object names, typedef names, enumeration constants). For example, this is valid ISO C because `Foo` the struct tag and `Foo` the function name are in different name spaces:

```
struct Foo {
  int bar;
};

int Foo(void) { return 42; }
```

SWIG doesn't currently implement this separate tag name space and for the above example you'll get:

```
foo.i:5: Warning 302: Redefinition of identifier 'Foo' as Foo(void) ignored,
foo.i:1: Warning 302: previous definition of 'Foo'.
```

In practice this rarely actually causes problems, particular because SWIG has special handling for `typedef` so cases such as this work:

```
typedef struct Foo {
  int bar;
} Foo;
```

If SWIG encounters the definition of a structure or union, it creates a set of accessor functions. Although SWIG does not need structure definitions to build an interface, providing definitions makes it possible to access structure members. The accessor functions generated by SWIG simply take a pointer to an object and allow access to an individual member. For example, the declaration :

```
struct Vector {
  double x, y, z;
};
```

gets transformed into the following set of accessor functions :

```
double Vector_x_get(struct Vector *obj) {
  return obj->x;
}
double Vector_y_get(struct Vector *obj) {
  return obj->y;
}
double Vector_z_get(struct Vector *obj) {
  return obj->z;
```

```
}
void Vector_x_set(struct Vector *obj, double value) {
  obj->x = value;
}
void Vector_y_set(struct Vector *obj, double value) {
  obj->y = value;
}
void Vector_z_set(struct Vector *obj, double value) {
  obj->z = value;
}
```

In addition, SWIG creates default constructor and destructor functions if none are defined in the interface. For example:

```
struct Vector *new_Vector() {
  return (Vector *) calloc(1, sizeof(struct Vector));
}
void delete_Vector(struct Vector *obj) {
  free(obj);
}
```

Using these low-level accessor functions, an object can be minimally manipulated from the target language using code like this:

```
v = new_Vector()
Vector_x_set(v, 2)
Vector_y_set(v, 10)
Vector_z_set(v, -5)
...
delete_Vector(v)
```

However, most of SWIG's language modules also provide a high-level interface that is more convenient. Keep reading.

### 5.5.1 Typedef and structures

SWIG supports the following construct which is quite common in C programs :

```
typedef struct {
  double x, y, z;
} Vector;
```

When encountered, SWIG assumes that the name of the object is `Vector' and creates accessor functions like before. The only difference is that the use of `typedef` allows SWIG to drop the `struct` keyword on its generated code. For example:

```
double Vector_x_get(Vector *obj) {
  return obj->x;
}
```

If two different names are used like this :

```
typedef struct vector_struct {
  double x, y, z;
} Vector;
```

the name `Vector` is used instead of `vector_struct` since this is more typical C programming style. If declarations defined later in the interface use the type `struct vector_struct`, SWIG knows that this is the same as `Vector` and it generates the appropriate type-checking code.

### 5.5.2 Character strings and structures

Structures involving character strings require some care. SWIG assumes that all members of type `char *` have been dynamically allocated using `malloc()` and that they are NULL-terminated ASCII strings. When such a member is modified, the previous contents will be released, and the new contents allocated. For example :

```
%module mymodule
...
struct Foo {
  char *name;
  ...
}
```

This results in the following accessor functions :

```
char *Foo_name_get(Foo *obj) {
  return Foo->name;
}

char *Foo_name_set(Foo *obj, char *c) {
  free(obj->name);
  obj->name = (char *) malloc(strlen(c)+1);
  strcpy(obj->name, c);
  return obj->name;
}
```

If this behavior differs from what you need in your applications, the SWIG "memberin" typemap can be used to change it. See the typemaps chapter for further details.

Note: If the `-c++` option is used, `new` and `delete` are used to perform memory allocation.

### 5.5.3 Array members

Arrays may appear as the members of structures, but they will be read-only. SWIG will write an accessor function that returns the pointer to the first element of the array, but will not write a function to change the contents of the array itself. When this situation is detected, SWIG may generate a warning message such as the following :

```
interface.i:116. Warning. Array member will be read-only
```

To eliminate the warning message, typemaps can be used, but this is discussed in a later chapter. In many cases, the warning message is harmless.

### 5.5.4 Structure data members

Occasionally, a structure will contain data members that are themselves structures. For example:

```
typedef struct Foo {
  int x;
} Foo;

typedef struct Bar {
  int y;
  Foo f;           /* struct member */
} Bar;
```

When a structure member is wrapped, it is handled as a pointer, unless the `%naturalvar` directive is used where it is handled more like a C++ reference (see C++ Member data). The accessors to the member variable as a pointer are effectively wrapped as follows:

```
Foo *Bar_f_get(Bar *b) {
  return &b->f;
}
void Bar_f_set(Bar *b, Foo *value) {
  b->f = *value;
}
```

The reasons for this are somewhat subtle but have to do with the problem of modifying and accessing data inside the data member. For example, suppose you wanted to modify the value of `f.x` of a `Bar` object like this:

```
Bar *b;
b->f.x = 37;
```

Translating this assignment to function calls (as would be used inside the scripting language interface) results in the following code:

```
Bar *b;
Foo_x_set(Bar_f_get(b), 37);
```

In this code, if the `Bar_f_get()` function were to return a `Foo` instead of a `Foo *`, then the resulting modification would be applied to a *copy* of `f` and not the data member `f` itself. Clearly that's not what you want!

It should be noted that this transformation to pointers only occurs if SWIG knows that a data member is a structure or class. For instance, if you had a structure like this,

```
struct Foo {
  WORD   w;
};
```

and nothing was known about `WORD`, then SWIG will generate more normal accessor functions like this:

```
WORD Foo_w_get(Foo *f) {
  return f->w;
}
void Foo_w_set(FOO *f, WORD value) {
  f->w = value;
}
```

If you have accessor methods that you want to use as attributes in the target language, you can make them appear as data members using attributes.i.

**Compatibility Note:** SWIG-1.3.11 and earlier releases transformed all non-primitive member datatypes to pointers. Starting in SWIG-1.3.12, this transformation *only* occurs if a datatype is known to be a structure, class, or union. This is unlikely to break existing code. However, if you need to tell SWIG that an undeclared datatype is really a struct, simply use a forward struct declaration such as `"struct Foo;"`.

### 5.5.5 C constructors and destructors

When wrapping structures, it is generally useful to have a mechanism for creating and destroying objects. If you don't do anything, SWIG will automatically generate functions for creating and destroying objects using `malloc()` and `free()`. Note: the use of`malloc()` only applies when SWIG is used on C code (i.e., when the`-c++` option is *not* supplied on the command line). C++ is handled differently.

If you don't want SWIG to generate default constructors for your interfaces, you can use the `%nodefaultctor` directive or the `-nodefaultctor` command line option. For example:

```
swig -nodefaultctor example.i
```

or

```
%module foo
...
%nodefaultctor;        // Don't create default constructors
... declarations ...
%clearnodefaultctor;   // Re-enable default constructors
```

If you need more precise control, `%nodefaultctor` can selectively target individual structure definitions. For example:

```
%nodefaultctor Foo;      // No default constructor for Foo
...
struct Foo {             // No default constructor generated.
};

struct Bar {             // Default constructor generated.
};
```

Since ignoring the implicit or default destructors most of the time produces memory leaks, SWIG will always try to generate them. If needed, however, you can selectively disable the generation of the default/implicit destructor by using `%nodefaultdtor`

```
%nodefaultdtor Foo; // No default/implicit destructor for Foo
...
struct Foo {              // No default destructor is generated.
};

struct Bar {              // Default destructor generated.
};
```

**Compatibility note:** Prior to SWIG-1.3.7, SWIG did not generate default constructors or destructors unless you explicitly turned them on. However, it appears that most users want to have constructor and destructor functions so it has now been enabled as the default behavior.

**Note:** There is also the `%nodefault` directive, which disables both the default or implicit destructor generation. This could lead to memory leaks across the target languages, and it is highly recommended you don't use them.

### 5.5.6 Adding member functions to C structures

Most languages provide a mechanism for creating classes and supporting object oriented programming. From a C standpoint, object oriented programming really just boils down to the process of attaching functions to structures. These functions normally operate on an instance of the structure (or object). Although there is a natural mapping of C++ to such a scheme, there is no direct mechanism for utilizing it with C code. However, SWIG provides a special `%extend` directive that makes it possible to attach methods to C structures for purposes of building an object oriented interface. Suppose you have a C header file with the following declaration :

```
/* file : vector.h */
...
typedef struct Vector {
  double x, y, z;
} Vector;
```

You can make a `Vector` look a lot like a class by writing a SWIG interface like this:

```
// file : vector.i
%module mymodule
%{
#include "vector.h"
%}

%include "vector.h"          // Just grab original C header file
%extend Vector {             // Attach these functions to struct Vector
  Vector(double x, double y, double z) {
    Vector *v;
    v = (Vector *) malloc(sizeof(Vector));
    v->x = x;
    v->y = y;
    v->z = z;
    return v;
  }
  ~Vector() {
    free($self);
  }
  double magnitude() {
    return sqrt($self->x*$self->x+$self->y*$self->y+$self->z*$self->z);
  }
  void print() {
    printf("Vector [%g, %g, %g]\n", $self->x, $self->y, $self->z);
  }
};
```

Note the usage of the `$self` special variable. Its usage is identical to a C++ 'this' pointer and should be used whenever access to the struct instance is required. Also note that C++ constructor and destructor syntax has been used to simulate a constructor and destructor, even for C code. There is one subtle difference to a normal C++ constructor implementation though and that is although the constructor declaration is as per a normal C++ constructor, the newly constructed object must be returned **as if** the constructor declaration had a return value, a `Vector *` in this case.

Now, when used with proxy classes in Python, you can do things like this :

```
>>> v = Vector(3, 4, 0)                 # Create a new vector
>>> print(v.magnitude())                # Print magnitude
5.0
>>> v.print()                   # Print it out
[ 3, 4, 0 ]
>>> del v                       # Destroy it
```

The `%extend` directive can also be used inside the definition of the Vector structure. For example:

```
// file : vector.i
%module mymodule
%{
#include "vector.h"
%}

typedef struct Vector {
```

```
  double x, y, z;
  %extend {
    Vector(double x, double y, double z) { ... }
    ~Vector() { ... }
    ...
  }
} Vector;
```

Note that `%extend` can be used to access externally written functions provided they follow the naming convention used in this example :

```
/* File : vector.c */
/* Vector methods */
#include "vector.h"
Vector *new_Vector(double x, double y, double z) {
  Vector *v;
  v = (Vector *) malloc(sizeof(Vector));
  v->x = x;
  v->y = y;
  v->z = z;
  return v;
}
void delete_Vector(Vector *v) {
  free(v);
}

double Vector_magnitude(Vector *v) {
  return sqrt(v->x*v->x+v->y*v->y+v->z*v->z);
}

// File : vector.i
// Interface file
%module mymodule
%{
#include "vector.h"
%}

typedef struct Vector {
  double x, y, z;
  %extend {
    Vector(int, int, int); // This calls new_Vector()
    ~Vector();           // This calls delete_Vector()
    double magnitude();  // This will call Vector_magnitude()
    ...
  }
} Vector;
```

You'll also need to use these names if you want to directly call methods added using `%extend` from other C/C++ code.

The name used for %extend should be the name of the struct and not the name of any typedef to the struct. For example:

```
typedef struct Integer {
  int value;
} Int;
%extend Integer { ...  } /* Correct name */
%extend Int { ...  } /* Incorrect name */

struct Float {
  float value;
};
typedef struct Float FloatValue;
%extend Float { ...  } /* Correct name */
%extend FloatValue { ...  } /* Incorrect name */
```

There is one exception to this rule and that is when the struct is anonymously named such as:

```
typedef struct {
  double value;
} Double;
%extend Double { ...  } /* Okay */
```

A little known feature of the `%extend` directive is that it can also be used to add synthesized attributes or to modify the behavior of existing data attributes. For example, suppose you wanted to make `magnitude` a read-only attribute of `Vector` instead of a method. To do this, you might write some code like this:

```
// Add a new attribute to Vector
%extend Vector {
  const double magnitude;
}
// Now supply the implementation of the Vector_magnitude_get function
%{
const double Vector_magnitude_get(Vector *v) {
  return (const double) sqrt(v->x*v->x+v->y*v->y+v->z*v->z);
}
%}
```

Now, for all practical purposes, `magnitude` will appear like an attribute of the object.

A similar technique can also be used to work with data members that you want to process. For example, consider this interface:

```
typedef struct Person {
  char name[50];
  ...
} Person;
```

Say you wanted to ensure `name` was always upper case, you can rewrite the interface as follows to ensure this occurs whenever a name is read or written to:

```
typedef struct Person {
  %extend {
    char name[50];
  }
  ...
} Person;

%{
#include <string.h>
#include <ctype.h>

void make_upper(char *name) {
  char *c;
  for (c = name; *c; ++c)
    *c = (char)toupper((int)*c);
}

/* Specific implementation of set/get functions forcing capitalization */

char *Person_name_get(Person *p) {
  make_upper(p->name);
  return p->name;
}

void Person_name_set(Person *p, char *val) {
  strncpy(p->name, val, 50);
  make_upper(p->name);
}
%}
```

Finally, it should be stressed that even though `%extend` can be used to add new data members, these new members can not require the allocation of additional storage in the object (e.g., their values must be entirely synthesized from existing attributes of the structure or obtained elsewhere).

### 5.5.7 Nested structures

Occasionally, a C program will involve structures like this :

```
typedef struct Object {
  int objtype;
  union {
    int ivalue;
    double dvalue;
    char *strvalue;
    void *ptrvalue;
  } intRep;
} Object;
```

When SWIG encounters this, it performs a structure splitting operation that transforms the declaration into the equivalent of the following:

```
typedef union {
  int ivalue;
  double dvalue;
  char *strvalue;
  void *ptrvalue;
} Object_intRep;

typedef struct Object {
  int objType;
  Object_intRep intRep;
} Object;
```

SWIG will then create an `Object_intRep` structure for use inside the interface file. Accessor functions will be created for both structures. In this case, functions like this would be created :

```
Object_intRep *Object_intRep_get(Object *o) {
  return (Object_intRep *) &o->intRep;
}
int Object_intRep_ivalue_get(Object_intRep *o) {
  return o->ivalue;
}
int Object_intRep_ivalue_set(Object_intRep *o, int value) {
  return (o->ivalue = value);
}
double Object_intRep_dvalue_get(Object_intRep *o) {
  return o->dvalue;
}
... etc ...
```

Although this process is a little hairy, it works like you would expect in the target scripting language--especially when proxy classes are used. For instance, in Perl:

```
# Perl5 script for accessing nested member
$o = CreateObject();                    # Create an object somehow
$o->{intRep}->{ivalue} = 7              # Change value of o.intRep.ivalue
```

If you have a lot of nested structure declarations, it is advisable to double-check them after running SWIG. Although, there is a good chance that they will work, you may have to modify the interface file in certain cases.

Finally, note that nesting is handled differently in C++ mode, see Nested classes.

### 5.5.8 Other things to note about structure wrapping

SWIG doesn't care if the declaration of a structure in a `.i` file exactly matches that used in the underlying C code (except in the case of nested structures). For this reason, there are no problems omitting problematic members or simply omitting the structure definition altogether. If you are happy passing pointers around, this can be done without ever giving SWIG a structure definition.

Starting with SWIG1.3, a number of improvements have been made to SWIG's code generator. Specifically, even though structure access has been described in terms of high-level accessor functions such as this,

```
double Vector_x_get(Vector *v) {
  return v->x;
}
```

most of the generated code is actually inlined directly into wrapper functions. Therefore, no function `Vector_x_get()` actually exists in the generated wrapper file. For example, when creating a Tcl module, the following function is generated instead:

```
static int
_wrap_Vector_x_get(ClientData clientData, Tcl_Interp *interp,
                   int objc, Tcl_Obj *const objv[]) {
  struct Vector *arg1 ;
  double result ;

  if (SWIG_GetArgs(interp, objc, objv, "p:Vector_x_get self ", &arg0,
                   SWIGTYPE_p_Vector) == TCL_ERROR)
    return TCL_ERROR;
  result = (double ) (arg1->x);
  Tcl_SetObjResult(interp, Tcl_NewDoubleObj((double) result));
  return TCL_OK;
}
```

The only exception to this rule are methods defined with `%extend` . In this case, the added code is contained in a separate function.

Finally, it is important to note that most language modules may choose to build a more advanced interface. Although you may never use the low-level interface described here, most of SWIG's language modules use it in some way or another.

## 5.6 Code Insertion

Sometimes it is necessary to insert special code into the resulting wrapper file generated by SWIG. For example, you may want to include additional C code to perform initialization or other operations. There are four common ways to insert code, but it's useful to know how the output of SWIG is structured first.

### 5.6.1 The output of SWIG

When SWIG creates its output C/C++ file, it is broken up into five sections corresponding to runtime code, headers, wrapper functions, and module initialization code (in that order).

- **Begin section**.
  A placeholder for users to put code at the beginning of the C/C++ wrapper file. This is most often used to define preprocessor macros that are used in later sections.
- **Runtime code**.
  This code is internal to SWIG and is used to include type-checking and other support functions that are used by the rest of the module.
- **Header section**.
  This is user-defined support code that has been included by the `%{ ... %}` directive. Usually this consists of header files and other helper functions.
- **Wrapper code**.
  These are the wrappers generated automatically by SWIG.
- **Module initialization**.
  The function generated by SWIG to initialize the module upon loading.

### 5.6.2 Code insertion blocks

The `%insert` directive enables inserting blocks of code into a given section of the generated code. It can be used in one of two ways:

```
%insert("section") "filename"
%insert("section") %{ ... %}
```

The first will dump the contents of the file in the given `filename` into the named `section`. The second inserts the code between the braces into the named `section`. For example, the following adds code into the runtime section:

```
%insert("runtime") %{
  ... code in runtime section ...
%}
```

There are the 5 sections, however, some target languages add in additional sections and some of these result in code being generated into a target language file instead of the C/C++ wrapper file. These are documented when available in the target language chapters. Macros named after the code sections are available as additional directives and these macro directives are normally used instead of `%insert` . For example, `%runtime` is used instead of `%insert("runtime")`. The valid sections and order of the sections in the generated C/C++ wrapper file is as shown:

```
%begin %{
  ... code in begin section ...
%}

%runtime %{
  ... code in runtime section ...
%}

%header %{
  ... code in header section ...
%}

%wrapper %{
  ... code in wrapper section ...
%}

%init %{
  ... code in init section ...
%}
```

The bare `%{ ... %}` directive is a shortcut that is the same as `%header %{ ... %}`.

The `%begin` section is effectively empty as it just contains the SWIG banner by default. This section is provided as a way for users to insert code at the top of the wrapper file before any other code is generated. Everything in a code insertion block is copied verbatim into the output file and is not parsed by SWIG. Most SWIG input files have at least one such block to include header files and support C code. Additional code blocks may be placed anywhere in a SWIG file as needed.

```
%module mymodule
%{
#include "my_header.h"
%}
... Declare functions here
%{

void some_extra_function() {
  ...
}
%}
```

A common use for code blocks is to write "helper" functions. These are functions that are used specifically for the purpose of building an interface, but which are generally not visible to the normal C program. For example :

```
%{
/* Create a new vector */
static Vector *new_Vector() {
  return (Vector *) malloc(sizeof(Vector));
}

%}
// Now wrap it
Vector *new_Vector();
```

### 5.6.3 Inlined code blocks

Since the process of writing helper functions is fairly common, there is a special inlined form of code block that is used as follows :

```
%inline %{
/* Create a new vector */
Vector *new_Vector() {
  return (Vector *) malloc(sizeof(Vector));
}
%}
```

This is the same as writing:

```
%{
/* Create a new vector */
Vector *new_Vector() {
  return (Vector *) malloc(sizeof(Vector));
}
%}

/* Create a new vector */
Vector *new_Vector() {
  return (Vector *) malloc(sizeof(Vector));
}
```

In other words, the `%inline` directive inserts all of the code that follows verbatim into the header portion of an interface file. The code is then parsed by both the SWIG preprocessor and parser. Thus, the above example creates a new command `new_Vector` using only one declaration. Since the code inside an `%inline %{ ... %}` block is given to both the C compiler and SWIG, it is illegal to include any SWIG directives inside a `%{ ... %}` block.

**Note:** The usual SWIG C preprocessor rules apply to code in`%apply` blocks when SWIG parses this code. For example, as mentioned earlier, SWIG's C Preprocessor does not follow `#include` directives by default.

### 5.6.4 Initialization blocks

When code is included in the `%init` section, it is copied directly into the module initialization function. For example, if you needed to perform some extra initialization on module loading, you could write this:

```
%init %{
  init_variables();
%}
```

Please note that some language backends (e.g. C# or Java) don't have any initialization function, hence you should define a global object performing the necessary initialization for them instead:

```
%init %{
  static struct MyInit { MyInit() { init_variables(); } } myInit;
%}
```

## 5.7 An Interface Building Strategy

This section describes the general approach for building interfaces with SWIG. The specifics related to a particular scripting language are found in later chapters.

### 5.7.1 Preparing a C program for SWIG

SWIG doesn't require modifications to your C code, but if you feed it a collection of raw C header files or source code, the results might not be what you expect---in fact, they might be awful. Here's a series of steps you can follow to make an interface for a C program :

- Identify the functions that you want to wrap. It's probably not necessary to access every single function of a C program--thus, a little forethought can dramatically simplify the resulting

scripting language interface. C header files are a particularly good source for finding things to wrap.
- Create a new interface file to describe the scripting language interface to your program.
- Copy the appropriate declarations into the interface file or use SWIG's `%include` directive to process an entire C source/header file.
- Make sure everything in the interface file uses ISO C/C++ syntax.
- Make sure all necessary `typedef' declarations and type-information is available in the interface file. In particular, ensure that the type information is specified in the correct order as required by a C/C++ compiler. Most importantly, define a type before it is used! A C compiler will tell you if the full type information is not available if it is needed, whereas SWIG will usually not warn or error out as it is designed to work without full type information. However, if type information is not specified correctly, the wrappers can be sub-optimal and even result in uncompilable C/C++ code.
- If your program has a main() function, you may need to rename it (read on).
- Run SWIG and compile.

Although this may sound complicated, the process turns out to be fairly easy once you get the hang of it.

In the process of building an interface, SWIG may encounter syntax errors or other problems. The best way to deal with this is to simply copy the offending code into a separate interface file and edit it. However, the SWIG developers have worked very hard to improve the SWIG parser--you should report parsing errors to the swig-devel mailing list or to the SWIG bug tracker.

### 5.7.2 The SWIG interface file

The preferred method of using SWIG is to generate a separate interface file. Suppose you have the following C header file :

```
/* File : header.h */

#include <stdio.h>
#include <math.h>

extern int foo(double);
extern double bar(int, int);
extern void dump(FILE *f);
```

A typical SWIG interface file for this header file would look like the following :

```
/* File : interface.i */
%module mymodule
%{
#include "header.h"
%}
extern int foo(double);
extern double bar(int, int);
extern void dump(FILE *f);
```

Of course, in this case, our header file is pretty simple so we could use a simpler approach and use an interface file like this:

```
/* File : interface.i */
%module mymodule
%{
#include "header.h"
%}
%include "header.h"
```

The main advantage of this approach is minimal maintenance of an interface file for when the header file changes in the future. In more complex projects, an interface file containing numerous `%include` and `#include` statements like this is one of the most common approaches to interface file design due to lower maintenance overhead.

### 5.7.3 Why use separate interface files?

Although SWIG can parse many header files, it is more common to write a special `.i` file defining the interface to a package. There are several reasons why you might want to do this:

- It is rarely necessary to access every single function in a large package. Many C functions might have little or no use in a scripted environment. Therefore, why wrap them?
- Separate interface files provide an opportunity to provide more precise rules about how an interface is to be constructed.
- Interface files can provide more structure and organization.
- SWIG can't parse certain definitions that appear in header files. Having a separate file allows you to eliminate or work around these problems.
- Interface files provide a more precise definition of what the interface is. Users wanting to extend the system can go to the interface file and immediately see what is available without having to dig it out of header files.

### 5.7.4 Getting the right header files

Sometimes, it is necessary to use certain header files in order for the code generated by SWIG to compile properly. Make sure you include certain header files by using a `%{ %}` block like this:

```
%module graphics
%{
#include <GL/gl.h>
#include <GL/glu.h>
%}

// Put the rest of the declarations here
...
```

### 5.7.5 What to do with main()

If your program defines a `main()` function, you may need to get rid of it or rename it in order to use a scripting language. Most scripting languages define their own `main()` procedure that is called instead. `main()` also makes no sense when working with dynamic loading. There are a few approaches to solving the `main()` conflict :

- Get rid of `main()` entirely.
- Rename `main()` to something else. You can do this by compiling your C program with an option like `-Dmain=oldmain`.
- Use conditional compilation to only include`main()` when not using a scripting language.

Getting rid of `main()` may cause potential initialization problems of a program. To handle this problem, you may consider writing a special function called `program_init()` that initializes your program upon startup. This function could then be called either from the scripting language as the first operation, or when the SWIG generated module is loaded.

As a general note, many C programs only use the `main()` function to parse command line options and to set parameters. However, by using a scripting language, you are probably trying to create a program that is more interactive. In many cases, the old `main()` program can be completely replaced by a Perl, Python, or Tcl script.

**Note:** In some cases, you might be inclined to create a scripting language wrapper for `main()`. If you do this, the compilation will probably work and your module might even load correctly. The only trouble is that when you call your `main()` wrapper, you will find that it actually invokes the `main()` of the scripting language interpreter itself! This behavior is a side effect of the

symbol binding mechanism used in the dynamic linker. The bottom line: don't do this.

# 6 SWIG and C++


- Comments on C++ Wrapping
- Approach
- Supported C++ features
- Command line options and compilation
- Proxy classes
  - Construction of proxy classes
  - Resource management in proxies
  - Language specific details
- Simple C++ wrapping
  - Constructors and destructors
  - Default constructors, copy constructors and implicit destructors
  - When constructor wrappers aren't created
  - Copy constructors
  - Member functions
  - Static members
  - Member data
- Protection
- Enums and constants
- Friends
  - Friend classes
  - Friend function definitions
  - Friend function declarations
  - Unqualified friend functions
- References and pointers
- Pass and return by value
- Inheritance
- A brief discussion of multiple inheritance, pointers, and type checking
- Default arguments
- Overloaded functions and methods
  - Dispatch function generation
  - Ambiguity in overloading
  - Renaming and ambiguity resolution
  - Comments on overloading
- Overloaded operators
- Class extension
  - Replacing class methods
- Templates
  - The %template directive
  - Function templates
  - Default template arguments
  - Template base classes
  - Empty template instantiation
  - Template specialization
  - Member templates
  - Scoping and templates
  - More on templates
- Namespaces
  - The nspace feature for namespaces
    - %nspace for mirroring namespace hierarchies
    - %nspacemove for modifying namespace hierarchies
    - More about the nspace feature
- Renaming templated types in namespaces
- Exception specifications
- Exception handling with %catches
- Pointers to Members
- Smart pointers and operator->()
- C++ reference counted objects - ref/unref feature
- Using declarations and inheritance
- Nested classes
- A brief rant about const-correctness
- Callbacks to the target language
  - Introduction to director classes
  - Using directors and target language callbacks
- Where to go for more information


This chapter describes SWIG's support for wrapping C++. It is mostly concerned about C++ as defined by the C++ 98 and 03 standards. For additions to the original C++ standard, please read the SWIG and C++11, SWIG and C++14, SWIG and C++17 and SWIG and C++20 chapters. As a prerequisite, you should first read the chapter SWIG Basics to see how SWIG wraps ISO C. Support for C++ builds upon ISO C wrapping and that material will be useful in understanding this chapter.

## 6.1 Comments on C++ Wrapping

Because of its complexity and the fact that C++ can be difficult to integrate with itself let alone other languages, SWIG only provides support for a subset of C++ features. Fortunately, this is now a rather large subset.

In part, the problem with C++ wrapping is that there is no semantically obvious (or automatic ) way to map many of its advanced features into other languages. As a simple example, consider the problem of wrapping C++ multiple inheritance to a target language with no such support. Similarly, the use of overloaded operators and overloaded functions can be problematic when no such capability exists in a target language.

A more subtle issue with C++ has to do with the way that some C++ programmers think about programming libraries. In the world of SWIG, you are really trying to create binary-level software components for use in other languages. In order for this to work, a "component" has to contain real executable instructions and there has to be some kind of binary linking mechanism for accessing its functionality. In contrast, C++ has increasingly relied upon generic programming and templates for much of its functionality. Although templates are a powerful feature, they are largely orthogonal to the whole notion of binary components and libraries. For example, an STL `vector` does not define any kind of binary object for which SWIG can just create a wrapper. To further complicate matters, these libraries often utilize a lot of behind the scenes magic in which the semantics of seemingly basic operations (e.g., pointer dereferencing, procedure call, etc.) can be changed in dramatic and sometimes non-obvious ways. Although this "magic" may present few problems in a C++-only universe, it greatly complicates the problem of crossing language boundaries and provides many opportunities to shoot yourself in the foot. You will just have to be careful.

## 6.2 Approach

To wrap C++, SWIG uses a layered approach to code generation. At the lowest level, SWIG generates a collection of procedural ISO C style wrappers. These wrappers take care of basic type conversion, type checking, error handling, and other low-level details of the C++ binding. These wrappers are also sufficient to bind C++ into any target language that supports built-in procedures. In some sense, you might view this layer of wrapping as providing a C library interface to C++. On top of the low-level procedural (flattened) interface, SWIG generates proxy classes that provide a natural object-oriented (OO) interface to the underlying code. The proxy classes are typically written in the target language itself. For instance, in Python, a real Python

class is used to provide a wrapper around the underlying C++ object.

It is important to emphasize that SWIG takes a deliberately conservative and non-intrusive approach to C++ wrapping. SWIG does not encapsulate C++ classes inside a special C++ adaptor, it does not rely upon templates, nor does it add in additional C++ inheritance when generating wrappers. The last thing that most C++ programs need is even more compiler magic. Therefore, SWIG tries to maintain a very strict and clean separation between the implementation of your C++ application and the resulting wrapper code. You might say that SWIG has been written to follow the principle of least surprise--it does not play sneaky tricks with the C++ type system, it doesn't mess with your class hierarchies, and it doesn't introduce new semantics. Although this approach might not provide the most seamless integration with C++, it is safe, simple, portable, and debuggable.

This approach results in generated code that uses just the most basic of C and C++ features, no matter how complex the input C++ is. SWIG will not force a newer standard of C++ to be required as the generated code complies with ISO standard C++98. However, there are a few features in later C++ standards that can be advantageous to use. For example, C++11 move semantics are available to provide performance improvements. So in some instances, SWIG will use features from newer C++ standards provided they are available. This is made possible via the `__cplusplus` macro, which is set by the C++ compiler to an appropriate standards defined value. This means that if your compiler is configured to support say the C++11 standard, then there may be some C++11 benefits available in the generated code, such as C++11 move semantics.

Some of this chapter focuses on the low-level procedural interface to C++ that is used as the foundation for all language modules. Keep in mind that the target languages also provide the high-level OO interface via proxy classes. More detailed coverage can be found in the documentation for each target language.

## 6.3 Supported C++ features

SWIG currently supports most C++ features including the following:

- Classes
- Constructors and destructors
- Virtual functions
- Public inheritance (including multiple inheritance)
- Static functions
- Function and method overloading
- Operator overloading for many standard operators
- References
- Templates (including specialization and member templates)
- Pointers to members
- Namespaces
- Default parameters
- Smart pointers

The following C++ features are not currently supported:

- Overloaded versions of certain operators (new, delete, etc.)

As a rule of thumb, SWIG should not be used on raw C++ source files, use header files only.

SWIG's C++ support is an ongoing project so some of these limitations may be lifted in future releases. However, we make no promises. Also, submitting a bug report is a very good way to get problems fixed (wink).

## 6.4 Command line options and compilation

When wrapping C++ code, it is critical that SWIG be called with the `-c++' option. This changes the way a number of critical features such as memory management are handled. It also enables the recognition of C++ keywords. Without the `-c++` flag, SWIG will either issue a warning or a large number of syntax errors if it encounters C++ code in an interface file.

When compiling and linking the resulting wrapper file, it is normal to use the C++ compiler. For example:

```
$ swig -c++ -tcl example.i
$ c++ -fPIC -c example_wrap.cxx
$ c++ example_wrap.o $(OBJS) -o example.so
```

Unfortunately, the process varies slightly on each platform. Make sure you refer to the documentation on each target language for further details. The SWIG Wiki also has further details.

**Compatibility Note:** Early versions of SWIG generated just a flattened low-level C style API to C++ classes by default. The `-noproxy` commandline option is recognised by some target languages and will generate just this interface as in earlier versions.

## 6.5 Proxy classes

In order to provide a natural mapping from C++ classes to the target language classes, SWIG's target languages mostly wrap C++ classes with special proxy classes. These proxy classes are typically implemented in the target language itself. For example, if you're building a Python module, each C++ class is wrapped by a Python proxy class. Or if you're building a Java module, each C++ class is wrapped by a Java proxy class.

### 6.5.1 Construction of proxy classes

Proxy classes are always constructed as an extra layer of wrapping that uses low-level accessor functions. To illustrate, suppose you had a C++ class like this:

```
class Foo {
  public:
    Foo();
    ~Foo();
    int  bar(int x);
    int  x;
};
```

Using C++ as pseudocode, a proxy class looks something like this:

```
class FooProxy {
  private:
    Foo    *self;
  public:
    FooProxy() {
      self = new_Foo();
    }
    ~FooProxy() {
      delete_Foo(self);
    }
    int bar(int x) {
      return Foo_bar(self, x);
    }
    int x_get() {
      return Foo_x_get(self);
    }
```

```
    void x_set(int x) {
      Foo_x_set(self, x);
    }
};
```

Of course, always keep in mind that the real proxy class is written in the target language. For example, in Python, the proxy might look roughly like this:

```
class Foo:
    def __init__(self):
        self.this = new_Foo()
    def __del__(self):
        delete_Foo(self.this)
    def bar(self, x):
        return Foo_bar(self.this, x)
    def __getattr__(self, name):
        if name == 'x':
            return Foo_x_get(self.this)
        ...
    def __setattr__(self, name, value):
        if name == 'x':
            Foo_x_set(self.this, value)
        ...
```

Again, it's important to emphasize that the low-level accessor functions are always used by the proxy classes. Whenever possible, proxies try to take advantage of language features that are similar to C++. This might include operator overloading, exception handling, and other features.

### 6.5.2 Resource management in proxies

A major issue with proxies concerns the memory management of wrapped objects. Consider the following C++ code:

```
class Foo {
public:
  Foo();
  ~Foo();
  int bar(int x);
  int x;
};

class Spam {
public:
  Foo *value;
  ...
};
```

Consider some script code that uses these classes:

```
f = Foo()               # Creates a new Foo
s = Spam()              # Creates a new Spam
s.value = f             # Stores a reference to f inside s
g = s.value             # Returns stored reference
g = 4                   # Reassign g to some other value
del f                   # Destroy f
```

Now, ponder the resulting memory management issues. When objects are created in the script, the objects are wrapped by newly created proxy classes. That is, there is both a new proxy class instance and a new instance of the underlying C++ class. In this example, both `f` and `s` are created in this way. However, the statement`s.value` is rather curious---when executed, a pointer to `f` is stored inside another object. This means that the scripting proxy class *AND* another C++ class share a reference to the same object. To make matters even more interesting, consider the statement `g = s.value`. When executed, this creates a new proxy class`g` that provides a wrapper around the C++ object stored in `s.value` . In general, there is no way to know where this object came from---it could have been created by the script, but it could also have been generated internally. In this particular example, the assignment of `g` results in a second proxy class for `f`. In other words, a reference to `f` is now shared by two proxy classes *and* a C++ class.

Finally, consider what happens when objects are destroyed. In the statement, `g=4`, the variable `g` is reassigned. In many languages, this makes the old value of `g` available for garbage collection. Therefore, this causes one of the proxy classes to be destroyed. Later on, the statement `del f` destroys the other proxy class. Of course, there is still a reference to the original object stored inside another C++ object. What happens to it? Is the object still valid?

To deal with memory management problems, proxy classes provide an API for controlling ownership. In C++ pseudocode, ownership control might look roughly like this:

```
class FooProxy {
  public:
    Foo    *self;
    int     thisown;

    FooProxy() {
      self = new_Foo();
      thisown = 1;       // Newly created object
    }
    ~FooProxy() {
      if (thisown) delete_Foo(self);
    }
    ...
    // Ownership control API
    void disown() {
      thisown = 0;
    }
    void acquire() {
      thisown = 1;
    }
};

class FooPtrProxy: public FooProxy {
public:
  FooPtrProxy(Foo *s) {
    self = s;
    thisown = 0;
  }
};
```

```
class SpamProxy {
  ...
  FooProxy *value_get() {
    return FooPtrProxy(Spam_value_get(self));
  }
  void value_set(FooProxy *v) {
    Spam_value_set(self, v->self);
    v->disown();
  }
  ...
};
```

Looking at this code, there are a few central features:

- Each proxy class keeps an extra flag to indicate ownership. C++ objects are only destroyed if the ownership flag is set.
- When new objects are created in the target language, the ownership flag is set.
- When a reference to an internal C++ object is returned, it is wrapped by a proxy class, but the proxy class does not have ownership.
- In certain cases, ownership is adjusted. For instance, when a value is assigned to the member of a class, ownership is lost.
- Manual ownership control is provided by special `disown()` and `acquire()` methods.

Given the tricky nature of C++ memory management, it is impossible for proxy classes to automatically handle every possible memory management problem. However, proxies do provide a mechanism for manual control that can be used (if necessary) to address some of the more tricky memory management problems.

### 6.5.3 Language specific details

Language specific details on proxy classes are contained in the chapters describing each target language. This chapter has merely introduced the topic in a very general way.

## 6.6 Simple C++ wrapping

The following code shows a SWIG interface file for a simple C++ class.

```
%module list
%{
#include "list.h"
%}

// Very simple C++ example for linked list

class List {
public:
  List();
  ~List();
  int  search(char *value);
  void insert(char *);
  void remove(char *);
  char *get(int n);
  int  length;
static void print(List *l);
};
```

To generate wrappers for this class, SWIG first reduces the class to a collection of low-level C-style accessor functions which are then used by the proxy classes.

### 6.6.1 Constructors and destructors

C++ constructors and destructors are translated into accessor functions such as the following :

```
List * new_List(void) {
  return new List;
}
void delete_List(List *l) {
  delete l;
}
```

### 6.6.2 Default constructors, copy constructors and implicit destructors

Following the C++ rules for implicit constructor and destructors, SWIG will automatically assume there is one even when they are not explicitly declared in the class interface.

In general then:

- If a C++ class does not declare any explicit constructor, SWIG will automatically generate a wrapper for one.
- If a C++ class does not declare an explicit copy constructor, SWIG will automatically generate a wrapper for one if `%copyctor` is used.
- If a C++ class does not declare an explicit destructor, SWIG will automatically generate a wrapper for one.

And as in C++, a few rules that alters the previous behavior:

- A default constructor is not created if a class already defines a constructor with arguments.
- Default constructors are not generated for classes with pure virtual methods or for classes that inherit from an abstract class, but don't provide definitions for all of the pure methods.
- A default constructor is not created unless all base classes support a default constructor.
- Default constructors and implicit destructors are not created if a class defines them in a `private` or `protected` section.
- Default constructors and implicit destructors are not created if any base class defines a non-public default constructor or destructor.

SWIG should never generate a default constructor, copy constructor or default destructor wrapper for a class in which it is illegal to do so. In some cases, however, it could be necessary (if the complete class declaration is not visible from SWIG, and one of the above rules is violated) or desired (to reduce the size of the final interface) by manually disabling the implicit constructor/destructor generation.

To manually disable these, the `%nodefaultctor` and `%nodefaultdtor` feature flag directives can be used. Note that these directives only affects the implicit generation, and they have no effect if the default/copy constructors or destructor are explicitly declared in the class interface.

For example:

```
%nodefaultctor Foo;  // Disable the default constructor for class Foo.
class Foo {          // No default constructor is generated, unless one is declared
...
};
```

```
class Bar {          // A default constructor is generated, if possible
...
};
```

The directive `%nodefaultctor` can also be applied "globally", as in:

```
%nodefaultctor; // Disable creation of default constructors
class Foo {     // No default constructor is generated, unless one is declared
...
};
class Bar {
public:
  Bar();        // The default constructor is generated, since one is declared
};
%clearnodefaultctor; // Enable the creation of default constructors again
```

The corresponding `%nodefaultdtor` directive can be used to disable the generation of the default or implicit destructor, if needed. Be aware, however, that this could lead to memory leaks in the target language. Hence, it is recommended to use this directive only in well known cases. For example:

```
%nodefaultdtor Foo;   // Disable the implicit/default destructor for class Foo.
class Foo {           // No destructor is generated, unless one is declared
...
};
```

**Compatibility Note:** The generation of default constructors/implicit destructors was made the default behavior in SWIG 1.3.7. This may break certain older modules, but the old behavior can be easily restored using `%nodefault`. Furthermore, in order for SWIG to properly generate (or not generate) default constructors, it must be able to gather information from both the `private` and `protected` sections (specifically, it needs to know if a private or protected constructor/destructor is defined). In older versions of SWIG, it was fairly common to simply remove or comment out the private and protected sections of a class due to parser limitations. However, this removal may now cause SWIG to erroneously generate constructors for classes that define a constructor in those sections. Consider restoring those sections in the interface or using `%nodefault` to fix the problem.

**Note:** The `%nodefault` directive described above, which disables both the default constructor and the implicit destructors, could lead to memory leaks, and so it is strongly recommended to not use it.

### 6.6.3 When constructor wrappers aren't created

If a class defines a constructor, SWIG normally tries to generate a wrapper for it. However, SWIG will not generate a constructor wrapper if it thinks that it will result in illegal wrapper code. There are really two cases where this might show up.

First, SWIG won't generate wrappers for protected or private constructors. For example:

```
class Foo {
protected:
  Foo();         // Not wrapped.
public:
  ...
};
```

Next, SWIG won't generate wrappers for a class if it appears to be abstract--that is, it has undefined pure virtual methods. Here are some examples:

```
class Bar {
public:
  Bar();               // Not wrapped.  Bar is abstract.
  virtual void spam(void) = 0;
};

class Grok : public Bar {
public:
  Grok();              // Not wrapped. No implementation of abstract spam().
};
```

Some users are surprised (or confused) to find missing constructor wrappers in their interfaces. In almost all cases, this is caused when classes are determined to be abstract. To see if this is the case, run SWIG with all of its warnings turned on:

```
% swig -Wall -python module.i
```

In this mode, SWIG will issue a warning for all abstract classes. It is possible to force a class to be non-abstract using this:

```
%feature("notabstract") Foo;

class Foo : public Bar {
public:
  Foo();    // Generated no matter what---not abstract.
  ...
};
```

More information about `%feature` can be found in the Customization features chapter.

### 6.6.4 Copy constructors

If a class defines more than one constructor, its behavior depends on the capabilities of the target language. If overloading is supported, the copy constructor is accessible using the normal constructor function. For example, if you have this:

```
class List {
public:
  List();
  List(const List &);      // Copy constructor
  ...
};
```

then the copy constructor can be used as follows:

```
x = List()              # Create a list
y = List(x)             # Copy list x
```

If the target language does not support overloading, then the copy constructor is available through a special function like this:

```
List *copy_List(List *f) {
  return new List(*f);
}
```

**Note:** For a class `X`, SWIG only treats a constructor as a copy constructor if it can be applied to an object of type `X` or `X *`. If more than one copy constructor is defined, only the first definition that appears is used as the copy constructor--other definitions will result in a name-clash. Constructors such as `X(const X &)`, `X(X &)`, and `X(X *)` are handled as copy constructors in SWIG.

**Note:** SWIG does *not* generate a copy constructor wrapper unless one is explicitly declared in the class. This differs from the treatment of default constructors and destructors. However, copy constructor wrappers can be generated if using the `copyctor` feature flag. For example:

```
%copyctor List;

class List {
public:
  List();
};
```

Will generate a copy constructor wrapper for `List`.

**Compatibility note:** Special support for copy constructors was not added until SWIG-1.3.12. In previous versions, copy constructors could be wrapped, but they had to be renamed. For example:

```
%rename(CopyFoo) Foo::Foo(const Foo &);

class Foo {
public:
  Foo();
  Foo(const Foo &);
  ...
};
```

For backwards compatibility, SWIG does not perform any special copy-constructor handling if the constructor has been manually renamed. For instance, in the above example, the name of the constructor is set to `new_CopyFoo()`. This is the same as in older versions.

### 6.6.5 Member functions

All member functions are roughly translated into accessor functions like this :

```
int List_search(List *obj, char *value) {
  return obj->search(value);
}
```

This translation is the same even if the member function has been declared as `virtual`.

It should be noted that SWIG does not *actually* create a C accessor function in the code it generates. Instead, member access such as `obj->search(value)` is directly inlined into the generated wrapper functions. However, the name and calling convention of the low-level procedural wrappers match the accessor function prototype described above.

### 6.6.6 Static members

Static member functions are called directly without making any special transformations. For example, the static member function `print(List *l)` directly invokes `List::print(List *l)` in the generated wrapper code.

### 6.6.7 Member data

Member data is handled in exactly the same manner as for C structures. A pair of accessor functions are effectively created. For example :

```
int List_length_get(List *obj) {
  return obj->length;
}
int List_length_set(List *obj, int value) {
  obj->length = value;
  return value;
}
```

A read-only member can be created using the `%immutable` and `%mutable` feature flag directive. For example, we probably wouldn't want the user to change the length of a list so we could do the following to make the value available, but read-only.

```
class List {
public:
...
%immutable;
  int length;
%mutable;
...
};
```

Alternatively, you can specify an immutable member in advance like this:

```
%immutable List::length;
```

```
...
class List {
  ...
  int length;         // Immutable by above directive
  ...
};
```

Similarly, all data attributes declared as `const` are wrapped as read-only members.

By default, SWIG uses the const reference typemaps for members that are primitive types. There are some subtle issues when wrapping data members that are not primitive types, such as classes. For instance, if you had another class like this,

```
class Foo {
public:
  List items;
  ...
```

then the low-level accessor to the `items` member actually uses pointers. For example:

```
List *Foo_items_get(Foo *self) {
  return &self->items;
}
void Foo_items_set(Foo *self, List *value) {
  self->items = *value;
}
```

More information about this can be found in the SWIG Basics chapter in the Structure data members and Creating read-only variables sections.

Additionally, any C++ type that is not assignable is also wrapped as a read-only member. Consider the class below which is non-assignable due to the private assignment operator:

```
class NonAssignable {
private:
  NonAssignable & operator=(const NonAssignable &);
public:
  NonAssignable();
};

struct ImmutableVars {
  NonAssignable non_assignable;
};
```

The `non_assignable` member variable is immutable by default as SWIG detects that `NonAssignable` is not assignable. SWIG must of course see the full type information in order to get this correct, otherwise you may have to use `%immutable` to make the variable read-only.

The wrapper code to generate the accessors for classes comes from the pointer typemaps. This can be somewhat unnatural for some types. For example, a user would expect the STL std::string class member variables to be wrapped as a string in the target language, rather than a pointer to this class. The const reference typemaps offer this type of marshalling, so there is a feature to tell SWIG to use the const reference typemaps rather than the pointer typemaps. It is the naturalvar feature and can be used to effectively change the way accessors are generated to the following:

```
const List &Foo_items_get(Foo *self) {
  return self->items;
}
void Foo_items_set(Foo *self, const List &value) {
  self->items = value;
}
```

The `%naturalvar` directive is a macro for, and hence equivalent to, `%feature("naturalvar")`. It can be used as follows:

```
// All List variables will use const List& typemaps
%naturalvar List;

// Only Foo::myList will use const List& typemaps
%naturalvar Foo::myList;
struct Foo {
  List myList;
};

// All non-primitive types will use const reference typemaps
%naturalvar;
```

The observant reader will notice that `%naturalvar` works like any other feature flag directive but with some extra flexibility. The first of the example usages above shows `%naturalvar` attaching to the `myList` 's variable type, that is the `List` class. The second usage shows `%naturalvar` attaching to the variable name. Hence the naturalvar feature can be used on either the variable's name or type. Note that using the naturalvar feature on a variable's name overrides any naturalvar feature attached to the variable's type.

It is generally a good idea to use this feature globally as the reference typemaps have extra NULL checking compared to the pointer typemaps. A pointer can be NULL, whereas a reference cannot, so the extra checking ensures that the target language user does not pass in a value that translates to a NULL pointer and thereby preventing any potential NULL pointer dereferences. The `%naturalvar` feature will apply to global variables in addition to member variables in some language modules, eg C# and Java.

The naturalvar behavior can also be turned on as a global setting via the `-naturalvar` commandline option or the module mode option, `%module(naturalvar=1)`. However, any use of `%feature("naturalvar")` will override the global setting.

**Compatibility note:** The `%naturalvar` feature was introduced in SWIG-1.3.28, prior to which it was necessary to manually apply the const reference typemaps, eg `%apply const std::string & { std::string * }`, but this example would also apply the typemaps to methods taking a `std::string` pointer.

**Compatibility note:** Prior to SWIG-1.3.12, all members of unknown type were wrapped into accessor functions using pointers. For example, if you had a structure like this

```
struct Foo {
  size_t  len;
};
```

and nothing was known about `size_t`, then accessors would be written to work with `size_t *`. Starting in SWIG-1.3.12, this behavior has been modified. Specifically, pointers will *only* be

used if SWIG knows that a datatype corresponds to a structure or class. Therefore, the above code would be wrapped into accessors involving `size_t`. This change is subtle, but it smooths over a few problems related to structure wrapping and some of SWIG's customization features.

## 6.7 Protection

SWIG wraps class members that are public following the C++ conventions, i.e., by explicit public declaration or by the use of `using` declarations. In general, anything specified in a private or protected section will be ignored, although the internal code generator sometimes looks at the contents of the private and protected sections so that it can properly generate code for default constructors and destructors. Directors could also modify the way non-public virtual protected members are treated.

By default, members of a class definition are assumed to be private until you explicitly give a `` `public:' `` declaration (This is the same convention used by C++).

## 6.8 Enums and constants

Enumerations and constants are handled differently by the different language modules and are described in detail in the appropriate language chapter. However, many languages map enums and constants in a class definition into constants with the classname as a prefix. For example :

```
class Swig {
public:
  enum {ALE, LAGER, PORTER, STOUT};
};
```

Generates the following set of constants in the target scripting language :

```
Swig_ALE = Swig::ALE
Swig_LAGER = Swig::LAGER
Swig_PORTER = Swig::PORTER
Swig_STOUT = Swig::STOUT
```

Members declared as `const` are wrapped as read-only members and do not create constants.

## 6.9 Friends

### 6.9.1 Friend classes

Friend classes are a C++ feature that do not affect SWIG wrappers. SWIG simply parses the friend class declarations, but they are effectively ignored as they have no meaningful effect for wrappers. An example of friend classes:

```
class X;
class Y;
class C {
  // Friend classes have no effect on generated wrappers
  friend class X;
  friend Y;
  ...
};
```

### 6.9.2 Friend function definitions

A friend function definition in a C++ class defines a non-member function of the class and simultaneously makes it a friend of the class. For example, if you have this code:

```
class Buddy {
  int val;
  friend int blah(Buddy *b) { return b->val; }
public:
  ...
};
```

then the `friend` function definition results in wrapper code equivalent to one generated for the following:

```
class Buddy {
  int val;
  friend int blah(Buddy *b);
public:
  ...
};

int blah(Buddy *b) { return b->val; }
```

Access from target languages is thus as if `blah` was wrapped as a non-member function. The function is available and wrapped even if the friend is defined with private or protected access.

A friend definition, as in C++, is understood to be in the same scope that the class is defined in, hence the scoping required for SWIG directives, such as `%ignore`, is as follows:

```
%ignore bar::blah(Buddy *b);
// Not: %ignore bar::Buddy::blah(Buddy *b);

namespace bar {
  class Buddy {
    int val;
    friend int blah(Buddy *b) { return b->val; }
  public:
    ...
  };
}
```

and a wrapper for `blah` will not be generated.

### 6.9.3 Friend function declarations

A C++ class can specify friends via friend function declarations. These functions are allowed access to the private and protected members of a class. This is pure C++ functionality and these friend function declarations are hence quietly ignored by SWIG and do not result in any wrappers. Well, not always! The C++ rules for friends that SWIG needs to follow are not that simple. Technically, only qualified function declarations are silently ignored by SWIG. Below are some examples of qualified friend declarations in `A` that are quietly ignored:

```
struct B {
  int f();
  B();
  ~B();
  ...
};

int g();

class A {
public:
  // The following friend function-declarations are silently ignored (including constructor and destructor friends)
  friend B::B();
  friend B::~B();
  friend int B::f();
  friend int ::g();
  ...
};
```

In the example above, if SWIG parses the struct `B` and global function `g()`, then they are of course wrapped as normal.

### 6.9.4 Unqualified friend functions

Further clarification is required regarding both friend function definitions and declarations. In C++, friend function definitions can only be unqualified, whereas, friend function declarations can be either unqualified or qualified. Qualified friend function declarations are silently ignored by SWIG as covered in the previous section. SWIG does generate wrappers for any unqualified friend functions that it parses. This section goes through some of the complexities of wrapping unqualified friend functions.

Consider an unqualified friend function definition:

```
class Chum {
  int val;
  friend int blah() { Chum c; c.private_function(); return c.val; }
  void private_function();
public:
  ...
};
```

The `Chum::blah()` friend is very similar to the `Buddy::blah(Buddy *)` friend presented earlier. However, the generated code to call `blah()` may not compile unlike the code to call `blah(Buddy *)`. The compiler error will be something like:

```
error: 'blah' was not declared in this scope
```

The reason one works and the other doesn't is due to the rules around unqualified friend definitions/declarations. Broadly, friends are not visible for lookup except via argument dependent lookup that considers the class that the friend is defined/declared in, unless there is a *matching declaration* at namespace scope. This will probably only make sense if you are conversant with this C++ concept, which is covered quite well at Argument-dependent lookup. In our examples, `blah(Buddy *)` is visible via argument dependent lookup, but `blah()` is not. The solution is thus to provide a *matching declaration* in order to make the function visible to the compiler. Simply add:

```
int blah();
```

SWIG does **not** have to parse it. In all likelihood, your code already has the *matching declaration* as it is required in order for the friend function definition to be usable from pure C++ code.

The same potential problem applies to unqualified friend function declarations, such as:

```
class Mate {
  int val;
  friend int blah(); // Unqualified friend function declaration
  void private_function();
public:
  ...
};
```

Again, the actual function declaration needs to be visible to the compiler. Or just the actual function definition as shown below. This must be defined in the same scope as `Mate`. Of course the function definition is necessary in order to avoid linking issues too.

```
int blah() { Mate m; m.private_function(); return m.val; }
```

## 6.10 References and pointers

C++ references are supported, but SWIG transforms them back into pointers. For example, a declaration like this :

```
class Foo {
public:
  double bar(double &a);
}
```

has a low-level accessor

```
double Foo_bar(Foo *obj, double *a) {
  obj->bar(*a);
}
```

As a special case, most language modules pass `const` references to primitive datatypes (`int`, `short`, `float`, etc.) by value instead of pointers. For example, if you have a function like this,

```
void foo(const int &x);
```

it is called from a script as follows:

```
foo(3)              # Notice pass by value
```

Functions that return a reference are remapped to return a pointer instead. For example:

```
class Bar {
public:
  Foo &spam();
};
```

Generates an accessor like this:

```
Foo *Bar_spam(Bar *obj) {
  Foo &result = obj->spam();
  return &result;
}
```

However, functions that return `const` references to primitive datatypes (`int`, `short`, etc.) normally return the result as a value rather than a pointer. For example, a function like this,

```
const int &bar();
```

will return integers such as 37 or 42 in the target scripting language rather than a pointer to an integer.

Don't return references to objects allocated as local variables on the stack. SWIG doesn't make a copy of the objects so this will probably cause your program to crash.

**Note:** The special treatment for references to primitive datatypes is necessary to provide more seamless integration with more advanced C++ wrapping applications---especially related to templates and the STL. This was first added in SWIG-1.3.12.

## 6.11 Pass and return by value

Occasionally, a C++ program will pass and return class objects by value. For example, a function like this might appear:

```
Vector cross_product(Vector a, Vector b);
```

If no information is supplied about `Vector`, SWIG creates a wrapper function similar to the following:

```
Vector *wrap_cross_product(Vector *a, Vector *b) {
  Vector x;
  Vector y;
  Vector r;
  x = *a;
  y = *b;
  r = cross_product(x, y);
  return new Vector(r);
}
```

In order for the wrapper code to compile, `Vector` must define a default constructor and a copy assignment operator (and/or a move assignment operator for C++11 and later). The Movable and move-only types section should be read regarding C++11 move semantics and return by value.

If `Vector` is defined as a class in the interface, but it does not support a default constructor and an assignment operator, SWIG changes the wrapper code by encapsulating the arguments inside a special C++ template wrapper class, through a process called the "Fulton Transform". This produces a wrapper that looks like this:

```
Vector cross_product(Vector *a, Vector *b) {
  SwigValueWrapper<Vector> x;
  SwigValueWrapper<Vector> y;
  SwigValueWrapper<Vector> r;
  x = *a;
  y = *b;
  r = cross_product(x, y);
  return new Vector(r);
}
```

This transformation is a little sneaky, but it provides support for pass-by-value even when a class is not default constructible nor assignable and it makes it possible to properly support a number of SWIG's customization options. The definition of `SwigValueWrapper` can be found by reading the SWIG wrapper code. This class is really nothing more than a thin wrapper around a pointer.

Although SWIG usually detects the classes to which the Fulton Transform should be applied, in some situations it's necessary to override it. That's done with `%feature("valuewrapper")` to ensure it is used and `%feature("novaluewrapper")` to ensure it is not used:

```
%feature("novaluewrapper") A;
class A;

%feature("valuewrapper") B;
struct B {
  B();
  // ....
};
```

It is well worth considering turning this feature on for classes that do have a default constructor. It will remove a redundant constructor call at the point of the variable declaration in the wrapper, so will generate notably better performance for large objects or for classes with expensive construction. Alternatively consider returning a reference or a pointer.

**Note:** this transformation has no effect on typemaps or any other part of SWIG---it should be transparent except that you may see this code when reading the SWIG output file.

**Note:** This template transformation is new in SWIG-1.3.11 and may be refined in future SWIG releases. In practice, it is only absolutely necessary to do this for classes that don't define a default constructor.

**Note:** The use of this template only occurs when objects are passed or returned by value. It is not used for C++ pointers or references.

## 6.12 Inheritance

SWIG supports C++ inheritance of classes and allows both single and multiple inheritance, as limited or allowed by the target language. The SWIG type-checker knows about the relationship between base and derived classes and allows pointers to any object of a derived class to be used in functions of a base class. The type-checker properly casts pointer values and is safe to use with multiple inheritance.

SWIG treats private or protected inheritance as close to the C++ spirit, and target language capabilities, as possible. In most cases, this means that SWIG will parse the non-public inheritance declarations, but that will have no effect in the generated code, besides the implicit policies derived for constructors and destructors.

The following example shows how SWIG handles inheritance. For clarity, the full C++ code has been omitted.

```
// shapes.i
%module shapes
%{
#include "shapes.h"
%}

class Shape {
public:
  double x, y;
  virtual double area() = 0;
  virtual double perimeter() = 0;
  void    set_location(double x, double y);
};
class Circle : public Shape {
public:
  Circle(double radius);
  ~Circle();
  double area();
  double perimeter();
};
class Square : public Shape {
public:
  Square(double size);
  ~Square();
  double area();
  double perimeter();
}
```

When wrapped into Python, we can perform the following operations (shown using the low level Python accessors):

```
$ python
>>> import shapes
>>> circle = shapes.new_Circle(7)
>>> square = shapes.new_Square(10)
>>> print(shapes.Circle_area(circle))
153.93804004599999757
>>> print(shapes.Shape_area(circle))
153.93804004599999757
>>> print(shapes.Shape_area(square))
100.00000000000000000
>>> shapes.Shape_set_location(square, 2, -3)
>>> print(shapes.Shape_perimeter(square))
40.00000000000000000
>>>
```

In this example, Circle and Square objects have been created. Member functions can be invoked on each object by making calls to `Circle_area`, `Square_area`, and so on. However, the same results can be accomplished by simply using the `Shape_area` function on either object.

One important point concerning inheritance is that the low-level accessor functions are only generated for classes in which they are actually declared. For instance, in the above example, the method `set_location()` is only accessible as `Shape_set_location()` and not as `Circle_set_location()` or `Square_set_location()` . Of course, the `Shape_set_location()` function will accept any kind of object derived from Shape. Similarly, accessor functions for the attributes `x` and `y` are generated as `Shape_x_get()`, `Shape_x_set()`, `Shape_y_get()`, and `Shape_y_set()`. Functions such as `Circle_x_get()` are not available--instead you should use `Shape_x_get()`.

Note that there is a one to one correlation between the low-level accessor functions and the proxy methods and therefore there is also a one to one correlation between the C++ class methods and the generated proxy class methods.

**Note:** For the best results, SWIG requires all base classes to be defined in an interface. Otherwise, you may get a warning message like this:

```
example.i:18: Warning 401: Nothing known about base class 'Foo'. Ignored.
```

If any base class is undefined, SWIG still generates correct type relationships. For instance, a function accepting a `Foo *` will accept any object derived from `Foo` regardless of whether or not SWIG actually wrapped the `Foo` class. If you really don't want to generate wrappers for the base class, but you want to silence the warning, you might consider using the `%import` directive to include the file that defines `Foo`. `%import` simply gathers type information, but doesn't generate wrappers. Alternatively, you could just define `Foo` as an empty class in the SWIG interface or use warning suppression .

**Note:** `typedef`-names *can* be used as base classes. For example:

```
class Foo {
...
};

typedef Foo FooObj;
class Bar : public FooObj {     // Ok.  Base class is Foo
...
};
```

Similarly, `typedef` allows unnamed structures to be used as base classes. For example:

```
typedef struct {
  ...
} Foo;

class Bar : public Foo {    // Ok.
...
};
```

**Compatibility Note:** Starting in version 1.3.7, SWIG only generates low-level accessor wrappers for the declarations that are actually defined in each class. This differs from SWIG1.1 which used to inherit all of the declarations defined in base classes and regenerate specialized accessor functions such as `Circle_x_get()`, `Square_x_get()`, `Circle_set_location()`, and `Square_set_location()`. This behavior resulted in huge amounts of replicated code for large class hierarchies and made it awkward to build applications spread across multiple modules (since accessor functions are duplicated in every single module). It is also unnecessary to have such wrappers when advanced features like proxy classes are used. **Note:** Further optimizations are enabled when using the `-fvirtual` option, which avoids the regenerating of wrapper functions for virtual members that are already defined in a base class.

## 6.13 A brief discussion of multiple inheritance, pointers, and type checking

When a target scripting language refers to a C++ object, it normally uses a tagged pointer object that contains both the value of the pointer and a type string. For example, in Tcl, a C++ pointer might be encoded as a string like this:

```
_808fea88_p_Circle
```

A somewhat common question is whether or not the type-tag could be safely removed from the pointer. For instance, to get better performance, could you strip all type tags and just use simple integers instead?

In general, the answer to this question is no. In the wrappers, all pointers are converted into a common data representation in the target language. Typically this is the equivalent of casting a pointer to `void *`. This means that any C++ type information associated with the pointer is lost in the conversion.

The problem with losing type information is that it is needed to properly support many advanced C++ features--especially multiple inheritance. For example, suppose you had code like this:

```
class A {
public:
  int x;
};

class B {
public:
  int y;
};

class C : public A, public B {
};

int A_function(A *a) {
  return a->x;
}

int B_function(B *b) {
  return b->y;
}
```

Now, consider the following code that uses `void *`.

```
C *c = new C();
void *p = (void *) c;
...
int x = A_function((A *) p);
int y = B_function((B *) p);
```

In this code, both `A_function()` and `B_function()` may legally accept an object of type `C *` (via inheritance). However, one of the functions will always return the wrong result when used as shown. The reason for this is that even though `p` points to an object of type `C`, the casting operation doesn't work like you would expect. Internally, this has to do with the data representation of `C`. With multiple inheritance, the data from each base class is stacked together. For example:

```
            ------------    <--- (C *),  (A *)
           |     A      |
           |------------|   <--- (B *)
           |     B      |
            ------------
```

Because of this stacking, a pointer of type `C *` may change value when it is converted to a `A *` or `B *`. However, this adjustment does *not* occur if you are converting from a `void *`.

The use of type tags marks all pointers with the real type of the underlying object. This extra information is then used by SWIG generated wrappers to correctly cast pointer values under inheritance (avoiding the above problem).

Some of the language modules are able to solve the problem by storing multiple instances of the pointer, for example, `A *`, in the A proxy class as well as `C *` in the C proxy class. The correct cast can then be made by choosing the correct `void *` pointer to use and is guaranteed to work as the cast to a void pointer and back to the same type does not lose any type information:

```
C *c = new C();
void *p = (void *) c;
void *pA = (void *) c;
void *pB = (void *) c;
...
int x = A_function((A *) pA);
int y = B_function((B *) pB);
```

In practice, the pointer is held as an integral number in the target language proxy class.

## 6.14 Default arguments

SWIG will wrap all types of functions that have default arguments. For example member functions:

```
class Foo {
public:
  void bar(int x, int y = 3, int z = 4);
};
```

SWIG handles default arguments by generating an extra overloaded method for each defaulted argument. SWIG is effectively handling methods with default arguments as if it was wrapping the equivalent overloaded methods. Thus for the example above, it is as if we had instead given the following to SWIG:

```
class Foo {
public:
  void bar(int x, int y, int z);
  void bar(int x, int y);
  void bar(int x);
};
```

The wrappers produced are exactly the same as if the above code was instead fed into SWIG. Details of this are covered in the next section Overloaded functions and methods . This approach allows SWIG to wrap all possible default arguments, but can be verbose. For example if a method has ten default arguments, then eleven wrapper methods are generated.

Please see the Features and default arguments section for more information on using `%feature` with functions with default arguments. TheRenaming and ambiguity resolution section also deals with using `%rename` and `%ignore` on methods with default arguments. If you are writing your own typemaps for types used in methods with default arguments, you may also need to write a `typecheck` typemap. See the Typemaps and overloading section for details or otherwise use the`compactdefaultargs` feature flag as mentioned below.

For C# please see the C# named and optional arguments section for information on special handling of default arguments available specifically for C#.

**Compatibility note:** Versions of SWIG prior to SWIG-1.3.23 wrapped default arguments slightly differently. Instead a single wrapper method was generated and the default values were copied into the C++ wrappers so that the method being wrapped was then called with all the arguments specified. If the size of the wrappers are a concern then this approach to wrapping methods with default arguments can be re-activated by using the `compactdefaultargs` feature flag.

```
%feature("compactdefaultargs") Foo::bar;
class Foo {
public:
  void bar(int x, int y = 3, int z = 4);
};
```

This is great for reducing the size of the wrappers, but the caveat is it does not work for the statically typed languages, such as C# and Java, which don't have optional arguments in the language, Another restriction of this feature is that it cannot handle default arguments that are not public. The following example illustrates this:

```
class Foo {
private:
  static const int spam;
public:
  void bar(int x, int y = spam);   // Won't work with %feature("compactdefaultargs") -
                                   // private default value
};
```

This produces uncompilable wrapper code because default values in C++ are evaluated in the same scope as the member function whereas SWIG evaluates them in the scope of a wrapper function (meaning that the values have to be public).

The `compactdefaultargs` feature is automatically turned on when wrapping C code with default arguments. Some target languages will also automatically turn on this feature if the keyword arguments feature (kwargs) is specified for either C or C++ functions, and the target language supports kwargs, the `compactdefaultargs` feature is also automatically turned on. Keyword arguments are a language feature of some scripting languages, for example Ruby and Python. SWIG is unable to support kwargs when wrapping overloaded methods, so the default approach cannot be used.

## 6.15 Overloaded functions and methods

In many language modules, SWIG provides partial support for overloaded functions, methods, and constructors. For example, if you supply SWIG with overloaded functions like this:

```
void foo(int x) {
  printf("x is %d\n", x);
}
void foo(char *x) {
  printf("x is '%s'\n", x);
}
```

The function is used in a completely natural way. For example:

```
>>> foo(3)
x is 3
>>> foo("hello")
x is 'hello'
>>>
```

Overloading works in a similar manner for methods and constructors. For example if you have this code,

```
class Foo {
public:
  Foo();
  Foo(const Foo &);   // Copy constructor
  void bar(int x);
  void bar(char *s, int y);
};
```

it might be used like this

```
>>> f = Foo()          # Create a Foo
>>> f.bar(3)
>>> g = Foo(f)         # Copy Foo
>>> f.bar("hello", 2)
```

### 6.15.1 Dispatch function generation

The implementation of overloaded functions and methods is somewhat complicated due to the dynamic nature of scripting languages. Unlike C++, which binds overloaded methods at compile time, SWIG must determine the proper function as a runtime check for scripting language targets. This check is further complicated by the typeless nature of certain scripting languages. For instance, in Tcl, all types are simply strings. Therefore, if you have two overloaded functions like this,

```
void foo(char *x);
void foo(int x);
```

the order in which the arguments are checked plays a rather critical role.

For statically typed languages, SWIG uses the language's method overloading mechanism. To implement overloading for the scripting languages, SWIG generates a dispatch function that checks the number of passed arguments and their types. To create this function, SWIG first examines all of the overloaded methods and ranks them according to the following rules:

1. **Number of required arguments.** Methods are sorted by increasing number of required arguments.
2. **Argument type precedence.** All C++ datatypes are assigned a numeric type precedence value (which is determined by the language module).

```
Type              Precedence
----------------  ----------
TYPE *            0     (High)
void *            20
Integers          40
Floating point    60
char              80
Strings           100   (Low)
```

Using these precedence values, overloaded methods with the same number of required arguments are sorted in increased order of precedence values.

This may sound very confusing, but an example will help. Consider the following collection of overloaded methods:

```
void foo(double);
void foo(int);
void foo(Bar *);
void foo();
void foo(int x, int y, int z, int w);
void foo(int x, int y, int z = 3);
void foo(double x, double y);
void foo(double x, Bar *z);
```

The first rule simply ranks the functions by required argument count. This would produce the following list:

```
rank
-----
[0]   foo()
[1]   foo(double);
[2]   foo(int);
[3]   foo(Bar *);
[4]   foo(int x, int y, int z = 3);
[5]   foo(double x, double y)
[6]   foo(double x, Bar *z)
[7]   foo(int x, int y, int z, int w);
```

The second rule, simply refines the ranking by looking at argument type precedence values.

```
rank
-----
[0]   foo()
[1]   foo(Bar *);
[2]   foo(int);
[3]   foo(double);
[4]   foo(int x, int y, int z = 3);
[5]   foo(double x, Bar *z)
[6]   foo(double x, double y)
[7]   foo(int x, int y, int z, int w);
```

Finally, to generate the dispatch function, the arguments passed to an overloaded method are simply checked in the same order as they appear in this ranking.

If you're still confused, don't worry about it---SWIG is probably doing the right thing.

### 6.15.2 Ambiguity in overloading

Regrettably, SWIG is not able to support every possible use of valid C++ overloading. Consider the following example:

```
void foo(int x);
void foo(long x);
```

In C++, this is perfectly legal. However, in a scripting language, there is generally only one kind of integer object. Therefore, which one of these functions do you pick? Clearly, there is no way to truly make a distinction just by looking at the value of the integer itself ( `int` and `long` may even be the same precision). Therefore, when SWIG encounters this situation, it may generate a warning message like this for scripting languages:

```
example.i:4: Warning 509: Overloaded method foo(long) effectively ignored,
example.i:3: Warning 509: as it is shadowed by foo(int).
```

or for statically typed languages like Java:

```
example.i:4: Warning 516: Overloaded method foo(long) ignored,
example.i:3: Warning 516: using foo(int) instead.
```

This means that the second overloaded function will be inaccessible from a scripting interface or the method won't be wrapped at all. This is done as SWIG does not know how to disambiguate it from an earlier method.

Ambiguity problems are known to arise in the following situations:

- Integer conversions. Datatypes such as `int`, `long`, and `short` cannot be disambiguated in some languages. Shown above.
- Floating point conversion. `float` and `double` can not be disambiguated in some languages.
- Pointers and references. For example, `Foo *` and `Foo &`.
- Pointers and arrays. For example, `Foo *` and `Foo [4]`.
- Pointers and instances. For example, `Foo` and `Foo *`. Note: SWIG converts all instances to pointers.
- Qualifiers. For example, `const Foo *` and `Foo *`.
- Default vs. non default arguments. For example, `foo(int a, int b)` and `foo(int a, int b = 3)`.

When an ambiguity arises, methods are checked in the same order as they appear in the interface file. Therefore, earlier methods will shadow methods that appear later.

When wrapping an overloaded function, there is a chance that you will get a warning message like this:

```
example.i:3: Warning 467: Overloaded foo(int) not supported (incomplete type checking rule -
no precedence level in typecheck typemap for 'int').
```

This error means that the target language module supports overloading, but for some reason there is no type-checking rule that can be used to generate a working dispatch function. The resulting behavior is then undefined. You should report this as a bug to the SWIG bug tracking database if this is due to one of the typemaps supplied with SWIG.

If you get an error message such as the following,

```
foo.i:6. Overloaded declaration ignored.  Spam::foo(double )
foo.i:5. Previous declaration is Spam::foo(int )
foo.i:7. Overloaded declaration ignored.  Spam::foo(Bar *, Spam *, int )
foo.i:5. Previous declaration is Spam::foo(int )
```

it means that the target language module has not yet implemented support for overloaded functions and methods. The only way to fix the problem is to read the next section.

### 6.15.3 Renaming and ambiguity resolution

If an ambiguity in overload resolution occurs or if a module doesn't allow overloading, there are a few strategies for dealing with the problem. First, you can tell SWIG to ignore one of the methods. This is easy---simply use the `%ignore` directive. For example:

```
%ignore foo(long);

void foo(int);
void foo(long);       // Ignored.  Oh well.
```

The other alternative is to rename one of the methods. This can be done using `%rename`. For example:

```
%rename("foo_short") foo(short);
%rename(foo_long) foo(long);

void foo(int);
void foo(short);      // Accessed as foo_short()
void foo(long);       // Accessed as foo_long()
```

Note that the quotes around the new name are optional, however, should the new name be a C/C++ keyword they would be essential in order to avoid a parsing error. The `%ignore` and `%rename` directives are both rather powerful in their ability to match declarations. When used in their simple form, they apply to both global functions and methods. For example:

```
/* Forward renaming declarations */
%rename(foo_i) foo(int);
%rename(foo_d) foo(double);
...
void foo(int);           // Becomes 'foo_i'
void foo(char *c);       // Stays 'foo' (not renamed)

class Spam {
public:
  void foo(int);      // Becomes 'foo_i'
  void foo(double);   // Becomes 'foo_d'
  ...
};
```

If you only want the renaming to apply to a certain scope, the C++ scope resolution operator (::) can be used. For example:

```
%rename(foo_i) ::foo(int);      // Only rename foo(int) in the global scope.
                                // (will not rename class members)

%rename(foo_i) Spam::foo(int);  // Only rename foo(int) in class Spam
```

When a renaming operator is applied to a class as in `Spam::foo(int)`, it is applied to that class and all derived classes. This can be used to apply a consistent renaming across an entire class hierarchy with only a few declarations. For example:

```
%rename(foo_i) Spam::foo(int);
%rename(foo_d) Spam::foo(double);

class Spam {
public:
  virtual void foo(int);      // Renamed to foo_i
  virtual void foo(double);   // Renamed to foo_d
  ...
};

class Bar : public Spam {
public:
```

```
  virtual void foo(int);      // Renamed to foo_i
  virtual void foo(double);   // Renamed to foo_d
  ...
};

class Grok : public Bar {
public:
  virtual void foo(int);      // Renamed to foo_i
  virtual void foo(double);   // Renamed to foo_d
  ...
};
```

It is also possible to include `%rename` specifications in the class definition itself. For example:

```
class Spam {
  %rename(foo_i) foo(int);
  %rename(foo_d) foo(double);
public:
  virtual void foo(int);      // Renamed to foo_i
  virtual void foo(double);   // Renamed to foo_d
  ...
};

class Bar : public Spam {
public:
  virtual void foo(int);      // Renamed to foo_i
  virtual void foo(double);   // Renamed to foo_d
...
};
```

In this case, the `%rename` directives still get applied across the entire inheritance hierarchy, but it's no longer necessary to explicitly specify the class prefix `Spam::`.

A special form of `%rename` can be used to apply a renaming just to class members (of all classes):

```
%rename(foo_i) *::foo(int);   // Only rename foo(int) if it appears in a class.
```

Note: the `*::` syntax is non-standard C++, but the '*' is meant to be a wildcard that matches any class name (we couldn't think of a better alternative so if you have a better idea, send email to the swig-devel mailing list.

Although this discussion has primarily focused on `%rename` all of the same rules also apply to `%ignore`. For example:

```
%ignore foo(double);          // Ignore all foo(double)
%ignore Spam::foo;            // Ignore foo in class Spam (and foo in any derived classes)
%ignore Spam::foo(double);    // Ignore foo(double) in class Spam (and foo in any derived classes)
%ignore *::foo(double);       // Ignore foo(double) in all classes
```

When applied to a base class, `%ignore` forces all definitions in derived classes to disappear. For example, `%ignore Spam::foo(double)` will eliminate `foo(double)` in `Spam` and all classes derived from `Spam`.

**Notes on %rename and %ignore:**

- Since, the `%rename` declaration is used to declare a renaming in advance, it can be placed at the start of an interface file. This makes it possible to apply a consistent name resolution without having to modify header files. For example:

```
%module foo

/* Rename these overloaded functions */
%rename(foo_i) foo(int);
%rename(foo_d) foo(double);

%include "header.h"
```

- The scope qualifier (::) can also be used on simple names. For example:

```
%rename(bar) ::foo;       // Rename foo to bar in global scope only
%rename(bar) Spam::foo;   // Rename foo to bar in class Spam only
%rename(bar) *::foo;      // Rename foo in classes only
```

- Name matching tries to find the most specific match that is defined. A qualified name such as `Spam::foo` always has higher precedence than an unqualified name `foo`. `Spam::foo` has higher precedence than `*::foo` and `*::foo` has higher precedence than `foo`. A parameterized name has higher precedence than an unparameterized name within the same scope level. However, an unparameterized name with a scope qualifier has higher precedence than a parameterized name in global scope (e.g., a renaming of `Spam::foo` takes precedence over a renaming of `foo(int)` ).

- Renaming a class member, using an unparameterized but qualified name, such as `Spam::foo`, also applies to members in all derived classes that have members with the same name. This can be used to simply rename a method, across an entire class hierarchy for all overloaded and non-overloaded methods. This also applies to methods introduced via `using` declarations, see Using declarations and inheritance. For example:

```
%rename(foo_new) Spam::foo;

class Spam {
public:
  virtual void foo(int);      // Renamed to foo_new
};

class Bar : public Spam {
public:
  virtual void foo(int);      // Renamed to foo_new
  void foo(bool, short, int); // Renamed to foo_new
};

class Grok : public Bar {
public:
```

```
  virtual void foo(int);      // Renamed to foo_new
  void foo(bool, int);        // Renamed to foo_new
  void foo(const char *);     // Renamed to foo_new
  void foo(Bar *);            // Renamed to foo_new
};

class Spok : public Grok {
public:
  void foo();                 // Renamed to foo_new
};

class Knock : public Spok {
public:
  using Grok::foo;            // Introduced methods renamed to foo_new
};
```

- The order in which `%rename` directives are defined does not matter as long as they appear before the declarations to be renamed. Thus, there is no difference between saying:

```
%rename(bar) foo;
%rename(foo_i) Spam::foo(int);
%rename(Foo) Spam::foo;
```

and this

```
%rename(Foo) Spam::foo;
%rename(bar) foo;
%rename(foo_i) Spam::foo(int);
```

(the declarations are not stored in a linked list and order has no importance). Of course, a repeated `%rename` directive will change the setting for a previous `%rename` directive if exactly the same name, scope, and parameters are supplied.

- For multiple inheritance where renaming rules are defined for multiple base classes, the first renaming rule found on a depth-first traversal of the class hierarchy is used.
- The name matching rules strictly follow member qualifier rules. For example, if you have a class and member with a member that is const qualified like this:

```
class Spam {
public:
  ...
  void bar() const;
  ...
};
```

the declaration

```
%rename(name) Spam::bar();
```

will not apply as there is no unqualified member `bar()`. The following will apply the rename as the qualifier matches correctly:

```
%rename(name) Spam::bar() const;
```

Similarly for combinations of cv-qualifiers and ref-qualifiers, all the qualifiers must be specified to match correctly:

```
%rename(name) Jam::bar();          // will not match
%rename(name) Jam::bar() &;        // will not match
%rename(name) Jam::bar() const;    // will not match
%rename(name) Jam::bar() const &;  // ok, will match

class Jam {
public:
  ...
  void bar() const &;
  ...
};
```

An often overlooked C++ feature is that classes can define two different overloaded members that differ only in their qualifiers, like this:

```
class Spam {
public:
  ...
  void bar();         // Unqualified member
  void bar() const;   // Qualified member
  ...
};
```

%rename can then be used to target each of the overloaded methods individually. For example we can give them separate names in the target language:

```
%rename(name1) Spam::bar();
%rename(name2) Spam::bar() const;
```

Similarly, if you merely wanted to ignore one of the declarations, use `%ignore` with the full qualifier. For example, the following directive would tell SWIG to ignore the `const` version of `bar()` above:

```
%ignore Spam::bar() const;   // Ignore bar() const, but leave other bar() alone
```

- Currently no resolution is performed in order to match function parameters. This means function parameter types must match exactly. For example, namespace qualifiers and typedefs will not work. The following usage of typedefs demonstrates this:

```
typedef int Integer;

%rename(foo_i) foo(int);

class Spam {
public:
  void foo(Integer);  // Stays 'foo' (not renamed)
};
class Ham {
public:
  void foo(int);      // Renamed to foo_i
};
```

- The name matching rules also use default arguments for finer control when wrapping methods that have default arguments. Recall that methods with default arguments are wrapped as if the equivalent overloaded methods had been parsed (Default arguments section). Let's consider the following example class:

```
class Spam {
public:
  ...
  void bar(int i=-1, double d=0.0);
  ...
};
```

The following `%rename` will match exactly and apply to all the target language overloaded methods because the declaration with the default arguments exactly matches the wrapped method:

```
%rename(newbar) Spam::bar(int i=-1, double d=0.0);
```

The C++ method can then be called from the target language with the new name no matter how many arguments are specified, for example: `newbar(2, 2.0)`, `newbar(2)` or `newbar()`. However, if the `%rename` does not contain the default arguments:

```
%rename(newbar) Spam::bar(int i, double d);
```

then only one of the three equivalent overloaded methods will be renamed and wrapped as if SWIG parsed:

```
void Spam::newbar(int i, double d);
void Spam::bar(int i);
void Spam::bar();
```

The C++ method must then be called from the target language with the new name `newbar(2, 2.0)` when both arguments are supplied or with the original name as `bar(2)` (one argument) or `bar()` (no arguments).

In fact it is possible to use `%rename` on the equivalent overloaded methods, to rename all the equivalent overloaded methods:

```
%rename(bar_2args)   Spam::bar(int i, double d);
%rename(bar_1arg)    Spam::bar(int i);
%rename(bar_default) Spam::bar();
```

Similarly, the extra overloaded methods can be selectively ignored using `%ignore`.

**Compatibility note:** The `%rename` directive introduced the default argument matching rules in SWIG-1.3.23 at the same time as the changes to wrapping methods with default arguments was introduced.

### 6.15.4 Comments on overloading

Support for overloaded methods was first added in SWIG-1.3.14. The implementation is somewhat unusual when compared to similar tools. For instance, the order in which declarations appear is largely irrelevant in SWIG. Furthermore, SWIG does not rely upon trial execution or exception handling to figure out which method to invoke.

Internally, the overloading mechanism is completely configurable by the target language module. Therefore, the degree of overloading support may vary from language to language. As a general rule, statically typed languages like Java are able to provide more support than dynamically typed languages like Perl, Python, Ruby, and Tcl.

## 6.16 Overloaded operators

C++ overloaded operator declarations can be wrapped. For example, consider a class like this:

```
class Complex {
private:
  double rpart, ipart;
public:
  Complex(double r = 0, double i = 0) : rpart(r), ipart(i) { }
  Complex(const Complex &c) : rpart(c.rpart), ipart(c.ipart) { }
  Complex &operator=(const Complex &c) {
    rpart = c.rpart;
    ipart = c.ipart;
    return *this;
  }
  Complex operator+(const Complex &c) const {
    return Complex(rpart+c.rpart, ipart+c.ipart);
  }
  Complex operator-(const Complex &c) const {
    return Complex(rpart-c.rpart, ipart-c.ipart);
  }
  Complex operator*(const Complex &c) const {
    return Complex(rpart*c.rpart - ipart*c.ipart,
                   rpart*c.ipart + c.rpart*ipart);
  }
  Complex operator-() const {
    return Complex(-rpart, -ipart);
  }
  double re() const { return rpart; }
```

```
  double im() const { return ipart; }
};
```

When operator declarations appear, they are handled in *exactly* the same manner as regular methods. However, the names of these methods are set to strings like "`operator +`" or "`operator -`". The problem with these names is that they are illegal identifiers in most scripting languages. For instance, you can't just create a method called "`operator +`" in Python--there won't be any way to call it.

Some language modules already know how to automatically handle certain operators (mapping them into operators in the target language). However, the underlying implementation of this is really managed in a very general way using the `%rename` directive. For example, in Python a declaration similar to this is used:

```
%rename(__add__) Complex::operator+;
```

This binds the + operator to a method called `__add__` (which is conveniently the same name used to implement the Python + operator). Internally, the generated wrapper code for a wrapped operator will look something like this pseudocode:

```
_wrap_Complex___add__(args) {
  ... get args ...
  obj->operator+(args);
  ...
}
```

When used in the target language, it may now be possible to use the overloaded operator normally. For example:

```
>>> a = Complex(3, 4)
>>> b = Complex(5, 2)
>>> c = a + b           # Invokes __add__ method
```

It is important to realize that there is nothing magical happening here. The `%rename` directive really only picks a valid method name. If you wrote this:

```
%rename(add) operator+;
```

The resulting scripting interface might work like this:

```
a = Complex(3, 4)
b = Complex(5, 2)
c = a.add(b)      # Call a.operator+(b)
```

All of the techniques described to deal with overloaded functions also apply to operators. For example:

```
%ignore Complex::operator=;             // Ignore = in class Complex
%ignore *::operator=;                   // Ignore = in all classes
%ignore operator=;                      // Ignore = everywhere.

%rename(__sub__) Complex::operator-;
%rename(__neg__) Complex::operator-();  // Unary -
```

The last part of this example illustrates how multiple definitions of the `operator-` method might be handled.

Handling operators in this manner is mostly straightforward. However, there are a few subtle issues to keep in mind:

- In C++, it is fairly common to define different versions of the operators to account for different types. For example, a class might also include a friend function like this:

  ```
  class Complex {
  public:
    friend Complex operator+(Complex &, double);
  };
  Complex operator+(Complex &, double);
  ```

  SWIG simply ignores all `friend` declarations. Furthermore, it doesn't know how to associate the associated `operator+` with the class (because it's not a member of the class).

  It's still possible to make a wrapper for this operator, but you'll have to handle it like a normal function. For example:

  ```
  %rename(add_complex_double) operator+(Complex &, double);
  ```

- Certain operators are ignored by default. For instance, `new` and `delete` operators are ignored as well as conversion and index operators. A warning such as the one below is shown:

  ```
  example.i:12: Warning 503: Can't wrap 'operator []' unless renamed to a valid identifier.
  ```

- The index operator, `operator[]`, is particularly difficult to overload due to differences in C++ implementations. Specifically, the get and set operators in other languages typically are separated into two methods such that additional logic can be packed into the operations; C# uses `this[type key] { get { ... } set { ... }}` , Python uses `__getitem__` and `__setitem__`, etc. In C++ if the return type of `operator[]` is a reference and the method is const, it is often indicative of the *setter,* and the *getter* is usually a const function return an object by value. In the absence of any hard and fast rules and the fact that there may be multiple index operators, it is up to the user to choose the getter and setter to use by using %rename as shown earlier.
- The semantics of certain C++ operators may not match those in the target language.

## 6.17 Class extension

New methods can be added to a class using the `%extend` directive. This directive is primarily used in conjunction with proxy classes to add additional functionality to an existing class. For example :

```
%module vector
%{
#include "vector.h"
```

```
%}

class Vector {
public:
  double x, y, z;
  Vector();
  ~Vector();
  ... bunch of C++ methods ...
  %extend {
    char *__str__() {
      static char temp[256];
      sprintf(temp, "[ %g, %g, %g ]", $self->x, $self->y, $self->z);
      return &temp[0];
    }
  }
};
```

This code adds a `__str__` method to our class for producing a string representation of the object. In Python, such a method would allow us to print the value of an object using the `print` command.

```
>>>
>>> v = Vector();
>>> v.x = 3
>>> v.y = 4
>>> v.z = 0
>>> print(v)
[ 3.0, 4.0, 0.0 ]
>>>
```

The C++ 'this' pointer is often needed to access member variables, methods etc. The `$self` special variable should be used wherever you could use 'this'. The example above demonstrates this for accessing member variables. Note that the members dereferenced by `$self` must be public members as the code is ultimately generated into a global function and so will not have any access to non-public members. The implicit 'this' pointer that is present in C++ methods is not present in `%extend` methods. In order to access anything in the extended class or its base class, an explicit 'this' is required. The following example shows how one could access base class members:

```
struct Base {
  virtual void method(int v) {
    ...
  }
  int value;
};
struct Derived : Base {
};
%extend Derived {
  virtual void method(int v) {
    $self->Base::method(v); // akin to this->Base::method(v);
    $self->value = v;       // akin to this->value = v;
    ...
  }
}
```

The following special variables are expanded if used within a %extend block: $name, $symname, $overname, $decl, $fulldecl, $parentclassname and $parentclasssymname. The Special variables section provides more information each of these special variables.

The `%extend` directive follows all of the same conventions as its use with C structures. Please refer to the Adding member functions to C structures section for further details.

### 6.17.1 Replacing class methods

Suppose there is a method in a class that you need to replace and keep the method name the same. This can be achieved combining the `%extend` and `%ignore` directives covered earlier. Here is an example to replace the `MyClass::mymethod()`:

```
%extend MyClass {
  void mymethod() {
    std::cout << "swig mymethod" << std::endl;
  }
}

%ignore MyClass::mymethod;

%inline %{
class MyClass {
public:
  void mymethod() {
    std::cout << "class mymethod" << std::endl;
  }
};
%}
```

Or if your code organization makes more sense to put the `%extend` after the class definition, you would need the following:

```
%rename("") MyClass::mymethod; // unignores the method
```

before the `%extend` or SWIG will continue to ignore`mymethod()`, even in an `%extend`.

Note that you can call the class method from the method in `%extend`, just use `self->mymethod()` and it will call the class method, not the one in `%extend`.

## 6.18 Templates

Template type names may appear anywhere a type is expected in an interface file. For example:

```
void foo(vector<int> *a, int n);
void bar(std::array<int, 100> *x);
```

There are some restrictions on the use of non-type arguments. Simple literals are supported, and so are most constant expressions. However, there are some limitations on the use of '<' and '>' in constant expressions (but note that '<=' and '>=' are fully supported). For example:

```
void bar(std::array<int, 100> *x);               // OK
void bar(std::array<int, 2*50> *x);              // OK
void bar(std::array<int, (1<2 ? 100 : 50)> *x)   // OK
void bar(std::array<int, 1<2 ? 100 : 50> *x)     // Not supported
void bar(std::array<int, (2>1 ? 100 : 50)> *x)   // Not supported
```

The type system is smart enough to figure out clever games you might try to play with `typedef`. For instance, consider this code:

```
typedef int Integer;
void foo(vector<int> *x, vector<Integer> *y);
```

In this case, `vector<Integer>` is exactly the same type as `vector<int>`. The wrapper for `foo()` will accept either variant.

### 6.18.1 The %template directive

There are a couple of important points about template wrapping. First, a bare C++ template does not define any sort of runnable object-code for which SWIG can normally create a wrapper. Therefore, in order to wrap a template, you need to give SWIG information about a particular template instantiation (e.g., `vector<int>`, `array<double>`, etc.). Second, an instantiation name such as `vector<int>` is generally not a valid identifier name in most target languages. Thus, you will need to give the template instantiation a more suitable name such as `intvector`.

To illustrate, consider the following class template definition:

```
template<class T> class List {
private:
  T *data;
  int nitems;
  int maxitems;
public:
  List(int max) {
    data = new T [max];
    nitems = 0;
    maxitems = max;
  }
  ~List() {
    delete [] data;
  };
  void append(T obj) {
    if (nitems < maxitems) {
      data[nitems++] = obj;
    }
  }
  int length() {
    return nitems;
  }
  T get(int n) {
    return data[n];
  }
};
```

By itself, this class template is useless--SWIG simply ignores it because it doesn't know how to generate any code unless a definition of `T` is provided. The `%template` directive is required to instantiate the template for use in a target language. The directive requires an identifier name for use in the target language plus the template for instantiation. The example below instantiates `List<int>` for use as a class named `intList`:

```
%template(intList) List<int>;
```

The instantiation expands the template code as a C++ compiler would do and then makes it available under the given identifier name. Essentially it is the same as wrapping the following concept code where the class template definition has `T` expanded to `int` (note that this is not entirely valid syntax):

```
%rename(intList) List<int>;       // Rename to a suitable identifier
class List<int> {
private:
  int *data;
  int nitems;
  int maxitems;
public:
  List(int max);
  ~List();
  void append(int obj);
  int length();
  int get(int n);
};
```

The argument to `%template()` is the name of the instantiation in the target language. The name you choose should not conflict with any other declarations in the interface file with one exception---it is okay for the template name to match that of a typedef declaration. For example:

```
%template(intList) List<int>;
...
typedef List<int> intList;    // OK
```

The `%template` directive must always appear *after* the definition of the template to be expanded, so the following will work:

```
template<class T> class List { ... };
%template(intList) List<int>;
```

but if %template is used before the template definition, such as:

```
%template(intList) List<int>;
template<class T> class List { ... };
```

SWIG will generate an error:

```
example.i:3: Error: Template 'List' undefined.
```

Since the type system knows how to handle `typedef`, it is generally not necessary to instantiate different versions of a template for typenames that are equivalent. For instance, consider this code:

```
%template(intList) List<int>;
typedef int Integer;
...
void foo(List<Integer> *x);
```

In this case, `List<Integer>` is exactly the same type as `List<int>`. Any use of `List<Integer>` is mapped back to the instantiation of `List<int>` created earlier. Therefore, it is not correct to instantiate a new class for the type `Integer`. An attempt to do so such as:

```
%template(intList) List<int>;
%template(IntegerList) List<Integer>;   // Ignored
```

will result in the duplicate instantiation being ignored with a warning:

```
example.i:48: Warning 404: Duplicate template instantiation of 'List< Integer >' with name 'IntegerList' ignored,
example.i:47: Warning 404: previous instantiation of 'List< int >' with name 'intList'.
```

The template provided to `%template` for instantiation must be the actual template and not a typedef to a template.

```
typedef List<int> ListOfInt;

%template(intList) List<int>; // ok
%template(intList) ListOfInt; // illegal - Syntax error
```

### 6.18.2 Function templates

SWIG can also generate wrappers for function templates using a similar technique to that shown above for class templates. For example:

```
// Function template
template<class T> T max(T a, T b) { return a > b ? a : b; }

// Make some different versions of this function
%template(maxint) max<int>;
%template(maxdouble) max<double>;
```

In this case, `maxint` and `maxdouble` become unique names for specific instantiations of the function.

SWIG even supports overloaded templated functions. As usual the `%template` directive is used to wrap templated functions. For example:

```
template<class T> void foo(T x) { };
template<class T> void foo(T x, T y) { };

%template(foo) foo<int>;
```

This will generate two overloaded wrapper methods, the first will take a single integer as an argument and the second will take two integer arguments.

### 6.18.3 Default template arguments

The number of arguments supplied to `%template` should match that in the original template definition. Template default arguments are supported. For example:

```
template <typename T, int max=100> class vector {
...
};

%template(intvec) vector<int>;            // OK
%template(vec1000) vector<int, 1000>;     // OK
```

The `%template` directive should not be used to wrap the same template instantiation more than once. This also applies to default parameters where a template parameter specified in the instantiation is the same as the default parameter. For example:

```
%template(vec) vector<double>;            // OK
%template(vec100) vector<double, 100>;    // Ignored
```

will warn:

```
example.i:59: Warning 404: Duplicate template instantiation of 'vector< double,100 >' with name 'vec100' ignored,
example.i:58: Warning 404: previous instantiation of 'vector< double >' with name 'vec'.
```

If this was not ignored, the template expansion would result in two identical classes. An identical instantiation is only wrapped once in order to reduce code bloat.

**Compatibility Note**: Versions prior to SWIG-4.2.0 would sometimes not detect and prevent duplicate instantiations, such as when the wrapped name was different.

### 6.18.4 Template base classes

When a template is instantiated using `%template`, information about that class is saved by SWIG and used elsewhere in the program. For example, if you wrote code like this,

```
...
%template(intList) List<int>;
...
class UltraList : public List<int> {
  ...
};
```

then SWIG knows that `List<int>` was already wrapped as a class called `intList` and arranges to handle the inheritance correctly. If, on the other hand, nothing is known about `List<int>` , you will get a warning message similar to this:

```
example.h:42: Warning 401. Nothing known about class 'List< int >'. Ignored.
example.h:42: Warning 401. Maybe you forgot to instantiate 'List< int >' using %template.
```

If a class template inherits from another class template, you need to make sure that base classes are instantiated before derived classes. For example:

```
template<class T> class Foo {
...
};

template<class T> class Bar : public Foo<T> {
...
};

// Instantiate base classes first
%template(intFoo) Foo<int>;
%template(doubleFoo) Foo<double>;

// Now instantiate derived classes
%template(intBar) Bar<int>;
%template(doubleBar) Bar<double>;
```

The order is important since SWIG uses the instantiation names to properly set up the inheritance hierarchy in the resulting wrapper code (and base classes need to be wrapped before derived classes). Don't worry--if you get the order wrong, SWIG should generate a warning message.

If you have to instantiate a lot of different classes for many different types, you might consider writing a SWIG macro. For example:

```
%define TEMPLATE_WRAP(prefix, T...)
%template(prefix ## Foo) Foo<T >;
%template(prefix ## Bar) Bar<T >;
...
%enddef

TEMPLATE_WRAP(int, int)
TEMPLATE_WRAP(double, double)
TEMPLATE_WRAP(String, char *)
TEMPLATE_WRAP(PairStringInt, std::pair<string, int>)
...
```

Note the use of a vararg macro for the type T. If this wasn't used, the comma in the templated type in the last example would not be possible.

### 6.18.5 Empty template instantiation

Occasionally, you may need to tell SWIG about classes that are defined by templates, but which aren't supposed to be wrapped. Since SWIG is not able to automatically instantiate templates for this purpose, you must do it manually. To do this, simply use `%template()`, that is the empty template instantiation that omits providing a name. For example:

```
template<typename T> struct Traits {
  typedef T type;
};
%}

%template() Traits<int>; // instantiate Traits<int>, but don't wrap it

void traitor(Traits<int>::type val);
```

Without a template instantiation, SWIG does not know that the first parameter to the `traitor` function is type int and passing an integer to this function from any target language won't work. The empty template instantiation adds the appropriate type information into SWIG's type system, without forcing one to wrap the `Traits` class.

Duplicate template instantiation are not allowed, as described in the Default template arguments section above. There is one exception where a named template instantiation can be followed by an empty template instantiation. Duplicate empty template instantiations are silently ignored, unlike duplicate named template instantiations.

Unlike template class instantiations, template function instantiations must have a name. Consider the following:

```
template<class T> T tfunc(T x) { };
%template() tfunc<double>;
```

The empty template instantiation will be ignored with:

```
example.i:9: Warning 519: %template() contains no name. Template method ignored: tfunc< double >(double)
```

### 6.18.6 Template specialization

The SWIG template mechanism *does* support specialization. For instance, if you define a class like this,

```
template<> class List<int> {
private:
  int *data;
  int nitems;
```

```
  int maxitems;
public:
  List(int max);
  ~List();
  void append(int obj);
  int length();
  int get(int n);
};
```

then SWIG will use this code whenever the user expands `List<int>` . In practice, this may have very little effect on the underlying wrapper code since specialization is often used to provide slightly modified method bodies (which are ignored by SWIG). However, special SWIG directives such as `%typemap`, `%extend`, and so forth can be attached to a specialization to provide customization for specific types.

Partial template specialization is partially supported by SWIG. For example, this code defines a template that is applied when the template argument is a pointer.

```
template<class T> class List<T*> {
private:
  T *data;
  int nitems;
  int maxitems;
public:
  List(int max);
  ~List();
  void append(T obj);
  int length();
  T get(int n);
};
```

SWIG supports both template explicit specialization and partial specialization. Consider:

```
template<class T1, class T2> class Foo { };                    // (1) primary template
template<>                   class Foo<double *, int *> { };   // (2) explicit specialization
template<class T1, class T2> class Foo<T1, T2 *> { };          // (3) partial specialization
```

SWIG is able to properly match explicit instantiations:

```
Foo<double *, int *>     // explicit specialization matching (2)
```

SWIG implements template argument deduction so that the following partial specialization examples work just like they would with a C++ compiler:

```
Foo<int *, int *>        // partial specialization matching (3)
Foo<int *, const int *>  // partial specialization matching (3)
Foo<int *, int **>       // partial specialization matching (3)
```

### 6.18.7 Member templates

Member templates are supported. The underlying principle is the same as for normal templates--SWIG can't create a wrapper unless you provide more information about types. For example, a class with a member function template might look like this:

```
class Foo {
public:
  template<class T> void bar(T x, T y) { ... };
  ...
};
```

To expand the template, simply use `%template` inside the class.

```
class Foo {
public:
  template<class T> void bar(T x, T y) { ... };
  ...
  %template(barint)    bar<int>;
  %template(bardouble) bar<double>;
};
```

Or, if you want to leave the original class definition alone, just do this:

```
class Foo {
public:
  template<class T> void bar(T x, T y) { ... };
  ...
};
...
%extend Foo {
  %template(barint)    bar<int>;
  %template(bardouble) bar<double>;
};
```

or simply

```
class Foo {
public:
  template<class T> void bar(T x, T y) { ... };
  ...
};
...

%template(bari) Foo::bar<int>;
```

```
%template(bard) Foo::bar<double>;
```

In this case, the `%extend` directive is not needed, and `%template` does exactly the same job, i.e., it adds two new methods to the Foo class.

Now, if your target language supports overloading, you can even try

```
%template(bar) Foo::bar<int>;
%template(bar) Foo::bar<double>;
```

and since the two new wrapped methods have the same name 'bar', they will be overloaded, and when called, the correct method will be dispatched depending on the argument type.

When used with members, the `%template` directive may be placed in another class template. Here is a slightly perverse example:

```
// A template
template<class T> class Foo {
public:
  // A member template
  template<class S> T bar(S x, S y) { ... };
  ...
};

// Expand a few member templates
%extend Foo {
  %template(bari) bar<int>;
  %template(bard) bar<double>;
}

// Create some wrappers for the template
%template(Fooi) Foo<int>;
%template(Food) Foo<double>;
```

Miraculously, you will find that each expansion of `Foo` has member functions `bari()` and `bard()` added.

A common use of member templates is to define constructors for copies and conversions. For example:

```
template<class T1, class T2> struct pair {
  T1 first;
  T2 second;
  pair() : first(T1()), second(T2()) { }
  pair(const T1 &x, const T2 &y) : first(x), second(y) { }
  template<class U1, class U2> pair(const pair<U1, U2> &x)
                                   : first(x.first), second(x.second) { }
};
```

This declaration is perfectly acceptable to SWIG, but the constructor template will be ignored unless you explicitly expand it. To do that, you could expand a few versions of the constructor in the class template itself. For example:

```
%extend pair {
  %template(pair) pair<T1, T2>;        // Generate default copy constructor
};
```

When using `%extend` in this manner, notice how you can still use the template parameters in the original template definition.

Alternatively, you could expand the constructor template in selected instantiations. For example:

```
// Instantiate a few versions
%template(pairii) pair<int, int>;
%template(pairdd) pair<double, double>;

// Create a default constructor only
%extend pair<int, int> {
  %template(paird) pair<int, int>;         // Default constructor
};

// Create default and conversion constructors
%extend pair<double, double> {
  %template(paird) pair<double, double>;   // Default constructor
  %template(pairc) pair<int, int>;         // Conversion constructor
};
```

And if your target language supports overloading, then you can try instead:

```
// Create default and conversion constructors
%extend pair<double, double> {
  %template(pair) pair<double, double>;   // Default constructor
  %template(pair) pair<int, int>;         // Conversion constructor
};
```

In this case, the default and conversion constructors have the same name. Hence, SWIG will overload them and define an unique visible constructor, that will dispatch the proper call depending on the argument type.

### 6.18.8 Scoping and templates

The `%template` directive for a class template is the equivalent to an explicit instantiation of a C++ class template. The scope for a valid `%template` instantiation is the same as the scope required for a valid explicit instantiation of a C++ template. A definition of the template for the explicit instantiation must be in scope where the instantiation is declared and must not be enclosed within a different namespace.

For example, a few `%template` instantiations and C++ explicit instantiations are shown below:

```
namespace N {
```

```
  template<typename T> class C {};
}

// valid
%template(cin) N::C<int>;
template class N::C<int>;

// valid
namespace N {
  %template(cin) C<int>;
  template class C<int>;
}

// valid
using namespace N;
%template(cin) C<int>;
template class C<int>;

// valid
using N::C;
%template(cin) C<int>;
template class C<int>;

// ill-formed
namespace unrelated {
  using N::C;
  %template(cin) C<int>;
  template class C<int>;
}

// ill-formed
namespace unrelated {
  using namespace N;
  %template(cin) C<int>;
  template class C<int>;
}

// ill-formed
namespace unrelated {
  namespace N {
    %template(cin) C<int>;
    template class C<int>;
  }
}

// ill-formed
namespace unrelated {
  %template(cin) N::C<int>;
  template class N::C<int>;
}
```

When the scope is incorrect, such as for the ill-formed examples above, an error occurs:

```
cpp_template_scope.i:34: Error: 'C' resolves to 'N::C' and was incorrectly instantiated
in scope 'unrelated' instead of within scope 'N'.
```

A note for the C++ standard geeks out there; a valid instantiation is one which conforms to the C++03 standard as C++11 made a change to disallow using declarations and using directives to find a template.

```
// valid C++03, ill-formed C++11
using N::C;
template class C<int>;
```

**Compatibility Note**: Versions prior to SWIG-4.0.0 did not error out with incorrectly scoped `%template` declarations, but this led to numerous subtle template scope problems.

### 6.18.9 More on templates

If all of this isn't quite enough and you really want to make someone's head explode, SWIG directives such as `%rename`, `%extend`, and `%typemap` can be included directly in template definitions. For example:

```
// File : list.h
template<class T> class List {
  ...
public:
  %rename(__getitem__) get(int);
  List(int max);
  ~List();
  ...
  T get(int index);
  %extend {
    char *__str__() {
      /* Make a string representation */
      ...
    }
  }
};
```

In this example, the extra SWIG directives are propagated to *every* template instantiation.

It is also possible to separate these declarations from the class template. For example:

```
%rename(__getitem__) List::get;
%extend List {
  char *__str__() {
    /* Make a string representation */
```

```
    ...
  }
  /* Make a copy */
  T *__copy__() {
    return new List<T>(*$self);
  }
};

...
template<class T> class List {
  ...
  public:
  List() { }
  T get(int index);
  ...
};
```

When `%extend` is decoupled from the class definition, it is legal to use the same template parameters as provided in the class definition. These are replaced when the template is expanded. In addition, the `%extend` directive can be used to add additional methods to a specific instantiation. For example:

```
%template(intList) List<int>;

%extend List<int> {
  void blah() {
    printf("Hey, I'm an List<int>!\n");
  }
};
```

It is even possible to extend a class via `%extend` with template methods, for example:

```
%include <std_string.i>

%inline %{
class ExtendMe {
public:
  template <typename T>
  T do_stuff_impl(int a, T b, double d) {
    return b;
  }
};
%}

%extend ExtendMe {
  template<typename T>
  T do_overloaded_stuff(T b) {
    return $self->do_stuff_impl(0, b, 4.0);
  }
}
%template(do_overloaded_stuff) ExtendMe::do_overloaded_stuff<std::string>;
%template(do_overloaded_stuff) ExtendMe::do_overloaded_stuff<double>;
```

The wrapped `ExtendMe` class will then have two (overloaded) methods called `do_overloaded_stuff`.

**Compatibility Note**: Extending a class with template methods was added in version 3.0.12

Needless to say, SWIG's template support provides plenty of opportunities to break the universe. That said, an important final point is that **SWIG does not perform extensive error checking of templates!** Specifically, SWIG does not perform type checking nor does it check to see if the actual contents of the template declaration make any sense. Since the C++ compiler checks this when it compiles the resulting wrapper file, there is no practical reason for SWIG to duplicate this functionality.

As SWIG's template support does not perform type checking `%template` can be used as early as after a template declaration. You can, and rarely have to, use `%template` before the template parameters have been declared. For example:

```
template <class T> class OuterTemplateClass {};

// The nested class OuterClass::InnerClass inherits from the class template
// OuterTemplateClass<OuterClass::InnerStruct> and thus the template needs
// to be expanded with %template before the OuterClass declaration.
%template(OuterTemplateClass_OuterClass__InnerStruct)
  OuterTemplateClass<OuterClass::InnerStruct>


// Don't forget to use %feature("flatnested") for OuterClass::InnerStruct and
// OuterClass::InnerClass if the target language doesn't support nested classes.
class OuterClass {
  public:
    // Forward declarations:
    struct InnerStruct;
    class InnerClass;
};

struct OuterClass::InnerStruct {};

// Expanding the template at this point with %template is too late as the
// OuterClass::InnerClass declaration is processed inside OuterClass.

class OuterClass::InnerClass : public OuterTemplateClass<InnerStruct> {};
```

**Compatibility Note**: The first implementation of template support relied heavily on macro expansion in the preprocessor. Templates have been more tightly integrated into the parser and type system in SWIG-1.3.12 and the preprocessor is no longer used. Code that relied on preprocessing features in template expansion will no longer work. However, SWIG still allows the # operator to be used to generate a string from a template argument.

**Compatibility Note**: In earlier versions of SWIG, the`%template` directive introduced a new class name. This name could then be used with other directives. For example:

```
%template(vectori) vector<int>;
%extend vectori {
```

```
  void somemethod() { }
};
```

This behavior is no longer supported. Instead, you should use the original template name as the class name. For example:

```
%template(vectori) vector<int>;
%extend vector<int> {
  void somemethod() { }
};
```

Similar changes apply to typemaps and other customization features.

## 6.19 Namespaces

Support for C++ namespaces is comprehensive, but by default simple, however, some target languages can turn on more advanced namespace support via the nspace feature, described later. Code within unnamed namespaces is ignored as there is no external access to symbols declared within the unnamed namespace. Before detailing the default implementation for named namespaces, it is worth noting that the semantics of C++ namespaces is extremely non-trivial--especially with regard to the C++ type system and class machinery. At a most basic level, namespaces are sometimes used to encapsulate common functionality. For example:

```
namespace math {
  double sin(double);
  double cos(double);

  class Complex {
    double im, re;
  public:
    ...
  };
  ...
};
```

Members of the namespace are accessed in C++ by prepending the namespace prefix to names. For example:

```
double x = math::sin(1.0);
double magnitude(math::Complex *c);
math::Complex c;
...
```

At this level, namespaces are relatively easy to manage. However, things start to get very ugly when you throw in the other ways a namespace can be used. For example, selective symbols can be exported from a namespace with a `using` declaration:

```
using math::Complex;               // Using declaration
double magnitude(Complex *c);      // Namespace prefix stripped
```

Similarly, the contents of an entire namespace can be made available via a `using` directive:

```
using namespace math;              // Using directive
double x = sin(1.0);
double magnitude(Complex *c);
```

Alternatively, a namespace can be aliased:

```
namespace M = math;
double x = M::sin(1.0);
double magnitude(M::Complex *c);
```

Using combinations of these features, it is possible to write head-exploding code like this:

```
namespace A {
  class Foo {
  };
}

namespace B {
  namespace C {
    using namespace A;
  }
  typedef C::Foo FooClass;
}

namespace BIGB = B;

namespace D {
  using BIGB::FooClass;
  class Bar : public FooClass {
  }
};

class Spam : public D::Bar {
};

void evil(A::Foo *a, B::FooClass *b, B::C::Foo *c, BIGB::FooClass *d,
          BIGB::C::Foo *e, D::FooClass *f);
```

Given the possibility for such perversion, it's hard to imagine how every C++ programmer might want such code wrapped into the target language. Clearly this code defines three different classes. However, one of those classes is accessible under at least six different names!

SWIG fully supports C++ namespaces in its internal type system and class handling code. If you feed SWIG the above code, it will be parsed correctly, it will generate compilable wrapper code, and it will produce a working scripting language module. However, the default wrapping behavior is to flatten namespaces in the target language. This means that the contents of all namespaces are merged together in the resulting scripting language module. For example, if you have code like this,

```
%module foo
namespace foo {
  void bar(int);
  void spam();
}

namespace bar {
  void blah();
}
```

then SWIG simply creates three wrapper functions `bar()`, `spam()`, and `blah()` in the target language. SWIG does not prepend the names with a namespace prefix nor are the functions packaged in any kind of nested scope. Note that the default handling of flattening all the namespace scopes in the target language can be changed via the nspace feature.

There is some rationale for taking this approach. Since C++ namespaces are often used to define modules in C++, there is a natural correlation between the likely contents of a SWIG module and the contents of a namespace. For instance, it would not be unreasonable to assume that a programmer might make a separate extension module for each C++ namespace. In this case, it would be redundant to prepend everything with an additional namespace prefix when the module itself already serves as a namespace in the target language. Or put another way, if you want SWIG to keep namespaces separate, simply wrap each namespace with its own SWIG interface.

Because namespaces are flattened, it is possible for symbols defined in different namespaces to generate a name conflict in the target language. For example:

```
namespace A {
  void foo(int);
}
namespace B {
  void foo(double);
}
```

When this conflict occurs, you will get an error message that resembles this:

```
example.i:26: Error: 'foo' is multiply defined in the generated target language module.
example.i:23: Error: Previous declaration of 'foo'
```

To resolve this error, simply use `%rename` to disambiguate the declarations. For example:

```
%rename(B_foo) B::foo;
...
namespace A {
  void foo(int);
}
namespace B {
  void foo(double);     // Gets renamed to B_foo
}
```

Similarly, `%ignore` can be used to ignore declarations.

C++ `using` directives and `using` declarations do not add any code to the generated wrapper code. However, there is an exception in one context, see Using declarations and inheritance for introducing members of a base class into a derived class definition. C++ `using` declarations and directives *are* used by the internal type system to track type-names. Therefore, if you have code like this:

```
namespace A {
  typedef int Integer;
}
using namespace A;
void foo(Integer x);
```

SWIG knows that `Integer` is the same as `A::Integer` which is the same as `int`.

Namespaces may be combined with templates. If necessary, the `%template` directive can be used to expand a template defined in a different namespace. For example:

```
namespace foo {
  template<typename T> T max(T a, T b) { return a > b ? a : b; }
}

using foo::max;

%template(maxint)   max<int>;           // Okay.
%template(maxfloat) foo::max<float>;    // Okay (qualified name).

namespace bar {
  using namespace foo;
  %template(maxdouble)  max<double>;    // Okay.
}
```

The combination of namespaces and other SWIG directives may introduce subtle scope-related problems. The key thing to keep in mind is that all SWIG generated wrappers are produced in the *global* namespace. Symbols from other namespaces are always accessed using fully qualified names---names are never imported into the global space unless the interface happens to do so with a `using` declaration. In almost all cases, SWIG adjusts typenames and symbols to be fully qualified. However, this is not done in code fragments such as function bodies, typemaps, exception handlers, and so forth. For example, consider the following:

```
namespace foo {
  typedef int Integer;
  class bar {
    public:
    ...
  };
}
```

```
%extend foo::bar {
  Integer add(Integer x, Integer y) {
    Integer r = x + y;        // Error. Integer not defined in this scope
    return r;
  }
};
```

In this case, SWIG correctly resolves the added method parameters and return type to `foo::Integer`. However, since function bodies aren't parsed and such code is emitted in the global namespace, this code produces a compiler error about `Integer`. To fix the problem, make sure you use fully qualified names. For example:

```
%extend foo::bar {
  Integer add(Integer x, Integer y) {
    foo::Integer r = x + y;        // Ok.
    return r;
  }
};
```

**Note:** SWIG does *not* propagate `using` declarations to the resulting wrapper code. If these declarations appear in an interface, they should *also* appear in any header files that might have been included in a `%{ ... %}` section. In other words, don't insert extra `using` declarations into a SWIG interface unless they also appear in the underlying C++ code.

**Note:** Code inclusion directives such as `%{ ... %}` or `%inline %{ ... %}` should not be placed inside a namespace declaration. The code emitted by these directives will not be enclosed in a namespace and you may get very strange results. If you need to use namespaces with these directives, consider the following:

```
// Good version
%inline %{
namespace foo {
  void bar(int) { ... }
  ...
}
%}

// Bad version.  Emitted code not placed in namespace.
namespace foo {
%inline %{
  void bar(int) { ... }   /* I'm bad */
  ...
  %}
}
```

**Note:** When the `%extend` directive is used inside a namespace, the namespace name is included in the generated functions. For example, if you have code like this,

```
namespace foo {
  class bar {
    public:
      %extend {
        int blah(int x);
      };
  };
}
```

the added method `blah()` is mapped to a function `int foo_bar_blah(foo::bar *self, int x)`. This function resides in the global namespace.

**Note:** Although namespaces are flattened in the target language, the SWIG generated wrapper code observes the same namespace conventions as used in the input file. Thus, if there are no symbol conflicts in the input, there will be no conflicts in the generated code.

**Note:** In the same way that no resolution is performed on parameters, a conversion operator name must match exactly to how it is defined. Do not change the qualification of the operator. For example, suppose you had an interface like this:

```
namespace foo {
  class bar;
  class spam {
    public:
    ...
    operator bar();      // Conversion of spam -> bar
    ...
  };
}
```

The following is how the feature is expected to be written for a successful match:

```
%rename(tofoo) foo::spam::operator bar();
```

The following does not work as no namespace resolution is performed in the matching of conversion operator names:

```
%rename(tofoo) foo::spam::operator foo::bar();
```

Note, however, that if the operator is defined using a qualifier in its name, then the feature must use it too...

```
%rename(tofoo) foo::spam::operator bar();      // will not match
%rename(tofoo) foo::spam::operator foo::bar(); // will match
namespace foo {
  class bar;
  class spam {
    public:
    ...
    operator foo::bar();
    ...
  };
}
```

**Compatibility Note:** Versions of SWIG prior to 1.3.32 were inconsistent in this approach. A fully qualified name was usually required, but would not work in some situations.

**Note:** The flattening of namespaces is only intended to serve as a basic namespace implementation. More advanced handling of namespaces is discussed next.

### 6.19.1 The nspace feature for namespaces

The `nspace` feature operates in two modes. Firstly, in a simple enable/disable mode to mirror the C++ namespaces into the target language specific concept of a C++ namespace. Secondly, a complex mode for modifying, renaming or moving the hierarchies of the language specific concept of a C++ namespace.

The types of symbols that the feature can be applied to are any class, struct, union or enum declared within a named namespace. This also includes templates of the aforementioned types. The feature wraps the type within the target language specific concept of a namespace, such as a Java package or C# namespace. Only some target languages provide support for the `nspace` feature. Please see the target language specific sections to see if the language you are interested in supports the nspace feature.

#### 6.19.1.1 %nspace for mirroring namespace hierarchies

In this simple mode the `nspace` feature works as a feature flag to enable or disable the feature for a given C++ symbol. As described earlier, all namespace are flattened by default, hence the `nspace` feature is disabled by default. When the feature is enabled, the C++ namespace hierarchies are mirrored into the target language concept of a namespace.

The feature flag is demonstrated below using the following example:

```
%feature("nspace") MyWorld::Material::Color;
%nspace MyWorld::Wrapping::Color; // %nspace is a macro for %feature("nspace")

namespace MyWorld {
  namespace Material {
    class Color {
    ...
    };
  }
  namespace Wrapping {
    class Color {
    ...
    };
  }
}
```

By default, without the above `nspace` feature flags (or an appropriate `%rename`), SWIG outputs the following warning resulting in just one of the `Color` classes being available for use from the target language:

```
example.i:9: Error: 'Color' is multiply defined in the generated target language module.
example.i:5: Error: Previous declaration of 'Color'
```

Let's consider C# as the target language. With the two `nspace` feature flags, the two C# `Color` proxy classes are generated into the equivalent C# namespaces that mirror the C++ namespace hierarchies. A fully qualified constructor call of each these two types in C# is then:

```
var materialColor = new MyWorld.Material.Color();
var wrappingColor = new MyWorld.Wrapping.Color();
```

Without the `nspace` feature flag, no namespaces are available in C# and the fully qualified constructor call of the type in C# would simply be:

```
var materialColor = new Color();
```

Note that the `nspace` feature does not apply to variables and functions simply declared in a namespace. For example, the following symbols cannot co-exist in the target language without renaming. This may change in a future version.

```
namespace MyWorld {
  namespace Material {
    int quantity;
    void dispatch();
  }
  namespace Wrapping {
    int quantity;
    void dispatch();
  }
}
```

**Compatibility Note:** The simple `%nspace` feature flag was first introduced in SWIG-2.0.0.

#### 6.19.1.2 %nspacemove for modifying namespace hierarchies

The more complex mode for `nspace` provides the ability to move a type into a differently named target language equivalent of a namespace. This allows a fully flexible approach to mapping C++ namespaces into a target language equivalent of a namespace, such as:

- Renaming a namespace.
- Renaming any number of sub-namespaces of a namespace.
- Moving a type to a completely different namespace.
- Splitting many types in one namespace into multiple different namespaces.
- Adding nested sub-namespaces to a namespace.
- Removing nested sub-namespaces of namespace.
- Combinations of the above.

The `nspace` feature attaches to a C++ symbol and provides a target language namespace for that symbol to move to. The `%nspacemove` directive is a macro for the `nspace` feature as follows:

```
#define %nspacemove(NAMESPACE) %feature("nspace", #NAMESPACE)
```

where the target namespace is provided in `NAMESPACE` and is specified in valid C++ syntax, such as `A::B::C`. Internally, SWIG converts the C++ namespace into a target language equivalent, so this might for example, result in a C# namespace called `A.B.C` or a Java package named `A.B.C`. Note that the target namespace does not have to actually exist in any of the C++ code that SWIG parses; it could be entirely made-up.

Either the `%nspacemove` macro or the more verbose `%feature` syntax can be used. Please see the Features and the %feature directive section for more details of SWIG features.

An example follows:

```
// Enable the nspace feature flag for all symbols
%nspace;

// Override the nspace feature for a subset of symbols by moving them into different namespaces
%nspacemove(A) A::B::C::Struct1;
%feature("nspace", "A::B::X") A::B::C::Struct2;
%nspacemove(A::B::C::D) A::B::C::Struct4;
%nspacemove(Somewhere::Else) A::B::C::Struct5;

%inline %{
namespace A {
  namespace B {
    namespace C {
      struct Struct1 {
        // ...
      };
      struct Struct2 {
        // ...
      };
      struct Struct3 {
        // ...
      };
      struct Struct4 {
        // ...
      };
      struct Struct5 {
        // ...
      };
    }
  }
}
%}
```

The C# code below constructs each of the above classes. It shows the namespaces that the C# proxy classes have been moved into, noting though that `Struct4` merely exactly mirrors the C++ namespace hierarchy as it has the `nspace` feature flag attached to it.

```
var s1 = new A.Struct1();
var s2 = new A.B.X.Struct2();
var s3 = new A.B.C.Struct3();
var s4 = new A.B.C.D.Struct4();
var s5 = new Somewhere.Else.Struct5();
```

#### 6.19.1.3 More about the nspace feature

When the `nspace` feature is attached to a class or enum, all contained symbols (members) are also automatically moved into the target language namespace. Contained symbols include all enum values, static and non-static class members as well as nested classes. There is no need for additional `nspace` features to be specified for all the contained symbols. Below is an example showing this. It also shows the nspace feature working for templates.

```
// Easy way to move all the Structure template instantiations into a different namespace
%nspacemove(A::Different::Space) Space::Structure;

%inline %{
namespace Space {
  template<typename T>
  struct Structure {
    static int Count;
    static void StaticMethod(T t) {
      // ...
    }
    struct NestedStruct {
      // ...
    };
  };
  template<typename T>
  int Structure<T>::Count = 0;
}
%}
%template(StructureInt) Space::Structure<int>;
%template(StructureString) Space::Structure<const char *>;
```

The C# code below shows the full C# namespace `A.Different.Space` being used as one would expect for all the contained symbols within a C# class.

```
var s = new A.Different.Space.StructureInt();
int count = A.Different.Space.StructureInt.Count;
var n = new A.Different.Space.StructureInt.NestedStruct();
A.Different.Space.StructureInt.StaticMethod(99);
A.Different.Space.StructureString.StaticMethod("hi");
```

Any attempt to give a different namespace value to a nested class or enum will issue a warning. For example, adding the following to the above in an attempt to move one of the instantiated nested classes into another namespace like this:

```
%nspacemove(Bad::Space) Space::Structure<int>::NestedStruct;
... rest of example above ...
```

will result in the following warning:

```
Warning 406: Ignoring nspace setting (Bad::Space) for 'Space::Structure< int >::NestedStruct',
Warning 406: as it conflicts with the nspace setting (A::Different::Space) for outer class 'Space::Structure< int >'.
```

**Compatibility Note:** Modifying namespace hierarchies via `%nspacemove` was first introduced in SWIG-4.3.0.

## 6.20 Renaming templated types in namespaces

As has been mentioned, when %rename includes parameters, the parameter types must match exactly (no typedef or namespace resolution is performed). SWIG treats templated types slightly differently and has an additional matching rule so unlike non-templated types, an exact match is not always required. If the fully qualified templated type is specified, it will have a higher precedence over the generic template type. In the example below, the generic template type is used to rename to `bbb` and the fully qualified type is used to rename to`ccc`.

```
%rename(bbb) Space::ABC::aaa(T t);                    // will match but with lower precedence than ccc
%rename(ccc) Space::ABC<Space::XYZ>::aaa(Space::XYZ t);// will match but with higher precedence
                                                      // than bbb

namespace Space {
  class XYZ {};
  template<typename T> struct ABC {
    void aaa(T t) {}
  };
}
%template(ABCXYZ) Space::ABC<Space::XYZ>;
```

It should now be apparent that there are many ways to achieve a renaming with %rename. This is demonstrated by the following two examples, which are effectively the same as the above example. Below shows how %rename can be placed inside a namespace.

```

namespace Space {
  %rename(bbb) ABC::aaa(T t);                    // will match but with lower precedence than ccc
  %rename(ccc) ABC<Space::XYZ>::aaa(Space::XYZ t);// will match but with higher precedence than bbb
  %rename(ddd) ABC<Space::XYZ>::aaa(XYZ t);       // will not match
}

namespace Space {
  class XYZ {};
  template<typename T> struct ABC {
    void aaa(T t) {}
  };
}
%template(ABCXYZ) Space::ABC<Space::XYZ>;
```

Note that `ddd` does not match as there is no namespace resolution for parameter types and the fully qualified type must be specified for template type expansion. The following example shows how %rename can be placed within %extend.

```
namespace Space {
  %extend ABC {
    %rename(bbb) aaa(T t);         // will match but with lower precedence than ccc
  }
  %extend ABC<Space::XYZ> {
    %rename(ccc) aaa(Space::XYZ t);// will match but with higher precedence than bbb
    %rename(ddd) aaa(XYZ t);       // will not match
  }
}

namespace Space {
  class XYZ {};
  template<typename T> struct ABC {
    void aaa(T t) {}
  };
}
%template(ABCXYZ) Space::ABC<Space::XYZ>;
```

## 6.21 Exception specifications

When C++ programs utilize exceptions, exceptional behavior is sometimes specified as part of a function or method declaration. For example:

```

class Error { };

class Foo {
public:
  ...
  void blah() throw(Error);
  ...
};
```

If an exception specification is used, SWIG automatically generates wrapper code for catching the indicated exception and, when possible, rethrowing it into the target language, or converting it into an error in the target language otherwise. For example, in Python, you can write code like this:

```
f = Foo()
try:
    f.blah()
except Error, e:
    # e is a wrapped instance of "Error"
```

Details of how to tailor code for handling the caught C++ exception and converting it into the target language's exception/error handling mechanism is outlined in the "throws" typemap section.

Since exception specifications are sometimes only used sparingly, this alone may not be enough to properly handle C++ exceptions. To do that, a different set of special SWIG directives are used. Consult the "Exception handling with %exception" section for details. The next section details a way of simulating an exception specification or replacing an existing one.

## 6.22 Exception handling with %catches

Exceptions are automatically handled for methods with an exception specification. Similar handling can be achieved for methods without exception specifications through the `%catches` feature. It is also possible to replace any declared exception specification using the `%catches` feature. In fact, `%catches` uses the same "throws" typemaps that SWIG uses for exception specifications in handling exceptions. The `%catches` feature must contain a list of possible types that can be thrown. For each type that is in the list, SWIG will generate a catch handler, in the

same way that it would for types declared in the exception specification. Note that the list can also include the catch all specification "...". For example,

```
struct EBase { virtual ~EBase(); };
struct Error1 : EBase { };
struct Error2 : EBase { };
struct Error3 : EBase { };
struct Error4 : EBase { };

%catches(Error1, Error2, ...) Foo::bar();
%catches(EBase) Foo::blah();

class Foo {
public:
  ...
  void bar();
  void blah() throw(Error1, Error2, Error3, Error4);
  ...
};
```

For the `Foo::bar()` method, which can throw anything, SWIG will generate catch handlers for `Error1`, `Error2` as well as a catch all handler (...). Each catch handler will convert the caught exception and convert it into a target language error/exception. The catch all handler will convert the caught exception into an unknown error/exception.

Without the `%catches` feature being attached to `Foo::blah()`, SWIG will generate catch handlers for all of the types in the exception specification, that is, `Error1, Error2, Error3, Error4`. However, with the `%catches` feature above, just a single catch handler for the base class, `EBase` will be generated to convert the C++ exception into a target language error/exception.

## 6.23 Pointers to Members

Starting with SWIG-1.3.7, there is limited parsing support for pointers to C++ class members. For example:

```
double do_op(Object *o, double (Object::*callback)(double, double));
extern double (Object::*fooptr)(double, double);
%constant double (Object::*FOO)(double, double) = &Object::foo;
```

Although these kinds of pointers can be parsed and represented by the SWIG type system, few language modules know how to handle them due to implementation differences from standard C pointers. Readers are *strongly* advised to consult an advanced text such as the "The Annotated C++ Manual" for specific details.

When pointers to members are supported, the pointer value might appear as a special string like this:

```
>>> print(example.FOO)
_ff0d54a800000000_m_Object__f_double_double__double
>>>
```

In this case, the hexadecimal digits represent the entire value of the pointer which is usually the contents of a small C++ structure on most machines.

SWIG's type-checking mechanism is also more limited when working with member pointers. Normally SWIG tries to keep track of inheritance when checking types. However, no such support is currently provided for member pointers.

## 6.24 Smart pointers and operator->()

In some C++ programs, objects are often encapsulated by smart-pointers or proxy classes. This is sometimes done to implement automatic memory management (reference counting) or persistence. Typically a smart-pointer is defined by a class template where the -> operator has been overloaded. This class is then wrapped around some other class. For example:

```
// Smart-pointer class
template<class T> class SmartPtr {
  T *pointee;
public:
  SmartPtr(T *p) : pointee(p) { ... }
  T *operator->() {
    return pointee;
  }
  ...
};

// Ordinary class
class Foo_Impl {
public:
  int x;
  virtual void bar();
  ...
};

// Smart-pointer wrapper
typedef SmartPtr<Foo_Impl> Foo;

// Create smart pointer Foo
Foo make_Foo() {
  return SmartPtr<Foo_Impl>(new Foo_Impl());
}

// Do something with smart pointer Foo
void do_something(Foo f) {
  printf("x = %d\n", f->x);
  f->bar();
}

// Call the wrapped smart pointer proxy class in the target language 'Foo'
%template(Foo) SmartPtr<Foo_Impl>;
```

A key feature of this approach is that by defining `operator->` the methods and attributes of the object wrapped by a smart pointer are transparently accessible. For example, expressions such as these (from the previous example),

```
f->x
f->bar()
```

are transparently mapped to the following

```
(f.operator->())->x;
(f.operator->())->bar();
```

When generating wrappers, SWIG tries to emulate this functionality to the extent that it is possible. To do this, whenever `operator->()` is encountered in a class, SWIG looks at its returned type and uses it to generate wrappers for accessing attributes of the underlying object. For example, wrapping the above code produces wrappers like this:

```
int Foo_x_get(Foo *f) {
  return (*f)->x;
}
void Foo_x_set(Foo *f, int value) {
  (*f)->x = value;
}
void Foo_bar(Foo *f) {
  (*f)->bar();
}
```

These wrappers take a smart-pointer instance as an argument, but dereference it in a way to gain access to the object returned by `operator->()`. You should carefully compare these wrappers to those in the first part of this chapter (they are slightly different).

The end result is that access looks very similar to C++. For example, you could do this in Python:

```
>>> f = make_Foo()
>>> print(f.x)
0
>>> f.bar()
>>>
```

When generating wrappers through a smart-pointer, SWIG tries to generate wrappers for all methods and attributes that might be accessible through `operator->()`. This includes any methods that might be accessible through inheritance. However, there are a number of restrictions:

- Member variables and methods are wrapped through a smart pointer. Enumerations, constructors, and destructors are not wrapped.
- If the smart-pointer class and the underlying object both define a method or variable of the same name, then the smart-pointer version has precedence. For example, if you have this code

  ```
  class Foo {
  public:
    int x;
  };

  class Bar {
  public:
    int x;
    Foo *operator->();
  };
  ```

  then the wrapper for `Bar::x` accesses the `x` defined in `Bar`, and not the `x` defined in `Foo`.

If your intent is to only expose the smart-pointer class in the interface, it is not necessary to wrap both the smart-pointer class and the class for the underlying object. However, you must still tell SWIG about both classes if you want the technique described in this section to work. To only generate wrappers for the smart-pointer class, you can use the %ignore directive. For example:

```
%ignore Foo;
class Foo {       // Ignored
};

class Bar {
public:
  Foo *operator->();
  ...
};
```

Alternatively, you can import the definition of `Foo` from a separate file using `%import`.

**Note:** When a class defines `operator->()`, the operator itself is wrapped as a method `__deref__()`. For example:

```
f = Foo()               # Smart-pointer
p = f.__deref__()       # Raw pointer from operator->
```

**Note:** To disable the smart-pointer behavior, use `%ignore` to ignore `operator->()`. For example:

```
%ignore Bar::operator->;
```

**Note:** Smart pointer support was first added in SWIG-1.3.14.

## 6.25 C++ reference counted objects - ref/unref feature

Another similar idiom in C++ is the use of reference counted objects. Consider for example:

```
class RCObj  {
  // implement the ref counting mechanism
  int add_ref();
```

```
  int del_ref();
  int ref_count();

public:
  virtual ~RCObj() = 0;

  int ref() const {
    return add_ref();
  }

  int unref() const {
    if (ref_count() == 0 || del_ref() == 0 ) {
      delete this;
      return 0;
    }
    return ref_count();
  }
};


class A : RCObj {
public:
  A();
  int foo();
};


class B {
  A *_a;

public:
  B(A *a) : _a(a) {
    a->ref();
  }

  ~B() {
    a->unref();
  }
};

int main() {
  A *a  = new A();       // (count: 0)
  a->ref();           // 'a' ref here (count: 1)

  B *b1 = new B(a);   // 'a' ref here (count: 2)
  if (1 + 1 == 2) {
    B *b2 = new B(a); // 'a' ref here (count: 3)
    delete b2;        // 'a' unref, but not deleted (count: 2)
  }

  delete b1;          // 'a' unref, but not deleted (count: 1)
  a->unref();         // 'a' unref and deleted (count: 0)
}
```

In the example above, the 'A' class instance 'a' is a reference counted object, which can't be deleted arbitrarily since it is shared between the objects 'b1' and 'b2'. 'A' is derived from a *Reference Counted Object* 'RCObj', which implements the ref/unref idiom.

To tell SWIG that 'RCObj' and all its derived classes are reference counted objects, use the "ref" and "unref" features. These are also available as `%refobject` and `%unrefobject`, respectively. For example:

```
%module example
...

%feature("ref")   RCObj "$this->ref();"
%feature("unref") RCObj "$this->unref();"

%include "rcobj.h"
%include "A.h"
...
```

where the code passed to the "ref" and "unref" features will be executed as needed whenever a new object is passed to Python, or when Python tries to release the proxy object instance, respectively.

On the Python side, the use of a reference counted object is no different to any other regular instance:

```
def create_A():
    a = A()         # SWIG ref 'a' - new object is passed to Python (count: 1)
    b1 = B(a)       # C++ ref 'a (count: 2)
    if 1 + 1 == 2:
        b2 = B(a)   # C++ ref 'a' (count: 3)
    return a        # 'b1' and 'b2' are released and deleted, C++ unref 'a' twice (count: 1)

a = create_A()      # (count: 1)
exit                # 'a' is released, SWIG unref 'a' called in the destructor wrapper (count: 0)
```

Note that the user doesn't explicitly need to call 'a->ref()' nor 'a->unref()' (and neither 'delete a'). Instead, SWIG takes cares of executing the "ref" and "unref" calls as needed. If the user doesn't specify the "ref/unref" feature for a type, SWIG will produce code equivalent to defining these features:

```
%feature("ref")   ""
%feature("unref") "delete $this;"
```

In other words, SWIG will not do anything special when a new object is passed to Python, and it will always 'delete' the underlying object when Python releases the proxy instance.

The %newobject feature is designed to indicate to the target language that it should take ownership of the returned object. When used in conjunction with a type that has the "ref" feature associated with it, it additionally emits the code in the "ref" feature into the C++ wrapper. Consider wrapping the following factory function in addition to the above:

```
%newobject AFactory;
A *AFactory() {
  return new A();
}
```

The `AFactory` function now acts much like a call to the`A` constructor with respect to memory handling:

```
a = AFactory()    # SWIG ref 'a' due to %newobject (count: 1)
exit              # 'a' is released, SWIG unref 'a' called in the destructor wrapper (count: 0)
```

## 6.26 Using declarations and inheritance

C++ `using` declarations are sometimes used to introduce members of base classes. For example:

```
class Foo {
public:
  int blah(int x);
};

class Bar {
public:
  double blah(double x);
};

class FooBar : public Foo, public Bar {
public:
  using Foo::blah;
  using Bar::blah;
  char *blah(const char *x);
};
```

In this example, the `using` declarations make different versions of the overloaded `blah()` method accessible from the derived class. For example:

```
FooBar *f;
f->blah(3);         // Ok. Invokes Foo::blah(int)
f->blah(3.5);       // Ok. Invokes Bar::blah(double)
f->blah("hello");   // Ok. Invokes FooBar::blah(const char *);
```

SWIG emulates the same functionality when creating wrappers. For example, if you wrap this code in Python, the module works just like you would expect:

```
>>> import example
>>> f = example.FooBar()
>>> f.blah(3)
>>> f.blah(3.5)
>>> f.blah("hello")
```

The C++11 standard supports using declarations for inheriting constructors and this is covered in Object construction improvement.

C++ `using` declarations can also be used to change access when applicable. For example, protected methods in a base class can be made public in a derived class:

```
class Foo {
protected:
  int x;
  int blah(int x);
};

class Bar : public Foo {
public:
  using Foo::x;       // Make x public
  using Foo::blah;    // Make blah public
};
```

This also works in SWIG---the exposed declarations will be wrapped normally.

When `using` declarations are used as shown in these examples, declarations from the base classes are copied into the derived class and wrapped normally. When copied, the declarations retain any properties that might have been attached using `%rename` , `%ignore`, or `%feature`. Thus, if a method is ignored in a base class, it will also be ignored by a `using` declaration.

Because a `using` declaration does not provide fine-grained control over the declarations that get imported, because a single `using` declaration may introduce multiple methods, it may be difficult to manage such declarations in applications that make heavy use of SWIG customization features. If you can't get `using` to work correctly, you can always modify the C++ code to handle SWIG differently such as:

```
class FooBar : public Foo, public Bar {
public:
#ifndef SWIG
  using Foo::blah;
  using Bar::blah;
#else
  int blah(int x);         // explicitly tell SWIG about other declarations
  double blah(double x);
#endif

  char *blah(const char *x);
};
```

If the C++ code being wrapped cannot be changed, make judicious usage of `%extend` and `%rename` to ignore and unignore declarations. The example below is effectively the same as above:

```
%extend FooBar {
  int blah(int x) { return $self->Foo::blah(x); }
  double blah(double x) { return $self->Bar::blah(x); }
```

```
}
%ignore FooBar::blah; // ignore all FooBar::blah below
%rename("") FooBar::blah(const char *x); // parameterized unignore

class FooBar : public Foo, public Bar {
public:
  using Foo::blah;
  using Bar::blah;
  char *blah(const char *x);
};
```

**Notes:**

- If a derived class introduces a method defined in a base class via a `using` declaration, there won't be a conflict due to incorrect additional methods. For example:

```
class Foo {
public:
  int blah(int );
  double blah(double);
};

class Bar : public Foo {
public:
  using Foo::blah;    // Only introduces blah(double);
  int blah(int);
};
```

- Renaming methods may prevent methods from being introduced into the derived class via `using` declarations. For example:

```
%rename(blah_long) Foo::blah(long);
class Foo {
public:
  int blah(int);
  long blah(long);  // Renamed to blah_long
};

class Bar : public Foo {
public:
  using Foo::blah;     // Only introduces blah(int)
  double blah(double x);
};
```

  The problem here is `Foo::blah` is renamed to `blah_long` in the target language, but the `using` declaration in Bar is not renamed in the target language and thinks all introduced methods should simply be called `blah`. It is not clear what target language names should be used in Bar and so the conflicting names are effectively ignored as they are not introduced into the derived class for the target language wrappers. In such situations SWIG will emit a warning:

```
example.i:15: Warning 526: Using declaration Foo::blah, with name 'blah', is not actually using
example.i:10: Warning 526: the method from Foo::blah(long), with name 'blah_long', as the names are different.
```

  **Compatibility Note:** This warning message was introduced in SWIG-4.1.0. Prior versions also effectively ignored the using declaration for the same reasons, but were silent about it.

  If methods really need different names, please use of combinations of `%rename`, `%ignore` and `%extend` to achieve the desired outcome.

## 6.27 Nested classes

If the target language supports the nested classes concept (like Java), the nested C++ classes are wrapped as nested target language proxy classes. (In case of Java - "static" nested classes.) Only public nested classes are wrapped. Otherwise there is little difference between nested and normal classes.

If the target language doesn't support nested classes directly, or the support is not implemented in the language module (like for Python currently), then the visible nested classes are moved to the same name space as the containing class (nesting hierarchy is "flattened"). The same behaviour may be turned on for C# and Java by the %feature ("flatnested"); If there is a class with the same name in the outer namespace the inner class (or the global one) may be renamed or ignored:

```
%rename (Bar_Foo) Bar::Foo;
class Foo {};
class Bar {
  public:
  class Foo {};
};
```

If a nested class, within an outer class, has to be used as a template parameter within the outer class, then the template will have to be instantiated with `%template` before the beginning of the outer class. An example can be found in the Templates section.

**Compatibility Note:** Prior to SWIG-3.0.0, there was limited nested class support. Nested classes were treated as opaque pointers. However, there was a workaround for nested class support in these older versions requiring the user to replicate the nested class in the global scope, adding in a typedef for the nested class in the global scope and using the "nestedworkaround" feature on the nested class. This resulted in approximately the same behaviour as the "flatnested" feature. With proper nested class support now available in SWIG-3.0.0, this feature was deprecated and has now been removed. If you see the following error:

```
example.i:8: Error: Unknown directive '%nestedworkaround'
```

consider using the "flatnested" feature discussed above which generates a non-nested proxy class, like the "nestedworkaround" feature did. Alternatively, use the default nested class code generation, which may generate an equivalent to a nested proxy class in the target language, depending on the target language support.

SWIG-1.3.40 and earlier versions did not have the `nestedworkaround` feature and the generated code resulting from parsing nested classes did not always compile. Nested class warnings could also not be suppressed using %warnfilter.

## 6.28 A brief rant about const-correctness

A common issue when working with C++ programs is dealing with all possible ways in which the `const` qualifier (or lack thereof) will break your program, all programs linked against your program, and all programs linked against those programs.

Although SWIG knows how to correctly deal with `const` in its internal type system and it knows how to generate wrappers that are free of const-related warnings, SWIG does not make any

attempt to preserve const-correctness in the target language. Thus, it is possible to pass `const` qualified objects to non-const methods and functions. For example, consider the following code in C++:

```
const Object * foo();
void bar(Object *);

...
// C++ code
void blah() {
  bar(foo());         // Error: bar discards const
};
```

Now, consider the behavior when wrapped into a Python module:

```
>>> bar(foo())         # Okay
>>>
```

Although this is clearly a violation of the C++ type-system, fixing the problem doesn't seem to be worth the added implementation complexity that would be required to support it in the SWIG run-time type system. There are no plans to change this in future releases (although we'll never rule anything out entirely).

The bottom line is that this particular issue does not appear to be a problem for most SWIG projects. Of course, you might want to consider using another tool if maintaining constness is the most important part of your project.

## 6.29 Callbacks to the target language

C/C++ function pointers are often used for callbacks and this is discussed in the Pointers to functions and callbacks section. The callback techniques described therein provide a way to control callbacks to a C/C++ function but not callbacks into the target language. The techniques described below show how the director feature can be used to support callbacks from C/C++ to the target language.

### 6.29.1 Introduction to director classes

The director feature enables the ability for a target language class to derive from a wrapped C++ class. The target language can override virtual methods of a wrapped C++ class, thereby supporting cross-language polymorphism. Code can 'call up' from C++ into the target language by simply calling a virtual method overridden in a derived class in the target language. The wrapped C++ classes that have this ability are termed 'director' classes. The director feature is documented individually in each target language and the reader should locate and read this to obtain a full understanding of directors.

### 6.29.2 Using directors and target language callbacks

SWIG's primary goal is to make it possible to call C/C++ code from a target language, however, the director feature enables the reverse. While there isn't simple direct support for calling target language code from C, the director feature makes this possible. It does require some work and additional wrapper code to be provided by the user. The additional code required must be C++ and not C code and hence may introduce a small dependency on C++ if using a pure C project. In a nutshell, the user must create a C++ base class and turn it into a director class. A virtual method in the director base class is required. SWIG generates the code to call up into the target language when wrapping the director virtual method.

Let's look at some details next. Consider the same function pointer for a callback called `binary_op` from the Pointers to functions and callbacks section. For completeness, the code required for the module and director feature is also shown:

```
%module(directors="1") example

%{
int binary_op(int a, int b, int (*op)(int, int)) {
  return op(a, b);
}
%}
```

The goal is to have a target language function that gets called by `binary_op`. The target language function should have the equivalent signature as the C/C++ function pointer `int (*op)(int, int)`. As we are using directors, we need a C++ virtual method with this signature, so let's define the C++ class and pure virtual method first and make it a director class via the director feature:

```
%feature("director") BinaryOp;

%inline %{
struct BinaryOp {
  virtual int handle(int a, int b) = 0;
  virtual ~BinaryOp() {}
};
%}
```

The following `handler_helper` function and `binary_op_wrapper` function completes the code needed in the C++/SWIG layer. The `binary_op_wrapper` function is wrapped by SWIG and is very similar to the `binary_op` function, however, it takes a pointer to the director base class `BinaryOp` instead of a C/C++ function pointer.

```
%{
static BinaryOp *handler_ptr = NULL;
static int handler_helper(int a, int b) {
  // Make the call up to the target language when handler_ptr
  // is an instance of a target language director class
  return handler_ptr->handle(a, b);
}
// If desired, handler_ptr above could be changed to a thread-local variable in order to make thread-safe
%}

%inline %{
int binary_op_wrapper(int a, int b, BinaryOp *handler) {
  handler_ptr = handler;
  int result = binary_op(a, b, &handler_helper);
  handler = NULL;
  return result;
}
%}
```

On the target language side, we need to derive a class from `BinaryOp` and override the `handle` method. In Python this could be as simple as:

```
import example

# PythonBinaryOp class is defined and derived from C++ class BinaryOp
class PythonBinaryOp(example.BinaryOp):

    # Define Python class 'constructor'
    def __init__(self):
        # Call C++ base class constructor
        example.BinaryOp.__init__(self)

    # Override C++ method: virtual int handle(int a, int b) = 0;
    def handle(self, a, b):
        # Return the product
        return a * b
```

For this to work from Python, an instance of the `PythonBinaryOp` class is created and then passed to `binary_op_wrapper`. The net result is the `binary_op` function will in turn be called which will call `handler_helper` which will call the virtual `handle` method, that is, the Python method `handle` in the PythonBinaryOp class. The result will be the product of 10 and 20 and make its way back to Python and hence 200 will be printed with the following code:

```
handler = PythonBinaryOp()
result = example.binary_op_wrapper(10, 20, handler)
print(result)
```

This has thus demonstrated a C/C++ function pointer calling back into a target language function. The code could be made a little more user friendly by using `%rename` to provide the original `binary_op` name from the target language instead of `binary_op_wrapper`. A C++ functor base class and Python functor class could also be used instead, but these are left as exercises for the reader.

### 6.30 Where to go for more information

If you're wrapping serious C++ code, you might want to pick up a copy of "The Annotated C++ Reference Manual" by Ellis and Stroustrup. This is the reference document we use to guide a lot of SWIG's C++ support.

## 7 SWIG and C++11


- Introduction
- Core language changes
  - Rvalue reference and move semantics
    - Rvalue reference inputs
    - Rvalue reference outputs
    - Movable and move-only types by value
  - Generalized constant expressions
  - Extern template
  - Initializer lists
  - Uniform initialization
  - Type inference
  - Range-based for-loop
  - Lambda functions and expressions
  - Alternate function syntax
  - Object construction improvement
  - Explicit overrides and final
  - Null pointer constant
  - Strongly typed enumerations
  - Double angle brackets
  - Explicit conversion operators
  - Type alias and alias templates
  - Unrestricted unions
  - Variadic templates
  - New character literals
  - New string literals
  - User-defined literals
  - Thread-local storage
  - Explicitly defaulted functions and deleted functions
  - Type long long int
  - Static assertions
  - Allow sizeof to work on members of classes without an explicit object
  - Exception specifications and noexcept
  - Control and query object alignment
  - Attributes
  - Methods with ref-qualifiers
- Standard library changes
  - Threading facilities
  - Tuple types
  - Hash tables
  - Regular expressions
  - General-purpose smart pointers
  - Extensible random number facility
  - Wrapper reference
  - Polymorphic wrappers for function objects
  - Type traits for metaprogramming
  - Uniform method for computing return type of function objects


### 7.1 Introduction

This chapter gives you a brief overview about the SWIG implementation of the C++11 standard.

SWIG supports the new C++ syntax changes with some minor limitations in some areas such as decltype expressions and variadic templates. Wrappers for the new STL types (unordered_ containers, result_of, tuples) are incomplete. The wrappers for the new containers would work much like the C++03 containers and users are welcome to help by adapting the existing container interface files and submitting them as a patch for inclusion in future versions of SWIG.

### 7.2 Core language changes

### 7.2.1 Rvalue reference and move semantics

SWIG correctly parses the rvalue reference syntax '&&', for example the typical usage of it in the move constructor and move assignment operator below:

```
class MyClass {
...
  std::vector<int> numbers;
public:
  MyClass() : numbers() {}
  MyClass(MyClass &&other) : numbers(std::move(other.numbers)) {}
  MyClass & operator=(MyClass &&other) {
    numbers = std::move(other.numbers);
    return *this;
  }
};
```

Rvalue references are designed for C++ temporaries and are not particularly useful when used from non-C++ target languages. One option is to just ignore them via `%ignore`. For example, ignore the move constructor:

```
%ignore MyClass::MyClass(MyClass &&);
```

#### 7.2.1.1 Rvalue reference inputs

Rvalue reference parameters are useful as input parameters in C++ for implementing move semantics, such as, in the move constructor and move assignment operator. This type of usage can be useful from target languages too to avoid copying large objects.

If you do wrap a function/constructor with an rvalue reference parameter and pass a proxy class to it, SWIG will assume that after the call, the rvalue reference parameter object will have been 'moved'. The proxy class passed as the rvalue reference, will own the underlying C++ object up until it is used as an rvalue reference parameter. Afterwards, the proxy class will have the underlying C++ pointer set to the nullptr so that the proxy class instance cannot be used again and the underlying (moved from) C++ object will be deleted after the function/constructor call has returned.

In this way, the SWIG proxy class works much like an exclusively owned smart pointer (think of `std::unique_ptr`), passing ownership to the called C++ function/constructor. Let's consider an example in Java using the wrapped proxy class from above:

```
MyClass mc = new MyClass();
MyClass mc1 = new MyClass(mc); // move constructor
MyClass mc2 = new MyClass(mc); // move constructor fails
```

The second call to the move constructor will fail as the `mc` proxy instance has been moved. Each target language handles the moved proxy class slightly differently when attempting to move it again, but typically you'll get an exception such as in Java:

```
Exception in thread "main" java.lang.RuntimeException: Cannot release ownership as memory is not owned
        at MyClass.swigRelease(MyClass.java:27)
        at MyClass.<init>(MyClass.java:55)
        at runme.main(runme.java:18)
```

Note that both normal copy assignment operators as well as move assignment operators are ignored by default in the target languages with the following warning:

```
example.i:18: Warning 503: Can't wrap 'operator =' unless renamed to a valid identifier.
```

Using a `%rename` will remove the warning and also makes the move assignment operator available from the target language:

```
%rename(MoveAssign) MyClass::operator=(MyClass &&);
```

You can then use it, but like the move constructor example above, you cannot use a proxy class once it has already been moved:

```
MyClass mc = new MyClass();
MyClass mc2 = mc.MoveAssign(mc);
MyClass mc3 = mc.MoveAssign(mc); // Use of mc again will fail
```

It is of course perfectly possible in C++ for a function/constructor to not move an object passed to it in an rvalue reference parameter. The assumption that SWIG makes would then not hold and customisation of the appropriate input typemaps would be required. For scripting languages, this would be for the 'in' typemap and for the non-scripting languages additional typemaps such as the 'javain' typemap, which is used to set the memory ownership of the underlying C++ object for Java, would also need copying and modifying appropriately.

**Compatibility note:** SWIG-4.1.0 changed the way that rvalue reference parameters were handled and implemented typemaps assuming that the proxy class owns the underlying C++ object and transfers ownership of the object when a function/constructor with an rvalue reference parameter is called.

#### 7.2.1.2 Rvalue reference outputs

While rvalue reference parameter inputs are not uncommon in C++ and can be usefully utilised from target languages, this cannot be said for rvalue reference outputs. Firstly, it is quite unusual in C++ to have functions that return an rvalue reference. Secondly, these cases are nigh on impossible to use from a target language. The main problem is these references are for C++ compiler temporaries used on the stack and the target languages use objects on the heap and the concept of compiler temporary objects doesn't make sense from another language.

Using `MyClass` from earlier and this C++ code:

```
void use(MyClass &&mc);
MyClass && get1();
MyClass & get2();
```

SWIG wraps the `get1` and `get2` functions more or less identically. The returned references are converted into pointers that are not owned by the target language. It means that the following perfectly valid C++ has no equivalent in any of the target languages:

```
use(get1());
use(std::move(get2()));
```

An attempt to call the equivalent `use(get1())` from one of the target languages will result in the ownership failure mentioned in the previous section as the object being passed to the `use`

function is not owned by the proxy class. In order to own the object, it would need to be cloned for the object to move from the stack to the heap, for which an appropriate clone function would be required, but may not even be available. Note that a move constructor or copy constructor may slice the object when inheritance is involved. Alternatively, customising the input rvalue reference typemap, as mentioned in the previous section, could remove the ownership requirement. Another alternative would be to modify the output rvalue reference typemap to always clone the rvalue reference object. Fortunately you're highly unlikely to have to solve any of these issues!

#### 7.2.1.3 Movable and move-only types by value

SWIG has traditionally relied on wrapped C++ types to be copy constructible or copy assignable, either via an explicit or implicit copy constructor and copy assignment operator. Prior to C++11, a function could not return nor take a type by value that was not copyable. In C++11 this is no longer the case. A type can also be movable if it has has a move constructor and a move assignment operator. A move-only type is movable but not copyable; it has both the copy constructor and copy assignment operator deleted. Movable types can appear in function signatures for passing 'by value' and in C++11 the object can then be moved rather than copied.

SWIG has support for both copyable and/or movable types. Support for move semantics is quite seamless when returning by value from a function. Support for move semantics is less so and may require some customisation when passing by value to a function. First let's consider returning by value from a function.

The support for function return values is generically implemented in the "out" `SWIGTYPE` typemap which supports any type, including copyable, movable and move-only types. The typemap code is very simple and written so that the compiler will call the move constructor if possible, otherwise the copy constructor:

```
%typemap(out) SWIGTYPE %{
  $result = new $1_ltype($1);
%}
```

The above typemap is for C# and when used to wrap a move-only type such as:

```
struct MoveOnly {
  int  val;
  MoveOnly(): val(0)  {}

  MoveOnly(const MoveOnly &) = delete;
  MoveOnly(MoveOnly &&) = default;

  MoveOnly & operator=(const MoveOnly &) = delete;
  MoveOnly & operator=(MoveOnly &&) = default;

  static MoveOnly create() { return MoveOnly(); }
  static void take(MoveOnly mo);
};
```

will generate wrapper code for the `create` factory method:

```
SWIGEXPORT void * SWIGSTDCALL CSharp_MoveOnly_create() {
  void * jresult ;
  SwigValueWrapper< MoveOnly > result;

  result = MoveOnly::create();
  jresult = new MoveOnly(result);
  return jresult;
}
```

`SwigValueWrapper` is covered in Pass and return by value and is automatically generated in this case as `MoveOnly` is not assignable. Note that the generated code could be optimised further using the "optimal" attribute in the "out" typemap, so if the above typemap is customised as follows (note that this is C# specific):

```
%typemap(out, optimal="1") MoveOnly %{
  $result = new $1_ltype($1);
%}
```

then the generated code will result in the object being optimally moved:

```
SWIGEXPORT void * SWIGSTDCALL CSharp_MoveOnly_create() {
  void * jresult ;
  jresult = new MoveOnly(MoveOnly::create());
  return jresult;
}
```

Now let's consider passing by value. We'll consider three cases; namely types that are:

1. Copyable and not movable - `CopyOnly`.
2. Copyable and movable - `MovableCopyable`.
3. Movable and not copyable - `MoveOnly`.

and for clarification, define these two additional types as follows:

```
struct CopyOnly {
  int  val;
  CopyOnly(): val(0)  {}

  CopyOnly(const CopyOnly &) = default;
  CopyOnly & operator=(const CopyOnly &) = default;

  static CopyOnly create() { return CopyOnly(); }
  static void take(CopyOnly co);
};

struct MovableCopyable {
  int  val;
  MovableCopyable(): val(0)  {}

  MovableCopyable(const MovableCopyable &) = default;
  MovableCopyable(MovableCopyable &&) = default;
  MovableCopyable & operator=(const MovableCopyable &) = default;
  MovableCopyable & operator=(MovableCopyable &&) = default;

  static MovableCopyable create() { return MovableCopyable(); }
```

```
  static void take(MovableCopyable mc);
};
```

The generated code is shown below for `CopyOnly::take` (with additional comments for when constructors and assignment operators are called). While the code shown is C# specific, the generated constructor and/or assignment operator calls are ultimately the same for all target languages.

```
SWIGEXPORT void SWIGSTDCALL CSharp_CopyOnly_take(void * jarg1) {
  CopyOnly arg1 ; // (a) Default constructor
  CopyOnly *argp1 ;

  argp1 = (CopyOnly *)jarg1;
  if (!argp1) {
    SWIG_CSharpSetPendingExceptionArgument(SWIG_CSharpArgumentNullException, "Attempt to dereference null CopyOnly", 0);
    return ;
  }
  arg1 = *argp1; // (b) Copy assignment
  CopyOnly::take(SWIG_STD_MOVE(arg1)); // (c) Copy constructor
}
```

Note that `SWIG_STD_MOVE` is a macro defined as shown below to use `std::move` which is only available from C++11 onwards:

```
#if __cplusplus >=201103L
# define SWIG_STD_MOVE(OBJ) std::move(OBJ)
#else
# define SWIG_STD_MOVE(OBJ) OBJ
#endif
```

Also note: *(c) Copy constructor.* Yes, when passing by value the copy constructor is called for all versions of C++, even C++11 and later even though std::move is specified. It's a C++ language feature for types that don't have move semantics!

The generated code for `MovableCopyable::take` is the same as for `CopyOnly::take`, however, the C++ compiler will choose the move constructor this time where commented *(c) Move constructor*:

```
SWIGEXPORT void SWIGSTDCALL CSharp_MovableCopyable_take(void * jarg1) {
  MovableCopyable arg1 ; // (a) Default constructor
  MovableCopyable *argp1 ;

  argp1 = (MovableCopyable *)jarg1;
  if (!argp1) {
    SWIG_CSharpSetPendingExceptionArgument(SWIG_CSharpArgumentNullException, "Attempt to dereference null MovableCopyable", 0);
    return ;
  }
  arg1 = *argp1; // (b) Copy assignment
  MovableCopyable::take(SWIG_STD_MOVE(arg1)); // (c) Move constructor
}
```

There are two optimisation opportunities available.

1. Remove the default constructor call with the `%feature("valuewrapper")` covered in Pass and return by value and replace it with `SwigValueWrapper`.
2. Apply the SWIGTYPE MOVE typemaps which are designed specifically to implement full move semantics when passing parameters by value. They replace the copy assignment with a call to `SwigValueWrapper::reset` , which works much like `std::unique_ptr::reset`. These typemaps could alternatively have replaced the copy assignment with a move assignment, but this is not maximally optimal.

Simply add the following before the `MovableCopyable::take` method is parsed:

```
%valuewrapper MovableCopyable;
%include <swigmove.i>
%apply SWIGTYPE MOVE { MovableCopyable }
```

will result in this optimal code where just one move constructor is invoked:

```
SWIGEXPORT void SWIGSTDCALL CSharp_MovableCopyable_take(void * jarg1) {
  SwigValueWrapper< MovableCopyable > arg1 ; // (a) No constructors invoked
  MovableCopyable *argp1 ;

  argp1 = (MovableCopyable *)jarg1;
  if (!argp1) {
    SWIG_CSharpSetPendingExceptionArgument(SWIG_CSharpArgumentNullException, "Attempt to dereference null MovableCopyable", 0);
    return ;
  }
  SwigValueWrapper< MovableCopyable >::reset(arg1, argp1);  // (b) No constructor or assignment operator invoked
  MovableCopyable::take(SWIG_STD_MOVE(arg1)); // (c) Move constructor
}
```

Note that `SwigValueWrapper` will call the destructor for the pointer passed to it in the `reset` function. This pointer is the underlying C++ object that the proxy class owns. The details aren't shown, but the 'csin' typemap also generates C# code to ensure that the proxy class releases ownership of the object. Please see the 'SWIGTYPE MOVE' typemaps in the swigmove.i file provided for each target language. Therefore full move semantics are implemented; ownership is moved from the proxy class into the C++ layer and the net effect is the same as using an rvalue reference parameter discussed earlier.

Lastly, let's consider the `MoveOnly::take` function defined earlier. By default the generated code fails to compile as `MoveOnly` does not have a copy assignment operator. SWIG is not designed to select a different typemap automatically for move-only types and the user must apply the SWIGTYPE MOVE typemaps to ensure that only move-only semantics are used. However, SWIG is able to automatically use `%feature("valuewrapper")` for move-only types so it is not necessary to explicitly use this feature. So in this move-only case, simply add the following before `MoveOnly::take` is parsed, which results in the same optimal code shown above for `MovableCopyable`:

```
%include <swigmove.i>
%apply SWIGTYPE MOVE { MoveOnly }
```

**Compatibility note:** SWIG-4.1.0 introduced support for taking advantage of types with move semantics and making it possible to easily use move only types.

### 7.2.2 Generalized constant expressions

SWIG parses and identifies the keyword `constexpr`, but cannot fully utilise it. These C++ compile time constants are usable as runtime constants from the target languages. Below shows example usage for assigning a C++ compile time constant from a compile time constant function:

```
constexpr int XXX() { return 10; }
constexpr int YYY = XXX() + 100;
```

When either of these is used from a target language, a runtime call is made to obtain the underlying constant.

### 7.2.3 Extern template

SWIG correctly parses `extern template` explicit instantiation declarations. However, this template instantiation suppression in a translation unit has no relevance outside of the C++ compiler and so is not used by SWIG. SWIG only uses `%template` for instantiating and wrapping templates. Consider the class template below:

```
// Class template
template class std::vector<int>;        // C++03 template explicit instantiation definition in C++
extern template class std::vector<int>; // C++11 template explicit instantiation declaration (extern template)
%template(VectorInt) std::vector<int>;  // SWIG template instantiation
```

The above result in warnings:

```
example.i:2: Warning 320: Explicit template instantiation ignored.
example.i:3: Warning 327: Extern template ignored.
```

Similarly for the function template below:

```
// Function template
template void Func<int>();              // C++03 template explicit instantiation definition in C++
extern template void Func<int>();       // C++11 template explicit instantiation declaration (extern template)
%template(FuncInt) Func<int>;           // SWIG template instantiation
```

### 7.2.4 Initializer lists

Initializer lists are very much a C++ compiler construct and are not very accessible from wrappers as they are intended for compile time initialization of classes using the special `std::initializer_list` type. SWIG detects usage of initializer lists and will emit a special informative warning each time one is used:

```
example.i:33: Warning 476: Initialization using std::initializer_list.
```

Initializer lists usually appear in constructors but can appear in any function or method. They often appear in constructors which are overloaded with alternative approaches to initializing a class, such as the std container's push_back method for adding elements to a container. The recommended approach then is to simply ignore the initializer-list constructor, for example:

```
%ignore Container::Container(std::initializer_list<int>);
class Container {
public:
  Container(std::initializer_list<int>); // initializer-list constructor
  Container();
  void push_back(const int &);
  ...
};
```

Alternatively you could modify the class and add another constructor for initialization by some other means, for example by a `std::vector`:

```
%include <std_vector.i>
class Container {
public:
  Container(const std::vector<int> &);
  Container(std::initializer_list<int>); // initializer-list constructor
  Container();
  void push_back(const int &);
  ...
};
```

And then call this constructor from your target language, for example, in Python, the following will call the constructor taking the `std::vector`:

```
>>> c = Container( [1, 2, 3, 4] )
```

If you are unable to modify the class being wrapped, consider ignoring the initializer-list constructor and using %extend to add in an alternative constructor:

```
%include <std_vector.i>
%extend Container {
  Container(const std::vector<int> &elements) {
    Container *c = new Container();
    for (int element : elements)
      c->push_back(element);
    return c;
  }
}

%ignore Container::Container(std::initializer_list<int>);

class Container {
public:
  Container(std::initializer_list<int>); // initializer-list constructor
  Container();
  void push_back(const int &);
  ...
};
```

The above makes the wrappers look is as if the class had been declared as follows:

```
%include <std_vector.i>
class Container {
public:
  Container(const std::vector<int> &);
//  Container(std::initializer_list<int>); // initializer-list constructor (ignored)
  Container();
  void push_back(const int &);
  ...
};
```

`std::initializer_list` is simply a container that can only be initialized at compile time. As it is just a C++ type, it is possible to write typemaps for a target language container to map onto `std::initializer_list`. However, this can only be done for a fixed number of elements as initializer lists are not designed to be constructed with a variable number of arguments at runtime. The example below is a very simple approach which ignores any parameters passed in and merely initializes with a fixed list of fixed integer values chosen at compile time:

```
%typemap(in) std::initializer_list<int> {
  $1 = {10, 20, 30, 40, 50};
}
class Container {
public:
  Container(std::initializer_list<int>); // initializer-list constructor
  Container();
  void push_back(const int &);
  ...
};
```

Any attempt at passing in values from the target language will be ignored and be replaced by `{10, 20, 30, 40, 50}`. Needless to say, this approach is very limited, but could be improved upon, but only slightly. A typemap could be written to map a fixed number of elements on to the `std::initializer_list`, but with values decided at runtime. The typemaps would be target language specific.

Note that the default typemap for `std::initializer_list` does nothing but issue the warning and hence any user supplied typemaps will override it and suppress the warning.

### 7.2.5 Uniform initialization

The curly brackets {} for member initialization are fully supported by SWIG:

```
struct BasicStruct {
 int x;
 double y;
};

struct AltStruct {
  AltStruct(int x, double y) : x_{x}, y_{y} {}

  int x_;
  double y_;
};

BasicStruct var1{5, 3.2}; // only fills the struct components
AltStruct var2{2, 4.3};   // calls the constructor
```

Uniform initialization does not affect usage from the target language, for example in Python:

```
>>> a = AltStruct(10, 142.15)
>>> a.x_
10
>>> a.y_
142.15
```

### 7.2.6 Type inference

`decltype()` is supported with a few limitations. SWIG can parse all uses, but can't deduce the type in every situation where a C++ compiler can. The cases SWIG can deduce have expanded with time and hopefully will continue to. For example, for the code

```
int i;
decltype(i) j;
decltype(i+j) k;
```

SWIG is able to deduce that the variable `i` and the expression `i+j` both have type `int`.

Using an expression for the decltype which SWIG can't handle results in a warning:

```
int foo(int);
decltype(foo(0)) k;  // Warning 344: Unable to deduce decltype for 'foo(0)'.
```

This warning should be viewed as a prompt to add in a manual ignore of the variable/function as in most cases the generated code will not compile. For the example above, ignore the symbol that is declared using `decltype` and perhaps additionally suppress the warning as follows:

```
#pragma SWIG nowarn=SWIGWARN_CPP11_DECLTYPE
%ignore k;
```

If an ignore is not acceptable, a workaround is to redefine the symbol with the actual type, for example:

```
int k; // define k with the actual type
%ignore k; // ignore the real definition of k
```

You would typically put one of these workarounds in your interface file before using `%include` to get SWIG to parse the header which defines `k`.

SWIG supports `auto` as a type specifier for variables (with the same limitations for actually deducing the type as for `decltype()`), and for specifying the return type of lambdas and functions.

### 7.2.7 Range-based for-loop

This feature is part of the implementation block only. SWIG ignores it.

### 7.2.8 Lambda functions and expressions

SWIG correctly parses most of the Lambda functions syntax. For example:

```
auto val = [] { return something; };
auto sum = [](int x, int y) { return x+y; };
auto sum = [](int x, int y) -> int { return x+y; };
```

The lambda functions are removed from the wrappers for now, because of the lack of support for closures (scope of the lambda functions) in the target languages.

Lambda functions used to create variables can also be parsed, but due to limited support of `auto` when the type is deduced from the expression, the variables are simply ignored.

```
auto six = [](int x, int y) { return x+y; }(4, 2);
```

Better support should be available in a later release.

### 7.2.9 Alternate function syntax

SWIG fully supports the new definition of functions. For example:

```
struct SomeStruct {
  int FuncName(int x, int y);
};
```

can now be written as in C++11:

```
struct SomeStruct {
  auto FuncName(int x, int y) -> int;
};

auto SomeStruct::FuncName(int x, int y) -> int {
  return x + y;
}
```

The usage in the target languages remains the same, for example in Python:

```
>>> a = SomeStruct()
>>> a.FuncName(10, 5)
15
```

SWIG will also deal with type inference for the return type, as per the limitations described earlier. For example:

```
auto square(float a, float b) -> decltype(a);
```

### 7.2.10 Object construction improvement

There are three parts to object construction improvement. The first improvement is constructor delegation such as the following:

```
class A {
public:
  int a;
  int b;
  int c;

  A() : A(10) {}
  A(int aa) : A(aa, 20) {}
  A(int aa, int bb) : A(aa, bb, 30) {}
  A(int aa, int bb, int cc) { a=aa; b=bb; c=cc; }
};
```

where peer constructors can be called. SWIG handles this without any issue.

The second improvement is constructor inheritance via a `using` declaration. The extra constructors provided by the `using` declaration will add the appropriate constructors into the target language proxy derived classes. In the example below a wrapper for the `DerivedClass(int)` constructor is added to `DerivedClass`:

```
class BaseClass {
public:
  BaseClass(int iValue);
};

class DerivedClass: public BaseClass {
  public:
  using BaseClass::BaseClass; // Adds DerivedClass(int) constructor
};
```

**Compatibility note:** SWIG-4.2.0 was the first version to generate wrappers for constructors inherited via `using` declarations.

The final part is member initialization at the site of the declaration. This kind of initialization is handled by SWIG.

```
class SomeClass {
public:
```

```
  SomeClass() {}
  explicit SomeClass(int new_value) : value(new_value) {}

  int value = 5;
};
```

### 7.2.11 Explicit overrides and final

The special identifiers `final` and `override` can be used on methods and destructors, such as in the following example:

```
struct BaseStruct {
  virtual void ab() const = 0;
  virtual void cd();
  virtual void ef();
  virtual ~BaseStruct();
};
struct DerivedStruct : BaseStruct {
  virtual void ab() const override;
  virtual void cd() final;
  virtual void ef() final override;
  virtual ~DerivedStruct() override;
};
```

Classes can also be marked as final, such as

```
struct FinalDerivedStruct final : BaseStruct {
  virtual void ab() const override;
};
```

**Compatibility note:** Final methods were supported much earlier than final classes. SWIG-4.1.0 was the first version to support classes marked as final.

### 7.2.12 Null pointer constant

The `nullptr` constant is mostly unimportant in wrappers. In the few places it has an effect, it is treated like `NULL`.

### 7.2.13 Strongly typed enumerations

SWIG supports strongly typed enumerations and parses the new`enum class` syntax and forward declarator for the enums, such as:

```
enum class MyEnum : unsigned int;
```

Strongly typed enums are often used to avoid name clashes such as the following:

```
struct Color {
  enum class RainbowColors : unsigned int {
    Red, Orange, Yellow, Green, Blue, Indigo, Violet
  };

  enum class WarmColors {
    Yellow, Orange, Red
  };

  // Note normal enum
  enum PrimeColors {
    Red=100, Green, Blue
  };
};
```

There are various ways that the target languages handle enums, so it is not possible to precisely state how they are handled in this section. However, generally, most scripting languages mangle in the strongly typed enumeration's class name, but do not use any additional mangling for normal enumerations. For example, in Python, the following code

```
print(Color.RainbowColors_Red, Color.WarmColors_Red, Color.Red)
```

results in

```
0 2 100
```

The strongly typed languages often wrap normal enums into an enum class and so treat normal enums and strongly typed enums the same. The equivalent in Java is:

```
System.out.println(Color.RainbowColors.Red.swigValue() + " " + Color.WarmColors.Red.swigValue() + " " + Color.PrimeColors.Red.swigValue());
```

The C++11 enum base type, such as `unsigned int`, in the example above, is used by some language modules and is missing support in others. For example, in C#, the enum base type in the example above is used and converted into a C# `uint` to specify the underlying C# enumeration type as follows:

```
  public enum RainbowColors : uint {
    Red,
    Orange,
    Yellow,
    Green,
    Blue,
    Indigo,
    Violet
  }
```

### 7.2.14 Double angle brackets

SWIG correctly parses the symbols >> as closing the template block, if found inside it at the top level, or as the right shift operator >> otherwise.

```
std::vector<std::vector<int>> myIntTable;
```

### 7.2.15 Explicit conversion operators

SWIG correctly parses the keyword `explicit` for operators in addition to constructors now. For example:

```
class U {
public:
  int u;
};

class V {
public:
  int v;
};

class TestClass {
public:
  //implicit converting constructor
  TestClass(U const &val) { t=val.u; }

  // explicit constructor
  explicit TestClass(V const &val) { t=val.v; }

  int t;
};

struct Testable {
  // explicit conversion operator
  explicit operator bool() const {
    return false;
  }
};
```

The effect of explicit constructors and operators has little relevance for the proxy classes as target languages don't have the same concepts of implicit conversions as C++. Conversion operators either with or without `explicit` need renaming to a valid identifier name in order to make them available as a normal proxy method.

### 7.2.16 Type alias and alias templates

A type alias is a statement of the form:

```
using PFD = void (*)(double); // New introduced syntax
```

which is equivalent to the old style typedef:

```
typedef void (*PFD)(double);  // The old style
```

The following is an example of an alias template:

```
template< typename T1, typename T2, int N >
class SomeType {
public:
  T1 a;
  T2 b;
};

template< typename T2 >
using TypedefName = SomeType<char*, T2, 5>;
```

SWIG supports both type aliasing and alias templates. However, in order to use an alias template, two `%template` directives must be used:

```
%template(SomeTypeBool) SomeType<char*, bool, 5>;
%template() TypedefName<bool>;
```

Firstly, the actual template is instantiated with a name to be used by the target language, as per any template being wrapped. Secondly, the empty template instantiation, `%template()`, is required for the alias template. This second requirement is necessary to add the appropriate instantiated template type into the type system as SWIG does not automatically instantiate templates. See the Templates section for more general information on wrapping templates.

### 7.2.17 Unrestricted unions

SWIG fully supports any type inside a union even if it does not define a trivial constructor. For example, the wrapper for the following code correctly provides access to all members in the union:

```
struct point {
  point() {}
  point(int x, int y) : x_(x), y_(y) {}
  int x_, y_;
};

#include <new> // For placement 'new' in the constructor below
union P {
  int z;
  double w;
  point p; // Illegal in C++03; legal in C++11.
  // Due to the point member, a constructor definition is required.
  P() {
    new(&p) point();
  }
```

```
} p1;
```

### 7.2.18 Variadic templates

SWIG supports the variadic templates including the <> variadic class inheritance, variadic methods, variadic constructors and initializers. Example:

```
template <typename... BaseClasses> class ClassName : public BaseClasses... {
public:
  ClassName(BaseClasses &&... baseClasses) : BaseClasses(baseClasses)... {}
  void InstanceMethod(const BaseClasses&... baseClasses) {}
};
```

The `%template` directive works as expected for variable template parameters.

```
struct A {
  virtual void amethod();
  virtual ~A();
};
struct B {
  virtual void bmethod();
  virtual ~B();
};
%template(ClassName0) ClassName<>
%template(ClassName1) ClassName<A>
%template(ClassName2) ClassName<A, B>
```

Example usage from say Python:

```
cn0 = ClassName0()
cn0.InstanceMethod()

a = A()
cn1 = ClassName1(a)
cn1.amethod()
cn1.InstanceMethod(a)

b = B()
cn2 = ClassName2(a, b)
cn2.InstanceMethod(a, b)
cn2.amethod()
cn2.bmethod()
```

Support for the variadic `sizeof()` function also works:

```
const int SIZE = sizeof...(ClassName<A, B>);
```

In the above example `SIZE` is of course wrapped as a constant.

**Compatibility note:** SWIG-4.2.0 was the first version to fully support variadic templates. SWIG-3.0.0 provided initial support and was limited to only one variadic parameter.

### 7.2.19 New character literals

C++11 adds support for UCS-2 and UCS-4 character literals. These character literals are preceded by either 'u' or 'U'.

```
char16_t a = u'a';
char32_t b = U'b';
```

**Compatibility note:** SWIG-4.0.0 was the first version to support these Universal Coded Character Set (UCS) character literals.

### 7.2.20 New string literals

SWIG supports wide string and Unicode string constants and raw string literals.

```
// New string literals
wstring         aa =  L"Wide string";
const char     *bb = u8"UTF-8 string";
const char16_t *cc =  u"UTF-16 string";
const char32_t *dd =  U"UTF-32 string";

// Raw string literals
const char      *xx =        ")I'm an \"ascii\" \\ string.";
const char      *ee =   R"XXX()I'm an "ascii" \ string.)XXX"; // same as xx
wstring          ff =  LR"XXX(I'm a "raw wide" \ string.)XXX";
const char      *gg = u8R"XXX(I'm a "raw UTF-8" \ string.)XXX";
const char16_t  *hh =  uR"XXX(I'm a "raw UTF-16" \ string.)XXX";
const char32_t  *ii =  UR"XXX(I'm a "raw UTF-32" \ string.)XXX";
```

Non-ASCII string support varies quite a bit among the various target languages though.

Note: There is a bug currently where SWIG's preprocessor incorrectly parses an odd number of double quotes inside raw string literals.

### 7.2.21 User-defined literals

SWIG parses the declaration of user-defined literals, that is, the `operator "" _mysuffix()` function syntax.

Some examples are the raw literal:

```
OutputType operator "" _myRawLiteral(const char * value);
```

numeric cooked literals:

```
OutputType operator "" _mySuffixIntegral(unsigned long long);
OutputType operator "" _mySuffixFloat(long double);
```

and cooked string literals:

```
OutputType operator "" _mySuffix(const char * string_values, size_t num_chars);
OutputType operator "" _mySuffix(const wchar_t * string_values, size_t num_chars);
OutputType operator "" _mySuffix(const char16_t * string_values, size_t num_chars);
OutputType operator "" _mySuffix(const char32_t * string_values, size_t num_chars);
```

Like other operators that SWIG parses, a warning is given about renaming the operator in order for it to be wrapped:

```
example.i:27: Warning 503: Can't wrap 'operator "" _myRawLiteral' unless renamed to a valid identifier.
```

If %rename is used, then it can be called like any other wrapped method. Currently you need to specify the full declaration including parameters for %rename:

```
%rename(MyRawLiteral)  operator"" _myRawLiteral(const char * value);
```

Or if you just wish to ignore it altogether:

```
%ignore operator "" _myRawLiteral(const char * value);
```

Note that use of user-defined literals such as the following still give a syntax error:

```
OutputType var1 = "1234"_suffix;
OutputType var2 = 1234_suffix;
OutputType var3 = 3.1416_suffix;
```

### 7.2.22 Thread-local storage

SWIG correctly parses the `thread_local` keyword. For example, variables reachable by the current thread can be defined as:

```
struct A {
  static thread_local int val;
};
thread_local int global_val;
```

The use of the `thread_local` storage specifier does not affect the wrapping process; it does not modify the wrapper code compared to when it is not specified. A variable will be thread local if accessed from different threads from the target language in the same way that it will be thread local if accessed from C++ code.

### 7.2.23 Explicitly defaulted functions and deleted functions

SWIG handles explicitly defaulted functions, that is, `= default` added to a function declaration. Deleted definitions, which are also called deleted functions, have `= delete` added to the function declaration. For example:

```
struct NonCopyable {
  NonCopyable & operator=(const NonCopyable &) = delete; /* Removes operator= */
  NonCopyable(const NonCopyable &) = delete;             /* Removes copy constructor */
  NonCopyable() = default;                               /* Explicitly allows the empty constructor */
};
```

Wrappers for deleted functions will not be available in the target language. Wrappers for defaulted functions will of course be available in the target language. Explicitly defaulted functions have no direct effect for SWIG wrapping as the declaration is handled much like any other method declaration parsed by SWIG.

Deleted functions are also designed to prevent implicit conversions when calling the function. For example, the C++ compiler will not compile any code which attempts to use an int as the type of the parameter passed to `f` below:

```
struct NoInt {
  void f(double i);
  void f(int) = delete;
};
```

This is a C++ compile time check and SWIG does not make any attempt to detect if the target language is using an int instead of a double though, so in this case it is entirely possible to pass an int instead of a double to `f` from Java, Python etc.

### 7.2.24 Type long long int

SWIG correctly parses and uses the new `long long` type already introduced in C99 some time ago.

### 7.2.25 Static assertions

SWIG correctly parses the new `static_assert` declarations (though 3.0.12 and earlier had a bug which meant this wasn't accepted at file scope). This is a C++ compile time directive so there isn't anything useful that SWIG can do with it.

```
template <typename T>
struct Check {
  static_assert(sizeof(int) <= sizeof(T), "not big enough");
};
```

### 7.2.26 Allow sizeof to work on members of classes without an explicit object

SWIG can parse the new sizeof() on types as well as on objects. For example:

```
struct A {
  int member;
};

const int SIZE = sizeof(A::member); // does not work with C++03. Okay with C++11
```

In Python:

```
>>> SIZE
8
```

### 7.2.27 Exception specifications and noexcept

C++11 added in the noexcept specification to exception specifications to indicate that a function simply may or may not throw an exception, without actually naming any exception. SWIG understands these, although there isn't any useful way that this information can be taken advantage of by target languages, so it is as good as ignored during the wrapping process. Below are some examples of noexcept in function declarations:

```
static void noex1() noexcept;
int noex2(int) noexcept(true);
int noex3(int, bool) noexcept(false);
```

### 7.2.28 Control and query object alignment

An `alignof` operator is used mostly within C++ to return alignment in number of bytes, but could be used to initialize a variable as shown below. The variable's value will be available for access by the target language as any other variable's compile time initialised value.

```
const int align1 = alignof(A::member);
```

The `alignas` specifier for variable alignment is not yet supported. Example usage:

```
struct alignas(16) S {
  int num;
};
alignas(double) unsigned char c[sizeof(double)];
```

Use the preprocessor to work around this for now:

```
#define alignas(T)
```

### 7.2.29 Attributes

Attributes such as those shown below, are supported since SWIG 4.1.0 but are currently crudely ignored by the parser's tokeniser so they have no effect on SWIG's code generation.

```
int [[attr1]] i [[attr2, attr3]];

[[noreturn, nothrow]] void f [[noreturn]] ();
```

### 7.2.30 Methods with ref-qualifiers

C++11 non-static member functions can be declared with ref-qualifiers. Member functions declared with a `&` lvalue ref-qualifiers are wrapped like any other function without ref-qualifiers. Member functions declared with a `&&` rvalue ref-qualifiers are ignored by default as they are unlikely to be required from non-C++ languages where the concept of *rvalue-ness* for the implied *this pointer does not apply. The warning is hidden by default, but can be displayed as described in the section on Enabling extra warnings.

Consider:

```
struct RQ {
  void m1(int x) &;
  void m2(int x) &&;
};
```

The only wrapped method will be the lvalue ref-qualified method `m1` and if SWIG is run with the `-Wextra` command-line option, the following warning will be issued indicating `m2` is not wrapped:

```
example.i:7: Warning 405: Method with rvalue ref-qualifier m2(int) && ignored.
```

If you unignore the method as follows, wrappers for `m2` will be generated:

```
%feature("ignore", "0") RQ::m2(int x) &&;
struct RQ {
  void m1(int x) &;
  void m2(int x) &&;
};
```

Inspection of the generated C++ code, will show that `std::move` is used on the instance of the `RQ *` class:

```
  RQ *arg1 = (RQ *) 0 ;
  int arg2 ;

  arg1 = ...marshalled from target language...
  arg2 = ...marshalled from target language...

  std::move(*arg1).m2(arg2);
```

This will compile but when run, the move effects may not be what you want. As stated earlier, rvalue ref-qualifiers aren't really applicable outside the world of C++. However, if you really know what you are doing, full control over the call to the method is possible via the low-level "action" feature. This feature completely replaces the call to the underlying function, that is, the last line in the snippet of code above.

```
%feature("ignore", "0") RQ::m2(int x) &&;
%feature("action") RQ::m2(int x) && %{
  RQ().m2(arg2);
%}
struct RQ {
  void m1(int x) &;
  void m2(int x) &&;
};
```

resulting in:

```
  RQ *arg1 = (RQ *) 0 ;
  int arg2 ;

  arg1 = ...marshalled from target language...
  arg2 = ...marshalled from target language...

  RQ().m2(arg2);
```

**Compatibility note:** SWIG-4.0.0 was the first version to support ref-qualifiers.

## 7.3 Standard library changes

### 7.3.1 Threading facilities

SWIG does not currently wrap or use any of the new threading classes introduced (thread, mutex, locks, condition variables, task). The main reason is that SWIG target languages offer their own threading facilities so there is limited use for them.

### 7.3.2 Tuple types

SWIG does not provide library files for the new tuple types yet. Variadic template support requires further work to provide substantial tuple wrappers.

### 7.3.3 Hash tables

The new hash tables in the STL are `unordered_set`, `unordered_multiset`, `unordered_map`, `unordered_multimap` . These are not available in all target languages. Any missing support can in principle be easily implemented by adapting the current STL containers.

### 7.3.4 Regular expressions

While SWIG could provide wrappers for the new C++11 regular expressions classes, there is little need as the target languages have their own regular expression facilities.

### 7.3.5 General-purpose smart pointers

SWIG provides special smart pointer handling for `std::shared_ptr` in the same way it has support for `boost::shared_ptr`. Please see the shared_ptr smart pointer and unique_ptr smart pointer library sections. There is no special smart pointer handling available for `std::weak_ptr`.

### 7.3.6 Extensible random number facility

This feature extends and standardizes the standard library only and does not affect the C++ language nor SWIG.

### 7.3.7 Wrapper reference

Wrapper references are similar to normal C++ references but are copy-constructible and copy-assignable. They could conceivably be used in public APIs. There is no special support for `std::reference_wrapper` in SWIG though. Users would need to write their own typemaps if wrapper references are being used and these would be similar to the plain C++ reference typemaps.

### 7.3.8 Polymorphic wrappers for function objects

SWIG supports functor classes in a few languages in a very natural way. However nothing is provided yet for the new `std::function` template. SWIG will parse usage of the template like any other template.

```
%rename(__call__) Test::operator(); // Default renaming used for Python

struct Test {
  bool operator()(int x, int y); // function object
};

#include <functional>
std::function<void (int, int)> pF = Test;   // function template wrapper
```

Example of supported usage of the plain functor from Python is shown below. It does not involve `std::function`.

```
t = Test()
b = t(1, 2) # invoke C++ function object
```

### 7.3.9 Type traits for metaprogramming

The type_traits functions to support C++ metaprogramming is useful at compile time and is aimed specifically at C++ development:

```
#include <type_traits>

// First way of operating.
template< bool B > struct algorithm {
  template< class T1, class T2 > static int do_it(T1 &, T2 &)  { /*...*/ return 1; }
};

// Second way of operating.
```

```
template<> struct algorithm<true> {
  template< class T1, class T2 > static int do_it(T1, T2)  { /*...*/ return 2; }
};

// Instantiating 'elaborate' will automatically instantiate the correct way to operate, depending on the types used.
template< class T1, class T2 > int elaborate(T1 A, T2 B) {
  // Use the second way only if 'T1' is an integer and if 'T2' is a floating point,
  // otherwise use the first way.
  return algorithm< std::is_integral<T1>::value && std::is_floating_point<T2>::value >::do_it(A, B);
}
```

SWIG correctly parses the template specialization, template types etc. However, metaprogramming and the additional support in the type_traits header is really for compile time and is not much use at runtime for the target languages. For example, as SWIG requires explicit instantiation of templates via `%template`, there isn't much that `std::is_integral<int>` is going to provide by itself. However, template functions using such metaprogramming techniques might be useful to wrap. For example, the following instantiations could be made:

```
%template(Elaborate) elaborate<int, int>;
%template(Elaborate) elaborate<int, double>;
```

Then the appropriate algorithm can be called for the subset of types given by the above `%template` instantiations from a target language, such as Python:

```
>>> Elaborate(0, 0)
1
>>> Elaborate(0, 0.0)
2
```

### 7.3.10 Uniform method for computing return type of function objects

The new `std::result_of` class introduced in the <functional> header provides a generic way to obtain the return type of a function type via `std::result_of::type`. There isn't any library interface file to support this type. With a bit of work, SWIG will deduce the return type of functions when used in `std::result_of` using the approach shown below. The technique basically forward declares the `std::result_of` template class, then partially specializes it for the function types of interest. SWIG will use the partial specialization and hence correctly use the `std::result_of::type` provided in the partial specialization.

```
%inline %{
#include <functional>
typedef double(*fn_ptr)(double);
%}

namespace std {
  // Forward declaration of result_of
  template<typename Func> struct result_of;
  // Add in a partial specialization of result_of
  template<> struct result_of< fn_ptr(double) > {
    typedef double type;
  };
}

%template() std::result_of< fn_ptr(double) >;

%inline %{

double square(double x) {
  return (x * x);
}

template<class Fun, class Arg>
typename std::result_of<Fun(Arg)>::type test_result_impl(Fun fun, Arg arg) {
  return fun(arg);
}
%}

%template(test_result) test_result_impl< fn_ptr, double >;
%constant double (*SQUARE)(double) = square;
```

Note the first use of `%template` which SWIG requires to instantiate the template. The empty template instantiation suffices as no proxy class is required for `std::result_of<Fun(Arg)>::type` as this type is really just a `double`. The second `%template` instantiates the template function which is being wrapped for use as a callback. The `%constant` can then be used for any callback function as described in Pointers to functions and callbacks.

Example usage from Python should give the not too surprising result:

```
>>> test_result(SQUARE, 5.0)
25.0
```

Phew, that is a lot of hard work to get a callback working. You could just go with the more attractive option of just using `double` as the return type in the function declaration instead of `result_of` !

# 8 SWIG and C++14


- Introduction
- Core language changes
  - Binary integer literals
  - Return type deduction
- Standard library changes


## 8.1 Introduction

This chapter gives you a brief overview about the SWIG implementation of the C++14 standard. There isn't much in C++14 that affects SWIG, however, work has only just begun on adding C++14 support.

**Compatibility note:** SWIG-4.0.0 is the first version to support any C++14 features.

## 8.2 Core language changes

### 8.2.1 Binary integer literals

C++14 added binary integer literals and SWIG supports these. Example:

```
int b = 0b101011;
```

### 8.2.2 Return type deduction

C++14 added the ability to specify `auto` for the return type of a function and have the compiler deduce it from the body of the function (in C++11 you had to explicitly specify a trailing return type if you used `auto` for the return type).

SWIG parses these types of functions, but with one significant limitation: SWIG can't actually deduce the return type! If you want to wrap such a function you will need to tell SWIG the return type explicitly.

The trick for specifying the return type is to use `%ignore` to tell SWIG to ignore the function with the deduced return type, but first provide SWIG with an alternative declaration of the function with an explicit return type. The generated wrapper will wrap this alternative declaration, and the call in the wrapper to the function will call the actual declaration. Here is an actual example:

```
std::tuple<int, int> va_static_cast();
%ignore va_static_cast();
#pragma SWIG nowarn=SWIGWARN_CPP14_AUTO

%inline %{
#include <tuple>

auto va_static_cast() {
    return std::make_tuple(0, 0);
}
%}
```

For member methods the trick is to use `%extend` to redeclare the method and call it as follows:

```
%extend X {
  const char * a() const { return $self->a(); }
}
%inline %{
struct X {
  auto a() const {
    return "a string";
  }
};
%}
```

**Compatibility note:** SWIG-4.2.0 first introduced support for functions declared with an auto return without a trailing return type.

## 8.3 Standard library changes

# 9 SWIG and C++17


- Introduction
- Core language changes
  - Nested namespace definitions
  - UTF-8 character literals
  - Hexadecimal floating literals
  - Fold expressions
- Standard library changes


## 9.1 Introduction

This chapter gives you a brief overview about the SWIG implementation of the C++17 standard. There isn't much in C++17 that affects SWIG, however, work has only just begun on adding C++17 support.

**Compatibility note:** SWIG-4.0.0 is the first version to support any C++17 features.

## 9.2 Core language changes

### 9.2.1 Nested namespace definitions

C++17 offers a more concise syntax for defining namespaces. SWIG has support for nested namespace definitions such as:

```
namespace A::B::C {
  ...
}
```

This is the equivalent to the C++98 namespace definitions:

```
namespace A {
  namespace B {
    namespace C {
      ...
    }
  }
}
```

### 9.2.2 UTF-8 character literals

C++17 added UTF-8 (u8) character literals. These are of type char. Example:

```
char a = u8'a';
```

### 9.2.3 Hexadecimal floating literals

C++17 added hexadecimal floating literals. For example:

```
double f = 0xF.68p2;
```

### 9.2.4 Fold expressions

C++17 added template fold expressions. SWIG 4.3.0 and later support parsing these with a few restrictions. Unary left fold expressions are not supported currently. Also the same restrictions that apply to other expressions apply here too.

## 9.3 Standard library changes

# 10 SWIG and C++20


- Introduction
- Core language changes
  - Spaceship operator
  - Lambda templates
  - Constexpr destructors
- Preprocessor changes
  - __VA_OPT__()
- Standard library changes


## 10.1 Introduction

This chapter gives you a brief overview about the SWIG implementation of the C++20 standard. Work has only just begun on adding C++20 support.

**Compatibility note:** SWIG-4.1.0 is the first version to support any C++20 features.

## 10.2 Core language changes

### 10.2.1 Spaceship operator

SWIG supports the spaceship operator <=> in constant expressions. To simplify handling of the return value type, it is currently treated as an integer rather than `std::strong_ordering` , etc. In practice we think that should do the right thing in most cases.

SWIG also recognises `operator<=>` which can be wrapped if renamed. There is not currently any default renaming for the operator or any attempt to automatically map it to a three-way comparison operator in any of the target languages.

### 10.2.2 Lambda templates

SWIG should parse lambda templates, but like non-templated lambdas they aren't currently wrapped.

### 10.2.3 Constexpr destructors

Destructors that are declared `constexpr` are parsed and handled like any other constructor. For example:

```
class DtorA {
public:
  constexpr ~DtorA() {}
};
```

## 10.3 Preprocessor changes

### 10.3.1 __VA_OPT__()

Support for `__VA_OPT__()` was added in SWIG 4.3.0.

## 10.4 Standard library changes

# 11 Preprocessing


- File inclusion
- File imports
- Conditional Compilation
- Macro Expansion
- SWIG Macros
- Variadic Macros
- Preprocessing and delimiters
  - Preprocessing and %{ ... %} & " ... " delimiters
  - Preprocessing and { ... } delimiters
- Preprocessor and Typemaps
- Viewing preprocessor output
- The #error and #warning directives

- Trigraphs
- Digraphs


SWIG includes its own enhanced version of the C preprocessor. The preprocessor supports the standard preprocessor directives and macro expansion rules. However, a number of modifications and enhancements have been made. This chapter describes some of these modifications.

## 11.1 File inclusion

To include another file into a SWIG interface, use the `%include` directive like this:

```
%include "cpointer.i"
```

Unlike, `#include`, `%include` includes each file once (and will not reload the file on subsequent `%include` declarations). Therefore, it is not necessary to use include-guards in SWIG interfaces.

By default, the `#include` is ignored unless you run SWIG with the `-includeall` option. The reason for ignoring traditional includes is that you often don't want SWIG to try and wrap everything included in standard header system headers and auxiliary files.

## 11.2 File imports

SWIG provides another file inclusion directive with the `%import` directive. For example:

```
%import "foo.i"
```

The purpose of `%import` is to collect certain information from another SWIG interface file or a header file without actually generating any wrapper code. Such information generally includes type declarations (e.g., `typedef`) as well as C++ classes that might be used as base-classes for class declarations in the interface. The use of `%import` is also important when SWIG is used to generate extensions as a collection of related modules. This is an advanced topic and is described in later in the Working with Modules chapter.

The `-importall` directive tells SWIG to follow all `#include` statements as imports. This might be useful if you want to extract type definitions from system header files without generating any wrappers.

## 11.3 Conditional Compilation

SWIG fully supports the use of `#if`, `#ifdef`, `#ifndef`, `#else`, `#endif` to conditionally include parts of an interface.

SWIG's preprocessor conditionals support the standard C/C++ preprocessor integer expressions. As a SWIG-specific extension, string equality and inequality tests are also supported, for example:

```
#if defined __cplusplus && (#__VA_ARGS__ != "" || #TYPE == "void")
```

The following symbols are predefined by SWIG when it is parsing the interface:

```
SWIG                            Always defined when SWIG is processing a file
SWIGIMPORTED                    Defined when SWIG is importing a file with %import
SWIG_VERSION                    Hexadecimal (binary-coded decimal) number containing SWIG version,
                                such as 0x010311 (corresponding to SWIG-1.3.11).

SWIGCSHARP                      Defined when using C#
SWIGD                           Defined when using D
SWIGGO                          Defined when using Go
SWIGGUILE                       Defined when using Guile
SWIGJAVA                        Defined when using Java
SWIGJAVASCRIPT                  Defined when using Javascript
SWIG_JAVASCRIPT_JSC             Defined when using Javascript with -jsc
SWIG_JAVASCRIPT_V8              Defined when using Javascript with -v8 or -node
SWIG_JAVASCRIPT_NAPI            Defined when using Javascript with -napi
SWIGLUA                         Defined when using Lua
SWIGMZSCHEME                    Defined when using Mzscheme
SWIGOCAML                       Defined when using OCaml
SWIGOCTAVE                      Defined when using Octave
SWIGPERL                        Defined when using Perl
SWIGPHP                         Defined when using PHP (any version)
SWIGPHP7                        Defined when using PHP 7 or later (with a compatible C API)
SWIGPYTHON                      Defined when using Python
SWIGR                           Defined when using R
SWIGRUBY                        Defined when using Ruby
SWIGSCILAB                      Defined when using Scilab
SWIGTCL                         Defined when using Tcl
SWIGXML                         Defined when using XML
```

SWIG also defines `SWIG_VERSION` and a target language macro in the generated wrapper file (since SWIG 4.1.0 - in older versions these were defined for some target languages but this wasn't consistent). Best practice is to use SWIG-time conditional checks because that results in smaller generated wrapper sources.

In addition, SWIG defines the following set of standard C/C++ macros:

```
__LINE__                        Current line number
__FILE__                        Current file name
__STDC__                        Defined to indicate ISO C/C++
__cplusplus                     Defined when -c++ option used, value controlled by -std=c++NN
__STDC_VERSION__                May be defined when -c++ option is not used, value controlled by -std=cNN
```

Since SWIG 4.2.0, `__STDC__` is defined to `1` to match the behaviour of ISO C/C++ compilers. Before this SWIG defined it to have an empty value.

Since SWIG 4.2.0, `__cplusplus` is defined to `199711L` (the value for C++98) by default. Before this SWIG always defined it to have the **value** `__cplusplus`.

Since SWIG 4.2.0, SWIG supports command line options `-std=cNN` and `-std=c++NN` to specify the C/C++ standards version. The only effect of these options is to set appropriate values for `__STDC_VERSION__` and `__cplusplus` respectively, which is useful if you're wrapping headers which have preprocessor checks based on their values.

If your code requires these macros to be set to a version of the standard that is not a final official version, or one that SWIG is not yet aware of, you can simply redefine the appropriate macro to an alternative value at the top of your interface file, for example:

```
#undef __cplusplus
#define __cplusplus 202211L
```

The following are language specific symbols that might be defined:

```
SWIG_D_VERSION                  Unsigned integer target version when using D
SWIGGO_CGO                      Defined when using Go for cgo
SWIGGO_GCCGO                    Defined when using Go for gccgo
SWIGGO_INTGO_SIZE               Size of the Go type int when using Go (32 or 64)
SWIGPYTHON_BUILTIN              Defined when using Python with -builtin
SWIG_RUBY_AUTORENAME            Defined when using Ruby with -autorename
```

Interface files can look at these symbols as necessary to change the way in which an interface is generated or to mix SWIG directives with C code.

## 11.4 Macro Expansion

Traditional preprocessor macros can be used in SWIG interfaces. Be aware that the `#define` statement is also used to try and detect constants. Therefore, if you have something like this in your file,

```
#ifndef FOO_H 1
#define FOO_H 1
...
#endif
```

you may get some extra constants such as `FOO_H` showing up in the scripting interface.

More complex macros can be defined in the standard way. For example:

```
#define EXTERN extern
#ifdef __STDC__
#define ISOC_(args)   (args)
#else
#define ISOC_(args) ()
#endif
```

The following operators can appear in macro definitions:

- `#x`
  Converts macro argument `x` to a string surrounded by double quotes ("x").
- `x ## y`
  Concatenates x and y together to form `xy`.
- `` `x` ``
  If `x` is a string surrounded by double quotes, do nothing. Otherwise, turn into a string like `#x`. This is a non-standard SWIG extension.

## 11.5 SWIG Macros

SWIG provides an enhanced macro capability with the `%define` and `%enddef` directives. For example:

```
%define ARRAYHELPER(type, name)
%inline %{
type *new_ ## name (int nitems) {
  return (type *) malloc(sizeof(type)*nitems);
}
void delete_ ## name(type *t) {
  free(t);
}
type name ## _get(type *t, int index) {
  return t[index];
}
void name ## _set(type *t, int index, type val) {
  t[index] = val;
}
%}
%enddef

ARRAYHELPER(int, IntArray)
ARRAYHELPER(double, DoubleArray)
```

The primary purpose of `%define` is to define large macros of code. Unlike normal C preprocessor macros, it is not necessary to terminate each line with a continuation character (\)--the macro definition extends to the first occurrence of `%enddef`. Furthermore, when such macros are expanded, they are reparsed through the C preprocessor. Thus, SWIG macros can contain all other preprocessor directives except for nested `%define` statements.

The SWIG macro capability is a very quick and easy way to generate large amounts of code. In fact, many of SWIG's advanced features and libraries are built using this mechanism (such as C++ template support).

## 11.6 Variadic Macros

SWIG-1.3.12 and newer releases support variadic preprocessor macros which were standardised by C99 and C++11. For example:

```
#define DEBUGF(fmt, ...)   fprintf(stderr, fmt, __VA_ARGS__)
```

When used, any extra arguments to `...` are placed into the special variable `__VA_ARGS__`. This also works with special SWIG macros defined using `%define`.

The variable arguments can be empty. However, this often results in an extra comma (`,`) and syntax error in the resulting expansion. For example:

```
DEBUGF("hello");   --> fprintf(stderr, "hello", );
```

C++20 and C23 added `__VA_OPT__()` as a solution to this, which SWIG 4.3.0 added support for. `__VA_OPT__()` expands to its argument if the variable arguments contain any tokens, and to nothing otherwise. It can be used to solve the problem above like so:

```
#define DEBUGF(fmt, ...)   fprintf(stderr, fmt __VA_OPT__(,) __VA_ARGS__)
```

An early non-standardised solution to this problem which gave a special meaning to the token sequence `, ## __VA_ARGS__` is supported by several C and C++ compilers, and also by SWIG 4.3.0 and later (it was documented as supported by earlier SWIG versions, but didn't actually work in at least SWIG 2.x and 3.x). Using this feature you can get rid of the extra comma like this:

```
#define DEBUGF(fmt, ...)   fprintf(stderr, fmt, ##__VA_ARGS__)
```

SWIG also supports GNU-style variadic macros, which specify a name for the variable arguments instead of using `__VA_ARGS__`. For example:

```
#define DEBUGF(fmt, args...)  fprintf(stdout, fmt, args)
```

SWIG supports `__VA_OPT__()` in combination with GNU-style variadic macros (following the lead of GCC and clang which also support this, albeit with a warning by default).

## 11.7 Preprocessing and delimiters

The preprocessor handles { }, " " and %{ %} delimiters differently.

### 11.7.1 Preprocessing and %{ ... %} & " ... " delimiters

The SWIG preprocessor does not process any text enclosed in a code block %{ ... %}. Therefore, if you write code like this,

```
%{
#ifdef NEED_BLAH
int blah() {
  ...
}
#endif
%}
```

the contents of the `%{ ... %}` block are copied without modification to the output (including all preprocessor directives).

### 11.7.2 Preprocessing and { ... } delimiters

SWIG always runs the preprocessor on text appearing inside `{ ... }`. However, sometimes it is desirable to make a preprocessor directive pass through to the output file. For example:

```
%extend Foo {
  void bar() {
    #ifdef DEBUG
      printf("I'm in bar\n");
    #endif
  }
}
```

By default, SWIG will interpret the `#ifdef DEBUG` statement. However, if you really wanted that code to actually go into the wrapper file, prefix the preprocessor directives with `%` like this:

```
%extend Foo {
  void bar() {
    %#ifdef DEBUG
      printf("I'm in bar\n");
    %#endif
  }
}
```

SWIG will strip the extra `%` and leave the preprocessor directive in the code.

## 11.8 Preprocessor and Typemaps

Typemaps support a special attribute called `noblock` where the { ... } delimiters can be used, but the delimiters are not actually generated into the code. The effect is then similar to using "" or %{ %} delimiters but the code **is** run through the preprocessor. For example:

```
#define SWIG_macro(CAST) (CAST)$input
%typemap(in) Int {$1= SWIG_macro(int);}
```

might generate

```
  {
    arg1=(int)jarg1;
  }
```

whereas

```
#define SWIG_macro(CAST) (CAST)$input
%typemap(in, noblock=1) Int {$1= SWIG_macro(int);}
```

might generate

```
  arg1=(int)jarg1;
```

and

```
#define SWIG_macro(CAST) (CAST)$input
%typemap(in) Int %{$1=SWIG_macro(int);%}
```

would generate

```
arg1=SWIG_macro(int);
```

## 11.9 Viewing preprocessor output

Like many compilers, SWIG supports a `-E` command line option to display the output from the preprocessor. When the `-E` option is used, SWIG will not generate any wrappers. Instead the results after the preprocessor has run are displayed. This might be useful as an aid to debugging and viewing the results of macro expansions.

## 11.10 The #error and #warning directives

SWIG supports the standard `#warning` and `#error` preprocessor directives. The `#warning` directive will cause SWIG to issue a warning then continue processing. It was standardised in C++23 and C23, and has been widely supported as an extension by most C and C++ compilers for a long time. The `#error` directive will cause SWIG to exit with a fatal error. Example usage:

```
#error "This is a fatal error message"
#warning "This is a warning message"
```

The `#error` behaviour can be made to work like `#warning` if the `-cpperraswarn` commandline option is used. Alternatively, the `#pragma` directive can be used to the same effect, for example:

```
/* Modified behaviour: #error does not cause SWIG to exit with error */
#pragma SWIG cpperraswarn=1
/* Normal behaviour: #error does cause SWIG to exit with error */
#pragma SWIG cpperraswarn=0
```

## 11.11 Trigraphs

SWIG's preprocessor does not implement trigraphs (such as `??!` being mapped to `|`). They are very rarely used deliberately but these character sequences sometimes occur in code where they aren't intended as trigraphs. Compilers typically don't enable trigraph support by default, and they've been removed in C++17 and C23.

## 11.12 Digraphs

SWIG's preprocessor does not currently implement digraphs (such as `<%` being an alternative way to write the token `{`). These are standard in C++ and C95, but they're intended to support working with code on systems with very restricted character sets which are really rare these days so digraphs just don't seem to be used in practice.

# 12 SWIG library


- The %include directive and library search path
- C arrays and pointers
  - argcargv.i
  - cpointer.i
  - carrays.i
  - cmalloc.i
  - cdata.i
- C string handling
  - Default string handling
  - Passing a string with length
  - Using %newobject to release memory
  - cstring.i
- C standard library
  - Complex floating types
- STL/C++ library
  - std::string
  - std::string_view
  - std::vector
  - STL exceptions
  - shared_ptr smart pointer
    - shared_ptr basics
    - shared_ptr and inheritance
    - shared_ptr and method overloading
    - shared_ptr and templates
    - shared_ptr and directors
  - unique_ptr smart pointer
    - unique_ptr passed by value
    - unique_ptr passed by reference
  - auto_ptr smart pointer
- Utility Libraries
  - exception.i
  - attribute.i
    - %attribute and C++ templates


To help build extension modules, SWIG is packaged with a library of support files that you can include in your own interfaces. These files often define new SWIG directives or provide utility functions that can be used to access parts of the standard C and C++ libraries. This chapter provides a reference to the current set of supported library files.

**Compatibility note:** Older versions of SWIG included a number of library files for manipulating pointers, arrays, and other structures. Most these files are now deprecated and have been removed from the distribution. Alternative libraries provide similar functionality. Please read this chapter carefully if you used the old libraries.

## 12.1 The %include directive and library search path

Library files are included using the `%include` directive. When searching for files, directories are searched in the following order:

1. The current directory
2. Directories specified with the `-I` command line option
3. `./swig_lib`
4. SWIG library install location as reported by `swig -swiglib`, for example `/usr/local/share/swig/1.3.30`
5. On Windows, a directory `Lib` relative to the location of `swig.exe` is also searched.

Within directories mentioned in points 3-5, SWIG first looks for a subdirectory corresponding to a target language (e.g., `python`, `tcl`, etc.). If found, SWIG will search the language specific directory first. This allows for language-specific implementations of library files.

You can ignore the installed SWIG library by setting the `SWIG_LIB` environment variable. Set the environment variable to hold an alternative library directory.

The directories that are searched are displayed when using `-verbose` commandline option.

## 12.2 C arrays and pointers

This section describes library modules for manipulating low-level C arrays and pointers. The primary use of these modules is in supporting C declarations that manipulate bare pointers such as `int *`, `double *`, or `void *`. The modules can be used to allocate memory, manufacture pointers, dereference memory, and wrap pointers as class-like objects. Since these functions provide direct access to memory, their use is potentially unsafe and you should exercise caution.

### 12.2.1 argcargv.i

The argcargv.i library is a simple library providing multi-argument typemaps for handling C argc argv command line argument C string arrays. The `argc` parameter contains the argument count and `argv` contains the argument vector array.

This library provides the following multi-argument typemap:

**`(int ARGC, char **ARGV)`**

Apply this multi-argument typemap to your use case, for example:

```
%apply (int ARGC, char **ARGV) { (size_t argc, const char **argv) }

int mainApp(size_t argc, const char **argv);
```

then from Ruby:

```
$args = ["myarg1", "myarg2"]
mainApp(args);
```

### 12.2.2 cpointer.i

The `cpointer.i` module defines macros that can be used to generate wrappers around simple C pointers. The primary use of this module is in generating pointers to primitive datatypes such as `int` and `double`.

**`%pointer_functions(type, name)`**

Generates a collection of four functions for manipulating a pointer `type *`:

`type *new_name()`

Creates a new object of type `type` and returns a pointer to it. In C, the object is created using `calloc()`. In C++, `new` is used.

`type *copy_name(type value)`

Creates a new object of type `type` and returns a pointer to it. An initial value is set by copying it from `value`. In C, the object is created using `calloc()`. In C++, `new` is used.

`type *delete_name(type *obj)`

Deletes an object type `type`.

`void name_assign(type *obj, type value)`

Assigns `*obj = value`.

`type name_value(type *obj)`

Returns the value of `*obj`.

When using this macro, `type` may be any type and `name` must be a legal identifier in the target language. `name` should not correspond to any other name used in the interface file.

Here is a simple example of using `%pointer_functions()`:

```
%module example
%include "cpointer.i"

/* Create some functions for working with "int *" */
%pointer_functions(int, intp);

/* A function that uses an "int *" */
void add(int x, int y, int *result);
```

Now, in Python:

```
>>> import example
>>> c = example.new_intp()     # Create an "int" for storing result
>>> example.add(3, 4, c)       # Call function
>>> example.intp_value(c)      # Dereference
7
>>> example.delete_intp(c)     # Delete
```

**`%pointer_class(type, name)`**

Wraps a pointer of `type *` inside a class-based interface. This interface is as follows:

```
struct name {
  name();                           // Create pointer object
  ~name();                          // Delete pointer object
  void assign(type value);          // Assign value
  type value();                     // Get value
  type *cast();                     // Cast the pointer to original type
  static name *frompointer(type *);  // Create class wrapper from existing
                                    // pointer
};
```

When using this macro, `type` is restricted to a simple type name like `int`, `float`, or `Foo`. Pointers and other complicated types are not allowed. `name` must be a valid identifier not already in use. When a pointer is wrapped as a class, the "class" may be transparently passed to any function that expects the pointer.

If the target language does not support proxy classes, the use of this macro will produce the example same functions as `%pointer_functions()` macro.

It should be noted that the class interface does introduce a new object or wrap a pointer inside a special structure. Instead, the raw pointer is used directly.

Here is the same example using a class instead:

```
%module example
%include "cpointer.i"

/* Wrap a class interface around an "int *" */
%pointer_class(int, intp);

/* A function that uses an "int *" */
void add(int x, int y, int *result);
```

Now, in Python (using proxy classes)

```
>>> import example
>>> c = example.intp()         # Create an "int" for storing result
>>> example.add(3, 4, c)       # Call function
>>> c.value()                  # Dereference
7
```

Of the two macros, `%pointer_class` is probably the most convenient when working with simple pointers. This is because the pointers are access like objects and they can be easily garbage collected (destruction of the pointer object destroys the underlying object).

**`%pointer_cast(type1, type2, name)`**

Creates a casting function that converts `type1` to `type2` . The name of the function is`name`. For example:

```
%pointer_cast(int *, unsigned int *, int_to_uint);
```

In this example, the function `int_to_uint()` would be used to cast types in the target language.

**Note:** None of these macros can be used to safely work with strings (`char *` or `char **`).

**Note:** When working with simple pointers, typemaps can often be used to provide more seamless operation.

### 12.2.3 carrays.i

This module defines macros that assist in wrapping ordinary C pointers as arrays. The module does not provide any safety or an extra layer of wrapping--it merely provides functionality for creating, destroying, and modifying the contents of raw C array data.

**`%array_functions(type, name)`**

Creates four functions.

`type *new_name(size_t nelements)`

Creates a new array of objects of type `type`. In C, the array is allocated using `calloc()`. In C++, `new []` is used.

`type *delete_name(type *ary)`

Deletes an array. In C, `free()` is used. In C++, `delete []` is used.

`type name_getitem(type *ary, size_t index)`

Returns the value `ary[index]`.

`void name_setitem(type *ary, size_t index, type value)`

Assigns `ary[index] = value`.

When using this macro, `type` may be any type and `name` must be a legal identifier in the target language. `name` should not correspond to any other name used in the interface file.

Here is an example of `%array_functions()`. Suppose you had a function like this:

```
void print_array(double x[10]) {
  int i;
  for (i = 0; i < 10; i++) {
    printf("[%d] = %g\n", i, x[i]);
  }
}
```

To wrap it, you might write this:

```
%module example

%include "carrays.i"
%array_functions(double, doubleArray);
```

```
void print_array(double x[10]);
```

Now, in a scripting language, you might write this:

```
a = new_doubleArray(10)              # Create an array
for i in range(0, 10):
    doubleArray_setitem(a, i, 2 * i)  # Set a value
print_array(a)                       # Pass to C
delete_doubleArray(a)                # Destroy array
```

**%array_class(type, name)**

Wraps a pointer of `type *` inside a class-based interface. This interface is as follows:

```
struct name {
  name(size_t nelements);              // Create an array
  ~name();                             // Delete array
  type getitem(size_t index);          // Return item
  void setitem(size_t index, type value);  // Set item
  type *cast();                        // Cast to original type
  static name *frompointer(type *);    // Create class wrapper from
                                       // existing pointer
};
```

When using this macro, `type` is restricted to a simple type name like `int` or `float`. Pointers and other complicated types are not allowed. `name` must be a valid identifier not already in use. When a pointer is wrapped as a class, it can be transparently passed to any function that expects the pointer.

When combined with proxy classes, the `%array_class()` macro can be especially useful. For example:

```
%module example
%include "carrays.i"
%array_class(double, doubleArray);

void print_array(double x[10]);
```

Allows you to do this:

```
import example
c = example.doubleArray(10)  # Create double[10]
for i in range(0, 10):
    c[i] = 2 * i             # Assign values
example.print_array(c)       # Pass to C
```

**Note:** These macros do not encapsulate C arrays inside a special data structure or proxy. There is no bounds checking or safety of any kind. If you want this, you should consider using a special array object rather than a bare pointer.

**Note:** `%array_functions()` and `%array_class()` should not be used with types of `char` or `char *`. SWIG's default handling of these types is to handle them as character strings and the two macros do not do enough to change this.

### 12.2.4 cmalloc.i

This module defines macros for wrapping the low-level C memory allocation functions `malloc()`, `calloc()`, `realloc()`, and `free()`.

**%malloc(type [, name=type])**

Creates a wrapper around `malloc()` with the following prototype:

```
type *malloc_name(int nbytes = sizeof(type));
```

If `type` is `void`, then the size parameter `nbytes` is required. The `name` parameter only needs to be specified when wrapping a type that is not a valid identifier (e.g., "`int *`", "`double **`", etc.).

**%calloc(type [, name=type])**

Creates a wrapper around `calloc()` with the following prototype:

```
type *calloc_name(int nobj =1, int sz = sizeof(type));
```

If `type` is `void`, then the size parameter `sz` is required.

**%realloc(type [, name=type])**

Creates a wrapper around `realloc()` with the following prototype:

```
type *realloc_name(type *ptr, int nitems);
```

Note: unlike the C `realloc()`, the wrapper generated by this macro implicitly includes the size of the corresponding type. For example, `realloc_int(p, 100)` reallocates `p` so that it holds 100 integers.

**%free(type [, name=type])**

Creates a wrapper around `free()` with the following prototype:

```
void free_name(type *ptr);
```

**%sizeof(type [, name=type])**

Creates the constant:

```
%constant int sizeof_name = sizeof(type);
```

**%allocators(type [, name=type])**

Generates wrappers for all five of the above operations.

Here is a simple example that illustrates the use of these macros:

```
// SWIG interface
%module example
%include "cmalloc.i"

%malloc(int);
%free(int);

%malloc(int *, intp);
%free(int *, intp);

%allocators(double);
```

Now, in a script:

```
>>> from example import *
>>> a = malloc_int()
>>> a
'_000efa70_p_int'
>>> free_int(a)
>>> b = malloc_intp()
>>> b
'_000efb20_p_p_int'
>>> free_intp(b)
>>> c = calloc_double(50)
>>> c
'_000fab98_p_double'
>>> c = realloc_double(100000)
>>> free_double(c)
>>> print sizeof_double
8
>>>
```

### 12.2.5 cdata.i

The `cdata.i` module defines functions for converting raw C data to and from a target language type. The target language type is either:

- A string for the scripting languages. The scripting language must support strings with embedded binary data in order for this to work.
- A byte array for the statically typed languages; namely, `byte[]` for C# and Java, `ubyte[]` for D, `[]byte` for Go.

The primary applications of this module would be packing/unpacking of binary data structures---for instance, if you needed to extract data from a buffer.

The available APIs are:

**const char *cdata(void *ptr, size_t nbytes)**

Converts `nbytes` of data at `ptr` into the target language type. `ptr` can be any pointer.

**void memmove(void *ptr, const char *s)**

This is actually a wrapper of the standard C library `memmove` function, **`void *memmove(void *ptr, const void *src, size_t n)`** , which copies `n` bytes of data from `src` into the destination pointed to by `ptr`. Multi-argument typemaps are used so that the last two parameters, `src` and `n` are replaced by `s`, a string or byte array target language type, mentioned earlier. The string/byte array may contain embedded NULL bytes. Unlike the C library function nothing is returned by the wrapper function.

One use of these functions is packing and unpacking data from memory. Here is a short example:

```
// SWIG interface
%module example
%include "carrays.i"
%include "cdata.i"

%array_class(int, intArray);
```

Python example:

```
>>> a = intArray(10)
>>> for i in range(0, 10):
...     a[i] = i
>>> b = cdata(a, 40)
>>> b
'\x00\x00\x00\x00\x00\x00\x00\x01\x00\x00\x00\x02\x00\x00\x00\x03\x00\x00\x00\x04
\x00\x00\x00\x05\x00\x00\x00\x06\x00\x00\x00\x07\x00\x00\x00\x08\x00\x00\x00\t'
>>> c = intArray(10)
>>> memmove(c, b)
>>> print c[4]
4
>>>
```

Since the size of data is not always known, the following macro is also defined:

**%cdata(type [, name=type])**

Generates the following function for extracting C data for a given type.

```
char *cdata_name(type* ptr, int nitems)
```

`nitems` is the number of items of the given type to extract.

**Note:** These functions provide direct access to memory and can be used to overwrite data. Clearly they are unsafe.

## 12.3 C string handling

A common problem when working with C programs is dealing with functions that manipulate raw character data using `char *`. In part, problems arise because there are different interpretations of `char *`---it could be a NULL-terminated string or it could point to binary data. Moreover, functions that manipulate raw strings may mutate data, perform implicit memory allocations, or utilize fixed-sized buffers.

The problems (and perils) of using `char *` are well-known. However, SWIG is not in the business of enforcing morality. The modules in this section provide basic functionality for manipulating raw C strings.

### 12.3.1 Default string handling

Suppose you have a C function with this prototype:

```
char *foo(char *s);
```

The default wrapping behavior for this function is to set `s` to a raw `char *` that refers to the internal string data in the target language. In other words, if you were using a language like Tcl, and you wrote this,

```
% foo Hello
```

then `s` would point to the representation of "Hello" inside the Tcl interpreter. When returning a `char *`, SWIG assumes that it is a NULL-terminated string and makes a copy of it. This gives the target language its own copy of the result.

There are obvious problems with the default behavior. First, since a `char *` argument points to data inside the target language, it is **NOT** safe for a function to modify this data (doing so may corrupt the interpreter and lead to a crash). Furthermore, the default behavior does not work well with binary data. Instead, strings are assumed to be NULL-terminated.

### 12.3.2 Passing a string with length

If you have a function that expects string with a length,

```
size_t parity(char *str, size_t len, size_t initial);
```

you can wrap the parameters `(char *str, size_t len)` as a single argument using a typemap. Just do this:

```
%apply (char *STRING, size_t LENGTH) { (char *str, size_t len) };
...
size_t parity(char *str, size_t len, size_t initial);
```

Now, in the target language, you can use the string like this:

```
>>> s = "H\x00\x15eg\x09\x20"
>>> parity(s, 0)
```

In the wrapper function, the passed string will be expanded to a pointer and length parameter. The `(char *STRING, int LENGTH)` multi-argument typemap is also available in addition to `(char *STRING, size_t LENGTH)`.

### 12.3.3 Using %newobject to release memory

If you have a function that allocates memory like this,

```
char *foo() {
  char *result = (char *) malloc(...);
  ...
  return result;
}
```

then the SWIG generated wrappers will have a memory leak--the returned data will be copied into a string object and the old contents ignored.

To fix the memory leak, use the `%newobject` directive.

```
%newobject foo;
...
char *foo();
```

This will release the result if the appropriate target language support is available. SWIG provides the appropriate "newfree" typemap for `char *` so that the memory is released, however, you may need to provide your own "newfree" typemap for other types. See Object ownership and %newobject for more details.

### 12.3.4 cstring.i

The `cstring.i` library file provides a collection of macros for dealing with functions that either mutate string arguments or which try to output string data through their arguments. An example of such a function might be this rather questionable implementation:

```
void get_path(char *s) {
  // Potential buffer overflow---uh, oh.
  sprintf(s, "%s/%s", base_directory, sub_directory);
}
...
// Somewhere else in the C program
{
  char path[1024];
  ...
  get_path(path);
  ...
}
```

(Off topic rant: If your program really has functions like this, you would be well-advised to replace them with safer alternatives involving bounds checking).

The macros defined in this module all expand to various combinations of typemaps. Therefore, the same pattern matching rules and ideas apply.

**%cstring_bounded_output(parm, maxsize)**

Turns parameter *parm* into an output value. The output string is assumed to be NULL-terminated and smaller than *maxsize* characters. Here is an example:

```
%cstring_bounded_output(char *path, 1024);
...
void get_path(char *path);
```

In the target language:

```
>>> get_path()
/home/beazley/packages/Foo/Bar
>>>
```

Internally, the wrapper function allocates a small buffer (on the stack) of the requested size and passes it as the pointer value. Data stored in the buffer is then returned as a function return value. If the function already returns a value, then the return value and the output string are returned together (multiple return values). **If more than *maxsize* bytes are written, your program will crash with a buffer overflow!**

**%cstring_chunk_output(parm, chunksize)**

Turns parameter *parm* into an output value. The output string is always *chunksize* and may contain binary data. Here is an example:

```
%cstring_chunk_output(char *packet, PACKETSIZE);
...
void get_packet(char *packet);
```

In the target language:

```
>>> get_packet()
'\xa9Y:\xf6\xd7\xe1\x87\xdbH;y\x97\x7f\xd3\x99\x14V\xec\x06\xea\xa2\x88'
>>>
```

This macro is essentially identical to `%cstring_bounded_output` . The only difference is that the result is always*chunksize* characters. Furthermore, the result can contain binary data. **If more than *maxsize* bytes are written, your program will crash with a buffer overflow!**

**%cstring_bounded_mutable(parm, maxsize)**

Turns parameter *parm* into a mutable string argument. The input string is assumed to be NULL-terminated and smaller than *maxsize* characters. The output string is also assumed to be NULL-terminated and less than *maxsize* characters.

```
%cstring_bounded_mutable(char *ustr, 1024);
...
void make_upper(char *ustr);
```

In the target language:

```
>>> make_upper("hello world")
'HELLO WORLD'
>>>
```

Internally, this macro is almost exactly the same as `%cstring_bounded_output`. The only difference is that the parameter accepts an input value that is used to initialize the internal buffer. It is important to emphasize that this function does not mutate the string value passed---instead it makes a copy of the input value, mutates it, and returns it as a result. **If more than *maxsize* bytes are written, your program will crash with a buffer overflow!**

**%cstring_mutable(parm [, expansion])**

Turns parameter *parm* into a mutable string argument. The input string is assumed to be NULL-terminated. An optional parameter *expansion* specifies the number of extra characters by which the string might grow when it is modified. The output string is assumed to be NULL-terminated and less than the size of the input string plus any expansion characters.

```
%cstring_mutable(char *ustr);
...
void make_upper(char *ustr);

%cstring_mutable(char *hstr, HEADER_SIZE);
...
void attach_header(char *hstr);
```

In the target language:

```
>>> make_upper("hello world")
'HELLO WORLD'
>>> attach_header("Hello world")
'header: Hello world'
>>>
```

This macro differs from `%cstring_bounded_mutable()` in that a buffer is dynamically allocated (on the heap using `malloc/new`). This buffer is always large enough to store a copy of the input value plus any expansion bytes that might have been requested. It is important to emphasize that this function does not directly mutate the string value passed---instead it makes a copy of the input value, mutates it, and returns it as a result. **If the function expands the result by more than *expansion* extra bytes, then the program will crash with a buffer overflow!**

**%cstring_output_maxsize(parm, maxparm)**

This macro is used to handle bounded character output functions where both a `char *` and a maximum length parameter are provided. As input, a user simply supplies the maximum

length. The return value is assumed to be a NULL-terminated string.

```
%cstring_output_maxsize(char *path, int maxpath);
...
void get_path(char *path, int maxpath);
```

In the target language:

```
>>> get_path(1024)
'/home/beazley/Packages/Foo/Bar'
>>>
```

This macro provides a safer alternative for functions that need to write string data into a buffer. User supplied buffer size is used to dynamically allocate memory on heap. Results are placed into that buffer and returned as a string object.

**%cstring_output_withsize(parm, maxparm)**

This macro is used to handle bounded character output functions where both a `char *` and a pointer `int *` are passed. Initially, the `int *` parameter points to a value containing the maximum size. On return, this value is assumed to contain the actual number of bytes. As input, a user simply supplies the maximum length. The output value is a string that may contain binary data.

```
%cstring_output_withsize(char *data, int *maxdata);
...
void get_data(char *data, int *maxdata);
```

In the target language:

```
>>> get_data(1024)
'x627388912'
>>> get_data(1024)
'xyzzy'
>>>
```

This macro is a somewhat more powerful version of `%cstring_output_chunk()`. Memory is dynamically allocated and can be arbitrary large. Furthermore, a function can control how much data is actually returned by changing the value of the `maxparm` argument.

**%cstring_output_allocate(parm, release)**

This macro is used to return strings that are allocated within the program and returned in a parameter of type `char **`. For example:

```
void foo(char **s) {
  *s = (char *) malloc(64);
  sprintf(*s, "Hello world\n");
}
```

The returned string is assumed to be NULL-terminated. *`release`* specifies how the allocated memory is to be released (if applicable). Here is an example:

```
%cstring_output_allocate(char **s, free(*$1));
...
void foo(char **s);
```

In the target language:

```
>>> foo()
'Hello world\n'
>>>
```

**%cstring_output_allocate_size(parm, szparm, release)**

This macro is used to return strings that are allocated within the program and returned in two parameters of type `char **` and `int *`. For example:

```
void foo(char **s, int *sz) {
  *s = (char *) malloc(64);
  *sz = 64;
  // Write some binary data
  ...
}
```

The returned string may contain binary data. *`release`* specifies how the allocated memory is to be released (if applicable). Here is an example:

```
%cstring_output_allocate_size(char **s, int *slen, free(*$1));
...
void foo(char **s, int *slen);
```

In the target language:

```
>>> foo()
'\xa9Y:\xf6\xd7\xe1\x87\xdbH;y\x97\x7f\xd3\x99\x14V\xec\x06\xea\xa2\x88'
>>>
```

This is the safest and most reliable way to return binary string data in SWIG. If you have functions that conform to another prototype, you might consider wrapping them with a helper function. For example, if you had this:

```
char  *get_data(int *len);
```

You could wrap it with a function like this:

```
void my_get_data(char **result, int *len) {
  *result = get_data(len);
}
```

**Comments:**

- Support for the `cstring.i` module depends on the target language. Not all SWIG modules currently support this library.
- Reliable handling of raw C strings is a delicate topic. There are many ways to accomplish this in SWIG. This library provides support for a few common techniques.
- If used in C++, this library uses `new` and `delete []` for memory allocation. If using C, the library uses `malloc()` and `free()`.
- Rather than manipulating `char *` directly, you might consider using a special string structure or class instead.

## 12.4 C standard library

### 12.4.1 Complex floating types

SWIG has some support for complex floating types. By default the keyword `_Complex` is understood by the parser but `complex` is not treated as a keyword because it may be used as an identifier.

SWIG is only able to fully wrap complex floating types for some target languages. If you are using such a target language for complex floating types, you should include `complex.i` to enable this support:

```
%include "complex.i"
```

If SWIG is in C++ code this will actually include `std_complex.i` for you; otherwise it will include `ccomplex.i`.

For target languages without support this will fail to find a file to include. In this case, if you are wrapping headers which use `complex` then you'll need to add a define to your SWIG interface file to get SWIG to parse the headers:

```
#define complex _Complex
```

Then if wrapping as opaque types is not useful to you, you can use `%ignore` to tell SWIG not to wrap the functions and/or variables which use complex floating types.

## 12.5 STL/C++ library

The library modules in this section provide access to parts of the standard C++ library including the STL. SWIG support for the STL is an ongoing effort. Support is quite comprehensive for some language modules but some of the lesser used modules do not have quite as much library code written.

The following table shows which C++ classes are supported and the equivalent SWIG interface library file for the C++ library.

| C++ class | C++ Library file | SWIG Interface library file |
|---|---|---|
| std::array (C++11) | array | std_array.i |
| std::auto_ptr | memory | std_auto_ptr.i |
| std::complex | complex | std_complex.i |
| std::deque | deque | std_deque.i |
| std::list | list | std_list.i |
| std::map | map | std_map.i |
| std::multimap (C++11) | multimap | std_multimap.i |
| std::multiset (C++11) | multiset | std_multiset.i |
| std::pair | utility | std_pair.i |
| std::set | set | std_set.i |
| std::shared_ptr (C++11) | shared_ptr | std_shared_ptr.i |
| std::string | string | std_string.i |
| std::string_view (C++17) | string_view | std_string_view.i |
| std::unordered_map (C++11) | unordered_map | std_unordered_map.i |
| std::unordered_multimap (C++11) | unordered_multimap | std_unordered_multimap.i |
| std::unordered_multiset (C++11) | unordered_multiset | std_unordered_multiset.i |
| std::unordered_set (C++11) | unordered_set | std_unordered_set.i |
| std::vector | vector | std_vector.i |
| std::wstring | wstring | std_wstring.i |

The list is by no means complete; some language modules support a subset of the above and some support additional STL classes. Please look for the library files in the appropriate language library directory.

### 12.5.1 std::string

The `std_string.i` library provides typemaps for converting C++ `std::string` objects to and from strings in the target scripting language. For example:

```
%module example
%include "std_string.i"

std::string foo();
void        bar(const std::string &x);
```

In the target language:

```
x = foo();               # Returns a string object
bar("Hello World");      # Pass string as std::string
```

A common problem that people encounter is that of classes/structures containing a `std::string`. This can be overcome by defining a typemap. For example:

```
%module example
```

```
%include "std_string.i"

%apply const std::string& {std::string* foo};

struct my_struct
{
  std::string foo;
};
```

In the target language:

```
x = my_struct();
x.foo = "Hello World";    # assign with string
print x.foo;              # print as string
```

This module only supports types `std::string` and `const std::string &`. Pointers and non-const references are left unmodified and returned as SWIG pointers.

This library file is fully aware of C++ namespaces. If you export `std::string` or rename it with a typedef, make sure you include those declarations in your interface. For example:

```
%module example
%include "std_string.i"

using namespace std;
typedef std::string String;
...
void foo(string s, const String &t);     // std_string typemaps still applied
```

### 12.5.2 std::string_view

The `std_string_view.i` library provides typemaps for converting C++17 `std::string_view` objects to and from strings in the target scripting language. For example:

```
%module example
%include "std_string_view.i"

std::string_view foo();
void        bar(std::string_view x);
```

In the target language:

```
x = foo();               # Returns a string object
bar("Hello World");      # Pass string as std::string_view
```

For target languages for which SWIG supports directors, `directorout` typemaps are provided for `std::string_view`, but these require extra care to use safely. The issue is that returning `std::string_view` effectively returns a pointer to string data but doesn't own the pointed to data. For target languages where there isn't a native narrow string representation (e.g. C#, Java) a `static std::string` is used to cache the data, which works but isn't thread/reentrant safe. For target languages where there is a native narrow string representation SWIG will return a `std::string_view` pointing to that data, so you need to store the string to return somewhere which will persist for the lifetime the caller needs (e.g. put it in a member variable) - you can't return a temporary target language string. In both cases SWIG will issue a warning by default.

### 12.5.3 std::vector

The `std_vector.i` library provides support for the C++`std::vector` class in the STL. Using this library involves the use of the `%template` directive. All you need to do is to instantiate different versions of `vector` for the types that you want to use. For example:

```
%module example
%include "std_vector.i"

namespace std {
  %template(vectori) vector<int>;
  %template(vectord) vector<double>;
};
```

When a template `vector<X>` is instantiated a number of things happen:

- A class that exposes the C++ API is created in the target language . This can be used to create objects, invoke methods, etc. This class is currently a subset of the real STL vector class.
- Input typemaps are defined for `vector<X>`, `const vector<X> &`, and `const vector<X> *`. For each of these, a pointer `vector<X> *` may be passed or a native list object in the target language.
- An output typemap is defined for `vector<X>`. In this case, the values in the vector are expanded into a list object in the target language.
- For all other variations of the type, the wrappers expect to receive a `vector<X> *` object in the usual manner.
- An exception handler for `std::out_of_range` is defined.
- Optionally, special methods for indexing, item retrieval, slicing, and element assignment may be defined. This depends on the target language.

To illustrate the use of this library, consider the following functions:

```
/* File : example.h */

#include <vector>
#include <algorithm>
#include <functional>
#include <numeric>

double average(std::vector<int> v) {
  return std::accumulate(v.begin(), v.end(), 0.0)/v.size();
}

std::vector<double> half(const std::vector<double>& v) {
  std::vector<double> w(v);
  for (unsigned int i=0; i<w.size(); i++)
    w[i] /= 2.0;
  return w;
}
```

```
void halve_in_place(std::vector<double>& v) {
  for (std::vector<double>::iterator it = v.begin(); it != v.end(); ++it)
    *it /= 2.0;
}
```

To wrap with SWIG, you might write the following:

```
%module example
%{
#include "example.h"
%}

%include "std_vector.i"
// Instantiate templates used by example
namespace std {
  %template(IntVector) vector<int>;
  %template(DoubleVector) vector<double>;
}

// Include the header file with above prototypes
%include "example.h"
```

Now, to illustrate the behavior in the scripting interpreter, consider this Python example:

```
>>> from example import *
>>> iv = IntVector(4)          # Create an vector<int>
>>> for i in range(0, 4):
...      iv[i] = i
>>> average(iv)                # Call method
1.5
>>> average([0, 1, 2, 3])         # Call with list
1.5
>>> half([1, 2, 3])              # Half a list
(0.5, 1.0, 1.5)
>>> halve_in_place([1, 2, 3])   # Oops
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
TypeError: Type error. Expected _p_std__vectorTdouble_t
>>> dv = DoubleVector(4)
>>> for i in range(0, 4):
...       dv[i] = i
>>> halve_in_place(dv)       # Ok
>>> for i in dv:
...       print i
...
0.0
0.5
1.0
1.5
>>> dv[20] = 4.5
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
  File "example.py", line 81, in __setitem__
    def __setitem__(*args): return apply(examplec.DoubleVector___setitem__, args)
IndexError: vector index out of range
>>>
```

This library module is fully aware of C++ namespaces. If you use vectors with other names, make sure you include the appropriate `using` or typedef directives. For example:

```
%include "std_vector.i"

namespace std {
  %template(IntVector) vector<int>;
}

using namespace std;
typedef std::vector Vector;

void foo(vector<int> *x, const Vector &x);
```

**Note:** This module makes use of several advanced SWIG features including templatized typemaps and template partial specialization. If you are trying to wrap other C++ code with templates, you might look at the code contained in `std_vector.i`. Alternatively, you can show them the code if you want to make their head explode.

**Note:** This module is defined for all SWIG target languages. However argument conversion details and the public API exposed to the interpreter vary.

### 12.5.4 STL exceptions

Many of the STL wrapper functions add parameter checking and will throw a language dependent error/exception should the values not be valid. The classic example is array bounds checking. The library wrappers are written to throw a C++ exception in the case of error. The C++ exception in turn gets converted into an appropriate error/exception for the target language. By and large this handling should not need customising, however, customisation can easily be achieved by supplying appropriate "throws" typemaps. For example:

```
%module example
%include "std_vector.i"
%typemap(throws) std::out_of_range {
  // custom exception handler
}
%template(VectInt) std::vector<int>;
```

The custom exception handler might, for example, log the exception then convert it into a specific error/exception for the target language.

When using the STL it is advisable to add in an exception handler to catch all STL exceptions. The `%exception` directive can be used by placing the following code before any other methods or libraries to be wrapped:

```
%include "exception.i"

%exception {
  try {
    $action
  } catch (const std::exception& e) {
    SWIG_exception(SWIG_RuntimeError, e.what());
  }
}
```

Any thrown STL exceptions will then be gracefully handled instead of causing a crash.

### 12.5.5 shared_ptr smart pointer

#### 12.5.5.1 shared_ptr basics

Some target languages have support for handling the shared_ptr reference counted smart pointer. This smart pointer is available in the standard C++11 library as `std::shared_ptr`. It was also in TR1 as `std::tr1::shared_ptr` before it was fully standardized. Support for the widely used `boost::shared_ptr` is also available.

In order to use `std::shared_ptr`, the `std_shared_ptr.i` library file should be included:

```
%include <std_shared_ptr.i>
```

The pre-standard `std::tr1::shared_ptr` can be used by including the following macro before including the `std_shared_ptr.i` library file:

```
#define SWIG_SHARED_PTR_SUBNAMESPACE tr1
%include <std_shared_ptr.i>
```

In order to use `boost::shared_ptr`, the `boost_shared_ptr.i` library file should be included:

```
%include <boost_shared_ptr.i>
```

You can only use one of these variants of shared_ptr in your interface file at a time (also SWIG doesn't currently support using both `%shared_ptr(T)` and `%unique_ptr<T>` on the same type `T`). All three variants must be used in conjunction with the `%shared_ptr(T)` macro, where `T` is the underlying pointer type equating to usage `shared_ptr<T>`. The type `T` must be non-primitive. A simple example demonstrates usage:

```
%module example
%include <boost_shared_ptr.i>
%shared_ptr(IntValue)

%inline %{
#include <boost/shared_ptr.hpp>

struct IntValue {
  int value;
  IntValue(int v) : value(v) {}
};

static int extractValue(const IntValue &t) {
  return t.value;
}

static int extractValueSmart(boost::shared_ptr<IntValue> t) {
  return t->value;
}
%}
```

Note that the `%shared_ptr(IntValue)` declaration occurs after the inclusion of the `boost_shared_ptr.i` library which provides the macro and, very importantly, before any usage or declaration of the type, `IntValue`. The `%shared_ptr` macro provides, a few things for handling this smart pointer, but mostly a number of typemaps. These typemaps override the default typemaps so that the underlying proxy class is stored and passed around as a pointer to a `shared_ptr` instead of a plain pointer to the underlying type. This approach means that any instantiation of the type can be passed to methods taking the type by value, reference, pointer or as a smart pointer. The interested reader might want to look at the generated code, however, usage is simple and no different handling is required from the target language. For example, a simple use case of the above code from Java would be:

```
IntValue iv = new IntValue(1234);
int val1 = example.extractValue(iv);
int val2 = example.extractValueSmart(iv);
System.out.println(val1 + " " + val2);
```

#### 12.5.5.2 shared_ptr and inheritance

The shared_ptr library works quite differently to SWIG's normal, but somewhat limited, smart pointer handling. The shared_ptr library does not generate extra wrappers, just for smart pointer handling, in addition to the proxy class. The normal proxy class including inheritance relationships is generated as usual. The only real change introduced by the `%shared_ptr` macro is that the proxy class stores a pointer to the shared_ptr instance instead of a raw pointer to the instance. A proxy class derived from a base which is being wrapped with shared_ptr can and **must** be wrapped as a shared_ptr too. In other words all classes in an inheritance hierarchy must all be used with the `%shared_ptr` macro. For example the following code can be used with the base class shown earlier:

```
%shared_ptr(DerivedIntValue)
%inline %{
struct DerivedIntValue : IntValue {
  DerivedIntValue(int value) : IntValue(value) {}
  ...
};
%}
```

A shared_ptr of the derived class can now be passed to a method where the base is expected in the target language, just as it can in C++:

```
DerivedIntValue div = new DerivedIntValue(5678);
int val3 = example.extractValue(div);
```

```
int val4 = example.extractValueSmart(div);
```

If the `%shared_ptr` macro is omitted for any class in the inheritance hierarchy, SWIG will warn about this and the generated code may or may not result in a C++ compilation error. For example, the following input:

```
%include "boost_shared_ptr.i"
%shared_ptr(Parent);

%inline %{
  #include <boost/shared_ptr.hpp>
  struct GrandParent {
    virtual ~GrandParent() {}
  };

  struct Parent : GrandParent {
    virtual ~Parent() {}
  };

  struct Child : Parent {
    virtual ~Child() {}
  };
%}
```

warns about the missing smart pointer information:

```
example.i:12: Warning 520: Base class 'GrandParent' of 'Parent' is not similarly marked as a smart pointer.
example.i:16: Warning 520: Derived class 'Child' of 'Parent' is not similarly marked as a smart pointer.
```

Adding the missing `%shared_ptr` macros will fix this:

```
%include <boost_shared_ptr.i>
%shared_ptr(GrandParent);
%shared_ptr(Parent);
%shared_ptr(Child);

... as before ...
```

#### 12.5.5.3 shared_ptr and method overloading

A C++ compiler can disambiguate a method overloaded by a shared_ptr and one using the raw underlying type. For example, either one of these methods can be called in C++:

```
int age(std::shared_ptr<GrandParent> num);
int age(GrandParent& num);
```

When wrapped by SWIG, disambiguation is not possible using the overloaded names as there is just one equivalent type (`GrandParent` ) in the target language. SWIG will choose to wrap just the first method by default. Ambiguity in overloading discusses ways to control which method(s) gets wrapped using `%ignore` or `%rename`. For the interested reader, SWIG detects that they are equivalent types via the typecheck typemaps in the shared_ptr library.

#### 12.5.5.4 shared_ptr and templates

The `%shared_ptr` macro should be used for all the required instantiations of the template before each of the `%template` instantiations. For example, consider `number.h` containing the following illustrative template:

```
#include <memory>

template<int N> struct Number {
  int num;
  Number() : num(N) {}
  static std::shared_ptr<Number<N>> make() { return std::make_shared<Number<N>>(); }
};
```

The SWIG code below shows the required ordering:

```
%include <std_shared_ptr.i>

%shared_ptr(Number<10>);
%shared_ptr(Number<42>);

%{
  #include "number.h"
%}
%include "number.h"

%template(Number10) Number<10>;
%template(Number42) Number<42>;
```

#### 12.5.5.5 shared_ptr and directors

The languages that support shared_ptr also have support for using shared_ptr with directors.

### 12.5.6 unique_ptr smart pointer

The `std_unique_ptr.i` library file provides SWIG's unique_ptr support. It provides move semantics for the smart pointer's underlying object, both from C++ to the target language and vice versa.

The library defines typemaps and a macro, `%unique_ptr(T)`, to use for handling `std::unique_ptr<T>` for a type `T`. The type `T` must be non-primitive. This macro should be used before any code declaring or using type `T`. Ordering requirements for using this smart pointer macro are the same as the equivalent `%shared_ptr(T)` macro covered in the previous section. The ownership and move semantics described here can of course be modified if not suitable by copying and customising the typemaps in the appropriate `std_unique_ptr.i` library file.

Note that SWIG doesn't currently support using both `%shared_ptr(T)` and `%unique_ptr<T>` on the same type `T` .

#### 12.5.6.1 unique_ptr passed by value

Example usage of a `std::unique_ptr` being returned from a function by value is shown below.

```
%include <std_unique_ptr.i>

%unique_ptr(Klass)
%inline %{
#include <memory>
class Klass {
public:
  // Factory function creating objects of this class:
  static std::unique_ptr<Klass> Create(int value) {
    return std::unique_ptr<Klass>(new Klass(value));
  }

  int getValue() const { return m_value; }

private:
  Klass(int value) : m_value(value) {}
  int m_value;
};
%}
```

The returned objects can be used naturally from the target language, e.g. from C#:

```
Klass k = Klass.Create(17);
int value = k.getValue();
```

The implementation simply calls `std::unique_ptr::release()` to obtain the underlying raw pointer. The pointer is then used to create a target language proxy class in the same way that SWIG handles a C++ function returning a class by value. The target language proxy class then owns the memory pointed to by the raw pointer and memory handling is identical to normal SWIG proxy class handling of the underlying C++ memory. Note that an object returned by value is first copied/moved from the stack onto the heap in order to obtain a raw pointer on the heap, whereas the underlying raw pointer in `std::unique_ptr` already points to an object the heap.

Note that the implementation is quite different to the `std::shared_ptr` smart pointer, where the proxy class manages the underlying C++ memory as a pointer to a shared_ptr instead of a plain raw pointer.

A possibly less common usage of this smart pointer is as a parameter to a function. When used like this it indicates that memory usage of the object pointed to by the underlying pointer is transferred to the function being called. The code that SWIG generates assumes this happens. First, it is assumed that a proxy class already owns the underlying C++ object and is used to pass the object to the C++ function being called. Second, the ownership is transferred from the proxy class to the C++ function being called and lifetime is then controlled by the function. Finally, it is assumed the lifetime of the object may not last beyond returning from the C++ function and hence the proxy class can no longer be used.

Consider expanding the example above with a function that takes a `std::unique_ptr` as follows:

```
void take(std::unique_ptr<Klass>);
```

and use from C#:

```
Klass k = Klass.Create(17); // create an instance of Klass any way you like
int value = k.getValue();   // ok
example.take(k);            // memory ownership passes from C# layer to C++ layer
int v = k.getValue();       // don't do this - invalid use of k
```

Attempts to use `k` after the ownership has been passed into the `take` function should not be attempted. The implementation sets the proxy class to an invalid state by setting the class's underlying C++ pointer to null after the return from the `take` function. Subsequent use of an invalid proxy class instance is very much dependent on the implementation in the target language and ranges from a segfault to giving a nice error. Consider implementing additional checks via the 'check' typemap.

Attempts to pass ownership from a proxy class to a `std::unique` parameter more than once will result in a "Cannot release ownership as memory is not owned" exception. For example, if `example.take(k)` in the example above is called twice.

#### 12.5.6.2 unique_ptr passed by reference

The effect of passing a `std::unique_ptr` by rvalue reference into a function is identical to passing it by value. The ownership of the memory of the object being pointed to by the underyling pointer is transferred from the proxy class to the C++ function being called. Example:

```
void grab(std::unique_ptr<Klass> &&);
```

Passing non-const lvalue references into a function is a bit quirky and not perfect due to ambiguities as to what the function may do. The approach taken is the ownership is transferred out of the target language from the proxy class into C++ space and the proxy class can then no longer be used after the wrapped function returns. In summary it works much like passing a `std::unique_ptr` by value into a function. The assumption is the function will not modify the `std::unique_ptr`. If this is not true and the underlying pointer is changed, such as calling the member functions, `swap`, `reset` or `release`, then the modified `std::unique_ptr` will effectively be ignored. It is destroyed when the function exits C++ space on return to the target language. Example:

```
void process(std::unique_ptr<Klass> &);
```

Passing const lvalue references into a function works much like passing any wrapped class. The proxy class owning the underling C++ object continues to own the underying C++ object after calling the function, the function cannot modify the `std::unique_ptr` or take ownership. Example:

```
void use(const std::unique_ptr<Klass> &);
```

Move semantics are not provided when wrapping a C++ function that returns a `std::unique_ptr` by reference. The target language proxy class wrapper that is returned does not own the underlying C++ object. This applies to all reference types, such as:

```
std::unique_ptr<Klass> & LvalueRefReturn();
std::unique_ptr<Klass> && RvalueRefReturn();
```

**Compatibility note:** Support for `std::unique_ptr` was first added in SWIG-4.1.0. This initial support contained the move semantics when passing a `std::unique_ptr` around by value. Support for passing a `std::unique_ptr` around by reference was added in SWIG-4.3.0.

### 12.5.7 auto_ptr smart pointer

While `std::auto_ptr` is deprecated in C++11, some existing code may still be using it. SWIG provides support for this class which is nearly identical to `std::unique_ptr`.

The `std_auto_ptr.i` library file provides SWIG's auto_ptr support. It defines typemaps and a macro, `%auto_ptr(T)`, to use for handling `std::auto_ptr<T>` for a type `T`. The type `T` must be non-primitive. This macro should be used before any code declaring or using type `T`. Ordering requirements for using this smart pointer macro are the same as the equivalent `%shared_ptr(T)` and `%unique_ptr` macros covered in the previous two sections.

Example usage of a `std::auto_ptr` being returned from a function is shown below.

```
%include <std_auto_ptr.i>

%auto_ptr(Klass)
%inline %{
#include <memory>
class Klass {
public:
  // Factory function creating objects of this class:
  static std::auto_ptr<Klass> Create(int value) {
    return std::auto_ptr<Klass>(new Klass(value));
  }

  int getValue() const { return m_value; }

private:
  Klass(int value) : m_value(value) {}
  int m_value;
};
%}
```

The returned objects can be used naturally from the target language, e.g. from C#:

```
Klass k = Klass.Create(17);
int value = k.getValue();
```

The implementation simply calls `std::auto_ptr::release()` to obtain the underlying raw pointer. That is, it works the same way covered in the previous section for `std::unique_ptr`.

Input parameters also work the same way as `std::unique_ptr` covered in the previous section.

## 12.6 Utility Libraries

### 12.6.1 exception.i

The `exception.i` library provides a language-independent function for raising a run-time exception in the target language. This library is largely used by the SWIG library writers. If possible, use the error handling scheme available to your target language as there is greater flexibility in what errors/exceptions can be thrown.

**`SWIG_exception(int code, const char *message)`**

Raises an exception in the target language. `code` is one of the following symbolic constants:

```
SWIG_MemoryError
SWIG_IOError
SWIG_RuntimeError
SWIG_IndexError
SWIG_TypeError
SWIG_DivisionByZero
SWIG_OverflowError
SWIG_SyntaxError
SWIG_ValueError
SWIG_SystemError
SWIG_NullReferenceError
```

`message` is a string indicating more information about the problem.

The primary use of this module is in writing language-independent exception handlers. For example:

```
%include "exception.i"
%exception std::vector::getitem {
  try {
    $action
  } catch (std::out_of_range& e) {
    SWIG_exception(SWIG_IndexError, const_cast<char*>(e.what()));
  }
}
```

### 12.6.2 attribute.i

The attribute library contains a set of macros to convert a pair of set/get methods into a "native" attribute/property.

Use `%attribute` when you have a pair of get/set methods to a primitive type like:

```
%include "attribute.i"
%attribute(A, int, a, get_a, set_a);

struct A {
  int get_a() const;
  void set_a(int aa);
};
```

and you want to provide that variable as an attribute in the target language. This example only works for primitive types, not derived types. Now you can use the attributes like so (in Python):

```
x = A()
x.a = 3        # calls A::set_a(3)
print(x.a)     # calls A::get_a() const
```

If you don't provide a 'set' method, a 'read-only' attribute is generated, ie, like:

```
%attribute(A, int, c, get_c);
```

Use `%attributeref` when you have const/non-const reference access methods for primitive types or class/structs, like:

```
%attributeref(A, int, b);

struct A {
  const int & b() const;
  int & b();
};

%attributeref(B, int, c);

struct B {
  int & c();
};
```

Use the attributes like so (in Python):

```
x = A()
x.b = 3        # calls A::b()
print(x.b)     # calls A::b() const
```

You can also use

```
%attributeref(Class, AttributeType, AttributeName, AccessorMethod)
```

if the internal C++ reference methods have a different name from the attribute you want, so

```
%attributeref(B, int, d, c);
```

is the same as the last example, but instead of the attribute 'c' being called 'c', it is called 'd'.

Use `%attribute2` instead of `%attribute` to indicate that reference-pointer translation is required. Use `%attribute2` instead of `%attribute` in cases like this:

```
%attribute2(MyClass, MyFoo, Foo, GetFoo, SetFoo);
%inline %{
  struct MyFoo {
    int x;
  };
  class MyClass {
    MyFoo foo;
  public:
    MyFoo & GetFoo() { return foo; }
    void SetFoo(const MyFoo &other) { foo = other; }
  };
%}
```

Here, the data type of the property is a wrapped type `MyFoo` and on the C++ side it is passed by reference. The problem is that the SWIG wrapper will pass around a pointer (MyFoo *) which is not compatible with the reference type of the accessors (MyFoo &). Therefore, if you use `%attribute`, you'll get an error from your C/C++ compiler. `%attribute2` translates between a pointer and a reference to eliminate the error. In case you're confused, let's make it simple: just use `%attribute` at first, but if the C/C++ compiler gives an error while compiling the wrapper, try `%attribute2` instead.

NOTE: remember that if the type contains commas, such as `std::pair<int, int>`, you need to use the macro like:

```
%attributeref(A, %arg(std::pair<int, int>), pval);
```

where `%arg()` 'normalizes' the type to be understood as a single argument, otherwise the macro will get confused by the comma.

The `%attributeval` is the same as `%attribute`, but should be used when the type is a class/struct (ie a non-primitive type) and when the get and set methods return/pass by value. The following is very similar to the above example, but note that the access is by value rather than reference.

```
%attributeval(MyClassVal, MyFoo, ReadWriteFoo, GetFoo, SetFoo);
%attributeval(MyClassVal, MyFoo, ReadOnlyFoo, GetFoo);
%inline %{
  class MyClassVal {
    MyFoo foo;
  public:
    MyFoo GetFoo() { return foo; }
    void SetFoo(MyFoo other) { foo = other; }
  };
%}
```

The `%attributestring` is the same as `%attributeval`, but should be used for string class types, which are unusual as they are a class on the C++ side, but normally an immutable/primitive type in the target language. Example usage for `std::string`:

```
%include <std_string.i>
%attributestring(MyStringyClass, std::string, ReadWriteString, GetString, SetString);
%attributestring(MyStringyClass, std::string, ReadOnlyString, GetString);
```

```
%inline %{
  class MyStringyClass {
    std::string str;
  public:
    MyStringyClass(const std::string &val) : str(val) {}
    std::string GetString() { return str; }
    void SetString(std::string other) { str = other; }
  };
%}
```

The `%attributestring` also works for class types that have `%naturalvar` turned on and so is also useful for shared_ptr which has `%naturalvar` turned on in `%shared_ptr`.

#### 12.6.2.1 %attribute and C++ templates

`%attribute` and friends have to be used on fully specified classes. For example

```
%attributeref(A<int>, int, a);
%inline %{
  template <class T> struct A {
    T a() const;
    void a(T &);
  };
%}
```

Note the use of a template-id (i.e., `A<int>` not `A<T>` or just `A`). This means that `%attribute` statements have to be repeated for any template-id that you want to use with `%template`.

---

# 13 Argument Handling


- The typemaps.i library
  - Introduction
  - Input parameters
  - Output parameters
  - Input/Output parameters
  - Using different names
- Applying constraints to input values
  - Simple constraint example
  - Constraint methods
  - Applying constraints to new datatypes


In Chapter 5, SWIG's treatment of basic datatypes and pointers was described. In particular, primitive types such as `int` and `double` are mapped to corresponding types in the target language. For everything else, pointers are used to refer to structures, classes, arrays, and other user-defined datatypes. However, in certain applications it is desirable to change SWIG's handling of a specific datatype. For example, you might want to return multiple values through the arguments of a function. This chapter describes some of the techniques for doing this.

## 13.1 The typemaps.i library

This section describes the `typemaps.i` library file--commonly used to change certain properties of argument conversion.

### 13.1.1 Introduction

Suppose you had a C function like this:

```
void add(double a, double b, double *result) {
  *result = a + b;
}
```

From reading the source code, it is clear that the function is storing a value in the `double *result` parameter. However, since SWIG does not examine function bodies, it has no way to know that this is the underlying behavior.

One way to deal with this is to use the `typemaps.i` library file and write interface code like this:

```
// Simple example using typemaps
%module example
%include "typemaps.i"

%apply double *OUTPUT { double *result };
%inline %{
extern void add(double a, double b, double *result);
%}
```

The `%apply` directive tells SWIG that you are going to apply a special type handling rule to a type. The "`double *OUTPUT`" specification is the name of a rule that defines how to return an output value from an argument of type `double *`. This rule gets applied to all of the datatypes listed in curly braces-- in this case "` double *result`".

When the resulting module is created, you can now use the function like this (shown for Python):

```
>>> a = add(3, 4)
>>> print a
7
>>>
```

In this case, you can see how the output value normally returned in the third argument has magically been transformed into a function return value. Clearly this makes the function much easier to use since it is no longer necessary to manufacture a special `double *` object and pass it to the function somehow.

Once a typemap has been applied to a type, it stays in effect for all future occurrences of the type and name. For example, you could write the following:

```
%module example
%include "typemaps.i"

%apply double *OUTPUT { double *result };
```

```
%inline %{
extern void add(double a, double b, double *result);
extern void sub(double a, double b, double *result);
extern void mul(double a, double b, double *result);
extern void div(double a, double b, double *result);
%}
...
```

In this case, the `double *OUTPUT` rule is applied to all of the functions that follow.

Typemap transformations can even be extended to multiple return values. For example, consider this code:

```
%include "typemaps.i"
%apply int *OUTPUT { int *width, int *height };

// Returns a pair (width, height)
void getwinsize(int winid, int *width, int *height);
```

In this case, the function returns multiple values, allowing it to be used like this:

```
>>> w, h = genwinsize(wid)
>>> print w
400
>>> print h
300
>>>
```

It should also be noted that although the `%apply` directive is used to associate typemap rules to datatypes, you can also use the rule names directly in arguments. For example, you could write this:

```
// Simple example using typemaps
%module example
%include "typemaps.i"

%{
extern void add(double a, double b, double *OUTPUT);
%}
extern void add(double a, double b, double *OUTPUT);
```

Typemaps stay in effect until they are explicitly deleted or redefined to something else. To clear a typemap, the `%clear` directive should be used. For example:

```
%clear double *result;      // Remove all typemaps for double *result
```

### 13.1.2 Input parameters

The following typemaps instruct SWIG that a pointer really only holds a single input value:

```
int *INPUT
short *INPUT
long *INPUT
unsigned int *INPUT
unsigned short *INPUT
unsigned long *INPUT
double *INPUT
float *INPUT
```

When used, it allows values to be passed instead of pointers. For example, consider this function:

```
double add(double *a, double *b) {
  return *a+*b;
}
```

Now, consider this SWIG interface:

```
%module example
%include "typemaps.i"
...
%{
extern double add(double *, double *);
%}
extern double add(double *INPUT, double *INPUT);
```

When the function is used in the scripting language interpreter, it will work like this:

```
result = add(3, 4)
```

### 13.1.3 Output parameters

The following typemap rules tell SWIG that pointer is the output value of a function. When used, you do not need to supply the argument when calling the function. Instead, one or more output values are returned.

```
int *OUTPUT
short *OUTPUT
long *OUTPUT
```

```
unsigned int *OUTPUT
unsigned short *OUTPUT
unsigned long *OUTPUT
double *OUTPUT
float *OUTPUT
```

These methods can be used as shown in an earlier example. For example, if you have this C function :

```
void add(double a, double b, double *c) {
  *c = a+b;
}
```

A SWIG interface file might look like this :

```
%module example
%include "typemaps.i"
...
%inline %{
extern void add(double a, double b, double *OUTPUT);
%}
```

In this case, only a single output value is returned, but this is not a restriction. An arbitrary number of output values can be returned by applying the output rules to more than one argument (as shown previously).

If the function also returns a value, it is returned along with the argument. For example, if you had this:

```
extern int foo(double a, double b, double *OUTPUT);
```

The function will return two values like this:

```
iresult, dresult = foo(3.5, 2)
```

### 13.1.4 Input/Output parameters

When a pointer serves as both an input and output value you can use the following typemaps :

```
int *INOUT
short *INOUT
long *INOUT
unsigned int *INOUT
unsigned short *INOUT
unsigned long *INOUT
double *INOUT
float *INOUT
```

A C function that uses this might be something like this:

```
void negate(double *x) {
  *x = -(*x);
}
```

To make x function as both and input and output value, declare the function like this in an interface file :

```
%module example
%include "typemaps.i"
...
%{
extern void negate(double *);
%}
extern void negate(double *INOUT);
```

Now within a script, you can simply call the function normally :

```
a = negate(3);         # a = -3 after calling this
```

One subtle point of the `INOUT` rule is that many scripting languages enforce mutability constraints on primitive objects (meaning that simple objects like integers and strings aren't supposed to change). Because of this, you can't just modify the object's value in place as the underlying C function does in this example. Therefore, the `INOUT` rule returns the modified value as a new object rather than directly overwriting the value of the original input object.

### 13.1.5 Using different names

As previously shown, the `%apply` directive can be used to apply the `INPUT`, `OUTPUT`, and `INOUT` typemaps to different argument names. For example:

```
// Make double *result an output value
%apply double *OUTPUT { double *result };

// Make Int32 *in an input value
%apply int *INPUT { Int32 *in };

// Make long *x inout
%apply long *INOUT {long *x};
```

```
```

To clear a rule, the `%clear` directive is used:

```
%clear double *result;
%clear Int32 *in, long *x;
```

Typemap declarations are lexically scoped so a typemap takes effect from the point of definition to the end of the file or a matching `%clear` declaration.

## 13.2 Applying constraints to input values

In addition to changing the handling of various input values, it is also possible to use typemaps to apply constraints. For example, maybe you want to insure that a value is positive, or that a pointer is non-NULL. This can be accomplished including the `constraints.i` library file.

### 13.2.1 Simple constraint example

The constraints library is best illustrated by the following interface file :

```
// Interface file with constraints
%module example
%include "constraints.i"

double exp(double x);
double log(double POSITIVE);         // Allow only positive values
double sqrt(double NONNEGATIVE);     // Non-negative values only
double inv(double NONZERO);          // Non-zero values
int    fclose(FILE *NONNULL);        // Non-NULL pointers only
```

The behavior of this file is exactly as you would expect. If any of the arguments violate the constraint condition, a scripting language exception will be raised. As a result, it is possible to catch bad values, prevent mysterious program crashes and so on.

### 13.2.2 Constraint methods

The following constraints are currently available

```
POSITIVE                    Any number > 0 (not zero)
NEGATIVE                    Any number < 0 (not zero)
NONNEGATIVE                 Any number >= 0
NONPOSITIVE                 Any number <= 0
NONZERO                     Nonzero number
NONNULL                     Non-NULL pointer (pointers only).
```

### 13.2.3 Applying constraints to new datatypes

The constraints library only supports the primitive C datatypes, but it is easy to apply it to new datatypes using `%apply`. For example :

```
// Apply a constraint to a Real variable
%apply Number POSITIVE { Real in };

// Apply a constraint to a pointer type
%apply Pointer NONNULL { Vector * };
```

The special types of "Number" and "Pointer" can be applied to any numeric and pointer variable type respectively. To later remove a constraint, the `%clear` directive can be used :

```
%clear Real in;
%clear Vector *;
```

# 14 Typemaps


- Introduction
  - Type conversion
  - Typemaps
  - Pattern matching
  - Reusing typemaps
  - What can be done with typemaps?
  - What can't be done with typemaps?
  - Similarities to Aspect Oriented Programming
  - The rest of this chapter
- Typemap specifications
  - Defining a typemap
  - Typemap scope
  - Copying a typemap
  - Deleting a typemap
  - Placement of typemaps
- Pattern matching rules
  - Basic matching rules
  - Typedef reductions matching
  - Default typemap matching rules
  - Multi-arguments typemaps
  - Matching rules compared to C++ templates
  - Debugging typemap pattern matching
- Code generation rules
  - Scope
  - Declaring new local variables
  - Special variables

- Type related special variables
- Non-type related special variables
- Special variable macros
  - $descriptor(type)
  - $typemap(method, typepattern)
  - $typemap(method:attribute, typepattern)
- Special variables and typemap attributes
- Special variables combined with special variable macros

- Common typemap methods
  - "in" typemap
  - "typecheck" typemap
  - "out" typemap
  - "arginit" typemap
  - "default" typemap
  - "check" typemap
  - "argout" typemap
  - "freearg" typemap
  - "newfree" typemap
  - "ret" typemap
  - "memberin" typemap
  - "varin" typemap
  - "varout" typemap
  - "throws" typemap
- Some typemap examples
  - Typemaps for arrays
  - Implementing constraints with typemaps
- Typemaps for multiple target languages
- Optimal code generation when returning by value
- Multi-argument typemaps
- Typemap warnings
- Typemap fragments
  - Fragment type specialization
  - Fragments and automatic typemap specialization
- The run-time type checker
  - Implementation
  - Usage
- Typemaps and overloading
  - SWIG_TYPECHECK_POINTER precedence level and the typecheck typemap
- More about %apply and %clear
- Passing data between typemaps
- C++ "this" pointer
- Where to go for more information?


## 14.1 Introduction

Chances are, you are reading this chapter for one of two reasons; you either want to customize SWIG's behavior or you overheard someone mumbling some incomprehensible drivel about "typemaps" and you asked yourself "typemaps, what are those?" That said, let's start with a short disclaimer that "typemaps" are an advanced customization feature that provide direct access to SWIG's low-level code generator. Not only that, they are an integral part of the SWIG C++ type system (a non-trivial topic of its own). Typemaps are generally *not* a required part of using SWIG. Therefore, you might want to re-read the earlier chapters if you have found your way to this chapter with only a vague idea of what SWIG already does by default.

### 14.1.1 Type conversion

One of the most important problems in wrapper code generation is the conversion or marshalling of datatypes between programming languages. Specifically, for every C/C++ declaration, SWIG must somehow generate wrapper code that allows values to be passed back and forth between languages. Since every programming language represents data differently, this is not a simple of matter of simply linking code together with the C linker. Instead, SWIG has to know something about how data is represented in each language and how it can be manipulated.

To illustrate, suppose you had a simple C function like this:

```
int factorial(int n);
```

To access this function from Python, a pair of Python API functions are used to convert integer values. For example:

```
long PyInt_AsLong(PyObject *obj);      /* Python --> C */
PyObject *PyInt_FromLong(long x);      /* C --> Python */
```

The first function is used to convert the input argument from a Python integer object to C `long`. The second function is used to convert a value from C back into a Python integer object.

Inside the wrapper function, you might see these functions used like this:

```
PyObject *wrap_factorial(PyObject *self, PyObject *args) {
  int       arg1;
  int       result;
  PyObject *obj1;
  PyObject *resultobj;

  if (!PyArg_ParseTuple("O:factorial", &obj1)) return NULL;
  arg1 = PyInt_AsLong(obj1);
  result = factorial(arg1);
  resultobj = PyInt_FromLong(result);
  return resultobj;
}
```

Every target language supported by SWIG has functions that work in a similar manner. For example, in Perl, the following functions are used:

```
IV SvIV(SV *sv);                    /* Perl --> C */
void sv_setiv(SV *sv, IV val);      /* C --> Perl */
```

In Tcl:

```
int Tcl_GetLongFromObj(Tcl_Interp *interp, Tcl_Obj *obj, long *value);
Tcl_Obj *Tcl_NewIntObj(long value);
```

The precise details are not so important. What is important is that all of the underlying type conversion is handled by collections of utility functions and short bits of C code like this---you simply have to read the extension documentation for your favorite language to know how it works (an exercise left to the reader).

### 14.1.2 Typemaps

Since type handling is so central to wrapper code generation, SWIG allows it to be completely defined (or redefined) by the user. To do this, a special `%typemap` directive is used. For example:

```
/* Convert from Python --> C */
%typemap(in) int {
  $1 = PyInt_AsLong($input);
}

/* Convert from C --> Python */
%typemap(out) int {
  $result = PyInt_FromLong($1);
}
```

At first glance, this code will look a little confusing. However, there is really not much to it. The first typemap (the "in" typemap) is used to convert a value from the target language to C. The second typemap (the "out" typemap) is used to convert in the other direction. The content of each typemap is a small fragment of code that is inserted directly into the SWIG generated wrapper functions. The code is usually C or C++ code which will be generated into the C/C++ wrapper functions. Note that this isn't always the case as some target language modules allow target language code within the typemaps which gets generated into target language specific files. Within this code, a number of special variables prefixed with a `$` are expanded. These are really just placeholders for C/C++ variables that are generated in the course of creating the wrapper function. In this case, `$input` refers to an input object that needs to be converted to C/C++ and `$result` refers to an object that is going to be returned by a wrapper function. `$1` refers to a C/C++ variable that has the same type as specified in the typemap declaration (an `int` in this example).

A short example might make this a little more clear. If you were wrapping a function like this:

```
int gcd(int x, int y);
```

A wrapper function would look approximately like this:

```
PyObject *wrap_gcd(PyObject *self, PyObject *args) {
  int arg1;
  int arg2;
  int result;
  PyObject *obj1;
  PyObject *obj2;
  PyObject *resultobj;

  if (!PyArg_ParseTuple("OO:gcd", &obj1, &obj2)) return NULL;

  /* "in" typemap, argument 1 */
  {
    arg1 = PyInt_AsLong(obj1);
  }

  /* "in" typemap, argument 2 */
  {
    arg2 = PyInt_AsLong(obj2);
  }

  result = gcd(arg1, arg2);

  /* "out" typemap, return value */
  {
    resultobj = PyInt_FromLong(result);
  }

  return resultobj;
}
```

In this code, you can see how the typemap code has been inserted into the function. You can also see how the special $ variables have been expanded to match certain variable names inside the wrapper function. This is really the whole idea behind typemaps--they simply let you insert arbitrary code into different parts of the generated wrapper functions. Because arbitrary code can be inserted, it possible to completely change the way in which values are converted.

### 14.1.3 Pattern matching

As the name implies, the purpose of a typemap is to "map" C datatypes to types in the target language. Once a typemap is defined for a C datatype, it is applied to all future occurrences of that type in the input file. For example:

```
/* Convert from Perl --> C */
%typemap(in) int {
  $1 = SvIV($input);
}

...
int factorial(int n);
int gcd(int x, int y);
int count(char *s, char *t, int max);
```

The matching of typemaps to C datatypes is more than a simple textual match. In fact, typemaps are fully built into the underlying type system. Therefore, typemaps are unaffected by `typedef`, namespaces, and other declarations that might hide the underlying type. For example, you could have code like this:

```
/* Convert from Ruby--> C */
%typemap(in) int {
  $1 = NUM2INT($input);
}
...
typedef int Integer;
namespace foo {
  typedef Integer Number;
};

int foo(int x);
```

```
int bar(Integer y);
int spam(foo::Number a, foo::Number b);
```

In this case, the typemap is still applied to the proper arguments even though typenames don't always match the text "int". This ability to track types is a critical part of SWIG--in fact, all of the target language modules work merely define a family of typemaps for the basic types. Yet, it is never necessary to write new typemaps for typenames introduced by `typedef`.

In addition to tracking typenames, typemaps may also be specialized to match against a specific argument name. For example, you could write a typemap like this:

```
%typemap(in) double nonnegative {
  $1 = PyFloat_AsDouble($input);
  if ($1 < 0) {
    PyErr_SetString(PyExc_ValueError, "argument must be nonnegative.");
    SWIG_fail;
  }
}

...
double sin(double x);
double cos(double x);
double sqrt(double nonnegative);

typedef double Real;
double log(Real nonnegative);
...
```

For certain tasks such as input argument conversion, typemaps can be defined for sequences of consecutive arguments. For example:

```
%typemap(in) (char *str, int len) {
  $1 = PyString_AsString($input);   /* char *str */
  $2 = PyString_Size($input);       /* int len   */
}
...
int count(char *str, int len, char c);
```

In this case, a single input object is expanded into a pair of C arguments. This example also provides a hint to the unusual variable naming scheme involving `$1`, `$2`, and so forth.

### 14.1.4 Reusing typemaps

Typemaps are normally defined for specific type and argument name patterns. However, typemaps can also be copied and reused. One way to do this is to use assignment like this:

```
%typemap(in) Integer = int;
%typemap(in) (char *buffer, int size) = (char *str, int len);
```

There is a more powerful way to copy a family of typemaps though. Consider the following family of two typemap methods, "in" and "out" for type `int`:

```
%typemap(in) int {
  /* Convert an integer argument */
  ...
}
%typemap(out) int {
  /* Return an integer value */
  ...
}
```

Each of the two typemap methods could be copied individually for type `size_t` as follows:

```
/* Apply all of the int typemaps to size_t */
%typemap(in) size_t = int;
%typemap(out) size_t = int;
```

A more powerful form of copying is available from the `%apply` directive. The code below is identical to the above:

```
/* Apply all of the int typemaps to size_t */
%apply int { size_t };
```

`%apply` merely takes *all* of the typemaps that are defined for one type and applies them to other types. Note: you can include a comma separated set of types in the `{ ... }` part of `%apply`.

It should be noted that it is not necessary to copy typemaps for types that are related by `typedef`. For example, if you have this,

```
typedef int size_t;
```

then SWIG already knows that the `int` typemaps apply. You don't have to do anything.

### 14.1.5 What can be done with typemaps?

The primary use of typemaps is for defining wrapper generation behavior at the level of individual C/C++ datatypes. There are currently six general categories of problems that typemaps address:

**Argument handling**

```
int foo(int x, double y, char *s);
```

- Input argument conversion ("in" typemap).
- Input argument type checking for types used in overloaded methods ("typecheck" typemap).
- Output argument handling ("argout" typemap).
- Input argument value checking ("check" typemap).
- Input argument initialization ("arginit" typemap).

- Default arguments ("default" typemap).
- Input argument resource management ("freearg" typemap).

**Return value handling**

```
int foo(int x, double y, char *s);
```

- Function return value conversion ("out" typemap).
- Return value resource management ("ret" typemap).
- Resource management for newly allocated objects ("newfree" typemap).

**Exception handling**

```
int foo(int x, double y, char *s) throw(MemoryError, IndexError);
```

- Handling of C++ exception specifications. ("throw" typemap).

**Global variables**

```
int foo;
```

- Assignment of a global variable. ("varin" typemap).
- Reading a global variable. ("varout" typemap).

**Member variables**

```
struct Foo {
  int x[20];
};
```

- Assignment of data to a class/structure member. ("memberin" typemap).

**Constant creation**

```
#define FOO 3
%constant int BAR = 42;
enum { ALE, LAGER, STOUT };
```

- Creation of constant values. ("consttab" or "constcode" typemap).

Details of each of these typemaps will be covered shortly. Also, certain language modules may define additional typemaps that expand upon this list. For example, the Java module defines a variety of typemaps for controlling additional aspects of the Java bindings. Consult language specific documentation for further details.

### 14.1.6 What can't be done with typemaps?

Typemaps can't be used to define properties that apply to C/C++ declarations as a whole. For example, suppose you had a declaration like this,

```
Foo *make_Foo(int n);
```

and you wanted to tell SWIG that `make_Foo(int n)` returned a newly allocated object (for the purposes of providing better memory management). Clearly, this property of `make_Foo(int n)` is *not* a property that would be associated with the datatype`Foo *` by itself. Therefore, a completely different SWIG customization mechanism (`%feature`) is used for this purpose. Consult the Customization Features chapter for more information about that.

Typemaps also can't be used to rearrange or transform the order of arguments. For example, if you had a function like this:

```
void foo(int, char *);
```

you can't use typemaps to interchange the arguments, allowing you to call the function like this:

```
foo("hello", 3)          # Reversed arguments
```

If you want to change the calling conventions of a function, write a helper function instead. For example:

```
%rename(foo) wrap_foo;
%inline %{
void wrap_foo(char *s, int x) {
  foo(x, s);
}
%}
```

### 14.1.7 Similarities to Aspect Oriented Programming

SWIG has parallels to Aspect Oriented Software Development (AOP). The AOP terminology with respect to SWIG typemaps can be viewed as follows:

- **Cross-cutting concerns**: The cross-cutting concerns are the modularization of the functionality that the typemaps implement, which is primarily marshalling of types from/to the target language and C/C++.
- **Advice**: The typemap body contains code which is executed whenever the marshalling is required.
- **Pointcut**: The pointcuts are the positions in the wrapper code that the typemap code is generated into.
- **Aspect**: Aspects are the combination of the pointcut and the advice, hence each typemap is an aspect.

SWIG can also be viewed as has having a second set of aspects based around %feature. Features such as `%exception` are also cross-cutting concerns as they encapsulate code that can be used to add logging or exception handling to any function.

### 14.1.8 The rest of this chapter

The rest of this chapter provides detailed information for people who want to write new typemaps. This information is of particular importance to anyone who intends to write a new SWIG target language module. Power users can also use this information to write application specific type conversion rules.

Since typemaps are strongly tied to the underlying C++ type system, subsequent sections assume that you are reasonably familiar with the basic details of values, pointers, references, arrays, type qualifiers (e.g., `const`), structures, namespaces, templates, and memory management in C/C++. If not, you would be well-advised to consult a copy of "The C Programming Language" by Kernighan and Ritchie or "The C++ Programming Language" by Stroustrup before going any further.

## 14.2 Typemap specifications

This section describes the behavior of the `%typemap` directive itself.

### 14.2.1 Defining a typemap

New typemaps are defined using the `%typemap` declaration. The general form of this declaration is as follows (parts enclosed in [ ... ] are optional):

```
%typemap(method [, modifiers]) typelist code ;
```

*method* is a simply a name that specifies what kind of typemap is being defined. It is usually a name like `"in"`, `"out"`, or `"argout"`. The purpose of these methods is described later.

*modifiers* is an optional comma separated list of`name="value"` values. These are sometimes to attach extra information to a typemap and is often target-language dependent. They are also known as typemap attributes.

*typelist* is a list of the C++ type patterns that the typemap will match. The general form of this list is as follows:

```
typelist    :  typepattern [, typepattern, typepattern, ... ] ;

typepattern :  type [ (parms) ]
            |  type name [ (parms) ]
            |  ( typelist ) [ (parms) ]
```

Each type pattern is either a simple type, a simple type and argument name, or a list of types in the case of multi-argument typemaps. In addition, each type pattern can be parameterized with a list of temporary variables (parms). The purpose of these variables will be explained shortly.

*code* specifies the code used in the typemap. Usually this is C/C++ code, but in the statically typed target languages, such as Java and C#, this can contain target language code for certain typemaps. It can take any one of the following forms:

```
code       : { ... }
           | " ... "
           | %{ ... %}
```

Note that the preprocessor will expand code within the {} delimiters, but not in the last two styles of delimiters, see Preprocessor and Typemaps. Here are some examples of valid typemap specifications:

```
/* Simple typemap declarations */
%typemap(in) int {
  $1 = PyInt_AsLong($input);
}
%typemap(in) int "$1 = PyInt_AsLong($input);"
%typemap(in) int %{
  $1 = PyInt_AsLong($input);
%}

/* Typemap with extra argument name */
%typemap(in) int nonnegative {
  ...
}

/* Multiple types in one typemap */
%typemap(in) int, short, long {
  $1 = SvIV($input);
}

/* Typemap with modifiers */
%typemap(in, doc="integer") int "$1 = scm_to_int($input);"

/* Typemap applied to patterns of multiple arguments */
%typemap(in) (char *str, int len),
             (char *buffer, int size)
{
  $1 = PyString_AsString($input);
  $2 = PyString_Size($input);
}

/* Typemap with extra pattern parameters */
%typemap(in, numinputs=0) int *output (int temp),
                          long *output (long temp)
{
  $1 = &temp;
}
```

Admittedly, it's not the most readable syntax at first glance. However, the purpose of the individual pieces will become clear.

### 14.2.2 Typemap scope

Once defined, a typemap remains in effect for all of the declarations that follow. A typemap may be redefined for different sections of an input file. For example:

```
// typemap1
%typemap(in) int {
...
}

int fact(int);                    // typemap1
int gcd(int x, int y);            // typemap1

// typemap2
```

```
%typemap(in) int {
...
}

int isprime(int);                // typemap2
```

One exception to the typemap scoping rules pertains to the `%extend` declaration. `%extend` is used to attach new declarations to a class or structure definition. Because of this, all of the declarations in an `%extend` block are subject to the typemap rules that are in effect at the point where the class itself is defined. For example:

```
class Foo {
  ...
};

%typemap(in) int {
 ...
}

%extend Foo {
  int blah(int x);    // typemap has no effect.  Declaration is attached to Foo which
                      // appears before the %typemap declaration.
};
```

### 14.2.3 Copying a typemap

A typemap is copied by using assignment. For example:

```
%typemap(in) Integer = int;
```

or this:

```
%typemap(in) Integer, Number, int32_t = int;
```

Types are often managed by a collection of different typemaps. For example:

```
%typemap(in)     int { ... }
%typemap(out)    int { ... }
%typemap(varin)  int { ... }
%typemap(varout) int { ... }
```

To copy all of these typemaps to a new type, use `%apply`. For example:

```
%apply int { Integer };            // Copy all int typemaps to Integer
%apply int { Integer, Number };    // Copy all int typemaps to both Integer and Number
```

The patterns for `%apply` follow the same rules as for `%typemap`. For example:

```
%apply int *output { Integer *output };                    // Typemap with name
%apply (char *buf, int len) { (char *buffer, int size) };  // Multiple arguments
```

### 14.2.4 Deleting a typemap

A particular typemap can be deleted / cleared by simply defining no code. For example:

```
%typemap(in) int;                // Clears the "in" typemap for int
%typemap(in) int, long, short;   // Clears the "in" typemap for int, long, short
%typemap(in) int *output;
```

The above syntax deletes a typemap for just one typemap method - the "in" method in each of the examples above. The `%clear` directive is more powerful and will delete / clear a family of typemaps, that is, all the typemap methods for a given type. For example:

```
%clear int;                      // Delete all typemaps ("in", "out", "varin", ...) for int
%clear int *output, long *output;
```

**Note:** Since SWIG's default behavior is defined by typemaps, clearing a fundamental type like `int` will make that type unusable unless you also define a new family of typemaps immediately after the clear operation.

### 14.2.5 Placement of typemaps

Typemap declarations can be declared in the global scope, within a C++ namespace, and within a C++ class. For example:

```
%typemap(in) int {
  ...
}

namespace std {
  class string;
  %typemap(in) string {
    ...
  }
}

class Bar {
public:
  typedef const int & const_reference;
  %typemap(out) const_reference {
    ...
```

```
  }
};
```

When a typemap appears inside a namespace or class, it stays in effect until the end of the SWIG input (just like before). However, the typemap takes the local scope into account. Therefore, this code

```
namespace std {
  class string;
  %typemap(in) string {
    ...
  }
}
```

is really defining a typemap for the type `std::string`. You could have code like this:

```
namespace std {
  class string;
  %typemap(in) string {          /* std::string */
    ...
  }
}

namespace Foo {
  class string;
  %typemap(in) string {          /* Foo::string */
    ...
  }
}
```

In this case, there are two completely distinct typemaps that apply to two completely different types (`std::string` and `Foo::string`).

It should be noted that for scoping to work, SWIG has to know that `string` is a typename defined within a particular namespace. In this example, this is done using the forward class declaration `class string`.

## 14.3 Pattern matching rules

The section describes the pattern matching rules by which C/C++ datatypes are associated with typemaps. The matching rules can be observed in practice by using the debugging options also described.

### 14.3.1 Basic matching rules

Typemaps are matched using both a type and a name (typically the name of an argument, but in the case of `out` typemaps, the name of a function, qualified by the class name if it's a class method). For a given `TYPE NAME` pair, the following rules are applied, in order, to find a match. The first typemap found is used.

- Typemaps that exactly match `TYPE` and `NAME`.
- Typemaps that exactly match `TYPE` only.
- If `TYPE` is a C++ template of type`T< TPARMS >`, where `TPARMS` are the template parameters, the type is stripped of the template parameters and the following checks are then made:
    - Typemaps that exactly match `T` and `NAME`.
    - Typemaps that exactly match `T` only.

If `TYPE` includes qualifiers (const, volatile, etc.), each qualifier is stripped one at a time to form a new stripped type and the matching rules above are repeated on the stripped type. The left-most qualifier is stripped first, resulting in the right-most (or top-level) qualifier being stripped last. For example `int const*const` is first stripped to `int *const` then `int *`.

If `TYPE` is an array. The following transformation is made:

- Replace all dimensions to `[ANY]` and look for a generic array typemap.

To illustrate, suppose that you had a function like this:

```
int foo(const char *s);
```

To find a typemap for the argument `const char *s`, SWIG will search for the following typemaps:

```
const char *s           Exact type and name match
const char *            Exact type match
char *s                 Type and name match (qualifier stripped)
char *                  Type match (qualifier stripped)
```

When more than one typemap rule might be defined, only the first match found is actually used. Here is an example that shows how some of the basic rules are applied:

```
%typemap(in) int *x {
  ... typemap 1
}

%typemap(in) int * {
  ... typemap 2
}

%typemap(in) const int *z {
  ... typemap 3
}

%typemap(in) int [4] {
  ... typemap 4
}

%typemap(in) int [ANY] {
  ... typemap 5
}

void A(int *x);        // int *x rule       (typemap 1)
void B(int *y);        // int * rule        (typemap 2)
```

```
void C(const int *x);  // int *x rule       (typemap 1)
void D(const int *z);  // const int *z rule (typemap 3)
void E(int x[4]);      // int [4] rule      (typemap 4)
void F(int x[1000]);   // int [ANY] rule    (typemap 5)
```

**Compatibility note:** SWIG-2.0.0 introduced stripping the qualifiers one step at a time. Prior versions stripped all qualifiers in one step.

### 14.3.2 Typedef reductions matching

If no match is found using the rules in the previous section, SWIG applies a typedef reduction to the type and repeats the typemap search for the reduced type. To illustrate, suppose you had code like this:

```
%typemap(in) int {
  ... typemap 1
}

typedef int Integer;
void blah(Integer x);
```

To find the typemap for `Integer x`, SWIG will first search for the following typemaps:

```
Integer x
Integer
```

Finding no match, it then applies a reduction `Integer -> int` to the type and repeats the search.

```
int x
int      --> match: typemap 1
```

Even though two types might be the same via typedef, SWIG allows typemaps to be defined for each typename independently. This allows for interesting customization possibilities based solely on the typename itself. For example, you could write code like this:

```
typedef double  pdouble;     // Positive double

// typemap 1
%typemap(in) double {
  ... get a double ...
}
// typemap 2
%typemap(in) pdouble {
  ... get a positive double ...
}
double sin(double x);           // typemap 1
pdouble sqrt(pdouble x);        // typemap 2
```

When reducing the type, only one typedef reduction is applied at a time. The search process continues to apply reductions until a match is found or until no more reductions can be made.

For complicated types, the reduction process can generate a long list of patterns. Consider the following:

```
typedef int Integer;
typedef Integer Row4[4];
void foo(Row4 rows[10]);
```

To find a match for the `Row4 rows[10]` argument, SWIG would check the following patterns, stopping only when it found a match:

```
Row4 rows[10]
Row4 [10]
Row4 rows[ANY]
Row4 [ANY]

# Reduce Row4 --> Integer[4]
Integer rows[10][4]
Integer [10][4]
Integer rows[ANY][ANY]
Integer [ANY][ANY]

# Reduce Integer --> int
int rows[10][4]
int [10][4]
int rows[ANY][ANY]
int [ANY][ANY]
```

For parameterized types like templates, the situation is even more complicated. Suppose you had some declarations like this:

```
typedef int Integer;
typedef foo<Integer, Integer> fooii;
void blah(fooii *x);
```

In this case, the following typemap patterns are searched for the argument `fooii *x`:

```
fooii *x
fooii *

# Reduce fooii --> foo<Integer, Integer>
foo<Integer, Integer> *x
foo<Integer, Integer> *

# Reduce Integer -> int
```

```
foo<int, Integer> *x
foo<int, Integer> *

# Reduce Integer -> int
foo<int, int> *x
foo<int, int> *
```

Typemap reductions are always applied to the left-most type that appears. Only when no reductions can be made to the left-most type are reductions made to other parts of the type. This behavior means that you could define a typemap for `foo<int, Integer>`, but a typemap for `foo<Integer, int>` would never be matched. Admittedly, this is rather esoteric--there's little practical reason to write a typemap quite like that. Of course, you could rely on this to confuse your coworkers even more.

As a point of clarification, it is worth emphasizing that typedef matching is a typedef **reduction** process only, that is, SWIG does not search for every single possible typedef. Given a type in a declaration, it will only reduce the type, it won't build it up looking for typedefs. For example, given the type `Struct`, the typemap below will not be used for the `aStruct` parameter, because `Struct` is fully reduced:

```
struct Struct {...};
typedef Struct StructTypedef;

%typemap(in) StructTypedef {
  ...
}

void go(Struct aStruct);
```

### 14.3.3 Default typemap matching rules

If the basic pattern matching rules result in no match being made, even after typedef reductions, the default typemap matching rules are used to look for a suitable typemap match. These rules match a generic typemap based on the reserved `SWIGTYPE` base type. For example pointers will use `SWIGTYPE *` and references will use `SWIGTYPE &`. More precisely, the rules are based on the C++ class template partial specialization matching rules used by C++ compilers when looking for an appropriate partial template specialization. This means that a match is chosen from the most specialized set of generic typemap types available. For example, when looking for a match to `int const *`, the rules will prefer to match `SWIGTYPE const *` if available before matching `SWIGTYPE *`, before matching `SWIGTYPE`.

Most SWIG language modules use typemaps to define the default behavior of the C primitive types. This is entirely straightforward. For example, a set of typemaps for primitives marshalled by value or const reference are written like this:

```
%typemap(in) int           "... convert to int ..."
%typemap(in) short         "... convert to short ..."
%typemap(in) float         "... convert to float ..."
...
%typemap(in) const int &   "... convert ..."
%typemap(in) const short & "... convert ..."
%typemap(in) const float & "... convert ..."
...
```

Since typemap matching follows all `typedef` declarations, any sort of type that is mapped to a primitive type by value or const reference through `typedef` will be picked up by one of these primitive typemaps. Most language modules also define typemaps for char pointers and char arrays to handle strings, so these non-default types will also be used in preference as the basic typemap matching rules provide a better match than the default typemap matching rules.

Below is a list of the typical default types supplied by language modules, showing what the "in" typemap would look like:

```
%typemap(in) SWIGTYPE &            { ... default reference handling ...                     }
%typemap(in) SWIGTYPE *            { ... default pointer handling ...                       }
%typemap(in) SWIGTYPE *const       { ... default pointer const handling ...                 }
%typemap(in) SWIGTYPE *const&      { ... default pointer const reference handling ...       }
%typemap(in) SWIGTYPE[ANY]         { ... 1D fixed size arrays handling ...                  }
%typemap(in) SWIGTYPE []           { ... unknown sized array handling ...                   }
%typemap(in) enum SWIGTYPE         { ... default handling for enum values ...               }
%typemap(in) const enum SWIGTYPE & { ... default handling for const enum reference values ... }
%typemap(in) SWIGTYPE (CLASS::*)   { ... default pointer member handling ...                }
%typemap(in) SWIGTYPE              { ... simple default handling ...                        }
```

If you wanted to change SWIG's default handling for simple pointers, you would simply redefine the rule for `SWIGTYPE *`. Note, the simple default typemap rule is used to match against simple types that don't match any other rules:

```
%typemap(in) SWIGTYPE              { ... simple default handling ...                        }
```

This typemap is important because it is the rule that gets triggered when call or return by value is used. For instance, if you have a declaration like this:

```
double dot_product(Vector a, Vector b);
```

The `Vector` type will usually just get matched against `SWIGTYPE`. The default implementation of `SWIGTYPE` is to convert the value into pointers (as described in this earlier section).

By redefining `SWIGTYPE` it may be possible to implement other behavior. For example, if you cleared all typemaps for `SWIGTYPE`, SWIG simply won't wrap any unknown datatype (which might be useful for debugging). Alternatively, you might modify SWIGTYPE to marshal objects into strings instead of converting them to pointers.

Let's consider an example where the following typemaps are defined and SWIG is looking for the best match for the enum shown below:

```
%typemap(in) const Hello &          { ... }
%typemap(in) const enum SWIGTYPE &  { ... }
%typemap(in) enum SWIGTYPE &        { ... }
%typemap(in) SWIGTYPE &             { ... }
%typemap(in) SWIGTYPE               { ... }

enum Hello {};
const Hello &hi;
```

The typemap at the top of the list will be chosen, not because it is defined first, but because it is the closest match for the type being wrapped. If any of the typemaps in the above list were not defined, then the next one on the list would have precedence.

The best way to explore the default typemaps is to look at the ones already defined for a particular language module. Typemap definitions are usually found in the SWIG library in a file such as `java.swg` , `csharp.swg` etc. However, for many of the target languages the typemaps are hidden behind complicated macros, so the best way to view the default typemaps, or any

typemaps for that matter, is to look at the preprocessed output by running `swig -E` on any interface file. Finally the best way to view the typemap matching rules in action is via the debugging typemap pattern matching options covered later on.

**Compatibility note:** The default typemap matching rules were modified in SWIG-2.0.0 from a slightly simpler scheme to match the current C++ class template partial specialization matching rules.

### 14.3.4 Multi-arguments typemaps

When multi-argument typemaps are specified, they take precedence over any typemaps specified for a single type. For example:

```
%typemap(in) (char *buffer, int len) {
  // typemap 1
}

%typemap(in) char *buffer {
  // typemap 2
}

void foo(char *buffer, int len, int count); // (char *buffer, int len)
void bar(char *buffer, int blah);           // char *buffer
```

Multi-argument typemaps are also more restrictive in the way that they are matched. Currently, the first argument follows the matching rules described in the previous section, but all subsequent arguments must match exactly.

### 14.3.5 Matching rules compared to C++ templates

For those intimately familiar with C++ templates, a comparison of the typemap matching rules and template type deduction is interesting. The two areas considered are firstly the default typemaps and their similarities to partial template specialization and secondly, non-default typemaps and their similarities to full template specialization.

For default (SWIGTYPE) typemaps the rules are inspired by C++ class template partial specialization. For example, given partial specialization for `T const&` :

```
template <typename T> struct X             { void a(); };
template <typename T> struct X< T const& > { void b(); };
```

The full (unspecialized) template is matched with most types, such as:

```
X< int & >            x1;  x1.a();
```

and the following all match the `T const&` partial specialization:

```
X< int *const& >      x2;  x2.b();
X< int const*const& > x3;  x3.b();
X< int const& >       x4;  x4.b();
```

Now, given just these two default typemaps, where T is analogous to SWIGTYPE:

```
%typemap(...) SWIGTYPE        { ... }
%typemap(...) SWIGTYPE const& { ... }
```

The generic default typemap `SWIGTYPE` is used with most types, such as

```
int &
```

and the following all match the `SWIGTYPE const&` typemap, just like the partial template matching:

```
int *const&
int const*const&
int const&
```

Note that the template and typemap matching rules are not identical for all default typemaps though, for example, with arrays.

For non-default typemaps, one might expect SWIG to follow the fully specialized template rules. This is nearly the case, but not quite. Consider a very similar example to the earlier partially specialized template but this time there is a fully specialized template:

```
template <typename T> struct Y       { void a(); };
template <> struct Y< int const & >  { void b(); };
```

Only the one type matches the specialized template exactly:

```
Y< int & >             y1;  y1.a();
Y< int *const& >       y2;  y2.a();
Y< int const *const& > y3;  y3.a();
Y< int const& >        y4;  y4.b(); // fully specialized match
```

Given typemaps with the same types used for the template declared above, where T is again analogous to SWIGTYPE:

```
%typemap(...) SWIGTYPE        { ... }
%typemap(...) int const&      { ... }
```

The comparison between non-default typemaps and fully specialized single parameter templates turns out to be the same, as just the one type will match the non-default typemap:

```
int &
int *const&
int const*const&
```

```
int const&        // matches non-default typemap int const&
```

However, if a non-const type is used instead:

```
%typemap(...) SWIGTYPE        { ... }
%typemap(...) int &           { ... }
```

then there is a clear difference to template matching as both the const and non-const types match the typemap:

```
int &             // matches non-default typemap int &
int *const&
int const*const&
int const&        // matches non-default typemap int &
```

There are other subtle differences such as typedef handling, but at least it should be clear that the typemap matching rules are similar to those for specialized template handling.

### 14.3.6 Debugging typemap pattern matching

There are two useful debug command line options available for debugging typemaps, `-debug-tmsearch` and `-debug-tmused` .

The `-debug-tmsearch` option is a verbose option for debugging typemap searches. This can be very useful for watching the pattern matching process in action and for debugging which typemaps are used. The option displays all the typemaps and types that are looked for until a successful pattern match is made. As the display includes searches for each and every type needed for wrapping, the amount of information displayed can be large. Normally you would manually search through the displayed information for the particular type that you are interested in.

For example, consider some of the code used in the Typedef reductions section already covered:

```
typedef int Integer;
typedef Integer Row4[4];
void foo(Row4 rows[10]);
```

A sample of the debugging output is shown below for the "in" typemap:

```
swig -perl -debug-tmsearch example.i
...
example.h:3: Searching for a suitable 'in' typemap for: Row4 rows[10]
  Looking for: Row4 rows[10]
  Looking for: Row4 [10]
  Looking for: Row4 rows[ANY]
  Looking for: Row4 [ANY]
  Looking for: Integer rows[10][4]
  Looking for: Integer [10][4]
  Looking for: Integer rows[ANY][ANY]
  Looking for: Integer [ANY][ANY]
  Looking for: int rows[10][4]
  Looking for: int [10][4]
  Looking for: int rows[ANY][ANY]
  Looking for: int [ANY][ANY]
  Looking for: SWIGTYPE rows[ANY][ANY]
  Looking for: SWIGTYPE [ANY][ANY]
  Looking for: SWIGTYPE rows[ANY][]
  Looking for: SWIGTYPE [ANY][]
  Looking for: SWIGTYPE *rows[ANY]
  Looking for: SWIGTYPE *[ANY]
  Looking for: SWIGTYPE rows[ANY]
  Looking for: SWIGTYPE [ANY]
  Looking for: SWIGTYPE rows[]
  Looking for: SWIGTYPE []
  Using: %typemap(in) SWIGTYPE []
...
```

showing that the best default match supplied by SWIG is the `SWIGTYPE []` typemap. As the example shows, the successful match displays the used typemap source including typemap method, type and optional name in one of these simplified formats:

- `Using: %typemap(method) type name`
- `Using: %typemap(method) type name = type2 name2`
- `Using: %apply type2 name2 { type name }`

This information might meet your debugging needs, however, you might want to analyze further. If you next invoke SWIG with the `-E` option to display the preprocessed output, and search for the particular typemap used, you'll find the full typemap contents (example shown below for Python):

```
%typemap(in, noblock=1) SWIGTYPE [] (void *argp = 0, int res = 0) {
  res = SWIG_ConvertPtr($input, &argp, $descriptor, $disown |  0 );
  if (!SWIG_IsOK(res)) {
    SWIG_exception_fail(SWIG_ArgError(res), "in method '" "$symname" "', argument "
                        "$argnum"" of type '" "$type""'");
  }
  $1 = ($ltype)(argp);
}
```

The generated code for the `foo` wrapper will then contain the snippets of the typemap with the special variables expanded. The rest of this chapter will need reading though to fully understand all of this, however, the relevant parts of the generated code for the above typemap can be seen below:

```
SWIGINTERN PyObject *_wrap_foo(PyObject *SWIGUNUSEDPARM(self), PyObject *args) {
...
  void *argp1 = 0 ;
  int res1 = 0 ;
...
  res1 = SWIG_ConvertPtr(obj0, &argp1, SWIGTYPE_p_a_4__int, 0 |  0 );
  if (!SWIG_IsOK(res1)) {
    SWIG_exception_fail(SWIG_ArgError(res1), "in method '" "foo" "', argument "
                        "1"" of type '" "int [10][4]""'");
  }
```

```
  arg1 = (int (*)[4])(argp1);
...
}
```

Searches for multi-argument typemaps are not mentioned unless a matching multi-argument typemap does actually exist. For example, the output for the code in the earlier multi-arguments section is as follows:

```
...
example.h:39: Searching for a suitable 'in' typemap for: char *buffer
  Looking for: char *buffer
  Multi-argument typemap found...
  Using: %typemap(in) (char *buffer, int len)
...
```

The second option for debugging is `-debug-tmused` and this displays the typemaps used. This option is a less verbose version of the `-debug-tmsearch` option as it only displays each successfully found typemap on a separate single line. The output displays the type, and name if present, the typemap method in brackets and then the actual typemap used in the same simplified format output by the `-debug-tmsearch` option. Below is the output for the example code at the start of this section on debugging.

```
$ swig -perl -debug-tmused example.i
example.h:3: Typemap for Row4 rows[10] (in) : %typemap(in) SWIGTYPE []
example.h:3: Typemap for Row4 rows[10] (typecheck) : %typemap(typecheck) SWIGTYPE *
example.h:3: Typemap for Row4 rows[10] (freearg) : %typemap(freearg) SWIGTYPE []
example.h:3: Typemap for void foo (out) : %typemap(out) void
```

Now, consider the following interface file:

```
%module example

%{
void set_value(const char* val) {}
%}

%typemap(check) char *NON_NULL {
  if (!$1) {
    /* ... error handling ... */
  }
}

// use default pointer handling instead of strings
%apply SWIGTYPE * { const char* val, const char* another_value }

%typemap(check) const char* val = char* NON_NULL;

%typemap(arginit, noblock=1) const char* val {
  $1 = "";
}

void set_value(const char* val);
```

and the output debug:

```
swig -perl5 -debug-tmused example.i
example.i:21: Typemap for char const *val (arginit) : %typemap(arginit) char const *val
example.i:21: Typemap for char const *val (in) : %apply SWIGTYPE * { char const *val }
example.i:21: Typemap for char const *val (typecheck) : %apply SWIGTYPE * { char const *val }
example.i:21: Typemap for char const *val (check) : %typemap(check) char const *val = char *NON_NULL
example.i:21: Typemap for char const *val (freearg) : %apply SWIGTYPE * { char const *val }
example.i:21: Typemap for void set_value (out) : %typemap(out) void
```

The following observations about what is displayed can be noted (the same applies for `-debug-tmsearch`):

- The relevant typemap is shown, but for typemap copying, the appropriate `%typemap` or `%apply` is displayed, for example, the "check" and "in" typemaps.
- The typemap modifiers are not shown, eg the `noblock=1` modifier in the "arginit" typemap.
- The exact `%apply` statement might look different to what is in the actual code. For example, the `const char* another_value` is not shown as it is not relevant here. Also the types may be displayed slightly differently - `char const *` and not `const char*`.

## 14.4 Code generation rules

This section describes rules by which typemap code is inserted into the generated wrapper code.

### 14.4.1 Scope

When a typemap is defined like this:

```
%typemap(in) int {
  $1 = PyInt_AsLong($input);
}
```

the typemap code is inserted into the wrapper function using a new block scope. In other words, the wrapper code will look like this:

```
wrap_whatever() {
  ...
  // Typemap code
  {
    arg1 = PyInt_AsLong(obj1);
  }
  ...
}
```

Because the typemap code is enclosed in its own block, it is legal to declare temporary variables for use during typemap execution. For example:

```
%typemap(in) short {
  long temp;          /* Temporary value */
  if (Tcl_GetLongFromObj(interp, $input, &temp) != TCL_OK) {
    return TCL_ERROR;
  }
  $1 = (short) temp;
}
```

Of course, any variables that you declare inside a typemap are destroyed as soon as the typemap code has executed (they are not visible to other parts of the wrapper function or other typemaps that might use the same variable names).

Occasionally, typemap code will be specified using a few alternative forms. For example:

```
%typemap(in) int "$1 = PyInt_AsLong($input);"
%typemap(in) int %{
$1 = PyInt_AsLong($input);
%}
%typemap(in, noblock=1) int {
$1 = PyInt_AsLong($input);
}
```

These three forms are mainly used for cosmetics--the specified code is not enclosed inside a block scope when it is emitted. This sometimes results in a less complicated looking wrapper function. Note that only the third of the three typemaps have the typemap code passed through the SWIG preprocessor.

### 14.4.2 Declaring new local variables

Sometimes it is useful to declare a new local variable that exists within the scope of the entire wrapper function. A good example of this might be an application in which you wanted to marshal strings. Suppose you had a C++ function like this

```
int foo(std::string *s);
```

and you wanted to pass a native string in the target language as an argument. For instance, in Perl, you wanted the function to work like this:

```
$x = foo("Hello World");
```

To do this, you can't just pass a raw Perl string as the `std::string *` argument. Instead, you have to create a temporary`std::string` object, copy the Perl string data into it, and then pass a pointer to the object. To do this, simply specify the typemap with an extra parameter like this:

```
%typemap(in) std::string * (std::string temp) {
  unsigned int len;
  char        *s;
  s = SvPV($input, len);         /* Extract string data */
  temp.assign(s, len);           /* Assign to temp */
  $1 = &temp;                   /* Set argument to point to temp */
}
```

In this case, `temp` becomes a local variable in the scope of the entire wrapper function. For example:

```
wrap_foo() {
  std::string temp;    <--- Declaration of temp goes here
  ...

  /* Typemap code */
  {
    ...
    temp.assign(s, len);
    ...
  }
  ...
}
```

When you set `temp` to a value, it persists for the duration of the wrapper function and gets cleaned up automatically on exit.

It is perfectly safe to use more than one typemap involving local variables in the same declaration. For example, you could declare a function as :

```
void foo(std::string *x, std::string *y, std::string *z);
```

This is safely handled because SWIG actually renames all local variable references by appending an argument number suffix. Therefore, the generated code would actually look like this:

```
wrap_foo() {
  int *arg1;    /* Actual arguments */
  int *arg2;
  int *arg3;
  std::string temp1;    /* Locals declared in the typemap */
  std::string temp2;
  std::string temp3;
  ...
  {
    char *s;
    unsigned int len;
    ...
    temp1.assign(s, len);
    arg1 = *temp1;
  }
  {
    char *s;
    unsigned int len;
```

```
    ...
    temp2.assign(s, len);
    arg2 = &temp2;
  }
  {
    char *s;
    unsigned int len;
    ...
    temp3.assign(s, len);
    arg3 = &temp3;
  }
  ...
}
```

There is an exception: if the variable name starts with the `_global_` prefix, the argument number is not appended. Such variables can be used throughout the generated wrapper function. For example, the above typemap could be rewritten to use `_global_temp` instead of `temp` and the generated code would then contain a single `_global_temp` variable instead of `temp1`, `temp2` and `temp3`:

```
%typemap(in) std::string * (std::string _global_temp) {
 ... as above ...
}
```

Some typemaps do not recognize local variables (or they may simply not apply). At this time, only typemaps that apply to argument conversion support this (input typemaps such as the "in" typemap).

**Note:**

When declaring a typemap for multiple types, each type must have its own local variable declaration.

```
%typemap(in) const std::string *, std::string * (std::string temp) // NO!
// only std::string * has a local variable
// const std::string * does not (oops)
....

%typemap(in) const std::string * (std::string temp), std::string * (std::string temp) // Correct
....
```

### 14.4.3 Special variables

Special variables are prefixed with a `$` and are expanded by SWIG as part of the code generation process.

#### 14.4.3.1 Type related special variables

Within all typemaps, the following special variables are expanded. These are related in some way to a typemap's type. This is by no means a complete list as some target languages have additional special variables which are documented in the language specific chapters.

| Variable | Meaning |
|---|---|
| $*n* | A C local variable corresponding to type *n* in the typemap pattern. |
| $argnum | Argument number. Only available in typemaps related to argument conversion |
| $*n*_name | Argument name |
| $*n*_type | Real C datatype of type *n*. |
| $*n*_ltype | ltype of type *n* |
| $*n*_mangle | Mangled form of type *n*. For example `_p_Foo` |
| $*n*_descriptor | Type descriptor structure for type *n*. For example `SWIGTYPE_p_Foo`. This is primarily used when interacting with the run-time type checker (described later). |
| $**n*_type | Real C datatype of type *n* with one pointer removed. |
| $**n*_ltype | ltype of type *n* with one pointer removed. |
| $**n*_mangle | Mangled form of type *n* with one pointer removed. |
| $**n*_descriptor | Type descriptor structure for type *n* with one pointer removed. |
| $&*n*_type | Real C datatype of type *n* with one pointer added. |
| $&*n*_ltype | ltype of type *n* with one pointer added. |
| $&*n*_mangle | Mangled form of type *n* with one pointer added. |
| $&*n*_descriptor | Type descriptor structure for type *n* with one pointer added. |
| $*n*_basetype | Base typename with all pointers and qualifiers stripped. |

Within the table, $*n* refers to a specific type within the typemap specification. For example, if you write this

```
%typemap(in) int *INPUT {

}
```

then $1 refers to `int *INPUT`. If you have a typemap like this,

```
%typemap(in) (int argc, char *argv[]) {
  ...
}
```

then $1 refers to `int argc` and $2 refers to `char *argv[]`.

Substitutions related to types and names always fill in values from the actual code that was matched. This is useful when a typemap might match multiple C datatype. For example:

```
%typemap(in)  int, short, long {
  $1 = ($1_ltype) PyInt_AsLong($input);
}
```

In this case, `$1_ltype` is replaced with the datatype that is actually matched.

When typemap code is emitted, the C/C++ datatype of the special variables `$1` and `$2` is always an "ltype." An "ltype" is simply a type that can legally appear on the left-hand side of a C assignment operation. Here are a few examples of types and ltypes:

```
type              ltype
------            ----------------
int               int
const int         int
const int *       int *
int [4]           int *
int [4][5]        int (*)[5]
```

In most cases a ltype is simply the C datatype with qualifiers stripped off. In addition, arrays are converted into pointers.

Variables such as `$&1_type` and `$*1_type` are used to safely modify the type by removing or adding pointers. Although not needed in most typemaps, these substitutions are sometimes needed to properly work with typemaps that convert values between pointers and values.

If necessary, type related substitutions can also be used when declaring locals. For example:

```
%typemap(in) int * ($*1_type temp) {
  temp = PyInt_AsLong($input);
  $1 = &temp;
}
```

There is one word of caution about declaring local variables in this manner. If you declare a local variable using a type substitution such as `$1_ltype temp`, it won't work like you expect for arrays and certain kinds of pointers. For example, if you wrote this,

```
%typemap(in) int [10][20] {
  $1_ltype temp;
}
```

then the declaration of `temp` will be expanded as

```
int (*)[20] temp;
```

This is illegal C syntax and won't compile. There is currently no straightforward way to work around this problem in SWIG due to the way that typemap code is expanded and processed. However, one possible workaround is to simply pick an alternative type such as `void *` and use casts to get the correct type when needed. For example:

```
%typemap(in) int [10][20] {
  void *temp;
  ...
  (($1_ltype) temp)[i][j] = x;    /* set a value */
  ...
}
```

Another approach, which only works for arrays is to use the `$1_basetype` substitution. For example:

```
%typemap(in) int [10][20] {
  $1_basetype temp[10][20];
  ...
  temp[i][j] = x;    /* set a value */
  ...
}
```

#### 14.4.3.2 Non-type related special variables

The following are additional special variables. These are not specifically related to a typemap's type.

| Variable | Meaning |
|---|---|
| $*isvoid* | Expands to 1 if the wrapped function has a void return, otherwise expands to 0. Note that it expands to 1 in destructors and 0 in constructors. |
| $*symname* | Name of function/method being wrapped. |

### 14.4.4 Special variable macros

Special variable macros are like macro functions in that they take one or more input arguments which are used for the macro expansion. They look like macro/function calls but use the special variable `$` prefix to the macro name. Note that unlike normal macros, the expansion is not done by the preprocessor, it is done during the SWIG parsing/compilation stages. The following special variable macros are available across all language modules.

#### 14.4.4.1 $descriptor(type)

This macro expands into the type descriptor structure for any C/C++ type specified in `type`. It behaves like the `$1_descriptor` special variable described above except that the type to expand is taken from the macro argument rather than inferred from the typemap type. For example, `$descriptor(std::vector<int> *)` will expand into `SWIGTYPE_p_std__vectorT_int_t`. This macro is mostly used in the scripting target languages and is demonstrated later in the Run-time type checker usage section.

#### 14.4.4.2 $typemap(method, typepattern)

This macro uses the pattern matching rules described earlier to lookup and then substitute the special variable macro with the code in the matched typemap. The typemap to search for is specified by the arguments, where `method` is the typemap method name and `typepattern` is a type pattern as per the `%typemap` specification in the Defining a typemap section.

The special variables within the matched typemap are expanded into those for the matched typemap type, not the typemap within which the macro is called. In practice, there is little use for this macro in the scripting target languages. It is mostly used in the target languages that are statically typed as a way to obtain the target language type given the C/C++ type and more commonly only when the C++ type is a template parameter.

The example below is for C# only and uses some typemap method names documented in the C# chapter, but it shows some of the possible syntax variations.

```
%typemap(cstype) unsigned long    "uint"
%typemap(cstype) unsigned long bb "bool"
```

```
%typemap(cscode) BarClass %{
  void foo($typemap(cstype, unsigned long aa) var1,
           $typemap(cstype, unsigned long bb) var2,
           $typemap(cstype, (unsigned long bb)) var3,
           $typemap(cstype, unsigned long) var4)
  {
    // do something
  }
%}
```

The result is the following expansion

```
%typemap(cstype) unsigned long    "uint"
%typemap(cstype) unsigned long bb "bool"
%typemap(cscode) BarClass %{
  void foo(uint var1,
           bool var2,
           bool var3,
           uint var4)
  {
    // do something
  }
%}
```

#### 14.4.4.3 $typemap(method:attribute, typepattern)

An enhanced version of `$typemap` provides access to typemap attributes by appending a colon and the attribute name after the method name. In the example below, "cstype" is the typemap method and "out" is the typemap attribute.

```
%typemap(cstype, out="object") XClass "XClass"
%typemap(cscode) BarClass %{
  $typemap(cstype:out, XClass) bar()
  {
    return null;
  }
```

which expands to

```
  object bar()
  {
    return null;
  }
```

**Compatibility note:** Support for typemap attributes in `$typemap` was introduced in SWIG-4.1.0.

### 14.4.5 Special variables and typemap attributes

As of SWIG-3.0.7 typemap attributes will also expand special variables and special variable macros.

Example usage showing the expansion in the 'out' attribute (C# specific) as well as the main typemap body:

```
%typemap(ctype, out="$*1_ltype") unsigned int& "$*1_ltype"
```

is equivalent to the following as `$*1_ltype` expands to `unsigned int`:

```
%typemap(ctype, out="unsigned int") unsigned int& "unsigned int"
```

### 14.4.6 Special variables combined with special variable macros

Special variables can also be used within special variable macros. The special variables are expanded before they are used in the special variable macros.

Consider the following C# typemaps:

```
%typemap(cstype) unsigned int "uint"
%typemap(cstype, out="$typemap(cstype, $*1_ltype)") unsigned int& "$typemap(cstype, $*1_ltype)"
```

Special variables are expanded first and hence the above is equivalent to:

```
%typemap(cstype) unsigned int "uint"
%typemap(cstype, out="$typemap(cstype, unsigned int)") unsigned int& "$typemap(cstype, unsigned int)"
```

which then expands to:

```
%typemap(cstype) unsigned int "uint"
%typemap(cstype, out="uint") unsigned int& "uint"
```

## 14.5 Common typemap methods

The family of typemaps recognized by a language module may vary. However, the following typemap methods are nearly universal:

### 14.5.1 "in" typemap

The "in" typemap is used to convert function arguments from the target language to C. For example:

```
%typemap(in) int {
```

```
  $1 = PyInt_AsLong($input);
}
```

The following additional special variables are available:

```
$input            - Input object holding value to be converted.
```

This is probably the most commonly redefined typemap because it can be used to implement customized conversions.

In addition, the "in" typemap allows the number of converted arguments to be specified. The `numinputs` attributes facilitates this. For example:

```
// Ignored argument.
%typemap(in, numinputs=0) int *out (int temp) {
  $1 = &temp;
}
```

At this time, only zero or one arguments may be converted. When `numinputs` is set to 0, the argument is effectively ignored and cannot be supplied from the target language. The argument is still required when making the C/C++ call and the above typemap shows the value used is instead obtained from a locally declared variable called `temp`. Usually `numinputs` is not specified, whereupon the default value is 1, that is, there is a one to one mapping of the number of arguments when used from the target language to the C/C++ call. Multi-argument typemaps provide a similar concept where the number of arguments mapped from the target language to C/C++ can be changed for multiple adjacent C/C++ arguments.

### 14.5.2 "typecheck" typemap

The "typecheck" typemap is used to support overloaded functions and methods. It merely checks an argument to see whether or not it matches a specific type. For example:

```
%typemap(typecheck, precedence=SWIG_TYPECHECK_INTEGER) int {
  $1 = PyInt_Check($input) ? 1 : 0;
}
```

For typechecking, the $1 variable is always a simple integer that is set to 1 or 0 depending on whether or not the input argument is the correct type. Set to 1 if the input argument is the correct type otherwise set to 0.

If you define new "in" typemaps *and* your program uses overloaded methods, you should also define a collection of "typecheck" typemaps. More details about this follow in the Typemaps and overloading section.

### 14.5.3 "out" typemap

The "out" typemap is used to convert function/method return values from C into the target language. For example:

```
%typemap(out) int {
  $result = PyInt_FromLong($1);
}
```

The following additional special variables are available.

```
$result           - Result object returned to target language.
```

The "out" typemap supports an optional attribute flag called "optimal". This is for code optimisation and is detailed in the Optimal code generation when returning by value section.

### 14.5.4 "arginit" typemap

The "arginit" typemap is used to set the initial value of a function argument--before any conversion has occurred. This is not normally necessary, but might be useful in highly specialized applications. For example:

```
// Set argument to NULL before any conversion occurs
%typemap(arginit) int *data {
  $1 = NULL;
}
```

### 14.5.5 "default" typemap

The "default" typemap is used to turn an argument into a default argument. For example:

```
%typemap(default) int flags {
  $1 = DEFAULT_FLAGS;
}
...
int foo(int x, int y, int flags);
```

The primary use of this typemap is to either change the wrapping of default arguments or specify a default argument in a language where they aren't supported (like C). Target languages that do not support optional arguments, such as Java and C#, effectively ignore the value specified by this typemap as all arguments must be given.

Once a default typemap has been applied to an argument, all arguments that follow must have default values. See the Default/optional arguments section for further information on default argument wrapping.

### 14.5.6 "check" typemap

The "check" typemap is used to supply value checking code during argument conversion. The typemap is applied *after* arguments have been converted. For example:

```
%typemap(check) int positive {
  if ($1 <= 0) {
    SWIG_exception(SWIG_ValueError, "Expected positive value.");
  }
}
```

### 14.5.7 "argout" typemap

The "argout" typemap is used to return values from arguments. This is most commonly used to write wrappers for C/C++ functions that need to return multiple values. The "argout" typemap is almost always combined with an "in" typemap---possibly to ignore the input value. For example:

```
/* Set the input argument to point to a temporary variable */
%typemap(in, numinputs=0) int *out (int temp) {
  $1 = &temp;
}

%typemap(argout) int *out {
  // Append output value $1 to $result
  ...
}
```

The following additional special variables are available.

```
$result           - Result object returned to target language.
$input            - The original input object passed.
```

The code supplied to the "argout" typemap is always placed after the "out" typemap. If multiple return values are used, the extra return values are often appended to return value of the function.

See the `typemaps.i` library file for examples.

### 14.5.8 "freearg" typemap

The "freearg" typemap is used to cleanup argument data. It is only used when an argument might have allocated resources that need to be cleaned up when the wrapper function exits. The "freearg" typemap usually cleans up argument resources allocated by the "in" typemap. For example:

```
// Get a list of integers
%typemap(in) int *items {
  int nitems = Length($input);
  $1 = (int *) malloc(sizeof(int)*nitems);
}
// Free the list
%typemap(freearg) int *items {
  free($1);
}
```

The "freearg" typemap inserted at the end of the wrapper function, just before control is returned back to the target language. This code is also placed into a special variable `$cleanup` that may be used in other typemaps whenever a wrapper function needs to abort prematurely.

### 14.5.9 "newfree" typemap

The "newfree" typemap is used in conjunction with the `%newobject` directive and is used to deallocate memory used by the return result of a function. For example:

```
%typemap(newfree) string * {
  delete $1;
}
%typemap(out) string * {
  $result = PyString_FromString($1->c_str());
}
...

%newobject foo;
...
string *foo();
```

See Object ownership and %newobject for further details.

### 14.5.10 "ret" typemap

The "ret" typemap is not used very often, but can be useful for anything associated with the return type, such as resource management, return value error checking, etc. Usually this can all be done in the "out" typemap, but sometimes it is handy to use the "out" typemap code untouched and add to the generated code using the code in the "ret" typemap. One such case is memory clean up. For example, a `stringheap_t` type is defined indicating that the returned memory must be deleted and a `string_t` type is defined indicating that the returned memory must not be deleted.

```
%typemap(ret) stringheap_t %{
  free($1);
%}

typedef char * string_t;
typedef char * stringheap_t;

string_t MakeString1();
stringheap_t MakeString2();
```

The "ret" typemap above will only be used for `MakeString2`, but both functions will use the default "out" typemap for `char *` provided by SWIG. The code above would ensure the appropriate memory is freed in all target languages as the need to provide custom "out" typemaps (which involve target language specific code) is not necessary.

This approach is an alternative to using the "newfree" typemap and `%newobject` as there is no need to list all the functions that require the memory cleanup, it is purely done on types.

### 14.5.11 "memberin" typemap

The "memberin" typemap is used to copy data from *an already converted input value* into a structure member. It is typically used to handle array members and other special cases. For example:

```
%typemap(memberin) int [4] {
  memmove($1, $input, 4*sizeof(int));
}
```

It is rarely necessary to write "memberin" typemaps---SWIG already provides a default implementation for arrays, strings, and other objects.

### 14.5.12 "varin" typemap

The "varin" typemap is used to convert objects in the target language to C for the purposes of assigning to a C/C++ global variable. This is implementation specific.

### 14.5.13 "varout" typemap

The "varout" typemap is used to convert a C/C++ object to an object in the target language when reading a C/C++ global variable. This is implementation specific.

### 14.5.14 "throws" typemap

The "throws" typemap is only used when SWIG parses a C++ method with an exception specification or has the `%catches` feature attached to the method (see Exception handling with %catches). It provides a default mechanism for handling C++ methods that have declared the exceptions they will throw. The purpose of this typemap is to convert a C++ exception into an error or exception in the target language. It is slightly different to the other typemaps as it is based around the exception type rather than the type of a parameter or variable. For example:

```
%typemap(throws) const char * %{
  PyErr_SetString(PyExc_RuntimeError, $1);
  SWIG_fail;
%}

// Either an exception specification on the method
  void bar() throw (const char *);

// Or a %catches feature attached to the method
  %catches(const char *) bar();
  void bar();
```

As can be seen from the resulting generated code below, SWIG generates an exception handler when wrapping the `bar` function with the catch block comprising the "throws" typemap content.

```
...
try {
  bar();
} catch(char const *_e) {
  PyErr_SetString(PyExc_RuntimeError, _e);
  SWIG_fail;
}
...
```

Note that if your methods do not have an exception specification but they do throw exceptions and you are not using `%catches`, SWIG cannot know how to deal with them. Please also see the Exception handling with %exception section for another way to handle exceptions.

## 14.6 Some typemap examples

This section contains a few examples. Consult language module documentation for more examples.

### 14.6.1 Typemaps for arrays

A common use of typemaps is to provide support for C arrays appearing both as arguments to functions and as structure members.

For example, suppose you had a function like this:

```
void set_vector(int type, float value[4]);
```

If you wanted to handle `float value[4]` as a list of floats, you might write a typemap similar to this:

```
%typemap(in) float value[4] (float temp[4]) {
  int i;
  if (!PySequence_Check($input)) {
    PyErr_SetString(PyExc_ValueError, "Expected a sequence");
    SWIG_fail;
  }
  if (PySequence_Length($input) != 4) {
    PyErr_SetString(PyExc_ValueError, "Size mismatch. Expected 4 elements");
    SWIG_fail;
  }
  for (i = 0; i < 4; i++) {
    PyObject *o = PySequence_GetItem($input, i);
    if (PyNumber_Check(o)) {
      temp[i] = (float) PyFloat_AsDouble(o);
    } else {
      PyErr_SetString(PyExc_ValueError, "Sequence elements must be numbers");
      SWIG_fail;
    }
  }
  $1 = temp;
}
```

In this example, the variable `temp` allocates a small array on the C stack. The typemap then populates this array and passes it to the underlying C function.

When used from Python, the typemap allows the following type of function call:

```
>>> set_vector(type, [ 1, 2.5, 5, 20 ])
```

If you wanted to generalize the typemap to apply to arrays of all dimensions you might write this:

```
%typemap(in) float value[ANY] (float temp[$1_dim0]) {
  int i;
  if (!PySequence_Check($input)) {
    PyErr_SetString(PyExc_ValueError, "Expected a sequence");
    SWIG_fail;
  }
```

```
  if (PySequence_Length($input) != $1_dim0) {
    PyErr_SetString(PyExc_ValueError, "Size mismatch. Expected $1_dim0 elements");
    SWIG_fail;
  }
  for (i = 0; i < $1_dim0; i++) {
    PyObject *o = PySequence_GetItem($input, i);
    if (PyNumber_Check(o)) {
      temp[i] = (float) PyFloat_AsDouble(o);
    } else {
      PyErr_SetString(PyExc_ValueError, "Sequence elements must be numbers");
      SWIG_fail;
    }
  }
  $1 = temp;
}
```

In this example, the special variable `$1_dim0` is expanded with the actual array dimensions. Multidimensional arrays can be matched in a similar manner. For example:

```
%typemap(in) float matrix[ANY][ANY] (float temp[$1_dim0][$1_dim1]) {
  ... convert a 2d array ...
}
```

For large arrays, it may be impractical to allocate storage on the stack using a temporary variable as shown. To work with heap allocated data, the following technique can be used.

```
%typemap(in) float value[ANY] {
  int i;
  if (!PySequence_Check($input)) {
    PyErr_SetString(PyExc_ValueError, "Expected a sequence");
    SWIG_fail;
  }
  if (PySequence_Length($input) != $1_dim0) {
    PyErr_SetString(PyExc_ValueError, "Size mismatch. Expected $1_dim0 elements");
    SWIG_fail;
  }
  $1 = (float *) malloc($1_dim0*sizeof(float));
  for (i = 0; i < $1_dim0; i++) {
    PyObject *o = PySequence_GetItem($input, i);
    if (PyNumber_Check(o)) {
      $1[i] = (float) PyFloat_AsDouble(o);
    } else {
      free($1);
      PyErr_SetString(PyExc_ValueError, "Sequence elements must be numbers");
      SWIG_fail;
    }
  }
}
%typemap(freearg) float value[ANY] {
  free($1);
}
```

In this case, an array is allocated using `malloc`. The `freearg` typemap is then used to release the argument after the function has been called.

Another common use of array typemaps is to provide support for array structure members. Due to subtle differences between pointers and arrays in C, you can't just "assign" to a array structure member. Instead, you have to explicitly copy elements into the array. For example, suppose you had a structure like this:

```
struct SomeObject {
  float  value[4];
  ...
};
```

When SWIG runs, it won't produce any code to set the `vec` member. You may even get a warning message like this:

```
$ swig -python  example.i
example.i:10: Warning 462: Unable to set variable of type float [4].
```

These warning messages indicate that SWIG does not know how you want to set the `vec` field.

To fix this, you can supply a special "memberin" typemap like this:

```
%typemap(memberin) float [ANY] {
  int i;
  for (i = 0; i < $1_dim0; i++) {
      $1[i] = $input[i];
  }
}
```

The memberin typemap is used to set a structure member from data that has already been converted from the target language to C. In this case, `$input` is the local variable in which converted input data is stored. This typemap then copies this data into the structure.

When combined with the earlier typemaps for arrays, the combination of the "in" and "memberin" typemap allows the following usage:

```
>>> s = SomeObject()
>>> s.x = [1, 2.5, 5, 10]
```

Related to structure member input, it may be desirable to return structure members as a new kind of object. For example, in this example, you will get very odd program behavior where the structure member can be set nicely, but reading the member simply returns a pointer:

```
>>> s = SomeObject()
>>> s.x = [1, 2.5, 5, 10]
>>> print s.x
```

```
_1008fea8_p_float
>>>
```

To fix this, you can write an "out" typemap. For example:

```
%typemap(out) float [ANY] {
  int i;
  $result = PyList_New($1_dim0);
  for (i = 0; i < $1_dim0; i++) {
    PyObject *o = PyFloat_FromDouble((double) $1[i]);
    PyList_SetItem($result, i, o);
  }
}
```

Now, you will find that member access is quite nice:

```
>>> s = SomeObject()
>>> s.x = [1, 2.5, 5, 10]
>>> print s.x
[ 1, 2.5, 5, 10]
```

**Compatibility Note:** SWIG1.1 used to provide a special "memberout" typemap. However, it was mostly useless and has since been eliminated. To return structure members, simply use the "out" typemap.

### 14.6.2 Implementing constraints with typemaps

One particularly interesting application of typemaps is the implementation of argument constraints. This can be done with the "check" typemap. When used, this allows you to provide code for checking the values of function arguments. For example:

```
%module math

%typemap(check) double posdouble {
  if ($1 < 0) {
    croak("Expecting a positive number");
  }
}

...
double sqrt(double posdouble);
```

This provides a sanity check to your wrapper function. If a negative number is passed to this function, a Perl exception will be raised and your program terminated with an error message.

This kind of checking can be particularly useful when working with pointers. For example:

```
%typemap(check) Vector * {
  if ($1 == 0) {
    PyErr_SetString(PyExc_TypeError, "NULL Pointer not allowed");
    SWIG_fail;
  }
}
```

will prevent any function involving a `Vector *` from accepting a NULL pointer. As a result, SWIG can often prevent a potential segmentation faults or other run-time problems by raising an exception rather than blindly passing values to the underlying C/C++ program.

## 14.7 Typemaps for multiple target languages

The code within typemaps is usually language dependent, however, many target languages support the same typemaps. In order to distinguish typemaps across different languages, the preprocessor should be used. For example, the "in" typemap for Perl and Ruby could be written as:

```
#if defined(SWIGPERL)
  %typemap(in) int "$1 = ($1_ltype) SvIV($input);"
#elif defined(SWIGRUBY)
  %typemap(in) int "$1 = NUM2INT($input);"
#else
  #warning no "in" typemap defined
#endif
```

The full set of language specific macros is defined in the Conditional Compilation section. The example above also shows a common approach of issuing a warning for an as yet unsupported language.

**Compatibility note:** In SWIG-1.1 different languages could be distinguished with the language name being put within the `%typemap` directive, but this was deprecated in SWIG 1.3.28 and support finally dropped completely in SWIG 4.1.0 so you'll need to update any remaining uses to use the approach above. For example, `%typemap(ruby, in) int "$1 = NUM2INT($input);"`.

## 14.8 Optimal code generation when returning by value

The "out" typemap is the main typemap for return types. This typemap supports an optional attribute flag called "optimal", which is for reducing the number of temporary variables and the amount of generated code, thereby giving the compiler the opportunity to use *return value optimization* for generating faster executing code. It only really makes a difference when returning objects by value and has some limitations on usage, as explained later on.

When a function returns an object by value, SWIG generates code that instantiates the default type on the stack then assigns the value returned by the function call to it. A copy of this object is then made on the heap and this is what is ultimately stored and used from the target language. This will be clearer considering an example. Consider running the following code through SWIG:

```
%typemap(out) SWIGTYPE %{
  $result = new $1_ltype($1);
%}

%inline %{
```

```
#include <iostream>
using namespace std;

struct XX {
  XX() { cout << "XX()" << endl; }
  XX(int i) { cout << "XX(" << i << ")" << endl; }
  XX(const XX &other) { cout << "XX(const XX &)" << endl; }
  XX & operator =(const XX &other) { cout << "operator=(const XX &)" << endl; return *this; }
  ~XX() { cout << "~XX()" << endl; }
  static XX create() {
    return XX(0);
  }
};
%}
```

The "out" typemap shown is the default typemap for C# when returning objects by value. When making a call to `XX::create()` from C#, the output is as follows:

```
XX()
XX(0)
operator=(const XX &)
~XX()
XX(const XX &)
~XX()
~XX()
```

Note that three objects are being created as well as an assignment. Wouldn't it be great if the `XX::create()` method was the only time a constructor was called? As the method returns by value, this is asking a lot and the code that SWIG generates by default makes it impossible for the compiler to use *return value optimisation (RVO)*. However, this is where the "optimal" attribute in the "out" typemap can help out. If the typemap code is kept the same and just the "optimal" attribute specified like this:

```
%typemap(out, optimal="1") SWIGTYPE %{
  $result = new $1_ltype($1);
%}
```

then when the code is run again, the output is simply:

```
XX(0)
~XX()
```

How the "optimal" attribute works is best explained using the generated code. Without "optimal", the generated code is:

```
SWIGEXPORT void * SWIGSTDCALL CSharp_XX_create() {
  void * jresult ;
  XX result;
  result = XX::create();
  jresult = new XX(result);
  return jresult;
}
```

With the "optimal" attribute, the code is:

```
SWIGEXPORT void * SWIGSTDCALL CSharp_XX_create() {
  void * jresult ;
  jresult = new XX(XX::create());
  return jresult;
}
```

The major difference is the `result` temporary variable holding the value returned from `XX::create()` is no longer generated and instead the copy constructor call is made directly from the value returned by `XX::create()`. With modern compilers implementing RVO, the copy is not actually done, in fact the object is never created on the stack in `XX::create()` at all, it is simply created directly on the heap. In the first instance, the `$1` special variable in the typemap is expanded into `result`. In the second instance, `$1` is expanded into `XX::create()` and this is essentially what the "optimal" attribute is telling SWIG to do.

The "optimal" attribute optimisation is not turned on by default as it has a number of restrictions. Firstly, some code cannot be condensed into a simple call for passing into the copy constructor. One common occurrence is when %exception is used. Consider adding the following `%exception` to the example:

```
%exception XX::create() %{
try {
  $action
} catch(const std::exception &e) {
  cout << e.what() << endl;
}
%}
```

SWIG can detect when the "optimal" attribute cannot be used and will ignore it and in this case will issue the following warning:

```
example.i:28: Warning 474: Method XX::create() usage of the optimal attribute ignored
example.i:14: Warning 474: in the out typemap as the following cannot be used to generate
optimal code:
try {
  result = XX::create();
} catch(const std::exception &e) {
  cout << e.what() << endl;
}
```

It should be clear that the above code cannot be used as the argument to the copy constructor call, that is, for the `$1` substitution.

Secondly, if the typemap uses `$1` more than once, then multiple calls to the wrapped function will be made. Obviously that is not very optimal. In fact SWIG attempts to detect this and will issue a warning something like:

```
example.i:21: Warning 475: Multiple calls to XX::create() might be generated due to
example.i:7: Warning 475: optimal attribute usage in the out typemap.
```

However, it doesn't always get it right, for example when `$1` is within some commented out code.

## 14.9 Multi-argument typemaps

So far, the typemaps presented have focused on the problem of dealing with single values. For example, converting a single input object to a single argument in a function call. However, certain conversion problems are difficult to handle in this manner. As an example, consider the example at the very beginning of this chapter:

```
int foo(int argc, char *argv[]);
```

Suppose that you wanted to wrap this function so that it accepted a single list of strings like this:

```
>>> foo(["ale", "lager", "stout"])
```

To do this, you not only need to map a list of strings to `char *argv[]`, but the value of `int argc` is implicitly determined by the length of the list. Using only simple typemaps, this type of conversion is possible, but extremely painful. Multi-argument typemaps help in this situation.

A multi-argument typemap is a conversion rule that specifies how to convert a *single* object in the target language to a set of consecutive function arguments in C/C++. For example, the following multi-argument maps perform the conversion described for the above example:

```
%typemap(in) (int argc, char *argv[]) {
  int i;
  if (!PyList_Check($input)) {
    PyErr_SetString(PyExc_ValueError, "Expecting a list");
    SWIG_fail;
  }
  $1 = PyList_Size($input);
  $2 = (char **) malloc(($1+1)*sizeof(char *));
  for (i = 0; i < $1; i++) {
    PyObject *s = PyList_GetItem($input, i);
    if (!PyString_Check(s)) {
      free($2);
      PyErr_SetString(PyExc_ValueError, "List items must be strings");
      SWIG_fail;
    }
    $2[i] = PyString_AsString(s);
  }
  $2[i] = 0;
}

%typemap(freearg) (int argc, char *argv[]) {
  free($2);
}

/* Required for C++ method overloading */
%typecheck(SWIG_TYPECHECK_STRING_ARRAY) (int argc, char *argv[]) {
  $1 = PyList_Check($input) ? 1 : 0;
}
```

A multi-argument map is always specified by surrounding the arguments with parentheses as shown. For example:

```
%typemap(in) (int argc, char *argv[]) { ... }
```

Within the typemap code, the variables `$1`, `$2`, and so forth refer to each type in the map. All of the usual substitutions apply--just use the appropriate `$1` or `$2` prefix on the variable name (e.g., `$2_type`, `$1_ltype`, etc.)

Multi-argument typemaps always have precedence over simple typemaps and SWIG always performs longest-match searching. Therefore, you will get the following behavior:

```
%typemap(in) int argc                             { ... typemap 1 ... }
%typemap(in) (int argc, char *argv[])             { ... typemap 2 ... }
%typemap(in) (int argc, char *argv[], char *env[]) { ... typemap 3 ... }

int foo(int argc, char *argv[]);                  // Uses typemap 2
int bar(int argc, int x);                         // Uses typemap 1
int spam(int argc, char *argv[], char *env[]);    // Uses typemap 3
```

It should be stressed that multi-argument typemaps can appear anywhere in a function declaration and can appear more than once. For example, you could write this:

```
%typemap(in) (int scount, char *swords[]) { ... }
%typemap(in) (int wcount, char *words[]) { ... }

void search_words(int scount, char *swords[], int wcount, char *words[], int maxcount);
```

Other directives such as `%apply` and `%clear` also work with multi-argument maps. For example:

```
%apply (int argc, char *argv[]) {
    (int scount, char *swords[]),
    (int wcount, char *words[])
};
...
%clear (int scount, char *swords[]), (int wcount, char *words[]);
...
```

Don't forget to also provide a suitable typemap for overloaded functions, such as `%typecheck` shown for foo above. This is only required if the function is overloaded in C++.

Although multi-argument typemaps may seem like an exotic, little used feature, there are several situations where they make sense. First, suppose you wanted to wrap functions similar to the

low-level `read()` and `write()` system calls. For example:

```
typedef unsigned int size_t;

int read(int fd, void *rbuffer, size_t len);
int write(int fd, void *wbuffer, size_t len);
```

As is, the only way to use the functions would be to allocate memory and pass some kind of pointer as the second argument---a process that might require the use of a helper function. However, using multi-argument maps, the functions can be transformed into something more natural. For example, you might write typemaps like this:

```
// typemap for an outgoing buffer
%typemap(in) (void *wbuffer, size_t len) {
  if (!PyString_Check($input)) {
    PyErr_SetString(PyExc_ValueError, "Expecting a string");
    SWIG_fail;
  }
  $1 = (void *) PyString_AsString($input);
  $2 = PyString_Size($input);
}

// typemap for an incoming buffer
%typemap(in) (void *rbuffer, size_t len) {
  if (!PyInt_Check($input)) {
    PyErr_SetString(PyExc_ValueError, "Expecting an integer");
    SWIG_fail;
  }
  $2 = PyInt_AsLong($input);
  if ($2 < 0) {
    PyErr_SetString(PyExc_ValueError, "Positive integer expected");
    SWIG_fail;
  }
  $1 = (void *) malloc($2);
}

// Return the buffer.  Discarding any previous return result
%typemap(argout) (void *rbuffer, size_t len) {
  Py_DecRef($result);   /* Blow away any previous result */
  if (result < 0) {      /* Check for I/O error */
    free($1);
    PyErr_SetFromErrno(PyExc_IOError);
    return NULL;
  }
  $result = PyString_FromStringAndSize($1, result);
  free($1);
}
```

(note: In the above example, `$result` and `result` are two different variables. `result` is the real C datatype that was returned by the function. `$result` is the scripting language object being returned to the interpreter.).

Now, in a script, you can write code that simply passes buffers as strings like this:

```
>>> f = example.open("Makefile")
>>> example.read(f, 40)
'TOP        = ../..\nSWIG       = $(TOP)/.'
>>> example.read(f, 40)
'./swig\nSRCS       = example.c\nTARGET    '
>>> example.close(f)
0
>>> g = example.open("foo", example.O_WRONLY | example.O_CREAT, 0644)
>>> example.write(g, "Hello world\n")
12
>>> example.write(g, "This is a test\n")
15
>>> example.close(g)
0
>>>
```

A number of multi-argument typemap problems also arise in libraries that perform matrix-calculations--especially if they are mapped onto low-level Fortran or C code. For example, you might have a function like this:

```
int is_symmetric(double *mat, int rows, int columns);
```

In this case, you might want to pass some kind of higher-level object as an matrix. To do this, you could write a multi-argument typemap like this:

```
%typemap(in) (double *mat, int rows, int columns) {
  MatrixObject *a;
  a = GetMatrixFromObject($input);     /* Get matrix somehow */

  /* Get matrix properties */
  $1 = GetPointer(a);
  $2 = GetRows(a);
  $3 = GetColumns(a);
}
```

This kind of technique can be used to hook into scripting-language matrix packages such as Numeric Python. However, it should also be stressed that some care is in order. For example, when crossing languages you may need to worry about issues such as row-major vs. column-major ordering (and perform conversions if needed). Note that multi-argument typemaps cannot deal with non-consecutive C/C++ arguments; a workaround such as a helper function re-ordering the arguments to make them consecutive will need to be written.

## 14.10 Typemap warnings

Warnings can be added to typemaps so that SWIG generates a warning message whenever the typemap is used. See the information in the issuing warnings section.

## 14.11 Typemap fragments

The primary purpose of fragments is to reduce code bloat that repeated use of typemap code can lead to. Fragments are snippets of code that can be thought of as code dependencies of a typemap. If a fragment is used by more than one typemap, then the snippet of code within the fragment is only generated once. Code bloat is typically reduced by moving typemap code into a support function and then placing the support function into a fragment.

For example, if you have a very long typemap

```
%typemap(in) MyClass * {
  MyClass *value = 0;

  ... many lines of marshalling code  ...

  $result = value;
}
```

the same marshalling code is often repeated in several typemaps, such as "in", "varin", "directorout", etc. SWIG copies the code for each argument that requires the typemap code, easily leading to code bloat in the generated code. To eliminate this, define a fragment that includes the common marshalling code:

```
%fragment("AsMyClass", "header") {
  MyClass *AsMyClass(PyObject *obj) {
    MyClass *value = 0;

    ... many lines of marshalling code  ...

    return value;
  }
}

%typemap(in, fragment="AsMyClass") MyClass * {
  $result = AsMyClass($input);
}

%typemap(varin, fragment="AsMyClass") MyClass * {
  $result = AsMyClass($input);
}
```

When the "in" or "varin" typemaps for MyClass are required, the contents of the fragment called "AsMyClass" are added to the "header" section within the generated code, and then the typemap code is emitted. Hence, the method `AsMyClass` will be generated into the wrapper code before any typemap code that calls it.

To define a fragment you need a fragment name, a section name for generating the fragment code into, and the code itself. See Code insertion blocks for a full list of section names. Usually the section name used is "header". Different delimiters can be used:

```
%fragment("my_name", "header") %{ ... %}
%fragment("my_name", "header") { ... }
%fragment("my_name", "header") " ... "
```

and these follow the usual preprocessing rules mentioned in the Preprocessing delimiters section. The following are some rules and guidelines for using fragments:

1. A fragment is added to the wrapping code only once. When using the `MyClass *` typemaps above and wrapping the method:

   ```
   void foo(MyClass *a, MyClass *b);
   ```

   the generated code will look something like:

   ```
   MyClass *AsMyClass(PyObject *obj) {
     ...
   }

   void _wrap_foo(...) {
     ....
     arg1 = AsMyClass(obj1);
     arg2 = AsMyClass(obj2);
     ...
     foo(arg1, arg2);
   }
   ```

   even as there is duplicated typemap code to process both `a` and `b`, the `AsMyClass` method will be defined only once.

2. A fragment should only be defined once. If there is more than one definition, the first definition is the one used. All other definitions are silently ignored. For example, if you have

   ```
   %fragment("AsMyClass", "header") { ...definition 1... }
   ....
   %fragment("AsMyClass", "header") { ...definition 2... }
   ```

   only the first definition is used. In this way you can override the default fragments in a SWIG library by defining your fragment before the library `%include`. Note that this behavior is the opposite to typemaps, where the last typemap defined/applied prevails. Fragments follow the first-in-first-out convention since they are intended to be global, while typemaps are intended to be locally specialized.

3. Fragment names cannot contain commas.

4. A fragment can use one or more additional fragments, for example:

   ```
   %fragment("<limits.h>", "header") %{
     #include <limits.h>
   %}


   %fragment("AsMyClass", "header", fragment="<limits.h>") {
     MyClass *AsMyClass(PyObject *obj) {
       MyClass *value = 0;
   ```

```
    ... some marshalling code  ...

    if  (ival < CHAR_MIN /*defined in <limits.h>*/) {
      ...
    } else {
      ...
    }
    ...
    return value;
  }
}
```

in this case, when the "AsMyClass" fragment is emitted, it also triggers the inclusion of the "<limits.h>" fragment.

5. A fragment can have dependencies on a number of other fragments, for example:

```
%fragment("bigfragment", "header", fragment="frag1", fragment="frag2", fragment="frag3") "";
```

When the "bigfragment" is used, the three dependent fragments "frag1", "frag2" and "frag3" are also pulled in. Note that as "bigframent" is empty (the empty string - ""), it does not add any code itself, but merely triggers the inclusion of the other fragments.

6. A typemap can also use more than one fragment:

```
%typemap("in", fragment="frag1", fragment="frag2", fragment="frag3") {...}
```

**Compatibility note:** The ability to use multiple `fragment` keys as shown above was introduced in SWIG-4.1.0.

Multiple fragments can alternatively be specified as a comma separated list value in a single `fragment` key. Note that no whitespace is allowed within this comma separated list. The following is the equivalent to the above:

```
%typemap(in, fragment="frag1,frag2,frag3") {...}
```

which in turn is functionally equivalent to:

```
%typemap(in, fragment="bigfragment") {...}
```

when used with the "bigfragment" defined above.

7. Finally, you can force the inclusion of a fragment at any point in the generated code as follows:

```
%fragment("bigfragment");
```

which, for example, is very useful inside a template class. Another useful case is when using `%extend` inside a class where the additional code in the `%extend` block depends on the contents of the fragment.

```
%fragment("<limits.h>", "header") %{
  #include <limits.h>
%}

struct X {
  ...
  %extend {
    %fragment("<limits.h>");
    bool check(short val) {
      if (val < SHRT_MIN /*defined in <limits.h>*/) {
        return true;
      } else {
        return false;
      }
    }
  }
};
```

Forced inclusion of fragments can be used as a replacement for code insertion block, ensuring the code block is only generated once. Consider the contents of FileA.i below which first uses a code insertion block and then a forced fragment inclusion to generate code:

```
// FileA.i
%{
  #include <stdio.h>
%}
%fragment("<limits.h>");
```

and another file including the above:

```
// FileB.i
%include "FileA.i"
```

The resulting code in the wrappers for FileB.i is:

```
  #include <stdio.h>

  #include <limits.h>
```

A note of caution must be mentioned when using `%fragment` forced inclusion or code insertion blocks with `%import`. If `%import` is used instead:

```
// FileC.i
%import "FileA.i"
```

then nothing is generated in the resulting code in the wrappers for FileC.i. This is because `%import` is for collecting type information and does not result in any code being generated, see File Imports.

Most readers will probably want to skip the next two sub-sections on advanced fragment usage unless a desire to really get to grips with some powerful but tricky macro and fragment usage that is used in parts of the SWIG typemap library.

### 14.11.1 Fragment type specialization

Fragments can be *type specialized*. The syntax is as follows:

```
%fragment("name", "header") { ...a type independent fragment... }
%fragment("name"{type}, "header") { ...a type dependent fragment...  }
```

where `type` is a C/C++ type. Like typemaps, fragments can also be used inside templates, for example:

```
template <class T>
struct A {
  %fragment("incode"{A<T>}, "header") {
    ... 'incode' specialized fragment ...
  }

  %typemap(in, fragment="incode"{A<T>}) {
    ... here we use the 'type specialized' fragment "incode"{A<T>} ...
  }
};
```

### 14.11.2 Fragments and automatic typemap specialization

Since fragments can be type specialized, they can be elegantly used to specialize typemaps. For example, if you have something like:

```
%fragment("incode"{float}, "header") {
  float in_method_float(PyObject *obj) {
    ...
  }
}

%fragment("incode"{long}, "header") {
  float in_method_long(PyObject *obj) {
    ...
  }
}

// %my_typemaps macro definition
%define %my_typemaps(Type)
%typemap(in, fragment="incode"{Type}) Type {
  value = in_method_##Type(obj);
}
%enddef

%my_typemaps(float);
%my_typemaps(long);
```

then the proper `"incode"{float}` or `"incode"{long}` fragment will be used, and the `in_method_float` and `in_method_long` methods will be called whenever the `float` or `long` types are used as input parameters.

This feature is used a lot in the typemaps shipped in the SWIG library for some scripting languages. The interested (or very brave) reader can take a look at the fragments.swg file shipped with SWIG to see this in action.

## 14.12 The run-time type checker

Most scripting languages need type information at run-time. This type information can include how to construct types, how to garbage collect types, and the inheritance relationships between types. If the language interface does not provide its own type information storage, the generated SWIG code needs to provide it.

Requirements for the type system:

- Store inheritance and type equivalence information and be able to correctly re-create the type pointer.
- Share type information between modules.
- Modules can be loaded in any order, regardless of actual type dependency.
- Avoid the use of dynamically allocated memory, and library/system calls in general.
- Provide a reasonably fast implementation, minimizing the lookup time for all language modules.
- Custom, language specific information can be attached to types.
- Modules can be unloaded from the type system.

### 14.12.1 Implementation

The run-time type checker is used by many, but not all, of SWIG's supported target languages. The run-time type checker features are not required and are thus not used for statically typed languages such as Java and C#. The scripting and scheme based languages rely on it and it forms a critical part of SWIG's operation for these languages.

When pointers, arrays, and objects are wrapped by SWIG, they are normally converted into typed pointer objects. For example, an instance of `Foo *` might be a string encoded like this:

```
_108e688_p_Foo
```

At a basic level, the type checker simply restores some type-safety to extension modules. However, the type checker is also responsible for making sure that wrapped C++ classes are handled correctly---especially when inheritance is used. This is especially important when an extension module makes use of multiple inheritance. For example:

```
class Foo {
public:
  int x;
};
```

```
class Bar {
public:
  int y;
};

class FooBar : public Foo, public Bar {
public:
  int z;
};
```

When the class `FooBar` is organized in memory, it contains the contents of the classes `Foo` and `Bar` as well as its own data members. For example:

```
FooBar --> | ------------|  <-- Foo
           |   int x     |
           |-------------|  <-- Bar
           |   int y     |
           |-------------|
           |   int z     |
           |-------------|
```

Because of the way that base class data is stacked together, the casting of a `Foobar *` to either of the base classes may change the actual value of the pointer. This means that it is generally not safe to represent pointers using a simple integer or a bare `void *` ---type tags are needed to implement correct handling of pointer values (and to make adjustments when needed).

In the wrapper code generated for each language, pointers are handled through the use of special type descriptors and conversion functions. For example, if you look at the wrapper code for Python, you will see code similar to the following (simplified for brevity):

```
if (!SWIG_IsOK(SWIG_ConvertPtr(obj0, (void **) &arg1, SWIGTYPE_p_Foo, 0))) {
  SWIG_exception_fail(SWIG_TypeError, "in method 'GrabVal', expecting type Foo");
}
```

In this code, `SWIGTYPE_p_Foo` is the type descriptor that describes `Foo *`. The type descriptor is actually a pointer to a structure that contains information about the type name to use in the target language, a list of equivalent typenames (via typedef or inheritance), and pointer value handling information (if applicable). The `SWIG_ConvertPtr()` function is simply a utility function that takes a pointer object in the target language and a type-descriptor object and uses this information to generate a C++ pointer. The `SWIG_IsOK` macro checks the return value for errors and `SWIG_exception_fail` can be called to raise an exception in the target language. However, the exact name and calling conventions of the conversion function depends on the target language (see language specific chapters for details).

The actual type code is in swigrun.swg, and gets inserted near the top of the generated swig wrapper file. The phrase "a type X that can cast into a type Y" means that given a type X, it can be converted into a type Y. In other words, X is a derived class of Y or X is a typedef of Y. The structure to store type information looks like this:

```
/* Structure to store information on one type */
typedef struct swig_type_info {
  const char *name;             /* mangled name of this type */
  const char *str;              /* human readable name for this type */
  swig_dycast_func dcast;       /* dynamic cast function down a hierarchy */
  struct swig_cast_info *cast;  /* Linked list of types that can cast into this type */
  void *clientdata;             /* Language specific type data */
} swig_type_info;

/* Structure to store a type and conversion function used for casting */
typedef struct swig_cast_info {
  swig_type_info *type;          /* pointer to type that is equivalent to this type */
  swig_converter_func converter; /* function to cast the void pointers */
  struct swig_cast_info *next;   /* pointer to next cast in linked list */
  struct swig_cast_info *prev;   /* pointer to the previous cast */
} swig_cast_info;
```

Each `swig_type_info` stores a linked list of types that it is equivalent to. Each entry in this doubly linked list stores a pointer back to another swig_type_info structure, along with a pointer to a conversion function. This conversion function is used to solve the above problem of the FooBar class, correctly returning a pointer to the type we want.

The basic problem we need to solve is verifying and building arguments passed to functions. So going back to the `SWIG_ConvertPtr()` function example from above, we are expecting a `Foo *` and need to check if `obj0` is in fact a `Foo *` . From before, `SWIGTYPE_p_Foo` is just a pointer to the `swig_type_info` structure describing `Foo *`. So we loop through the linked list of `swig_cast_info` structures attached to `SWIGTYPE_p_Foo`. If we see that the type of`obj0` is in the linked list, we pass the object through the associated conversion function and then return a positive. If we reach the end of the linked list without a match, then `obj0` can not be converted to a `Foo *` and an error is generated.

Another issue needing to be addressed is sharing type information between multiple modules. More explicitly, we need to have ONE `swig_type_info` for each type. If two modules both use the type, the second module loaded must lookup and use the swig_type_info structure from the module already loaded. Because no dynamic memory is used and the circular dependencies of the casting information, loading the type information is somewhat tricky, and not explained here. A complete description is in the `Lib/swiginit.swg` file (and near the top of any generated file).

Each module has one swig_module_info structure which looks like this:

```
/* Structure used to store module information
 * Each module generates one structure like this, and the runtime collects
 * all of these structures and stores them in a circularly linked list.*/
typedef struct swig_module_info {
  swig_type_info **types;         /* Array of pointers to swig_type_info structs in this module */
  int size;                       /* Number of types in this module */
  struct swig_module_info *next;  /* Pointer to next element in circularly linked list */
  swig_type_info **type_initial;  /* Array of initially generated type structures */
  swig_cast_info **cast_initial;  /* Array of initially generated casting structures */
  void *clientdata;               /* Language specific module data */
} swig_module_info;
```

Each module stores an array of pointers to `swig_type_info` structures and the number of types in this module. So when a second module is loaded, it finds the `swig_module_info` structure for the first module and searches the array of types. If any of its own types are in the first module and have already been loaded, it uses those `swig_type_info` structures rather than creating new ones. These `swig_module_info` structures are chained together in a circularly linked list.

### 14.12.2 Usage

This section covers how to use these functions from typemaps. To learn how to call these functions from external files (not the generated _wrap.c file), see the External access to the run-time system section.

When pointers are converted in a typemap, the typemap code often looks similar to this:

```
%typemap(in) Foo * {
  if (!SWIG_IsOK(SWIG_ConvertPtr($input, (void **) &$1, $1_descriptor, 0))) {
    SWIG_exception_fail(SWIG_TypeError, "in method '$symname', expecting type Foo");
  }
}
```

The most critical part is the typemap is the use of the `$1_descriptor` special variable. When placed in a typemap, this is expanded into the `SWIGTYPE_*` type descriptor object above. As a general rule, you should always use `$1_descriptor` instead of trying to hard-code the type descriptor name directly.

There is another reason why you should always use the `$1_descriptor` variable. When this special variable is expanded, SWIG marks the corresponding type as "in use." When type-tables and type information is emitted in the wrapper file, descriptor information is only generated for those datatypes that were actually used in the interface. This greatly reduces the size of the type tables and improves efficiency.

Occasionally, you might need to write a typemap that needs to convert pointers of other types. To handle this, the special variable macro `$descriptor(type)` covered earlier can be used to generate the SWIG type descriptor name for any C datatype. For example:

```
%typemap(in) Foo * {
  if (!SWIG_IsOK(SWIG_ConvertPtr($input, (void **) &$1, $1_descriptor, 0))) {
    Bar *temp;
    if (!SWIG_IsOK(SWIG_ConvertPtr($input, (void **) &temp, $descriptor(Bar *), 0))) {
      SWIG_exception_fail(SWIG_TypeError, "in method '$symname', expecting type Foo or Bar");
    }
    $1 = (Foo *)temp;
  }
}
```

The primary use of `$descriptor(type)` is when writing typemaps for container objects and other complex data structures. There are some restrictions on the argument---namely it must be a fully defined C datatype. It can not be any of the special typemap variables.

In certain cases, SWIG may not generate type-descriptors like you expect. For example, if you are converting pointers in some non-standard way or working with an unusual combination of interface files and modules, you may find that SWIG omits information for a specific type descriptor. To fix this, you may need to use the `%types` directive. For example:

```
%types(int *, short *, long *, float *, double *);
```

When `%types` is used, SWIG generates type-descriptor information even if those datatypes never appear elsewhere in the interface file.

Further details about the run-time type checking can be found in the documentation for individual language modules. Reading the source code may also help. The file `Lib/swigrun.swg` in the SWIG library contains all of the source of the generated code for type-checking. This code is also included in every generated wrapped file so you probably just look at the output of SWIG to get a better sense for how types are managed.

## 14.13 Typemaps and overloading

This section does not apply to the statically typed languages like Java and C#, where overloading of the types is handled much like C++ by generating overloaded methods in the target language. In many of the other target languages, SWIG still fully supports C++ overloaded methods and functions. For example, if you have a collection of functions like this:

```
int foo(int x);
int foo(double x);
int foo(char *s, int y);
```

You can access the functions in a normal way from the scripting interpreter:

```
# Python
foo(3)           # foo(int)
foo(3.5)         # foo(double)
foo("hello", 5)  # foo(char *, int)

# Tcl
foo 3            # foo(int)
foo 3.5          # foo(double)
foo hello 5      # foo(char *, int)
```

To implement overloading, SWIG generates a separate wrapper function for each overloaded method. For example, the above functions would produce something roughly like this:

```
// wrapper pseudocode
_wrap_foo_0(argc, args[]) {       // foo(int)
  int arg1;
  int result;
  ...
  arg1 = FromInteger(args[0]);
  result = foo(arg1);
  return ToInteger(result);
}

_wrap_foo_1(argc, args[]) {       // foo(double)
  double arg1;
  int result;
  ...
  arg1 = FromDouble(args[0]);
  result = foo(arg1);
  return ToInteger(result);
}

_wrap_foo_2(argc, args[]) {       // foo(char *, int)
  char *arg1;
  int   arg2;
  int result;
  ...
  arg1 = FromString(args[0]);
  arg2 = FromInteger(args[1]);
  result = foo(arg1, arg2);
```

```
  return ToInteger(result);
}
```

Next, a dynamic dispatch function is generated:

```
_wrap_foo(argc, args[]) {
  if (argc == 1) {
    if (IsInteger(args[0])) {
      return _wrap_foo_0(argc, args);
    }
    if (IsDouble(args[0])) {
      return _wrap_foo_1(argc, args);
    }
  }
  if (argc == 2) {
    if (IsString(args[0]) && IsInteger(args[1])) {
      return _wrap_foo_2(argc, args);
    }
  }
  error("No matching function!\n");
}
```

The purpose of the dynamic dispatch function is to select the appropriate C++ function based on argument types---a task that must be performed at runtime in most of SWIG's target languages.

The generation of the dynamic dispatch function is a relatively tricky affair. Not only must input typemaps be taken into account (these typemaps can radically change the types of arguments accepted), but overloaded methods must also be sorted and checked in a very specific order to resolve potential ambiguity. A high-level overview of this ranking process is found in the "SWIG and C++ " chapter. What isn't mentioned in that chapter is the mechanism by which it is implemented---as a collection of typemaps.

To support dynamic dispatch, SWIG first defines a general purpose type hierarchy as follows:

```
Symbolic Name                  Precedence Value
-----------------------------  ----------------
SWIG_TYPECHECK_POINTER          0
SWIG_TYPECHECK_ITERATOR         5
SWIG_TYPECHECK_VOIDPTR          10
SWIG_TYPECHECK_BOOL             15
SWIG_TYPECHECK_UINT8            20
SWIG_TYPECHECK_INT8             25
SWIG_TYPECHECK_UINT16           30
SWIG_TYPECHECK_INT16            35
SWIG_TYPECHECK_UINT32           40
SWIG_TYPECHECK_INT32            45
SWIG_TYPECHECK_SIZE             47
SWIG_TYPECHECK_PTRDIFF          48
SWIG_TYPECHECK_UINT64           50
SWIG_TYPECHECK_INT64            55
SWIG_TYPECHECK_UINT128          60
SWIG_TYPECHECK_INT128           65
SWIG_TYPECHECK_INTEGER          70
SWIG_TYPECHECK_FLOAT            80
SWIG_TYPECHECK_DOUBLE           90
SWIG_TYPECHECK_CPLXFLT          95
SWIG_TYPECHECK_CPLXDBL          100
SWIG_TYPECHECK_COMPLEX          105
SWIG_TYPECHECK_UNICHAR          110
SWIG_TYPECHECK_STDUNISTRING     115
SWIG_TYPECHECK_UNISTRING        120
SWIG_TYPECHECK_CHAR             130
SWIG_TYPECHECK_STDSTRING        135
SWIG_TYPECHECK_STRING           140
SWIG_TYPECHECK_PAIR             150
SWIG_TYPECHECK_STDARRAY         155
SWIG_TYPECHECK_VECTOR           160
SWIG_TYPECHECK_DEQUE            170
SWIG_TYPECHECK_LIST             180
SWIG_TYPECHECK_SET              190
SWIG_TYPECHECK_MULTISET         200
SWIG_TYPECHECK_MAP              210
SWIG_TYPECHECK_MULTIMAP         220
SWIG_TYPECHECK_STACK            230
SWIG_TYPECHECK_QUEUE            240
SWIG_TYPECHECK_BOOL_ARRAY       1015
SWIG_TYPECHECK_INT8_ARRAY       1025
SWIG_TYPECHECK_INT16_ARRAY      1035
SWIG_TYPECHECK_INT32_ARRAY      1045
SWIG_TYPECHECK_INT64_ARRAY      1055
SWIG_TYPECHECK_INT128_ARRAY     1065
SWIG_TYPECHECK_FLOAT_ARRAY      1080
SWIG_TYPECHECK_DOUBLE_ARRAY     1090
SWIG_TYPECHECK_CHAR_ARRAY       1130
SWIG_TYPECHECK_STRING_ARRAY     1140
SWIG_TYPECHECK_OBJECT_ARRAY     1150
SWIG_TYPECHECK_BOOL_PTR         2015
SWIG_TYPECHECK_UINT8_PTR        2020
SWIG_TYPECHECK_INT8_PTR         2025
SWIG_TYPECHECK_UINT16_PTR       2030
SWIG_TYPECHECK_INT16_PTR        2035
SWIG_TYPECHECK_UINT32_PTR       2040
SWIG_TYPECHECK_INT32_PTR        2045
SWIG_TYPECHECK_UINT64_PTR       2050
SWIG_TYPECHECK_INT64_PTR        2055
SWIG_TYPECHECK_FLOAT_PTR        2080
SWIG_TYPECHECK_DOUBLE_PTR       2090
SWIG_TYPECHECK_CHAR_PTR         2130
SWIG_TYPECHECK_SWIGOBJECT       5000
```

(These precedence levels are defined in `swig.swg`, a library file that's included by all target language modules.)

In this table, the precedence-level determines the order in which types are going to be checked. Low values are always checked before higher values. For example, integers are checked before floats, single values are checked before arrays, and so forth.

Using the above table as a guide, each target language defines a collection of "typecheck" typemaps. The following excerpt from the Python module illustrates this:

```
/* Python type checking rules */
/* Note:  %typecheck(X) is a macro for %typemap(typecheck, precedence=X) */

%typecheck(SWIG_TYPECHECK_INTEGER)
  int, short, long,
  unsigned int, unsigned short, unsigned long,
  signed char, unsigned char,
  long long, unsigned long long,
  const int &, const short &, const long &,
  const unsigned int &, const unsigned short &, const unsigned long &,
  const long long &, const unsigned long long &,
  enum SWIGTYPE,
  bool, const bool &
{
  $1 = (PyInt_Check($input) || PyLong_Check($input)) ? 1 : 0;
}

%typecheck(SWIG_TYPECHECK_DOUBLE)
  float, double,
  const float &, const double &
{
  $1 = (PyFloat_Check($input) || PyInt_Check($input) || PyLong_Check($input)) ? 1 : 0;
}

%typecheck(SWIG_TYPECHECK_CHAR) char {
  $1 = (PyString_Check($input) && (PyString_Size($input) == 1)) ? 1 : 0;
}

%typecheck(SWIG_TYPECHECK_STRING) char * {
  $1 = PyString_Check($input) ? 1 : 0;
}

%typemap(typecheck, precedence=SWIG_TYPECHECK_POINTER, noblock=1) SWIGTYPE * {
  void *vptr = 0;
  int res = SWIG_ConvertPtr($input, &vptr, $1_descriptor, 0);
  $1 = SWIG_IsOK(res) ? 1 : 0;
}

%typecheck(SWIG_TYPECHECK_POINTER) PyObject * {
  $1 = ($input != 0);
}
```

It might take a bit of contemplation, but this code has merely organized all of the basic C++ types, provided some simple type-checking code, and assigned each type a precedence value.

Finally, to generate the dynamic dispatch function, SWIG uses the following algorithm:

- Overloaded methods are first sorted by the number of required arguments.
- Methods with the same number of arguments are then sorted by precedence values of argument types.
- Typecheck typemaps are then emitted to produce a dispatch function that checks arguments in the correct order.

If you haven't written any typemaps of your own, it is unnecessary to worry about the typechecking rules. However, if you have written new input typemaps, you might have to supply a typechecking rule as well. An easy way to do this is to simply copy one of the existing typechecking rules. Here is an example,

```
// Typemap for a C++ string
%typemap(in) std::string {
  if (PyString_Check($input)) {
    $1 = std::string(PyString_AsString($input));
  } else {
    SWIG_exception(SWIG_TypeError, "string expected");
  }
}
// Copy the typecheck code for "char *".
%typemap(typecheck) std::string = char *;
```

The bottom line: If you are writing new typemaps and you are using overloaded methods, you will probably have to write new typecheck code or copy and modify existing typecheck code.

If you write a typecheck typemap and omit the precedence level, for example commenting it out as shown below:

```
%typemap(typecheck /*, precedence=SWIG_TYPECHECK_INTEGER*/) int {
  $1 = PyInt_Check($input) ? 1 : 0;
}
```

then the type is given a precedence higher than any other known precedence level and a warning is issued:

```
example.i:18: Warning 467: Overloaded method foo(int) not supported (incomplete type
checking rule - no precedence level in typecheck typemap for 'int').
```

**Notes:**

- Typecheck typemaps are not used for non-overloaded methods. Because of this, it is still always necessary to check types in any "in" typemaps.
- The dynamic dispatch process is only meant to be a heuristic. There are many corner cases where SWIG simply can't disambiguate types to the same degree as C++. The only way to resolve this ambiguity is to use the %rename directive to rename one of the overloaded methods (effectively eliminating overloading).
- Typechecking may be partial. For example, if working with arrays, the typecheck code might simply check the type of the first array element and use that to dispatch to the correct function. Subsequent "in" typemaps would then perform more extensive type-checking.
- Make sure you read the section on overloading in the SWIG and C++ chapter.

### 14.13.1 SWIG_TYPECHECK_POINTER precedence level and the typecheck typemap

When it comes to overloading of a particular type passed by value, pointer or reference (const and non-const), a C++ compiler can disambiguate which overloaded function to call. However,

SWIG effectively treats these as pointers in the target language and thus as equivalent types. For example, consider:

```
class X { ... };
void m(X const &c); // equivalent: void m(X *c);
void m(X &r);       // equivalent: void m(X *r);
void m(X *p);       // equivalent: void m(X *p);
```

These cannot be disambiguated in the target languages and so SWIG will choose the first method and ignore the subsequent two methods. The scripting languages do this by using the overload dispatch mechanism described earlier and warnings indicate this:

```
example.i:6: Warning 509: Overloaded method m(X &) effectively ignored,
example.i:5: Warning 509: as it is shadowed by m(X const &).
example.i:7: Warning 509: Overloaded method m(X *) effectively ignored,
example.i:5: Warning 509: as it is shadowed by m(X const &).
```

The statically typed languages like Java and C# automatically ignore all but the first equivalent overloaded methods with warnings:

```
example.i:6: Warning 516: Overloaded method m(X &) ignored,
example.i:5: Warning 516: using m(X const &) instead.
example.i:7: Warning 516: Overloaded method m(X *) ignored,
example.i:5: Warning 516: using m(X const &) instead.
```

You can select the overloaded method you would like to wrap by ignoring the other two with `%ignore` or rename two of them with `%rename` and this will of course remove the warnings too. The problem of ambiguity is also discussed in the C++ chapter on overloading.

So how does this work with respect to typemaps? The typemaps SWIG provides to handle overloading for these three methods are from the SWIGTYPE family. As discussed earlier, in Default typemap matching rules, the `SWIGTYPE &` typemaps are used for references and `SWIGTYPE *` typemaps are used for pointers. SWIG uses the special `SWIG_TYPECHECK_POINTER` (0) precedence level to handle these types in the "typecheck" typemap:

```
%typemap(typecheck, precedence=SWIG_TYPECHECK_POINTER) SWIGTYPE & "..."
%typemap(typecheck, precedence=SWIG_TYPECHECK_POINTER) SWIGTYPE * "..."
```

When the SWIGTYPE "typecheck" typemaps use the `SWIG_TYPECHECK_POINTER` precedence level, SWIG converts the type to a pointer equivalent type and then uses the equivalent type to detect if it can be disambiguated in an overloaded method in the target language. In our example above, the equivalent types for `X const &` , `X &` and `X *` are all `X *`. As they are the same, they cannot be disambiguated and so just the first overloaded method is chosen.

The automatic conversion to equivalent types and subsequent type comparison is triggered via the use of the special `SWIG_TYPECHECK_POINTER` precedence level and works for types passed by value, pointer and reference. Alas, there are more ways to overload a method that also need handling. C++ smart pointers are such a type which can be disambiguated by a C++ compiler but not automatically by SWIG. SWIG does not automatically know that a smart pointer has an equivalent type, but it can be told manually. Just specify the 'equivalent' attribute in the "typecheck" typemap with a pointer to the underlying type.

```
%typemap(typecheck, precedence=SWIG_TYPECHECK_POINTER, equivalent="X *") MySmartPtr<X> " ... "

void m(X &r);            // equivalent: void m(X *r);
void m(MySmartPtr<X> s); // equivalent: void m(X *s);
```

Now SWIG will detect the two types are equivalent and generate valid code by wrapping just the first overloaded method. You can of course choose which method to wrap by ignoring one of them with `%ignore` . Otherwise both can be wrapped by removing the overloading name ambiguity by renaming one of them with `%rename`.

The 'equivalent' attribute is used in the implementation for the shared_ptr smart pointer library.

## 14.14 More about %apply and %clear

In order to implement certain kinds of program behavior, it is sometimes necessary to write a family of typemap methods. For example, to support output arguments, one often writes a family of typemaps like this:

```
%typemap(in, numinputs=0) int *OUTPUT (int temp) {
  $1 = &temp;
}
%typemap(argout) int *OUTPUT {
  // return value somehow
}
```

To make it easier to apply the typemap to different argument types and names, the `%apply` directive performs a copy of all typemaps from a source type to one or more set of target types. For example, if you specify this,

```
%apply int *OUTPUT { int *retvalue, int32 *output };
```

then all of the `int *OUTPUT` (source) typemap methods are copied to `int *retvalue` and `int32 *output` (the targets).

However, there is a subtle aspect of `%apply` that needs clarification. Namely, if a target contains a typemap method that the source does not, the target typemap method remains in place and unchanged. This behavior allows you to do two things:

- You can specialize parts of a complex typemap rule by first defining a few typemaps and then using `%apply` to incorporate the remaining pieces.
- Different typemaps can be applied to the same datatype using repeated `%apply` directives.

For example:

```
%typemap(in) int *INPUT (int temp) {
  temp = ... get value from $input ...;
  $1 = &temp;
}

%typemap(check) int *POSITIVE {
  if (*$1 <= 0) {
    SWIG_exception(SWIG_ValueError, "Expected a positive number!\n");
    return NULL;
  }
}
```

```
%typemap(arginit) int *invalue %{
  $1 = NULL;
%}

...
%apply int *INPUT     { int *invalue };
%apply int *POSITIVE  { int *invalue };
```

In this example, neither of the two `%apply` directives will overwrite / delete the "arginit" typemap as neither has an "arginit" typemap. The result is a family of three relevant typemaps for `int *invalue`. Since `%apply` does not overwrite / delete any existing rules, the only way to reset behavior is to delete them, such as with the `%clear` directive. For example:

```
%clear int *invalue;
```

will delete the typemaps for all the typemap methods; namely "in", "check" and "arginit". Alternatively delete each one individually:

```
%typemap(in) int *invalue;
%typemap(check) int *invalue;
%typemap(arginit) int *invalue;
```

### 14.15 Passing data between typemaps

It is also important to note that the primary use of local variables is to create stack-allocated objects for temporary use inside a wrapper function (this is faster and less-prone to error than allocating data on the heap). In general, the variables are not intended to pass information between different types of typemaps. However, this can be done if you realize that local names have the argument number appended to them. For example, you could do this:

```
%typemap(in) int *(int temp) {
  temp = (int) PyInt_AsLong($input);
  $1 = &temp;
}

%typemap(argout) int * {
  PyObject *o = PyInt_FromLong(temp$argnum);
  ...
}
```

In this case, the `$argnum` variable is expanded into the argument number. Therefore, the code will reference the appropriate local such as `temp1` and `temp2`. It should be noted that there are plenty of opportunities to break the universe here and that accessing locals in this manner should probably be avoided. At the very least, you should make sure that the typemaps sharing information have exactly the same types and names.

### 14.16 C++ "this" pointer

All the rules discussed for typemaps apply to C++ as well as C. However in addition C++ passes an extra parameter into every non-static class method -- the `this` pointer. Occasionally it can be useful to apply a typemap to this pointer (for example to check and make sure `this` is non-null before deferencing). Actually, C also has an the equivalent of the `this` pointer which is used when accessing variables in a C struct.

In order to customise the `this` pointer handling, target a variable named `self` in your typemaps. `self` is the name SWIG uses to refer to the extra parameter in wrapped functions.

For example, if wrapping for Java generation:

```
%typemap(check) SWIGTYPE *self %{
if (!$1) {
  SWIG_JavaThrowException(jenv, SWIG_JavaNullPointerException,
    "invalid native object; delete() likely already called");
  return $null;
}
%}
```

In the above case, the `$1` variable is expanded into the argument name that SWIG is using as the `this` pointer. SWIG will then insert the check code before the actual C++ class method is called, and will raise an exception rather than crash the Java virtual machine. The generated code will look something like:

```
  if (!arg1) {
    SWIG_JavaThrowException(jenv, SWIG_JavaNullPointerException,
      "invalid native object; delete() likely already called");
    return ;
  }
  (arg1)->wrappedFunction(...);
```

Note that if you have a parameter named `self` then it will also match the typemap. One work around is to create an interface file that wraps the method, but gives the argument a name other than `self`.

### 14.17 Where to go for more information?

The best place to find out more information about writing typemaps is to look in the SWIG library. Most language modules define all of their default behavior using typemaps. These are found in files such as `python.swg`, `perl5.swg`, `tcl8.swg` and so forth. The `typemaps.i` file in the library also contains numerous examples. You should look at these files to get a feel for how to define typemaps of your own. Some of the language modules support additional typemaps and further information is available in the individual chapters for each target language. There you may also find more hands-on practical examples.

## 15 Customization Features


- Exception handling with %exception
  - Handling exceptions in C code
  - Exception handling with longjmp()
  - Handling C++ exceptions
  - Exception handlers for variables
  - Defining different exception handlers

- Special variables for %exception
- Using The SWIG exception library
- Object ownership and %newobject
- Features and the %feature directive
  - Feature attributes
  - Feature flags
  - Clearing features
  - Features and default arguments
  - Feature example


In many cases, it is desirable to change the default wrapping of particular declarations in an interface. For example, you might want to provide hooks for catching C++ exceptions, add assertions, or provide hints to the underlying code generator. This chapter describes some of these customization techniques. First, a discussion of exception handling is presented. Then, a more general-purpose customization mechanism known as "features" is described.

## 15.1 Exception handling with %exception

The `%exception` directive allows you to define a general purpose exception handler. For example, you can specify the following:

```
%exception {
  try {
    $action
  }
  catch (RangeError) {
    ... handle error ...
  }
}
```

How the exception is handled depends on the target language, for example, Python:

```
%exception {
  try {
    $action
  }
  catch (RangeError) {
    PyErr_SetString(PyExc_IndexError, "index out-of-bounds");
    SWIG_fail;
  }
}
```

When defined, the code enclosed in braces is inserted directly into the low-level wrapper functions. The special variable `$action` is one of a few %exception special variables supported and gets replaced with the actual operation to be performed (a function call, method invocation, attribute access, etc.). An exception handler remains in effect until it is explicitly deleted. This is done by using either `%exception` or `%noexception` with no code. For example:

```
%exception;   // Deletes any previously defined handler
```

### 15.1.1 Handling exceptions in C code

C has no formal exception handling mechanism so there are several approaches that might be used. A somewhat common technique is to simply set a special error code. For example:

```
/* File : except.c */

static char error_message[256];
static int error_status = 0;

void throw_exception(char *msg) {
  strncpy(error_message, msg, 256);
  error_status = 1;
}

void clear_exception() {
  error_status = 0;
}
char *check_exception() {
  if (error_status)
    return error_message;
  else
    return NULL;
}
```

To use these functions, functions simply call `throw_exception()` to indicate an error occurred. For example :

```
double inv(double x) {
  if (x != 0)
    return 1.0/x;
  else {
    throw_exception("Division by zero");
    return 0;
  }
}
```

To catch the exception, you can write a simple exception handler such as the following (shown for Perl5) :

```
%exception {
  char *err;
  clear_exception();
  $action
  if ((err = check_exception())) {
    croak(err);
  }
```

```
}
```

In this case, when an error occurs, it is translated into a Perl error. Each target language has its own approach to creating a runtime error/exception in and for Perl it is the `croak` method shown above.

### 15.1.2 Exception handling with longjmp()

Exception handling can also be added to C code using the `<setjmp.h>` library. Here is a minimalistic implementation that relies on the C preprocessor :

```
/* File : except.c
   Just the declaration of a few global variables we're going to use */

#include <setjmp.h>
jmp_buf exception_buffer;
int exception_status;

/* File : except.h */
#include <setjmp.h>
extern jmp_buf exception_buffer;
extern int exception_status;

#define try if ((exception_status = setjmp(exception_buffer)) == 0)
#define catch(val) else if (exception_status == val)
#define throw(val) longjmp(exception_buffer, val)
#define finally else

/* Exception codes */

#define RangeError     1
#define DivisionByZero 2
#define OutOfMemory    3
```

Now, within a C program, you can do the following :

```
double inv(double x) {
  if (x)
    return 1.0/x;
  else
    throw(DivisionByZero);
}
```

Finally, to create a SWIG exception handler, write the following :

```
%{
#include "except.h"
%}

%exception {
  try {
    $action
  } catch(RangeError) {
    croak("Range Error");
  } catch(DivisionByZero) {
    croak("Division by zero");
  } catch(OutOfMemory) {
    croak("Out of memory");
  } finally {
    croak("Unknown exception");
  }
}
```

Note: This implementation is only intended to illustrate the general idea. To make it work better, you'll need to modify it to handle nested `try` declarations.

### 15.1.3 Handling C++ exceptions

Handling C++ exceptions is also straightforward. For example:

```
%exception {
  try {
    $action
  } catch(RangeError) {
    croak("Range Error");
  } catch(DivisionByZero) {
    croak("Division by zero");
  } catch(OutOfMemory) {
    croak("Out of memory");
  } catch(...) {
    croak("Unknown exception");
  }
}
```

The exception types need to be declared as classes elsewhere, possibly in a header file :

```
class RangeError {};
class DivisionByZero {};
class OutOfMemory {};
```

### 15.1.4 Exception handlers for variables

By default all variables will ignore `%exception`, so it is effectively turned off for all variables wrappers. This applies to global variables, member variables and static member variables. The approach is certainly a logical one when wrapping variables in C. However, in C++, it is quite possible for an exception to be thrown while the variable is being assigned. To ensure `%exception` is used when wrapping variables, it needs to be 'turned on' using the `%allowexception` feature. Note that `%allowexception` is just a macro for `%feature("allowexcept")`, that is, it is a feature called "allowexcept". Any variable which has this feature attached to it, will then use the `%exception` feature, but of course, only if there is a `%exception` attached to the variable in the first place. The `%allowexception` feature works like any other feature and so can be used globally or for selective variables.

```
%allowexception;                // turn on globally
%allowexception Klass::MyVar;   // turn on for a specific variable

%noallowexception Klass::MyVar; // turn off for a specific variable
%noallowexception;              // turn off globally
```

### 15.1.5 Defining different exception handlers

By default, the `%exception` directive creates an exception handler that is used for all wrapper functions that follow it. Unless there is a well-defined (and simple) error handling mechanism in place, defining one universal exception handler may be unwieldy and result in excessive code bloat since the handler is inlined into each wrapper function.

To fix this, you can be more selective about how you use the `%exception` directive. One approach is to only place it around critical pieces of code. For example:

```
%exception {
  ... your exception handler ...
}
/* Define critical operations that can throw exceptions here */

%exception;

/* Define non-critical operations that don't throw exceptions */
```

More precise control over exception handling can be obtained by attaching an exception handler to specific declaration name. For example:

```
%exception allocate {
  try {
    $action
  }
  catch (MemoryError) {
    croak("Out of memory");
  }
}
```

In this case, the exception handler is only attached to declarations named "allocate". This would include both global and member functions. The names supplied to `%exception` follow the same rules as for `%rename` described in the section on Renaming and ambiguity resolution. For example, if you wanted to define an exception handler for a specific class, you might write this:

```
%exception Object::allocate {
  try {
    $action
  }
  catch (MemoryError) {
    croak("Out of memory");
  }
}
```

When a class prefix is supplied, the exception handler is applied to the corresponding declaration in the specified class as well as for identically named functions appearing in derived classes.

`%exception` can even be used to pinpoint a precise declaration when overloading is used. For example:

```
%exception Object::allocate(int) {
  try {
    $action
  }
  catch (MemoryError) {
    croak("Out of memory");
  }
}
```

Attaching exceptions to specific declarations is a good way to reduce code bloat. It can also be a useful way to attach exceptions to specific parts of a header file. For example:

```
%module example
%{
#include "someheader.h"
%}

// Define a few exception handlers for specific declarations
%exception Object::allocate(int) {
  try {
    $action
  }
  catch (MemoryError) {
    croak("Out of memory");
  }
}

%exception Object::getitem {
  try {
    $action
  }
  catch (RangeError) {
    croak("Index out of range");
  }
}
...
// Read a raw header file
%include "someheader.h"
```

### 15.1.6 Special variables for %exception

The %exception directive supports a few special variables which are placeholders for code substitution. The following table shows the available special variables and details what the special variables are replaced with.

| | |
|---|---|
| $action | The actual operation to be performed (a function call, method invocation, variable access, etc.) |
| $decl | The fully qualified C/C++ declaration of the method being wrapped without the return type. |
| $fulldecl | The fully qualified C/C++ declaration of the method being wrapped including the return type. |
| $isvoid | Expands to 1 if the wrapped function has a void return, otherwise expands to 0. Note that it expands to 1 in destructors and 0 in constructors. |
| $name | The C/C++ symbol name for the function. |
| $overname | The extra mangling used in the symbol name for overloaded method. Expands to nothing if the wrapped method is not overloaded. |
| $parentclassname | The parent class name (if any) for a method. |
| $parentclasssymname | The target language parent class name (if any) for a method. |
| $symname | The symbol name used internally by SWIG. |
| $wrapname | The language specific wrapper name (usually a C function name exported from the shared object/dll). |

The special variables are often used in situations where method calls are logged. Exactly which form of the method call needs logging is up to individual requirements, but the example code below shows all the possible expansions, plus how an exception message could be tailored to show the C++ method declaration:

```
%exception Special::something {
  log("symname: $symname");
  log("overname: $overname");
  log("wrapname: $wrapname");
  log("decl: $decl");
  log("fulldecl: $fulldecl");
  try {
    $action
  }
  catch (MemoryError) {
    croak("Out of memory in $decl");
  }
}
void log(const char *message);
struct Special {
  void something(const char *c);
  void something(int i);
};
```

Below shows the expansions for the 1st of the overloaded `something` wrapper methods for Perl:

```
  log("symname: Special_something");
  log("overname: __SWIG_0");
  log("wrapname: _wrap_Special_something__SWIG_0");
  log("decl: Special::something(char const *)");
  log("fulldecl: void Special::something(char const *)");
  try {
    (arg1)->something((char const *)arg2);
  }
  catch (MemoryError) {
    croak("Out of memory in Special::something(char const *)");
  }
```

### 15.1.7 Using The SWIG exception library

The `exception.i` library file provides support for creating language independent exceptions in your interfaces. To use it, simply put an "`%include exception.i`" in your interface file. This provides a function `SWIG_exception()` that can be used to raise common scripting language exceptions in a portable manner. For example :

```
// Language independent exception handler
%include exception.i

%exception {
  try {
    $action
  } catch(RangeError) {
    SWIG_exception(SWIG_ValueError, "Range Error");
  } catch(DivisionByZero) {
    SWIG_exception(SWIG_DivisionByZero, "Division by zero");
  } catch(OutOfMemory) {
    SWIG_exception(SWIG_MemoryError, "Out of memory");
  } catch(...) {
    SWIG_exception(SWIG_RuntimeError, "Unknown exception");
  }
}
```

As arguments, `SWIG_exception()` takes an error type code (an integer) and an error message string. The currently supported error types are :

```
SWIG_UnknownError
SWIG_IOError
SWIG_RuntimeError
SWIG_IndexError
SWIG_TypeError
SWIG_DivisionByZero
SWIG_OverflowError
SWIG_SyntaxError
SWIG_ValueError
SWIG_SystemError
SWIG_AttributeError
SWIG_MemoryError
SWIG_NullReferenceError
```

The `SWIG_exception()` function can also be used in typemaps.

## 15.2 Object ownership and %newobject

A common problem in some applications is managing proper ownership of objects. For example, consider a function like this:

```
Foo *blah() {
  Foo *f = new Foo();
  return f;
}
```

If you wrap the function `blah()`, SWIG has no idea that the return value is a newly allocated object. As a result, the resulting extension module may produce a memory leak (SWIG is conservative and will never delete objects unless it knows for certain that the returned object was newly created).

To fix this, you can provide an extra hint to the code generator using the `%newobject` directive. For example:

```
%newobject blah;
Foo *blah();
```

`%newobject` works exactly like `%rename` and `%exception`. In other words, you can attach it to class members and parameterized declarations as before. For example:

```
%newobject ::blah();                   // Only applies to global blah
%newobject Object::blah(int, double);  // Only blah(int, double) in Object
%newobject *::copy;                    // Copy method in all classes
...
```

When `%newobject` is supplied, many language modules will arrange to take ownership of the return value. This allows the value to be automatically garbage-collected when it is no longer in use. However, this depends entirely on the target language (a language module may also choose to ignore the `%newobject` directive).

Closely related to `%newobject` is a special typemap. The "newfree" typemap can be used to deallocate a newly allocated return value. It is only available on methods for which `%newobject` has been applied and is commonly used to clean-up string results. For example:

```
%typemap(newfree) char * "free($1);"
...
%newobject strdup;
...
char *strdup(const char *s);
```

In this case, the result of the function is a string in the target language. Since this string is a copy of the original result, the data returned by `strdup()` is no longer needed. The "newfree" typemap in the example simply releases this memory.

As a complement to the `%newobject`, from SWIG 1.3.28, you can use the `%delobject` directive. For example, if you have two methods, one to create objects and one to destroy them, you can use:

```
%newobject create_foo;
%delobject destroy_foo;
...
Foo *create_foo();
void destroy_foo(Foo *foo);
```

or in a member method as:

```
%delobject Foo::destroy;

class Foo {
public:
  void destroy() { delete this;}

private:
  ~Foo();
};
```

`%delobject` instructs SWIG that the first argument passed to the method will be destroyed, and therefore, the target language should not attempt to deallocate it twice. This is similar to use the DISOWN typemap in the first method argument, and in fact, it also depends on the target language on implementing the 'disown' mechanism properly.

The use of `%newobject` is also integrated with reference counting and is covered in the C++ reference counted objects section.

## 15.3 Features and the %feature directive

Both `%exception` and `%newobject` are examples of a more general purpose customization mechanism known as "features." A feature is simply a user-definable property that is attached to specific declarations. Features are attached using the `%feature` directive. For example:

```
%feature("except") Object::allocate {
  try {
    $action
  }
  catch (MemoryError) {
    croak("Out of memory");
  }
}

%feature("new", "1") *::copy;
```

In fact, the `%exception` and `%newobject` directives are really nothing more than macros involving `%feature`:

```
#define %exception %feature("except")
#define %newobject %feature("new", "1")
```

The name matching rules outlined in the Renaming and ambiguity resolution section applies to all `%feature` directives. In fact the `%rename` directive is just a special form of `%feature`. The matching rules mean that features are very flexible and can be applied with pinpoint accuracy to specific declarations if needed. Additionally, if no declaration name is given, a global feature is said to be defined. This feature is then attached to *every* declaration that follows. This is how global exception handlers are defined. For example:

```
/* Define a global exception handler */
%feature("except") {
  try {
    $action
  }
  ...
}

... bunch of declarations ...
```

The `%feature` directive can be used with different syntax. The following are all equivalent:

```
%feature("except") Object::method { $action };
%feature("except") Object::method %{ $action %};
%feature("except") Object::method " $action ";
%feature("except", "$action") Object::method;
```

The syntax in the first variation will generate the `{ }` delimiters used whereas the other variations will not.

### 15.3.1 Feature attributes

The `%feature` directive also accepts XML style attributes in the same way that typemaps do. Any number of attributes can be specified. The following is the generic syntax for features:

```
%feature("name", "value", attribute1="AttributeValue1") symbol;
%feature("name", attribute1="AttributeValue1") symbol {value};
%feature("name", attribute1="AttributeValue1") symbol %{value%};
%feature("name", attribute1="AttributeValue1") symbol "value";
```

More than one attribute can be specified using a comma separated list. The Java module is an example that uses attributes in `%feature("except")`. The `throws` attribute specifies the name of a Java class to add to a proxy method's throws clause. In the following example, `MyExceptionClass` is the name of the Java class for adding to the throws clause.

```
%feature("except", throws="MyExceptionClass") Object::method {
  try {
    $action
  } catch (...) {
    ... code to throw a MyExceptionClass Java exception ...
  }
};
```

Further details can be obtained from the Java exception handling section.

### 15.3.2 Feature flags

Feature flags are used to enable or disable a particular feature. Feature flags are a common but simple usage of `%feature` and the feature value should be either `1` to enable or `0` to disable the feature.

```
%feature("featurename")          // enables feature
%feature("featurename", "1")     // enables feature
%feature("featurename", "x")     // enables feature
%feature("featurename", "0")     // disables feature
%feature("featurename", "")      // clears feature
```

Actually any value other than zero will enable the feature. Note that if the value is omitted completely, the default value becomes `1`, thereby enabling the feature. A feature is cleared by specifying no value, see Clearing features. The `%immutable` directive described in the Creating read-only variables section, is just a macro for `%feature("immutable")`, and can be used to demonstrates feature flags:

```
                              // features are disabled by default
int red;                      // mutable

%feature("immutable");        // global enable
int orange;                   // immutable

%feature("immutable", "0");   // global disable
int yellow;                   // mutable

%feature("immutable", "1");   // another form of global enable
int green;                    // immutable

%feature("immutable", "");    // clears the global feature
int blue;                     // mutable
```

Note that features are disabled by default and must be explicitly enabled either globally or by specifying a targeted declaration. The above intersperses SWIG directives with C code. Of course you can target features explicitly, so the above could also be rewritten as:

```
%feature("immutable", "1") orange;
%feature("immutable", "1") green;
int red;                      // mutable
int orange;                   // immutable
int yellow;                   // mutable
int green;                    // immutable
int blue;                     // mutable
```

The above approach allows for the C declarations to be separated from the SWIG directives for when the C declarations are parsed from a C header file. The logic above can of course be inverted and rewritten as:

```
%feature("immutable", "1");
%feature("immutable", "0") red;
%feature("immutable", "0") yellow;
%feature("immutable", "0") blue;
int red;                        // mutable
int orange;                     // immutable
int yellow;                     // mutable
int green;                      // immutable
int blue;                       // mutable
```

As hinted above for `%immutable`, most feature flags can also be specified via alternative syntax. The alternative syntax is just a macro in the `swig.swg` Library file. The following shows the alternative syntax for the imaginary `featurename` feature:

```
%featurename       // equivalent to %feature("featurename", "1") ie enables feature
%nofeaturename     // equivalent to %feature("featurename", "0") ie disables feature
%clearfeaturename  // equivalent to %feature("featurename", "")  ie clears feature
```

The concept of clearing features is discussed next.

### 15.3.3 Clearing features

A feature stays in effect until it is explicitly cleared. A feature is cleared by supplying a `%feature` directive with no value. For example `%feature("name", "")`. A cleared feature means that any feature exactly matching any previously defined feature is no longer used in the name matching rules. So if a feature is cleared, it might mean that another name matching rule will apply. To clarify, let's consider the `except` feature again (`%exception`):

```
// Define global exception handler
%feature("except") {
  try {
    $action
  } catch (...) {
    croak("Unknown C++ exception");
  }
}

// Define exception handler for all clone methods to log the method calls
%feature("except") *::clone() {
  try {
    logger.info("$action");
    $action
  } catch (...) {
    croak("Unknown C++ exception");
  }
}

... initial set of class declarations with clone methods ...

// clear the previously defined feature
%feature("except", "") *::clone();

... final set of class declarations with clone methods ...
```

In the above scenario, the initial set of clone methods will log all method invocations from the target language. This specific feature is cleared for the final set of clone methods. However, these clone methods will still have an exception handler (without logging) as the next best feature match for them is the global exception handler.

Note that clearing a feature is not always the same as disabling it. Clearing the feature above with `%feature("except", "") *::clone()` is not the same as specifying `%feature("except", "0") *::clone()` . The former will disable the feature for clone methods - the feature is still a better match than the global feature. If on the other hand, no global exception handler had been defined at all, then clearing the feature would be the same as disabling it as no other feature would have matched.

Note that the feature must match exactly for it to be cleared by any previously defined feature. For example the following attempt to clear the initial feature will not work:

```
%feature("except") clone() { logger.info("$action"); $action }
%feature("except", "") *::clone();
```

but this will:

```
%feature("except") clone() { logger.info("$action"); $action }
%feature("except", "") clone();
```

SWIG provides macros for disabling and clearing features. Many of these can be found in the `swig.swg` library file. The typical pattern is to define three macros; one to define the feature itself, one to disable the feature and one to clear the feature. The three macros below show this for the "except" feature:

```
#define %exception      %feature("except")
#define %noexception    %feature("except", "0")
#define %clearexception %feature("except", "")
```

### 15.3.4 Features and default arguments

SWIG treats methods with default arguments as separate overloaded methods as detailed in the default arguments section. Any `%feature` targeting a method with default arguments will apply to all the extra overloaded methods that SWIG generates if the default arguments are specified in the feature. If the default arguments are not specified in the feature, then the feature will match that exact wrapper method only and not the extra overloaded methods that SWIG generates. For example:

```
%feature("except") hello(int i=0, double d=0.0) { ... }
void hello(int i=0, double d=0.0);
```

will apply the feature to all three wrapper methods, that is:

```
void hello(int i, double d);
void hello(int i);
```

```
void hello();
```

If the default arguments are not specified in the feature:

```
%feature("except") hello(int i, double d) { ... }
void hello(int i=0, double d=0.0);
```

then the feature will only apply to this wrapper method:

```
void hello(int i, double d);
```

and not these wrapper methods:

```
void hello(int i);
void hello();
```

If compactdefaultargs are being used, then the difference between specifying or not specifying default arguments in a feature is not applicable as just one wrapper is generated.

**Compatibility note:** The different behaviour of features specified with or without default arguments was introduced in SWIG-1.3.23 when the approach to wrapping methods with default arguments was changed.

### 15.3.5 Feature example

As has been shown earlier, the intended use for the `%feature` directive is as a highly flexible customization mechanism that can be used to annotate declarations with additional information for use by specific target language modules. Another example is in the Python module. You might use `%feature` to rewrite proxy/shadow class code as follows:

```
%module example
%rename(bar_id) bar(int, double);

// Rewrite bar() to allow some nice overloading

%feature("shadow") Foo::bar(int) %{
def bar(*args):
    if len(args) == 3:
        return apply(examplec.Foo_bar_id, args)
    return apply(examplec.Foo_bar, args)
%}

class Foo {
public:
  int bar(int x);
  int bar(int x, double y);
}
```

Further details of `%feature` usage is described in the documentation for specific language modules.

# 16 Contracts


- The %contract directive
- %contract and classes
- Constant aggregation and %aggregate_check
- Notes


A common problem that arises when wrapping C libraries is that of maintaining reliability and checking for errors. The fact of the matter is that many C programs are notorious for not providing error checks. Not only that, when you expose the internals of an application as a library, it often becomes possible to crash it simply by providing bad inputs or using it in a way that wasn't intended.

This chapter describes SWIG's support for software contracts. In the context of SWIG, a contract can be viewed as a runtime constraint that is attached to a declaration. For example, you can easily attach argument checking rules, check the output values of a function and more. When one of the rules is violated by a script, a runtime exception is generated rather than having the program continue to execute.

## 16.1 The %contract directive

Contracts are added to a declaration using the %contract directive. Here is a simple example:

```
%contract sqrt(double x) {
require:
  x >= 0;
ensure:
  sqrt >= 0;
}

...
double sqrt(double);
```

In this case, a contract is being added to the `sqrt()` function. The `%contract` directive must always appear before the declaration in question. Within the contract there are two sections, both of which are optional. The `require:` section specifies conditions that must hold before the function is called. Typically, this is used to check argument values. The `ensure:` section specifies conditions that must hold after the function is called. This is often used to check return values or the state of the program. In both cases, the conditions that must hold must be specified as boolean expressions.

In the above example, we're simply making sure that sqrt() returns a non-negative number (if it didn't, then it would be broken in some way).

Once a contract has been specified, it modifies the behavior of the resulting module. For example:

```
>>> example.sqrt(2)
1.4142135623730951
>>> example.sqrt(-2)
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
RuntimeError: Contract violation: require: (arg1>=0)
>>>
```

## 16.2 %contract and classes

The `%contract` directive can also be applied to class methods and constructors. For example:

```
%contract Foo::bar(int x, int y) {
require:
  x > 0;
ensure:
  bar > 0;
}

%contract Foo::Foo(int a) {
require:
  a > 0;
}

class Foo {
public:
  Foo(int);
  int bar(int, int);
};
```

The way in which `%contract` is applied is exactly the same as the `%feature` directive. Thus, any contract that you specified for a base class will also be attached to inherited methods. For example:

```
class Spam : public Foo {
public:
  int bar(int, int);    // Gets contract defined for Foo::bar(int, int)
};
```

In addition to this, separate contracts can be applied to both the base class and a derived class. For example:

```
%contract Foo::bar(int x, int) {
require:
  x > 0;
}

%contract Spam::bar(int, int y) {
require:
  y > 0;
}

class Foo {
public:
  int bar(int, int);   // Gets Foo::bar contract.
};

class Spam : public Foo {
public:
  int bar(int, int);   // Gets Foo::bar and Spam::bar contract
};
```

When more than one contract is applied, the conditions specified in a "require:" section are combined together using a logical-AND operation. In other words conditions specified for the base class and conditions specified for the derived class all must hold. In the above example, this means that both the arguments to `Spam::bar` must be positive.

## 16.3 Constant aggregation and %aggregate_check

Consider an interface file that contains the following code:

```
#define  UP     1
#define  DOWN   2
#define  RIGHT  3
#define  LEFT   4

void move(SomeObject *, int direction, int distance);
```

One thing you might want to do is impose a constraint on the direction parameter to make sure it's one of a few accepted values. To do that, SWIG provides an easy to use macro %aggregate_check() that works like this:

```
%aggregate_check(int, check_direction, UP, DOWN, LEFT, RIGHT);
```

This merely defines a utility function of the form

```
int check_direction(int x);
```

That checks the argument x to see if it is one of the values listed. This utility function can be used in contracts. For example:

```
%aggregate_check(int, check_direction, UP, DOWN, RIGHT, LEFT);

%contract move(SomeObject *, int direction, in) {
require:
  check_direction(direction);
}
```

```
#define  UP     1
#define  DOWN   2
#define  RIGHT  3
#define  LEFT   4

void move(SomeObject *, int direction, int distance);
```

Alternatively, it can be used in typemaps and other directives. For example:

```
%aggregate_check(int, check_direction, UP, DOWN, RIGHT, LEFT);

%typemap(check) int direction {
  if (!check_direction($1)) SWIG_exception(SWIG_ValueError, "Bad direction");
}

#define  UP     1
#define  DOWN   2
#define  RIGHT  3
#define  LEFT   4

void move(SomeObject *, int direction, int distance);
```

Regrettably, there is no automatic way to perform similar checks with enums values. Maybe in a future release.

## 16.4 Notes

Contract support was implemented by Songyan (Tiger) Feng and first appeared in SWIG-1.3.20.

# 17 Variable Length Arguments


- Introduction
- The Problem
- Default varargs support
- Argument replacement using %varargs
- Varargs and typemaps
- Varargs wrapping with libffi
- Wrapping of va_list
- C++ Issues
- Discussion


**(a.k.a, "The horror. The horror.")**

This chapter describes the problem of wrapping functions that take a variable number of arguments. For instance, generating wrappers for the C `printf()` family of functions.

This topic is sufficiently advanced to merit its own chapter. In fact, support for varargs is an often requested feature that was first added in SWIG-1.3.12. Most other wrapper generation tools have wisely chosen to avoid this issue.

## 17.1 Introduction

Some C and C++ programs may include functions that accept a variable number of arguments. For example, most programmers are familiar with functions from the C library such as the following:

```
int printf(const char *fmt, ...)
int fprintf(FILE *, const char *fmt, ...);
int sprintf(char *s, const char *fmt, ...);
```

Although there is probably little practical purpose in wrapping these specific C library functions in a scripting language (what would be the point?), a library may include its own set of special functions based on a similar API. For example:

```
int  traceprintf(const char *fmt, ...);
```

In this case, you may want to have some kind of access from the target language.

Before describing the SWIG implementation, it is important to discuss the common uses of varargs that you are likely to encounter in real programs. Obviously, there are the `printf()` style output functions as shown. Closely related to this would be `scanf()` style input functions that accept a format string and a list of pointers into which return values are placed. However, variable length arguments are also sometimes used to write functions that accept a NULL-terminated list of pointers. A good example of this would be a function like this:

```
int execlp(const char *path, const char *arg1, ...);
...

/* Example */
execlp("ls", "ls", "-l", NULL);
```

In addition, varargs is sometimes used to fake default arguments in older C libraries. For instance, the low level `open()` system call is often declared as a varargs function so that it will accept two or three arguments:

```
int open(const char *path, int oflag, ...);
...

/* Examples */
f = open("foo", O_RDONLY);
g = open("bar", O_WRONLY | O_CREAT, 0644);
```

Finally, to implement a varargs function, recall that you have to use the C library functions defined in `<stdarg.h>`. For example:

```
List make_list(const char *s, ...) {
  va_list ap;
  List    x;
  ...
  va_start(ap, s);
  while (s) {
    x.append(s);
    s = va_arg(ap, const char *);
  }
  va_end(ap);
  return x;
}
```

## 17.2 The Problem

Generating wrappers for a variable length argument function presents a number of special challenges. Although C provides support for implementing functions that receive variable length arguments, there are no functions that can go in the other direction. Specifically, you can't write a function that dynamically creates a list of arguments and which invokes a varargs function on your behalf.

Although it is possible to write functions that accept the special type `va_list`, this is something entirely different. You can't take a `va_list` structure and pass it in place of the variable length arguments to another varargs function. It just doesn't work.

The reason this doesn't work has to do with the way that function calls get compiled. For example, suppose that your program has a function call like this:

```
printf("Hello %s. Your number is %d\n", name, num);
```

When the compiler looks at this, it knows that you are calling `printf()` with exactly three arguments. Furthermore, it knows that the number of arguments as well are their types and sizes is *never* going to change during program execution. Therefore, this gets turned to machine code that sets up a three-argument stack frame followed by a call to `printf()`.

In contrast, suppose you attempted to make some kind of wrapper around `printf()` using code like this:

```
int wrap_printf(const char *fmt, ...) {
  va_list ap;
  va_start(ap, fmt);
  ...
  printf(fmt, ap);
  ...
  va_end(ap);
};
```

Although this code might compile, it won't do what you expect. This is because the call to `printf()` is compiled as a procedure call involving only two arguments. However, clearly a two-argument configuration of the call stack is completely wrong if your intent is to pass an arbitrary number of arguments to the real `printf()`. Needless to say, it won't work.

Unfortunately, the situation just described is exactly the problem faced by wrapper generation tools. In general, the number of passed arguments will not be known until run-time. To make matters even worse, you won't know the types and sizes of arguments until run-time as well. Needless to say, there is no obvious way to make the C compiler generate code for a function call involving an unknown number of arguments of unknown types.

In theory, it *is* possible to write a wrapper that does the right thing. However, this involves knowing the underlying ABI for the target platform and language as well as writing special purpose code that manually constructed the call stack before making a procedure call. Unfortunately, both of these tasks require the use of inline assembly code. Clearly, that's the kind of solution you would much rather avoid.

With this nastiness in mind, SWIG provides a number of solutions to the varargs wrapping problem. Most of these solutions are compromises that provide limited varargs support without having to resort to assembly language. However, SWIG can also support real varargs wrapping (with stack-frame manipulation) if you are willing to get hands dirty. Keep reading.

## 17.3 Default varargs support

When variable length arguments appear in an interface, the default behavior is to drop the variable argument list entirely, replacing them with a single NULL pointer. For example, if you had this function,

```
void traceprintf(const char *fmt, ...);
```

it would be wrapped as if it had been declared as follows:

```
void traceprintf(const char *fmt);
```

When the function is called inside the wrappers, it is called as follows:

```
traceprintf(arg1, NULL);
```

Arguably, this approach seems to defeat the whole point of variable length arguments. However, this actually provides enough support for many simple kinds of varargs functions to still be useful, however it does come with a caveat. For instance, you could make function calls like this (in Python):

```
>>> traceprintf("Hello World")
>>> traceprintf("Hello %s. Your number is %d\n" % (name, num))
>>> traceprintf("Your result is 90%%.")
```

Notice how string formatting is being done in Python instead of C. The caveat is the strings passed must be safe to use in C though. For example if name was to contain a "%" it should be double escaped in order to avoid unpredictable behaviour:

```
>>> traceprintf("Your result is 90%.\n")  # unpredictable behaviour
>>> traceprintf("Your result is 90%%.\n") # good
```

Read on for further solutions.

## 17.4 Argument replacement using %varargs

Instead of dropping the variable length arguments, an alternative approach is to replace `(...)` with a set of suitable arguments. SWIG provides a special `%varargs` directive that can be used to do this. For example,

```
%varargs(int mode = 0) open;
...
int open(const char *path, int oflags, ...);
```

is equivalent to this:

```
int open(const char *path, int oflags, int mode = 0);
```

In this case, `%varargs` is simply providing more specific information about the extra arguments that might be passed to a function. If the arguments to a varargs function are of uniform type, `%varargs` can also accept a numerical argument count as follows:

```
%varargs(3, char *str = NULL) execlp;
...
int execlp(const char *path, const char *arg, ...);
```

and is effectively seen as:

```
int execlp(const char *path, const char *arg,
           char *str1 = NULL,
           char *str2 = NULL,
           char *str3 = NULL);
```

This would wrap `execlp()` as a function that accepted up to 3 optional arguments. Depending on the application, this may be more than enough for practical purposes.

The handling of default arguments can be changed via the `compactdefaultargs` feature. If this feature is used, for example

```
%feature("compactdefaultargs") execlp;
%varargs(3, char *str = NULL) execlp;
...
int execlp(const char *path, const char *arg, ...);
```

a call from the target language which does not provide the maximum number of arguments, such as, `execlp("a", "b", "c")` will generate C code which includes the missing default values, that is, `execlp("a", "b", "c", NULL, NULL)`. If `compactdefaultargs` is not used, then the generated code will be `execlp("a", "b", "c")` . The former is useful for helping providing a sentinel to terminate the argument list. However, this is not guaranteed, for example when a user passes a non-NULL value for all the parameters. When using `compactdefaultargs` it is possible to guarantee the NULL sentinel is passed through the, `numinputs=0` 'in' typemap attribute, naming the **last parameter**. For example,

```
%feature("compactdefaultargs") execlp;
%varargs(3, char *str = NULL) execlp;
%typemap(in, numinputs=0) char *str3 ""
...
int execlp(const char *path, const char *arg, ...);
```

Note that `str3` is the name of the last argument, as we have used `%varargs` with 3. Now `execlp("a", "b", "c", "d", "e")` will result in an error as one too many arguments has been passed, as now only 2 additional 'str' arguments can be passed with the 3rd one always using the specified default `NULL`.

Argument replacement is most appropriate in cases where the types of the extra arguments are uniform and the maximum number of arguments are known. Argument replacement is not as useful when working with functions that accept mixed argument types such as `printf()`. Providing general purpose wrappers to such functions presents special problems (covered shortly).

## 17.5 Varargs and typemaps

Variable length arguments may be used in typemap specifications. For example:

```
%typemap(in) (...) {
  // Get variable length arguments (somehow)
  ...
}

%typemap(in) (const char *fmt, ...) {
    // Multi-argument typemap
}
```

However, this immediately raises the question of what "type" is actually used to represent `(...)`. For lack of a better alternative, the type of `(...)` is set to `void *`. Since there is no way to dynamically pass arguments to a varargs function (as previously described), the `void *` argument value is intended to serve as a place holder for storing some kind of information about the extra arguments (if any). In addition, the default behavior of SWIG is to pass the `void *` value as an argument to the function. Therefore, you could use the pointer to hold a valid argument value if you wanted.

To illustrate, here is a safer version of wrapping `printf()` in Python:

```
%typemap(in) (const char *fmt, ...) {
    $1 = "%s";                                /* Fix format string to %s */
    $2 = (void *) PyString_AsString($input);  /* Get string argument */
};
...
int printf(const char *fmt, ...);
```

In this example, the format string is implicitly set to `"%s"` . This prevents a program from passing a bogus format string to the extension. Then, the passed input object is decoded and placed in the `void *` argument defined for the `(...)` argument. When the actual function call is made, the underlying wrapper code will look roughly like this:

```
wrap_printf() {
  char *arg1;
  void *arg2;
  int   result;
```

```
  arg1 = "%s";
  arg2 = (void *) PyString_AsString(arg2obj);
  ...
  result = printf(arg1, arg2);
  ...
}
```

Notice how both arguments are passed to the function and it does what you would expect.

The next example illustrates a more advanced kind of varargs typemap. Disclaimer: this requires special support in the target language module and is not guaranteed to work with all SWIG modules at this time. It also starts to illustrate some of the more fundamental problems with supporting varargs in more generality.

If a typemap is defined for any form of `(...)`, many SWIG modules will generate wrappers that accept a variable number of arguments as input and will make these arguments available in some form. The precise details of this depends on the language module being used (consult the appropriate chapter for more details). However, suppose that you wanted to create a Python wrapper for the `execlp()` function shown earlier. To do this using a typemap instead of using `%varargs`, you might first write a typemap like this:

```
%typemap(in) (...)(char *vargs[10]) {
  int i;
  Py_ssize_t argc;
  for (i = 0; i < 10; i++) vargs[i] = 0;
  argc = PyTuple_Size(varargs);
  if (argc > 10) {
    PyErr_SetString(PyExc_ValueError, "Too many arguments");
    SWIG_fail;
  }
  for (i = 0; i < argc; i++) {
    PyObject *pyobj = PyTuple_GetItem(varargs, i);
    char *str = 0;
%#if PY_VERSION_HEX>=0x03000000
    const char *strtmp = 0;
    PyObject *pystr;
    if (!PyUnicode_Check(pyobj)) {
      PyErr_SetString(PyExc_ValueError, "Expected a string");
      SWIG_fail;
    }
    pystr = PyUnicode_AsUTF8String(pyobj);
    if (!pystr) {
      SWIG_fail;
    }
    strtmp = PyBytes_AsString(pystr);
    str = (char *)malloc(strlen(strtmp) + 1);
    if (str)
      strcpy(str, strtmp);
    Py_DecRef(pystr);
%#else
    if (!PyString_Check(pyobj)) {
      PyErr_SetString(PyExc_ValueError, "Expected a string");
      SWIG_fail;
    }
    str = PyString_AsString(pyobj);
%#endif
    vargs[i] = str;
  }
  $1 = (void *)vargs;
}

%typemap(freearg) (...) {
%#if PY_VERSION_HEX>=0x03000000
  int i;
  for (i = 0; i < 10; i++) {
    free(vargs$argnum[i]);
  }
%#endif
}
```

In the 'in' typemap, the special variable `varargs` is a tuple holding all of the extra arguments passed (this is specific to the Python module). The typemap then pulls this apart and sticks the values into the array of strings `args`. Then, the array is assigned to `$1` (recall that this is the `void *` variable corresponding to `(...)`). However, this assignment is only half of the picture---clearly this alone is not enough to make the function work. The 'freearg' typemap cleans up memory allocated in the 'in' typemap; this code is generated to be called after the `execlp` function is called. To patch everything up, you have to rewrite the underlying action code using the `%feature` directive like this:

```
%feature("action") execlp {
  char **vargs = (char **) arg3;
  result = execlp(arg1, arg2, vargs[0], vargs[1], vargs[2], vargs[3], vargs[4],
                  vargs[5], vargs[6], vargs[7], vargs[8], vargs[9], NULL);
}

int execlp(const char *path, const char *arg, ...);
```

This patches everything up and creates a function that more or less works. However, don't try explaining this to your coworkers unless you know for certain that they've had several cups of coffee. If you really want to elevate your guru status and increase your job security, continue to the next section.

## 17.6 Varargs wrapping with libffi

All of the previous examples have relied on features of SWIG that are portable and which don't rely upon any low-level machine-level details. In many ways, they have all dodged the real issue of variable length arguments by recasting a varargs function into some weaker variation with a fixed number of arguments of known types. In many cases, this works perfectly fine. However, if you want more generality than this, you need to bring out some bigger guns.

One way to do this is to use a special purpose library such as libffi ( https://www.sourceware.org/libffi/). libffi is a library that allows you to dynamically construct call-stacks and invoke procedures in a relatively platform independent manner. Details about the library can be found in the libffi distribution and are not repeated here.

To illustrate the use of libffi, suppose that you *really* wanted to create a wrapper for `execlp()` that accepted *any* number of arguments. To do this, you might make a few adjustments to the previous example. For example:

```
/* Take an arbitrary number of extra arguments and place into an array
   of strings */
```

```
%typemap(in) (...) {
  char **argv;
  int    argc;
  int    i;

  argc = PyTuple_Size(varargs);
  argv = (char **) malloc(sizeof(char *)*(argc+1));
  for (i = 0; i < argc; i++) {
    PyObject *o = PyTuple_GetItem(varargs, i);
    if (!PyString_Check(o)) {
      free(argv);
      PyErr_SetString(PyExc_ValueError, "Expected a string");
      SWIG_fail;
    }
    argv[i] = PyString_AsString(o);
  }
  argv[i] = NULL;
  $1 = (void *) argv;
}

/* Rewrite the function call, using libffi */

%feature("action") execlp {
  int       i, vc;
  ffi_cif   cif;
  ffi_type  **types;
  void      **values;
  char      **args;

  vc = PyTuple_Size(varargs);
  types  = (ffi_type **) malloc((vc+3)*sizeof(ffi_type *));
  values = (void **) malloc((vc+3)*sizeof(void *));
  args   = (char **) arg3;

  /* Set up path parameter */
  types[0] = &ffi_type_pointer;
  values[0] = &arg1;

  /* Set up first argument */
  types[1] = &ffi_type_pointer;
  values[1] = &arg2;

  /* Set up rest of parameters */
  for (i = 0; i <= vc; i++) {
    types[2+i] = &ffi_type_pointer;
    values[2+i] = &args[i];
  }
  if (ffi_prep_cif(&cif, FFI_DEFAULT_ABI, vc+3,
                   &ffi_type_uint, types) == FFI_OK) {
    ffi_call(&cif, (void (*)()) execlp, &result, values);
  } else {
    free(types);
    free(values);
    free(arg3);
    PyErr_SetString(PyExc_RuntimeError, "Whoa!!!!!");
    SWIG_fail;
  }
  free(types);
  free(values);
  free(arg3);
}

/* Declare the function. Whew! */
int execlp(const char *path, const char *arg1, ...);
```

Looking at this example, you may start to wonder if SWIG is making life any easier. Given the amount of code involved, you might also wonder why you didn't just write a hand-crafted wrapper! Either that or you're wondering "why in the hell am I trying to wrap this varargs function in the first place?!?" Obviously, those are questions you'll have to answer for yourself.

As a more extreme example of libffi, here is some code that attempts to wrap `printf()`,

```
/* A wrapper for printf() using libffi */

%{
/* Structure for holding passed arguments after conversion */
  typedef struct {
    int type;
    union {
      int    ivalue;
      double dvalue;
      void   *pvalue;
    } val;
  } vtype;
  enum { VT_INT, VT_DOUBLE, VT_POINTER };
%}

%typemap(in) (const char *fmt, ...) {
  vtype *argv;
  int    argc;
  int    i;

  /* Format string */
  $1 = PyString_AsString($input);

  /* Variable length arguments */
  argc = PyTuple_Size(varargs);
  argv = (vtype *) malloc(argc*sizeof(vtype));
  for (i = 0; i < argc; i++) {
    PyObject *o = PyTuple_GetItem(varargs, i);
    if (PyInt_Check(o)) {
      argv[i].type = VT_INT;
```

```
      argv[i].val.ivalue = PyInt_AsLong(o);
    } else if (PyFloat_Check(o)) {
      argv[i].type = VT_DOUBLE;
      argv[i].val.dvalue = PyFloat_AsDouble(o);
    } else if (PyString_Check(o)) {
      argv[i].type = VT_POINTER;
      argv[i].val.pvalue = (void *) PyString_AsString(o);
    } else {
      free(argv);
      PyErr_SetString(PyExc_ValueError, "Unsupported argument type");
      return NULL;
    }
  }
  $2 = (void *) argv;
}

/* Rewrite the function call using libffi */
%feature("action") printf {
  int       i, vc;
  ffi_cif   cif;
  ffi_type  **types;
  void      **values;
  vtype     *args;

  vc = PyTuple_Size(varargs);
  types  = (ffi_type **) malloc((vc+1)*sizeof(ffi_type *));
  values = (void **) malloc((vc+1)*sizeof(void *));
  args   = (vtype *) arg2;

  /* Set up fmt parameter */
  types[0] = &ffi_type_pointer;
  values[0] = &arg1;

  /* Set up rest of parameters */
  for (i = 0; i < vc; i++) {
    switch(args[i].type) {
    case VT_INT:
      types[1+i] = &ffi_type_uint;
      values[1+i] = &args[i].val.ivalue;
      break;
    case VT_DOUBLE:
      types[1+i] = &ffi_type_double;
      values[1+i] = &args[i].val.dvalue;
      break;
    case VT_POINTER:
      types[1+i] = &ffi_type_pointer;
      values[1+i] = &args[i].val.pvalue;
      break;
    default:
      abort();    /* Whoa! We're seriously hosed */
      break;
    }
  }
  if (ffi_prep_cif(&cif, FFI_DEFAULT_ABI, vc+1,
                   &ffi_type_uint, types) == FFI_OK) {
    ffi_call(&cif, (void (*)()) printf, &result, values);
  } else {
    free(types);
    free(values);
    free(args);
    PyErr_SetString(PyExc_RuntimeError, "Whoa!!!!!");
    SWIG_fail;
  }
  free(types);
  free(values);
  free(args);
}

/* The function */
int printf(const char *fmt, ...);
```

Much to your amazement, it even seems to work if you try it:

```
>>> import example
>>> example.printf("Grade: %s   %d/60 = %0.2f%%\n", "Dave", 47, 47.0*100/60)
Grade: Dave   47/60 = 78.33%
>>>
```

Of course, there are still some limitations to consider:

```
>>> example.printf("la de da de da %s", 42)
Segmentation fault (core dumped)
```

And, on this note, we leave further exploration of libffi to the reader as an exercise. Although Python has been used as an example, most of the techniques in this section can be extrapolated to other language modules with a bit of work. The only details you need to know is how the extra arguments are accessed in each target language. For example, in the Python module, we used the special `varargs` variable to get these arguments. Modules such as Tcl8 and Perl5 simply provide an argument number for the first extra argument. This can be used to index into an array of passed arguments to get values. Please consult the chapter on each language module for more details.

## 17.7 Wrapping of va_list

Closely related to variable length argument wrapping, you may encounter functions that accept a parameter of type `va_list`. For example:

```
int vprintf(const char *fmt, va_list ap);
```

As far as we know, there is no obvious way to wrap these functions with SWIG. This is because there is no documented way to assemble the proper va_list structure (there are no C library

functions to do it and the contents of va_list are opaque). Not only that, the contents of a `va_list` structure are closely tied to the underlying call-stack. It's not clear that exporting a `va_list` would have any use or that it would work at all.

A workaround can be implemented by writing a simple varargs C wrapper and then using the techniques discussed earlier in this chapter for varargs. Below is a simple wrapper for `vprintf` renamed so that it can still be called as `vprintf` from your target language. The `%varargs` used in the example restricts the function to taking one string argument.

```
%{
int vprintf(const char *fmt, va_list ap);
%}

%varargs(const char *) my_vprintf;
%rename(vprintf) my_vprintf;

%inline %{
int my_vprintf(const char *fmt, ...) {
  va_list ap;
  int result;

  va_start(ap, fmt);
  result = vprintf(fmt, ap);
  va_end(ap);
  return result;
}
%}
```

## 17.8 C++ Issues

Wrapping of C++ member functions that accept a variable number of arguments presents a number of challenges. By far, the easiest way to handle this is to use the `%varargs` directive. This is portable and it fully supports classes much like the `%rename` directive. For example:

```
%varargs (10, char * = NULL) Foo::bar;

class Foo {
public:
  virtual void bar(char *arg, ...);   // gets varargs above
};

class Spam: public Foo {
public:
  virtual void bar(char *arg, ...);   // gets varargs above
};
```

`%varargs` also works with constructors, operators, and any other C++ programming construct that accepts variable arguments.

Doing anything more advanced than this is likely to involve a serious world of pain. In order to use a library like libffi, you will need to know the underlying calling conventions and details of the C++ ABI. For instance, the details of how `this` is passed to member functions as well as any hidden arguments that might be used to pass additional information. These details are implementation specific and may differ between compilers and even different versions of the same compiler. Also, be aware that invoking a member function is further complicated if it is a virtual method. In this case, invocation might require a table lookup to obtain the proper function address (although you might be able to obtain an address by casting a bound pointer to a pointer to function as described in the C++ ARM section 18.3.4).

If you do decide to change the underlying action code, be aware that SWIG always places the `this` pointer in `arg1`. Other arguments are placed in `arg2`, `arg3`, and so forth. For example:

```
%feature("action") Foo::bar {
  ...
  result = arg1->bar(arg2, arg3, etc.);
  ...
}
```

Given the potential to shoot yourself in the foot, it is probably easier to reconsider your design or to provide an alternative interface using a helper function than it is to create a fully general wrapper to a varargs C++ member function.

## 17.9 Discussion

This chapter has provided a number of techniques that can be used to address the problem of variable length argument wrapping. If you care about portability and ease of use, the `%varargs` directive is probably the easiest way to tackle the problem. However, using typemaps, it is possible to do some very advanced kinds of wrapping.

One point of discussion concerns the structure of the libffi examples in the previous section. Looking at that code, it is not at all clear that this is the easiest way to solve the problem. However, there are a number of subtle aspects of the solution to consider--mostly concerning the way in which the problem has been decomposed. First, the example is structured in a way that tries to maintain separation between wrapper-specific information and the declaration of the function itself. The idea here is that you might structure your interface like this:

```
%typemap(const char *fmt, ...) {
  ...
}
%feature("action") traceprintf {
  ...
}

/* Include some header file with traceprintf in it */
%include "someheader.h"
```

Second, careful scrutiny will reveal that the typemaps involving `(...)` have nothing whatsoever to do with the libffi library. In fact, they are generic with respect to the way in which the function is actually called. This decoupling means that it will be much easier to consider other library alternatives for making the function call. For instance, if libffi wasn't supported on a certain platform, you might be able to use something else instead. You could use conditional compilation to control this:

```
#ifdef USE_LIBFFI
%feature("action") printf {
  ...
}
#endif
#ifdef USE_OTHERFFI
%feature("action") printf {
...
```

```
}
#endif
```

Finally, even though you might be inclined to just write a hand-written wrapper for varargs functions, the techniques used in the previous section have the advantage of being compatible with all other features of SWIG such as exception handling.

As a final word, some C programmers seem to have the assumption that the wrapping of variable length argument functions is an easily solved problem. However, this section has hopefully dispelled some of these myths. All things being equal, you are better off avoiding variable length arguments if you can. If you can't avoid them, please consider some of the simple solutions first. If you can't live with a simple solution, proceed with caution. At the very least, make sure you carefully read the section "A7.3.2 Function Calls" in Kernighan and Ritchie and make sure you fully understand the parameter passing conventions used for varargs. Also, be aware of the platform dependencies and reliability issues that this will introduce. Good luck.

# 18 SWIG and Doxygen Translation


- Doxygen translation overview
- Preparations
  - Enabling Doxygen translation
  - Doxygen-specific %feature directives
    - doxygen:notranslate
    - doxygen:alias:<command-name>
    - doxygen:ignore:<command-name>
    - doxygen:nolinktranslate
    - doxygen:nostripparams
  - Additional command line options
- Doxygen to Javadoc
  - Basic Javadoc example
  - Javadoc tags
  - Unsupported tags for Javadoc
- Doxygen to Pydoc
  - Basic Pydoc example
  - Pydoc translator
  - Unsupported tags for Pydoc
- Doxygen to XML C# documentation
  - Basic C# example
  - C# tags
  - Unsupported tags for C#
- Troubleshooting
  - Problem with conditional compilation
- Developer information
  - Doxygen translator design
  - Debugging the Doxygen parser and translator
  - Tests
- Extending to other languages


This chapter describes SWIG's support for translating Doxygen comments found in interface and header files into a target language's normal documentation language. Currently only Javadoc and Pydoc is supported.

## 18.1 Doxygen translation overview

The Doxygen Translation module of SWIG adds an extra layer of functionality to SWIG, allowing automated translation of Doxygen formatted comments from input files into a documentation language more suited for the target language. Currently this module only translates into Javadoc and Pydoc for the SWIG Java and Python modules. Other extensions could be added at a later date. The Doxygen Translation module originally started as a Google Summer of Code proposal from Summer 2008.

## 18.2 Preparations

To make use of the comment translation system, your documentation comments must be in properly formatted Doxygen. Doxygen comments can be present in your main SWIG interface file or any header file that it imports. You are advised to be validate that your comments compile properly with Doxygen before you try to translate them. Doxygen itself is a more comprehensive tool and can provide you better feedback for correcting any syntax errors that may be present. Please look at Doxygen's Documenting the code for the full comment format specifications. However, SWIG's Doxygen parser will still report many errors and warnings found in comments (like unterminated strings or missing ending tags).

Currently, the whole subset of Doxygen comment styles is supported (See Documenting the code). Here they are:

```
/**
 * Javadoc style comment, multiline
 */
/*!
 * QT-style comment, multiline
 */
/**
 Any of the above, but without intermediate *'s
 */
/// Single-line comment
//! Another single-line comment
```

Also any of the above with '<' added after comment-starting symbol, like `/**<`, `/*!<`, `///<`, or `//!<` will be treated as a post-comment and will be assigned to the code before the comment. Any number of '*' or '/' within a Doxygen comment is considered to be a separator and is not included in the final comment, so you may safely use comments like `/*********/` or `//////////`.

Please note, as SWIG parses the input file by itself with strict grammar, there is only a limited support for various cases of comment placement in the file.

Comments can be placed before C/C++ expressions on separate lines:

```
/**
 * Some comment
 */
void someOtherFunction();
/**
 * Some comment
 */
void someFunction();

class Shape {
  /*
   * Calculate the area in cm^2
```

```
   */
  int getArea();
}
```

After C/C++ expressions at the end of the line:

```
int someVariable = 9; ///< This is a var holding magic number 9
void doNothing(); ///< This does nothing, nop
```

and in some special cases, like function parameter comments:

```
void someFunction(
         int a ///< Some parameter
     );
```

or enum element comments:

```
enum E_NUMBERS
{
    EN_ZERO, ///< The first enum item, gets zero as its value
    EN_ONE, ///< The second, EN_ONE=1
    EN_THREE
};
```

Currently only comments directly before or after the code items are supported. Doxygen also supports comments containing structural commands, where the comments for a code item are not put directly before or after the code item. These structural commands are stripped out by SWIG and are not assigned to anything.

### 18.2.1 Enabling Doxygen translation

Doxygen comments translation is disabled by default and needs to be explicitly enabled using the command line `-doxygen` option for the languages that do support it (currently Java and Python).

### 18.2.2 Doxygen-specific %feature directives

Translation of Doxygen comments is influenced by the following %feature directives:

#### 18.2.2.1 doxygen:notranslate

Turns off translation of Doxygen comments to the target language syntax: the original comment will be copied to the output unchanged. This is useful if you want to use Doxygen itself to generate documentation for the target language instead of the corresponding language tool (`javadoc`, `sphinx`, ...).

#### 18.2.2.2 doxygen:alias:<command-name>

Specify an alias for a Doxygen command with the given name. This can be useful for custom Doxygen commands which can be defined using `ALIASES` option for Doxygen itself but which are unknown to SWIG. `"command-name"` is the name of the command in the Doxyfile, e.g. if it contains

```
ALIASES = "sideeffect=\par Side Effects:\n"
```

Then you could also specify the same expansion for SWIG with:

```
%feature("doxygen:alias:sideeffect") "\par Side Effects:\n"
```

Please note that command arguments are not currently supported with this feature.

Notice that it is perfectly possible and potentially useful to define the alias expansion differently depending on the target language, e.g. with

```
#ifdef SWIGJAVA
%feature("doxygen:alias:not_for_java") "This functionality is not available for Java"
#else
%feature("doxygen:alias:not_for_java") ""
#endif
```

you could use `@not_for_java` in the documentation comments of all functions which can't, for whatever reason, be currently exposed in Java wrappers of the C++ API.

#### 18.2.2.3 doxygen:ignore:<command-name>

This feature makes it possible to just ignore an unknown Doxygen command, instead of replacing it with the predefined text that `doxygen:alias` does. For example, you could use

```
%feature("doxygen:ignore:transferfull") Fantastic();
/**
    A fantastic function.

    @transferfull Command ignored, but anything here is still included.
 */
int * Fantastic();
```

if you use a custom Doxygen `transferfull` command to indicate that the return value ownership is transferred to the caller, as this information doesn't make much sense for the other languages without explicit ownership management.

Doxygen syntax is rather rich and, in addition to simple commands such as `@transferfull`, it is also possible to define commands with arguments. As explained in Doxygen documentation, the arguments can have a range of a single word, everything until the end of line or everything until the end of the next paragraph. Currently, only the "end of line" case is supported using the `range="line"` argument of the feature directive:

```
// Ignore occurrences of
//
//    @compileroptions Some special C++ compiler options.
//
```

```
// in Doxygen comments as C++ options are not interesting for the target language
// developers.
%feature("doxygen:ignore:compileroptions", range="line") Amazing();

/**
    An amazing function.

    @compileroptions This function must be compiled with /EHa when using MSVC.
 */
void Amazing();
```

In addition, it is also possible to have custom pairs of begin/end tags, similarly to the standard Doxygen `@code/@endcode`, for example. Such tags can also be ignored using the special value of `range` starting with `end` to indicate that the range is an interval, for example:

```
%feature("doxygen:ignore:forcpponly", range="end"); // same as "end:endforcpponly"
/**
    An incredible function.

    @forcpponly
    This is C++-specific.
    @endforcpponly
 */
void Incredible();
```

would ignore everything between `@forcpponly` and `@endforcpponly` commands in Doxygen comments. By default, the name of the end command is the same as of the start one with "end" prefix, following Doxygen conventions, but this can be overridden by providing the end command name after the colon.

This example shows how custom tags can be used to bracket anything specific to C++ and prevent it from appearing in the target language documentation. Conversely, another pair of custom tags could be used to put target language specific information in the C++ comments. In this case, only the custom tags themselves should be ignored, but their contents should be parsed as usual and `contents="parse"` can be used for this:

```
%feature("doxygen:ignore:beginPythonOnly", range="end:endPythonOnly", contents="parse");
/**
    A splendid function.

    @beginPythonOnly
    This is specific to @b Python.
    @endPythonOnly
 */
void Splendid();
```

Putting everything together, if these directives are in effect:

```
%feature("doxygen:ignore:transferfull");
%feature("doxygen:ignore:compileroptions", range="line");
%feature("doxygen:ignore:forcpponly", range="end");
%feature("doxygen:ignore:beginPythonOnly", range="end:endPythonOnly", contents="parse");
```

then the following C++ Doxygen comment:

```
/**
    A contrived example of ignoring too many commands in one comment.

    @forcpponly
    This is C++-specific.
    @endforcpponly

    @beginPythonOnly
    This is specific to @b Python.
    @endPythonOnly

    @transferfull Command ignored, but anything here is still included.

    @compileroptions This function must be compiled with /EHa when using MSVC.
 */
int * Contrived();
```

would be translated to this comment in Python:

```
def func():
    r"""
    A contrived example of ignoring too many commands in one comment.

    This is specific to **Python**.

    Command ignored, but anything here is still included.
    """
    ...
```

#### 18.2.2.4 doxygen:nolinktranslate

Turn off automatic link-objects translation. This is only applicable to Java at the moment.

#### 18.2.2.5 doxygen:nostripparams

Turn off stripping of `@param` and `@tparam` Doxygen commands if the parameter is not found in the function signature. This is only applicable to Java at the moment.

### 18.2.3 Additional command line options

ALSO TO BE ADDED (Javadoc auto brief?)

## 18.3 Doxygen to Javadoc

If translation is enabled, Javadoc formatted comments should be automatically placed in the correct locations in the resulting module and proxy files.

### 18.3.1 Basic Javadoc example

Here is an example segment from an included header file

```
/*! This is describing class Shape
 \author Bob
 */

class Shape {
public:
  Shape() {
    nshapes++;
  }
  virtual ~Shape() {
    nshapes--;
  };
  double  x, y; /*!< Important Variables */
  void    move(double dx, double dy); /*!< Moves the Shape */
  virtual double area(void) = 0; /*!< \return the area */
  virtual double perimeter(void) = 0; /*!< \return the perimeter */
  static  int nshapes;
};
```

Simply running SWIG should result in the following code being present in Shapes.java

```


/**
 * This is describing class Shape
 * @author Bob
 *
 */

public class Shape {

...

/**
 * Important Variables
 */
  public void setX(double value) {
    ShapesJNI.Shape_x_set(swigCPtr, this, value);
  }

/**
 * Important Variables
 */
  public double getX() {
    return ShapesJNI.Shape_x_get(swigCPtr, this);
  }

/**
 * Moves the Shape
 */
  public void move(double dx, double dy) {
    ShapesJNI.Shape_move(swigCPtr, this, dx, dy);
  }

/**
 * @return the area
 */
  public double area() {
    return ShapesJNI.Shape_area(swigCPtr, this);
  }

/**
 * @return the perimeter
 */
  public double perimeter() {
    return ShapesJNI.Shape_perimeter(swigCPtr, this);
  }
}


```

The code Java-wise should be identical to what would have been generated without the doxygen functionality enabled. When the Doxygen Translator module encounters a comment that contains nothing useful or a doxygen comment that it cannot parse, it will not affect the functionality of the SWIG generated code.

The Javadoc translator will handle most of the tags conversions (see the table below). It will also automatically translate link-objects params, in `\see` and `\link...\endlink` commands. For example, `someFunction(std::string)` will be converted to `someFunction(String)`. If you don't want such behaviour, you could turn this off by using the `doxygen:nolinktranslate` feature. Also all `\param` and `\tparam` commands are stripped out, if the specified parameter is not present in the function. Use `doxygen:nostripparams` to avoid.

Javadoc translator features summary (see %feature directives):

### 18.3.2 Javadoc tags

Here is the list of all Doxygen tags and the description of how they are translated to Javadoc

```
   Doxygen tags
   \a                    wrapped with <i> html tag
   \arg                  wrapped with <li> html tag
   \author               translated to @author
   \authors              translated to @author
```

```
   \b                      wrapped with <b> html tag
   \c                      wrapped with <code> html tag
   \cite                   wrapped with <i> html tag
   \code                   translated to {@code ...}
   \code{<ext>}            translated to {@code ...}; code language extension is ignored
   \cond                   translated to 'Conditional comment: <condition>'
   \copyright              replaced with 'Copyright:'
   \deprecated             translated to @deprecated
   \e                      wrapped with <i> html tag
   \else                   replaced with '}Else:{'
   \elseif                 replaced with '}Else if: <condition>{'
   \em                     wrapped with <i> html tag
   \endcode                see note for \code
   \endcond                replaced with 'End of conditional comment.'
   \endif                  replaced with '}'
   \endlink                see note for \link
   \endverbatim            see note for \verbatim
   \exception              translated to @exception
   \f$, \f[, \f], \f{, \f} LateX formulas are left unchanged
   \if                     replaced with 'If: <condition> {'
   \ifnot                  replaced with 'If not: <condition> {'
   \image                  translated to <img/> html tag only if target=HTML
   \li                     wrapped with <li> html tag
   \link                   translated to {@link ...}
   \n                      replaced with newline char
   \note                   replaced with 'Note:'
   \overload               prints 'This is an overloaded ...' according to Doxygen docs
   \p                      wrapped with <code> html tag
   \par                    replaced with <p alt='title'>...</p>
   \param                  translated to @param
   \param[<dir>]           translated to @param; parameter direction ('in'; 'out'; or 'in,out') is ignored
   \remark                 replaced with 'Remarks:'
   \remarks                replaced with 'Remarks:'
   \result                 translated to @return
   \return                 translated to @return
   \returns                translated to @return
   \sa                     translated to @see
   \see                    translated to @see
   \since                  translated to @since
   \throw                  translated to @throws
   \throws                 translated to @throws
   \todo                   replaced with 'TODO:'
   \tparam                 translated to @param
   \verbatim               translated to {@literal ...}
   \version                translated to @version
   \warning                translated to 'Warning:'
   \$                      prints $ char
   \@                      prints @ char
   \\                      prints \ char
   \&                      prints & char
   \~                      prints ~ char
   \<                      prints < char
   \>                      prints > char
   \#                      prints # char
   \%                      prints % char
   \"                      prints " char
   \.                      prints . char
   \::                     prints ::
```

### 18.3.3 Unsupported tags for Javadoc

Doxygen has a wealth of tags such as `@latexonly` that have no equivalent in Javadoc (all supported tags are listed in Javadoc documentation). As a result several tags have no translation or particular use, such as some linking and section tags. These are suppressed with their content just printed out (if the tag has any sense, typically text content). Here is the list of these tags:

```
Unsupported Doxygen tags

    \addindex
    \addtogroup
    \anchor
    \attention
    \brief
    \bug
    \callergraph
    \callgraph
    \category
    \class
    \copybrief
    \copydetails
    \copydoc
    \date
    \def
    \defgroup
    \details
    \dir
    \dontinclude
    \dot
    \dotfile
    \enddot
    \endhtmlonly
```

```
    \endinternal
    \endlatexonly
    \endmanonly
    \endmsc
    \endrtfonly
    \endxmlonly
    \enum
    \example
    \extends
    \file
    \fn
    \headerfile
    \hideinitializer
    \htmlinclude
    \htmlonly
    \implements
    \include
    \includelineno
    \ingroup
    \interface
    \internal
    \invariant
    \latexonly
    \line
    \mainpage
    \manonly
    \memberof
    \msc
    \mscfile
    \name
    \namespace
    \nosubgrouping
    \package
    \page
    \paragraph
    \post
    \pre
    \private
    \privatesection
    \property
    \protected
    \protectedsection
    \protocol
    \public
    \publicsection
    \ref
    \related
    \relatedalso
    \relates
    \relatesalso
    \retval
    \rtfonly
    \section
    \short
    \showinitializer
    \skip
    \skipline
    \snippet
    \struct
    \subpage
    \subsection
    \subsubsection
    \tableofcontents
    \test
    \typedef
    \union
    \until
    \var
    \verbinclude
    \weakgroup
    \xmlonly
    \xrefitem
```

If one of the following Doxygen tags appears as the first tag in a comment, the whole comment block is ignored:

**Ignored Doxygen tags**

```
    \addtogroup
    \callergraph
    \callgraph
    \category
    \class
    \def
    \defgroup
    \dir
    \enum
    \example
    \file
    \fn
    \headerfile
    \hideinitializer
    \interface
    \internal
    \mainpage
    \name
    \namespace
    \nosubgrouping
    \overload
    \package
    \page
    \property
```

```
    \protocol
    \relates
    \relatesalso
    \showinitializer
    \struct
    \typedef
    \union
    \var
    \weakgroup
```

## 18.4 Doxygen to Pydoc

If translation is enabled, Pydoc formatted comments should be automatically placed in the correct locations in the resulting module and proxy files. The problem is that Pydoc has no tag mechanism like Doxygen or Javadoc, so most of Doxygen commands are translated by merely copying the appropriate command text.

### 18.4.1 Basic Pydoc example

Here is an example segment from an included header file

```
/*! This is describing class Shape
 \author Bob
 */

class Shape {
public:
  Shape() {
    nshapes++;
  }
  virtual ~Shape() {
    nshapes--;
  };
  double  x, y; /*!< Important Variables */
  void    move(double dx, double dy); /*!< Moves the Shape */
  virtual double area(void) = 0; /*!< \return the area */
  virtual double perimeter(void) = 0; /*!< \return the perimeter */
  static  int nshapes;
};
```

Simply running SWIG should result in the following code being present in Shapes.py

```
...

class Shape(_object):
    """
    This is describing class Shape
    Authors:
    Bob

    """

    ...

    def move(self, *args):
        """
        Moves the Shape
        """
        return _Shapes.Shape_move(self, *args)

    def area(self):
        """
        Return:
        the area
        """
        return _Shapes.Shape_area(self)

    def perimeter(self):
        """
        Return:
        the perimeter
        """
        return _Shapes.Shape_perimeter(self)
```

If any parameters of a function or a method are documented in the Doxygen comment, their description is copied into the generated output using Sphinx documentation conventions. For example

```
/**
    Set a breakpoint at the given location.

    @param filename The full path to the file.
    @param line_number The line number in the file.
 */
bool SetBreakpoint(const char* filename, int line_number);
```

would be translated to

```
def SetBreakpoint(filename, line_number):
    r"""
    Set a breakpoint at the given location.

    :type filename: string
    :param filename: The full path to the file.
    :type line_number: int
    :param line_number: The line number in the file.
```

```
    """
```

The types used for the parameter documentation come from the "doctype" typemap which is defined for all the primitive types and a few others (e.g. `std::string` and `shared_ptr<T>`) but for non-primitive types is taken to be just the C++ name of the type with namespace scope delimiters (`::`) replaced with a dot. To change this, you can define your own typemaps for the custom types, e.g:

```
%typemap(doctype) MyDate "datetime.date"
```

Currently Doxygen comments assigned to global variables and static member variables are not present in generated code, so they have no comment translated for them.

**Whitespace and tables** Whitespace is preserved when translating comments, so it makes sense to have Doxygen comments formatted in a readable way. This includes tables, where tags <th>, <td> and </tr>are translated to '|'. The line after line with <th> tags contains dashes. If we take care about whitespace, comments in Python are much more readable. Example:

```
/**
 * <table border = '1'>
 * <caption>Animals</caption>
 * <tr><th> Column 1 </th><th> Column 2 </th></tr>
 * <tr><td> cow      </td><td> dog      </td></tr>
 * <tr><td> cat      </td><td> mouse    </td></tr>
 * <tr><td> horse    </td><td> parrot   </td></tr>
 * </table>
 */
```

translates to Python as:

```
Animals
| Column 1 | Column 2 |
-----------------------
| cow      | dog      |
| cat      | mouse    |
| horse    | parrot   |
```

**Overloaded functions** Since all the overloaded functions in c++ are wrapped into one Python function, Pydoc translator will combine every comment of every overloaded function and put it into the comment for the one wrapper function.

If you intend to use resulting generated Python file with the Doxygen docs generator, rather than Pydoc, you may want to turn off translation completely (doxygen:notranslate feature). Then SWIG will just copy the comments to the proxy file and reformat them if needed, but all the comment content will be left as is. As Doxygen doesn't support special commands in Python comments (see Doxygen docs), you may want to use some tool like doxypy (doxypy) to do the work.

### 18.4.2 Pydoc translator

Here is the list of all Doxygen tags and the description of how they are translated to Pydoc

```
  Doxygen tags
  \a            wrapped with '*'
  \arg          prepended with '* '
  \author       prints 'Author:'
  \authors      prints 'Authors:'
  \b            wrapped with '**'
  \c            wrapped with '``'
  \cite         wrapped with single quotes
  \code         replaced with '.. code-block:: c++'
  \code{<ext>}  replaced with '.. code-block:: <lang>', where the following doxygen code languages are recognized: .c -> C, .py
                -> python, .java > java
  \cond         translated to 'Conditional comment: <condition>'
  \copyright    prints 'Copyright:'
  \deprecated   prints 'Deprecated:'
  \e            wrapped with '*'
  \else         replaced with '}Else:{'
  \elseif       replaced with '}Else if: <condition>{'
  \em           wrapped with '*'
  \endcond      replaced with 'End of conditional comment.'
  \endif        replaced with '}'
  \example      replaced with 'Example:'
  \exception    replaced with ':raises:'
  \f$           rendered using ':math:``'
  \f[           rendered using '.. math::'
  \f{           rendered using '.. math::'
  \if           replaced with 'If: <condition> {'
  \ifnot        replaced with 'If not: <condition> {'
  \li           prepended with '* '
  \n            replaced with newline char
  \note         replaced with 'Note:'
  \overload     prints 'This is an overloaded ...' according to Doxygen docs
  \p            wrapped with '``'
  \par          replaced with 'Title: ...'
  \param        add ':type:' and ':param:' directives
  \param[<dir>] same as \param, but direction ('in'; 'out'; 'in,out') is included in ':type:' directive
  \remark       replaced with 'Remarks:'
  \remarks      replaced with 'Remarks:'
  \result       add ':rtype:' and ':return:' directives
  \return       add ':rtype:' and ':return:' directives
  \returns      add ':rtype:' and ':return:' directives
  \sa           replaced with 'See also:'
  \see          replaced with 'See also:'
  \since        replaced with 'Since:'
  \throw        replaced with ':raises:'
```

```
\throws      replaced with ':raises:'
\todo        replaced with 'TODO:'
\tparam      add ':type:' and ':param:' directives
\verbatim    content copied verbatim
\version     replaced with 'Version:'
\warning     translated to 'Warning:'
\$           prints $ char
\@           prints @ char
\\           prints \ char
\&           prints & char
\~           prints ~ char
\<           prints < char
\>           prints > char
\#           prints # char
\%           prints % char
\"           prints " char
\.           prints . character
\::          prints ::
```

### 18.4.3 Unsupported tags for Pydoc

Doxygen has a wealth of tags such as `@latexonly` that have no equivalent in Pydoc. As a result several tags that have no translation (or particular use, such as some linking and section tags) are suppressed with their content just printed out (if it has any sense, typically text content). Here is the list of these tags:

```
Unsupported Python Doxygen tags

    \addindex
    \addtogroup
    \anchor
    \attention
    \brief
    \bug
    \callergraph
    \callgraph
    \category
    \class
    \copybrief
    \copydetails
    \copydoc
    \date
    \def
    \defgroup
    \details
    \dir
    \dontinclude
    \dot
    \dotfile
    \enddot
    \endhtmlonly
    \endinternal
    \endlatexonly
    \endlink
    \endmanonly
    \endmsc
    \endrtfonly
    \endxmlonly
    \enum
    \extends
    \file
    \fn
    \headerfile
    \hideinitializer
    \htmlinclude
    \htmlonly
    \image
    \implements
    \include
    \includelineno
    \ingroup
    \interface
    \internal
    \invariant
    \latexonly
    \line
    \link
    \mainpage
    \manonly
    \memberof
    \msc
    \mscfile
    \name
    \namespace
    \nosubgrouping
    \package
    \page
    \paragraph
    \post
    \pre
    \private
    \privatesection
    \property
    \protected
    \protectedsection
    \protocol
    \public
    \publicsection
    \ref
```

```
    \related
    \relatedalso
    \relates
    \relatesalso
    \retval
    \rtfonly
    \section
    \short
    \showinitializer
    \skip
    \skipline
    \snippet
    \struct
    \subpage
    \subsection
    \subsubsection
    \tableofcontents
    \test
    \typedef
    \union
    \until
    \var
    \verbinclude
    \weakgroup
    \xmlonly
    \xrefitem
```

## 18.5 Doxygen to XML C# documentation

If translation is enabled, XML formatted comments should be automatically placed in the correct locations in the resulting module and proxy files.

### 18.5.1 Basic C# example

Here is an example segment from an included header file

```
/*! This is describing class Shape
 \author Bob
 */

class Shape {
public:
  Shape() {
    nshapes++;
  }
  virtual ~Shape() {
    nshapes--;
  };
   /** Important Variables x*/
  double  x;
   /** Important Variables y*/
   double y;
   /** Moves the Shape
   * @param dx delta on x
   * @param dy delta on y */
  void    move(double dx, double dy);
  /** \return the area */
  virtual double area(void) = 0;
  /** \return the perimeter */
  virtual double perimeter(void) = 0;
  static  int nshapes;
};
```

Simply running SWIG should result in the following code being present in Shapes.cs

```


/// <summary>This is describing class Shape
/// Author: Bob
/// </summary>
public class Shape {

...
  ///  Important Variables x
  public double x {
    set {
      ShapesCsPINVOKE.Shape_x_set(swigCPtr, value);
    }
    get {
      double ret = ShapesCsPINVOKE.Shape_x_get(swigCPtr);
      return ret;
    }
  }

  ///  Important Variables y
  public double y {
    set {
      ShapesCsPINVOKE.Shape_y_set(swigCPtr, value);
    }
    get {
      double ret = ShapesCsPINVOKE.Shape_y_get(swigCPtr);
      return ret;
    }
  }

  ///  Moves the Shape
  /// <param name="dx"> delta on x</param>
  /// <param name="dy"> delta on y</param>
  public void move(double dx, double dy) {
```

```
    ShapesCsPINVOKE.Shape_move(swigCPtr, dx, dy);
  }

  /// <returns>the area</returns>
  public virtual double area() {
    double ret = ShapesCsPINVOKE.Shape_area(swigCPtr);
    return ret;
  }

  /// <returns>the perimeter</returns>
  public virtual double perimeter() {
    double ret = ShapesCsPINVOKE.Shape_perimeter(swigCPtr);
    return ret;
  }

  public static int nshapes {
    set {
      ShapesCsPINVOKE.Shape_nshapes_set(value);
    }
    get {
      int ret = ShapesCsPINVOKE.Shape_nshapes_get();
      return ret;
    }
  }
}
```

Otherwise, the code should be identical to what would have been generated without the Doxygen functionality enabled. When the Doxygen Translator module encounters a comment that contains nothing useful or a Doxygen comment that it cannot parse, it will not affect the functionality of the SWIG generated code.

The translator will handle most of the tags conversions (see the table below). It will also automatically translate link-objects params, in `<seealso>` tags . Also all `\param` and `\tparam` commands are stripped out, if the specified parameter is not present in the function. Use `doxygen:nostripparams` to avoid.

C# translator features summary (see %feature directives):

### 18.5.2 C# tags

Here is the list of all Doxygen tags and the description of how they are translated to C# XML

```
  Doxygen tags
  \a                    wrapped with '*' markdown character
  \arg                  prefixed with '*' markdown character
  \author               Made simple text with prefix 'Author:'
  \authors              Made simple text with prefix 'Author:'
  \b                    wrapped with '**' markdown character
  \c                    wrapped with `` markdown character
  \cite                 started with ' markdown character
  \code                 wrapped in <code> tag
  \code{<ext>}          wrapped in <code> tag. code language extension is ignored
  \cond                 Ignored
  \copyright            wrapped in <remark> tag
  \deprecated           Made simple text with prefix 'Deprecated:'
  \e                    prefixed with '*' markdown character
  \else                 Ignored
  \elseif               Ignored
  \em                   prefixed with '*' markdown character
  \endcode              wrapped in <code> tag. code language extension is ignored
  \endcond              Ignored
  \endif                Ignored
  \endlink              Ignored
  \endverbatim          see note for \verbatim
  \exception            translated to <exception> section
  \f$, \f[, \f], \f{, \f} Ignored
  \if                   Ignored
  \ifnot                Ignored
  \image                Ignored
  \li                   Ignored
  \link                 Ignored
  \n                    replaced with newline char
  \note                 Content copied
  \overload             Ignored
  \p                    wrapped with `` markdown characters
  \par                  Made simple text with prefix "Title:"
  \param                translated to <param> xml section
  \param[<dir>]         translated to <param> xml section; parameter direction ('in'; 'out'; or 'in,out') is ignored
  \remark               Made simple text with prefix "remarks:"
  \remarks              Made simple text with prefix "remarks:"
  \result               translated to <return> xml section
  \return               translated to <return> xml section
  \returns              translated to <return> xml section
  \sa                   translated to <seealso> xml section
  \see                  translated to <seealso> xml section
  \since                Content copied
  \throw                translated to <exception>
  \throws               translated to <exception>
  \todo                 Prefixed with 'TODO:'

  \tparam               translated to <exception> xml section
  \verbatim             copied without any processing
  \version              Made simple text with prefix 'Version:"
```

```
\warning                Made simple text with prefix 'remarks:"
\$                      prints $ char
\@                      prints @ char
\\                      prints \ char
\&                      prints & char
\~                      prints ~ char
\<                      prints < char
\>                      prints > char
\#                      prints # char
\%                      prints % char
\"                      prints " char
\.                      prints . char
\::                     prints ::
```

### 18.5.3 Unsupported tags for C#

Doxygen has a wealth of tags such as `@latexonly` that have no equivalent in C# XML (all supported tags are listed in XML documentation). As a result several tags have no translation or particular use, such as some linking and section tags. These are suppressed with their content just printed out (if the tag has any sense, typically text content). Here is the list of these tags:

```
Unsupported Doxygen tags

    \addindex
    \addtogroup
    \anchor
    \attention
    \brief
    \bug
    \callergraph
    \callgraph
    \category
    \class
    \copybrief
    \copydetails
    \copydoc
    \date
    \def
    \defgroup
    \details
    \dir
    \dontinclude
    \dot
    \dotfile
    \enddot
    \endhtmlonly
    \endinternal
    \endlatexonly
    \endmanonly
    \endmsc
    \endrtfonly
    \endxmlonly
    \enum
    \example
    \extends
    \file
    \fn
    \headerfile
    \hideinitializer
    \htmlinclude
    \htmlonly
    \implements
    \include
    \includelineno
    \ingroup
    \interface
    \internal
    \invariant
    \latexonly
    \line
    \mainpage
    \manonly
    \memberof
    \msc
    \mscfile
    \name
    \namespace
    \nosubgrouping
    \package
    \page
    \paragraph
    \post
    \pre
    \private
    \privatesection
    \property
    \protected
    \protectedsection
    \protocol
    \public
    \publicsection
    \ref
    \related
    \relatedalso
    \relates
    \relatesalso
    \retval
    \rtfonly
    \section
    \short
    \showinitializer
```

```
    \skip
    \skipline
    \snippet
    \struct
    \subpage
    \subsection
    \subsubsection
    \tableofcontents
    \test
    \typedef
    \union
    \until
    \var
    \verbinclude
    \weakgroup
    \xmlonly
    \xrefitem
```

If one of the following Doxygen tags appears as the first tag in a comment, the whole comment block is ignored:

```
Ignored Doxygen tags

    \addtogroup
    \callergraph
    \callgraph
    \category
    \class
    \def
    \defgroup
    \dir
    \enum
    \example
    \file
    \fn
    \headerfile
    \hideinitializer
    \interface
    \internal
    \mainpage
    \name
    \namespace
    \nosubgrouping
    \overload
    \package
    \page
    \property
    \protocol
    \relates
    \relatesalso
    \showinitializer
    \struct
    \typedef
    \union
    \var
    \weakgroup
```

## 18.6 Troubleshooting

When running SWIG with command line option `-doxygen`, it may happen that SWIG will fail to parse the code, which is valid C++ code and is parsed without problems without the option. The problem is, that Doxygen comments are not tokens (the C/C++ compiler actually never sees them) and that they can appear anywhere in the code. That's why it is practically impossible to handle all corner cases with the parser. However, these problems can usually be avoided by minor changes in the code or comment. Known problems and solutions are shown in this section.

Recommended approach is to first run SWIG without command line option `-doxygen`. When it successfully processes the code, include the option and fix problems with Doxygen comments.

### 18.6.1 Problem with conditional compilation

Inserting a conditional compilation preprocessor directive between a Doxygen comment and a commented item may break parsing:

```
class A {
  /**
   * Some func.
   */
  #ifndef SWIG
  void myfunc()
  {
  }
  #endif
};
```

The solution is to move the directive above the comment:

```
class A {
  #ifndef SWIG
  /**
   * Some func.
   */
  void myfunc()
  {
  }
  #endif
};
```

## 18.7 Developer information

This section contains information for developers enhancing the Doxygen translator.

### 18.7.1 Doxygen translator design

If this functionality is turned on, SWIG places all comments found into the SWIG parse tree. Nodes contain an additional attribute called `doxygen` when a comment is present. Individual nodes containing Doxygen with Structural Indicators, such as `@file`, as their first command, are also present in the parse tree. These individual "blobs" of Doxygen such as :

```
/*! This is describing function Foo
 \param x some random variable
 \author Bob
 \return Foo
 */
```

are passed on individually to the Doxygen Translator module. This module builds its own private parse tree and hands it to a separate class for translation into the target documentation language. For example, `JavaDocConverter` is the Javadoc module class.

### 18.7.2 Debugging the Doxygen parser and translator

There are two handy command line options, that enable lots of detailed debug information printing.

```
-debug-doxygen-parser     - Display Doxygen parser module debugging information
-debug-doxygen-translator - Display Doxygen translator module debugging information
```

### 18.7.3 Tests

Doxygen tests have been added to the regular SWIG test-suite. There are a number of tests beginning `doxygen_` in the Examples/test-suite sub-directory.

Like any other SWIG test case, the tests are included in Examples/test-suite/common.mk and can be tested with commands like `make check-test-suite` or `make check-python-test-suite`. To run them individually, type `make -s <testname>.cpptest` in the language-specific sub-directory in `Examples/test-suite` directory. For example:

```
Examples/test-suite/java $ make -s doxygen_parsing.cpptest
```

If the test fails, both expected and translated comments are printed to std out, but also written to files *expected.txt* and *got.txt*. Since it is often difficult to find a single character difference in several lines of text, we can use some diff tool, for example:

```
Examples/test-suite/java $ kdiff3 expected.txt got.txt
```

Runtime tests in Java are implemented using Javadoc doclets. To make that work, you should have tools.jar from the JDK in your classpath. Or you should have JAVA_HOME environment variable defined and pointing to the JDK location.

The Java's comment parsing code (the testing part) is located in commentParser.java. It checks the generated code. It is possible to run this file as a stand-alone program, with `java commentParser <some java package>`, and it will print the list of comments found in the specified directory (in the format it has used in the runtime tests). So, when you want to create a new Doxygen test case, just copy an existing one and replace the actual comment content (section of entries in form 'wantedComments.put(...)' with the output of the above command.

Runtime tests in Python are just plain string comparisons of the __doc__ properties.

## 18.8 Extending to other languages

In general, an extension to another language requires a fairly deep understanding of the target language module, such as Modules/python.cxx for Python. Searching for "doxygen" in the java.cxx module can give you a good idea of the process for placing documentation comments into the correct areas. The basic gist is that anywhere a comment may reside on a node, there needs to be a catch for it in front of where that function, class, or other object is written out to a target language file. The other half of extension is building a target documentation language comment generator that handles one blob at a time. However, this is relatively simple and nowhere near as complex as the wrapper generating modules in SWIG. See Source/Doxygen/javadoc.cxx for a good example. The target language module passes the Doxygen Translator the blob to translate, and receives back the translated text.

**What is given to the Doxygen Translator**

```
/*! This is describing function Foo
 \param x some random variable
 \author Bob
 \return Foo
 */
```

**What is received back by java.cxx**

```
/** This is describing function Foo
 *
 * @param x some random variable
 * @author Bob
 * @return Foo
 */
```

Development of the comment translator itself is simplified by the fact that the Doxygen Translator module can easily include a `main` function and thus be developed, compiled, and tested independently of SWIG.

# 19 Warning Messages


- Introduction
- Warning message suppression
- Enabling extra warnings
- Issuing a warning message
- Symbolic symbols
- Commentary
- Warnings as errors
- Message output format
- Warning number reference
  - Deprecated features (100-199)
  - Preprocessor (200-299)
  - C/C++ Parser (300-399)
  - Types and typemaps (400-499)

- Code generation (500-559)
- Doxygen comments (560-599)
- Language module specific (700-899)
- User defined (900-999)

- History


## 19.1 Introduction

During compilation, SWIG may generate a variety of warning messages. For example:

```
example.i:16: Warning 501: Overloaded declaration ignored.  bar(double)
example.i:15: Warning 501: Previous declaration is bar(int)
```

Typically, warning messages indicate non-fatal problems with the input where the generated wrapper code will probably compile, but it may not work like you expect.

## 19.2 Warning message suppression

All warning messages have a numeric code that is shown in the warning message itself. To suppress the printing of a warning message, a number of techniques can be used. First, you can run SWIG with the `-w` command line option. For example:

```
% swig -python -w501 example.i
% swig -python -w501,505,401 example.i
```

Alternatively, warnings can be suppressed by inserting a special preprocessor pragma into the input file:

```
%module example
#pragma SWIG nowarn=501
#pragma SWIG nowarn=501,505,401
```

Finally, code-generation warnings can be disabled on a declaration by declaration basis using the `%warnfilter` directive. For example:

```
%module example
%warnfilter(501) foo;
...
int foo(int);
int foo(double);              // Silently ignored.
```

The `%warnfilter` directive has the same semantics as other declaration modifiers like `%rename`, `%ignore` and `%feature`, see the %feature directive section. For example, if you wanted to suppress a warning for a method in a class hierarchy, you could do this:

```
%warnfilter(501) Object::foo;
class Object {
public:
  int foo(int);
  int foo(double);      // Silently ignored
  ...
};

class Derived : public Object {
public:
  int foo(int);
  int foo(double);      // Silently ignored
  ...
};
```

Warnings can be suppressed for an entire class by supplying a class name. For example:

```
%warnfilter(501) Object;

class Object {
public:
  ...                     // All 501 warnings ignored in class
};
```

There is no option to suppress all SWIG warning messages. The warning messages are there for a reason---to tell you that something may be *broken* in your interface. Ignore the warning messages at your own peril.

## 19.3 Enabling extra warnings

Some warning messages are disabled by default and are generated only to provide additional diagnostics. These warnings can be turned on using the `-Wextra` option. For example:

```
% swig -Wextra -python example.i
```

Preprocessor warning 202 ("Could not evaluate expression *expr*.") was formally off by default and enabled by `-Wextra`, but since SWIG 4.1.0 this warning is on by default because suppressing it tends to hide genuine problems. If you really don't want to see it, you can suppress it with `-w202` or using `%warnfilter` as described below. Both will work with older versions of SWIG too.

To selectively turn on extra warning messages, you can use the directives and options in the previous section--simply add a "+" to all warning numbers. For example:

```
% swig -w+309,+452 example.i
```

or in your interface file use either

```
#pragma SWIG nowarn=+309,+452
```

or

```
%warnfilter(+309,+452) foo;
```

Note: selective enabling of warnings with `%warnfilter` overrides any global settings you might have made using `-w` or `#pragma`.

You can of course also enable all warnings and suppress a select few, for example:

```
% swig -Wextra -w309,452 example.i
```

The warnings on the right take precedence over the warnings on the left, so in the above example `-Wextra` adds numerous warnings including 452, but then `-w309,452` overrides this and so 452 is suppressed.

If you would like all warnings to appear, regardless of the warning filters used, then use the `-Wall` option. The `-Wall` option also turns on the extra warnings that `-Wextra` adds, however, it is subtely different. When `-Wall` is used, it also disables all other warning filters, that is, any warnings suppressed or added in `%warnfilter`, `#pragma SWIG nowarn` or the `-w` option.

## 19.4 Issuing a warning message

Warning messages can be issued from an interface file using a number of directives. The `%warn` directive is the most simple:

```
%warn "900:This is your last warning!"
```

All warning messages are optionally prefixed by the warning number to use. If you are generating your own warnings, make sure you don't use numbers defined in the table at the end of this section.

The `%ignorewarn` directive is the same as `%ignore` except that it issues a warning message whenever a matching declaration is found. For example:

```
%ignorewarn("362:operator= ignored") operator=;
```

Warning messages can be associated with typemaps using the `warning` attribute of a typemap declaration. For example:

```
%typemap(in, warning="901:You are really going to regret this usage of $1_type $1_name") blah * {
  ...
}
```

In this case, the warning message will be printed whenever the typemap is actually used and the special variables will be expanded as appropriate, for example:

```
example.i:23: Warning 901: You are really going to regret this usage of blah * self
example.i:24: Warning 901: You are really going to regret this usage of blah * stuff
```

## 19.5 Symbolic symbols

The `swigwarn.swg` file that is installed with SWIG contains symbol constants that could also be used in `%warnfilter` and `#pragma SWIG nowarn`. For example this file contains the following line:

```
%define SWIGWARN_TYPE_UNDEFINED_CLASS 401 %enddef
```

so `SWIGWARN_TYPE_UNDEFINED_CLASS` could be used instead of 401, for example:

```
#pragma SWIG nowarn=SWIGWARN_TYPE_UNDEFINED_CLASS
```

or

```
%warnfilter(SWIGWARN_TYPE_UNDEFINED_CLASS) Foo;
```

## 19.6 Commentary

The ability to suppress warning messages is really only provided for advanced users and is not recommended in normal use. You are advised to modify your interface to fix the problems highlighted by the warnings wherever possible instead of suppressing warnings.

Certain types of SWIG problems are errors. These usually arise due to parsing errors (bad syntax) or semantic problems for which there is no obvious recovery. There is no mechanism for suppressing error messages.

## 19.7 Warnings as errors

Warnings can be handled as errors by using the `-Werror` command line option. This will cause SWIG to exit with a non successful exit code if a warning is encountered.

## 19.8 Message output format

The output format for both warnings and errors can be selected for integration with your favourite IDE/editor. Editors and IDEs can usually parse error messages and if in the appropriate format will easily take you directly to the source of the error. The standard format is used by default except on Windows where the Microsoft format is used by default. These can be overridden using command line options, for example:

```
$ swig -python -Fstandard example.i
example.i:4: Syntax error in input(1).
$ swig -python -Fmicrosoft example.i
example.i(4) : Syntax error in input(1).
```

## 19.9 Warning number reference

### 19.9.1 Deprecated features (100-199)

None currently.

### 19.9.2 Preprocessor (200-299)

- 201. Unable to find *filename*.
- 202. Could not evaluate expression *expr*.
- 203. Both includeall and importall are defined: using includeall.
- 204. CPP #warning, "*warning*".
- 205. CPP #error, "*error*".
- 206. Unexpected tokens after *#directive* directive.

### 19.9.3 C/C++ Parser (300-399)

- 301. `class` keyword used, but not in C++ mode.
- 302. Redefinition of identifier *'name*' as *decl* ignored.
- 303. `%extend` defined for an undeclared class *'name* '.
- 304. Unsupported constant value (ignored).
- 305. Bad constant value (ignored).
- 308. Namespace alias *'name*' not allowed here. Assuming '*name*'
- 309. [private | protected] inheritance ignored.
- 312. Unnamed nested class not currently supported (ignored).
- 313. Unrecognized extern type "*name*" (ignored).
- 314. '*identifier*' is a *lang* keyword.
- 315. Nothing known about *'identifier*'.
- 317. Specialization of non-template *'name*'.
- 318. Instantiation of template *'name*' is ambiguous, instantiation *templ* used, instantiation *templ* ignored.
- 319. No access specifier given for base class *name* (ignored).
- 320. Explicit template instantiation ignored.
- 321. *identifier* conflicts with a built-in name.
- 322. Redundant redeclaration of identifier *'name*' as *decl* ignored.
- 323. Recursive scope inheritance of *'name*'.
- 324. Named nested template instantiations not supported. Processing as if no name was given to %template().
- 325. Nested *kind* not currently supported (*name* ignored).
- 326. Deprecated %extend name used - the *kind* name '*name* ' should be used instead of the typedef name *hame*'.
- 327. Extern template ignored.
- 328. Value assigned to *name* not used due to limited parsing implementation.
- 329. Using declaration '*name*' for inheriting constructors uses base '*name*' which is not an immediate base of *'name* '.
- 330. Template forward class *'name*' cannot be used to instantiate a full template class with name '*name*'.
- 331. Unsupported template nested class *'name*' cannot be used to instantiate a full template class with name '*name*'.
- 340. Lambda expressions and closures are not fully supported yet.
- 344. Unable to deduce decltype for *'expr*'.
- 345. Unable to deduce auto return type for *'name*' (ignored).
- 346. Unable to deduce auto type for variable *'name*' (ignored).
- 350. operator new ignored.
- 351. operator delete ignored.
- 352. operator+ ignored.
- 353. operator- ignored.
- 354. operator* ignored.
- 355. operator/ ignored.
- 356. operator% ignored.
- 357. operator^ ignored.
- 358. operator& ignored.
- 359. operator| ignored.
- 360. operator~ ignored.
- 361. operator! ignored.
- 362. operator= ignored.
- 363. operator< ignored.
- 364. operator> ignored.
- 365. operator+= ignored.
- 366. operator-= ignored.
- 367. operator*= ignored.
- 368. operator/= ignored.
- 369. operator%= ignored.
- 370. operator^= ignored.
- 371. operator&= ignored.
- 372. operator|= ignored.
- 373. operator<< ignored.
- 374. operator>>ignored.
- 375. operator<<= ignored.
- 376. operator>>= ignored.
- 377. operator== ignored.
- 378. operator!= ignored.
- 379. operator<= ignored.
- 380. operator>= ignored.
- 381. operator&& ignored.
- 382. operator|| ignored.
- 383. operator++ ignored.
- 384. operator-- ignored.
- 385. operator, ignored.
- 386. operator-<* ignored.
- 387. operator-< ignored.
- 388. operator() ignored.
- 389. operator[] ignored.
- 390. operator+ ignored (unary).
- 391. operator- ignored (unary).
- 392. operator* ignored (unary).
- 393. operator& ignored (unary).
- 394. operator new[] ignored.
- 395. operator delete[] ignored.
- 396. operator*() ignored.
- 397. operator<=> delete[] ignored.

### 19.9.4 Types and typemaps (400-499)

- 401. Nothing known about class 'name'. Ignored.
- 402. Base class 'name' is incomplete.
- 403. Class 'name' might be abstract.
- 404. Duplicate template instantiation of '*type*' with name ' *name*' ignored, previous instantiation of '*type*' with name '*name*'.
- 405. Method with rvalue ref-qualifier *name* ignored.
- 406. Ignoring nspace setting (*setting*) for '*type*', as it conflicts with the nspace setting (*setting*) for outer class '*type*'
- 450. Reserved
- 451. Setting const char * variable may leak memory.
- 452. Reserved
- 453. Can't apply (pattern). No typemaps are defined.
- 454. Setting a pointer/reference variable may leak memory
- 455. Setting a const wchar_t * variable may leak memory.
- 460. Unable to use type *type* as a function argument.
- 461. Unable to use return type *type* in function *name* .
- 462. Unable to set variable of type *type*.
- 463. Unable to read variable of type *type*.
- 464. Unsupported constant value.
- 465. Unable to handle type *type*.
- 466. Unsupported variable type *type*.
- 467. Overloaded *declaration* not supported (incomplete type checking rule - no precedence level in typecheck typemap for '*type* ')
- 468. No 'throw' typemap defined for exception type *type*
- 469. No or improper directorin typemap defined for *type*
- 470. Thread/reentrant unsafe wrapping, consider returning by value instead.
- 471. Unable to use return type *type* in director method
- 472. Overloaded method *method* with no explicit typecheck typemap for arg *number* of type '*type*'. Dispatching calls to this method may not work correctly, see the 'Typemaps and Overloading' section in the Typemaps chapter of the SWIG documentation.
- 473. Returning a reference, pointer or pointer wrapper in a director method is not recommended.
- 474. Method *method* usage of the optimal attribute ignored in the out typemap as the following cannot be used to generate optimal code: *code*
- 475. Multiple calls to *method* might be generated due to optimal attribute usage in the out typemap.
- 476. Initialization using std::initializer_list.
- 477. No directorthrows typemap defined for *type*
- 490. Fragment '*name*' not found.

### 19.9.5 Code generation (500-559)

- 501. Overloaded declaration ignored. *decl*. Previous declaration is *decl*.
- 502. Overloaded constructor ignored. *decl*. Previous declaration is *decl*.
- 503. Can't wrap '*identifier*' unless renamed to a valid identifier.
- 504. Function *name* must have a return type. Ignored.
- 505. Variable length arguments discarded.
- 506. Can't wrap varargs with keyword arguments enabled.
- 507. Adding native function *name* not supported (ignored).
- 508. Declaration of '*name*' shadows declaration accessible via operator->(), previous declaration of '*declaration*'.
- 509. Overloaded method *declaration* effectively ignored, as it is shadowed by *declaration*.
- 510. Friend function '*name*' ignored.
- 511. Can't use keyword arguments with overloaded functions.
- 512. Overloaded method *declaration* ignored, using non-const method *declaration* instead.
- 513. Can't generate wrappers for unnamed struct/class.
- 514.
- 515.
- 516. Overloaded method *declaration* ignored, using *declaration* instead.
- 517.
- 518. Portability warning: File *file1* will be overwritten by *file2* on case insensitive filesystems such as Windows' FAT32 and NTFS unless the class/module name is renamed.
- 519. %template() contains no name. Template method ignored: *declaration*
- 520. *Base/Derived* class '*classname1*' of ' *classname2*' is not similarly marked as a smart pointer.
- 521. Illegal destructor name *name*. Ignored.
- 522. Use of an illegal constructor name '*name*' in %extend is deprecated, the constructor name should be '*name*'.
- 523. Use of an illegal destructor name '*name*' in %extend is deprecated, the destructor name should be '*name*'.
- 524. Experimental target language. Target language *language* specified by *lang* is an experimental language. See the 'Target Languages' section in the Introduction chapter of the SWIG documentation.
- 525. Destructor *declaration* is final, *name* cannot be a director class.
- 526. Using declaration *declaration*, with name '*name* ', is not actually using the method from *declaration*, with name '*name*', as the names are different.
- 527. Deprecated target language. Target language *language* specified by *lang* is a deprecated language. It will be removed in the next release of SWIG unless a new maintainer steps forward to bring it up to at least experimental status. See the 'Target Languages' section in the Introduction chapter of the SWIG documentation.

### 19.9.6 Doxygen comments (560-599)

- 560: Unknown Doxygen command: *command*.
- 561: Unexpected end of Doxygen comment encountered.
- 562: Expected Doxygen command: *command*
- 563: Doxygen HTML error for tag *tag*: *error text*.
- 564: Error parsing Doxygen command *command*: *error text* . Command ignored."

### 19.9.7 Language module specific (700-899)

- 801. Wrong name (corrected to '*name*'). (Ruby).

- 810. No jni typemap defined for *type* (Java).
- 811. No jtype typemap defined for *type* (Java).
- 812. No jstype typemap defined for *type* (Java).
- 813. Warning for *classname*, base *baseclass* ignored. Multiple inheritance is not supported in Java. (Java).
- 814.
- 815. No javafinalize typemap defined for *type* (Java).
- 816. No javabody typemap defined for *type* (Java).
- 817. No javaout typemap defined for *type* (Java).
- 818. No javain typemap defined for *type* (Java).
- 819. No javadirectorin typemap defined for *type* (Java).
- 820. No javadirectorout typemap defined for *type* (Java).
- 821.
- 822. Covariant return types not supported in Java. Proxy method will return *basetype* (Java).
- 823. No javaconstruct typemap defined for *type* (Java).

- 824. Missing JNI descriptor in directorin typemap defined for *type* (Java).
- 825. "directorconnect" attribute missing in *type* "javaconstruct" typemap. (Java).
- 826. The nspace feature is used on '*type*' without -package. The generated code may not compile as Java does not support types declared in a named package accessing types declared in an unnamed package. (Java).

- 830. No ctype typemap defined for *type* (C#).
- 831. No cstype typemap defined for *type* (C#).
- 832. No cswtype typemap defined for *type* (C#).
- 833. Warning for *classname*, base *baseclass* ignored. Multiple inheritance is not supported in C#. (C#).
- 834.
- 835. No csfinalize typemap defined for *type* (C#).
- 836. No csbody typemap defined for *type* (C#).
- 837. No csout typemap defined for *type* (C#).
- 838. No csin typemap defined for *type* (C#).
- 839.
- 840.
- 841.
- 842. Covariant return types not supported in C#. Proxy method will return *basetype* (C#).
- 843. No csconstruct typemap defined for *type* (C#).
- 844. C# exception may not be thrown - no $excode or excode attribute in *typemap* typemap. (C#).
- 845. Unmanaged code contains a call to a SWIG_CSharpSetPendingException method and C# code does not handle pending exceptions via the canthrow attribute. (C#).

- 870. Warning for *classname*: Base *baseclass* ignored. Multiple inheritance is not supported in PHP. (Php).
- 871. Unrecognized pragma *pragma*. (Php).

#### 19.9.8 User defined (900-999)

These numbers can be used by your own application.

### 19.10 History

The ability to control warning messages was first added to SWIG-1.3.12.

---

# 20 Working with Modules


- Modules Introduction
- Basics
- The SWIG runtime code
- External access to the runtime
- A word of caution about static libraries
- References
- Reducing the wrapper file size


## 20.1 Modules Introduction

Each invocation of SWIG requires a module name to be specified. The module name is used to name the resulting target language extension module. Exactly what this means and what the name is used for depends on the target language, for example the name can define a target language namespace or merely be a useful name for naming files or helper classes. Essentially, a module comprises target language wrappers for a chosen collection of global variables/functions, structs/classes and other C/C++ types.

The module name can be supplied in one of two ways. The first is to specify it with the special `%module` directive. This directive must appear at the beginning of the interface file. The general form of this directive is:

```
%module(option1="value1", option2="value2", ...) modulename
```

where the modulename is mandatory and the options add one or more optional additional features. Typically no options are specified, for example:

```
%module mymodule
```

The second way to specify the module name is with the `-module` command line option, for example `-module mymodule`. If the module name is supplied on the command line, it overrides the name specified by the `%module` directive.

When first working with SWIG, users commonly start by creating a single module. That is, you might define a single SWIG interface that wraps some set of C/C++ code. You then compile all of the generated wrapper code together and use it. For large applications, however, this approach is problematic---the size of the generated wrapper code can be rather large. Moreover, it is probably easier to manage the target language interface when it is broken up into smaller pieces.

This chapter describes the problem of using SWIG in programs where you want to create a collection of modules. Each module in the collection is created via separate invocations of SWIG.

## 20.2 Basics

The basic usage case with multiple modules is when modules do not have cross-references (ie. when wrapping multiple independent C APIs). In that case, swig input files should just work out of the box - you simply create multiple wrapper .cxx files, link them into your application, and insert/load each in the scripting language runtime as you would do for the single module case.

A bit more complex is the case in which modules need to share information. For example, when one module extends the class of another by deriving from it:

```
// File: base.h
class base {
public:
  int foo();
};
```

```
// File: base_module.i
%module base_module

%{
#include "base.h"
%}
%include "base.h"
```

```
// File: derived_module.i
%module derived_module

%{
#include "base.h"
%}

%import "base_module.i"

%inline %{
class derived : public base {
public:
  int bar();
};
%}
```

To create the wrapper properly, module `derived_module` needs to know about the `base` class and that its interface is covered in another module. The line `%import "base_module.i"` lets SWIG know exactly that. Often the `.h` file is passed to `%import` instead of the `.i`, which unfortunately doesn't work for all language modules. For example, Python requires the name of module that the base class exists in so that the proxy classes can fully inherit the base class's methods. Typically you will get a warning when the module name is missing, eg:

```
derived_module.i:8: Warning 401: Base class 'base' ignored - unknown module name for base. Either
import
the appropriate module interface file or specify the name of the module in the %import directive.
```

It is sometimes desirable to import the header file rather than the interface file and overcome the above warning. For example in the case of the imported interface being quite large, it may be desirable to simplify matters and just import a small header file of dependent types. This can be done by specifying the optional `module` attribute in the `%import` directive. The `derived_module.i` file shown above could be replaced with the following:

```
// File: derived_module.i
%module derived_module

%{
#include "base.h"
%}

%import(module="base_module") "base.h"

%inline %{
class derived : public base {
public:
  int bar();
};
```

Note that "base_module" is the module name and is the same as that specified in `%module` in `base_module.i` as well as the `%import` in `derived_module.i`.

Another issue to beware of is that multiple dependent wrappers should not be linked/loaded in parallel from multiple threads as SWIG provides no locking - for more on that issue, read on.

## 20.3 The SWIG runtime code

Many of SWIG's target languages generate a set of functions commonly known as the "SWIG runtime." These functions are primarily related to the runtime type system which checks pointer types and performs other tasks such as proper casting of pointer values in C++. As a general rule, the statically typed target languages, such as Java, use the language's built in static type checking and have no need for a SWIG runtime. All the dynamically typed / interpreted languages rely on the SWIG runtime.

The runtime functions are private to each SWIG-generated module. That is, the runtime functions are declared with "static" linkage and are visible only to the wrapper functions defined in that module. The only problem with this approach is that when more than one SWIG module is used in the same application, those modules often need to share type information. This is especially true for C++ programs where SWIG must collect and share information about inheritance relationships that cross module boundaries.

To solve the problem of sharing information across modules, a pointer to the type information is stored in a global variable in the target language namespace. During module initialization, type information is loaded into the global data structure of type information from all modules.

There are a few trade offs with this approach. This type information is global across all SWIG modules loaded, and can cause type conflicts between modules that were not designed to work together. To solve this approach, the SWIG runtime code uses a define SWIG_TYPE_TABLE to provide a unique type table. This behavior can be enabled when compiling the generated _wrap.cxx or _wrap.c file by adding -DSWIG_TYPE_TABLE=myprojectname to the command line argument.

Then, only modules compiled with SWIG_TYPE_TABLE set to myprojectname will share type information. So if your project has three modules, all three should be compiled with -DSWIG_TYPE_TABLE=myprojectname, and then these three modules will share type information. But any other project's types will not interfere or clash with the types in your module.

Another issue relating to the global type table is thread safety. If two modules try and load at the same time, the type information can become corrupt. SWIG currently does not provide any locking, and if you use threads, you must make sure that modules are loaded serially. Be careful if you use threads and the automatic module loading that some scripting languages provide. One solution is to load all modules before spawning any threads, or use SWIG_TYPE_TABLE to separate type tables so they do not clash with each other.

Lastly, SWIG uses a #define SWIG_RUNTIME_VERSION, located in Lib/swigrun.swg and near the top of every generated module. This number gets incremented when the data structures change, so that SWIG modules generated with different versions can peacefully coexist. So the type structures are separated by the (SWIG_TYPE_TABLE, SWIG_RUNTIME_VERSION) pair, where by default SWIG_TYPE_TABLE is empty. Only modules compiled with the same pair will share type information.

## 20.4 External access to the runtime

As described in The run-time type checker, the functions `SWIG_TypeQuery`, `SWIG_NewPointerObj`, and others sometimes need to be called. Calling these functions from a typemap is supported, since the typemap code is embedded into the `_wrap.c` file, which has those declarations available. If you need to call the SWIG run-time functions from another C file, there is one header you need to include. To generate the header that needs to be included, SWIG can be run in a different mode via `-external-runtime` to generate the run-time instead of the normal mode of processing an input interface file. For example:

```
$ swig -python -external-runtime <filename>
```

The filename argument is optional and if it is not passed, then the default filename will be something like `swigpyrun.h`, depending on the language. This header file should be treated like any of the other _wrap.c output files, and should be regenerated when the _wrap files are. After including this header, your code will be able to call `SWIG_TypeQuery`, `SWIG_NewPointerObj`, `SWIG_ConvertPtr` and others. The exact argument parameters for these functions might differ between language modules; please check the language module chapters for more information.

Inside this header the functions are declared static and are included inline into the file, and thus the file does not need to be linked against any SWIG libraries or code (you might still need to link against the language libraries like libpython-2.3). Data is shared between this file and the _wrap.c files through a global variable in the scripting language. It is also possible to copy this header file along with the generated wrapper files into your own package, so that you can distribute a package that can be compiled without SWIG installed (this works because the header file is self-contained, and does not need to link with anything).

This header will also use the -DSWIG_TYPE_TABLE described above, so when compiling any code which includes the generated header file should define the SWIG_TYPE_TABLE to be the same as the module whose types you are trying to access.

## 20.5 A word of caution about static libraries

When working with multiple SWIG modules, you should take care not to use static libraries. For example, if you have a static library `libfoo.a` and you link a collection of SWIG modules with that library, each module will get its own private copy of the library code inserted into it. This is very often **NOT** what you want and it can lead to unexpected or bizarre program behavior. When working with dynamically loadable modules, you should try to work exclusively with shared libraries.

## 20.6 References

Due to the complexity of working with shared libraries and multiple modules, it might be a good idea to consult an outside reference. John Levine's "Linkers and Loaders" is highly recommended.

## 20.7 Reducing the wrapper file size

Using multiple modules with the `%import` directive is the most common approach to modularising large projects. In this way a number of different wrapper files can be generated, thereby avoiding the generation of a single large wrapper file. There are a couple of alternative solutions for reducing the size of a wrapper file through the use of command line options and features.

**-fcompact**
This command line option will compact the size of the wrapper file without changing the code generated into the wrapper file. It simply removes blank lines and joins lines of code together. This is useful for compilers that have a maximum file size that can be handled.

**-fvirtual**
This command line option will remove the generation of superfluous virtual method wrappers. Consider the following inheritance hierarchy:

```
struct Base {
  virtual void method();
  ...
};

struct Derived : Base {
  virtual void method();
  ...
};
```

Normally wrappers are generated for both methods, whereas this command line option will suppress the generation of a wrapper for `Derived::method`. Normal polymorphic behaviour remains as `Derived::method` will still be called should you have a`Derived` instance and call the wrapper for `Base::method`.

**%feature("compactdefaultargs")**
This feature can reduce the number of wrapper methods when wrapping methods with default arguments. The section on default arguments discusses the feature and its limitations.

# 21 Using SWIG with ccache - ccache-swig(1) manpage


- NAME
- SYNOPSIS
- DESCRIPTION
- OPTIONS SUMMARY
- OPTIONS
- INSTALLATION
- EXTRA OPTIONS
- ENVIRONMENT VARIABLES
- CACHE SIZE MANAGEMENT
- CACHE COMPRESSION
- HOW IT WORKS
- USING CCACHE WITH DISTCC
- SHARING A CACHE
- HISTORY
- DIFFERENCES FROM COMPILERCACHE
- CREDITS
- AUTHOR


## 21.1 NAME

ccache-swig - a fast compiler cache

## 21.2 SYNOPSIS

ccache-swig [OPTION]

ccache-swig <compiler> [COMPILER OPTIONS]

<compiler> [COMPILER OPTIONS]

## 21.3 DESCRIPTION

ccache-swig is a compiler cache. It speeds up re-compilation of C/C++/SWIG code by caching previous compiles and detecting when the same compile is being done again. ccache-swig is ccache plus support for SWIG. ccache and ccache-swig are used interchangeably in this document.

## 21.4 OPTIONS SUMMARY

Here is a summary of the options to ccache-swig.

```
-s                      show statistics summary
-z                      zero statistics
-c                      run a cache cleanup
-C                      clear the cache completely
-F <n>                  set maximum files in cache
-M <n>                  set maximum size of cache (use G, M or K)
```

```
-h                      this help page
-V                      print version number
```

## 21.5 OPTIONS

These options only apply when you invoke ccache as "ccache-swig". When invoked as a compiler none of these options apply. In that case your normal compiler options apply and you should refer to your compilers documentation.

**-h**
Print a options summary page

**-s**
Print the current statistics summary for the cache. The statistics are stored spread across the subdirectories of the cache. Using "ccache-swig -s" adds up the statistics across all subdirectories and prints the totals.

**-z**
Zero the cache statistics.

**-V**
Print the ccache version number

**-c**
Clean the cache and re-calculate the cache file count and size totals. Normally the -c option should not be necessary as ccache keeps the cache below the specified limits at runtime and keeps statistics up to date on each compile. This option is mostly useful if you manually modify the cache contents or believe that the cache size statistics may be inaccurate.

**-C**
Clear the entire cache, removing all cached files.

**-F <maxfiles>**
This sets the maximum number of files allowed in the cache. The value is stored inside the cache directory and applies to all future compiles. Due to the way the value is stored the actual value used is always rounded down to the nearest multiple of 16.

**-M <maxsize>**
This sets the maximum cache size. You can specify a value in gigabytes, megabytes or kilobytes by appending a G, M or K to the value. The default is gigabytes. The actual value stored is rounded down to the nearest multiple of 16 kilobytes.

## 21.6 INSTALLATION

There are two ways to use ccache. You can either prefix your compile commands with "ccache-swig" or you can create a symbolic link between ccache-swig and the names of your compilers. The first method is most convenient if you just want to try out ccache or wish to use it for some specific projects. The second method is most useful for when you wish to use ccache for all your compiles.

To install for usage by the first method just copy ccache-swig to somewhere in your path.

To install for the second method do something like this:

```
cp ccache-swig /usr/local/bin/
ln -s /usr/local/bin/ccache-swig /usr/local/bin/gcc
ln -s /usr/local/bin/ccache-swig /usr/local/bin/g++
ln -s /usr/local/bin/ccache-swig /usr/local/bin/cc
ln -s /usr/local/bin/ccache-swig /usr/local/bin/swig
```

This will work as long as /usr/local/bin comes before the path to gcc (which is usually in /usr/bin). After installing you may wish to run "which gcc" to make sure that the correct link is being used.

Note! Do not use a hard link, use a symbolic link. A hardlink will cause "interesting" problems.

## 21.7 EXTRA OPTIONS

When run as a compiler front end ccache usually just takes the same command line options as the compiler you are using. The only exception to this is the option '--ccache-skip'. That option can be used to tell ccache that the next option is definitely not a input filename, and should be passed along to the compiler as-is.

The reason this can be important is that ccache does need to parse the command line and determine what is an input filename and what is a compiler option, as it needs the input filename to determine the name of the resulting object file (among other things). The heuristic ccache uses in this parse is that any string on the command line that exists as a file is treated as an input file name (usually a C file). By using --ccache-skip you can force an option to not be treated as an input file name and instead be passed along to the compiler as a command line option.

## 21.8 ENVIRONMENT VARIABLES

ccache uses a number of environment variables to control operation. In most cases you won't need any of these as the defaults will be fine.

**CCACHE_DIR**
the CCACHE_DIR environment variable specifies where ccache will keep its cached compiler output. The default is "$HOME/.ccache".

**CCACHE_TEMPDIR**
the CCACHE_TEMPDIR environment variable specifies where ccache will put temporary files. The default is the same as CCACHE_DIR. Note that the CCACHE_TEMPDIR path must be on the same filesystem as the CCACHE_DIR path, so that renames of files between the two directories can work.

**CCACHE_LOGFILE**
If you set the CCACHE_LOGFILE environment variable then ccache will write some log information on cache hits and misses in that file. This is useful for tracking down problems.

**CCACHE_VERBOSE**
If you set the CCACHE_VERBOSE environment variable then ccache will display on stdout all the compiler invocations that it makes. This can useful for debugging unexpected problems.

**CCACHE_PATH**
You can optionally set CCACHE_PATH to a colon separated path where ccache will look for the real compilers. If you don't do this then ccache will look for the first executable matching the compiler name in the normal PATH that isn't a symbolic link to ccache itself.

**CCACHE_CC**
You can optionally set CCACHE_CC to force the name of the compiler to use. If you don't do this then ccache works it out from the command line.

**CCACHE_PREFIX**
This option adds a prefix to the command line that ccache runs when invoking the compiler. Also see the section below on using ccache with distcc.

**CCACHE_DISABLE**
If you set the environment variable CCACHE_DISABLE then ccache will just call the real compiler, bypassing the cache completely.

**CCACHE_READONLY**
the CCACHE_READONLY environment variable tells ccache to attempt to use existing cached object files, but not to try to add anything new to the cache. If you are using this because your CCACHE_DIR is read-only, then you may find that you also need to set CCACHE_TEMPDIR as otherwise ccache will fail to create the temporary files.

**CCACHE_CPP2**
If you set the environment variable CCACHE_CPP2 then ccache will not use the optimisation of avoiding the 2nd call to the pre-processor by compiling the pre-processed output that was used for finding the hash in the case of a cache miss. This is primarily a debugging option, although it is possible that some unusual compilers will have problems with the intermediate filename extensions used in this optimisation, in which case this option could allow ccache to be used.

**CCACHE_NOCOMPRESS**
If you set the environment variable CCACHE_NOCOMPRESS then there is no compression used on files that go into the cache. However, this setting has no effect on how files are retrieved from the cache, compressed results will still be usable.

**CCACHE_NOSTATS**
If you set the environment variable CCACHE_NOSTATS then ccache will not update the statistics files on each compile.

**CCACHE_NLEVELS**
The environment variable CCACHE_NLEVELS allows you to choose the number of levels of hash in the cache directory. The default is 2. The minimum is 1 and the maximum is 8.

**CCACHE_HARDLINK**
If you set the environment variable CCACHE_HARDLINK then ccache will attempt to use hard links from the cache directory when creating the compiler output rather than using a file copy. Using hard links is faster, but can confuse programs like 'make' that rely on modification times. Hard links are never made for compressed cache files.

**CCACHE_RECACHE**
This forces ccache to not use any cached results, even if it finds them. New results are still cached, but existing cache entries are ignored.

**CCACHE_UMASK**
This sets the umask for ccache and all child processes (such as the compiler). This is mostly useful when you wish to share your cache with other users. Note that this also affects the file permissions set on the object files created from your compilations.

**CCACHE_HASHDIR**
This tells ccache to hash the current working directory when calculating the hash that is used to distinguish two compiles. This prevents a problem with the storage of the current working directory in the debug info of a object file, which can lead ccache to give a cached object file that has the working directory in the debug info set incorrectly. This option is off by default as the incorrect setting of this debug info rarely causes problems. If you strike problems with gdb not using the correct directory then enable this option.

**CCACHE_UNIFY**
If you set the environment variable CCACHE_UNIFY then ccache will use the C/C++ unifier when hashing the pre-processor output if -g is not used in the compile. The unifier is slower than a normal hash, so setting this environment variable loses a little bit of speed, but it means that ccache can take advantage of not recompiling when the changes to the source code consist of reformatting only. Note that using CCACHE_UNIFY changes the hash, so cached compiles with CCACHE_UNIFY set cannot be used when CCACHE_UNIFY is not set and vice versa. The reason the unifier is off by default is that it can give incorrect line number information in compiler warning messages.

**CCACHE_EXTENSION**
Normally ccache tries to automatically determine the extension to use for intermediate C pre-processor files based on the type of file being compiled. Unfortunately this sometimes doesn't work, for example when using the aCC compiler on HP-UX. On systems like this you can use the CCACHE_EXTENSION option to override the default. On HP-UX set this environment variable to "i" if you use the aCC compiler.

**CCACHE_STRIPC**
If you set the environment variable CCACHE_STRIPC then ccache will strip the -c option when invoking the preprocessor. This option is primarily for the Sun Workshop C++ compiler as without this option an unwarranted warning is displayed: CC: Warning: "-E" redefines product from "object" to "source (stdout)" when -E and -c is used together.

**CCACHE_SWIG**
When using SWIG as the compiler and it does not have 'swig' in the executable name, then the CCACHE_SWIG environment variable needs to be set in order for ccache to work correctly with SWIG. The use of CCACHE_CPP2 is also recommended for SWIG due to some preprocessor quirks, however, use of CCACHE_CPP2 can often be skipped -- check your generated code with and without this option set. Known problems are using preprocessor directives within %inline blocks and the use of '#pragma SWIG'.

## 21.9 CACHE SIZE MANAGEMENT

By default ccache has a one gigabyte limit on the cache size and no maximum number of files. You can set a different limit using the "ccache -M" and "ccache -F" options, which set the size and number of files limits.

When these limits are reached ccache will reduce the cache to 20% below the numbers you specified in order to avoid doing the cache clean operation too often.

## 21.10 CACHE COMPRESSION

By default on most platforms ccache will compress all files it puts into the cache using the zlib compression. While this involves a negligible performance slowdown, it significantly increases the number of files that fit in the cache. You can turn off compression setting the CCACHE_NOCOMPRESS environment variable.

## 21.11 HOW IT WORKS

The basic idea is to detect when you are compiling exactly the same code a 2nd time and use the previously compiled output. You detect that it is the same code by forming a hash of:

- the pre-processor output from running the compiler with -E
- the command line options
- the real compilers size and modification time
- any stderr output generated by the compiler

These are hashed using md4 (a strong hash) and a cache file is formed based on that hash result. When the same compilation is done a second time ccache is able to supply the correct compiler output (including all warnings etc) from the cache.

ccache has been carefully written to always produce exactly the same compiler output that you would get without the cache. If you ever discover a case where ccache changes the output of your compiler then please let me know.

## 21.12 USING CCACHE WITH DISTCC

distcc is a very useful program for distributing compilation across a range of compiler servers. It is often useful to combine distcc with ccache, so that compiles that are done are sped up by distcc, but that ccache avoids the compile completely where possible.

To use distcc with ccache I recommend using the CCACHE_PREFIX option. You just need to set the environment variable CCACHE_PREFIX to 'distcc' and ccache will prefix the command line used with the compiler with the command 'distcc'.

## 21.13 SHARING A CACHE

A group of developers can increase the cache hit rate by sharing a cache directory. The hard links however cause unwanted side effects, as all links to a cached file share the file's modification timestamp. This results in false dependencies to be triggered by timestamp-based build systems whenever another user links to an existing file. Typically, users will see that their libraries and binaries are relinked without reason. To share a cache without side effects, the following conditions need to be met:

- Use the same **CCACHE_DIR** environment variable setting
- Unset the **CCACHE_HARDLINK** environment variable
- Make sure everyone sets the CCACHE_UMASK environment variable to 002, this ensures that cached files are accessible to everyone in the group.
- Make sure that all users have write permission in the entire cache directory (and that you trust all users of the shared cache).
- Make sure that the setgid bit is set on all directories in the cache. This tells the filesystem to inherit group ownership for new directories. The command "chmod g+s `find $CCACHE_DIR -type d`" might be useful for this.
- Set **CCACHE_NOCOMPRESS** for all users, if there are users with versions of ccache that do not support compression.

## 21.14 HISTORY

ccache was inspired by the compilercache shell script written by Erik Thiele and I would like to thank him for an excellent piece of work. See http://www.erikyyy.de/compilercache/ for the Erik's scripts. ccache-swig is a port of the original ccache with support added for use with SWIG.

I wrote ccache because I wanted to get a bit more speed out of a compiler cache and I wanted to remove some of the limitations of the shell-script version.

## 21.15 DIFFERENCES FROM COMPILERCACHE

The biggest differences between Erik's compilercache script and ccache are:

- ccache is written in C, which makes it a bit faster (calling out to external programs is mostly what slowed down the scripts).
- ccache can automatically find the real compiler
- ccache keeps statistics on hits/misses
- ccache can do automatic cache management
- ccache can cache compiler output that includes warnings. In many cases this gives ccache a much higher cache hit rate.
- ccache can handle a much wider ranger of compiler options
- ccache avoids a double call to cpp on a cache miss

## 21.16 CREDITS

Thanks to the following people for their contributions to ccache

- Erik Thiele for the original compilercache script
- Luciano Rocha for the idea of compiling the pre-processor output to avoid a 2nd cpp pass
- Paul Russell for many suggestions and the debian packaging

## 21.17 AUTHOR

ccache was written by Andrew Tridgell https://www.samba.org/~tridge/. ccache was adapted to create ccache-swig for use with SWIG by William Fulton.

If you wish to report a problem or make a suggestion then please email the SWIG developers on the swig-devel mailing list, see https://www.swig.org/mail.html

ccache is released under the GNU General Public License version 2 or later. Please see the file COPYING for license details.

# 22 SWIG and Android


- Overview
- Android examples
  - Examples introduction
  - Simple C example
  - C++ class example
  - Other examples
- C++ STL


This chapter describes SWIG's support of Android.

## 22.1 Overview

The Android chapter is fairly short as support for Android is the same as for Java, where the Java Native Interface (JNI) is used to call from Android Java into C or C++ compiled code. Everything in the Java chapter applies to generating code for access from Android Java code. This chapter contains a few Android specific notes and examples.

## 22.2 Android examples

### 22.2.1 Examples introduction

The examples require the Android SDK and Android NDK which can be installed as per instructions in the links. The Eclipse version is not required for these examples as just the command line tools are used (shown for Linux as the host, but Windows will be very similar, if not identical in most places). Add the SDK tools and NDK tools to your path and create a directory somewhere for your Android projects (adjust PATH as necessary to where you installed the tools):

```
$ export PATH=$HOME/android/android-sdk-linux_x86/tools:$HOME/android/android-sdk-linux_x86/platform-tools:$HOME/android/android-ndk-r6b:$PATH
$ mkdir AndroidApps
$ cd AndroidApps
```

The examples use a target id of 1. This might need changing depending on your setup. After installation of the Android SDK, the available target ids can be viewed by running the command below. Please adjust the id to suit your target device.

```
$ android list targets
```

The following examples are shipped with SWIG under the Examples/android directory and include a Makefile to build and install each example.

### 22.2.2 Simple C example

This simple C example shows how to call a C function as well as read and modify a global variable. First we'll create and build a pure Java Android app. Afterwards the JNI code will be generated by SWIG and built into the app. First create and build an app called `SwigSimple` in a subdirectory called `simple` using the commands below. Adjust the `--target` id as mentioned earlier in the Examples introduction. Managing Projects from the Command Line on the Android developer's site is a useful reference for these steps.

```
$ android create project --target 1 --name SwigSimple --path ./simple --activity SwigSimple --package org.swig.simple
$ cd simple
$ ant debug
```

Modify `src/org/swig/simple/SwigSimple.java` from the default to:

```
package org.swig.simple;

import android.app.Activity;
import android.os.Bundle;
```

```
import android.view.View;
import android.widget.Button;
import android.widget.TextView;
import android.widget.ScrollView;
import android.text.method.ScrollingMovementMethod;

public class SwigSimple extends Activity
{
    TextView outputText = null;
    ScrollView scroller = null;

    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);

        outputText = (TextView)findViewById(R.id.OutputText);
        outputText.setText("Press 'Run' to start...\n");
        outputText.setMovementMethod(new ScrollingMovementMethod());

        scroller = (ScrollView)findViewById(R.id.Scroller);
    }

    public void onRunButtonClick(View view)
    {
      outputText.append("Started...\n");
      nativeCall();
      outputText.append("Finished!\n");

      // Ensure scroll to end of text
      scroller.post(new Runnable() {
        public void run() {
          scroller.fullScroll(ScrollView.FOCUS_DOWN);
        }
      });
    }

    /** Calls into C/C++ code */
    public void nativeCall()
    {
        // TODO
    }
}
```

The above simply adds a *Run* button and scrollable text view as the GUI aspects of the program. The associated resources need to be created, modify `res/layout/main.xml` as follows:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    >
<Button
    android:id="@+id/RunButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Run..."
    android:onClick="onRunButtonClick"
    />
<ScrollView
    android:id="@+id/Scroller"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    >
<TextView
    android:id="@+id/OutputText"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    />
</ScrollView>
</LinearLayout>
```

Rebuild the project with your changes:

```
$ ant debug
```

Although there are no native function calls in the code, yet, you may want to check that this simple pure Java app runs before adding in the native calls. First, set up your Android device for hardware debugging, see Using hardware devices on the Android developer's site. When complete your device should be listed in those attached, something like:

```
$ adb devices
List of devices attached
A32-6DBE0001-9FF80000-015D62C3-02018028 device
```

This means you are now ready to install the application...

```
$ adb install bin/SwigSimple-debug.apk
95 KB/s (4834 bytes in 0.049s)
        pkg: /data/local/tmp/SwigSimple-debug.apk
Success
```

The newly installed 'SwigSimple' app will be amongst all your other applications on the home screen. Run the app and it will show a *Run* button text box below it. Press the *Run* button to see the simple text output.

The application can be uninstalled like any other application and in fact must be uninstalled before installing an updated version. Uninstalling is quite easy too from your host computer:

```
$ adb uninstall org.swig.simple
Success
```

Now that you have a pure Java Android app working, let's add some JNI code generated from SWIG.

First create a `jni` subdirectory and then create some C source code in `jni/example.c`:

```
/* File : example.c */

/* A global variable */
double Foo = 3.0;

/* Compute the greatest common divisor of positive integers */
int gcd(int x, int y) {
  int g;
  g = y;
  while (x > 0) {
    g = x;
    x = y % x;
    y = g;
  }
  return g;
}
```

Create a SWIG interface file for this C code, `jni/example.i` :

```
/* File : example.i */
%module example

%inline %{
extern int    gcd(int x, int y);
extern double Foo;
%}
```

Invoke SWIG as follows:

```
$ swig -java -package org.swig.simple -outdir src/org/swig/simple -o jni/example_wrap.c jni/example.i
```

SWIG generates the following files:

- `src/org/swig/simple/exampleJNI.java`
- `src/org/swig/simple/example.java`
- `jni/example_wrap.c`

Next we need to create a standard Android NDK build system file `jni/Android.mk`:

```
# File: Android.mk
LOCAL_PATH := $(call my-dir)

include $(CLEAR_VARS)

LOCAL_MODULE    := example
LOCAL_SRC_FILES := example_wrap.c example.c

include $(BUILD_SHARED_LIBRARY)
```

See the Android NDK documentation for more on the NDK build system and getting started with the NDK. A simple invocation of ndk-build will compile the .c files and generate a shared object/system library. Output will be similar to:

```
$ ndk-build
Compile thumb  : example <= example_wrap.c
Compile thumb  : example <= example.c
SharedLibrary  : libexample.so
Install        : libexample.so => libs/armeabi/libexample.so
```

Now that the C JNI layer has been built, we can write Java code to call into the this layer. Modify the `nativeCall` method in `src/org/swig/simple/SwigSimple.java` to call the JNI code as follows and add the static constructor to load the system library containing the compiled JNI C code:

```
   /** Calls into C/C++ code */
   public void nativeCall()
   {
     // Call our gcd() function

     int x = 42;
     int y = 105;
     int g = example.gcd(x, y);
     outputText.append("The greatest common divisor of " + x + " and " + y + " is " + g + "\n");

     // Manipulate the Foo global variable

     // Output its current value
     double foo = example.getFoo();
     outputText.append("Foo = " + foo + "\n");

     // Change its value
     example.setFoo(3.1415926);

     // See if the change took effect
     outputText.append("Foo = " + example.getFoo() + "\n");
```

```
    // Restore value
    example.setFoo(foo);
  }

  /** static constructor */
  static {
      System.loadLibrary("example");
  }
```

Compile the Java code as usual, uninstall the old version of the app if still installed and re-install the new app:

```
$ ant debug
$ adb uninstall org.swig.simple
$ adb install bin/SwigSimple-debug.apk
```

Run the app again and this time you will see the output pictured below, showing the result of calls into the C code:




### 22.2.3 C++ class example

The steps for calling C++ code are almost identical to those in the previous C code example. All the steps required to compile and use a simple hierarchy of classes for shapes are shown in this example.

First create an Android project called `SwigClass` in a subdirectory called `class`. The steps below create and build the JNI C++ app. Adjust the `--target` id as mentioned earlier in the Examples introduction.

```
$ android create project --target 1 --name SwigClass --path ./class --activity SwigClass --package org.swig.classexample
$ cd class
```

Now create a `jni` subdirectory and then create a C++ header file `jni/example.h` which defines our hierarchy of shape classes:

```
/* File : example.h */

class Shape {
public:
  Shape() {
    nshapes++;
  }
  virtual ~Shape() {
    nshapes--;
  }
  double  x, y;
  void    move(double dx, double dy);
  virtual double area() = 0;
  virtual double perimeter() = 0;
  static  int nshapes;
};

class Circle : public Shape {
private:
  double radius;
public:
  Circle(double r) : radius(r) { }
  virtual double area();
  virtual double perimeter();
};

class Square : public Shape {
private:
  double width;
public:
  Square(double w) : width(w) { }
  virtual double area();
  virtual double perimeter();
};
```

and create the implementation in the `jni/example.cpp` file:

```
/* File : example.cpp */

#include "example.h"
#define M_PI 3.14159265358979323846
```

```
/* Move the shape to a new location */
void Shape::move(double dx, double dy) {
  x += dx;
  y += dy;
}

int Shape::nshapes = 0;

double Circle::area() {
  return M_PI*radius*radius;
}

double Circle::perimeter() {
  return 2*M_PI*radius;
}

double Square::area() {
  return width*width;
}

double Square::perimeter() {
  return 4*width;
}
```

Create a SWIG interface file for this C++ code in `jni/example.i` :

```
/* File : example.i */
%module example

%{
#include "example.h"
%}

/* Let's just grab the original header file here */
%include "example.h"
```

Invoke SWIG as follows, note that the -c++ option is required for C++ code:

```
$ swig -c++ -java -package org.swig.classexample -outdir src/org/swig/classexample -o jni/example_wrap.cpp jni/example.i
```

SWIG generates the following files:

- `src/org/swig/classexample/Square.java`
- `src/org/swig/classexample/exampleJNI.java`
- `src/org/swig/classexample/example.java`
- `src/org/swig/classexample/Circle.java`
- `src/org/swig/classexample/Shape.java`
- `jni/example_wrap.cpp`

Next we need to create an Android NDK build system file for compiling the C++ code `jni/Android.mk`. The `-frtti` compiler flag isn't strictly needed for this example, but is needed for any code that uses C++ RTTI:

```
# File: Android.mk
LOCAL_PATH := $(call my-dir)

include $(CLEAR_VARS)

LOCAL_MODULE    := example
LOCAL_SRC_FILES := example_wrap.cpp example.cpp
LOCAL_CFLAGS    := -frtti

include $(BUILD_SHARED_LIBRARY)
```

A simple invocation of ndk-build will compile the .cpp files and generate a shared object/system library. Output will be similar to:

```
$ ndk-build
Compile++ thumb  : example <= example_wrap.cpp
Compile++ thumb  : example <= example.cpp
StaticLibrary  : libstdc++.a
SharedLibrary  : libexample.so
Install        : libexample.so => libs/armeabi/libexample.so
```

Now that the C JNI layer has been built, we can write Java code to call into this layer. Modify `src/org/swig/classexample/SwigClass.java` from the default to:

```
package org.swig.classexample;

import android.app.Activity;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;
import android.widget.ScrollView;
import android.text.method.ScrollingMovementMethod;

public class SwigClass extends Activity
{
    TextView outputText = null;
    ScrollView scroller = null;

    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState)
    {
```

```
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);

        outputText = (TextView)findViewById(R.id.OutputText);
        outputText.setText("Press 'Run' to start...\n");
        outputText.setMovementMethod(new ScrollingMovementMethod());

        scroller = (ScrollView)findViewById(R.id.Scroller);
    }

    public void onRunButtonClick(View view)
    {
      outputText.append("Started...\n");
      nativeCall();
      outputText.append("Finished!\n");

      // Ensure scroll to end of text
      scroller.post(new Runnable() {
        public void run() {
          scroller.fullScroll(ScrollView.FOCUS_DOWN);
        }
      });
    }

    /** Calls into C/C++ code */
    public void nativeCall()
    {
      // ----- Object creation -----

      outputText.append( "Creating some objects:\n" );
      Circle c = new Circle(10);
      outputText.append( "    Created circle " + c + "\n");
      Square s = new Square(10);
      outputText.append( "    Created square " + s + "\n");

      // ----- Access a static member -----

      outputText.append( "\nA total of " + Shape.getNshapes() + " shapes were created\n" );

      // ----- Member data access -----

      // Notice how we can do this using functions specific to
      // the 'Circle' class.
      c.setX(20);
      c.setY(30);

      // Now use the same functions in the base class
      Shape shape = s;
      shape.setX(-10);
      shape.setY(5);

      outputText.append( "\nHere is their current position:\n" );
      outputText.append( "    Circle = (" + c.getX() + " " + c.getY() + ")\n" );
      outputText.append( "    Square = (" + s.getX() + " " + s.getY() + ")\n" );

      // ----- Call some methods -----

      outputText.append( "\nHere are some properties of the shapes:\n" );
      Shape[] shapes = {c, s};
      for (int i=0; i<shapes.length; i++)
      {
        outputText.append( "   " + shapes[i].toString() + "\n" );
        outputText.append( "        area      = " + shapes[i].area() + "\n" );
        outputText.append( "        perimeter = " + shapes[i].perimeter() + "\n" );
      }

      // Notice how the area() and perimeter() functions really
      // invoke the appropriate virtual method on each object.

      // ----- Delete everything -----

      outputText.append( "\nGuess I'll clean up now\n" );

      // Note: this invokes the virtual destructor
      // You could leave this to the garbage collector
      c.delete();
      s.delete();

      outputText.append( Shape.getNshapes() + " shapes remain\n" );
      outputText.append( "Goodbye\n" );
    }

    /** static constructor */
    static {
        System.loadLibrary("example");
    }
}
```

Note the static constructor and the interesting JNI code is in the `nativeCall` method. The remaining code deals with the GUI aspects which are identical to the previous C simple example. Modify `res/layout/main.xml` to contain the xml for the 'Run' button and scrollable text view:

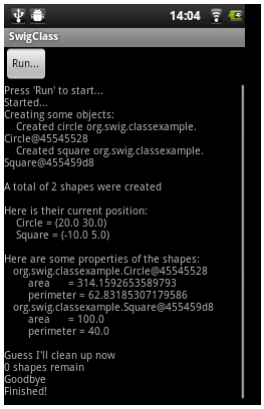
```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    >
<Button
    android:id="@+id/RunButton"
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
    android:text="Run..."
    android:onClick="onRunButtonClick"
    />
<ScrollView
    android:id="@+id/Scroller"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    >
<TextView
    android:id="@+id/OutputText"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    />
</ScrollView>
</LinearLayout>
```

Compile the Java code as usual, uninstall the old version of the app if installed and re-install the new app:

```
$ ant debug
$ adb uninstall org.swig.classexample
$ adb install bin/SwigClass-debug.apk
```

Run the app to see the result of calling the C++ code from Java:




### 22.2.4 Other examples

The Examples/android directory contains further examples which can be run and installed in a similar manner to the previous two examples.

Note that the 'extend' example is demonstrates the directors feature. Normally C++ exception handling and the STL is not available by default in the version of g++ shipped with Android, but this example turns these features on as described in the next section.

## 22.3 C++ STL

Should the C++ Standard Template Library (STL) be required, an `Application.mk` file needs to be created in the same directory as the `Android.mk` directory containing information about the STL to use. See the NDK documentation in the $NDKROOT/docs folder especially CPLUSPLUS-SUPPORT.html. Below is an example of the `Application.mk` file to make the STLport static library available for use:

```
# File: Application.mk
APP_STL := gnustl_static
```

# 23 SWIG and C#


- Introduction
  - SWIG 2 Compatibility
  - Additional command line options
- Differences to the Java module
- Type mapping
  - Primitive types
  - Other types
  - Void pointers
- C# Arrays
  - The SWIG C arrays library
  - Managed arrays using P/Invoke default array marshalling
  - Managed arrays using pinning
- C# Exceptions
  - C# exception example using "check" typemap
  - C# exception example using %exception
  - C# exception example using exception specifications
  - Custom C# ApplicationException example
- C# Directors
  - Directors example
  - Directors implementation
  - Director caveats
- Multiple modules
- C# named and optional arguments
- C# Typemap examples
  - Memory management when returning references to member variables
  - Memory management for objects passed to the C++ layer
  - Date marshalling using the csin typemap and associated attributes
  - A date example demonstrating marshalling of C# properties

- Date example demonstrating the 'pre' and 'post' typemap attributes for directors
- Turning proxy classes into partial classes
- Turning proxy classes into sealed classes
- Extending proxy classes with additional C# code
- Underlying type for enums


## 23.1 Introduction

The purpose of the C# module is to offer an automated way of accessing existing C/C++ code from .NET languages. The wrapper code implementation uses C# and the Platform Invoke (PInvoke) interface to access natively compiled C/C++ code. The PInvoke interface has been chosen over Microsoft's Managed C++ interface as it is portable to both Microsoft Windows and non-Microsoft platforms. PInvoke is part of the ECMA/ISO C# specification. It is also better suited for robust production environments due to the Managed C++ flaw called the Mixed DLL Loading Problem. SWIG C# works equally well on non-Microsoft operating systems such as Linux, Solaris and Apple Mac using Mono.

SWIG 3 and later requires .NET 2.0 at a minimum. There are some minor exceptions, where the minimum required is .NET 4.0. This is when using the `std::complex` and `std::list` STL containers.

To get the most out of this chapter an understanding of interop is required. The Microsoft Developer Network (MSDN) has a good reference guide in a section titled "Interop Marshaling". Monodoc, available from the Mono project, has a very useful section titled Interop with native libraries.

### 23.1.1 SWIG 2 Compatibility

In order to minimize name collisions between names generated based on input to SWIG and names used in the generated code from the .NET framework, SWIG 3 fully qualifies the use of all .NET types. Furthermore, SWIG 3 avoids `using` directives in generated code. This breaks backwards compatibility with typemaps, pragmas, etc written for use with SWIG 2 that assume the presence of `using System;` or `using System.Runtime.InteropServices;` directives in the intermediate class imports, module imports, or proxy imports. SWIG 3 supports backwards compatibility though the use of the `SWIG2_CSHARP` macro. If `SWIG2_CSHARP` is defined, SWIG 3 generates `using` directives in the intermediate class, module class, and proxy class code similar to those generated by SWIG 2. This can be done without modifying any of the input code by passing the `-DSWIG2_CSHARP` commandline parameter when executing `swig`.

### 23.1.2 Additional command line options

The following table lists the additional commandline options available for the C# module. They can also be seen by using:

```
swig -csharp -help
```

**C# specific options**

| | |
|---|---|
| -dllimport <dl> | Override DllImport attribute name to <dl> |
| -namespace <nm> | Generate wrappers into C# namespace <nm> |
| -noproxy | Generate the low-level functional interface instead of proxy classes |
| -oldvarnames | Old intermediary method names for variable wrappers |
| -outfile <file> | Write all C# into a single <file> located in the output directory |
| -doxygen | Convert C++ doxygen comments to CSharp comments |

The -outfile option combines all the generated C# code into a single output file instead of creating multiple C# files. The default, when this option is not provided, is to generate separate .cs files for the module class, intermediary class and each of the generated proxy and type wrapper classes. Note that the file extension (.cs) will not be automatically added and needs to be provided. Due to possible compiler limits it is not advisable to use `-outfile` for large projects.

## 23.2 Differences to the Java module

The C# module is very similar to the Java module, so until some more complete documentation has been written, please use the Java documentation as a guide to using SWIG with C#. The C# module has the same major SWIG features as the Java module. The rest of this section should be read in conjunction with the Java documentation as it lists the main differences. The most notable differences to Java are the following:

- When invoking SWIG use the `-csharp` command line option instead of `-java`.
- The `-nopgcpp` command line option does not exist.
- The `-package` command line option does not exist.
- The `-namespace <name>` commandline option will generate all code into the namespace specified by `<name>`. C# supports nested namespaces that are not lexically nested, so nested namespaces will of course also work. For example: `-namespace com.bloggs.widget` , will generate code into C# namespaces:

  ```
  namespace com.bloggs.widget {
    ...
  }
  ```

  Note that by default, the generated C# classes have no namespace and the module name is unrelated to namespaces. The module name is just like in Java and is merely used to name some of the generated classes.
- The nspace feature is also supported as described in this general section with a C# example. Unlike Java which requires the use of the -package option when using the `nspace` feature, the -namespace option is not mandatory for C#.
- The `-dllimport <name>` commandline option specifies the name of the DLL for the `DllImport` attribute for every PInvoke method. If this commandline option is not given, the `DllImport` DLL name is the same as the module name. This option is useful for when one wants to invoke SWIG multiple times on different modules, yet compile all the resulting code into a single DLL.
- C/C++ variables are wrapped with C# properties and not JavaBean style getters and setters.
- Global constants are generated into the module class. There is no constants interface.
- There is no implementation for type unsafe enums - not deemed necessary.
- The default enum wrapping approach is proper C# enums, not typesafe enums.
  Note that %csconst(0) will be ignored when wrapping C/C++ enums with proper C# enums. This is because C# enum items must be initialised from a compile time constant. If an enum item has an initialiser and the initialiser doesn't compile as C# code, then the %csconstvalue directive must be used as %csconst(0) will have no effect. If it was used, it would generate an illegal runtime initialisation via a PInvoke call.
- C# doesn't support the notion of throws clauses. Therefore there is no 'throws' typemap attribute support for adding exception classes to a throws clause. Likewise there is no need for an equivalent to `%javaexception`. In fact, throwing C# exceptions works quite differently, see C# Exceptions below.
- The majority of the typemaps are in csharp.swg, not java.swg.

- Typemap equivalent names:

```
jni                     -> ctype
jtype                   -> imtype
jstype                  -> cstype
javain                  -> csin
javaout                 -> csout
javadirectorin          -> csdirectorin
javadirectorout         -> csdirectorout
javainterfaces          -> csinterfaces and csinterfaces_derived
javabase                -> csbase
javaclassmodifiers      -> csclassmodifiers
javacode                -> cscode
javaimports             -> csimports
javabody                -> csbody
```

```
javafinalize                -> csfinalize
javadestruct                -> csdisposing and csdispose
javadestruct_derived        -> csdisposing_derived and csdispose_derived
javainterfacemodifiers      -> csinterfacemodifiers
javainterfacecode           -> csinterfacecode
```

- Typemap macros:

```
SWIG_JAVABODY_PROXY         -> SWIG_CSBODY_PROXY
SWIG_JAVABODY_TYPEWRAPPER   -> SWIG_CSBODY_TYPEWRAPPER
```

- Additional typemaps:

```
csvarin                     C# code property set typemap
csvarout                    C# code property get typemap
csattributes                C# attributes for attaching to proxy classes/enums
```

- Additional typemap attributes:

  The "null" attribute in the "out" typemap can be specified to provide a value for `$null` to expand into for wrapped functions that return non-void. Normally the default value of `0` is used. For example this is needed if you change the return type to void:

```
%typemap(ctype) Status "void"
%typemap(out, null="") Status { ... }
```

- Feature equivalent names:

```
%javaconst                  -> %csconst
%javaconstvalue             -> %csconstvalue
%javamethodmodifiers        -> %csmethodmodifiers
```

- Pragma equivalent names:

```
%pragma(java)               -> %pragma(csharp)
jniclassbase                -> imclassbase
jniclassclassmodifiers      -> imclassclassmodifiers
jniclasscode                -> imclasscode
jniclassimports             -> imclassimports
jniclassinterfaces          -> imclassinterfaces
```

- Special variable equivalent names:

```
$javaclassname              -> $csclassname
$&javaclassname             -> $&csclassname
$*javaclassname             -> $*csclassname
$javaclazzname              -> $csclazzname
$javainput                  -> $csinput
$jnicall                    -> $imcall
$javainterfacename          -> $csinterfacename
$&javainterfacename         -> $&csinterfacename
$*javainterfacename         -> $*csinterfacename
```

- Unlike the "javain" typemap, the "csin" typemap does not support the 'pgcpp' attribute as the C# module does not have a premature garbage collection prevention parameter. The "csin" typemap supports additional optional attributes called 'cshin' and 'terminator'. The "csdirectorin" typemap supports additional optional attributes called 'terminator'. The 'cshin' attribute should contain the parameter type and name whenever a constructor helper function is generated due to the 'pre' or 'post' attributes. The 'terminator' attribute normally just contains a closing brace for when the 'pre' attribute contains an opening brace, such as when a C# `using` or `fixed` block is started. Note that 'pre', 'post', 'terminator' and 'cshin' attributes are not used for marshalling the property set. Please see the Date marshalling example and Date marshalling of properties example for further understanding of these "csin" applicable attributes. Please see the Date marshalling director example for further understanding of the "csdirectorin" attributes.

- Support for asymmetric type marshalling. The 'ctype', 'imtype' and 'cstype' typemaps support an optional `out` attribute which is used for output types. If this typemap attribute is specified, then the type specified in the attribute is used for output types and the type specified in the typemap itself is used for the input type. If this typemap attribute is not specified, then the type used for both input and output is the type specified in the typemap. An example shows that `char *` could be marshalled in different ways,

```
%typemap(imtype, out="global::System.IntPtr") char * "string"
char * function(char *);
```

  The output type is thus IntPtr and the input type is string. The resulting intermediary C# code is:

```
public static extern global::System.IntPtr function(string jarg1);
```

- Support for type attributes. The 'imtype' and 'cstype' typemaps can have an optional `inattributes` and `outattributes` typemap attribute. The 'imtype' typemap can also have an optional `directorinattributes` and `directoroutattributes` typemap attribute which attaches to director delegates, an implementation detail of directors, see directors implementation. Note that there are C# attributes and typemap attributes, don't get confused between the two!! The C# attributes specified in these typemap attributes are generated wherever the type is used in the C# wrappers. These can be used to specify any C# attribute associated with a C/C++ type, but are more typically used for the C# `MarshalAs` attribute. For example:

```
%typemap(imtype,
         inattributes="[global::System.Runtime.InteropServices.MarshalAs(UnmanagedType.LPStr)]",
         outattributes="[return: global::System.Runtime.InteropServices.MarshalAs(UnmanagedType.LPStr)]") const char * "String"

const char * GetMsg() {}
void SetMsg(const char *msg) {}
```

  The intermediary class will then have the marshalling as specified by everything in the 'imtype' typemap:

```
class examplePINVOKE {
  ...
  [global::System.Runtime.InteropServices.DllImport("example", EntryPoint="CSharp_GetMsg")]
  [return: global::System.Runtime.InteropServices.MarshalAs(UnmanagedType.LPStr)]
  public static extern String GetMsg();

  [global::System.Runtime.InteropServices.DllImport("example", EntryPoint="CSharp_SetMsg")]
  public static extern void SetMsg([global::System.Runtime.InteropServices.MarshalAs(UnmanagedType.LPStr)]String jarg1);
}
```

Note that the `DllImport` attribute is always generated, irrespective of any additional attributes specified.

These attributes are associated with the C/C++ parameter type or return type, which is subtly different to the attribute features and typemaps covered next. Note that all these different C# attributes can be combined so that a method has more than one attribute.

The `directorinattributes` and `directoroutattributes` typemap attribute are attached to the delegates in the director class, for example, the SwigDelegateBase_0

- Support for attaching C# attributes to wrapped methods, variables and enum values. This is done using the `%csattributes` feature, see %feature directives. Note that C# attributes are attached to proxy classes and enums using the `csattributes` typemap. For example, imagine we have a custom attribute class, `ThreadSafeAttribute`, for labelling thread safety. The following SWIG code shows how to attach this C# attribute to some methods and the class declaration itself:

```
%typemap(csattributes) AClass          "[ThreadSafe]"
%csattributes AClass::AClass(double d) "[ThreadSafe(false)]"
%csattributes AClass::AMethod()        "[ThreadSafe(true)]"

%inline %{
class AClass {
public:
  AClass(double a) {}
  void AMethod() {}
};
%}
```

will generate a C# proxy class:

```
[ThreadSafe]
public class AClass : global::System.IDisposable {
  ...
  [ThreadSafe(false)]
  public AClass(double a) ...

  [ThreadSafe(true)]
  public void AMethod() ...
}
```

If C# attributes need adding to the `set` or `get` part of C# properties, when wrapping C/C++ variables, they can be added using the 'csvarin' and 'csvarout' typemaps respectively. Note that the type used for the property is specified in the 'cstype' typemap. If the 'out' attribute exists in this typemap, then the type used is from the 'out' attribute.

An example for attaching attributes to the enum and enum values is shown below.

```
%typemap(csattributes) Couleur "[global::System.ComponentModel.Description(\"Colours\")]"
%csattributes Rouge "[global::System.ComponentModel.Description(\"Red\")]"
%csattributes Vert "[global::System.ComponentModel.Description(\"Green\")]"
%inline %{
  enum Couleur { Rouge, Orange, Vert };
%}
```

which will result in the following C# enum:

```
[global::System.ComponentModel.Description("Colours")]
public enum Couleur {
  [global::System.ComponentModel.Description("Red")]
  Rouge,
  Orange,
  [global::System.ComponentModel.Description("Green")]
  Vert
}
```

- The intermediary classname has `PINVOKE` appended after the module name instead of `JNI`, for example `modulenamePINVOKE` .
- The `%csmethodmodifiers` feature can also be applied to variables as well as methods. In addition to the default `public` modifier that SWIG generates when `%csmethodmodifiers` is not specified, the feature will also replace the `virtual`/`new` /`override` modifiers that SWIG thinks is appropriate. This feature is useful for some obscure cases where SWIG might get the `virtual`/`new`/`override` modifiers incorrect, for example with multiple inheritance.
- The name of the intermediary class can be changed from its default, that is, the module name with PINVOKE appended after it. The module directive attribute `imclassname` is used to achieve this:

```
%module(imclassname="name") modulename
```

If `name` is the same as `modulename` then the module class name gets changed from `modulename` to `modulenameModule`.

- The `%module` directive supports the `csbegin` option for adding code to the start of every generated C# file. This is useful for adding common comments, using statements and/or preprocessor statements into all generated .cs files. For example, C# 8 nullable reference types can be enabled via a C# preprocessor directive by adding `#nullable enable` into C# files as follows:

```
%module(csbegin="#nullable enable\n") mymodule
```

It might be easier to use a macro for multiple lines of code, for example:

```
%define CSBEGIN_CODE
"
/* Copyright statement */
using System.Text;
#nullable enable
"
%enddef

%module(csbegin=CSBEGIN_CODE) mymodule
```

- There is no additional 'premature garbage collection prevention parameter' as the marshalling of the `HandleRef` object takes care of ensuring a reference to the proxy class is held until the unmanaged call completed.

**`$dllimport`**
This is a C# only special variable that can be used in typemaps, pragmas, features etc. The special variable will get translated into the value specified by the `-dllimport` commandline option if specified, otherwise it is equivalent to the **$module** special variable.

**`$imclassname`**
This special variable expands to the intermediary class name. For C# this is usually the same as '$modulePINVOKE' ('$moduleJNI' for Java), unless the imclassname attribute is specified in the %module directive.

**`$imfuncname`**
This special variable expands to the name of the function in the intermediary class that will be used in $imcall. Like, $imcall, this special variable is only expanded in the "csout", "csvarin" and "csvarout" typemaps.

The directory `Examples/csharp` has a number of simple examples. Visual Studio .NET 2003 solution and project files are available for compiling with the Microsoft .NET C# compiler on Windows. This also works with newer versions of Visual Studio if you allow it to convert the solution to the latest version. If your SWIG installation went well on a Unix environment and your C# compiler was detected, you should be able to type `make` in each example directory. After SWIG has run and both the C# and C/C++ compilers have finished building, the examples will be run, by either running `runme.exe` or by running `mono runme.exe` (Mono C# compiler). Windows users can also get the examples working using a Cygwin or MinGW environment for automatic configuration of the example makefiles. Any one of the C# compilers (Mono or Microsoft) can be detected from within a Cygwin or MinGW environment if installed in your path.

## 23.3 Type mapping

The marshalling of the types and typemaps used for marshalling across the managed/unmanaged layers are discussed in this section. The interested reader will find the implementation in the csharp.swg file.

### 23.3.1 Primitive types

Primitive types are marshalled between the unmanaged and managed layers as blittable types.

- The first column in the table below shows various C/C++ types that might be parsed by SWIG.
- The second column contains the default type provided by the 'ctype' typemap, that is, the type used for marshalling on the C side.
- The third column shows the default type provided by both the 'imtype' and the 'cstype' typemaps, that is, the equivalent type on the C# side.
- The fourth column shows the size or number of bytes of the 'imtype'/'cstype', which may or may not match the size of the C/C++ type and is discussed next.

| C/C++ type | ctype | imtype/cstype | Size |
|---|---|---|---|
| bool<br>const bool & | unsigned int | bool | 1 |
| char<br>const char & | char | char | 1 |
| signed char<br>const signed char & | signed char | sbyte | 1 |
| unsigned char<br>const unsigned char & | unsigned char | byte | 1 |
| short<br>const short & | short | short | 2 |
| unsigned short<br>const unsigned short & | unsigned short | ushort | 2 |
| int<br>const int & | int | int | 4 |
| unsigned int<br>const unsigned int & | unsigned int | uint | 4 |
| long<br>const long & | int | int | 4 |
| unsigned long<br>const unsigned long & | unsigned int | uint | 4 |
| long long<br>const long long & | long long | long | 8 |
| unsigned long long<br>const unsigned long long & | unsigned long long | ulong | 8 |
| float<br>const float & | float | float | 4 |
| double<br>const double & | double | double | 8 |
| size_t<br>const size_t & | unsigned int | uint | 4 |

The size in bytes of the C type, 'ctype', should match the C# type, 'imtype' for blitting across the managed/unmanaged layers. They do match across the common 32-bit and 64-bit operating systems, Unix, Windows and MacOS, except for the C long/unsigned long and size_t types. From the table above the default is to handle C long and size_t as a 32-bit (4 byte) type, so large numbers could be truncated on some 64-bit operating systems. If `SWIGWORDSIZE64` is defined the C long type is instead handled as a 64-bit (8 byte) type, as per the table below.

| C/C++ type | ctype | imtype/cstype | Size |
|---|---|---|---|
| long<br>const long & | long long | long | 8 |
| unsigned long<br>const unsigned long & | unsigned long long | ulong | 8 |

However, truncation may then occur when the C long type is actually 32-bits (4 bytes). It's best to avoid using C long for portability across different operating systems!

If you need to support long on a range of systems where the size of long varies, then steps must be taken before invoking SWIG to determine whether or not to define `SWIGWORDSIZE64` when invoking SWIG.

In order to treat the C size_t type as a 64-bit (8 byte) type, apply the 64-bit typemaps as follows:

```
%apply unsigned long long { size_t };
%apply const unsigned long long & { const size_t & };
```

The net effect then changes from the default shown earlier to:

| C/C++ type | ctype | imtype/cstype | Size |
|---|---|---|---|
| size_t<br>const size_t & | unsigned long<br>long | ulong | 8 |

If you need to support size_t on a range of systems where the size of size_t varies, then steps must be taken before invoking SWIG to determine whether or not to apply the typemaps. Conditionally applying the typemaps using a macro is easily done. For example define `MY_SIZET_WORDSIZE64` to generate 64-bit (8 byte) handling using the following:

```
#if defined(MY_SIZET_WORDSIZE64)
%apply unsigned long long { size_t };
%apply const unsigned long long & { const size_t & };
#endif
```

### 23.3.2 Other types

The table below shows the equivalent mappings for pointers and strings. Classes and structs are marshalled using a pointer to the instance of the object. Note the types in the 'imtype' and 'cstype' typemaps can be different.

- The 'imtype' type is used for marshalling across the managed and unmanaged boundary.
- The 'cstype' is the final C# proxy or intermediary class type.
- In the table below, the 'imtype' type is used for marshalling from the managed to the unmanaged layer.
- The 'imtype out' type is used for marshalling from the unmanaged to the managed layer.

| C/C++ type | ctype | imtype | imtype out | cstype |
|---|---|---|---|---|
| char *<br>char [] | char * | string | string | string |
| void * | void * | System.Runtime.InteropServices.HandleRef | System.IntPtr | SWIGTYPE_p_void |

### 23.3.3 Void pointers

By default SWIG treats `void *` as any other pointer and hence marshalls it as a type wrapper class called `SWIGTYPE_p_void` , as shown in the table in the previous section. If you want to marshall with the .NET `System.IntPtr` type instead, there is a simple set of named typemaps called `void *VOID_INT_PTR` that can be used. The net effect is then:

| C/C++ type | ctype | imtype | imtype out | cstype |
|---|---|---|---|---|
| void *VOID_INT_PTR | void * | System.IntPtr | System.IntPtr | System.IntPtr |

This is achieved by applying them like any other named typemaps:

```
%apply void *VOID_INT_PTR { void * }
void * f(void *v);
```

## 23.4 C# Arrays

There are various ways to pass arrays from C# to C/C++. The default wrapping treats arrays as pointers and as such simple type wrapper classes are generated, eg `SWIGTYPE_p_int` when wrapping the C type `int []` or `int *`. This gives a rather restricted use of the underlying unmanaged code and the most practical way to use arrays is to enhance or customise with one of the following three approaches; namely the SWIG C arrays library, P/Invoke default array marshalling or pinned arrays.

### 23.4.1 The SWIG C arrays library

The C arrays library keeps all the array memory in the unmanaged layer. The library is available to all language modules and is documented in the carrays.i library section. Please refer to this section for details, but for convenience, the C# usage for the two examples outlined there is shown below.

For the `%array_functions` example, the equivalent usage would be:

```
SWIGTYPE_p_double a = example.new_doubleArray(10);  // Create an array
for (int i=0; i<10; i++)
  example.doubleArray_setitem(a, i, 2*i);           // Set a value
example.print_array(a);                             // Pass to C
example.delete_doubleArray(a);                      // Destroy array
```

and for the `%array_class` example, the equivalent usage would be:

```
doubleArray c = new doubleArray(10);    // Create double[10]
for (int i=0; i<10; i++)
  c.setitem(i, 2*i);                    // Assign values
example.print_array(c.cast());          // Pass to C
```

### 23.4.2 Managed arrays using P/Invoke default array marshalling

In the P/Invoke default marshalling scheme, one needs to designate whether the invoked function will treat a managed array parameter as input, output, or both. When the function is invoked, the CLR allocates a separate chunk of memory as big as the given managed array, which is automatically released at the end of the function call. If the array parameter is marked as being input, the content of the managed array is copied into this buffer when the call is made. Correspondingly, if the array parameter is marked as being output, the contents of the reserved buffer are copied back into the managed array after the call returns. A pointer to this buffer is passed to the native function.

The reason for allocating a separate buffer is to leave the CLR free to relocate the managed array object during garbage collection. If the overhead caused by the copying is causing a significant performance penalty, consider pinning the managed array and passing a direct reference as described in the next section.

For more information on the subject, see the Default Marshaling for Arrays article on MSDN.

The P/Invoke default marshalling is supported by the `arrays_csharp.i` library via the INPUT, OUTPUT and INOUT typemaps. Let's look at some example usage. Consider the following C function:

```
void myArrayCopy(int *sourceArray, int *targetArray, int nitems);
```

We can now instruct SWIG to use the default marshalling typemaps by

```
%include "arrays_csharp.i"

%apply int INPUT[]  {int *sourceArray}
%apply int OUTPUT[] {int *targetArray}
```

As a result, we get the following method in the module class:

```
public static void myArrayCopy(int[] sourceArray, int[] targetArray, int nitems) {
  examplePINVOKE.myArrayCopy(sourceArray, targetArray, nitems);
}
```

If we look beneath the surface at the corresponding intermediary class code, we see that SWIG has generated code that uses attributes (from the System.Runtime.InteropServices namespace) to tell the CLR to use default marshalling for the arrays:

```
[global::System.Runtime.InteropServices.DllImport("example", EntryPoint="CSharp_myArrayCopy")]
public static extern void myArrayCopy([global::System.Runtime.InteropServices.In, global::System.Runtime.InteropServices.MarshalAs(UnmanagedT
                                      [global::System.Runtime.InteropServices.Out, global::System.Runtime.InteropServices.MarshalAs(Unmanaged
                                       int jarg3);
```

As an example of passing an inout array (i.e. the target function will both read from and write to the array), consider this C function that swaps a given number of elements in the given arrays:

```
void myArraySwap(int *array1, int *array2, int nitems);
```

Now, we can instruct SWIG to wrap this by

```
%include "arrays_csharp.i"

%apply int INOUT[] {int *array1}
%apply int INOUT[] {int *array2}
```

This results in the module class method

```
  public static void myArraySwap(int[] array1, int[] array2, int nitems) {
    examplePINVOKE.myArraySwap(array1, array2, nitems);
  }
```

and intermediary class method

```
  [global::System.Runtime.InteropServices.DllImport("example", EntryPoint="CSharp_myArraySwap")]
  public static extern void myArraySwap([global::System.Runtime.InteropServices.In, global::System.Runtime.InteropServices.Out, global::Syste
                                        [global::System.Runtime.InteropServices.In, global::System.Runtime.InteropServices.Out, global::Syste
                                         int jarg3);
```

### 23.4.3 Managed arrays using pinning

It is also possible to pin a given array in memory (i.e. fix its location in memory), obtain a direct pointer to it, and then pass this pointer to the wrapped C/C++ function. This approach involves no copying, but it makes the work of the garbage collector harder as the managed array object can not be relocated before the fix on the array is released. You should avoid fixing arrays in memory in cases where the control may re-enter the managed side via a callback and/or another thread may produce enough garbage to trigger garbage collection.

For more information, see the fixed statement in the C# language reference.

Now let's look at an example using pinning, thus avoiding the CLR making copies of the arrays passed as parameters. The `arrays_csharp.i` library file again provides the required support via the `FIXED` typemaps. Let's use the same function from the previous section:

```
void myArrayCopy(int *sourceArray, int *targetArray, int nitems);
```

We now need to declare the module class method unsafe, as we are using pointers:

```
%csmethodmodifiers myArrayCopy "public unsafe";
```

Apply the appropriate typemaps to the array parameters:

```
%include "arrays_csharp.i"

%apply int FIXED[] {int *sourceArray}
%apply int FIXED[] {int *targetArray}
```

Notice that there is no need for separate in, out or inout typemaps as is the case when using P/Invoke default marshalling.

As a result, we get the following method in the module class:

```
  public unsafe static void myArrayCopy(int[] sourceArray, int[] targetArray, int nitems) {
    fixed ( int *swig_ptrTo_sourceArray = sourceArray ) {
    fixed ( int *swig_ptrTo_targetArray = targetArray ) {
    {
      examplePINVOKE.myArrayCopy((global::System.IntPtr)swig_ptrTo_sourceArray, (global::System.IntPtr)swig_ptrTo_targetArray,
                                 nitems);
    }
    }
    }
```

```
  }
```

On the method signature level the only difference to the version using P/Invoke default marshalling is the "unsafe" quantifier, which is required because we are handling pointers.

Also the intermediary class method looks a little different from the default marshalling example - the method is expecting an IntPtr as the parameter type.

```
[global::System.Runtime.InteropServices.DllImport("example", EntryPoint="CSharp_myArrayCopy")]
public static extern void myArrayCopy(global::System.IntPtr jarg1, global::System.IntPtr jarg2, int jarg3);
```

## 23.5 C# Exceptions

It is possible to throw a C# Exception from C/C++ code. SWIG already provides the framework for throwing C# exceptions if it is able to detect that a C++ exception could be thrown. Automatically detecting that a C++ exception could be thrown is only possible when a C++ exception specification is used, see Exception specifications. The Exception handling with %exception section details the `%exception` feature. Customised code for handling exceptions with or without a C++ exception specification is possible and the details follow. However anyone wishing to do this should be familiar with the contents of the sections referred to above.

Unfortunately a C# exception cannot simply be thrown from unmanaged code for a variety of reasons. Most notably being that throwing a C# exception results in exceptions being thrown across the C PInvoke interface and C does not understand exceptions. The design revolves around a C# exception being constructed and stored as a pending exception, to be thrown only when the unmanaged code has completed. Implementing this is a tad involved and there are thus some unusual typemap constructs. Some practical examples follow and they should be read in conjunction with the rest of this section.

First some details about the design that must be followed. Each typemap or feature that generates **unmanaged code** supports an attribute called `canthrow`. This is simply a flag which when set indicates that the code in the typemap/feature has code which might want to throw a C# exception. The code in the typemap/feature can then raise a C# exception by calling one of the C functions, `SWIG_CSharpSetPendingException()` or `SWIG_CSharpSetPendingExceptionArgument()`. When called, the function makes a callback into the managed world via a delegate. The callback creates and stores an exception ready for throwing when the unmanaged code has finished. The typemap/feature unmanaged code is then expected to force an immediate return from the unmanaged wrapper function, so that the pending managed exception can then be thrown. The support code has been carefully designed to be efficient as well as thread-safe. However to achieve the goal of efficiency requires some optional code generation in the **managed code** typemaps. Code to check for pending exceptions is generated if and only if the unmanaged code has code to set a pending exception, that is if the `canthrow` attribute is set. The optional managed code is generated using the `excode` typemap attribute and `$excode` special variable in the relevant managed code typemaps. Simply, if any relevant unmanaged code has the `canthrow` attribute set, then any occurrences of `$excode` is replaced with the code in the `excode` attribute. If the `canthrow` attribute is not set, then any occurrences of `$excode` are replaced with nothing.

The prototypes for the `SWIG_CSharpSetPendingException()` and `SWIG_CSharpSetPendingExceptionArgument()` functions are

```
static void SWIG_CSharpSetPendingException(SWIG_CSharpExceptionCodes code,
                                           const char *msg);

static void SWIG_CSharpSetPendingExceptionArgument(SWIG_CSharpExceptionArgumentCodes code,
                                                   const char *msg,
                                                   const char *param_name);
```

The first parameter defines which .NET exceptions can be thrown:

```
typedef enum {
  SWIG_CSharpApplicationException,
  SWIG_CSharpArithmeticException,
  SWIG_CSharpDivideByZeroException,
  SWIG_CSharpIndexOutOfRangeException,
  SWIG_CSharpInvalidCastException,
  SWIG_CSharpInvalidOperationException,
  SWIG_CSharpIOException,
  SWIG_CSharpNullReferenceException,
  SWIG_CSharpOutOfMemoryException,
  SWIG_CSharpOverflowException,
  SWIG_CSharpSystemException
} SWIG_CSharpExceptionCodes;

typedef enum {
  SWIG_CSharpArgumentException,
  SWIG_CSharpArgumentNullException,
  SWIG_CSharpArgumentOutOfRangeException,
} SWIG_CSharpExceptionArgumentCodes;
```

where, for example, `SWIG_CSharpApplicationException` corresponds to the .NET exception, `ApplicationException`. The `msg` and `param_name` parameters contain the C# exception message and parameter name associated with the exception.

The `%exception` feature in C# has the `canthrow` attribute set. The `%csnothrowexception` feature is like `%exception`, but it does not have the `canthrow` attribute set so should only be used when a C# exception is not created.

### 23.5.1 C# exception example using "check" typemap

Let's say we have the following simple C++ method:

```
void positivesonly(int number);
```

and we want to check that the input `number` is always positive and if not throw a C# `ArgumentOutOfRangeException`. The "check" typemap is designed for checking input parameters. Below you will see the `canthrow` attribute is set because the code contains a call to `SWIG_CSharpSetPendingExceptionArgument()`. The full example follows:

```
%module example

%typemap(check, canthrow=1) int number %{
if ($1 < 0) {
  SWIG_CSharpSetPendingExceptionArgument(SWIG_CSharpArgumentOutOfRangeException,
                                         "only positive numbers accepted", "number");
  return $null;
}
// SWIGEXCODE is a macro used by many other csout typemaps
%define SWIGEXCODE
 "\n    if ($modulePINVOKE.SWIGPendingException.Pending)"
 "\n      throw $modulePINVOKE.SWIGPendingException.Retrieve();"
%enddef
%typemap(csout, excode=SWIGEXCODE) void {
    $imcall;$excode
```

```
  }
%}

%inline %{

void positivesonly(int number) {
}

%}
```

When the following C# code is executed:

```
public class runme {
  static void Main() {
    example.positivesonly(-1);
  }
}
```

The exception is thrown:

```
Unhandled Exception: System.ArgumentOutOfRangeException: only positive numbers accepted
Parameter name: number
in <0x00034> example:positivesonly (int)
in <0x0000c> runme:Main ()
```

Now let's analyse the generated code to gain a fuller understanding of the typemaps. The generated unmanaged C++ code is:

```
SWIGEXPORT void SWIGSTDCALL CSharp_positivesonly(int jarg1) {
  int arg1 ;

  arg1 = (int)jarg1;

  if (arg1 < 0) {
    SWIG_CSharpSetPendingExceptionArgument(SWIG_CSharpArgumentOutOfRangeException,
      "only positive numbers accepted", "number");
    return ;
  }

  positivesonly(arg1);

}
```

This largely comes from the "check" typemap. The managed code in the module class is:

```
public class example {
  public static void positivesonly(int number) {
    examplePINVOKE.positivesonly(number);
    if (examplePINVOKE.SWIGPendingException.Pending)
      throw examplePINVOKE.SWIGPendingException.Retrieve();
  }

}
```

This comes largely from the "csout" typemap.

The "csout" typemap is the same as the default void "csout" typemap so is not strictly necessary for the example. However, it is shown to demonstrate what managed output code typemaps should contain, that is, a `$excode` special variable and an `excode` attribute. Also note that `$excode` is expanded into the code held in the`excode` attribute. The `$imcall` as always expands into `examplePINVOKE.positivesonly(number)`. The exception support code in the intermediary class, `examplePINVOKE`, is not shown, but is contained within the inner classes, `SWIGPendingException` and `SWIGExceptionHelper` and is always generated. These classes can be seen in any of the generated wrappers. However, all that is required of a user is as demonstrated in the "csin" typemap above. That is, is to check `SWIGPendingException.Pending` and to throw the exception returned by `SWIGPendingException.Retrieve()`.

If the "check" typemap did not exist, then the following module class would instead be generated:

```
public class example {
  public static void positivesonly(int number) {
    examplePINVOKE.positivesonly(number);
  }

}
```

Here we see the pending exception checking code is omitted. In fact, the code above would be generated if the `canthrow` attribute was not in the "check" typemap, such as:

```
%typemap(check) int number %{
if ($1 < 0) {
  SWIG_CSharpSetPendingExceptionArgument(SWIG_CSharpArgumentOutOfRangeException,
                                         "only positive numbers accepted", "number");
  return $null;
}
%}
```

Note that if SWIG detects you have used `SWIG_CSharpSetPendingException()` or `SWIG_CSharpSetPendingExceptionArgument()` without setting the `canthrow` attribute you will get a warning message similar to

```
example.i:21: Warning 845: Unmanaged code contains a call to a SWIG_CSharpSetPendingException
method and C# code does not handle pending exceptions via the canthrow attribute.
```

Actually it will issue this warning for any function beginning with `SWIG_CSharpSetPendingException`.

### 23.5.2 C# exception example using %exception

Let's consider a similar, but more common example that throws a C++ exception from within a wrapped function. We can use `%exception` as mentioned in Exception handling with %exception.

```
%exception negativesonly(int value) %{
try {
  $action
} catch (std::out_of_range e) {
  SWIG_CSharpSetPendingException(SWIG_CSharpApplicationException, e.what());
  return $null;
}
%}

%inline %{
#include <stdexcept>
void negativesonly(int value) {
  if (value >= 0)
    throw std::out_of_range("number should be negative");
}
%}
```

The generated unmanaged code this time catches the C++ exception and converts it into a C# `ApplicationException`.

```
SWIGEXPORT void SWIGSTDCALL CSharp_negativesonly(int jarg1) {
  int arg1 ;

  arg1 = (int)jarg1;

  try {
    negativesonly(arg1);

  } catch (std::out_of_range e) {
    SWIG_CSharpSetPendingException(SWIG_CSharpApplicationException, e.what());
    return ;
  }
}
```

The managed code generated does check for the pending exception as mentioned earlier as the C# version of `%exception` has the `canthrow` attribute set by default:

```
  public static void negativesonly(int value) {
    examplePINVOKE.negativesonly(value);
    if (examplePINVOKE.SWIGPendingException.Pending)
      throw examplePINVOKE.SWIGPendingException.Retrieve();
  }
```

### 23.5.3 C# exception example using exception specifications

When C++ exception specifications are used, SWIG is able to detect that the method might throw an exception. By default SWIG will automatically generate code to catch the exception and convert it into a managed `ApplicationException`, as defined by the default "throws" typemaps. The following example has a user supplied "throws" typemap which is used whenever an exception specification contains a `std::out_of_range`, such as the `evensonly` method below.

```
%typemap(throws, canthrow=1) std::out_of_range {
  SWIG_CSharpSetPendingExceptionArgument(SWIG_CSharpArgumentException, $1.what(), NULL);
  return $null;
}

%inline %{
#include <stdexcept>
void evensonly(int input) throw (std::out_of_range) {
  if (input%2 != 0)
    throw std::out_of_range("number is not even");
}
%}
```

Note that the type for the throws typemap is the type in the exception specification. SWIG generates a try catch block with the throws typemap code in the catch handler.

```
SWIGEXPORT void SWIGSTDCALL CSharp_evensonly(int jarg1) {
  int arg1 ;

  arg1 = (int)jarg1;
  try {
    evensonly(arg1);
  }
  catch(std::out_of_range &_e) {
    {
      SWIG_CSharpSetPendingExceptionArgument(SWIG_CSharpArgumentException, (&_e)->what(), NULL);
      return ;
    }
  }
}
```

Multiple catch handlers are generated should there be more than one exception specifications declared.

### 23.5.4 Custom C# ApplicationException example

This example involves a user defined exception. The conventional .NET exception handling approach is to create a custom `ApplicationException` and throw it in your application. The goal in this example is to convert the STL `std::out_of_range` exception into one of these custom .NET exceptions.

The default exception handling is quite easy to use as the `SWIG_CSharpSetPendingException()` and `SWIG_CSharpSetPendingExceptionArgument()` methods are provided by SWIG. However, for a custom C# exception, the boiler plate code that supports these functions needs replicating. In essence this consists of some C/C++ code and C# code. The C/C++ code can be generated into the wrapper file using the `%insert(runtime)` directive and the C# code can be generated into the intermediary class using the `imclasscode` pragma as follows:

```
%insert(runtime) %{
```

```
  // Code to handle throwing of C# CustomApplicationException from C/C++ code.
  // The equivalent delegate to the callback, CSharpExceptionCallback_t, is CustomExceptionDelegate
  // and the equivalent customExceptionCallback instance is customDelegate
  typedef void (SWIGSTDCALL* CSharpExceptionCallback_t)(const char *);
  CSharpExceptionCallback_t customExceptionCallback = NULL;

  extern "C" SWIGEXPORT
  void SWIGSTDCALL CustomExceptionRegisterCallback(CSharpExceptionCallback_t customCallback) {
    customExceptionCallback = customCallback;
  }

  // Note that SWIG detects any method calls named starting with
  // SWIG_CSharpSetPendingException for warning 845
  static void SWIG_CSharpSetPendingExceptionCustom(const char *msg) {
    customExceptionCallback(msg);
  }
%}

%pragma(csharp) imclasscode=%{
  class CustomExceptionHelper {
    // C# delegate for the C/C++ customExceptionCallback
    public delegate void CustomExceptionDelegate(string message);
    static CustomExceptionDelegate customDelegate =
                                  new CustomExceptionDelegate(SetPendingCustomException);

    [global::System.Runtime.InteropServices.DllImport("$dllimport", EntryPoint="CustomExceptionRegisterCallback")]
    public static extern
           void CustomExceptionRegisterCallback(CustomExceptionDelegate customCallback);

    static void SetPendingCustomException(string message) {
      SWIGPendingException.Set(new CustomApplicationException(message));
    }

    static CustomExceptionHelper() {
      CustomExceptionRegisterCallback(customDelegate);
    }
  }
  static CustomExceptionHelper exceptionHelper = new CustomExceptionHelper();
%}
```

The method stored in the C# delegate instance, `customDelegate` is what gets called by the C/C++ callback. However, the equivalent to the C# delegate, that is the C/C++ callback, needs to be assigned before any unmanaged code is executed. This is achieved by putting the initialisation code in the intermediary class. Recall that the intermediary class contains all the PInvoke methods, so the static variables in the intermediary class will be initialised before any of the PInvoke methods in this class are called. The `exceptionHelper` static variable ensures the C/C++ callback is initialised with the value in `customDelegate` by calling the `CustomExceptionRegisterCallback` method in the `CustomExceptionHelper` static constructor. Once this has been done, unmanaged code can make callbacks into the managed world as `customExceptionCallback` will be initialised with a valid callback/delegate. Any calls to `SWIG_CSharpSetPendingExceptionCustom()` will make the callback to create the pending exception in the same way that `SWIG_CSharpSetPendingException()` and `SWIG_CSharpSetPendingExceptionArgument()` does. In fact the method has been similarly named so that SWIG can issue the warning about missing `canthrow` attributes as discussed earlier. It is an invaluable warning as it is easy to forget the `canthrow` attribute when writing typemaps/features.

The `SWIGPendingException` helper class is not shown, but is generated as an inner class into the intermediary class. It stores the pending exception in Thread Local Storage so that the exception handling mechanism is thread safe.

The boiler plate code above must be used in addition to a handcrafted `CustomApplicationException`:

```
// Custom C# Exception
class CustomApplicationException : global::System.ApplicationException {
  public CustomApplicationException(string message)
    : base(message) {
  }
}
```

and the SWIG interface code:

```
%typemap(throws, canthrow=1) std::out_of_range {
  SWIG_CSharpSetPendingExceptionCustom($1.what());
  return $null;
}

%inline %{
void oddsonly(int input) throw (std::out_of_range) {
  if (input%2 != 1)
    throw std::out_of_range("number is not odd");
}
%}
```

The "throws" typemap now simply calls our new `SWIG_CSharpSetPendingExceptionCustom()` function so that the exception can be caught, as such:

```
try {
  example.oddsonly(2);
} catch (CustomApplicationException e) {
  ...
}
```

## 23.6 C# Directors

The SWIG directors feature adds extra code to the generated C# proxy classes that enable these classes to be used in cross-language polymorphism. Essentially, it enables unmanaged C++ code to call back into managed code for virtual methods so that a C# class can derive from a wrapped C++ class.

The following sections provide information on the C# director implementation and contain most of the information required to use the C# directors. However, the Java directors section should also be read in order to gain more insight into directors.

### 23.6.1 Directors example

Imagine we are wrapping a C++ base class, `Base`, from which we would like to inherit in C#. Such a class is shown below as well as another class, `Caller`, which calls the virtual method `UIntMethod` from pure unmanaged C++ code.

```
// file: example.h
class Base {
public:
  virtual ~Base() {}

  virtual unsigned int UIntMethod(unsigned int x) {
    std::cout << "Base - UIntMethod(" << x << ")" << std::endl;
    return x;
  }
  virtual void BaseBoolMethod(const Base &b, bool flag) {}
};

class Caller {
public:
  Caller(): m_base(0) {}
  ~Caller() { delBase(); }
  void set(Base *b) { delBase(); m_base = b; }
  void reset() { m_base = 0; }
  unsigned int UIntMethodCall(unsigned int x) { return m_base->UIntMethod(x); }

private:
  Base *m_base;
  void delBase() { delete m_base; m_base = 0; }
};
```

The director feature is turned off by default and the following simple interface file shows how directors are enabled for the class `Base`.

```
/* File : example.i */
%module(directors="1") example
%{
#include "example.h"
%}

%feature("director") Base;

%include "example.h"
```

The following is a C# class inheriting from `Base`:

```
public class CSharpDerived : Base
{
  public override uint UIntMethod(uint x)
  {
    global::System.Console.WriteLine("CSharpDerived - UIntMethod({0})", x);
    return x;
  }
}
```

The `Caller` class can demonstrate the `UIntMethod` method being called from unmanaged code using the following C# code:

```
public class runme
{
  static void Main()
  {
    Caller myCaller = new Caller();

    // Test pure C++ class
    using (Base myBase = new Base())
    {
      makeCalls(myCaller, myBase);
    }

    // Test director / C# derived class
    using (Base myBase = new CSharpDerived())
    {
      makeCalls(myCaller, myBase);
    }
  }

  static void makeCalls(Caller myCaller, Base myBase)
  {
    myCaller.set(myBase);
    myCaller.UIntMethodCall(123);
    myCaller.reset();
  }
}
```

If the above is run, the output is then:

```
Base - UIntMethod(123)
CSharpDerived - UIntMethod(123)
```

### 23.6.2 Directors implementation

The previous section demonstrated a simple example where the virtual `UIntMethod` method was called from C++ code, even when the overridden method is implemented in C#. The intention of this section is to gain an insight into how the director feature works. It shows the generated code for the two virtual methods, `UIntMethod` and `BaseBoolMethod`, when the director feature is enabled for the `Base` class.

Below is the generated C# `Base` director class.

```
public class Base : global::System.IDisposable {
```

```
  private global::System.Runtime.InteropServices.HandleRef swigCPtr;
  protected bool swigCMemOwn;

  internal Base(global::System.IntPtr cPtr, bool cMemoryOwn) {
    swigCMemOwn = cMemoryOwn;
    swigCPtr = new global::System.Runtime.InteropServices.HandleRef(this, cPtr);
  }

  internal static global::System.Runtime.InteropServices.HandleRef getCPtr(Base obj) {
    return (obj == null) ? new global::System.Runtime.InteropServices.HandleRef(null, global::System.IntPtr.Zero) : obj.swigCPtr;
  }

  ~Base() {
    Dispose();
  }

  public virtual void Dispose() {
    lock(this) {
      if(swigCPtr.Handle != global::System.IntPtr.Zero && swigCMemOwn) {
        swigCMemOwn = false;
        examplePINVOKE.delete_Base(swigCPtr);
      }
      swigCPtr = new global::System.Runtime.InteropServices.HandleRef(null, global::System.IntPtr.Zero);
      global::System.GC.SuppressFinalize(this);
    }
  }

  public virtual uint UIntMethod(uint x) {
    uint ret = examplePINVOKE.Base_UIntMethod(swigCPtr, x);
    return ret;
  }

  public virtual void BaseBoolMethod(Base b, bool flag) {
    examplePINVOKE.Base_BaseBoolMethod(swigCPtr, Base.getCPtr(b), flag);
    if (examplePINVOKE.SWIGPendingException.Pending)
      throw examplePINVOKE.SWIGPendingException.Retrieve();
  }

  public Base() : this(examplePINVOKE.new_Base(), true) {
    SwigDirectorConnect();
  }

  private void SwigDirectorConnect() {
    if (SwigDerivedClassHasMethod("UIntMethod", swigMethodTypes0))
      swigDelegate0 = new SwigDelegateBase_0(SwigDirectorMethodUIntMethod);
    if (SwigDerivedClassHasMethod("BaseBoolMethod", swigMethodTypes1))
      swigDelegate1 = new SwigDelegateBase_1(SwigDirectorMethodBaseBoolMethod);
    examplePINVOKE.Base_director_connect(swigCPtr, swigDelegate0, swigDelegate1);
  }

  private bool SwigDerivedClassHasMethod(string methodName, global::System.global::System.Type[] methodTypes) {
    System.Reflection.MethodInfo methodInfo = this.GetType().GetMethod(methodName, methodTypes);
    bool hasDerivedMethod = methodInfo.DeclaringType.IsSubclassOf(typeof(Base));
    return hasDerivedMethod;
  }

  private uint SwigDirectorMethodUIntMethod(uint x) {
    return UIntMethod(x);
  }

  private void SwigDirectorMethodBaseBoolMethod(global::System.IntPtr b, bool flag) {
    BaseBoolMethod(new Base(b, false), flag);
  }

  public delegate uint SwigDelegateBase_0(uint x);
  public delegate void SwigDelegateBase_1(global::System.IntPtr b, bool flag);

  private SwigDelegateBase_0 swigDelegate0;
  private SwigDelegateBase_1 swigDelegate1;

  private static global::System.Type[] swigMethodTypes0 = new global::System.Type[] { typeof(uint) };
  private static global::System.Type[] swigMethodTypes1 = new global::System.Type[] { typeof(Base), typeof(bool) };
}
```

Everything from the `SwigDirectorConnect()` method and below is code that is only generated when directors are enabled. The design comprises a C# delegate being initialised for each virtual method on construction of the class. Let's examine the `BaseBoolMethod`.

In the `Base` constructor a call is made to `SwigDirectorConnect()` which contains the initialisation code for all the virtual methods. It uses a support method, `SwigDerivedClassHasMethod()`, which simply uses reflection to determine if the named method, BaseBoolMethod, with the list of required parameter types, exists in a subclass. If it does not exist, the delegate is not initialised as there is no need for unmanaged code to call back into managed C# code. However, if there is an overridden method in any subclass, the delegate is required. It is then initialised to the `SwigDirectorMethodBaseBoolMethod` which in turn will call `BaseBoolMethod` if invoked. The delegate is not initialised to the `BaseBoolMethod` directly as quite often types will need marshalling from the unmanaged type to the managed type in which case an intermediary method ( `SwigDirectorMethodBaseBoolMethod`) is required for the marshalling. In this case, the C# `Base` class needs to be created from the unmanaged `IntPtr` type.

The last thing that `SwigDirectorConnect()` does is to pass the delegates to the unmanaged code. It calls the intermediary method `Base_director_connect()` which is really a call to the C function `CSharp_Base_director_connect()`. This method simply maps each C# delegate onto a C function pointer.

```
SWIGEXPORT void SWIGSTDCALL CSharp_Base_director_connect(void *objarg,
                                    SwigDirector_Base::SWIG_Callback0_t callback0,
                                    SwigDirector_Base::SWIG_Callback1_t callback1) {
  Base *obj = (Base *)objarg;
  SwigDirector_Base *director = dynamic_cast<SwigDirector_Base *>(obj);
  if (director) {
    director->swig_connect_director(callback0, callback1);
  }
}

class SwigDirector_Base : public Base, public Swig::Director {
public:
  SwigDirector_Base();
```

```
  virtual unsigned int UIntMethod(unsigned int x);
  virtual ~SwigDirector_Base();
  virtual void BaseBoolMethod(Base const &b, bool flag);

  typedef unsigned int (SWIGSTDCALL* SWIG_Callback0_t)(unsigned int);
  typedef void (SWIGSTDCALL* SWIG_Callback1_t)(void *, unsigned int);
  void swig_connect_director(SWIG_Callback0_t callbackUIntMethod,
                             SWIG_Callback1_t callbackBaseBoolMethod);

private:
  SWIG_Callback0_t swig_callbackUIntMethod;
  SWIG_Callback1_t swig_callbackBaseBoolMethod;
  void swig_init_callbacks();
};

void SwigDirector_Base::swig_connect_director(SWIG_Callback0_t callbackUIntMethod,
                                              SWIG_Callback1_t callbackBaseBoolMethod) {
  swig_callbackUIntMethod = callbackUIntMethod;
  swig_callbackBaseBoolMethod = callbackBaseBoolMethod;
}
```

Note that for each director class SWIG creates an unmanaged director class for making the callbacks. For example `Base` has `SwigDirector_Base` and `SwigDirector_Base` is derived from `Base`. Should a C# class be derived from `Base`, the underlying C++ `SwigDirector_Base` is created rather than `Base`. The `SwigDirector_Base` class then implements all the virtual methods, redirecting calls up to managed code if the callback/delegate is non-zero. The implementation of `SwigDirector_Base::BaseBoolMethod` shows this - the callback is made by invoking the `swig_callbackBaseBoolMethod` function pointer:

```
void SwigDirector_Base::BaseBoolMethod(Base const &b, bool flag) {
  void * jb = 0 ;
  unsigned int jflag  ;

  if (!swig_callbackBaseBoolMethod) {
    Base::BaseBoolMethod(b, flag);
    return;
  } else {
    jb = (Base *) &b;
    jflag = flag;
    swig_callbackBaseBoolMethod(jb, jflag);
  }
}
```

The delegates from the above example are `public` by default:

```
  public delegate uint SwigDelegateBase_0(uint x);
  public delegate void SwigDelegateBase_1(global::System.IntPtr b, bool flag);
```

These can be changed if desired via the `csdirectordelegatemodifiers` %feature directive. For example, using `%feature("csdirectordelegatemodifiers") "internal"` before SWIG parses the Base class will change all the delegates to `internal` :

```
  internal delegate uint SwigDelegateBase_0(uint x);
  internal delegate void SwigDelegateBase_1(global::System.IntPtr b, bool flag);
```

### 23.6.3 Director caveats

There is a subtle gotcha with directors. If default parameters are used, it is recommended to follow a pattern of always calling a single method in any C# derived class. An example will clarify this and the reasoning behind the recommendation. Consider the following C++ class wrapped as a director class:

```
class Defaults {
public:
  virtual ~Defaults();
  virtual void DefaultMethod(int a=-100);
};
```

Recall that C++ methods with default parameters generate overloaded methods for each defaulted parameter, so a C# derived class can be created with two `DefaultMethod` override methods:

```
public class CSharpDefaults : Defaults
{
  public override void DefaultMethod()
  {
    DefaultMethod(-100); // note C++ default value used
  }
  public override void DefaultMethod(int x)
  {
  }
}
```

It may not be clear at first, but should a user intend to call `CSharpDefaults.DefaultMethod()` from C++, a call is actually made to `CSharpDefaults.DefaultMethod(int)`. This is because the initial call is made in C++ and therefore the `DefaultMethod(int)` method will be called as is expected with C++ calls to methods with defaults, with the default being set to -100. The callback/delegate matching this method is of course the overloaded method `DefaultMethod(int)`. However, a call from C# to `CSharpDefaults.DefaultMethod()` will of course call this exact method and in order for behaviour to be consistent with calls from C++, the implementation should pass the call on to `CSharpDefaults.DefaultMethod(int)` using the C++ default value, as shown above.

## 23.7 Multiple modules

When using multiple modules it is possible to compile each SWIG generated wrapper into a different assembly. However, by default the generated code may not compile if generated classes in one assembly use generated classes in another assembly. The visibility of the `getCPtr()` and pointer constructor generated from the `csbody` typemaps needs changing. The default visibility is `internal` but it needs to be `public` for access from a different assembly. Just changing 'internal' to 'public' in the typemap achieves this. Two macros are available in `csharp.swg` to make this easier and using them is the preferred approach over simply copying the typemaps and modifying as this is forward compatible with any changes in the `csbody` typemap in future versions of SWIG. The macros are for the proxy and typewrapper classes and can respectively be used to to make the method and constructor public:

```
SWIG_CSBODY_PROXY(public, public, SWIGTYPE)
SWIG_CSBODY_TYPEWRAPPER(public, public, public, SWIGTYPE)
```

Alternatively, instead of exposing these as public, consider using the `[assembly:InternalsVisibleTo("Name")]` attribute available in the .NET framework when you know which assemblies these can be exposed to.

Another approach would be to make these public, but also to hide them from intellisense by using the `[System.ComponentModel.EditorBrowsable(System.ComponentModel.EditorBrowsableState.Never)]` attribute if you don't want users to easily stumble upon these so called 'internal workings' of the wrappers. You can do this like so:

```
#define PUBLIC_BUT_HIDDEN [System.ComponentModel.EditorBrowsable(System.ComponentModel.EditorBrowsableState.Never)] public
SWIG_CSBODY_PROXY(PUBLIC_BUT_HIDDEN, PUBLIC_BUT_HIDDEN, SWIGTYPE)
SWIG_CSBODY_TYPEWRAPPER(PUBLIC_BUT_HIDDEN, PUBLIC_BUT_HIDDEN, PUBLIC_BUT_HIDDEN, SWIGTYPE)
```

## 23.8 C# named and optional arguments

In C++ you can specify default arguments for functions, methods, and constructors. C# offers named arguments. This feature, specific to C#, lets you bind default argument functions with default arguments in C#. SWIG also allows you to add default arguments to C# functions which don't have default arguments in C++.

The `cs:defaultargs` feature enables C# named arguments with C# default values. Using this feature will turn off SWIG's default handling for default arguments, which would create an override for each defaulted argument.

For this feature, you first specify the function/method/constructor you want it to impact. Inside the feature call you specify each argument you want to override. If you specify none, it will take the literal text from c++ for each argument and apply that in C#. That often works fine, but when it doesn't you can supply a string literal with a C# expression to be used as an override. Or you can supply an int literal, or a float literal. If you want to give a literal string you need to include an escaped quote at the start and end of the literal such as `"\"a string\""`.

Let's consider an example:

```
%feature("cs:defaultargs") Foo::Foo;
%feature("cs:defaultargs", x=0, z=4) Foo::bar;
%feature("cs:defaultargs", x="\"five\"") Foo::zoo;

%inline %{
class Foo {
public:
    Foo(int a, int b=1, int c=2)
    {
    }
    int bar(int x, int y=2, int z=3)
    {
        return x+y+z;
    }
    int bat(int x=1, int y=2, int z=3)
    {
        return x+y+z;
    }
    int zoo(std::string x="four")
    {
        return (int)x.size();
    }
};
%}
```

The generated C# proxy class contains:

```
public class Foo : global::System.IDisposable {
  ...
  public Foo(int a, int b=1, int c=2) ...

  public int bar(int x=0, int y=2, int z=4) ...

  public int bat(int x, int y, int z) ...
  public int bat(int x, int y) ...
  public int bat(int x) ...
  public int bat() ...

  public int zoo(string x="five") ...
}
```

Note that:

1. The constructor uses the default arguments exactly as taken from C++.
2. The `bar` method uses the default arguments exactly as taken from C++ apart from `z` whose default value is changed to 4, and `x` did not have a default value in C++, but does in the generated C#.
3. The `bat` method does not use the `cs:defaultargs` feature and so the default handling of one overloaded C# method per defaulted C++ argument is generated.
4. The `zoo` method's string value is overridden by the new string value from the feature.

**Compatibility Note:** SWIG-4.2.0 added support for the `cs:defaultargs` feature.

## 23.9 C# Typemap examples

This section includes a few examples of typemaps. For more examples, you might look at the files "`csharp.swg`" and "`typemaps.i` " in the SWIG library.

### 23.9.1 Memory management when returning references to member variables

This example shows how to prevent premature garbage collection of objects when the underlying C++ class returns a pointer or reference to a member variable. The example is a direct equivalent to this Java equivalent.

Consider the following C++ code:

```
struct Wheel {
```

```
  int size;
  Wheel(int sz = 0) : size(sz) {}
};

class Bike {
  Wheel wheel;
public:
  Bike(int val) : wheel(val) {}
  Wheel& getWheel() { return wheel; }
};
```

and the following usage from C# after running the code through SWIG:

```
  Wheel wheel = new Bike(10).getWheel();
  global::System.Console.WriteLine("wheel size: " + wheel.size);
  // Simulate a garbage collection
  global::System.GC.Collect();
  global::System.GC.WaitForPendingFinalizers();
  global::System.Console.WriteLine("wheel size: " + wheel.size);
```

Don't be surprised that if the resulting output gives strange results such as...

```
wheel size: 10
wheel size: 135019664
```

What has happened here is the garbage collector has collected the `Bike` instance as it doesn't think it is needed any more. The proxy instance, `wheel`, contains a reference to memory that was deleted when the `Bike` instance was collected. In order to prevent the garbage collector from collecting the `Bike` instance a reference to the `Bike` must be added to the`wheel` instance. You can do this by adding the reference when the `getWheel()` method is called using the following typemaps.

```
%typemap(cscode) Wheel %{
  // Ensure that the GC doesn't collect any Bike instance set from C#
  private Bike bikeReference;
  internal void addReference(Bike bike) {
    bikeReference = bike;
  }
%}

// Add a C# reference to prevent premature garbage collection and resulting use
// of dangling C++ pointer. Intended for methods that return pointers or
// references to a member variable.
%typemap(csout, excode=SWIGEXCODE) Wheel& getWheel {
    global::System.IntPtr cPtr = $imcall;$excode
    $csclassname ret = null;
    if (cPtr != global::System.IntPtr.Zero) {
      ret = new $csclassname(cPtr, $owner);
      ret.addReference(this);
    }
    return ret;
  }
```

The code in the first typemap gets added to the `Wheel` proxy class. The code in the second typemap constitutes the bulk of the code in the generated `getWheel()` function:

```
public class Wheel : global::System.IDisposable {
  ...
  // Ensure that the GC doesn't collect any Bike instance set from C#
  private Bike bikeReference;
  internal void addReference(Bike bike) {
    bikeReference = bike;
  }
}

public class Bike : global::System.IDisposable {
  ...
  public Wheel getWheel() {
    global::System.IntPtr cPtr = examplePINVOKE.Bike_getWheel(swigCPtr);
    Wheel ret = null;
    if (cPtr != global::System.IntPtr.Zero) {
      ret = new Wheel(cPtr, false);
      ret.addReference(this);
    }
    return ret;
  }
}
```

Note the `addReference` call.

### 23.9.2 Memory management for objects passed to the C++ layer

The example is a direct equivalent to this Java equivalent. Managing memory can be tricky when using C++ and C# proxy classes. The previous example shows one such case and this example looks at memory management for a class passed to a C++ method which expects the object to remain in scope after the function has returned. Consider the following two C++ classes:

```
struct Element {
  int value;
  Element(int val) : value(val) {}
};
class Container {
  Element* element;
public:
  Container() : element(0) {}
  void setElement(Element* e) { element = e; }
  Element* getElement() { return element; }
};
```

and usage from C++

```
Container container;
Element element(20);
container.setElement(&element);
cout << "element.value: " << container.getElement()->value << endl;
```

and more or less equivalent usage from C#

```
Container container = new Container();
Element element = new Element(20);
container.setElement(element);
```

The C++ code will always print out 20, but the value printed out may not be this in the C# equivalent code. In order to understand why, consider a garbage collection occurring...

```
Container container = new Container();
Element element = new Element(20);
container.setElement(element);
global::System.Console.WriteLine("element.value: " + container.getElement().value);
// Simulate a garbage collection
global::System.GC.Collect();
global::System.GC.WaitForPendingFinalizers();
global::System.Console.WriteLine("element.value: " + container.getElement().value);
```

The temporary element created with `new Element(20)` could get garbage collected which ultimately means the `container` variable is holding a dangling pointer, thereby printing out any old random value instead of the expected value of 20. One solution is to add in the appropriate references in the C# layer...

```
public class Container : global::System.IDisposable {

  ...

  // Ensure that the GC doesn't collect any Element set from C#
  // as the underlying C++ class stores a shallow copy
  private Element elementReference;

  public void setElement(Element e) {
    examplePINVOKE.Container_setElement(swigCPtr, Element.getCPtr(e));
    elementReference = e;
  }
}
```

The following typemaps can be used to generate this code:

```
%typemap(cscode) Container %{
  // Ensure that the GC doesn't collect any Element set from C#
  // as the underlying C++ class stores a shallow copy
  private Element elementReference;
%}

%typemap(csin,
         post="      elementReference = $csinput;"
         ) Element *e "Element.getCPtr($csinput)"
```

The 'cscode' typemap simply adds in the specified code into the C# proxy class. The 'csin' typemap matches the input parameter type and name for the `setElement` method and the 'post' typemap attribute allows adding code after the PInvoke call. The 'post' code is generated into a finally block after the PInvoke call so the resulting code isn't quite as mentioned earlier, `setElement` is actually:

```
  public void setElement(Element e) {
    try {
      examplePINVOKE.Container_setElement(swigCPtr, Element.getCPtr(e));
    } finally {
      elementReference = e;
    }
  }
```

### 23.9.3 Date marshalling using the csin typemap and associated attributes

The NaN Exception example is a simple example of the "javain" typemap and its 'pre' attribute. This example demonstrates how a C++ date class, say `CDate`, can be mapped onto the standard .NET date class, `System.DateTime` by using the 'pre', 'post' and 'pgcppname' attributes of the "csin" typemap (the C# equivalent to the "javain" typemap). The example is an equivalent to the Java Date marshalling example. The idea is that the `System.DateTime` is used wherever the C++ API uses a `CDate`. Let's assume the code being wrapped is as follows:

```
class CDate {
public:
  CDate();
  CDate(int year, int month, int day);
  int getYear();
  int getMonth();
  int getDay();
  ...
};
struct Action {
  static int doSomething(const CDate &dateIn, CDate &dateOut);
  Action(const CDate &date, CDate &dateOut);
};
```

Note that `dateIn` is const and therefore read only and `dateOut` is a non-const output type.

First let's look at the code that is generated by default, where the C# proxy class `CDate` is used in the proxy interface:

```
public class Action : global::System.IDisposable {
  ...
  public Action(CDate dateIn, CDate dateOut)
      : this(examplePINVOKE.new_Action(CDate.getCPtr(dateIn), CDate.getCPtr(dateOut)), true) {
    if (examplePINVOKE.SWIGPendingException.Pending)
      throw examplePINVOKE.SWIGPendingException.Retrieve();
  }

  public int doSomething(CDate dateIn, CDate dateOut) {
    int ret = examplePINVOKE.Action_doSomething(swigCPtr,
                                                CDate.getCPtr(dateIn),
                                                CDate.getCPtr(dateOut));
    if (examplePINVOKE.SWIGPendingException.Pending)
      throw examplePINVOKE.SWIGPendingException.Retrieve();
    return ret;
  }
}
```

The `CDate &` and `const CDate &` C# code is generated from the following two default typemaps:

```
%typemap(cstype) SWIGTYPE & "$csclassname"
%typemap(csin) SWIGTYPE & "$csclassname.getCPtr($csinput)"
```

where '$csclassname' is translated into the proxy class name, `CDate` and '$csinput' is translated into the name of the parameter, eg `dateIn`. From C#, the intention is then to call into a modified API with something like:

```
System.DateTime dateIn = new System.DateTime(2011, 4, 13);
System.DateTime dateOut = new System.DateTime();

// Note in calls below, dateIn remains unchanged and dateOut
// is set to a new value by the C++ call
Action action = new Action(dateIn, out dateOut);
dateIn = new System.DateTime(2012, 7, 14);
```

To achieve this mapping, we need to alter the default code generation slightly so that at the C# layer, a `System.DateTime` is converted into a `CDate`. The intermediary layer will still take a pointer to the underlying `CDate` class. The typemaps to achieve this are shown below.

```
%typemap(cstype) const CDate & "System.DateTime"
%typemap(csin,
         pre="    CDate temp$csinput = new CDate($csinput.Year, $csinput.Month, $csinput.Day);"
        ) const CDate &
         "$csclassname.getCPtr(temp$csinput)"

%typemap(cstype) CDate & "out System.DateTime"
%typemap(csin,
         pre="    CDate temp$csinput = new CDate();",
         post="      $csinput = new System.DateTime(temp$csinput.getYear(),"
              " temp$csinput.getMonth(), temp$csinput.getDay(), 0, 0, 0);",
         cshin="out $csinput"
        ) CDate &
         "$csclassname.getCPtr(temp$csinput)"
```

The resulting generated proxy code in the `Action` class follows:

```
public class Action : global::System.IDisposable {
  ...
  public int doSomething(System.DateTime dateIn, out System.DateTime dateOut) {
    CDate tempdateIn = new CDate(dateIn.Year, dateIn.Month, dateIn.Day);
    CDate tempdateOut = new CDate();
    try {
      int ret = examplePINVOKE.Action_doSomething(swigCPtr,
                                                  CDate.getCPtr(tempdateIn),
                                                  CDate.getCPtr(tempdateOut));
      if (examplePINVOKE.SWIGPendingException.Pending)
        throw examplePINVOKE.SWIGPendingException.Retrieve();
      return ret;
    } finally {
      dateOut = new System.DateTime(tempdateOut.getYear(),
                                    tempdateOut.getMonth(), tempdateOut.getDay(), 0, 0, 0);
    }
  }

  static private global::System.IntPtr SwigConstructAction(System.DateTime dateIn, out System.DateTime dateOut) {
    CDate tempdateIn = new CDate(dateIn.Year, dateIn.Month, dateIn.Day);
    CDate tempdateOut = new CDate();
    try {
      return examplePINVOKE.new_Action(CDate.getCPtr(tempdateIn), CDate.getCPtr(tempdateOut));
    } finally {
      dateOut = new System.DateTime(tempdateOut.getYear(),
                                    tempdateOut.getMonth(), tempdateOut.getDay(), 0, 0, 0);
    }
  }

  public Action(System.DateTime dateIn, out System.DateTime dateOut)
      : this(Action.SwigConstructAction(dateIn, out dateOut), true) {
    if (examplePINVOKE.SWIGPendingException.Pending)
      throw examplePINVOKE.SWIGPendingException.Retrieve();
  }
}
```

A few things to note:

- The "cstype" typemap has changed the parameter type to `System.DateTime` instead of the default generated `CDate` proxy.
- The non-const `CDate &` type is marshalled as a reference parameter in C# as the date cannot be explicitly set once the object has been created, so a new object is created instead.
- The code in the 'pre' attribute appears before the intermediary call (`examplePINVOKE.new_Action` / `examplePINVOKE.Action_doSomething`).
- The code in the 'post' attribute appears after the intermediary call.
- A try .. finally block is generated with the intermediary call in the try block and 'post' code in the finally block. The alternative of just using a temporary variable for the return value from the intermediary call and the 'post' code being inserted before the return statement is not possible given that the intermediary call and method return comes from a single source (the "csout" typemap).
- The temporary variables in the "csin" typemaps are called `temp$csin`, where "$csin" is replaced with the parameter name. "$csin" is used to mangle the variable name so that more than one `CDate &` type can be used as a parameter in a method, otherwise two or more local variables with the same name would be generated.
- The use of the "csin" typemap causes a constructor helper function (`SwigConstructAction`) to be generated. This allows C# code to be called before the intermediary call made in the constructor initialization list.
- The 'cshin' attribute is required for the `SwigConstructAction` constructor helper function so that the 2nd parameter is declared as `out dateOut` instead of just `dateOut`.

So far we have considered the date as an input only and an output only type. Now let's consider `CDate *` used as an input/output type. Consider the following C++ function which modifies the date passed in:

```
void addYears(CDate *pDate, int years) {
  *pDate = CDate(pDate->getYear() + years, pDate->getMonth(), pDate->getDay());
}
```

If usage of `CDate *` commonly follows this input/output pattern, usage from C# like the following

```
System.DateTime christmasEve = new System.DateTime(2000, 12, 24);
example.addYears(ref christmasEve, 10); // christmasEve now contains 2010-12-24
```

will be possible with the following `CDate *` typemaps

```
%typemap(cstype, out="System.DateTime") CDate * "ref System.DateTime"

%typemap(csin,
         pre="    CDate temp$csinput = new CDate($csinput.Year, $csinput.Month, $csinput.Day);",
         post="      $csinput = new System.DateTime(temp$csinput.getYear(),"
              " temp$csinput.getMonth(), temp$csinput.getDay(), 0, 0, 0);",
         cshin="ref $csinput"
        ) CDate *
         "$csclassname.getCPtr(temp$csinput)"
```

Globals are wrapped by the module class and for a module called example, the typemaps result in the following code:

```
public class example {
  public static void addYears(ref System.DateTime pDate, int years) {
    CDate temppDate = new CDate(pDate.Year, pDate.Month, pDate.Day);
    try {
      examplePINVOKE.addYears(CDate.getCPtr(temppDate), years);
    } finally {
      pDate = new System.DateTime(temppDate.getYear(), temppDate.getMonth(), temppDate.getDay(),
                                  0, 0, 0);
    }
  }
  ...
}
```

The following typemap is the same as the previous but demonstrates how a using block can be used for the temporary variable. The only change to the previous typemap is the introduction of the 'terminator' attribute to terminate the `using` block. The `subtractYears` method is nearly identical to the above `addYears` method.

```
%typemap(csin,
  pre="    using (CDate temp$csinput = new CDate($csinput.Year, $csinput.Month, $csinput.Day)) {",
  post="      $csinput = new System.DateTime(temp$csinput.getYear(),"
       " temp$csinput.getMonth(), temp$csinput.getDay(), 0, 0, 0);",
  terminator="    } // terminate temp$csinput using block",
  cshin="ref $csinput"
 ) CDate *
  "$csclassname.getCPtr(temp$csinput)"

void subtractYears(CDate *pDate, int years) {
  *pDate = CDate(pDate->getYear() - years, pDate->getMonth(), pDate->getDay());
}
```

The resulting generated code shows the termination of the `using` block:

```
public class example {
  public static void subtractYears(ref System.DateTime pDate, int years) {
    using (CDate temppDate = new CDate(pDate.Year, pDate.Month, pDate.Day)) {
    try {
      examplePINVOKE.subtractYears(CDate.getCPtr(temppDate), years);
    } finally {
      pDate = new System.DateTime(temppDate.getYear(), temppDate.getMonth(), temppDate.getDay(),
                                  0, 0, 0);
    }
    } // terminate temppDate using block
  }
  ...
}
```

### 23.9.4 A date example demonstrating marshalling of C# properties

The previous section looked at converting a C++ date class to `System.DateTime` for parameters. This section extends this idea so that the correct marshalling is obtained when wrapping C++ variables. Consider the same `CDate` class from the previous section and a global variable:

```
CDate ImportantDate = CDate(1999, 12, 31);
```

The aim is to use `System.DateTime` from C# when accessing this date as shown in the following usage where the module name is 'example':

```
example.ImportantDate = new System.DateTime(2000, 11, 22);
System.DateTime importantDate = example.ImportantDate;
Console.WriteLine("Important date: " + importantDate);
```

When SWIG wraps a variable that is a class/struct/union, it is wrapped using a pointer to the type for the reasons given in Structure data members. The typemap type required is thus `CDate *`. Given that the previous section already designed `CDate *` typemaps, we'll use those same typemaps plus the 'csvarin' and 'csvarout' typemaps.

```
%typemap(cstype, out="System.DateTime") CDate * "ref System.DateTime"

%typemap(csin,
         pre="    CDate temp$csinput = new CDate($csinput.Year, $csinput.Month, $csinput.Day);",
         post="      $csinput = new System.DateTime(temp$csinput.getYear()," 
              " temp$csinput.getMonth(), temp$csinput.getDay(), 0, 0, 0);",
         cshin="ref $csinput"
        ) CDate *
         "$csclassname.getCPtr(temp$csinput)"

%typemap(csvarin, excode=SWIGEXCODE2) CDate * %{
    /* csvarin typemap code */
    set {
      CDate temp$csinput = new CDate($csinput.Year, $csinput.Month, $csinput.Day);
      $imcall;$excode
    } %}

%typemap(csvarout, excode=SWIGEXCODE2) CDate * %{
    /* csvarout typemap code */
    get {
      global::System.IntPtr cPtr = $imcall;
      CDate tempDate = (cPtr == global::System.IntPtr.Zero) ? null : new CDate(cPtr, $owner);$excode
      return new System.DateTime(tempDate.getYear(), tempDate.getMonth(), tempDate.getDay(),
                                 0, 0, 0);
    } %}
```

For a module called example, the typemaps result in the following code:

```
public class example {
  public static System.DateTime ImportantDate {
    /* csvarin typemap code */
    set {
      CDate tempvalue = new CDate(value.Year, value.Month, value.Day);
      examplePINVOKE.ImportantDate_set(CDate.getCPtr(tempvalue));
    }
    /* csvarout typemap code */
    get {
      global::System.IntPtr cPtr = examplePINVOKE.ImportantDate_get();
      CDate tempDate = (cPtr == global::System.IntPtr.Zero) ? null : new CDate(cPtr, false);
      return new System.DateTime(tempDate.getYear(), tempDate.getMonth(), tempDate.getDay(),
                                 0, 0, 0);
    }
  }
  ...
}
```

Some points to note:

- The property set comes from the 'csvarin' typemap and the property get comes from the 'csvarout' typemap.
- The type used for the property comes from the 'cstype' typemap. This particular example has the 'out' attribute set in the typemap and as it is specified, it is used in preference to the type in the typemap body. This is because the type in the 'out' attribute can never include modifiers such as 'ref', thereby avoiding code such as `public static ref System.DateTime ImportantDate { ...`, which would of course not compile.
- The `$excode` special variable expands to nothing as there are no exception handlers specified in any of the unmanaged code typemaps (in fact the marshalling was done using the default unmanaged code typemaps.)
- The `$imcall` typemap expands to the appropriate intermediary method call in the `examplePINVOKE` class.
- The `$csinput` special variable in the 'csin' typemap always expands to `value` for properties. In this case `$csclassname.getCPtr(temp$csinput)` expands to `CDate.getCPtr(tempvalue)`.
- The 'csin' typemap has 'pre', 'post' and 'cshin' attributes, and these are all ignored in the property set. The code in these attributes must instead be replicated within the 'csvarin' typemap. The line creating the `temp$csinput` variable is such an example; it is identical to what is in the 'pre' attribute.

### 23.9.5 Date example demonstrating the 'pre' and 'post' typemap attributes for directors

The 'pre' and 'post' attributes in the "csdirectorin" typemap act like the attributes of the same name in the "csin" typemap. For example if we modify the Date marshalling example like this:

```
class CDate {
  ...
  void setYear(int);
  void setMonth(int);
  void setDay(int);
};
struct Action {
  virtual void someCallback(CDate &date);
  virtual ~Action();
  ...
};
```

and declare `%feature ("director")` for the `Action` class, we would have to define additional marshalling rules for `CDate &` parameter. The typemap may look like this:

```
%typemap(csdirectorin,
         pre="System.DateTime temp$iminput = new System.DateTime();",
         post="CDate temp2$iminput = new CDate($iminput, false);\n"
              "temp2$iminput.setYear(tempdate.Year);\n"
              "temp2$iminput.setMonth(tempdate.Month);\n"
```

```
              "temp2$iminput.setDay(tempdate.Day);"
        ) CDate &date "out temp$iminput"
```

The generated proxy class code will then contain the following wrapper for calling user-overloaded `someCallback()`:

```
...
  private void SwigDirectorMethodsomeCallback(global::System.IntPtr date) {
    System.DateTime tempdate = new System.DateTime();
    try {
      someCallback(out tempdate);
    } finally {
      // we create a managed wrapper around the existing C reference, just for convenience
      CDate temp2date = new CDate(date, false);
      temp2date.setYear(tempdate.Year);
      temp2date.setMonth(tempdate.Month);
      temp2date.setDay(tempdate.Day);
    }
  }
...
```

Pay special attention to the memory management issues, using these attributes.

### 23.9.6 Turning proxy classes into partial classes

C# supports the notion of partial classes whereby a class definition can be split into more than one file. It is possible to turn the wrapped C++ class into a partial C# class using the `csclassmodifiers` typemap. Consider a C++ class called `ExtendMe` :

```
class ExtendMe {
public:
  int Part1() { return 1; }
};
```

The default C# proxy class generated is:

```
public class ExtendMe : global::System.IDisposable {
  ...
  public int Part1() {
    ...
  }
}
```

The default csclassmodifiers typemap shipped with SWIG is

```
%typemap(csclassmodifiers) SWIGTYPE "public class"
```

Note that the type used is the special catch all type `SWIGTYPE` . If instead we use the following typemap to override this for just the `ExtendMe` class:

```
%typemap(csclassmodifiers) ExtendMe "public partial class"
```

The C# proxy class becomes a partial class:

```
public partial class ExtendMe : global::System.IDisposable {
  ...
  public int Part1() {
    ...
  }
}
```

You can then of course declare another part of the partial class elsewhere, for example:

```
public partial class ExtendMe : global::System.IDisposable {
  public int Part2() {
    return 2;
  }
}
```

and compile the following code:

```
ExtendMe em = new ExtendMe();
Console.WriteLine("part1: {0}", em.Part1());
Console.WriteLine("part2: {0}", em.Part2());
```

demonstrating that the class contains methods calling both unmanaged code - `Part1()` and managed code - `Part2()`. The following example is an alternative approach to adding managed code to the generated proxy class.

### 23.9.7 Turning proxy classes into sealed classes

The technique in the previous section can be used to make the proxy class a sealed class. Consider a C++ class `NotABaseClass` that you don't want to be derived from in C#:

```
struct NotABaseClass {
  NotABaseClass();
  ~NotABaseClass();
};
```

The default C# proxy class method generated with Dispose method is:

```
public class NotABaseClass : global::System.IDisposable {
  ...
  public virtual void Dispose() {
    ...
  }
}
```

The `csclassmodifiers` typemap can be used to modify the class modifiers and the `csmethodmodifiers` feature can be used on the destructor to modify the proxy's `Dispose` method:

```
%typemap(csclassmodifiers) NotABaseClass "public sealed class"
%csmethodmodifiers NotABaseClass::~NotABaseClass "public /*virtual*/";
```

The relevant generated code is thus:

```
public sealed class NotABaseClass : global::System.IDisposable {
  ...
  public /*virtual*/ void Dispose() {
    ...
  }
}
```

Any attempt to derive from the `NotABaseClass` in C# will result in a C# compiler error, for example:

```
public class Derived : NotABaseClass {
};
```

```
runme.cs(6,14): error CS0509: `Derived': cannot derive from sealed type `NotABaseClass'
```

Finally, if you get a warning about use of 'protected' in the generated base class:

```
NotABaseClass.cs(14,18): warning CS0628: `NotABaseClass.swigCMemOwn': new protected member declared in sealed class
```

Either suppress the warning or modify the generated code by copying and tweaking the default 'csbody' typemap code in csharp.swg by modifying swigCMemOwn to not be protected.

### 23.9.8 Extending proxy classes with additional C# code

The previous example showed how to use partial classes to add functionality to a generated C# proxy class. It is also possible to extend a wrapped struct/class with C/C++ code by using the %extend directive. A third approach is to add some C# methods into the generated proxy class with the `cscode` typemap. If we declare the following typemap before SWIG parses the `ExtendMe` class used in the previous example

```
%typemap(cscode) ExtendMe %{
  public int Part3() {
    return 3;
  }
%}
```

The generated C# proxy class will instead be:

```
public class ExtendMe : global::System.IDisposable {
  ...
  public int Part3() {
    return 3;
  }
  public int Part1() {
    ...
  }
}
```

### 23.9.9 Underlying type for enums

C# enums use int as the underlying type for each enum item, unless there is a C++11 enum base specifying the underlying C++ enum type. If there is a C++ base enum then this is automatically converted to the equivalent C# integral type. If you wish to change the underlying type to something else, then use the `csbase` typemap. For example when your C++ code uses a value larger than int, this is necessary as the C# compiler will not compile values which are too large to fit into an int. Here is an example:

```
%typemap(csbase) BigNumbers "uint"
%inline %{
  enum BigNumbers { big=0x80000000, bigger };
%}
```

The generated enum will then use the given underlying type and compile correctly:

```
public enum BigNumbers : uint {
  big = 0x80000000,
  bigger
}
```

If a C++11 enum base is specified, such as `unsigned short` in the following:

```
%inline %{
  enum SmallNumbers : unsigned short { tiny, small=1  };
%}
```

The underlying type is automatically converted to the C# equivalent, `ushort`:

```
public enum SmallNumbers : ushort {
  tiny,
  small = 1
}
```

The underlying C# type can still be changed to something else using the csbase typemap but the `replace` attribute must be set to avoid an ignored warnings as there are effectively two specified bases, which of course is not possible. For example:

```
%typemap(csbase, replace="1") SmallNumbers "byte"
%inline %{
  enum SmallNumbers : unsigned short { tiny, small=1  };
%}
```

which generates the desired underlying enum type:

```
public enum SmallNumbers : byte {
  tiny,
  small = 1
}
```

If SWIG cannot find appropriate type information for the C++ enum base, then a type wrapper class (something like SWIGTYPE_p_xxxx) is generated for the C# enum base. This will not compile in C#, as a C# integral base is required. To fix this, make sure the C++ enum base type information is parsed by SWIG; this is usually a typedef to an integral type. Note that SWIG uses the cstype typemap to lookup the C# type for any given C++ type. Using the csbase typemap, as described above, overrides the default cstype for looking up the C# integral base type.

---

# 24 SWIG and D


- Introduction
- Command line invocation
- Typemaps
  - C# <-> D name comparison
  - ctype, imtype, dtype
  - in, out, directorin, directorout
  - din, dout, ddirectorin, ddirectorout
  - typecheck typemaps
  - Code injection typemaps
  - Special variable macros
- Other D code control features
  - D begin
  - D and %feature
- Pragmas
- D Exceptions
- D Directors
- Other features
  - Extended namespace support (nspace)
  - Native pointer support
  - Operator overloading
  - Running the test-suite
- D Typemap examples
- Work in progress and planned features


## 24.1 Introduction

From the D Programming Language web site: *D is a systems programming language. Its focus is on combining the power and high performance of C and C++ with the programmer productivity of modern languages like Ruby and Python. [...] The D language is statically typed and compiles directly to machine code.* As such, it is not very surprising that D is able to directly interface with C libraries. Why would a SWIG module for D be needed then in the first place?

Well, besides the obvious downside that the C header files have to be manually converted to D modules for this to work, there is one major inconvenience with this approach: D code usually is on a higher abstraction level than C, and many of the features that make D interesting are simply not available when dealing with C libraries, requiring you e.g. to manually convert strings between pointers to `\0`-terminated char arrays and D char arrays, making the algorithms from the D2 standard library unusable with C arrays and data structures, and so on.

While these issues can be worked around relatively easy by hand-coding a thin wrapper layer around the C library in question, there is another issue where writing wrapper code per hand is not feasible: C++ libraries. Support for C++ was added in D2 via `extern(C++)` but this support is still very limited, and a custom wrapper layer is still required in many cases.

To help addressing these issues, the SWIG C# module has been forked to support D. Is has evolved quite a lot since then, but there are still many similarities, so if you do not find what you are looking for on this page, it might be worth having a look at the chapter on C# (and also on Java, since the C# module was in turn forked from it).

## 24.2 Command line invocation

To activate the D module, pass the `-d` option to SWIG at the command line. The same standard command line options as with any other language module are available, plus the following D specific ones:

`-d2`

Prior to SWIG 4.2.0, SWIG generated wrappers for D1/Tango by default and `-d2` could be used to generate D2/Phobos wrappers instead. SWIG 4.2.0 dropped support for D1, and D2 wrappers are now produced by default. This option is still recognised to allow existing build systems calling SWIG to work, but is now a no-op.

`-splitproxy`

By default, SWIG generates two D modules: the *proxy* module, named like the source module (either specified via the `%module` directive or via the `module` command line option), which contains all the proxy classes, functions, enums, etc., and the *intermediary* module (named like the proxy module, but suffixed with `_im`), which contains all the `extern(C)` function declarations and other private parts only used internally by the proxy module.

If the split proxy mode is enabled by passing this option at the command line, all proxy classes and enums are emitted to their own D module instead. The main proxy module only contains free functions and constants in this case.

`-package <pkg>`

By default, the proxy D modules and the intermediary D module are written to the root package. Using this option, you can specify another target package instead.

`-wrapperlibrary <wl>`

The code SWIG generates to dynamically load the C/C++ wrapper layer looks for a library called `$module_wrap` by default. With this option, you can override the name of the file the wrapper code loads at runtime (the `lib` prefix and the suffix for shared libraries are appended automatically, depending on the OS).

This might especially be useful if you want to invoke SWIG several times on separate modules, but compile the resulting code into a single shared library.

## 24.3 Typemaps

### 24.3.1 C# <-> D name comparison

If you already know the SWIG C# module, you might find the following name comparison table useful:

```
ctype                  <->  ctype
imtype                 <->  imtype
cstype                 <->  dtype
csin                   <->  din
csout                  <->  dout
csdirectorin           <->  ddirectorin
csdirectorout          <->  ddirectorout
csinterfaces           <->  dinterfaces
csinterfaces_derived   <->  dinterfaces_derived
csbase                 <->  dbase
csclassmodifiers       <->  dclassmodifiers
cscode                 <->  dcode
csimports              <->  dimports
csbody                 <->  dbody
csfinalize             <->  ddestructor
csdisposing            <->  ddispose
csdisposing_derived    <->  ddispose_derived
```

### 24.3.2 ctype, imtype, dtype

Mapping of types between the C/C++ library, the C/C++ library wrapper exposing the C functions, the D wrapper module importing these functions and the D proxy code.

The `ctype` typemap is used to determine the types to use in the C wrapper functions. The types from the `imtype` typemap are used in the extern(C) declarations of these functions in the intermediary D module. The `dtype` typemap contains the D types used in the D proxy module/class.

### 24.3.3 in, out, directorin, directorout

Used for converting between the types for C/C++ and D when generating the code for the wrapper functions (on the C++ side).

The code from the `in` typemap is used to convert arguments to the C wrapper function to the type used in the wrapped code (`ctype` ->original C++ type), the `out` typemap is utilized to convert values from the wrapped code to wrapper function return types (original C++ type->`ctype`).

The `directorin` typemap is used to convert parameters to the type used in the D director callback function, its return value is processed by `directorout` (see below).

### 24.3.4 din, dout, ddirectorin, ddirectorout

Typemaps for code generation in D proxy and type wrapper classes.

The `din` typemap is used for converting function parameter types from the type used in the proxy module or class to the type used in the intermediary D module (the $dinput macro is replaced). To inject further parameter processing code before or after the call to the intermediary layer, the `pre` , `post` and `terminator` attributes can be used (please refer to the C# date marshalling example for more information on these).

The `dout` typemap is used for converting function return values from the return type used in the intermediary D module to the type returned by the proxy function. The `$excode` special variable in `dout` typemaps is replaced by the `excode` typemap attribute code if the method can throw any exceptions from unmanaged code, otherwise by nothing (the `$imcall` and `$owner` macros are replaced).

The code from the `ddirectorin` and `ddirectorout` typemaps is used for conversion in director callback functions. Arguments are converted to the type used in the proxy class method they are calling by using the code from `ddirectorin`, the proxy class method return value is converted to the type the C++ code expects via the `ddirectorout` typemap (the `$dcall` and `$winput` macros are replaced).

The full chain of type conversions when a director callback is invoked looks like this:

```
    type             CPPClass::method(type a)
     ↑                        ↓
  <directorout>          <directorin>
      ↑                        ↓
    ctype            methodCallback(ctype a)           C++
::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
    imtype           methodCallback(imtype a)           D
     ↑                        ↓
 <ddirectorout>         <ddirectorin>
      ↑                        ↓
    dtype            DClass.method(dtype a)
```

### 24.3.5 typecheck typemaps

Because, unlike many scripting languages supported by SWIG, D does not need any dynamic dispatch helper to access an overloaded function, the purpose of these is merely to issue a warning for overloaded C++ functions that cannot be overloaded in D (as more than one C++ type maps to a single D type).

### 24.3.6 Code injection typemaps

These typemaps are used for generating the skeleton of proxy classes for C++ types.

By overriding `dbase`, `dinterfaces` or `dinterfaces_derived`, the inheritance chain of the generated proxy class for a type can be modified. `dclassmodifiers` allows you to add any custom modifiers around the class keyword.

Using `dcode` and `dimports`, you can specify additional D code which will be emitted into the class body respectively the imports section of the D module the class is written to.

`dconstructor`, `ddestructor`, `ddispose` and `ddispose_derived` are used to generate the class constructor, destructor and `dispose()` method, respectively. The auxiliary code for handling the pointer to the C++ object is stored in `dbody` and `dbody_derived`. You can override them for specific types.

Code can also be injected into the D proxy class using `%proxycode`.

### 24.3.7 Special variable macros

The standard SWIG special variables are available for use within typemaps as described in the Typemaps documentation , for example `$1`, `$input`, `$result` etc.

When generating D wrappers, a few additional macros are available:

`$dclassname` (C#: `$csclassname`)

This special variable works similar to `$n_type` in that it returns the name of a type - it expands to the D proxy class name of the type being wrapped. If the type does not have an associated proxy class, it expands to the type wrapper class name, for example, `SWIGTYPE_p_p_SomeCppClass` is generated when wrapping `SomeCppClass **`.

There are two other variants available, `$&dclassname` and `$*dclassname`. The former adds a level of indirection, while the latter removes one. For instance, when wrapping `Foo **`, `$*dclassname` would be replaced by the proxy class name corresponding to `Foo *`.

`$dclazzname` (C#: `$csclazzname`)

This special variable expands the fully qualified C++ class into the package name, if used by the `nspace` feature, and the proxy class name, mangled for use as a function name. For example, `Namespace1::Namespace2::Klass` is expanded into `Namespace1_Namespace2_Klass_`.

This special variable might be useful for calling certain functions in the wrapper layer (e.g. upcast wrappers) which are mangled like this.

`$null`

In code inserted into the generated C/C++ wrapper functions, this variable is replaced by either `0` or nothing at all, depending on whether the function has a return value or not. It can be used to bail out early e.g. in case of errors (`return $null;`).

`$dinput` (C#: `$csinput`)

This variable is used in `din` typemaps and is replaced by the expression which is to be passed to C/C++.

For example, this input

```
%typemap(din) SomeClass * "SomeClass.getCPointer($dinput)"

%inline %{
  class SomeClass {};
  void foo(SomeClass *arg);
%}
```

leads to the following D proxy code being generated:

```
void foo(SomeClass arg) {
  example_im.foo(SomeClass.getCPointer(arg));
}
```

`$imcall` and `$owner` (C#: `$imcall`)

These variables are used in `dout` typemaps. `$imcall` contains the call to the intermediary module which provides the value to be used, and `$owner` signals if the caller is responsible for managing the object lifetime (that is, if the called method is a constructor or has been marked via `%newobject` ).

Consider the following example:

```
%typemap(dout) SomeClass * {
  return new SomeClass($imcall, $owner);
}

%inline %{
  class SomeClass;
  SomeClass *foo();

  %newobject bar();
  SomeClass *bar();
%}
```

The code generated for `foo()` and `bar()` looks like this:

```
SomeClass foo() {
  return new SomeClass(example_im.foo(), false);
}

SomeClass bar() {
  return new SomeClass(example_im.bar(), true);
}
```

`$dcall` and `$winput` (C#: `$cscall`, `$iminput`)

These variables are used in the director-specific typemaps `ddirectorin` and `ddirectorout`. They are more or less the reverse of the `$imcall` and `$dinput` macros: `$dcall` contains the invocation of the D proxy method of which the return value is to be passed back to C++, `$winput` contains the parameter value from C++.

`$excode`

This variable is used in `dout` and `dconstructor` typemaps and is filled with the contents of the `excode` typemap attribute if an exception could be thrown from the C++ side. See the C# documentation for details.

`$dbaseclass`

Currently for internal use only, it contains the D name of the C++ base class (if any) inside proxy classes.

`$directorconnect`

This macro is only valid inside the `dconstructor` typemap and contains the value of the `dconstructor` typemap attribute if the currently wrapped class has directors enabled.

This is how the default `dconstructor` typemap looks like (you usually do not want to specify a custom one):

```
%typemap(dconstructor, excode=SWIGEXCODE,
         directorconnect="\n  swigDirectorConnect();") SWIGTYPE {
  this($imcall, true);$excode$directorconnect
}
```

`$imfuncname`

This special variable expands to the name of the function in the intermediary class that will be used in $imcall. Like, $imcall, this special variable is only expanded in the "dout" typemap.

`$importtype(SomeDType)`

This macro is used in the `dimports` typemap if a dependency on another D type generated by SWIG is added by a custom typemap.

Consider the following code snippet:

```
%typemap(dinterfaces) SomeClass "AnInterface, AnotherInterface"
```

This causes SWIG to add `AnInterface` and `AnotherInterface` to the base class list of `SomeClass`:

```
class SomeClass : AnInterface, AnotherInterface {
  ...
}
```

For this to work, `AnInterface` and `AnotherInterface` have to be in scope. If SWIG is not in split proxy mode, this is already the case, but if it is, they have to be added to the import list via the `dimports` typemap. Additionally, the import statement depends on the package SWIG is configured to emit the modules to.

The `$importtype` macro helps you to elegantly solve this problem:

```
%typemap(dimports) RemoteMpe %{
$importtype(AnInterface)
$importtype(AnotherInterface)
%}
```

If SWIG is in split proxy mode, it expands to an `import` statement for the specified type, to nothing if not.

`$module`

Expands to the name of the main proxy D module.

`$imdmodule`

Contains the fully qualified name of the intermediary D module.

## 24.4 Other D code control features

### 24.4.1 D begin

It is possible to add a common comment at the start of every generated D file. The `%module` directive supports the `dbegin` option for this. The provided text is generated at the very beginning of each generated D file. As it is generated before the D module statement, is only really useful for adding in a common comment into all generated D files. For example, copyright text for each file:

```
%module(dbegin="/* Common comment. Copyright (C) 2000 Mr Nobody. */\n") nobodymodule
```

### 24.4.2 D and %feature

The D module defines a number of directives which modify the SWIG features set globally or for a specific declaration:

`%dmanifestconst` and `%dconstvalue(value)`

Out of the box, SWIG generates accessor methods for C `#defines` and C++ constants. The `%dmanifestconst` directive enables wrapping these constants as D manifest constants (`enum` in D2).

For this to work, the C/C++ code for the constant value must directly compile as D code, though. If this is not the case, you can manually override the expression written to the D proxy module using the `%dconstvalue` directive, passing the new value as parameter.

For `enums`, again `%dconstvalue` can be used to override the value of an enum item if the initializer should not compile in D.

`%dmethodmodifiers`

This directive can be used to override the modifiers for a proxy function. For instance, you could make a `public` C++ member function `private` in D like this:

```
%dmethodmodifiers A::foo "private";

%inline %{
struct A {
  void foo();
};
%}
```

## 24.5 Pragmas

There are a few SWIG pragmas specific to the D module, which you can use to influence the D code SWIG generates:

`%pragma(d) imdmodulecode`

The passed text (D code) is copied verbatim to the intermediary D module. For example, it can be (and is, internally) used to emit additional private helper code for the use by proxy typemaps.

`%pragma(d) imdmoduleimports`

Additional code to be emitted to the imports section of the intermediary D module (the $importtype macro can be used here). You probably want to use this in conjunction with the `imdmodulecode` pragma.

`%pragma(d) proxydmodulecode`

Just like `proxydmodulecode`, the argument is copied to the proxy D module (if SWIG is in split proxy mode and/or the `nspace` feature is used, it is emitted to the main proxy D module only).

`%pragma(d) globalproxyimports`

The D module currently does not support specifying dependencies on external modules (e.g. from the standard library) for the D typemaps. To add the import statements to the proxy modules (resp. to *all* proxy modules if in split proxy mode), you can use the `globalproxyimports` directive.

For example:

```
%typemap(din) char[] "($dinput ? tango.stdc.stringz.toStringz($dinput) : null)"
%pragma(d) globalproxyimports = "static import tango.stdc.stringz;";
```

`%pragma(d) wrapperloadercode`

The D code for loading the wrapper library (it is copied to the intermediary D module). The `$wrapperloaderbindcode` variable is replaced by the list of commands for binding the functions from the wrapper library to the symbols in the intermediary D module.

Each time this pragma is specified, the previous value is overwritten.

`%pragma(d) wrapperloaderbindcommand`

The D command to use for binding the wrapper functions from the C/C++ library to the symbols in the intermediary D module. The `$function` variable contains the name of the D function in the wrap module, the `$symbol` variable is replaced by the name of the symbol in the library.

Each time this pragma is specified, the previous value is overwritten.

## 24.6 D Exceptions

Out of the box, C++ exceptions are fundamentally incompatible to their equivalent in the D world and cannot simply be propagated to a calling D method. There is, however, an easy way to solve this problem: Just catch the exception in the C/C++ wrapper layer, pass the contents to D, and make the wrapper code rethrow the exception in the D world.

The implementation details of this are a bit crude, but the SWIG D module automatically takes care of this, as long as it is able to detect that an exception could potentially be thrown (e.g. because the C++ method has a `throw(...)` exception specification).

As this feature is implemented in exactly the same way it is for C#, please see the C# documentation for a more detailed explanation.

## 24.7 D Directors

When the directors feature is activated, SWIG generates extra code on both the C++ and the D side to enable cross-language polymorphism. Essentially, this means that if you subclass a proxy class in D, C++ code can access any overridden virtual methods just as if you created a derived class in C++.

There is no D specific documentation yet, but the way the feature is implemented is very similar to how it is done in Java and C#.

## 24.8 Other features

### 24.8.1 Extended namespace support (nspace)

By default, SWIG flattens all C++ namespaces into a flattened D module hierarchy, but as for Java and C#, the `nspace` feature is supported for D. If the feature is active, C++ namespaces are mapped to D packages/modules. This includes the `nspace` feature flag for mirroring the C++ namespaces into D namespaces as a scoped D module/package name. It also includes `%nspacemove` for transforming a C++ namespace into a completely different scoped D module/package name. Note, however, that like for the other languages, *free* variables and functions are not supported yet; currently, they are all available in the main proxy D module.

### 24.8.2 Native pointer support

Contrary to many of the scripting languages supported by SWIG, D fully supports C-style pointers. The D module thus includes a custom mechanism to wrap C pointers directly as D pointers where applicable, that is, if the type that is pointed to is represented the same in C and D (on the bit-level), dubbed a *primitive type* below.

Central to this custom pointer handling scheme are two typemap attributes: the `cprimitive` attribute on the `dtype` typemap and the `nativepointer` attribute on all the typemaps which influence the D side of the code (`dtype`, `din`, `dout`, ...). When a D typemap is looked up, the following happens behind the scenes:

First, the matching typemap is determined by the usual typemap lookup rules. Then, it is checked if the result has the `nativepointer` attribute set. If it is present, it means that its value should replace the typemap value *if and only if* the actual type the typemap is looked up for is a primitive type, a pointer to a primitive type (through an arbitrary level of indirections), or a function pointer with only primitive types in its signature.

To determine if a type should be considered primitive, the `cprimitive` attribute on its `dtype` attribute is used. For example, the `dtype` typemap for `float` has `cprimitive="1"`, so the code from the `nativepointer` attribute is taken into account e.g. for `float **` or the function pointer `float (*)(float *)`.

### 24.8.3 Operator overloading

The D module comes with basic operator overloading support. There are, however, a few limitations arising from conceptual differences between C++ and D:

The first key difference is that C++ supports free functions as operators (along with argument-dependent lookup), while D requires operators to be member functions of the class they are operating on. SWIG can only automatically generate wrapping code for member function operators; if you want to use operators defined as free functions in D, you need to handle them manually.

Another set of differences between C++ and D concerns individual operators. For example, there are quite a few operators which are overloadable in C++, but not in D, for example `&&` and `||`, but also `!`, and postfix increment/decrement operators (see the D documentation for details).

There are also some cases where the operators can be translated to D, but the differences in the implementation details are big enough that a rather involved scheme would be required for automatic wrapping them, which has not been implemented yet. This affects, for example, the array subscript operator, `[]`, in combination with assignments - while `operator []` in C++ simply returns a reference which is then written to, D resorts to a separate `opIndexAssign` method -, or implicit casting (which was introduced in D2 via `alias this`). Despite the lack of automatic support, manually handling these cases should be perfectly possible.

### 24.8.4 Running the test-suite

As with any other language, the SWIG test-suite can be built for D using the `*-d-test-suite` targets of the top-level Makefile.

Note: If you want to use GDC on Linux or another platform which requires you to link `libdl` for dynamically loading the shared library, you might have to add `-ldl` manually to the `d_compile` target in `Examples/Makefile`, because GDC does not currently honor the `pragma(lib, ...)` statement.

## 24.9 D Typemap examples

There are no D-specific typemap examples yet. However, with the above name comparison table, you should be able to get an idea what can be done by looking at the corresponding C# section.

## 24.10 Work in progress and planned features

There are a couple of features which are not implemented yet, but would be very useful and might be added in the near future:

- *Static linking:* Currently, the C wrapper code is compiled into a dynamic library, out of which the symbol addresses are looked up at runtime by the D part. If statically linking the different languages into one binary was supported, a tool-chain capable of performing IPO at link time could inline the wrapping code, effectively reducing the overhead for simple calls to zero.

- *C array handling:* Many data structures in some C/C++ libraries contain array containing of a pointer to the first element and the element count. Currently, one must manually writing wrapper code to be able to access these from D. It should be possible to add a set of SWIG macros to semi-automatically generate conversion code.

Some generated code might also be a bit rough around the edges, particularly in the following areas:

- *Memory management:* Although the currently generated wrapper code works fine with regard to the GC for the test-suite, there might be issues coming up in real-world multi-threaded usage.
- *D2 support*: Originally, the module has been developed for the use with D1, D2/Phobos support has been added in later. The basic features should work equally well for both, but there *could* be issues concerning const-correctness etc.

---

# 25 SWIG and Go


- Overview
- Examples
- Running SWIG with Go
  - Go-specific Commandline Options
  - Generated Wrapper Files
- A tour of basic C/C++ wrapping
  - Go Package Name
  - Go Names
  - Go Constants
  - Go Enumerations
  - Go Classes
    - Go Class Memory Management
    - Go Class Inheritance
  - Go Templates
  - Go and C/C++ Threads
  - Go and C++ Exceptions
  - Go Director Classes
    - Example C++ code
    - Enable director feature
    - Constructor and destructor
    - Override virtual methods
    - Call base methods
    - Subclass via embedding
    - Memory management with runtime.SetFinalizer
    - Complete FooBarGo example class
  - Default Go primitive type mappings
  - Output arguments
  - Adding additional go code
  - Go typemaps


This chapter describes SWIG's support of Go. For more information on the Go programming language see golang.org.

## 25.1 Overview

Go does not support direct calling of functions written in C/C++. The cgo program may be used to generate wrappers to call C code from Go, but there is no convenient way to call C++ code. SWIG fills this gap.

There are (at least) two different Go compilers. The first is the gc compiler of the Go distribution, normally invoked via thego tool. SWIG supports the gc compiler version 1.2 or later. The second Go compiler is the gccgo compiler, which is a frontend to the GCC compiler suite. The interface to C/C++ code is completely different for the two Go compilers. SWIG supports both Go compilers, selected by the `-gccgo` command line option.

Go is a type-safe compiled language and the wrapper code generated by SWIG is type-safe as well. In case of type issues the build will fail and hence SWIG's runtime library and runtime type checking are not used.

## 25.2 Examples

Working examples can be found in the SWIG source tree .

Please note that the examples in the SWIG source tree use makefiles with the .i SWIG interface file extension for backwards compatibility with Go 1.

## 25.3 Running SWIG with Go

Most Go programs are built using the go tool. Since Go 1.1 the go tool has support for SWIG. To use it, give your SWIG interface file the extension .swig (for C code) or .swigcxx (for C++ code). Put that file in a GOPATH/src directory as usual for Go sources. Put other C/C++ code in the same directory with extensions of .c and .cxx. The `go build` and `go install` commands will automatically run SWIG for you and compile the generated wrapper code. To check the SWIG command line options the go tool uses run `go build -x`. To access the automatically generated files run `go build -work`. You'll find the files under the temporary WORK directory.

To manually generate and compile C/C++ wrapper code for Go, use the `-go` option with SWIG. By default SWIG will generate code for the Go compiler of the Go distribution. To generate code for gccgo, you should also use the `-gccgo` option.

By default SWIG will generate files that can be used directly by `go build`. This requires Go 1.2 or later. Put your SWIG interface file in a directory under GOPATH/src, and give it a name that does **not** end in the .swig or .swigcxx extension. Typically the SWIG interface file extension is .i in this case.

```
% swig -go example.i
% go install
```

You will now have a Go package that you can import from other Go packages as usual.

### 25.3.1 Go-specific Commandline Options

These are the command line options for SWIG's Go module. They can also be seen by using:

```
swig -go -help
```

| Go-specific options | |
|---|---|
| -cgo | Generate files to be used as input for the Go cgo tool. This is the default. |
| -no-cgo | This option is no longer supported. |

| | |
|---|---|
| -intgosize <s> | Set the size for the Go type `int` . This controls the size that the C/C++ code expects to see. The <s> argument should be 32 or 64. This option was required during the transition from Go 1.0 to Go 1.1, as the size of `int` on 64-bit x86 systems changed between those releases (from 32 bits to 64 bits). It was made optional in SWIG 4.1.0 and if not specified SWIG will assume that the size of `int` is the size of a C pointer. |
| -gccgo | Generate code for gccgo. The default is to generate code for the Go compiler of the Go distribution. |
| -package <name> | Set the name of the Go package to <name>. The default package name is the SWIG module name. |
| -use-shlib | Tell SWIG to emit code that uses a shared library. This is only meaningful for the Go compiler of the Go distribution, which needs to know at compile time whether a shared library will be used. |
| -soname <name> | Set the runtime name of the shared library that the dynamic linker should include at runtime. The default is the package name with ".so" appended. This is only used when generating code for the Go compiler of the Go distribution; when using gccgo, the equivalent name will be taken from the `-soname` option passed to the linker. Using this option implies the -use-shlib option. |
| -go-pkgpath <pkgpath> | When generating code for gccgo, set the pkgpath to use. This corresponds to the `-fgo-pkgpath` option to gccgo. |
| -go-prefix <prefix> | When generating code for gccgo, set the prefix to use. This corresponds to the `-fgo-prefix` option to gccgo. If `-go-pkgpath` is used, `-go-prefix` will be ignored. |
| -import-prefix <prefix> | A prefix to add when turning a %import prefix in the SWIG interface file into an import statement in the Go file. For example, with `-import-prefix mymodule`, a SWIG interface file `%import mypackage` will become a Go import statement `import "mymodule/mypackage"`. |

### 25.3.2 Generated Wrapper Files

SWIG will generate the following files when generating wrapper code:

- MODULE.go will contain the Go functions that your Go code will call. These functions will be wrappers for the C++ functions defined by your module. This file should, of course, be compiled with the Go compiler.
- MODULE_wrap.c or MODULE_wrap.cxx will contain C/C++ functions will be invoked by the Go wrapper code. This file should be compiled with the usual C or C++ compiler.
- MODULE_wrap.h will be generated if you use the directors feature. It provides a definition of the generated C++ director classes. It is generally not necessary to use this file, but in some special cases it may be helpful to include it in your code, compiled with the usual C or C++ compiler.

## 25.4 A tour of basic C/C++ wrapping

By default, SWIG attempts to build a natural Go interface to your C/C++ code. However, the languages are somewhat different, so some modifications have to occur. This section briefly covers the essential aspects of this wrapping.

### 25.4.1 Go Package Name

All Go source code lives in a package. The name of this package will default to the name of the module from SWIG's `%module` directive. You may override this by using SWIG's `-package` command line option.

### 25.4.2 Go Names

In Go, a function is only visible outside the current package if the first letter of the name is uppercase. This is quite different from C/C++. Because of this, C/C++ names are modified when generating the Go interface: the first letter is forced to be uppercase if it is not already. This affects the names of functions, methods, variables, constants, enums, and classes.

C/C++ variables are wrapped with setter and getter functions in Go. First the first letter of the variable name will be forced to uppercase, and then `Get` or `Set` will be prepended. For example, if the C/C++ variable is called `var`, then SWIG will define the functions `GetVar` and `SetVar`. If a variable is declared as `const`, or if SWIG's `%immutable` directive is used for the variable, then only the getter will be defined.

C++ classes will be discussed further below. Here we'll note that the first letter of the class name will be forced to uppercase to give the name of a type in Go. A constructor will be named `New` followed by that name, and the destructor will be named `Delete` followed by that name.

### 25.4.3 Go Constants

C/C++ constants created via `#define` or the `%constant` directive become Go constants, declared with a `const` declaration.

### 25.4.4 Go Enumerations

C/C++ enumeration types will cause SWIG to define an integer type with the name of the enumeration (with first letter forced to uppercase as usual). The values of the enumeration will become variables in Go; code should avoid modifying those variables.

### 25.4.5 Go Classes

Go has interfaces, methods and inheritance, but it does not have classes in the same sense as C++. This sections describes how SWIG represents C++ classes represented in Go.

For a C++ class `ClassName`, SWIG will define two types in Go: an underlying type, which will just hold a pointer to the C++ type, and an interface type. The interface type will be named `ClassName` . SWIG will define a function `NewClassName` which will take any constructor arguments and return a value of the interface type `ClassName`. SWIG will also define a destructor `DeleteClassName` .

SWIG will represent any methods of the C++ class as methods on the underlying type, and also as methods of the interface type. Thus C++ methods may be invoked directly using the usual `val.MethodName` syntax. Public members of the C++ class will be given getter and setter functions defined as methods of the class.

SWIG will represent static methods of C++ classes as ordinary Go functions. SWIG will use names like `ClassNameMethodName`. SWIG will give static members getter and setter functions with names like `GetClassNameVarName`.

Given a value of the interface type, Go code can retrieve the pointer to the C++ type by calling the `Swigcptr` method. This will return a value of type `SwigcptrClassName`, which is just a name for `uintptr`. A Go type conversion can be used to convert this value to a different C++ type, but note that this conversion will not be type checked and is essentially equivalent to `reinterpret_cast`. This should only be used for very special cases, such as where C++ would use a `dynamic_cast`.

Note that C++ pointers to compound objects are represented in go as objects themselves, not as go pointers. So, for example, if you wrap the following function:

```
class MyClass {
  int MyMethod();
  static MyClass *MyFactoryFunction();
};
```

You will get go code that looks like this:

```
type MyClass interface {
  Swigcptr() uintptr
  SwigIsMyClass()
  MyMethod() int
}
```

```
func MyClassMyFactoryFunction() MyClass {
  // swig magic here
}
```

Note that the factory function does not return a go pointer; it actually returns a go interface. If the returned pointer can be null, you can check for this by calling the Swigcptr() method.

#### 25.4.5.1 Go Class Memory Management

Calling `NewClassName` for a C++ class `ClassName` will allocate memory using the C++ memory allocator. This memory will not be automatically freed by Go's garbage collector as the object ownership is not tracked. When you are done with the C++ object you must free it using `DeleteClassName`.

The most Go idiomatic way to manage the memory for some C++ class is to call `NewClassName` followed by a `defer` of the `DeleteClassName` call. Using `defer` ensures that the memory of the C++ object is freed as soon as the function containing the `defer` statement returns. Furthermore `defer` works great for short-lived objects and fits nicely C++'s RAII idiom. Example:

```
func UseClassName(...) ... {
  o := NewClassName(...)
  defer DeleteClassName(o)
  // Use the ClassName object
  return ...
}
```

With increasing complexity, especially complex C++ object hierarchies, the correct placement of `defer` statements becomes harder and harder as C++ objects need to be freed in the correct order. This problem can be eased by keeping a C++ object function local so that it is only available to the function that creates a C++ object and functions called by this function. Example:

```
func WithClassName(constructor args, f func(ClassName, ...interface{}) error, data ...interface{}) error {
  o := NewClassName(constructor args)
  defer DeleteClassName(o)
  return f(o, data...)
}

func UseClassName(o ClassName, data ...interface{}) (err error) {
  // Use the ClassName object and additional data and return error.
}

func main() {
  WithClassName(constructor args, UseClassName, additional data)
}
```

Using `defer` has limitations though, especially when it comes to long-lived C++ objects whose lifetimes are hard to predict. For such C++ objects a common technique is to store the C++ object into a Go object, and to use the Go function `runtime.SetFinalizer` to add a finalizer which frees the C++ object when the Go object is freed. It is strongly recommended to read the runtime.SetFinalizer documentation before using this technique to understand the `runtime.SetFinalizer` limitations.

Common pitfalls with `runtime.SetFinalizer` are:

- If a hierarchy of C++ objects will be automatically freed by Go finalizers then the Go objects that store the C++ objects need to replicate the hierarchy of the C++ objects to prevent that C++ objects are freed prematurely while other C++ objects still rely on them.
- The usage of Go finalizers is problematic with C++'s RAII idiom as it isn't predictable when the finalizer will run and this might require a Close or Delete method to be added the Go object that stores a C++ object to mitigate.

`runtime.SetFinalizer` Example:

```
import (
  "runtime"
  "wrap" // SWIG generated wrapper code
)

type GoClassName struct {
  wcn wrap.ClassName
}

func NewGoClassName() *GoClassName {
  o := &GoClassName{wcn: wrap.NewClassName()}
  runtime.SetFinalizer(o, deleteGoClassName)
  return o
}

func deleteGoClassName(o *GoClassName) {
  // Runs typically in a different OS thread!
  wrap.DeleteClassName(o.wcn)
  o.wcn = nil
}

func (o *GoClassName) Close() {
  // If the C++ object has a Close method.
  o.wcn.Close()

  // If the GoClassName object is no longer in an usable state.
  runtime.SetFinalizer(o, nil) // Remove finalizer.
  deleteGoClassName() // Free the C++ object.
}
```

#### 25.4.5.2 Go Class Inheritance

C++ class inheritance is automatically represented in Go due to its use of interfaces. The interface for a child class will be a superset of the interface of its parent class. Thus a value of the child class type in Go may be passed to a function which expects the parent class. Doing the reverse will require an explicit type assertion, which will be checked dynamically.

### 25.4.6 Go Templates

In order to use C++ templates in Go, you must tell SWIG to create wrappers for a particular template instantiation. To do this, use the `%template` directive.

### 25.4.7 Go and C/C++ Threads

C and C++ code can use operating system threads and thread local storage. Go code uses goroutines, which are multiplexed onto operating system threads. This multiplexing means that Go code can change to run on a different thread at any time. C/C++ code, on the other hand, may assume that it runs on a single thread; this is true in particular if the C/C++ code uses thread local storage.

In order to use Go code with C/C++ code that expects to run on a single thread, the Go code must call the `runtime.LockOSThread` function to lock the goroutine onto a single thread.

### 25.4.8 Go and C++ Exceptions

C++ exceptions do not interoperate with Go code. Attempts to throw C++ exceptions through a Go caller are unreliable: in many cases the C++ exception handler will be unable to unwind the stack, and the program will crash. The only safe way to handle C++ exceptions is to catch them in C++ before returning to Go.

### 25.4.9 Go Director Classes

SWIG's director feature permits a Go type to act as the subclass of a C++ class. This is complicated by the fact that C++ and Go define inheritance differently. SWIG normally represents the C++ class inheritance automatically in Go via interfaces but with a Go type representing a subclass of a C++ class some manual work is necessary.

This subchapter gives a step by step guide how to properly subclass a C++ class with a Go type. In general it is strongly recommended to follow this guide completely to avoid common pitfalls with directors in Go.

#### 25.4.9.1 Example C++ code

The step by step guide is based on two example C++ classes. FooBarAbstract is an abstract C++ class and the FooBarCpp class inherits from it. This guide explains how to implement a FooBarGo class similar to the FooBarCpp class.

`FooBarAbstract` abstract C++ class:

```
class FooBarAbstract
{
public:
  FooBarAbstract() {};
  virtual ~FooBarAbstract() {};

  std::string FooBar() {
    return this->Foo() + ", " + this->Bar();
  };

protected:
  virtual std::string Foo() {
    return "Foo";
  };

  virtual std::string Bar() = 0;
};
```

`FooBarCpp` C++ class:

```
class FooBarCpp : public FooBarAbstract
{
protected:
  virtual std::string Foo() {
    return "C++ " + FooBarAbstract::Foo();
  }

  virtual std::string Bar() {
    return "C++ Bar";
  }
};
```

Returned string by the `FooBarCpp::FooBar` method is:

```
C++ Foo, C++ Bar
```

The complete example, including the `FooBarGoo` class implementation, can be found in the end of the guide.

#### 25.4.9.2 Enable director feature

The director feature is disabled by default. To use directors you must make two changes to the interface file. First, add the "directors" option to the %module directive, like this:

```
%module(directors="1") modulename
```

Second, you must use the %feature("director") directive to tell SWIG which classes should get directors. In the example the FooBarAbstract class needs the director feature enabled so that the FooBarGo class can inherit from it, like this:

```
%feature("director") FooBarAbstract;
```

For a more detailed documentation of the director feature and how to enable or disable it for specific classes and virtual methods see SWIG's Java documentation on directors.

#### 25.4.9.3 Constructor and destructor

SWIG creates an additional set of constructor and destructor functions once the director feature has been enabled for a C++ class. `NewDirectorClassName` allows overriding virtual methods on the new object instance and `DeleteDirectorClassName` needs to be used to free a director object instance created with `NewDirectorClassName`. More on overriding virtual methods follows later in this guide under overriding virtual methods.

The default constructor and destructor functions `NewClassName` and `DeleteClassName` can still be used as before so that existing code doesn't break just because the director feature has been enabled for a C++ class. The behavior is undefined if the default and director constructor and destructor functions get mixed and so great care needs to be taken that only one of the constructor and destructor function pairs is used for any object instance. Both constructor functions, the default and the director one, return the same interface type. This makes it potentially hard to know which destructor function, the default or the director one, needs to be called to delete an object instance.

In **theory** the `DirectorInterface` method could be used to determine if an object instance was created via `NewDirectorClassName`:

```
if o.DirectorInterface() != nil {
  DeleteDirectorClassName(o)
} else {
  DeleteClassName(o)
}
```

In **practice** it is strongly recommended to embed a director object instance in a Go struct so that a director object instance will be represented as a distinct Go type that subclasses a C++ class. For this Go type custom constructor and destructor functions take care of the director constructor and destructor function calls and the resulting Go class will appear to the user as any other SWIG wrapped C++ class. More on properly subclassing a C++ class follows later in this guide under subclass via embedding .

#### 25.4.9.4 Override virtual methods

In order to override virtual methods on a C++ class with Go methods the `NewDirectorClassName` constructor functions receives a `DirectorInterface` argument. The methods in the `DirectorInterface` are a subset of the public and protected virtual methods of the C++ class. Virtual methods that have a final specifier are unsurprisingly excluded. If the `DirectorInterface` contains a method with a matching signature to a virtual method of the C++ class then the virtual C++ method will be overwritten with the Go method. As Go doesn't support protected methods all overridden protected virtual C++ methods will be public in Go.

As an example see part of the `FooBarGo` class:

```
type overwrittenMethodsOnFooBarAbstract struct {
  fb FooBarAbstract
}

func (om *overwrittenMethodsOnFooBarAbstract) Foo() string {
  ...
}

func (om *overwrittenMethodsOnFooBarAbstract) Bar() string {
  ...
}

func NewFooBarGo() FooBarGo {
  om := &overwrittenMethodsOnFooBarAbstract{}
  fb := NewDirectorFooBarAbstract(om)
  om.fb = fb
  ...
}
```

The complete example, including the `FooBarGoo` class implementation, can be found in the end of the guide. In this part of the example the virtual methods `FooBarAbstract::Foo` and `FooBarAbstract::Bar` have been overwritten with Go methods similarly to how the `FooBarAbstract` virtual methods are overwritten by the `FooBarCpp` class.

The `DirectorInterface` in the example is implemented by the `overwrittenMethodsOnFooBarAbstract` Go struct type. A pointer to a `overwrittenMethodsOnFooBarAbstract` struct instance will be given to the `NewDirectorFooBarAbstract` constructor function. The constructor return value implements the `FooBarAbstract` interface. `overwrittenMethodsOnFooBarAbstract` could in theory be any Go type but in practice a struct is used as it typically contains at least a value of the C++ class interface so that the overwritten methods can use the rest of the C++ class. If the `FooBarGo` class would receive additional constructor arguments then these would also typically be stored in the `overwrittenMethodsOnFooBarAbstract` struct so that they can be used by the Go methods.

#### 25.4.9.5 Call base methods

Often a virtual method will be overwritten to extend the original behavior of the method in the base class. This is also the case for the `FooBarCpp::Foo` method of the example code:

```
virtual std::string Foo() {
  return "C++ " + FooBarAbstract::Foo();
}
```

To use base methods the `DirectorClassNameMethodName` wrapper functions are automatically generated by SWIG for public and protected virtual methods. The `FooBarGo.Foo` implementation in the example looks like this:

```
func (om *overwrittenMethodsOnFooBarAbstract) Foo() string {
  return "Go " + DirectorFooBarAbstractFoo(om.fb)
}
```

The complete example, including the `FooBarGoo` class implementation, can be found in the end of the guide.

#### 25.4.9.6 Subclass via embedding

As previously mentioned in this guide the default and director constructor functions return the same interface type. To properly subclass a C++ class with a Go type the director object instance returned by the `NewDirectorClassName` constructor function should be embedded into a Go struct so that it represents a distinct but compatible type in Go's type system. This Go struct should be private and the constructor and destructor functions should instead work with a public interface type so that the Go class that subclasses a C++ class can be used as a compatible drop in.

The subclassing part of the `FooBarGo` class for an example looks like this:

```
type FooBarGo interface {
  FooBarAbstract
  deleteFooBarAbstract()
  IsFooBarGo()
}

type fooBarGo struct {
  FooBarAbstract
}

func (fbgs *fooBarGo) deleteFooBarAbstract() {
  DeleteDirectorFooBarAbstract(fbgs.FooBarAbstract)
}

func (fbgs *fooBarGo) IsFooBarGo() {}

func NewFooBarGo() FooBarGo {
  om := &overwrittenMethodsOnFooBarAbstract{}
  fb := NewDirectorFooBarAbstract(om)
  om.fb = fb

  return &fooBarGo{FooBarAbstract: fb}
}

func DeleteFooBarGo(fbg FooBarGo) {
  fbg.deleteFooBarAbstract()
}
```

The complete example, including the `FooBarGoo` class implementation, can be found in the end of the guide. In this part of the example the private `fooBarGo` struct embeds `FooBarAbstract` which lets the `fooBarGo` Go type "inherit" all the methods of the `FooBarAbstract` C++ class by means of embedding. The public `FooBarGo` interface type includes the `FooBarAbstract` interface and hence `FooBarGo` can be used as a drop in replacement for `FooBarAbstract` while the reverse isn't possible and would raise a compile time error. Furthermore the constructor and destructor functions `NewFooBarGo` and `DeleteFooBarGo` take care of all the director specifics and to the user the class appears as any other SWIG wrapped C++ class.

#### 25.4.9.7 Memory management with runtime.SetFinalizer

In general all guidelines for C++ class memory management apply as well to director classes. One often overlooked limitation with `runtime.SetFinalizer` is that a finalizer doesn't run in case of a cycle and director classes typically have a cycle. The cycle in the `FooBarGo` class is here:

```
type overwrittenMethodsOnFooBarAbstract struct {
  fb FooBarAbstract
}

func NewFooBarGo() FooBarGo {
  om := &overwrittenMethodsOnFooBarAbstract{}
  fb := NewDirectorFooBarAbstract(om) // fb.v = om
  om.fb = fb // Backlink causes cycle as fb.v = om!
  ...
}
```

In order to be able to use `runtime.SetFinalizer` nevertheless the finalizer needs to be set on something that isn't in a cycle and that references the director object instance. In the `FooBarGo` class example the `FooBarAbstract` director instance can be automatically deleted by setting the finalizer on `fooBarGo`:

```
type fooBarGo struct {
  FooBarAbstract
}

type overwrittenMethodsOnFooBarAbstract struct {
  fb FooBarAbstract
}

func NewFooBarGo() FooBarGo {
  om := &overwrittenMethodsOnFooBarAbstract{}
  fb := NewDirectorFooBarAbstract(om)
  om.fb = fb // Backlink causes cycle as fb.v = om!

  fbgs := &fooBarGo{FooBarAbstract: fb}
  runtime.SetFinalizer(fbgs, FooBarGo.deleteFooBarAbstract)
  return fbgs
}
```

Furthermore if `runtime.SetFinalizer` is in use either the `DeleteClassName` destructor function needs to be removed or the `fooBarGo` struct needs additional data to prevent double deletion. Please read the C++ class memory management subchapter before using `runtime.SetFinalizer` to know all of its gotchas.

#### 25.4.9.8 Complete FooBarGo example class

The complete and annotated `FooBarGo` class looks like this:

```
// FooBarGo is a superset of FooBarAbstract and hence FooBarGo can be used as a
// drop in replacement for FooBarAbstract but the reverse causes a compile time
// error.
type FooBarGo interface {
  FooBarAbstract
  deleteFooBarAbstract()
  IsFooBarGo()
}

// Via embedding fooBarGo "inherits" all methods of FooBarAbstract.
type fooBarGo struct {
  FooBarAbstract
}

func (fbgs *fooBarGo) deleteFooBarAbstract() {
  DeleteDirectorFooBarAbstract(fbgs.FooBarAbstract)
}

// The IsFooBarGo method ensures that FooBarGo is a superset of FooBarAbstract.
// This is also how the class hierarchy gets represented by the SWIG generated
// wrapper code.  For an instance FooBarCpp has the IsFooBarAbstract and
// IsFooBarCpp methods.
func (fbgs *fooBarGo) IsFooBarGo() {}

// Go type that defines the DirectorInterface. It contains the Foo and Bar
// methods that overwrite the respective virtual C++ methods on FooBarAbstract.
type overwrittenMethodsOnFooBarAbstract struct {
  // Backlink to FooBarAbstract so that the rest of the class can be used by
  // the overridden methods.
  fb FooBarAbstract

  // If additional constructor arguments have been given they are typically
  // stored here so that the overridden methods can use them.
}

func (om *overwrittenMethodsOnFooBarAbstract) Foo() string {
  // DirectorFooBarAbstractFoo calls the base method FooBarAbstract::Foo.
  return "Go " + DirectorFooBarAbstractFoo(om.fb)
}

func (om *overwrittenMethodsOnFooBarAbstract) Bar() string {
  return "Go Bar"
}

func NewFooBarGo() FooBarGo {
  // Instantiate FooBarAbstract with selected methods overridden.  The methods
  // that will be overwritten are defined on
```

```
  // overwrittenMethodsOnFooBarAbstract and have a compatible signature to the
  // respective virtual C++ methods. Furthermore additional constructor
  // arguments will be typically stored in the
  // overwrittenMethodsOnFooBarAbstract struct.
  om := &overwrittenMethodsOnFooBarAbstract{}
  fb := NewDirectorFooBarAbstract(om)
  om.fb = fb // Backlink causes cycle as fb.v = om!

  fbgs := &fooBarGo{FooBarAbstract: fb}
  // The memory of the FooBarAbstract director object instance can be
  // automatically freed once the FooBarGo instance is garbage collected by
  // uncommenting the following line.  Please make sure to understand the
  // runtime.SetFinalizer specific gotchas before doing this.  Furthermore
  // DeleteFooBarGo should be deleted if a finalizer is in use or the fooBarGo
  // struct needs additional data to prevent double deletion.
  // runtime.SetFinalizer(fbgs, FooBarGo.deleteFooBarAbstract)
  return fbgs
}

// Recommended to be removed if runtime.SetFinalizer is in use.
func DeleteFooBarGo(fbg FooBarGo) {
  fbg.deleteFooBarAbstract()
}
```

Returned string by the `FooBarGo.FooBar` method is:

```
Go Foo, Go Bar
```

For comparison the `FooBarCpp` class looks like this:

```
class FooBarCpp : public FooBarAbstract
{
protected:
  virtual std::string Foo() {
    return "C++ " + FooBarAbstract::Foo();
  }

  virtual std::string Bar() {
    return "C++ Bar";
  }
};
```

For comparison the returned string by the `FooBarCpp::FooBar` method is:

```
C++ Foo, C++ Bar
```

The complete source of this example can be found under SWIG/Examples/go/director/.

### 25.4.10 Default Go primitive type mappings

The following table lists the default type mapping from C/C++ to Go. This table will tell you which Go type to expect for a function which uses a given C/C++ type.

| C/C++ type | Go type |
|---|---|
| bool | bool |
| char | byte |
| signed char | int8 |
| unsigned char | byte |
| short | int16 |
| unsigned short | uint16 |
| int | int |
| unsigned int | uint |
| long | int64 |
| unsigned long | uint64 |
| long long | int64 |
| unsigned long long | uint64 |
| float | float32 |
| double | float64 |
| char *<br>char [] | string |

Note that SWIG wraps the C `char` type as a character. Pointers and arrays of this type are wrapped as strings. The `signed char` type can be used if you want to treat`char` as a signed number rather than a character. Also note that all const references to primitive types are treated as if they are passed by value.

These type mappings are defined by the "gotype" typemap. You may change that typemap, or add new values, to control how C/C++ types are mapped into Go types.

### 25.4.11 Output arguments

Because of limitations in the way output arguments are processed in swig, a function with output arguments will not have multiple return values. Instead, you must pass a pointer into the C++ function to tell it where to store the output value. In go, you supply a slice in the place of the output argument.

For example, suppose you were trying to wrap the modf() function in the C math library which splits x into integral and fractional parts (and returns the integer part in one of its parameters):

```
double modf(double x, double *ip);
```

You could wrap it with SWIG as follows:

```
%include <typemaps.i>
```

```
double modf(double x, double *OUTPUT);
```

or you can use the `%apply` directive:

```
%include <typemaps.i>
%apply double *OUTPUT { double *ip };
double modf(double x, double *ip);
```

In Go you would use it like this:

```
ptr := []float64{0.0}
fraction := modulename.Modf(5.0, ptr)
```

Since this is ugly, you may want to wrap the swig-generated API with some additional functions written in go that hide the ugly details.

There are no `char *OUTPUT` typemaps. However you can apply the `signed char *` typemaps instead:

```
%include <typemaps.i>
%apply signed char *OUTPUT {char *output};
void f(char *output);
```

### 25.4.12 Adding additional go code

Often the APIs generated by swig are not very natural in go, especially if there are output arguments. You can insert additional go wrapping code to add new APIs with `%insert(go_wrapper)`, like this:

```
%include <typemaps.i>
// Change name of what swig generates to Wrapped_modf.  This function will
// have the following signature in go:
//   func Wrapped_modf(float64, []float64) float64
%rename(wrapped_modf) modf(double x, double *ip);

%apply double *OUTPUT { double *ip };
double modf(double x, double *ip);

%insert(go_wrapper) %{

// The improved go interface to this function, which has two return values,
// in the more natural go idiom:
func Modf(x float64) (fracPart float64, intPart float64) {
  ip := []float64{0.0}
  fracPart = Wrapped_modf(x, ip)
  intPart = ip[0]
  return
}

%}
```

For classes, since swig generates an interface, you can add additional methods by defining another interface that includes the swig-generated interface. For example,

```
%rename(Wrapped_MyClass) MyClass;
%rename(Wrapped_GetAValue) MyClass::GetAValue(int *x);
%apply int *OUTPUT { int *x };

class MyClass {
 public:
  MyClass();
  int AFineMethod(const char *arg); // Swig's wrapping is fine for this one.
  bool GetAValue(int *x);
};

%insert(go_wrapper) %{

type MyClass interface {
  Wrapped_MyClass
  GetAValue() (int, bool)
}

func (arg SwigcptrWrapped_MyClass) GetAValue() (int, bool) {
  ip := []int{0}
  ok := arg.Wrapped_GetAValue(ip)
  return ip[0], ok
}

%}
```

Of course, if you have to rewrite most of the methods, instead of just a few, then you might as well define your own struct that includes the swig-wrapped object, instead of adding methods to the swig-generated object.

If you need to import other go packages, you can do this with `%go_import`. For example,

```
%go_import("fmt", _ "unusedPackage", rp "renamed/package")

%insert(go_wrapper) %{

func foo() {
  fmt.Println("Some string:", rp.GetString())
}

// Importing the same package twice is permitted,
// Go code will be generated with only the first instance of the import.
```

```
%go_import("fmt")

%insert(go_wrapper) %{

func bar() {
  fmt.Println("Hello world!")
}

%}
```

### 25.4.13 Go typemaps

You can use the `%typemap` directive to modify SWIG's default wrapping behavior for specific C/C++ types. You need to be familiar with the material in the general "Typemaps" chapter. That chapter explains how to define a typemap. This section describes some specific typemaps used for Go.

In general type conversion code may be written either in C/C++ or in Go. The choice to make normally depends on where memory should be allocated. To allocate memory controlled by the Go garbage collector, write Go code. To allocate memory in the C/C++ heap, write C code.

| Typemap | Description |
|---|---|
| gotype | The Go type to use for a C++ type. This type will appear in the generated Go wrapper function. If this is not defined SWIG will use a default as described above. |
| imtype | An intermediate Go type used by the "goin", "goout", "godirectorin", and "godirectorout" typemaps. If this typemap is not defined for a C/C++ type, the gotype typemap will be used. This is useful when gotype is best converted to C/C++ using Go code. |
| goin | Go code to convert from gotype to imtype when calling a C/C++ function. SWIG will then internally convert imtype to a C/C++ type and pass it down. If this is not defined, or is the empty string, no conversion is done. |
| in | C/C++ code to convert the internally generated C/C++ type, based on imtype, into the C/C++ type that a function call expects. If this is not defined the value will simply be cast to the desired type. |
| out | C/C++ code to convert the C/C++ type that a function call returns into the internally generated C/C++ type, based on imtype, that will be returned to Go. If this is not defined the value will simply be cast to the desired type. |
| goout | Go code to convert a value returned from a C/C++ function from imtype to gotype. If this is not defined, or is the empty string, no conversion is done. |
| argout | C/C++ code to adjust an argument value when returning from a function. This is called after the real C/C++ function has run. This uses the internally generated C/C++ type, based on imtype. This is only useful for a pointer type of some sort. If this is not defined nothing will be done. |
| goargout | Go code to adjust an argument value when returning from a function. This is called after the real C/C++ function has run. The value will be in imtype. This is only useful for a pointer type of some sort. If this is not defined, or is the empty string, nothing will be done. |
| directorin | C/C++ code to convert the C/C++ type used to call a director method into the internally generated C/C++ type, based on imtype, that will be passed to Go. If this is not defined the value will simply be cast to the desired type. |
| godirectorin | Go code to convert a value used to call a director method from imtype to gotype. If this is not defined, or is the empty string, no conversion is done. |
| godirectorout | Go code to convert a value returned from a director method from gotype to imtype. If this is not defined, or is the empty string, no conversion is done. |
| directorout | C/C++ code to convert a value returned from a director method from the internally generated C/C++ type, based on imtype, into the type that the method should return If this is not defined the value will simply be cast to the desired type. |

# 26 SWIG and Guile


- Supported Guile Versions
- Meaning of "Module"
- Old GH Guile API
- Linkage
  - Simple Linkage
  - Passive Linkage
  - Native Guile Module Linkage
  - Old Auto-Loading Guile Module Linkage
  - Hobbit4D Linkage
- Underscore Folding
- Typemaps
- Representation of pointers as smobs
  - Smobs
  - Garbage Collection
- Native Guile pointers
- Exception Handling
- Procedure documentation
- Procedures with setters
- GOOPS Proxy Classes
  - Naming Issues
  - Linking


This section details guile-specific support in SWIG.

## 26.1 Supported Guile Versions

SWIG is known to work with Guile versions 2.0.x, 2.2.x and 3.0.x (these are all tested via CI). SWIG probably still works with Guile 1.8.x but we're no longer able to regularly test this either in CI or by hand. Support for Guile 1.6.x has been dropped (SWIG 2.0.9 was the last version of SWIG to support it).

Note that starting with guile 2.0, the guile sources can be compiled for improved performance. This is currently not tested with swig so your mileage may vary. To be safe set environment variable `GUILE_AUTO_COMPILE` to 0 when using swig generated guile code.

## 26.2 Meaning of "Module"

There are three different concepts of "module" involved, defined separately for SWIG, Guile, and Libtool. To avoid horrible confusion, we explicitly prefix the context, e.g., "guile-module".

## 26.3 Old GH Guile API

Guile 1.8 and older could be interfaced using two different api's, the SCM or the GH API. The GH interface to guile is deprecated. Read more about why in the Guile manual.

Support for the guile GH wrapper code generation has been dropped from SWIG. The last version of SWIG that can still generate guile GH wrapper code is 2.0.9. Please use that version if you really need the GH wrapper code.

## 26.4 Linkage

Guile support is complicated by a lack of user community cohesiveness, which manifests in multiple shared-library usage conventions. A set of policies implementing a usage convention is called a **linkage**.

### 26.4.1 Simple Linkage

The default linkage is the simplest; nothing special is done. In this case the function `SWIG_init()` is exported. Simple linkage can be used in several ways:

- **Embedded Guile, no modules.** You want to embed a Guile interpreter into your program; all bindings made by SWIG shall show up in the root module. Then call `SWIG_init()` in the `inner_main()` function. See the "simple" and "matrix" examples under `Examples/guile`.
- **Dynamic module mix-in.** You want to create a Guile module using `define-module`, containing both Scheme code and bindings made by SWIG; you want to load the SWIG modules as shared libraries into Guile.

  ```
  (define-module (my module))
  (define my-so (dynamic-link "./libexample.so"))
  (dynamic-call "SWIG_init" my-so) ; make SWIG bindings
  ;; Scheme definitions can go here
  ```

  Newer Guile versions provide a shorthand for `dynamic-link` and `dynamic-call`:

  ```
  (load-extension "./libexample.so" "SWIG_init")
  ```

  A more portable approach would be to drop the shared library extension:

  ```
  (load-extension "./libexample" "SWIG_init")
  ```

  You need to explicitly export those bindings made by SWIG that you want to import into other modules:

  ```
  (export foo bar)
  ```

  In this example, the procedures `foo` and `bar` would be exported. Alternatively, you can export all bindings with the following module-system hack:

  ```
  (module-map (lambda (sym var)
                (module-export! (current-module) (list sym)))
              (current-module))
  ```

  SWIG can also generate this Scheme stub (from `define-module` up to `export`) semi-automagically if you pass it the command-line argument `-scmstub`. The code will be exported in a file called *`module`*`.scm` in the directory specified by `-outdir` or the current directory if `-outdir` is not specified. Since SWIG doesn't know how to load your extension module (with `dynamic-link` or `load-extension`), you need to supply this information by including a directive like this in the interface file:

  ```
  %scheme %{ (load-extension "./libexample.so" "SWIG_init") %}
  ```

  (The `%scheme` directive allows inserting arbitrary Scheme code into the generated file *`module.scm`*; it is placed between the `define-module` form and the `export` form.)

If you want to include several SWIG modules, you would need to rename `SWIG_init` via a preprocessor define to avoid symbol clashes. For this case, however, passive linkage is available.

### 26.4.2 Passive Linkage

Passive linkage is just like simple linkage, but it generates an initialization function whose name is derived from the module and package name (see below).

You should use passive linkage rather than simple linkage when you are using multiple modules.

### 26.4.3 Native Guile Module Linkage

SWIG can also generate wrapper code that does all the Guile module declarations on its own if you pass it the `-Linkage module` command-line option.

The module name is set with the `-package` and `-module` command-line options. Suppose you want to define a module with name `(my lib foo)`; then you would have to pass the options `-package` *`my/lib`* `-module` *`foo`* . Note that the last part of the name can also be set via the SWIG directive `%module`.

You can use this linkage in several ways:

- **Embedded Guile with SWIG modules.** You want to embed a Guile interpreter into your program; the SWIG bindings shall be put into different modules. Simply call the function `scm_init_`*`my`*` _`*`modules`*`_foo_module` in the `inner_main()` function.
- **Dynamic Guile modules.** You want to load the SWIG modules as shared libraries into Guile; all bindings are automatically put in newly created Guile modules.

  ```
  (define my-so (dynamic-link "./libfoo"))
  ;; create new module and put bindings there:
  (dynamic-call "scm_init_my_modules_foo_module" my-so)
  ```

  Newer Guile versions have a shorthand procedure for this:

  ```
  (load-extension "./libfoo.so" "scm_init_my_modules_foo_module")
  ```

### 26.4.4 Old Auto-Loading Guile Module Linkage

Guile used to support an autoloading facility for object-code modules, but this support was deprecated and removed in Guile version 1.4.1. SWIG supported this via option `-Linkage ltdlmod`, but this support is no longer useful and was removed in SWIG 4.2.0.

### 26.4.5 Hobbit4D Linkage

The only other linkage supported at this time creates shared object libraries suitable for use by hobbit's `(hobbit4d link)` guile module. This is called the "hobbit" linkage, and requires also using the "-package" command line option to set the part of the module name before the last symbol. For example, both command lines:

```
swig -guile -package my/lib foo.i
swig -guile -package my/lib -module foo foo.i
```

would create module `(my lib foo)` (assuming in the first case foo.i declares the module to be "foo"). The installed files are my/lib/libfoo.so.X.Y.Z and friends. This scheme is still very experimental; the (hobbit4d link) conventions are not well understood.

## 26.5 Underscore Folding

Underscores are converted to dashes in identifiers. Guile support may grow an option to inhibit this folding in the future, but no one has complained so far.

You can use the SWIG directive `%rename` to specify the Guile names of the wrapped functions and variables.

## 26.6 Typemaps

The Guile module handles all types via typemaps. This information is read from `Lib/guile/typemaps.i`. Some non-standard typemap substitutions are supported:

- `$descriptor` expands to a type descriptor for use with the `SWIG_NewPointerObj()` and `SWIG_ConvertPtr` functions.
- For pointer types, `$*descriptor` expands to a descriptor for the direct base type (i.e., one pointer is stripped), whereas `$basedescriptor` expands to a descriptor for the base type (i.e., all pointers are stripped).

A function returning `void` (more precisely, a function whose `out` typemap returns `SCM_UNSPECIFIED`) is treated as returning no values. In `argout` typemaps, one can use the macro `GUILE_APPEND_RESULT` in order to append a value to the list of function return values.

Multiple values can be passed up to Scheme in one of three ways:

- *Multiple values as lists.* By default, if more than one value is to be returned, a list of the values is created and returned; to switch back to this behavior, use

  ```
  %values_as_list;
  ```

- *Multiple values as vectors.* By issuing

  ```
  %values_as_vector;
  ```

  vectors instead of lists will be used.

- *Multiple values for multiple-value continuations.* **This is the most elegant way.** By issuing

  ```
  %multiple_values;
  ```

  multiple values are passed to the multiple-value continuation, as created by `call-with-values` or the convenience macro `receive`. The latter is available if you issue `(use-modules (srfi srfi-8))`. Assuming that your `divide` function wants to return two values, a quotient and a remainder, you can write:

  ```
  (receive (quotient remainder)
      (divide 35 17)
    body...)
  ```

  In *`body`*, the first result of `divide` will be bound to the variable `quotient`, and the second result to `remainder`.

See also the "multivalue" example.

Constants are exported as a function that returns the value. The %feature("constasvar") can be applied to any constant, immutable variable, or enum. Instead of exporting the constant as a function that must be called, the constant will appear as a scheme variable. See Features and the %feature directive for info on how to apply the %feature.

## 26.7 Representation of pointers as smobs

For pointer types, SWIG uses Guile smobs. SWIG smobs print like this: `#<swig struct xyzzy * 0x1234affe>` Two of them are `equal?` if and only if they have the same type and value.

To construct a Scheme object from a C pointer, the wrapper code calls the function `SWIG_NewPointerObj()`, passing a pointer to a struct representing the pointer type. The type index to store in the upper half of the CAR is read from this struct. To get the pointer represented by a smob, the wrapper code calls the function `SWIG_ConvertPtr()`, passing a pointer to a struct representing the expected pointer type. See also The run-time type checker. If the Scheme object passed was not a SWIG smob representing a compatible pointer, a `wrong-type-arg` exception is raised.

### 26.7.1 Smobs

In earlier versions of SWIG, C pointers were represented as Scheme strings containing a hexadecimal rendering of the pointer value and a mangled type name. As Guile allows registering user types, so-called "smobs" (small objects), a much cleaner representation has been implemented now. The details will be discussed in the following.

The whole type system, when it is first initialized, creates two smobs named "swig" and "collected_swig". The swig smob is used for non-garbage collected smobs, while the collected_swig smob is used as described below. Each smob has the same format, which is a double cell created by SCM_NEWSMOB2() The first word of data is the pointer to the object and the second word of data is the swig_type_info * structure describing this type. If a generated GOOPS module has been loaded, smobs will be wrapped by the corresponding GOOPS class.

### 26.7.2 Garbage Collection

Garbage collection is a feature of Guile since version 1.6. As SWIG now requires Guile > 1.8, it is automatically included. Garbage collection works like this. Every swig_type_info structure stores in its clientdata field a pointer to the destructor for this type. The destructor is the generated wrapper around the delete function. So swig still exports a wrapper for the destructor, it just does not call scm_c_define_gsubr() for the wrapped delete function. So the only way to delete an object is from the garbage collector, since the delete function is not available to scripts. How swig determines if a type should be garbage collected is exactly like described in Object ownership and %newobject in the SWIG manual. All typemaps use an $owner var, and the guile module replaces $owner with 0 or 1 depending on feature:new.

## 26.8 Native Guile pointers

In addition to SWIG smob pointers, Guile's native pointer type are accepted as arguments to wrapped SWIG functions. This can be useful for passing pointers to bytevector data to wrapped functions.

## 26.9 Exception Handling

SWIG code calls `scm_error` on exception, using the following mapping:

```
    MAP(SWIG_MemoryError,         "swig-memory-error");
    MAP(SWIG_IOError,             "swig-io-error");
    MAP(SWIG_RuntimeError,        "swig-runtime-error");
    MAP(SWIG_IndexError,          "swig-index-error");
    MAP(SWIG_TypeError,           "swig-type-error");
    MAP(SWIG_DivisionByZero,      "swig-division-by-zero");
    MAP(SWIG_OverflowError,       "swig-overflow-error");
    MAP(SWIG_SyntaxError,         "swig-syntax-error");
    MAP(SWIG_ValueError,          "swig-value-error");
    MAP(SWIG_SystemError,         "swig-system-error");
```

```
      MAP(SWIG_NullReferenceError,  "swig-null-reference-error");
```

The default when not specified here is to use "swig-error". See Lib/exception.i for details.

## 26.10 Procedure documentation

If invoked with the command-line option `-procdoc file` , SWIG creates documentation strings for the generated wrapper functions, describing the procedure signature and return value, and writes them to *file*.

SWIG can generate documentation strings in three formats, which are selected via the command-line option `-procdocformat format` :

- `guile-1.4` (default): Generates a format suitable for Guile 1.4.
- `plain`: Generates a format suitable for Guile 1.4.1 and later.
- `texinfo`: Generates texinfo source, which must be run through texinfo in order to get a format suitable for Guile 1.4.1 and later.

You need to register the generated documentation file with Guile like this:

```
(use-modules (ice-9 documentation))
(set! documentation-files
      (cons "file" documentation-files))
```

Documentation strings can be configured using the Guile-specific typemap argument `doc`. See `Lib/guile/typemaps.i` for details.

## 26.11 Procedures with setters

For global variables, SWIG creates a single wrapper procedure `( variable :optional value)`, which is used for both getting and setting the value. For struct members, SWIG creates two wrapper procedures `(struct-member-get pointer)` and `(struct-member-set pointer value)`.

If invoked with the command-line option `-emit-setters` ( *recommended*), SWIG will additionally create procedures with setters. For global variables, the procedure-with-setter `variable` is created, so you can use `(variable )` to get the value and `(set! (variable) value)` to set it. For struct members, the procedure-with-setter `struct-member` is created, so you can use `(struct-member pointer)` to get the value and `(set! (struct -member pointer) value)` to set it.

If invoked with the command-line option `-only-setters`, SWIG will *only* create procedures with setters, i.e., for struct members, the procedures `(struct-member -get pointer)` and `(struct-member-set pointer value)` are *not* generated.

## 26.12 GOOPS Proxy Classes

SWIG can also generate classes and generic functions for use with Guile's Object-Oriented Programming System (GOOPS). GOOPS is a sophisticated object system in the spirit of the Common Lisp Object System (CLOS).

To enable GOOPS support, pass the `-proxy` argument to swig. This will export the GOOPS wrapper definitions into the `module.scm` file in the directory specified by -outdir or the current directory. GOOPS support requires either passive or module linkage.

The generated file will contain definitions of GOOPS classes mimicking the C++ class hierarchy.

Enabling GOOPS support implies `-emit-setters`.

If `-emit-slot-accessors` is also passed as an argument, then the generated file will contain accessor methods for all the slots in the classes and for global variables. The input class

```
  class Foo {
    public:
      Foo(int i) : a(i) {}
      int a;
      int getMultBy(int i) { return a * i; }
      Foo getFooMultBy(int i) { return Foo(a * i); }
  };
  Foo getFooPlus(int i) { return Foo(a + i); }
```

will produce (if `-emit-slot-accessors` is not passed as a parameter)

```
(define-class <Foo> (<swig>)
  (a #:allocation #:swig-virtual
     #:slot-ref primitive:Foo-a-get
     #:slot-set! primitive:Foo-a-set)
  #:metaclass <swig-metaclass>
  #:new-function primitive:new-Foo
)
(define-method (getMultBy (swig_smob <Foo>) i)
  (primitive:Foo-getMultBy  (slot-ref swig_smob 'smob) i))
(define-method (getFooMultBy (swig_smob <Foo>) i)
  (make <Foo> #:init-smob (primitive:Foo-getFooMultBy  (slot-ref swig_smob 'smob) i)))

(define-method (getFooPlus i)
  (make <Foo> #:init-smob (primitive:getFooPlus i)))

(export <Foo> getMultBy getFooMultBy getFooPlus )
```

and will produce (if `-emit-slot-accessors` is passed as a parameter)

```
(define-class <Foo> (<swig>)
  (a #:allocation #:swig-virtual
     #:slot-ref primitive:Foo-a-get
     #:slot-set! primitive:Foo-a-set
     #:accessor a)
  #:metaclass <swig-metaclass>
  #:new-function primitive:new-Foo
)
(define-method (getMultBy (swig_smob <Foo>) i)
  (primitive:Foo-getMultBy  (slot-ref swig_smob 'smob) i))
(define-method (getFooMultBy (swig_smob <Foo>) i)
  (make <Foo> #:init-smob (primitive:Foo-getFooMultBy  (slot-ref swig_smob 'smob) i)))
```

```
(define-method (getFooPlus i)
  (make <Foo> #:init-smob (primitive:getFooPlus i)))

(export <Foo> a getMultBy getFooMultBy getFooPlus )
```

which can then be used by this code

```
;; not using getters and setters
(define foo (make <Foo> #:args '(45)))
(slot-ref foo 'a)
(slot-set! foo 'a 3)
(getMultBy foo 4)
(define foo2 (getFooMultBy foo 7))
(slot-ref foo 'a)
(slot-ref (getFooPlus foo 4) 'a)

;; using getters and setters
(define foo (make <Foo> #:args '(45)))
(a foo)
(set! (a foo) 5)
(getMultBy foo 4)
(a (getFooMultBy foo 7))
```

Notice that constructor arguments are passed as a list after the `#:args` keyword. Hopefully in the future the following will be valid `(make <Foo> #:a 5 #:b 4)`

Also note that the order the declarations occur in the .i file make a difference. For example,

```
%module test

%{ #include "foo.h" %}

%inline %{
  int someFunc(Foo &a) {
    ...
  }
%}

%include "foo.h"
```

This is a valid SWIG file it will work as you think it will for primitive support, but the generated GOOPS file will be broken. Since the `someFunc` definition is parsed by SWIG before all the declarations in foo.h, the generated GOOPS file will contain the definition of `someFunc()` before the definition of <Foo>. The generated GOOPS file would look like

```
;;...

(define-method (someFunc (swig_smob <Foo>))
  (primitive:someFunc (slot-ref swig_smob 'smob)))

;;...

(define-class <Foo> (<swig>)
  ;;...
)

;;...
```

Notice that <Foo> is used before it is defined. The fix is to just put the `%import "foo.h"` before the `%inline` block.

### 26.12.1 Naming Issues

As you can see in the example above, there are potential naming conflicts. The default exported accessor for the `Foo::a` variable is named `a`. The name of the wrapper global function is `getFooPlus`. If the `-useclassprefix` option is passed to swig, the name of all accessors and member functions will be prepended with the class name. So the accessor will be called `Foo-a` and the member functions will be called `Foo-getMultBy`. Also, if the `-goopsprefix goops:` argument is passed to swig, every identifier will be prefixed by `goops:`

Two guile-modules are created by SWIG. The first module contains the primitive definitions of all the wrapped functions and variables, and is located either in the _wrap.cxx file (with `-Linkage module`) or in the scmstub file (if `-Linkage passive -scmstub`). The name of this guile-module is the swig-module name (given on the command line with the -module argument or with the %module directive) concatenated with the string "-primitive". For example, if `%module Test` is set in the swig interface file, the name of the guile-module in the scmstub or `-Linkage module` will be `Test-primitive`. Also, the scmstub file will be named `Test-primitive.scm`. The string "primitive" can be changed by the `-primsuffix` swig argument. So the same interface, with the `-primsuffix base` will produce a module called `Test-base` . The second generated guile-module contains all the GOOPS class definitions and is located in a file named *module*.scm in the directory specified with -outdir or the current directory. The name of this guile-module is the name of the swig-module (given on the command line or with the `%module` directive). In the previous example, the GOOPS definitions will be in a file named Test.scm.

Because of the naming conflicts, you can't in general use both the `-primitive` and the GOOPS guile-modules at the same time. To do this, you need to rename the exported symbols from one or both guile-modules. For example,

```
(use-modules ((Test-primitive) #:renamer (symbol-prefix-proc 'primitive:)))
(use-modules ((Test) #:renamer (symbol-prefix-proc 'goops:)))
```

### 26.12.2 Linking

The guile-modules generated above all need to be linked together. GOOPS support requires either passive or module linkage. The exported GOOPS guile-module will be the name of the swig-module and should be located in a file called *Module*.scm. This should be installed on the autoload path for guile, so that `(use-modules (Package Module))` will load everything needed. Thus, the top of the GOOPS guile-module will contain code to load everything needed by the interface (the shared library, the scmstub module, etc.). The `%goops` directive inserts arbitrary code into the generated GOOPS guile-module, and should be used to load the dependent libraries.

This breaks up into three cases

- **Passive Linkage without -scmstub**: Note that this linkage style has the potential for naming conflicts, since the primitive exported function and variable names are not wrapped in a guile-module and might conflict with names from the GOOPS guile-module (see above). Pass the -goopsprefix argument to solve this problem. If the `-exportprimitive` option is passed to SWIG the `(export ...)` code that would be exported into the scmstub file is exported at the bottom of the generated GOOPS guile-module. The `%goops` directive should contain code to load the shared library.

  ```
  %goops %{ (load-extension "./libfoo.so" "scm_init_my_modules_foo_module") %}
  ```

Produces the following code at the top of the generated GOOPS guile-module (with the `-package my/modules -module foo` command line arguments)

```
(define-module (my modules foo))

;; %goops directive goes here
(load-extension "./libfoo.so" "scm_init_my_modules_foo_module")

(use-modules (oop goops) (Swig common))
```

- **Passive Linkage with -scmstub**: Here, the name of the scmstub file should be `Module-primitive.scm` (with *primitive* replaced with whatever is given with the `-primsuffix` argument. The code to load the shared library should be located in the `%scheme` directive, which will then be added to the scmstub file. SWIG will automatically generate the line `(use-modules ( Package Module-primitive))` into the GOOPS guile-module. So if *Module-primitive.scm* is on the autoload path for guile, the `%goops` directive can be empty. Otherwise, the `%goops` directive should contain whatever code is needed to load the *Module-primitive.scm* file into guile.

```
%scheme %{ (load-extension "./libfoo.so" "scm_init_my_modules_foo_module") %}
// only include the following definition if (my modules foo) cannot
// be loaded automatically
%goops %{
  (primitive-load "/path/to/foo-primitive.scm")
  (primitive-load "/path/to/Swig/common.scm")
%}
```

Produces the following code at the top of the generated GOOPS guile-module

```
(define-module (my modules foo))

;; %goops directive goes here (if any)
(primitive-load "/path/to/foo-primitive.scm")
(primitive-load "/path/to/Swig/common.scm")

(use-modules (oop goops) (Swig common))
(use-modules ((my modules foo-primitive) :renamer (symbol-prefix-proc
                                                   'primitive:)))
```

- **Module Linkage**: This is very similar to passive linkage with a scmstub file. SWIG will also automatically generate the line `(use-modules (Package Module-primitive))` into the GOOPS guile-module. Again the `%goops` directive should contain whatever code is needed to get that module loaded into guile.

```
%goops %{ (load-extension "./libfoo.so" "scm_init_my_modules_foo_module") %}
```

Produces the following code at the top of the generated GOOPS guile-module

```
(define-module (my modules foo))

;; %goops directive goes here (if any)
(load-extension "./libfoo.so" "scm_init_my_modules_foo_module")

(use-modules (oop goops) (Swig common))
(use-modules ((my modules foo-primitive) :renamer (symbol-prefix-proc
                                                     'primitive:)))
```

**(Swig common)**: The generated GOOPS guile-module also imports definitions from the (Swig common) guile-module. This module is included with SWIG and should be installed by SWIG into the autoload path for guile (based on the configure script and whatever arguments are passed). If it is not, then the `%goops` directive also needs to contain code to load the `common.scm` file into guile. Also note that if you are trying to install the generated wrappers on a computer without SWIG installed, you will need to include the common.swg file along with the install.

**Multiple Modules**: Type dependencies between modules is supported. For example, if `mod1` includes definitions of some classes, and `mod2` includes some classes derived from classes in `mod1`, the generated GOOPS file for `mod2` will declare the correct superclasses. The only problem is that since `mod2` uses symbols from `mod1`, the `mod2` GOOPS file must include a `(use-modules (mod2))`. Currently, SWIG does not automatically export this line; it must be included in the `%goops` directive of `mod2`. Maybe in the future SWIG can detect dependencies and export this line. (how do other language modules handle this problem?)

---

# 27 SWIG and Java


- Overview
- Preliminaries
  - Running SWIG
  - Additional Commandline Options
  - Getting the right header files
  - Compiling a dynamic module
  - Using your module
  - Dynamic linking problems
  - Compilation problems and compiling with C++
  - Building on Windows
    - Running SWIG from Visual Studio
    - Using NMAKE
- A tour of basic C/C++ wrapping
  - Modules, packages and generated Java classes
  - Functions
  - Global variables
  - Constants
  - Enumerations
    - Anonymous enums
    - Typesafe enums
    - Proper Java enums
    - Type unsafe enums
    - Simple enums
  - Pointers
  - Structures
  - C++ classes

- C++ inheritance
- Pointers, references, arrays and pass by value
  - Null pointers
- C++ overloaded functions
- C++ default arguments
- C++ namespaces
- C++ templates
- C++ Smart Pointers
  - The shared_ptr Smart Pointer
  - Generic Smart Pointers

- Further details on the generated Java classes
  - The intermediary JNI class
    - The intermediary JNI class pragmas
  - The Java module class
    - The Java module class pragmas
  - The Java constants interface
    - The Java constants interface pragmas
  - Java proxy classes
    - Memory management
    - Inheritance
    - Proxy classes and garbage collection
    - The premature garbage collection prevention parameter for proxy class marshalling
    - Single threaded applications and thread safety
  - Type wrapper classes
  - Enum classes
    - Typesafe enum classes
    - Proper Java enum classes
    - Type unsafe enum classes
  - Interfaces
- Cross language polymorphism using directors
  - Enabling directors
  - Director classes
  - Overhead and code bloat
  - Simple directors example
  - Director threading issues
  - Director performance tuning
  - Java exceptions from directors
    - Customizing director exceptions
  - Accessing virtual protected methods
  - Accessing non-virtual protected members
- Common customization features
  - C/C++ helper functions
  - Class extension with %extend
  - Class extension with %proxycode
  - Exception handling with %exception and %javaexception
  - Method access with %javamethodmodifiers
  - Java begin
- Tips and techniques
  - Input and output parameters using primitive pointers and references
  - Simple pointers
  - Wrapping C arrays with Java arrays
  - Unbounded C Arrays
  - Passing a string with length
  - Overriding new and delete to allocate from Java heap
- Java typemaps
  - Default primitive type mappings
  - Default typemaps for non-primitive types
  - Sixty four bit JVMs
  - What is a typemap?
  - Typemaps for mapping C/C++ types to Java types
  - Java typemap attributes
  - Java special variables
  - Typemaps for both C and C++ compilation
  - Java code typemaps
  - Director specific typemaps
- Typemap Examples
  - Simpler Java enums for enums without initializers
  - Handling C++ exception specifications as Java exceptions
  - NaN Exception - exception handling for a particular type
  - Converting Java String arrays to char **
  - Expanding a Java object to multiple arguments
  - Using typemaps to return arguments
  - Adding Java downcasts to polymorphic return types
  - Adding an equals method to the Java classes
  - Void pointers and a common Java base class
  - Struct pointer to pointer
  - Memory management when returning references to member variables
  - Memory management for objects passed to the C++ layer
  - Date marshalling using the javain typemap and associated attributes
- Living with Java Directors
- Odds and ends
  - JavaDoc comments
  - Functional interface without proxy classes
  - Using your own JNI functions
  - Performance concerns and hints
  - Debugging
- Java Examples


This chapter describes SWIG's support of Java. It covers most SWIG features, but certain low-level details are covered in less depth than in earlier chapters.

## 27.1 Overview

The 100% Pure Java effort is a commendable concept, however in the real world programmers often either need to re-use their existing code or in some situations want to take advantage of Java but are forced into using some native (C/C++) code. The Java extension to SWIG makes it very easy to plumb in existing C/C++ code for access from Java, as SWIG writes the Java Native Interface (JNI) code for you. It is different to using the 'javah' tool as SWIG will wrap existing C/C++ code, whereas javah takes 'native' Java function declarations and creates C/C++ function prototypes. SWIG wraps C/C++ code using Java proxy classes and is very useful if you want to have access to large amounts of C/C++ code from Java. If only one or two JNI functions are needed then using SWIG may be overkill. SWIG enables a Java program to easily call into C/C++ code from Java. Historically, SWIG was not able to generate any code to call into Java code from C++. However, SWIG now supports full cross language polymorphism and code is generated to call up from C++ to Java when wrapping C++ virtual methods via the director feature.

Java is one of the few non-scripting language modules in SWIG. As SWIG utilizes the type safety that the Java language offers, it takes a somewhat different approach to that used for scripting languages. In particular runtime type checking and the runtime library are not used by Java. This should be borne in mind when reading the rest of the SWIG documentation. This chapter on Java is relatively self contained and will provide you with nearly everything you need for using SWIG and Java. However, the "SWIG Basics" chapter will be a useful read in conjunction with this one.

This chapter starts with a few practicalities on running SWIG and compiling the generated code. If you are looking for the minimum amount to read, have a look at the sections up to and including the tour of basic C/C++ wrapping section which explains how to call the various C/C++ code constructs from Java. Following this section are details of the C/C++ code and Java classes that SWIG generates. Due to the complexities of C and C++ there are different ways in which C/C++ code could be wrapped and called from Java. SWIG is a powerful tool and the rest of the chapter details how the default code wrapping can be tailored. Various customisation tips and techniques using SWIG directives are covered. The latter sections cover the advanced techniques of using typemaps for complete control of the wrapping process.

## 27.2 Preliminaries

SWIG 1.1 works with JDKs from JDK 1.1 to JDK1.4 (Java 2 SDK1.4) and should also work with any later versions. Given the choice, you should probably use the latest version of Sun's JDK. The SWIG Java module is known to work using Sun's JVM on Solaris, Linux and the various flavours of Microsoft Windows including Cygwin. The Kaffe JVM is known to give a few problems and at the time of writing was not a fully fledged JVM with full JNI support. The generated code is also known to work on vxWorks using WindRiver's PJava 3.1. The best way to determine whether your combination of operating system and JDK will work is to test the examples and test-suite that comes with SWIG. Run `make -k check` from the SWIG root directory after installing SWIG on Unix systems.

The Java module requires your system to support shared libraries and dynamic loading. This is the commonly used method to load JNI code so your system will more than likely support this.

Android uses Java JNI and also works with SWIG. Please read the Android chapter in conjunction with this one if you are targeting Android.

### 27.2.1 Running SWIG

Suppose that you defined a SWIG module such as the following:

```
/* File: example.i */
%module test
%{
#include "stuff.h"
%}
int fact(int n);
```

To build a Java module, run SWIG using the `-java` option :

```
%swig -java example.i
```

If building C++, add the `-c++` option:

```
$ swig -c++ -java example.i
```

This creates two different files; a C/C++ source file `example_wrap.c` or `example_wrap.cxx` and numerous Java files. The generated C/C++ source file contains the JNI wrapper code that needs to be compiled and linked with the rest of your C/C++ application.

The name of the wrapper file is derived from the name of the input file. For example, if the input file is `example.i`, the name of the wrapper file is `example_wrap.c`. To change this, you can use the `-o` option. It is also possible to change theoutput directory that the Java files are generated into using`-outdir`.

The module name, specified with `%module`, determines the name of various generated classes as discussed later. Note that the module name does not define a Java package and by default, the generated Java classes do not have a Java package. The `-package` option described below can specify a Java package name to use.

The following sections have further practical examples and details on how you might go about compiling and using the generated files.

### 27.2.2 Additional Commandline Options

The following table lists the additional commandline options available for the Java module. They can also be seen by using:

```
swig -java -help
```

**Java specific options**

| | |
|---|---|
| -nopgcpp | suppress the premature garbage collection prevention parameter |
| -noproxy | generate the low-level functional interface instead of proxy classes |
| -package <name> | set name of the Java package to <name> |

Their use will become clearer by the time you have finished reading this section on SWIG and Java.

### 27.2.3 Getting the right header files

In order to compile the C/C++ wrappers, the compiler needs the `jni.h` and `jni_md.h` header files which are part of the JDK. They are usually in directories like this:

```
/usr/java/include
/usr/java/include/<operating_system>
```

The exact location may vary on your machine, but the above locations are typical.

### 27.2.4 Compiling a dynamic module

The JNI code exists in a dynamic module or shared library (DLL on Windows) and gets loaded by the JVM. Assuming you have code you need to link to in a file called `example.c`, in order to build a shared library file, you need to compile your module in a manner similar to the following (shown for Solaris):

```
$ swig -java example.i
$ gcc -fPIC -c example_wrap.c -I/usr/java/include -I/usr/java/include/solaris
$ gcc -fPIC -c example.c
$ ld -G example_wrap.o example.o -o libexample.so
```

The exact commands for doing this vary from platform to platform. However, SWIG tries to guess the right options when it is installed. Therefore, you may want to start with one of the examples in the `Examples/java` directory. If that doesn't work, you will need to read the man-pages for your compiler and linker to get the right set of options. You might also check the SWIG Wiki for additional information.

**Important**

If you are going to use optimisations turned on with gcc (for example -O2), ensure you also compile with -fno-strict-aliasing. The GCC optimisations have become more aggressive from gcc-4.0 onwards and will result in code that fails with strict aliasing optimisations turned on. See the C/C++ to Java typemaps section for more details.

The name of the shared library output file is important. If the name of your SWIG module is "example", the name of the corresponding shared library file should be "libexample.so" (or equivalent depending on your machine, see Dynamic linking problems for more information). The name of the module is specified using the %module directive or -module command line option.

### 27.2.5 Using your module

To load your shared native library module in Java, simply use Java's System.loadLibrary method in a Java class:

```
// runme.java

public class runme {
  static {
    System.loadLibrary("example");
  }

  public static void main(String argv[]) {
    System.out.println(example.fact(4));
  }
}
```

Compile all the Java files and run:

```
$ javac *.java
$ java runme
24
$
```

If it doesn't work have a look at the following section which discusses problems loading the shared library.

### 27.2.6 Dynamic linking problems

As shown in the previous section the code to load a native library (shared library) is System.loadLibrary("name"). This can fail with an UnsatisfiedLinkError exception and can be due to a number of reasons.

You may get an exception similar to this:

```
$ java runme
Exception in thread "main" java.lang.UnsatisfiedLinkError: no example in java.library.path
        at java.lang.ClassLoader.loadLibrary(ClassLoader.java:1312)
        at java.lang.Runtime.loadLibrary0(Runtime.java:749)
        at java.lang.System.loadLibrary(System.java:820)
        at runme.<clinit>(runme.java:5)
```

The most common cause for this is an incorrect naming of the native library for the name passed to the loadLibrary function. The string passed to the loadLibrary function must not include the file extension name in the string, that is *.dll* or *.so*. The string must be *name* and not *libname* for all platforms. On Windows the native library must then be called *name.dll* and on most Unix systems it must be called *libname.so* .

Another common reason for the native library not loading is because it is not in your path. On Windows make sure the *path* environment variable contains the path to the native library. On Unix make sure that your *LD_LIBRARY_PATH* contains the path to the native library. Adding paths to *LD_LIBRARY_PATH* can slow down other programs on your system so you may want to consider alternative approaches. For example you could recompile your native library with extra path information using -rpath if you're using GNU, see the GNU linker documentation (ld man page). You could use a command such as ldconfig (Linux) or crle (Solaris) to add additional search paths to the default system configuration (this requires root access and you will need to read the man pages).

The native library will also not load if there are any unresolved symbols in the compiled C/C++ code. The following exception is indicative of this:

```
$ java runme
Exception in thread "main" java.lang.UnsatisfiedLinkError: libexample.so: undefined
symbol: fact
        at java.lang.ClassLoader$NativeLibrary.load(Native Method)
        at java.lang.ClassLoader.loadLibrary0(ClassLoader.java, Compiled Code)
        at java.lang.ClassLoader.loadLibrary(ClassLoader.java, Compiled Code)
        at java.lang.Runtime.loadLibrary0(Runtime.java, Compiled Code)
        at java.lang.System.loadLibrary(System.java, Compiled Code)
        at runme.<clinit>(runme.java:5)
$
```

This error usually indicates that you forgot to include some object files or libraries in the linking of the native library file. Make sure you compile both the SWIG wrapper file and the code you are wrapping into the native library file. If you forget to compile and link in the SWIG wrapper file into your native library file, you will get a message similar to the following:

```
$ java runme
Exception in thread "main" java.lang.UnsatisfiedLinkError: exampleJNI.gcd(II)I
        at exampleJNI.gcd(Native Method)
        at example.gcd(example.java:12)
        at runme.main(runme.java:18)
```

where gcd is the missing JNI function that SWIG generated into the wrapper file. Also make sure you pass all of the required libraries to the linker. The java -verbose:jni commandline option is also a great way to get more information on unresolved symbols. One last piece of advice is to beware of the common faux pas of having more than one native library version in your path.

In summary, ensure that you are using the correct C/C++ compiler and linker combination and options for successful native library loading. If you are using the examples that ship with SWIG, then the Examples/Makefile must have these set up correctly for your system. The SWIG installation package makes a best attempt at getting these correct but does not get it right 100% of the time. The SWIG Wiki also has some settings for commonly used compiler and operating system combinations. The following section also contains some C++ specific linking problems and solutions.

### 27.2.7 Compilation problems and compiling with C++

On most machines, shared library files should be linked using the C++ compiler. For example:

```
% swig -c++ -java example.i
```

```
% g++ -c -fPIC example.cxx
% g++ -c -fPIC example_wrap.cxx -I/usr/java/j2sdk1.4.1/include -I/usr/java/j2sdk1.4.1/include/linux
% g++ -shared example.o example_wrap.o -o libexample.so
```

In addition to this, you may need to include additional library files to make it work. For example, if you are using the Sun C++ compiler on Solaris, you often need to add an extra library `-lCrun` like this:

```
% swig -c++ -java example.i
% CC -c example.cxx
% CC -c example_wrap.cxx -I/usr/java/include -I/usr/java/include/solaris
% CC -G example.o example_wrap.o -L/opt/SUNWspro/lib -o libexample.so -lCrun
```

If you aren't entirely sure about the linking for C++, you might look at an existing C++ program. On many Unix machines, the `ldd` command will list library dependencies. This should give you some clues about what you might have to include when you link your shared library. For example:

```
$ ldd swig
        libstdc++-libc6.1-1.so.2 => /usr/lib/libstdc++-libc6.1-1.so.2 (0x40019000)
        libm.so.6 => /lib/libm.so.6 (0x4005b000)
        libc.so.6 => /lib/libc.so.6 (0x40077000)
        /lib/ld-linux.so.2 => /lib/ld-linux.so.2 (0x40000000)
$
```

Finally make sure the version of JDK header files matches the version of Java that you are running as incompatibilities could lead to compilation problems or unpredictable behaviour.

### 27.2.8 Building on Windows

Building on Windows is roughly similar to the process used with Unix. You will want to produce a DLL that can be loaded by the Java Virtual Machine. This section covers the process of using SWIG with Microsoft Visual C++ 6 although the procedure may be similar with other compilers. In order for everything to work, you will need to have a JDK installed on your machine in order to read the JNI header files.

#### 27.2.8.1 Running SWIG from Visual Studio

If you are developing your application within Microsoft Visual studio, SWIG can be invoked as a custom build option. The Examples\java directory has a few Windows Examples containing Visual Studio project (.dsp) files. The process to re-create the project files for a C project are roughly:

- Open up a new workspace and use the AppWizard to select a DLL project.
- Add both the SWIG interface file (the .i file), any supporting C files, and the name of the wrapper file that will be created by SWIG (ie. `example_wrap.c`). Don't worry if the wrapper file doesn't exist yet--Visual Studio will keep a reference to it.
- Select the SWIG interface file and go to the settings menu. Under settings, select the "Custom Build" option.
- Enter "SWIG" in the description field.
- Enter "`swig -java -o $(ProjDir)\$(InputName)_wrap.c $(InputPath)` " in the "Build command(s) field"
- Enter "`$(ProjDir)\$(InputName)_wrap.c`" in the "Output files(s) field".
- Next, select the settings for the entire project and go to C/C++ tab and select the Preprocessor category. Add the include directories to the JNI header files under "Additional include directories", eg "C:\jdk1.3\include, C:\jdk1.3\include\win32".
- Next, select the settings for the entire project and go to Link tab and select the General category. Set the name of the output file to match the name of your Java module (ie. example.dll).
- Next, select the example.c and example_wrap.c files and go to the C/C++ tab and select the Precompiled Headers tab in the project settings. Disabling precompiled headers for these files will overcome any precompiled header errors while building.
- Finally, add the java compilation as a post build rule in the Post-build step tab in project settings, eg, "c:\jdk1.3\bin\javac *.java"
- Build your project.

Note: If using C++, choose a C++ suffix for the wrapper file, for example `example_wrap.cxx`. Use `_wrap.cxx` instead of `_wrap.c` in the instructions above and add -c++ when invoking swig.

Now, assuming all went well, SWIG will be automatically invoked when you build your project. When doing a build, any changes made to the interface file will result in SWIG being automatically invoked to produce a new version of the wrapper file.

The Java classes that SWIG output should also be compiled into .class files. To run the native code in the DLL (example.dll), make sure that it is in your path then run your Java program which uses it, as described in the previous section. If the library fails to load have a look at Dynamic linking problems.

#### 27.2.8.2 Using NMAKE

Alternatively, a Makefile for use by NMAKE can be written. Make sure the environment variables for MSVC++ are available and the MSVC++ tools are in your path. Now, just write a short Makefile like this :

```
# Makefile for using SWIG and Java for C code

SRCS          = example.c
IFILE         = example
INTERFACE     = $(IFILE).i
WRAPFILE      = $(IFILE)_wrap.c

# Location of the Visual C++ tools (32 bit assumed)

TOOLS         = c:\msdev
TARGET        = example.dll
CC            = $(TOOLS)\bin\cl.exe
LINK          = $(TOOLS)\bin\link.exe
INCLUDE32     = -I$(TOOLS)\include
MACHINE       = IX86

# C Library needed to build a DLL

DLLIBC        = msvcrt.lib oldnames.lib

# Windows libraries that are apparently needed
WINLIB        = kernel32.lib advapi32.lib user32.lib gdi32.lib comdlg32.lib winspool.lib

# Libraries common to all DLLs
LIBS          = $(DLLIBC) $(WINLIB)

# Linker options
LOPT      = -debug:full -debugtype:cv /NODEFAULTLIB /RELEASE /NOLOGO \
            /MACHINE:$(MACHINE) -entry:_DllMainCRTStartup@12 -dll

# C compiler flags
```

```
CFLAGS        = /Z7 /Od /c /nologo
JAVA_INCLUDE    = -ID:\jdk1.3\include -ID:\jdk1.3\include\win32

java::
        swig -java -o $(WRAPFILE) $(INTERFACE)
        $(CC) $(CFLAGS) $(JAVA_INCLUDE) $(SRCS) $(WRAPFILE)
        set LIB=$(TOOLS)\lib
        $(LINK) $(LOPT) -out:example.dll $(LIBS) example.obj example_wrap.obj
        javac *.java
```

To build the DLL and compile the java code, run NMAKE (you may need to run `vcvars32` first). This is a pretty simplistic Makefile, but hopefully it's enough to get you started. Of course you may want to make changes for it to work for C++ by adding in the -c++ command line option for swig and replacing .c with .cxx.

## 27.3 A tour of basic C/C++ wrapping

By default, SWIG attempts to build a natural Java interface to your C/C++ code. Functions are wrapped as functions, classes are wrapped as classes, variables are wrapped with JavaBean type getters and setters and so forth. This section briefly covers the essential aspects of this wrapping.

### 27.3.1 Modules, packages and generated Java classes

The SWIG `%module` directive specifies the name of the Java module. When you specify `%module example', the *module name* determines the name of some of the generated files in the module. The generated code consists of a *module class* file `example.java` , an *intermediary JNI class* file, `exampleJNI.java` as well as numerous other Java *proxy class* files. Each proxy class is named after the structs, unions and classes you are wrapping. You may also get a *constants interface* file if you are wrapping any unnamed enumerations or constants, for example `exampleConstants.java`. When choosing a module name, make sure you don't use the same name as one of the generated proxy class files nor a Java keyword. Sometimes a C/C++ type cannot be wrapped by a proxy class, for example a pointer to a primitive type. In these situations a *type wrapper class* is generated. Wrapping an enum generates an *enum class*, either a proper Java enum or a Java class that simulates the enums pattern. Details of all these generated classes will unfold as you read this section.

The JNI (C/C++) code is generated into a file which also contains the module name, for example `example_wrap.cxx` or `example_wrap.c`. These C or C++ files complete the contents of the module.

The generated Java classes can be placed into a Java package by using the `-package` commandline option. This is often combined with the `-outdir` to specify a package directory for generating the Java files.

```
swig -java -package com.bloggs.swig -outdir com/bloggs/swig example.i
```

SWIG won't create the directory, so make sure it exists beforehand.

### 27.3.2 Functions

There is no such thing as a global Java function so global C functions are wrapped as static methods in the module class. For example,

```
%module example
int fact(int n);
```

creates a static function that works exactly like you think it might:

```
public class example {
  public static int fact(int n) {
    // makes call using JNI to the C function
  }
}
```

The Java class `example` is the *module class*. The function can be used as follows from Java:

```
System.out.println(example.fact(4));
```

### 27.3.3 Global variables

C/C++ global variables are fully supported by SWIG. Java does not allow the overriding of the dot operator so all variables are accessed through getters and setters. Again because there is no such thing as a Java global variable, access to C/C++ global variables is done through static getter and setter functions in the module class.

```
// SWIG interface file with global variables
%module example
...
%inline %{
extern int My_variable;
extern double density;
%}
...
```

Now in Java :

```
// Print out value of a C global variable
System.out.println("My_variable = " + example.getMy_variable());
// Set the value of a C global variable
example.setDensity(0.8442);
```

The value returned by the getter will always be up to date even if the value is changed in C. Note that the getters and setters produced follow the JavaBean property design pattern. That is the first letter of the variable name is capitalized and preceded with set or get. If you have the misfortune of wrapping two variables that differ only in the capitalization of their first letters, use %rename to change one of the variable names. For example:

```
%rename Clash RenamedClash;
float Clash;
int clash;
```

If a variable is declared as `const`, it is wrapped as a read-only variable. That is only a getter is produced.

To make ordinary variables read-only, you can use the `%immutable` directive. For example:

```
%{
extern char *path;
%}
%immutable;
extern char *path;
%mutable;
```

The `%immutable` directive stays in effect until it is explicitly disabled or cleared using `%mutable`. See the Creating read-only variables section for further details.

If you just want to make a specific variable immutable, supply a declaration name. For example:

```
%{
extern char *path;
%}
%immutable path;
...
extern char *path;      // Read-only (due to %immutable)
```

### 27.3.4 Constants

C/C++ constants are wrapped as Java static final variables. To create a constant, use `#define` or the `%constant` directive. For example:

```
#define PI 3.14159
#define VERSION "1.0"
%constant int FOO = 42;
%constant const char *path = "/usr/local";
```

By default the generated static final variables are initialized by making a JNI call to get their value. The constants are generated into the constants interface and look like this:

```
public interface exampleConstants {
  public final static double PI = exampleJNI.PI_get();
  public final static String VERSION = exampleJNI.VERSION_get();
  public final static int FOO = exampleJNI.FOO_get();
  public final static String path = exampleJNI.path_get();
}
```

Note that SWIG has inferred the C type and used an appropriate Java type that will fit the range of all possible values for the C type. By default SWIG generates **runtime constants**. They are not **compiler constants** that can, for example, be used in a switch statement. This can be changed by using the `%javaconst(flag)` directive. It works like all the other %feature directives. The default is `%javaconst(0)`. It is possible to initialize all wrapped constants from pure Java code by placing a `%javaconst(1)` **before** SWIG parses the constants. Putting it at the top of your interface file would ensure this. Here is an example:

```
%javaconst(1);
%javaconst(0) BIG;
%javaconst(0) LARGE;

#define EXPRESSION (0x100+5)
#define BIG 1000LL
#define LARGE 2000ULL
```

generates:

```
public interface exampleConstants {
  public final static int EXPRESSION = (0x100+5);
  public final static long BIG = exampleJNI.BIG_get();
  public final static java.math.BigInteger LARGE = exampleJNI.LARGE_get();
}
```

Note that SWIG has inferred the C `long long` type from `BIG` and used an appropriate Java type (`long`) as a Java `long` is the smallest sized Java type that will take all possible values for a C `long long`. Similarly for `LARGE`.

Be careful using the `%javaconst(1)` directive as not all C code will compile as Java code. For example neither the `1000LL` value for `BIG` nor `2000ULL` for `LARGE` above would generate valid Java code. The example demonstrates how you can target particular constants (`BIG` and `LARGE`) with `%javaconst`. SWIG doesn't use `%javaconst(1)` as the default as it tries to generate code that will always compile. However, using a `%javaconst(1)` at the top of your interface file is strongly recommended as the preferred compile time constants will be generated and most C constants will compile as Java code and in any case the odd constant that doesn't can be fixed using `%javaconst(0)`.

There is an alternative directive which can be used for these rare constant values that won't compile as Java code. This is the `%javaconstvalue(value)` directive, where `value` is a Java code replacement for the C constant and can be either a string or a number. This is useful if you do not want to use either the parsed C value nor a JNI call, such as when the C parsed value will not compile as Java code and a compile time constant is required. The same example demonstrates this:

```
%javaconst(1);
%javaconstvalue("new java.math.BigInteger(\"2000\")") LARGE;
%javaconstvalue(1000) BIG;

#define EXPRESSION (0x100+5)
#define BIG 1000LL
#define LARGE 2000ULL
```

Note the string quotes for `"2000"` are escaped. The following is then generated:

```
public interface exampleConstants {
  public final static int EXPRESSION = (0x100+5);
  public final static long BIG = 1000;
  public final static java.math.BigInteger LARGE = new java.math.BigInteger("2000");
}
```

Note: declarations declared as `const` are wrapped as read-only variables and will be accessed using a getter as described in the previous section. They are not wrapped as constants. The

exception to this rule are static const integral values defined within a class/struct, where they are wrapped as constants, eg:.

```
struct Maths {
  static const int FIVE = 5;
};
```

**Compatibility Note:** In SWIG-1.3.19 and earlier releases, the constants were generated into the module class and the constants interface didn't exist. Backwards compatibility is maintained as the module class implements the constants interface (even though some consider this type of interface implementation to be bad practice):

```
public class example implements exampleConstants {
}
```

You thus have the choice of accessing these constants from either the module class or the constants interface, for example, `example.EXPRESSION` or `exampleConstants.EXPRESSION`. Or if you decide this practice isn't so bad and your own class implements `exampleConstants`, you can of course just use `EXPRESSION`.

### 27.3.5 Enumerations

SWIG handles both named and unnamed (anonymous) enumerations. There is a choice of approaches to wrapping named C/C++ enums. This is due to historical reasons as SWIG's initial support for enums was limited and Java did not originally have support for enums. Each approach has advantages and disadvantages and it is important for the user to decide which is the most appropriate solution. There are four approaches of which the first is the default approach based on the so called Java typesafe enum pattern. The second generates proper Java enums. The final two approaches use simple integers for each enum item. Before looking at the various approaches for wrapping named C/C++ enums, anonymous enums are considered.

#### 27.3.5.1 Anonymous enums

There is no name for anonymous enums and so they are handled like constants. For example:

```
enum { ALE, LAGER=10, STOUT, PILSNER, PILZ=PILSNER };
```

is wrapped into the constants interface, in a similar manner as constants (see previous section):

```
public interface exampleConstants {
  public final static int ALE = exampleJNI.ALE_get();
  public final static int LAGER = exampleJNI.LAGER_get();
  public final static int STOUT = exampleJNI.STOUT_get();
  public final static int PILSNER = exampleJNI.PILSNER_get();
  public final static int PILZ = exampleJNI.PILZ_get();
}
```

The `%javaconst(flag)` and `%javaconstvalue(value)` directive introduced in the previous section on constants can also be used with enums. As is the case for constants, the default is `%javaconst(0)` as not all C values will compile as Java code. However, it is strongly recommended to add in a `%javaconst(1)` directive at the top of your interface file as it is only on very rare occasions that this will produce code that won't compile under Java. Using `%javaconst(1)` will ensure compile time constants are generated, thereby allowing the enum values to be used in Java switch statements. Example usage:

```
%javaconst(1);
%javaconst(0) PILSNER;
enum { ALE, LAGER=10, STOUT, PILSNER, PILZ=PILSNER };
```

generates:

```
public interface exampleConstants {
  public final static int ALE = 0;
  public final static int LAGER = 10;
  public final static int STOUT = LAGER + 1;
  public final static int PILSNER = exampleJNI.PILSNER_get();
  public final static int PILZ = PILSNER;
}
```

As in the case of constants, you can access them through either the module class or the constants interface, for example, `example.ALE` or `exampleConstants.ALE`.

#### 27.3.5.2 Typesafe enums

This is the default approach to wrapping named enums. The typesafe enum pattern is a relatively well known construct to work around the lack of enums in versions of Java prior to JDK 1.5. It basically defines a class for the enumeration and permits a limited number of final static instances of the class. Each instance equates to an enum item within the enumeration. The implementation is in the "enumtypesafe.swg" file. Let's look at an example:

```
%include "enumtypesafe.swg" // optional as typesafe enums are the default
enum Beverage { ALE, LAGER=10, STOUT, PILSNER, PILZ=PILSNER };
```

will generate:

```
public final class Beverage {
  public final static Beverage ALE = new Beverage("ALE");
  public final static Beverage LAGER = new Beverage("LAGER", exampleJNI.LAGER_get());
  public final static Beverage STOUT = new Beverage("STOUT");
  public final static Beverage PILSNER = new Beverage("PILSNER");
  public final static Beverage PILZ = new Beverage("PILZ", exampleJNI.PILZ_get());
  [... additional support methods omitted for brevity ...]
}
```

See Typesafe enum classes to see the omitted support methods. Note that the enum item with an initializer (LAGER) is initialized with the enum value obtained via a JNI call. However, as with anonymous enums and constants, use of the `%javaconst` directive is strongly recommended to change this behaviour:

```
%include "enumtypesafe.swg" // optional as typesafe enums are the default
%javaconst(1);
enum Beverage { ALE, LAGER=10, STOUT, PILSNER, PILZ=PILSNER };
```

will generate:

```
public final class Beverage {
  public final static Beverage ALE = new Beverage("ALE");
  public final static Beverage LAGER = new Beverage("LAGER", 10);
  public final static Beverage STOUT = new Beverage("STOUT");
  public final static Beverage PILSNER = new Beverage("PILSNER");
  public final static Beverage PILZ = new Beverage("PILZ", PILSNER);
  [... additional support methods omitted for brevity ...]
}
```

The generated code is easier to read and more efficient as a true constant is used instead of a JNI call. As is the case for constants, the default is `%javaconst(0)` as not all C values will compile as Java code. However, it is recommended to add in a `%javaconst(1)` directive at the top of your interface file as it is only on very rare occasions that this will produce code that won't compile under Java. The `%javaconstvalue(value)` directive can also be used for typesafe enums. Note that global enums are generated into a Java class within whatever package you are using. C++ enums defined within a C++ class are generated into a static final inner Java class within the Java proxy class.

Typesafe enums have their advantages over using plain integers in that they can be used in a typesafe manner. However, there are limitations. For example, they cannot be used in switch statements and serialization is an issue. Please look at the following references for further information: Replace Enums with Classes in *Effective Java Programming* on the Sun website, Create enumerated constants in Java JavaWorld article, Java Tip 133: More on typesafe enums and Java Tip 122: Beware of Java typesafe enumerations JavaWorld tips.

Note that the syntax required for using typesafe enums is the same as that for proper Java enums. This is useful during the period that a project has to support legacy versions of Java. When upgrading to JDK 1.5 or later, proper Java enums could be used instead, without users having to change their code. The following section details proper Java enum generation.

#### 27.3.5.3 Proper Java enums

Proper Java enums were only introduced in JDK 1.5 so this approach is only compatible with more recent versions of Java. Java enums have been designed to overcome all the limitations of both typesafe and type unsafe enums and should be the choice solution, provided older versions of Java do not have to be supported. In this approach, each named C/C++ enum is wrapped by a Java enum. Java enums, by default, do not support enums with initializers. Java enums are in many respects similar to Java classes in that they can be customised with additional methods. SWIG takes advantage of this feature to facilitate wrapping C/C++ enums that have initializers. In order to wrap all possible C/C++ enums using proper Java enums, the "enums.swg" file must be used. Let's take a look at an example.

```
%include "enums.swg"
%javaconst(1);
enum Beverage { ALE, LAGER=10, STOUT, PILSNER, PILZ=PILSNER };
```

will generate:

```
public enum Beverage {
  ALE,
  LAGER(10),
  STOUT,
  PILSNER,
  PILZ(PILSNER);
  [... additional support methods omitted for brevity ...]
}
```

See Proper Java enum classes to see the omitted support methods. The generated Java enum has numerous additional methods to support enums with initializers, such as `LAGER` above. Note that as with the typesafe enum pattern, enum items with initializers are by default initialized with the enum value obtained via a JNI call. However, this is not the case above as we have used the recommended `%javaconst(1)` to avoid the JNI call. The `%javaconstvalue(value)` directive covered in the Constants section can also be used for proper Java enums.

The additional support methods need not be generated if none of the enum items have initializers and this is covered later in the Simpler Java enums for enums without initializers section.

#### 27.3.5.4 Type unsafe enums

In this approach each enum item in a named enumeration is wrapped as a static final integer in a class named after the C/C++ enum name. This is a commonly used pattern in Java to simulate C/C++ enums, but it is not typesafe. However, the main advantage over the typesafe enum pattern is enum items can be used in switch statements. In order to use this approach, the "enumtypeunsafe.swg" file must be used. Let's take a look at an example.

```
%include "enumtypeunsafe.swg"
%javaconst(1);
enum Beverage { ALE, LAGER=10, STOUT, PILSNER, PILZ=PILSNER };
```

will generate:

```
public final class Beverage {
  public final static int ALE = 0;
  public final static int LAGER = 10;
  public final static int STOUT = LAGER + 1;
  public final static int PILSNER = STOUT + 1;
  public final static int PILZ = PILSNER;
}
```

As is the case previously, the default is `%javaconst(0)` as not all C/C++ values will compile as Java code. However, again it is recommended to add in a `%javaconst(1)` directive. and the `%javaconstvalue(value)` directive covered in the Constants section can also be used for type unsafe enums. Note that global enums are generated into a Java class within whatever package you are using. C++ enums defined within a C++ class are generated into a static final inner Java class within the Java proxy class.

Note that unlike typesafe enums, this approach requires users to mostly use different syntax compared with proper Java enums. Thus the upgrade path to proper enums provided in JDK 1.5 is more painful.

#### 27.3.5.5 Simple enums

This approach is similar to the type unsafe approach. Each enum item is also wrapped as a static final integer. However, these integers are not generated into a class named after the C/C++ enum. Instead, global enums are generated into the constants interface. Also, enums defined in a C++ class have their enum items generated directly into the Java proxy class rather than an inner class within the Java proxy class. In fact, this approach is effectively wrapping the enums as if they were anonymous enums and the resulting code is as per anonymous enums. The implementation is in the "enumsimple.swg" file.

**Compatibility Note:** SWIG-1.3.21 and earlier versions wrapped all enums using this approach. The type unsafe approach is preferable to this one and this simple approach is only included for backwards compatibility with these earlier versions of SWIG.

### 27.3.6 Pointers

C/C++ pointers are fully supported by SWIG. Furthermore, SWIG has no problem working with incomplete type information. Here is a rather simple interface:

```
%module example

FILE *fopen(const char *filename, const char *mode);
int fputs(const char *, FILE *);
int fclose(FILE *);
```

When wrapped, you will be able to use the functions in a natural way from Java. For example:

```
SWIGTYPE_p_FILE f = example.fopen("junk", "w");
example.fputs("Hello World\n", f);
example.fclose(f);
```

C pointers in the Java module are stored in a Java `long` and cross the JNI boundary held within this 64 bit number. Many other SWIG language modules use an encoding of the pointer in a string. These scripting languages use the SWIG runtime type checker for dynamic type checking as they do not support static type checking by a compiler. In order to implement static type checking of pointers within Java, they are wrapped by a simple Java class. In the example above the `FILE *` pointer is wrapped with a *type wrapper class* called `SWIGTYPE_p_FILE`.

Once obtained, a type wrapper object can be freely passed around to different C functions that expect to receive an object of that type. The only thing you can't do is dereference the pointer from Java. Of course, that isn't much of a concern in this example.

As much as you might be inclined to modify a pointer value directly from Java, don't. The value is not necessarily the same as the logical memory address of the underlying object. The value will vary depending on the native byte-ordering of the platform (i.e., big-endian vs. little-endian). Most JVMs are 32 bit applications so any JNI code must also be compiled as 32 bit. The net result is pointers in JNI code are also 32 bits and are stored in the high order 4 bytes on big-endian machines and in the low order 4 bytes on little-endian machines. By design it is also not possible to manually cast a pointer to a new type by using Java casts as it is particularly dangerous especially when casting C++ objects. If you need to cast a pointer or change its value, consider writing some helper functions instead. For example:

```
%inline %{
/* C-style cast */
Bar *FooToBar(Foo *f) {
  return (Bar *) f;
}

/* C++-style cast */
Foo *BarToFoo(Bar *b) {
  return dynamic_cast<Foo*>(b);
}

Foo *IncrFoo(Foo *f, int i) {
  return f+i;
}
%}
```

Also, if working with C++, you should always try to use the new C++ style casts. For example, in the above code, the C-style cast may return a bogus result whereas as the C++-style cast will return a NULL pointer if the conversion can't be performed.

### 27.3.7 Structures

If you wrap a C structure, it is wrapped by a Java class with getters and setters for access to the member variables. For example,

```
struct Vector {
  double x, y, z;
};
```

is used as follows:

```
Vector v = new Vector();
v.setX(3.5);
v.setY(7.2);
double x = v.getX();
double y = v.getY();
```

The variable setters and getters are also based on the JavaBean design pattern already covered under the Global variables section. Similar access is provided for unions and the public data members of C++ classes.

This object is actually an instance of a Java class that has been wrapped around a pointer to the C structure. This instance doesn't actually do anything--it just serves as a proxy. The pointer to the C object is held in the Java proxy class in much the same way as pointers are held by type wrapper classes. Further details about Java proxy classes are covered a little later.

`const` members of a structure are read-only. Data members can also be forced to be read-only using the `%immutable` directive. For example:

```
struct Foo {
  ...
  %immutable;
  int x;        /* Read-only members */
  char *name;
  %mutable;
  ...
};
```

When `char *` members of a structure are wrapped, the contents are assumed to be dynamically allocated using `malloc` or `new` (depending on whether or not SWIG is run with the `-c++` option). When the structure member is set, the old contents will be released and a new value created. If this is not the behavior you want, you will have to use a typemap (described later).

If a structure contains arrays, access to those arrays is managed through pointers. For example, consider this:

```
struct Bar {
  int  x[16];
};
```

If accessed in Java, you will see behavior like this:

```
Bar b = new Bar();
```

```
SWIGTYPE_p_int x = b.getX();
```

This pointer can be passed around to functions that expect to receive an `int *` (just like C). You can also set the value of an array member using another pointer. For example:

```
Bar b = new Bar();
SWIGTYPE_p_int x = b.getX();
Bar c = new Bar();
c.setX(x);                    // Copy contents of b.x to c.x
```

For array assignment (setters not getters), SWIG copies the entire contents of the array starting with the data pointed to by `b.x` . In this example, 16 integers would be copied. Like C, SWIG makes no assumptions about bounds checking---if you pass a bad pointer, you may get a segmentation fault or access violation. The default wrapping makes it hard to set or get just one element of the array and so array access from Java is somewhat limited. This can be changed easily though by using the approach outlined later in the Wrapping C arrays with Java arrays and Unbounded C Arrays sections.

When a member of a structure is itself a structure, it is handled as a pointer. For example, suppose you have two structures like this:

```
struct Foo {
  int a;
};

struct Bar {
  Foo f;
};
```

Now, suppose that you access the `f` member of `Bar` like this:

```
Bar b = new Bar();
Foo x = b.getF();
```

In this case, `x` is a pointer that points to the `Foo` that is inside `b`. This is the same value as generated by this C code:

```
Bar b;
Foo *x = &b->f;       /* Points inside b */
```

Because the pointer points inside the structure, you can modify the contents and everything works just like you would expect. For example:

```
Bar b = new Bar();
b.getF().setA(3);   // Modify b.f.a
Foo x = b.getF();
x.setA(3);          // Modify x.a - this is the same as b.f.a
```

### 27.3.8 C++ classes

C++ classes are wrapped by Java classes as well. For example, if you have this class,

```
class List {
public:
  List();
  ~List();
  int  search(char *item);
  void insert(char *item);
  void remove(char *item);
  char *get(int n);
  int  length;
};
```

you can use it in Java like this:

```
List l = new List();
l.insert("Ale");
l.insert("Stout");
l.insert("Lager");
String item = l.get(2);
int length = l.getLength();
```

Class data members are accessed in the same manner as C structures.

Static class members are unsurprisingly wrapped as static members of the Java class:

```
class Spam {
public:
  static void foo();
  static int bar;
};
```

The static members work like any other Java static member:

```
Spam.foo();
int bar = Spam.getBar();
```

### 27.3.9 C++ inheritance

SWIG is fully aware of issues related to C++ inheritance. Therefore, if you have classes like this

```
class Foo {
...
};

class Bar : public Foo {
...
};
```

those classes are wrapped into a hierarchy of Java classes that reflect the same inheritance structure:

```
Bar b = new Bar();
Class c = b.getClass();
System.out.println(c.getSuperclass().getName());
```

will of course display:

```
Foo
```

Furthermore, if you have functions like this

```
void spam(Foo *f);
```

then the Java function `spam()` accepts instances of `Foo` or instances of any other proxy classes derived from `Foo`.

Note that Java does not support multiple inheritance so any multiple inheritance in the C++ code is not going to work. A warning is given when multiple inheritance is detected and only the first base class is used.

Private and protected methods are not wrapped as they are inaccessible outside of the class. Protected methods can be made accessible though via the directors feature, see the Accessing virtual protected methods in the directors section.

### 27.3.10 Pointers, references, arrays and pass by value

In C++, there are many different ways a function might receive and manipulate objects. For example:

```
void spam1(Foo *x);      // Pass by pointer
void spam2(Foo &x);      // Pass by reference
void spam3(Foo x);       // Pass by value
void spam4(Foo x[]);     // Array of objects
```

In Java, there is no detailed distinction like this--specifically, there are only instances of classes. There are no pointers nor references. Because of this, SWIG unifies all of these types together in the wrapper code. For instance, if you actually had the above functions, it is perfectly legal to do this from Java:

```
Foo f = new Foo();  // Create a Foo
example.spam1(f);   // Ok. Pointer
example.spam2(f);   // Ok. Reference
example.spam3(f);   // Ok. Value.
example.spam4(f);   // Ok. Array (1 element)
```

Similar behavior occurs for return values. For example, if you had functions like this,

```
Foo *spam5();
Foo &spam6();
Foo  spam7();
```

then all three functions will return a pointer to some `Foo` object. Since the third function (spam7) returns a value, newly allocated memory is used to hold the result and a pointer is returned (Java will release this memory when the returned object's finalizer is run by the garbage collector).

#### 27.3.10.1 Null pointers

Working with null pointers is easy. A Java `null` can be used whenever a method expects a proxy class or typewrapper class. However, it is not possible to pass null to C/C++ functions that take parameters by value or by reference. If you try you will get a NullPointerException.

```
example.spam1(null);   // Pointer - ok
example.spam2(null);   // Reference - NullPointerException
example.spam3(null);   // Value - NullPointerException
example.spam4(null);   // Array - ok
```

For `spam1` and `spam4` above the Java `null` gets translated into a NULL pointer for passing to the C/C++ function. The converse also occurs, that is, NULL pointers are translated into `null` Java objects when returned from a C/C++ function.

### 27.3.11 C++ overloaded functions

C++ overloaded functions, methods, and constructors are mostly supported by SWIG. For example, if you have two functions like this:

```
%module example

void foo(int);
void foo(char *c);
```

You can use them in Java in a straightforward manner:

```
example.foo(3);           // foo(int)
example.foo("Hello");     // foo(char *c)
```

Similarly, if you have a class like this,

```
class Foo {
public:
  Foo();
  Foo(const Foo &);
  ...
};
```

you can write Java code like this:

```
Foo f = new Foo();        // Create a Foo
Foo g = new Foo(f);       // Copy f
```

Overloading support is not quite as flexible as in C++. Sometimes there are methods that SWIG cannot disambiguate as there can be more than one C++ type mapping onto a single Java type. For example:

```
void spam(int);
void spam(unsigned short);
```

Here both int and unsigned short map onto a Java int. Here is another example:

```
void foo(Bar *b);
void foo(Bar &b);
```

If declarations such as these appear, you will get a warning message like this:

```
example.i:12: Warning 515: Overloaded method spam(unsigned short) ignored.
Method spam(int) at example.i:11 used.
```

To fix this, you either need to either rename or ignore one of the methods. For example:

```
%rename(spam_ushort) spam(unsigned short);
...
void spam(int);
void spam(unsigned short);   // Now renamed to spam_ushort
```

or

```
%ignore spam(unsigned short);
...
void spam(int);
void spam(unsigned short);   // Ignored
```

### 27.3.12 C++ default arguments

Any function with a default argument is wrapped by generating an additional function for each argument that is defaulted. For example, if we have the following C++:

```
%module example

void defaults(double d=10.0, int i=0);
```

The following methods are generated in the Java module class:

```
public class example {
  public static void defaults(double d, int i) { ... }
  public static void defaults(double d) { ... }
  public static void defaults() { ... }
}
```

It is as if SWIG had parsed three separate overloaded methods. The same approach is taken for static methods, constructors and member methods.

**Compatibility note:** Versions of SWIG prior to SWIG-1.3.23 wrapped these with a single wrapper method and so the default values could not be taken advantage of from Java. Further details on default arguments and how to restore this approach are given in the more general Default arguments section.

### 27.3.13 C++ namespaces

SWIG is aware of named C++ namespaces and they can be mapped to Java packages, however, the default wrapping flattens the namespaces, effectively ignoring them. So by default, the namespace names do not appear in the module nor do namespaces result in a module that is broken up into submodules or packages. For example, if you have a file like this,

```
%module example

namespace foo {
  int fact(int n);
  struct Vector {
    double x, y, z;
  };
};
```

it works in Java as follows:

```
int f = example.fact(3);
Vector v = new Vector();
v.setX(3.4);
```

```
double y = v.getY();
```

If your program has more than one namespace, name conflicts (if any) can be resolved using `%rename` For example:

```
%rename(Bar_spam) Bar::spam;

namespace Foo {
  int spam();
}

namespace Bar {
  int spam();
}
```

If you have more than one namespace and you want to keep their symbols separate, consider wrapping them as separate SWIG modules. Each SWIG module can be placed into a separate package.

The default behaviour described above can be improved via the nspace feature. Note that it only works for classes, structs, unions and enums declared within a named C++ namespace. When the nspace feature is used (either the `nspace` feature flag or `%nspacemove`), the C++ namespaces are converted into Java packages of the same name. Proxy classes are thus declared within a package and this proxy makes numerous calls to the JNI intermediary class which is declared in the unnamed package by default. As Java does not support types declared in a named package accessing types declared in an unnamed package, the `-package` commandline option described earlier generally should be used to provide a parent package. So if SWIG is run using the `-package com.myco` option, a wrapped class, `MyWorld::Material::Color`, can then be accessed as `com.myco.MyWorld.Material.Color`. If you don't specify a package, you will get the following warning:

```
example.i:16: Warning 826: The nspace feature is used on 'MyWorld::Material::Color' without -package. The generated code
may not compile as Java does not support types declared in a named package accessing types declared in an unnamed package.
```

If it is undesirable to have a single top level package, the nspace feature may be used without the `-package` commandline option (and the resulting warning ignored) if all of the types exposed using SWIG are placed in a package using the nspace feature and the 'jniclasspackage' pragma is used to specify a package for the JNI intermediary class.

If the resulting use of the nspace feature and hence packages results in a proxy class in one package deriving or using a proxy class from another package, you will need to open up the visibility for the pointer constructor and `getCPtr` method from the default 'protected' to 'public' with the `SWIG_JAVABODY_PROXY` macro. See Java code typemaps.

### 27.3.14 C++ templates

C++ templates don't present a huge problem for SWIG. However, in order to create wrappers, you have to tell SWIG to create wrappers for a particular template instantiation. To do this, you use the `%template` directive. For example:

```
%module example
%{
#include <utility>
%}

template<class T1, class T2>
struct pair {
  typedef T1 first_type;
  typedef T2 second_type;
  T1 first;
  T2 second;
  pair();
  pair(const T1&, const T2&);
  ~pair();
};

%template(pairii) pair<int, int>;
```

In Java:

```
pairii p = new pairii(3, 4);
int first = p.getFirst();
int second = p.getSecond();
```

Obviously, there is more to template wrapping than shown in this example. More details can be found in the SWIG and C++ chapter.

### 27.3.15 C++ Smart Pointers

#### 27.3.15.1 The shared_ptr Smart Pointer

The C++11 standard provides `std::shared_ptr` which was derived from the Boost implementation, `boost::shared_ptr`. Both of these are available for Java in the SWIG library and usage is outlined in the shared_ptr smart pointer library section.

#### 27.3.15.2 Generic Smart Pointers

In certain C++ programs, it is common to use classes that have been wrapped by so-called "smart pointers." Generally, this involves the use of a template class that implements `operator->()` like this:

```
template<class T> class SmartPtr {
  ...
  T *operator->();
  ...
}
```

Then, if you have a class like this,

```
class Foo {
public:
  int x;
  int bar();
};
```

A smart pointer would be used in C++ as follows:

```
SmartPtr<Foo> p = CreateFoo();   // Created somehow (not shown)
...
p->x = 3;                        // Foo::x
int y = p->bar();                // Foo::bar
```

To wrap this in Java, simply tell SWIG about the `SmartPtr` class and the low-level `Foo` object. Make sure you instantiate `SmartPtr` using `%template` if necessary. For example:

```
%module example
...
%template(SmartPtrFoo) SmartPtr<Foo>;
...
```

Now, in Java, everything should just "work":

```
SmartPtrFoo p = example.CreateFoo(); // Create a smart-pointer somehow
p.setX(3);                           // Foo::x
int y = p.bar();                     // Foo::bar
```

If you ever need to access the underlying pointer returned by `operator->()` itself, simply use the `__deref__()` method. For example:

```
Foo f = p.__deref__();               // Returns underlying Foo *
```

## 27.4 Further details on the generated Java classes

In the previous section, a high-level view of Java wrapping was presented. A key component of this wrapping is that structures and classes are wrapped by Java proxy classes and type wrapper classes are used in situations where no proxies are generated. This provides a very natural, type safe Java interface to the C/C++ code and fits in with the Java programming paradigm. However, a number of low-level details were omitted. This section provides a brief overview of how the proxy classes work and then covers the type wrapper classes. Finally enum classes are covered. First, the crucial intermediary JNI class is considered.

### 27.4.1 The intermediary JNI class

In the "SWIG basics" and "SWIG and C++" chapters, details of low-level structure and class wrapping are described. To summarize those chapters, if you have a global function and class like this

```
class Foo {
public:
  int x;
  int spam(int num, Foo* foo);
};
void egg(Foo* chips);
```

then SWIG transforms the class into a set of low-level procedural wrappers. These procedural wrappers essentially perform the equivalent of this C++ code:

```
Foo *new_Foo() {
  return new Foo();
}
void delete_Foo(Foo *f) {
  delete f;
}
int Foo_x_get(Foo *f) {
  return f->x;
}
void Foo_x_set(Foo *f, int value) {
  f->x = value;
}
int Foo_spam(Foo *f, int num, Foo* foo) {
  return f->spam(num, foo);
}
```

These procedural function names don't actually exist, but their functionality appears inside the generated JNI functions. The JNI functions have to follow a particular naming convention so the function names are actually:

```
SWIGEXPORT jlong JNICALL Java_exampleJNI_new_1Foo(JNIEnv *jenv, jclass jcls);
SWIGEXPORT void JNICALL Java_exampleJNI_delete_1Foo(JNIEnv *jenv, jclass jcls,
                                                    jlong jarg1);
SWIGEXPORT void JNICALL Java_exampleJNI_Foo_1x_1set(JNIEnv *jenv, jclass jcls,
                                                    jlong jarg1, jobject jarg1_, jint jarg2);
SWIGEXPORT jint JNICALL Java_exampleJNI_Foo_1x_1get(JNIEnv *jenv, jclass jcls,
                                                    jlong jarg1, jobject jarg1_);
SWIGEXPORT jint JNICALL Java_exampleJNI_Foo_1spam(JNIEnv *jenv, jclass jcls,
                                                  jlong jarg1, jobject jarg1_, jint jarg2,
                                                  jlong jarg3, jobject jarg3_);
SWIGEXPORT void JNICALL Java_exampleJNI_egg(JNIEnv *jenv, jclass jcls,
                                            jlong jarg1, jobject jarg1_);
```

For every JNI C function there has to be a static native Java function. These appear in the intermediary JNI class:

```
class exampleJNI {
  public final static native long new_Foo();
  public final static native void delete_Foo(long jarg1);
  public final static native void Foo_x_set(long jarg1, Foo jarg1_, int jarg2);
  public final static native int Foo_x_get(long jarg1, Foo jarg1_);
  public final static native int Foo_spam(long jarg1, Foo jarg1_, int jarg2,
                                          long jarg3, Foo jarg3_);
  public final static native void egg(long jarg1, Foo jarg1_);
}
```

This class contains the complete Java - C/C++ interface so all function calls go via this class. As this class acts as a go-between for all JNI calls to C/C++ code from the Java proxy classes, type wrapper classes and module class, it is known as the intermediary JNI class.

You may notice that SWIG uses a Java long wherever a pointer or class object needs to be marshalled across the Java-C/C++ boundary. This approach leads to minimal JNI code which makes for better performance as JNI code involves a lot of string manipulation. SWIG favours generating Java code over JNI code as Java code is compiled into byte code and avoids the costly string operations needed in JNI code. This approach has a downside though as the proxy class might get collected before the native method has completed. You might notice above that there is an additional parameters with a underscore postfix, eg `jarg1_`. These are added in order to prevent premature garbage collection when marshalling proxy classes.

The functions in the intermediary JNI class cannot be accessed outside of its package. Access to them is gained through the module class for globals otherwise the appropriate proxy class.

The name of the intermediary JNI class can be changed from its default, that is, the module name with JNI appended after it. The module directive attribute `jniclassname` is used to achieve this:

```
%module(jniclassname="name") modulename
```

If `name` is the same as `modulename` then the module class name gets changed from `modulename` to `modulenameModule`.

#### 27.4.1.1 The intermediary JNI class pragmas

The intermediary JNI class can be tailored through the use of pragmas, but is not commonly done. The pragmas for this class are:

| Pragma | Description |
|---|---|
| jniclassbase | Base class for the intermediary JNI class |
| jniclasspackage | Package in which to place the intermediary JNI class |
| jniclassclassmodifiers | Class modifiers and class type for the intermediary JNI class |
| jniclasscode | Java code is copied verbatim into the intermediary JNI class |
| jniclassimports | Java code, usually one or more import statements, placed before the intermediary JNI class definition |
| jniclassinterfaces | Comma separated interface classes for the intermediary JNI class |

The pragma code appears in the generated intermediary JNI class where you would expect:

```
[ jniclassimports pragma ]
[ jniclassclassmodifiers pragma ] jniclassname extends [ jniclassbase pragma ]
                                        implements [ jniclassinterfaces pragma ] {
[ jniclasscode pragma ]
... SWIG generated native methods ...
}
```

The `jniclasscode` pragma is quite useful for adding in a static block for loading the shared library / dynamic link library and demonstrates how pragmas work:

```
%pragma(java) jniclasscode=%{
  static {
    try {
      System.loadLibrary("example");
    } catch (UnsatisfiedLinkError e) {
      System.err.println("Native code library failed to load. \n" + e);
      System.exit(1);
    }
  }
%}
```

Pragmas will take either `""` or `%{ %}` as delimiters. For example, let's change the intermediary JNI class access to just the default package-private access.

```
%pragma(java) jniclassclassmodifiers="class"
```

All the methods in the intermediary JNI class will then not be callable outside of the package as the method modifiers have been changed from public access to default access. This is useful if you want to prevent users calling these low level functions.

### 27.4.2 The Java module class

All global functions and variable getters/setters appear in the module class. For our example, there is just one function:

```
public class example {
  public static void egg(Foo chips) {
    exampleJNI.egg(Foo.getCPtr(chips), chips);
  }
}
```

The module class is necessary as there is no such thing as a global in Java so all the C globals are put into this class. They are generated as static functions and so must be accessed as such by using the module name in the static function call:

```
example.egg(new Foo());
```

The primary reason for having the module class wrapping the calls in the intermediary JNI class is to implement static type checking. In this case only a `Foo` can be passed to the `egg` function, whereas any `long` can be passed to the `egg` function in the intermediary JNI class.

#### 27.4.2.1 The Java module class pragmas

The module class can be tailored through the use of pragmas, in the same manner as the intermediary JNI class. The pragmas are similarly named and are used in the same way. The complete list follows:

| Pragma | Description |
|---|---|
| modulebase | Base class for the module class |
| moduleclassmodifiers | Class modifiers and class type for the module class |
| modulecode | Java code is copied verbatim into the module class |

| | |
|---|---|
| moduleimports | Java code, usually one or more import statements, placed before the module class definition |
| moduleinterfaces | Comma separated interface classes for the module class |

The pragma code appears in the generated module class like this:

```
[ moduleimports pragma ]
[ modulemodifiers pragma ] modulename extends [ modulebase pragma ]
                                    implements [ moduleinterfaces pragma ] {
[ modulecode pragma ]
... SWIG generated wrapper functions ...
}
```

See The intermediary JNI class pragmas section for further details on using pragmas.

### 27.4.3 The Java constants interface

C/C++ constants are generated as final static members in a constants interface as mentioned in the Constants section, such as the example in this section:

```
public interface exampleConstants {
  public final static int EXPRESSION = (0x100+5);
  public final static long BIG = exampleJNI.BIG_get();
  public final static java.math.BigInteger LARGE = exampleJNI.LARGE_get();
}
```

C/C++ enums can also be generated into the same constants interface as described in the Enumerations section.

#### 27.4.3.1 The Java constants interface pragmas

Scope for tailoring the generated interface is limited to one pragma, in the same manner as the intermediary JNI class pragmas and module class pragmas. The pragma details are:

| Pragma | Description |
|---|---|
| constantsmodifiers | Class modifiers and class type for the constants interface, defaults to `public interface` |

The pragma code appears in the generated constants interface like this:

```
[ constantsmodifiers pragma ] constantsname {
... SWIG generated final static variables ...
}
```

where `constantsname` is a name created by concatenating the module name and `Constants`, for example, `exampleConstants` for a module named `example`.

The only real use for this pragma is to change the visibility from public to default with:

```
%pragma(java) constantsmodifiers="interface"
```

### 27.4.4 Java proxy classes

A Java proxy class is generated for each structure, union or C++ class that is wrapped. Proxy classes have also been called peer classes. The default proxy class for our previous example looks like this:

```
public class Foo {
  private transient long swigCPtr;
  protected transient boolean swigCMemOwn;

  protected Foo(long cPtr, boolean cMemoryOwn) {
    swigCMemOwn = cMemoryOwn;
    swigCPtr = cPtr;
  }

  protected static long getCPtr(Foo obj) {
    return (obj == null) ? 0 : obj.swigCPtr;
  }

  protected void finalize() {
    delete();
  }

  public synchronized void delete() {
    if(swigCPtr != 0 && swigCMemOwn) {
      swigCMemOwn = false;
      exampleJNI.delete_Foo(swigCPtr);
    }
    swigCPtr = 0;
  }

  public void setX(int value) {
    exampleJNI.Foo_x_set(swigCPtr, this, value);
  }

  public int getX() {
    return exampleJNI.Foo_x_get(swigCPtr, this);
  }

  public int spam(int num, Foo foo) {
    return exampleJNI.Foo_spam(swigCPtr, this, num, Foo.getCPtr(foo), foo);
  }

  public Foo() {
    this(exampleJNI.new_Foo(), true);
  }
```

```
}
```

This class merely holds a pointer to the underlying C++ object ( `swigCPtr`). It also contains all the methods in the C++ class it is proxying plus getters and setters for public member variables. These functions call the native methods in the intermediary JNI class. The advantage of having this extra layer is the type safety that the proxy class functions offer. It adds static type checking which leads to fewer surprises at runtime. For example, you can see that if you attempt to use the `spam()` function it will only compile when the parameters passed are an `int` and a `Foo`. From a user's point of view, it makes the class work as if it were a Java class:

```
Foo f = new Foo();
f.setX(3);
int y = f.spam(5, new Foo());
```

#### 27.4.4.1 Memory management

Each proxy class has an ownership flag `swigCMemOwn`. The value of this flag determines who is responsible for deleting the underlying C++ object. If set to `true`, the proxy class's finalizer will destroy the C++ object when the proxy class is garbage collected. If set to false, then the destruction of the proxy class has no effect on the C++ object.

When an object is created by a constructor or returned by value, Java automatically takes ownership of the result. On the other hand, when pointers or references are returned to Java, there is often no way to know where they came from. Therefore, the ownership is set to false. For example:

```
class Foo {
public:
  Foo();
  Foo bar1();
  Foo &bar2();
  Foo *bar2();
};
```

In Java:

```
Foo f = new Foo();   //  f.swigCMemOwn = true
Foo f1 = f.bar1();   // f1.swigCMemOwn = true
Foo f2 = f.bar2();   // f2.swigCMemOwn = false
Foo f3 = f.bar3();   // f3.swigCMemOwn = false
```

This behavior for pointers and references is especially important for classes that act as containers. For example, if a method returns a pointer to an object that is contained inside another object, you definitely don't want Java to assume ownership and destroy it!

For the most part, memory management issues remain hidden. However, there are situations where you might have to manually change the ownership of an object. For instance, consider code like this:

```
class Obj {};
class Node {
  Obj *value;
public:
  void set_value(Obj *v) { value = v; }
};
```

Now, consider the following Java code:

```
Node n = new Node();    // Create a node
{
  Obj o = new Obj();    // Create an object
  n.set_value(o);       // Set value
}                       // o goes out of scope
```

In this case, the Node `n` is holding a reference to `o` internally. However, SWIG has no way to know that this has occurred. The Java proxy class still thinks that it has ownership of `o`. As `o` has gone out of scope, it could be garbage collected in which case the C++ destructor will be invoked and `n` will then be holding a stale-pointer to `o`. If you're lucky, you will only get a segmentation fault.

To work around this, the ownership flag of `o` needs changing to `false`. The ownership flag is a private member variable of the proxy class so this is not possible without some customization of the proxy class. This can be achieved by using a typemap to customise the proxy class with pure Java code as detailed later in the section on Java typemaps.

Sometimes a function will create memory and return a pointer to a newly allocated object. SWIG has no way of knowing this so by default the proxy class does not manage the returned object. However, you can tell the proxy class to manage the memory if you specify the `%newobject` directive. Consider:

```
class Obj {...};
class Factory {
public:
  static Obj *createObj() { return new Obj(); }
};
```

If we call the factory function, then we have to manually delete the memory:

```
Obj obj = Factory.createObj();   // obj.swigCMemOwn = false
...
obj.delete();
```

Now add in the %newobject directive:

```
%newobject Factory::createObj();

class Obj {...};
class Factory {
public:
  static Obj *createObj() { return new Obj(); }
};
```

A call to `delete()` is no longer necessary as the garbage collector will make the C++ destructor call because `swigCMemOwn` is now true.

```
Obj obj = Factory.createObj();   // obj.swigCMemOwn = true;
...
```

Some memory management issues are quite tricky to fix and may only be noticeable after using for a long time. One such issue is premature garbage collection of an object created from Java and resultant usage from C++ code. The section on typemap examples cover two such scenarios, Memory management for objects passed to the C++ layer and Memory management when returning references to member variables

#### 27.4.4.2 Inheritance

Java proxy classes will mirror C++ inheritance chains. For example, given the base class `Base` and its derived class `Derived` :

```
class Base {
public:
  virtual double foo();
};

class Derived : public Base {
public:
  virtual double foo();
};
```

The base class is generated much like any other proxy class seen so far:

```
public class Base {
  private transient long swigCPtr;
  protected transient boolean swigCMemOwn;

  protected Base(long cPtr, boolean cMemoryOwn) {
    swigCMemOwn = cMemoryOwn;
    swigCPtr = cPtr;
  }

  protected static long getCPtr(Base obj) {
    return (obj == null) ? 0 : obj.swigCPtr;
  }

  protected void finalize() {
    delete();
  }

  public synchronized void delete() {
    if(swigCPtr != 0 && swigCMemOwn) {
      swigCMemOwn = false;
      exampleJNI.delete_Base(swigCPtr);
    }
    swigCPtr = 0;
  }

  public double foo() {
    return exampleJNI.Base_foo(swigCPtr, this);
  }

  public Base() {
    this(exampleJNI.new_Base(), true);
  }

}
```

The `Derived` class extends `Base` mirroring the C++ class inheritance hierarchy.

```
public class Derived extends Base {
  private transient long swigCPtr;

  protected Derived(long cPtr, boolean cMemoryOwn) {
    super(exampleJNI.SWIGDerivedUpcast(cPtr), cMemoryOwn);
    swigCPtr = cPtr;
  }

  protected static long getCPtr(Derived obj) {
    return (obj == null) ? 0 : obj.swigCPtr;
  }

  protected void finalize() {
    delete();
  }

  public synchronized void delete() {
    if(swigCPtr != 0 && swigCMemOwn) {
      swigCMemOwn = false;
      exampleJNI.delete_Derived(swigCPtr);
    }
    swigCPtr = 0;
    super.delete();
  }

  public double foo() {
    return exampleJNI.Derived_foo(swigCPtr, this);
  }

  public Derived() {
    this(exampleJNI.new_Derived(), true);
  }
```

```
}
```

Note the memory ownership is controlled by the base class. However each class in the inheritance hierarchy has its own pointer value which is obtained during construction. The `SWIGDerivedUpcast()` call converts the pointer from a `Derived *` to a `Base *`. This is a necessity as C++ compilers are free to implement pointers in the inheritance hierarchy with different values.

It is of course possible to extend `Base` using your own Java classes. If `Derived` is provided by the C++ code, you could for example add in a pure Java class `Extended` derived from `Base`. There is a caveat and that is any C++ code will not know about your pure Java class `Extended` so this type of derivation is restricted. However, true cross language polymorphism can be achieved using the directors feature.

#### 27.4.4.3 Proxy classes and garbage collection

By default each proxy class has a `delete()` and a `finalize()` method. The `finalize()` method calls `delete()` which frees any malloc'd memory for wrapped C structs or calls the C++ class destructors. The idea is for `delete()` to be called when you have finished with the C/C++ object. Ideally you need not call `delete()`, but rather leave it to the garbage collector to call it from the finalizer. When a program exits, the garbage collector does not guarantee to call all finalizers. An insight into the reasoning behind this can be obtained from Hans Boehm's Destructors, Finalizers, and Synchronization paper. Depending on what the finalizers do and which operating system you use, this may or may not be a problem.

If the `delete()` call into JNI code is just for memory handling, there is not a problem when run on most operating systems, for example Windows and Unix. Say your JNI code creates memory on the heap which your finalizers should clean up, the finalizers may or may not be called before the program exits. In Windows and Unix all memory that a process uses is returned to the system on exit, so this isn't a problem. This is not the case in some operating systems like vxWorks. If however, your finalizer calls into JNI code invoking the C++ destructor which in turn releases a TCP/IP socket for example, there is no guarantee that it will be released. Note that with long running programs the garbage collector will eventually run, thereby calling any unreferenced object's finalizers.

Some not so ideal solutions are:

1. Call the `System.runFinalizersOnExit(true)` or `Runtime.getRuntime().runFinalizersOnExit(true)` to ensure the finalizers are called before the program exits. The catch is that this is a deprecated function call as the documentation says:

   ```
   This method is inherently unsafe. It may result in finalizers being called on live objects while other threads are
   concurrently manipulating those objects, resulting in erratic behavior or deadlock.
   ```

   In many cases you will be lucky and find that it works, but it is not to be advocated. Have a look at Java web site and search for `runFinalizersOnExit`.

2. From jdk1.3 onwards a new function, `addShutdownHook()`, was introduced which is guaranteed to be called when your program exits. You can encourage the garbage collector to call the finalizers, for example, add this static block to the class that has the `main()` function:

   ```
   static {
     Runtime.getRuntime().addShutdownHook(
       new Thread() {
         public void run() { System.gc(); System.runFinalization(); }
       }
     );
   }
   ```

   Although this usually works, the documentation doesn't guarantee that `runFinalization()` will actually call the finalizers. As the shutdown hook is guaranteed you could also make a JNI call to clean up any resources that are being tracked by the C/C++ code.

3. Call the `delete()` function manually which will immediately invoke the C++ destructor. As a suggestion it may be a good idea to set the object to null so that should the object be inadvertently used again a Java null pointer exception is thrown, the alternative would crash the JVM by using a null C pointer. For example given a SWIG generated class A:

   ```
   A myA = new A();
   // use myA ...
   myA.delete();
   // any use of myA here would crash the JVM
   myA=null;
   // any use of myA here would cause a Java null pointer exception to be thrown
   ```

   The SWIG generated code ensures that the memory is not deleted twice, in the event the finalizers get called in addition to the manual `delete()` call.

4. Write your own object manager in Java. You could derive all SWIG classes from a single base class which could track which objects have had their finalizers run, then call the rest of them on program termination. The section on Java typemaps details how to specify a pure Java base class.

See the How to Handle Java Finalization's Memory-Retention Issues article for alternative approaches to managing memory by avoiding finalizers altogether.

#### 27.4.4.4 The premature garbage collection prevention parameter for proxy class marshalling

As covered earlier, the C/C++ struct/class pointer is stored in the proxy class as a Java long and when needed is passed into the native method where it is cast into the appropriate type. This approach provides very fast marshalling but could be susceptible to premature garbage collection. Consider the following C++ code:

```
class Wibble {
};
void wobble(Wibble &w);
```

The module class contains the Java wrapper for the global `wobble` method:

```
public class example {
  ...
  public static void wobble(Wibble w) {
    exampleJNI.wobble(Wibble.getCPtr(w), w);
  }
}
```

where `example` is the name of the module. All native methods go through the intermediary class which has the native method declared as such:

```
public class exampleJNI {
  ...
  public final static native void wobble(long jarg1, Wibble jarg1_);
}
```

The second parameter, `jarg1_`, is the premature garbage collection prevention parameter and is added to the native method parameter list whenever a C/C++ struct or class is marshalled as a Java long. In order to understand why, consider the alternative where the intermediary class method is declared without the additional parameter:

```
public class exampleJNI {
  ...
  public final static native void wobble(long jarg1);
}
```

and the following simple call to `wobble`:

```
{
  Wibble w = new Wibble();
  example.wobble(w);
}
```

The hotspot compiler effectively sees something like:

```
{
  Wibble w = new Wibble();
  long w_ptr = Wibble.getCPtr(w);
  // w is no longer reachable
  exampleJNI.wobble(w_ptr);
}
```

The `Wibble` object is no longer reachable after the point shown as in this bit of code, the `Wibble` object is not referenced again after this point. This means that it is a candidate for garbage collection. Should `wobble` be a long running method, it is quite likely that the finalizer for the `Wibble` instance will be called. This in turn will call its underlying C++ destructor which is obviously disastrous while the method `wobble` is running using this object. Even if `wobble` is not a long running method, it is possible for the `Wibble` instance to be finalized. By passing the `Wibble` instance into the native method, it will not be finalized as the JVM guarantees not to finalize any objects until the native method returns. Effectively, the code then becomes

```
{
  Wibble w = new Wibble();
  long w_ptr = Wibble.getCPtr(w);
  exampleJNI.wobble(w_ptr, w);
  // w is no longer reachable
}
```

and therefore there is no possibility of premature garbage collection. In practice, this premature garbage collection was only ever observed in Sun's server JVM from jdk-1.3 onwards and in Sun's client JVM from jdk-1.6 onwards.

The premature garbage collection prevention parameter for proxy classes is generated by default whenever proxy classes are passed by value, reference or with a pointer. The implementation for this extra parameter generation requires the "jtype" typemap to contain `long` and the "jstype" typemap to contain the name of a proxy class.

The additional parameter does impose a slight performance overhead and the parameter generation can be suppressed globally with the `-nopgcpp` commandline option. More selective suppression is possible with the 'nopgcpp' attribute in the "jtype" Java typemap. The attribute is a flag and so should be set to "1" to enable the suppression, or it can be omitted or set to "0" to disable. For example:

```
%typemap(jtype, nopgcpp="1") Wibble & "long"
```

**Compatibility note:** The generation of this additional parameter did not occur in versions prior to SWIG-1.3.30.

#### 27.4.4.5 Single threaded applications and thread safety

Single threaded Java applications using JNI need to consider thread safety. The same applies for the C# module where the .NET wrappers use PInvoke. Consider the C++ class:

```
class Test {
  string str;
public:
  Test() : str("initial") {}
};
```

and the Java proxy class generated by SWIG:

```
public class Test {
  private transient long swigCPtr;
  protected transient boolean swigCMemOwn;

  protected Test(long cPtr, boolean cMemoryOwn) {
    swigCMemOwn = cMemoryOwn;
    swigCPtr = cPtr;
  }

  protected static long getCPtr(Test obj) {
    return (obj == null) ? 0 : obj.swigCPtr;
  }

  protected void finalize() {
    delete();
  }

  // Call C++ destructor
  public synchronized void delete() {
    if(swigCPtr != 0 && swigCMemOwn) {
      swigCMemOwn = false;
      exampleJNI.delete_Test(swigCPtr);
    }
    swigCPtr = 0;
  }

  // Call C++ constructor
  public Test() {
    this(exampleJNI.new_Test(), true);
  }
```

```
}
```

It has two methods that call JNI methods, namely, `exampleJNI.new_Test()` for the C++ constructor and `exampleJNI.delete_Test()` for the C++ destructor. If the garbage collector collects an instance of this class, ie `delete()` is not explicitly called, then the C++ destructor will be run in a different thread to the main thread. This is because when an object is marked for garbage collection, any objects with finalizers are added to a finalization queue and the objects in the finalization queue have their `finalize()` methods run in a separate finalization thread. Therefore, if the C memory allocator is not thread safe, then the heap will get corrupted sooner or later, when a concurrent C++ delete and new are executed. It is thus essential, even in single threaded usage, to link to the C multi-thread runtime libraries, for example, use the /MD option for Visual C++ on Windows. Alternatively, lock all access to C++ functions that have heap allocation/deallocation.

Note that some of the STL in Visual C++ 6 is not thread safe, so although code might be linked to the multithread runtime libraries, undefined behaviour might still occur in a single threaded Java program. Similarly some older versions of Sun Studio have bugs in the multi-threaded implementation of the std::string class and so will lead to undefined behaviour in these supposedly single threaded Java applications.

The following innocuous Java usage of Test is an example that will crash very quickly on a multiprocessor machine if the JNI compiled code is linked against the single thread C runtime libraries.

```
for (int i=0; i<100000; i++) {
  System.out.println("Iteration " + i);
  for (int k=0; k<10; k++) {
    Test test = new Test();
  }
  System.gc();
}
```

### 27.4.5 Type wrapper classes

The generated type wrapper class, for say an `int *`, looks like this:

```
public class SWIGTYPE_p_int {
  private transient long swigCPtr;

  protected SWIGTYPE_p_int(long cPtr, boolean bFutureUse) {
    swigCPtr = cPtr;
  }

  protected SWIGTYPE_p_int() {
    swigCPtr = 0;
  }

  protected static long getCPtr(SWIGTYPE_p_int obj) {
    return obj.swigCPtr;
  }
}
```

The methods do not have public access, so by default it is impossible to do anything with objects of this class other than pass them around. The methods in the class are part of the inner workings of SWIG. If you need to mess around with pointers you will have to use some typemaps specific to the Java module to achieve this. The section on Java typemaps details how to modify the generated code.

Note that if you use a pointer or reference to a proxy class in a function then no type wrapper class is generated because the proxy class can be used as the function parameter. If however, you need anything more complicated like a pointer to a pointer to a proxy class then a typewrapper class is generated for your use.

Note that SWIG generates a type wrapper class and not a proxy class when it has not parsed the definition of a type that gets used. For example, say SWIG has not parsed the definition of `class Snazzy` because it is in a header file that you may have forgotten to use the `%include` directive on. Should SWIG parse `Snazzy *` being used in a function parameter, it will then generates a type wrapper class around a `Snazzy` pointer. Also recall from earlier that SWIG will use a pointer when a class is passed by value or by reference:

```
void spam(Snazzy *x, Snazzy &y, Snazzy z);
```

Should SWIG not know anything about `Snazzy` then a `SWIGTYPE_p_Snazzy` must be used for all 3 parameters in the `spam` function. The Java function generated is:

```
public static void spam(SWIGTYPE_p_Snazzy x, SWIGTYPE_p_Snazzy y, SWIGTYPE_p_Snazzy z) {
 ...
}
```

Note that typedefs are tracked by SWIG and the typedef name is used to construct the type wrapper class name. For example, consider the case where `Snazzy` is a typedef to an `int` which SWIG does parse:

```
typedef int Snazzy;
void spam(Snazzy *x, Snazzy &y, Snazzy z);
```

Because the typedefs have been tracked the Java function generated is:

```
public static void spam(SWIGTYPE_p_int x, SWIGTYPE_p_int y, int z) { ... }
```

### 27.4.6 Enum classes

SWIG can generate three types of enum classes. The Enumerations section discussed these but omitted all the details. The following sub-sections detail the various types of enum classes that can be generated.

#### 27.4.6.1 Typesafe enum classes

The following example demonstrates the typesafe enum classes which SWIG generates:

```
%include "enumtypesafe.swg"
%javaconst(1);
enum Beverage { ALE, LAGER=10, STOUT, PILSNER, PILZ=PILSNER };
```

The following is the code that SWIG generates:

```
public final class Beverage {
  public final static Beverage ALE = new Beverage("ALE");
  public final static Beverage LAGER = new Beverage("LAGER", 10);
  public final static Beverage STOUT = new Beverage("STOUT");
  public final static Beverage PILSNER = new Beverage("PILSNER");
  public final static Beverage PILZ = new Beverage("PILZ", PILSNER);

  public final int swigValue() {
    return swigValue;
  }

  public String toString() {
    return swigName;
  }

  public static Beverage swigToEnum(int swigValue) {
    if (swigValue < swigValues.length && swigValue >= 0 &&
        swigValues[swigValue].swigValue == swigValue)
      return swigValues[swigValue];
    for (int i = 0; i < swigValues.length; i++)
      if (swigValues[i].swigValue == swigValue)
        return swigValues[i];
    throw new IllegalArgumentException("No enum " + Beverage.class + " with value " +
                                                                        swigValue);
  }

  private Beverage(String swigName) {
    this.swigName = swigName;
    this.swigValue = swigNext++;
  }

  private Beverage(String swigName, int swigValue) {
    this.swigName = swigName;
    this.swigValue = swigValue;
    swigNext = swigValue+1;
  }

  private Beverage(String swigName, Beverage swigEnum) {
    this.swigName = swigName;
    this.swigValue = swigEnum.swigValue;
    swigNext = this.swigValue+1;
  }

  private static Beverage[] swigValues = { ALE, LAGER, STOUT, PILSNER, PILZ };
  private static int swigNext = 0;
  private final int swigValue;
  private final String swigName;
}
```

As can be seen, there are a fair number of support methods for the typesafe enum pattern. The typesafe enum pattern involves creating a fixed number of static instances of the enum class. The constructors are private to enforce this. Three constructors are available - two for C/C++ enums with an initializer and one for those without an initializer. Note that the two enums with initializers, `LAGER` and `PILZ`, each call one the two different initializer constructors. In order to use one of these typesafe enums, the `swigToEnum` static method must be called to return a reference to one of the static instances. The JNI layer returns the enum value from the C/C++ world as an integer and this method is used to find the appropriate Java enum static instance. The `swigValue` method is used for marshalling in the other direction. The `toString` method is overridden so that the enum name is available.

#### 27.4.6.2 Proper Java enum classes

The following example demonstrates the Java enums approach:

```
%include "enums.swg"
%javaconst(1);
enum Beverage { ALE, LAGER=10, STOUT, PILSNER, PILZ=PILSNER };
```

SWIG will generate the following Java enum:

```
public enum Beverage {
  ALE,
  LAGER(10),
  STOUT,
  PILSNER,
  PILZ(PILSNER);

  public final int swigValue() {
    return swigValue;
  }

  public static Beverage swigToEnum(int swigValue) {
    Beverage[] swigValues = Beverage.class.getEnumConstants();
    if (swigValue < swigValues.length && swigValue >= 0 &&
        swigValues[swigValue].swigValue == swigValue)
      return swigValues[swigValue];
    for (Beverage swigEnum : swigValues)
      if (swigEnum.swigValue == swigValue)
        return swigEnum;
    throw new IllegalArgumentException("No enum " + Beverage.class +
                                       " with value " + swigValue);
  }

  private Beverage() {
    this.swigValue = SwigNext.next++;
  }

  private Beverage(int swigValue) {
    this.swigValue = swigValue;
    SwigNext.next = swigValue+1;
  }
```

```
  private Beverage(Beverage swigEnum) {
    this.swigValue = swigEnum.swigValue;
    SwigNext.next = this.swigValue+1;
  }

  private final int swigValue;

  private static class SwigNext {
    private static int next = 0;
  }
}
```

The enum items appear first. Like the typesafe enum pattern, the constructors are private. The constructors are required to handle C/C++ enums with initializers. The `next` variable is in the `SwigNext` inner class rather than in the enum class as static primitive variables cannot be modified from within enum constructors. Marshalling between Java enums and the C/C++ enum integer value is handled via the `swigToEnum` and `swigValue` methods. All the constructors and methods in the Java enum are required just to handle C/C++ enums with initializers. These needn't be generated if the enum being wrapped does not have any initializers and the Simpler Java enums for enums without initializers section describes how typemaps can be used to achieve this.

#### 27.4.6.3 Type unsafe enum classes

The following example demonstrates type unsafe enums:

```
%include "enumtypeunsafe.swg"
%javaconst(1);
enum Beverage { ALE, LAGER=10, STOUT, PILSNER, PILZ=PILSNER };
```

SWIG will generate the following simple class:

```
public final class Beverage {
  public final static int ALE = 0;
  public final static int LAGER = 10;
  public final static int STOUT = LAGER + 1;
  public final static int PILSNER = STOUT + 1;
  public final static int PILZ = PILSNER;
}
```

### 27.4.7 Interfaces

By default SWIG wraps all C++ classes as Java classes. As Java only supports derivation from a single base class, SWIG has to ignore all bases except the first when a C++ class inherits from more than one base class. However, there is a family of SWIG macros that change the default wrapping and allows a C++ class to be wrapped as a Java interface instead of a Java class. These macros provide a way to support some sort of multiple inheritance as there is no limit to the number of interfaces that a Java class can inherit from.

When a C++ class is wrapped as a Java interface, a Java proxy class is still needed. The `swiginterface.i` library file provides three macros for marking a C++ class to be wrapped as a Java interface. There is more than one macro in order to provide a choice for choosing the Java interface and Java proxy names.

| Interface Macro Name | Description |
|---|---|
| `%interface(CTYPE)` | For C++ class `CTYPE`, proxy class name is unchanged without any suffix added, interface name has `SwigInterface` added as a suffix. |
| `%interface_impl(CTYPE)` | For C++ class `CTYPE` , proxy class name has `SwigImpl` added as a suffix, interface name has no added suffix. |
| `%interface_custom("PROXY", "INTERFACE", CTYPE)` | For C++ class `CTYPE`, proxy class name is given by the string `PROXY`, interface name is given by the string `INTERFACE`. The `PROXY` and `INTERFACE` names can use the string formatting functions used in `%rename`. |
| `%interface_additional("PROXY", "INTERFACE", "ADDITIONAL", CTYPE)` | For C++ class `CTYPE`, proxy class name is given by the string `PROXY`, interface name is given by the string `INTERFACE`. The `PROXY` and `INTERFACE` names can use the string formatting functions used in `%rename`. `ADDITIONAL` should contain a comma separated list of interfaces for the interface class to additionally extend. This is for adding interfaces not parsed by SWIG and is useful for adding in pure Java interfaces |

The table below has a few examples showing the resulting proxy and interface names for a C++ class called `Base`.

| Example Usage | Proxy Class Name | Interface Class Name |
|---|---|---|
| `%interface(Base)` | `Base` | `BaseSwigInterface` |
| `%interface_impl(Base)` | `BaseSwigImpl` | `Base` |
| `%interface_custom("BaseProxy", "IBase", Base)` | `BaseProxy` | `IBase` |
| `%interface_custom("%sProxy", "IBase", Base)` | `BaseProxy` | `IBase` |
| `%interface_custom("%sProxy", "%sInterface", Base)` | `BaseProxy` | `BaseProxyInterface` |
| `%interface_custom("%sProxy", "%(rstrip:[Proxy])sInterface", Base)` | `BaseProxy` | `BaseInterface` |

The 2nd last example shows the names used in the string formatting functions. The input for `PROXY` that `"%s"` expands to is the proxy name, that is, Base. The input for `INTERFACE` that `"%s"` expands to is the proxy name, that is, `BaseProxy` .

The last example shows `rstrip` and in this case strips the `Proxy` suffix and then adds on `Interface`.

Consider the following C++ code:

```
namespace Space {
  struct Base1 {
    virtual void Method1();
    virtual Base1();
  };
  struct Base2 {
    virtual void Method2();
    virtual Base2();
  };
  struct Derived : Base1, Base2 {
  };
  void UseBases(const Base1 &b1, const Base2 &b2);
}
```

By default all classes are wrapped and are available in Java, but, `Derived` has all bases ignored except the first. SWIG generates a warning for the above code:

```
example.i:12: Warning 813: Warning for Derived, base Base2 ignored.
Multiple inheritance is not supported in Java.
```

If we decide to wrap the two base classes as interfaces and add the following before SWIG parses the above example code:

```
%include <swiginterface.i>
%interface_impl(Space::Base1);
%interface_impl(Space::Base2);
```

then two interface files are generated, Base1.java and Base2.java in addition to proxy class files, Base1SwigImpl.java and Base2SwigImpl.java. The contents of interface file Base1.java for `Base1` is shown below:

```
public interface Base1 {
  long Base1_GetInterfaceCPtr();
  void Method1();
}
```

The proxy class in Base1SwigImpl.java for Base1 is as it would have been if `%interface` was not used, except the name has changed to `Base1SwigImpl` and it implements the appropriate base:

```
public class Base1SwigImpl implements Base1 {
...
  public long Base1_GetInterfaceCPtr() {
    return exampleJNI.Base1SwigImpl_Base1_GetInterfaceCPtr(swigCPtr);
  }

  public void Method1() {
    exampleJNI.Base1SwigImpl_Method1(swigCPtr, this);
  }
...
}
```

In fact any class using `Base` as an immediate base class will now implement the interface instead of deriving from it (or ignoring the base in the case of multiple base classes). Hence the `Derived` proxy class will now implement both bases:

```
public class Derived implements Base1, Base2 {
...
  public long Base1_GetInterfaceCPtr() {
    return exampleJNI.Derived_Base1_GetInterfaceCPtr(swigCPtr);
  }

  public long Base2_GetInterfaceCPtr() {
    return exampleJNI.Derived_Base2_GetInterfaceCPtr(swigCPtr);
  }

  public void Method1() {
    exampleJNI.Derived_Method1(swigCPtr, this);
  }

  public void Method2() {
    exampleJNI.Derived_Method2(swigCPtr, this);
  }
...
}
```

The proxy class has methods added to it, from the implemented bases, so that the underlying C++ implementation can be called. In the example above, `Method1` and `Method2` have been added from the implemented bases. If a method is ignored in the base, such as via `%ignore`, then that method will be excluded from the interface and there will not be an additional method added to the proxy class implementing that interface.

The Java interface only ever contains virtual and non-virtual instance methods from the wrapped C++ class. Any static methods, enums or variables in the wrapped C++ class are not supported and are not added to the interface. They are of course still available in the Java proxy class.

Wherever a class marked as an interface is used, such as the `UseBases` method in the example, the interface name is used as the type in the Java layer:

```
  public static void UseBases(Base1 b1, Base2 b2) {
    exampleJNI.UseBases(b1.Base1_GetInterfaceCPtr(), b1, b2.Base2_GetInterfaceCPtr(), b2);
  }
```

Note that each Java interface has a method added to obtain the correct C++ pointer for passing to the native function - `Base1_GetInterfaceCPtr` for `Base1`. This method is similar to the `getCPtr` method in the proxy classes. In fact, as shown above in the `Derived` class, the proxy classes implement this generated interface by calling a native method ( `Derived_Base1_GetInterfaceCPtr`) which calls an appropriate C++ cast of the pointer up the inheritance chain.

The `%interface_additional` macro is useful for adding additional interfaces to the Java interface class. Consider a simple class hierarchy, where `Whizz` inherits from `Bang` and we want these two classes to be wrapped as Java interfaces and the `Whizz` interface to additionally extend from the pure Java interface `java.util.EventListener`.

```
%interface_custom("Bang", "IBang", Space::Bang);
%interface_additional("Whizz", "IWhizz", "java.util.EventListener", Space::Whizz)

namespace Space {
  struct Bang {
    virtual void bang() = 0;
  };
  struct Whizz : Bang {
    virtual void whizz() = 0;
  };
}
```

The classes and interfaces generated are shown below, noting that `IWhizz` extends the additional interface `java.util.EventListener` :

```
public interface IBang { ... }

public interface IWhizz extends java.util.EventListener, IBang { ... }

public class Bang implements IBang { ... }
```

```
public class Whizz implements IWhizz, IBang { ... }
```

Note the subtle difference to the `javainterfaces` typemap discussed elsewhere. The typemap applies to the generated Java proxy class only, not the Java interface. A slight change to our example above as follows:

```
%interface_custom("Bang", "IBang", Space::Bang);
%interface_custom("Whizz", "IWhizz", Space::Whizz)
%typemap(javainterfaces) Space::Whizz "java.util.EventListener"

namespace Space {
... as shown above ...
}
```

will then the generate different code for the proxy class `Whizz` and the interface `IWhizz` as follows:

```
public interface IBang { ... }

public interface IWhizz extends IBang { ... }

public class Bang implements IBang { ... }

public class Whizz implements IWhizz, IBang, java.util.EventListener { ... }
```

Finally, a note on the implementation of the interface macros. The interface macros are implemented using the `interface` feature and typemaps. A couple of examples:

```
%define %interface(CTYPE...)
%feature("interface", name="%sSwigInterface") CTYPE;
INTERFACE_TYPEMAPS(CTYPE)
%enddef

%define %interface_additional(PROXY, INTERFACE, ADDITIONAL, CTYPE...)
%rename(PROXY) CTYPE;
%feature("interface", name=INTERFACE, additional=ADDITIONAL) CTYPE;
INTERFACE_TYPEMAPS(CTYPE)
%enddef
```

The feature accepts two attributes named `name` and `additional`. The `name` attribute should contain the name of the Java interface. The `additional` attribute should contain additional interfaces for the interface class, not parsed by SWIG. The `INTERFACE_TYPEMAPS` macro implements the typemaps and can be viewed in the `swiginterface.i` file and contains the usual Java typemaps for generating code plus the `javainterfacecode` typemap which is only used when a class is marked with the `interface` feature. See Java code typemaps for details.

**Compatibility note:** The `additional` attribute and `%interface_additional` macro was added in SWIG-4.3.0.

## 27.5 Cross language polymorphism using directors

Proxy classes provide a natural, object-oriented way to wrap C++ classes. as described earlier, each proxy instance has an associated C++ instance, and method calls from Java to the proxy are passed to the C++ instance transparently via C wrapper functions.

This arrangement is asymmetric in the sense that no corresponding mechanism exists to pass method calls down the inheritance chain from C++ to Java. In particular, if a C++ class has been extended in Java (by deriving from the proxy class), these classes will not be visible from C++ code. Virtual method calls from C++ are thus not able to access the lowest implementation in the inheritance chain.

SWIG can address this problem and make the relationship between C++ classes and proxy classes more symmetric. To achieve this goal, new classes called directors are introduced at the bottom of the C++ inheritance chain. The job of the directors is to route method calls correctly, either to C++ implementations higher in the inheritance chain or to Java implementations lower in the inheritance chain. The upshot is that C++ classes can be extended in Java and from C++ these extensions look exactly like native C++ classes. Neither C++ code nor Java code needs to know where a particular method is implemented: the combination of proxy classes, director classes, and C wrapper functions transparently takes care of all the cross-language method routing.

### 27.5.1 Enabling directors

The director feature is disabled by default. To use directors you must make two changes to the interface file. First, add the "directors" option to the %module directive, like this:

```
%module(directors="1") modulename
```

Without this option no director code will be generated. Second, you must use the %feature("director") directive to tell SWIG which classes and methods should get directors. The %feature directive can be applied globally, to specific classes, and to specific methods, like this:

```
// generate directors for all classes that have virtual methods
%feature("director");

// generate directors for the virtual methods in class Foo
%feature("director") Foo;
```

You can use the %feature("nodirector") directive to turn off directors for specific classes or methods. So for example,

```
%feature("director") Foo;
%feature("nodirector") Foo::bar;
```

will generate directors for the virtual methods of class Foo except bar().

Directors can also be generated implicitly through inheritance. In the following, class Bar will get a director class that handles the methods one() and two() (but not three()):

```
%feature("director") Foo;
class Foo {
public:
  virtual void one();
  virtual void two();
};
```

```
class Bar: public Foo {
public:
  virtual void three();
};
```

### 27.5.2 Director classes

For each class that has directors enabled, SWIG generates a new class that derives from both the class in question and a special `Swig::Director` class. These new classes, referred to as director classes, can be loosely thought of as the C++ equivalent of the Java proxy classes. The director classes store a pointer to their underlying Java proxy classes.

For simplicity let's ignore the `Swig::Director` class and refer to the original C++ class as the director's base class. By default, a director class extends all virtual methods in the inheritance chain of its base class (see the preceding section for how to modify this behavior). Virtual methods that have a final specifier are unsurprisingly excluded. Thus the virtual method calls, whether they originate in C++ or in Java via proxy classes, eventually end up in at the implementation in the director class. The job of the director methods is to route these method calls to the appropriate place in the inheritance chain. By "appropriate place" we mean the method that would have been called if the C++ base class and its Java derived classes were seamlessly integrated. That seamless integration is exactly what the director classes provide, transparently skipping over all the messy JNI glue code that binds the two languages together.

In reality, the "appropriate place" is one of only two possibilities: C++ or Java. Once this decision is made, the rest is fairly easy. If the correct implementation is in C++, then the lowest implementation of the method in the C++ inheritance chain is called explicitly. If the correct implementation is in Java, the Java API is used to call the method of the underlying Java object (after which the usual virtual method resolution in Java automatically finds the right implementation).

### 27.5.3 Overhead and code bloat

Enabling directors for a class will generate a new director method for every virtual method in the class' inheritance chain. This alone can generate a lot of code bloat for large hierarchies. Method arguments that require complex conversions to and from Java types can result in large director methods. For this reason it is recommended that directors are selectively enabled only for specific classes that are likely to be extended in Java and used in C++.

Although directors make it natural to mix native C++ objects with Java objects (as director objects), one should be aware of the obvious fact that method calls to Java objects from C++ will be much slower than calls to C++ objects. Additionally, compared to classes that do not use directors, the call routing in the director methods adds a small overhead. This situation can be optimized by selectively enabling director methods (using the %feature directive) for only those methods that are likely to be extended in Java.

### 27.5.4 Simple directors example

Consider the following SWIG interface file:

```
%module(directors="1") example;

%feature("director") DirectorBase;

class DirectorBase {
public:
  virtual ~DirectorBase() {}
  virtual void upcall_method() {}
};

void callup(DirectorBase *director) {
  director->upcall_method();
}
```

The following `DirectorDerived` Java class is derived from the Java proxy class `DirectorBase` and overrides `upcall_method()`. When C++ code invokes `upcall_method()`, the SWIG-generated C++ code redirects the call via JNI to the Java `DirectorDerived` subclass. Naturally, the SWIG generated C++ code and the generated Java intermediary class marshal and convert arguments between C++ and Java when needed.

```
class DirectorDerived extends DirectorBase {
  @Override
  public void upcall_method() {
    System.out.println("DirectorDerived.upcall_method() invoked.");
  }
}
```

Running the following Java code

```
DirectorDerived director = new DirectorDerived();
example.callup(director);
```

will result in the following being output:

```
DirectorDerived.upcall_method() invoked.
```

### 27.5.5 Director threading issues

Depending on your operating system and version of Java and how you are using threads, you might find the JVM hangs on exit. There are a couple of solutions to try out. The preferred solution requires jdk-1.4 and later and uses `AttachCurrentThreadAsDaemon` instead of `AttachCurrentThread` whenever a call into the JVM is required. This can be enabled by defining the SWIG_JAVA_ATTACH_CURRENT_THREAD_AS_DAEMON macro when compiling the C++ wrapper code. For older JVMs define SWIG_JAVA_NO_DETACH_CURRENT_THREAD instead, to avoid the `DetachCurrentThread` call but this will result in a memory leak instead. For further details inspect the source code in the java/director.swg library file.

Macros can be defined on the commandline when compiling your C++ code, or alternatively added to the C++ wrapper file as shown below:

```
%insert("runtime") %{
#define SWIG_JAVA_NO_DETACH_CURRENT_THREAD
%}
```

### 27.5.6 Director performance tuning

When a new instance of a director (or subclass) is created in Java, the C++ side of the director performs a runtime check per director method to determine if that particular method is overridden in Java or if it should invoke the C++ base implementation directly. Although this makes initialization slightly more expensive, it is generally a good overall tradeoff.

However, if all director methods are expected to usually be overridden by Java subclasses, then initialization can be made faster by avoiding these checks via the `assumeoverride` attribute. For example:

```
%feature("director", assumeoverride=1) Foo;
```

The disadvantage is that invocation of director methods from C++ when Java doesn't actually override the method will require an additional call up into Java and back to C++. As such, this option is only useful when overrides are extremely common and instantiation is frequent enough that its performance is critical.

### 27.5.7 Java exceptions from directors

With directors routing method calls to Java, and proxies routing them to C++, the handling of exceptions is an important concern. The default behavior for Java exceptions thrown in a director method overridden in Java is to store the thrown Java exception into a SWIG defined `Swig::DirectorException` C++ class exception in the C++ layer and then throw this C++ exception.

Of course, should this exception be thrown, your C++ code must catch it and handle it before returning back to Java. The default generated code **does not** attempt to handle the C++ exception, but there is a simple way to make this all work by catching the C++ exception and extracting the original Java exception by using `%catches` for `Swig::DirectorException`. Consider the example shown earlier with a modification to the `upcall_method` Java method to throw a Java exception:

```
class DirectorDerived extends DirectorBase {
  @Override
  public void upcall_method() {
    System.out.println("DirectorDerived.upcall_method() invoked.");
    throw new RuntimeException("There was a problem!");
  }
}
```

Now, by default, the JVM will abort when `example.callup(director)` is called as the C++ `Swig::DirectorException` (storing the Java exception) is thrown and not handled by the `callup` method. Needless to say this is not very user friendly and so the recommendation is to add the following simple `%catches` directive before SWIG parses the `callup` function:

```
%catches(Swig::DirectorException) callup;
```

Or target all wrapped methods using:

```
%catches(Swig::DirectorException);
```

This tells SWIG to generate a C++ catch handler using some code from the throws typemap for `Swig::DirectorException` that SWIG supplies by default, see Exception handling with %catches. This typemap code is written to simply catch the C++ `Swig::DirectorException` class and immediately return to Java throwing the original Java exception that it has stored. The net result is a stack trace containing the original Java exception including the location that the exception was thrown from.

```
DirectorDerived.upcall_method() invoked.
Exception in thread "main" java.lang.RuntimeException: There was a problem!
    at DirectorDerived.upcall_method(runme.java:4)
    at exampleJNI.SwigDirector_DirectorBase_upcall_method(exampleJNI.java:20)
    at exampleJNI.callup(Native Method)
    at example.callup(example.java:12)
    at runme.main(runme.java:21)
```

More on the `Swig::DirectorException` class can be found in the next section which details how to customize the handling of director exceptions.

#### 27.5.7.1 Customizing director exceptions

This section is for advanced customization of director exceptions. The recommendation for most users is to use the simple `%catches` directive described above as it should be sufficient for most users needs.

The conversion of Java exceptions into C++ exceptions can be customized in two different ways using the `director:except` feature. In the first approach, a code block is attached to each director method to handle the mapping of Java exceptions into C++ exceptions. The code block is generated just after the call up from the C++ director method into the overloaded method in Java. Its primary function is to check if a Java exception has been thrown and then handle it in C++. The example below converts a `java.lang.IndexOutOfBoundsException` into a C++ `std::out_of_range` exception and converts a user's Java `MyJavaException` into a C++ `MyCppException` exception. If the Java exception doesn't match either of these, a fallback `std::runtime_error` C++ exception is thrown.

```
%feature("director:except") MyClass::dirmethod(int x) {
  jthrowable $error = jenv->ExceptionOccurred();
  if ($error) {
    if (Swig::ExceptionMatches(jenv, $error, "java/lang/IndexOutOfBoundsException"))
      throw std::out_of_range(Swig::JavaExceptionMessage(jenv, $error).message());
    if (Swig::ExceptionMatches(jenv, $error, "$packagepath/MyJavaException"))
      throw MyCppException(Swig::JavaExceptionMessage(jenv, $error).message());
    throw std::runtime_error("Unexpected exception thrown in MyClass::dirmethod");
  }
}

class MyClass {
public:
  /** Throws either a std::out_of_range or MyCppException on error */
  virtual void dirmethod(int x);
  virtual ~MyClass();
};
```

A few special variables are expanded within the `director:except` feature.

- The special variable `$error` is expanded into a unique variable name (swigerror) and should be used for the assignment of the jthrowable exception that occurred.
- The special variable `$packagepath` is replaced by the outer package provided for SWIG generation by the -package option.
- The special variable `$directorthrowshandlers` is not shown above, but is replaced by applicable "directorthrows" typemap contents (covered later in this section).
- The special variable `$null` is not shown above, but is replaced by a suitable default constructed object for returning from the director method (or nothing if the director method has a void return).

Utility functions/classes in director.swg are provided to aid the exception conversion as follows:

```
namespace Swig {

  // Helper method to determine if a Java throwable matches a particular Java class type
  // Note side effect of clearing any pending exceptions
  bool ExceptionMatches(JNIEnv *jenv, jthrowable throwable, const char *classname);

  // Helper class to extract the exception message from a Java throwable
  class JavaExceptionMessage {
  public:
```

```
    JavaExceptionMessage(JNIEnv *jenv, jthrowable throwable);

    // Return a C string of the exception message in the jthrowable passed in the constructor
    // If no message is available, null_string is return instead
    const char *message(const char *null_string =
                        "Could not get exception message in JavaExceptionMessage") const;
  };

  // C++ Exception class for handling Java exceptions thrown during a director method Java upcall
  class DirectorException : public std::exception {
  public:

    // Construct exception from a Java throwable
    DirectorException(JNIEnv *jenv, jthrowable throwable);

    // More general constructor for handling as a java.lang.RuntimeException
    DirectorException(const char *msg);

    // Return exception message extracted from the Java throwable
    const char *what() const throw();

    // Reconstruct and raise/throw the Java Exception that caused the DirectorException
    // Note that any error in the JNI exception handling results in a Java RuntimeException
    void throwException(JNIEnv *jenv) const;

    // Create and throw the DirectorException
    static void raise(JNIEnv *jenv, jthrowable throwable) {
      throw DirectorException(jenv, throwable);
    }
  };

}
```

The utility function `Swig::ExceptionMatches` and class `Swig::JavaExceptionMessage` are provided to simplify writing code for wrappers that use the `director:except` feature. The function `Swig::ExceptionMatches` matches the type of the `jthrowable` thrown against a **fully qualified** JNI style class name, such as `"java/lang/IOError"`. If the throwable class is the same type, or derives from the given type, `Swig::ExceptionMatches` will return true. Care must be taken to provide the correct fully qualified name, since for wrapped exceptions the generated proxy class will have an additional package qualification, depending on the '-package' argument and use of the nspace feature. The utility class `Swig::JavaExceptionMessage` is a holder providing access to the message from the thrown Java exception. The `message()` method returns the exception message as a `const char *`, which is only valid during the lifetime of the holder. Any code using this message needs to copy it, for example into a std::string or a newly constructed C++ exception.

Using the first approach above to write handlers for a large number of methods will require repetitive duplication of the `director:except` feature code for each director method. To mitigate this, a second approach is provided via typemaps in a fashion analogous to the "throws" typemap . The "throws" typemap provides a way to map all the C++ exceptions listed in a method's defined exceptions (either from a C++ *exception specification* or a `%catches` feature) into Java exceptions. The "directorthrows" typemap provides the inverse mapping and should contain code to convert a suitably matching Java exception into a C++ exception. Only use this typemap if you wish to write custom conversions of Java exceptions into C++ exceptions and apply them to many different methods. The default handling which uses the `Swig::DirectorException` class should otherwise meet your needs.

The example below converts a Java `java.lang.IndexOutOfBoundsException` exception to the typemap's type, that is a `std::out_of_range` C++ exception:

```
%typemap(directorthrows) std::out_of_range %{
  if (Swig::ExceptionMatches(jenv, $error, "java/lang/IndexOutOfBoundsException")) {
    throw std::out_of_range(Swig::JavaExceptionMessage(jenv, $error).message());
  }
%}
```

The "directorthrows" typemap is then used in conjunction with the `director:except` feature if the `$directorthrowshandlers` special variable is used in the code block. Consider the following, which also happens to be the default:

```
%feature("director:except") %{
  jthrowable $error = jenv->ExceptionOccurred();
  if ($error) {
    $directorthrowshandlers
    Swig::DirectorException::raise(jenv, $error);
  }
%}
```

where `Swig::DirectorException::raise` is the helper method to throw a C++ `Swig::DirectorException`, see above. The code generated from the `director:except` feature has the `$directorthrowshandlers` special variable replaced with the code in the relevant "directorthrows" typemaps, for each and every exception defined for the method. The relevant exceptions can be defined either with a C++ exception specification or `%catches` as described for the "throws" typemap .

Let's try and put all this together by considering the following director method:

```
struct X {
  virtual void doSomething(int index) throw (std::out_of_range);
  ...
};

OR

%catches(std::out_of_range) X::doSomething;
struct X {
  virtual void doSomething(int index);
  ...
};
```

When combined with the default `director:except` feature and the "directorthrows" typemap above, the resulting code generated in the director method after calling up to Java will be:

```
jthrowable swigerror = jenv->ExceptionOccurred();
if (swigerror) {
  if (Swig::ExceptionMatches(jenv, swigerror, "java/lang/IndexOutOfBoundsException")) {
    throw std::out_of_range(Swig::JavaExceptionMessage(jenv, swigerror).message());
  }
  Swig::DirectorException::raise(jenv, swigerror);
}
```

*Note: Beware of using exception specifications as the SWIG director methods will be generated with the same exception specifications and if the director method throws an exception that is not specified in the exception specifications list it is likely to terminate your program. See the C++ standard for more details. Using the %catches feature instead to define the handled exceptions does not suffer this potential fate.*

Because the default code generation maps any unhandled Java exceptions to `Swig::DirectorException`, any director methods that have exception specifications may cause program termination as this exception class won't be in the exception specifications list. You can avoid throwing `Swig::DirectorException` by changing the default handling for all methods by adding a `director:except` feature without any method name. For example, you can just ignore them:

```
%feature("director:except") %{
  jthrowable $error = jenv->ExceptionOccurred();
  if ($error) {
    $directorthrowshandlers
    jenv->ExceptionClear();
    return $null; // exception is ignored
  }
%}
```

Alternatively an exception compatible with the existing director method exception specifications can be thrown. Assuming that all methods allow std::runtime_error to be thrown, the `return $null` line above could be changed to:

```
  throw std::runtime_error(Swig::JavaExceptionMessage(jenv, $error).message());
```

In more complex situations, a separate `director:except` feature may need to be attached to specific methods by providing a method name to the `director:except` feature.

This is all no doubt quite hard to follow without seeing a full example and some code. Below is a complete example demonstrating the use of most of the exception customizations one can use, that is, "directorthrows" and "throws" typemaps, %exception and %catches. See the Exception handling with %exception and %javaexception section for more on converting C++ exceptions to Java exceptions. The example also has a user defined C++ exception class called `MyNS::MyException` and this is wrapped as a Java exception. The director class being wrapped is `MyClass` and the director method is called `MyClass::dirmethod`. A number of `std::cout` calls have been added to help understand code flow. You can copy the code below into an interface file and run SWIG on it and examine the generated code.

```
%module(directors="1") example

%{
  #include <stdexcept>
  #include <iostream>
%}

// Generic catch handler for all wrapped methods
%exception %{
  try {
    $action
  } catch (const std::exception &e) {
    std::cout << "Generic std::exception catch handler" << std::endl;
    jclass clazz = jenv->FindClass("java/lang/RuntimeException");
    jenv->ThrowNew(clazz, e.what());
    return $null;
  }
%}

// Expose C++ exception as a Java Exception by changing the Java base class and providing a getMessage()
%typemap(javabase) MyNS::MyException "java.lang.RuntimeException"
%rename(getMessage) MyNS::MyException::whatsup;

%inline %{
namespace MyNS {
  class MyException {
    std::string msg;
  public:
    MyException(const char *msg) : msg(msg) {}
    const char * whatsup() const { return msg.c_str(); }
  };
}
%}

%typemap(directorthrows) MyNS::MyException %{
  if (Swig::ExceptionMatches(jenv, $error, "$packagepath/MyException")) {
    std::cout << "$1_type exception matched (directorthrows typemap)" << std::endl;
    throw $1_type(Swig::JavaExceptionMessage(jenv, $error).message());
  }
%}

%typemap(throws) MyNS::MyException %{
  std::cout << "$1_type caught (throws typemap)" << std::endl;
  jclass excep = jenv->FindClass("MyException");
  if (excep) {
    std::cout << "$1_type class found (throws typemap)" << std::endl;
    jenv->ThrowNew(excep, $1.whatsup());
  }
  return $null;
%}

// These are the exceptions that the director method MyClass::dirmethod will have catch handlers for.
// Note that this is also a virtual method / director method and the C++ exceptions listed can be
// thrown after converting them from Java exceptions.
%catches(MyNS::MyException, Swig::DirectorException) MyClass::dirmethod;

// These are the exceptions that call_dirmethod C++ wrapper will have catch handlers for.
// Note that this is not a virtual method, hence not a director method.
%catches(MyNS::MyException, Swig::DirectorException) call_dirmethod;

%feature("director") MyClass;

%feature("director:except") MyClass::dirmethod(int x) {
  jthrowable $error = jenv->ExceptionOccurred();
  if ($error) {
    std::cout << "Upcall finished, an exception was thrown in Java" << std::endl;
    $directorthrowshandlers
```

```
    std::cout << "Upcall finished, no exception conversion, throwing DirectorException" << std::endl;
    Swig::DirectorException::raise(jenv, $error);
  }
}

%inline %{
class MyClass {
public:
  /** Throws either a std::out_of_range or MyException on error */
  virtual void dirmethod(int x) {
    if (x <= 0)
      throw std::out_of_range("MyClass::dirmethod index is out of range");
    else if (x == 1)
      throw MyNS::MyException("MyClass::dirmethod some problem!");
  }
  virtual ~MyClass() {}
  static void call_dirmethod(MyClass& c, int x) {
    return c.dirmethod(x);
  }
};
%}
```

The generated code for the `call_dirmethod` wrapper contains the various exception handlers. The outer exception handler is from the `%exception` directive and the others are from the "throws" typemaps.

```
SWIGEXPORT void JNICALL Java_exampleJNI_MyClass_1call_1dirmethod(JNIEnv *jenv, jclass jcls, jlong jarg1, jobject jarg1_, jint jarg2) {
  ...
  try {
    try {
      MyClass::call_dirmethod(*arg1,arg2);
    } catch(MyNS::MyException &_e) {
      std::cout << "MyNS::MyException caught (throws typemap)" << std::endl;
      jclass excep = jenv->FindClass("MyException");
      if (excep) {
        std::cout << "MyNS::MyException class found (throws typemap)" << std::endl;
        jenv->ThrowNew(excep, (&_e)->whatsup());
      }
      return ;

    } catch(Swig::DirectorException &_e) {
      (&_e)->throwException(jenv);
      return ;
    }
  } catch (const std::exception &e) {
    std::cout << "Generic std::exception catch handler" << std::endl;
    jclass clazz = jenv->FindClass("java/lang/RuntimeException");
    jenv->ThrowNew(clazz, e.what());
    return ;
  }
```

The director method calling up to Java contains the exception handling code from the "directorthrows" typemaps and `director:except` feature.

```
void SwigDirector_MyClass::dirmethod(int x) {
      ... [call up to Java using CallStaticVoidMethod]
      jthrowable swigerror = jenv->ExceptionOccurred();
      if (swigerror) {
        std::cout << "Upcall finished, an exception was thrown in Java" << std::endl;

        if (Swig::ExceptionMatches(jenv, swigerror, "MyException")) {
          std::cout << "MyNS::MyException exception matched (directorthrows typemap)" << std::endl;
          throw MyNS::MyException(Swig::JavaExceptionMessage(jenv, swigerror).message());
        }

        std::cout << "Upcall finished, no exception conversion, throwing DirectorException" << std::endl;
        Swig::DirectorException::raise(jenv, swigerror);
      }
```

Let's use the following Java class to override the director method.

```
class DerivedClass extends MyClass {
  @Override
  public void dirmethod(int x) {
    if (x < 0)
      throw new IndexOutOfBoundsException("Index is negative");
    else if (x == 0)
      throw new MyException("MyException: bad dirmethod");
  }
}
public class runme {
  public static void main(String argv[]) {
    System.loadLibrary("example");
    ... code snippets shown below ...
  }
}
```

Consider the output using the Java code in the four slightly different scenarios below.

1. Non-director C++ class is used, thus, no upcall to a Java director method is made. A `std::out_of_range` exception is thrown, which is derived from `std::exception`, and hence caught by the generic exception handler in the `call_dirmethod` wrapper. The Java code snippet and resulting output is:

```
MyClass.call_dirmethod(new MyClass(), 0);
```

```
Generic std::exception catch handler
Exception in thread "main" java.lang.RuntimeException: MyClass::dirmethod index is out of range
```

```
        at exampleJNI.MyClass_call_dirmethod(Native Method)
        at MyClass.call_dirmethod(MyClass.java:57)
        at runme.main(runme.java:14)
```

2. Non-director C++ class again but this time the `MyNS::MyException` class is thrown and caught:

```
MyClass.call_dirmethod(new MyClass(), 1);
```

```
MyNS::MyException caught (throws typemap)
MyNS::MyException class found (throws typemap)
Exception in thread "main" MyException: MyClass::dirmethod some problem!
        at exampleJNI.MyClass_call_dirmethod(Native Method)
        at MyClass.call_dirmethod(MyClass.java:57)
        at runme.main(runme.java:15)
```

3. The `DerivedClass` director class is used so the upcall to Java occurs, but it throws a Java `MyException`, which gets converted into a C++ `MyNS::MyException`, then caught and converted back into a Java `MyException`:

```
MyClass.call_dirmethod(new DerivedClass(), 0);
```

```
Upcall finished, an exception was thrown in Java
MyNS::MyException exception matched (directorthrows typemap)
MyNS::MyException caught (throws typemap)
MyNS::MyException class found (throws typemap)
Exception in thread "main" MyException: MyException: bad dirmethod
        at exampleJNI.MyClass_call_dirmethod(Native Method)
        at MyClass.call_dirmethod(MyClass.java:57)
        at runme.main(runme.java:16)
```

4. The director class is used again, but this time the director method throws a Java `IndexOutOfBoundsException` exception which is converted into a C++ `Swig::DirectorException`, thrown and caught again. This time the original Java exception is extracted from the `Swig::DirectorException` and rethrown. Note that this approach keeps the stack trace information of the original exception, so it has the exact location of where the `IndexOutOfBoundsException` exception was thrown. This is arguably an improvement over the approach above that converts from a Java excepton to C++ exception and then back to a new Java exception, losing the location of the original exception.

```
MyClass.call_dirmethod(new DerivedClass(), -1);
```

```
Upcall finished, an exception was thrown in Java
Upcall finished, no exception conversion, throwing DirectorException
Exception in thread "main" java.lang.IndexOutOfBoundsException: Index is negative
        at DerivedClass.dirmethod(runme.java:5)
        at exampleJNI.SwigDirector_MyClass_dirmethod(exampleJNI.java:23)
        at exampleJNI.MyClass_call_dirmethod(Native Method)
        at MyClass.call_dirmethod(MyClass.java:57)
        at runme.main(runme.java:17)
```

### 27.5.8 Accessing virtual protected methods

By default, without enabling the director feature, protected methods are not wrapped and so cannot be accessed from Java. When using directors, the protected virtual methods are wrapped. These methods are wrapped with a protected Java proxy method, so the only way that Java code can access these is from within a Java class derived from the director class.

Unfortunately the rules for classes in an inheritance chain when changing the access specifiers in C++ are different to Java access modifiers. They are similar, but, unlike in C+++, one cannot assign weaker access modifiers in Java. Consider the following inheritance chain and using directors to turn on wrapping protected methods:

```
%feature("director") BaseClass;
%feature("director") DerivedClass;

%inline %{
class BaseClass {
public:
  virtual ~BaseClass();
  virtual void baseMethod1();
  virtual void baseMethod2();
protected:
  virtual void baseMethod3();
  virtual void baseMethod4();
};

class DerivedClass : public BaseClass {
public:
  virtual ~DerivedClass();
  virtual void baseMethod1();
protected:
  virtual void baseMethod2();
  virtual void baseMethod3();
public:
  virtual void baseMethod4();
};
%}
```

SWIG will generate Java access modifiers that are a one-to-one mapping of the C++ access specifiers. So if a method is `public` in C++ it will also be `public` in Java, and similarly `protected` in C++ becomes `protected` in Java. This is fine for all of the methods in the above example, except for the `DerivedClass.baseMethod2`. The Java compiler issues an error:

```
DerivedClass.java:69: error: baseMethod2() in DerivedClass cannot override baseMethod2() in BaseClass
  protected void baseMethod2() {
                 ^
  attempting to assign weaker access privileges; was public
```

This requires fixing with one of two solutions:

1. Change the access modifier to public for the derived class by adding the following before SWIG parses the `DerivedClass:`

```
%feature("java:methodmodifiers") DerivedClass::baseMethod2 "public"
```

2. Simply ignore the method in the derived class altogether as it cannot be used as a `protected` method in Java as the C++ design intended:

```
%ignore DerivedClass::baseMethod2;
```

### 27.5.9 Accessing non-virtual protected members

Members which are protected and non-virtual can also be accessed when using the 'allprotected' mode. The allprotected mode requires directors and is turned on by setting the `allprotected` option in addition to the `directors` option in the %module directive, like this:

```
%module(directors="1", allprotected="1") modulename
```

Protected member variables and methods (both static and non-static) will then be wrapped with protected access in the Java proxy class.

**Note:** Neither the directors option nor the allprotected mode support types defined with protected scope. This includes any enums or typedefs declared in the protected section of the C++ class.

The following simple example is a class with numerous protected members, including the constructor and destructor:

```
%module(directors="1", allprotected="1") example

%feature("director") ProtectedBase;

// Ignore use of unsupported types (those defined in the protected section)
%ignore ProtectedBase::typedefs;

%inline %{

class ProtectedBase {
protected:
  ProtectedBase() {}
  virtual ~ProtectedBase() {}
  virtual void virtualMethod() const {}
  void nonStaticMethod(double d) const {}
  static void staticMethod(int i) {}
  int instanceMemberVariable;
  static int staticMemberVariable;

  // unsupported: types defined with protected access and the methods/variables which use them
  typedef int IntegerType;
  IntegerType typedefs(IntegerType it) { return it; }
};
int ProtectedBase::staticMemberVariable = 10;

%}
```

Note that the `IntegerType` has protected scope and the members which use this type must be ignored as they cannot be wrapped.

The proxy methods are protected, so the only way the protected members can be accessed is within a class that derives from the director class, such as the following:

```
class MyProtectedBase extends ProtectedBase
{
  public MyProtectedBase() {
  }

  public void accessProtected() {
    virtualMethod();
    nonStaticMethod(1.2);
    staticMethod(99);

    setInstanceMemberVariable(5);
    int i = getInstanceMemberVariable();

    setStaticMemberVariable(10);
    i = getStaticMemberVariable();
  }
}
```

## 27.6 Common customization features

An earlier section presented the absolute basics of C/C++ wrapping. If you do nothing but feed SWIG a header file, you will get an interface that mimics the behavior described. However, sometimes this isn't enough to produce a nice module. Certain types of functionality might be missing or the interface to certain functions might be awkward. This section describes some common SWIG features that are used to improve the interface to existing C/C++ code.

### 27.6.1 C/C++ helper functions

Sometimes when you create a module, it is missing certain bits of functionality. For example, if you had a function like this

```
typedef struct Image {...};
void set_transform(Image *im, double m[4][4]);
```

it would be accessible from Java, but there may be no easy way to call it. The problem here is that a type wrapper class is generated for the two dimensional array parameter so there is no easy way to construct and manipulate a suitable `double [4][4]` value. To fix this, you can write some extra C helper functions. Just use the `%inline` directive. For example:

```
%inline %{
/* Note: double[4][4] is equivalent to a pointer to an array double (*)[4] */
```

```
double (*new_mat44())[4] {
  return (double (*)[4]) malloc(16*sizeof(double));
}
void free_mat44(double (*x)[4]) {
  free(x);
}
void mat44_set(double x[4][4], int i, int j, double v) {
  x[i][j] = v;
}
double mat44_get(double x[4][4], int i, int j) {
  return x[i][j];
}
%}
```

From Java, you could then write code like this:

```
Image im = new Image();
SWIGTYPE_p_a_4__double a = example.new_mat44();
example.mat44_set(a, 0, 0, 1.0);
example.mat44_set(a, 1, 1, 1.0);
example.mat44_set(a, 2, 2, 1.0);
...
example.set_transform(im, a);
example.free_mat44(a);
```

Admittedly, this is not the most elegant looking approach. However, it works and it wasn't too hard to implement. It is possible to improve on this using Java code, typemaps, and other customization features as covered in later sections, but sometimes helper functions are a quick and easy solution to difficult cases.

### 27.6.2 Class extension with %extend

One of the more interesting features of SWIG is that it can extend structures and classes with new methods or constructors. Here is a simple example:

```
%module example
%{
#include "someheader.h"
%}

struct Vector {
  double x, y, z;
};

%extend Vector {
  char *toString() {
    static char tmp[1024];
    sprintf(tmp, "Vector(%g, %g, %g)", $self->x, $self->y, $self->z);
    return tmp;
  }
  Vector(double x, double y, double z) {
    Vector *v = (Vector *) malloc(sizeof(Vector));
    v->x = x;
    v->y = y;
    v->z = z;
    return v;
  }
};
```

Now, in Java

```
Vector v = new Vector(2, 3, 4);
System.out.println(v);
```

will display

```
Vector(2, 3, 4)
```

`%extend` works with both C and C++ code. It does not modify the underlying object in any way---the extensions only show up in the Java interface.

### 27.6.3 Class extension with %proxycode

The previous section described how to extend a wrapped class with C or C++ code. This section describes how to extend a wrapped class with Java code instead of C/C++ code. The `%proxycode` directive is used and is just a macro for `%insert("proxycode")`. The Code insertion block section describes the `%insert` directive. The section of code for insertion is "proxycode", that is, the Java proxy class. This directive must hence only be used within the scope of a class, otherwise it is silently ignored. There are two common ways to get the scope correct.

The first is to use `%proxycode` inside a class that SWIG parses, for example a `toString()` method can be added to a C++ class using pure Java code. A C++ header file can mix C++ and Java code inside the C++ class as follows:

```
// flag.h header file
class Flag {
  bool flag;
public:
  Flag(bool flag) : flag(flag) {}
  bool FetchFlag() { return flag; }
#if defined(SWIG)
%proxycode %{
  public String toString() {
    boolean flag = FetchFlag();
    return Boolean.toString(flag);
  }
%}
#endif
};
```

and wrapped using:

```
%{
#include "flag.h"
%}
%include "flag.h"
```

The second is to use `%proxycode` within `%extend` as everything within a `%extend` block is effectively within the scope of the class, for example:

```
// flag.h header file
class Flag {
  bool flag;
public:
  Flag(bool flag) : flag(flag) {}
  bool FetchFlag() { return flag; }
};
```

and wrapped using:

```
%{
#include "flag.h"
%}
%include "flag.h"

%extend Flag {
#if defined(SWIG)
%proxycode %{
  public String toString() {
    boolean flag = FetchFlag();
    return Boolean.toString(flag);
  }
%}
#endif
}
```

There is some very limited support of typemaps within a `%proxycode` block. A useful trick is to obtain the Java type for a given C/C++ type using the $typemap special macro. The following C++ template demonstrates this:

```
%inline %{
template<typename T> struct Value {
  T value;
  Value(const T& val) : value(val) {}
};
%}

%extend Value {
%proxycode %{
  public String toString() {
    // Note template type expansion is supported, so T is expanded to 'unsigned int' in this example
    // and $typemap(jstype, unsigned int) in turn is expanded to 'long'
    $typemap(jstype, T) val = getValue();
    return "$javaclassname value: " + val + " Java type: $typemap(jstype, T) JNI type: $typemap(jni, T)";
  }
%}
}
%template(ValueUnsignedInt) Value<unsigned int>;
```

The generated Java contains the expanded special variable and macro resulting in Java proxy code:

```
public class ValueUnsignedInt {
  ...
  public String toString() {
    long val = getValue();
    return "ValueUnsignedInt value: " + val + " Java type: long JNI type: jlong";
  }

}
```

### 27.6.4 Exception handling with %exception and %javaexception

If a C or C++ function throws an error, you may want to convert that error into a Java exception. To do this, you can use the `%exception` directive. The `%exception` directive simply lets you rewrite part of the generated wrapper code to include an error check. It is detailed in full in the Exception handling with %exception section.

In C, a function often indicates an error by returning a status code (a negative number or a NULL pointer perhaps). Here is a simple example of how you might handle that:

```
%exception malloc {
  $action
  if (!result) {
    jclass clazz = (*jenv)->FindClass(jenv, "java/lang/OutOfMemoryError");
    (*jenv)->ThrowNew(jenv, clazz, "Not enough memory");
    return $null;
  }
}
void *malloc(size_t nbytes);
```

In Java,

```
SWIGTYPE_p_void a = example.malloc(2000000000);
```

will produce a familiar looking Java exception:

```
Exception in thread "main" java.lang.OutOfMemoryError: Not enough memory
        at exampleJNI.malloc(Native Method)
        at example.malloc(example.java:16)
        at runme.main(runme.java:112)
```

If a library provides some kind of general error handling framework, you can also use that. For example:

```
%exception malloc {
  $action
  if (err_occurred()) {
    jclass clazz = (*jenv)->FindClass(jenv, "java/lang/OutOfMemoryError");
    (*jenv)->ThrowNew(jenv, clazz, "Not enough memory");
    return $null;
  }
}
void *malloc(size_t nbytes);
```

If no declaration name is given to `%exception`, it is applied to all wrapper functions. The `$action` is a SWIG special variable and is replaced by the C/C++ function call being wrapped. The `return $null;` handles all native method return types, namely those that have a void return and those that do not. This is useful for typemaps that will be used in native method returning all return types. See the section on Java special variables for further explanation.

C++ exceptions are also easy to handle. We can catch the C++ exception and rethrow it as a Java exception like this:

```
%exception getitem {
  try {
    $action
  } catch (std::out_of_range &e) {
    jclass clazz = jenv->FindClass("java/lang/Exception");
    jenv->ThrowNew(clazz, "Range error");
    return $null;
  }
}

class FooClass {
  public:
    FooClass *getitem(int index);      // Might throw std::out_of_range exception
    ...
};
```

In the example above, `java.lang.Exception` is a checked exception class and so ought to be declared in the throws clause of `getitem`. Classes can be specified for adding to the throws clause using `%javaexception(classes)` instead of `%exception`, where `classes` is a string containing one or more comma separated Java classes. The `%clearjavaexception` feature is the equivalent to `%clearexception` and clears previously declared exception handlers. The `%nojavaexception` feature is the equivalent to `%noexception` and disables the exception handler. See Clearing features for the difference on disabling and clearing features.

```
%javaexception("java.lang.Exception") getitem {
  try {
    $action
  } catch (std::out_of_range &e) {
    jclass clazz = jenv->FindClass("java/lang/Exception");
    jenv->ThrowNew(clazz, "Range error");
    return $null;
  }
}

class FooClass {
public:
  FooClass *getitem(int index);      // Might throw std::out_of_range exception
  ...
};
```

The generated proxy method now generates a throws clause containing `java.lang.Exception`:

```
public class FooClass {
  ...
  public FooClass getitem(int index) throws java.lang.Exception { ... }
  ...
}
```

The examples above first use the C JNI calling syntax then the C++ JNI calling syntax. The C++ calling syntax will not compile as C and also vice versa. It is however possible to write JNI calls which will compile under both C and C++ and is covered in the Typemaps for both C and C++ compilation section.

The language-independent `exception.i` library file can also be used to raise exceptions. See the SWIG Library chapter. The typemap example Handling C++ exception specifications as Java exceptions provides further exception handling capabilities.

### 27.6.5 Method access with %javamethodmodifiers

A Java feature called `%javamethodmodifiers` can be used to change the method modifiers from the default `public`. It applies to both module class methods and proxy class methods. For example:

```
%javamethodmodifiers protect_me() "protected";
void protect_me();
```

Will produce the method in the module class with protected access.

```
protected static void protect_me() {
  exampleJNI.protect_me();
}
```

### 27.6.6 Java begin

It is possible to add a common comment at the start of every generated Java file. The `%module` directive supports the `javabegin` option for this. The provided text is generated at the very beginning of each generated .java file. As it is generated before the package statement, is only really useful for adding in a common comment into all generated .java files. For example, copyright text for each file:

```
%module(javabegin="/* Common comment. Copyright (C) 2000 Mr Nobody. */\n") nobodymodule
```

## 27.7 Tips and techniques

Although SWIG is largely automatic, there are certain types of wrapping problems that require additional user input. Examples include dealing with output parameters, strings and arrays. This chapter discusses the common techniques for solving these problems.

### 27.7.1 Input and output parameters using primitive pointers and references

A common problem in some C programs is handling parameters passed as simple pointers or references. For example:

```
void add(int x, int y, int *result) {
  *result = x + y;
}
```

or perhaps

```
int sub(int *x, int *y) {
  return *x-*y;
}
```

The `typemaps.i` library file will help in these situations. For example:

```
%module example
%include "typemaps.i"

void add(int, int, int *OUTPUT);
int  sub(int *INPUT, int *INPUT);
```

In Java, this allows you to pass simple values. For example:

```
int result = example.sub(7, 4);
System.out.println("7 - 4 = " + result);
int[] sum = {0};
example.add(3, 4, sum);
System.out.println("3 + 4 = " + sum[0]);
```

Which will display:

```
7 - 4 = 3
3 + 4 = 7
```

Notice how the `INPUT` parameters allow integer values to be passed instead of pointers and how the `OUTPUT` parameter will return the result in the first element of the integer array.

If you don't want to use the names `INPUT` or `OUTPUT` , use the `%apply` directive. For example:

```
%module example
%include "typemaps.i"

%apply int *OUTPUT { int *result };
%apply int *INPUT  { int *x, int *y};

void add(int x, int y, int *result);
int  sub(int *x, int *y);
```

If a function mutates one of its parameters like this,

```
void negate(int *x) {
  *x = -(*x);
}
```

you can use `INOUT` like this:

```
%include "typemaps.i"
...
void negate(int *INOUT);
```

In Java, the input parameter is the first element in a 1 element array and is replaced by the output of the function. For example:

```
int[] neg = {3};
example.negate(neg);
System.out.println("Negative of 3 = " + neg[0]);
```

And no prizes for guessing the output:

```
Negative of 3 = -3
```

These typemaps can also be applied to C++ references. The above examples would work the same if they had been defined using references instead of pointers. For example, the Java code

to use the `negate` function would be the same if it were defined either as it is above:

```
void negate(int *INOUT);
```

or using a reference:

```
void negate(int &INOUT);
```

Note: Since most Java primitive types are immutable and are passed by value, it is not possible to perform in-place modification of a type passed as a parameter.

Be aware that the primary purpose of the `typemaps.i` file is to support primitive datatypes. Writing a function like this

```
void foo(Bar *OUTPUT);
```

will not have the intended effect since `typemaps.i` does not define an OUTPUT rule for `Bar`.

### 27.7.2 Simple pointers

If you must work with simple pointers such as `int *` or `double *` another approach to using `typemaps.i` is to use the `cpointer.i` pointer library file. For example:

```
%module example
%include "cpointer.i"

%inline %{
extern void add(int x, int y, int *result);
%}

%pointer_functions(int, intp);
```

The `%pointer_functions(type, name)` macro generates five helper functions that can be used to create, destroy, copy, assign, and dereference a pointer. In this case, the functions are as follows:

```
int  *new_intp();
int  *copy_intp(int *x);
void  delete_intp(int *x);
void  intp_assign(int *x, int value);
int   intp_value(int *x);
```

In Java, you would use the functions like this:

```
SWIGTYPE_p_int intPtr = example.new_intp();
example.add(3, 4, intPtr);
int result = example.intp_value(intPtr);
System.out.println("3 + 4 = " + result);
```

If you replace `%pointer_functions(int, intp)` by `%pointer_class(int, intp)`, the interface is more class-like.

```
intp intPtr = new intp();
example.add(3, 4, intPtr.cast());
int result = intPtr.value();
System.out.println("3 + 4 = " + result);
```

See the SWIG Library chapter for further details.

### 27.7.3 Wrapping C arrays with Java arrays

SWIG can wrap arrays in a more natural Java manner than the default by using the `arrays_java.i` library file. Let's consider an example:

```
%include "arrays_java.i";
int array[4];
void populate(int x[]) {
  int i;
  for (i=0; i<4; i++)
    x[i] = 100 + i;
}
```

These one dimensional arrays can then be used as if they were Java arrays:

```
int[] array = new int[4];
example.populate(array);

System.out.print("array: ");
for (int i=0; i<array.length; i++)
  System.out.print(array[i] + " ");

example.setArray(array);

int[] global_array = example.getArray();

System.out.print("\nglobal_array: ");
for (int i=0; i<array.length; i++)
  System.out.print(global_array[i] + " ");
```

Java arrays are always passed by reference, so any changes a function makes to the array will be seen by the calling function. Here is the output after running this code:

```
array: 100 101 102 103
global_array: 100 101 102 103
```

Note that for assigning array variables the length of the C variable is used, so it is possible to use a Java array that is bigger than the C code will cope with. Only the number of elements in the C array will be used. However, if the Java array is not large enough then you are likely to get a segmentation fault or access violation, just like you would in C. When arrays are used in functions like `populate`, the size of the C array passed to the function is determined by the size of the Java array.

Please be aware that the typemaps in this library are not efficient as all the elements are copied from the Java array to a C array whenever the array is passed to and from JNI code. There is an alternative approach using the SWIG array library and this is covered in the next section.

### 27.7.4 Unbounded C Arrays

Sometimes a C function expects an array to be passed as a pointer. For example,

```
int sumitems(int *first, int nitems) {
  int i, sum = 0;
  for (i = 0; i < nitems; i++) {
    sum += first[i];
  }
  return sum;
}
```

One of the ways to wrap this is to apply the Java array typemaps that come in the `arrays_java.i` library file:

```
%include "arrays_java.i"
%apply int[] {int *};
```

The `ANY` size will ensure the typemap is applied to arrays of all sizes. You could narrow the typemap matching rules by specifying a particular array size. Now you can use a pure Java array and pass it to the C code:

```
int[] array = new int[10000000];          // Array of 10-million integers
for (int i=0; i<array.length; i++) {      // Set some values
  array[i] = i;
}
int sum = example.sumitems(array, 10000);
System.out.println("Sum = " + sum);
```

and the sum would be displayed:

```
Sum = 49995000
```

This approach is probably the most natural way to use arrays. However, it suffers from performance problems when using large arrays as a lot of copying of the elements occurs in transferring the array from the Java world to the C++ world. An alternative approach to using Java arrays for C arrays is to use an alternative SWIG library file `carrays.i`. This approach can be more efficient for large arrays as the array is accessed one element at a time. For example:

```
%include "carrays.i"
%array_functions(int, intArray);
```

The `%array_functions(type, name)` macro generates four helper functions that can be used to create and destroy arrays and operate on elements. In this case, the functions are as follows:

```
int *new_intArray(size_t nelements);
void delete_intArray(int *x);
int intArray_getitem(int *x, size_t index);
void intArray_setitem(int *x, size_t index, int value);
```

In Java, you would use the functions like this:

```
SWIGTYPE_p_int array = example.new_intArray(10000000);  // Array of 10-million integers
for (int i=0; i<10000; i++) {                           // Set some values
  example.intArray_setitem(array, i, i);
}
int sum = example.sumitems(array, 10000);
System.out.println("Sum = " + sum);
```

If you replace `%array_functions(int, intp)` by `%array_class(int, intp)`, the interface is more class-like and a couple more helper functions are available for casting between the array and the type wrapper class.

```
%include "carrays.i"
%array_class(int, intArray);
```

The `%array_class(type, name)` macro creates wrappers for an unbounded array object that can be passed around as a simple pointer like `int *` or `double *`. For instance, you will be able to do this in Java:

```
intArray array = new intArray(10000000);  // Array of 10-million integers
for (int i=0; i<10000; i++) {             // Set some values
  array.setitem(i, i);
}
int sum = example.sumitems(array.cast(), 10000);
System.out.println("Sum = " + sum);
```

The array "object" created by `%array_class()` does not encapsulate pointers inside a special array object. In fact, there is no bounds checking or safety of any kind (just like in C). Because of this, the arrays created by this library are extremely low-level indeed. You can't iterate over them nor can you even query their length. In fact, any valid memory address can be accessed if you want (negative indices, indices beyond the end of the array, etc.). Needless to say, this approach is not going to suit all applications. On the other hand, this low-level approach is

extremely efficient and well suited for applications in which you need to create buffers, package binary data, etc.

### 27.7.5 Passing a string with length

SWIG provides multi-argument typemap available as mentioned in Passing a string with length. The following simple example demonstrates passing a string to the wrapped function.

```
%apply (char *STRING, size_t LENGTH) { (const char data[], size_t len) }
%inline %{
void binaryChar1(const char data[], size_t len) {
  printf("len: %d data: ", len);
  for (size_t i=0; i<len; ++i)
    printf("%x ", data[i]);
  printf("\n");
}
%}
```

Calling from Java requires a string to be passed in as the multi-argument typemap being applied reduces the number of arguments in the target language to one, from the original two:

```
String str = "hi\0jk";
example.binaryChar1(str);
```

resulting in the output

```
$ java runme
len: 5 data: 68 69 0 6a 6b
```

The typemap uses Java `String::getBytes()` to convert the string to the default system encoding. If you wish to use another encoding, you can modify the typemap. For example to convert the string to UTF-8 use:

```
%typemap(javain, throws="java.io.IllegalCharsetNameException")


  (const char *STRING, size_t LENGTH) %{($javainput == null) ? null : $javainput.getBytes("UTF-8")%}
```

### 27.7.6 Overriding new and delete to allocate from Java heap

Unlike some languages supported by SWIG, Java has a true garbage collection subsystem. Other languages will free SWIG wrapped objects when their reference count reaches zero. Java only schedules these objects for finalization, which may not occur for some time. Because SWIG objects are allocated on the C heap, Java users may find the JVM memory use quickly exceeds the assigned limits, as memory fills with unfinalized proxy objects. Forcing garbage collection is clearly an undesirable solution.

An elegant fix for C++ users is to override new and delete using the following code (here shown included in a SWIG interface file)

```
/* File: java_heap.i */
%module test
%{
#include <stdexcept>
#include "jni.h"

/**
 *  A stash area embedded in each allocation to hold java handles
 */
struct Jalloc {
  jbyteArray jba;
  jobject ref;
};

static JavaVM *cached_jvm = 0;

JNIEXPORT jint JNICALL JNI_OnLoad(JavaVM *jvm, void *reserved) {
  cached_jvm = jvm;
  return JNI_VERSION_1_2;
}

static JNIEnv * JNU_GetEnv() {
  JNIEnv *env;
  jint rc = cached_jvm->GetEnv((void **)&env, JNI_VERSION_1_2);
  if (rc == JNI_EDETACHED)
    throw std::runtime_error("current thread not attached");
  if (rc == JNI_EVERSION)
    throw std::runtime_error("jni version not supported");
  return env;
}

void * operator new(size_t t) {
  if (cached_jvm != 0) {
    JNIEnv *env = JNU_GetEnv();
    jbyteArray jba = env->NewByteArray((int) t + sizeof(Jalloc));
    if (env->ExceptionOccurred())
      throw bad_alloc();
    void *jbuffer = static_cast<void *>(env->GetByteArrayElements(jba, 0));
    if (env->ExceptionOccurred())
      throw bad_alloc();
    Jalloc *pJalloc = static_cast<Jalloc *>(jbuffer);
    pJalloc->jba = jba;
    /* Assign a global reference so byte array will persist until delete'ed */
    pJalloc->ref = env->NewGlobalRef(jba);
    if (env->ExceptionOccurred())
      throw bad_alloc();
    return static_cast<void *>(static_cast<char *>(jbuffer) + sizeof(Jalloc));
  }
  else { /* JNI_OnLoad not called, use malloc and mark as special */
    Jalloc *pJalloc = static_cast<Jalloc *>(malloc((int) t + sizeof(Jalloc)));
    if (!pJalloc)
      throw bad_alloc();
```

```
    pJalloc->ref = 0;
    return static_cast<void *>(
      static_cast<char *>(static_cast<void *>(pJalloc)) + sizeof(Jalloc));
  }
}

void operator delete(void *v) {
  if (v != 0) {
    void *buffer = static_cast<void *>( static_cast<char *>(v) - sizeof(Jalloc));
    Jalloc *pJalloc = static_cast<Jalloc *>(buffer);
    if (pJalloc->ref) {
      JNIEnv *env = JNU_GetEnv();
      env->DeleteGlobalRef(pJalloc->ref);
      env->ReleaseByteArrayElements(pJalloc->jba, static_cast<jbyte *>(buffer), 0);
    }
    else {
      free(buffer);
    }
  }
}
%}
...
```

This code caches the Java environment during initialization, and when new is called, a Java ByteArray is allocated to provide the SWIG objects with space in the Java heap. This has the combined effect of re-asserting the Java virtual machine's limit on memory allocation, and puts additional pressure on the garbage collection system to run more frequently. This code is made slightly more complicated because allowances must be made if new is called before the JNI_OnLoad is executed. This can happen during static class initialization, for example.

Unfortunately, because most Java implementations call malloc and free, this solution will not work for C wrapped structures. However, you are free to make functions that allocate and free memory from the Java heap using this model and use these functions in place of malloc and free in your own code.

## 27.8 Java typemaps

This section describes how you can modify SWIG's default wrapping behavior for various C/C++ datatypes using the `%typemap` directive. You are advised to be familiar with the material in the "Typemaps" chapter. While not absolutely essential knowledge, this section assumes some familiarity with the Java Native Interface (JNI). JNI documentation can be consulted either online at the Java web site or from a good JNI book. The following two books are recommended:

- Title: 'Essential JNI: Java Native Interface.' Author: Rob Gordon. Publisher: Prentice Hall. ISBN: 0-13-679895-0.
- Title: 'The Java Native Interface: Programmer's Guide and Specification.' Author: Sheng Liang. Publisher: Addison-Wesley. ISBN: 0-201-32577-2. Also available online at the Sun Developer Network.

Before proceeding, it should be stressed that typemaps are not a required part of using SWIG---the default wrapping behavior is enough in most cases. Typemaps are only used if you want to change some aspect of the generated code.

### 27.8.1 Default primitive type mappings

The following table lists the default type mapping from Java to C/C++.

| C/C++ type | Java type | JNI type |
|---|---|---|
| bool<br>const bool & | boolean | jboolean |
| char<br>const char & | char | jchar |
| signed char<br>const signed char & | byte | jbyte |
| unsigned char<br>const unsigned char & | short | jshort |
| short<br>const short & | short | jshort |
| unsigned short<br>const unsigned short & | int | jint |
| int<br>const int & | int | jint |
| unsigned int<br>const unsigned int & | long | jlong |
| long<br>const long & | int | jint |
| unsigned long<br>const unsigned long & | long | jlong |
| long long<br>const long long & | long | jlong |
| unsigned long long<br>const unsigned long long & | java.math.BigInteger | jobject |
| float<br>const float & | float | jfloat |
| double<br>const double & | double | jdouble |
| size_t<br>const size_t & | long | jlong |
| char *<br>char [] | String | jstring |

Note that default mappings for the C long type is more suitable for 32-bit systems. If long is 64-bit, the full range can be obtained by defining `SWIGWORDSIZE64` when invoking SWIG. The long type will instead be mapped as follows:

| C/C++ type | Java type | JNI type |
|---|---|---|
| long<br>const long & | long | jlong |
| unsigned long<br>const unsigned long & | java.math.BigInteger | jobject |

Note that when size_t is 64-bit in C, the full unsigned range is not available. This can be fixed by applying the 64-bit unsigned long long typemaps as follows:

```
%apply unsigned long long { size_t };
%apply const unsigned long long & { const size_t & };
```

The net effect then changes from the default shown earlier to:

| C/C++ type | Java type | JNI type |
|---|---|---|
| size_t<br>const size_t & | java.math.BigInteger | jobject |

Note that SWIG wraps the C `char` type as a character. Pointers and arrays of this type are wrapped as strings. The `signed char` type can be used if you want to treat `char` as a signed number rather than a character. Also note that all const references to primitive types are treated as if they are passed by value.

Given the following C function:

```
void func(unsigned short a, char *b, const long &c, unsigned long long d);
```

The module class method would be:

```
public static void func(int a, String b, int c, java.math.BigInteger d) {...}
```

The intermediary JNI class would use the same types:

```
public final static native void func(int jarg1, String jarg2, int jarg3,
                                     java.math.BigInteger jarg4);
```

and the JNI function would look like this:

```
SWIGEXPORT void JNICALL Java_exampleJNI_func(JNIEnv *jenv, jclass jcls,
                jint jarg1, jstring jarg2, jint jarg3, jobject jarg4) {...}
```

The mappings for C `int` and C `long` are appropriate for 32 bit applications which are used in the 32 bit JVMs. There is no perfect mapping between Java and C as Java doesn't support all the unsigned C data types. However, the mappings allow the full range of values for each C type from Java.

### 27.8.2 Default typemaps for non-primitive types

The previous section covered the primitive type mappings. Non-primitive types such as classes and structs are mapped using pointers on the C/C++ side and storing the pointer into a Java `long` variable which is held by the proxy class or type wrapper class. This applies whether the type is marshalled as a pointer, by reference or by value. It also applies for any unknown/incomplete types which use type wrapper classes.

So in summary, the C/C++ pointer to non-primitive types is cast into the 64 bit Java `long` type and therefore the JNI type is a `jlong`. The Java type is either the proxy class or type wrapper class.

### 27.8.3 Sixty four bit JVMs

If you are using a 64 bit JVM you may have to override the C long, but probably not C int default mappings. Mappings will be system dependent, for example long will need remapping on Unix LP64 systems (long, pointer 64 bits, int 32 bits), but not on Microsoft 64 bit Windows which will be using a P64 IL32 (pointer 64 bits and int, long 32 bits) model. This may be automated in a future version of SWIG. Note that the Java write once run anywhere philosophy holds true for all pure Java code when moving to a 64 bit JVM. Unfortunately it won't of course hold true for JNI code.

### 27.8.4 What is a typemap?

A typemap is nothing more than a code generation rule that is attached to a specific C datatype. For example, to convert integers from Java to C, you might define a typemap like this:

```
%module example

%typemap(in) int {
  $1 = $input;
  printf("Received an integer : %d\n",  $1);
}
%inline %{
extern int fact(int nonnegative);
%}
```

Typemaps are always associated with some specific aspect of code generation. In this case, the "in" method refers to the conversion of input arguments to C/C++. The datatype `int` is the datatype to which the typemap will be applied. The supplied C code is used to convert values. In this code a number of special variables prefaced by a `$` are used. The `$1` variable is a placeholder for a local variable of type `int`. The `$input` variable contains the Java data, the JNI `jint` in this case.

When this example is compiled into a Java module, it can be used as follows:

```
System.out.println(example.fact(6));
```

and the output will be:

```
Received an integer : 6
720
```

In this example, the typemap is applied to all occurrences of the `int` datatype. You can refine this by supplying an optional parameter name. For example:

```
%module example

%typemap(in) int nonnegative {
  $1 = $input;
  printf("Received an integer : %d\n",  $1);
}

%inline %{
extern int fact(int nonnegative);
```

```
%}
```

In this case, the typemap code is only attached to arguments that exactly match `int nonnegative`.

The application of a typemap to specific datatypes and argument names involves more than simple text-matching--typemaps are fully integrated into the SWIG C++ type-system. When you define a typemap for `int`, that typemap applies to `int` and qualified variations such as `const int`. In addition, the typemap system follows `typedef` declarations. For example:

```
%typemap(in) int nonnegative {
  $1 = $input;
  printf("Received an integer : %d\n",  $1);
}
%inline %{
typedef int Integer;
extern int fact(Integer nonnegative);    // Above typemap is applied
%}
```

However, the matching of `typedef` only occurs in one direction. If you defined a typemap for `Integer`, it is not applied to arguments of type `int`.

Typemaps can also be defined for groups of consecutive arguments. For example:

```
%typemap(in) (char *str, int len) {
...
};

int count(char c, char *str, int len);
```

When a multi-argument typemap is defined, the arguments are always handled as a single Java parameter. This allows the function to be used like this (notice how the length parameter is omitted):

```
int c = example.count('e', "Hello World");
```

### 27.8.5 Typemaps for mapping C/C++ types to Java types

The typemaps available to the Java module include the common typemaps listed in the main typemaps section. There are a number of additional typemaps which are necessary for using SWIG with Java. The most important of these implement the mapping of C/C++ types to Java types:

| Typemap | Description |
|---|---|
| jni | JNI C types. These provide the default mapping of types from C/C++ to JNI for use in the JNI (C/C++) code. |
| jtype | Java intermediary types. These provide the default mapping of types from C/C++ to Java for use in the native functions in the intermediary JNI class. The type must be the equivalent Java type for the JNI C type specified in the "jni" typemap. |
| jstype | Java types. These provide the default mapping of types from C/C++ to Java for use in the Java module class, proxy classes and type wrapper classes. |
| javain | Conversion from jstype to jtype. These are Java code typemaps which transform the type used in the Java module class, proxy classes and type wrapper classes (as specified in the "jstype" typemap) to the type used in the Java intermediary JNI class (as specified in the "jtype" typemap). In other words the typemap provides the conversion to the native method call parameter types. |
| javaout | Conversion from jtype to jstype. These are Java code typemaps which transform the type used in the Java intermediary JNI class (as specified in the "jtype" typemap) to the Java type used in the Java module class, proxy classes and type wrapper classes (as specified in the "jstype" typemap). In other words the typemap provides the conversion from the native method call return type. |
| jboxtype | Java boxed type. These are Java code typemaps to provide the Java boxed type, such as, `Integer` for C type `int`. As autoboxing is only relevant to the Java primitive types, these are only provided for the C types that map to Java primitive types. This typemap is usually only used by C++ STL container wrappers that are wrapped by Java generic types as the boxed type must be used instead of the unboxed/primitive type when declaring a Java generic type. |
| javadirectorin | Conversion from jtype to jstype for director methods. These are Java code typemaps which transform the type used in the Java intermediary JNI class (as specified in the "jtype" typemap) to the Java type used in the Java module class, proxy classes and type wrapper classes (as specified in the "jstype" typemap). This typemap provides the conversion for the parameters in the director methods when calling up from C++ to Java. See Director typemaps. |
| javadirectorout | Conversion from jstype to jtype for director methods. These are Java code typemaps which transform the type used in the Java module class, proxy classes and type wrapper classes (as specified in the "jstype" typemap) to the type used in the Java intermediary JNI class (as specified in the "jtype" typemap). This typemap provides the conversion for the return type in the director methods when returning from the C++ to Java upcall. See Director typemaps. |
| directorin | Conversion from C++ type to jni type for director methods. These are C++ typemaps which convert the parameters used in the C++ director method to the appropriate JNI intermediary type. The conversion is done in JNI code prior to calling the Java function from the JNI code. See Director typemaps. |
| directorout | Conversion from jni type to C++ type for director methods. These are C++ typemaps which convert the JNI return type used in the C++ director method to the appropriate C++ return type. The conversion is done in JNI code after calling the Java function from the JNI code. See Director typemaps. |

If you are writing your own typemaps to handle a particular type, you will normally have to write a collection of them. The default typemaps are in "`java.swg`" and so might be a good place for finding typemaps to base any new ones on.

The "jni", "jtype" and "jstype" typemaps are usually defined together to handle the Java to C/C++ type mapping. An "in" typemap should be accompanied by a "javain" typemap and likewise an "out" typemap by a "javaout" typemap. If an "in" typemap is written, a "freearg" and "argout" typemap may also need to be written as some types have a default "freearg" and/or "argout" typemap which may need overriding. The "freearg" typemap sometimes releases memory allocated by the "in" typemap. The "argout" typemap sometimes sets values in function parameters which are passed by reference in Java.

Note that the "in" typemap marshals the JNI type held in the "jni" typemap to the real C/C++ type and for the opposite direction, the "out" typemap marshals the real C/C++ type to the JNI type held in the "jni" typemap. For non-primitive types the "in" and "out" typemaps are responsible for casting between the C/C++ pointer and the 64 bit `jlong` type. There is no portable way to cast a pointer into a 64 bit integer type and the approach taken by SWIG is mostly portable, but breaks C/C++ aliasing rules. In summary, these rules state that a pointer to any type must never be dereferenced by a pointer to any other incompatible type. The following code snippet might aid in understand aliasing rules better:

```
short a;
short* pa = 0;
int i = 0x1234;

a = (short)i;    /* okay */
a = *(short*)&i; /* breaks aliasing rules */
```

An email posting, Aliasing, pointer casts and gcc 3.3 elaborates further on the subject. In SWIG, the "in" and "out" typemaps for pointers are typically

```
%typemap(in) struct Foo * %{
  $1 = *(struct Foo **)&$input; /* cast jlong into C ptr */
%}
%typemap(out) struct Bar * %{
  *(struct Bar **)&$result = $1; /* cast C ptr into jlong */
```

```
%}
struct Bar {...};
struct Foo {...};
struct Bar * FooBar(struct Foo *f);
```

resulting in the following code which breaks the aliasing rules:

```
SWIGEXPORT jlong JNICALL Java_exampleJNI_FooBar(JNIEnv *jenv, jclass jcls,
                                                jlong jarg1, jobject jarg1_) {
  jlong jresult = 0 ;
  struct Foo *arg1 = (struct Foo *) 0 ;
  struct Bar *result = 0 ;

  (void)jenv;
  (void)jcls;
  (void)jarg1_;
  arg1 = *(struct Foo **)&jarg1;
  result = (struct Bar *)FooBar(arg1);
  *(struct Bar **)&jresult = result;
  return jresult;
}
```

If you are using gcc as your C compiler, you might get a "dereferencing type-punned pointer will break strict-aliasing rules" warning about this. Please see Compiling a dynamic module to avoid runtime problems with these strict aliasing rules.

The default code generated by SWIG for the Java module comes from the typemaps in the "`java.swg`" library file which implements the Default primitive type mappings and Default typemaps for non-primitive types covered earlier. There are other type mapping typemaps in the Java library. These are listed below:

| C Type | Typemap | File | Kind | Java Type | Function |
|---|---|---|---|---|---|
| primitive pointers and references | INPUT | typemaps.i | input | Java basic types | Allows values to be used for C functions taking pointers for data input. |
| primitive pointers and references | OUTPUT | typemaps.i | output | Java basic type arrays | Allows values held within an array to be used for C functions taking pointers for data output. |
| primitive pointers and references | INOUT | typemaps.i | input output | Java basic type arrays | Allows values held within an array to be used for C functions taking pointers for data input and output. |
| string<br>wstring | [unnamed] | std_string.i | input output | String | Use for std::string mapping to Java String. |
| arrays of primitive types | [unnamed] | arrays_java.i | input output | arrays of primitive Java types | Use for mapping C arrays to Java arrays. |
| arrays of classes/structs/unions | JAVA_ARRAYSOFCLASSES macro | arrays_java.i | input output | arrays of proxy classes | Use for mapping C arrays to Java arrays. |
| arrays of enums | ARRAYSOFENUMS | arrays_java.i | input output | int[] | Use for mapping C arrays to Java arrays (typeunsafe and simple enum wrapping approaches only). |
| char * | BYTE | various.i | input | byte[] | Java byte array is converted to char array |
| char ** | STRING_ARRAY | various.i | input output | String[] | Use for mapping NULL terminated arrays of C strings to Java String arrays |
| unsigned char * | NIOBUFFER | various.i | input output | java.nio.Buffer | Use for mapping directly allocated buffers to c/c++. useful with directors and long lived memory objects |

### 27.8.6 Java typemap attributes

There are a few additional typemap attributes that the Java module supports.

The first of these is the 'throws' attribute. The throws attribute is optional and specified after the typemap name and contains one or more comma separated classes for adding to the throws clause for any methods that use that typemap. It is analogous to the %javaexception feature's throws attribute.

```
%typemap(typemapname, throws="ExceptionClass1, ExceptionClass2") type { ... }
```

The attribute is necessary for supporting Java checked exceptions and can be added to just about any typemap. The list of typemaps include all the C/C++ (JNI) typemaps in the " Typemaps" chapter and the Java specific typemaps listed in the previous section, barring the "jni", "jtype" and "jstype" typemaps as they could never contain code to throw an exception.

The throws clause is generated for the proxy method as well as the JNI method in the JNI intermediary class. If a method uses more than one typemap and each of those typemaps have classes specified in the throws clause, the union of the exception classes is added to the throws clause ensuring there are no duplicate classes. See the NaN exception example for further usage.

The "jtype" typemap has the optional 'nopgcpp' attribute which can be used to suppress the generation of the premature garbage collection prevention parameter.

The "javain" typemap has the optional 'pre', 'post' and 'pgcppname' attributes. These are used for generating code before and after the JNI call in the proxy class or module class. The 'pre' attribute contains code that is generated before the JNI call and the 'post' attribute contains code generated after the JNI call. The 'pgcppname' attribute is used to change the premature garbage collection prevention parameter name passed to the JNI function. This is sometimes needed when the 'pre' typemap creates a temporary variable which is then passed to the JNI function.

Note that when the 'pre' or 'post' attributes are specified and the associated type is used in a constructor, a constructor helper function is generated. This is necessary as the Java proxy constructor wrapper makes a call to a support constructor using a *this* call. In Java the *this* call must be the first statement in the constructor body. The constructor body thus calls the helper function and the helper function instead makes the JNI call, ensuring the 'pre' code is called before the JNI call is made. There is a Date marshalling example showing 'pre', 'post' and 'pgcppname' attributes in action.

### 27.8.7 Java special variables

The standard SWIG special variables are available for use within typemaps as described in the Typemaps documentation , for example `$1`, `$input`, `$result` etc.

The Java module uses a few additional special variables:

**`$javaclassname`**
This special variable works like the other special variables and `$javaclassname` is similar to `$1_type`. It expands to the class name for use in Java given a pointer. SWIG wraps unions, structs and classes using pointers and in this case it expands to the Java proxy class name. For example, `$javaclassname` is replaced by the proxy classname`Foo` when wrapping a `Foo *` and `$&javaclassname` expands to the proxy classname when wrapping the C/C++ type `Foo` and `$*javaclassname` expands to the proxy classname when wrapping`Foo *&`. If the type does not have an associated proxy class, it expands to the type wrapper class name, for example, `SWIGTYPE_p_unsigned_short` is generated when wrapping `unsigned short *`. The class name is fully qualified with the package name when using the nspace feature.

**`$javaclazzname`**
This special variable works like `$javaclassname`, but expands the fully qualified C++ class into the package name, if used by the nspace feature, and the proxy class name, mangled for use as a function name. For example, `Namespace1::Namespace2::Klass` is expanded into `Namespace1_Namespace2_Klass_`. This special variable is usually used for making calls to a

function in the intermediary JNI class, as they are mangled with this prefix.

**$null**
Used in input typemaps to return early from JNI functions that have either void or a non-void return type. Example:

```
%typemap(check) int * %{
  if (error) {
    SWIG_JavaThrowException(jenv, SWIG_JavaIndexOutOfBoundsException, "Array element error");
    return $null;
  }
%}
```

If the typemap gets put into a function with void as return, $null will expand to nothing:

```
SWIGEXPORT void JNICALL Java_jnifn(...) {
  if (error) {
    SWIG_JavaThrowException(jenv, SWIG_JavaIndexOutOfBoundsException, "Array element error");
    return ;
  }
  ...
}
```

otherwise $null expands to *NULL*

```
SWIGEXPORT jobject JNICALL Java_jnifn(...) {
  if (error) {
    SWIG_JavaThrowException(jenv, SWIG_JavaIndexOutOfBoundsException, "Array element error");
    return NULL;
  }
  ...
}
```

**$javainput, $jnicall and $owner**
The $javainput special variable is used in "javain" typemaps and $jnicall and $owner are used in "javaout" typemaps. $jnicall is analogous to $action in %exception. It is replaced by the call to the native method in the intermediary JNI class. $owner is replaced by either `true` if %newobject has been used, otherwise `false` . $javainput is analogous to the $input special variable. It is replaced by the parameter name.

Here is an example:

```
%typemap(javain) Class "Class.getCPtr($javainput)"
%typemap(javain) unsigned short "$javainput"
%typemap(javaout) Class * {
    return new Class($jnicall, $owner);
  }

%inline %{
  class Class {...};
  Class * bar(Class cls, unsigned short ush) { return new Class(); };
%}
```

The generated proxy code is then:

```
public static Class bar(Class cls, int ush) {
  return new Class(exampleJNI.bar(Class.getCPtr(cls), cls, ush), false);
}
```

Here $javainput has been replaced by `cls` and `ush`. $jnicall has been replaced by the native method call, `exampleJNI.bar(...)` and $owner has been replaced by `false`. If %newobject is used by adding the following at the beginning of our example:

```
%newobject bar(Class cls, unsigned short ush);
```

The generated code constructs the return type using `true` indicating the proxy class `Class` is responsible for destroying the C++ memory allocated for it in `bar`:

```
public static Class bar(Class cls, int ush) {
  return new Class(exampleJNI.bar(Class.getCPtr(cls), cls, ush), true);
}
```

**$static**
This special variable expands to either *static* or nothing depending on whether the class is an inner Java class or not. It is used in the "javaclassmodifiers" typemap so that global classes can be wrapped as Java proxy classes and nested C++ classes/enums can be wrapped with the Java equivalent, that is, static inner proxy classes.

**$error, $jniinput, $javacall and $packagepath**
These special variables are used in the directors typemaps. See Director specific typemaps for details.

**$module**
This special variable expands to the module name, as specified by `%module` or the `-module` commandline option.

**$imclassname**
This special variable expands to the intermediary class name. Usually this is the same as '$moduleJNI', unless the jniclassname attribute is specified in the %module directive.

**$imfuncname**
This special variable expands to the name of the function in the intermediary class that will be used in $jnicall. Like, $jnicall, this special variable is only expanded in the "javaout" typemap.

**$javainterfacename**
This special variable is only expanded when the `interface` feature is applied to a class. It works much like `$javaclassname` , but instead of expanding to the proxy classname, it expands to the value in the `name` attribute in the `interface` feature. For example:

```
%feature("interface", name="MyInterface") MyClass;
%typemap(jstype) MyClass  "$&javainterfacename"
```

```
%typemap(jstype) MyClass * "$javainterfacename"
```

will result in the `jstype` typemap expanding to `MyInterface` for both `MyClass` and `MyClass *`. The interface name is fully qualified with the package name when using the nspace feature.

**`$interfacename`**
This special variable is only expanded when the `interface` feature is applied to a class. It expands to just the interface name and is thus different to `$javainterfacename` in that it is not fully qualified with the package name when using the nspace feature.

### 27.8.8 Typemaps for both C and C++ compilation

JNI calls must be written differently depending on whether the code is being compiled as C or C++. For example C compilation requires the pointer to a function pointer struct member syntax like

```
const jclass clazz = (*jenv)->FindClass(jenv, "java/lang/String");
```

whereas C++ code compilation of the same function call is a member function call using a class pointer like

```
const jclass clazz = jenv->FindClass("java/lang/String");
```

To enable typemaps to be used for either C or C++ compilation, a set of JCALLx macros have been defined in Lib/java/javahead.swg, where x is the number of arguments in the C++ version of the JNI call. The above JNI calls would be written in a typemap like this

```
const jclass clazz = JCALL1(FindClass, jenv, "java/lang/String");
```

Note that the SWIG preprocessor expands these into the appropriate C or C++ JNI calling convention. The C calling convention is emitted by default and the C++ calling convention is emitted when using the -c++ SWIG commandline option. If you do not intend your code to be targeting both C and C++ then your typemaps can use the appropriate JNI calling convention and need not use the JCALLx macros.

### 27.8.9 Java code typemaps

Most of SWIG's typemaps are used for the generation of C/C++ code. The typemaps in this section are used solely for the generation of Java code. Elements of proxy classes and type wrapper classes come from the following typemaps (the defaults).

`%typemap(javabase)`

base (extends) for Java class: empty default
Note that this typemap accepts a `replace` attribute as an optional flag. When set to "1", it will replace/override any C++ base classes that might have been parsed. If this flag is not specified and there are C++ base classes, then a multiple inheritance warning is issued and the code in the typemap is ignored. The typemap also accepts a `notderived` attribute as an optional flag. When set to "1", it will not apply to classes that are derived from a C++ base. When used with the SWIGTYPE type, it is useful for giving a common base for all proxy classes, that is, providing a base class that sits in between all proxy classes and the Java base class `Object` for example: `%typemap(javabase, notderived="1") SWIGTYPE "CommonBase"`.

`%typemap(javabody)`

the essential support body for proxy classes (proxy base classes only), typewrapper classes and enum classes. Default contains extra constructors, memory ownership control member variables (`swigCMemOwn`, `swigCPtr`), the `getCPtr` method etc.

`%typemap(javabody_derived)`

the essential support body for proxy classes (derived classes only). Same as "javabody" typemap, but only used for proxy derived classes.

`%typemap(javaclassmodifiers)`

class modifiers for the Java class: default is "public class"

`%typemap(javacode)`

Java code is copied verbatim to the Java class: empty default As there can only be one "javacode" typemap per class, also consider using the %proxycode directive which can be used multiple times per class and offers nearly identical functionality.

`%typemap(javadestruct, methodname="delete", methodmodifiers="public synchronized", parameters="")`

destructor wrapper - the `delete()` method (proxy classes only), used for all proxy classes except those which have a base class : default calls C++ destructor (or frees C memory) and resets `swigCPtr` and `swigCMemOwn` flags

Note that the `delete()` method name is configurable and is specified by the `methodname` attribute. The method modifiers are also configurable via the `methodmodifiers` attribute. If a `%javamethodmodifiers` is attached to the class' destructor, it will be used in preference to the `methodmodifiers` typemap attribute for the class. The `delete` method's parameters declaration can be provided in the optional `parameters` typemap attribute.

`%typemap(javadestruct_derived, methodname="delete", methodmodifiers="public synchronized", parameters="")`

destructor wrapper - the `delete()` method (proxy classes only), same as "javadestruct" but only used for derived proxy classes : default calls C++ destructor (or frees C memory) and resets `swigCPtr` and `swigCMemOwn` flags

Note that the `delete()` method name is configurable and is specified by the `methodname` attribute. The method modifiers are also configurable via the `methodmodifiers` attribute. If a `%javamethodmodifiers` is attached to the class' destructor, it will be used in preference to the `methodmodifiers` typemap attribute for the class. The `delete` method's parameters declaration can be provided in the optional `parameters` typemap attribute.

`%typemap(javaimports)`

import statements for Java class: empty default

`%typemap(javainterfaces)`

interfaces (implements) for Java class: empty default

`%typemap(javafinalize)`

the `finalize()` method (proxy classes only): default calls the `delete()` method

Note that the default javafinalize typemap must contain the full implementation of the finalize method. Any customization to this typemap must still declare a java finalize method with the correct signature. Note also that the name of the generated "delete" method may be affected by `javadestruct` and `javadestruct_derived` typemaps. Below shows an example modifying the finalizer, assuming the `delete` method has been renamed to `swig_delete`.

```
%typemap(javafinalize) SWIGTYPE %{
  protected void finalize() {
    swig_delete();  // renamed to prevent conflict with existing delete method
```

```
  }
]%
```

`%typemap(javainterfacemodifiers)`

Interface modifiers for the Java interface generated when using the `interface` feature, see Java interfaces section. The default is "public interface".

**Compatibility note:** This typemap was added in SWIG-4.1.0.

`%typemap(javainterfacecode, declaration="...", cptrmethod="...")`

The code in this typemap is added to the body of a Java proxy class but only when a class is marked with the `interface` feature. The typemap is used in the proxy class marked with the interface feature as well as all proxy classes derived from the marked C++ class, as they are all generated as implementing the Java interface. The default typemap used in the `%interface` family of macros mentioned in the Java interfaces section, where `CTYPE` is the C++ class macro argument, is as follows:

```
%typemap(javainterfacecode,
         declaration="  long $interfacename_GetInterfaceCPtr();\n",
         cptrmethod="$interfacename_GetInterfaceCPtr") CTYPE %{
  public long $interfacename_GetInterfaceCPtr() {
    return $imclassname.$javaclazzname$interfacename_GetInterfaceCPtr(swigCPtr);
  }
%}
```

The special variable `$interfacename` is expanded into the name specified in the `interface` feature.

**Compatibility Note:** In SWIG-1.3.21 and earlier releases, typemaps called "javagetcptr" and "javaptrconstructormodifiers" were available. These are deprecated and the "javabody" typemap can be used instead. The `javainterfacecode` typemap and interface feature was introduced in SWIG-3.0.9.

In summary the contents of the typemaps make up a proxy class like this:

```
[ javaimports typemap ]
[ javaclassmodifiers typemap ] javaclassname extends [ javabase typemap ]
                                              implements [ javainterfaces typemap ] {
[ javabody or javabody_derived typemap ]
[ javafinalize typemap ]
public synchronized void delete() [ javadestruct OR javadestruct_derived typemap ]
[ javacode typemap ]
[ javainterfacecode typemap]
... proxy functions ...
}
```

Note the `delete()` methodname and method modifiers are configurable, see "javadestruct" and "javadestruct_derived" typemaps above.

The `javainterfacecode` typemap is only used when bases are marked by the `interface` feature and the `implements` list will also then be expanded to include these Java interfaces.

The type wrapper class is similar in construction:

```
[ javaimports typemap ]
[ javaclassmodifiers typemap ] javaclassname extends [ javabase typemap ]
                                              implements [ javainterfaces typemap ] {
[ javabody typemap ]
[ javacode typemap ]
}
```

The enum class is also similar in construction:

```
[ javaimports typemap ]
[ javaclassmodifiers typemap ] javaclassname extends [ javabase typemap ]
                                              implements [ javainterfaces typemap ] {
... Enum values ...
[ javabody typemap ]
[ javacode typemap ]
}
```

The "javaimports" typemap is ignored if the enum class is wrapped by an inner Java class, that is when wrapping an enum declared within a C++ class.

The Java interface turned on by the `interface` feature is fairly simple:

```
[ javaimports typemap ]
[ javainterfacemodifiers typemap ] [ name ] [extends additional [, ...] ] {
[ javainterfacecode:cptrmethod typemap attribute ]
... interface declarations ...
}
```

where `name` is the `name` attribute and `additional` is the `additional` attribute in the interface feature.

The defaults can be overridden to tailor the generated classes. Here is an example which will change the `getCPtr` method and constructor from the default public access to protected access. If the classes in one package are not using the classes in another package, then these methods need not be public and removing access to these low level implementation details, is a good thing. If you are invoking SWIG more than once and generating the wrapped classes into different packages in each invocation, then you cannot do this as you will then have different packages.

```
%typemap(javabody) SWIGTYPE %{
  private transient long swigCPtr;
  protected transient boolean swigCMemOwn;

  protected $javaclassname(long cPtr, boolean cMemoryOwn) {
    swigCMemOwn = cMemoryOwn;
    swigCPtr = cPtr;
  }

  protected static long getCPtr($javaclassname obj) {
    return (obj == null) ? 0 : obj.swigCPtr;
  }
```

```
%}
```

The typemap code is the same that is in "java.swg", barring the last two method modifiers. Note that SWIGTYPE will target all proxy classes, but not the type wrapper classes. Also the above typemap is only used for proxy classes that are potential base classes. To target proxy classes that are derived from a wrapped class as well, the "javabody_derived" typemap should also be overridden.

For the typemap to be used in all type wrapper classes, all the different types that type wrapper classes could be used for should be targeted:

```
%typemap(javabody) SWIGTYPE *, SWIGTYPE &, SWIGTYPE [], SWIGTYPE (CLASS::*) %{
  private transient long swigCPtr;

  protected $javaclassname(long cPtr, boolean bFutureUse) {
    swigCPtr = cPtr;
  }

  protected $javaclassname() {
    swigCPtr = 0;
  }

  protected static long getCPtr($javaclassname obj) {
    return (obj == null) ? 0 : obj.swigCPtr;
  }
%}
```

Again this is the same that is in "java.swg", barring the method modifier for getCPtr.

When using multiple modules or the nspace feature it is common to invoke SWIG with a different -package command line option for each module. However, by default the generated code may not compile if generated classes in one package use generated classes in another package. The visibility of the getCPtr() and pointer constructor generated from the javabody typemaps needs changing. The default visibility is protected but it needs to be public for access from a different package. Just changing 'protected' to 'public' in the typemap achieves this. Two macros are available in java.swg to make this easier and using them is the preferred approach over simply copying the typemaps and modifying as this is forward compatible with any changes in the javabody typemap in future versions of SWIG. The macros are for the proxy and typewrapper classes and can respectively be used to to make the method and constructor public:

```
SWIG_JAVABODY_PROXY(public, public, SWIGTYPE)
SWIG_JAVABODY_TYPEWRAPPER(public, public, public, SWIGTYPE)
```

### 27.8.10 Director specific typemaps

The Java directors feature requires the "javadirectorin", "javadirectorout", "directorin" and the "directorout" typemaps in order to work properly. The "javapackage" typemap is an optional typemap used to identify the Java package path for individual SWIG generated proxy classes used in director methods.

%typemap(directorin)

The "directorin" typemap is used for converting arguments in the C++ director class to the appropriate JNI type before the upcall to Java. This typemap also specifies the JNI field descriptor for the type in the "descriptor" attribute. For example, integers are converted as follows:

```
%typemap(directorin, descriptor="I") int "$input = (jint) $1;"
```

$input is the SWIG name of the JNI temporary variable passed to Java in the upcall. The descriptor="I" will put an I into the JNI field descriptor that identifies the Java method that will be called from C++. For more about JNI field descriptors and their importance, refer to the JNI documentation mentioned earlier. A typemap for C character strings is:

```
%typemap(directorin, descriptor="Ljava/lang/String;", noblock=1) char * {
  $input = 0;
  if ($1) {
    $input = JCALL1(NewStringUTF, jenv, (const char *)$1);
    if (!$input) return $null;
  }
  Swig::LocalRefGuard $1_refguard(jenv, $input);
}
```

The Swig::LocalRefGuard class should be used in directorin typemaps for newly allocated objects. It is used to control local reference counts ensuring the count is decremented after the call up into Java has completed. Its destructor simply calls jenv->DeleteLocalRef(obj) on the obj passed in during construction.

User-defined types have the default "descriptor" attribute " L$packagepath/$javaclassname;" where $packagepath is the package name passed from the SWIG command line and $javaclassname is the Java proxy class' name. If the -package commandline option is not used to specify the package, then '$packagepath/' will be removed from the resulting output JNI field descriptor. **Do not forget the terminating ';' for JNI field descriptors starting with 'L'.** If the ';' is left out, Java will generate a "method not found" runtime error. Note that the $packagepath substitution always uses the path separator '/' when expanded. The $javaclassname expansion can be confusing as it is normally expanded using the '.' separator. However, $javaclassname is expanded using the path separator '/' in typemap's "descriptor" attribute as well as in the "directorthrows" typemap.

%typemap(directorout)

The "directorout" typemap is used for converting the JNI return type in the C++ director class to the appropriate C++ type after the upcall to Java. For example, integers are converted as follows:

```
%typemap(directorout) int %{ $result = (int)$input; %}
```

$input is the SWIG name of the JNI temporary variable returned from Java after the upcall. $result is the resulting output. A typemap for C character strings is:

```
%typemap(directorout) char * {
  $1 = 0;
  if ($input) {
    $result = (char *)jenv->GetStringUTFChars($input, 0);
    if (!$1) return $null;
  }
}
```

%typemap(javadirectorin)

Conversion from jtype to jstype for director methods. These are Java code typemaps which transform the type used in the Java intermediary JNI class (as specified in the "jtype" typemap) to the Java type used in the Java module class, proxy classes and type wrapper classes (as specified in the "jstype" typemap). This typemap provides the conversion for the parameters in the director methods when calling up from C++ to Java.

For primitive types, this typemap is usually specified as:

```
%typemap(javadirectorin) int "$jniinput"
```

The `$jniinput` special variable is analogous to `$javainput` special variable. It is replaced by the input parameter name.

`%typemap(javadirectorout)`

Conversion from jstype to jtype for director methods. These are Java code typemaps which transform the type used in the Java module class, proxy classes and type wrapper classes (as specified in the "jstype" typemap) to the type used in the Java intermediary JNI class (as specified in the "jtype" typemap). This typemap provides the conversion for the return type in the director methods when returning from the C++ to Java upcall.

For primitive types, this typemap is usually specified as:

```
%typemap(javadirectorout) int "$javacall"
```

The `$javacall` special variable is analogous to the `$jnicall` special variable. It is replaced by the call to the target Java method. The target method is the method in the Java proxy class which overrides the virtual C++ method in the C++ base class.

`%typemap(directorthrows)`

Conversion of Java exceptions to C++ exceptions in director method's exception handling. This typemap is expected to test the `$error` special variable for a matching Java exception and if successful convert and throw it into a C++ exception given by the typemap's type. The `$error` special variable is of type `jthrowable` and is substituted with a unique variable name in the generated code.

The example below converts a Java `java.lang.IndexOutOfBoundsException` exception to the typemap's type, that is `std::out_of_range`:

```
%typemap(directorthrows) std::out_of_range %{
  if (Swig::ExceptionMatches(jenv, $error, "java/lang/IndexOutOfBoundsException")) {
    throw std::out_of_range(Swig::JavaExceptionMessage(jenv, $error).message());
  }
%}
```

The utility function `Swig::ExceptionMatches` and class `Swig::JavaExceptionMessage` are helpers available when using directors and are described in the Java Exceptions from Directors section.

`%typemap(javapackage)`

The "javapackage" typemap is optional; it serves to identify a class's Java package. This typemap should be used in conjunction with classes that are defined outside of the current SWIG interface file. The typemap is only used if the type is used in a director method, that is, in a virtual method in a director class. For example:

```
// class Foo is handled in a different interface file:
%import "Foo.i"

%feature("director") Example;

%inline {
  class Bar { };

  class Example {
  public:
    virtual ~Example();
    virtual void ping(Foo *arg1, Bar *arg2);
  };
}
```

Assume that the Foo class is part of the Java package *com.wombat.foo* but the above interface file is part of the Java package *com.wombat.example*. Without the "javapackage" typemap, SWIG will assume that the Foo class belongs to *com.wombat.example* class. The corrected interface file looks like:

```
// class Foo is handled in a different interface file:
%import "Foo.i"
%typemap("javapackage") Foo, Foo *, Foo & "com.wombat.foo"
%feature("director") Example;

%inline {
  class Bar { };

  class Example {
  public:
    virtual ~Example();
    virtual void ping(Foo *arg1, Bar *arg2);
  };
}
```

SWIG looks up the package based on the **actual** type (plain Foo, Foo pointer and Foo reference), so it is important to associate all three types with the desired package. Practically speaking, you should create a separate SWIG interface file, which is %import-ed into each SWIG interface file, when you have multiple Java packages. Note the helper macros below, `OTHER_PACKAGE_SPEC` and `ANOTHER_PACKAGE_SPEC`, which reduce the amount of extra typing. "`TYPE...`" is useful when passing templated types to the macro, since multiargument template types appear to the SWIG preprocessor as multiple macro arguments.

```
%typemap("javapackage") SWIGTYPE, SWIGTYPE *, SWIGTYPE &
                                        "package.for.most.classes";

%define OTHER_PACKAGE_SPEC(TYPE...)
%typemap("javapackage") TYPE, TYPE *, TYPE & "package.for.other.classes"
%enddef

%define ANOTHER_PACKAGE_SPEC(TYPE...)
%typemap("javapackage") TYPE, TYPE *, TYPE & "package.for.another.set"
%enddef

OTHER_PACKAGE_SPEC(Package_2_class_one)
ANOTHER_PACKAGE_SPEC(Package_3_class_two)
/* etc */
```

The basic strategy here is to provide a default package typemap for the majority of the classes, only providing "javapackage" typemaps for the exceptions.

## 27.9 Typemap Examples

This section includes a few examples of typemaps. For more examples, you might look at the files "`java.swg`" and "`typemaps.i` " in the SWIG library.

### 27.9.1 Simpler Java enums for enums without initializers

The default Proper Java enums approach to wrapping enums is somewhat verbose. This is to handle all possible C/C++ enums, in particular enums with initializers. The generated code can be simplified if the enum being wrapped does not have any initializers.

The following shows how to remove the support methods that are generated by default and instead use the methods in the Java enum base class `java.lang.Enum` and `java.lang.Class` for marshalling enums between C/C++ and Java. The type used for the typemaps below is `enum SWIGTYPE` which is the default type used for all enums. The "enums.swg" file should be examined in order to see the original overridden versions of the typemaps.

```
%include "enums.swg"

%typemap(javain) enum SWIGTYPE "$javainput.ordinal()"
%typemap(javaout) enum SWIGTYPE {
    return $javaclassname.class.getEnumConstants()[$jnicall];
  }
%typemap(javabody) enum SWIGTYPE ""

%inline %{
  enum HairType { blonde, ginger, brunette };
  void setHair(HairType h);
  HairType getHair();
%}
```

SWIG will generate the following Java enum, which is somewhat simpler than the default:

```
public enum HairType {
  blonde,
  ginger,
  brunette;
}
```

and the two Java proxy methods will be:

```
public static void setHair(HairType h) {
  exampleJNI.setHair(h.ordinal());
}

public static HairType getHair() {
  return HairType.class.getEnumConstants()[exampleJNI.getHair()];
}
```

For marshalling Java enums to C/C++ enums, the `ordinal` method is used to convert the Java enum into an integer value for passing to the JNI layer, see the "javain" typemap. For marshalling C/C++ enums to Java enums, the C/C++ enum value is cast to an integer in the C/C++ typemaps (not shown). This integer value is then used to index into the array of enum constants that the Java language provides. See the `getEnumConstants` method in the "javaout" typemap.

These typemaps can often be used as the default for wrapping enums as in many cases there won't be any enum initializers. In fact a good strategy is to always use these typemaps and to specifically handle enums with initializers using %apply. This would be done by using the original versions of these typemaps in "enums.swg" under another typemap name for applying using %apply.

### 27.9.2 Handling C++ exception specifications as Java exceptions

This example demonstrates various ways in which C++ exceptions can be tailored and converted into Java exceptions. Let's consider a simple file class `SimpleFile` and an exception class `FileException` which it may throw on error:

```
%include "std_string.i" // for std::string typemaps
#include <string>

class FileException {
  std::string message;
public:
  FileException(const std::string& msg) : message(msg) {}
  std::string what() {
    return message;
  }
};

class SimpleFile {
  std::string filename;
public:
  SimpleFile(const std::string& filename) : filename(filename) {}
  void open() throw(FileException) {
  ...
  }
};
```

As the `open` method has a C++ exception specification, SWIG will parse this and know that the method can throw an exception. The "throws" typemap is then used when SWIG encounters an exception specification. The default generic "throws" typemap looks like this:

```
%typemap(throws) SWIGTYPE, SWIGTYPE &, SWIGTYPE *, SWIGTYPE [ANY] %{
  SWIG_JavaThrowException(jenv, SWIG_JavaRuntimeException,
                          "C++ $1_type exception thrown");
  return $null;
%}
```

Basically SWIG will generate a C++ try catch block and the body of the "throws" typemap constitutes the catch block. The above typemap calls a SWIG supplied method which throws a

`java.lang.RuntimeException`. This exception class is a runtime exception and therefore not a checked exception. If, however, we wanted to throw a checked exception, say `java.io.IOException`, then we could use the following typemap:

```
%typemap(throws, throws="java.io.IOException") FileException {
  jclass excep = jenv->FindClass("java/io/IOException");
  if (excep)
    jenv->ThrowNew(excep, $1.what().c_str());
  return $null;
}
```

Note that this typemap uses the 'throws' typemap attribute to ensure a throws clause is generated. The generated proxy method then specifies the checked exception by containing `java.io.IOException` in the throws clause:

```
public class SimpleFile {
  ...
  public void open() throws java.io.IOException { ... }
}
```

Lastly, if you don't want to map your C++ exception into one of the standard Java exceptions, the C++ class can be wrapped and turned into a custom Java exception class. If we go back to our example, the first thing we must do is get SWIG to wrap `FileException` and ensure that it derives from `java.lang.Exception`. Additionally, we might want to override the `java.lang.Exception.getMessage()` method. The typemaps to use then are as follows:

```
%typemap(javabase) FileException "java.lang.Exception"
%typemap(javacode) FileException %{
  public String getMessage() {
    return what();
  }
%}
```

This generates:

```
public class FileException extends java.lang.Exception {
  ...
  public String getMessage() {
    return what();
  }

  public FileException(String msg) { ... }

  public String what() {
    return exampleJNI.FileException_what(swigCPtr, this);
  }
}
```

We could alternatively have used `%rename` to rename `what()` into `getMessage()`.

### 27.9.3 NaN Exception - exception handling for a particular type

A Java exception can be thrown from any Java or JNI code. Therefore, as most typemaps contain either Java or JNI code, just about any typemap could throw an exception. The following example demonstrates exception handling on a type by type basis by checking for 'Not a number' (NaN) whenever a parameter of type `float` is wrapped.

Consider the following C++ code:

```
bool calculate(float first, float second);
```

To validate every `float` being passed to C++, we could precede the code being wrapped by the following typemap which throws a runtime exception whenever the `float` is 'Not a Number':

```
%module example
%typemap(javain) float "$module.CheckForNaN($javainput)"
%pragma(java) modulecode=%{
  /** Simply returns the input value unless it is not a number,
      whereupon an exception is thrown. */
  static protected float CheckForNaN(float num) {
    if (Float.isNaN(num))
      throw new RuntimeException("Not a number");
    return num;
  }
%}
```

Note that the `CheckForNaN` support method has been added to the module class using the `modulecode` pragma. The following shows the generated code of interest:

```
public class example {
  ...

  /** Simply returns the input value unless it is not a number,
      whereupon an exception is thrown. */
  static protected float CheckForNaN(float num) {
    if (Float.isNaN(num))
      throw new RuntimeException("Not a number");
    return num;
  }

  public static boolean calculate(float first, float second) {
    return exampleJNI.calculate(example.CheckForNaN(first), example.CheckForNaN(second));
  }
}
```

Note that the "javain" typemap is used for every occurrence of a `float` being used as an input. Of course, we could have targeted the typemap at a particular parameter by using `float first`, say, instead of just `float`.

The exception checking could alternatively have been placed into the 'pre' attribute that the "javain" typemap supports. The "javain" typemap above could be replaced with the following:

```
%typemap(javain, pre="    $module.CheckForNaN($javainput);") float "$javainput"
```

which would modify the `calculate` function to instead be generated as:

```
public class example {
  ...
  public static boolean calculate(float first, float second) {
    example.CheckForNaN(first);
    example.CheckForNaN(second);
    {
      return exampleJNI.calculate(first, second);
    }
  }
}
```

See the Date marshalling example for an example using further "javain" typemap attributes.

If we decide that what we actually want is a checked exception instead of a runtime exception, we can change this easily enough. The proxy method that uses `float` as an input, must then add the exception class to the throws clause. SWIG can handle this as it supports the 'throws' typemap attribute for specifying classes for the throws clause. Thus we can modify the pragma and the typemap for the throws clause:

```
%typemap(javain, throws="java.lang.Exception") float "$module.CheckForNaN($javainput)"
%pragma(java) modulecode=%{
  /** Simply returns the input value unless it is not a number,
      whereupon an exception is thrown. */
  static protected float CheckForNaN(float num) throws java.lang.Exception {
    if (Float.isNaN(num))
      throw new RuntimeException("Not a number");
    return num;
  }
%}
```

The `calculate` method now has a throws clause and even though the typemap is used twice for both `float first` and `float second`, the throws clause contains a single instance of `java.lang.Exception`:

```
public class example {
  ...

  /** Simply returns the input value unless it is not a number,
      whereupon an exception is thrown. */
  static protected float CheckForNaN(float num) throws java.lang.Exception {
    if (Float.isNaN(num))
      throw new RuntimeException("Not a number");
    return num;
  }

  public static boolean calculate(float first, float second) throws java.lang.Exception {
    return exampleJNI.calculate(example.CheckForNaN(first), example.CheckForNaN(second));
  }
}
```

If we were a martyr to the JNI cause, we could replace the succinct code within the "javain" typemap with a few pages of JNI code. If we had, we would have put it in the "in" typemap which, like all JNI and Java typemaps, also supports the 'throws' attribute.

### 27.9.4 Converting Java String arrays to char **

A common problem in many C programs is the processing of command line arguments, which are usually passed in an array of NULL terminated strings. The following SWIG interface file allows a Java String array to be used as a `char **` object.

```
%module example

/* This tells SWIG to treat char ** as a special case when used as a parameter
   in a function call */
%typemap(in) char ** (jint size) {
  int i = 0;
  size = (*jenv)->GetArrayLength(jenv, $input);
  $1 = (char **) malloc((size+1)*sizeof(char *));
  /* make a copy of each string */
  for (i = 0; i<size; i++) {
    jstring j_string = (jstring)(*jenv)->GetObjectArrayElement(jenv, $input, i);
    const char * c_string = (*jenv)->GetStringUTFChars(jenv, j_string, 0);
    $1[i] = malloc((strlen(c_string)+1)*sizeof(char));
    strcpy($1[i], c_string);
    (*jenv)->ReleaseStringUTFChars(jenv, j_string, c_string);
    (*jenv)->DeleteLocalRef(jenv, j_string);
  }
  $1[i] = 0;
}

/* This cleans up the memory we malloc'd before the function call */
%typemap(freearg) char ** {
  int i;
  for (i=0; i<size$argnum-1; i++)
    free($1[i]);
  free($1);
}

/* This allows a C function to return a char ** as a Java String array */
%typemap(out) char ** {
  int i;
  int len=0;
  jstring temp_string;
```

```
  const jclass clazz = (*jenv)->FindClass(jenv, "java/lang/String");

  while ($1[len]) len++;
  jresult = (*jenv)->NewObjectArray(jenv, len, clazz, NULL);
  /* exception checking omitted */

  for (i=0; i<len; i++) {
    temp_string = (*jenv)->NewStringUTF(jenv, *result++);
    (*jenv)->SetObjectArrayElement(jenv, jresult, i, temp_string);
    (*jenv)->DeleteLocalRef(jenv, temp_string);
  }
}

/* These 3 typemaps tell SWIG what JNI and Java types to use */
%typemap(jni) char ** "jobjectArray"
%typemap(jtype) char ** "String[]"
%typemap(jstype) char ** "String[]"

/* These 2 typemaps handle the conversion of the jtype to jstype typemap type
   and vice versa */
%typemap(javain) char ** "$javainput"
%typemap(javaout) char ** {
    return $jnicall;
  }

/* Now a few test functions */
%inline %{

int print_args(char **argv) {
  int i = 0;
  while (argv[i]) {
    printf("argv[%d] = %s\n", i, argv[i]);
    i++;
  }
  return i;
}

char **get_args() {
  static char *values[] = { "Dave", "Mike", "Susan", "John", "Michelle", 0};
  return &values[0];
}

%}
```

Note that the 'C' JNI calling convention is used. Checking for any thrown exceptions after JNI function calls has been omitted. When this module is compiled, our wrapped C functions can be used by the following Java program:

```
// File runme.java

public class runme {

  static {
    try {
      System.loadLibrary("example");
    } catch (UnsatisfiedLinkError e) {
      System.err.println("Native code library failed to load. " + e);
      System.exit(1);
    }
  }

  public static void main(String argv[]) {
    String animals[] = {"Cat", "Dog", "Cow", "Goat"};
    example.print_args(animals);
    String args[] = example.get_args();
    for (int i=0; i<args.length; i++)
      System.out.println(i + ":" + args[i]);
  }
}
```

When compiled and run we get:

```
$ java runme
argv[0] = Cat
argv[1] = Dog
argv[2] = Cow
argv[3] = Goat
0:Dave
1:Mike
2:Susan
3:John
4:Michelle
```

In the example, a few different typemaps are used. The "in" typemap is used to receive an input argument and convert it to a C array. Since dynamic memory allocation is used to allocate memory for the array, the "freearg" typemap is used to later release this memory after the execution of the C function. The "out" typemap is used for function return values. Lastly the "jni", "jtype" and "jstype" typemaps are also required to specify what Java types to use.

### 27.9.5 Expanding a Java object to multiple arguments

Suppose that you had a collection of C functions with arguments such as the following:

```
int foo(int argc, char **argv);
```

In the previous example, a typemap was written to pass a Java String array as the `char **argv`. This allows the function to be used from Java as follows:

```
example.foo(4, new String[]{"red", "green", "blue", "white"});
```

Although this works, it's a little awkward to specify the argument count. To fix this, a multi-argument typemap can be defined. This is not very difficult--you only have to make slight modifications to the previous example's typemaps:

```
%typemap(in) (int argc, char **argv) {
  int i = 0;
  $1 = (*jenv)->GetArrayLength(jenv, $input);
  $2 = (char **) malloc(($1+1)*sizeof(char *));
  /* make a copy of each string */
  for (i = 0; i<$1; i++) {
    jstring j_string = (jstring)(*jenv)->GetObjectArrayElement(jenv, $input, i);
    const char * c_string = (*jenv)->GetStringUTFChars(jenv, j_string, 0);
    $2[i] = malloc((strlen(c_string)+1)*sizeof(char));
    strcpy($2[i], c_string);
    (*jenv)->ReleaseStringUTFChars(jenv, j_string, c_string);
    (*jenv)->DeleteLocalRef(jenv, j_string);
  }
  $2[i] = 0;
}

%typemap(freearg) (int argc, char **argv) {
  int i;
  for (i=0; i<$1-1; i++)
    free($2[i]);
  free($2);
}

%typemap(jni) (int argc, char **argv) "jobjectArray"
%typemap(jtype) (int argc, char **argv) "String[]"
%typemap(jstype) (int argc, char **argv) "String[]"

%typemap(javain) (int argc, char **argv) "$javainput"
```

When writing a multiple-argument typemap, each of the types is referenced by a variable such as `$1` or `$2`. The typemap code simply fills in the appropriate values from the supplied Java parameter.

With the above typemap in place, you will find it no longer necessary to supply the argument count. This is automatically set by the typemap code. For example:

```
example.foo(new String[]{"red", "green", "blue", "white"});
```

### 27.9.6 Using typemaps to return arguments

A common problem in some C programs is that values may be returned in function parameters rather than in the return value of a function. The `typemaps.i` file defines INPUT, OUTPUT and INOUT typemaps which can be used to solve some instances of this problem. This library file uses an array as a means of moving data to and from Java when wrapping a C function that takes non const pointers or non const references as parameters.

Now we are going to outline an alternative approach to using arrays for C pointers. The INOUT typemap uses a `double[]` array for receiving and returning the `double*` parameters. In this approach we are able to use a Java class `myDouble` instead of `double[]` arrays where the C pointer `double*` is required.

Here is our example function:

```
/* Returns a status value and two values in out1 and out2 */
int spam(double a, double b, double *out1, double *out2);
```

If we define a structure `MyDouble` containing a `double` member variable and use some typemaps we can solve this problem. For example we could put the following through SWIG:

```
%module example

/* Define a new structure to use instead of double * */
%inline %{
typedef struct {
  double value;
} MyDouble;
%}


%{
/* Returns a status value and two values in out1 and out2 */
int spam(double a, double b, double *out1, double *out2) {
  int status = 1;
  *out1 = a*10.0;
  *out2 = b*100.0;
  return status;
}
%}

/*
This typemap will make any double * function parameters with name OUTVALUE take an
argument of MyDouble instead of double *. This will
allow the calling function to read the double * value after returning from the function.
*/
%typemap(in) double *OUTVALUE {
  jclass clazz = jenv->FindClass("MyDouble");
  jfieldID fid = jenv->GetFieldID(clazz, "swigCPtr", "J");
  jlong cPtr = jenv->GetLongField($input, fid);
  MyDouble *pMyDouble = NULL;
  *(MyDouble **)&pMyDouble = *(MyDouble **)&cPtr;
  $1 = &pMyDouble->value;
}

%typemap(jtype) double *OUTVALUE "MyDouble"
%typemap(jstype) double *OUTVALUE "MyDouble"
%typemap(jni) double *OUTVALUE "jobject"

%typemap(javain) double *OUTVALUE "$javainput"
```

```
/* Now we apply the typemap to the named variables */
%apply double *OUTVALUE { double *out1, double *out2 };
int spam(double a, double b, double *out1, double *out2);
```

Note that the C++ JNI calling convention has been used this time and so must be compiled as C++ and the `-c++` commandline must be passed to SWIG. JNI error checking has been omitted for clarity.

What the typemaps do are make the named `double*` function parameters use our new `MyDouble` wrapper structure. The "in" typemap takes this structure, gets the C++ pointer to it, takes the `double value` member variable and passes it to the C++`spam` function. In Java, when the function returns, we use the SWIG created `getValue()` function to get the output value. The following Java program demonstrates this:

```
// File: runme.java

public class runme {

  static {
    try {
      System.loadLibrary("example");
    } catch (UnsatisfiedLinkError e) {
      System.err.println("Native code library failed to load. " + e);
      System.exit(1);
    }
  }

  public static void main(String argv[]) {
    MyDouble out1 = new MyDouble();
    MyDouble out2 = new MyDouble();
    int ret = example.spam(1.2, 3.4, out1, out2);
    System.out.println(ret + "  " + out1.getValue() + "  " + out2.getValue());
  }
}
```

When compiled and run we get:

```
$ java runme
1 12.0  340.0
```

### 27.9.7 Adding Java downcasts to polymorphic return types

SWIG support for polymorphism works in that the appropriate virtual function is called. However, the default generated code does not allow for downcasting. Let's examine this with the following code:

```
%include "std_string.i"

#include <iostream>
using namespace std;
class Vehicle {
public:
  virtual void start() = 0;
...
};

class Ambulance : public Vehicle {
  string vol;
public:
  Ambulance(string volume) : vol(volume) {}
  virtual void start() {
    cout << "Ambulance started" << endl;
  }
  void sound_siren() {
    cout << vol << " siren sounded!" << endl;
  }
...
};

Vehicle *vehicle_factory() {
  return new Ambulance("Very loud");
}
```

If we execute the following Java code:

```
Vehicle vehicle = example.vehicle_factory();
vehicle.start();

Ambulance ambulance = (Ambulance)vehicle;
ambulance.sound_siren();
```

We get:

```
Ambulance started
java.lang.ClassCastException
        at runme.main(runme.java:16)
```

Even though we know from examination of the C++ code that `vehicle_factory` returns an object of type `Ambulance`, we are not able to use this knowledge to perform the downcast in Java. This occurs because the runtime type information is not completely passed from C++ to Java when returning the type from `vehicle_factory()`. Usually this is not a problem as virtual functions do work by default, such as in the case of `start()`. There are a few solutions to getting downcasts to work.

The first is not to use a Java cast but a call to C++ to make the cast. Add this to your code:

```
%exception Ambulance::dynamic_cast(Vehicle *vehicle) {
```

```
  $action
    if (!result) {
      jclass excep = jenv->FindClass("java/lang/ClassCastException");
      if (excep) {
        jenv->ThrowNew(excep, "dynamic_cast exception");
      }
    }
}
%extend Ambulance {
  static Ambulance *dynamic_cast(Vehicle *vehicle) {
    return dynamic_cast<Ambulance *>(vehicle);
  }
};
```

It would then be used from Java like this

```
Ambulance ambulance = Ambulance.dynamic_cast(vehicle);
ambulance.sound_siren();
```

Should `vehicle` not be of type `ambulance` then a Java `ClassCastException` is thrown. The next solution is a purer solution in that Java downcasts can be performed on the types. Add the following before the definition of `vehicle_factory`:

```
%typemap(out) Vehicle * {
  Ambulance *downcast = dynamic_cast<Ambulance *>($1);
  *(Ambulance **)&$result = downcast;
}

%typemap(javaout) Vehicle * {
    return new Ambulance($jnicall, $owner);
  }
```

Here we are using our knowledge that `vehicle_factory` always returns type `Ambulance` so that the Java proxy is created as a type `Ambulance`. If `vehicle_factory` can manufacture any type of `Vehicle` and we want to be able to downcast using Java casts for any of these types, then a different approach is needed. Consider expanding our example with a new Vehicle type and a more flexible factory function:

```
class FireEngine : public Vehicle {
public:
  FireEngine() {}
  virtual void start() {
    cout << "FireEngine started" << endl;
  }
  void roll_out_hose() {
    cout << "Hose rolled out" << endl;
  }
  ...
};
Vehicle *vehicle_factory(int vehicle_number) {
  if (vehicle_number == 0)
    return new Ambulance("Very loud");
  else
    return new FireEngine();
}
```

To be able to downcast with this sort of Java code:

```
FireEngine fireengine = (FireEngine)example.vehicle_factory(1);
fireengine.roll_out_hose();
Ambulance ambulance = (Ambulance)example.vehicle_factory(0);
ambulance.sound_siren();
```

the following typemaps targeted at the `vehicle_factory` function will achieve this. Note that in this case, the Java class is constructed using JNI code rather than passing a pointer across the JNI boundary in a Java long for construction in Java code.

```
%typemap(jni) Vehicle *vehicle_factory "jobject"
%typemap(jtype) Vehicle *vehicle_factory "Vehicle"
%typemap(jstype) Vehicle *vehicle_factory "Vehicle"
%typemap(javaout) Vehicle *vehicle_factory {
    return $jnicall;
  }

%typemap(out) Vehicle *vehicle_factory {
  Ambulance *ambulance = dynamic_cast<Ambulance *>($1);
  FireEngine *fireengine = dynamic_cast<FireEngine *>($1);
  if (ambulance) {
    // call the Ambulance(long cPtr, boolean cMemoryOwn) constructor
    jclass clazz = jenv->FindClass("Ambulance");
    if (clazz) {
      jmethodID mid = jenv->GetMethodID(clazz, "<init>", "(JZ)V");
      if (mid) {
        jlong cptr = 0;
        *(Ambulance **)&cptr = ambulance;
        $result = jenv->NewObject(clazz, mid, cptr, false);
      }
    }
  } else if (fireengine) {
    // call the FireEngine(long cPtr, boolean cMemoryOwn) constructor
    jclass clazz = jenv->FindClass("FireEngine");
    if (clazz) {
      jmethodID mid = jenv->GetMethodID(clazz, "<init>", "(JZ)V");
      if (mid) {
        jlong cptr = 0;
        *(FireEngine **)&cptr = fireengine;
        $result = jenv->NewObject(clazz, mid, cptr, false);
```

```
      }
    }
  }
  else {
    cout << "Unexpected type " << endl;
  }

  if (!$result)
    cout << "Failed to create new java object" << endl;
}
```

Better error handling would need to be added into this code. There are other solutions to this problem, but this last example demonstrates some more involved JNI code. SWIG usually generates code which constructs the proxy classes using Java code as it is easier to handle error conditions and is faster. Note that the JNI code above uses a number of string lookups to call a constructor, whereas this would not occur using byte compiled Java code.

### 27.9.8 Adding an equals method to the Java classes

When a pointer is returned from a JNI function, it is wrapped using a new Java proxy class or type wrapper class. Even when the pointers are the same, it will not be possible to know that the two Java classes containing those pointers are actually the same object. It is common in Java to use the `equals()` method to check whether two objects are equivalent. The `equals()` method is usually accompanied by a `hashCode()` method in order to fulfill the requirement that the hash code is equal for equal objects. Pure Java code methods like these can be easily added:

```
%typemap(javacode) SWIGTYPE %{
  public boolean equals(Object obj) {
    boolean equal = false;
    if (obj instanceof $javaclassname)
      equal = (($javaclassname)obj).swigCPtr == this.swigCPtr);
    return equal;
  }
  public int hashCode() {
    return (int)getPointer();
  }
%}

class Foo { };
Foo* returnFoo(Foo *foo) { return foo; }
```

The following would display `false` without the `javacode` typemap above. With the typemap defining the `equals` method the result is `true`.

```
Foo foo1 = new Foo();
Foo foo2 = example.returnFoo(foo1);
System.out.println("foo1? " + foo1.equals(foo2));
```

### 27.9.9 Void pointers and a common Java base class

One might wonder why the common code that SWIG emits for the proxy and type wrapper classes is not pushed into a base class. The reason is that although `swigCPtr` could be put into a common base class for all classes wrapping C structures, it would not work for C++ classes involved in an inheritance chain. Each class derived from a base needs a separate `swigCPtr` because C++ compilers sometimes use a different pointer value when casting a derived class to a base. Additionally as Java only supports single inheritance, it would not be possible to derive wrapped classes from your own pure Java classes if the base class has been 'used up' by SWIG. However, you may want to move some of the common code into a base class. Here is an example which uses a common base class for all proxy classes and type wrapper classes:

```
%typemap(javabase) SWIGTYPE, SWIGTYPE *, SWIGTYPE &, SWIGTYPE [],
                                                    SWIGTYPE (CLASS::*) "SWIG"

%typemap(javacode) SWIGTYPE, SWIGTYPE *, SWIGTYPE &, SWIGTYPE [],
                                                    SWIGTYPE (CLASS::*) %{
  protected long getPointer() {
    return swigCPtr;
  }
%}
```

Define new base class called SWIG:

```
public abstract class SWIG {
  protected abstract long getPointer();

  public boolean equals(Object obj) {
    boolean equal = false;
    if (obj instanceof SWIG)
      equal = (((SWIG)obj).getPointer() == this.getPointer());
    return equal;
  }

  SWIGTYPE_p_void getVoidPointer() {
    return new SWIGTYPE_p_void(getPointer(), false);
  }
}
```

This example contains some useful functionality which you may want in your code.

- It has an `equals()` method. Unlike the previous example, the method code isn't replicated in all classes.
- It also has a function which effectively implements a cast from the type of the proxy/type wrapper class to a void pointer. This is necessary for passing a proxy class or a type wrapper class to a function that takes a void pointer.

### 27.9.10 Struct pointer to pointer

Pointers to pointers are often used as output parameters in C factory type functions. These are a bit more tricky to handle. Consider the following situation where a `Butler` can be hired and fired:

```
typedef struct {
  int hoursAvailable;
  char *greeting;
} Butler;
```

```
// Note: HireButler will allocate the memory
// The caller must free the memory by calling FireButler()!!
extern int HireButler(Butler **ppButler);
extern void FireButler(Butler *pButler);
```

C code implementation:

```
int HireButler(Butler **ppButler) {
  Butler *pButler = (Butler *)malloc(sizeof(Butler));
  pButler->hoursAvailable = 24;
  pButler->greeting = (char *)malloc(32);
  strcpy(pButler->greeting, "At your service Sir");
  *ppButler = pButler;
  return 1;
}
void FireButler(Butler *pButler) {
  free(pButler->greeting);
  free(pButler);
}
```

Let's take two approaches to wrapping this code. The first is to provide a functional interface, much like the original C interface. The following Java code shows how we intend the code to be used:

```
Butler jeeves = new Butler();
example.HireButler(jeeves);
System.out.println("Greeting:     " + jeeves.getGreeting());
System.out.println("Availability: " + jeeves.getHoursAvailable() + " hours per day");
example.FireButler(jeeves);
```

Resulting in the following output when run:

```
Greeting:     At your service Sir
Availability: 24 hours per day
```

Note the usage is very much like it would be used if we were writing C code, that is, explicit memory management is needed. No C memory is allocated in the construction of the `Butler` proxy class and the proxy class will not destroy the underlying C memory when it is collected. A number of typemaps and features are needed to implement this approach. The following interface file code should be placed before SWIG parses the above C code.

```
%module example

// Do not generate the default proxy constructor or destructor
%nodefaultctor Butler;
%nodefaultdtor Butler;

// Add in pure Java code proxy constructor
%typemap(javacode) Butler %{
  /** This constructor creates the proxy which initially does not create nor own any C memory */
  public Butler() {
    this(0, false);
  }
%}

// Type typemaps for marshalling Butler **
%typemap(jni) Butler ** "jobject"
%typemap(jtype) Butler ** "Butler"
%typemap(jstype) Butler ** "Butler"

// Typemaps for Butler ** as a parameter output type
%typemap(in) Butler ** (Butler *ppButler = 0) %{
  $1 = &ppButler;
%}
%typemap(argout) Butler ** {
  // Give Java proxy the C pointer (of newly created object)
  jclass clazz = (*jenv)->FindClass(jenv, "Butler");
  jfieldID fid = (*jenv)->GetFieldID(jenv, clazz, "swigCPtr", "J");
  jlong cPtr = 0;
  *(Butler **)&cPtr = *$1;
  (*jenv)->SetLongField(jenv, $input, fid, cPtr);
}
%typemap(javain) Butler ** "$javainput"
```

Note that the JNI code sets the proxy's `swigCPtr` member variable to point to the newly created object. The `swigCMemOwn` remains unchanged (at false), so that the proxy does not own the memory.

Note: The old %nodefault directive disabled the default constructor and destructor at the same time. This is unsafe in most of the cases, and you can use the explicit %nodefaultctor and %nodefaultdtor directives to achieve the same result if needed.

The second approach offers a more object oriented interface to the Java user. We do this by making the Java proxy class's constructor call the `HireButler()` method to create the underlying C object. Additionally we get the proxy to take ownership of the memory so that the finalizer will call the `FireButler()` function. The proxy class will thus take ownership of the memory and clean it up when no longer needed. We will also prevent the user from being able to explicitly call the `HireButler()` and `FireButler()` functions. Usage from Java will simply be:

```
Butler jeeves = new Butler();
System.out.println("Greeting:     " + jeeves.getGreeting());
System.out.println("Availability: " + jeeves.getHoursAvailable() + " hours per day");
```

Note that the Butler class is used just like any other Java class and no extra coding by the user needs to be written to clear up the underlying C memory as the finalizer will be called by the garbage collector which in turn will call the `FireButler()` function. To implement this, we use the above interface file code but remove the `javacode` typemap and add the following:

```
// Don't expose the memory allocation/de-allocation functions
%ignore FireButler(Butler *pButler);
```

```
%ignore HireButler(Butler **ppButler);

// Add in a custom proxy constructor and destructor
%extend Butler {
  Butler() {
    Butler *pButler = 0;
    HireButler(&pButler);
    return pButler;
  }
  ~Butler() {
    FireButler($self);
  }
}
```

Note that the code in `%extend` is using a C++ type constructor and destructor, yet the generated code will still compile as C code, see Adding member functions to C structures. The C functional interface has been completely morphed into an object-oriented interface and the Butler class would behave much like any pure Java class and feel more natural to Java users.

### 27.9.11 Memory management when returning references to member variables

This example shows how to prevent premature garbage collection of objects when the underlying C++ class returns a pointer or reference to a member variable.

Consider the following C++ code:

```
struct Wheel {
  int size;
  Wheel(int sz = 0) : size(sz) {}
};

class Bike {
  Wheel wheel;
public:
  Bike(int val) : wheel(val) {}
  Wheel& getWheel() { return wheel; }
};
```

and the following usage from Java after running the code through SWIG:

```
Wheel wheel = new Bike(10).getWheel();
System.out.println("wheel size: " + wheel.getSize());
// Simulate a garbage collection
System.gc();
System.runFinalization();
System.out.println("wheel size: " + wheel.getSize());
```

Don't be surprised that if the resulting output gives strange results such as...

```
wheel size: 10
wheel size: 135019664
```

What has happened here is the garbage collector has collected the `Bike` instance as it doesn't think it is needed any more. The proxy instance, `wheel`, contains a reference to memory that was deleted when the `Bike` instance was collected. In order to prevent the garbage collector from collecting the `Bike` instance a reference to the `Bike` must be added to the`wheel` instance. You can do this by adding the reference when the `getWheel()` method is called using the following typemaps.

```
%typemap(javacode) Wheel %{
  // Ensure that the GC doesn't collect any Bike instance set from Java
  private Bike bikeReference;
  protected void addReference(Bike bike) {
    bikeReference = bike;
  }
%}

// Add a Java reference to prevent premature garbage collection and resulting use
// of dangling C++ pointer. Intended for methods that return pointers or
// references to a member variable.
%typemap(javaout) Wheel& getWheel {
  long cPtr = $jnicall;
  $javaclassname ret = null;
  if (cPtr != 0) {
    ret = new $javaclassname(cPtr, $owner);
    ret.addReference(this);
  }
  return ret;
  }
```

The code in the first typemap gets added to the `Wheel` proxy class. The code in the second typemap constitutes the bulk of the code in the generated `getWheel()` function:

```
public class Wheel {
  ...
  // Ensure that the GC doesn't collect any bike set from Java
  private Bike bikeReference;
  protected void addReference(Bike bike) {
    bikeReference = bike;
  }
}

public class Bike {
  ...
  public Wheel getWheel() {
    long cPtr = exampleJNI.Bike_getWheel(swigCPtr, this);
    Wheel ret = null;
    if (cPtr != 0) {
      ret = new Wheel(cPtr, false);
      ret.addReference(this);
```

```
    }
    return ret;
  }
}
```

Note the `addReference` call.

### 27.9.12 Memory management for objects passed to the C++ layer

Managing memory can be tricky when using C++ and Java proxy classes. The previous example shows one such case and this example looks at memory management for a class passed to a C++ method which expects the object to remain in scope after the function has returned. Consider the following two C++ classes:

```
struct Element {
  int value;
  Element(int val) : value(val) {}
};
class Container {
  Element* element;
public:
  Container() : element(0) {}
  void setElement(Element* e) { element = e; }
  Element* getElement() { return element; }
};
```

and usage from C++

```
Container container;
Element element(20);
container.setElement(&element);
cout << "element.value: " << container.getElement()->value << endl;
```

and more or less equivalent usage from Java

```
Container container = new Container();
container.setElement(new Element(20));
System.out.println("element value: " + container.getElement().getValue());
```

The C++ code will always print out 20, but the value printed out may not be this in the Java equivalent code. In order to understand why, consider a garbage collection occurring...

```
Container container = new Container();
container.setElement(new Element(20));
// Simulate a garbage collection
System.gc();
System.runFinalization();
System.out.println("element value: " + container.getElement().getValue());
```

The temporary element created with `new Element(20)` could get garbage collected which ultimately means the `container` variable is holding a dangling pointer, thereby printing out any old random value instead of the expected value of 20. One solution is to add in the appropriate references in the Java layer...

```
public class Container {

  ...

  // Ensure that the GC doesn't collect any Element set from Java
  // as the underlying C++ class stores a shallow copy
  private Element elementReference;

  public void setElement(Element e) {
    exampleJNI.Container_setElement(swigCPtr, this, Element.getCPtr(e), e);
    elementReference = e;
  }
}
```

The following typemaps can be used to generate this code:

```
%typemap(javacode) Container %{
  // Ensure that the GC doesn't collect any element set from Java
  // as the underlying C++ class stores a shallow copy
  private Element elementReference;
%}

%typemap(javain,
         post="      elementReference = $javainput;"
         ) Element *e "Element.getCPtr($javainput)"
```

The 'javacode' typemap simply adds in the specified code into the Java proxy class. The 'javain' typemap matches the input parameter type and name for the `setElement` method and the 'post' typemap attribute allows adding code after the JNI call. The 'post' code is generated into a finally block after the JNI call so the resulting code isn't quite as mentioned earlier, `setElement` is actually:

```
  public void setElement(Element e) {
    try {
      exampleJNI.Container_setElement(swigCPtr, this, Element.getCPtr(e), e);
    } finally {
      elementReference = e;
    }
  }
```

### 27.9.13 Date marshalling using the javain typemap and associated attributes

The NaN Exception example is a simple example of the "javain" typemap and its 'pre' attribute. This example demonstrates how a C++ date class, say `CDate`, can be mapped onto the standard Java date class, `java.util.GregorianCalendar` by using the 'pre', 'post' and 'pgcppname' attributes of the "javain" typemap. The idea is that the `GregorianCalendar` is used wherever the C++ API uses a `CDate` . Let's assume the code being wrapped is as follows:

```
class CDate {
public:
  CDate(int year, int month, int day);
  int getYear();
  int getMonth();
  int getDay();
  ...
};
struct Action {
  static int doSomething(const CDate &dateIn, CDate &dateOut);
  Action(const CDate &date, CDate &dateOut);
};
```

Note that `dateIn` is const and therefore read only and `dateOut` is a non-const output type.

First let's look at the code that is generated by default, where the Java proxy class `CDate` is used in the proxy interface:

```
public class Action {
  ...
  public static int doSomething(CDate dateIn, CDate dateOut) {
    return exampleJNI.Action_doSomething(CDate.getCPtr(dateIn), dateIn,
                                         CDate.getCPtr(dateOut), dateOut);
  }

  public Action(CDate date, CDate dateOut) {
    this(exampleJNI.new_Action(CDate.getCPtr(date), date,
                               CDate.getCPtr(dateOut), dateOut), true);
  }
}
```

The `CDate &` and `const CDate &` Java code is generated from the following two default typemaps:

```
%typemap(jstype) SWIGTYPE & "$javaclassname"
%typemap(javain) SWIGTYPE & "$javaclassname.getCPtr($javainput)"
```

where '$javaclassname' is translated into the proxy class name, `CDate` and '$javainput' is translated into the name of the parameter, eg `dateIn`. From Java, the intention is then to call into a modified API with something like:

```
java.util.GregorianCalendar calendarIn =
  new java.util.GregorianCalendar(2011, java.util.Calendar.APRIL, 13, 0, 0, 0);
java.util.GregorianCalendar calendarOut = new java.util.GregorianCalendar();

// Note in calls below, calendarIn remains unchanged and calendarOut
// is set to a new value by the C++ call
Action.doSomething(calendarIn, calendarOut);
Action action = new Action(calendarIn, calendarOut);
```

To achieve this mapping, we need to alter the default code generation slightly so that at the Java layer, a `GregorianCalendar` is converted into a `CDate`. The JNI intermediary layer will still take a pointer to the underlying `CDate` class. The typemaps to achieve this are shown below.

```
%typemap(jstype) const CDate& "java.util.GregorianCalendar"
%typemap(javain,
         pre="    CDate temp$javainput = new CDate($javainput.get(java.util.Calendar.YEAR), "
             "$javainput.get(java.util.Calendar.MONTH), $javainput.get(java.util.Calendar.DATE));",
         pgcppname="temp$javainput") const CDate &
         "$javaclassname.getCPtr(temp$javainput)"

%typemap(jstype) CDate& "java.util.Calendar"
%typemap(javain,
         pre="    CDate temp$javainput = new CDate($javainput.get(java.util.Calendar.YEAR), "
             "$javainput.get(java.util.Calendar.MONTH), $javainput.get(java.util.Calendar.DATE));",
         post="      $javainput.set(temp$javainput.getYear(), temp$javainput.getMonth(), "
              "temp$javainput.getDay(), 0, 0, 0);",
         pgcppname="temp$javainput") CDate &
         "$javaclassname.getCPtr(temp$javainput)"
```

The resulting generated proxy code in the `Action` class follows:

```
public class Action {
  ...
  public static int doSomething(java.util.GregorianCalendar dateIn,
                                java.util.Calendar dateOut) {
    CDate tempdateIn = new CDate(dateIn.get(java.util.Calendar.YEAR),
                                 dateIn.get(java.util.Calendar.MONTH),
                                 dateIn.get(java.util.Calendar.DATE));
    CDate tempdateOut = new CDate(dateOut.get(java.util.Calendar.YEAR),
                                  dateOut.get(java.util.Calendar.MONTH),
                                  dateOut.get(java.util.Calendar.DATE));
    try {
      return exampleJNI.Action_doSomething(CDate.getCPtr(tempdateIn), tempdateIn,
                                           CDate.getCPtr(tempdateOut), tempdateOut);
    } finally {
      dateOut.set(tempdateOut.getYear(), tempdateOut.getMonth(), tempdateOut.getDay(), 0, 0, 0);
    }
  }

  static private long SwigConstructAction(java.util.GregorianCalendar date,
                                          java.util.Calendar dateOut) {
```

```
    CDate tempdate = new CDate(date.get(java.util.Calendar.YEAR),
                               date.get(java.util.Calendar.MONTH),
                               date.get(java.util.Calendar.DATE));
    CDate tempdateOut = new CDate(dateOut.get(java.util.Calendar.YEAR),
                                  dateOut.get(java.util.Calendar.MONTH),
                                  dateOut.get(java.util.Calendar.DATE));
    try {
      return exampleJNI.new_Action(CDate.getCPtr(tempdate), tempdate,
                                   CDate.getCPtr(tempdateOut), tempdateOut);
    } finally {
      dateOut.set(tempdateOut.getYear(), tempdateOut.getMonth(), tempdateOut.getDay(), 0, 0, 0);
    }
  }

  public Action(java.util.GregorianCalendar date, java.util.Calendar dateOut) {
    this(Action.SwigConstructAction(date, dateOut), true);
  }
}
```

A few things to note:

- The "javatype" typemap has changed the parameter type to `java.util.GregorianCalendar` or `java.util.Calendar` instead of the default generated `CDate` proxy.
- The code in the 'pre' attribute appears before the JNI call ( `exampleJNI.new_Action` / `exampleJNI.Action_doSomething`).
- The code in the 'post' attribute appears after the JNI call.
- A try .. finally block is generated with the JNI call in the try block and 'post' code in the finally block. The alternative of just using a temporary variable for the return value from the JNI call and the 'post' code being generated before the return statement is not possible given that the JNI call is in one line and comes from the "javaout" typemap.
- The temporary variables in the "javain" typemaps are called `temp$javain`, where "$javain" is replaced with the parameter name. "$javain" is used to mangle the variable name so that more than one `CDate &` type can be used as a parameter in a method, otherwise two or more local variables with the same name would be generated.
- The use of the "javain" typemap causes a constructor helper function (`SwigConstructAction`) to be generated. This allows Java code to be called before the JNI call and is required as the Java compiler won't compile code inserted before the 'this' call.
- The 'pgcppname' attribute is used to modify the object being passed as the premature garbage collection prevention parameter (the 2nd and 4th parameters in the JNI calls).

## 27.10 Living with Java Directors

This section is intended to address frequently asked questions and frequently encountered problems when using Java directors.

1. *When my program starts up, it complains that* method_foo *cannot be found in a Java method called* swig_module_init. *How do I fix this?*

   Open up the C++ wrapper source code file and look for `"method_foo"` (include the double quotes, they are important!) Look at the JNI field descriptor and make sure that each class that occurs in the descriptor has the correct package name in front of it. If the package name is incorrect, put a "javapackage" typemap in your SWIG interface file.

2. *I'm compiling my code and I'm using templates. I provided a javapackage typemap, but SWIG doesn't generate the right JNI field descriptor.*

   Use the template's renamed name as the argument to the "javapackage" typemap:

   ```
   %typemap(javapackage)  std::vector<int>  "your.package.here"
   %template(VectorOfInt) std::vector<int>;
   ```

3. *When I pass class pointers or references through a C++ upcall and I try to type cast them, Java complains with a ClassCastException. What am I doing wrong?*

   Normally, a non-director generated Java proxy class creates temporary Java objects as follows:

   ```
   public static void MyClass_method_upcall(MyClass self, long jarg1)
   {
     Foo darg1 = new Foo(jarg1, false);

     self.method_upcall(darg1);
   }
   ```

   Unfortunately, this loses the Java type information that is part of the underlying Foo director proxy class's Java object pointer causing the type cast to fail. The SWIG Java module's director code attempts to correct the problem, **but only for director-enabled classes**, since the director class retains a global reference to its Java object. Thus, for director-enabled classes **and only for director-enabled classes**, the generated proxy Java code looks something like:

   ```
   public static void MyClass_method_upcall(MyClass self, long jarg1,
                                            Foo jarg1_object)
   {
     Foo darg1 = (jarg1_object != null ? jarg1_object : new Foo(jarg1, false));

     self.method_upcall(darg1);
   }
   ```

   When you import a SWIG interface file containing class definitions, the classes you want to be director-enabled must be have the `feature("director")` enabled for type symmetry to work. This applies even when the class being wrapped isn't a director-enabled class but takes parameters that are director-enabled classes.

   The current "type symmetry" design will work for simple C++ inheritance, but will most likely fail for anything more complicated such as tree or diamond C++ inheritance hierarchies. Those who are interested in challenging problems are more than welcome to hack the `Java::Java_director_declaration` method in `Source/Modules/java.cxx`.

   If all else fails, you can use the downcastXXXXX() method to attempt to recover the director class's Java object pointer. For the Java Foo proxy class, the Foo director class's java object pointer can be accessed through the javaObjectFoo() method. The generated method's signature is:

   ```
   public static Foo javaObjectFoo(Foo obj);
   ```

   From your code, this method is invoked as follows:

   ```
   public class MyClassDerived {
     public void method_upcall(Foo foo_object)
     {
       FooDerived    derived = (foo_object != null ?
                   (FooDerived) Foo.downcastFoo(foo_object) : null);
       /* rest of your code here */
     }
   }
   ```

An good approach for managing downcasting is placing a static method in each derived class that performs the downcast from the superclass, e.g.,

```
public class FooDerived extends Foo {
  /* ... */
  public static FooDerived downcastFooDerived(Foo foo_object)
  {
    try {
      return foo_object != null ? (FooDerived) Foo.downcastFoo(foo_object);
    }

    catch (ClassCastException exc) {
      // Wasn't a FooDerived object, some other subclass of Foo
      return null;
    }
  }
}
```

Then change the code in MyClassDerived as follows:

```
public class MyClassDerived extends MyClass {
  /* ... */
  public void method_upcall(Foo foo_object)
  {
    FooDerived    derived = FooDerived.downcastFooDerived(foo_object);
    /* rest of your code here */
  }
}
```

4. *Why isn't the proxy class declared abstract? Why aren't the director upcall methods in the proxy class declared abstract?*

   Declaring the proxy class and its methods abstract would break the JNI argument marshalling and SWIG's downcall functionality (going from Java to C++.) Create an abstract Java subclass that inherits from the director-enabled class instead. Using the previous Foo class example:

```
public abstract class UserVisibleFoo extends Foo {
  /** Make sure user overrides this method, it's where the upcall
   * happens.
   */
  public abstract void method_upcall(Foo foo_object);

  /// Downcast from Foo to UserVisibleFoo
  public static UserVisibleFoo downcastUserVisibleFoo(Foo foo_object)
  {
    try {
      return foo_object != null ? (FooDerived) Foo.downcastFoo(foo_object) : null;
    } catch (ClassCastException exc) {
      // Wasn't a FooDerived object, some other subclass of Foo
      return null;
    }
  }
}
```

   This doesn't prevent the user from creating subclasses derived from Foo, however, UserVisibleFoo provides the safety net that reminds the user to override the `method_upcall()` method.

## 27.11 Odds and ends

### 27.11.1 JavaDoc comments

SWIG can translate Doxygen comments in the C/C++ headers being wrapped to JavaDoc. For details of this, see the Doxygen to Javadoc section.

If you don't have Doxygen comments to translate then it's possible to add JavaDoc comments from your interface file. The `%javamethodmodifiers` feature can be used for adding proxy class method comments and module class method comments. The "javaimports" typemap can be hijacked for adding in proxy class JavaDoc comments. The `jniclassimports` or `jniclassclassmodifiers` pragmas can also be used for adding intermediary JNI class comments and likewise the `moduleimports` or `moduleclassmodifiers` pragmas for the module class. Here is an example adding in a proxy class and method comment:

```
%javamethodmodifiers Barmy::lose_marbles() "
  /**
    * Calling this method will make you mad.
    * Use with <b>utmost</b> caution.
    */
  public";

%typemap(javaimports) Barmy "
/** The crazy class. Use as a last resort. */"

class Barmy {
public:
  void lose_marbles() {}
};
```

Note the "public" added at the end of the `%javamethodmodifiers` as this is the default for this feature. The generated proxy class with JavaDoc comments is then as follows:

```
/** The crazy class. Use as a last resort. */
public class Barmy {
...
  /**
    * Calling this method will make you mad.
    * Use with <b>utmost</b> caution.
    */
  public void lose_marbles() {
    ...
  }
...
```

```
}
```

### 27.11.2 Functional interface without proxy classes

It is possible to run SWIG in a mode that does not produce proxy classes by using the -noproxy commandline option. The interface is rather primitive when wrapping structures or classes and is accessed through function calls to the module class. All the functions in the module class are wrapped by functions with identical names as those in the intermediary JNI class.

Consider the example we looked at when examining proxy classes:

```
class Foo {
public:
  int x;
  int spam(int num, Foo* foo);
};
```

When using `-noproxy`, type wrapper classes are generated instead of proxy classes. Access to all the functions and variables is through a C like set of functions where the first parameter passed is the pointer to the class, that is an instance of a type wrapper class. Here is what the module class looks like:

```
public class example {
  public static void Foo_x_get(SWIGTYPE_p_Foo self, int x) {...}
  public static int Foo_x_get(SWIGTYPE_p_Foo self) {...}
  public static int Foo_spam(SWIGTYPE_p_Foo self, int num, SWIGTYPE_p_Foo foo) {...}
  public static SWIGTYPE_p_Foo new_Foo() {...}
  public static void delete_Foo(SWIGTYPE_p_Foo self) {...}
}
```

This approach is not nearly as natural as using proxy classes as the functions need to be used like this:

```
SWIGTYPE_p_Foo foo = example.new_Foo();
example.Foo_x_set(foo, 10);
int var = example.Foo_x_get(foo);
example.Foo_spam(foo, 20, foo);
example.delete_Foo(foo);
```

Unlike proxy classes, there is no attempt at tracking memory. All destructors have to be called manually for example the `delete_Foo(foo)` call above.

### 27.11.3 Using your own JNI functions

You may have some hand written JNI functions that you want to use in addition to the SWIG generated JNI functions. Adding these to your SWIG generated package is possible using the `%native` directive. If you don't want SWIG to wrap your JNI function then of course you can simply use the `%ignore` directive. However, if you want SWIG to generate just the Java code for a JNI function then use the `%native` directive. The C types for the parameters and return type must be specified in place of the JNI types and the function name must be the native method name. For example:

```
%native (HandRolled) void HandRolled(int, char *);
%{
JNIEXPORT void JNICALL Java_packageName_moduleName_HandRolled(JNIEnv *, jclass,
                                                              jlong, jstring);
%}
```

No C JNI function will be generated and the `Java_packageName_moduleName_HandRolled` function will be accessible using the SWIG generated Java native method call in the intermediary JNI class which will look like this:

```
  public final static native void HandRolled(int jarg1, String jarg2);
```

and as usual this function is wrapped by another which for a global C function would appear in the module class:

```
  public static void HandRolled(int arg0, String arg1) {
    exampleJNI.HandRolled(arg0, arg1);
  }
```

The `packageName` and `moduleName` must of course be correct else you will get linker errors when the JVM dynamically loads the JNI function. You may have to add in some "jtype", "jstype", "javain" and "javaout" typemaps when wrapping some JNI types. Here the default typemaps work for `int` and `char *`.

Note that if you're wanting to effectively **replace** the JNI code generated for a C/C++ function then you'll need to use `%ignore` as well to tell SWIG not to automatically generate a JNI wrapper for it.

In summary the `%native` directive is telling SWIG to generate the Java code to access the JNI C code, but not the JNI C function itself. This directive is only really useful if you want to mix your own hand crafted JNI code and the SWIG generated code into one Java class or package.

### 27.11.4 Performance concerns and hints

If you're directly manipulating huge arrays of complex objects from Java, performance may suffer greatly when using the array functions in `arrays_java.i`. Try and minimise the expensive JNI calls to C/C++ functions, perhaps by using temporary Java variables instead of accessing the information directly from the C/C++ object.

Java classes without any finalizers generally speed up code execution as there is less for the garbage collector to do. Finalizer generation can be stopped by using an empty `javafinalize` typemap:

```
%typemap(javafinalize) SWIGTYPE ""
```

However, you will have to be careful about memory management and make sure that you code in a call to the `delete()` member function. This method normally calls the C++ destructor or `free()` for C code.

### 27.11.5 Debugging

The generated code can be debugged using both a Java debugger and a C++ debugger using the usual debugging techniques. Breakpoints can be set in either Java or C++ code and so both can be debugged simultaneously. Most debuggers do not understand both Java and C++, with one notable exception of Sun Studio, where it is possible to step from Java code into a JNI method within one environment.

Alternatively, debugging can involve placing debug printout statements in the JNI layer using the `%exception` directive. See the special variables for %exception section. Many of the default

typemaps can also be overridden and modified for adding in extra logging/debug display information.

The `-Xcheck:jni` and `-Xcheck:nabounds` Java executable options are useful for debugging to make sure the JNI code is behaving. The -verbose:jni and -verbose:gc are also useful options for monitoring code behaviour.

## 27.12 Java Examples

The directory Examples/java has a number of further examples. Take a look at these if you want to see some of the techniques described in action. The Examples/index.html file in the parent directory contains the SWIG Examples Documentation and is a useful starting point. If your SWIG installation went well Unix users should be able to type `make` in each example directory, then `java main` to see them running. For the benefit of Windows users, there are also Visual C++ project files in a couple of the Windows Examples . There are also many regression tests in the Examples/test-suite directory. Many of these have runtime tests in the java subdirectory.

# 28 SWIG and Javascript


- Overview
- Preliminaries
  - Running SWIG
  - Running Tests and Examples
  - Known Issues
- Integration
  - Creating node.js Extensions
    - Using `yeoman` to generate a Node-API skeleton
    - Troubleshooting
  - Embedded Webkit
    - Mac OS X
    - GTK
  - Creating Applications with node-webkit
- Examples
  - Simple
  - Class
- Implementation
  - Source Code
  - Code Templates
  - Emitter
  - Emitter states
  - Handling Exceptions in JavascriptCore
  - Handling Exceptions in Node-API


This chapter describes SWIG's support of Javascript. It does not cover SWIG basics, but only information that is specific to this module.

## 28.1 Overview

Javascript is a prototype-based scripting language that is dynamic, weakly typed and has first-class functions. Its arguably the most popular language for web development. Javascript has gone beyond being a browser-based scripting language and can be used as a backend development language with node.js.

Native Javascript extensions can be used for applications that embed a web-browser view or that embed a Javascript engine (such as *node.js*). Extending a general purpose web-browser is not possible as this would be a severe security issue.

SWIG/Javascript currently supports:

- **Node.js** 12 or later. SWIG can optionally generate Node-API code, which needs Node.js 12.17 or later.
- **JavascriptCore** 4.0 or later. JavascriptCore is used by Safari/Webkit
- **v8** 7.4 or later (we currently CI test with 7.8 and 10.2). v8 is used by Chromium and Node.js.

WebKit is a modern browser implementation available as open-source which can be embedded into an application. NW.jst provides an app runtime which uses Google's Chromium as Web-Browser widget and node.js for javascript extensions.

## 28.2 Preliminaries

### 28.2.1 Running SWIG

Suppose that you defined a SWIG module such as the following:

```
%module example
%{
#include "example.h"
%}
int gcd(int x, int y);
extern double Foo;
```

To build a Javascript module, run SWIG using the `-javascript` option and a desired target engine `-jsc`, `-v8`, `-node` or `-napi`. `-v8` allows for interfacing with a raw embedded version of V8. In this case, it is up to the user to implement a binary module loading protocol. There are two generators supporting Node.js. The older generator for `node` is essentially delegating to the `v8` generator and adds some necessary preprocessor definitions. The more recent `-napi` generator produces `node-addon-api` that interfaces to Node.js through Node-API. The V8 generator is more mature, while the Node-API generator offers a number of advantages such as binary stable ABI allowing for publishing of universal binary modules on npm, Electron support and automatic multi-threading.

```
$ swig -javascript -jsc example.i
```

If building a C++ extension, add the -c++ option:

```
$ swig -c++ -javascript -jsc example.i
```

To generate code for V8, you would run swig like so:

```
$ swig -c++ -javascript -v8 example.i
```

This creates a C/C++ source file `example_wrap.c` or `example_wrap.cxx`. The generated C source file contains the low-level wrappers that need to be compiled and linked with the rest of your C/C++ application to create an extension module.

The name of the wrapper file is derived from the name of the input file. For example, if the input file is `example.i`, the name of the wrapper file is `example_wrap.c`. To change this, you can use the -o option. The wrapped module will export one function which must be called to register the module with the Javascript interpreter. For example, if your module is named `example` the corresponding initializer for JavascriptCore would be

```
bool example_initialize(JSGlobalContextRef context, JSObjectRef *exports)
```

and for v8:

```
void example_initialize(v8::Handle<v8::Object> exports)
```

**Note**: be aware that `v8` has a C++ API, and thus, the generated modules must be compiled as C++.

### 28.2.2 Running Tests and Examples

The configuration for tests and examples currently supports Linux and Mac only and not MinGW (Windows) yet.

The default interpreter is `node.js` as it is available on all platforms and convenient to use.

Running the examples with JavascriptCore requires `libjavascriptcoregtk-1.0` to be installed, e.g., under Ubuntu with

```
$ sudo apt-get install libjavascriptcoregtk-1.0-dev
```

Running with `V8` requires `libv8`:

```
$ sudo apt-get install libnode-dev
```

Running with Node-API requires `node-addon-api`:

```
$ sudo npm install -g node-addon-api
```

Examples can be run using

```
$ make check-javascript-examples ENGINE=jsc
```

`ENGINE` can be `node`, `jsc`, `v8`, or `napi`.

The test-suite can be run using

```
$ make check-javascript-test-suite ENGINE=jsc
```

### 28.2.3 Known Issues

At the moment, the Javascript generators pass all tests syntactically, i.e., the generated source code compiles. However, there are still remaining runtime issues.

- Default optional arguments do not work for all targeted interpreters except Node-API
- Multiple output arguments do not work for JSC
- C89 incompatibility: the JSC generator might still generate C89 violating code
- `long long` is not supported except with Node-API
- Javascript callbacks are not supported
- `instanceOf` does not work under JSC

The primary development environment has been Linux (Ubuntu 22.04). Windows and Mac OS X have been tested sporadically. Therefore, the generators might have more issues on those platforms. Please report back any problem you observe to help us improving this module quickly.

## 28.3 Integration

This chapter gives a short introduction how to use a native Javascript extension: as a `node.js` module, and as an extension for an embedded Webkit.

### 28.3.1 Creating node.js Extensions

To install `node.js` you can download an installer from their web-site for Mac OS X and Windows. For Linux you can either build the source yourself and run `sudo checkinstall` or keep to the (probably stone-age) packaged version. For Ubuntu there is a PPA available.

```
$ sudo add-apt-repository ppa:chris-lea/node.js
$ sudo apt-get update
$ sudo apt-get install nodejs
```

As `v8` is written in C++ and comes as a C++ library it is crucial to compile your module using the same compiler flags as used for building v8. To make things easier, `node.js` provides a build tool called `node-gyp`.

You have to install it using `npm`:

```
$ sudo npm install -g node-gyp
```

`node-gyp` expects a configuration file named `binding.gyp` which is basically in JSON format and conforms to the same format that is used with Google's build-tool `gyp`.

`binding.gyp`:

```
{
  "targets": [
```

```
    {
      "target_name": "example",
      "sources": [ "example.cxx", "example_wrap.cxx" ]
    }
  ]
}
```

First create the wrapper using SWIG:

```
$ swig -javascript -node -c++ example.i
```

Then run `node-gyp build` to actually create the module:

```
$ node-gyp build
```

This will create a `build` folder containing the native module. To use the extension you need to 'require' it in your Javascript source file:

```
require("./build/Release/example")
```

A more detailed explanation is given in the Examples section.

#### 28.3.1.1 Using `yeoman` to generate a Node-API skeleton

If targeting Node-API, the easiest way to bootstrap a project is by using the `yeoman` generator:

```
$ sudo npm install -g yo
$ sudo npm install -g generator-napi-module
$ mkdir example
$ cd example
$ yo napi-module                    # the choice of template is irrelevant, SWIG will replace the C++ code
$ npm install node-addon-api@latest  # the yeoman version is outdated
$ swig -javascript -napi -c++ -o src/example.cc example.i
$ node-gyp configure
$ node-gyp build
```

There is also the `node-magickwand` project that can be used as a tutorial for building and publishing a complex C++ library to npm as a ready-to-use real-world binary module.

#### 28.3.1.2 Troubleshooting

- *'module' object has no attribute 'script_main'*

This error happens when `gyp` is installed as a distribution package. It seems to be outdated. Removing it resolves the problem.

```
$ sudo apt-get remove gyp
```

### 28.3.2 Embedded Webkit

Webkit is pre-installed on Mac OS X and available as a library for GTK.

#### 28.3.2.1 Mac OS X

There is general information about programming with WebKit on Apple Developer Documentation. Details about `Cocoa` programming are not covered here.

An integration of a native extension 'example' would look like this:

```
#import "appDelegate.h"

extern bool example_initialize(JSGlobalContextRef context, JSObjectRef* exports);


@implementation ExampleAppDelegate

@synthesize webView;

- (void)addGlobalObject:(JSContextRef) context:(NSString *)objectName:(JSObjectRef) theObject {
  JSObjectRef global = JSContextGetGlobalObject(context);
  JSStringRef objectJSName = JSStringCreateWithCFString( (CFStringRef) objectName )
  if ( objectJSName != NULL ) {
    JSObjectSetProperty(context, global, objectJSName, theObject, kJSPropertyAttributeReadOnly, NULL);
    JSStringRelease( objectJSName );
  }
}

- (void)applicationDidFinishLaunching:(NSNotification *)aNotification {

  // Start a webview with the bundled index.html file
  NSString *path = [[NSBundle mainBundle] bundlePath];
  NSString *url =  [NSString stringWithFormat: @"file://%@/Contents/Assets/index.html", path];

  WebFrame *webframe = [webView mainFrame];
  JSGlobalContextRef context = [webframe globalContext];

  JSObjectRef example;
  example_initialize(context, &example);
  [self addGlobalObject:context:@"example":example]

  JSObjectSetProperty(context, global, JSStringRef propertyName, example, JSPropertyAttributes attributes, NULL);

  [ [webView mainFrame] loadRequest:
    [NSURLRequest requestWithURL: [NSURL URLWithString:url] ]
  ];
```

```
}

@end
```

#### 28.3.2.2 GTK

There is general information about programming GTK at GTK documentation and in the GTK tutorial, and for Webkit there is a Webkit GTK+ API Reference.

An integration of a native extension 'example' would look like this:

```
#include <gtk/gtk.h>
#include <webkit/webkit.h>

extern bool example_initialize(JSGlobalContextRef context);

int main(int argc, char* argv[])
{
  // Initialize GTK+
  gtk_init(&argc, &argv);

  ...

  // Create a browser instance
  WebKitWebView *webView = WEBKIT_WEB_VIEW(webkit_web_view_new());
  WebFrame *webframe = webkit_web_view_get_main_frame(webView);
  JSGlobalContextRef context = webkit_web_frame_get_global_context(webFrame);
  JSObjectRef global = JSContextGetGlobalObject(context);

  JSObjectRef exampleModule;
  example_initialize(context, &exampleModule);
  JSStringRef jsName = JSStringCreateWithUTF8CString("example");
  JSObjectSetProperty(context, global, jsName, exampleModule, kJSPropertyAttributeReadOnly, NULL);
  JSStringRelease(jsName);

  ...

  // Load a web page into the browser instance
  webkit_web_view_load_uri(webView, "https://www.webkitgtk.org/");

  ...

  // Run the main GTK+ event loop
  gtk_main();

  return 0;
}
```

### 28.3.3 Creating Applications with node-webkit

To get started with `node-webkit` there is a very informative set of wiki pages.

Similar to `node.js`, `node-webkit` is started from command line within a `node.js` project directory. Native extensions are created in the very same way as for `node.js` , except that a customized `gyp` derivate has to be used: nw-gyp.

A simple example would have the following structure:

```
- package.json
- app.html
- app.js
- node_modules
  / example
  ... (as known from node.js)
```

The configuration file essentially conforms to `node.js` syntax. It has some extras to configure `node-webkit`. See the Manifest specification for more details.

`package.json`:

```
{
  "name": "example",
  "main": "app.html",
  "window": {
    "show": true,
    "width": 800,
    "height": 600
  }
}
```

The `'main'` property of `package.json` specifies a web-page to be rendered in the main window.

`app.html`:

```
<html>
  <head>
    <script src="app.js"></script>
  </head>
  <body>
    <div>
      The greatest common divisor of
      <span id="x"></span> and
      <span id="y"></span> is
      <span id="z"></span>.
    </div>
  </body>
</html>
```

As known from `node.js` one can use `require` to load javascript modules. Additionally, `node-webkit` provides an API that allows manipulating the window's menu, open new windows, and many more things.

`app.js`:

```
window.onload = function() {
  var example = require("example");
  var x = 18;
  var y = 24;
  var z = example.gcd(x, y);
  document.querySelector('#x').innerHTML = x;
  document.querySelector('#y').innerHTML = y;
  document.querySelector('#z').innerHTML = z;
};
```

## 28.4 Examples

Some basic examples are shown here in more detail.

### 28.4.1 Simple

The common example `simple` looks like this:

```
/* File : example.i */
%module example

%inline %{
extern int    gcd(int x, int y);
extern double Foo;
%}
```

To make this available as a node extension `abinding.gyp` has to be created:

```
{
  "targets": [
    {
      "target_name": "example",
      "sources": [ "example.cxx", "example_wrap.cxx" ]
    }
  ]
}
```

Then `node-gyp` is used to build the extension:

```
$ node-gyp configure build
```

From a 'nodejs` application the extension would be used like this:

```
// import the extension via require
var example = require("./build/Release/example");

// calling the global method
var x = 42;
var y = 105;
var g = example.gcd(x, y);

// Accessing the global variable
var f = example.Foo;
example.Foo = 3.1415926;
```

First the module `example` is loaded from the previously built extension. Global methods and variables are available in the scope of the module.

**Note**: ECMAScript 5, the currently implemented Javascript standard, does not have modules. `node.js` and other implementations provide this mechanism defined by the CommonJS group. For browsers this is provided by Browserify, for instance.

### 28.4.2 Class

The common example `class` defines three classes, `Shape`, `Circle`, and `Square`:

```
class Shape {
public:
  Shape() {
    nshapes++;
  }
  virtual ~Shape() {
    nshapes--;
  }
  double  x, y;
  void    move(double dx, double dy);
  virtual double area(void) = 0;
  virtual double perimeter(void) = 0;
  static  int nshapes;
};

class Circle : public Shape {
private:
  double radius;
public:
  Circle(double r) : radius(r) { }
  virtual double area(void);
  virtual double perimeter(void);
```

```
};

class Square : public Shape {
private:
  double width;
public:
  Square(double w) : width(w) { }
  virtual double area(void);
  virtual double perimeter(void);
};
```

`Circle` and `Square` inherit from `Shape` . `Shape` has a static variable `nshapes`, a function `move` that can't be overridden (non-virtual), and two abstract functions `area` and `perimeter` (pure virtual) that must be overridden by the sub-classes.

A `nodejs` extension is built the same way as for the `simple` example.

In Javascript it can be used as follows:

```
var example = require("./build/Release/example");

// local aliases for convenience
var Shape = example.Shape;
var Circle = example.Circle;
var Square = example.Square;

// creating new instances using the 'new' operator
var c = new Circle(10);
var s = new Square(10);

// accessing a static member
Shape.nshapes;

// accessing member variables
c.x = 20;
c.y = 30;
s.x = -10;
s.y = 5;

// calling some methods
c.area();
c.perimeter();
s.area();
s.perimeter();

// instantiation of Shape is not permitted
new Shape();
```

Running these commands in an interactive node shell results in the following output:

```
$ node -i
& var example = require("./build/Release/example");
undefined
& var Shape = example.Shape;
undefined
& var Circle = example.Circle;
undefined
& var Square = example.Square;
undefined
& var c = new Circle(10);
undefined
& var s = new Square(10);
undefined
& Shape.nshapes;
2
& c.x = 20;
20
& c.y = 30;
30
& s.x = -10;
-10
& s.y = 5;
5
& c.area();
314.1592653589793
& c.perimeter();
62.83185307179586
& s.area();
100
& s.perimeter();
40
& c.move(40, 40)
undefined
& c.x
60
& c.y
70
& new Shape()
Error: Class Shape can not be instantiated
at repl:1:2
at REPLServer.self.eval (repl.js:110:21)
at Interface.<anonymous> (repl.js:239:12)
at Interface.EventEmitter.emit (events.js:95:17)
at Interface._onLine (readline.js:202:10)
at Interface._line (readline.js:531:8)
at Interface._ttyWrite (readline.js:760:14)
at ReadStream.onkeypress (readline.js:99:10)
at ReadStream.EventEmitter.emit (events.js:98:17)
at emitKey (readline.js:1095:12)
```

**Note**: In ECMAScript 5 there is no concept for classes. Instead each function can be used as a constructor function which is executed by the 'new' operator. Furthermore, during construction the key property `prototype` of the constructor function is used to attach a prototype instance to the created object. A prototype is essentially an object itself that is the first-class delegate of a class used whenever the access to a property of an object fails. The very same prototype instance is shared among all instances of one type. Prototypal inheritance is explained in more detail on in Inheritance and the prototype chain, for instance.

## 28.5 Implementation

The Javascript Module implementation has taken a very different approach compared to other language modules in order to support different Javascript interpreters.

### 28.5.1 Source Code

The Javascript module is implemented in `Source/Modules/javascript.cxx`. It dispatches the code generation to a `JSEmitter` instance, `V8Emitter`, `JSCEmitter` or `NAPIEmitter`. Additionally there are some helpers: `Template`, for templated code generation, and `JSEmitterState`, which is used to manage state information during AST traversal. This rough map shall make it easier to find a way through this huge source file:

```
// module wide defines

#define NAME "name"
...

// ################################
// #  Helper class declarations

class JSEmitterState { ... };

class Template { ... };

// ################################
// # JSEmitter declaration

class JSEmitter { ... };

// Emitter factory declarations

JSEmitter *swig_javascript_create_JSCEmitter();
JSEmitter *swig_javascript_create_V8Emitter();
JSEmitter *swig_javascript_create_NAPIEmitter();

// ################################
// # Javascript module

// Javascript module declaration

class JAVASCRIPT:public Language { ... };

// Javascript module implementation

int JAVASCRIPT::functionWrapper(Node *n) { ... }
...

// Module factory implementation

static Language *new_swig_javascript() { ... }

extern "C" Language *swig_javascript(void) { ... }

// ################################
// # JSEmitter base implementation

JSEmitter::JSEmitter() { ... }

Template JSEmitter::getTemplate(const String *name) { ... }
...

// ################################
// # JSCEmitter

// JSCEmitter declaration

class JSCEmitter: public JSEmitter { ... };

// JSCEmitter implementation

JSCEmitter::JSCEmitter() { ... }

void JSCEmitter::marshalInputArgs(Node *n, ParmList *parms, Wrapper *wrapper, MarshallingMode mode, bool is_member, bool is_static) { ... }
...

// JSCEmitter factory

JSEmitter *swig_javascript_create_JSCEmitter() { ... }


// ################################
// # V8Emitter

// V8Emitter declaration

class V8Emitter: public JSEmitter { ... };

// V8Emitter implementation

V8Emitter::V8Emitter() { ... }

int V8Emitter::initialize(Node *n) { ... }

// V8Emitter factory

JSEmitter *swig_javascript_create_V8Emitter() { ... }
```

```
// ###############################
// # Helper implementation (JSEmitterState, Template)

JSEmitterState::JSEmitterState() { ... }
...

Template::Template(const String *code_) { ... }
...
```

### 28.5.2 Code Templates

All generated code is created on the basis of code templates. The templates for *JavascriptCore* can be found in `Lib/javascript/jsc/javascriptcode.swg`, for *v8* in `Lib/javascript/v8/javascriptcode.swg` and for *Node-API* in `Lib/javascript/napi/javascriptcode.swg`.

To track the originating code template for generated code you can run

```
$ swig -javascript -jsc -debug-codetemplates
```

which wraps generated code with a descriptive comment

```
/* begin fragment("template_name") */

...generated code ...

/* end fragment("template_name") */
```

The Template class is used like this:

```
Template t_register = getTemplate("jsv8_register_static_variable");
t_register.replace("$jsparent", state.clazz(NAME_MANGLED))
    .replace("$jsname", state.variable(NAME))
    .replace("$jsgetter", state.variable(GETTER))
    .replace("$jssetter", state.variable(SETTER))
    .trim().
    print(f_init_static_wrappers);
```

A code template is registered with the *JSEmitter* via `fragment(name, "template")`, e.g.,

```
%fragment ("jsc_variable_declaration", "templates")
%{
  {"$jsname", $jsgetter, $jssetter, kJSPropertyAttributeNone},
%}
```

`Template` creates a copy of that string and `Template::replace` uses Swig's `Replaceall` to replace variables in the template. `Template::trim` can be used to eliminate leading and trailing whitespaces. `Template::print` is used to write the final template string to a Swig `DOH` (based on `Printv`). All methods allow chaining.

### 28.5.3 Emitter

The Javascript module delegates code generation to a `JSEmitter` instance. The following extract shows the essential interface:

```
class JSEmitter {
  ...

  /**
   * Opens output files and temporary output DOHs.
   */
  virtual int initialize(Node *n);

  /**
   * Writes all collected code into the output file(s).
   */
  virtual int dump(Node *n) = 0;

  /**
   * Cleans up all open output DOHs.
   */
  virtual int close() = 0;

  ...

  /**
   * Invoked at the beginning of the classHandler.
   */
  virtual int enterClass(Node *);

  /**
   * Invoked at the end of the classHandler.
   */
  virtual int exitClass(Node *) {
    return SWIG_OK;
  }

  /**
   * Invoked at the beginning of the variableHandler.
   */
  virtual int enterVariable(Node *);

  /**
   * Invoked at the end of the variableHandler.
   */
  virtual int exitVariable(Node *) {
```

```
    return SWIG_OK;
  }

  /**
   * Invoked at the beginning of the functionHandler.
   */
  virtual int enterFunction(Node *);

  /**
   * Invoked at the end of the functionHandler.
   */
  virtual int exitFunction(Node *) {
    return SWIG_OK;
  }

  /**
   * Invoked by functionWrapper callback after call to Language::functionWrapper.
   */
  virtual int emitWrapperFunction(Node *n);

  /**
   * Invoked from constantWrapper after call to Language::constantWrapper.
   **/
  virtual int emitConstant(Node *n);

  /**
   * Registers a given code snippet for a given key name.
   *
   * This method is called by the fragmentDirective handler
   * of the JAVASCRIPT language module.
   **/
  int registerTemplate(const String *name, const String *code);

  /**
   * Retrieve the code template registered for a given name.
   */
  Template getTemplate(const String *name);

  State &getState();

  ...

}
```

The module calls `initialize`, `dump`, and `close` from within the `top` method:

```
int JAVASCRIPT::top(Node *n) {
  emitter->initialize(n);

  Language::top(n);

  emitter->dump(n);
  emitter->close();

  return SWIG_OK;
}
```

The methods `enterClass` and `exitClass` are called from within the `classHandler` method:

```
int JAVASCRIPT::classHandler(Node *n) {

  emitter->enterClass(n);
  Language::classHandler(n);
  emitter->exitClass(n);

  return SWIG_OK;
}
```

In `enterClass` the emitter stores state information that is necessary when processing class members. In `exitClass` the wrapper code for the whole class is generated.

### 28.5.4 Emitter states

For storing information during the AST traversal the emitter provides a `JSEmitterState` with different slots to store data representing the scopes global, class, function, and variable.

```
class JSEmitterState {

public:

  JSEmitterState();

  ~JSEmitterState();

  DOH *global();

  DOH *global(const char* key, DOH *initial = 0);

  DOH *clazz(bool reset = false);

  DOH *clazz(const char* key, DOH *initial = 0);

  DOH *function(bool reset = false);

  DOH *function(const char* key, DOH *initial = 0);

  DOH *variable(bool reset = false);
```

```
  DOH *variable(const char* key, DOH *initial = 0);

  static int IsSet(DOH *val);

  ...
};
```

When entering a scope, such as in `enterClass`, the corresponding state is reset and new data is stored:

```
state.clazz(RESET);
state.clazz(NAME, Getattr(n, "sym:name"));
```

State information can be retrieved using `state.clazz(NAME)` or with `Getattr` on `state.clazz()` which actually returns a `Hash` instance.

### 28.5.5 Handling Exceptions in JavascriptCore

Applications with an embedded JavascriptCore should be able to present detailed exception messages that occur in the Javascript engine. Below is an example derived from code provided by Brian Barnes on how these exception details can be extracted.

```
void script_exception_to_string(JSContextRef js_context, JSValueRef exception_value_ref, char* return_error_string, int return_error_string_ma
{
  JSObjectRef exception_object;
  JSValueRef value_ref;
  JSStringRef jsstring_property_name = NULL;
  JSValueRef temporary_exception = NULL;
  JSStringRef js_return_string = NULL;
  size_t bytes_needed;
  char* c_result_string = NULL;
  exception_object = JSValueToObject(js_context, exception_value_ref, NULL);

  /* source url */
  strcpy(return_error_string, "[");
  jsstring_property_name = JSStringCreateWithUTF8CString("sourceURL");
  value_ref = JSObjectGetProperty(js_context, exception_object, jsstring_property_name, &temporary_exception);
  JSStringRelease(jsstring_property_name);
  js_return_string = JSValueToStringCopy(js_context, value_ref, NULL);
  bytes_needed = JSStringGetMaximumUTF8CStringSize(js_return_string);
  c_result_string = (char*)calloc(bytes_needed, sizeof(char));
  JSStringGetUTF8CString(js_return_string, c_result_string, bytes_needed);
  JSStringRelease(js_return_string);
  strncat(return_error_string, c_result_string, return_error_string_max_length-1);
  free(c_result_string);

  strncat(return_error_string, ":", return_error_string_max_length-1);

  /* line number */

  jsstring_property_name = JSStringCreateWithUTF8CString("line");
  value_ref = JSObjectGetProperty(js_context, exception_object, jsstring_property_name, &temporary_exception);
  JSStringRelease(jsstring_property_name);
  js_return_string = JSValueToStringCopy(js_context, value_ref, NULL);
  bytes_needed = JSStringGetMaximumUTF8CStringSize(js_return_string);
  c_result_string = (char*)calloc(bytes_needed, sizeof(char));
  JSStringGetUTF8CString(js_return_string, c_result_string, bytes_needed);
  JSStringRelease(js_return_string);
  strncat(return_error_string, c_result_string, return_error_string_max_length-1);
  free(c_result_string);

  strncat(return_error_string, "]", return_error_string_max_length-1);

  /* error message */

  jsstring_property_name = JSStringCreateWithUTF8CString("message");
  value_ref = JSObjectGetProperty(js_context, exception_object, jsstring_property_name, &temporary_exception);
  JSStringRelease(jsstring_property_name);
  if(NULL == value_ref)
  {
    strncat(return_error_string, "Unknown Error", return_error_string_max_length-1);
  }
  else
  {
    js_return_string = JSValueToStringCopy(js_context, value_ref, NULL);
    bytes_needed = JSStringGetMaximumUTF8CStringSize(js_return_string);
    c_result_string = (char*)calloc(bytes_needed, sizeof(char));
    JSStringGetUTF8CString(js_return_string, c_result_string, bytes_needed);
    JSStringRelease(js_return_string);
    strncat(return_error_string, c_result_string, return_error_string_max_length-1);
    free(c_result_string);
  }
}
```

It would be used in the following way:

```
if(js_exception)
{
  char return_error_string[256];
  script_exception_to_string(js_context, js_exception, return_error_string, 256);
  printf("Compile error is %s", return_error_string);
}
```

### 28.5.6 Handling Exceptions in Node-API

Node-API is the only generator that provides fully automatic conversion of C++ exceptions to JavaScript exceptions when building with C++ exceptions enabled in `binding.gyp`:

```
'conditions': [
```

```
  ['OS=="mac"',
    {
      'xcode_settings': {
        'GCC_ENABLE_CPP_RTTI': 'YES',
        'GCC_ENABLE_CPP_EXCEPTIONS' : 'YES'
      }
    }
  ],
  ['OS=="linux" or OS=="freebsd" or OS=="openbsd" or OS=="solaris"',
    {
      'cflags!': [ '-fno-exceptions' ],
      'cflags_cc!': [ '-fno-exceptions', '-fno-rtti' ]
    }
  ]
]
```

In this case, nothing else is needed for the C++ exceptions to be passed to JavaScript.

---

# 29 SWIG and Lua


- Preliminaries
- Running SWIG
  - Additional command line options
  - Compiling and Linking and Interpreter
  - Compiling a dynamic module
  - Using your module
- A tour of basic C/C++ wrapping
  - Modules
  - Functions
  - Global variables
  - Constants and enums
    - Constants/enums and classes/structures
  - Pointers
  - Structures
  - C++ classes
  - C++ inheritance
  - Pointers, references, values, and arrays
  - C++ overloaded functions
  - C++ operators
  - Class extension with %extend
  - Using %newobject to release memory
  - C++ templates
  - C++ Smart Pointers
  - C++ Exceptions
  - Namespaces
    - Compatibility Note
    - Names
    - Inheritance
- Typemaps
  - What is a typemap?
  - Using typemaps
  - Typemaps and arrays
  - Typemaps and pointer-pointer functions
- Writing typemaps
  - Typemaps you can write
  - SWIG's Lua-C API
- Customization of your Bindings
  - Writing your own custom wrappers
  - Adding additional Lua code
- Details on the Lua binding
  - Binding global data into the module.
  - Userdata and Metatables
  - Memory management


Lua is an extension programming language designed to support general procedural programming with data description facilities. It also offers good support for object-oriented programming, functional programming, and data-driven programming. Lua is intended to be used as a powerful, light-weight configuration language for any program that needs one. Lua is implemented as a library, written in clean C (that is, in the common subset of ISO C and C++). It's also a *really* tiny language, less than 6000 lines of code, which compiles to <100 kilobytes of binary code. It can be found at https://www.lua.org

eLua stands for Embedded Lua (can be thought of as a flavor of Lua) and offers the full implementation of the Lua programming language to the embedded world, extending it with specific features for efficient and portable software embedded development. eLua runs on smaller devices like microcontrollers and provides the full features of the regular Lua desktop version. More information on eLua can be found here: http://www.eluaproject.net

## 29.1 Preliminaries

The current SWIG implementation is designed to work with Lua 5.0.x, 5.1.x and 5.2.x. It should work with later versions of Lua, but certainly not with Lua 4.0 due to substantial API changes. It is possible to either static link or dynamic link a Lua module into the interpreter (normally Lua static links its libraries, as dynamic linking is not available on all platforms). SWIG also has support for eLua starting from eLua 0.8. Due to substantial changes between SWIG 2.x and SWIG 3.0 and unavailability of testing platform, eLua status was downgraded to 'experimental'.

## 29.2 Running SWIG

Suppose that you defined a SWIG module such as the following:

```
%module example
%{
#include "example.h"
%}
int gcd(int x, int y);
extern double Foo;
```

To build a Lua module, run SWIG using the `-lua` option.

```
$ swig -lua example.i
```

If building a C++ extension, add the `-c++` option:

```
$ swig -c++ -lua example.i
```

This creates a C/C++ source file `example_wrap.c` or `example_wrap.cxx`. The generated C source file contains the low-level wrappers that need to be compiled and linked with the rest of your C/C++ application to create an extension module.

The name of the wrapper file is derived from the name of the input file. For example, if the input file is `example.i`, the name of the wrapper file is `example_wrap.c`. To change this, you can use the -o option. The wrapped module will export one function `"int luaopen_example(lua_State* L)"` which must be called to register the module with the Lua interpreter. The name "luaopen_example" depends upon the name of the module.

To build an eLua module, run SWIG using `-lua` and add either `-elua` or `-eluac`.

```
$ swig -lua -elua example.i
```

or

```
$ swig -lua -eluac example.i
```

The `-elua` option puts all the C function wrappers and variable get/set wrappers in rotables. It also generates a metatable which will control the access to these variables from eLua. It also offers a significant amount of module size compression. On the other hand, the `-eluac` option puts all the wrappers in a single rotable. With this option, no matter how huge the module, it will consume no additional microcontroller SRAM (crass compression). There is a catch though: Metatables are not generated with `-eluac`. To access any value from eLua, one must directly call the wrapper function associated with that value.

### 29.2.1 Additional command line options

The following table list the additional commandline options available for the Lua module. They can also be seen by using:

```
swig -lua -help
```

| Lua specific options | |
|---|---|
| -elua | Generates LTR compatible wrappers for smaller devices running elua. |
| -eluac | LTR compatible wrappers in "crass compress" mode for elua. |
| -nomoduleglobal | Do not register the module name as a global variable but return the module table from calls to require. |
| -no-old-metatable-bindings | Disable backward compatibility: old-style binding names generations and a few other things. Explanations are included in appropriate later sections. This option is considered deprecated and will be removed in the near future. |
| -squash-bases | Squashes symbols from all inheritance tree of a given class into itself. Emulates pre-SWIG3.0 inheritance. Insignificantly speeds things up, but increases memory consumption. |

### 29.2.2 Compiling and Linking and Interpreter

Normally Lua is embedded into another program and will be statically linked. An extremely simple stand-alone interpreter (`min.c`) is given below:

```
#include <stdio.h>
#include "lua.h"
#include "lualib.h"
#include "lauxlib.h"

extern int luaopen_example(lua_State* L); // declare the wrapped module

int main(int argc, char* argv[])
{
 lua_State *L;
 if (argc<2)
 {
  printf("%s: <filename.lua>\n", argv[0]);
  return 0;
 }
 L=lua_open();
 luaopen_base(L);       // load basic libs (eg. print)
 luaopen_example(L);    // load the wrapped module
 if (luaL_loadfile(L, argv[1])==0) // load and run the file
  lua_pcall(L, 0, 0, 0);
 else
  printf("unable to load %s\n", argv[1]);
 lua_close(L);
 return 0;
}
```

A much improved set of code can be found in the Lua distribution `src/lua/lua.c`. Include your module, just add the external declaration & add a `#define LUA_EXTRALIBS {"example", luaopen_example}`, at the relevant place.

The exact commands for compiling and linking vary from platform to platform. Here is a possible set of commands of doing this:

```
$ swig -lua example.i -o example_wrap.c
$ gcc -I/usr/include/lua -c min.c -o min.o
$ gcc -I/usr/include/lua -c example_wrap.c -o example_wrap.o
$ gcc -c example.c -o example.o
$ gcc -I/usr/include/lua -L/usr/lib/lua min.o example_wrap.o example.o -o my_lua
```

For eLua, the source must be built along with the wrappers generated by SWIG. Make sure the eLua source files `platform_conf.h` and `auxmods.h` are updated with the entries of your new module. Please note: `"mod"` is the module name.

```
/* Sample platform_conf.h */
#define LUA_PLATFORM_LIBS_ROM\
```

```
  _ROM( AUXLIB_PIO, luaopen_pio, pio_map )\
  _ROM( AUXLIB_TMR, luaopen_tmr, tmr_map )\
  _ROM( AUXLIB_MOD, luaopen_mod, mod_map )\
  ....
```

```
/* Sample auxmods.h */
#define AUXLIB_PIO       "pio"
LUALIB_API int ( luaopen_pio )(lua_State *L );

#define AUXLIB_MOD       "mod"
LUALIB_API int ( luaopen_mod )(lua_State *L );
....
```

More information on building and configuring eLua can be found here: http://www.eluaproject.net/doc/v0.8/en_building.html

### 29.2.3 Compiling a dynamic module

Most, but not all platforms support the dynamic loading of modules (Windows & Linux do). Refer to the Lua manual to determine if your platform supports it. For compiling a dynamically loaded module the same wrapper can be used. Assuming you have code you need to link to in a file called `example.c`, the commands will be something like this:

```
$ swig -lua example.i -o example_wrap.c
$ gcc -fPIC -I/usr/include/lua -c example_wrap.c -o example_wrap.o
$ gcc -fPIC -c example.c -o example.o
$ gcc -shared -I/usr/include/lua -L/usr/lib/lua example_wrap.o example.o -o example.so
```

The wrappers produced by SWIG can be compiled and linked with Lua 5.1.x and later. The loading is extremely simple.

```
require("example")
```

For those using Lua 5.0.x, you will also need an interpreter with the loadlib function (such as the default interpreter compiled with Lua). In order to dynamically load a module you must call the loadlib function with two parameters: the filename of the shared library, and the function exported by SWIG. Calling loadlib should return the function, which you then call to initialise the module

```
my_init=loadlib("example.so", "luaopen_example") -- for Unix/Linux
--my_init=loadlib("example.dll", "luaopen_example") -- for Windows
assert(my_init) -- make sure it's not nil
my_init()       -- call the init fn of the lib
```

Or can be done in a single line of Lua code

```
assert(loadlib("example.so", "luaopen_example"))()
```

If the code didn't work, don't panic. The best thing to do is to copy the module and your interpreter into a single directory and then execute the interpreter and try to manually load the module (take care, all this code is case sensitive).

```
a, b, c=package.loadlib("example.so", "luaopen_example") -- for Unix/Linux
--a, b, c=package.loadlib("example.dll", "luaopen_example") -- for Windows
print(a, b, c)
```

Note: for Lua 5.0:
The loadlib() function is in the global namespace, not in a package. So it's just loadlib().

if 'a' is a function, this is all working fine, all you need to do is call it

```
  a()
```

to load your library which will add a table 'example' with all the functions added.

If it doesn't work, look at the error messages, in particular message 'b'
`The specified module could not be found.`
Means that is cannot find the module, check your the location and spelling of the module.
`The specified procedure could not be found.`
Means that it loaded the module, but cannot find the named function. Again check the spelling, and if possible check to make sure the functions were exported correctly.
`'loadlib' not installed/supported`
Is quite obvious (Go back and consult the Lua documents on how to enable loadlib for your platform).

### 29.2.4 Using your module

Assuming all goes well, you will be able to this:

```
$ ./my_lua
> print(example.gcd(4, 6))
2
> print(example.Foo)
3
> example.Foo=4
> print(example.Foo)
4
>
```

## 29.3 A tour of basic C/C++ wrapping

By default, SWIG tries to build a very natural Lua interface to your C/C++ code. This section briefly covers the essential aspects of this wrapping.

### 29.3.1 Modules

The SWIG module directive specifies the name of the Lua module. If you specify `module example', then everything is wrapped into a Lua table 'example' containing all the functions and variables. When choosing a module name, make sure you don't use the same name as a built-in Lua command or standard module name.

### 29.3.2 Functions

Global functions are wrapped as new Lua built-in functions. For example,

```
%module example
int fact(int n);
```

creates a built-in function `example.fact(n)` that works exactly like you think it does:

```
> print example.fact(4)
24
>
```

To avoid name collisions, SWIG create a Lua table which keeps all the functions, constants, classes and global variables in. It is possible to copy the functions, constants and classes (but not variables) out of this and into the global environment with the following code. This can easily overwrite existing functions, so this must be used with care.

```
> for k, v in pairs(example) do _G[k]=v end
> print(fact(4))
24
>
```

It is also possible to rename the module with an assignment.

```
> e=example
> print(e.fact(4))
24
> print(example.fact(4))
24
```

### 29.3.3 Global variables

Global variables (which are linked to C code) are supported, and appear to be just another variable in Lua. However the actual mechanism is more complex. Given a global variable:

```
%module example
extern double Foo;
```

SWIG will effectively generate two functions `example.Foo_set()` and `example.Foo_get()`. It then adds a metatable to the table 'example' to call these functions at the correct time (when you attempt to set or get examples.Foo). Therefore if you were to attempt to assign the global to another variable, you will get a local copy within the interpreter, which is no longer linked to the C code.

```
> print(example.Foo)
3
> c=example.Foo   -- c is a COPY of example.Foo, not the same thing
> example.Foo=4
> print(c)
3
> c=5 -- this will not affect the original example.Foo
> print(example.Foo, c)
4    5
```

It is therefore not possible to 'move' the global variable into the global namespace as it is with functions. It is however, possible to rename the module with an assignment, to make it more convenient.

```
> e=example
> -- e and example are the same table
> -- so e.Foo and example.Foo are the same thing
> example.Foo=4
> print(e.Foo)
4
```

If a variable is marked with the %immutable directive then any attempts to set this variable will cause a Lua error. Given a global variable:

```
%module example
%immutable;
extern double Foo;
%mutable;
```

SWIG will allow the reading of `Foo` but when a set attempt is made, an error function will be called.

```
> print(e.Foo) -- reading works ok
4
> example.Foo=40 -- but writing does not
This variable is immutable
stack traceback:
        [C]: ?
        [C]: ?
        stdin:1: in main chunk
        [C]: ?
```

For those people who would rather that SWIG silently ignore the setting of immutables (as previous versions of the Lua bindings did), adding a `-DSWIGLUA_IGNORE_SET_IMMUTABLE` compile option will remove this.

Unlike earlier versions of the binding, it is now possible to add new functions or variables to the module, just as if it were a normal table. This also allows the user to rename/remove existing functions and constants (but not linked variables, mutable or immutable). Therefore users are recommended to be careful when doing so.

```
> -- example.PI does not exist
> print(example.PI)
nil
> example.PI=3.142 -- new value added
> print(example.PI)
3.142
```

If you have used the `-eluac` option for your eLua module, you will have to follow a different approach while manipulating global variables. (This is not applicable for wrappers generated with `-elua`)

```
> -- Applicable only with -eluac. (num is defined)
> print(example.num_get())
20
> example.num_set(50) -- new value added
> print(example.num_get())
50
```

In general, functions of the form `"variable_get()"` and `"variable_set()"` are automatically generated by SWIG for use with `-eluac`.

### 29.3.4 Constants and enums

Because Lua doesn't really have the concept of constants, C/C++ constants are not really constant in Lua. They are actually just a copy of the value into the Lua interpreter. Therefore they can be changed just as any other value. For example given some constants:

```
%module example
%constant int ICONST=42;
#define    SCONST      "Hello World"
enum Days{SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY};
```

This is 'effectively' converted into the following Lua code:

```
example.ICONST=42
example.SCONST="Hello World"
example.SUNDAY=0
....
```

Constants are not guaranteed to remain constant in Lua. The name of the constant could be accidentally reassigned to refer to some other object. Unfortunately, there is no easy way for SWIG to generate code that prevents this. You will just have to be careful.

If you're using eLua and have used `-elua` or `-eluac` to generate your wrapper, macro constants and enums should be accessed through a rotable called `"const"`. In eLua, macro constants and enums are guaranteed to remain constants since they are all contained within a rotable. A regular C constant is accessed from eLua just as if it were a regular global variable, just that the property of value immutability is demonstrated if an attempt at modifying a C constant is made.

```
> print(example.ICONST)
10
> print(example.const.SUNDAY)
0
> print(example.const.SCONST)
Hello World
```

#### 29.3.4.1 Constants/enums and classes/structures

Enums are exported into a class table. For example, given some enums:

```
%module example
enum Days { SUNDAY = 0, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY };
struct Test {
  enum { TEST1 = 10, TEST2 = 20 };
#ifdef __cplusplus // There are no static members in C
  static const int ICONST = 12;
#endif
};
```

There is a slight difference in behaviour wrapping C and C++ code due to the different scoping rules of C and C++. The wrapped C++ code is used as follows from Lua code:

```
> print(example.SUNDAY)
0
> print(example.Test.TEST1)
10
> print(example.Test.ICONST)
12
```

Enums within a C struct are in the global namespace and are used as follows from Lua

```
> print(example.SUNDAY)
0
> -- See the difference here
> print(example.TEST1)
10
```

**Compatibility Note:** Versions of SWIG prior to SWIG-3.0.0 did not generate the class table members above. There is no change in the C wrappers, but the following code was the only way to access these constants/enums when wrapping C++ member constants:

```
> print(example.Test_TEST1)
10
> print(example.Test_ICONST)
12
```

The old-style bindings are still generated in addition to the new ones. If the `-no-old-metatable-bindings` option is used, then these old-style bindings are not generated.

It is worth mentioning, that `example.Test.TEST1` and `example.Test_TEST1` are different entities and changing one does not change the other. Given the fact that these are constants and they are not supposed to be changed, it is up to you to avoid such issues.

### 29.3.5 Pointers

C/C++ pointers are fully supported by SWIG. Furthermore, SWIG has no problem working with incomplete type information. Given a wrapping of the <file.h> interface:

```
%module example

FILE *fopen(const char *filename, const char *mode);
int fputs(const char *, FILE *);
int fclose(FILE *);
```

When wrapped, you will be able to use the functions in a natural way from Lua. For example:

```
> f=example.fopen("junk", "w")
> example.fputs("Hello World", f)
> example.fclose(f)
```

Unlike many scripting languages, Lua has had support for pointers to C/C++ object built in for a long time. They are called 'userdata'. Unlike many other SWIG versions which use some kind of encoded character string, all objects will be represented as a userdata. The SWIG-Lua bindings provides a special function `swig_type()`, which if given a userdata object will return the type of object pointed to as a string (assuming it was a SWIG wrapped object).

```
> print(f)
userdata: 003FDA80
> print(swig_type(f))
FILE * -- it's a FILE*
```

Lua enforces the integrity of its userdata, so it is virtually impossible to corrupt the data. But as the user of the pointer, you are responsible for freeing it, or closing any resources associated with it (just as you would in a C program). This does not apply so strictly to classes & structs (see below). One final note: if a function returns a NULL pointer, this is not encoded as a userdata, but as a Lua nil.

```
> f=example.fopen("not there", "r") -- this will return a NULL in C
> print(f)
nil
```

### 29.3.6 Structures

If you wrap a C structure, it is also mapped to a Lua userdata. By adding a metatable to the userdata, this provides a very natural interface. For example,

```
struct Point{
  int x, y;
};
```

is used as follows:

```
> p=example.new_Point()
> p.x=3
> p.y=5
> print(p.x, p.y)
3       5
>
```

Similar access is provided for unions and the data members of C++ classes.
C structures can be created using a function `new_Point()`, and both C structures and C++ classes can be created using just the name `Point()`.

If you print out the value of p in the above example, you will see something like this:

```
> print(p)
userdata: 003FA320
```

Like the pointer in the previous section, this is held as a userdata. However, additional features have been added to make this more usable. SWIG effectively creates some accessor/mutator functions to get and set the data. These functions will be added to the userdata's metatable. This provides the natural access to the member variables that were shown above (see end of the document for full details).

`const` members of a structure are read-only. Data members can also be forced to be read-only using the immutable directive. As with other immutables, setting attempts will be cause an error. For example:

```
struct Foo {
  ...
  %immutable;
  int x;        // Read-only members
  char *name;
  %mutable;
  ...
};
```

The mechanism for managing char* members as well as array members is similar to other languages. It is somewhat cumbersome and should probably be better handled by defining of typemaps (described later).

When a member of a structure is itself a structure, it is handled as a pointer. For example, suppose you have two structures like this:

```
struct Foo {
  int a;
```

```
};

struct Bar {
  Foo f;
};
```

Now, suppose that you access the f attribute of Bar like this:

```
> b = Bar()
> x = b.f
```

In this case, x is a pointer that points to the Foo that is inside b. This is the same value as generated by this C code:

```
Bar b;
Foo *x = &b->f;       // Points inside b
```

Because the pointer points inside the structure, you can modify the contents and everything works just like you would expect. For example:

```
> b = Bar()
> b.f.a = 3               -- Modify attribute of structure member
> x = b.f
> x.a = 3                 -- Modifies the same structure
```

For eLua with the `-eluac` option, structure manipulation has to be performed with specific structure functions generated by SWIG. Let's say you have the following structure definition:

```
struct data {
  int x, y;
  double z;
};

> --From eLua
> a = example.new_data()
> example.data_x_set(a, 10)
> example.data_y_set(a, 20)
> print(example.data_x_get(a), example.data_y_get(a))
10 20
```

In general, functions of the form `"new_struct()"`, `"struct_member_get()"`, `"struct_member_set()"` and `"free_struct()"` are automatically generated by SWIG for each structure defined in C. (Please note: This doesn't apply for modules generated with the `-elua` option)

### 29.3.7 C++ classes

C++ classes are wrapped by a Lua userdata as well. For example, if you have this class,

```
class List {
public:
  List();
  ~List();
  int  search(char *item);
  void insert(char *item);
  void remove(char *item);
  char *get(int n);
  int  length;
};
```

you can use it in Lua like this:

```
> l = example.List()
> l:insert("Ale")
> l:insert("Stout")
> l:insert("Lager")
> print(l:get(1))
Stout
> print(l:length)
3
>
```

(Note: for calling methods of a class, you use `class:method(args)`, not `class.method(args)`, it's an easy mistake to make. However for data attributes it is `class.attribute` )

Class data members are accessed in the same manner as C structures. Static class members present a special problem for Lua, as Lua doesn't have support for such features. Therefore, SWIG generates wrappers that try to work around some of these issues. To illustrate, suppose you have a class like this:

```
class Spam {
public:
  static void foo();
  static int bar;
};
```

In Lua, C++ static members can be accessed as follows:

```
> example.Spam.foo()            -- calling Spam::foo()
> a=example.Spam.bar            -- reading Spam::bar
> example.Spam.bar=b            -- writing to Spam::bar
```

It is not (currently) possible to access static members of an instance:

```
> s=example.Spam()              -- s is a Spam instance
> s.foo()                       -- Spam::foo() via an instance
                                -- does NOT work
```

**Compatibility Note:** In versions prior to SWIG-3.0.0 only the following names would work:

```
> example.Spam_foo()            -- calling Spam::foo()
> a=example.Spam_bar            -- reading Spam::bar
> example.Spam_bar=b            -- writing to Spam::bar
```

Both style names are generated by default now. However, if the `-no-old-metatable-bindings` option is used, then the backward compatible names are not generated in addition to ordinary ones.

### 29.3.8 C++ inheritance

SWIG is fully aware of issues related to C++ inheritance. Therefore, if you have classes like this

```
class Foo {
...
};

class Bar : public Foo {
...
};
```

And if you have functions like this

```
void spam(Foo *f);
```

then the function `spam()` accepts a Foo pointer or a pointer to any class derived from Foo.

It is safe to use multiple inheritance with SWIG.

### 29.3.9 Pointers, references, values, and arrays

In C++, there are many different ways a function might receive and manipulate objects. For example:

```
void spam1(Foo *x);      // Pass by pointer
void spam2(Foo &x);      // Pass by reference
void spam3(Foo x);       // Pass by value
void spam4(Foo x[]);     // Array of objects
```

In SWIG, there is no detailed distinction like this--specifically, there are only "objects". There are no pointers, references, arrays, and so forth. Because of this, SWIG unifies all of these types together in the wrapper code. For instance, if you actually had the above functions, it is perfectly legal to do this:

```
> f = Foo()           -- Create a Foo
> spam1(f)            -- Ok. Pointer
> spam2(f)            -- Ok. Reference
> spam3(f)            -- Ok. Value.
> spam4(f)            -- Ok. Array (1 element)
```

Similar behaviour occurs for return values. For example, if you had functions like this,

```
Foo *spam5();
Foo &spam6();
Foo  spam7();
```

then all three functions will return a pointer to some Foo object. Since the third function (spam7) returns a value, newly allocated memory is used to hold the result and a pointer is returned (Lua will release this memory when the return value is garbage collected). The other two are pointers which are assumed to be managed by the C code and so will not be garbage collected.

### 29.3.10 C++ overloaded functions

C++ overloaded functions, methods, and constructors are mostly supported by SWIG. For example, if you have two functions like this:

```
void foo(int);
void foo(char *c);
```

You can use them in Lua in a straightforward manner:

```
> foo(3)           -- foo(int)
> foo("Hello")     -- foo(char *c)
```

However due to Lua's coercion mechanism is can sometimes do strange things.

```
> foo("3")           -- "3" can be coerced into an int, so it calls foo(int)!
```

As this coercion mechanism is an integral part of Lua, there is no easy way to get around this other than renaming of functions (see below).

Similarly, if you have a class like this,

```
class Foo {
public:
  Foo();
  Foo(const Foo &);
```

```
  ...
};
```

you can write Lua code like this:

```
> f = Foo()          -- Create a Foo
> g = Foo(f)         -- Copy f
```

Overloading support is not quite as flexible as in C++. Sometimes there are methods that SWIG can't disambiguate. For example:

```
void spam(int);
void spam(short);
```

or

```
void foo(Bar *b);
void foo(Bar &b);
```

If declarations such as these appear, you will get a warning message like this:

```
example.i:12: Warning 509: Overloaded method spam(short) effectively ignored,
example.i:11: Warning 509: as it is shadowed by spam(int).
```

To fix this, you either need to ignore or rename one of the methods. For example:

```
%rename(spam_short) spam(short);
...
void spam(int);
void spam(short);   // Accessed as spam_short
```

or

```
%ignore spam(short);
...
void spam(int);
void spam(short);   // Ignored
```

SWIG resolves overloaded functions and methods using a disambiguation scheme that ranks and sorts declarations according to a set of type-precedence rules. The order in which declarations appear in the input does not matter except in situations where ambiguity arises--in this case, the first declaration takes precedence.

Please refer to the "SWIG and C++" chapter for more information about overloading.

Dealing with the Lua coercion mechanism, the priority is roughly (integers, floats, strings, userdata). But it is better to rename the functions rather than rely upon the ordering.

### 29.3.11 C++ operators

Certain C++ overloaded operators can be handled automatically by SWIG. For example, consider a class like this:

```
class Complex {
private:
  double rpart, ipart;
public:
  Complex(double r = 0, double i = 0) : rpart(r), ipart(i) { }
  Complex(const Complex &c) : rpart(c.rpart), ipart(c.ipart) { }
  Complex &operator=(const Complex &c);
  Complex operator+(const Complex &c) const;
  Complex operator-(const Complex &c) const;
  Complex operator*(const Complex &c) const;
  Complex operator-() const;

  double re() const { return rpart; }
  double im() const { return ipart; }
};
```

When wrapped, it works like you expect:

```
> c = Complex(3, 4)
> d = Complex(7, 8)
> e = c + d
> e:re()
10.0
> e:im()
12.0
```

One restriction with operator overloading support is that SWIG is not able to fully handle operators that aren't defined as part of the class. For example, if you had code like this

```
class Complex {
...
friend Complex operator+(double, const Complex &c);
...
};
```

then SWIG doesn't know what to do with the friend function--in fact, it simply ignores it and issues a warning. You can still wrap the operator, but you may have to encapsulate it in a special function. For example:

```
%rename(Complex_add_dc) operator+(double, const Complex &);
...
Complex operator+(double, const Complex &c);
```

There are ways to make this operator appear as part of the class using the `%extend` directive. Keep reading.

Also, be aware that certain operators don't map cleanly to Lua, and some Lua operators don't map cleanly to C++ operators. For instance, overloaded assignment operators don't map to Lua semantics and will be ignored, and C++ doesn't support Lua's concatenation operator (`..` ).

In order to keep maximum compatibility within the different languages in SWIG, the Lua bindings uses the same set of operator names as Python. Although internally it renames the functions to something else (on order to work with Lua).

The current list of operators which can be overloaded (and the alternative function names) are:

- `__add__` operator+
- `__sub__` operator-
- `__mul__` operator *
- `__div__` operator/
- `__unm__` unary minus
- `__call__` operator`()` (often used in functor classes)
- `__pow__` the exponential fn (no C++ equivalent, Lua uses^)
- `__concat__` the concatenation operator (Lua's `..`)
- `__eq__` operator==
- `__lt__` operator<
- `__le__` operator<=

Note: in Lua, only the equals, less than, and less than equals operators are defined. The other operators (!=, >, >=) are achieved by using a logical not applied to the results of other operators.

The following operators cannot be overloaded (mainly because they are not supported in Lua)

- ++ and --
- +=, -=, *= etc
- % operator (you have to use math.mod)
- assignment operator
- all bitwise/logical operations

SWIG also accepts the `__str__()` member function which converts an object to a string. This function should return a const char*, preferably to static memory. This will be used for the `print()` and `tostring()` functions in Lua. Assuming the complex class has a function

```
const char* __str__() {
  static char buffer[255];
  sprintf(buffer, "Complex(%g, %g)", this->re(), this->im());
  return buffer;
}
```

Then this will support the following code in Lua

```
> c = Complex(3, 4)
> d = Complex(7, 8)
> e = c + d
> print(e)
Complex(10, 12)
> s=tostring(e) -- s is the number in string form
> print(s)
Complex(10, 12)
```

It is also possible to overload the operator`[]`, but currently this cannot be automatically performed. To overload the operator`[]` you need to provide two functions,`__getitem__()` and `__setitem__()`

```
class Complex {
  //....
  double __getitem__(int i)const; // i is the index, returns the data
  void __setitem__(int i, double d); // i is the index, d is the data
};
```

C++ operators are mapped to Lua predefined metafunctions. Class inherits from its bases the following list of metafunctions ( thus inheriting the folloging operators and pseudo-operators):

- `__add__`
- `__sub__`
- `__mul__`
- `__div__`
- `__unm__`
- `__mod__`
- `__call__`
- `__pow__`
- `__concat__`
- `__eq__`
- `__lt__`
- `__le__`
- `__len__`
- `__getitem__`
- `__setitem__`
- `__tostring` used internally by Lua for tostring() function. __str__ is mapped to this function

No other lua metafunction is inherited. For example, __gc is not inherited and must be redefined in every class. `__tostring` is subject to a special handling. If absent in class and in class bases, a default one will be provided by SWIG.

### 29.3.12 Class extension with %extend

One of the more interesting features of SWIG is that it can extend structures and classes with new methods. In the previous section, the Complex class would have benefited greatly from an __str__() method as well as some repairs to the operator overloading. It can also be used to add additional functions to the class if they are needed.

Take the original Complex class

```
class Complex {
private:
  double rpart, ipart;
public:
  Complex(double r = 0, double i = 0) : rpart(r), ipart(i) { }
  Complex(const Complex &c) : rpart(c.rpart), ipart(c.ipart) { }
  Complex &operator=(const Complex &c);
  Complex operator+(const Complex &c) const;
  Complex operator-(const Complex &c) const;
  Complex operator*(const Complex &c) const;
  Complex operator-() const;

  double re() const { return rpart; }
  double im() const { return ipart; }
};
```

Now we extend it with some new code

```
%extend Complex {
  const char *__str__() {
    static char tmp[1024];
    sprintf(tmp, "Complex(%g, %g)", $self->re(), $self->im());
    return tmp;
  }
  bool operator==(const Complex& c) {
    return ($self->re()==c.re() && $self->im()==c.im());
  }
};
```

Now, in Lua

```
> c = Complex(3, 4)
> d = Complex(7, 8)
> e = c + d
> print(e)      -- print uses __str__ to get the string form to print
Complex(10, 12)
> print(e==Complex(10, 12))  -- testing the == operator
true
> print(e!=Complex(12, 12))  -- the != uses the == operator
true
```

Extend works with both C and C++ code, on classes and structs. It does not modify the underlying object in any way---the extensions only show up in the Lua interface. The only item to take note of is the code has to use the '$self' instead of 'this', and that you cannot access protected/private members of the code (as you are not officially part of the class).

### 29.3.13 Using %newobject to release memory

If you have a function that allocates memory like this,

```
char *foo() {
  char *result = (char *) malloc(...);
  ...
  return result;
}
```

then the SWIG generated wrappers will have a memory leak--the returned data will be copied into a string object and the old contents ignored.

To fix the memory leak, use the %newobject directive.

```
%newobject foo;
...
char *foo();
```

This will release the allocated memory.

### 29.3.14 C++ templates

C++ templates don't present a huge problem for SWIG. However, in order to create wrappers, you have to tell SWIG to create wrappers for a particular template instantiation. To do this, you use the template directive. For example:

```
%module example
%{
#include "pair.h"
%}

template<class T1, class T2>
struct pair {
  typedef T1 first_type;
  typedef T2 second_type;
  T1 first;
  T2 second;
  pair();
  pair(const T1&, const T2&);
  ~pair();
};

%template(pairii) pair<int, int>;
```

In Lua:

```
> p = example.pairii(3, 4)
> print(p.first, p.second)
```

```
3    4
```

Obviously, there is more to template wrapping than shown in this example. More details can be found in the SWIG and C++ chapter. Some more complicated examples will appear later.

### 29.3.15 C++ Smart Pointers

In certain C++ programs, it is common to use classes that have been wrapped by so-called "smart pointers." Generally, this involves the use of a template class that implements operator->() like this:

```
template<class T> class SmartPtr {
  ...
  T *operator->();
  ...
}
```

Then, if you have a class like this,

```
class Foo {
public:
  int x;
  int bar();
};
```

A smart pointer would be used in C++ as follows:

```
SmartPtr<Foo> p = CreateFoo();   // Created somehow (not shown)
...
p->x = 3;                        // Foo::x
int y = p->bar();                // Foo::bar
```

To wrap this, simply tell SWIG about the SmartPtr class and the low-level Foo object. Make sure you instantiate SmartPtr using template if necessary. For example:

```
%module example
...
%template(SmartPtrFoo) SmartPtr<Foo>;
...
```

Now, in Lua, everything should just "work":

```
> p = example.CreateFoo()          -- Create a smart-pointer somehow
> p.x = 3                          -- Foo::x
> print(p:bar())                   -- Foo::bar
```

If you ever need to access the underlying pointer returned by `operator->()` itself, simply use the `__deref__()` method. For example:

```
> f = p:__deref__()     -- Returns underlying Foo *
```

### 29.3.16 C++ Exceptions

Lua does not natively support exceptions, but it has errors which are similar. When a Lua function terminates with an error it returns one value back to the caller. SWIG automatically maps any basic type which is thrown into a Lua error. Therefore for a function:

```
int message() throw(const char *) {
  throw("I died.");
  return 1;
}
```

SWIG will automatically convert this to a Lua error.

```
> message()
I died.
stack traceback:
        [C]: in function 'message'
        stdin:1: in main chunk
        [C]: ?
>
```

If you want to catch an exception, you must use either pcall() or xpcall(), which are documented in the Lua manual. Using xpcall will allow you to obtain additional debug information (such as a stacktrace).

```
> function a() b() end -- function a() calls function b()
> function b() message() end -- function b() calls C++ function message(), which throws
> ok, res=pcall(a)  -- call the function
> print(ok, res)
false   I died.
> ok, res=xpcall(a, debug.traceback)  -- call the function
> print(ok, res)
false   I died.
stack traceback:
        [C]: in function 'message'
        runme.lua:70: in function 'b'
        runme.lua:67: in function <runme.lua:66>
        [C]: in function 'xpcall'
        runme.lua:95: in main chunk
        [C]: ?
```

SWIG is able to throw numeric types, enums, chars, char*'s and std::string's without problem. It has also written typemaps for std::exception and its derived classes, which convert the exception into an error string.

However it's not so simple to throw other types of objects. Thrown objects are not valid outside the 'catch' block. Therefore they cannot be returned to the interpreter. The obvious ways to overcome this would be to either return a copy of the object, or to convert the object to a string and return that. Though it seems obvious to perform the former, in some cases this is not possible, most notably when SWIG has no information about the object, or the object is not copyable/creatable.

Therefore by default SWIG converts all thrown object into strings and returns them. So given a function:

```
void throw_A() throw(A*) {
  throw new A();
}
```

SWIG will just convert it (poorly) to a string and use that as its error. (This is not that useful, but it always works).

```
> throw_A()
object exception:A *
stack traceback:
        [C]: in function 'unknown'
        stdin:1: in main chunk
        [C]: ?
>
```

To get a more useful behaviour out of SWIG you must either: provide a way to convert your exceptions into strings, or throw objects which can be copied.

If you have your own class which you want output as a string you will need to add a typemap something like this:

```
%typemap(throws) my_except
%{
  lua_pushstring(L, $1.what()); // assuming my_except::what() returns a const char* message
  SWIG_fail; // trigger the error handler
%}
```

If you wish your exception to be returned to the interpreter, it must firstly be copyable. Then you must have an additional `%apply` statement, to tell SWIG to return a copy of this object to the interpreter. For example:

```
%apply SWIGTYPE EXCEPTION_BY_VAL {Exc}; // tell SWIG to return Exc by value to interpreter

class Exc {
public:
  Exc(int c, const char *m) {
    code = c;
    strncpy(msg, m, 256);
  }
  int code;
  char msg[256];
};

void throw_exc() throw(Exc) {
  throw(Exc(42, "Hosed"));
}
```

Then the following code can be used (note: we use pcall to catch the error so we can process the exception).

```
> ok, res=pcall(throw_exc)
> print(ok)
false
> print(res)
userdata: 0003D880
> print(res.code, res.msg)
42      Hosed
>
```

Note: it is also possible (though tedious) to have a function throw several different kinds of exceptions. To process this will require a pcall, followed by a set of if statements checking the type of the error.

All of this code assumes that your C++ code uses exception specification (which a lot doesn't). If it doesn't consult the " Exception handling with %catches" section and the " Exception handling with %exception" section, for more details on how to add exception specification to functions or globally (respectively).

### 29.3.17 Namespaces

C++ namespaces are supported via the %nspace feature.

Namespaces are mapped into Lua tables. Each of those tables contains names that were defined within an appropriate namespace. Namespace hierarchies (a.k.a nested namespaces) are preserved. Consider the following C++ code:

```
%module example

%nspace MyWorld::World;
%nspace MyWorld::Nested::Dweller;

int module_function() { return 7; }
int module_variable = 9;

namespace MyWorld {
  class World {
  public:
    World() : world_max_count(9) {}
    int create_world() { return 17; }
    const int world_max_count; // = 9
  };
  namespace Nested {
    class Dweller {
```

```
    public:
      enum Gender { MALE = 0, FEMALE = 1 };
      static int count() { return 19; }
  };
 }
}
```

Now, from Lua usage is as follows:

```
> print(example.module_function())
7.0
> print(example.module_variable)
9.0
> print(example.MyWorld.World():create_world())
17.0
> print(example.MyWorld.World().world_max_count)
9.0
> print(example.MyWorld.Nested.Dweller.MALE)
0
> print(example.MyWorld.Nested.Dweller.count())
19.0
```

The hierarchies in Lua are the same as the C++ hierarchies when using `%nspace`, however, we could instead completely change these hierarchies with `%nspacemove`. Consider the code above with the two `%nspace` lines of code removed and replaced with the following:

```
%nspacemove(DifferentWorld::SubSpace1::SubSpace2) MyWorld::World;
%nspacemove(DifferentWorld) MyWorld::Nested::Dweller;
```

The Lua code uses the completely modified hierarchies:

```
> print(example.module_function())
7.0
> print(example.module_variable)
9.0
> print(example.DifferentWorld.SubSpace1.SubSpace2.World():create_world())
17.0
> print(example.DifferentWorld.SubSpace1.SubSpace2.World().world_max_count)
9.0
> print(example.DifferentWorld.Dweller.MALE)
0
> print(example.DifferentWorld.Dweller.count())
19.0
```

#### 29.3.17.1 Compatibility Note

The nspace feature was first supported by the Lua module in SWIG-3.0.0

If SWIG is running in a backwards compatible way, i.e. without the `-no-old-metatable-bindings` option, then additional old-style names are generated (notice the underscore):

```
> print(example.MyWorld.Nested.Dweller_MALE)
0
> print(example.MyWorld.Nested.Dweller_count())
19.0
```

The ability to move symbols into different namespaces via `%nspacemove` was introduced in SWIG-4.3.0.

#### 29.3.17.2 Names

If SWIG is launched without `-no-old-metatable-bindings` option, then it enters backward-compatible mode. While in this mode, it tries to generate additional names for static functions, class static constants and class enums. Those names are in a form `$classname_$symbolname` and are added to the scope surrounding the class. If %nspace is enabled, then class namespace is taken as scope. If there is no namespace, or %nspace is disabled, then module is considered a class namespace.

Consider the following C++ code

```
%module example
%nspace MyWorld::Test;
namespace MyWorld {
class Test {
  public:
  enum { TEST1 = 10, TEST2 }
  static const int ICONST = 12;
};
class Test2 {
  public:
  enum { TEST3 = 20, TEST4 }
  static const int ICONST2 = 23;
}
```

When in backward compatible mode, in addition to the usual names, the following ones will be generated (notice the underscore):

```
9
> print(example.MyWorld.Test_TEST1) -- Test has %nspace enabled
10
> print(example.MyWorld.Test_ICONST) -- Test has %nspace enabled
12
> print(example.Test2_TEST3) -- Test2 doesn't have %nspace enabled
20
> print(example.Test2_ICONST2) -- Test2 doesn't have %nspace enabled
23
>
```

There is a slight difference with enums when in C mode. As per C standard, enums from C structures are exported to surrounding scope without any prefixing. Pretending that Test2 is a struct, not class, that would be:

```
> print(example.TEST3) -- NOT Test2_TEST3
20
>
```

#### 29.3.17.3 Inheritance

The internal organization of inheritance has changed. Consider the following C++ code:

```
%module example
class Base {
  public:
  int base_func()
};
class Derived : public Base {
  public:
  int derived_func()
}
```

Lets assume for a moment that class member functions are stored in `.fn` table. Previously, when classes were exported to Lua during module initialization, for every derived class all service tables `ST(i.e. ".fn")` were squashed and added to corresponding derived class `ST`: Everything from `.fn` table of class Base was copied to `.fn` table of class Derived and so on. This was a recursive procedure, so in the end the whole inheritance tree of derived class was squashed into derived class.

That means that any changes done to class Base after module initialization wouldn't affect class Derived:

```
base = example.Base()
der = example.Derived()
> print(base.base_func)
function: 0x1367940
> getmetatable(base)[".fn"].new_func = function (x) return x -- Adding new function to class Base (to class, not to an instance!)
> print(base.new_func) -- Checking this function
function
> print(der.new_func) -- Wouldn't work. Derived doesn't check Base any more.
nil
>
```

This behaviour was changed. Now unless -squash-bases option is provided, Derived stores a list of its bases and if some symbol is not found in its own service tables then its bases are searched for it. Option -squash-bases will effectively return old behaviour.

```
> print(der.new_func) -- Now it works
function
>
```

## 29.4 Typemaps

This section explains what typemaps are and how to use them. The default wrapping behaviour of SWIG is enough in most cases. However sometimes SWIG may need a little additional assistance to know which typemap to apply to provide the best wrapping. This section will be explaining how to use typemaps to best effect

### 29.4.1 What is a typemap?

A typemap is nothing more than a code generation rule that is attached to a specific C datatype. For example, to convert integers from Lua to C, you might define a typemap like this:

```
%module example

%typemap(in) int {
  $1 = (int) lua_tonumber(L, $input);
  printf("Received an integer : %d\n", $1);
}
%inline %{
extern int fact(int n);
%}
```

*Note: you shouldn't use this typemap, as SWIG already has a typemap for this task. This is purely for example.*

Typemaps are always associated with some specific aspect of code generation. In this case, the "in" method refers to the conversion of input arguments to C/C++. The datatype int is the datatype to which the typemap will be applied. The supplied C code is used to convert values. In this code a number of special variable prefaced by a $ are used. The $1 variable is placeholder for a local variable of type int. The $input is the index on the Lua stack for the value to be used.

When this example is compiled into a Lua module, it operates as follows:

```
> require "example"
> print(example.fact(6))
Received an integer : 6
720
```

### 29.4.2 Using typemaps

There are many ready written typemaps built into SWIG for all common types (int, float, short, long, char*, enum and more), which SWIG uses automatically, with no effort required on your part.

However for more complex functions which use input/output parameters or arrays, you will need to make use of <typemaps.i>, which contains typemaps for these situations. For example, consider these functions:

```
void add(int x, int y, int *result) {
  *result = x + y;
}
```

```
int sub(int *x1, int *y1) {
  return *x1-*y1;
}

void swap(int *sx, int *sy) {
  int t=*sx;
  *sx=*sy;
  *sy=t;
}
```

It is clear to the programmer, that 'result' is an output parameter, 'x1' and 'y1' are input parameters and 'sx' and 'sy' are input/output parameters. However is not apparent to SWIG, so SWIG must to informed about which kind they are, so it can wrapper accordingly.

One means would be to rename the argument name to help SWIG, eg`void add(int x, int y, int *OUTPUT)`, however it is easier to use the `%apply` to achieve the same result, as shown below.

```
%include <typemaps.i>
%apply int* OUTPUT {int *result}; // int *result is output
%apply int* INPUT {int *x1, int *y1}; // int *x1 and int *y1 are input
%apply int* INOUT {int *sx, int *sy}; // int *sx and int *sy are input and output

void add(int x, int y, int *result);
int sub(int *x1, int *y1);
void swap(int *sx, int *sy);
```

When wrapped, it gives the following results:

```
> require "example"
> print(example.add(1, 2))
3
> print(demo.sub(1, 2))
-1
> a, b=1, 2
> c, d=demo.swap(a, b)
> print(a, b, c, d)
1       2       2       1
```

Notice, that 'result' is not required in the arguments to call the function, as it an output parameter only. For 'sx' and 'sy' they must be passed in (as they are input), but the original value is not modified (Lua does not have a pass by reference feature). The modified results are then returned as two return values. All INPUT/OUTPUT/INOUT arguments will behave in a similar manner.

Note: C++ references must be handled exactly the same way. However SWIG will automatically wrap a `const int&` as an input parameter (since that it obviously input).

### 29.4.3 Typemaps and arrays

Arrays present a challenge for SWIG, because like pointers SWIG does not know whether these are input or output values, nor does SWIG have any indication of how large an array should be. However with the proper guidance SWIG can easily wrapper arrays for convenient usage.

Given the functions:

```
extern void sort_int(int* arr, size_t len);
extern void sort_double(double* arr, size_t len);
```

There are basically two ways that SWIG can deal with this. The first way, uses the `<carrays.i>` library to create an array in C/C++ then this can be filled within Lua and passed into the function. It works, but it's a bit tedious. More details can be found in the carrays.i documentation.

The second and more intuitive way, would be to pass a Lua table directly into the function, and have SWIG automatically convert between Lua-table and C-array. Within the `<typemaps.i>` file there are typemaps ready written to perform this task. To use them is again a matter of using %apply in the correct manner.

The wrapper file below, shows both the use of carrays as well as the use of the typemap to wrap arrays.

```
// using the C-array
%include <carrays.i>
// this declares a batch of functions for manipulating C integer arrays
%array_functions(int, int)

extern void sort_int(int* arr, size_t len); // the function to wrap

// using typemaps
%include <typemaps.i>
%apply (double *INOUT, int) {(double* arr, size_t len)};

extern void sort_double(double* arr, size_t len); // the function to wrap
```

Once wrapped, the functions can both be called, though with different ease of use:

```
require "example"
ARRAY_SIZE=10

-- passing a C array to the sort_int()
arr=example.new_int(ARRAY_SIZE) -- create the array
for i=0, ARRAY_SIZE-1 do -- index 0..9 (just like C)
    example.int_setitem(arr, i, math.random(1000))
end
example.sort_int(arr, ARRAY_SIZE)  -- call the function
example.delete_int(arr) -- must delete the allocated memory

-- use a typemap to call with a Lua-table
-- one item of note: the typemap creates a copy, rather than edit-in-place
t={} -- a Lua table
for i=1, ARRAY_SIZE do -- index 1..10 (Lua style)
    t[i]=math.random(1000)/10
end
t=example.sort_double(t) -- replace t with the result
```

Obviously the first version could be made less tedious by writing a Lua function to perform the conversion from a table to a C-array. The `%luacode` directive is good for this. See SWIG\Examples\lua\arrays for an example of this.

**Warning:** in C indexes start at ZERO, in Lua indexes start at ONE. SWIG expects C-arrays to be filled for 0..N-1 and Lua tables to be 1..N, (the indexing follows the norm for the language). In the typemap when it converts the table to an array it quietly changes the indexing accordingly. Take note of this behaviour if you have a C function which returns indexes.

Note: SWIG also can support arrays of pointers in a similar manner.

### 29.4.4 Typemaps and pointer-pointer functions

Several C++ libraries use a pointer-pointer functions to create its objects. These functions require a pointer to a pointer which is then filled with the pointer to the new object. Microsoft's COM and DirectX as well as many other libraries have this kind of function. An example is given below:

```
struct iMath;    // some structure
int Create_Math(iMath** pptr); // its creator (assume it mallocs)
```

Which would be used with the following C code:

```
iMath* ptr;
int ok;
ok=Create_Math(&ptr);
// do things with ptr
//...
free(ptr); // dispose of iMath
```

SWIG has a ready written typemap to deal with such a kind of function in <typemaps.i>. It provides the correct wrapping as well as setting the flag to inform Lua that the object in question should be garbage collected. Therefore the code is simply:

```
%include <typemaps.i>
%apply SWIGTYPE** OUTPUT{iMath **pptr }; // tell SWIG it's an output

struct iMath;    // some structure
int Create_Math(iMath** pptr); // its creator (assume it mallocs)
```

The usage is as follows:

```
ok, ptr=Create_Math() -- ptr is an iMath* which is returned with the int (ok)
ptr=nil -- the iMath* will be GC'ed as normal
```

## 29.5 Writing typemaps

This section describes how you can modify SWIG's default wrapping behavior for various C/C++ datatypes using the `%typemap` directive. This is an advanced topic that assumes familiarity with the Lua C API as well as the material in the "Typemaps " chapter.

Before proceeding, it should be stressed that writing typemaps is rarely needed unless you want to change some aspect of the wrapping, or to achieve an effect which in not available with the default bindings.

Before proceeding, you should read the previous section on using typemaps, and look at the existing typemaps found in luatypemaps.swg and typemaps.i. These are both well documented and fairly easy to read. You should not attempt to write your own typemaps until you have read and can understand both of these files (they may well also give you an idea to base your work on).

### 29.5.1 Typemaps you can write

There are many different types of typemap that can be written, the full list can be found in the "Typemaps" chapter. However the following are the most commonly used ones.

- `in` this is for input arguments to functions
- `out` this is for return types from functions
- `argout` this is for a function argument which is actually returning something
- `typecheck` this is used to determine which overloaded function should be called (the syntax for the typecheck is different from the typemap, see typemaps for details).

### 29.5.2 SWIG's Lua-C API

This section explains the SWIG specific Lua-C API. It does not cover the main Lua-C api, as this is well documented and not worth covering.

`int SWIG_ConvertPtr(lua_State* L, int index, void** ptr, swig_type_info *type, int flags);`

This is the standard function used for converting a Lua userdata to a void*. It takes the value at the given index in the Lua state and converts it to a userdata. It will then provide the necessary type checks, confirming that the pointer is compatible with the type given in 'type'. Then finally setting '*ptr' to the pointer. If flags is set to SWIG_POINTER_DISOWN, this is will clear any ownership flag set on the object.
This returns a value which can be checked with the macro SWIG_IsOK()

`void SWIG_NewPointerObj(lua_State* L, void* ptr, swig_type_info *type, int own);`

This is the opposite of SWIG_ConvertPtr, as it pushes a new userdata which wrappers the pointer 'ptr' of type 'type'. The parameter 'own' specifies if the object is owned be Lua and if it is 1 then Lua will GC the object when the userdata is disposed of.

`void* SWIG_MustGetPtr(lua_State* L, int index, swig_type_info *type, int flags, int argnum, const char* func_name);`

This function is a version of SWIG_ConvertPtr(), except that it will either work, or it will trigger a lua_error() with a text error message. This function is rarely used, and may be deprecated in the future.

`SWIG_fail`

This macro, when called within the context of a SWIG wrapped function, will jump to the error handler code. This will call any cleanup code (freeing any temp variables) and then triggers a lua_error.
A common use for this code is:

```
if (!SWIG_IsOK(SWIG_ConvertPtr( .....)){
 lua_pushstring(L, "something bad happened");
 SWIG_fail;
}
```

`SWIG_fail_arg(char* func_name, int argnum, char* type)`

This macro, when called within the context of a SWIG wrapped function, will display the error message and jump to the error handler code. The error message is of the form

```
    "Error in func_name (arg argnum), expected 'type' got 'whatever the type was'"
SWIG_fail_ptr(const char* fn_name, int argnum, swig_type_info* type);
```

Similar to SWIG_fail_arg, except that it will display the swig_type_info information instead.

## 29.6 Customization of your Bindings

This section covers adding of some small extra bits to your module to add the last finishing touches.

### 29.6.1 Writing your own custom wrappers

Sometimes, it may be necessary to add your own special functions, which bypass the normal SWIG wrapper method, and just use the native Lua API calls. These 'native' functions allow direct adding of your own code into the module. This is performed with the `%native` directive as follows:

```
%native(my_func) int native_function(lua_State*L);  // registers native_function() with SWIG
...
%{
int native_function(lua_State*L) // my native code
{
 ...
}
%}
```

The `%native` directive in the above example, tells SWIG that there is a function `int native_function(lua_State*L);` which is to be added into the module under the name '`my_func`'. SWIG will not add any wrapper for this function, beyond adding it into the function table. How you write your code is entirely up to you.

### 29.6.2 Adding additional Lua code

As well as adding additional C/C++ code, it's also possible to add your own Lua code to the module as well. This code is executed once all other initialisation, including the %init code has been called.

The directive `%luacode` adds code into the module which is executed upon loading. Normally you would use this to add your own functions to the module. Though you could easily perform other tasks.

```
%module example;

%luacode {
  function example.greet()
    print "hello world"
  end

  print "Module loaded ok"
}
...
%}
```

Notice that the code is not part of the module table. Therefore any references to the module must have the module name added.

Should there be an error in the Lua code, this will *not* stop loading of the module. The default behaviour of SWIG is to print an error message to stderr and then continue. It is possible to change this behaviour by using a `#define SWIG_DOSTRING_FAIL(STR)` to define a different behaviour should the code fail.

Good uses for this feature is adding of new code, or writing helper functions to simplify some of the code. See Examples/lua/arrays for an example of this code.

## 29.7 Details on the Lua binding

In the previous section, a high-level view of Lua wrapping was presented. Obviously a lot of stuff happens behind the scenes to make this happen. This section will explain some of the low-level details on how this is achieved.

*If you just want to use SWIG and don't care how it works, then stop reading here. This is going into the guts of the code and how it works. It's mainly for people who need to know what's going on within the code.*

### 29.7.1 Binding global data into the module.

Assuming that you had some global data that you wanted to share between C and Lua. How does SWIG do it?

```
%module example;
extern double Foo;
```

SWIG will effectively generate the pair of functions

```
void Foo_set(double);
double Foo_get();
```

At initialisation time, it will then add to the interpreter a table called 'example', which represents the module. It will then add all its functions to the module. (Note: older versions of SWIG actually added the Foo_set() and Foo_get() functions, current implementation does not add these functions any more.) But it also adds a metatable to this table, which has two functions (`__index` and `__newindex` ) as well as two tables (`.get` and `.set`) The following Lua code will show these hidden features.

```
> print(example)
table: 003F8F90
> m=getmetatable(example)
> table.foreach(m, print)
.set    table: 003F9088
.get    table: 003F9038
__index function: 003F8FE0
__newindex      function: 003F8FF8
> g=m['.get']
> table.foreach(g, print)
Foo     function: 003FAFD8
>
```

The .get and .set tables are lookups connecting the variable name 'Foo' to the accessor/mutator functions (Foo_set, Foo_get)

The Lua equivalent of the code for the `__index` and `__newindex` looks a bit like this

```
function __index(mod, name)
        local g=getmetatable(mod)['.get'] -- gets the table
        if not g then return nil end
        local f=g[name] -- looks for the function
        -- calls it & returns the value
        if type(f)=="function" then return f() end
        return nil
end

function __newindex(mod, name, value)
        local s=getmetatable(mod)['.set'] -- gets the table
        if not s then return end
        local f=s[name] -- looks for the function
        -- calls it to set the value
        if type(f)=="function" then f(value)
        else rawset(mod, name, value) end
end
```

That way when you call '`a=example.Foo`', the interpreter looks at the table 'example' sees that there is no field 'Foo' and calls __index. This will in turn check in '.get' table and find the existence of 'Foo' and then return the value of the C function call 'Foo_get()'. Similarly for the code '`example.Foo=10`', the interpreter will check the table, then call the __newindex which will then check the '.set' table and call the C function 'Foo_set(10)'.

### 29.7.2 Userdata and Metatables

As mentioned earlier, classes and structures, are all held as pointer, using the Lua 'userdata' structure. This structure is actually a pointer to a C structure 'swig_lua_userdata', which contains the pointer to the data, a pointer to the swig_type_info (an internal SWIG struct) and a flag which marks if the object is to be disposed of when the interpreter no longer needs it. The actual accessing of the object is done via the metatable attached to this userdata.

The metatable is a Lua 5.0 feature (which is also why SWIG cannot wrap Lua 4.0). It's a table which holds a list of functions, operators and attributes. This is what gives the userdata the feeling that it is a real object and not just a hunk of memory.

Given a class

```
%module excpp;

class Point
{
public:
 int x, y;
 Point(){x=y=0;}
 ~Point(){}
 virtual void Print(){printf("Point @%p (%d, %d)\n", this, x, y);}
};
```

SWIG will create a module excpp, with all the various functions inside. However to allow the intuitive use of the userdata, SWIG also creates up a set of metatables. As seen in the above section on global variables, use of the metatables allows for wrappers to be used intuitively. To save effort, the code creates one metatable per class and stores it inside Lua's registry. Then when a new object is instantiated, the metatable is found in the registry and the userdata associated with the metatable. Currently, derived classes make a complete copy of the base class' table and then add on their own additional functions.

Some of the internals can be seen by looking at the metatable of a class:

```
> p=excpp.Point()
> print(p)
userdata: 003FDB28
> m=getmetatable(p)
> table.foreach(m, print)
.type   Point
__gc    function: 003FB6C8
__newindex      function: 003FB6B0
__index function: 003FB698
.get    table: 003FB4D8
.set    table: 003FB500
.fn     table: 003FB528
```

The '.type' attribute is the name of the class. The '.get' and '.set' tables work in a similar manner to the modules, the main difference is the '.fn' table which also holds all the member functions. (The '__gc' function is the class' destructor function)

The Lua equivalent of the code for enabling functions looks a little like this

```
function __index(obj, name)
        local m=getmetatable(obj) -- gets the metatable
        if not m then return nil end
        local g=m['.get'] -- gets the attribute table
        if not g then return nil end
        local f=g[name] -- looks for the get_attribute function
        -- calls it & returns the value
        if type(f)=="function" then return f() end
        -- ok, so it not an attribute, maybe it's a function
        local fn=m['.fn'] -- gets the function table
        if not fn then return nil end
        local f=fn[name] -- looks for the function
        -- if found the fn then return the function
        -- so the interpreter can call it
        if type(f)=="function" then return f end
        return nil
end
```

So when 'p:Print()' is called, the __index looks on the object metatable for a 'Print' attribute, then looks for a 'Print' function. When it finds the function, it returns the function, and then interpreter can call 'Point_Print(p)'

In theory, you can play with this usertable & add new features, but remember that it is a shared table between all instances of one class, and you could very easily corrupt the functions in all the instances.

Note: Both the opaque structures (like the FILE*) and normal wrapped classes/structs use the same 'swig_lua_userdata' structure. Though the opaque structures do not have a metatable attached, or any information on how to dispose of them when the interpreter has finished with them.

Note: Operator overloads are basically done in the same way, by adding functions such as '__add' & '__call' to the class' metatable. The current implementation is a bit rough as it will add any member function beginning with '__' into the metatable too, assuming it's an operator overload.

### 29.7.3 Memory management

Lua is very helpful with the memory management. The 'swig_lua_userdata' is fully managed by the interpreter itself. This means that neither the C code nor the Lua code can damage it. Once a piece of userdata has no references to it, it is not instantly collected, but will be collected when Lua deems is necessary. (You can force collection by calling the Lua function `collectgarbage()`). Once the userdata is about to be free'ed, the interpreter will check the userdata for a metatable and for a function '__gc'. If this exists this is called. For all complete types (ie normal wrapped classes & structs) this should exist. The '__gc' function will check the 'swig_lua_userdata' to check for the 'own' field and if this is true (which is will be for all owned data) it will then call the destructor on the pointer.

It is currently not recommended to edit this field or add some user code, to change the behaviour. Though for those who wish to try, here is where to look.

It is also currently not possible to change the ownership flag on the data (unlike most other scripting languages, Lua does not permit access to the data from within the interpreter).

---

# 30 SWIG and Octave


- Preliminaries
- Running SWIG
  - Command-line options
  - Compiling a dynamic module
  - Using your module
- A tour of basic C/C++ wrapping
  - Modules
  - Functions
  - Global variables
  - Constants and enums
  - Pointers
  - Structures and C++ classes
  - C++ inheritance
  - C++ overloaded functions
  - C++ operators
  - Class extension with %extend
  - C++ templates
  - C++ Smart Pointers
    - The shared_ptr Smart Pointer
    - Generic Smart Pointers
  - Directors (calling Octave from C++ code)
  - Threads
  - Memory management
  - STL support
  - Matrix typemaps


Octave is a high-level language intended for numerical programming that is mostly compatible with MATLAB. More information can be found at Octave web site.

This chapter is intended to give an introduction to using the module. You should also read the SWIG documentation that is not specific to Octave. Also, there are a dozen or so examples in the Examples/octave directory, and hundreds in the test suite (Examples/test-suite and Examples/test-suite/octave).

## 30.1 Preliminaries

SWIG is regularly tested against the following versions of Octave: 5.2, 6.4. SWIG 4.2.1 has also been manually tested with 7.3, 8.4 and 9.0.

Every effort is made to maintain backward compatibility with older versions of Octave. This cannot be guaranteed however, as in recent times new Octave releases have required nontrivial updates to SWIG, which may break backward compatibility for older Octave versions against which SWIG is not regularly tested.

The SWIG runtime exports the function `swig_octave_prereq()` for checking the version of Octave.

## 30.2 Running SWIG

Let's start with a very simple SWIG interface file, example.i:

```
%module swigexample
%{
#include "example.h"
%}
int gcd(int x, int y);
extern double Foo;
```

To build an Octave module when wrapping C code, run SWIG using the `-octave` option:

```
$ swig -octave -o example_wrap.cpp example.i
```

The `-c++` option is also required when wrapping C++ code:

```
$ swig -octave -c++ -o example_wrap.cpp example.i
```

### 30.2.1 Command-line options

The swig command line has a number of options you can use, like to redirect its output. Use `swig -help` to learn about these. Options specific to the Octave module are:

```
$ swig -octave -help
...
Octave Options (available with -octave)
     -globals name - Set name used to access C global variables [default: 'cvar']
                     Use '.' to load C global variables into module namespace
     -opprefix str - Prefix str for global operator functions [default: 'op_']
```

The *-globals* option sets the name of the variable which is the namespace for C global variables exported by the module. The special name "." loads C global variables into the module namespace, i.e. alongside C functions and structs exported by the module. The *-opprefix* options sets the prefix of the names of global/friendoperator functions.

### 30.2.2 Compiling a dynamic module

Octave modules are DLLs/shared objects having the ".oct" suffix. Building an oct file is usually done with the mkoctfile command (either within Octave itself, or from the shell). For example,

```
$ swig -octave -c++ -o example_wrap.cpp example.i
$ mkoctfile example_wrap.cpp example.c
```

where "example.c" is the file containing the gcd() implementation.

mkoctfile can also be used to extract the build parameters required to invoke the compiler and linker yourself. See the Octave manual and mkoctfile man page.

mkoctfile will produce "swigexample.oct", which contains the compiled extension module. Loading it into Octave is then a matter of invoking

```
octave:1> swigexample
```

### 30.2.3 Using your module

Assuming all goes well, you will be able to do this:

```
$ octave -q
octave:1> swigexample
octave:2> swigexample.gcd(4, 6)
ans =  2
octave:3> swigexample.cvar.Foo
ans =  3
octave:4> swigexample.cvar.Foo=4;
octave:5> swigexample.cvar.Foo
ans =  4
```

## 30.3 A tour of basic C/C++ wrapping

### 30.3.1 Modules

The SWIG module directive specifies the name of the Octave module. If you specify "module swigexample", then in Octave everything in the module will be accessible under "swigexample", as in the above example. When choosing a module name, make sure you don't use the same name as a built-in Octave command or standard module name.

When Octave is asked to invoke `swigexample`, it will try to find the ".m" or ".oct" file that defines the function "swigexample". You therefore need to make sure that "swigexample.oct" is in Octave's search path, which can be specified with the environment variable "OCTAVE_PATH".

To load an Octave module, simply type its name:

```
octave:1> swigexample;
octave:2> gcd(4, 6)
ans =  2
octave:3> cvar.Foo
ans =  3
octave:4> cvar.Foo=4;
octave:5> cvar.Foo
ans =  4
```

Modules can also be loaded from within functions, even before being loaded in the base context. If the module is also used in the base context, however, it must first be loaded again:

```
octave:1> function l = my_lcm(a, b)
> swigexample
> l = abs(a*b)/swigexample.gcd(a, b);
> endfunction
octave:2> my_lcm(4, 6)
ans =  12
octave:3> swigexample.gcd(4, 6)
error: can't perform indexing operations for <unknown type> type
octave:3> swigexample;
octave:4> swigexample.gcd(4, 6)
ans =  2
```

### 30.3.2 Functions

Global functions are wrapped as new Octave built-in functions. For example,

```
%module swigexample
int fact(int n);
```

creates a built-in function `swigexample.fact(n)` that works exactly like you think it does:

```
octave:1> swigexample.fact(4)
24
```

### 30.3.3 Global variables

Global variables are a little special in Octave. Given a global variable:

```
%module swigexample
extern double Foo;
```

To expose variables, SWIG actually generates two functions, to get and set the value. In this case, Foo_set and Foo_set would be generated. SWIG then automatically calls these functions when you get and set the variable-- in the former case creating a local copy in the interpreter of the C variables, and in the latter case copying an interpreter variables onto the C variable.

```
octave:1> swigexample;
octave:2> c=swigexample.cvar.Foo
c =  3
octave:3> swigexample.cvar.Foo=4;
octave:4> c
c =  3
octave:5> swigexample.cvar.Foo
ans =  4
```

If a variable is marked with the %immutable directive then any attempts to set this variable will cause an Octave error. Given a global variable:

```
%module swigexample
%immutable;
extern double Foo;
%mutable;
```

SWIG will allow the reading of `Foo` but when a set attempt is made, an error function will be called.

```
octave:1> swigexample
octave:2> swigexample.Foo=4
error: attempt to set immutable member variable
error: assignment failed, or no method for `swig_type = scalar'
error: evaluating assignment expression near line 2, column 12
```

It is possible to add new functions or variables to the module. This also allows the user to rename/remove existing functions and constants (but not linked variables, mutable or immutable). Therefore users are recommended to be careful when doing so.

```
octave:1> swigexample;
octave:2> swigexample.PI=3.142;
octave:3> swigexample.PI
ans =  3.1420
```

### 30.3.4 Constants and enums

Because Octave doesn't really have the concept of constants, C/C++ constants are not really constant in Octave. They are actually just a copy of the value into the Octave interpreter. Therefore they can be changed just as any other value. For example given some constants:

```
%module swigexample
%constant int ICONST=42;
#define    SCONST      "Hello World"
enum Days{SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY};
```

This is 'effectively' converted into the following Octave code:

```
swigexample.ICONST=42
swigexample.SCONST="Hello World"
swigexample.SUNDAY=0
....
```

### 30.3.5 Pointers

C/C++ pointers are fully supported by SWIG. Furthermore, SWIG has no problem working with incomplete type information. Given a wrapping of the <file.h> interface: C/C++ pointers are fully supported by SWIG. Furthermore, SWIG has no problem working with incomplete type information. Given a wrapping of the <file.h> interface:

```
%module swigexample
FILE *fopen(const char *filename, const char *mode);
int fputs(const char *, FILE *);
int fclose(FILE *);
```

When wrapped, you will be able to use the functions in a natural way from Octave. For example:

```
octave:1> swigexample;
octave:2> f=swigexample.fopen("w", "junk");
octave:3> swigexample.fputs("Hello world", f);
octave:4> swigexample.fclose(f);
```

Simply printing the value of a wrapped C++ type will print its typename. E.g.,

```
octave:1> swigexample;
octave:2> f=swigexample.fopen("junk", "w");
octave:3> f
f =

{
  _p_FILE, ptr = 0x9b0cd00
}
```

As the user of the pointer, you are responsible for freeing it, or closing any resources associated with it (just as you would in a C program). This does not apply so strictly to classes and structs (see below).

```
octave:1> swigexample;
octave:2> f=swigexample.fopen("not there", "r");
```

```
error: value on right hand side of assignment is undefined
error: evaluating assignment expression near line 2, column 2
```

NULL C/C++ pointers are represented by the Octave null matrix, `[]`.

### 30.3.6 Structures and C++ classes

SWIG wraps C structures and C++ classes by using a special Octave type called a `swig_ref`. A `swig_ref` contains a reference to one or more instances of C/C++ objects, or just the type information for an object. For each wrapped structure and class, a `swig_ref` will be exposed that has the name of the type. When invoked as a function, it creates a new object of its type and returns a `swig_ref` that points to that instance. This provides a very natural interface. For example,

```
struct Point{
  int x, y;
};
```

is used as follows:

```
octave:1> swigexample;
octave:2> p=swigexample.Point();
octave:3> p.x=3;
octave:4> p.y=5;
octave:5> p.x, p.y
ans =  3
ans =  5
```

In C++, invoking the type object in this way calls the object's constructor. `swig_ref` objects can also be acquired by having a wrapped function return a pointer, reference, or value of a non-primitive type.

The swig_ref type handles indexing operations such that usage maps closely to what you would have in C/C++. Structure members are accessed as in the above example, by calling set and get methods for C++ variables. Methods also work as expected. For example, code wrapped in the following way

```
class Point{
public:
  int x, y;
  Point(int _x, int _y) : x(_x), y(_y) {}
  double distance(const Point& rhs) {
    return sqrt(pow(x-rhs.x, 2)+pow(y-rhs.y, 2));
  }
  void set(int _x, int _y) {
    x=_x; y=_y;
  }
};
```

can be used from Octave like this

```
octave:1> swigexample;
octave:2> p1=swigexample.Point(3, 5);
octave:3> p2=swigexample.Point(1, 2);
octave:4> p1.distance(p2)
ans =  3.6056
```

By using the `swig_this()` and `swig_type()` functions, one can discover the pointers to and types of the underlying C/C++ object.

```
octave:5> swig_this(p1)
ans = 162504808
octave:6> swig_type(p1)
ans = Point
```

Note that `swig_ref` is a reference-counted pointer to a C/C++ object/type, and as such has pass-by-reference semantics. For example if one has a allocated a single object but has two `swig_ref`'s pointing to it, modifying the object through either of them will change the single allocated object. This differs from the usual pass-by-value (copy-on-write) semantics that Octave maintains for built-in types. For example, in the following snippet, modifying `b` does not modify `a`,

```
octave:7> a=struct('x', 4)
a =
{
  x =  4
}

octave:8> b=a
b =
{
  x =  4
}

octave:9> b.y=4
b =
{
  x =  4
  y =  4
}

octave:10> a
a =
{
  x =  4
}
```

However, when dealing with wrapped objects, one gets the behavior

```
octave:2> a=Point(3, 5)
```

```
a =

{
  Point, ptr = 0x9afbbb0
}

octave:3> b=a
b =

{
  Point, ptr = 0x9afbbb0
}

octave:4> b.set(2, 1);
octave:5> b.x, b.y
ans =  2
ans =  1
octave:6> a.x, a.y
ans =  2
ans =  1
```

Depending on the ownership setting of a `swig_ref`, it may call C++ destructors when its reference count goes to zero. See the section on memory management below for details.

### 30.3.7 C++ inheritance

Single and multiple inheritance are fully supported. The `swig_ref` type carries type information along with any C++ object pointer it holds. This information contains the full class hierarchy. When an indexing operation (such as a method invocation) occurs, the tree is walked to find a match in the current class as well as any of its bases. The lookup is then cached in the `swig_ref`.

### 30.3.8 C++ overloaded functions

Overloaded functions are supported, and handled as in other modules. That is, each overload is wrapped separately (under internal names), and a dispatch function is also emitted under the external/visible name. The dispatch function selects which overload to call (if any) based on the passed arguments. `typecheck` typemaps are used to analyze each argument, as well as assign precedence. See the chapter on typemaps for details.

### 30.3.9 C++ operators

C++ operator overloading is supported, in a way similar to other modules. The `swig_ref` type supports all unary and binary operators between itself and all other types that exist in the system at module load time. When an operator is used (where one of the operands is a `swig_ref`), the runtime routes the call to either a member function of the given object, or to a global function whose named is derived from the types of the operands (either both or just the lhs or rhs).

For example, if `a` and `b` are SWIG variables in Octave, `a+b` becomes `a.__add__(b)`. The wrapper is then free to implement __add__ to do whatever it wants. A wrapper may define the `__add__` function manually, %rename some other function to it, or %rename a C++ operator to it.

By default the C++ operators are renamed to their corresponding Octave operators. So without doing any work, the following interface

```
%inline {
struct A {
  int value;
  A(int _value) : value(_value) {}
  A operator+ (const A& x) {
    return A(value+x.value);
  }
};
}
```

is usable from Octave like this:

```
a=A(2), b=A(3), c=a+b
assert(c.value==5);
```

Octave operators are mapped in the following way:

```
__brace__       a{args}
__brace_asgn__ a{args} = rhs
__paren__       a(args)
__paren_asgn__ a(args) = rhs
__str__         generates string rep
__not__         !a
__uplus__       +a
__uminus__      -a
__transpose__   a.'
__hermitian__   a'
__incr__        a++
__decr__        a--
__add__         a + b
__sub__         a - b
__mul__         a * b
__div__         a / b
__pow__         a ^ b
__ldiv__        a \ b
__lshift__      a << b
__rshift__      a >> b
__lt__          a < b
__le__          a <= b
__eq__          a == b
__ge__          a >= b
__gt__          a > b
__ne__          a != b
__el_mul__      a .* b
__el_div__      a ./ b
__el_pow__      a .^ b
__el_ldiv__     a .\ b
__el_and__      a & b
__el_or__       a | b
```

On the C++ side, the default mappings are as follows:

```
%rename(__add__)       *::operator+;
%rename(__add__)       *::operator+();
%rename(__add__)       *::operator+() const;
%rename(__sub__)       *::operator-;
%rename(__uminus__)    *::operator-();
%rename(__uminus__)    *::operator-() const;
%rename(__mul__)       *::operator*;
%rename(__div__)       *::operator/;
%rename(__mod__)       *::operator%;
%rename(__lshift__)    *::operator<<;
%rename(__rshift__)    *::operator>>;
%rename(__el_and__)    *::operator&&;
%rename(__el_or__)     *::operator||;
%rename(__xor__)       *::operator^;
%rename(__invert__)    *::operator~;
%rename(__lt__)        *::operator<;
%rename(__le__)        *::operator<=;
%rename(__gt__)        *::operator>;
%rename(__ge__)        *::operator>=;
%rename(__eq__)        *::operator==;
%rename(__ne__)        *::operator!=;
%rename(__not__)       *::operator!;
%rename(__incr__)      *::operator++;
%rename(__decr__)      *::operator--;
%rename(__paren__)     *::operator();
%rename(__brace__)     *::operator[];
```

Octave can also utilise friend (i.e. non-member) operators with a simple %rename: see the example in the Examples/octave/operator directory.

Octave has several operators for which no corresponding C++ operators exist. For example, the Octave code

```
x=[a,b,c];
```

calls the Octave operator `horzcat` of the class of `a` . Hence, if `a` is of type `swig_ref` you can write an overload for this operator for your wrapped C++ class by placing a file `@swig_ref/horzcat.m` in the Octave load path (like for every Octave class, see Creating a Class). This Octave function file is then called whenever the above Octave code is executed for a variable of type `swig_ref` .

### 30.3.10 Class extension with %extend

The %extend directive works the same as in other modules.

You can use it to define special behavior, like for example defining Octave operators not mapped to C++ operators, or defining certain Octave mechanisms such as how an object prints. For example, the `octave_value::{is_string, string_value, print}` functions are routed to a special method `__str__` that can be defined inside an %extend.

```
%extend A {
string __str__() {
  stringstream sout;
  sout<<$self->value;
  return sout.str();
}
}
```

Then in Octave one gets,

```
octave:1> a=A(4);
octave:2> a
a = 4
octave:3> printf("%s\n", a);
4
octave:4> a.__str__()
4
```

Similarly, Octave can use the `__float__` method to convert an object to a numeric value.

Octave 3.8.0 and later versions will also map unary functions X() to the corresponding `__X__` method, where X includes: abs(), acos(), acosh(), angle(), arg(), asin(), asinh(), atan(), atanh(), cbrt(), ceil(), conj(), cos(), cosh(), dawson(), erf(), erfc(), erfcinv(), erfcx(), erfi(), erfinv(), exp(), expm1(), finite(), fix(), floor(), gamma(), imag(), isalnum(), isalpha(), isascii(), iscntrl(), isdigit(), isgraph(), isinf(), islower(), isna(), isnan(), isprint(), ispunct(), isspace(), isupper(), isxdigit(), lgamma(), log(), log10(), log1p(), log2(), real(), round(), roundb(), signbit(), signum(), sin(), sinh(), sqrt(), tan(), tanh(), toascii(), tolower(), toupper()

### 30.3.11 C++ templates

C++ class and function templates are fully supported as in other modules, in that the %template directive may used to create explicit instantiations of templated types. For example, function templates can be instantiated as follows:

```
%module swigexample
%inline {
  template<class __scalar>
    __scalar mul(__scalar a, __scalar b) {
    return a*b;
  }
}
%include <std_complex.i>
%template(mul) mul<std::complex<double> >
%template(mul) mul<double>
```

and then used from Octave

```
octave:1> mul(4, 3)
ans =  12
octave:2> mul(4.2, 3.6)
```

```
ans =  15.120
octave:3> mul(3+4i, 10+2i)
ans =  22 + 46i
```

Similarly, class templates can be instantiated as in the following example,

```
%module swigexample
%include <std_complex.i>
%include <std_string.i>
%inline {
  #include <sstream>
  template<class __scalar> class sum {
    __scalar s;
  public:
    sum(__scalar _s=0) : s(_s) {}
    sum& add(__scalar _s) {
      s+=_s;
      return *this;
    }
    std::string __str__() const {
      std::stringstream sout;
      sout<<s;
      return sout.str();
    }
  };
}
%template(sum_complex) sum<std::complex<double> >;
%template(sum_double) sum<double>;
```

and then used from Octave

```
octave:2> a=sum_complex(2+3i);
octave:3> a.add(2)
ans =

(4, 3)
octave:4> a.add(3+i)
ans =

(7, 4)
```

### 30.3.12 C++ Smart Pointers

#### 30.3.12.1 The shared_ptr Smart Pointer

The C++11 standard provides `std::shared_ptr` which was derived from the Boost implementation, `boost::shared_ptr`. Both of these are available for Octave in the SWIG library and usage is outlined in the shared_ptr smart pointer library section.

#### 30.3.12.2 Generic Smart Pointers

C++ smart pointers are fully supported as in other modules.

### 30.3.13 Directors (calling Octave from C++ code)

There is full support for SWIG Directors, which permits Octave code to subclass C++ classes, and implement their virtual methods.

Octave has no direct support for object oriented programming, however the `swig_ref` type provides some of this support. You can manufacture a `swig_ref` using the `subclass` function (provided by the SWIG/Octave runtime).

For example,

```
octave:1> a=subclass();
octave:2> a.my_var = 4;
octave:3> a.my_method = @(self) printf("my_var = ", self.my_var);
octave:4> a.my_method();
my_var = 4
```

`subclass()` can also be used to subclass one or more C++ types. Suppose you have an interface defined by

```
%inline {
class A {
public:
  virtual my_method() {
    printf("c-side routine called\n");
  }
};
void call_your_method(A& a) {
  a.my_method();
}
}
```

Then from Octave you can say:

```
octave:1> B=@() subclass(A(), @my_method);
octave:2> function my_method(self)
octave:3>   printf("octave-side routine called\n");
octave:4> end
octave:5> call_your_method(B());
octave-side routine called
```

or more concisely,

```
octave:1> B=@() subclass(A(), 'my_method', @(self) printf("octave-side routine called\n"));
octave:2> call_your_method(B());
octave-side routine called
```

Note that you have to enable directors via the %feature directive (see other modules for this).

`subclass()` will accept any number of C++ bases or other`subclass()`'ed objects, `(string, octave_value)` pairs, and `function_handles`. In the first case, these are taken as base classes; in the second case, as named members (either variables or functions, depending on whether the given value is a function handle); in the third case, as member functions whose name is taken from the given function handle. E.g.,

```
octave:1> B=@(some_var=2) subclass(A(), 'some_var', some_var, @some_func, 'another_func',
@(self) do_stuff())
```

You can also assign non-C++ member variables and functions after construct time. There is no support for non-C++ static members.

There is limited support for explicitly referencing C++ bases. So, in the example above, we could have

```
octave:1> B=@() subclass(A(), @my_method);
octave:2> function my_method(self)
octave:3>   self.A.my_method();
octave:4>   printf("octave-side routine called\n");
octave:5> end
octave:6> call_your_method(B());
c-side routine called
octave-side routine called
```

### 30.3.14 Threads

The use of threads in wrapped Director code is not supported; i.e., an Octave-side implementation of a C++ class must be called from the Octave interpreter's thread. Anything fancier (apartment/queue model, whatever) is left to the user. Without anything fancier, this amounts to the limitation that Octave must drive the module... like, for example, an optimization package that calls Octave to evaluate an objective function.

### 30.3.15 Memory management

As noted above, `swig_ref` represents a reference counted pointer to a C/C++-side object. It also contains a flag indicating whether Octave or the C/C++ code owns the object. If Octave owns it, any destructors will be called when the reference count reaches zero. If the C/C++ side owns the object, then destructors will not be called when the reference count goes to zero. As noted above, `swig_ref` represents a reference counted pointer to a C/C++-side object. It also contains a flag indicating whether Octave or the C/C++ code owns the object. If Octave owns it, any destructors will be called when the reference count reaches zero. If the C/C++ side owns the object, then destructors will not be called when the reference count goes to zero.

For example,

```
%inline {
class A {
public:
  A() { printf("A constructing\n"); }
  ~A() { printf("A destructing\n"); }
};
}
```

Would produce this behavior in Octave:

```
octave:1> a=A();
A constructing
octave:2> b=a;
octave:3> clear a;
octave:4> b=4;
A destructing
```

The %newobject directive may be used to control this behavior for pointers returned from functions.

In the case where one wishes for the C++ side to own an object that was created in Octave (especially a Director object), one can use the __disown() method to invert this logic. Then letting the Octave reference count go to zero will not destroy the object, but destroying the object will invalidate the Octave-side object if it still exists (and call destructors of other C++ bases in the case of multiple inheritance/`subclass()`'ing).

### 30.3.16 STL support

Various STL library files are provided for wrapping STL containers.

### 30.3.17 Matrix typemaps

Octave provides a rich set of classes for dealing with matrices. Currently there are no built-in typemaps to deal with those. However, these are relatively straight forward for users to add themselves (see the docs on typemaps). Without much work (a single typemap decl-- say, 5 lines of code in the interface file), it would be possible to have a function

```
double my_det(const double* mat, int m, int n);
```

that is accessed from Octave as,

```
octave:1> my_det(rand(4));
ans = -0.18388
```

# 31 SWIG and Perl5


- Overview
- Preliminaries
  - Getting the right header files
  - Compiling a dynamic module
  - Building a dynamic module with MakeMaker

- Building a static version of Perl
- Using the module
- Compilation problems and compiling with C++
- Compiling for 64-bit platforms
- Building Perl Extensions under Windows
  - Running SWIG from Developer Studio
  - Using other compilers
- The low-level interface
  - Functions
  - Global variables
  - Constants
  - Pointers
  - Structures
  - C++ classes
  - C++ classes and type-checking
  - C++ overloaded functions
  - Operators
  - Modules and packages
- Input and output parameters
- Exception handling
- Remapping datatypes with typemaps
  - A simple typemap example
  - Perl5 typemaps
  - Typemap variables
  - Useful functions
- Typemap Examples
  - Converting a Perl5 array to a char **
  - Return values
  - Returning values from arguments
  - Accessing array structure members
  - Turning Perl references into C pointers
  - Pointer handling
- Proxy classes
  - Preliminaries
  - Structure and class wrappers
  - Object Ownership
  - Nested Objects
  - Proxy Functions
  - Inheritance
  - Modifying the proxy methods
- Adding additional Perl code
- Cross language polymorphism
  - Enabling directors
  - Director classes
  - Ownership and object destruction
  - Exception unrolling
  - Overhead and code bloat
  - Typemaps


**Caution: This chapter is under repair!**

This chapter describes SWIG's support of Perl5. Although the Perl5 module is one of the earliest SWIG modules, it has continued to evolve and has been improved greatly with the help of SWIG users. As of SWIG 4.1.0, the minimum version of Perl we aim to support is Perl 5.8.0. We can no longer easily test with older versions, and they no longer seem to be in active use.

## 31.1 Overview

To build Perl extension modules, SWIG uses a layered approach. At the lowest level, simple procedural wrappers are generated for functions, classes, methods, and other declarations in the input file. Then, for structures and classes, an optional collection of Perl proxy classes can be generated in order to provide a more natural object oriented Perl interface. These proxy classes simply build upon the low-level interface.

In describing the Perl interface, this chapter begins by covering the essentials. First, the problem of configuration, compiling, and installing Perl modules is discussed. Next, the low-level procedural interface is presented. Finally, proxy classes are described. Advanced customization features, typemaps, and other options are found near the end of the chapter.

## 31.2 Preliminaries

To build a Perl5 module, run SWIG using the `-perl` or `-perl5` option as follows:

```
swig -perl example.i
```

This produces two files. The first file, `example_wrap.c` contains all of the C code needed to build a Perl5 module. The second file, `example.pm` contains supporting Perl code needed to properly load the module.

To build the module, you will need to compile the file `example_wrap.c` and link it with the rest of your program.

### 31.2.1 Getting the right header files

In order to compile, SWIG extensions need the following Perl5 header files:

```
#include "Extern.h"
#include "perl.h"
#include "XSUB.h"
```

These are typically located in a directory like this

```
/usr/lib/perl/5.14/CORE
```

The SWIG configuration script automatically tries to locate this directory so that it can compile examples. However, if you need to find out where the directory is located, an easy way to find out is to ask Perl itself:

```
$ perl -e 'use Config; print "$Config{archlib}\n";'
/usr/lib/perl/5.14
```

### 31.2.2 Compiling a dynamic module

The preferred approach to building an extension module is to compile it into a shared object file or DLL. Assuming you have code you need to link to in a file called `example.c`, you will need to compile your program using commands like this (shown for Linux):

```
$ swig -perl example.i
$ gcc -fPIC example.c
$ gcc -fPIC -c example_wrap.c -I/usr/lib/perl/5.14/CORE -Dbool=char
$ gcc -shared example.o example_wrap.o -o example.so
```

The exact compiler options vary from platform to platform. SWIG tries to guess the right options when it is installed. Therefore, you may want to start with one of the examples in the `SWIG/Examples/perl5` directory. If that doesn't work, you will need to read the man-pages for your compiler and linker to get the right set of options. You might also check the SWIG Wiki for additional information.

When linking the module, the name of the shared object file must match the module name used in the SWIG interface file. If you used ` %module example`', then the target should be named `example.so', `example.sl', or the appropriate dynamic module name on your system.

### 31.2.3 Building a dynamic module with MakeMaker

It is also possible to use Perl to build dynamically loadable modules for you using the MakeMaker utility. To do this, write a Perl script such as the following:

```
# File : Makefile.PL
use ExtUtils::MakeMaker;
WriteMakefile(
        `NAME'    => `example',                # Name of package
        `LIBS'    => [`-lm'],                  # Name of custom libraries
        `OBJECT'  => `example.o example_wrap.o'  # Object files
);
```

Now, to build a module, simply follow these steps:

```
$ perl Makefile.PL
$ make
$ make install
```

If you are planning to distribute a SWIG-generated module, this is the preferred approach to compilation. More information about MakeMaker can be found in "Programming Perl, 2nd ed." by Larry Wall, Tom Christiansen, and Randal Schwartz.

### 31.2.4 Building a static version of Perl

If you machine does not support dynamic loading or if you've tried to use it without success, you can build a new version of the Perl interpreter with your SWIG extensions added to it. To build a static extension, you first need to invoke SWIG as follows:

```
$ swig -perl -static example.i
```

By default SWIG includes code for dynamic loading, but the `-static` option takes it out.

Next, you will need to supply a `main()` function that initializes your extension and starts the Perl interpreter. While, this may sound daunting, SWIG can do this for you automatically as follows:

```
%module example

%inline %{
extern double My_variable;
extern int fact(int);
%}

// Include code for rebuilding Perl
%include <perlmain.i>
```

The same thing can be accomplished by running SWIG as follows:

```
$ swig -perl -static -lperlmain.i example.i
```

The `perlmain.i` file inserts Perl's`main()` function into the wrapper code and automatically initializes the SWIG generated module. If you just want to make a quick a dirty module, this may be the easiest way. By default, the `perlmain.i` code does not initialize any other Perl extensions. If you need to use other packages, you will need to modify it appropriately. You can do this by just copying `perlmain.i` out of the SWIG library, placing it in your own directory, and modifying it to suit your purposes.

To build your new Perl executable, follow the exact same procedure as for a dynamic module, but change the link line to something like this:

```
$ gcc example.o example_wrap.o -L/usr/lib/perl/5.14/CORE \
        -lperl -lsocket -lnsl -lm -o myperl
```

This will produce a new version of Perl called `myperl`. It should be functionality identical to Perl with your C/C++ extension added to it. Depending on your machine, you may need to link with additional libraries such as `-lsocket, -lnsl, -ldl`, etc.

### 31.2.5 Using the module

To use the module, simply use the Perl `use` statement. If all goes well, you will be able to do this:

```
$ perl
use example;
print example::fact(4), "\n";
24
```

A common error received by first-time users is the following:

```
use example;
Can't locate example.pm in @INC (@INC contains: /etc/perl /usr/local/lib/perl/5.14.2 /usr/local/share/perl/5.14.2 /usr/lib/perl5 /usr/share/p
BEGIN failed--compilation aborted at - line 1.
```

This error is almost caused when the name of the shared object file you created doesn't match the module name you specified with the `%module` directive.

A somewhat related, but slightly different error is this:

```
use example;
Can't find 'boot_example' symbol in ./example.so
 at - line 1
BEGIN failed--compilation aborted at - line 1.
```

This error is generated because Perl can't locate the module bootstrap function in the SWIG extension module. This could be caused by a mismatch between the module name and the shared library name. However, another possible cause is forgetting to link the SWIG-generated wrapper code with the rest of your application when you linked the extension module.

Another common error is the following:

```
use example;
Can't load './example.so' for module example: ./example.so:
undefined symbol: Foo at /usr/lib/perl/5.14/i386-linux/DynaLoader.pm line 169.

 at - line 1
BEGIN failed--compilation aborted at - line 1.
```

This error usually indicates that you forgot to include some object files or libraries in the linking of the shared library file. Make sure you compile both the SWIG wrapper file and your original program into a shared library file. Make sure you pass all of the required libraries to the linker.

Sometimes unresolved symbols occur because a wrapper has been created for a function that doesn't actually exist in a library. This usually occurs when a header file includes a declaration for a function that was never actually implemented or it was removed from a library without updating the header file. To fix this, you can either edit the SWIG input file to remove the offending declaration or you can use the `%ignore` directive to ignore the declaration. Better yet, update the header file so that it doesn't have an undefined declaration.

Finally, suppose that your extension module is linked with another library like this:

```
$ gcc -shared example.o example_wrap.o -L/home/beazley/projects/lib -lfoo \
      -o example.so
```

If the `foo` library is compiled as a shared library, you might get the following error when you try to use your module:

```
use example;
Can't load './example.so' for module example: libfoo.so: cannot open shared object file:
No such file or directory at /usr/lib/perl/5.14/i386-linux/DynaLoader.pm line 169.

 at - line 1
BEGIN failed--compilation aborted at - line 1.
>>>
```

This error is generated because the dynamic linker can't locate the `libfoo.so` library. When shared libraries are loaded, the system normally only checks a few standard locations such as `/usr/lib` and `/usr/local/lib`. To get the loader to look in other locations, there are several things you can do. First, you can recompile your extension module with extra path information. For example, on Linux you can do this:

```
$ gcc -shared example.o example_wrap.o -L/home/beazley/projects/lib -lfoo \
      -Xlinker -rpath /home/beazley/projects/lib \
      -o example.so
```

Alternatively, you can set the `LD_LIBRARY_PATH` environment variable to include the directory with your shared libraries. If setting `LD_LIBRARY_PATH`, be aware that setting this variable can introduce a noticeable performance impact on all other applications that you run. To set it only for Perl, you might want to do this instead:

```
$ env LD_LIBRARY_PATH=/home/beazley/projects/lib perl
```

Finally, you can use a command such as `ldconfig` (Linux) or `crle` (Solaris) to add additional search paths to the default system configuration (this requires root access and you will need to read the man pages).

### 31.2.6 Compilation problems and compiling with C++

Compilation of C++ extensions has traditionally been a tricky problem. Since the Perl interpreter is written in C, you need to take steps to make sure C++ is properly initialized and that modules are compiled correctly.

On most machines, C++ extension modules should be linked using the C++ compiler. For example:

```
$ swig -c++ -perl example.i
$ g++ -fPIC -c example.cxx
$ g++ -fPIC -c example_wrap.cxx -I/usr/lib/perl/5.14/i386-linux/CORE
$ g++ -shared example.o example_wrap.o -o example.so
```

In addition to this, you may need to include additional library files to make it work. For example, if you are using the Sun C++ compiler on Solaris, you often need to add an extra library `-lCrun` like this:

```
$ swig -c++ -perl example.i
$ CC -Kpic -c example.cxx
$ CC -Kpic -c example_wrap.cxx -I/usr/lib/perl/5.14/i386-linux/CORE
$ CC -shared example.o example_wrap.o -o example.so -lCrun
```

Of course, the names of the extra libraries are completely non-portable---you will probably need to do some experimentation.

Another possible compile problem comes from recent versions of Perl (5.8.0) and the GNU tools. If you see errors having to do with _crypt_struct, that means _GNU_SOURCE is not defined and it needs to be. So you should compile the wrapper like:

```
$ g++ -fPIC -c example_wrap.cxx -I/usr/lib/perl/5.8.0/CORE -D_GNU_SOURCE
```

-D_GNU_SOURCE is also included in the Perl ccflags, which can be found by running

```
$ perl -e 'use Config; print "$Config{ccflags}\n";'
```

So you could also compile the wrapper like

```
$ g++ -fPIC -c example_wrap.cxx -I/usr/lib/perl/5.8.0/CORE \
`perl -MConfig -e 'print $Config{ccflags}'`
```

Sometimes people have suggested that it is necessary to relink the Perl interpreter using the C++ compiler to make C++ extension modules work. In the experience of this author, this has never actually appeared to be necessary on most platforms. Relinking the interpreter with C++ really only includes the special run-time libraries described above---as long as you link your extension modules with these libraries, it should not be necessary to rebuild Perl.

If you aren't entirely sure about the linking of a C++ extension, you might look at an existing C++ program. On many Unix machines, the `ldd` command will list library dependencies. This should give you some clues about what you might have to include when you link your extension module. For example, notice the first line of output here:

```
$ ldd swig
        libstdc++-libc6.1-1.so.2 => /usr/lib/libstdc++-libc6.1-1.so.2 (0x40019000)
        libm.so.6 => /lib/libm.so.6 (0x4005b000)
        libc.so.6 => /lib/libc.so.6 (0x40077000)
        /lib/ld-linux.so.2 => /lib/ld-linux.so.2 (0x40000000)
$
```

If linking wasn't enough of a problem, another major complication of C++ is that it does not define any sort of standard for binary linking of libraries. This means that C++ code compiled by different compilers will not link together properly as libraries nor is the memory layout of classes and data structures implemented in any kind of portable manner. In a monolithic C++ program, this problem may be unnoticed. However, in Perl, it is possible for different extension modules to be compiled with different C++ compilers. As long as these modules are self-contained, this probably won't matter. However, if these modules start sharing data, you will need to take steps to avoid segmentation faults and other erratic program behavior. Also, be aware that certain C++ features, especially RTTI, can behave strangely when working with multiple modules.

It should be noted that you may get a lot of error messages about the '`bool`' datatype when compiling a C++ Perl module. If you experience this problem, you can try the following:

- Use `-DHAS_BOOL` when compiling the SWIG wrapper code
- Or use `-Dbool=char` when compiling.

Finally, recent versions of Perl (5.8.0) have namespace conflict problems. Perl defines a bunch of short macros to make the Perl API function names shorter. For example, in /usr/lib/perl/5.8.0/CORE/embed.h there is a line:

```
#define do_open Perl_do_open
```

The problem is, in the <iostream> header from GNU libstdc++v3 there is a private function named do_open. If <iostream> is included after the perl headers, then the Perl macro causes the iostream do_open to be renamed, which causes compile errors. Hopefully in the future Perl will support a PERL_NO_SHORT_NAMES flag, but for now the only solution is to undef the macros that conflict. Lib/perl5/noembed.h in the SWIG source has a list of macros that are known to conflict with either standard headers or other headers. But if you get macro type conflicts from other macros not included in Lib/perl5/noembed.h while compiling the wrapper, you will have to find the macro that conflicts and add an #undef into the .i file. Please report any conflicting macros you find to swig-user mailing list.

### 31.2.7 Compiling for 64-bit platforms

On platforms that support 64-bit applications (Solaris, Irix, etc.), special care is required when building extension modules. On these machines, 64-bit applications are compiled and linked using a different set of compiler/linker options. In addition, it is not generally possible to mix 32-bit and 64-bit code together in the same application.

To utilize 64-bits, the Perl executable will need to be recompiled as a 64-bit application. In addition, all libraries, wrapper code, and every other part of your application will need to be compiled for 64-bits. If you plan to use other third-party extension modules, they will also have to be recompiled as 64-bit extensions.

If you are wrapping commercial software for which you have no source code, you will be forced to use the same linking standard as used by that software. This may prevent the use of 64-bit extensions. It may also introduce problems on platforms that support more than one linking standard (e.g., -o32 and -n32 on Irix).

## 31.3 Building Perl Extensions under Windows

Building a SWIG extension to Perl under Windows is roughly similar to the process used with Unix. Normally, you will want to produce a DLL that can be loaded into the Perl interpreter. This section assumes you are using SWIG with Microsoft Visual C++ although the procedure may be similar with other compilers.

### 31.3.1 Running SWIG from Developer Studio

If you are developing your application within Microsoft developer studio, SWIG can be invoked as a custom build option. The process roughly requires these steps:

- Open up a new workspace and use the AppWizard to select a DLL project.
- Add both the SWIG interface file (the .i file), any supporting C files, and the name of the wrapper file that will be created by SWIG (ie. `example_wrap.c`). Note: If using C++, choose a different suffix for the wrapper file such as `example_wrap.cxx`. Don't worry if the wrapper file doesn't exist yet--Developer studio will keep a reference to it around.
- Select the SWIG interface file and go to the settings menu. Under settings, select the "Custom Build" option.
- Enter "SWIG" in the description field.
- Enter "`swig -perl5 -o $(ProjDir)\$(InputName)_wrap.cxx $(InputPath)`" in the "Build command(s) field"
- Enter "`$(ProjDir)\$(InputName)_wrap.cxx`" in the "Output files(s) field".
- Next, select the settings for the entire project and go to "C++:Preprocessor". Add the include directories for your Perl 5 installation under "Additional include directories".
- Define the symbols WIN32 and MSWIN32 under preprocessor options. Note that all extensions to the ActiveWare port must be compiled with the C++ compiler since Perl has been encapsulated in a C++ class.
- Finally, select the settings for the entire project and go to "Link Options". Add the Perl library file to your link libraries. For example "perl.lib". Also, set the name of the output file to match the name of your Perl module (ie. example.dll).
- Build your project.

Now, assuming you made it this far, SWIG will be automatically invoked when you build your project. Any changes made to the interface file will result in SWIG being automatically invoked to produce a new version of the wrapper file. To run your new Perl extension, simply run Perl and use the use command as normal. For example:

```
DOS > perl
use example;
$a = example::fact(4);
print "$a\n";
```

### 31.3.2 Using other compilers

SWIG is known to work with Cygwin and may work with other compilers on Windows. For general hints and suggestions refer to the Windows chapter.

## 31.4 The low-level interface

At its core, the Perl module uses a simple low-level interface to C function, variables, constants, and classes. This low-level interface can be used to control your application. However, it is also used to construct more user-friendly proxy classes as described in the next section.

### 31.4.1 Functions

C functions are converted into new Perl built-in commands (or subroutines). For example:

```
%module example
int fact(int a);
...
```

Now, in Perl:

```
use example;
$a = &example::fact(2);
```

### 31.4.2 Global variables

Global variables are handled using Perl's magic variable mechanism. SWIG generates a pair of functions that intercept read/write operations and attaches them to a Perl variable with the same name as the C global variable. Thus, an interface like this

```
%module example;
...
double Spam;
...
```

is accessed as follows:

```
use example;
print $example::Spam, "\n";
$example::Spam = $example::Spam + 4
# ... etc ...
```

If a variable is declared as `const`, it is wrapped as a read-only variable. Attempts to modify its value will result in an error.

To make ordinary variables read-only, you can also use the `%immutable` directive. For example:

```
%{
extern char *path;
%}
%immutable;
extern char *path;
%mutable;
```

The `%immutable` directive stays in effect until it is explicitly disabled or cleared using `%mutable`. See the Creating read-only variables section for further details.

It is also possible to tag a specific variable as read-only like this:

```
%{
extern char *path;
%}
%immutable path;
...
...
extern char *path;       // Declared later in the input
```

### 31.4.3 Constants

By default, constants are wrapped as read-only Perl variables. For example:

```
%module example

#define FOO 42
```

In Perl:

```
use example;
print $example::FOO, "\n";   # OK
$example::FOO = 2;           # Error
```

Alternatively, if you use swig's `-const` option, constants are wrapped such that the leading $ isn't required (by using a constant subroutine), which usually gives a more natural Perl interface, for example:

```
use example;
print example::FOO, "\n";
```

### 31.4.4 Pointers

SWIG represents pointers as blessed references. A blessed reference is the same as a Perl reference except that it has additional information attached to it indicating what kind of reference it is. That is, if you have a C declaration like this:

```
Matrix *new_Matrix(int n, int m);
```

The module returns a value generated as follows:

```
$ptr = new_Matrix(int n, int m);     # Save pointer return result
bless $ptr, "p_Matrix";              # Bless it as a pointer to Matrix
```

SWIG uses the "blessing" to check the datatype of various pointers. In the event of a mismatch, an error or warning message is generated.

To check to see if a value is the NULL pointer, use the `defined()` command:

```
if (defined($ptr)) {
  print "Not a NULL pointer.";
} else {
  print "Is a NULL pointer.";
}
```

To create a NULL pointer, you should pass the `undef` value to a function.

The "value" of a Perl reference is not the same as the underlying C pointer that SWIG wrapper functions return. Suppose that `$a` and `$b` are two references that point to the same C object. In general, `$a` and `$b` will be different--since they are different references. Thus, it is a mistake to check the equality of `$a` and `$b` to check the equality of two C pointers. The correct method to check equality of C pointers is to dereference them as follows:

```
if ($$a == $$b) {
  print "a and b point to the same thing in C";
} else {
  print "a and b point to different objects.";
}
```

As much as you might be inclined to modify a pointer value directly from Perl, don't. Manipulating pointer values is architecture dependent and could cause your program to crash. Similarly, don't try to manually cast a pointer to a new type by reblessing a pointer. This may not work like you expect and it is particularly dangerous when casting C++ objects. If you need to cast a pointer or change its value, consider writing some helper functions instead. For example:

```
%inline %{
/* C-style cast */
Bar *FooToBar(Foo *f) {
  return (Bar *) f;
}

/* C++-style cast */
Foo *BarToFoo(Bar *b) {
  return dynamic_cast<Foo*>(b);
}

Foo *IncrFoo(Foo *f, int i) {
  return f+i;
}
%}
```

Also, if working with C++, you should always try to use the new C++ style casts. For example, in the above code, the C-style cast may return a bogus result whereas as the C++-style cast will return `NULL` if the conversion can't be performed.

**Compatibility Note:** In earlier versions, SWIG tried to preserve the same pointer naming conventions as XS and `xsubpp`. Given the advancement of the SWIG typesystem and the growing differences between SWIG and XS, this is no longer supported.

### 31.4.5 Structures

Access to the contents of a structure are provided through a set of low-level accessor functions as described in the "SWIG Basics" chapter. For example,

```
struct Vector {
  double x, y, z;
};
```

gets mapped into the following collection of accessor functions:

```
struct Vector *new_Vector();
void           delete_Vector(Vector *v);
double         Vector_x_get(Vector *obj)
void           Vector_x_set(Vector *obj, double x)
double         Vector_y_get(Vector *obj)
void           Vector_y_set(Vector *obj, double y)
double         Vector_z_get(Vector *obj)
void           Vector_z_set(Vector *obj, double z)
```

These functions are then used to access structure data from Perl as follows:

```
$v = example::new_Vector();
print example::Vector_x_get($v), "\n";   # Get x component
example::Vector_x_set($v, 7.8);          # Change x component
```

Similar access is provided for unions and the data members of C++ classes.

`const` members of a structure are read-only. Data members can also be forced to be read-only using the `%immutable` directive. For example:

```
struct Foo {
  ...
  %immutable;
  int x;        /* Read-only members */
  char *name;
  %mutable;
  ...
};
```

When `char *` members of a structure are wrapped, the contents are assumed to be dynamically allocated using `malloc` or `new` (depending on whether or not SWIG is run with the -c++ option). When the structure member is set, the old contents will be released and a new value created. If this is not the behavior you want, you will have to use a typemap (described later).

Array members are normally wrapped as read-only. For example,

```
struct Foo {
  int x[50];
};
```

produces a single accessor function like this:

```
int *Foo_x_get(Foo *self) {
  return self->x;
};
```

If you want to set an array member, you will need to supply a "memberin" typemap described later in this chapter. As a special case, SWIG does generate code to set array members of type `char` (allowing you to store a Perl string in the structure).

When structure members are wrapped, they are handled as pointers. For example,

```
struct Foo {
  ...
};

struct Bar {
  Foo f;
};
```

generates accessor functions such as this:

```
Foo *Bar_f_get(Bar *b) {
  return &b->f;
}

void Bar_f_set(Bar *b, Foo *val) {
  b->f = *val;
}
```

### 31.4.6 C++ classes

C++ classes are wrapped by building a set of low level accessor functions. Consider the following class:

```
class List {
public:
  List();
  ~List();
  int  search(char *item);
  void insert(char *item);
  void remove(char *item);
  char *get(int n);
  int  length;
static void print(List *l);
};
```

When wrapped by SWIG, the following functions are created:

```
List    *new_List();
void     delete_List(List *l);
int      List_search(List *l, char *item);
void     List_insert(List *l, char *item);
void     List_remove(List *l, char *item);
char    *List_get(List *l, int n);
int      List_length_get(List *l);
void     List_length_set(List *l, int n);
void     List_print(List *l);
```

In Perl, these functions are used in a straightforward manner:

```
use example;
$l = example::new_List();
example::List_insert($l, "Ale");
example::List_insert($l, "Stout");
example::List_insert($l, "Lager")
example::List_print($l)
Lager
Stout
Ale
print example::List_length_get($l), "\n";
3
```

At this low level, C++ objects are really just typed pointers. Member functions are accessed by calling a C-like wrapper with an instance pointer as the first argument. Although this interface is fairly primitive, it provides direct access to C++ objects. A higher level interface using Perl proxy classes can be built using these low-level accessors. This is described shortly.

### 31.4.7 C++ classes and type-checking

The SWIG type-checker is fully aware of C++ inheritance. Therefore, if you have classes like this

```
class Foo {
...
};

class Bar : public Foo {
...
};
```

and a function

```
void spam(Foo *f);
```

then the function `spam()` accepts `Foo *` or a pointer to any class derived from `Foo`. If necessary, the type-checker also adjusts the value of the pointer (as is necessary when multiple inheritance is used).

### 31.4.8 C++ overloaded functions

If you have a C++ program with overloaded functions or methods, you will need to disambiguate those methods using `%rename`. For example:

```
/* Forward renaming declarations */
%rename(foo_i) foo(int);
%rename(foo_d) foo(double);
...
void foo(int);           // Becomes 'foo_i'
void foo(char *c);       // Stays 'foo' (not renamed)

class Spam {
public:
  void foo(int);      // Becomes 'foo_i'
  void foo(double);   // Becomes 'foo_d'
  ...
};
```

Now, in Perl, the methods are accessed as follows:

```
use example;
example::foo_i(3);
$s = example::new_Spam();
example::Spam_foo_i($s, 3);
example::Spam_foo_d($s, 3.14);
```

Please refer to the "SWIG Basics" chapter for more information.

### 31.4.9 Operators

As of version 1.3.27 SWIG automatically renames the most common C++ operators, and maps them into the perl module with the proper 'use overload ...' so you don't need to do any work.

The following C++ operators are currently supported by the Perl module:

- operator++
- operator--
- operator+
- operator-
- operator*
- operator/
- operator==
- operator!=
- operator%
- operator>
- operator<
- operator and
- operator or

### 31.4.10 Modules and packages

When you create a SWIG extension, everything gets placed into a single Perl module. The name of the module is determined by the `%module` directive. To use the module, do the following:

```
$ perl5
use example;                    # load the example module
print example::fact(4), "\n"    # Call a function in it
24
```

Usually, a module consists of a collection of code that is contained within a single file. A package, on the other hand, is the Perl equivalent of a namespace. A package is a lot like a module, except that it is independent of files. Any number of files may be part of the same package--or a package may be broken up into a collection of modules if you prefer to think about it in this way.

SWIG installs its functions into a package with the same name as the module.

**Incompatible Change:** previous versions of SWIG enabled you to change the name of the package by using the -package option, this feature has been removed in order to properly support modules that used nested namespaces, e.g. Foo::Bar::Baz. To give your module a nested namespace simply provide the fully qualified name in your %module directive:

```
%module "Foo::Bar::Baz"
```

**NOTE:** the double quotes are necessary.

Using the `package` option of the `%module` directive allows you to specify what Perl namespace that the module will be living in when installed. This is useful in the situation where a module maintainer wants to split a large module into smaller pieces to make maintenance easier, but doesn't want to have that affect the module name used by applications. So for example, if I wanted to split `XML::Xerces` into `XML::Xerces::SAX`, etc. , but I wanted all the applications to be able to access the classes using the `XML::Xerces` namespace I could use:

```
%module(package="XML::Xerces") "XML::Xerces::SAX
```

And now all the applications could use the class `XML::Xerces::SAXParser`. Without the `package` directive splitting the module would force applications to use the class `XML::Xerces::SAX::SAXParser`. This could break compatibility for existing applications that are already using the class under the name `XML::Xerces::SAXParser`.

## 31.5 Input and output parameters

A common problem in some C programs is handling parameters passed as simple pointers. For example:

```
void add(int x, int y, int *result) {
  *result = x + y;
}
```

or perhaps

```
int sub(int *x, int *y) {
  return *x+*y;
}
```

The easiest way to handle these situations is to use the `typemaps.i` file. For example:

```
%module example
%include "typemaps.i"

void add(int, int, int *OUTPUT);
int  sub(int *INPUT, int *INPUT);
```

In Perl, this allows you to pass simple values. For example:

```
$a = example::add(3, 4);
print "$a\n";
7
$b = example::sub(7, 4);
print "$b\n";
3
```

Notice how the `INPUT` parameters allow integer values to be passed instead of pointers and how the `OUTPUT` parameter creates a return result.

If you don't want to use the names `INPUT` or `OUTPUT` , use the `%apply` directive. For example:

```
%module example
%include "typemaps.i"

%apply int *OUTPUT { int *result };
%apply int *INPUT  { int *x, int *y};

void add(int x, int y, int *result);
int  sub(int *x, int *y);
```

If a function mutates one of its parameters like this,

```
void negate(int *x) {
  *x = -(*x);
}
```

you can use `INOUT` like this:

```
%include "typemaps.i"
...
void negate(int *INOUT);
```

In Perl, a mutated parameter shows up as a return value. For example:

```
$a = example::negate(3);
print "$a\n";
-3
```

The most common use of these special typemap rules is to handle functions that return more than one value. For example, sometimes a function returns a result as well as a special error code:

```
/* send message, return number of bytes sent, along with success code */
int send_message(char *text, int *success);
```

To wrap such a function, simply use the `OUTPUT` rule above. For example:

```
%module example
%include "typemaps.i"
%apply int *OUTPUT { int *success };
...
```

```
int send_message(char *text, int *success);
```

When used in Perl, the function will return multiple values.

```
($bytes, $success) = example::send_message("Hello World");
```

Another common use of multiple return values are in query functions. For example:

```
void get_dimensions(Matrix *m, int *rows, int *columns);
```

To wrap this, you might use the following:

```
%module example
%include "typemaps.i"
%apply int *OUTPUT { int *rows, int *columns };
...
void get_dimensions(Matrix *m, int *rows, *columns);
```

Now, in Perl:

```
($r, $c) = example::get_dimensions($m);
```

In certain cases, it is possible to treat Perl references as C pointers. To do this, use the `REFERENCE` typemap. For example:

```
%module example
%include "typemaps.i"

void add(int x, int y, int *REFERENCE);
```

In Perl:

```
use example;
$c = 0.0;
example::add(3, 4, \$c);
print "$c\n";
7
```

**Note:** The `REFERENCE` feature is only currently supported for numeric types (integers and floating point).

## 31.6 Exception handling

The SWIG `%exception` directive can be used to create a user-definable exception handler for converting exceptions in your C/C++ program into Perl exceptions. The chapter on customization features contains more details, but suppose you have a C++ class like the following:

```
class RangeError {};   // Used for an exception

class DoubleArray {
  private:
    int n;
    double *ptr;
  public:
    // Create a new array of fixed size
    DoubleArray(int size) {
      ptr = new double[size];
      n = size;
    }
    // Destroy an array
    ~DoubleArray() {
      delete ptr;
    }
    // Return the length of the array
    int   length() {
      return n;
    }

    // Get an item from the array and perform bounds checking.
    double getitem(int i) {
      if ((i >= 0) && (i < n))
        return ptr[i];
      else
        throw RangeError();
    }

    // Set an item in the array and perform bounds checking.
    void setitem(int i, double val) {
      if ((i >= 0) && (i < n))
        ptr[i] = val;
      else
        throw RangeError();
    }
  };
```

Since several methods in this class can throw an exception for an out-of-bounds access, you might want to catch this in the Perl extension by writing the following in an interface file:

```
%exception {
  try {
    $action
```

```
  }
  catch (RangeError) {
    croak("Array index out-of-bounds");
  }
}

class DoubleArray {
...
};
```

The exception handling code is inserted directly into generated wrapper functions. The `$action` variable is replaced with the C/C++ code being executed by the wrapper. When an exception handler is defined, errors can be caught and used to gracefully generate a Perl error instead of forcing the entire program to terminate with an uncaught error.

As shown, the exception handling code will be added to every wrapper function. Since this is somewhat inefficient. You might consider refining the exception handler to only apply to specific methods like this:

```
%exception getitem {
  try {
    $action
  }
  catch (RangeError) {
    croak("Array index out-of-bounds");
  }
}

%exception setitem {
  try {
    $action
  }
  catch (RangeError) {
    croak("Array index out-of-bounds");
  }
}
```

In this case, the exception handler is only attached to methods and functions named `getitem` and `setitem`.

If you had a lot of different methods, you can avoid extra typing by using a macro. For example:

```
%define RANGE_ERROR
{
  try {
    $action
  }
  catch (RangeError) {
    croak("Array index out-of-bounds");
  }
}
%enddef

%exception getitem RANGE_ERROR;
%exception setitem RANGE_ERROR;
```

Since SWIG's exception handling is user-definable, you are not limited to C++ exception handling. See the chapter on "Customization features" for more examples.

## 31.7 Remapping datatypes with typemaps

This section describes how you can modify SWIG's default wrapping behavior for various C/C++ datatypes using the `%typemap` directive. This is an advanced topic that assumes familiarity with the Perl C API as well as the material in the "Typemaps " chapter.

Before proceeding, it should be stressed that typemaps are *not* a required part of using SWIG---the default wrapping behavior is enough in most cases. Typemaps are only used if you want to change some aspect of the primitive C-Perl interface.

### 31.7.1 A simple typemap example

A typemap is nothing more than a code generation rule that is attached to a specific C datatype. For example, to convert integers from Perl to C, you might define a typemap like this:

```
%module example

%typemap(in) int {
  $1 = (int) SvIV($input);
  printf("Received an integer : %d\n", $1);
}
...
%inline %{
extern int fact(int n);
%}
```

Typemaps are always associated with some specific aspect of code generation. In this case, the "in" method refers to the conversion of input arguments to C/C++. The datatype `int` is the datatype to which the typemap will be applied. The supplied C code is used to convert values. In this code a number of special variable prefaced by a `$` are used. The `$1` variable is placeholder for a local variable of type `int`. The `$input` variable is the input object (usually a `SV *`).

When this example is used in Perl5, it will operate as follows:

```
use example;
$n = example::fact(6);
print "$n\n";
...

Output:
Received an integer : 6
720
```

The application of a typemap to specific datatypes and argument names involves more than simple text-matching--typemaps are fully integrated into the SWIG type-system. When you define a typemap for `int`, that typemap applies to `int` and qualified variations such as `const int`. In addition, the typemap system follows `typedef` declarations. For example:

```
%typemap(in) int n {
  $1 = (int) SvIV($input);
  printf("n = %d\n", $1);
}
%inline %{
typedef int Integer;
extern int fact(Integer n);    // Above typemap is applied
%}
```

It should be noted that the matching of `typedef` only occurs in one direction. If you defined a typemap for `Integer`, it is not applied to arguments of type `int`.

Typemaps can also be defined for groups of consecutive arguments. For example:

```
%typemap(in) (char *str, unsigned len) {
    $1 = SvPV($input, $2);
};

int count(char c, char *str, unsigned len);
```

When a multi-argument typemap is defined, the arguments are always handled as a single Perl object. This allows the function to be used like this (notice how the length parameter is omitted):

```
example::count("e", "Hello World");
1
>>>
```

### 31.7.2 Perl5 typemaps

The previous section illustrated an "in" typemap for converting Perl objects to C. A variety of different typemap methods are defined by the Perl module. For example, to convert a C integer back into a Perl object, you might define an "out" typemap like this:

```
%typemap(out) int {
  $result = sv_newmortal();
  sv_setiv($result, (IV) $1);
  argvi++;
}
```

The following typemap methods are available:

`%typemap(in)`

Converts Perl5 object to input function arguments.

`%typemap(out)`

Converts function return value to a Perl5 value.

`%typemap(varin)`

Converts a Perl5 object to a global variable.

`%typemap(varout)`

Converts a global variable to a Perl5 object.

`%typemap(freearg)`

Cleans up a function argument after a function call

`%typemap(argout)`

Output argument handling

`%typemap(ret)`

Clean up return value from a function.

`%typemap(memberin)`

Setting of C++ member data (all languages).

`%typemap(check)`

Check value of input parameter.

### 31.7.3 Typemap variables

Within typemap code, a number of special variables prefaced with a `$` may appear. A full list of variables can be found in the "Typemaps" chapter. This is a list of the most common variables:

`$1`

A C local variable corresponding to the actual type specified in the `%typemap` directive. For input values, this is a C local variable that's supposed to hold an argument value. For output values, this is the raw result that's supposed to be returned to Perl.

`$input`

A Perl object holding the value of an argument of variable value.

`$result`

A Perl object that holds the result to be returned to Perl.

`$1_name`

The parameter name that was matched.

`$1_type`

The actual C datatype matched by the typemap.

`$1_ltype`

An assignable version of the datatype matched by the typemap (a type that can appear on the left-hand-side of a C assignment operation). This type is stripped of qualifiers and may be an altered version of `$1_type`. All arguments and local variables in wrapper functions are declared using this type so that their values can be properly assigned.

`$symname`

The Perl name of the wrapper function being created.

### 31.7.4 Useful functions

When writing typemaps, it is necessary to work directly with Perl5 objects. This, unfortunately, can be a daunting task. Consult the "perlguts" man-page for all of the really ugly details. A short summary of commonly used functions is provided here for reference. It should be stressed that SWIG can be used quite effectively without knowing any of these details--especially now that there are typemap libraries that can already been written.

**Perl Integer Functions**

```
int   SvIV(SV *);
void  sv_setiv(SV *sv, IV value);
SV   *newSViv(IV value);
int   SvIOK(SV *);
```

**Perl Floating Point Functions**

```
double SvNV(SV *);
void   sv_setnv(SV *, double value);
SV    *newSVnv(double value);
int    SvNOK(SV *);
```

**Perl String Functions**

```
char     *SvPV(SV *, STRLEN len);
void      sv_setpv(SV *, char *val);
void      sv_setpvn(SV *, char *val, STRLEN len);
SV       *newSVpv(char *value, STRLEN len);
int       SvPOK(SV *);
void      sv_catpv(SV *, char *);
void      sv_catpvn(SV *, char *, STRLEN);
```

**Perl References**

```
void      sv_setref_pv(SV *, char *, void *ptr);
int       sv_isobject(SV *);
SV       *SvRV(SV *);
int       sv_isa(SV *, char *0;
```

## 31.8 Typemap Examples

This section includes a few examples of typemaps. For more examples, you might look at the files "`perl5.swg`" and "`typemaps.i` " in the SWIG library.

### 31.8.1 Converting a Perl5 array to a char **

A common problem in many C programs is the processing of command line arguments, which are usually passed in an array of NULL terminated strings. The following SWIG interface file allows a Perl5 array reference to be used as a char ** datatype.

```
%module argv

// This tells SWIG to treat char ** as a special case
%typemap(in) char ** {
  AV *tempav;
  I32 len;
  int i;
  SV  **tv;
  if (!SvROK($input))
    croak("Argument $argnum is not a reference.");
  if (SvTYPE(SvRV($input)) != SVt_PVAV)
    croak("Argument $argnum is not an array.");
  tempav = (AV*)SvRV($input);
  len = av_len(tempav);
  $1 = (char **) malloc((len+2)*sizeof(char *));
  for (i = 0; i <= len; i++) {
    tv = av_fetch(tempav, i, 0);
    $1[i] = (char *) SvPV(*tv, PL_na);
  }
  $1[i] = NULL;
};

// This cleans up the char ** array after the function call
%typemap(freearg) char ** {
  free($1);
}

// Creates a new Perl array and places a NULL-terminated char ** into it
%typemap(out) char ** {
  AV *myav;
  SV **svs;
  int i = 0, len = 0;
  /* Figure out how many elements we have */
  while ($1[len])
    len++;
```

```
  svs = (SV **) malloc(len*sizeof(SV *));
  for (i = 0; i < len ; i++) {
    svs[i] = sv_newmortal();
    sv_setpv((SV*)svs[i], $1[i]);
  };
  myav = av_make(len, svs);
  free(svs);
  $result = newRV_noinc((SV*)myav);
  sv_2mortal($result);
  argvi++;
}

// Now a few test functions
%inline %{
  int print_args(char **argv) {
    int i = 0;
    while (argv[i]) {
      printf("argv[%d] = %s\n", i, argv[i]);
      i++;
    }
    return i;
  }

  // Returns a char ** list
  char **get_args() {
    static char *values[] = { "Dave", "Mike", "Susan", "John", "Michelle", 0};
    return &values[0];
  }
%}
```

When this module is compiled, the wrapped C functions can be used in a Perl script as follows:

```
use argv;
@a = ("Dave", "Mike", "John", "Mary");           # Create an array of strings
argv::print_args(\@a);                           # Pass it to our C function
$b = argv::get_args();                           # Get array of strings from C
print @$b, "\n";                                 # Print it out
```

### 31.8.2 Return values

Return values are placed on the argument stack of each wrapper function. The current value of the argument stack pointer is contained in a variable `argvi`. Whenever a new output value is added, it is critical that this value be incremented. For multiple output values, the final value of `argvi` should be the total number of output values.

The total number of return values should not exceed the number of input values unless you explicitly extend the argument stack. This can be done using the `EXTEND()` macro as in:

```
%typemap(argout) int *OUTPUT {
  if (argvi >= items) {
    EXTEND(sp, 1);              /* Extend the stack by 1 object */
  }
  $result = sv_newmortal();
  sv_setiv($result, (IV) *($1));
  argvi++;
}
```

### 31.8.3 Returning values from arguments

Sometimes it is desirable for a function to return a value in one of its arguments. This example describes the implementation of the `OUTPUT` typemap.

```
%module return

// This tells SWIG to treat an double * argument with name 'OutDouble' as
// an output value.

%typemap(argout) double *OUTPUT {
  $result = sv_newmortal();
  sv_setnv($result, *$input);
  argvi++;                    /* Increment return count -- important! */
}

// We don't care what the input value is. Ignore, but set to a temporary variable

%typemap(in, numinputs=0) double *OUTPUT(double junk) {
  $1 = &junk;
}

// Now a function to test it
%{
/* Returns the first two input arguments */
int multout(double a, double b, double *out1, double *out2) {
  *out1 = a;
  *out2 = b;
  return 0;
};
%}

// If we name both parameters OutDouble both will be output

int multout(double a, double b, double *OUTPUT, double *OUTPUT);
...
```

When this function is called, the output arguments are appended to the stack used to return results. This shows up an array in Perl. For example:

```
@r = multout(7, 13);
```

```
print "multout(7, 13) = @r\n";
($x, $y) = multout(7, 13);
```

### 31.8.4 Accessing array structure members

Consider the following data structure:

```
#define SIZE  8
typedef struct {
  int   values[SIZE];
  ...
} Foo;
```

By default, SWIG doesn't know how to the handle the values structure member because it's an array, not a pointer. In this case, SWIG makes the array member read-only. Reading will simply return a pointer to the first item in the array. To make the member writable, a "memberin" typemap can be used.

```
%typemap(memberin) int [SIZE] {
  int i;
  for (i = 0; i < SIZE; i++) {
    $1[i] = $input[i];
  }
}
```

Whenever a `int [SIZE]` member is encountered in a structure or class, this typemap provides a safe mechanism for setting its value.

As in the previous example, the typemap can be generalized for any dimension. For example:

```
%typemap(memberin) int [ANY] {
  int i;
  for (i = 0; i < $1_dim0; i++) {
    $1[i] = $input[i];
  }
}
```

When setting structure members, the input object is always assumed to be a C array of values that have already been converted from the target language. Because of this, the `memberin` typemap is almost always combined with the use of an "in" typemap. For example, the "in" typemap in the previous section would be used to convert an `int[]` array to C whereas the "memberin" typemap would be used to copy the converted array into a C data structure.

### 31.8.5 Turning Perl references into C pointers

A frequent confusion on the SWIG mailing list is errors caused by the mixing of Perl references and C pointers. For example, suppose you have a C function that modifies its arguments like this:

```
void add(double a, double b, double *c) {
  *c = a + b;
}
```

A common misinterpretation of this function is the following Perl script:

```
# Perl script
$a = 3.5;
$b = 7.5;
$c = 0.0;          # Output value
add($a, $b, \$c);    # Place result in c (Except that it doesn't work)
```

To make this work with a reference, you can use a typemap such as this:

```
%typemap(in) double * (double dvalue) {
  SV* tempsv;
  if (!SvROK($input)) {
    croak("expected a reference\n");
  }
  tempsv = SvRV($input);
  if ((!SvNOK(tempsv)) && (!SvIOK(tempsv))) {
    croak("expected a double reference\n");
  }
  dvalue = SvNV(tempsv);
  $1 = &dvalue;
}

%typemap(argout) double * {
  SV *tempsv;
  tempsv = SvRV($input);
  sv_setnv(tempsv, *$1);
}
```

Now, if you place this before the add function, you can do this:

```
$a = 3.5;
$b = 7.5;
$c = 0.0;
add($a, $b, \$c);            # Now it works!
print "$c\n";
```

### 31.8.6 Pointer handling

Occasionally, it might be necessary to convert pointer values that have been stored using the SWIG typed-pointer representation. To convert a pointer from Perl to C, the following function is used:

```
int SWIG_ConvertPtr(SV *obj, void **ptr, swig_type_info *ty, int flags)
```

Converts a Perl object `obj` to a C pointer. The result of the conversion is placed into the pointer located at `ptr`. `ty` is a SWIG type descriptor structure. `flags` is used to handle error checking and other aspects of conversion. `flags` is currently undefined and reserved for future expansion. Returns 0 on success and -1 on error.

```
void *SWIG_MakePtr(SV *obj, void *ptr, swig_type_info *ty, int flags)
```

Creates a new Perl pointer object. `obj` is a Perl SV that has been initialized to hold the result, `ptr` is the pointer to convert, `ty` is the SWIG type descriptor structure that describes the type, and `flags` is a flag that controls properties of the conversion. `flags` is currently undefined and reserved.

Both of these functions require the use of a special SWIG type-descriptor structure. This structure contains information about the mangled name of the datatype, type-equivalence information, as well as information about converting pointer values under C++ inheritance. For a type of `Foo *`, the type descriptor structure is usually accessed as follows:

```
Foo *f;
if (!SWIG_IsOK(SWIG_ConvertPtr($input, (void **) &f, SWIGTYPE_p_Foo, 0))) {
  SWIG_exception_fail(SWIG_TypeError, "in method '$symname', expecting type Foo");
}

SV *sv = sv_newmortal();
SWIG_MakePtr(sv, f, SWIGTYPE_p_Foo, 0);
```

In a typemap, the type descriptor should always be accessed using the special typemap variable `$1_descriptor`. For example:

```
%typemap(in) Foo * {
  if (!SWIG_IsOK(SWIG_ConvertPtr($input, (void **) &$1, $1_descriptor, 0))) {
    SWIG_exception_fail(SWIG_TypeError, "in method '$symname', expecting type Foo");
  }
}
```

If necessary, the descriptor for any type can be obtained using the `$descriptor()` macro in a typemap. For example:

```
%typemap(in) Foo * {
  if (!SWIG_IsOK(SWIG_ConvertPtr($input, (void **) &$1, $descriptor(Foo *), 0))) {
    SWIG_exception_fail(SWIG_TypeError, "in method '$symname', expecting type Foo");
  }
}
```

## 31.9 Proxy classes

**Out of date. Needs update.**

Using the low-level procedural interface, SWIG can also construct a high-level object oriented interface to C structures and C++ classes. This is done by constructing a Perl proxy class (also known as a shadow class) that provides an OO wrapper to the underlying code. This section describes the implementation details of the proxy interface.

### 31.9.1 Preliminaries

Proxy classes, are generated by default. If you want to turn them off, use the `-noproxy` command line option. For example:

```
$ swig -c++ -perl -noproxy example.i
```

When proxy classes are used, SWIG moves all of the low-level procedural wrappers to another package name. By default, this package is named 'modulec' where 'module' is the name of the module you provided with the `%module` directive. Then, in place of the original module, SWIG creates a collection of high-level Perl wrappers. In your scripts, you will use these high level wrappers. The wrappers, in turn, interact with the low-level procedural module.

### 31.9.2 Structure and class wrappers

Suppose you have the following SWIG interface file:

```
%module example
struct Vector {
  Vector(double x, double y, double z);
  ~Vector();
  double x, y, z;
};
```

When wrapped, SWIG creates the following set of low-level accessor functions as described in previous sections.

```
Vector *new_Vector(double x, double y, double z);
void    delete_Vector(Vector *v);
double  Vector_x_get(Vector *v);
double  Vector_x_set(Vector *v, double value);
double  Vector_y_get(Vector *v);
double  Vector_y_set(Vector *v, double value);
double  Vector_z_get(Vector *v);
double  Vector_z_set(Vector *v, double value);
```

However, when proxy classes are enabled, these accessor functions are wrapped inside a Perl class like this:

```
package example::Vector;
@ISA = qw( example );
%OWNER = ();
%BLESSEDMEMBERS = ();

sub new () {
  my $self = shift;
```

```
  my @args = @_;
  $self = vectorc::new_Vector(@args);
  return undef if (!defined($self));
  bless $self, "example::Vector";
  $OWNER{$self} = 1;
  my %retval;
  tie %retval, "example::Vector", $self;
  return bless \%retval, "Vector";
}

sub DESTROY {
  return unless $_[0]->isa('HASH');
  my $self = tied(%{$_[0]});
  delete $ITERATORS{$self};
  if (exists $OWNER{$self}) {
    examplec::delete_Vector($self));
    delete $OWNER{$self};
  }
}

sub FETCH {
  my ($self, $field) = @_;
  my $member_func = "vectorc::Vector_${field}_get";
  my $val = &$member_func($self);
  if (exists $BLESSEDMEMBERS{$field}) {
    return undef if (!defined($val));
    my %retval;
    tie %retval, $BLESSEDMEMBERS{$field}, $val;
    return bless \%retval, $BLESSEDMEMBERS{$field};
  }
  return $val;
}

sub STORE {
  my ($self, $field, $newval) = @_;
  my $member_func = "vectorc::Vector_${field}_set";
  if (exists $BLESSEDMEMBERS{$field}) {
    &$member_func($self, tied(%{$newval}));
  } else {
    &$member_func($self, $newval);
  }
}
```

Each structure or class is mapped into a Perl package of the same name. The C++ constructors and destructors are mapped into constructors and destructors for the package and are always named "new" and "DESTROY". The constructor always returns a tied hash table. This hash table is used to access the member variables of a structure in addition to being able to invoke member functions. The `%OWNER` and `%BLESSEDMEMBERS` hash tables are implementation details used internally and described shortly.

To use our new proxy class we can simply do the following:

```
# Perl code using Vector class
$v = new Vector(2, 3, 4);
$w = Vector->new(-1, -2, -3);

# Assignment of a single member
$v->{x} = 7.5;

# Assignment of all members
%$v = ( x=>3,
        y=>9,
        z=>-2);

# Reading members
$x = $v->{x};

# Destruction
$v->DESTROY();
```

### 31.9.3 Object Ownership

In order for proxy classes to work properly, it is necessary for Perl to manage some mechanism of object ownership. Here's the crux of the problem---suppose you had a function like this:

```
Vector *Vector_get(Vector *v, int index) {
  return &v[i];
}
```

This function takes a Vector pointer and returns a pointer to another Vector. Such a function might be used to manage arrays or lists of vectors (in C). Now contrast this function with the constructor for a Vector object:

```
Vector *new_Vector(double x, double y, double z) {
  Vector *v;
  v = new Vector(x, y, z);        // Call C++ constructor
  return v;
}
```

Both functions return a Vector, but the constructor is returning a brand-new Vector while the other function is returning a Vector that was already created (hopefully). In Perl, both vectors will be indistinguishable---clearly a problem considering that we would probably like the newly created Vector to be destroyed when we are done with it.

To manage these problems, each class contains two methods that access an internal hash table called `%OWNER`. This hash keeps a list of all of the objects that Perl knows that it has created. This happens in two cases: (1) when the constructor has been called, and (2) when a function implicitly creates a new object (as is done when SWIG needs to return a complex datatype by value). When the destructor is invoked, the Perl proxy class module checks the `%OWNER` hash to see if Perl created the object. If so, the C/C++ destructor is invoked. If not, we simply destroy the Perl object and leave the underlying C object alone (under the assumption that someone else must have created it).

This scheme works remarkably well in practice but it isn't foolproof. In fact, it will fail if you create a new C object in Perl, pass it on to a C function that remembers the object, and then destroy the corresponding Perl object (this situation turns out to come up frequently when constructing objects like linked lists and trees). When C takes possession of an object, you can change Perl's ownership by calling the `DISOWN` method (which will delete the object from the internal `%OWNER` hash).

The %OWNER hash is an implementation detail, discussed here only to help clarify the operation of ACQUIRE and DISOWN . You should not access %OWNER directly - the details of how it works (and possibly even its existence) may change in future SWIG versions.

```
# Perl code to change ownership of an object
$v = new Vector(x, y, z);
$v->DISOWN();
```

To acquire ownership of an object, the ACQUIRE method can be used.

```
# Given Perl ownership of a file
$u = Vector_get($v);
$u->ACQUIRE();
```

As always, a little care is in order. SWIG does not provide reference counting, garbage collection, or advanced features one might find in sophisticated languages.

### 31.9.4 Nested Objects

Suppose that we have a new object that looks like this:

```
struct Particle {
  Vector r;
  Vector v;
  Vector f;
  int type;
}
```

In this case, the members of the structure are complex objects that have already been encapsulated in a Perl proxy class. To handle these correctly, we use the %BLESSEDMEMBERS hash which would look like this (along with some supporting code):

```
package Particle;
...
%BLESSEDMEMBERS = (
        r => `Vector',
        v => `Vector',
        f => `Vector',
);
```

When fetching members from the structure, %BLESSEDMEMBERS is checked. If the requested field is present, we create a tied-hash table and return it. If not, we just return the corresponding member unmodified.

This implementation allows us to operate on nested structures as follows:

```
# Perl access of nested structure
$p = new Particle();
$p->{f}->{x} = 0.0;
%${$p->{v}} = ( x=>0, y=>0, z=>0);
```

### 31.9.5 Proxy Functions

When functions take arguments involving a complex object, it is sometimes necessary to write a proxy function. For example:

```
double dot_product(Vector *v1, Vector *v2);
```

Since Vector is an object already wrapped into a proxy class, we need to modify this function to accept arguments that are given in the form of tied hash tables. This is done by creating a Perl function like this:

```
sub dot_product {
  my @args = @_;
  $args[0] = tied(%{$args[0]});         # Get the real pointer values
  $args[1] = tied(%{$args[1]});
  my $result = vectorc::dot_product(@args);
  return $result;
}
```

This function replaces the original function, but operates in an identical manner.

### 31.9.6 Inheritance

Simple C++ inheritance is handled using the Perl @ISA array in each class package. For example, if you have the following interface file:

```
// shapes.i
// SWIG interface file for shapes class
%module shapes
%{
#include "shapes.h"
%}

class Shape {
public:
  virtual double area() = 0;
  virtual double perimeter() = 0;
  void    set_location(double x, double y);
};
class Circle : public Shape {
public:
  Circle(double radius);
```

```
  ~Circle();
  double area();
  double perimeter();
};
class Square : public Shape {
public:
  Square(double size);
  ~Square();
  double area();
  double perimeter();
}
```

The resulting, Perl wrapper class will create the following code:

```
Package Shape;
@ISA = (shapes);
...
Package Circle;
@ISA = (shapes Shape);
...
Package Square;
@ISA = (shapes Shape);
```

The `@ISA` array determines where to look for methods of a particular class. In this case, both the `Circle` and `Square` classes inherit functions from `Shape` so we'll want to look in the `Shape` base class for them. All classes also inherit from the top-level module `shapes`. This is because certain common operations needed to implement proxy classes are implemented only once and reused in the wrapper code for various classes and structures.

Since SWIG proxy classes are implemented in Perl, it is easy to subclass from any SWIG generated class. To do this, simply put the name of a SWIG class in the `@ISA` array for your new class. However, be forewarned that this is not a trivial problem. In particular, inheritance of data members is extremely tricky (and I'm not even sure if it really works).

### 31.9.7 Modifying the proxy methods

It is possible to override the SWIG generated proxy/shadow methods, using `%feature("shadow")`. It works like all the other%feature directives. Here is a simple example showing how to add some Perl debug code to the constructor:

```
/* Let's make the constructor of the class Square more verbose */
%feature("shadow") Square(double w)
%{
  sub new {
    my $pkg = shift;
    my $self = examplec::new_Square(@_);
    print STDERR "Constructed an @{[ref($self)]}\n";
    bless $self, $pkg if defined($self);
  }
%}

class Square {
public:
  Square(double w);
  ...
};
```

## 31.10 Adding additional Perl code

If writing support code in C isn't enough, it is also possible to write code in Perl. This code gets inserted in to the `.pm` file created by SWIG. One use of Perl code might be to supply a high-level interface to certain functions. For example:

```
void set_transform(Image *im, double x[4][4]);

...
/* Rewrite the high level interface to set_transform */
%perlcode %{
sub set_transform
{
  my ($im, $x) = @_;
  my $a = new_mat44();
  for (my $i = 0; $i < 4, $i++)
  {
    for (my $j = 0; $j < 4, $j++)
    {
      mat44_set($a, $i, $j, $x->[i][j])
    }
  }
  example.set_transform($im, $a);
  free_mat44($a);
}
%}
```

In this example, `set_transform()` provides a high-level Perl interface built on top of low-level helper functions. For example, this code now seems to work:

```
my $a =
  [[1, 0, 0, 0],
  [0, 1, 0, 0],
  [0, 0, 1, 0],
  [0, 0, 0, 1]];
set_transform($im, $a);
```

## 31.11 Cross language polymorphism

Proxy classes provide a more natural, object-oriented way to access extension classes. As described above, each proxy instance has an associated C++ instance, and method calls to the

proxy are passed to the C++ instance transparently via C wrapper functions.

This arrangement is asymmetric in the sense that no corresponding mechanism exists to pass method calls down the inheritance chain from C++ to Perl. In particular, if a C++ class has been extended in Perl (by extending the proxy class), these extensions will not be visible from C++ code. Virtual method calls from C++ are thus not able access the lowest implementation in the inheritance chain.

Changes have been made to SWIG to address this problem and make the relationship between C++ classes and proxy classes more symmetric. To achieve this goal, new classes called directors are introduced at the bottom of the C++ inheritance chain. The job of the directors is to route method calls correctly, either to C++ implementations higher in the inheritance chain or to Perl implementations lower in the inheritance chain. The upshot is that C++ classes can be extended in Perl and from C++ these extensions look exactly like native C++ classes. Neither C++ code nor Perl code needs to know where a particular method is implemented: the combination of proxy classes, director classes, and C wrapper functions takes care of all the cross-language method routing transparently.

### 31.11.1 Enabling directors

The director feature is disabled by default. To use directors you must make two changes to the interface file. First, add the "directors" option to the %module directive, like this:

```
%module(directors="1") modulename
```

Without this option no director code will be generated. Second, you must use the %feature("director") directive to tell SWIG which classes and methods should get directors. The %feature directive can be applied globally, to specific classes, and to specific methods, like this:

```
// generate directors for all classes that have virtual methods
%feature("director");

// generate directors for the virtual methods in class Foo
%feature("director") Foo;
```

You can use the %feature("nodirector") directive to turn off directors for specific classes or methods. So for example,

```
%feature("director") Foo;
%feature("nodirector") Foo::bar;
```

will generate directors for the virtual methods of class Foo except bar().

Directors can also be generated implicitly through inheritance. In the following, class Bar will get a director class that handles the methods one() and two() (but not three()):

```
%feature("director") Foo;
class Foo {
public:
  Foo(int foo);
  virtual void one();
  virtual void two();
};

class Bar: public Foo {
public:
  virtual void three();
};
```

then at the Perl side you can define

```
use mymodule;

package MyFoo;
use base 'mymodule::Foo';

sub one {
  print "one from Perl\n";
}
```

### 31.11.2 Director classes

For each class that has directors enabled, SWIG generates a new class that derives from both the class in question and a special `Swig::Director` class. These new classes, referred to as director classes, can be loosely thought of as the C++ equivalent of the Perl proxy classes. The director classes store a pointer to their underlying Perl object and handle various issues related to object ownership.

For simplicity let's ignore the `Swig::Director` class and refer to the original C++ class as the director's base class. By default, a director class extends all virtual methods in the inheritance chain of its base class (see the preceding section for how to modify this behavior). Virtual methods that have a final specifier are unsurprisingly excluded. Thus the virtual method calls, whether they originate in C++ or in Perl via proxy classes, eventually end up in at the implementation in the director class. The job of the director methods is to route these method calls to the appropriate place in the inheritance chain. By "appropriate place" we mean the method that would have been called if the C++ base class and its extensions in Perl were seamlessly integrated. That seamless integration is exactly what the director classes provide, transparently skipping over all the messy extension API glue that binds the two languages together.

In reality, the "appropriate place" is one of only two possibilities: C++ or Perl. Once this decision is made, the rest is fairly easy. If the correct implementation is in C++, then the lowest implementation of the method in the C++ inheritance chain is called explicitly. If the correct implementation is in Perl, the Perl API is used to call the method of the underlying Perl object (after which the usual virtual method resolution in Perl automatically finds the right implementation).

Now how does the director decide which language should handle the method call? The basic rule is to handle the method in Perl, unless there's a good reason not to. The reason for this is simple: Perl has the most "extended" implementation of the method. This assertion is guaranteed, since at a minimum the Perl proxy class implements the method. If the method in question has been extended by a class derived from the proxy class, that extended implementation will execute exactly as it should. If not, the proxy class will route the method call into a C wrapper function, expecting that the method will be resolved in C++. The wrapper will call the virtual method of the C++ instance, and since the director extends this the call will end up right back in the director method. Now comes the "good reason not to" part. If the director method were to blindly call the Perl method again, it would get stuck in an infinite loop. We avoid this situation by adding special code to the C wrapper function that tells the director method to not do this. The C wrapper function compares the pointer to the Perl object that called the wrapper function to the pointer stored by the director. If these are the same, then the C wrapper function tells the director to resolve the method by calling up the C++ inheritance chain, preventing an infinite loop.

One more point needs to be made about the relationship between director classes and proxy classes. When a proxy class instance is created in Perl, SWIG creates an instance of the original C++ class. This is exactly what happens without directors and is true even if directors are enabled for the particular class in question. When a class *derived* from a proxy class is created, however, SWIG then creates an instance of the corresponding C++ director class. The reason for this difference is that user-defined subclasses may override or extend methods of the original class, so the director class is needed to route calls to these methods correctly. For unmodified proxy classes, all methods are ultimately implemented in C++ so there is no need for the extra overhead involved with routing the calls through Perl.

### 31.11.3 Ownership and object destruction

Memory management issues are slightly more complicated with directors than for proxy classes alone. Perl instances hold a pointer to the associated C++ director object, and the director in turn holds a pointer back to a Perl object. By default, proxy classes own their C++ director object and take care of deleting it when they are garbage collected.

This relationship can be reversed by calling the special `DISOWN()` method of the proxy class. After calling this method the director class increments the reference count of the Perl object. When the director class is deleted it decrements the reference count. Assuming no outstanding references to the Perl object remain, the Perl object will be destroyed at the same time. This is a good thing, since directors and proxies refer to each other and so must be created and destroyed together. Destroying one without destroying the other will likely cause your program to segfault.

Also note that due to the proxy implementation, the `DESTROY()` method on directors can be called for several reasons, many of which have little to do with the teardown of an object instance. To help disambiguate this, a second argument is added to the `DESTROY()` call when a C++ director object is being released. So, to avoid running your clean-up code when an object is not really going away, or after it has already been reclaimed, it is suggested that custom destructors in Perl subclasses looks something like:

```
sub DESTROY {
  my($self, $final) = @_;
  if($final) {
    # real teardown code
  }
  shift->SUPER::DESTROY(@_);
}
```

### 31.11.4 Exception unrolling

With directors routing method calls to Perl, and proxies routing them to C++, the handling of exceptions is an important concern. By default, the directors ignore exceptions that occur during method calls that are resolved in Perl. To handle such exceptions correctly, it is necessary to temporarily translate them into C++ exceptions. This can be done with the %feature("director:except") directive. The following code should suffice in most cases:

```
%feature("director:except") {
  if ($error != NULL) {
    throw Swig::DirectorMethodException();
  }
}
```

This code will check the Perl error state after each method call from a director into Perl, and throw a C++ exception if an error occurred. This exception can be caught in C++ to implement an error handler.

It may be the case that a method call originates in Perl, travels up to C++ through a proxy class, and then back into Perl via a director method. If an exception occurs in Perl at this point, it would be nice for that exception to find its way back to the original caller. This can be done by combining a normal %exception directive with the `director:except` handler shown above. Here is an example of a suitable exception handler:

```
%exception {
  try { $action }
  catch (Swig::DirectorException &e) { SWIG_fail; }
}
```

The class Swig::DirectorException used in this example is actually a base class of Swig::DirectorMethodException, so it will trap this exception. Because the Perl error state is still set when Swig::DirectorMethodException is thrown, Perl will register the exception as soon as the C wrapper function returns.

### 31.11.5 Overhead and code bloat

Enabling directors for a class will generate a new director method for every virtual method in the class' inheritance chain. This alone can generate a lot of code bloat for large hierarchies. Method arguments that require complex conversions to and from target language types can result in large director methods. For this reason it is recommended that you selectively enable directors only for specific classes that are likely to be extended in Perl and used in C++.

Compared to classes that do not use directors, the call routing in the director methods does add some overhead. In particular, at least one dynamic cast and one extra function call occurs per method call from Perl. Relative to the speed of Perl execution this is probably completely negligible. For worst case routing, a method call that ultimately resolves in C++ may take one extra detour through Perl in order to ensure that the method does not have an extended Perl implementation. This could result in a noticeable overhead in some cases.

Although directors make it natural to mix native C++ objects with Perl objects (as director objects) via a common base class pointer, one should be aware of the obvious fact that method calls to Perl objects will be much slower than calls to C++ objects. This situation can be optimized by selectively enabling director methods (using the %feature directive) for only those methods that are likely to be extended in Perl.

### 31.11.6 Typemaps

Typemaps for input and output of most of the basic types from director classes have been written. These are roughly the reverse of the usual input and output typemaps used by the wrapper code. The typemap operation names are 'directorin', 'directorout', and 'directorargout'. The director code does not currently use any of the other kinds of typemaps. It is not clear at this point which kinds are appropriate and need to be supported.

---

# 32 SWIG and PHP


- Generating PHP Extensions
  - Building a loadable extension
  - Using PHP Extensions
- Basic PHP interface
  - Constants
  - Global Variables
  - Functions
  - Overloading
  - Pointers and References
  - Structures and C++ classes
    - Using -noproxy
    - Constructors and Destructors
    - Static Member Variables
    - Static Member Functions
    - Specifying Implemented Interfaces
    - Dynamic Properties
  - PHP Pragmas, Startup and Shutdown code
- Cross language polymorphism
  - Enabling directors
  - Director classes
  - Ownership and object destruction
  - Exception unrolling
  - Overhead and code bloat
  - Typemaps
  - Miscellaneous


In this chapter, we discuss SWIG's support of PHP. Currently any PHP8 release should work.

Support for PHP8 was added in SWIG 4.1.0. Support for PHP7 was added in SWIG 3.0.11 and removed in 4.2.0. Support for PHP5 was removed in SWIG 4.0.0 and support for PHP4 was removed in SWIG 1.3.37. There never was a PHP6 release.

In order to use this module, you will need to have a copy of the PHP include files to compile the SWIG generated C/C++ sources. If you installed PHP from a binary package, you may need to install a "php-dev" or "php-devel" package for these to be installed. You can find out where these files are by running `php-config --includes` . To use the built PHP module you will need either the php binary or the Apache php module. If you want to build your extension into php directly, you will need the complete PHP source tree available.

## 32.1 Generating PHP Extensions

To build a PHP extension, run swig using the `-php` option (you can also use `-php7` - PHP7 and PHP8 have a largely compatible C extension API, hence the same option name has been used for both). For example:

```
swig -php example.i
```

This will produce 2 files: example_wrap.c and php_example.h. The first file, `example_wrap.c` contains all of the C code needed to build a PHP extension. The second file, `php_example.h` contains the header information needed if you wish to statically link the extension into the php interpreter.

If the interface file uses `%pragma(php) include=...` or `%pragma(php) code=...` then SWIG will also generate a third file,`example.php` to contain what these specify. In SWIG < 4.1.0, this third file was always generated as it defined the PHP classes, etc (but this is now done via C code in `example_wrap.c`) and also contained code to dynamically load the extension (but this used the PHP `dl()` function, which isn't recommended nowadays).

SWIG can generate PHP extensions from C++ libraries as well when given the `-c++` option. The support for C++ is discussed in more detail in section 27.2.6. The generated C++ wrapper will be called example_wrap.cxx. You can specify a different extension for the C++ wrapper using `-cppext` - e.g. if you want example_wrap.cc use `-cppext cc`.

The usual (and recommended) way is to build the extension as a separate dynamically loaded module (which is supported by all modern operating systems).

It is also possible to rebuild PHP from source so that your module is statically linked into the php executable/library. This is a lot more work, and also requires a full rebuild of PHP to update your module, and it doesn't play nicely with package system. We don't recommend this approach, or provide explicit support for it.

### 32.1.1 Building a loadable extension

To build your module as a dynamically loadable extension, use compilation commands like these (if you aren't using GCC, the commands will be different, and there may be some variation between platforms - these commands should at least work for Linux though):

```
$ gcc `php-config --includes` -fPIC -c example_wrap.c example.c
$ gcc -shared example_wrap.o example.o -o example.so
```

### 32.1.2 Using PHP Extensions

To test the extension from a PHP script, you first need to tell PHP to load it. The recommended (and simplest!) way to do this is to copy it to PHP's default extension directory and add a line like this to the `[PHP]` section of `php.ini`:

```
        extension=modulename
```

If the module is not in PHP's default extension directory, you also need to specify the path, in which case you'll also need to deal with platform-specific naming - for example, on Linux:

```
        extension=/path/to/modulename.so
```

but on Microsoft Windows you'd need to use:

```
        extension=/path/to/php_modulename.dll
```

If you're using the PHP CLI SAPI it's possible (but not recommended) to use the dl() function to load an extension at run time, by adding a line like this to the start of each PHP script which uses your extension:

```
        dl("modulename"); // Load the module
```

Again, if the module isn't in PHP's default extension directory you'll also need to specify the path and deal with that varying by platform.

For security reasons PHP no longer supports `dl()` when running PHP through a webserver, so this isn't an option there.

## 32.2 Basic PHP interface

It is important to understand that PHP uses a single global namespace into which all symbols from extension modules are loaded. It is quite possible for names of symbols in one extension module to clash with other symbols unless care is taken to `%rename` them. At present SWIG doesn't have support for generating wrappers which make use of PHP's namespace feature.

### 32.2.1 Constants

These work in much the same way as in C/C++. Constants can be defined by using either the normal C pre-processor declarations, or the `%constant` SWIG directive. These will then be available from your PHP script as a PHP constant, (i.e. no dollar sign is needed to access them.) For example, with a swig interface file like this,

```
%module example

#define PI 3.14159

%constant int E  = 2.71828
```

you can access the constants in your PHP script like this,

```
echo "PI = " . PI . "\n";
echo "E = " . E . "\n";
```

### 32.2.2 Global Variables

Because PHP does not provide a mechanism to intercept access and assignment of global variables, global variables are supported through the use of automatically generated accessor functions.

```
%module example;

%inline %{
  double seki = 2;
  void print_seki() {
    zend_printf("seki is now %f\n", seki);
  }
%}
```

is accessed as follows:

```
print seki_get();
seki_set( seki_get() * 2); # The C variable is now 4.
print seki_get();
```

SWIG supports global variables of all C datatypes including pointers and complex objects. To support additional types, you just need to supply the standard `in` and `out` typemaps, which get used because of the wrapping as `_get()` and `_set()` functions.

SWIG honors the `%immutable` modifier by not generating a `_set` method (so attempting to call it will give a PHP fatal error). A `_get` method is still generated so this provides read-only access to the variable from the PHP script.

At this time SWIG does not support custom accessor methods.

### 32.2.3 Functions

C functions are converted into PHP functions. Default/optional arguments are also allowed. An interface file like this :

```
%module example
int foo(int a);
double bar(double, double b = 3.0);
...
```

Will be accessed in PHP like this :

```
$a = foo(2);
$b = bar(3.5, -1.5);
$c = bar(3.5);  # Use default argument for 2nd parameter
```

SWIG generates PHP type declarations for function parameters and return types for PHP 8 and later.

You can control the generation of PHP type declarations using the "php:type" %feature. This has three settings:

- If unset or set to "0" then no type declarations are generated, e.g.: `%feature("php:type", "0");`
- If set to "1" then type declarations are generated for both parameters and return types, e.g.: `%feature("php:type", "1");`
- The default setting is "compat", which is the same as "1" except no return type declarations are generated for virtual methods for which directors are enabled. This provides better compatibility for PHP subclasses of wrapped virtual methods in existing SWIG-generated bindings, e.g.: `%feature("php:type", "compat");`

If you have an existing PHP interface and are upgrading to SWIG >= 4.1.0 then the default "compat" setting should work well.

If you're writing a new set of bindings and **only targeting PHP8 or newer** then enabling type declarations everywhere probably makes sense. It will only actually make a difference if you enable directors and are wrapping C++ classes with virtual methods, but doing it anyway means you won't forget to if the code you are wrapping later evolves to have such classes and methods.

The type declaration information will make the generated source code and compiler extension module larger, so you might want to turn off type declarations if keeping these small is important to you. If you find you need to turn off type declarations to fix a problem, please let us know via our github issue tracker.

Note that being a SWIG feature this can be specified globally (like above) or per class, per method, etc. See the %feature directives section for full details of how to control at a fine-grained level.

The PHP type information is specified via a "phptype" attribute on "in" and "out" typemaps, and these have been added for all the typemaps we supply for PHP. We don't currently support this for "argout" templates, but probably will in a future version.

If you have written custom SWIG typemaps for PHP and want to add PHP type declarations, then the syntax is very like how you'd specify the type in PHP code, e.g. `%typemap(in, phptype="int|string|Foo")` means the typemap accepts a PHP int or string or an object of class Foo, `%typemap(in, phptype="?int")` means a PHP int or NULL, etc. As well as the standard PHP type declaration types, SWIG also understands the special type "SWIGTYPE" as an entry in phptype, which means the PHP type corresponding to the type that this typemap matched on - for a object this will give you the PHP class for the object, and for a pointer to a non-class type it will give you the name of the PHP class SWIG created for that pointer type.

### 32.2.4 Overloading

Although PHP does not support overloading functions natively, swig will generate dispatch functions which will use `%typecheck` typemaps to allow overloading. This dispatch function's operation and precedence is described in Overloaded functions and methods.

### 32.2.5 Pointers and References

Since SWIG 4.1.0, SWIG wraps C/C++ classes directly with PHP objects. Pointers to other types are also wrapped as PHP objects - mostly this is an implementation detail, but it's visible from PHP via `is_object()` and similar. In earlier SWIG versions, PHP resources were used to wrap both classes and pointers to other types.

There are multiple ways to wrap pointers to simple types. Given the following C method:

```
 void add( int *in1, int *in2, int *result);
```

One can include **cpointer.i** to generate PHP wrappers to `int *`.

```
%module example
%include "cpointer.i"
%pointer_functions(int, intp)

void add( int *in1, int *in2, int *result);
```

This will result in the following usage in PHP:

```
<?php

$in1=copy_intp(3);
$in2=copy_intp(5);
$result=new_intp();

add( $in1, $in2, $result );

echo "The sum " . intp_value($in1) . " + " . intp_value($in2) . " = " . intp_value( $result) . "\n";
```

An alternative would be to use the include **typemaps.i** which defines named typemaps for INPUT, OUTPUT and INOUT variables. One needs to either `%apply` the appropriate typemap or adjust the parameter names as appropriate.

```
%module example
%include "typemaps.i"

void add( int *INPUT, int *INPUT, int *OUTPUT);
```

This will result in the following usage in PHP:

```
<?php

$in1 = 3;
$in2 = 5;
$result= add($in1, $in2);  # Note using variables for the input is unnecessary.

echo "The sum $in1 + $in2 = $result\n";
```

Because PHP has a native concept of reference, it may seem more natural to the PHP developer to use references to pass pointers. To enable this, one needs to include **phppointers.i** which defines the named typemap REF.

In case you write your own typemaps, SWIG supports an attribute called `byref`: if you set that, then SWIG will make sure that the generated wrapper function will want the input parameter as a reference.

```
%module example
%include "phppointers.i"

void add( int *REF, int *REF, int *REF);
```

This will result in the following usage in PHP:

```
<?php

$in1 = 3;
$in2 = 5;
$result = 0;
add($in1, $in2, $result);

echo "The sum $in1 + $in2 = $result\n";
```

It is important to note that a php variable which is NULL when passed by reference would end up passing a NULL pointer into the function. In PHP, an unassigned variable (i.e. where the first reference to the variable is not an assignment) is NULL. In the above example, if any of the three variables had not been assigned, a NULL pointer would have been passed into `add`. Depending on the implementation of the function, this may or may not be a good thing.

We chose to allow passing NULL pointers into functions because that is sometimes required in C libraries. A NULL pointer can be created in PHP in a number of ways: by using `unset` on an existing variable, or assigning `NULL` to a variable.

### 32.2.6 Structures and C++ classes

SWIG wraps C++ structs and classes with PHP classes. Since SWIG 4.1.0, this is done entirely via PHP's C API - earlier SWIG versions generated a PHP wrapper script which defined proxy classes which called a set of flat functions which actually wrapped the C++ class.

This interface file

```
%module vector

class Vector {
public:
  double x, y, z;
  Vector();
  ~Vector();
  double magnitude();
};

struct Complex {
 double re, im;
};
```

Would be used in the following way from PHP:

```
<?php

  $v = new Vector();
  $v->x = 3;
  $v->y = 4;
  $v->z = 5;

  echo "Magnitude of ($v->x, $v->y, $v->z) = " . $v->magnitude() . "\n";
```

```
$v = NULL;   # destructor called.

$c = new Complex();

$c->re = 0;
$c->im = 0;

# $c destructor called when $c goes out of scope.
```

Member variables and methods are accessed using the -> operator.

#### 32.2.6.1 Using -noproxy

SWIG/PHP used to support a `-noproxy` option to flatten the class structure and generate collections of named flat functions. This is no longer supported as of SWIG 4.1.0.

#### 32.2.6.2 Constructors and Destructors

The constructor is called when `new Object()` is used to create an instance of the object. If multiple constructors are defined for an object, function overloading will be used to determine which constructor to execute.

Because PHP uses reference counting, simple assignment of one variable to another such as:

```
$ref = $v;
```

causes the symbol `$ref` to refer to the same underlying object as `$v`. This does not result in a call to the C++ copy constructor or copy assignment operator.

One can force execution of the copy constructor by using:

```
$o_copy = new Object($o);
```

Destructors are automatically called when all variables referencing the instance are reassigned or go out of scope. The destructor is not available to be called manually. To force a destructor to be called the programmer can either reassign the variable or call `unset($v)`

#### 32.2.6.3 Static Member Variables

Static member variables in C++ are not wrapped as such in PHP as it does not appear to be possible to intercept accesses to such variables. Therefore, static member variables are wrapped using a class function with the same name, which returns the current value of the class variable. For example

```
%module example

class Ko {
  static int threats;
};
```

would be accessed in PHP as,

```
echo "There have now been " . Ko::threats() . " threats\n";
```

To set the static member variable, pass the value as the argument to the class function, e.g.

```

Ko::threats(10);

echo "There have now been " . Ko::threats() . " threats\n";
```

#### 32.2.6.4 Static Member Functions

Static member functions are supported in PHP using the `class::function()` syntax. For example

```
%module example
class Ko {
  static void threats();
};
```

would be executed in PHP as

```
Ko::threats();
```

#### 32.2.6.5 Specifying Implemented Interfaces

PHP supports the concept of abstract interfaces which a class can implement. Since SWIG 3.0.3, you can tell SWIG that a wrapped class (for example `MyIterator`) implements the `Iterator` interface like so:

```
%typemap("phpinterfaces") MyIterator "Iterator"
```

If there are multiple interfaces, just list them separated by commas.

#### 32.2.6.6 Dynamic Properties

Historically PHP has supported dynamic class properties and SWIG has implemented them too (because we implement the magic `__get()`, `__set()` and `__isset()` methods we need to include explicit handling).

PHP 8.2 deprecates dynamic class properties - initially they'll warn, and apparently they'll not work by default in PHP 9.0.

In PHP code dynamic properties can be enabled for a class by marking that class with the attribute `#[AllowDynamicProperties]`.

To follow this PHP change, as of SWIG 4.1.0 you now need enable dynamic properties for any classes you want to support them. To enable for class `Foo`:

```
%feature("php:allowdynamicproperties", 1) Foo;
```

or to enable them for all wrapped classes:

```
%feature("php:allowdynamicproperties", 1);
```

Note that unknown features are ignored, so you can add use these unconditionally in your interface file and it'll work with older SWIG too.

There was a bug in SWIG 4.1.0 where setting this feature to any value enabled it - SWIG 4.2.0 fixed this and you can now set it to 0 to turn it off (for example, you might want to enabled it for everything and then selectively turn it off for specific classes).

### 32.2.7 PHP Pragmas, Startup and Shutdown code

You can get SWIG to generate an "example.php" file by specifying the code to put in it using the **code** pragma.

```
%module example
%pragma(php) code="
# This code is inserted into example.php
echo \"example.php execution\\n\";
"
```

Results in the following in "example.php"

```
# This code is inserted into example.php
echo "example.php execution\n";
```

The **version** pragma can be used to add version to generated PHP extension module. The version is inserted in the zend_module_entry block.

```
%module example
%pragma(php) version="1.5"
```

The **include** pragma is a short cut to add include statements to the example.php file.

```
%module example
%pragma(php) code="
include \"include.php\";
"
%pragma(php) include="include.php"   // equivalent.
```

The **phpinfo** pragma inserts code in the `PHP_MINFO_FUNCTION` which is called from PHP's phpinfo() function.

```
%module example;
%pragma(php) phpinfo="
  zend_printf("An example of PHP support through SWIG\n");
  php_info_print_table_start();
  php_info_print_table_header(2, \"Directive\", \"Value\");
  php_info_print_table_row(2, \"Example support\", \"enabled\");
  php_info_print_table_end();
"
```

To insert code into the `PHP_MINIT_FUNCTION`, one can use either `%init` or `%minit`.

```
%module example;
%init {
  zend_printf("Inserted into PHP_MINIT_FUNCTION\n");
}
%minit {
  zend_printf("Inserted into PHP_MINIT_FUNCTION\n");
}
```

To insert code into the `PHP_MSHUTDOWN_FUNCTION`, one can use either `%shutdown` or `%mshutdown`.

```
%module example;
%mshutdown {
  zend_printf("Inserted into PHP_MSHUTDOWN_FUNCTION\n");
}
```

The `%rinit` and `%rshutdown` statements are very similar but insert code into the request init (PHP_RINIT_FUNCTION) and request shutdown (PHP_RSHUTDOWN_FUNCTION) code respectively.

## 32.3 Cross language polymorphism

Proxy classes provide a more natural, object-oriented way to access extension classes. As described above, each proxy instance has an associated C++ instance, and method calls to the proxy are passed to the C++ instance transparently.

This arrangement is asymmetric in the sense that no corresponding mechanism exists to pass method calls down the inheritance chain from C++ to PHP. In particular, if a C++ class has been extended in PHP (by extending the proxy class), these extensions will not be visible from C++ code. Virtual method calls from C++ are thus not able access the lowest implementation in the inheritance chain.

Changes have been made to SWIG 1.3.18 to address this problem and make the relationship between C++ classes and proxy classes more symmetric. To achieve this goal, new classes called directors are introduced at the bottom of the C++ inheritance chain. Support for generating PHP classes has been added in SWIG 1.3.40. The job of the directors is to route method calls correctly, either to C++ implementations higher in the inheritance chain or to PHP implementations lower in the inheritance chain. The upshot is that C++ classes can be extended in PHP and

from C++ these extensions look exactly like native C++ classes. Neither C++ code nor PHP code needs to know where a particular method is implemented: the combination of proxy classes, director classes, and C wrapper functions takes care of all the cross-language method routing transparently.

### 32.3.1 Enabling directors

The director feature is disabled by default. To use directors you must make two changes to the interface file. First, add the "directors" option to the %module directive, like this:

```
%module(directors="1") modulename
```

Without this option no director code will be generated. Second, you must use the %feature("director") directive to tell SWIG which classes and methods should get directors. The %feature directive can be applied globally, to specific classes, and to specific methods, like this:

```
// generate directors for all classes that have virtual methods
%feature("director");

// generate directors for the virtual methods in class Foo
%feature("director") Foo;
```

You can use the %feature("nodirector") directive to turn off directors for specific classes or methods. So for example,

```
%feature("director") Foo;
%feature("nodirector") Foo::bar;
```

will generate directors for the virtual methods of class Foo except bar().

Directors can also be generated implicitly through inheritance. In the following, class Bar will get a director class that handles the methods one() and two() (but not three()):

```
%feature("director") Foo;
class Foo {
public:
  Foo(int foo);
  virtual void one();
  virtual void two();
};

class Bar: public Foo {
public:
  virtual void three();
};
```

then at the PHP side you can define

```
class MyFoo extends Foo {
  function one() {
    print "one from php\n";
  }
}
```

### 32.3.2 Director classes

For each class that has directors enabled, SWIG generates a new class that derives from both the class in question and a special `Swig::Director` class. These new classes, referred to as director classes, can be loosely thought of as the C++ equivalent of the PHP proxy classes. The director classes store a pointer to their underlying PHP object. Indeed, this is quite similar to `struct swig_object_wrapper` which is used to implement the PHP proxy classes.

For simplicity let's ignore the `Swig::Director` class and refer to the original C++ class as the director's base class. By default, a director class extends all virtual methods in the inheritance chain of its base class (see the preceding section for how to modify this behavior). Virtual methods that have a final specifier are unsurprisingly excluded. Thus the virtual method calls, whether they originate in C++ or in PHP via proxy classes, eventually end up in at the implementation in the director class. The job of the director methods is to route these method calls to the appropriate place in the inheritance chain. By "appropriate place" we mean the method that would have been called if the C++ base class and its extensions in PHP were seamlessly integrated. That seamless integration is exactly what the director classes provide, transparently skipping over all the messy extension API glue that binds the two languages together.

In reality, the "appropriate place" is one of only two possibilities: C++ or PHP. Once this decision is made, the rest is fairly easy. If the correct implementation is in C++, then the lowest implementation of the method in the C++ inheritance chain is called explicitly. If the correct implementation is in PHP, the Zend API is used to call the method of the underlying PHP object (after which the usual virtual method resolution in PHP automatically finds the right implementation).

Now how does the director decide which language should handle the method call? The basic rule is to handle the method in PHP, unless there's a good reason not to. The reason for this is simple: PHP has the most "extended" implementation of the method. This assertion is guaranteed, since at a minimum the PHP proxy class implements the method. If the method in question has been extended by a class derived from the proxy class, that extended implementation will execute exactly as it should. If not, the proxy class will route the method call into a C wrapper function, expecting that the method will be resolved in C++. The wrapper will call the virtual method of the C++ instance, and since the director extends this the call will end up right back in the director method. Now comes the "good reason not to" part. If the director method were to blindly call the PHP method again, it would get stuck in an infinite loop. We avoid this situation by adding special code to the C wrapper function that tells the director method to not do this. The C wrapper function compares the called and the declaring class name of the given method. If these are not the same, then the C wrapper function tells the director to resolve the method by calling up the C++ inheritance chain, preventing an infinite loop.

One more point needs to be made about the relationship between director classes and proxy classes. When a proxy class instance is created in PHP, SWIG creates an instance of the original C++ class and stores it in the `struct swig_object_wrapper`. This is true whether or not directors are enabled for the particular class in question. However when a class *derived* from a proxy class is created, SWIG instead creates an instance of the corresponding C++ director class. The reason for this difference is that user-defined subclasses may override or extend methods of the original class, so the director class is needed to route calls to these methods correctly. For unmodified proxy classes, all methods are ultimately implemented in C++ so there is no need for the extra overhead involved with routing the calls through PHP.

### 32.3.3 Ownership and object destruction

Memory management issues are slightly more complicated with directors than for proxy classes alone. PHP instances hold a pointer to the associated C++ director object, and the director in turn holds a pointer back to the PHP object. By default, proxy classes own their C++ director object and take care of deleting it when they are garbage collected.

This relationship can be reversed by calling the special `->thisown` property of the proxy class. After setting this property to `0`, the director class no longer destroys the PHP object. Assuming no outstanding references to the PHP object remain, the PHP object will be destroyed at the same time. This is a good thing, since directors and proxies refer to each other and so must be created and destroyed together. Destroying one without destroying the other will likely cause your program to segfault.

Here is an example:

```
class Foo {
public:
  ...
};
```

```
class FooContainer {
public:
  void addFoo(Foo *);
  ...
};
```

```
$c = new FooContainer();
$a = new Foo();
$a->thisown = 0;
$c->addFoo($a);
```

In this example, we are assuming that FooContainer will take care of deleting all the Foo pointers it contains at some point.

### 32.3.4 Exception unrolling

With directors routing method calls to PHP, and proxies routing them to C++, the handling of exceptions is an important concern. By default, an exception thrown in PHP code called from C++ causes the PHP interpreter to flag that an exception is thrown, then return passes to C++ as if the PHP function had returned `Null`. Assuming the directorout typemaps handle this (those SWIG defines by default should) then once control returns to PHP code again, the PHP exception will actually propagate.

Sometimes this control flow is problematic, and you want to skip any handling in the C++ code. To achieve this, it is necessary to temporarily translate the PHP exception into a C++ exception. This can be achieved using the %feature("director:except") directive. The following code should suffice in most cases:

```
%feature("director:except") {
#if SWIG_VERSION >= 0x040100
  if ($error != NULL)
#else
  if ($error == FAILURE)
#endif
  {
    throw Swig::DirectorMethodException();
  }
}
```

If you only need to support SWIG >= 4.1.0, you can just use the `($error != NULL)` condition.

In SWIG 4.1.0, `$error` was changed in the SWIG/PHP director implementation to make it work more like how it does for other languages. Previously, `$error` didn't actually indicate an exception, but instead was only set to `FAILURE` if there was a problem calling the PHP method. Now `$error` indicates if the PHP method threw a PHP exception, and directorout typemaps for PHP no longer need to be gated by `if (EG(exception))`.

This code will check the PHP error state after each method call from a director into PHP, and throw a C++ exception if an error occurred. This exception can be caught in C++ to implement an error handler. Currently no information about the PHP error is stored in the Swig::DirectorMethodException object, but this will likely change in the future.

It may be the case that a method call originates in PHP, travels up to C++ through a proxy class, and then back into PHP via a director method. If an exception occurs in PHP at this point, it would be nice for that exception to find its way back to the original caller. This can be done by combining a normal %exception directive with the `director:except` handler shown above. Here is an example of a suitable exception handler:

```
%exception {
  try { $action }
  catch (Swig::DirectorException &e) { SWIG_fail; }
}
```

The class Swig::DirectorException used in this example is actually a base class of Swig::DirectorMethodException, so it will trap this exception. Because the PHP error state is still set when Swig::DirectorMethodException is thrown, PHP will register the exception as soon as the C wrapper function returns.

### 32.3.5 Overhead and code bloat

Enabling directors for a class will generate a new director method for every virtual method in the class' inheritance chain. This alone can generate a lot of code bloat for large hierarchies. Method arguments that require complex conversions to and from target language types can result in large director methods. For this reason it is recommended that you selectively enable directors only for specific classes that are likely to be extended in PHP and used in C++.

Compared to classes that do not use directors, the call routing in the director methods does add some overhead. In particular, at least one dynamic cast and one extra function call occurs per method call from PHP. Relative to the speed of PHP execution this is probably completely negligible. For worst case routing, a method call that ultimately resolves in C++ may take one extra detour through PHP in order to ensure that the method does not have an extended PHP implementation. This could result in a noticeable overhead in some cases.

Although directors make it natural to mix native C++ objects with PHP objects (as director objects) via a common base class pointer, one should be aware of the obvious fact that method calls to PHP objects will be much slower than calls to C++ objects. This situation can be optimized by selectively enabling director methods (using the %feature directive) for only those methods that are likely to be extended in PHP.

### 32.3.6 Typemaps

Typemaps for input and output of most of the basic types from director classes have been written. These are roughly the reverse of the usual input and output typemaps used by the wrapper code. The typemap operation names are 'directorin', 'directorout', and 'directorargout'. The director code does not currently use any of the other kinds of typemaps. It is not clear at this point which kinds are appropriate and need to be supported.

### 32.3.7 Miscellaneous

Director typemaps for STL classes are mostly in place, and hence you should be able to use std::string, etc., as you would any other type.

---

# 33 SWIG and Python


- Overview
- Preliminaries
  - Running SWIG
  - Using distutils
  - Hand compiling a dynamic module
  - Static linking
  - Using your module
  - Compilation of C++ extensions
  - Compiling for 64-bit platforms
  - Building Python extensions under Windows
  - Additional Python commandline options
- A tour of basic C/C++ wrapping
  - Modules

- Functions
- Global variables
- Constants and enums
- Pointers
- Structures
- C++ classes
- C++ inheritance
- Pointers, references, values, and arrays
- C++ overloaded functions
- C++ operators
- C++ namespaces
- C++ templates
- C++ Smart Pointers
  - The shared_ptr Smart Pointer
  - Generic Smart Pointers
- C++ reference counted objects

- Further details on the Python class interface
  - Proxy classes
  - Built-in Types
    - Limitations
    - Operator overloads and slots -- use them!
  - Memory management
- Cross language polymorphism
  - Enabling directors
  - Director classes
  - Ownership and object destruction
  - Exception unrolling
  - Overhead and code bloat
  - Typemaps
  - Thread safety
  - Miscellaneous
- Common customization features
  - C/C++ helper functions
  - Adding additional Python code
  - Class extension with %extend
  - Exception handling with %exception
  - Optimization options
    - -fastproxy
  - Stable ABI
- Tips and techniques
  - Input and output parameters
  - Simple pointers
  - Unbounded C Arrays
  - String handling
  - Default arguments
- Typemaps
  - What is a typemap?
  - Python typemaps
  - Typemap variables
  - Useful Python Functions
- Typemap Examples
  - Converting a Python list to a char **
  - Expanding a Python object into multiple arguments
  - Using typemaps to return arguments
  - Mapping Python tuples into small arrays
  - Mapping sequences to C arrays
  - Pointer handling
  - Memory management when returning references to member variables
- Docstring Features
  - Module docstring
  - %feature("autodoc")
    - %feature("autodoc", "0")
    - %feature("autodoc", "1")
    - %feature("autodoc", "2")
    - %feature("autodoc", "3")
    - %feature("autodoc", "docstring")
  - %feature("docstring")
  - Doxygen comments
- Python Packages
  - Setting the Python package
  - Absolute and relative imports
  - Enforcing absolute import semantics
  - Importing from __init__.py
  - Implicit namespace packages
  - Location of modules
    - Both modules in the same package
    - Both modules are global
    - Split modules custom configuration
    - More on customizing the module import code
    - Statically linked C modules
- Python 3 Support
  - Python function annotations and variable annotations
    - C/C++ annotation types
  - Buffer interface
  - Abstract base classes
  - Byte string output conversion
  - Python 2 Unicode
- Support for Multithreaded Applications
  - UI for Enabling Multithreading Support
  - Multithread Performance


This chapter describes SWIG's support of Python. SWIG is compatible with all recent Python versions (Python 2.7 and Python >= 3.5). Python 3.3 and 3.4 may still work but we are no longer regularly testing with them. SWIG 4.0.x supported Python 3.2. SWIG 3.0.x supported older Python 2.x and 3.x.

This chapter covers most SWIG features, but certain low-level details are covered in less depth than in earlier chapters. At the very least, make sure you read the "SWIG Basics" chapter.

## 33.1 Overview

To build Python extension modules, SWIG uses a layered approach in which parts of the extension module are defined in C and other parts are defined in Python. The C layer contains low-level wrappers whereas Python code is used to define high-level features.

This layered approach recognizes the fact that certain aspects of extension building are better accomplished in each language (instead of trying to do everything in C or C++). Furthermore, by generating code in both languages, you get a lot more flexibility since you can enhance the extension module with support code in either language.

In describing the Python interface, this chapter starts by covering the basics of configuration, compiling, and installing Python modules. Next, the Python interface to common C and C++ programming features is described. Advanced customization features such as typemaps are then described followed by a discussion of low-level implementation details.

## 33.2 Preliminaries

### 33.2.1 Running SWIG

Suppose that you defined a SWIG module such as the following:

```
/* File: example.i */
%module example

%{
#include "example.h"
%}

int fact(int n);
```

This `.i` file wraps the following simple C file:

```
/* File: example.c */

#include "example.h"

int fact(int n) {
  if (n < 0) { /* This should probably return an error, but this is simpler */
    return 0;
  }
  if (n == 0) {
    return 1;
  } else {
    /* testing for overflow would be a good idea here */
    return n * fact(n-1);
  }
}
```

With the header file:

```
/* File: example.h */

int fact(int n);
```

To build a Python module, run SWIG using the `-python` option:

```
$ swig -python example.i
```

If building a C++ extension, add the `-c++` option:

```
$ swig -c++ -python example.i
```

This creates two different files; a C/C++ source file `example_wrap.c` or `example_wrap.cxx` and a Python source file `example.py`. The generated C source file contains the low-level wrappers that need to be compiled and linked with the rest of your C/C++ application to create an extension module. The Python source file contains high-level support code. This is the file that you will import to use the module.

The name of the wrapper file is derived from the name of the input file. For example, if the input file is `example.i`, the name of the wrapper file is `example_wrap.c`. To change this, you can use the `-o` option. The name of the Python file is derived from the module name specified with `%module`. If the module name is `example`, then a file `example.py` is created.

The following sections have further practical examples and details on how you might go about compiling and using the generated files.

### 33.2.2 Using distutils

The preferred approach to building an extension module for Python is to compile it with distutils, which comes with all recent versions of Python ( Distutils Docs).

Distutils takes care of making sure that your extension is built with all the correct flags, headers, etc. for the version of Python it is run with. Distutils will compile your extension into a shared object file or DLL (`.so` on Linux, `.pyd` on Windows, etc). In addition, distutils can handle installing your package into site-packages, if that is desired. A configuration file (conventionally called: `setup.py`) describes the extension (and related Python modules). The distutils will then generate all the right compiler directives to build it for you.

Here is a sample `setup.py` file for the above example:

```
#!/usr/bin/env python3

"""
setup.py file for SWIG example
"""

from distutils.core import setup, Extension


example_module = Extension('_example',
                           sources=['example_wrap.c', 'example.c'],
                           )

setup (name = 'example',
       version = '0.1',
       author      = "SWIG Docs",
       description = """Simple swig example from docs""",
       ext_modules = [example_module],
       py_modules = ["example"],
```

```
        )
```

In this example, the line: `example_module = Extension(....)` creates an Extension module object, defining the name as `_example` , and using the source code files: `example_wrap.c`, generated by swig, and `example.c`, your original c source. The swig (and other Python extension modules) tradition is for the compiled extension to have the name of the Python portion, prefixed by an underscore. If the name of your Python module is "`example.py`", then the name of the corresponding object file will be"`_example.so`"

The `setup` call then sets up distutils to build your package, defining some meta data, and passing in your extension module object. Once this is saved as `setup.py`, you can build your extension with these commands:

```
$ swig -python example.i
$ python setup.py build_ext --inplace
```

And a .so, or .pyd or... will be created for you. It will build a version that matches the Python that you run the command with. Taking apart the command line:

- `python` -- the version of Python you want to build for
- `setup.py` -- the name of your setup script (it can be called anything, but setup.py is the tradition)
- `build_ext` -- telling distutils to build extensions
- `--inplace` -- this tells distutils to put the extension lib in the current dir. Otherwise, it will put it inside a build hierarchy, and you'd have to move it to use it.

The distutils have many other features, consult the Python distutils docs for details.

This same approach works on all platforms if the appropriate compiler is installed. (it can even build extensions to the standard Windows Python using MingGW)

### 33.2.3 Hand compiling a dynamic module

While the preferred approach to building an extension module is to use the distutils, some people like to integrate building extensions with a larger build system, and thus may wish to compile their modules without the distutils. To do this, you need to compile your program using commands like this (shown for Linux):

```
$ swig -python example.i
$ gcc -O2 -fPIC -c example.c
$ gcc -O2 -fPIC -c example_wrap.c -I/usr/local/include/python2.5
$ gcc -shared example.o example_wrap.o -o _example.so
```

The exact commands for doing this vary from platform to platform. However, SWIG tries to guess the right options when it is installed. Therefore, you may want to start with one of the examples in the `SWIG/Examples/python` directory. If that doesn't work, you will need to read the man-pages for your compiler and linker to get the right set of options. You might also check the SWIG Wiki for additional information.

When linking the module, **the name of the output file has to match the name of the module prefixed by an underscore**. If the name of your module is "`example`", then the name of the corresponding object file should be "`_example.so`" or "`_examplemodule.so` ". The name of the module is specified using the`%module` directive or the `-module` command line option.

**Compatibility Note:** In SWIG-1.3.13 and earlier releases, module names did not include the leading underscore. This is because modules were normally created as C-only extensions without the extra Python support file (instead, creating Python code was supported as an optional feature). This has been changed in SWIG-1.3.14 and is consistent with other Python extension modules. For example, the `socket` module actually consists of two files;`socket.py` and `_socket.so`. Many other built-in Python modules follow a similar convention.

### 33.2.4 Static linking

An alternative approach to dynamic linking is to rebuild the Python interpreter with your extension module added to it. In the past, this approach was sometimes necessary due to limitations in dynamic loading support on certain machines. However, the situation has improved greatly over the last few years and you should not consider this approach unless there is really no other option.

The usual procedure for adding a new module to Python involves finding the Python source, adding an entry to the `Modules/Setup` file, and rebuilding the interpreter using the Python Makefile. However, newer Python versions have changed the build process. You may need to edit the 'setup.py' file in the Python distribution instead.

In earlier versions of SWIG, the `embed.i` library file could be used to rebuild the interpreter. For example:

```
%module example

%inline %{
extern int fact(int);
extern int mod(int, int);
extern double My_variable;
%}

%include "embed.i"       // Include code for a static version of Python
```

The `embed.i` library file includes supporting code that contains everything needed to rebuild Python. To rebuild the interpreter, you simply do something like this:

```
$ swig -python -lembed.i example.i
$ gcc example.c example_wrap.c \
        -Xlinker -export-dynamic \
        -DHAVE_CONFIG_H -I/usr/include/python2.7 \
        -I/usr/lib/python2.7/config-x86_64-linux-gnu \
        -I/usr/lib/python2.7/config \
        -L/usr/lib/python2.7/config -lpython2.7 -lm -ldl \
        -o mypython
```

You will need to supply the same libraries that were used to build Python the first time. This may include system libraries such as `-lsocket`, `-lnsl`, and `-lpthread`. Assuming this actually works, the new version of Python should be identical to the default version except that your extension module will be a built-in part of the interpreter.

**Comment:** In practice, you should probably try to avoid static linking if possible. Some programmers may be inclined to use static linking in the interest of getting better performance. However, the performance gained by static linking tends to be rather minimal in most situations (and quite frankly not worth the extra hassle in the opinion of this author).

**Compatibility note:** The `embed.i` library file is deprecated and has not been actively maintained for many years. Even though it appears to "work" with Python 2.7, no future support is guaranteed. If using static linking, you might want to rely on a different approach (perhaps using distutils).

### 33.2.5 Using your module

To use your module, simply use the Python `import` statement. If all goes well, you will be able to run this:

```
$ python
>>> import example
>>> example.fact(4)
24
```

```
>>>
```

A common error received by first-time users is the following:

```
>>> import example
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
  File "example.py", line 2, in ?
    import _example
ImportError: No module named _example
```

If you get this message, it means that you either forgot to compile the wrapper code into an extension module or you didn't give the extension module the right name. Make sure that you compiled the wrappers into a module called `_example.so`. And don't forget the leading underscore (_).

Another possible error is the following:

```
>>> import example
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
ImportError: dynamic module does not define init function (init_example)
>>>
```

This error is almost always caused when a bad name is given to the shared object file. For example, if you created a file `example.so` instead of `_example.so` you would get this error. Alternatively, this error could arise if the name of the module is inconsistent with the module name supplied with the `%module` directive. Double-check the interface to make sure the module name and the shared object filename match. Another possible cause of this error is forgetting to link the SWIG-generated wrapper code with the rest of your application when creating the extension module.

Another common error is something similar to the following:

```
Traceback (most recent call last):
  File "example.py", line 3, in ?
    import example
ImportError: ./_example.so: undefined symbol: fact
```

This error usually indicates that you forgot to include some object files or libraries in the linking of the shared library file. Make sure you compile both the SWIG wrapper file and your original program into a shared library file. Make sure you pass all of the required libraries to the linker.

Sometimes unresolved symbols occur because a wrapper has been created for a function that doesn't actually exist in a library. This usually occurs when a header file includes a declaration for a function that was never actually implemented or it was removed from a library without updating the header file. To fix this, you can either edit the SWIG input file to remove the offending declaration or you can use the `%ignore` directive to ignore the declaration.

Finally, suppose that your extension module is linked with another library like this:

```
$ gcc -shared example.o example_wrap.o -L/home/beazley/projects/lib -lfoo \
      -o _example.so
```

If the `foo` library is compiled as a shared library, you might encounter the following problem when you try to use your module:

```
>>> import example
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
ImportError: libfoo.so: cannot open shared object file: No such file or directory
>>>
```

This error is generated because the dynamic linker can't locate the `libfoo.so` library. When shared libraries are loaded, the system normally only checks a few standard locations such as `/usr/lib` and `/usr/local/lib`. To fix this problem, there are several things you can do. First, you can recompile your extension module with extra path information. For example, on Linux you can do this:

```
$ gcc -shared example.o example_wrap.o -L/home/beazley/projects/lib -lfoo \
      -Xlinker -rpath /home/beazley/projects/lib  \
      -o _example.so
```

Alternatively, you can set the `LD_LIBRARY_PATH` environment variable to include the directory with your shared libraries. If setting `LD_LIBRARY_PATH`, be aware that setting this variable can introduce a noticeable performance impact on all other applications that you run. To set it only for Python, you might want to do this instead:

```
$ env LD_LIBRARY_PATH=/home/beazley/projects/lib python
```

Finally, you can use a command such as `ldconfig` (Linux) or `crle` (Solaris) to add additional search paths to the default system configuration (this requires root access and you will need to read the man pages).

### 33.2.6 Compilation of C++ extensions

Compilation of C++ extensions has traditionally been a tricky problem. Since the Python interpreter is written in C, you need to take steps to make sure C++ is properly initialized and that modules are compiled correctly. This should be a non-issue if you're using distutils, as it takes care of all that for you. The following is included for historical reasons, and in case you need to compile on your own.

On most machines, C++ extension modules should be linked using the C++ compiler. For example:

```
$ swig -c++ -python example.i
$ g++ -O2 -fPIC -c example.cxx
$ g++ -O2 -fPIC -c example_wrap.cxx -I/usr/local/include/python2.5
$ g++ -shared example.o example_wrap.o -o _example.so
```

The -fPIC option tells GCC to generate position-independent code (PIC) which is required for most architectures (it's not vital on x86, but still a good idea as it allows code pages from the library to be shared between processes). Other compilers may need a different option specified instead of -fPIC.

In addition to this, you may need to include additional library files to make it work. For example, if you are using the Sun C++ compiler on Solaris, you often need to add an extra library `-lCrun` like this:

```
$ swig -c++ -python example.i
$ CC -c example.cxx
$ CC -c example_wrap.cxx -I/usr/local/include/python2.5
$ CC -G example.o example_wrap.o -L/opt/SUNWspro/lib -o _example.so -lCrun
```

Of course, the extra libraries to use are completely non-portable---you will probably need to do some experimentation.

Sometimes people have suggested that it is necessary to relink the Python interpreter using the C++ compiler to make C++ extension modules work. In the experience of this author, this has never actually appeared to be necessary. Relinking the interpreter with C++ really only includes the special run-time libraries described above---as long as you link your extension modules with these libraries, it should not be necessary to rebuild Python.

If you aren't entirely sure about the linking of a C++ extension, you might look at an existing C++ program. On many Unix machines, the `ldd` command will list library dependencies. This should give you some clues about what you might have to include when you link your extension module. For example:

```
$ ldd swig
        libstdc++-libc6.1-1.so.2 => /usr/lib/libstdc++-libc6.1-1.so.2 (0x40019000)
        libm.so.6 => /lib/libm.so.6 (0x4005b000)
        libc.so.6 => /lib/libc.so.6 (0x40077000)
        /lib/ld-linux.so.2 => /lib/ld-linux.so.2 (0x40000000)
```

As a final complication, a major weakness of C++ is that it does not define any sort of standard for binary linking of libraries. This means that C++ code compiled by different compilers will not link together properly as libraries nor is the memory layout of classes and data structures implemented in any kind of portable manner. In a monolithic C++ program, this problem may be unnoticed. However, in Python, it is possible for different extension modules to be compiled with different C++ compilers. As long as these modules are self-contained, this probably won't matter. However, if these modules start sharing data, you will need to take steps to avoid segmentation faults and other erratic program behavior. If working with lots of software components, you might want to investigate using a more formal standard such as COM.

### 33.2.7 Compiling for 64-bit platforms

On platforms that support 64-bit applications (Solaris, Irix, etc.), special care is required when building extension modules. On these machines, 64-bit applications are compiled and linked using a different set of compiler/linker options. In addition, it is not generally possible to mix 32-bit and 64-bit code together in the same application.

To utilize 64-bits, the Python executable will need to be recompiled as a 64-bit application. In addition, all libraries, wrapper code, and every other part of your application will need to be compiled for 64-bits. If you plan to use other third-party extension modules, they will also have to be recompiled as 64-bit extensions.

If you are wrapping commercial software for which you have no source code, you will be forced to use the same linking standard as used by that software. This may prevent the use of 64-bit extensions. It may also introduce problems on platforms that support more than one linking standard (e.g., -o32 and -n32 on Irix).

On the Linux x86_64 platform (Opteron or EM64T), besides of the required compiler option -fPIC discussed above, you will need to be careful about the libraries you link with or the library path you use. In general, a Linux distribution will have two set of libraries, one for native x86_64 programs (under /usr/lib64), and another for 32 bits compatibility (under /usr/lib). Also, the compiler options -m32 and -m64 allow you to choose the desired binary format for your Python extension.

### 33.2.8 Building Python extensions under Windows

Building a SWIG extension to Python under Windows is roughly similar to the process used with Unix. Using the distutils, it is essentially identical. If you have the same version of the MS compiler that Python was built with (the python2.4 and python2.5 distributed by python.org are built with Visual Studio 2003), the standard `python setup.py build` should just work.

As of python2.5, the distutils support building extensions with MingGW out of the box. Following the instruction here: Building Python extensions for Windows with only free tools should get you started.

If you need to build it on your own, the following notes are provided:

You will need to create a DLL that can be loaded into the interpreter. This section briefly describes the use of SWIG with Microsoft Visual C++. As a starting point, many of SWIG's examples include project files (.dsp files) for Visual C++ 6. These can be opened by more recent versions of Visual Studio. You might want to take a quick look at these examples in addition to reading this section.

In Developer Studio, SWIG should be invoked as a custom build option. This is usually done as follows:

- Open up a new workspace and use the AppWizard to select a DLL project.
- Add both the SWIG interface file (the .i file), any supporting C files, and the name of the wrapper file that will be created by SWIG (ie. `example_wrap.c`). Note : If using C++, choose a different suffix for the wrapper file such as `example_wrap.cxx`. Don't worry if the wrapper file doesn't exist yet--Developer Studio keeps a reference to it.
- Select the SWIG interface file and go to the settings menu. Under settings, select the "Custom Build" option.
- Enter "SWIG" in the description field.
- Enter "`swig -python -o $(ProjDir)\$(InputName)_wrap.c $(InputPath)`" in the "Build command(s) field"
- Enter "`$(ProjDir)\$(InputName)_wrap.c`" in the "Output files(s) field".
- Next, select the settings for the entire project and go to "C++:Preprocessor". Add the include directories for your Python installation under "Additional include directories".
- Define the symbol __WIN32__ under preprocessor options.
- Finally, select the settings for the entire project and go to "Link Options". Add the Python library file to your link libraries. For example "python27.lib". Also, set the name of the output file to match the name of your Python module, i.e. `_example.pyd`
- Build your project.

If all went well, SWIG will be automatically invoked whenever you build your project. Any changes made to the interface file will result in SWIG being automatically executed to produce a new version of the wrapper file.

To run your new Python extension, simply run Python and use the `import` command as normal. For example :

```
$ python
>>> import example
>>> print(example.fact(4))
24
>>>
```

If you get an `ImportError` exception when importing the module, you may have forgotten to include additional library files when you built your module. If you get an access violation or some kind of general protection fault immediately upon import, you have a more serious problem. This is often caused by linking your extension module against the wrong set of Win32 debug or thread libraries. You will have to fiddle around with the build options of project to try and track this down.

A 'Debug' build of the wrappers requires a debug build of the Python interpreter. This normally requires building the Python interpreter from source, which is not a job for the feint-hearted. Alternatively you can use the 'Release' build of the Python interpreter with a 'Debug' build of your wrappers by defining the `SWIG_PYTHON_INTERPRETER_NO_DEBUG` symbol under the preprocessor options. Or you can ensure this macro is defined at the beginning of the wrapper code using the following in your interface file, where `_MSC_VER` ensures it is only used by the Visual Studio compiler:

```
%begin %{
#ifdef _MSC_VER
#define SWIG_PYTHON_INTERPRETER_NO_DEBUG
#endif
%}
```

Some users have reported success in building extension modules using Cygwin and other compilers. However, the problem of building usable DLLs with these compilers tends to be rather problematic. For the latest information, you may want to consult the SWIG Wiki.

### 33.2.9 Additional Python commandline options

The following table lists the additional commandline options available for the Python module. They can also be seen by using:

```
swig -python -help
```

| Python specific options | |
|---|---|
| -builtin | Create Python built-in types rather than proxy classes, for better performance |
| -castmode | Enable the casting mode, which allows implicit cast between types in Python |
| -debug-doxygen-parser | Display doxygen parser module debugging information |
| -debug-doxygen-translator | Display doxygen translator module debugging information |
| -dirvtable | Generate a pseudo virtual table for directors for faster dispatch (warning: currently the vtable is per-object and never freed so this option causes memory leaks) |
| -doxygen | Convert C++ doxygen comments to pydoc comments in proxy classes |
| -extranative | Return extra native wrappers for C++ std containers wherever possible |
| -fastproxy | Use fast proxy mechanism for member methods |
| -flatstaticmethod | Generate additional flattened Python methods for C++ static methods |
| -globals <name> | Set <name> used to access C global variable (default: 'cvar') |
| -interface <mod> | Set low-level C/C++ module name to <mod> (default: module name prefixed by '_') |
| -keyword | Use keyword arguments |
| -nofastunpack | Use traditional UnpackTuple method to parse the argument functions |
| -noh | Don't generate the output header file |
| -noproxy | Don't generate proxy classes |
| -nortti | Disable the use of the native C++ RTTI with directors |
| -nothreads | Disable thread support for the entire interface |
| -olddefs | Keep the old method definitions when using -fastproxy |
| -relativeimport | Use relative Python imports |
| -threads | Add thread support for all the interface |
| -O | Enable the following optimization options: -fastdispatch -fastproxy -fvirtual |

Many of these options are covered later on and their use should become clearer by the time you have finished reading this section on SWIG and Python.

## 33.3 A tour of basic C/C++ wrapping

By default, SWIG tries to build a very natural Python interface to your C/C++ code. Functions are wrapped as functions, classes are wrapped as classes, and so forth. This section briefly covers the essential aspects of this wrapping.

### 33.3.1 Modules

The SWIG `%module` directive specifies the name of the Python module. If you specify `` `%module example` '', then everything is wrapped into a Python '`example`' module. Underneath the covers, this module consists of a Python source file `example.py` and a low-level extension module `_example.so`. When choosing a module name, make sure you don't use the same name as a built-in Python command or standard module name.

### 33.3.2 Functions

Global functions are wrapped as new Python built-in functions. For example,

```
%module example
int fact(int n);
```

creates a built-in function `example.fact(n)` that works exactly like you think it does:

```
>>> import example
>>> print(example.fact(4))
24
>>>
```

### 33.3.3 Global variables

C/C++ global variables are fully supported by SWIG. However, the underlying mechanism is somewhat different than you might expect due to the way that Python assignment works. When you type the following in Python

```
a = 3.4
```

"a" becomes a name for an object containing the value 3.4. If you later type

```
b = a
```

then "a" and "b" are both names for the object containing the value 3.4. Thus, there is only one object containing 3.4 and "a" and "b" are both names that refer to it. This is quite different than C where a variable name refers to a memory location in which a value is stored (and assignment copies data into that location). Because of this, there is no direct way to map variable assignment in C to variable assignment in Python.

To provide access to C global variables, SWIG creates a special object called `` `cvar` '' that is added to each SWIG generated module. Global variables are then accessed as attributes of this object. For example, consider this interface

```
// SWIG interface file with global variables
%module example
...
%inline %{
extern int My_variable;
extern double density;
%}
...
```

Now look at the Python interface:

```
>>> import example
>>> # Print out value of a C global variable
>>> print(example.cvar.My_variable)
4
>>> # Set the value of a C global variable
>>> example.cvar.density = 0.8442
>>> # Use in a math operation
>>> example.cvar.density = example.cvar.density*1.10
```

If you make an error in variable assignment, you will receive an error message. For example:

```
>>> example.cvar.density = "Hello"
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
TypeError: C variable 'density (double )'
>>>
```

If a variable is declared as `const`, it is wrapped as a read-only variable. Attempts to modify its value will result in an error.

To make ordinary variables read-only, you can use the `%immutable` directive. For example:

```
%{
extern char *path;
%}
%immutable;
extern char *path;
%mutable;
```

The `%immutable` directive stays in effect until it is explicitly disabled or cleared using `%mutable`. See the Creating read-only variables section for further details.

If you just want to make a specific variable immutable, supply a declaration name. For example:

```
%{
extern char *path;
%}
%immutable path;
...
extern char *path;      // Read-only (due to %immutable)
```

If you would like to access variables using a name other than "`cvar`", it can be changed using the `-globals` option :

```
$ swig -python -globals myvar example.i
```

Some care is in order when importing multiple SWIG modules. If you use the "`from <file> import *`" style of importing, you will get a name clash on the variable `` `cvar' `` and you will only be able to access global variables from the last module loaded. To prevent this, you might consider renaming `cvar` or making it private to the module by giving it a name that starts with a leading underscore. SWIG does not create `cvar` if there are no global variables in a module.

### 33.3.4 Constants and enums

C/C++ constants are installed as Python objects containing the appropriate value. To create a constant, use `#define`, `enum` , or the `%constant` directive. For example:

```
#define PI 3.14159
#define VERSION "1.0"

enum Beverage { ALE, LAGER, STOUT, PILSNER };

%constant int FOO = 42;
%constant const char *path = "/usr/local";
```

For enums, make sure that the definition of the enumeration actually appears in a header file or in the wrapper file somehow---if you just stick an enum in a SWIG interface without also telling the C compiler about it, the wrapper code won't compile.

Note: declarations declared as `const` are wrapped as read-only variables and will be accessed using the `cvar` object described in the previous section. They are not wrapped as constants. For further discussion about this, see the SWIG Basics chapter.

Constants are not guaranteed to remain constant in Python---the name of the constant could be accidentally reassigned to refer to some other object. Unfortunately, there is no easy way for SWIG to generate code that prevents this. You will just have to be careful.

### 33.3.5 Pointers

C/C++ pointers are fully supported by SWIG. Furthermore, SWIG has no problem working with incomplete type information. Here is a rather simple interface:

```
%module example

FILE *fopen(const char *filename, const char *mode);
int fputs(const char *, FILE *);
int fclose(FILE *);
```

When wrapped, you will be able to use the functions in a natural way from Python. For example:

```
>>> import example
>>> f = example.fopen("junk", "w")
>>> example.fputs("Hello World\n", f)
>>> example.fclose(f)
```

If this makes you uneasy, rest assured that there is no deep magic involved. Underneath the covers, pointers to C/C++ objects are simply represented as opaque values using an especial

Python container object:

```
>>> print(f)
<Swig Object of type 'FILE *' at 0xb7d6f470>
```

This pointer value can be freely passed around to different C functions that expect to receive an object of type `FILE *`. The only thing you can't do is dereference the pointer from Python. Of course, that isn't much of a concern in this example.

In older versions of SWIG (1.3.22 or older), pointers were represented using a plain string object. If you have an old package that still requires that representation, or you just feel nostalgic, you can always retrieve it by casting the pointer object to a string:

```
>>> print(str(f))
_c0671108_p_FILE
```

Also, if you need to pass the raw pointer value to some external Python library, you can do it by casting the pointer object to an integer:

```
>>> print(int(f))
135833352
```

However, the inverse operation is not possible, i.e., you can't build a SWIG pointer object from a raw integer value.

Note also that the '0' or NULL pointer is always represented by `None`, no matter what type swig is addressing. In the previous example, you can call:

```
>>> example.fclose(None)
```

and that will be equivalent to the following, but not really useful, C code:

```
FILE *f = NULL;
fclose(f);
```

As much as you might be inclined to modify a pointer value directly from Python, don't. The hexadecimal encoding is not necessarily the same as the logical memory address of the underlying object. Instead it is the raw byte encoding of the pointer value. The encoding will vary depending on the native byte-ordering of the platform (i.e., big-endian vs. little-endian). Similarly, don't try to manually cast a pointer to a new type by simply replacing the type-string. This may not work like you expect, it is particularly dangerous when casting C++ objects. If you need to cast a pointer or change its value, consider writing some helper functions instead. For example:

```
%inline %{
/* C-style cast */
Bar *FooToBar(Foo *f) {
  return (Bar *) f;
}

/* C++-style cast */
Foo *BarToFoo(Bar *b) {
  return dynamic_cast<Foo*>(b);
}

Foo *IncrFoo(Foo *f, int i) {
  return f+i;
}
%}
```

Also, if working with C++, you should always try to use the new C++ style casts. For example, in the above code, the C-style cast may return a bogus result whereas as the C++-style cast will return `None` if the conversion can't be performed.

### 33.3.6 Structures

If you wrap a C structure, it is wrapped by a Python class. This provides a very natural interface. For example,

```
struct Vector {
  double x, y, z;
};
```

is used as follows:

```
>>> v = example.Vector()
>>> v.x = 3.5
>>> v.y = 7.2
>>> print(v.x, v.y, v.z)
3.5 7.2 0.0
>>>
```

Similar access is provided for unions and the data members of C++ classes.

If you print out the value of `v` in the above example, you will see something like this:

```
>>> print(v)
<C Vector instance at _18e31408_p_Vector>
```

This object is actually a Python instance that has been wrapped around a pointer to the low-level C structure. This instance doesn't actually do anything--it just serves as a proxy. The pointer to the C object can be found in the `.this` attribute. For example:

```
>>> print(v.this)
_18e31408_p_Vector
>>>
```

Further details about the Python proxy class are covered a little later.

`const` members of a structure are read-only. Data members can also be forced to be read-only using the `%immutable` directive. For example:

```
struct Foo {
  ...
  %immutable;
  int x;        /* Read-only members */
  char *name;
  %mutable;
  ...
};
```

When `char *` members of a structure are wrapped, the contents are assumed to be dynamically allocated using `malloc` or `new` (depending on whether or not SWIG is run with the `-c++` option). When the structure member is set, the old contents will be released and a new value created. If this is not the behavior you want, you will have to use a typemap (described later).

If a structure contains arrays, access to those arrays is managed through pointers. For example, consider this:

```
struct Bar {
  int  x[16];
};
```

If accessed in Python, you will see behavior like this:

```
>>> b = example.Bar()
>>> print(b.x)
_801861a4_p_int
>>>
```

This pointer can be passed around to functions that expect to receive an `int *` (just like C). You can also set the value of an array member using another pointer. For example:

```
>>> c = example.Bar()
>>> c.x = b.x             # Copy contents of b.x to c.x
```

For array assignment, SWIG copies the entire contents of the array starting with the data pointed to by `b.x`. In this example, 16 integers would be copied. Like C, SWIG makes no assumptions about bounds checking---if you pass a bad pointer, you may get a segmentation fault or access violation.

When a member of a structure is itself a structure, it is handled as a pointer. For example, suppose you have two structures like this:

```
struct Foo {
  int a;
};

struct Bar {
  Foo f;
};
```

Now, suppose that you access the `f` attribute of `Bar` like this:

```
>>> b = Bar()
>>> x = b.f
```

In this case, `x` is a pointer that points to the `Foo` that is inside `b`. This is the same value as generated by this C code:

```
Bar b;
Foo *x = &b->f;       /* Points inside b */
```

Because the pointer points inside the structure, you can modify the contents and everything works just like you would expect. For example:

```
>>> b = Bar()
>>> b.f.a = 3               # Modify attribute of structure member
>>> x = b.f
>>> x.a = 3                 # Modifies the same structure
```

Note that there is a limitation with structs within structs that will cause a problem if the outer struct is not a named variable in Python. The following will cause a segfault:

```
Bar().f.a = 3
```

because the unnamed Python proxy class for `Bar()` has its reference count decremented by the Python interpreter after `f` has been obtained from it and before `f` is used to obtain `a` . This results in the underlying `Bar` instance being deleted, which of course also deletes `f` inside it. Hence the pointer to `f` points to deleted memory and use of it results in a segfault or some sort of other undefined behaviour.

### 33.3.7 C++ classes

C++ classes are wrapped by Python classes as well. For example, if you have this class,

```
class List {
public:
  List();
  ~List();
  int  search(char *item);
  void insert(char *item);
  void remove(char *item);
  char *get(int n);
  int  length;
};
```

you can use it in Python like this:

```
>>> l = example.List()
>>> l.insert("Ale")
>>> l.insert("Stout")
>>> l.insert("Lager")
>>> l.get(1)
'Stout'
>>> print(l.length)
3
>>>
```

Class data members are accessed in the same manner as C structures.

Static class members present a special problem for Python. Prior to Python-2.2, Python classes had no support for static methods and no version of Python supports static member variables in a manner that SWIG can utilize. Therefore, SWIG generates wrappers that try to work around some of these issues. To illustrate, suppose you have a class like this:

```
class Spam {
public:
  static void foo();
  static int bar;
};
```

In Python, the static member can be accessed in three different ways:

```
>>> s = example.Spam()
>>> s.foo()               # Spam::foo() via an instance
>>> example.Spam.foo()    # Spam::foo() using class method
>>> example.Spam_foo()    # Spam::foo() "flattened" name
```

The last technique is only available when using the `-flatstaticmethod` option. This option is not recommended, it is only available for backwards compatibility as ancient versions of Python did not have Python class methods.

Static member variables are currently accessed as global variables. This means, they are accessed through `cvar` or via an instance property:

```
>>> example.cvar.Spam_bar # Spam::bar
7
>>> s = example.Spam()
>>> s.bar                 # Spam::bar via an instance property
7
```

The `-builtin` option uses a metaclass to additionally provide access as follows:

```
>>> example.Spam.bar      # Spam::bar using -builtin option only
7
```

### 33.3.8 C++ inheritance

SWIG is fully aware of issues related to C++ inheritance. Therefore, if you have classes like this

```
class Foo {
...
};

class Bar : public Foo {
...
};
```

those classes are wrapped into a hierarchy of Python classes that reflect the same inheritance structure. All of the usual Python utility functions work normally:

```
>>> b = Bar()
>>> instance(b, Foo)
1
>>> issubclass(Bar, Foo)
1
>>> issubclass(Foo, Bar)
0
```

Furthermore, if you have functions like this

```
void spam(Foo *f);
```

then the function `spam()` accepts `Foo *` or a pointer to any class derived from `Foo`.

It is safe to use multiple inheritance with SWIG.

### 33.3.9 Pointers, references, values, and arrays

In C++, there are many different ways a function might receive and manipulate objects. For example:

```
void spam1(Foo *x);      // Pass by pointer
void spam2(Foo &x);      // Pass by reference
void spam3(const Foo &x);// Pass by const reference
void spam4(Foo x);       // Pass by value
void spam5(Foo x[]);     // Array of objects
```

In Python, there is no detailed distinction like this--specifically, there are only "objects". There are no pointers, references, arrays, and so forth. Because of this, SWIG unifies all of these types together in the wrapper code. For instance, if you actually had the above functions, it is perfectly legal to do this:

```
>>> f = Foo()           # Create a Foo
>>> spam1(f)            # Ok. Pointer
>>> spam2(f)            # Ok. Reference
>>> spam3(f)            # Ok. Const reference
>>> spam4(f)            # Ok. Value.
>>> spam5(f)            # Ok. Array (1 element)
```

Similar behavior occurs for return values. For example, if you had functions like this,

```
Foo *spam6();
Foo &spam7();
Foo  spam8();
const Foo &spam9();
```

then all three functions will return a pointer to some `Foo` object. Since the third function (spam8) returns a value, newly allocated memory is used to hold the result and a pointer is returned (Python will release this memory when the return value is garbage collected). The fourth case (spam9) which returns a const reference, in most of the cases will be treated as a returning value, and it will follow the same allocation/deallocation process.

### 33.3.10 C++ overloaded functions

C++ overloaded functions, methods, and constructors are mostly supported by SWIG. For example, if you have two functions like this:

```
void foo(int);
void foo(char *c);
```

You can use them in Python in a straightforward manner:

```
>>> foo(3)           # foo(int)
>>> foo("Hello")     # foo(char *c)
```

Similarly, if you have a class like this,

```
class Foo {
public:
  Foo();
  Foo(const Foo &);
  ...
};
```

you can write Python code like this:

```
>>> f = Foo()           # Create a Foo
>>> g = Foo(f)          # Copy f
```

Overloading support is not quite as flexible as in C++. Sometimes there are methods that SWIG can't disambiguate. For example:

```
void spam(int);
void spam(short);
```

or

```
void foo(Bar *b);
void foo(Bar &b);
```

If declarations such as these appear, you will get a warning message like this:

```
example.i:12: Warning 509: Overloaded method spam(short) effectively ignored,
example.i:11: Warning 509: as it is shadowed by spam(int).
```

To fix this, you either need to ignore or rename one of the methods. For example:

```
%rename(spam_short) spam(short);
...
void spam(int);
void spam(short);   // Accessed as spam_short
```

or

```
%ignore spam(short);
...
void spam(int);
void spam(short);   // Ignored
```

SWIG resolves overloaded functions and methods using a disambiguation scheme that ranks and sorts declarations according to a set of type-precedence rules. The order in which declarations appear in the input does not matter except in situations where ambiguity arises--in this case, the first declaration takes precedence.

Please refer to the "SWIG and C++" chapter for more information about overloading.

### 33.3.11 C++ operators

Certain C++ overloaded operators can be handled automatically by SWIG. For example, consider a class like this:

```
class Complex {
private:
  double rpart, ipart;
public:
  Complex(double r = 0, double i = 0) : rpart(r), ipart(i) { }
  Complex(const Complex &c) : rpart(c.rpart), ipart(c.ipart) { }
  Complex &operator=(const Complex &c);

  Complex operator+=(const Complex &c) const;
  Complex operator+(const Complex &c) const;
  Complex operator-(const Complex &c) const;
  Complex operator*(const Complex &c) const;
  Complex operator-() const;

  double re() const { return rpart; }
  double im() const { return ipart; }
};
```

When wrapped, it works like you expect:

```
>>> c = Complex(3, 4)
>>> d = Complex(7, 8)
>>> e = c + d
>>> e.re()
10.0
>>> e.im()
12.0
>>> c += d
>>> c.re()
10.0
>>> c.im()
12.0
```

One restriction with operator overloading support is that SWIG is not able to fully handle operators that aren't defined as part of the class. For example, if you had code like this

```
class Complex {
...
friend Complex operator+(double, const Complex &c);
...
};
```

then SWIG ignores it and issues a warning. You can still wrap the operator, but you may have to encapsulate it in a special function. For example:

```
%rename(Complex_add_dc) operator+(double, const Complex &);
```

There are ways to make this operator appear as part of the class using the `%extend` directive. Keep reading.

Also, be aware that certain operators don't map cleanly to Python. For instance, overloaded assignment operators don't map to Python semantics and will be ignored.

Operator overloading is implemented in the `pyopers.swg` library file. In particular overloaded operators are marked with the `python:maybecall` feature, also known as `%pythonmaybecall`. This feature forces SWIG to generate code that returns an instance of Python's `NotImplemented` instead of raising the usual `TypeError` exception when an incorrect type is passed to a SWIG wrapped method. This follows the guidelines in PEP 207 - Rich Comparisons and NotImplemented Python constant.

### 33.3.12 C++ namespaces

SWIG is aware of C++ namespaces, but namespace names do not appear in the module nor do namespaces result in a module that is broken up into submodules or packages. For example, if you have a file like this,

```
%module example

namespace foo {
  int fact(int n);
  struct Vector {
    double x, y, z;
  };
};
```

it works in Python as follows:

```
>>> import example
>>> example.fact(3)
6
>>> v = example.Vector()
>>> v.x = 3.4
>>> print(v.y)
0.0
>>>
```

If your program has more than one namespace, name conflicts (if any) can be resolved using `%rename` For example:

```
%rename(Bar_spam) Bar::spam;

namespace Foo {
  int spam();
}

namespace Bar {
  int spam();
```

```
}
```

If you have more than one namespace and your want to keep their symbols separate, consider wrapping them as separate SWIG modules. For example, make the module name the same as the namespace and create extension modules for each namespace separately. If your program utilizes thousands of small deeply nested namespaces each with identical symbol names, well, then you get what you deserve.

### 33.3.13 C++ templates

C++ templates don't present a huge problem for SWIG. However, in order to create wrappers, you have to tell SWIG to create wrappers for a particular template instantiation. To do this, you use the `%template` directive. For example:

```
%module example
%{
#include "pair.h"
%}

template<class T1, class T2>
struct pair {
  typedef T1 first_type;
  typedef T2 second_type;
  T1 first;
  T2 second;
  pair();
  pair(const T1&, const T2&);
 ~pair();
};

%template(pairii) pair<int, int>;
```

In Python:

```
>>> import example
>>> p = example.pairii(3, 4)
>>> p.first
3
>>> p.second
4
```

Obviously, there is more to template wrapping than shown in this example. More details can be found in the SWIG and C++ chapter. Some more complicated examples will appear later.

### 33.3.14 C++ Smart Pointers

#### 33.3.14.1 The shared_ptr Smart Pointer

The C++11 standard provides `std::shared_ptr` which was derived from the Boost implementation, `boost::shared_ptr`. Both of these are available for Python in the SWIG library and usage is outlined in the shared_ptr smart pointer library section.

#### 33.3.14.2 Generic Smart Pointers

In certain C++ programs, it is common to use classes that have been wrapped by so-called "smart pointers." Generally, this involves the use of a template class that implements `operator->()` like this:

```
template<class T> class SmartPtr {
  ...
  T *operator->();
  ...
}
```

Then, if you have a class like this,

```
class Foo {
public:
  int x;
  int bar();
};
```

A smart pointer would be used in C++ as follows:

```
SmartPtr<Foo> p = CreateFoo();   // Created somehow (not shown)
...
p->x = 3;                        // Foo::x
int y = p->bar();                // Foo::bar
```

To wrap this in Python, simply tell SWIG about the `SmartPtr` class and the low-level `Foo` object. Make sure you instantiate `SmartPtr` using `%template` if necessary. For example:

```
%module example
...
%template(SmartPtrFoo) SmartPtr<Foo>;
...
```

Now, in Python, everything should just "work":

```
>>> p = example.CreateFoo()          # Create a smart-pointer somehow
>>> p.x = 3                          # Foo::x
>>> p.bar()                          # Foo::bar
```

If you ever need to access the underlying pointer returned by `operator->()` itself, simply use the `__deref__()` method. For example:

```
>>> f = p.__deref__()     # Returns underlying Foo *
```

### 33.3.15 C++ reference counted objects

The C++ reference counted objects section contains Python examples of memory management using referencing counting.

## 33.4 Further details on the Python class interface

In the previous section, a high-level view of Python wrapping was presented. A key component of this wrapping is that structures and classes are wrapped by Python proxy classes. This provides a very natural Python interface and allows SWIG to support a number of advanced features such as operator overloading. However, a number of low-level details were omitted. This section provides a brief overview of how the proxy classes work.

**New in SWIG version 2.0.4:** The use of Python proxy classes has performance implications that may be unacceptable for a high-performance library. The new `-builtin` option instructs SWIG to forego the use of proxy classes, and instead create wrapped types as new built-in Python types. When this option is used, the following section ("Proxy classes") does not apply. Details on the use of the `-builtin` option are in the Built-in Types section.

### 33.4.1 Proxy classes

In the "SWIG basics" and "SWIG and C++" chapters, details of low-level structure and class wrapping are described. To summarize those chapters, if you have a class like this

```
class Foo {
public:
  int x;
  int spam(int);
  ...
```

then SWIG transforms it into a set of low-level procedural wrappers. For example:

```
Foo *new_Foo() {
  return new Foo();
}
void delete_Foo(Foo *f) {
  delete f;
}
int Foo_x_get(Foo *f) {
  return f->x;
}
void Foo_x_set(Foo *f, int value) {
  f->x = value;
}
int Foo_spam(Foo *f, int arg1) {
  return f->spam(arg1);
}
```

These wrappers can be found in the low-level extension module (e.g., `_example`).

Using these wrappers, SWIG generates a high-level Python proxy class (also known as a shadow class) like this (shown for Python 2.2):

```
import _example

class Foo(object):
    def __init__(self):
        self.this = _example.new_Foo()
        self.thisown = True
    def __del__(self):
        if self.thisown:
            _example.delete_Foo(self.this)
    def spam(self, arg1):
        return _example.Foo_spam(self.this, arg1)
    x = property(_example.Foo_x_get, _example.Foo_x_set)
```

This class merely holds a pointer to the underlying C++ object ( `.this`) and dispatches methods and member variable access to that object using the low-level accessor functions. From a user's point of view, it makes the class work normally:

```
>>> f = example.Foo()
>>> f.x = 3
>>> y = f.spam(5)
```

The fact that the class has been wrapped by a real Python class offers certain advantages. For instance, you can attach new Python methods to the class and you can even inherit from it (something not supported by Python built-in types until Python 2.2).

### 33.4.2 Built-in Types

The `-builtin` option provides a significant performance improvement in the wrapped code. To understand the difference between proxy classes and built-in types, let's take a look at what a wrapped object looks like under both circumstances.

When proxy classes are used, each wrapped object in Python is an instance of a pure Python class. As a reminder, here is what the `__init__` method looks like in a proxy class:

```
class Foo(object):
    def __init__(self):
        self.this = _example.new_Foo()
        self.thisown = True
```

When a `Foo` instance is created, the call to `_example.new_Foo()` creates a new C++ `Foo` instance; wraps that C++ instance inside an instance of a Python built-in type called `SwigPyObject`; and stores the `SwigPyObject` instance in the 'this' field of the Python Foo object. Did you get all that? So, the Python `Foo` object is composed of three parts:

- The Python `Foo` instance, which contains...
- ... an instance of `struct SwigPyObject`, which contains...
- ... a C++ `Foo` instance

When `-builtin` is used, the pure Python layer is stripped off. Each wrapped class is turned into a new Python built-in type which inherits from `SwigPyObject`, and `SwigPyObject`

instances are returned directly from the wrapped methods. For more information about Python built-in extensions, please refer to the Python documentation:

https://docs.python.org/3/extending/newtypes.html

#### 33.4.2.1 Limitations

Use of the `-builtin` option implies a couple of limitations:

- Some legacy syntax is no longer supported; in particular:
  - The functional interface is no longer exposed. For example, you may no longer call `Whizzo.new_CrunchyFrog()`. Instead, you must use `Whizzo.CrunchyFrog()`.
  - Static member variables are no longer accessed through the 'cvar' field (e.g., `Dances.cvar.FishSlap`). They are instead accessed in the idiomatic way (`Dances.FishSlap`).
- Wrapped types may not be raised as Python exceptions. Here's why: the Python internals expect that all sub-classes of Exception will have this struct layout:

```
typedef struct {
  PyObject_HEAD
  PyObject *dict;
  PyObject *args;
  PyObject *message;
} PyBaseExceptionObject;
```

  But swig-generated wrappers expect that all swig-wrapped classes will have this struct layout:

```
typedef struct {
  PyObject_HEAD
  void *ptr;
  swig_type_info *ty;
  int own;
  PyObject *next;
  PyObject *dict;
} SwigPyObject;
```

  There are workarounds for this. For example, if you wrap this class:

```
class MyException {
public:
  MyException (const char *msg_);
  ~MyException ();

  const char *what () const;

private:
  char *msg;
};
```

  ... you can define this Python class, which may be raised as an exception:

```
class MyPyException(Exception):
    def __init__(self, msg, *args):
        Exception.__init__(self, *args)
        self.myexc = MyException(msg)
    def what(self):
        return self.myexc.what()
```

- Reverse binary operators (e.g., `__radd__`) are not supported.

  To illustrate this point, if you have a wrapped class called `MyString`, and you want to use instances of `MyString` interchangeably with native Python strings, you can define an `'operator+ (const char*)'` method :

```
class MyString {
public:
  MyString (const char *init);
  MyString operator+ (const char *other) const;
  ...
};
```

  SWIG will automatically create an operator overload in Python that will allow this:

```
from MyModule import MyString

mystr = MyString("No one expects")
episode = mystr + " the Spanish Inquisition"
```

  This works because the first operand (`mystr`) defines a way to add a native string to itself. However, the following will **not** work:

```
from MyModule import MyString

mystr = MyString("Parrot")
episode = "Dead " + mystr
```

  The above code fails, because the first operand -- a native Python string -- doesn't know how to add an instance of `MyString` to itself.

- If you have multiple SWIG modules that share type information (more info), the `-builtin` option requires a bit of extra discipline to ensure that base classes are initialized before derived classes. Specifically:
  - There must be an unambiguous dependency graph for the modules.
  - Module dependencies must be explicitly stated with `%import` statements in the SWIG interface file.

As an example, suppose module `A` has this interface in `A.i` :

```
%module "A";

class Base {
...
};
```

If you want to wrap another module containing a class that inherits from `A`, this is how it would look :

```
%module "B";

%import "A.i"

class Derived : public Base {
...
};
```

The `import "A.i"` statement is required, because module `B` depends on module `A`.

As long as you obey these requirements, your Python code may import the modules in any order :

```
import B
import A

assert(issubclass(B.Derived, A.Base))
```

- Python annotations are not supported.

#### 33.4.2.2 Operator overloads and slots -- use them!

The entire justification for the `-builtin` option is improved performance. To that end, the best way to squeeze maximum performance out of your wrappers is to **use operator overloads.** Named method dispatch is slow in Python, even when compared to other scripting languages. However, Python built-in types have a large number of "slots", analogous to C++ operator overloads, which allow you to short-circuit named method dispatch for certain common operations.

By default, SWIG will translate most C++ arithmetic operator overloads into Python slot entries. For example, suppose you have this class:

```
class Twit {
public:
  Twit operator+ (const Twit& twit) const;

  // Forward to operator+
  Twit add (const Twit& twit) const {
    return *this + twit;
  }
};
```

SWIG will automatically register `operator+` as a Python slot operator for addition. You may write Python code like this:

```
from MyModule import Twit

nigel = Twit()
emily = Twit()
percival = nigel + emily
percival = nigel.add(emily)
```

The last two lines of the Python code are equivalent, but **the line that uses the '+' operator is much faster**.

In-place operators (e.g., `operator+=`) and comparison operators (`operator==, operator<`, etc.) are also converted to Python slot operators. For a complete list of C++ operators that are automatically converted to Python slot operators, refer to the file `python/pyopers.swg` in the SWIG library.

Read about all of the available Python slots here: https://docs.python.org/3/c-api/typeobj.html

There are two ways to define a Python slot function: dispatch to a statically defined function; or dispatch to a method defined on the operand.

To dispatch to a statically defined function, use %feature("python:<slot>"), where <slot> is the name of a field in a `PyTypeObject, PyNumberMethods, PyMappingMethods, PySequenceMethods` or `PyBufferProcs`. You may override (almost) all of these slots.

Let's consider an example setting the `tp_hash` slot for the `MyClass` type. This is akin to providing a `__hash__` method (for non-builtin types) to make a type hashable. The hashable type can then for example be added to a Python `dict`.

```
%feature("python:tp_hash") MyClass "myHashFunc";

class MyClass {
public:
  long field1;
  long field2;
  ...
};

%{
#if PY_VERSION_HEX >= 0x03020000
  static Py_hash_t myHashFunc(PyObject *pyobj)
#else
  static long myHashFunc(PyObject *pyobj)
#endif
  {
    MyClass *cobj;
    // Convert pyobj to cobj
    return (cobj->field1 * (cobj->field2 << 7));
  }
%}
```

If you examine the generated code, the supplied hash function will now be the function callback in the tp_hash slot for the builtin type for `MyClass`:

```
static PyHeapTypeObject SwigPyBuiltin__MyClass_type = {
  ...
  (hashfunc) myHashFunc,       /* tp_hash */
  ...
```

NOTE: It is the responsibility of the programmer (that's you!) to ensure that a statically defined slot function has the correct signature, the `hashfunc` typedef in this case.

If, instead, you want to dispatch to an instance method, you can use %feature("python:slot"). For example:

```
%feature("python:slot", "tp_hash", functype="hashfunc") MyClass::myHashFunc;

#if PY_VERSION_HEX < 0x03020000
  #define Py_hash_t long
#endif

class MyClass {
  public:
    Py_hash_t myHashFunc() const;
    ...
};
```

NOTE: Some Python slots use a method signature which does not match the signature of SWIG-wrapped methods. For those slots, SWIG will automatically generate a "closure" function to re-marshal the arguments before dispatching to the wrapped method. Setting the "functype" attribute of the feature enables SWIG to generate the chosen closure function.

There is further information on `%feature("python:slot")` in the file `python/pyopers.swg` in the SWIG library.

### 33.4.3 Memory management

NOTE: Although this section refers to proxy objects, everything here also applies when the `-builtin` option is used.

Associated with proxy object, is an ownership flag `thisown`. The value of this flag determines who is responsible for deleting the underlying C++ object. If set to `True`, the Python interpreter will destroy the C++ object when the proxy class is garbage collected. If set to `False` (or if the attribute is missing), then the destruction of the proxy class has no effect on the C++ object.

When an object is created by a constructor or returned by value, Python automatically takes ownership of the result. For example:

```
class Foo {
public:
  Foo();
  ~Foo();
};
Foo CreateFoo();
```

In Python:

```
>>> f = Foo()
>>> f.thisown
True
>>> g = CreateFoo()
>>> g.thisown
True
```

The type information is a Python class, a proxy of the underlying C++ type:

```
>>> type(f)
<class 'example.Foo'>
>>> f
<example.Foo; proxy of <Swig Object of type 'Foo *' at 0x7f5a86d48600> >
True
```

Unlike a C or C++ compiler, SWIG can work with incomplete type information and generate wrapper code even if the definition of `class Foo` is not provided to SWIG. However, this has negative consequences in the form of a memory leak.

```
>>> h = CreateFoo()
>>> type(h)
<class 'SwigPyObject'>
>>> h
<Swig Object of type 'Foo *' at 0x7fc341b4d6f0>
>>> quit()
swig/python detected a memory leak of type 'Foo *', no destructor found.
```

Note that the type is no longer a proxy Python class, but is instead a generic `SwigPyObject` type. The memory leak warning is shown as SWIG has no way of knowing how to call the destructor with this generic type. The recommendation is to fix the leak by providing the missing type information. However, for those who don't care about memory leaks, the warning can also be suppressed by by defining `SWIG_PYTHON_SILENT_MEMLEAK` when compiling the generated C/C++ wrapper code, such as passing `-DSWIG_PYTHON_SILENT_MEMLEAK` to the compiler or getting SWIG to define this macro at the beginning of the generated wrapper code by adding the following to the interface file using:

```
%begin %{
#define SWIG_PYTHON_SILENT_MEMLEAK
%}
```

The generic `SwigPyObject` type also has a `disown()` method which can be called to deactivate the warning.

Next, let's consider returning raw pointers even when the full type information is available to SWIG. When pointers or references are returned to Python, there is often no way to know where they came from and how to manage the memory that they point to. Therefore, the ownership is set to `False`. For example:

```
class Foo {
public:
  Foo();
```

```
  ~Foo();
};
Foo *PointerCreate();
Foo &ReferenceCreate();
```

```
>>> p = PointerCreate()
>>> p.thisown
False
>>> r = ReferenceCreate()
>>> r.thisown
False
```

This behavior is especially important for classes that act as containers. For example, if a method returns a pointer to an object that is contained inside another object, you definitely don't want Python to assume ownership and destroy it!

A good way to indicate that ownership should be set for a returned pointer is to use the %newobject directive.

Related to containers, ownership issues can arise whenever an object is assigned to a member or global variable. For example, consider this interface:

```
%module example

struct Foo {
  int  value;
  Foo  *next;
};

Foo *head = 0;
```

When wrapped in Python, careful observation will reveal that ownership changes whenever an object is assigned to a global variable. For example:

```
>>> f = example.Foo()
>>> f.thisown
True
>>> example.cvar.head = f
>>> f.thisown
False
>>>
```

In this case, C is now holding a reference to the object---you probably don't want Python to destroy it. Similarly, this occurs for members. For example:

```
>>> f = example.Foo()
>>> g = example.Foo()
>>> f.thisown
True
>>> g.thisown
True
>>> f.next = g
>>> g.thisown
False
>>>
```

For the most part, memory management issues remain hidden. However, there are occasionally situations where you might have to manually change the ownership of an object. For instance, consider code like this:

```
class Node {
  Object *value;
public:
  void set_value(Object *v) { value = v; }
  ...
};
```

Now, consider the following Python code:

```
>>> v = Object()          # Create an object
>>> n = Node()            # Create a node
>>> n.set_value(v)        # Set value
>>> v.thisown
True
>>> del v
```

In this case, the object `n` is holding a reference to `v` internally. However, SWIG has no way to know that this has occurred. Therefore, Python still thinks that it has ownership of the object. Should the proxy object be destroyed, then the C++ destructor will be invoked and `n` will be holding a stale-pointer. If you're lucky, you will only get a segmentation fault.

To work around this, it is always possible to flip the ownership flag. For example,

```
>>> v.thisown = False
```

It is also possible to deal with situations like this using typemaps--an advanced topic discussed later.

## 33.5 Cross language polymorphism

Proxy classes provide a more natural, object-oriented way to access extension classes. As described above, each proxy instance has an associated C++ instance, and method calls to the proxy are passed to the C++ instance transparently via C wrapper functions.

This arrangement is asymmetric in the sense that no corresponding mechanism exists to pass method calls down the inheritance chain from C++ to Python. In particular, if a C++ class has been extended in Python (by extending the proxy class), these extensions will not be visible from C++ code. Virtual method calls from C++ are thus not able access the lowest implementation in the inheritance chain.

Changes have been made to SWIG 1.3.18 to address this problem and make the relationship between C++ classes and proxy classes more symmetric. To achieve this goal, new classes

called directors are introduced at the bottom of the C++ inheritance chain. The job of the directors is to route method calls correctly, either to C++ implementations higher in the inheritance chain or to Python implementations lower in the inheritance chain. The upshot is that C++ classes can be extended in Python and from C++ these extensions look exactly like native C++ classes. Neither C++ code nor Python code needs to know where a particular method is implemented: the combination of proxy classes, director classes, and C wrapper functions takes care of all the cross-language method routing transparently.

### 33.5.1 Enabling directors

The director feature is disabled by default. To use directors you must make two changes to the interface file. First, add the "directors" option to the %module directive, like this:

```
%module(directors="1") modulename
```

Without this option no director code will be generated. Second, you must use the %feature("director") directive to tell SWIG which classes and methods should get directors. The %feature directive can be applied globally, to specific classes, and to specific methods, like this:

```
// generate directors for all classes that have virtual methods
%feature("director");

// generate directors for the virtual methods in class Foo
%feature("director") Foo;
```

You can use the %feature("nodirector") directive to turn off directors for specific classes or methods. So for example,

```
%feature("director") Foo;
%feature("nodirector") Foo::bar;
```

will generate directors for the virtual methods of class Foo except bar().

Directors can also be generated implicitly through inheritance. In the following, class Bar will get a director class that handles the methods one() and two() (but not three()):

```
%feature("director") Foo;
class Foo {
public:
  Foo(int foo);
  virtual ~Foo();
  virtual void one();
  virtual void two();
};

class Bar: public Foo {
public:
  virtual void three();
};
```

then at the Python side you can define

```
import mymodule

class MyFoo(mymodule.Foo):
    def __init__(self, foo):
        mymodule.Foo.__init__(self, foo)
#       super().__init__(foo) # Alternative construction for Python3

    def one(self):
        print("one from Python")
```

### 33.5.2 Director classes

For each class that has directors enabled, SWIG generates a new class that derives from both the class in question and a special `Swig::Director` class. These new classes, referred to as director classes, can be loosely thought of as the C++ equivalent of the Python proxy classes. The director classes store a pointer to their underlying Python object and handle various issues related to object ownership. Indeed, this is quite similar to the "this" and "thisown" members of the Python proxy classes.

For simplicity let's ignore the `Swig::Director` class and refer to the original C++ class as the director's base class. By default, a director class extends all virtual methods in the inheritance chain of its base class (see the preceding section for how to modify this behavior). Virtual methods that have a final specifier are unsurprisingly excluded. Thus the virtual method calls, whether they originate in C++ or in Python via proxy classes, eventually end up in at the implementation in the director class. The job of the director methods is to route these method calls to the appropriate place in the inheritance chain. By "appropriate place" we mean the method that would have been called if the C++ base class and its extensions in Python were seamlessly integrated. That seamless integration is exactly what the director classes provide, transparently skipping over all the messy extension API glue that binds the two languages together.

In reality, the "appropriate place" is one of only two possibilities: C++ or Python. Once this decision is made, the rest is fairly easy. If the correct implementation is in C++, then the lowest implementation of the method in the C++ inheritance chain is called explicitly. If the correct implementation is in Python, the Python API is used to call the method of the underlying Python object (after which the usual virtual method resolution in Python automatically finds the right implementation).

Now how does the director decide which language should handle the method call? The basic rule is to handle the method in Python, unless there's a good reason not to. The reason for this is simple: Python has the most "extended" implementation of the method. This assertion is guaranteed, since at a minimum the Python proxy class implements the method. If the method in question has been extended by a class derived from the proxy class, that extended implementation will execute exactly as it should. If not, the proxy class will route the method call into a C wrapper function, expecting that the method will be resolved in C++. The wrapper will call the virtual method of the C++ instance, and since the director extends this the call will end up right back in the director method. Now comes the "good reason not to" part. If the director method were to blindly call the Python method again, it would get stuck in an infinite loop. We avoid this situation by adding special code to the C wrapper function that tells the director method to not do this. The C wrapper function compares the pointer to the Python object that called the wrapper function to the pointer stored by the director. If these are the same, then the C wrapper function tells the director to resolve the method by calling up the C++ inheritance chain, preventing an infinite loop.

One more point needs to be made about the relationship between director classes and proxy classes. When a proxy class instance is created in Python, SWIG creates an instance of the original C++ class and assigns it to `.this`. This is exactly what happens without directors and is true even if directors are enabled for the particular class in question. When a class *derived* from a proxy class is created, however, SWIG then creates an instance of the corresponding C++ director class. The reason for this difference is that user-defined subclasses may override or extend methods of the original class, so the director class is needed to route calls to these methods correctly. For unmodified proxy classes, all methods are ultimately implemented in C++ so there is no need for the extra overhead involved with routing the calls through Python.

### 33.5.3 Ownership and object destruction

Memory management issues are slightly more complicated with directors than for proxy classes alone. Python instances hold a pointer to the associated C++ director object, and the director in turn holds a pointer back to the Python object. By default, proxy classes own their C++ director object and take care of deleting it when they are garbage collected.

This relationship can be reversed by calling the special `__disown__()` method of the proxy class. After calling this method, the `thisown` flag is set to `False`, and the director class increments the reference count of the Python object. When the director class is deleted it decrements the reference count. Assuming no outstanding references to the Python object remain, the Python object will be destroyed at the same time. This is a good thing, since directors and proxies refer to each other and so must be created and destroyed together. Destroying one

without destroying the other will likely cause your program to segfault.

To help ensure that no references to the Python object remain after calling `__disown__()`, this method returns a weak reference to the Python object. Here is an example:

```
class Foo {
public:
  ...
};
class FooContainer {
public:
  void addFoo(Foo *);
  ...
};
```

```
>>> c = FooContainer()
>>> a = Foo().__disown__()
>>> c.addFoo(a)
>>> b = Foo()
>>> b = b.__disown__()
>>> c.addFoo(b)
>>> c.addFoo(Foo().__disown__())
```

In this example, we are assuming that FooContainer will take care of deleting all the Foo pointers it contains at some point. Note that no hard references to the Foo objects remain in Python.

### 33.5.4 Exception unrolling

With directors routing method calls to Python, and proxies routing them to C++, the handling of exceptions is an important concern. By default, the directors ignore exceptions that occur during method calls that are resolved in Python. To handle such exceptions correctly, it is necessary to temporarily translate them into C++ exceptions. This can be done with the %feature("director:except") directive. The following code should suffice in most cases:

```
%feature("director:except") {
  if ($error != NULL) {
    throw Swig::DirectorMethodException();
  }
}
```

This code will check the Python error state after each method call from a director into Python, and throw a C++ exception if an error occurred. This exception can be caught in C++ to implement an error handler. Currently no information about the Python error is stored in the Swig::DirectorMethodException object, but this will likely change in the future.

It may be the case that a method call originates in Python, travels up to C++ through a proxy class, and then back into Python via a director method. If an exception occurs in Python at this point, it would be nice for that exception to find its way back to the original caller. This can be done by combining a normal %exception directive with the `director:except` handler shown above. Here is an example of a suitable exception handler:

```
%exception {
  try { $action }
  catch (Swig::DirectorException &e) { SWIG_fail; }
}
```

The class Swig::DirectorException used in this example is actually a base class of Swig::DirectorMethodException, so it will trap this exception. Because the Python error state is still set when Swig::DirectorMethodException is thrown, Python will register the exception as soon as the C wrapper function returns.

### 33.5.5 Overhead and code bloat

Enabling directors for a class will generate a new director method for every virtual method in the class' inheritance chain. This alone can generate a lot of code bloat for large hierarchies. Method arguments that require complex conversions to and from target language types can result in large director methods. For this reason it is recommended that you selectively enable directors only for specific classes that are likely to be extended in Python and used in C++.

Compared to classes that do not use directors, the call routing in the director methods does add some overhead. In particular, at least one dynamic cast and one extra function call occurs per method call from Python. Relative to the speed of Python execution this is probably completely negligible. For worst case routing, a method call that ultimately resolves in C++ may take one extra detour through Python in order to ensure that the method does not have an extended Python implementation. This could result in a noticeable overhead in some cases.

Although directors make it natural to mix native C++ objects with Python objects (as director objects) via a common base class pointer, one should be aware of the obvious fact that method calls to Python objects will be much slower than calls to C++ objects. This situation can be optimized by selectively enabling director methods (using the %feature directive) for only those methods that are likely to be extended in Python.

### 33.5.6 Typemaps

Typemaps for input and output of most of the basic types from director classes have been written. These are roughly the reverse of the usual input and output typemaps used by the wrapper code. The typemap operation names are 'directorin', 'directorout', and 'directorargout'. The director code does not currently use any of the other kinds of typemaps. It is not clear at this point which kinds are appropriate and need to be supported.

### 33.5.7 Thread safety

The director implementation uses a `std::map` to manage C++ pointer ownership. This code is not thread-safe by default. See Thread safety for turning on thread-safety. This defines a macro called `SWIG_THREADS` which forces the C++ code to then use a mutex for managing the pointer ownership, which in turn makes the directors implementation thread-safe.

Please see the `director_guard.swg` Library file for details of the implementation.

### 33.5.8 Miscellaneous

Director typemaps for STL classes are in place, and hence you should be able to use std::vector, std::string, etc., as you would any other type.

**Note:** The director typemaps for return types based on const references, such as

```
class Foo {
…
  virtual const int& bar();
…
};
```

will work only for simple call scenarios. Usually the resulting code is neither thread nor reentrant safe. Hence, the user is advised to avoid returning const references in director methods. For example, the user could modify the method interface to use return by value semantics instead.

```
class Foo {
…
  virtual int bar();
…
};
```

If that is not possible, the user should avoid enabling the director feature for reentrant, recursive or threaded member methods that return const references.

## 33.6 Common customization features

The last section presented the absolute basics of C/C++ wrapping. If you do nothing but feed SWIG a header file, you will get an interface that mimics the behavior described. However, sometimes this isn't enough to produce a nice module. Certain types of functionality might be missing or the interface to certain functions might be awkward. This section describes some common SWIG features that are used to improve your the interface to an extension module.

### 33.6.1 C/C++ helper functions

Sometimes when you create a module, it is missing certain bits of functionality. For example, if you had a function like this

```
void set_transform(Image *im, double m[4][4]);
```

it would be accessible from Python, but there may be no easy way to call it. For example, you might get errors like this:

```
>>> a = [
...   [1, 0, 0, 0],
...   [0, 1, 0, 0],
...   [0, 0, 1, 0],
...   [0, 0, 0, 1]]
>>> set_transform(im, a)
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
TypeError: Type error. Expected _p_a_4__double
```

The problem here is that there is no easy way to construct and manipulate a suitable `double [4][4]` value to use. To fix this, you can write some extra C helper functions. Just use the `%inline` directive. For example:

```
%inline %{
/* Note: double[4][4] is equivalent to a pointer to an array double (*)[4] */
double (*new_mat44())[4] {
  return (double (*)[4]) malloc(16*sizeof(double));
}
void free_mat44(double (*x)[4]) {
  free(x);
}
void mat44_set(double x[4][4], int i, int j, double v) {
  x[i][j] = v;
}
double mat44_get(double x[4][4], int i, int j) {
  return x[i][j];
}
%}
```

From Python, you could then write code like this:

```
>>> a = new_mat44()
>>> mat44_set(a, 0, 0, 1.0)
>>> mat44_set(a, 1, 1, 1.0)
>>> mat44_set(a, 2, 2, 1.0)
...
>>> set_transform(im, a)
>>>
```

Admittedly, this is not the most elegant looking approach. However, it works and it wasn't too hard to implement. It is possible to clean this up using Python code, typemaps, and other customization features as covered in later sections.

### 33.6.2 Adding additional Python code

If writing support code in C isn't enough, it is also possible to write code in Python. This code gets inserted in to the `.py` file created by SWIG. One use of Python code might be to supply a high-level interface to certain functions. For example:

```
/* Rename the SWIG-generated wrapper. */
%rename _set_transform set_transform;

...

void set_transform(Image *im, double x[4][4]);

...

/* Rewrite the high level interface to set_transform */
%pythoncode %{
def set_transform(im, x):
    a = new_mat44()
    for i in range(4):
        for j in range(4):
            mat44_set(a, i, j, x[i][j])
    _example._set_transform(im, a)
    free_mat44(a)
%}
```

In this example, `set_transform()` provides a high-level Python interface built on top of low-level helper functions. For example, this code now seems to work:

```
>>> a = [
...   [1, 0, 0, 0],
...   [0, 1, 0, 0],
...   [0, 0, 1, 0],
...   [0, 0, 0, 1]]
>>> set_transform(im, a)
>>>
```

Admittedly, this whole scheme for wrapping the two-dimension array argument is rather ad-hoc. Besides, shouldn't a Python list or a Numeric Python array just work normally? We'll get to those examples soon enough. For now, think of this example as an illustration of what can be done without having to rely on any of the more advanced customization features.

There is also `%pythonbegin` which is another directive very similar to `%pythoncode`, but generates the given Python code at the beginning of the `.py` file. This directive works in the same way as `%pythoncode`, except the code is copied just after the SWIG banner (comment) at the top of the file, before any real code. This provides an opportunity to add your own description in a comment near the top of the file as well as Python imports that have to appear at the top of the file, such as "`from __future__ import`" statements.

The following example for Python 2.x shows how to insert code into the generated wrapper to enable `print` to be used as a Python3-compatible function instead of a statement:

```
%pythonbegin %{
# This module provides wrappers to the Whizz Bang library
%}

%pythonbegin %{
from __future__ import print_function
print("Loading", "Whizz", "Bang", sep=' ... ')
%}
```

The insert code can be seen at the start of the generated `.py` file:

```
# This file was automatically generated by SWIG (https://www.swig.org).
# Version 4.0.0
#
# Do not make changes to this file unless you know what you are doing--modify
# the SWIG interface file instead.

# This module provides wrappers to the Whizz Bang library

from __future__ import print_function
print("Loading", "Whizz", "Bang", sep=' ... ')
```

When using `%pythoncode` and `%pythonbegin` you generally want to make sure that the block is delimited by `%{` and `%}`. If you delimit it with `{` and `}` then any lines with a leading `#` will be handled by SWIG as preprocessor directives, when you probably meant them as Python comments. Prior to SWIG 3.0.3, invalid preprocessor directives were silently ignored, so generally using the wrong delimiters resulted in such comments not appearing in the generated output (though a comment starting with a valid preprocessor directive could cause problems, for example: `# error handling`). SWIG 3.0.3 and later report an error for invalid preprocessor directives, so you may have to update existing interface files to delimit blocks of Python code correctly.

As an alternative to providing a block containing Python code, you can include Python code from a file. The code is inserted exactly as in the file, so this avoids any issues with the SWIG preprocessor. It's a good approach if you have a non-trivial chunk of Python code to insert. To use this feature you specify a filename in double quotes, for example:

```
%pythoncode "somecode.py"
```

Sometimes you may want to replace or modify the wrapper function that SWIG creates in the proxy `.py` file. The Python module in SWIG provides some features that enable you to do this. First, to entirely replace a proxy function you can use `%feature("shadow")`. For example:

```
%module example

// Rewrite bar() Python code

%feature("shadow") Foo::bar(int) %{
def bar(*args):
    #do something before
    $action
    #do something after
%}

class Foo {
public:
  int bar(int x);
};
```

where `$action` will be replaced by the call to the C/C++ proper method. Note that this does not include the arguments to the C/C++ method.

Often the proxy function created by SWIG is fine, but you simply want to add code to it without touching the rest of the generated function body. For these cases SWIG provides the `pythonprepend` and `pythonappend` features which do exactly as their names suggest. The `pythonprepend` feature will insert its value at the beginning of the proxy function, and `pythonappend` will insert code at the end of the proxy, just before the return statement.

```
%module example

// Add Python code to bar()

%feature("pythonprepend") Foo::bar(int) %{
    #do something before C++ call
%}

%feature("pythonappend") Foo::bar(int) %{
    #do something after C++ call
    #the 'val' variable holds the return value
%}


class Foo {
public:
  int bar(int x);
};
```

Notes: Usually the `pythonappend` and `pythonprepend` features are safer to use than the `shadow` feature. Also, from SWIG version 1.3.28 you can use the directive forms `%pythonappend` and `%pythonprepend` as follows:

```
%module example

// Add Python code to bar()

%pythonprepend Foo::bar(int) %{
    #do something before C++ call
%}

%pythonappend Foo::bar(int) %{
    #do something after C++ call
    #the 'val' variable holds the return value
%}


class Foo {
public:
  int bar(int x);
};
```

Note that when the underlying C++ method is overloaded, there is only one proxy Python method for multiple C++ methods. In this case, only one of parsed methods is examined for the feature. You are better off specifying the feature without the argument list to ensure it will get used, as it will then get attached to all the overloaded C++ methods. For example:

```
%module example

// Add Python code to bar()

%pythonprepend Foo::bar %{
    #do something before C++ call
%}

%pythonappend Foo::bar %{
    #do something after C++ call
%}


class Foo {
public:
  int bar(int x);
  int bar();
};
```

The same applies for overloaded constructors.

### 33.6.3 Class extension with %extend

One of the more interesting features of SWIG is that it can extend structures and classes with new methods--at least in the Python interface. Here is a simple example:

```
%module example
%{
#include "someheader.h"
%}

struct Vector {
  double x, y, z;
};

%extend Vector {
  char *__str__() {
    static char tmp[1024];
    sprintf(tmp, "Vector(%g, %g, %g)", $self->x, $self->y, $self->z);
    return tmp;
  }
  Vector(double x, double y, double z) {
    Vector *v = (Vector *) malloc(sizeof(Vector));
    v->x = x;
    v->y = y;
    v->z = z;
    return v;
  }
};
```

Now, in Python

```
>>> v = example.Vector(2, 3, 4)
>>> print(v)
Vector(2, 3, 4)
>>>
```

`%extend` can be used for many more tasks than this. For example, if you wanted to overload a Python operator, you might do this:

```
%extend Vector {
  Vector __add__(Vector *other) {
    Vector v;
    v.x = $self->x + other->x;
    v.y = $self->y + other->y;
    v.z = $self->z + other->z;
    return v;
  }
};
```

Use it like this:

```
>>> import example
>>> v = example.Vector(2, 3, 4)
>>> w = example.Vector(10, 11, 12)
>>> print(v+w)
Vector(12, 14, 16)
>>>
```

`%extend` works with both C and C++ code. It does not modify the underlying object in any way---the extensions only show up in the Python interface.

### 33.6.4 Exception handling with %exception

If a C or C++ function throws an error, you may want to convert that error into a Python exception. To do this, you can use the `%exception` directive. `%exception` simply lets you rewrite part of the generated wrapper code to include an error check.

In C, a function often indicates an error by returning a status code (a negative number or a NULL pointer perhaps). Here is a simple example of how you might handle that:

```
%exception malloc {
  $action
  if (!result) {
    PyErr_SetString(PyExc_MemoryError, "Not enough memory");
    SWIG_fail;
  }
}
void *malloc(size_t nbytes);
```

In Python,

```
>>> a = example.malloc(2000000000)
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
MemoryError: Not enough memory
>>>
```

If a library provides some kind of general error handling framework, you can also use that. For example:

```
%exception {
  $action
  if (err_occurred()) {
    PyErr_SetString(PyExc_RuntimeError, err_message());
    SWIG_fail;
  }
}
```

No declaration name is given to `%exception`, it is applied to all wrapper functions.

C++ exceptions are also easy to handle. For example, you can write code like this:

```
%exception getitem {
  try {
    $action
  } catch (std::out_of_range &e) {
    PyErr_SetString(PyExc_IndexError, const_cast<char*>(e.what()));
    SWIG_fail;
  }
}

class Base {
public:
  Foo *getitem(int index);      // Exception handled added
  ...
};
```

When raising a Python exception from C, use the `PyErr_SetString()` function as shown above followed by `SWIG_fail` . The following exception types can be used as the first argument.

```
PyExc_ArithmeticError
PyExc_AssertionError
PyExc_AttributeError
PyExc_EnvironmentError
PyExc_EOFError
PyExc_Exception
PyExc_FloatingPointError
PyExc_ImportError
PyExc_IndexError
PyExc_IOError
PyExc_KeyError
PyExc_KeyboardInterrupt
PyExc_LookupError
PyExc_MemoryError
PyExc_NameError
PyExc_NotImplementedError
PyExc_OSError
PyExc_OverflowError
PyExc_RuntimeError
PyExc_StandardError
PyExc_SyntaxError
PyExc_SystemError
PyExc_TypeError
PyExc_UnicodeError
PyExc_ValueError
PyExc_ZeroDivisionError
```

`SWIG_fail` is a C macro which when called within the context of SWIG wrapper function, will jump to the error handler code. This will call any cleanup code (freeing any temp variables) and then return from the wrapper function so that the Python interpreter can raise the Python exception. This macro should always be called after setting a Python error in code snippets, such as typemaps and `%exception` , that are ultimately generated into the wrapper function.

The language-independent `exception.i` library file can also be used to raise exceptions. See the SWIG Library chapter.

### 33.6.5 Optimization options

#### 33.6.5.1 -fastproxy

The `-fastproxy` command line option enables faster method calling as the call is made directly into the C/C++ layer rather than going through a method wrapper.

Consider wrapping a C++ class:

```
struct Go {
  void callme0() {}
  void callme4(int a, int b, int c, int d) {}
  void callme8(double a, double b, double c, double d, double e, double f, double g, double i) {}
};
```

The default generated proxy class is:

```
class Go(object):
    def callme0(self):
        return _example.Go_callme0(self)

    def callme4(self, a, b, c, d):
        return _example.Go_callme4(self, a, b, c, d)

    def callme8(self, a, b, c, d, e, f, g, i):
        return _example.Go_callme8(self, a, b, c, d, e, f, g, i)
    ...
```

Each method in the Python class contains a Python proxy method which passes the arguments on to the underlying function in the low-level C/C++ module (_example in this case). The generated proxy class when using `-fastproxy` is:

```
%module example
class Go(object):
    callme0 = _swig_new_instance_method(_example.Go_callme0)
    callme4 = _swig_new_instance_method(_example.Go_callme4)
    callme8 = _swig_new_instance_method(_example.Go_callme8)
    ...
```

where `_swig_new_instance_method` adds the method to the proxy class via C API calls for direct access to the underlying function in the low-level C/C++ module. Note that for some methods it is not possible to generate the direct access call and so `-fastproxy` is ignored. This happens, for example, when adding additional code to Python proxy methods, such as using `%pythonprepend`.

The overhead calling into C/C++ from Python is reduced slightly using `-fastproxy`. Below are some timings in microseconds calling the 3 functions in the example above. Also included in the table for comparison is using the `-builtin` option covered in the Built-in Types.

| Method name | Default | -fastproxy | -builtin |
|---|---|---|---|
| callme0 | 0.15 | 0.09 | 0.07 |
| callme4 | 0.26 | 0.16 | 0.14 |
| callme8 | 0.32 | 0.20 | 0.17 |

Although the `-fastproxy` option results in faster code over the default, the generated proxy code is not as user-friendly as docstring/doxygen comments, Python annotations and functions with default values are not visible in the generated Python proxy class. The `-olddefs` option can rectify this.

The generated proxy class for the example above when using `-fastproxy -olddefs` is:

```
class Go(object):
    def callme0(self):
        return _example.Go_callme0(self)
    callme0 = _swig_new_instance_method(_example.Go_callme0)

    def callme4(self, a, b, c, d):
        return _example.Go_callme4(self, a, b, c, d)
    callme4 = _swig_new_instance_method(_example.Go_callme4)

    def callme8(self, a, b, c, d, e, f, g, i):
        return _example.Go_callme8(self, a, b, c, d, e, f, g, i)
    callme8 = _swig_new_instance_method(_example.Go_callme8)
    ...
```

The class defines each method in two different ways. The first definition is replaced by the second definition and so the second definition is the one used when the method is called. While this possibly provides the best of both worlds, the time to import the module will be slightly slower when the class is defined due to the additional method definitions.

The command line options mentioned above also apply to wrapped C/C++ global functions, not just class methods.

### 33.6.6 Stable ABI

By default, the version of Python used to compile the wrappers needs to be the same as that used during runtime. Alternatvely, the Python Stable ABI enables a single compiled binary to be used by different versions of Python. This is enabled by defining `Py_LIMITED_API` during the compilation of the C/C++ wrapper code and setting this macro to a particular minimum version of Python that one wants to support.

SWIG supports the stable ABI, but only version 3.4 of Python and later is supported. There are two recommended approaches for using SWIG and the stable ABI and both require setting the `Py_LIMITED_API` macro to be set to `0x03040000` as a minimum value (Python 3.4). Either set this using `%begin` by adding the following into your interface file so that this macro appears at the beginning of the generated C/C++ code:

```
%begin %{
#define Py_LIMITED_API 0x03040000
%}
```

or simply define the macro using your C/C++ compiler's `-D` command line option, for example, `-DPy_LIMITED_API=0x03040000`.

The default SWIG command line options generate code that enable the limited API/stable ABI. Some options, such as `-builtin`, `-fast` (used by `-O`) do not use the limited API and hence when `Py_LIMITED_API` is defined there will be missing symbols during compilation. Compatibility of user's custom typemaps is of course dependent on the Python APIs used in the typemaps.

As discussed in the official Python Stable ABI documentation, there is no guarantee that code conforms to the Limited API or Stable ABI when defining `Py_LIMITED_API`. There are a number of recommendations and caveats detailed there which are worth studying. The Stable ABI is also evolving and being modified with each Python release. In fact there is a Python C API working group, capi-workgroup, which was setup in the first half of 2023. Thecapi-workgroup/problems issues is very enlightening reading for anyone using the Stable ABI. PEP 733 came out of the working group and is an evaluation of Python's C API.

There is a lot of information to digest in this space, but a simple approach would be to follow the offical recommendation of testing an extension with all minor Python versions you want to support, and to build with the lowest supported version. SWIG supports at least Python 3.4 for the Stable ABI so this would be a good version to compile a SWIG extension against. There is also a useful tool abi3audit which can be used to check the compiled extension for Stable ABI conformance. It can be easily installed with pip and then used to check a SWIG generated shared object or wheel even. Note that if you are using it to check a shared object, the shared object must be renamed to have abi3 in the name. For example, below is the output on Linux for the class example shipped with SWIG, if `Py_LIMITED_API` is set as described earlier:

```
~/swig/Examples/python/class $ mv _example.so _example.abi3.so
~/swig/Examples/python/class $ abi3audit --assume-minimum-abi3 3.4 _example.abi3.so
[22:12:18] WARNING  no wheel to infer abi3 version from; assuming (3.4)
[22:12:18] _example.abi3.so: 1 extensions scanned; 0 ABI version mismatches and 0 ABI violations found
```

If `Py_LIMITED_API` is not set, the check fails, but this should be expected:

```
~/swig/Examples/python/class $ mv _example.so _example.abi3.so
~/swig/Examples/python/class $ abi3audit --report --assume-minimum-abi3 3.4 _example.abi3.so
[22:22:27] WARNING  no wheel to infer abi3 version from; assuming (3.4)
[22:22:27] _example.abi3.so: 1 extensions scanned; 1 ABI version mismatches and 0 ABI violations found
{"specs": {"_example.abi3.so": {"kind": "object", "object": {"name": "_example.abi3.so",
"result": {"is_abi3": true, "is_abi3_baseline_compatible": false, "baseline": "3.4", "computed": "3.10",
"non_abi3_symbols": [], "future_abi3_objects": {"PyUnicode_AsUTF8AndSize": "3.10"}}}}}}
```

Note that ABI conformance as measured by `abi3audit` is far from simple and can depend on function inlining. There are functions in Python.h, which if inlined, become ABI compliant. Your chances of your C or C++ compiler inlining functions is greater with higher compiler optimisation levels. So merely by changing your compiler optimisation level may switch your Python extension from being identified as ABI compliant or not.

**Compatibility Note:** Support for the stable ABI was added in SWIG-4.2.0.

## 33.7 Tips and techniques

Although SWIG is largely automatic, there are certain types of wrapping problems that require additional user input. Examples include dealing with output parameters, strings, binary data, and arrays. This chapter discusses the common techniques for solving these problems.

### 33.7.1 Input and output parameters

A common problem in some C programs is handling parameters passed as simple pointers. For example:

```
void add(int x, int y, int *result) {
  *result = x + y;
}
```

or perhaps

```
int sub(int *x, int *y) {
  return *x-*y;
}
```

The easiest way to handle these situations is to use the `typemaps.i` file. For example:

```
%module example
%include "typemaps.i"

void add(int, int, int *OUTPUT);
int  sub(int *INPUT, int *INPUT);
```

In Python, this allows you to pass simple values. For example:

```
>>> a = add(3, 4)
>>> print(a)
7
>>> b = sub(7, 4)
>>> print(b)
3
>>>
```

Notice how the `INPUT` parameters allow integer values to be passed instead of pointers and how the `OUTPUT` parameter creates a return result.

If you don't want to use the names `INPUT` or `OUTPUT` , use the `%apply` directive. For example:

```
%module example
%include "typemaps.i"

%apply int *OUTPUT { int *result };
%apply int *INPUT  { int *x, int *y};

void add(int x, int y, int *result);
int  sub(int *x, int *y);
```

If a function mutates one of its parameters like this,

```
void negate(int *x) {
  *x = -(*x);
}
```

you can use `INOUT` like this:

```
%include "typemaps.i"
...
void negate(int *INOUT);
```

In Python, a mutated parameter shows up as a return value. For example:

```
>>> a = negate(3)
>>> print(a)
-3
>>>
```

Note: Since most primitive Python objects are immutable, it is not possible to perform in-place modification of a Python object passed as a parameter.

The most common use of these special typemap rules is to handle functions that return more than one value. For example, sometimes a function returns a result as well as a special error code:

```
/* send message, return number of bytes sent, along with success code */
int send_message(char *text, int *success);
```

To wrap such a function, simply use the `OUTPUT` rule above. For example:

```
%module example
%include "typemaps.i"
%apply int *OUTPUT { int *success };
...
int send_message(char *text, int *success);
```

When used in Python, the function will return multiple values.

```
bytes, success = send_message("Hello World")
if not success:
    print("Whoa!")
else:
    print("Sent", bytes)
```

Another common use of multiple return values are in query functions. For example:

```
void get_dimensions(Matrix *m, int *rows, int *columns);
```

To wrap this, you might use the following:

```
%module example
%include "typemaps.i"
%apply int *OUTPUT { int *rows, int *columns };
...
void get_dimensions(Matrix *m, int *rows, *columns);
```

Now, in Python:

```
>>> r, c = get_dimensions(m)
```

Be aware that the primary purpose of the `typemaps.i` file is to support primitive datatypes. Writing a function like this

```
void foo(Bar *OUTPUT);
```

may not have the intended effect since `typemaps.i` does not define an OUTPUT rule for `Bar`.

### 33.7.2 Simple pointers

If you must work with simple pointers such as `int *` or `double *` and you don't want to use `typemaps.i`, consider using the `cpointer.i` library file. For example:

```
%module example
%include "cpointer.i"

%inline %{
extern void add(int x, int y, int *result);
%}

%pointer_functions(int, intp);
```

The `%pointer_functions(type, name)` macro generates five helper functions that can be used to create, destroy, copy, assign, and dereference a pointer. In this case, the functions are as follows:

```
int  *new_intp();
```

```
int  *copy_intp(int *x);
void  delete_intp(int *x);
void  intp_assign(int *x, int value);
int   intp_value(int *x);
```

In Python, you would use the functions like this:

```
>>> result = new_intp()
>>> print(result)
_108fea8_p_int
>>> add(3, 4, result)
>>> print(intp_value(result))
7
>>>
```

If you replace `%pointer_functions()` by `%pointer_class(type, name)`, the interface is more class-like.

```
>>> result = intp()
>>> add(3, 4, result)
>>> print(result.value())
7
```

See the SWIG Library chapter for further details.

### 33.7.3 Unbounded C Arrays

Sometimes a C function expects an array to be passed as a pointer. For example,

```
int sumitems(int *first, int nitems) {
  int i, sum = 0;
  for (i = 0; i < nitems; i++) {
    sum += first[i];
  }
  return sum;
}
```

To wrap this into Python, you need to pass an array pointer as the first argument. A simple way to do this is to use the `carrays.i` library file. For example:

```
%include "carrays.i"
%array_class(int, intArray);
```

The `%array_class(type, name)` macro creates wrappers for an unbounded array object that can be passed around as a simple pointer like `int *` or `double *`. For instance, you will be able to do this in Python:

```
>>> a = intArray(10000000)         # Array of 10-million integers
>>> for i in xrange(10000):        # Set some values
...     a[i] = i
>>> sumitems(a, 10000)
49995000
>>>
```

The array "object" created by `%array_class()` does not encapsulate pointers inside a special array object. In fact, there is no bounds checking or safety of any kind (just like in C). Because of this, the arrays created by this library are extremely low-level indeed. You can't iterate over them nor can you even query their length. In fact, any valid memory address can be accessed if you want (negative indices, indices beyond the end of the array, etc.). Needless to say, this approach is not going to suit all applications. On the other hand, this low-level approach is extremely efficient and well suited for applications in which you need to create buffers, package binary data, etc.

### 33.7.4 String handling

If a C function has an argument of `char *`, then a Python string can be passed as input. For example:

```
// C
void foo(char *s);
```

```
# Python
>>> foo("Hello")
```

When a Python string is passed as a parameter, the C function receives a pointer to the raw data contained in the string. Since Python strings are immutable, it is illegal for your program to change the value. In fact, doing so will probably crash the Python interpreter.

If your program modifies the input parameter or uses it to return data, consider using the `cstring.i` library file described in the SWIG Library chapter.

When functions return a `char *`, it is assumed to be a NULL-terminated string. Data is copied into a new Python string and returned.

If your program needs to work with binary data, you can use a typemap to expand a Python string into a pointer/length argument pair. As luck would have it, just such a typemap is already defined. Just do this:

```
%apply (char *STRING, size_t LENGTH) { (char *data, size_t size) };
...
int parity(char *data, size_t size, int initial);
```

Now in Python:

```
>>> parity("e\x09ffss\x00\x00\x01\nx", 0)
```

If you need to return binary data, you might use the `cstring.i` library file. The `cdata.i` library can also be used to extra binary data from arbitrary pointers.

### 33.7.5 Default arguments

C++ default argument code generation is documented in the main Default arguments section. There is also an optional Python specific feature that can be used called the `python:cdefaultargs` feature flag. By default, SWIG attempts to convert C++ default argument values into Python values and generates code into the Python layer containing these values. For example:

```
struct CDA {
  int fff(int a = 1, bool b = false);
};
```

From Python this can be called as follows:

```
>>> CDA().fff()        # C++ layer receives a=1 and b=false
>>> CDA().fff(2)       # C++ layer receives a=2 and b=false
>>> CDA().fff(3, True) # C++ layer receives a=3 and b=true
```

The default code generation in the Python layer is:

```
class CDA(object):
    ...
    def fff(self, a=1, b=False):
        return _default_args.CDA_fff(self, a, b)
```

Adding the feature:

```
%feature("python:cdefaultargs") CDA::fff;
struct CDA {
  int fff(int a = 1, bool b = false);
```

results in identical behaviour when called from Python, however, it results in different code generation:

```
class CDA(object):
    ...
    def fff(self, *args):
        return _default_args.CDA_fff(self, *args)
```

The default arguments are obtained in the C++ wrapper layer instead of the Python layer. Some code generation modes are quite different, eg `-builtin` and `-fastproxy`, and are unaffected by `python:cdefaultargs` as the default values are always obtained from the C++ layer.

Note that not all default arguments can be converted into a Python equivalent. When SWIG does not convert them, it will generate code to obtain them from the C++ layer as if `python:cdefaultargs` was specified. This will happen if just one argument cannot be converted into a Python equivalent. This occurs typically when the argument is not fully numeric, such as `int(1)`:

```
struct CDA {
  int fff(int a = int(1), bool b = false);
};
```

**Compatibility Note:** SWIG-3.0.6 introduced the `python:cdefaultargs` feature. Versions of SWIG prior to this varied in their ability to convert C++ default values into equivalent Python default argument values.

## 33.8 Typemaps

This section describes how you can modify SWIG's default wrapping behavior for various C/C++ datatypes using the `%typemap` directive. This is an advanced topic that assumes familiarity with the Python C API as well as the material in the "Typemaps" chapter.

Before proceeding, it should be stressed that typemaps are not a required part of using SWIG---the default wrapping behavior is enough in most cases. Typemaps are only used if you want to change some aspect of the primitive C-Python interface or if you want to elevate your guru status.

### 33.8.1 What is a typemap?

A typemap is nothing more than a code generation rule that is attached to a specific C datatype. For example, to convert integers from Python to C, you might define a typemap like this:

```
%module example

%typemap(in) int {
  $1 = (int) PyLong_AsLong($input);
  printf("Received an integer : %d\n", $1);
}
%inline %{
extern int fact(int n);
%}
```

Typemaps are always associated with some specific aspect of code generation. In this case, the "in" method refers to the conversion of input arguments to C/C++. The datatype `int` is the datatype to which the typemap will be applied. The supplied C code is used to convert values. In this code a number of special variable prefaced by a `$` are used. The `$1` variable is placeholder for a local variable of type `int`. The `$input` variable is the input object of type `PyObject *`.

When this example is compiled into a Python module, it operates as follows:

```
>>> from example import *
>>> fact(6)
Received an integer : 6
720
```

In this example, the typemap is applied to all occurrences of the `int` datatype. You can refine this by supplying an optional parameter name. For example:

```
%module example
```

```
%typemap(in) int nonnegative {
  $1 = (int) PyLong_AsLong($input);
  if ($1 < 0) {
    PyErr_SetString(PyExc_ValueError, "Expected a nonnegative value.");
    SWIG_fail;
  }
}
%inline %{
extern int fact(int nonnegative);
%}
```

In this case, the typemap code is only attached to arguments that exactly match `int nonnegative`.

The application of a typemap to specific datatypes and argument names involves more than simple text-matching--typemaps are fully integrated into the SWIG C++ type-system. When you define a typemap for `int`, that typemap applies to `int` and qualified variations such as `const int`. In addition, the typemap system follows `typedef` declarations. For example:

```
%typemap(in) int n {
  $1 = (int) PyLong_AsLong($input);
  printf("n = %d\n", $1);
}
%inline %{
typedef int Integer;
extern int fact(Integer n);    // Above typemap is applied
%}
```

Typemaps can also be defined for groups of consecutive arguments. For example:

```
%typemap(in) (char *str, int len) {
  $1 = PyString_AsString($input);
  $2 = PyString_Size($input);
};

int count(char c, char *str, int len);
```

When a multi-argument typemap is defined, the arguments are always handled as a single Python object. This allows the function to be used like this (notice how the length parameter is omitted):

```
>>> example.count('e', 'Hello World')
1
>>>
```

### 33.8.2 Python typemaps

The previous section illustrated an "in" typemap for converting Python objects to C. A variety of different typemap methods are defined by the Python module. For example, to convert a C integer back into a Python object, you might define an "out" typemap like this:

```
%typemap(out) int {
    $result = PyInt_FromLong((long) $1);
}
```

A detailed list of available methods can be found in the "Typemaps" chapter.

However, the best source of typemap information (and examples) is probably the Python module itself. In fact, all of SWIG's default type handling is defined by typemaps. You can view these typemaps by looking at the files in the SWIG library. Just take into account that in the latest versions of swig (1.3.22+), the library files are not very pristine clear for the casual reader, as they used to be. The extensive use of macros and other ugly techniques in the latest version produce a very powerful and consistent Python typemap library, but at the cost of simplicity and pedagogic value.

To learn how to write a simple or your first typemap, you better take a look at the SWIG library version 1.3.20 or so.

### 33.8.3 Typemap variables

Within typemap code, a number of special variables prefaced with a `$` may appear. A full list of variables can be found in the "Typemaps" chapter. This is a list of the most common variables:

`$1`

A C local variable corresponding to the actual type specified in the `%typemap` directive. For input values, this is a C local variable that's supposed to hold an argument value. For output values, this is the raw result that's supposed to be returned to Python.

`$input`

A `PyObject *` holding a raw Python object with an argument or variable value.

`$result`

A `PyObject *` that holds the result to be returned to Python.

`$1_name`

The parameter name that was matched.

`$1_type`

The actual C datatype matched by the typemap.

`$1_ltype`

An assignable version of the datatype matched by the typemap (a type that can appear on the left-hand-side of a C assignment operation). This type is stripped of qualifiers and may be an altered version of `$1_type`. All arguments and local variables in wrapper functions are declared using this type so that their values can be properly assigned.

`$symname`

The Python name of the wrapper function being created.

### 33.8.4 Useful Python Functions

When you write a typemap, you usually have to work directly with Python objects. The following functions may prove to be useful.

**Python Integer Functions**

```
PyObject *PyInt_FromLong(long l);
long      PyInt_AsLong(PyObject *);
int       PyInt_Check(PyObject *);
```

**Python Floating Point Functions**

```
PyObject *PyFloat_FromDouble(double);
double    PyFloat_AsDouble(PyObject *);
int       PyFloat_Check(PyObject *);
```

**Python String Functions**

```
PyObject *PyString_FromString(char *);
PyObject *PyString_FromStringAndSize(char *, lint len);
int       PyString_Size(PyObject *);
char     *PyString_AsString(PyObject *);
int       PyString_Check(PyObject *);
```

**Python List Functions**

```
PyObject *PyList_New(int size);
int       PyList_Size(PyObject *list);
PyObject *PyList_GetItem(PyObject *list, int i);
int       PyList_SetItem(PyObject *list, int i, PyObject *item);
int       PyList_Insert(PyObject *list, int i, PyObject *item);
int       PyList_Append(PyObject *list, PyObject *item);
PyObject *PyList_GetSlice(PyObject *list, int i, int j);
int       PyList_SetSlice(PyObject *list, int i, int , PyObject *list2);
int       PyList_Sort(PyObject *list);
int       PyList_Reverse(PyObject *list);
PyObject *PyList_AsTuple(PyObject *list);
int       PyList_Check(PyObject *);
```

**Python Tuple Functions**

```
PyObject *PyTuple_New(int size);
int       PyTuple_Size(PyObject *);
PyObject *PyTuple_GetItem(PyObject *, int i);
int       PyTuple_SetItem(PyObject *, int i, PyObject *item);
PyObject *PyTuple_GetSlice(PyObject *t, int i, int j);
int       PyTuple_Check(PyObject *);
```

**Python Dictionary Functions**

```
PyObject *PyDict_New();
int       PyDict_Check(PyObject *);
int       PyDict_SetItem(PyObject *p, PyObject *key, PyObject *val);
int       PyDict_SetItemString(PyObject *p, const char *key, PyObject *val);
int       PyDict_DelItem(PyObject *p, PyObject *key);
int       PyDict_DelItemString(PyObject *p, char *key);
PyObject* PyDict_Keys(PyObject *p);
PyObject* PyDict_Values(PyObject *p);
PyObject* PyDict_GetItem(PyObject *p, PyObject *key);
PyObject* PyDict_GetItemString(PyObject *p, const char *key);
int       PyDict_Next(PyObject *p, Py_ssize_t *ppos, PyObject **pkey, PyObject **pvalue);
Py_ssize_t PyDict_Size(PyObject *p);
int       PyDict_Update(PyObject *a, PyObject *b);
int       PyDict_Merge(PyObject *a, PyObject *b, int override);
PyObject* PyDict_Items(PyObject *p);
```

**Python File Conversion Functions**

```
PyObject *PyFile_FromFile(FILE *f);
FILE     *PyFile_AsFile(PyObject *);
int       PyFile_Check(PyObject *);
```

**Abstract Object Interface**

```
write me
```

## 33.9 Typemap Examples

This section includes a few examples of typemaps. For more examples, you might look at the files "`python.swg`" and "`typemaps.i`" in the SWIG library.

### 33.9.1 Converting a Python list to a char **

A common problem in many C programs is the processing of command line arguments, which are usually passed in an array of NULL terminated strings. SWIG provides typemaps which allow passing a Python list or tuple - see argcargv.i.

### 33.9.2 Expanding a Python object into multiple arguments

Suppose that you had a collection of C functions with arguments such as the following:

```
int foo(int argc, char **argv);
```

In the previous example, a typemap was written to pass a Python list as the `char **argv`. This allows the function to be used from Python as follows:

```
>>> foo(4, ["foo", "bar", "spam", "1"])
```

Although this works, it's a little awkward to specify the argument count. To fix this, a multi-argument typemap can be defined. This is not very difficult--you only have to make slight modifications to the previous example:

```
%typemap(in) (int argc, char **argv) {
  /* Check if is a list */
  if (PyList_Check($input)) {
    int i;
    $1 = PyList_Size($input);
    $2 = (char **) malloc(($1+1)*sizeof(char *));
    for (i = 0; i < $1; i++) {
      PyObject *o = PyList_GetItem($input, i);
      if (PyString_Check(o)) {
        $2[i] = PyString_AsString(PyList_GetItem($input, i));
      } else {
        PyErr_SetString(PyExc_TypeError, "list must contain strings");
        SWIG_fail;
      }
    }
    $2[i] = 0;
  } else {
    PyErr_SetString(PyExc_TypeError, "not a list");
    SWIG_fail;
  }
}

%typemap(freearg) (int argc, char **argv) {
  free((char *) $2);
}
```

When writing a multiple-argument typemap, each of the types is referenced by a variable such as `$1` or `$2`. The typemap code simply fills in the appropriate values from the supplied Python object.

With the above typemap in place, you will find it no longer necessary to supply the argument count. This is automatically set by the typemap code. For example:

```
>>> foo(["foo", "bar", "spam", "1"])
```

If your function is overloaded in C++, for example:

```
int foo(int argc, char **argv);
int foo();
```

don't forget to also provide a suitable typecheck typemap for overloading such as:

```
%typecheck(SWIG_TYPECHECK_STRING_ARRAY) (int argc, char **argv) {
  $1 = PyList_Check($input) ? 1 : 0;
}
```

If you don't you'll get an error message along the lines of:

```
Traceback (most recent call last):
  File "runme.py", line 3, in >module<
    example.foo(["foo", "bar", "spam", "1"])
TypeError: Wrong number or type of arguments for overloaded function 'foo'.
  Possible C/C++ prototypes are:
    foo(int, char **)
    foo()
```

### 33.9.3 Using typemaps to return arguments

A common problem in some C programs is that values may be returned in arguments rather than in the return value of a function. For example:

```
/* Returns a status value and two values in out1 and out2 */
int spam(double a, double b, double *out1, double *out2) {
  ... Do a bunch of stuff ...
  *out1 = result1;
  *out2 = result2;
  return status;
}
```

A typemap can be used to handle this case as follows :

```
%module outarg

// This tells SWIG to treat an double * argument with name 'OutValue' as
// an output value.  We'll append the value to the current result which
// is guaranteed to be a List object by SWIG.

%typemap(argout) double *OutValue {
  PyObject *o, *o2, *o3;
  o = PyFloat_FromDouble(*$1);
  if ((!$result) || ($result == Py_None)) {
    $result = o;
```

```
  } else {
    if (!PyTuple_Check($result)) {
      PyObject *o2 = $result;
      $result = PyTuple_New(1);
      PyTuple_SetItem($result, 0, o2);
    }
    o3 = PyTuple_New(1);
    PyTuple_SetItem(o3, 0, o);
    o2 = $result;
    $result = PySequence_Concat(o2, o3);
    Py_DecRef(o2);
    Py_DecRef(o3);
  }
}

int spam(double a, double b, double *OutValue, double *OutValue);
```

The typemap works as follows. First, a check is made to see if any previous result exists. If so, it is turned into a tuple and the new output value is concatenated to it. Otherwise, the result is returned normally. For the sample function `spam()`, there are three output values--meaning that the function will return a 3-tuple of the results.

As written, the function must accept 4 arguments as input values, last two being pointers to doubles. If these arguments are only used to hold output values (and have no meaningful input value), an additional typemap can be written. For example:

```
%typemap(in, numinputs=0) double *OutValue(double temp) {
  $1 = &temp;
}
```

By specifying numinputs=0, the input value is ignored. However, since the argument still has to be set to some meaningful value before calling C, it is set to point to a local variable `temp`. When the function stores its output value, it will simply be placed in this local variable. As a result, the function can now be used as follows:

```
>>> a = spam(4, 5)
>>> print(a)
(0, 2.45, 5.0)
>>> x, y, z = spam(4, 5)
>>>
```

### 33.9.4 Mapping Python tuples into small arrays

In some applications, it is sometimes desirable to pass small arrays of numbers as arguments. For example :

```
extern void set_direction(double a[4]);       // Set direction vector
```

This too, can be handled used typemaps as follows :

```
// Grab a 4 element array as a Python 4-tuple
%typemap(in) double[4](double temp[4]) {   // temp[4] becomes a local variable
  int i;
  if (PyTuple_Check($input)) {
    if (!PyArg_ParseTuple($input, "dddd", temp, temp+1, temp+2, temp+3)) {
      PyErr_SetString(PyExc_TypeError, "tuple must have 4 elements");
      SWIG_fail;
    }
    $1 = &temp[0];
  } else {
    PyErr_SetString(PyExc_TypeError, "expected a tuple.");
    SWIG_fail;
  }
}
```

This allows our `set_direction` function to be called from Python as follows :

```
>>> set_direction((0.5, 0.0, 1.0, -0.25))
```

Since our mapping copies the contents of a Python tuple into a C array, such an approach would not be recommended for huge arrays, but for small structures, this approach works fine.

### 33.9.5 Mapping sequences to C arrays

Suppose that you wanted to generalize the previous example to handle C arrays of different sizes. To do this, you might write a typemap as follows:

```
// Map a Python sequence into any sized C double array
%typemap(in) double[ANY](double temp[$1_dim0]) {
  int i;
  if (!PySequence_Check($input)) {
    PyErr_SetString(PyExc_TypeError, "Expecting a sequence");
    SWIG_fail;
  }
  if (PyObject_Length($input) != $1_dim0) {
    PyErr_SetString(PyExc_ValueError, "Expecting a sequence with $1_dim0 elements");
    SWIG_fail;
  }
  for (i =0; i < $1_dim0; i++) {
    PyObject *o = PySequence_GetItem($input, i);
    if (!PyFloat_Check(o)) {
      Py_DecRef(o);
      PyErr_SetString(PyExc_ValueError, "Expecting a sequence of floats");
      SWIG_fail;
    }
    temp[i] = PyFloat_AsDouble(o);
```

```
    Py_DecRef(o);
  }
  $1 = &temp[0];
}
```

In this case, the variable `$1_dim0` is expanded to match the array dimensions actually used in the C code. This allows the typemap to be applied to types such as:

```
void foo(double x[10]);
void bar(double a[4], double b[8]);
```

Since the above typemap code gets inserted into every wrapper function where used, it might make sense to use a helper function instead. This will greatly reduce the amount of wrapper code. For example:

```
%{
static int convert_darray(PyObject *input, double *ptr, int size) {
  int i;
  if (!PySequence_Check(input)) {
    PyErr_SetString(PyExc_TypeError, "Expecting a sequence");
    return 0;
  }
  if (PyObject_Length(input) != size) {
    PyErr_SetString(PyExc_ValueError, "Sequence size mismatch");
    return 0;
  }
  for (i =0; i < size; i++) {
    PyObject *o = PySequence_GetItem(input, i);
    if (!PyFloat_Check(o)) {
      Py_DecRef(o);
      PyErr_SetString(PyExc_ValueError, "Expecting a sequence of floats");
      return 0;
    }
    ptr[i] = PyFloat_AsDouble(o);
    Py_DecRef(o);
  }
  return 1;
}
%}

%typemap(in) double [ANY](double temp[$1_dim0]) {
  if (!convert_darray($input, temp, $1_dim0)) {
    SWIG_fail;
  }
  $1 = &temp[0];
}
```

### 33.9.6 Pointer handling

Occasionally, it might be necessary to convert pointer values that have been stored using the SWIG typed-pointer representation. Since there are several ways in which pointers can be represented, the following two functions are used to safely perform this conversion:

```
int SWIG_ConvertPtr(PyObject *obj, void **ptr, swig_type_info *ty, int flags)
```

Converts a Python object `obj` to a C pointer. The result of the conversion is placed into the pointer located at `ptr`. `ty` is a SWIG type descriptor structure. `flags` is used to handle error checking and other aspects of conversion. It is the bitwise-or of several flag values including `SWIG_POINTER_DISOWN` (which steals ownership of the object) and `SWIG_POINTER_NO_NULL` (which makes the conversion fail if the C pointer would be `NULL`). Returns 0 on success and -1 on error.

```
PyObject *SWIG_NewPointerObj(void *ptr, swig_type_info *ty, int own)
```

Creates a new Python pointer object. `ptr` is the pointer to convert, `ty` is the SWIG type descriptor structure that describes the type, and `own` is a flag that indicates whether or not Python should take ownership of the pointer.

Both of these functions require the use of a special SWIG type-descriptor structure. This structure contains information about the mangled name of the datatype, type-equivalence information, as well as information about converting pointer values under C++ inheritance. For a type of `Foo *`, the type descriptor structure is usually accessed as follows:

```
Foo *f;
if (!SWIG_IsOK(SWIG_ConvertPtr($input, (void **) &f, SWIGTYPE_p_Foo, 0))) {
  SWIG_exception_fail(SWIG_TypeError, "in method '$symname', expecting type Foo");
}

PyObject *obj;
obj = SWIG_NewPointerObj(f, SWIGTYPE_p_Foo, 0);
```

In a typemap, the type descriptor should always be accessed using the special typemap variable `$1_descriptor`. For example:

```
%typemap(in) Foo * {
  if (!SWIG_IsOK(SWIG_ConvertPtr($input, (void **) &$1, $1_descriptor, 0))) {
    SWIG_exception_fail(SWIG_TypeError, "in method '$symname', expecting type Foo");
  }
}
```

If necessary, the descriptor for any type can be obtained using the `$descriptor()` macro in a typemap. For example:

```
%typemap(in) Foo * {
  if (!SWIG_IsOK(SWIG_ConvertPtr($input, (void **) &$1, $descriptor(Foo *), 0))) {
    SWIG_exception_fail(SWIG_TypeError, "in method '$symname', expecting type Foo");
  }
}
```

Although the pointer handling functions are primarily intended for manipulating low-level pointers, both functions are fully aware of Python proxy classes. Specifically, `SWIG_ConvertPtr()` will retrieve a pointer from any object that has a `this` attribute. In addition, `SWIG_NewPointerObj()` can automatically generate a proxy class object (if applicable).

### 33.9.7 Memory management when returning references to member variables

This example shows how to prevent premature garbage collection of objects when the underlying C++ class returns a pointer or reference to a member variable. The example is a direct equivalent to this Java equivalent.

Consider the following C++ code:

```
#include <iostream>
struct Wheel {
  int size;
  Wheel(int sz = 0) : size(sz) {}
  ~Wheel() { std::cout << "~Wheel" << std::endl; }
};

class Bike {
  Wheel wheel;
public:
  Bike(int val) : wheel(val) {}
  Wheel& getWheel() { return wheel; }
};
```

and the following usage from Python after running the code through SWIG:

```
bike = Bike(10)
wheel = bike.getWheel()
print("wheel size: {}".format(wheel.size))

del bike  # Allow bike to be garbage collected
print("wheel size: {}".format(wheel.size))
```

Don't be surprised that if the resulting output gives strange results such as...

```
wheel size: 10
~Wheel
wheel size: 135019664
```

What has happened here is the garbage collector has collected the `Bike` instance as it doesn't think it is needed any more. The proxy instance, `wheel`, contains a reference to memory that was deleted when the `Bike` instance was collected. In order to prevent the garbage collector from collecting the `Bike` instance, a reference to the `Bike` must be added to the `wheel` instance.

You can do this by adding the reference when the `getWheel()` method is called using one of three approaches:

The easier, but less optimized, way is to use the `%pythonappend` directive (see Adding additional Python code ):

```
%pythonappend getWheel %{
  # val is the Wheel proxy, self is the Bike instance
  val.__bike_reference = self
%}
```

The code gets appended to the Python code generated for the `Bike::getWheel` wrapper function, where we store the `Bike` proxy instance onto the `Wheel` proxy instance before it is returned to the caller as follows.

```
class Bike(object):
    ...
    def getWheel(self):
        val = _example.Bike_getWheel(self)

        # val is the Wheel proxy, self is the Bike instance
        val.__bike_reference = self

        return val
```

The second option, which performs better and is required if you use the `-builtin` option, is to set the reference in the CPython implementation:

```
%extend Wheel {
// A reference to the parent class is added to ensure the underlying C++
// object is not deleted while the item is in use
%typemap(ret) Wheel& getWheel {
  PyObject *bike_reference_string = SWIG_Python_str_FromChar("__bike_reference");
  PyObject_SetAttr($result, bike_reference_string, $self);
  Py_DecRef(bike_reference_string);
}
}
```

The third approach, shown below, is an optimization of the above approach and creates the "__bike_reference" Python string object just once. While this looks more complex, it is just a small variation on the above typemap plus a support function `bike_reference()` in a fragment called `bike_reference_function`. The `bike_reference_init` typemap generates code into the "init" section for an initial call to `bike_reference()` when the module is initialized and is done to create the "__bike_reference" Python string singleton in a thread-safe manner.

```
%fragment("bike_reference_init", "init") {
  // Thread-safe initialization - initialize during Python module initialization
  bike_reference();
}

%fragment("bike_reference_function", "header", fragment="bike_reference_init") {

static PyObject *bike_reference() {
  static PyObject *bike_reference_string = SWIG_Python_str_FromChar("__bike_reference");
  return bike_reference_string;
}

}

%extend Wheel {
// A reference to the parent class is added to ensure the underlying C++
```

```
// object is not deleted while the item is in use
%typemap(ret, fragment="bike_reference_function") Wheel& getWheel %{
  PyObject_SetAttr($result, bike_reference(), $self);
%}
}
```

## 33.10 Docstring Features

Using docstrings in Python code is becoming more and more important and more tools are coming on the scene that take advantage of them, everything from full-blown documentation generators to class browsers and popup call-tips in Python-aware IDEs. Given the way that SWIG generates the proxy code by default, your users will normally get something like `"function_name(*args)"` in the popup calltip of their IDE which is next to useless when the real function prototype might be something like this:

```
bool function_name(int x, int y, Foo* foo=NULL, Bar* bar=NULL);
```

The features described in this section make it easy for you to add docstrings to your modules, functions and methods that can then be used by the various tools out there to make the programming experience of your users much simpler.

### 33.10.1 Module docstring

Python allows a docstring at the beginning of the `.py` file before any other statements, and it is typically used to give a general description of the entire module. SWIG supports this by setting an option of the `%module` directive. For example:

```
%module(docstring="This is the example module's docstring") example
```

When you have more than just a line or so then you can retain the easy readability of the `%module` directive by using a macro. For example:

```
%define DOCSTRING
"The `XmlResource` class allows program resources defining menus,
layout of controls on a panel, etc. to be loaded from an XML file."
%enddef

%module(docstring=DOCSTRING) xrc
```

### 33.10.2 %feature("autodoc")

As alluded to above SWIG will generate all the function and method proxy wrappers with just "*args" (or "*args, **kwargs" if the -keyword option is used) for a parameter list and will then sort out the individual parameters in the C wrapper code. This is nice and simple for the wrapper code, but makes it difficult to be programmer and tool friendly as anyone looking at the `.py` file will not be able to find out anything about the parameters that the functions accept.

But since SWIG does know everything about the function it is possible to generate a docstring containing the parameter types, names and default values. Since many of the docstring tools are adopting a standard of recognizing if the first thing in the docstring is a function prototype then using that instead of what they found from introspection, then life is good once more.

SWIG's Python module provides support for the "autodoc" feature, which when attached to a node in the parse tree will cause a docstring to be generated that includes the name of the function, parameter names, default values if any, and return type if any. There are also four levels for autodoc controlled by the value given to the feature, `%feature("autodoc", "`*`level`*`")`. The four values for *level* are covered in the following sub-sections.

#### 33.10.2.1 %feature("autodoc", "0")

When level "0" is used then the types of the parameters will *not* be included in the autodoc string. For example, given this function prototype:

```
%feature("autodoc", "0");
bool function_name(int x, int y, Foo* foo=NULL, Bar* bar=NULL);
```

Then Python code like this will be generated:

```
def function_name(*args, **kwargs):
    """function_name(x, y, foo=None, bar=None) -> bool"""
    ...
```

#### 33.10.2.2 %feature("autodoc", "1")

When level "1" is used then the parameter types *will* be used in the autodoc string. In addition, an attempt is made to simplify the type name such that it makes more sense to the Python user. Pointer, reference and const info is removed if the associated type is has an associated Python type (`%rename`'s are thus shown correctly). This works most of the time, otherwise a C/C++ type will be used. See the next section for the "docstring" feature for tweaking the docstrings to your liking. Given the example above, then turning on the parameter types with level "1" will result in Python code like this:

```
def function_name(*args, **kwargs):
    """function_name(int x, int y, Foo foo=None, Bar bar=None) -> bool"""
    ...
```

#### 33.10.2.3 %feature("autodoc", "2")

Level "2" results in the function prototype as per level "0". In addition, a line of documentation is generated for each parameter using numpydoc style. Using the previous example, the generated code will be:

```
def function_name(*args, **kwargs):
    """
    function_name(x, y, foo=None, bar=None) -> bool

    Parameters
    ----------
    x: int
    y: int
    foo: Foo *
    bar: Bar *

    """
```

```
    ...
```

Note that the documentation for each parameter is sourced from the "doc" typemap which by default shows the C/C++ type rather than the simplified Python type name described earlier for level "1". Typemaps can of course change the output for any particular type, for example the `int x` parameter:

```
%feature("autodoc", "2");
%typemap("doc") int x "$1_name (C++ type: $1_type) -- Input $1_name dimension"
bool function_name(int x, int y, Foo* foo=NULL, Bar* bar=NULL);
```

resulting in

```
def function_name(*args, **kwargs):
    """
    function_name(x, y, foo=None, bar=None) -> bool

    Parameters
    ----------
    x (C++ type: int) -- Input x dimension
    y: int
    foo: Foo *
    bar: Bar *

    """
```

#### 33.10.2.4 %feature("autodoc", "3")

Level "3" results in the function prototype as per level "1" but also contains the same additional line of documentation for each parameter as per level "2". Using our earlier example again, the generated code will be:

```
def function_name(*args, **kwargs):
    """
    function_name(int x, int y, Foo foo=None, Bar bar=None) -> bool

    Parameters
    ----------
    x: int
    y: int
    foo: Foo *
    bar: Bar *

    """
    ...
```

#### 33.10.2.5 %feature("autodoc", "docstring")

Finally, there are times when the automatically generated autodoc string will make no sense for a Python programmer, particularly when a typemap is involved. So if you give an explicit value for the autodoc feature then that string will be used in place of the automatically generated string. For example:

```
%feature("autodoc", "GetPosition() -> (x, y)") GetPosition;
void GetPosition(int* OUTPUT, int* OUTPUT);
```

### 33.10.3 %feature("docstring")

In addition to the autodoc strings described above, you can also attach any arbitrary descriptive text to a node in the parse tree with the "docstring" feature. When the proxy module is generated then any docstring associated with classes, function or methods are output. If an item already has an autodoc string then it is combined with the docstring and they are output together. If the docstring is all on a single line then it is output like this::

```
"""This is the docstring"""
```

Otherwise, to aid readability it is output like this:

```
"""
This is a multi-line docstring
with more than one line.
"""
```

### 33.10.4 Doxygen comments

Please see the separate Doxygen chapter for information on making use of C++ Doxygen comments and translating them into Python docstring comments.

Note that when generating docstrings and Doxygen comments have also been turned on, the docstring feature will take precedence over a Doxygen comment. If the autodoc feature is also turned on, then it will be used in conjunction with the docstring feature. However, if there is no docstring feature present and there is a Doxygen comment, then the autodoc docstring will not be generated. The Doxygen comment alone will be used.

This way, if the autodoc feature is specified globally it will fill in any missing Doxygen documentation comments. Doxygen comments can be overridden by using the docstring feature.

## 33.11 Python Packages

Python has concepts of modules and packages. Modules are separate units of code and may be grouped together to form a package. Packages may be nested, that is they may contain subpackages. This leads to tree-like hierarchy, with packages as intermediate nodes and modules as leaf nodes.

The hierarchy of Python packages/modules follows the hierarchy of `*.py` files found in a source tree (or, more generally, in the Python path). Normally, the developer creates new module by placing a `*.py` file somewhere under Python path; the module is then named after that `*.py` file. A package is created by placing an `__init__.py` file within a directory; the package is then named after that directory. For example, the following source tree:

```
mod1.py
pkg1/__init__.py
pkg1/mod2.py
```

```
pkg1/pkg2/__init__.py
pkg1/pkg2/mod3.py
```

defines the following Python packages and modules:

```
pkg1            # package
pkg1.pkg2       # package
mod1            # module
pkg1.mod2       # module
pkg1.pkg2.mod3  # module
```

The purpose of an `__init__.py` file is two-fold. First, the existence of `__init__.py` in a directory informs the Python interpreter that this directory contains a Python package. Second, the code in `__init__.py` is loaded/executed automatically when the package is initialized (when it or its submodule/subpackage gets `import`'ed). By default, SWIG generates proxy Python code – one `*.py` file for each `*.i` interface. The `__init__.py` files, however, are not generated by SWIG. They should be created by other means. Both files (module `*.py` and `__init__.py`) should be installed in appropriate destination directories in order to obtain a desirable package/module hierarchy.

Python3 adds another option for packages with PEP 0420 (implicit namespace packages). Implicit namespace packages no longer use __init__.py files. SWIG generated Python modules support implicit namespace packages. See Implicit namespace packages for more information.

You can place a SWIG generated module into a Python package or keep as a global module, details are covered a little later in Location of modules.

The way Python defines its modules and packages impacts SWIG users. Some users may need to use special features such as the `package` option in the `%module` directive or import related command line options. These are explained in the following sections.

### 33.11.1 Setting the Python package

Using the `package` option in the `%module` directive allows you to specify a Python package that the module will be in when installed.

```
%module(package="wx") xrc
```

This is useful when the `.i` file is `%import`ed by another `.i` file. By default SWIG will assume that the importer is able to find the importee with just the module name, but if they live in separate Python packages then this won't work. However if the importee specifies what its package is with the `%module` option then the Python code generated for the importer will use that package name when importing the other module and in base class declarations, etc..

SWIG assumes that the `package` option provided to `%module` together with the `module` name (that is, `wx.xrc` in the above example) forms a fully qualified (absolute) name of a module (in Python terms). This is important especially for Python 3, where absolute imports are used by default. It's up to you to place the generated module files (`.py`, `.so`) in appropriate subdirectories. For example, if you have an interface file `foo.i` with:

```
%module(package="pkg1.pkg2") foo
```

then the resulting directory layout should be

```
pkg1/
pkg1/__init__.py
pkg1/pkg2/__init__.py
pkg1/pkg2/foo.py        # (generated by SWIG)
pkg1/pkg2/_foo.so       # (shared library built from C/C++ code generated by SWIG)
```

### 33.11.2 Absolute and relative imports

Suppose, we have the following hierarchy of files:

```
pkg1/
pkg1/__init__.py
pkg1/mod2.py
pkg1/pkg2/__init__.py
pkg1/pkg2/mod3.py
```

Let the contents of `pkg1/pkg2/mod3.py` be

```
class M3: pass
```

We edit `pkg1/mod2.py` and want to import module `pkg1/pkg2/mod3.py` in order to derive from class `M3`. We can write appropriate Python code in several ways, for example:

1. Using "`import <>`" syntax with absolute package name:

   ```
   # pkg1/mod2.py
   import pkg1.pkg2.mod3
   class M2(pkg1.pkg2.mod3.M3): pass
   ```

2. Using "`import <>`" syntax with package name relative to `pkg1` (only in Python 2.7 and earlier):

   ```
   # pkg1/mod2.py
   import pkg2.mod3
   class M2(pkg2.mod3.M3): pass
   ```

3. Using "`from <> import <>`" syntax (relative import syntax, only in Python 2.5 and later):

   ```
   # pkg1/mod2.py
   from .pkg2 import mod3
   class M2(mod3.M3): pass
   ```

4. Other variants, for example the following construction in order to have the `pkg2.mod3.M3` symbol available in `mod2` as in point 2 above (but now under Python 3):

```
# pkg1/mod2.py
from . import pkg2
from .pkg2 import mod3
class M2(pkg2.mod3.M3): pass
```

Now suppose we have `mod2.i` with

```
// mod2.i
%module (package="pkg1") mod2
%import "mod3.i"
// ...
```

and `mod3.i` with

```
// mod3.i
%module (package="pkg1.pkg2") mod3
// ...
```

By default, SWIG will generate `mod2.py` proxy file with `import` directive as in point 1. This can be changed with the `-relativeimport` command line option. The `-relativeimport` instructs SWIG to organize imports as in point 4 for Python 2.7.0 and newer.

**Compatibility Note:** Versions of SWIG prior to SWIG-4.0.0 supported Python < 2.7.0 and would organize the imports as in point 2 if an older version of Python was detected at runtime.

In short, if you have `mod2.i` and `mod3.i` as above, then without `-relativeimport` SWIG will write

```
import pkg1.pkg2.mod3
```

to `mod2.py` proxy file, and with `-relativeimport` it will write

```
from . import pkg2
from .pkg2 import mod3
```

You should avoid using relative imports and use absolute ones whenever possible. There are some cases, however, when relative imports may be necessary. The first example is, when some (legacy) Python code refers entities imported by proxy files generated by SWIG, and it assumes that the proxy file uses relative imports. Second case is, when one puts import directives in `__init__.py` to import symbols from submodules or subpackages and the submodule depends on other submodules (discussed later).

### 33.11.3 Enforcing absolute import semantics

As you may know, there is an incompatibility in import semantics (for the `import <>` syntax) between Python 2 and 3. In Python 2.4 and earlier it is not clear whether

```
import foo
```

refers to a top-level module or to another module inside the current package. In Python 3 it always refers to a top-level module (see PEP 328). To instruct Python 2.5 through 2.7 to use new semantics (that is `import foo` is interpreted as absolute import), one has to put the following line

```
from __future__ import absolute_import
```

at the very beginning of your proxy`*.py` file. In SWIG, it may be accomplished with `%pythonbegin` directive as follows:

```
%pythonbegin %{
from __future__ import absolute_import
%}
```

### 33.11.4 Importing from __init__.py

Imports in `__init__.py` are handy when you want to populate a package's namespace with names imported from other modules. In SWIG based projects this approach may also be used to split large pieces of code into smaller modules, compile them in parallel and then re-assemble everything at runtime by importing submodules' contents in `__init__.py`, for example.

Unfortunately import directives in `__init__.py` may cause problems, especially if they refer to a package's submodules. This is caused by the way Python initializes packages. If you spot problems with imports from `__init__.py` try using `-relativeimport` option. Below we explain in detail one issue, for which the `-relativeimport` workaround may be helpful.

Consider the following example (Python 3):

```
pkg1/__init__.py        # (empty)
pkg1/pkg2/__init__.py   # (imports something from bar.py)
pkg1/pkg2/foo.py
pkg1/pkg2/bar.py        # (imports foo.py)
```

If the file contents are:

- `pkg1/pkg2/__init__.py:`

```
# pkg1/pkg2/__init__.py
from .bar import Bar
```

- `pkg1/pkg2/foo.py:`

```
# pkg1/pkg2/foo.py
class Foo: pass
```

- `pkg1/pkg2/bar.py:`

```
# pkg1/pkg2/bar.py
import pkg1.pkg2.foo
class Bar(pkg1.pkg2.foo.Foo): pass
```

Now if one simply used `import pkg1.pkg2`, it will usually fail:

```
>>> import pkg1.pkg2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "./pkg1/pkg2/__init__.py", line 2, in <module>
    from .bar import Bar
  File "./pkg1/pkg2/bar.py", line 3, in <module>
    class Bar(pkg1.pkg2.foo.Foo): pass
AttributeError: 'module' object has no attribute 'pkg2'
```

Surprisingly, if we execute the `import pkg1.pkg2` directive for the second time, it succeeds. The reason seems to be following: when Python spots the `from .bar import Bar` directive in `pkg1/pkg2/__init__.py` it starts loading `pkg1/pkg2/bar.py`. This module imports `pkg1.pkg2.foo` in turn and tries to use `pkg1.pkg2.foo.Foo`, but the package `pkg1` is not fully initialized yet (the initialization procedure is actually in progress) and it seems like the effect of the already seen `import pkg1.pkg2.pkg3.foo` is "delayed" or ignored. Exactly the same may happen to a proxy module generated by SWIG.

One workaround for this case is to use a relative import in `pkg1/pkg2/bar.py`. If we change `bar.py` to be:

```
from .pkg3 import foo
class Bar(foo.Foo): pass
```

or

```
from . import pkg3
from .pkg3 import foo
class Bar(pkg3.foo.Foo): pass
```

then the example works again. With SWIG, you need to enable the `-relativeimport` option in order to have the above workaround in effect (note, that the Python 2 case also needs the `-relativeimport` workaround).

### 33.11.5 Implicit namespace packages

Python 3.3 introduced PEP 0420 which implements implicit namespace packages. In a nutshell, implicit namespace packages remove the requirement of an __init__.py file and allow packages to be split across multiple PATH elements. For example:

```
/fragment1/pkg1/mod1.py
/fragment2/pkg1/mod2.py
/fragment3/pkg1/mod3.py
```

If PYTHONPATH is set to "/fragment1:/fragment2:/fragment3", then mod1, mod2 and mod3 will be part of pkg1. This allows for splitting of packages into separate pieces. This can be useful for SWIG generated wrappers in the following way.

Suppose you create a SWIG wrapper for a module called robin. The SWIG generated code consists of two files robin.py and _robin.so. You wish to make these modules part of a subpackage (brave.sir). With implicit namespace packages you can place these files in the following configurations:

Using PYTHONPATH="/some/path"

```
/some/path/brave/sir/robin.py
/some/path/brave/sir/_robin.so
```

Using PYTHONPATH="/some/path:/some/other/path"

```
/some/path/brave/sir/robin.py
/some/other/path/brave/sir/_robin.so
```

Finally suppose that your pure Python code is stored in a .zip file or some other way (database, web service connection, etc). Python can load the robin.py module using a custom importer. But the _robin.so module will need to be located on a file system. Implicit namespace packages make this possible. For example, using PYTHONPATH="/some/path/foo.zip:/some/other/path"

Contents of foo.zip

```
brave/
brave/sir/
brave/sir/robin.py
```

File system contents

```
/some/other/path/brave/sir/_robin.so
```

Support for implicit namespace packages was added to python-3.3. The zipimporter requires python-3.5.1 or newer to work with subpackages.

**Compatibility Note:** Support for implicit namespace packages was added in SWIG-3.0.9.

### 33.11.6 Location of modules

When SWIG creates wrappers from an interface file, say foo.i, two Python modules are created. There is a pure Python module (foo.py) and C/C++ code which is compiled and linked into a dynamically (or statically) loaded low-level module _foo (see the Preliminaries section for details). So, the interface file really defines two Python modules. How these two modules are loaded is covered next.

The pure Python module needs to load the C/C++ module in order to call the wrapped C/C++ methods. To do this it must make some assumptions about the location of the C/C++ module. There are two configurations that are supported by default.

1. Both modules in the same package

2. Both modules are global

Additional configurations are supported but require custom import code.

The following sub-sections look more closely at the two default configurations as well as some customized configurations. An input interface file, foo.i, results in the two modules foo.py and _foo.so for each of the configurations.

#### 33.11.6.1 Both modules in the same package

In this configuration, the pure Python module, foo.py, tries to load the C/C++ module, _foo, from the same package foo.py is located in. The package name is determined from the `__package__` attribute if available, see PEP 366. Otherwise it is derived from the`__name__` attribute given to foo.py by the Python loader that imported foo.py. The interface file for this configuration would contain:

```
%module(package="mypackage") foo
```

The location of the files could be as follows:

```
/dir/mypackage/foo.py
/dir/mypackage/__init__.py
/dir/mypackage/_foo.so
```

Assuming /dir/ is in PYTHONPATH, the module can be imported using

```
from mypackage import foo
```

#### 33.11.6.2 Both modules are global

In this configuration, there are no packages. If foo.py is not in a package, that is, it is a global module, then _foo is loaded as a global module. The interface file for this configuration would contain:

```
%module foo
```

The location of the files could be as follows:

```
/dir/foo.py
/dir/_foo.so
```

Assuming /dir/ is in PYTHONPATH, the module can be imported using

```
import foo
```

#### 33.11.6.3 Split modules custom configuration

In this non-standard 'split module' configuration, the pure Python module is in a package and the low level C/C++ module is global. This configuration is not generally recommended and is not supported by default as it needs a custom configuration. The module import code customization required is via the `moduleimport` attribute in the `%module` directive. The next sub-section elaborates further on this. The interface file for this split module configuration would contain:

```
%module(package="mypackage", moduleimport="import _foo") foo
```

When using `-builtin`, use the following instead (the reasons are also covered shortly in the next sub-section):

```
%module(package="mypackage", moduleimport="from _foo import *") foo
```

The location of the files could be as follows:

```
/dir/mypackage/foo.py
/dir/mypackage/__init__.py
/dir/_foo.so
```

Assuming /dir/ is in PYTHONPATH, the module can be imported using

```
from mypackage import foo
```

**Compatibility Note:** Versions of SWIG prior to SWIG-4.0.0 supported split modules without the above customization. However, this had to be removed as the default import code often led to confusion due to obfuscation of genuine Python `ImportError` problems. Using one of the two default configurations is the recommended approach now.

#### 33.11.6.4 More on customizing the module import code

The Python code implementing the default import logic is shown below. It supports the two configurations described earlier, that is, either both modules are in a package or loading both as global modules. The code is generated into the pure Python module, foo.py, and merely imports the low-level _foo module.

```
if __package__ or '.' in __name__:
    from . import _foo
else:
    import _foo
```

This import code implementation is non-trivial but it can be replaced with custom code providing opportunities to make it simpler and/or more flexible. This is not normally recommended though unless you have a good understanding of the intricacies of importing Python modules. The custom code can be specified by setting the `moduleimport` option of the `%module` directive with the appropriate import code. For example:

```
%module(moduleimport="import _foo") foo
```

This will replace the default import logic above and generate the following into the pure Python module, foo.py:

```
import _foo
```

In fact the above is a simplification customization for the configuration where both modules are global; it removes the logic for also handling the modules being in a package.

There is a special variable, `$module`, which is expanded into the low-level C/C++ module name, `_foo` in the case above. The identical output would be generated if instead the following had been used:

```
%module(moduleimport="import $module") foo
```

When you have many lines you can retain the easy readability of the `%module` directive by using a macro. For example:

```
%define MODULEIMPORT
"
print('Loading low-level module $module')
import $module
print('Module has loaded')
"
%enddef

%module(moduleimport=MODULEIMPORT) foo
```

This will of course generate the following into the pure Python module:

```
print('Loading low-level module $module')
import _foo
print('Module has loaded')
```

When using the `-builtin` option, the link between the pure Python module and the low-level C/C++ module is slightly different as all the objects from the low-level module are imported directly into the pure Python module. The default import loading code is thus different:

```
if __package__ or '.' in __name__:
    from ._foo import *
else:
    from _foo import *
```

Any customizations must import the code in a similar manner. The best way to support both with and without `-builtin` is to make use of the `SWIGPYTHON_BUILTIN` macro which is defined when `-builtin` is specified. The following will do this for the split modules case above.

```
#if defined(SWIGPYTHON_BUILTIN) /* defined when using -builtin */
%module(package="mypackage", moduleimport="from $module import *") foo
#else
%module(package="mypackage", moduleimport="import $module") foo
#endif
```

#### 33.11.6.5 Statically linked C modules

It is strongly recommended to use dynamically linked modules for the C portion of your pair of Python modules. If for some reason you still need to link the C module of the pair of Python modules generated by SWIG into your interpreter, then this section provides some details on how this impacts the pure Python modules ability to locate the other part of the pair. Please also see the Static Linking section.

When Python is extended with C code the Python interpreter needs to be informed about details of the new C functions that have been linked into the executable. The code to do this is created by SWIG and is automatically called in the correct way when the module is dynamically loaded. However when the code is not dynamically loaded (because it is statically linked) Then the initialization method for the module created by SWIG is not called automatically and the Python interpreter has no idea that the new SWIG C module exists.

Before Python 3, one could simply call the init method created by SWIG which would have normally been called when the shared object was dynamically loaded. The specific name of this method is not given here because statically linked modules are not encouraged with SWIG ( Static Linking). However one can find this init function in the C file generated by SWIG.

If you are really keen on static linking there are two ways to initialize the SWIG generated C module with the init method. Which way you use depends on what version of Python your module is being linked with. Python 2 and Python 3 treat this init function differently. And the way they treat it affects how the pure Python module will be able to locate the C module.

The details concerning this are covered completely in the documentation for Python itself. Links to the relevant sections follow:

- Extending in python2
- Extending in python3

There are two keys things to understand. The first is that in Python 2 the init() function returns void. In Python 3 the init() function returns a `PyObject *` which points to the new module. Secondly, when you call the init() method manually, you are the Python importer. So, you determine which package the C module will be located in.

So, if you are using Python 3 it is important that you follow what is described in the Python documentation linked above. In particular, you can't simply call the init() function generated by SWIG and cast the `PyObject` pointer it returns over the side. If you do then Python 3 will have no idea that your C module exists and the pure Python half of your wrapper will not be able to find it. You need to register your module with the Python interpreter as described in the Python docs.

With Python 2 things are somewhat more simple. In this case the init() function returns void. Calling it will register your new C module as a **global** module. The pure Python part of the SWIG wrapper will be able to find it because it tries both the pure Python module it is part of and the global module. If you wish not to have the statically linked module be a global module then you will either need to refer to the Python documentation on how to do this (remember you are now the Python importer) or use dynamic linking.

## 33.12 Python 3 Support

SWIG is able to support Python 3.x. The wrapper code generated by SWIG can be compiled with both Python 2.x or 3.x.

The list of known-to-be-broken features around Python 3 are:

- No more support for FILE* typemaps, because PyFile_AsFile has been dropped in Python 3.
- The `-apply` command line option is removed and generating code using apply() is no longer supported.

The following are Python 3 new features that are currently supported by SWIG.

### 33.12.1 Python function annotations and variable annotations

Python 3 supports function annotations as defined in PEP 3107. Python 3.6 and later additionally support variable annotations as defined in PEP 526. Note that currently there is no annotations support in SWIG for the `-builtin` nor the `-fastproxy` option. Annotations are added via the `python:annotations` %feature directives. SWIG currently supports one type of function annotation.

#### 33.12.1.1 C/C++ annotation types

The `%feature("python:annotations", "c")` directive generates annotations containing C/C++ types. For example:

```
%feature("python:annotations", "c") global_ints;
int *global_ints(int &ri);
```

The generated code then contains function annotations containing the C++ types:

```
def global_ints(ri: "int &") -> "int *":
    return _example.global_ints(ri)
```

There are some limitations with function annotations support, for example, overloaded functions use `*args` or `**kwargs` when keyword arguments are enabled. The parameter names and types are then not shown. For example, with input:

```
int *global_overloaded(int &ri);
int *global_overloaded();
```

The generated Python function including annotations is shown below. Only the return type is annotated.

```
def global_overloaded(*args) -> "int *":
    return _example.global_overloaded(*args)
```

Below is an example demonstrating variable annotations.

```
%feature("python:annotations", "c");

struct V {
  float val;
};
```

The generated code contains a variable annotation containing the C `float` type:

```
class V(object):
    val: "float" = property(_example.V_val_get, _example.V_val_set)
    ...
```

Variable annotations are only supported from Python 3.6. If you need to support earlier versions of Python, you'll need to turn variable annotations off via the `python:annotations:novar` feature flag. It is quite easy to support function annotations but turn off variable annotations. The next example shows how to do this for all variables.

```
%feature("python:annotations", "c");  // Turn on function annotations and variable annotations globally
%feature("python:annotations:novar"); // Turn off variable annotations globally

struct V {
  float val;
  void vv(float *v) const;
};
```

The resulting code will work with versions older than Python 3.6 as the variable annotations are turned off:

```
class V(object):
    val = property(_example.V_val_get, _example.V_val_set)

    def vv(self, v: "float *") -> "void":
        return _example.V_vv(self, v)
    ...
```

**Compatibility Note:** SWIG-4.1.0 changed the way that function annotations are generated. Prior versions required the (now removed) `-py3` option to generate function annotation support containing C/C++ types instead of supporting `%feature("python:annotations", "c")`. Variable annotations were also added in SWIG-4.1.0.

### 33.12.2 Buffer interface

SWIG has a series of typemaps to support buffer interfaces. These typemap macros are defined in `pybuffer.i`, which must be included in order to use them. By using these typemaps, your wrapped function will be able to accept any Python object that exposes a suitable buffer interface.

For example, the `get_path()` function puts the path string into the memory pointed to by its argument:

```
void get_path(char *s);
```

Then you can write a typemap like this: (the following example is applied to both Python 2 and 3, since the `bytearray` type was backported to 2.6.

```
%include <pybuffer.i>
%pybuffer_mutable_string(char *str);
void get_path(char *str);
```

And then on the Python side the wrapped `get_path` could be used in this way:

```
>>> p = bytearray(10)
>>> get_path(p)
```

```
>>> print(p)
bytearray(b'/Foo/Bar/\x00')
```

The macros defined in `pybuffer.i` are similar to those in `cstring.i`:

**%pybuffer_mutable_binary(parm, size_parm)**

The macro can be used to generate a typemap which maps a buffer of an object to a pointer provided by `parm` and a size argument provided by `size_parm`. For example:

```
%pybuffer_mutable_binary(char *str, size_t size);
...
int snprintf(char *str, size_t size, const char *format, ...);
```

In Python:

```
>>> buf = bytearray(6)
>>> snprintf(buf, "Hello world!")
>>> print(buf)
bytearray(b'Hello\x00')
>>>
```

**%pybuffer_mutable_string(parm)**

This typemap macro requires the buffer to be a zero terminated string, and maps the pointer of the buffer to `parm`. For example:

```
%pybuffer_mutable_string(char *str);
...
size_t make_upper(char *str);
```

In Python:

```
>>> buf = bytearray(b'foo\x00')
>>> make_upper(buf)
>>> print(buf)
bytearray(b'FOO\x00')
>>>
```

Both `%pybuffer_mutable_binary` and `%pybuffer_mutable_string` require the provided buffer to be mutable, eg. they can accept a `bytearray` type but can't accept an immutable `byte` type.

**%pybuffer_binary(parm, size_parm)**

This macro maps an object's buffer to a pointer `parm` and a size `size_parm`. It is similar to `%pybuffer_mutable_binary` , except the `%pybuffer_binary` can accept both mutable and immutable buffers. As a result, the wrapped function should not modify the buffer.

**%pybuffer_string(parm)**

This macro maps an object's buffer as a string pointer `parm` . It is similar to `%pybuffer_mutable_string` but the buffer could be both mutable and immutable. And your function should not modify the buffer.

### 33.12.3 Abstract base classes

By including `pyabc.i` in your interface file, the proxy classes of the STL containers will automatically gain an appropriate abstract base class from the `collections.abc` module for Python 3.3 and later, otherwise from the `collections` module. For example, the following SWIG interface:

```
%include <pyabc.i>
%include <std_map.i>
%include <std_list.i>

namespace std {
  %template(Mapii) map<int, int>;
  %template(IntList) list<int>;
}
```

will generate a Python proxy class `Mapii` inheriting from `collections.abc.MutableMap` for Python 3.3 and later, or `collections.MutableMap` for earlier versions and a proxy class `IntList` inheriting from `collections.abc.MutableSequence` for Python 3.3 or later, or `collections.MutableSequence` for earlier versions.

`pyabc.i` also provides a macro `%pythonabc` that could be used to define an abstract base class for your own C++ class:

```
%pythonabc(MySet, collections.abc.MutableSet); # Python 3.3 and later
%pythonabc(MySet, collections.MutableSet);     # Prior to Python 3.3
%pythonabc(MySet, "collections.abc.MutableSet if _swig_python_version_info >= (3, 3) else collections.MutableSet"); # All Python versions
```

For details of abstract base class, please see PEP 3119.

**Compatibility Note:** SWIG-4.0.0 changed the base classes to use the `collections.abc` module instead of `collections` due to the deprecation of the classes in the `collections` module in Python 3.7. The `collections.abc` module was introduced in Python 3.3 and hence this feature requires Python 3.3 or later. SWIG-4.1.0 introduced the flexibility of using either the `collections.abc` module for Python 3.3 and later or the `collections` module for earlier Python versions.

### 33.12.4 Byte string output conversion

By default, any byte string (`char*` or `std::string`) returned from C or C++ code is decoded to text as UTF-8. This decoding uses the `surrogateescape` error handler under Python 3.1 or higher -- this error handler decodes invalid byte sequences to high surrogate characters in the range U+DC80 to U+DCFF. As an example, consider the following SWIG interface, which exposes a byte string that cannot be completely decoded as UTF-8:

```
%module example

%inline %{

const char * non_utf8_c_str(void) {
```

```
  return "h\xe9llo w\xc3\xb6rld";
}

void instring(const char *s) {
  ...
}

%}
```

Note that "\xe9" is an invalid UTF-8 encoding, but "\xc3\xb6" is valid. When this method is called from Python 3, the return value is the following text string:

```
>>> s = example.non_utf8_c_str()
>>> s
'h\udce9llo wörld'
```

Since the C string contains bytes that cannot be decoded as UTF-8, those raw bytes are represented as high surrogate characters that can be used to obtain the original byte sequence:

```
>>> b = s.encode('utf-8', errors='surrogateescape')
>>> b
b'h\xe9llo w\xc3\xb6rld'
```

One can then attempt a different encoding, if desired (or simply leave the byte string as a raw sequence of bytes for use in binary protocols):

```
>>> b.decode('latin-1')
'héllo wÃ¶rld'
```

Note, however, that text strings containing surrogate characters are rejected with the default `strict` codec error handler. For example:

```
>>> with open('test', 'w') as f:
...     print(s, file=f)
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
UnicodeEncodeError: 'utf-8' codec can't encode character '\udce9' in position 1: surrogates not allowed
```

This requires the user to check most strings returned by SWIG bindings, but the alternative is for a non-UTF8 byte string to be completely inaccessible in Python 3 code.

For more details about the `surrogateescape` error handler, please see PEP 383 .

When Python 3 strings are passed to the C/C++ layer, they are expected to be valid UTF8 Unicode strings too. For example, when the `instring` method above is wrapped and called, any invalid UTF8 Unicode code strings will result in a TypeError because the attempted conversion fails:

```
>>> example.instring('h\xe9llo')
>>> example.instring('h\udce9llo')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: in method 'instring', argument 1 of type 'char const *'
```

In some cases, users may wish to instead handle all byte strings as bytes objects in Python 3. This can be accomplished by adding `SWIG_PYTHON_STRICT_BYTE_CHAR` to the generated code:

```
%module char_to_bytes
%begin %{
#define SWIG_PYTHON_STRICT_BYTE_CHAR
%}

char *charstring(char *s) {
  return s;
}
```

This will modify the behavior so that only Python 3 bytes objects will be accepted and converted to a C/C++ string, and any string returned from C/C++ will be converted to a bytes object in Python 3:

```
>>> from char_to_bytes import *
>>> charstring(b"hi") # Byte string
b'hi'
>>> charstring("hi")  # Unicode string
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
TypeError: in method 'charstring', argument 1 of type 'char *'
```

Note that in Python 2, defining `SWIG_PYTHON_STRICT_BYTE_CHAR` has no effect, since strings in Python 2 are equivalent to Python 3 bytes objects. However, there is a similar capability to force unicode-only handling for wide characters C/C++ strings (`wchar_t *` or `std::wstring` types) in Python 2. By default, in Python 2 both strings and unicode strings are converted to C/C++ wide strings, and returned wide strings are converted to a Python unicode string. To instead only convert unicode strings to wide strings, users can add `SWIG_PYTHON_STRICT_UNICODE_WCHAR` to the generated code:

```
%module wchar_to_unicode
%begin %{
#define SWIG_PYTHON_STRICT_UNICODE_WCHAR
%}

wchar_t *wcharstring(wchar_t *s) {
  return s;
}
```

This ensures that only unicode strings are accepted by wcharstring in both Python 2 and Python 3:

```
>>> from wchar_to_unicode import *
>>> wcharstring(u"hi") # Unicode string
u'hi'
>>> wcharstring(b"hi") # Byte string
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
TypeError: in method 'charstring', argument 1 of type 'wchar_t *'
```

By defining both `SWIG_PYTHON_STRICT_BYTE_CHAR` and `SWIG_PYTHON_STRICT_UNICODE_WCHAR`, Python wrapper code can support overloads taking both std::string (as Python bytes) and std::wstring (as Python unicode).

#### 33.12.5 Python 2 Unicode

A Python 3 string is a Unicode string so by default a Python 3 string that contains Unicode characters passed to C/C++ will be accepted and converted to a C/C++ string (`char *` or `std::string` types). A Python 2 string is not a unicode string by default and should a Unicode string be passed to C/C++ it will fail to convert to a C/C++ string (`char *` or `std::string` types). The Python 2 behavior can be made more like Python 3 by defining `SWIG_PYTHON_2_UNICODE` when compiling the generated C/C++ code. By default when the following is wrapped:

```
%module unicode_strings
char *charstring(char *s) {
  return s;
}
```

An error will occur when using Unicode strings in Python 2:

```
>>> from unicode_strings import *
>>> charstring("hi")
'hi'
>>> charstring(u"hi")
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
TypeError: in method 'charstring', argument 1 of type 'char *'
```

When the `SWIG_PYTHON_2_UNICODE` macro is added to the generated code:

```
%module unicode_strings
%begin %{
#define SWIG_PYTHON_2_UNICODE
%}

char *charstring(char *s) {
  return s;
}
```

Unicode strings will be successfully accepted and converted from UTF-8, but note that they are returned as a normal Python 2 string:

```
>>> from unicode_strings import *
>>> charstring("hi")
'hi'
>>> charstring(u"hi")
'hi'
>>>
```

Note that defining both `SWIG_PYTHON_2_UNICODE` and `SWIG_PYTHON_STRICT_BYTE_CHAR` at the same time is not allowed, since the first is allowing unicode conversion and the second is explicitly prohibiting it.

## 33.13 Support for Multithreaded Applications

By default, SWIG does not enable support for multithreaded Python applications. More specifically, the Python wrappers generated by SWIG will not release the Python's interpreter's Global Interpreter Lock (GIL) when wrapped C/C++ code is entered. Hence, while any of the wrapped C/C++ code is executing, the Python interpreter will not be able to run any other threads, even if the wrapped C/C++ code is waiting in a blocking call for something like network or disk IO. Fortunately, SWIG does have the ability to enable multithreaded support and automatic release of the GIL either for all wrapped code in a module or on a more selective basis. The user interface for this is described in the next section.

#### 33.13.1 UI for Enabling Multithreading Support

The user interface is as follows:

1. Module thread support can be enabled in two ways:
   - The `-threads` SWIG Python option at the command line (or in `setup.py`):

     ```
     $ swig -python -threads example.i
     ```

   - The `threads` module option in the *.i template file:

     ```
     %module(threads="1")
     ```

2. You can disable thread support for a given method:

   ```
   %feature("nothread") method;
   ```

   or

   ```
   %nothread method;
   ```

3. You can partially disable thread support for a given method:
   - To disable the C++/Python thread protection:

```
%feature("nothreadblock") method;
```

or

```
%nothreadblock method;
```

- To disable the Python/C++ thread protection

```
%feature("nothreadallow") method;
```

or

```
%nothreadallow method;
```

### 33.13.2 Multithread Performance

For the curious about performance, here are some numbers for the profiletest.i test, which is used to check the speed of the wrapped code:

| Thread Mode | Execution Time (sec) | Comment |
|---|---|---|
| Single Threaded | 9.6 | no "-threads" option given |
| Fully Multithreaded | 15.5 | "-threads" option = 'allow' + 'block' |
| No Thread block | 12.2 | only 'allow' |
| No Thread Allow | 13.6 | only block' |

Fully threaded code decreases the wrapping performance by around 60%. If that is important to your application, you can tune each method using the different 'nothread', 'nothreadblock' or 'nothreadallow' features as needed. Note that for some methods deactivating the 'thread block' or 'thread allow' code is not an option, so, be careful.

---

# 34 SWIG and R


- Bugs
- Using R and SWIG
- Precompiling large R files
- General policy
- Language conventions
- C++ classes
  - Examples
- Enumerations


R is a GPL'ed open source statistical and plotting environment. Information about R can be found at www.r-project.org.

The R bindings are under active development. They have been used to compile and run an R interface to QuantLib running on Mandriva Linux with gcc. They are also used to create the SimpleITK R package, which runs on Linux and MacOS. SWIG is used to create all wrapper interfaces to SimpleITK. The R bindings also work on Microsoft Windows using Visual C++.

## 34.1 Bugs

Currently the following features are not implemented or broken:

- Garbage collection of some created objects. Finalizers are available for wrapped C++ classes and are called by the garbage collection system.
- C Array wrappings

## 34.2 Using R and SWIG

To use R and SWIG in C mode, execute the following commands where example.c is the name of the file with the functions in them

```
swig -r example.i
R CMD SHLIB example_wrap.c example.c
```

The corresponding options for C++ mode are

```
swig -c++ -r -o example_wrap.cpp example.i
R CMD SHLIB example_wrap.cpp example.cpp
```

Note that R is sensitive to the names of the files. The name of the wrapper file must be the name of the library unless you use the -o option to R when building the library, for example:

```
swig -c++ -r -o example_wrap.cpp example.i
R CMD SHLIB -o example.so example_wrap.cpp example.cpp
```

R is also sensitive to the name of the file extension in C and C++ mode. In C++ mode, the file extension must be .cpp rather than .cxx for the R compile command to recognize it. If your C++ code is in a file using something other than a .cpp extension, then it may still work using PKG_LIBS:

```
swig -c++ -r -o example_wrap.cpp example.i
PKG_LIBS="example.cxx" R CMD SHLIB -o example example_wrap.cpp
```

The commands produces two files. A dynamic shared object file called example.so, or example.dll, and an R wrapper file called example.R. To load these files, start up R and type in the following commands

```
dyn.load(paste("example", .Platform$dynlib.ext, sep=""))
source("example.R")
cacheMetaData(1)
```

The cacheMetaData(1) will cause R to refresh its object tables. Without it, inheritance of wrapped objects may fail. These two files can be loaded in any order.

If you are compiling code yourself (not using R itself), there are a few things to watch out for:

- The output shared library name (to the left of the file extension) MUST match the module name, or alternatively, you can also set the -package NAME command line argument. See swig -r -help for more information
- If you do not set the output file name appropriately, you might see errors like

```
> fact(4)
Error in .Call("R_swig_fact", s_arg1, as.logical(.copy), PACKAGE = "example") :
  "R_swig_fact" not available for .Call() for package "example"
```

- Make sure the architecture of the shared library(x64 for instance), matches the architecture of the R program you want to load your shared library into

## 34.3 Precompiling large R files

In cases where the R file is large, one make save a lot of loading time by precompiling the R wrapper. This can be done by creating the file makeRData.R which contains the following

```
source('BigFile.R')
save(list=ls(all=TRUE), file="BigFile.RData", compress=TRUE)
q(save="no")
```

This will generate a compiled R file called BigFile.RData that will save a large amount of loading time.

There is no need to precompile large R files if the SWIG-generated code is being included in an R package. The package infrastructure provides this service during package installation.

## 34.4 General policy

The general policy of the module is to treat the C/C++ as a basic wrapping over the underlying functions and rely on the R type system to provide R syntax.

## 34.5 Language conventions

getitem and setitem use C++ conventions (i.e. zero based indices). [<- and [ are overloaded to allow for R syntax (one based indices and slices)

## 34.6 C++ classes

Wrapping of C++ classes for R works quite well. R has a special type, known as an external reference, that can be used as a pointer to arbitrary things, including C++ classes. The proxy layers generated for other classes are not required.

SWIG currently creates a custom hierarchy of R classes derived from the external reference type and implements type checking and function overloading in the R code it generates. In the future we hope to utilise the built in R6 class structures.

The R interface has the following capabilities:

- Destructor methods are registered and called automatically by the R garbage collector.
- A range of std::vector types are converted automatically to R equivalents via the std_vector.i library.
- The $ operator is used for method access.
- Variable accessors are automatically generated and called via the $, [, [[, $<-, [<-, [[<- operators.

### 34.6.1 Examples

Consider the following simple example:

```
class Vehicle {
private:
  int m_axles;

public:
  int Axles() {
    return(m_axles);
  }

  bool Available;

  Vehicle() {
    Available=false;
    m_axles=2;
  }

  Vehicle(int ax) {
    Available=false;
    m_axles=ax;
  }
};
```

The following options are available in R:

```
v1 <- Vehicle()
v2 <- Vehicle(4)
# access members
v1$Axles()
[1] 2
v2$Axles
[1] 4
v1$Available
[1] FALSE
# Set availability
v1$Available <- TRUE
v1$Available
[1] TRUE
```

A useful trick to determine the methods that are available is to query the R method definition as follows:

```
# display the methods for the class
getMethod("$", class(v1))
```

```
Method Definition:

function (x, name)
{
    accessorFuns = list(Axles = Vehicle_Axles, Available = Vehicle_Available_get)
    vaccessors = c("Available")
    idx = pmatch(name, names(accessorFuns))
    if (is.na(idx))
        return(callNextMethod(x, name))
    f = accessorFuns[[idx]]
    if (is.na(match(name, vaccessors)))
        function(...) {
            f(x, ...)
        }
    else f(x)
}

Signatures:
        x
target  "_p_Vehicle"
defined "_p_Vehicle"
```

The names in the `accessorFuns` list correspond to class methods while names in the `vaccessors` section correspond to variables that may be modified.

### 34.7 Enumerations

R doesn't have a native enumeration type. Enumerations are represented as character strings in R, with calls to R functions that convert back and forth between integers.

The details of enumeration names and contents are stored in hidden R environments, which are named according to the enumeration name - for example, an enumeration colour:

```
enum colour { red=-1, blue, green = 10 };
```

will be initialized by the following call in R:

```
defineEnumeration("_colour",
 .values=c("red" = .Call('R_swig_colour_red_get',FALSE, PACKAGE='enum_thorough'),
"blue" = .Call('R_swig_colour_blue_get',FALSE, PACKAGE='enum_thorough'),
"green" = .Call('R_swig_colour_green_get',FALSE, PACKAGE='enum_thorough')))
```

which will create an environment named `.__E___colour`. The enumeration values are initialised via calls to C/C++ code, allowing complex values for enumerations to be used. Calls to the C/C++ code require the compiled library to be loaded, so a `delayedAssign` is employed within `defineEnumeration` in order to allow the code to be easily used in R packages.

The user typically does not need to access the enumeration lookup functions or know the name of the enumeration type used by R. Attributes containing the type information are attached by swig to functions requiring enumeration arguments or returning enumeration values, and those attributes are used to identify and access the appropriate environments and thus translate between characters and integers.

The relevant functions, for debugging purposes, are `enumToInteger` and `enumFromInteger`.

Anonymous enumerations are ignored by the binding generation process, leaving no way of accessing the value of anonymous enumerations from R code.

## 35 SWIG and Ruby


- Preliminaries
  - Running SWIG
  - Getting the right header files
  - Compiling a dynamic module
  - Using your module
  - Static linking
  - Compilation of C++ extensions
- Building Ruby Extensions under Windows 95/NT
  - Running SWIG from Developer Studio
- The Ruby-to-C/C++ Mapping
  - Modules
  - Functions
  - Variable Linking
  - Constants
  - Pointers
  - Structures
  - C++ classes
  - C++ Inheritance
  - C++ Overloaded Functions
  - C++ Operators
  - C++ namespaces
  - C++ templates
  - C++ Standard Template Library (STL)
  - C++ STL Functors
  - C++ STL Iterators
  - C++ Smart Pointers
    - The shared_ptr Smart Pointer
    - Generic Smart Pointers
  - Cross-Language Polymorphism
    - Exception Unrolling
- Naming
  - Defining Aliases
  - Predicate Methods
  - Bang Methods
  - Getters and Setters
- Input and output parameters
- Exception handling
  - Using the %exception directive
  - Handling Ruby Blocks
  - Raising exceptions

- Exception classes
- Typemaps
  - What is a typemap?
  - Typemap scope
  - Copying a typemap
  - Deleting a typemap
  - Placement of typemaps
  - Ruby typemaps
    - "in" typemap
    - "typecheck" typemap
    - "out" typemap
    - "arginit" typemap
    - "default" typemap
    - "check" typemap
    - "argout" typemap
    - "freearg" typemap
    - "newfree" typemap
    - "memberin" typemap
    - "varin" typemap
    - "varout" typemap
    - "throws" typemap
    - directorin typemap
    - directorout typemap
    - directorargout typemap
    - ret typemap
    - globalin typemap
  - Typemap variables
  - Useful Functions
    - C Datatypes to Ruby Objects
    - Ruby Objects to C Datatypes
    - Macros for VALUE
    - Exceptions
    - Iterators
  - Typemap Examples
  - Converting a Ruby array to a char **
  - Collecting arguments in a hash
  - Pointer handling
    - Ruby Datatype Wrapping
  - Example: STL Vector to Ruby Array
- Docstring Features
  - Module docstring
  - %feature("autodoc")
    - %feature("autodoc", "0")
    - %feature("autodoc", "1")
    - %feature("autodoc", "2")
    - %feature("autodoc", "3")
    - %feature("autodoc", "docstring")
  - %feature("docstring")
- Advanced Topics
  - Operator overloading
  - Creating Multi-Module Packages
  - Specifying Mixin Modules
- Memory Management
  - Mark and Sweep Garbage Collector
  - Object Ownership
  - Object Tracking
  - Mark Functions
  - Free Functions
  - Embedded Ruby and the C++ Stack


This chapter describes SWIG's support of Ruby.

## 35.1 Preliminaries

SWIG 4.2 is known to work with Ruby versions 2.0 and later. Given the choice, you should use the latest stable version of Ruby. You should also determine if your system supports shared libraries and dynamic loading. SWIG will work with or without dynamic loading, but the compilation process will vary.

This chapter covers most SWIG features, but in less depth than is found in earlier chapters. At the very least, make sure you also read the "SWIG Basics" chapter. It is also assumed that the reader has a basic understanding of Ruby.

### 35.1.1 Running SWIG

To build a Ruby module, run SWIG using the `-ruby` option:

```
$ swig -ruby example.i
```

If building a C++ extension, add the `-c++` option:

```
$ swig -c++ -ruby example.i
```

This creates a file `example_wrap.c` (`example_wrap.cxx` if compiling a C++ extension) that contains all of the code needed to build a Ruby extension module. To finish building the module, you need to compile this file and link it with the rest of your program.

### 35.1.2 Getting the right header files

In order to compile the wrapper code, the compiler needs the `ruby.h` header file and its dependencies, notably `ruby/config.h` which is found in a different, architecture-dependent, directory. The best way to find the compiler options needed to compile the code is to ask Ruby itself:

```
$ ruby -rrbconfig -e 'puts "-I#{RbConfig::CONFIG[%q{rubyhdrdir}]} -I#{RbConfig::CONFIG[%q{rubyarchhdrdir}]}"'
-I/usr/include/ruby-2.1.0 -I/usr/include/x86_64-linux-gnu/ruby-2.1.0
```

### 35.1.3 Compiling a dynamic module

Ruby extension modules are typically compiled into shared libraries that the interpreter loads dynamically at runtime. Since the exact commands for doing this vary from platform to platform,

your best bet is to follow the steps described in the `README.EXT` file from the Ruby distribution:

1. Create a file called `extconf.rb` that looks like the following:

```
require 'mkmf'
create_makefile('example')
```

2. Type the following to build the extension:

```
$ ruby extconf.rb
$ make
$ make install
```

Of course, there is the problem that mkmf does not work correctly on all platforms, e.g, HPUX. If you need to add your own make rules to the file that `extconf.rb` produces, you can add this:

```
open("Makefile", "a") { |mf|
 puts <<EOM
 # Your make rules go here
 EOM
}
```

to the end of the `extconf.rb` file. If for some reason you don't want to use the standard approach, you'll need to determine the correct compiler and linker flags for your build platform. For example, assuming you have code you need to link to in a file called `example.c`, a typical sequence of commands for the Linux operating system would look something like this:

```
$ swig -ruby example.i
$ gcc -O2 -fPIC -c example.c
$ gcc -O2 -fPIC -c example_wrap.c -I/usr/include/ruby-2.1.0
$ gcc -shared example.o example_wrap.o -o example.so
```

The -fPIC option tells GCC to generate position-independent code (PIC) which is required for most architectures (it's not vital on x86, but still a good idea as it allows code pages from the library to be shared between processes). Other compilers may need a different option specified instead of -fPIC.

If in doubt, consult the manual pages for your compiler and linker to determine the correct set of options. You might also check the SWIG Wiki for additional information.

### 35.1.4 Using your module

Ruby *module* names must be capitalized, but the convention for Ruby *feature* names is to use lowercase names. So, for example, the **Etc** extension module is imported by requiring the**etc** feature:

```
# The feature name begins with a lowercase letter...
require 'etc'

# ... but the module name begins with an uppercase letter
puts "Your login name: #{Etc.getlogin}"
```

To stay consistent with this practice, you should always specify a **lowercase** module name with SWIG's `%module` directive. SWIG will automatically correct the resulting Ruby module name for your extension. So for example, a SWIG interface file that begins with:

```
%module example
```

will result in an extension module using the feature name "example" and Ruby module name "Example".

### 35.1.5 Static linking

An alternative approach to dynamic linking is to rebuild the Ruby interpreter with your extension module added to it. In the past, this approach was sometimes necessary due to limitations in dynamic loading support on certain machines. However, the situation has improved greatly over the last few years and you should not consider this approach unless there is really no other option.

The usual procedure for adding a new module to Ruby involves finding the Ruby source, adding an entry to the `ext/Setup` file, adding your directory to the list of extensions in the file, and finally rebuilding Ruby.

### 35.1.6 Compilation of C++ extensions

On most machines, C++ extension modules should be linked using the C++ compiler. For example:

```
$ swig -c++ -ruby example.i
$ g++ -fPIC -c example.cxx
$ g++ -fPIC -c example_wrap.cxx -I/usr/include/ruby-2.1.0
$ g++ -shared example.o example_wrap.o -o example.so
```

If you've written an `extconf.rb` script to automatically generate a `Makefile` for your C++ extension module, keep in mind that (as of this writing) Ruby still uses `gcc` and not `g++` as its linker. As a result, the required C++ runtime library support will not be automatically linked into your extension module and it may fail to load on some platforms. A workaround for this problem is use the `mkmf` module's `append_library()` method to add one of the C++ runtime libraries to the list of libraries linked into your extension, e.g.

```
require 'mkmf'
$libs = append_library($libs, "supc++")
create_makefile('example')
```

## 35.2 Building Ruby Extensions under Windows 95/NT

Building a SWIG extension to Ruby under Windows 95/NT is roughly similar to the process used with Unix. Normally, you will want to produce a DLL that can be loaded into the Ruby interpreter. For all recent versions of Ruby, the procedure described above (i.e. using an `extconf.rb` script) will work with Windows as well; you should be able to build your code into a DLL by typing:

```
C:\swigtest> ruby extconf.rb
```

```
C:\swigtest> nmake
C:\swigtest> nmake install
```

The remainder of this section covers the process of compiling SWIG-generated Ruby extensions with Microsoft Visual C++ 6 (i.e. within the Developer Studio IDE, instead of using the command line tools). In order to build extensions, you may need to download the source distribution to the Ruby package, as you will need the Ruby header files.

### 35.2.1 Running SWIG from Developer Studio

If you are developing your application within Microsoft developer studio, SWIG can be invoked as a custom build option. The process roughly follows these steps :

- Open up a new workspace and use the AppWizard to select a DLL project.
- Add both the SWIG interface file (the .i file), any supporting C files, and the name of the wrapper file that will be created by SWIG (i.e. `example_wrap.c`). Note : If using C++, choose a different suffix for the wrapper file such as `example_wrap.cxx`. Don't worry if the wrapper file doesn't exist yet--Developer Studio will keep a reference to it around.
- Select the SWIG interface file and go to the settings menu. Under settings, select the "Custom Build" option.
- Enter "SWIG" in the description field.
- Enter "`swig -ruby -o $(ProjDir)\$(InputName)_wrap.c $(InputPath)`" in the "Build command(s) field". You may have to include the path to swig.exe.
- Enter "`$(ProjDir)\$(InputName)_wrap.c`" in the "Output files(s) field".
- Next, select the settings for the entire project and go to the C/C++ tab and select the Preprocessor category. Add NT=1 to the Preprocessor definitions. This must be set else you will get compilation errors. Also add IMPORT to the preprocessor definitions, else you may get runtime errors. Also add the include directories for your Ruby installation under "Additional include directories".
- Next, select the settings for the entire project and go to the Link tab and select the General category. Set the name of the output file to match the name of your Ruby module (i.e.. example.dll). Next add the Ruby library file to your link libraries under Object/Library modules. For example "mswin32-ruby16.lib. You also need to add the path to the library under the Input tab - Additional library path.
- Build your project.

Now, assuming all went well, SWIG will be automatically invoked when you build your project. Any changes made to the interface file will result in SWIG being automatically invoked to produce a new version of the wrapper file. To run your new Ruby extension, simply run Ruby and use the `require` command as normal. For example if you have this ruby file run.rb:

```
# file: run.rb
require 'Example'

# Call a c function
print "Foo = ", Example.Foo, "\n"
```

Ensure the dll just built is in your path or current directory, then run the Ruby script from the DOS/Command prompt:

```
C:\swigtest> ruby run.rb
Foo = 3.0
```

## 35.3 The Ruby-to-C/C++ Mapping

This section describes the basics of how SWIG maps C or C++ declarations in your SWIG interface files to Ruby constructs.

### 35.3.1 Modules

The SWIG `%module` directive specifies the name of the Ruby module. If you specify:

```
%module example
```

then everything is wrapped into a Ruby module named `Example` that is nested directly under the global module. You can specify a more deeply nested module by specifying the fully-qualified module name in quotes, e.g.

```
%module "foo::bar::spam"
```

An alternate method of specifying a nested module name is to use the `-prefix` option on the SWIG command line. The prefix that you specify with this option will be prepended to the module name specified with the `%module` directive in your SWIG interface file. So for example, this declaration at the top of your SWIG interface file:

```
%module "foo::bar::spam"
```

will result in a nested module name of `Foo::Bar::Spam`, but you can achieve the *same* effect by specifying:

```
%module spam
```

and then running SWIG with the `-prefix` command line option:

```
$ swig -ruby -prefix "foo::bar::" example.i
```

Starting with SWIG 1.3.20, you can also choose to wrap everything into the global module by specifying the `-globalmodule` option on the SWIG command line, i.e.

```
$ swig -ruby -globalmodule example.i
```

Note that this does not relieve you of the requirement of specifying the SWIG module name with the `%module` directive (or the `-module` command-line option) as described earlier.

When choosing a module name, do not use the same name as a built-in Ruby command or standard module name, as the results may be unpredictable. Similarly, if you're using the `-globalmodule` option to wrap everything into the global module, take care that the names of your constants, classes and methods don't conflict with any of Ruby's built-in names.

### 35.3.2 Functions

Global functions are wrapped as Ruby module methods. For example, given the SWIG interface file `example.i`:

```
%module example

int fact(int n);
```

and C source file `example.c`:

```
int fact(int n) {
  if (n == 0)
  return 1;
  return (n * fact(n-1));
}
```

SWIG will generate a method *fact* in the *Example* module that can be used like so:

```
$ irb
irb(main):001:0> require 'example'
true
irb(main):002:0> Example.fact(4)
24
```

### 35.3.3 Variable Linking

C/C++ global variables are wrapped as a pair of singleton methods for the module: one to get the value of the global variable and one to set it. For example, the following SWIG interface file declares two global variables:

```
// SWIG interface file with global variables
%module example
...
%inline %{
  extern int variable1;
  extern double Variable2;
%}
...
```

Now look at the Ruby interface:

```
$ irb
irb(main):001:0> require 'Example'
true
irb(main):002:0> Example.variable1 = 2
2
irb(main):003:0> Example.Variable2 = 4 * 10.3
41.2
irb(main):004:0> Example.Variable2
41.2
```

If you make an error in variable assignment, you will receive an error message. For example:

```
irb(main):005:0> Example.Variable2 = "hello"
TypeError: no implicit conversion to float from string
from (irb):5:in `Variable2='
from (irb):5
```

If a variable is declared as `const`, it is wrapped as a read-only variable. Attempts to modify its value will result in an error.

To make ordinary variables read-only, you can also use the `%immutable` directive. For example:

```
%immutable;
%inline %{
  extern char *path;
%}
%mutable;
```

The `%immutable` directive stays in effect until it is explicitly disabled using `%mutable`.

Note: When SWIG is invoked with the `-globalmodule` option in effect, the C/C++ global variables will be translated into Ruby global variables. Type-checking and the optional read-only characteristic are available in the same way as described above. However the example would then have to be modified and executed in the following way:

```
$ irb
irb(main):001:0> require 'Example'
true
irb(main):002:0> $variable1 = 2
2
irb(main):003:0> $Variable2 = 4 * 10.3
41.2
irb(main):004:0> $Variable2
41.2
```

### 35.3.4 Constants

C/C++ constants are wrapped as module constants initialized to the appropriate value. To create a constant, use `#define` or the `%constant` directive. For example:

```
#define PI 3.14159
#define VERSION "1.0"

%constant int FOO = 42;
%constant const char *path = "/usr/local";

const int BAR = 32;
```

Remember to use the :: operator in Ruby to get at these constant values, e.g.

```
$ irb
irb(main):001:0> require 'Example'
true
irb(main):002:0> Example::PI
3.14159
```

### 35.3.5 Pointers

"Opaque" pointers to arbitrary C/C++ types (i.e. types that aren't explicitly declared in your SWIG interface file) are wrapped as data objects. So, for example, consider a SWIG interface file containing only the declarations:

```
Foo *get_foo();
void set_foo(Foo *foo);
```

For this case, the *get_foo()* method returns an instance of an internally generated Ruby class:

```
irb(main):001:0> foo = Example::get_foo()
#<SWIG::TYPE_p_Foo:0x402b1654>
```

A `NULL` pointer is always represented by the Ruby `nil` object.

### 35.3.6 Structures

C/C++ structs are wrapped as Ruby classes, with accessor methods (i.e. "getters" and "setters") for all of the struct members. For example, this struct declaration:

```
struct Vector {
  double x, y;
};
```

gets wrapped as a `Vector` class, with Ruby instance methods `x`, `x=`, `y` and `y=`. These methods can be used to access structure data from Ruby as follows:

```
$ irb
irb(main):001:0> require 'Example'
true
irb(main):002:0> f = Example::Vector.new
#<Example::Vector:0x4020b268>
irb(main):003:0> f.x = 10
nil
irb(main):004:0> f.x
10.0
```

Similar access is provided for unions and the public data members of C++ classes.

`const` members of a structure are read-only. Data members can also be forced to be read-only using the `%immutable` directive (in C++, `private` may also be used). For example:

```
struct Foo {
  ...
  %immutable;
  int x; /* Read-only members */
  char *name;
  %mutable;
  ...
};
```

When `char *` members of a structure are wrapped, the contents are assumed to be dynamically allocated using `malloc` or `new` (depending on whether or not SWIG is run with the `-c++` option). When the structure member is set, the old contents will be released and a new value created. If this is not the behavior you want, you will have to use a typemap (described shortly).

Array members are normally wrapped as read-only. For example, this code:

```
struct Foo {
  int x[50];
};
```

produces a single accessor function like this:

```
int *Foo_x_get(Foo *self) {
  return self->x;
};
```

If you want to set an array member, you will need to supply a "memberin" typemap described in the section on typemaps. As a special case, SWIG does generate code to set array members of type `char` (allowing you to store a Ruby string in the structure).

When structure members are wrapped, they are handled as pointers. For example,

```
struct Foo {
  ...
};

struct Bar {
  Foo f;
};
```

generates accessor functions such as this:

```
Foo *Bar_f_get(Bar *b) {
  return &b->f;
```

```
}

void Bar_f_set(Bar *b, Foo *val) {
  b->f = *val;
}
```

### 35.3.7 C++ classes

Like structs, C++ classes are wrapped by creating a new Ruby class of the same name with accessor methods for the public class member data. Additionally, public member functions for the class are wrapped as Ruby instance methods, and public static member functions are wrapped as Ruby singleton methods. So, given the C++ class declaration:

```
class List {
public:
  List();
  ~List();
  int search(char *item);
  void insert(char *item);
  void remove(char *item);
  char *get(int n);
  int length;
  static void print(List *l);
};
```

SWIG would create a `List` class with:

- instance methods *search*, *insert*, *remove*, and *get*;
- instance methods *length* and *length=* (to get and set the value of the *length* data member); and,
- a *print* singleton method for the class.

In Ruby, these functions are used as follows:

```
require 'Example'

l = Example::List.new

l.insert("Ale")
l.insert("Stout")
l.insert("Lager")
Example.print(l)
l.length()
----- produces the following output
Lager
Stout
Ale
3
```

### 35.3.8 C++ Inheritance

The SWIG type-checker is fully aware of C++ inheritance. Therefore, if you have classes like this:

```
class Parent {
  ...
};

class Child : public Parent {
  ...
};
```

those classes are wrapped into a hierarchy of Ruby classes that reflect the same inheritance structure. All of the usual Ruby utility methods work normally:

```
irb(main):001:0> c = Child.new
#<Bar:0x4016efd4>
irb(main):002:0> c.instance_of? Child
true
irb(main):003:0> b.instance_of? Parent
false
irb(main):004:0> b.is_a? Child
true
irb(main):005:0> b.is_a? Parent
true
irb(main):006:0> Child < Parent
true
irb(main):007:0> Child > Parent
false
```

Furthermore, if you have a function like this:

```
void spam(Parent *f);
```

then the function `spam()` accepts `Parent*` or a pointer to any class derived from `Parent`.

Until recently, the Ruby module for SWIG didn't support multiple inheritance, and this is still the default behavior. This doesn't mean that you can't wrap C++ classes which inherit from multiple base classes; it simply means that only the **first** base class listed in the class declaration is considered, and any additional base classes are ignored. As an example, consider a SWIG interface file with a declaration like this:

```
class Derived : public Base1, public Base2
{
  ...
};
```

For this case, the resulting Ruby class (`Derived`) will only consider `Base1` as its superclass. It won't inherit any of `Base2`'s member functions or data and it won't recognize `Base2` as an "ancestor" of `Derived` (i.e. the *is_a?* relationship would fail). When SWIG processes this interface file, you'll see a warning message like:

```
example.i:5: Warning 802: Warning for Derived: Base Base2 ignored.
Multiple inheritance is not supported in Ruby.
```

Starting with SWIG 1.3.20, the Ruby module for SWIG provides limited support for multiple inheritance. Because the approach for dealing with multiple inheritance introduces some limitations, this is an optional feature that you can activate with the `-minherit` command-line option:

```
$ swig -c++ -ruby -minherit example.i
```

Using our previous example, if your SWIG interface file contains a declaration like this:

```
class Derived : public Base1, public Base2
{
  ...
};
```

and you run SWIG with the `-minherit` command-line option, then you will end up with a Ruby class `Derived` that appears to "inherit" the member data and functions from both `Base1` and `Base2`. What actually happens is that three different top-level classes are created, with Ruby's `Object` class as their superclass. Each of these classes defines a nested module named `Impl`, and it's in these nested `Impl` modules that the actual instance methods for the classes are defined, i.e.

```
class Base1
  module Impl
  # Define Base1 methods here
  end
  include Impl
end

class Base2
  module Impl
  # Define Base2 methods here
  end
  include Impl
end

class Derived
  module Impl
  include Base1::Impl
  include Base2::Impl
  # Define Derived methods here
  end
  include Impl
end
```

Observe that after the nested `Impl` module for a class is defined, it is mixed-in to the class itself. Also observe that the `Derived::Impl` module first mixes-in its base classes' `Impl` modules, thus "inheriting" all of their behavior.

The primary drawback is that, unlike the default mode of operation, neither `Base1` nor `Base2` is a true superclass of `Derived` anymore:

```
obj = Derived.new
obj.is_a? Base1 # this will return false...
obj.is_a? Base2 # ... and so will this
```

In most cases, this is not a serious problem since objects of type `Derived` will otherwise behave as though they inherit from both `Base1` and `Base2` (i.e. they exhibit "Duck Typing").

### 35.3.9 C++ Overloaded Functions

C++ overloaded functions, methods, and constructors are mostly supported by SWIG. For example, if you have two functions like this:

```
void foo(int);
void foo(char *c);
```

You can use them in Ruby in a straightforward manner:

```
irb(main):001:0> foo(3) # foo(int)
irb(main):002:0> foo("Hello") # foo(char *c)
```

Similarly, if you have a class like this,

```
class Foo {
public:
  Foo();
  Foo(const Foo &);
  ...
};
```

you can write Ruby code like this:

```
irb(main):001:0> f = Foo.new # Create a Foo
irb(main):002:0> g = Foo.new(f) # Copy f
```

Overloading support is not quite as flexible as in C++. Sometimes there are methods that SWIG can't disambiguate. For example:

```
void spam(int);
```

```
void spam(short);
```

or

```
void foo(Bar *b);
void foo(Bar &b);
```

If declarations such as these appear, you will get a warning message like this:

```
example.i:12: Warning 509: Overloaded method spam(short) effectively ignored,
example.i:11: Warning 509: as it is shadowed by spam(int).
```

To fix this, you either need to ignore or rename one of the methods. For example:

```
%rename(spam_short) spam(short);
...
void spam(int);
void spam(short); // Accessed as spam_short
```

or

```
%ignore spam(short);
...
void spam(int);
void spam(short); // Ignored
```

SWIG resolves overloaded functions and methods using a disambiguation scheme that ranks and sorts declarations according to a set of type-precedence rules. The order in which declarations appear in the input does not matter except in situations where ambiguity arises--in this case, the first declaration takes precedence.

Please refer to the "SWIG and C++" chapter for more information about overloading.

### 35.3.10 C++ Operators

For the most part, overloaded operators are handled automatically by SWIG and do not require any special treatment on your part. So if your class declares an overloaded addition operator, e.g.

```
class Complex {
  ...
  Complex operator+(Complex &);
  ...
};
```

the resulting Ruby class will also support the addition (+) method correctly.

For cases where SWIG's built-in support is not sufficient, C++ operators can be wrapped using the `%rename` directive (available on SWIG 1.3.10 and later releases). All you need to do is give the operator the name of a valid Ruby identifier. For example:

```
%rename(add_complex) operator+(Complex &, Complex &);
...
Complex operator+(Complex &, Complex &);
```

Now, in Ruby, you can do this:

```
a = Example::Complex.new(2, 3)
b = Example::Complex.new(4, -1)
c = Example.add_complex(a, b)
```

More details about wrapping C++ operators into Ruby operators is discussed in the section on operator overloading.

### 35.3.11 C++ namespaces

SWIG is aware of C++ namespaces, but namespace names do not appear in the module nor do namespaces result in a module that is broken up into submodules or packages. For example, if you have a file like this,

```
%module example

namespace foo {
  int fact(int n);
  struct Vector {
    double x, y, z;
  };
};
```

it works in Ruby as follows:

```
irb(main):001:0> require 'example'
true
irb(main):002:0> Example.fact(3)
6
irb(main):003:0> v = Example::Vector.new
#<Example::Vector:0x4016f4d4>
irb(main):004:0> v.x = 3.4
3.4
irb(main):004:0> v.y
0.0
```

If your program has more than one namespace, name conflicts (if any) can be resolved using `%rename` For example:

```
%rename(Bar_spam) Bar::spam;

namespace Foo {
  int spam();
}

namespace Bar {
  int spam();
}
```

If you have more than one namespace and your want to keep their symbols separate, consider wrapping them as separate SWIG modules. For example, make the module name the same as the namespace and create extension modules for each namespace separately. If your program utilizes thousands of small deeply nested namespaces each with identical symbol names, well, then you get what you deserve.

### 35.3.12 C++ templates

C++ templates don't present a huge problem for SWIG. However, in order to create wrappers, you have to tell SWIG to create wrappers for a particular template instantiation. To do this, you use the `%template` directive. For example:

```
%module example

%{
#include "pair.h"
%}

template<class T1, class T2>
struct pair {
  typedef T1 first_type;
  typedef T2 second_type;
  T1 first;
  T2 second;
  pair();
  pair(const T1&, const T2&);
  ~pair();
};

%template(Pairii) pair<int, int>;
```

In Ruby:

```
irb(main):001:0> require 'example'
true
irb(main):002:0> p = Example::Pairii.new(3, 4)
#<Example:Pairii:0x4016f4df>
irb(main):003:0> p.first
3
irb(main):004:0> p.second
4
```

### 35.3.13 C++ Standard Template Library (STL)

On a related note, the standard SWIG library contains a number of modules that provide typemaps for standard C++ library classes (such as `std::pair`, `std::string` and `std::vector`). These library modules don't provide wrappers around the templates themselves, but they do make it convenient for users of your extension module to pass Ruby objects (such as arrays and strings) to wrapped C++ code that expects instances of standard C++ templates. For example, suppose the C++ library you're wrapping has a function that expects a vector of floats:

```
%module example

float sum(const std::vector<float>& values);
```

Rather than go through the hassle of writing an "in" typemap to convert an array of Ruby numbers into a std::vector<float>, you can just use the `std_vector.i` module from the standard SWIG library:

```
%module example

%include std_vector.i
float sum(const std::vector<float>& values);
```

Ruby's STL wrappings provide additional methods to make them behave more similarly to Ruby's native classes.

Thus, you can do, for example:

```
v = IntVector.new
v << 2
v << 3
v << 4
v.each { |x| puts x }

=> 2
3
4
v.delete_if { |x| x == 3 }
=> [2, 4]
```

The SWIG Ruby module provides also the ability for all the STL containers to carry around Ruby native objects (Fixnum, Classes, etc) making them act almost like Ruby's own Array, Hash, etc. To do that, you need to define a container that contains a swig::GC_VALUE, like:

```
%module nativevector
```

```
%{
std::vector< swig::GC_VALUE > NativeVector;
%}

%template(NativeVector) std::vector< swig::GC_VALUE >;
```

This vector can then contain any Ruby object, making them almost identical to Ruby's own Array class.

```
require 'nativevector'
include NativeVector

v = NativeVector.new
v << 1
v << [1, 2]
v << 'hello'

class A; end

v << A.new

puts v
=> [1, [1, 2], 'hello', #<A:0x245325>]
```

Obviously, there is a lot more to template wrapping than shown in these examples. More details can be found in the SWIG and C++ chapter.

### 35.3.14 C++ STL Functors

Some containers in the STL allow you to modify their default behavior by using so called functors or function objects. Functors are often just a very simple struct with `operator()` redefined or an actual C/C++ function. This allows you, for example, to always keep the sort order of a STL container to your liking.

The Ruby STL mappings allows you to modify those containers that support functors using Ruby procs or methods, instead. Currently, this includes `std::set`, `set::map`, `std::multiset` and `std::multimap`.

The functors in swig are called `swig::UnaryFunction` and `swig::BinaryFunction`. For C++ predicates (ie. functors that must return bool as a result) `swig::UnaryPredicate` and `swig::BinaryPredicate` are provided.

As an example, if given this swig file:

```
%module intset;

%include <std_set.i>

%template(IntSet) std::set< int, swig::BinaryPredicate >;
```

You can then use the set from Ruby with or without a proc object as a predicate:

```
require 'intset'
include Intset

# Default sorting behavior defined in C++
a = IntSet.new
a << 1
a << 2
a << 3
a
=> [1, 2, 3]

# Custom sorting behavior defined by a Ruby proc
b = IntSet.new( proc { |a, b| a > b } )
b << 1
b << 2
b << 3
b
=>  [3, 2, 1]
```

### 35.3.15 C++ STL Iterators

The STL is well known for the use of iterators. There are a number of iterators possible with different properties, but in general there are two main categories: const iterators and non-const iterators. The const iterators can access and not modify the values they point at, while the non-const iterators can both read and modify the values.

The Ruby STL wrappings support both type of iterators by using a proxy class in-between. This proxy class is `swig::Iterator` or `swig::ConstIterator`. Derived from them are template classes that need to be initialized with the actual iterator for the container you are wrapping and often times with the beginning and ending points of the iteration range.

The SWIG STL library already provides typemaps to all the standard containers to do this wrapping automatically for you, but if you have your own STL-like iterator, you will need to write your own typemap for them. For out typemaps, the special functions `make_const_iterator` and `make_nonconst_iterator` are provided.

These can be used either like:

```
make_const_iterator( iterator, rubyclass );
make_const_iterator( iterator, iterator_begin, iterator_end, rubyclass );
```

The iterators support a `next()` and `previous()` member function to just change the iterator without returning anything. `previous()` should obviously only be used for bidirectional iterators. You can also advance the iterator multiple steps by using standard math operations like `+=`.

The value the iterator points at can be accessed with `value()` -- this is equivalent to dereferencing it with `*i`. For non-const iterators, a `value=()` function is also provided which allows you to change the value pointed by the iterator. This is equivalent to the C++ construct of dereferencing and assignment, like `*i = something`.

Thus, given say a vector class of doubles defined as:

```
%module doublevector

%include std_vector.i

%template(DoubleVector) std::vector<double>;
```

Its iterator can then be used from Ruby like:

```
require 'doublevector'
include Doublevector

v = DoubleVector.new
v << 1
v << 2
v << 3

#
# an elaborate and less efficient way of doing v.map! { |x| x+2 }
#
i = v.begin
e = v.end
while i != e
  val = i.value
  val += 2
  i.value = val
  i.next
end
i
>> [3, 4, 5 ]
```

If you'd rather have STL classes without any iterators, you should define `-DSWIG_NO_EXPORT_ITERATOR_METHODS` when running swig.

### 35.3.16 C++ Smart Pointers

#### 35.3.16.1 The shared_ptr Smart Pointer

The C++11 standard provides `std::shared_ptr` which was derived from the Boost implementation, `boost::shared_ptr`. Both of these are available for Ruby in the SWIG library and usage is outlined in the shared_ptr smart pointer library section.

#### 35.3.16.2 Generic Smart Pointers

In certain C++ programs, it is common to use classes that have been wrapped by so-called "smart pointers." Generally, this involves the use of a template class that implements `operator->()` like this:

```
template<class T> class SmartPtr {
  ...
  T *operator->();
  ...
}
```

Then, if you have a class like this,

```
class Foo {
public:
  int x;
  int bar();
};
```

A smart pointer would be used in C++ as follows:

```
SmartPtr<Foo> p = CreateFoo(); // Created somehow (not shown)
...
p->x = 3; // Foo::x
int y = p->bar(); // Foo::bar
```

To wrap this in Ruby, simply tell SWIG about the `SmartPtr` class and the low-level `Foo` object. Make sure you instantiate `SmartPtr` using `%template` if necessary. For example:

```
%module example
...
%template(SmartPtrFoo) SmartPtr<Foo>;
...
```

Now, in Ruby, everything should just "work":

```
irb(main):001:0> p = Example::CreateFoo() # Create a smart-pointer somehow
#<Example::SmartPtrFoo:0x4016f4df>
irb(main):002:0> p.x = 3 # Foo::x
3
irb(main):003:0> p.bar() # Foo::bar
```

If you ever need to access the underlying pointer returned by `operator->()` itself, simply use the `__deref__()` method. For example:

```
irb(main):004:0> f = p.__deref__() # Returns underlying Foo *
```

### 35.3.17 Cross-Language Polymorphism

SWIG's Ruby module supports cross-language polymorphism (a.k.a. the "directors" feature) similar to that for SWIG's Python module. Rather than duplicate the information presented in the Python chapter, this section just notes the differences that you need to be aware of when using this feature with Ruby.

#### 35.3.17.1 Exception Unrolling

Whenever a C++ director class routes one of its virtual member function calls to a Ruby instance method, there's always the possibility that an exception will be raised in the Ruby code. By default, those exceptions are ignored, which simply means that the exception will be exposed to the Ruby interpreter. If you would like to change this behavior, you can use the

`%feature("director:except")` directive to indicate what action should be taken when a Ruby exception is raised. The following code should suffice in most cases:

```
%feature("director:except") {
  throw Swig::DirectorMethodException($error);
}
```

When this feature is activated, the call to the Ruby instance method is "wrapped" using the `rb_rescue2()` function from Ruby's C API. If any Ruby exception is raised, it will be caught here and a C++ exception is raised in its place.

## 35.4 Naming

Ruby has several common naming conventions. Constants are generally in upper case, module and class names are in camel case and methods are in lower case with underscores. For example:

- **MATH::PI** is a constant name
- **MyClass** is a class name
- **my_method** is a method name

Prior to version 1.3.28, SWIG did not support these Ruby conventions. The only modifications it made to names was to capitalize the first letter of constants (which includes module and class names).

SWIG 1.3.28 introduces the new -autorename command line parameter. When this parameter is specified, SWIG will automatically change constant, class and method names to conform with the standard Ruby naming conventions. For example:

```
$ swig -ruby -autorename example.i
```

To disable renaming use the -noautorename command line option.

Since this change significantly changes the wrapper code generated by SWIG, it is turned off by default in SWIG 1.3.28. However, it is planned to become the default option in future releases.

### 35.4.1 Defining Aliases

It's a fairly common practice in the Ruby built-ins and standard library to provide aliases for method names. For example, *Array#size* is an alias for *Array#length*. If you would like to provide an alias for one of your class' instance methods, one approach is to use SWIG's `%extend` directive to add a new method of the aliased name that calls the original function. For example:

```
class MyArray {
public:
  // Construct an empty array
  MyArray();

  // Return the size of this array
  size_t length() const;
};

%extend MyArray {
  // MyArray#size is an alias for MyArray#length
  size_t size() const {
    return $self->length();
  }
}
```

A better solution is to use the `%alias` directive (unique to SWIG's Ruby module). The previous example could then be rewritten as:

```
// MyArray#size is an alias for MyArray#length
%alias MyArray::length "size";

class MyArray {
public:
  // Construct an empty array
  MyArray();

  // Return the size of this array
  size_t length() const;
};
```

Multiple aliases can be associated with a method by providing a comma-separated list of aliases to the `%alias` directive, e.g.

```
%alias MyArray::length "amount, quantity, size";
```

From an end-user's standpoint, there's no functional difference between these two approaches; i.e. they should get the same result from calling either *MyArray#size* or *MyArray#length*. However, when the `%alias` directive is used, SWIG doesn't need to generate all of the wrapper code that's usually associated with added methods like our *MyArray::size()* example.

Note that the `%alias` directive is implemented using SWIG's "features" mechanism and so the same name matching rules used for other kinds of features apply (see the chapter on "Customization Features") for more details).

### 35.4.2 Predicate Methods

Ruby methods that return a boolean value and end in a question mark are known as predicate methods. Examples of predicate methods in standard Ruby classes include *Array#empty?* (which returns `true` for an array containing no elements) and *Object#instance_of?* (which returns `true` if the object is an instance of the specified class). For consistency with Ruby conventions, methods that return boolean values should be marked as predicate methods.

One cumbersome solution to this problem is to rename the method (using SWIG's `%rename` directive) and provide a custom typemap that converts the function's actual return type to Ruby's `true` or `false`. For example:

```
%rename("is_it_safe?") is_it_safe();

%typemap(out) int is_it_safe "$result = ($1 != 0) ? Qtrue : Qfalse;"
```

```
int is_it_safe();
```

A better solution is to use the `%predicate` directive (unique to SWIG's Ruby module) to designate a method as a predicate method. For the previous example, this would look like:

```
%predicate is_it_safe();

int is_it_safe();
```

This method would be invoked from Ruby code like this:

```
irb(main):001:0> Example::is_it_safe?
true
```

The `%predicate` directive is implemented using SWIG's "features" mechanism and so the same name matching rules used for other kinds of features apply (see the chapter on "Customization Features") for more details).

### 35.4.3 Bang Methods

Ruby methods that modify an object in-place and end in an exclamation mark are known as bang methods. An example of a bang method is *Array#sort!* which changes the ordering of items in an array. Contrast this with *Array#sort*, which returns a copy of the array with the items sorted instead of modifying the original array. For consistency with Ruby conventions, methods that modify objects in place should be marked as bang methods.

Bang methods can be marked using the`%bang` directive which is unique to the Ruby module and was introduced in SWIG 1.3.28. For example:

```
%bang sort(int arr[]);

int sort(int arr[]);
```

This method would be invoked from Ruby code like this:

```
irb(main):001:0> Example::sort!(arr)
```

The `%bang` directive is implemented using SWIG's "features" mechanism and so the same name matching rules used for other kinds of features apply (see the chapter on "Customization Features") for more details).

### 35.4.4 Getters and Setters

Often times a C++ library will expose properties through getter and setter methods. For example:

```
class Foo {
  Foo() {}
  int getValue() { return value_; }
  void setValue(int value) { value_ = value; }

private:
  int value_;
};
```

By default, SWIG will expose these methods to Ruby as`get_value` and `set_value`. However, it more natural for these methods to be exposed in Ruby as `value` and `value=`. That allows the methods to be used like this:

```
irb(main):001:0> foo = Foo.new()
irb(main):002:0> foo.value = 5
irb(main):003:0> puts foo.value
```

This can be done by using the %rename directive:

```
%rename("value") Foo::getValue();
%rename("value=") Foo::setValue(int value);
```

## 35.5 Input and output parameters

A common problem in some C programs is handling parameters passed as simple pointers. For example:

```
void add(int x, int y, int *result) {
  *result = x + y;
}
```

or

```
int sub(int *x, int *y) {
  return *x-*y;
}
```

The easiest way to handle these situations is to use the `typemaps.i` file. For example:

```
%module Example
%include "typemaps.i"

void add(int, int, int *OUTPUT);
int sub(int *INPUT, int *INPUT);
```

In Ruby, this allows you to pass simple values. For example:

```
a = Example.add(3, 4)
puts a
7
b = Example.sub(7, 4)
puts b
3
```

Notice how the `INPUT` parameters allow integer values to be passed instead of pointers and how the `OUTPUT` parameter creates a return result.

If you don't want to use the names `INPUT` or `OUTPUT`, use the `%apply` directive. For example:

```
%module Example
%include "typemaps.i"

%apply int *OUTPUT { int *result };
%apply int *INPUT { int *x, int *y};

void add(int x, int y, int *result);
int sub(int *x, int *y);
```

If a function mutates one of its parameters like this,

```
void negate(int *x) {
 *x = -(*x);
}
```

you can use `INOUT` like this:

```
%include "typemaps.i"
...
void negate(int *INOUT);
```

In Ruby, a mutated parameter shows up as a return value. For example:

```
a = Example.negate(3)
print a
-3
```

The most common use of these special typemap rules is to handle functions that return more than one value. For example, sometimes a function returns a result as well as a special error code:

```
/* send message, return number of bytes sent, success code, and error_code */
int send_message(char *text, int *success, int *error_code);
```

To wrap such a function, simply use the `OUTPUT` rule above. For example:

```
%module example
%include "typemaps.i"
...
int send_message(char *, int *OUTPUT, int *OUTPUT);
```

When used in Ruby, the function will return an array of multiple values.

```
bytes, success, error_code = send_message("Hello World")
if not success
  print "error #{error_code} : in send_message"
else
  print "Sent", bytes
end
```

Another way to access multiple return values is to use the `%apply` rule. In the following example, the parameters rows and columns are related to SWIG as `OUTPUT` values through the use of `%apply`

```
%module Example
%include "typemaps.i"
%apply int *OUTPUT { int *rows, int *columns };
...
void get_dimensions(Matrix *m, int *rows, int*columns);
```

In Ruby:

```
r, c = Example.get_dimensions(m)
```

## 35.6 Exception handling

### 35.6.1 Using the %exception directive

The SWIG `%exception` directive can be used to define a user-definable exception handler that can convert C/C++ errors into Ruby exceptions. The chapter on Customization Features contains more details, but suppose you have a C++ class like the following :

```
class DoubleArray {
```

```
private:
  int n;
  double *ptr;
public:
  // Create a new array of fixed size
  DoubleArray(int size) {
    ptr = new double[size];
    n = size;
  }

  // Destroy an array
  ~DoubleArray() {
    delete ptr;
  }

  // Return the length of the array
  int length() {
    return n;
  }

  // Get an array item and perform bounds checking.
  double getitem(int i) {
    if ((i >= 0) && (i < n))
      return ptr[i];
    else
      throw RangeError();
  }

  // Set an array item and perform bounds checking.
  void setitem(int i, double val) {
    if ((i >= 0) && (i < n))
      ptr[i] = val;
    else {
      throw RangeError();
    }
  }
};
```

Since several methods in this class can throw an exception for an out-of-bounds access, you might want to catch this in the Ruby extension by writing the following in an interface file:

```
%exception {
  try {
    $action
  }
  catch (const RangeError&) {
    static VALUE cpperror = rb_define_class("CPPError", rb_eStandardError);
    rb_raise(cpperror, "Range error.");
  }
}

class DoubleArray {
  ...
};
```

The exception handling code is inserted directly into generated wrapper functions. When an exception handler is defined, errors can be caught and used to gracefully raise a Ruby exception instead of forcing the entire program to terminate with an uncaught error.

As shown, the exception handling code will be added to every wrapper function. Because this is somewhat inefficient, you might consider refining the exception handler to only apply to specific methods like this:

```
%exception getitem {
  try {
    $action
  } catch (const RangeError&) {
    static VALUE cpperror = rb_define_class("CPPError", rb_eStandardError);
    rb_raise(cpperror, "Range error in getitem.");
  }
}

%exception setitem {
  try {
    $action
  } catch (const RangeError&) {
    static VALUE cpperror = rb_define_class("CPPError", rb_eStandardError);
    rb_raise(cpperror, "Range error in setitem.");
  }
}
```

In this case, the exception handler is only attached to methods and functions named `getitem` and `setitem`.

Since SWIG's exception handling is user-definable, you are not limited to C++ exception handling. See the chapter on Customization Features for more examples.

### 35.6.2 Handling Ruby Blocks

One of the highlights of Ruby and most of its standard library is the use of blocks, which allow the easy creation of continuations and other niceties. Blocks in ruby are also often used to simplify the passing of many arguments to a class.

In order to make your class constructor support blocks, you can take advantage of the %exception directive, which will get run after the C++ class' constructor was called.

For example, this yields the class over after its construction:

```
class Window
{
public:
  Window(int x, int y, int w, int h);
  // .... other methods here ....
};
```

```
// Add support for yielding self in the Class' constructor.
%exception Window::Window {
  $action
  if (rb_block_given_p()) {
    rb_yield(self);
  }
}
```

Then, in ruby, it can be used like:

```
Window.new(0, 0, 360, 480) { |w|
  w.color = Fltk::RED
  w.border = false
}
```

For other methods, you can usually use a dummy parameter with a special in typemap, like:

```
//
// original function was:
//
// void func(int x);

%typemap(in, numinputs=0) int RUBY_YIELD_SELF {
  if ( !rb_block_given_p() )
    rb_raise("No block given");
  return rb_yield(self);
}

%extend {
  void func(int x, int RUBY_YIELD_SELF );
}
```

For more information on typemaps, see Typemaps .

### 35.6.3 Raising exceptions

There are three ways to raise exceptions from C++ code to Ruby.

The first way is to use `SWIG_exception(int code, const char *msg)` . The following table shows the mappings from SWIG error codes to Ruby exceptions:

| | |
|---|---|
| SWIG_MemoryError | rb_eNoMemError |
| SWIG_IOError | rb_eIOError |
| SWIG_RuntimeError | rb_eRuntimeError |
| SWIG_IndexError | rb_eIndexError |
| SWIG_TypeError | rb_eTypeError |
| SWIG_DivisionByZero | rb_eZeroDivError |
| SWIG_OverflowError | rb_eRangeError |
| SWIG_SyntaxError | rb_eSyntaxError |
| SWIG_ValueError | rb_eArgError |
| SWIG_SystemError | rb_eFatal |
| SWIG_AttributeError | rb_eRuntimeError |
| SWIG_NullReferenceError | rb_eNullReferenceError* |
| SWIG_ObjectPreviouslyDeletedError | rb_eObjectPreviouslyDeleted* |
| SWIG_UnknownError | rb_eRuntimeError |
| * These error classes are created by SWIG and are not built-in Ruby exception classes | |

The second way to raise errors is to use `SWIG_Raise(obj, type, desc)`. Obj is a C++ instance of an exception class, type is a string specifying the type of exception (for example, "MyError") and desc is the SWIG description of the exception class. For example:

```
%raise(SWIG_NewPointerObj(e, SWIGTYPE_p_AssertionFailedException, 0), ":AssertionFailedException", SWIGTYPE_p_AssertionFailedException);
```

This is useful when you want to pass the current exception object directly to Ruby, particularly when the object is an instance of class marked as an `%exceptionclass` (see the next section for more information).

Last, you can raise an exception by directly calling Ruby's C api. This is done by invoking the `rb_raise()` function. The first argument passed to `rb_raise()` is the exception type. You can raise a custom exception type or one of the built-in Ruby exception types.

### 35.6.4 Exception classes

Starting with SWIG 1.3.28, the Ruby module supports the`%exceptionclass` directive, which is used to identify C++ classes that are used as exceptions. Classes that are marked with the `%exceptionclass` directive are exposed in Ruby as child classes of`rb_eRuntimeError`. This allows C++ exceptions to be directly mapped to Ruby exceptions, providing for a more natural integration between C++ code and Ruby code.

```
%exceptionclass CustomError;

%inline %{
  class CustomError { };

  class Foo {
  public:
    void test() { throw CustomError; }
  };
%}
```

From Ruby you can now call this method like this:

```
foo = Foo.new
```

```
begin
  foo.test()
rescue CustomError => e
  puts "Caught custom error"
end
```

For another example look at swig/Examples/ruby/exception_class.

## 35.7 Typemaps

This section describes how you can modify SWIG's default wrapping behavior for various C/C++ datatypes using the `%typemap` directive. This is an advanced topic that assumes familiarity with the Ruby C API as well as the material in the "Typemaps " chapter.

Before proceeding, it should be stressed that typemaps are not a required part of using SWIG---the default wrapping behavior is enough in most cases. Typemaps are only used if you want to change some aspect of the primitive C-Ruby interface.

### 35.7.1 What is a typemap?

A typemap is nothing more than a code generation rule that is attached to a specific C datatype. The general form of this declaration is as follows ( parts enclosed in [...] are optional ):

```
%typemap( method [, modifiers...] ) typelist code;
```

*method* is a simply a name that specifies what kind of typemap is being defined. It is usually a name like `"in"`, `"out"`, or `"argout"` (or its director variations). The purpose of these methods is described later.

*modifiers* is an optional comma separated list of`name="value"` values. These are sometimes to attach extra information to a typemap and is often target-language dependent.

*typelist* is a list of the C++ type patterns that the typemap will match. The general form of this list is as follows:

```
typelist : typepattern [, typepattern, typepattern, ... ] ;

typepattern : type [ (parms) ]
  | type name [ (parms) ]
  | ( typelist ) [ (parms) ]
```

Each type pattern is either a simple type, a simple type and argument name, or a list of types in the case of multi-argument typemaps. In addition, each type pattern can be parameterized with a list of temporary variables (parms). The purpose of these variables will be explained shortly.

*code* specifies the C code used in the typemap. It can take any one of the following forms:

```
code : { ... }
  | " ... "
  | %{ ... %}
```

For example, to convert integers from Ruby to C, you might define a typemap like this:

```
%module example

%typemap(in) int {
  $1 = (int) NUM2INT($input);
  printf("Received an integer : %d\n", $1);
}

%inline %{
  extern int fact(int n);
%}
```

Typemaps are always associated with some specific aspect of code generation. In this case, the "in" method refers to the conversion of input arguments to C/C++. The datatype `int` is the datatype to which the typemap will be applied. The supplied C code is used to convert values. In this code a number of special variables prefaced by a `$` are used. The `$1` variable is placeholder for a local variable of type `int`. The `$input` variable is the input Ruby object.

When this example is compiled into a Ruby module, the following sample code:

```
require 'example'

puts Example.fact(6)
```

prints the result:

```
Received an integer : 6
720
```

In this example, the typemap is applied to all occurrences of the `int` datatype. You can refine this by supplying an optional parameter name. For example:

```
%module example

%typemap(in) int n {
  $1 = (int) NUM2INT($input);
  printf("n = %d\n", $1);
}

%inline %{
  extern int fact(int n);
%}
```

In this case, the typemap code is only attached to arguments that exactly match "`int n`".

The application of a typemap to specific datatypes and argument names involves more than simple text-matching--typemaps are fully integrated into the SWIG type-system. When you define a

typemap for `int`, that typemap applies to `int` and qualified variations such as `const int`. In addition, the typemap system follows `typedef` declarations. For example:

```
%typemap(in) int n {
  $1 = (int) NUM2INT($input);
  printf("n = %d\n", $1);
}

typedef int Integer;
extern int fact(Integer n); // Above typemap is applied
```

However, the matching of `typedef` only occurs in one direction. If you defined a typemap for `Integer`, it is not applied to arguments of type `int`.

Typemaps can also be defined for groups of consecutive arguments. For example:

```
%typemap(in) (char *str, int len) {
  $1 = StringValuePtr($input);
  $2 = (int) RSTRING($input)->len;
};

int count(char c, char *str, int len);
```

When a multi-argument typemap is defined, the arguments are always handled as a single Ruby object. This allows the function `count` to be used as follows (notice how the length parameter is omitted):

```
puts Example.count('o', 'Hello World')
2
```

### 35.7.2 Typemap scope

Once defined, a typemap remains in effect for all of the declarations that follow. A typemap may be redefined for different sections of an input file. For example:

```
// typemap1
%typemap(in) int {
  ...
}

int fact(int); // typemap1
int gcd(int x, int y); // typemap1

// typemap2
%typemap(in) int {
  ...
}

int isprime(int); // typemap2
```

One exception to the typemap scoping rules pertains to the `%extend` declaration. `%extend` is used to attach new declarations to a class or structure definition. Because of this, all of the declarations in an `%extend` block are subject to the typemap rules that are in effect at the point where the class itself is defined. For example:

```
class Foo {
  ...
};

%typemap(in) int {
  ...
}

%extend Foo {
  int blah(int x); // typemap has no effect. Declaration is attached to Foo which
  // appears before the %typemap declaration.
};
```

### 35.7.3 Copying a typemap

A typemap is copied by using assignment. For example:

```
%typemap(in) Integer = int;
```

or this:

```
%typemap(in) Integer, Number, int32_t = int;
```

Types are often managed by a collection of different typemaps. For example:

```
%typemap(in) int { ... }
%typemap(out) int { ... }
%typemap(varin) int { ... }
%typemap(varout) int { ... }
```

To copy all of these typemaps to a new type, use `%apply`. For example:

```
%apply int { Integer }; // Copy all int typemaps to Integer
%apply int { Integer, Number }; // Copy all int typemaps to both Integer and Number
```

The patterns for `%apply` follow the same rules as for `%typemap`. For example:

```
%apply int *output { Integer *output }; // Typemap with name
%apply (char *buf, int len) { (char *buffer, int size) }; // Multiple arguments
```

### 35.7.4 Deleting a typemap

A typemap can be deleted by simply defining no code. For example:

```
%typemap(in) int; // Clears typemap for int
%typemap(in) int, long, short; // Clears typemap for int, long, short
%typemap(in) int *output;
```

The `%clear` directive clears all typemaps for a given type. For example:

```
%clear int; // Removes all types for int
%clear int *output, long *output;
```

**Note:** Since SWIG's default behavior is defined by typemaps, clearing a fundamental type like `int` will make that type unusable unless you also define a new set of typemaps immediately after the clear operation.

### 35.7.5 Placement of typemaps

Typemap declarations can be declared in the global scope, within a C++ namespace, and within a C++ class. For example:

```
%typemap(in) int {
  ...
}

namespace std {
  class string;
  %typemap(in) string {
    ...
  }
}

class Bar {
public:
  typedef const int & const_reference;
  %typemap(out) const_reference {
    ...
  }
};
```

When a typemap appears inside a namespace or class, it stays in effect until the end of the SWIG input (just like before). However, the typemap takes the local scope into account. Therefore, this code

```
namespace std {
  class string;
  %typemap(in) string {
    ...
  }
}
```

is really defining a typemap for the type `std::string`. You could have code like this:

```
namespace std {
  class string;
  %typemap(in) string { /* std::string */
  ...
  }
}

namespace Foo {
  class string;
  %typemap(in) string { /* Foo::string */
  ...
  }
}
```

In this case, there are two completely distinct typemaps that apply to two completely different types (`std::string` and `Foo::string`).

It should be noted that for scoping to work, SWIG has to know that `string` is a typename defined within a particular namespace. In this example, this is done using the class declaration `class string`.

### 35.7.6 Ruby typemaps

The following list details all of the typemap methods that can be used by the Ruby module:

#### 35.7.6.1 "in" typemap

Converts Ruby objects to input function arguments. For example:

```
%typemap(in) int {
  $1 = NUM2INT($input);
}
```

The following special variables are available:

| | |
|---|---|
| $input | Input object holding value to be converted. |
| $symname | Name of function/method being wrapped |

| | |
|---|---|
| $1...n | Argument being sent to the function |
| $1_name | Name of the argument (if provided) |
| $1_type | The actual C datatype matched by the typemap. |
| $1_ltype | The assignable version of the C datatype matched by the typemap. |

This is probably the most commonly redefined typemap because it can be used to implement customized conversions.

In addition, the "in" typemap allows the number of converted arguments to be specified. For example:

```
// Ignored argument.
%typemap(in, numinputs=0) int *out (int temp) {
  $1 = &temp;
}
```

At this time, only zero or one arguments may be converted.

#### 35.7.6.2 "typecheck" typemap

The "typecheck" typemap is used to support overloaded functions and methods. It merely checks an argument to see whether or not it matches a specific type. For example:

```
%typemap(typecheck, precedence=SWIG_TYPECHECK_INTEGER) int {
  $1 = FIXNUM_P($input) ? 1 : 0;
}
```

For typechecking, the $1 variable is always a simple integer that is set to 1 or 0 depending on whether or not the input argument is the correct type.

If you define new "in" typemaps *and* your program uses overloaded methods, you should also define a collection of "typecheck" typemaps. More details about this follow in a later section on "Typemaps and Overloading."

#### 35.7.6.3 "out" typemap

Converts return value of a C function to a Ruby object.

```
%typemap(out) int {
  $result = INT2NUM( $1 );
}
```

The following special variables are available.

| | |
|---|---|
| $result | Result object returned to target language. |
| $symname | Name of function/method being wrapped |
| $1...n | Argument being wrapped |
| $1_name | Name of the argument (if provided) |
| $1_type | The actual C datatype matched by the typemap. |
| $1_ltype | The assignable version of the C datatype matched by the typemap. |

#### 35.7.6.4 "arginit" typemap

The "arginit" typemap is used to set the initial value of a function argument--before any conversion has occurred. This is not normally necessary, but might be useful in highly specialized applications. For example:

```
// Set argument to NULL before any conversion occurs
%typemap(arginit) int *data {
  $1 = NULL;
}
```

#### 35.7.6.5 "default" typemap

The "default" typemap is used to turn an argument into a default argument. For example:

```
%typemap(default) int flags {
  $1 = DEFAULT_FLAGS;
}
...
int foo(int x, int y, int flags);
```

The primary use of this typemap is to either change the wrapping of default arguments or specify a default argument in a language where they aren't supported (like C). Target languages that do not support optional arguments, such as Java and C#, effectively ignore the value specified by this typemap as all arguments must be given.

Once a default typemap has been applied to an argument, all arguments that follow must have default values. See the Default/optional arguments section for further information on default argument wrapping.

#### 35.7.6.6 "check" typemap

The "check" typemap is used to supply value checking code during argument conversion. The typemap is applied *after* arguments have been converted. For example:

```
%typemap(check) int positive {
  if ($1 <= 0) {
    SWIG_exception(SWIG_ValueError, "Expected positive value.");
  }
}
```

#### 35.7.6.7 "argout" typemap

The "argout" typemap is used to return values from arguments. This is most commonly used to write wrappers for C/C++ functions that need to return multiple values. The "argout" typemap is

almost always combined with an "in" typemap---possibly to ignore the input value. For example:

```
/* Set the input argument to point to a temporary variable */
%typemap(in, numinputs=0) int *out (int temp) {
  $1 = &temp;
}

%typemap(argout) int *out {
  // Append output value $1 to $result (assuming a single integer in this case)
  $result = SWIG_AppendOutput( $result, INT2NUM(*$1) );
}
```

The following special variables are available.

| | |
|---|---|
| $result | Result object returned to target language. |
| $input | The original input object passed. |
| $symname | Name of function/method being wrapped. |

The code supplied to the "argout" typemap is always placed after the "out" typemap. If multiple return values are used, the extra return values are often appended to return value of the function.

Output helper is a fragment that usually defines a macro to some function like SWIG_Ruby_AppendOutput.

See the `typemaps.i` library for examples.

#### 35.7.6.8 "freearg" typemap

The "freearg" typemap is used to cleanup argument data. It is only used when an argument might have allocated resources that need to be cleaned up when the wrapper function exits. The "freearg" typemap usually cleans up argument resources allocated by the "in" typemap. For example:

```
// Get a list of integers
%typemap(in) int *items {
  int nitems = Length($input);
  $1 = (int *) malloc(sizeof(int)*nitems);
}
// Free the list
%typemap(freearg) int *items {
  free($1);
}
```

The "freearg" typemap inserted at the end of the wrapper function, just before control is returned back to the target language. This code is also placed into a special variable `$cleanup` that may be used in other typemaps whenever a wrapper function needs to abort prematurely.

#### 35.7.6.9 "newfree" typemap

The "newfree" typemap is used in conjunction with the `%newobject` directive and is used to deallocate memory used by the return result of a function. For example:

```
%typemap(newfree) string * {
  delete $1;
}
%typemap(out) string * {
  $result = PyString_FromString($1->c_str());
}
...

%newobject foo;
...
string *foo();
```

See Object ownership and %newobject for further details.

#### 35.7.6.10 "memberin" typemap

The "memberin" typemap is used to copy data from *an already converted input value* into a structure member. It is typically used to handle array members and other special cases. For example:

```
%typemap(memberin) int [4] {
  memmove($1, $input, 4*sizeof(int));
}
```

It is rarely necessary to write "memberin" typemaps---SWIG already provides a default implementation for arrays, strings, and other objects.

#### 35.7.6.11 "varin" typemap

The "varin" typemap is used to convert objects in the target language to C for the purposes of assigning to a C/C++ global variable. This is implementation specific.

#### 35.7.6.12 "varout" typemap

The "varout" typemap is used to convert a C/C++ object to an object in the target language when reading a C/C++ global variable. This is implementation specific.

#### 35.7.6.13 "throws" typemap

The "throws" typemap is only used when SWIG parses a C++ method with an exception specification or has the `%catches` feature attached to the method. It provides a default mechanism for handling C++ methods that have declared the exceptions they will throw. The purpose of this typemap is to convert a C++ exception into an error or exception in the target language. It is slightly different to the other typemaps as it is based around the exception type rather than the type of a parameter or variable. For example:

```
%typemap(throws) const char * %{
  rb_raise(rb_eRuntimeError, $1);
  SWIG_fail;
%}
void bar() throw (const char *);
```

As can be seen from the generated code below, SWIG generates an exception handler with the catch block comprising the "throws" typemap content.

```
...
try {
  bar();
}
catch(char const *_e) {
  rb_raise(rb_eRuntimeError, _e);
  SWIG_fail;
}
...
```

Note that if your methods do not have an exception specification yet they do throw exceptions, SWIG cannot know how to deal with them. For a neat way to handle these, see the Exception handling with %exception section.

#### 35.7.6.14 directorin typemap

Converts C++ objects in director member functions to ruby objects. It is roughly the opposite of the "in" typemap, making its typemap rule often similar to the "out" typemap.

```
%typemap(directorin) int {
  $result = INT2NUM($1);
}
```

The following special variables are available.

| | |
|---|---|
| $result | Result object returned to target language. |
| $symname | Name of function/method being wrapped |
| $1...n | Argument being wrapped |
| $1_name | Name of the argument (if provided) |
| $1_type | The actual C datatype matched by the typemap. |
| $1_ltype | The assignable version of the C datatype matched by the typemap. |
| this | C++ this, referring to the class itself. |

#### 35.7.6.15 directorout typemap

Converts Ruby objects in director member functions to C++ objects. It is roughly the opposite of the "out" typemap, making its rule often similar to the "in" typemap.

```
%typemap(directorout) int {
  $result = NUM2INT($1);
}
```

The following special variables are available:

| | |
|---|---|
| $input | Ruby object being sent to the function |
| $symname | Name of function/method being wrapped |
| $1...n | Argument being sent to the function |
| $1_name | Name of the argument (if provided) |
| $1_type | The actual C datatype matched by the typemap. |
| $1_ltype | The assignable version of the C datatype matched by the typemap. |
| this | C++ this, referring to the class itself. |

Currently, the directorout nor the out typemap support the option`numoutputs`, but the Ruby module provides that functionality through a %feature directive. Thus, a function can be made to return "nothing" if you do:

```
%feature("numoutputs", "0") MyClass::function;
```

This feature can be useful if a function returns a status code, which you want to discard but still use the typemap to raise an exception.

#### 35.7.6.16 directorargout typemap

Output argument processing in director member functions.

```
%typemap(directorargout) int {
  $result = SWIG_AppendOutput( $result, NUM2INT($1) );
}
```

The following special variables are available:

| | |
|---|---|
| $result | Result that the director function returns |
| $input | Ruby object being sent to the function |
| $symname | name of the function/method being wrapped |
| $1...n | Argument being sent to the function |
| $1_name | Name of the argument (if provided) |
| $1_type | The actual C datatype matched by the typemap |
| $1_ltype | The assignable version of the C datatype matched by the typemap |
| this | C++ this, referring to the instance of the class itself |

#### 35.7.6.17 ret typemap

Cleanup of function return values

#### 35.7.6.18 globalin typemap

Setting of C global variables

### 35.7.7 Typemap variables

Within a typemap, a number of special variables prefaced with a `$` may appear. A full list of variables can be found in the "Typemaps" chapter. This is a list of the most common variables:

`$1`

A C local variable corresponding to the actual type specified in the `%typemap` directive. For input values, this is a C local variable that is supposed to hold an argument value. For output values, this is the raw result that is supposed to be returned to Ruby.

`$input`

A `VALUE` holding a raw Ruby object with an argument or variable value.

`$result`

A `VALUE` that holds the result to be returned to Ruby.

`$1_name`

The parameter name that was matched.

`$1_type`

The actual C datatype matched by the typemap.

`$1_ltype`

An assignable version of the datatype matched by the typemap (a type that can appear on the left-hand-side of a C assignment operation). This type is stripped of qualifiers and may be an altered version of `$1_type`. All arguments and local variables in wrapper functions are declared using this type so that their values can be properly assigned.

`$symname`

The Ruby name of the wrapper function being created.

### 35.7.8 Useful Functions

When you write a typemap, you usually have to work directly with Ruby objects. The following functions may prove to be useful. (These functions plus many more can be found in *Programming Ruby* book, by David Thomas and Andrew Hunt.)

In addition, we list equivalent functions that SWIG defines, which provide a language neutral conversion (these functions are defined for each swig language supported). If you are trying to create a swig file that will work under multiple languages, it is recommended you stick to the swig functions instead of the native Ruby functions. That should help you avoid having to rewrite a lot of typemaps across multiple languages.

#### 35.7.8.1 C Datatypes to Ruby Objects

| RUBY | SWIG | |
|---|---|---|
| INT2NUM(long or int) | SWIG_From_int(int x) | int to Fixnum or Bignum |
| INT2FIX(long or int) | | int to Fixnum (faster than INT2NUM) |
| CHR2FIX(char) | SWIG_From_char(char x) | char to Fixnum |
| rb_str_new2(char*) | SWIG_FromCharPtrAndSize(char*, size_t) | char* to String |
| rb_float_new(double) | SWIG_From_double(double),<br>SWIG_From_float(float) | float/double to Float |

#### 35.7.8.2 Ruby Objects to C Datatypes

Here, while the Ruby versions return the value directly, the SWIG versions do not, but return a status value to indicate success ( `SWIG_OK`). While more awkward to use, this allows you to write typemaps that report more helpful error messages, like:

```
%typemap(in) size_t (int ok)
  ok = SWIG_AsVal_size_t($input, &$1);
  if (!SWIG_IsOK(ok)) {
    SWIG_exception_fail(SWIG_ArgError(ok), Ruby_Format_TypeError( "$1_name", "$1_type", "$symname", $argnum, $input));
  }
}
```

| | |
|---|---|
| int NUM2INT(Numeric) | SWIG_AsVal_int(VALUE, int*) |
| int FIX2INT(Numeric) | SWIG_AsVal_int(VALUE, int*) |
| unsigned int NUM2UINT(Numeric) | SWIG_AsVal_unsigned_SS_int(VALUE, int*) |
| unsigned int FIX2UINT(Numeric) | SWIG_AsVal_unsigned_SS_int(VALUE, int*) |
| long NUM2LONG(Numeric) | SWIG_AsVal_long(VALUE, long*) |
| long FIX2LONG(Numeric) | SWIG_AsVal_long(VALUE, long*) |
| unsigned long FIX2ULONG(Numeric) | SWIG_AsVal_unsigned_SS_long(VALUE, unsigned long*) |
| char NUM2CHR(Numeric or String) | SWIG_AsVal_char(VALUE, int*) |
| char * StringValuePtr(String) | SWIG_AsCharPtrAndSize(VALUE, char**, size_t*, int* alloc) |
| char * rb_str2cstr(String, int*length) | |
| double NUM2DBL(Numeric) | (double) SWIG_AsVal_int(VALUE) or similar |

#### 35.7.8.3 Macros for VALUE

`RSTRING_LEN(str)`

length of the Ruby string

`RSTRING_PTR(str)`

pointer to string storage

`RARRAY_LEN(arr)`

length of the Ruby array

`RARRAY(arr)->capa`

capacity of the Ruby array

`RARRAY_PTR(arr)`

pointer to array storage

#### 35.7.8.4 Exceptions

`void rb_raise(VALUE exception, const char *fmt, ...)`

Raises an exception. The given format string *fmt* and remaining arguments are interpreted as with `printf()`.

`void rb_fatal(const char *fmt, ...)`

Raises a fatal exception, terminating the process. No rescue blocks are called, but ensure blocks will be called. The given format string *fmt* and remaining arguments are interpreted as with `printf()`.

`void rb_bug(const char *fmt, ...)`

Terminates the process immediately -- no handlers of any sort will be called. The given format string *fmt* and remaining arguments are interpreted as with `printf()`. You should call this function only if a fatal bug has been exposed.

`void rb_sys_fail(const char *msg)`

Raises a platform-specific exception corresponding to the last known system error, with the given string *msg*.

`VALUE rb_rescue(VALUE (*body)(VALUE), VALUE args, VALUE(*rescue)(VALUE, VALUE), VALUE rargs)`

Executes *body* with the given *args*. If a `StandardError` exception is raised, then execute *rescue* with the given *rargs*.

`VALUE rb_ensure(VALUE(*body)(VALUE), VALUE args, VALUE(*ensure)(VALUE), VALUE eargs)`

Executes *body* with the given *args*. Whether or not an exception is raised, execute *ensure* with the given *rargs* after *body* has completed.

`VALUE rb_protect(VALUE (*body)(VALUE), VALUE args, int *result)`

Executes *body* with the given *args* and returns nonzero in result if any exception was raised.

`void rb_notimplement()`

Raises a `NotImpError` exception to indicate that the enclosed function is not implemented yet, or not available on this platform.

`void rb_exit(int status)`

Exits Ruby with the given *status*. Raises a `SystemExit` exception and calls registered exit functions and finalizers.

`void rb_warn(const char *fmt, ...)`

Unconditionally issues a warning message to standard error. The given format string *fmt* and remaining arguments are interpreted as with `printf()`.

`void rb_warning(const char *fmt, ...)`

Conditionally issues a warning message to standard error if Ruby was invoked with the `-w` flag. The given format string *fmt* and remaining arguments are interpreted as with `printf()`.

#### 35.7.8.5 Iterators

`void rb_iter_break()`

Breaks out of the enclosing iterator block.

`VALUE rb_each(VALUE obj)`

Invokes the `each` method of the given *obj*.

`VALUE rb_yield(VALUE arg)`

Transfers execution to the iterator block in the current context, passing *arg* as an argument. Multiple values may be passed in an array.

`int rb_block_given_p()`

Returns `true` if `yield` would execute a block in the current context; that is, if a code block was passed to the current method and is available to be called.

`VALUE rb_iterate(VALUE (*method)(VALUE), VALUE args, VALUE (*block)(VALUE, VALUE), VALUE arg2)`

Invokes *method* with argument *args* and block *block*. A `yield` from that method will invoke *block* with the argument given to `yield`, and a second argument *arg2*.

`VALUE rb_catch(const char *tag, VALUE (*proc)(VALUE, VALUE), VALUE value)`

Equivalent to Ruby's `catch`.

`void rb_throw(const char *tag, VALUE value)`

Equivalent to Ruby's `throw`.

### 35.7.9 Typemap Examples

This section includes a few examples of typemaps. For more examples, you might look at the examples in the `Example/ruby` directory.

### 35.7.10 Converting a Ruby array to a char **

A common problem in many C programs is the processing of command line arguments, which are usually passed in an array of `NULL` terminated strings. The following SWIG interface file allows a Ruby Array instance to be used as a `char **` object.

```
%module argv

// This tells SWIG to treat char ** as a special case
%typemap(in) char ** {
  /* Get the length of the array */
  int size = RARRAY($input)->len;
  int i;
```

```
  $1 = (char **) malloc((size+1)*sizeof(char *));
  /* Get the first element in memory */
  VALUE *ptr = RARRAY($input)->ptr;
  for (i=0; i < size; i++, ptr++) {
    /* Convert Ruby Object String to char* */
    $1[i]= StringValuePtr(*ptr);
  }
  $1[i]=NULL; /* End of list */
}

// This cleans up the char ** array created before
// the function call

%typemap(freearg) char ** {
  free((char *) $1);
}

// Now a test function
%inline %{
int print_args(char **argv) {
  int i = 0;
  while (argv[i]) {
    printf("argv[%d] = %s\n", i, argv[i]);
    i++;
  }
  return i;
}
%}
```

When this module is compiled, the wrapped C function now operates as follows :

```
require 'Argv'
Argv.print_args(["Dave", "Mike", "Mary", "Jane", "John"])
argv[0] = Dave
argv[1] = Mike
argv[2] = Mary
argv[3] = Jane
argv[4] = John
```

In the example, two different typemaps are used. The "in" typemap is used to receive an input argument and convert it to a C array. Since dynamic memory allocation is used to allocate memory for the array, the "freearg" typemap is used to later release this memory after the execution of the C function.

### 35.7.11 Collecting arguments in a hash

Ruby's solution to the "keyword arguments" capability of some other languages is to allow the programmer to pass in one or more key-value pairs as arguments to a function. All of those key-value pairs are collected in a single `Hash` argument that's presented to the function. If it makes sense, you might want to provide similar functionality for your Ruby interface. For example, suppose you'd like to wrap this C function that collects information about people's vital statistics:

```
void setVitalStats(const char *person, int nattributes, const char **names, int *values);
```

and you'd like to be able to call it from Ruby by passing in an arbitrary number of key-value pairs as inputs, e.g.

```
setVitalStats("Fred",
  'weight' => 270,
  'age' => 42
)
```

To make this work, you need to write a typemap that expects a Ruby `Hash` as its input and somehow extracts the last three arguments (*nattributes*, *names* and *values*) needed by your C function. Let's start with the basics:

```
%typemap(in) (int nattributes, const char **names, const int *values)
  (VALUE keys_arr, int i, VALUE key, VALUE val) {
}
```

This `%typemap` directive tells SWIG that we want to match any function declaration that has the specified types and names of arguments somewhere in the argument list. The fact that we specified the argument names (*nattributes*, *names* and *values*) in our typemap is significant; this ensures that SWIG won't try to apply this typemap to *other* functions it sees that happen to have a similar declaration with different argument names. The arguments that appear in the second set of parentheses (*keys_arr*, *i* , *key* and *val*) define local variables that our typemap will need.

Since we expect the input argument to be `aHash`, let's next add a check for that:

```
%typemap(in) (int nattributes, const char **names, const int *values)
  (VALUE keys_arr, int i, VALUE key, VALUE val) {
    Check_Type($input, T_HASH);
}
```

`Check_Type()` is just a macro (defined in the Ruby header files) that confirms that the input argument is of the correct type; if it isn't, an exception will be raised.

The next task is to determine how many key-value pairs are present in the hash; we'll assign this number to the first typemap argument ( `$1`). This is a little tricky since the Ruby/C API doesn't provide a public function for querying the size of a hash, but we can get around that by calling the hash's *size* method directly and converting its result to a C `int` value:

```
%typemap(in) (int nattributes, const char **names, const int *values)
  (VALUE keys_arr, int i, VALUE key, VALUE val) {
    Check_Type($input, T_HASH);
    $1 = NUM2INT(rb_funcall($input, rb_intern("size"), 0, Qnil));
}
```

So now we know the number of attributes. Next we need to initialize the second and third typemap arguments (i.e. the two C arrays) to `NULL` and set the stage for extracting the keys and values from the hash:

```
%typemap(in) (int nattributes, const char **names, const int *values)
  (VALUE keys_arr, int i, VALUE key, VALUE val) {
    Check_Type($input, T_HASH);
    $1 = NUM2INT(rb_funcall($input, rb_intern("size"), 0, Qnil));
    $2 = NULL;
    $3 = NULL;
    if ($1 > 0) {
      $2 = (char **) malloc($1*sizeof(char *));
      $3 = (int *) malloc($1*sizeof(int));
    }
}
```

There are a number of ways we could extract the keys and values from the input hash, but the simplest approach is to first call the hash's *keys* method (which returns a Ruby array of the keys) and then start looping over the elements in that array:

```
%typemap(in) (int nattributes, const char **names, const int *values)
  (VALUE keys_arr, int i, VALUE key, VALUE val) {
    Check_Type($input, T_HASH);
    $1 = NUM2INT(rb_funcall($input, rb_intern("size"), 0, Qnil));
    $2 = NULL;
    $3 = NULL;
    if ($1 > 0) {
      $2 = (char **) malloc($1*sizeof(char *));
      $3 = (int *) malloc($1*sizeof(int));
      keys_arr = rb_funcall($input, rb_intern("keys"), 0, Qnil);
      for (i = 0; i < $1; i++) {
      }
    }
}
```

Recall that *keys_arr* and *i* are local variables for this typemap. For each element in the *keys_arr* array, we want to get the key itself, as well as the value corresponding to that key in the hash:

```
%typemap(in) (int nattributes, const char **names, const int *values)
  (VALUE keys_arr, int i, VALUE key, VALUE val) {
    Check_Type($input, T_HASH);
    $1 = NUM2INT(rb_funcall($input, rb_intern("size"), 0, Qnil));
    $2 = NULL;
    $3 = NULL;
    if ($1 > 0) {
      $2 = (char **) malloc($1*sizeof(char *));
      $3 = (int *) malloc($1*sizeof(int));
      keys_arr = rb_funcall($input, rb_intern("keys"), 0, Qnil);
      for (i = 0; i < $1; i++) {
        key = rb_ary_entry(keys_arr, i);
        val = rb_hash_aref($input, key);
      }
    }
}
```

To be safe, we should again use the `Check_Type()` macro to confirm that the key is a `String` and the value is a `Fixnum` :

```
%typemap(in) (int nattributes, const char **names, const int *values)
  (VALUE keys_arr, int i, VALUE key, VALUE val) {
    Check_Type($input, T_HASH);
    $1 = NUM2INT(rb_funcall($input, rb_intern("size"), 0, Qnil));
    $2 = NULL;
    $3 = NULL;
    if ($1 > 0) {
      $2 = (char **) malloc($1*sizeof(char *));
      $3 = (int *) malloc($1*sizeof(int));
      keys_arr = rb_funcall($input, rb_intern("keys"), 0, Qnil);
      for (i = 0; i < $1; i++) {
        key = rb_ary_entry(keys_arr, i);
        val = rb_hash_aref($input, key);
        Check_Type(key, T_STRING);
        Check_Type(val, T_FIXNUM);
      }
    }
}
```

Finally, we can convert these Ruby objects into their C equivalents and store them in our local C arrays:

```
%typemap(in) (int nattributes, const char **names, const int *values)
  (VALUE keys_arr, int i, VALUE key, VALUE val) {
  Check_Type($input, T_HASH);
  $1 = NUM2INT(rb_funcall($input, rb_intern("size"), 0, Qnil));
  $2 = NULL;
  $3 = NULL;
  if ($1 > 0) {
    $2 = (char **) malloc($1*sizeof(char *));
    $3 = (int *) malloc($1*sizeof(int));
    keys_arr = rb_funcall($input, rb_intern("keys"), 0, Qnil);
    for (i = 0; i < $1; i++) {
      key = rb_ary_entry(keys_arr, i);
      val = rb_hash_aref($input, key);
      Check_Type(key, T_STRING);
      Check_Type(val, T_FIXNUM);
      $2[i] = StringValuePtr(key);
      $3[i] = NUM2INT(val);
    }
  }
}
```

We're not done yet. Since we used `malloc()` to dynamically allocate the memory used for the *names* and *values* arguments, we need to provide a corresponding "freearg" typemap to free

that memory so that there is no memory leak. Fortunately, this typemap is a lot easier to write:

```
%typemap(freearg) (int nattributes, const char **names, const int *values) {
  free((void *) $2);
  free((void *) $3);
}
```

All of the code for this example, as well as a sample Ruby program that uses the extension, can be found in the `Examples/ruby/hashargs` directory of the SWIG distribution.

### 35.7.12 Pointer handling

Occasionally, it might be necessary to convert pointer values that have been stored using the SWIG typed-pointer representation. Since there are several ways in which pointers can be represented, the following two functions are used to safely perform this conversion:

`int SWIG_ConvertPtr(VALUE obj, void **ptr, swig_type_info *ty, int flags)`

Converts a Ruby object *obj* to a C pointer whose address is *ptr* (i.e. *ptr* is a pointer to a pointer). The third argument, *ty*, is a pointer to a SWIG type descriptor structure. If *ty* is not `NULL`, that type information is used to validate type compatibility and other aspects of the type conversion. If *flags* is non-zero, any type errors encountered during this validation result in a Ruby `TypeError` exception being raised; if *flags* is zero, such type errors will cause `SWIG_ConvertPtr()` to return -1 but not raise an exception. If *ty* is `NULL`, no type-checking is performed.

`VALUE SWIG_NewPointerObj(void *ptr, swig_type_info *ty, int own)`

Creates a new Ruby pointer object. Here, *ptr* is the pointer to convert, *ty* is the SWIG type descriptor structure that describes the type, and *own* is a flag that indicates whether or not Ruby should take ownership of the pointer (i.e. whether Ruby should free this data when the corresponding Ruby instance is garbage-collected).

Both of these functions require the use of a special SWIG type-descriptor structure. This structure contains information about the mangled name of the datatype, type-equivalence information, as well as information about converting pointer values under C++ inheritance. For a type of `Foo *`, the type descriptor structure is usually accessed as follows:

```
Foo *foo;
SWIG_ConvertPtr($input, (void **) &foo, SWIGTYPE_p_Foo, 1);

VALUE obj;
obj = SWIG_NewPointerObj(f, SWIGTYPE_p_Foo, 0);
```

In a typemap, the type descriptor should always be accessed using the special typemap variable `$1_descriptor`. For example:

```
%typemap(in) Foo * {
  SWIG_ConvertPtr($input, (void **) &$1, $1_descriptor, 1);
}
```

#### 35.7.12.1 Ruby Datatype Wrapping

`VALUE Data_Wrap_Struct(VALUE class, void (*mark)(void *), void (*free)(void *), void *ptr)`

Given a pointer *ptr* to some C data, and the two garbage collection routines for this data (*mark* and *free*), return a `VALUE` for the Ruby object.

`VALUE Data_Make_Struct(VALUE class, c-type, void (*mark)(void *), void (*free)(void *), c-type *ptr)`

Allocates a new instance of a C data type *c-type*, assigns it to the pointer *ptr*, then wraps that pointer with `Data_Wrap_Struct()` as above.

`Data_Get_Struct(VALUE obj, c-type, c-type *ptr)`

Retrieves the original C pointer of type *c-type* from the data object *obj* and assigns that pointer to *ptr*.

### 35.7.13 Example: STL Vector to Ruby Array

Another use for macros and type maps is to create a Ruby array from a STL vector of pointers. In essence, copy of all the pointers in the vector into a Ruby array. The use of the macro is to make the typemap so generic that any vector with pointers can use the type map. The following is an example of how to construct this type of macro/typemap and should give insight into constructing similar typemaps for other STL structures:

```
%define PTR_VECTOR_TO_RUBY_ARRAY(vectorclassname, classname)
%typemap(out) vectorclassname &, const vectorclassname & {
  VALUE arr = rb_ary_new2($1->size());
  vectorclassname::iterator i = $1->begin(), iend = $1->end();
  for ( ; i!=iend; i++ )
    rb_ary_push(arr, Data_Wrap_Struct(c ## classname.klass, 0, 0, *i));
  $result = arr;
}
%typemap(out) vectorclassname, const vectorclassname {
  VALUE arr = rb_ary_new2($1.size());
  vectorclassname::iterator i = $1.begin(), iend = $1.end();
  for ( ; i!=iend; i++ )
    rb_ary_push(arr, Data_Wrap_Struct(c ## classname.klass, 0, 0, *i));
  $result = arr;
}
%enddef
```

Note, that the "`c ## classname.klass`" is used in the preprocessor step to determine the actual object from the class name.

To use the macro with a class Foo, the following is used:

```
PTR_VECTOR_TO_RUBY_ARRAY(vector<foo *="">, Foo)
```

It is also possible to create a STL vector of Ruby objects:

```
%define RUBY_ARRAY_TO_PTR_VECTOR(vectorclassname, classname)
%typemap(in) vectorclassname &, const vectorclassname & {
  Check_Type($input, T_ARRAY);
  vectorclassname *vec = new vectorclassname;
  int len = RARRAY($input)->len;
  for (int i=0; i!=len; i++) {
```

```
    VALUE inst = rb_ary_entry($input, i);
    //The following _should_ work but doesn't on HPUX
    // Check_Type(inst, T_DATA);
    classname *element = NULL;
    Data_Get_Struct(inst, classname, element);
    vec->push_back(element);
  }
  $1 = vec;
}

%typemap(freearg) vectorclassname &, const vectorclassname & {
  delete $1;
}
%enddef
```

It is also possible to create a Ruby array from a vector of static data types:

```
%define VECTOR_TO_RUBY_ARRAY(vectorclassname, classname)
%typemap(out) vectorclassname &, const vectorclassname & {
  VALUE arr = rb_ary_new2($1->size());
  vectorclassname::iterator i = $1->begin(), iend = $1->end();
  for ( ; i!=iend; i++ )
    rb_ary_push(arr, Data_Wrap_Struct(c ## classname.klass, 0, 0, &(*i)));
  $result = arr;
}
%typemap(out) vectorclassname, const vectorclassname {
  VALUE arr = rb_ary_new2($1.size());
  vectorclassname::iterator i = $1.begin(), iend = $1.end();
  for ( ; i!=iend; i++ )
    rb_ary_push(arr, Data_Wrap_Struct(c ## classname.klass, 0, 0, &(*i)));
  $result = arr;
}
%enddef
```

Note that this is mostly an example of typemaps. If you want to use the STL with ruby, you are advised to use the standard swig STL library, which does much more than this. Refer to the section called the C++ Standard Template Library.

## 35.8 Docstring Features

Using ri and rdoc web pages in Ruby libraries is a common practice. Given the way that SWIG generates the extensions by default, your users will normally not get any documentation for it, even if they run 'rdoc' on the resulting .c or .cxx file.

The features described in this section make it easy for you to add rdoc strings to your modules, functions and methods that can then be read by Ruby's rdoc tool to generate html web pages, ri documentation, Windows chm file and an .xml description.

rdoc can then be run from a console or shell window on a swig generated file.

For example, to generate html web pages from a C++ file, you'd do:

```
$ rdoc -E cxx=c -f html file_wrap.cxx
```

To generate ri documentation from a c wrap file, you could do:

```
$ rdoc -r file_wrap.c
```

### 35.8.1 Module docstring

Ruby allows a docstring at the beginning of the file before any other statements, and it is typically used to give a general description of the entire module. SWIG supports this by setting an option of the `%module` directive. For example:

```
%module(docstring="This is the example module's docstring") example
```

When you have more than just a line or so then you can retain the easy readability of the `%module` directive by using a macro. For example:

```
%define DOCSTRING
"The `XmlResource` class allows program resources defining menus,
layout of controls on a panel, etc. to be loaded from an XML file."
%enddef

%module(docstring=DOCSTRING) xrc
```

### 35.8.2 %feature("autodoc")

Since SWIG does know everything about the function it wraps, it is possible to generate an rdoc containing the parameter types, names and default values. Since Ruby ships with one of the best documentation systems of any language, it makes sense to take advantage of it.

SWIG's Ruby module provides support for the "autodoc" feature, which when attached to a node in the parse tree will cause an rdoc comment to be generated in the wrapper file that includes the name of the function, parameter names, default values if any, and return type if any. There are also several options for autodoc controlled by the value given to the feature, described below.

#### 35.8.2.1 %feature("autodoc", "0")

When the "0" option is given then the types of the parameters will *not* be included in the autodoc string. For example, given this function prototype:

```
%feature("autodoc", "0");
bool function_name(int x, int y, Foo* foo=NULL, Bar* bar=NULL);
```

Then Ruby code like this will be generated:

```
function_name(x, y, foo=nil, bar=nil) -> bool
  ...
```

#### 35.8.2.2 %feature("autodoc", "1")

When the "1" option is used then the parameter types *will* be used in the rdoc string. In addition, an attempt is made to simplify the type name such that it makes more sense to the Ruby user. Pointer, reference and const info is removed, `%rename`'s are evaluated, etc. (This is not always successful, but works most of the time. See the next section for what to do when it doesn't.) Given the example above, then turning on the parameter types with the "1" option will result in rdoc code like this:

```
function_name(int x, int y, Foo foo=nil, Bar bar=nil) -> bool
  ...
```

#### 35.8.2.3 %feature("autodoc", "2")

When the "2" option is used then the parameter types will not be used in the rdoc string. However, they will be listed in full after the function. Given the example above, then turning on the parameter types with the "2" option will result in Ruby code like this:

#### 35.8.2.4 %feature("autodoc", "3")

When the "3" option is used then the function will be documented using a combination of "1" and "2" above. Given the example above, then turning on the parameter types with the "2" option will result in Ruby code like this:

```
function_name(int x, int y, Foo foo=nil, Bar bar=nil) -> bool

Parameters:
        x - int
        y - int
        foo - Foo
        bar - Bar
```

#### 35.8.2.5 %feature("autodoc", "docstring")

Finally, there are times when the automatically generated autodoc string will make no sense for a Ruby programmer, particularly when a typemap is involved. So if you give an explicit value for the autodoc feature then that string will be used in place of the automatically generated string. For example:

```
%feature("autodoc", "GetPosition() -> (x, y)") GetPosition;
void GetPosition(int* OUTPUT, int* OUTPUT);
```

### 35.8.3 %feature("docstring")

In addition to the autodoc strings described above, you can also attach any arbitrary descriptive text to a node in the parse tree with the "docstring" feature. When the proxy module is generated then any docstring associated with classes, function or methods are output. If an item already has an autodoc string then it is combined with the docstring and they are output together.

## 35.9 Advanced Topics

### 35.9.1 Operator overloading

SWIG allows operator overloading with, by using the `%extend` or `%rename` commands in SWIG and the following operator names (derived from Python):

| | |
|---|---|
| **General** | |
| __repr__ | inspect |
| __str__ | to_s |
| __cmp__ | <=> |
| __hash__ | hash |
| __nonzero__ | nonzero? |
| **Callable** | |
| __call__ | call |
| **Collection** | |
| __len__ | length |
| __getitem__ | [] |
| __setitem__ | []= |
| **Numeric** | |
| __add__ | + |
| __sub__ | - |
| __mul__ | * |
| __div__ | / |
| __mod__ | % |
| __divmod__ | divmod |
| __pow__ | ** |
| __lshift__ | << |
| __rshift__ | >> |
| __and__ | & |
| __xor__ | ^ |
| __or__ | \| |
| __neg__ | -@ |
| __pos__ | +@ |
| __abs__ | abs |
| __invert__ | ~ |
| __int__ | to_i |
| __float__ | to_f |
| __coerce__ | coerce |

| Additions in 1.3.13 | |
|---|---|
| __lt__ | < |
| __le__ | <= |
| __eq__ | == |
| __gt__ | > |
| __ge__ | >= |

Note that although SWIG supports the `__eq__` magic method name for defining an equivalence operator, there is no separate method for handling *inequality* since Ruby parses the expression *a != b* as *!(a == b)*.

### 35.9.2 Creating Multi-Module Packages

The chapter on Working with Modules discusses the basics of creating multi-module extensions with SWIG, and in particular the considerations for sharing runtime type information among the different modules.

As an example, consider one module's interface file (`shape.i`) that defines our base class:

```
%module shape

%{
#include "Shape.h"
%}

class Shape {
protected:
  double xpos;
  double ypos;
protected:
  Shape(double x, double y);
public:
  double getX() const;
  double getY() const;
};
```

We also have a separate interface file (`circle.i`) that defines a derived class:

```
%module circle

%{
#include "Shape.h"
#include "Circle.h"
%}

// Import the base class definition from Shape module
%import shape.i

class Circle : public Shape {
protected:
  double radius;
public:
  Circle(double x, double y, double r);
  double getRadius() const;
};
```

We'll start by building the **Shape** extension module:

```
$ swig -c++ -ruby shape.i
```

SWIG generates a wrapper file named `shape_wrap.cxx`. To compile this into a dynamically loadable extension for Ruby, prepare an `extconf.rb` script using this template:

```
require 'mkmf'

# Since the SWIG runtime support library for Ruby
# depends on the Ruby library, make sure it's in the list
# of libraries.
$libs = append_library($libs, Config::CONFIG['RUBY_INSTALL_NAME'])

# Create the makefile
create_makefile('shape')
```

Run this script to create a `Makefile` and then type `make` to build the shared library:

```
$ ruby extconf.rb
creating Makefile
$ make
g++ -fPIC -g -O2 -I. -I/usr/include/ruby-2.1.0 \
-I. -c shape_wrap.cxx
gcc -shared -L/usr/local/lib -o shape.so shape_wrap.o -L. \
-lruby -lruby -lc
```

Note that depending on your installation, the outputs may be slightly different; these outputs are those for a Linux-based development environment. The end result should be a shared library (here, `shape.so`) containing the extension module code. Now repeat this process in a separate directory for the **Circle** module:

1. Run SWIG to generate the wrapper code (`circle_wrap.cxx`);
2. Write an `extconf.rb` script that your end-users can use to create a platform-specific `Makefile` for the extension;
3. Build the shared library for this extension by typing `make` .

Once you've built both of these extension modules, you can test them interactively in IRB to confirm that the `Shape` and `Circle` modules are properly loaded and initialized:

```
$ irb
```

```
irb(main):001:0> require 'shape'
true
irb(main):002:0> require 'circle'
true
irb(main):003:0> c = Circle::Circle.new(5, 5, 20)
#<Circle::Circle:0xa097208>
irb(main):004:0> c.kind_of? Shape::Shape
true
irb(main):005:0> c.getX()
5.0
```

### 35.9.3 Specifying Mixin Modules

The Ruby language doesn't support multiple inheritance, but it does allow you to mix one or more modules into a class using Ruby's `include` method. For example, if you have a Ruby class that defines an *each* instance method, e.g.

```
class Set
  def initialize
  @members = []
  end

  def each
  @members.each { |m| yield m }
  end
end
```

then you can mix-in Ruby's `Enumerable` module to easily add a lot of functionality to your class:

```
class Set
  include Enumerable
  def initialize
    @members = []
  end
  def each
    @members.each { |m| yield m }
  end
end
```

To get the same benefit for your SWIG-wrapped classes, you can use the `%mixin` directive to specify the names of one or more modules that should be mixed-in to a class. For the above example, the SWIG interface specification might look like this:

```
%mixin Set "Enumerable";

class Set {
public:
  // Constructor
  Set();

  // Iterates through set members
  void each();
};
```

Multiple modules can be mixed into a class by providing a comma-separated list of module names to the `%mixin` directive, e.g.

```
%mixin Set "Fee, Fi, Fo, Fum";
```

Note that the `%mixin` directive is implemented using SWIG's "features" mechanism and so the same name matching rules used for other kinds of features apply (see the chapter on "Customization Features") for more details).

## 35.10 Memory Management

One of the most common issues in generating SWIG bindings for Ruby is proper memory management. The key to proper memory management is clearly defining whether a wrapper Ruby object owns the underlying C struct or C++ class. There are two possibilities:

- The Ruby object is responsible for freeing the C struct or C++ object
- The Ruby object should not free the C struct or C++ object because it will be freed by the underlying C or C++ code

To complicate matters, object ownership may transfer from Ruby to C++ (or vice versa) depending on what function or methods are invoked. Clearly, developing a SWIG wrapper requires a thorough understanding of how the underlying library manages memory.

### 35.10.1 Mark and Sweep Garbage Collector

Ruby uses a mark and sweep garbage collector. When the garbage collector runs, it finds all the "root" objects, including local variables, global variables, global constants, hardware registers and the C stack. For each root object, the garbage collector sets its mark flag to true and calls `rb_gc_mark` on the object. The job of`rb_gc_mark` is to recursively mark all the objects that a Ruby object has a reference to (ignoring those objects that have already been marked). Those objects, in turn, may reference other objects. This process will continue until all active objects have been "marked." After the mark phase comes the sweep phase. In the sweep phase, all objects that have not been marked will be garbage collected.

The Ruby C/API provides extension developers two hooks into the garbage collector - a "mark" function and a "sweep" function. By default these functions are set to NULL.

If a C struct or C++ class references any other Ruby objects, then it must provide a "mark" function. The "mark" function should identify any referenced Ruby objects by calling the rb_gc_mark function for each one. Unsurprisingly, this function will be called by the Ruby garbage during the "mark" phase.

During the sweep phase, Ruby destroys any unused objects. If any memory has been allocated in creating the underlying C struct or C++ struct, then a "free" function must be defined that deallocates this memory.

### 35.10.2 Object Ownership

As described above, memory management depends on clearly defining who is responsible for freeing the underlying C struct or C++ class. If the Ruby object is responsible for freeing the C++ object, then a "free" function must be registered for the object. If the Ruby object is not responsible for freeing the underlying memory, then a "free" function must not be registered for the object.

For the most part, SWIG takes care of memory management issues. The rules it uses are:

- When calling a C++ object's constructor from Ruby, SWIG will assign a "free" function thereby making the Ruby object responsible for freeing the C++ object
- When calling a C++ member function that returns a pointer, SWIG will not assign a "free" function thereby making the underlying library responsible for freeing the object.

To make this clearer, let's look at an example. Assume we have a Foo and a Bar class.

```
/* File "RubyOwernshipExample.h" */

class Foo
{
public:
  Foo() {}
  ~Foo() {}
};

class Bar
{
  Foo *foo_;
public:
  Bar(): foo_(new Foo) {}
  ~Bar() { delete foo_; }
  Foo* get_foo() { return foo_; }
  Foo* get_new_foo() { return new Foo; }
  void set_foo(Foo *foo) { delete foo_; foo_ = foo; }
};
```

First, consider this Ruby code:

```
foo = Foo.new
```

In this case, the Ruby code calls the underlying`Foo` C++ constructor, thus creating a new `foo` object. By default, SWIG will assign the new Ruby object a "free" function. When the Ruby object is garbage collected, the "free" function will be called. It in turn will call `Foo`'s destructor.

Next, consider this code:

```
bar = Bar.new
foo = bar.get_foo()
```

In this case, the Ruby code calls a C++ member function,`get_foo` . By default, SWIG will not assign the Ruby object a "free" function. Thus, when the Ruby object is garbage collected the underlying C++ `foo` object is not affected.

Unfortunately, the real world is not as simple as the examples above. For example:

```
bar = Bar.new
foo = bar.get_new_foo()
```

In this case, the default SWIG behavior for calling member functions is incorrect. The Ruby object should assume ownership of the returned object. This can be done by using the %newobject directive. See Object ownership and %newobject for more information.

The SWIG default mappings are also incorrect in this case:

```
foo = Foo.new
bar = Bar.new
bar.set_foo(foo)
```

Without modification, this code will cause a segmentation fault. When the Ruby `foo` object goes out of scope, it will free the underlying C++ `foo` object. However, when the Ruby bar object goes out of scope, it will call the C++ bar destructor which will also free the C++ `foo` object. The problem is that object ownership is transferred from the Ruby object to the C++ object when the `set_foo` method is called. This can be done by using the special DISOWN type map, which was added to the Ruby bindings in SWIG-1.3.26.

Thus, a correct SWIG interface file correct mapping for these classes is:

```
/* File RubyOwnershipExample.i */

%module RubyOwnershipExample

%{
#include "RubyOwnershipExample.h"
%}

class Foo
{
public:
  Foo();
  ~Foo();
};

class Bar
{
  Foo *foo_;
public:
  Bar();
  ~Bar();
  Foo* get_foo();

  %newobject get_new_foo;
  Foo* get_new_foo();

  %apply SWIGTYPE *DISOWN {Foo *foo};
  void set_foo(Foo *foo);
  %clear Foo *foo;
};
```

This code can be seen in swig/examples/ruby/tracking.

### 35.10.3 Object Tracking

The remaining parts of this section will use the class library shown below to illustrate different memory management techniques. The class library models a zoo and the animals it contains.

```
%module zoo

%{
#include <string>
#include <vector>

#include "zoo.h"
%}

class Animal
{
private:
  typedef std::vector<Animal*> AnimalsType;
  typedef AnimalsType::iterator IterType;
protected:
  AnimalsType animals;
protected:
  std::string name_;
public:
  // Construct an animal with this name
  Animal(const char* name) : name_(name) {}

  // Return the animal's name
  const char* get_name() const { return name.c_str(); }
};

class Zoo
{
protected:
 std::vector<Animal *> animals;

public:
  // Construct an empty zoo
  Zoo() {}

  /* Create a new animal. */
  static Animal* Zoo::create_animal(const char* name) {
    return new Animal(name);
  }

  // Add a new animal to the zoo
  void add_animal(Animal* animal) {
    animals.push_back(animal);
  }

  Animal* remove_animal(size_t i) {
    Animal* result = this->animals[i];
    IterType iter = this->animals.begin();
    std::advance(iter, i);
    this->animals.erase(iter);

    return result;
  }

  // Return the number of animals in the zoo
  size_t get_num_animals() const {
    return animals.size();
  }

  // Return a pointer to the ith animal
  Animal* get_animal(size_t i) const {
    return animals[i];
  }
};
```

Let's say you SWIG this code and then run IRB:

```
$ irb
irb(main):001:0> require 'example'
=> true

irb(main):002:0> tiger1 = Example::Animal.new("tiger1")
=> #<Example::Animal:0x2be3820>

irb(main):004:0> tiger1.get_name()
=> "tiger1"

irb(main):003:0> zoo = Example::Zoo.new()
=> #<Example::Zoo:0x2be0a60>

irb(main):006:0> zoo.add_animal(tiger)
=> nil

irb(main):007:0> zoo.get_num_animals()
=> 1

irb(main):007:0> tiger2 = zoo.remove_animal(0)
=> #<Example::Animal:0x2bd4a18>

irb(main):008:0> tiger2.get_name()
=> "tiger1"

irb(main):009:0> tiger1.equal?(tiger2)
=> false
```

Pay particular attention to the code `tiger1.equal?(tiger2)`. Note that the two Ruby objects are not the same - but they reference the same underlying C++ object. This can cause problems. For example:

```
irb(main):010:0> tiger1 = nil
=> nil

irb(main):011:0> GC.start
=> nil

irb(main):012:0> tiger2.get_name()
(irb):12: [BUG] Segmentation fault
```

After the garbage collector runs, as a result of our call to `GC.start`, calling `tiger2.get_name()` causes a segmentation fault. The problem is that when `tiger1` is garbage collected, it frees the underlying C++ object. Thus, when `tiger2` calls the `get_name()` method it invokes it on a destroyed object.

This problem can be avoided if SWIG enforces a one-to-one mapping between Ruby objects and C++ classes. This can be done via the use of the `%trackobjects` functionality available in SWIG-1.3.26. and later.

When the `%trackobjects` is turned on, SWIG automatically keeps track of mappings between C++ objects and Ruby objects. Note that enabling object tracking causes a slight performance degradation. Test results show this degradation to be about 3% to 5% when creating and destroying 100,000 animals in a row.

Since `%trackobjects` is implemented as a `%feature`, it uses the same name matching rules as other kinds of features (see the chapter on "Customization Features") . Thus it can be applied on a class-by-class basis if needed. To fix the example above:

```
%module example

%{
#include "example.h"
%}

/* Tell SWIG that create_animal creates a new object */
%newobject Zoo::create_animal;

/* Tell SWIG to keep track of mappings between C/C++ structs/classes. */
%trackobjects;

%include "example.h"
```

When this code runs we see:

```
$ irb
irb(main):001:0> require 'example'
=> true

irb(main):002:0> tiger1 = Example::Animal.new("tiger1")
=> #<Example::Animal:0x2be37d8>

irb(main):003:0> zoo = Example::Zoo.new()
=> #<Example::Zoo:0x2be0a18>

irb(main):004:0> zoo.add_animal(tiger1)
=> nil

irb(main):006:0> tiger2 = zoo.remove_animal(0)
=> #<Example::Animal:0x2be37d8>

irb(main):007:0> tiger1.equal?(tiger2)
=> true

irb(main):008:0> tiger1 = nil
=> nil

irb(main):009:0> GC.start
=> nil

irb(main):010:0> tiger.get_name()
=> "tiger1"
irb(main):011:0>
```

For those who are interested, object tracking is implemented by storing Ruby objects in a hash table and keying them on C++ pointers. The underlying API is:

```
static void SWIG_RubyAddTracking(void* ptr, VALUE object);
static VALUE SWIG_RubyInstanceFor(void* ptr) ;
static void SWIG_RubyRemoveTracking(void* ptr);
static void SWIG_RubyUnlinkObjects(void* ptr);
```

When an object is created, SWIG will automatically call the `SWIG_RubyAddTracking` method. Similarly, when an object is deleted, SWIG will call the `SWIG_RubyRemoveTracking`. When an object is returned to Ruby from C++, SWIG will use the `SWIG_RubyInstanceFor` method to ensure a one-to-one mapping from Ruby to C++ objects. Last, the `RubyUnlinkObjects` method unlinks a Ruby object from its underlying C++ object.

In general, you will only need to use the `SWIG_RubyInstanceFor` , which is required for implementing mark functions as shown below. However, if you implement your own free functions (see below) you may also have to call the `SWIG_RubyRemoveTracking` and `RubyUnlinkObjects` methods.

### 35.10.4 Mark Functions

With a bit more testing, we see that our class library still has problems. For example:

```
$ irb
irb(main):001:0> require 'example'
=> true

irb(main):002:0> tiger1 = Example::Animal.new("tiger1")
=> #<Example::Animal:0x2bea6a8>
```

```
irb(main):003:0> zoo = Example::Zoo.new()
=> #<Example::Zoo:0x2be7960>

irb(main):004:0> zoo.add_animal(tiger1)
=> nil

irb(main):007:0> tiger1 = nil
=> nil

irb(main):007:0> GC.start
=> nil

irb(main):005:0> tiger2 = zoo.get_animal(0)
(irb):12: [BUG] Segmentation fault
```

The problem is that Ruby does not know that the `zoo` object contains a reference to a Ruby object. Thus, when Ruby garbage collects `tiger1` it frees the underlying C++ object.

This can be fixed by implementing a `mark` function as described above in the Mark and Sweep Garbage Collector section. You can specify a mark function by using the `%markfunc` directive. Since the `%markfunc` directive is implemented using SWIG's' "features" mechanism it uses the same name matching rules as other kinds of features (see the chapter on "Customization Features" for more details).

A `mark` function takes a single argument, which is a pointer to the C++ object being marked; it should, in turn, call `rb_gc_mark()` for any instances that are reachable from the current object. The mark function for our `Zoo` class should therefore loop over all of the C++ animal objects in the zoo object, look up their Ruby object equivalent, and then call `rb_gc_mark()`. One possible implementation is:

```
%module example

%{
#include "example.h"
%}

/* Keep track of mappings between C/C++ structs/classes
 and Ruby objects so we can implement a mark function. */
%trackobjects;

/* Specify the mark function */
%markfunc Zoo "mark_Zoo";

%include "example.h"

%header %{

static void mark_Zoo(void* ptr) {
  Zoo* zoo = (Zoo*) ptr;

  /* Loop over each object and tell the garbage collector
  that we are holding a reference to them. */
  int count = zoo->get_num_animals();

  for(int i = 0; i < count; ++i) {
    Animal* animal = zoo->get_animal(i);
    VALUE object = SWIG_RubyInstanceFor(animal);

    if (object != Qnil) {
      rb_gc_mark(object);
    }
  }
}
%}
```

Note the `mark` function is dependent on the `SWIG_RUBY_InstanceFor` method, and thus requires that `%trackobjects` is enabled. For more information, please refer to the ruby_track_objects.i test case in the SWIG test suite.

When this code is compiled we now see:

```
$ irb
irb(main):002:0> tiger1=Example::Animal.new("tiger1")
=> #<Example::Animal:0x2be3bf8>

irb(main):003:0> Example::Zoo.new()
=> #<Example::Zoo:0x2be1780>

irb(main):004:0> zoo = Example::Zoo.new()
=> #<Example::Zoo:0x2bde9c0>

irb(main):005:0> zoo.add_animal(tiger1)
=> nil

irb(main):009:0> tiger1 = nil
=> nil

irb(main):010:0> GC.start
=> nil
irb(main):014:0> tiger2 = zoo.get_animal(0)
=> #<Example::Animal:0x2be3bf8>

irb(main):015:0> tiger2.get_name()
=> "tiger1"
irb(main):016:0>
```

This code can be seen in swig/examples/ruby/mark_function.

### 35.10.5 Free Functions

By default, SWIG creates a "free" function that is called when a Ruby object is garbage collected. The free function simply calls the C++ object's destructor.

However, sometimes an appropriate destructor does not exist or special processing needs to be performed before the destructor is called. Therefore, SWIG allows you to manually specify a "free" function via the use of the `%freefunc` directive. The `%freefunc` directive is implemented using SWIG's' "features" mechanism and so the same name matching rules used for other

kinds of features apply (see the chapter on "Customization Features") for more details).

IMPORTANT ! - If you define your own free function, then you must ensure that you call the underlying C++ object's destructor. In addition, if object tracking is activated for the object's class, you must also call the `SWIG_RubyRemoveTracking` function (of course call this before you destroy the C++ object). Note that it is harmless to call this method if object tracking if off so it is advised to always call it.

Note there is a subtle interaction between object ownership and free functions. A custom defined free function will only be called if the Ruby object owns the underlying C++ object. This also to Ruby objects which are created, but then transfer ownership to C++ objects via the use of the `disown` typemap described above.

To show how to use the `%freefunc` directive, let's slightly change our example. Assume that the zoo object is responsible for freeing any animal that it contains. This means that the `Zoo::add_animal` function should be marked with `aDISOWN` typemap and the destructor should be updated as below:

```
Zoo::~Zoo() {
  IterType iter = this->animals.begin();
  IterType end = this->animals.end();

  for(iter; iter != end; ++iter) {
    Animal* animal = *iter;
    delete animal;
  }
}
```

When we use these objects in IRB we see:

```
$irb
irb(main):002:0> require 'example'
=> true

irb(main):003:0> zoo = Example::Zoo.new()
=> #<Example::Zoo:0x2be0fe8>

irb(main):005:0> tiger1 = Example::Animal.new("tiger1")
=> #<Example::Animal:0x2bda760>

irb(main):006:0> zoo.add_animal(tiger1)
=> nil

irb(main):007:0> zoo = nil
=> nil

irb(main):008:0> GC.start
=> nil

irb(main):009:0> tiger1.get_name()
(irb):12: [BUG] Segmentation fault
```

The error happens because the C++ `animal` object is freed when the `zoo` object is freed. Although this error is unavoidable, we can at least prevent the segmentation fault. To do this requires enabling object tracking and implementing a custom free function that calls the `SWIG_RubyUnlinkObjects` function for each animal object that is destroyed. The `SWIG_RubyUnlinkObjects` function notifies SWIG that a Ruby object's underlying C++ object is no longer valid. Once notified, SWIG will intercept any calls from the existing Ruby object to the destroyed C++ object and raise an exception.

```
%module example

%{
#include "example.h"
%}

/* Specify that ownership is transferred to the zoo when calling add_animal */
%apply SWIGTYPE *DISOWN { Animal* animal };

/* Track objects */
%trackobjects;

/* Specify the mark function */
%freefunc Zoo "free_Zoo";

%include "example.h"

%header %{
  static void free_Zoo(void* ptr) {
    Zoo* zoo = (Zoo*) ptr;

    /* Loop over each animal */
    int count = zoo->get_num_animals();

    for(int i = 0; i < count; ++i) {
      /* Get an animal */
      Animal* animal = zoo->get_animal(i);

      /* Unlink the Ruby object from the C++ object */
      SWIG_RubyUnlinkObjects(animal);

      /* Now remove the tracking for this animal */
      SWIG_RubyRemoveTracking(animal);
    }

    /* Now call SWIG_RubyRemoveTracking for the zoo */
    SWIG_RubyRemoveTracking(ptr);
    /* Now free the zoo which will free the animals it contains */
    delete zoo;
  }
%}
```

Now when we use these objects in IRB we see:

```
$irb
```

```
irb(main):002:0> require 'example'
=> true

irb(main):003:0> zoo = Example::Zoo.new()
=> #<Example::Zoo:0x2be0fe8>

irb(main):005:0> tiger1 = Example::Animal.new("tiger1")
=> #<Example::Animal:0x2bda760>

irb(main):006:0> zoo.add_animal(tiger1)
=> nil

irb(main):007:0> zoo = nil
=> nil

irb(main):008:0> GC.start
=> nil

irb(main):009:0> tiger1.get_name()
RuntimeError: This Animal * already released
 from (irb):10:in `get_name'
 from (irb):10
irb(main):011:0>
```

Notice that SWIG can now detect the underlying C++ object has been freed, and thus raises a runtime exception.

This code can be seen in swig/examples/ruby/free_function.

### 35.10.6 Embedded Ruby and the C++ Stack

As has been said, the Ruby GC runs and marks objects before its sweep phase. When the garbage collector is called, it will also try to mark any Ruby objects (VALUE) it finds in the machine registers and in the C++ stack.

The stack is basically the history of the functions that have been called and also contains local variables, such as the ones you define whenever you do inside a function:

```
VALUE obj;
```

For ruby to determine where its stack space begins, during initialization a normal Ruby interpreter will call the ruby_init() function which in turn will call a function called Init_stack or similar. This function will store a pointer to the location where the stack points at that point in time.

ruby_init() is presumed to always be called within the main() function of your program and whenever the GC is called, ruby will assume that the memory between the current location in memory and the pointer that was stored previously represents the stack, which may contain local (and temporary) VALUE ruby objects. Ruby will then be careful not to remove any of those objects in that location.

So far so good. For a normal Ruby session, all the above is completely transparent and magic to the extensions developer.

However, with an embedded Ruby, it may not always be possible to modify main() to make sure ruby_init() is called there. As such, ruby_init() will likely end up being called from within some other function. This can lead Ruby to measure incorrectly where the stack begins and can result in Ruby incorrectly collecting those temporary VALUE objects that are created once another function is called. The end result: random crashes and segmentation faults.

This problem will often be seen in director functions that are used for callbacks, for example.

To solve the problem, SWIG can now generate code with director functions containing the optional macros SWIG_INIT_STACK and SWIG_RELEASE_STACK. These macros will try to force Ruby to reinitialize the beginning of the stack the first time a director function is called. This will lead Ruby to measure and not collect any VALUE objects defined from that point on.

To mark functions to either reset the ruby stack or not, you can use:

```
%initstack Class::memberfunction;   // only re-init the stack in this director method
%ignorestack Class::memberfunction; // do not re-init the stack in this director method
%initstack Class;                   // init the stack on all the methods of this class
%initstack;                         // all director functions will re-init the stack
```

# 36 SWIG and Scilab


- Preliminaries
- Running SWIG
  - Generating the module
  - Building the module
  - Loading the module
  - Using the module
  - Scilab command line options
- A basic tour of C/C++ wrapping
  - Overview
  - Identifiers
  - Functions
    - Argument passing
    - Multiple output arguments
  - Global variables
  - Constants and enumerations
    - Constants
    - Enumerations
  - Pointers
    - Utility functions
    - Null pointers:
  - Structures
  - C++ classes
  - C++ inheritance
  - C++ overloading
  - Pointers, references, values, and arrays
  - C++ templates
  - C++ operators
  - C++ namespaces
  - C++ exceptions
  - C++ STL
- Type mappings and libraries
  - Default primitive type mappings
  - Arrays

- Pointer-to-pointers
- Matrices
- STL
- Module initialization
- Building modes
  - No-builder mode
  - Builder mode
- Generated scripts
  - Builder script
  - Loader script
  - Gateway XML files
- Other resources


Scilab is a scientific software package for numerical computations providing a powerful open computing environment for engineering and scientific applications that is mostly compatible with MATLAB. More information can be found at www.scilab.org.

This chapter explains how to use SWIG for Scilab. After this introduction, you should be able to generate with SWIG a Scilab external module from a C/C++ library.

## 36.1 Preliminaries

SWIG for Scilab supports Linux. Other operating systems haven't been tested.

Scilab is supported from version 5.5.2 onwards, the SWIG generated code is supported on both Scilab 5, Scilab 6 and more recent versions.

SWIG for Scilab supports C language. C++ is partially supported. See A basic tour of C/C++ wrapping for further details.

## 36.2 Running SWIG

Let's see how to use SWIG for Scilab on a small example.
In this example we bind from C a function and a global variable into Scilab. The SWIG interface (stored in a file named `example.i` ), is the following:

```
%module example

%inline %{
double Foo = 3.0;

int fact(int n) {
  if (n < 0) {
    return 0;
  }
  else if (n == 0) {
    return 1;
  }
  else {
    return n * fact(n-1);
  }
}
%}
```

Note: a code in an `%inline` section is both parsed and wrapped by SWIG, and inserted as is in the wrapper source file.

### 36.2.1 Generating the module

The module is generated using the `swig` executable and its `-scilab` option.

```
$ swig -scilab example.i
```

This command generates two files:

- `example_wrap.c`: a C source file containing the wrapping code and also here the wrapped code (the `fact()` and `Foo` definitions)
- `loader.sce`: a Scilab script used to load the module into Scilab

Note: if the following error is returned:

```
:1: Error: Unable to find 'swig.swg'
:3: Error: Unable to find 'scilab.swg'
```

it may be because the SWIG library is not found. Check the `SWIG_LIB` environment variable or your SWIG installation.

Note: SWIG for Scilab can work in two modes related to the way the module is built, see the Building modes section for details. This example uses the `builder` mode.

The `swig` executable has several other command line options you can use. See Scilab command line options for further details.

### 36.2.2 Building the module

To be loaded in Scilab, the wrapper has to be built into a dynamic module (or shared library).

The commands to compile and link the wrapper (with `gcc`) into the shared library `libexample.so` are:

```
$ gcc -fPIC -c -I/usr/local/include/scilab example_wrap.c
$ gcc -shared example_wrap.o -o libexample.so
```

Note: we supposed in this example that the path to the Scilab include directory is `/usr/local/include/scilab` (which is the case in a Debian environment), this should be changed for another environment.

### 36.2.3 Loading the module

Loading a module is done by running the loader script in Scilab:

```
--> exec loader.sce
```

Scilab should output the following messages:

```
Shared archive loaded.
Link done.
```

which means that Scilab has successfully loaded the shared library. The module functions and other symbols are now available in Scilab.

### 36.2.4 Using the module

In Scilab, the function `fact()` is simply called as following:

```
--> fact(5)
ans  =

    120.
```

For the `Foo` global variable, the accessors need to be used:

```
--> Foo_get
ans  =

    3.

--> Foo_set(4);

--> Foo_get
ans  =

    4.
```

Note: for conciseness, we assume in the subsequent Scilab code examples that the modules have been beforehand built and loaded in Scilab.

### 36.2.5 Scilab command line options

The following table lists the Scilab specific command line options in addition to the generic SWIG options:

| | |
|---|---|
| `-builder` | Generate the Scilab builder script |
| `-buildercflags <cflags>` | Add <cflags> to the builder compiler flags |
| `-builderldflags <ldflags>` | Add <ldlags> to the builder linker flags |
| `-buildersources <files>` | Add the (comma separated) files <files> to the builder sources |
| `-builderverbositylevel <level>` | Set the build verbosity level to <level> (default 0: off, 2: high) |
| `-builderflagscript <file>` | Use the Scilab script <file> to configure the compiler and linker flags |
| `-gatewayxml <gateway_id>` | Generate the gateway XML with the given <gateway_id> |
| `-gatewayxml6` | Generate a gateway XML file compatible with Scilab 6 |

These options can be displayed with:

```
$ swig -scilab -help
```

## 36.3 A basic tour of C/C++ wrapping

### 36.3.1 Overview

SWIG for Scilab provides only a low-level C interface for Scilab (see Scripting Languages for the general approach to wrapping). This means that functions, structs, classes, variables, etc... are interfaced through C functions. These C functions are mapped as Scilab functions. There are a few exceptions, such as constants and enumerations, which can be wrapped directly as Scilab variables.

### 36.3.2 Identifiers

In Scilab 5.x, identifier names are composed of 24 characters maximum (this limitation disappears from Scilab 6.0 onwards).
By default, variable, member, and function names longer than 24 characters are truncated, and a warning is produced for each truncation.

This can cause ambiguities, especially when wrapping structs/classes, for which the wrapped function name is composed of the struct/class name and field names. In these cases, the %rename directive can be used to choose a different Scilab name.

### 36.3.3 Functions

Functions are wrapped as new Scilab built-in functions. For example:

```
%module example

%inline %{
int fact(int n) {
  if (n > 1)
    return n * fact(n - 1);
  else
    return 1;
}
%}
```

creates a built-in function `fact(n)` in Scilab:

```
--> fact(4)
ans  =

    24.
```

#### 36.3.3.1 Argument passing

In the above example, the function parameter is a primitive type and is marshalled by value. So this function is wrapped without any additional customization. Argument values are converted between C types and Scilab types through type mappings. There are several default type mappings for primitive and complex types, described later in the Scilab typemaps section.

When a parameter is not passed by value, such as a pointer or reference, SWIG does not know if it is an input, output (or both) parameter. The INPUT, OUTPUT, INOUT typemaps defined in the `typemaps.i` library can be used to specify this.

Let's see this on two simple functions: `sub()` which has an output parameter, and `inc()`, which as input/output parameter:

```
%module example

%include <typemaps.i>

extern void sub(int *INPUT, int *INPUT, int *OUTPUT);
extern void inc(int *INOUT, int *INPUT);

%{
void sub(int *x, int *y, int *result) {
  *result = *x - *y;
}
void inc(int *x, int *delta) {
  *x = *x + *delta;
}
%}
```

In Scilab, parameters are passed by value. The output (and inout) parameters are returned as the result of the functions:

```
--> sub(5, 3)
 ans  =

    2.

--> inc(4, 3)
 ans  =

    7.
```

#### 36.3.3.2 Multiple output arguments

A C function can have several output parameters. They can all be returned as results of the wrapped function as Scilab supports multiple return values from a function when using the `typemaps.i` library. If the C function itself returns a result, this is returned first before the parameter outputs.

The example below shows this for a C function returning 2 values and a result:

```
%module example

%include <typemaps.i>

int divide(int n, int d, int *OUTPUT, int *OUTPUT);

%{
  int divide(int n, int d, int q*, int *r) {
    if (d != 0) {
      *q = n / d;
      *r = n % d;
      return 1;
    } else {
      return 0;
    }
  }
%}
```

```
--> [ret, q, r] = divide(20, 6)
 r  =

    2.
 q  =

    3.
 ret  =

    1.
```

### 36.3.4 Global variables

Global variables are manipulated through generated accessor functions. For example, for a given `Foo` global variable, SWIG actually generates two functions: `Foo_get()` to get the value of `Foo`, and `Foo_set()` to set the value. These functions are used as following:

```
--> exec loader.sce;
--> c = Foo_get();

--> Foo_set(4);

--> c
c =

    3.

--> Foo_get()
ans  =

    4.
```

It works for variables of primitive type, but also for non-primitive types: arrays, and structs/classes which are described later. For now, an example with two global primitive arrays x and y is shown:

```
%module example

%inline %{
int x[10];
double y[7];

void initArrays()
{
  int i;
  for (i = 0; i < 10; i++)
    x[i] = 1;
  for (i = 0; i < 7; i++)
    y[i] = 1.0f;
}
%}
```

It works the same:

```
--> exec loader.sce

--> initArrays();
--> x_get()
 ans  =

    1.    1.    1.    1.    1.    1.    1.    1.    1.    1.

--> y_set([0:6] / 10);
--> y_get()
 ans  =

    0.    0.1    0.2    0.3    0.4    0.5    0.6
```

### 36.3.5 Constants and enumerations

#### 36.3.5.1 Constants

There is not any constant in Scilab. By default, C/C++ constants are wrapped as getter functions. For example, for the following constants:

```
%module example
#define    ICONST      42
#define    FCONST      2.1828
#define    CCONST      'x'
#define    CCONST2     '\n'
#define    SCONST      "Hello World"
#define    SCONST2     "\"Hello World\""
```

the following getter functions are generated:

```
--> exec loader.sce;
--> ICONST_get();
 ans  =

    42.

--> FCONST_get();
 ans  =

    2.1828

--> CCONST_get();
 ans  =

    x

--> CCONST2_get();
 ans  =

--> SCONST_get();
 ans  =

    Hello World

--> SCONST2_get();
 ans  =

    "Hello World"

--> EXPR_get();
 ans  =

    48.5484

--> iconst_get();
 ans  =

    37.

--> fconst_get();
 ans  =

    3.14
```

There is another mode in which constants are wrapped as Scilab variables. The variables are easier to use than functions, but the drawback is that variables are not constant and so can be modified.

This mode can be enabled/disabled at any time in the interface file with `%scilabconst()`, which works like all the other%feature directives. Use the argument value "1" to enable and "0" to disable this mode. For example in this mode the previous constants:

```
%module example

%scilabconst(1);
#define    ICONST      42
#define    FCONST      2.1828
#define    CCONST      'x'
#define    CCONST2     '\n'
#define    SCONST      "Hello World"
#define    SCONST2     "\"Hello World\""
```

are mapped to Scilab variables, with the same name:

```
--> exec loader.sce;
--> ICONST
 ans  =

    42

--> FCONST
 ans  =

    2.1828

--> CCONST
 ans  =

    x

--> CCONST2
 ans  =

--> SCONST
 ans  =

    Hello World

--> SCONST2
 ans  =

    "Hello World"

--> EXPR
 ans  =

    48.5484

--> iconst
 ans  =

    37

--> fconst
 ans  =

    3.14
```

#### 36.3.5.2 Enumerations

The wrapping of enums is the same as for constants. By default, enums are wrapped as getter functions. For example, with the following enumeration:

```
%module example
typedef enum { RED, BLUE, GREEN } color;
```

a getter function will be generated for each value of the enumeration:

```
--> exec loader.sce;
--> RED_get()
 ans  =

    0.

--> BLUE_get()
 ans  =

    1.

--> GREEN_get()
 ans  =

    2.
```

The `%scilabconst()` feature is also available for enumerations:

```
%module example
%scilabconst(1) color;
typedef enum { RED, BLUE, GREEN } color;
```

```
--> exec loader.sce;
--> RED
 ans  =

    0.

--> BLUE
 ans  =

    1.

--> GREEN
 ans  =

    2.
```

### 36.3.6 Pointers

Pointers are supported by SWIG. A pointer can be returned from a wrapped C/C++ function, stored in a Scilab variable, and used in input argument of another C/C++ function.

Also, thanks to the SWIG runtime which stores information about types, pointer types are tracked between exchanges Scilab and the native code. Indeed pointer types are stored alongside the pointer address. A pointer is mapped to a Scilab structure ( tlist), which contains as fields the pointer address and the pointer type (in fact a pointer to the type information structure in the SWIG runtime).
Why a native pointer is not mapped to a Scilab pointer (type name: "pointer", type ID: 128) ? The big advantage of mapping to a `tlist` is that it exposes a new type for the pointer in Scilab, type which can be accessed in Scilab with the typeof function, and manipulated using theoverloading mechanism.

Notes:

- type tracking needs the SWIG runtime to be first initialized with the appropriate function (see the Module initialization section).
- for any reason, if a wrapped pointer type is unknown (or if the SWIG runtime is not initialized), SWIG maps it to a Scilab pointer. Also, a Scilab pointer is always accepted as a pointer argument of a wrapped function. The drawback is that pointer type is lost.

Following is an example of the wrapping of the C `FILE*` pointer:

```
%module example

%{
#include <stdio.h>
%}

FILE *fopen(const char *filename, const char *mode);
int fputs(const char *, FILE *);
int fclose(FILE *);
```

These functions can be used the same way as in C from Scilab:

```
--> example_Init();

--> f = fopen("junk", "w");
--> typeof(f)
 ans  =

  _p_FILE

--> fputs("Hello World", f);
--> fclose(f);
```

Note: the type name `_p_FILE` which means "pointer to FILE".

The user of a pointer is responsible for freeing it or, like in the example, closing any resources associated with it (just as is required in a C program).

#### 36.3.6.1 Utility functions

As a scripting language, Scilab does not provide functions to manipulate pointers. However, in some cases it can be useful, such as for testing or debugging.

SWIG comes with two pointer utility functions:

- `SWIG_this()`: returns the address value of a pointer
- `SWIG_ptr()`: creates a pointer from an address value

Note: a pointer created by `SWIG_ptr()` does not have any type and is mapped as a Scilab pointer.

Following we use the utility functions on the previous example:

```
--> f = fopen("junk", "w");
--> fputs("Hello", f);
--> addr = SWIG_this(f)
 ans  =

    8219088.

--> p = SWIG_ptr(addr);
--> typeof(p)
ans  =

  pointer

--> fputs(" World", p);
--> fclose(f);
```

#### 36.3.6.2 Null pointers:

Using the previous `SWIG_this()` and `SWIG_ptr()`, it is possible to create and check null pointers:

```
--> p = SWIG_ptr(0);
--> SWIG_this(p) == 0
 ans  =

    T
```

### 36.3.7 Structures

Structs exist in Scilab, but C structs are not (at least in this version of SWIG) mapped to Scilab structs. A C structure is wrapped through low-level accessor functions, i.e. functions that give access to the member variables of this structure. In Scilab, a structure is manipulated through a pointer which is passed as an argument to the accessor functions.

Let's see it on an example of a struct with two members:

```
%module example

%inline %{

typedef struct {
  int x;
  int arr[4];
} Foo;

%}
```

Several functions are generated:

- a constructor function `new_Foo()` which returns a pointer to a newly created struct `Foo`.
- two member getter functions `Foo_x_get()`, `Foo_arr_get()` , to get the values of`x` and `y` for the struct pointer (provided as the first parameter to these functions)
- two member setter functions `Foo_x_set()`, `Foo_arr_set()` , to set the values of`x` and `y` for the struct pointer (provided as the first parameter to these functions).
- a destructor function `delete_Foo()` to release the struct pointer.

Usage example:

```
--> f = new_Foo();
--> Foo_x_set(f, 100);
--> Foo_x_get(f)
ans  =

    100.

--> Foo_arr_set(f, [0:3]);
--> Foo_arr_get(f)
ans  =

    0.    1.    2.    3.

--> delete_Foo(f);
```

Members of a structure that are also structures are also accepted and wrapped as a pointer:

```
%module example

%inline %{

typedef struct {
  int x;
} Bar;

typedef struct {
  Bar b;
} Foo;

%}
```

```
--> b = new_Bar();
--> Bar_x_set(b, 20.);

--> f = new_Foo();
--> Foo_b_set(f, b);

--> b2 = Foo_b_get(f);
--> Bar_x_get(b2);
ans  =

    20.
```

Note: the pointer to the struct works as described in Pointers. For example, the type of the struct pointer can be get with `typeof`, as following:

```
--> example_Init();
--> b = new_Bar();
--> typeof(b)
 ans  =

    _p_Bar
--> delete_Bar(b);
```

### 36.3.8 C++ classes

Classes do not exist in Scilab. The classes are wrapped the same way as structs. Low-level accessor functions are generated for class members. Also, constructor and destructor functions are generated to create and destroy an instance of the class.

For example, the following class:

```
%module example

%inline %{

class Point {
public:
  int x, y;
  Point(int _x, int _y) : x(_x), y(_y) {}
  double distance(const Point& rhs) {
    return sqrt(pow(x-rhs.x, 2) + pow(y-rhs.y, 2));
  }
  void set(int _x, int _y) {
    x=_x;
    y=_y;
  }
};

%}
```

can be used in Scilab like this:

```
--> p1 = Point_new(3, 5);
--> p2 = Point_new(1, 2);
--> p1.distance(p2)
ans  =

    3.6056

--> delete_Point(p1);
--> delete_Point(p2);
```

Note: like structs, class pointers are mapped as described in Pointers. Let's give an example which shows that each class pointer type is a new type in Scilab that can be used for example (through overloading) to implement a custom print for the `Point` class:

```
--> function %_p_Point_p(p)
-->     mprintf('[%d, %d]\n', Point_x_get(p), Point_y_get(p));
--> endfunction

--> example_Init();
--> p = new_Point(1, 2)
 p  =

[1, 2]

--> delete_Point(p);
```

### 36.3.9 C++ inheritance

Inheritance is supported. SWIG knows the inheritance relationship between classes.

A function is only generated for the class in which it is actually declared. But if one of its parameters is a class, any instance of a derived class is accepted as the argument.

This mechanism also applies for accessor functions: they are generated only in the class in which they are defined. But any instance of a derived class can be used as the argument to these accessor functions.

For example, let's take a base class `Shape` and two derived classes `Circle` and `Square`:

```
%module example

%inline %{

class Shape {
public:
  double x, y;
  void set_location(double _x, double _y) { x = _x; y = _y; }
  virtual double get_perimeter() { return 0; };
};

class Circle : public Shape {
public:
  int radius;
  Circle(int _radius): radius(_radius) {};
  virtual double get_perimeter() { return 6.28 * radius; }
};

class Square : public Shape {
public:
  int size;
  Square(int _size): size(_size) {};
  virtual double get_perimeter() { return 4 * size; }
};

%}
```

To set the location of the `Circle`, we have to use the function `set_location()` of the parent `Shape`. But we can use either use the `get_perimeter()` function of the parent class or the derived class:

```
--> c = new_Circle(3);

--> Shape_set_location(c, 2, 3);
--> Shape_x_get(c)
 ans  =
```

```
    2.

--> Circle_get_perimeter(c)
 ans  =

    18.84

--> Shape_get_perimeter(c)
 ans  =

    18.84
```

### 36.3.10 C++ overloading

As explained in Overloaded functions and methods SWIG provides support for overloaded functions and constructors.

As SWIG knows pointer types, the overloading works also with pointer types, here is an example with a function `magnify` overloaded for the previous classes `Shape` and `Circle`:

```
%module example

void magnify(Square *square, double factor) {
  square->size *= factor;
};

void magnify(Circle *circle, double factor) {
  square->radius *= factor;
};
```

```
--> example_Init();
--> c = new_Circle(3);
--> s = new_Square(2);

--> magnify(c, 10);
--> Circle_get_radius(c)
 ans  =

   30;
--> magnify(s, 10);
--> Square_get_size(s)
 ans  =

   20;
```

### 36.3.11 Pointers, references, values, and arrays

In C++ objects can be passed by value, pointer, reference, or by an array:

```
%module example

%{
#include <sciprint.h>
%}

%inline %{

class Foo {
public:
  Foo(int _x) : x(_x) {}
  int x;
};

void spam1(Foo *f)  { sciprint("%d\n", f->x); }   // Pass by pointer
void spam2(Foo &f)  { sciprint("%d\n", f.x); }    // Pass by reference
void spam3(Foo f)   { sciprint("%d\n", f.x); }    // Pass by value
void spam4(Foo f[]) { sciprint("%d\n", f[0].x); } // Array of objects

%}
```

In SWIG, there is no real distinction between these. So in Scilab, it is perfectly legal to do this:

```
--> f = new_Foo()
--> spam1(f)
3
--> spam2(f)
3
--> spam3(f)
3
--> spam4(f)
3
```

Similar behaviour occurs for return values. For example, if you had functions like this:

```
Foo *spam5();
Foo &spam6();
Foo  spam7();
```

All these functions will return a pointer to an instance of `Foo` . As the function `spam7` returns a value, new instance of `Foo` has to be allocated, and a pointer on this instance is returned.

### 36.3.12 C++ templates

As in other languages, function and class templates are supported in SWIG Scilab.

You have to tell SWIG to create wrappers for a particular template instantiation. The `%template` directive is used for this purpose. For example:

```
%module example

template<class T1, class T2, class T3>
struct triplet {
  T1 first;
  T2 second;
  T3 third;
  triplet(const T1& a, const T2& b, const T3& c) {
    third = a; second = b; third = c;
  }
};

%template(IntTriplet) triplet<int, int, int>;
```

Then in Scilab:

```
--> t = new_IntTriplet(3, 4, 1);

--> IntTriplet_first_get(t)
 ans  =

    3.

--> IntTriplet_second_get(t)
 ans  =

    4.

--> IntTriplet_third_get(t)
 ans  =

    1.

--> delete_IntTriplet(t);
```

More details on template support can be found in the templates documentation.

### 36.3.13 C++ operators

C++ operators are partially supported. Operator overloading exists in Scilab, but a C++ operator is not (in this version) wrapped by SWIG as a Scilab operator, but as a function. It is not automatic, you have to rename each operator (with the instruction `%rename`) with the suitable wrapper name.

Let's see it with an example of class with two operators + and `double()`:

```
%module example

%rename(plus) operator +;
%rename(toDouble) operator double();

%inline %{

class Complex {
public:
  Complex(double re, double im) : real(re), imag(im) {};

  Complex operator+(const Complex& other) {
    double result_real = real + other.real;
    double result_imaginary = imag + other.imag;
    return Complex(result_real, result_imaginary);
  }
  operator double() { return real; }
private:
  double real;
  double imag;
};

%}
```

```
--> c1 = new_Complex(3, 7);

--> c2 = Complex_plus(c, new_Complex(1, 1));

--> Complex_toDouble(c2)
 ans  =

    4.
```

### 36.3.14 C++ namespaces

SWIG is aware of C++ namespaces, but does not use it for wrappers. The module is not broken into submodules, nor do namespace appear in functions names. All the namespaces are all flattened in the module. For example with one namespace `Foo`:

```
%module example

%inline %{

namespace foo {
  int fact(int n) {
    if (n > 1)
      return n * fact(n-1);
    else
```

```
    return 1;
  }

  struct Vector {
    double x, y, z;
  };
};

%}
```

In Scilab, there is no need to the specify the `Foo` namespace:

```
--> fact(3)
 ans  =

   6.

--> v = new_Vector();
--> Vector_x_set(v, 3.4);
--> Vector_y_get(v)
 ans  =

   0.
```

If your program has more than one namespace, name conflicts can be resolved using `%rename`. For example:

```
%rename(Bar_spam) Bar::spam;

namespace Foo {
  int spam();
}

namespace Bar {
  int spam();
}
```

Note: the nspace feature is not supported.

### 36.3.15 C++ exceptions

Scilab does not natively support exceptions, but has errors. When an exception is thrown, SWIG catches it, and sets a Scilab error. An error message is displayed in Scilab. For example:

```
%module example

%inline %{
void throw_exception() throw(char const *) {
  throw "Bye world !";
}
%}
```

```
-->throw_exception()
  !--error 999
SWIG/Scilab: Exception (char const *) occurred: Bye world !
```

Scilab has a `try-catch` mechanism (and a similar instruction `execstr()`) to handle exceptions. It can be used with the`lasterror()` function as following:

```
--> execstr('throw_exception()', 'errcatch');
 ans  =

    999.

--> lasterror()
 ans  =

    SWIG/Scilab: Exception (char const *) occurred: Bye world !
```

If the function has a `throw` exception specification, SWIG can automatically map the exception type and set an appropriate Scilab error message. It works for a few primitive types, and also for STL exceptions (the library `std_except.i` has to be included to get the STL exception support):

```
%module example

%include <std_except.i>

%inline %{
void throw_int() throw(int) {
  throw 12;
}

void throw_stl_invalid_arg(int i) throw(std::invalid_argument) {
  if (i < 0)
    throw std::invalid_argument("argument is negative.");
}
%}
```

```
--> throw_int();
          !--error 999
SWIG/Scilab: Exception (int) occurred: 12
```

```
-->throw_stl_invalid_arg(-1);
                          !--error 999
SWIG/Scilab: ValueError: argument is negative.
```

More complex or custom exception types require specific exception typemaps to be implemented in order to specifically handle a thrown type. See the SWIG C++ documentation for more details.

### 36.3.16 C++ STL

The Standard Template Library (STL) is partially supported. See STL for more details.

## 36.4 Type mappings and libraries

### 36.4.1 Default primitive type mappings

The following table provides the equivalent Scilab type for C/C++ primitive types.

| C/C++ type | Scilab type |
|---|---|
| bool | boolean |
| char | string |
| signed char | double or int8 |
| unsigned char | double or uint8 |
| short | double or int16 |
| unsigned short | double or uint16 |
| int | double or int32 |
| unsigned int | double or uint32 |
| long | double or int32 |
| unsigned long | double or uint32 |
| signed long long | not supported in Scilab 5.x |
| unsigned long long | not supported in Scilab 5.x |
| float | double |
| double | double |
| char * or char[] | string |

Notes:

- In Scilab the `double` type is far more used than any integer type. This is why integer values (`int32`, `uint32`, ...) are automatically converted to Scilab `double` values when marshalled from C into Scilab. Additionally on input to a C function, Scilab `double` values are converted into the related integer type.
- When an integer is expected, if the input is a double, the value must be an integer, i.e. it must not have any decimal part, otherwise a SWIG value error occurs.
- In SWIG for Scilab 5.x, the `long long` type is not supported, since Scilab 5.x does not have a 64-bit integer type. The default behaviour is for SWIG to generate code that will give a runtime error if `long long` type arguments are used from Scilab.

### 36.4.2 Arrays

Typemaps are available by default for arrays. Primitive type arrays are automatically converted to/from Scilab matrices. Typemaps are also provided to handle members of a struct or class that are arrays.

In input, the matrix is usually one-dimensional (it can be either a row or column vector). But it can also be a two-dimensional matrix. Warning: in Scilab, the values are column-major ordered, unlike in C, which is row-major ordered.

The type mappings used for arrays is the same for primitive types, described earlier. This means that, if needed, a Scilab `double` vector is converted in input into the related C integer array and this C integer array is automatically converted on output into a Scilab `double` vector. Note that unlike scalars, no control is done for arrays when a `double` is converted into an integer.

The following example illustrates all this:

```
%module example

%#include <stdio.h>

%inline %{

void printArray(int values[], int len) {
  int i = 0;
  for (i = 0; i < len; i++) {
    printf("%s %d %s", i==0?"[":"", values[i], i==len-1?"]\n":"");
  }
}
%}
```

```
--> printArray([0 1 2 3], 4)
[ 0  1  2  3 ]

--> printArray([0.2; -1.8; 2; 3.7], 4)
[ 0  -1  2  3 ]

--> printArray([0 1; 2 3], 4)
[ 0  2  1  3 ]

--> printArray([0; 1; 2; 3], 4)
[ 0  1  2  3 ]
```

### 36.4.3 Pointer-to-pointers

There are no specific typemaps for pointer-to-pointers, they are mapped as pointers in Scilab.

Pointer-to-pointers are sometimes used to implement matrices in C. The following is an example of this:

```
%module example
```

```
%inline %{

// Returns the matrix [1 2; 3 4];
double **create_matrix() {
  double **M;
  int i;
  M = (double **) malloc(2 * sizeof(double *));
  for (i = 0; i < 2; i++) {
    M[i] = (double *) malloc(2 * sizeof(double));
    M[i][0] = 2 * i + 1;
    M[i][1] = 2 * i + 2;
  }
  return M;
}

// Gets the item M(i, j) value
double get_matrix(double **M, int i, int j) {
  return M[i][j];
}

// Sets the item M(i, j) value to be val
void set_matrix(double **M, int i, int j, double val) {
  M[i][j] = val;
}

// Prints a matrix (2, 2) to console
void print_matrix(double **M, int nbRows, int nbCols) {
  int i, j;
  for (i = 0; i < 2; i++) {
    for (j = 0; j < 2; j++) {
      printf("%3g ", M[i][j]);
    }
    printf("\n");
  }
}

%}
```

These functions are used like this in Scilab:

```
--> m = create_matrix();

--> print_matrix(m);
   1.   2.
   3.   4.

--> set_matrix(m, 1, 1, 5.);

--> get_matrix(m, 1, 1)
 ans  =

    5.
```

### 36.4.4 Matrices

The `matrix.i` library provides a set of typemaps which can be useful when working with one-dimensional and two-dimensional matrices.

In order to use this library, just include it in the interface file:

```
 %include <matrix.i>
```

Several typemaps are available for the common Scilab matrix types:

- `double`
- `int`
- `char *`
- `bool`

For example: for a matrix of `int`, we have the typemaps, for input:

- `(int *IN, int IN_ROWCOUNT, int IN_COLCOUNT)`
- `(int IN_ROWCOUNT, int IN_COLCOUNT, int *IN)`
- `(int *IN, int IN_SIZE)`
- `(int IN_SIZE, int *IN)`

and output:

- `(int **OUT, int *OUT_ROWCOUNT, int *OUT_COLCOUNT)`
- `(int *OUT_ROWCOUNT, int *OUT_COLCOUNT, int **OUT)`
- `(int **OUT, int *OUT_SIZE)`
- `(int *OUT_SIZE, int **OUT)`

They marshall a Scilab matrix type into the appropriate 2 or 3 C parameters. The following is an example using the typemaps in this library:

```
%module example

%include <matrix.i>

%apply (int *IN, int IN_ROWCOUNT, int IN_COLCOUNT) { (int *matrix, int matrixNbRow, int matrixNbCol) };
%apply (int **OUT, int *OUT_ROWCOUNT, int *OUT_COLCOUNT) { (int **outMatrix, int *outMatrixNbRow, int *outMatrixNbCol) };

%inline %{

void absolute(int *matrix, int matrixNbRow, int matrixNbCol,
  int **outMatrix, int *outMatrixNbRow, int *outMatrixNbCol) {
  int i, j;
  *outMatrixNbRow = matrixNbRow;
```

```
  *outMatrixNbCol = matrixNbCol;
  *outMatrix = malloc(matrixNbRow * matrixNbCol * sizeof(int));
  for (i=0; i < matrixNbRow * matrixNbCol; i++) {
    (*outMatrix)[i] = matrix[i] > 0 ? matrix[i]:-matrix[i];
  }
}

%}
```

```
--> absolute([-0 1 -2; 3 4 -5])
 ans  =

    0.    1.    2.
    3.    4.    5.
```

The remarks made earlier for arrays also apply here:

- The values of matrices in Scilab are column-major orderered,
- There is no control while converting `double` values to integers, `double` values are truncated without any checking or warning.

### 36.4.5 STL

The STL library wraps some containers defined in the STL (Standard Template Library), so that they can be manipulated in Scilab. This library also provides the appropriate typemaps to use the containers in functions and variables.

The list of wrapped sequence containers are:

- `std::vector`
- `std::list`
- `std::deque`

And associative containers are:

- `std::set`
- `std::multiset`

Typemaps are available for the following container types:

- `double`
- `float`
- `int`
- `string`
- `bool`
- `pointer`

Containers of other item types are not supported. Using them does not break compilation, but provokes a runtime error. Containers of enum are not supported yet.

In order to use the STL, the library must first be included in the SWIG interface file:

```
%include <stl.i>
```

Then for each container used, the appropriate template must be instantiated, in the `std` namespace:

```
namespace std {
    %template(IntVector)    vector<int>;
    %template(DoubleVector) vector<double>;
}
```

Additionally, the module initialization function has to be executed first in Scilab, so that all the types are known to Scilab. See the Module initialization section for more details.

Because in Scilab matrices exist for basic types only, a sequence container of pointers is mapped to a Scilab list. For other item types (double, int, string...) the sequence container is mapped to a Scilab matrix.

The first example below shows how to create a vector (of `int` ) in Scilab, add some values to the vector and pass it as an argument of a function. It also shows, thanks to the typemaps, that we can also pass a Scilab matrix of values directly into the function:

```
%module example

%include <stl.i>

namespace std {
  %template(IntVector) vector<int>;
}

%{
#include <numeric>
%}

%inline %{

double average(std::vector<int> v) {
  return std::accumulate(v.begin(), v.end(), 0.0) / v.size();
}

%}
```

```
--> example_Init();

--> v = new_IntVector();

--> for i = 1:4
-->     IntVector_push_back(v, i);
--> end;
```

```
--> average(v)
 ans  =

    2.5

--> average([0 1 2 3])
 ans  =

    2.5

--> delete_IntVector();
```

In the second example, a set of struct (`Person`) is wrapped. A function performs a search in this set, and returns a subset. As one can see, the result in Scilab is a list of pointers:

```
%module example

%include <stl.i>

%{
#include <string>
%}

%inline %{

struct Person {
  Person(std::string _name, int _age) : name(_name), age(_age) {};
  std::string name;
  int age;
};
typedef Person * PersonPtr;

%}

namespace std {
  %template(PersonPtrSet) set<PersonPtr>;
}

%inline %{

std::set<PersonPtr> findPersonsByAge(std::set<PersonPtr> persons, int minAge, int maxAge) {
  std::set<PersonPtr> foundPersons;
  for (std::set<PersonPtr>::iterator it = persons.begin(); it != persons.end(); it++) {
    if (((*it)->age >= minAge) && ((*it)->age <= maxAge)) {
      foundPersons.insert(*it);
    }
  }
  return foundPersons;
}

%}
```

```
--> example_Init();

--> joe = new_Person("Joe", 25);
--> susan = new_Person("Susan", 32);
--> bill = new_Person("Bill", 50);

--> p = new_PersonPtrSet();
--> PersonPtrSet_insert(p, susan);
--> PersonPtrSet_insert(p, joe);
--> PersonPtrSet_insert(p, bill);

--> l = findPersonsByAge(p, 20, 40);

--> size(l)
 ans  =

    2.

--> Person_name_get(l(1))
ans  =

    Susan

--> Person_name_get(l(2))
 ans  =

    Joe

--> delete_PersonPtrSet(p);
```

## 36.5 Module initialization

The wrapped module contains an initialization function to:

- initialize the SWIG runtime, needed for pointer type tracking or when working with the STL
- initialize the module constants and enumerations declared with `%scilabconst()`

This initialization function should be executed at the start of a script, before the wrapped library has to be used.

The function has the name of the module suffixed by `_Init`. For example, to initialize the module `example`:

```
--> example_Init();
```

## 36.6 Building modes

The mechanism to load an external module in Scilab is called *Dynamic Link* and works with dynamic modules (or shared libraries, `.so` files).

To produce a dynamic module, when generating the wrapper, there are two possibilities, or build modes:

- the `nobuilder` mode, this is the default mode in SWIG. The user is responsible of the build.
- the `builder` mode. In this mode, Scilab is responsible of building.

### 36.6.1 No-builder mode

In this mode, used by default, SWIG generates the wrapper sources, which have to be manually compiled and linked. A loader script `loader.sce` is also produced, this one is executed further in Scilab to load the module.

This mode is the best option to use when you have to integrate the module build into a larger build process.

### 36.6.2 Builder mode

In this mode, in addition to the wrapper sources, SWIG produces a builder Scilab script (`builder.sce`), which is executed in Scilab to build the module. In a few words, the Scilab `ilib_build()` command is used, which produces the shared library file, and the loader script `loader.sce` (and also a cleaner script`cleaner.sce` ).

An advantage of this mode is that it hides all the complexity of the build and other platform issues. Also it allows the module to conform to a Scilab external module convention which is that an external module should be simply built by calling a builder script.

The builder mode is activated with the `-builder` SWIG option. In this mode, the following SWIG options may be used to setup the build:

- **`-buildersources`**: to add sources to the build (several files must be separated by a comma)
- **`-buildercflags`**: to add flags to the builder compiler flags, for example to set library dependencies include paths
- **`-builderldflags`**: to add flags to the linker flags, for example to set library dependency names and paths

Let's give an example how to build a module `example`, composed of two sources, and using a library dependency:

- the sources are `baa1.c` and `baa2.c` (and are stored in the current directory)
- the library is `libfoo` in `/opt/foo` (headers stored in `/opt/foo/include`, and shared library in `/opt/foo/lib` )

The command is:

```
$ swig -scilab -builder -buildercflags -I/opt/foo/include -builderldflags "-L/opt/foo/lib -lfoo" -buildersources baa1.cxx, baa2.cxx example.i
```

## 36.7 Generated scripts

In this part we give some details about the generated Scilab scripts.

### 36.7.1 Builder script

`builder.sce` is the name of the builder script generated by SWIG in `builder` mode. It contains code like this:

```
ilib_name = "examplelib";
files = ["example_wrap.c"];
libs = [];
table = ["fact", "_wrap_fact";"Foo_set", "_wrap_Foo_set";"Foo_get", "_wrap_Foo_get";];
ilib_build(ilib_name, table, files, libs);
```

`ilib_build(lib_name, table, files, libs)` is used to create shared libraries, and to generate a loader file used to dynamically load the shared library into Scilab.

- **`ilib_name`**: a character string, the generic name of the library without path and extension.
- **`files`**: string matrix containing objects files needed for shared library creation.
- **`libs`**: string matrix containing extra libraries needed for shared library creation.
- **`table`**: two column string matrix containing a table of pairs of 'scilab function name', 'C function name'.

### 36.7.2 Loader script

The loader script is used to load in Scilab all the module functions. When loaded, these functions can be used as other Scilab functions.

The loader script `loader.sce` contains code similar to:

```
// ------------------------------------------------------
// generated by builder.sce: Please do not edit this file
// ------------------------------------------------------

libexamplelib_path = get_file_path('loader.sce');
list_functions = [             'fact';
            'Foo_set';
            'Foo_get';
];
addinter(libexamplelib_path+'/libexamplelib.so', 'libexamplelib', list_functions);
// remove temp. variables on stack
clear libexamplelib_path;
clear list_functions;
clear get_file_path;
// ------------------------------------------------------
```

`addinter(files, spname, fcts)` performs dynamic linking of a compiled C interface function.

- **`files`**: a character string or a vector of character strings defining the object files (containing the C interface functions) to link with.
- **`spname`**: a character string. Name of interface routine entry point.
- **`fcts`**: vector of character strings. The name of new Scilab function.

### 36.7.3 Gateway XML files

If you need to post-process the entry points, Scilab gateway files are XML files that can be used to retrieve all SWIG-generated entry points. With these XML files you can write your own `builder_swig.sce` file to add custom Scilab for building or linking the generated code. Documentation stubs can also be generated thanks to these function listings.

As an example, for a SWIG module named `fmuswig` the Scilab code below can be used to store all SWIG-generated functions in a variable named `funs`.

```
// src_swig_path is a path to the directory containing the fmuswig.i file

doc = xmlRead(src_swig_path + "/fmuswig_gateway.xml");
names = xmlAsText(xmlXPath(doc, "//gateway/@name"));
funs = xmlAsText(xmlXPath(doc, "//gateway/@function"));
xmlDelete(doc);
```

## 36.8 Other resources

- Example use cases can be found in the `Examples/scilab` directory.
- The test suite in the `Examples/test-suite/scilab` can be another source of useful use cases.
- The Scilab API is used in the generated code and is a useful reference when examining the output.
- This guide describes the Scilab external modules structure and files, in particular the files that are generated by SWIG for Scilab.

# 37 SWIG and Tcl


- Preliminaries
  - Getting the right header files
  - Compiling a dynamic module
  - Static linking
  - Using your module
  - Compilation of C++ extensions
  - Compiling for 64-bit platforms
  - Setting a package prefix
  - Using namespaces
- Building Tcl/Tk Extensions under Windows 95/NT
  - Running SWIG from Developer Studio
  - Using NMAKE
- A tour of basic C/C++ wrapping
  - Modules
  - Functions
  - Global variables
  - Constants and enums
  - Pointers
  - Structures
  - C++ classes
  - C++ inheritance
  - Pointers, references, values, and arrays
  - C++ overloaded functions
  - C++ operators
  - C++ namespaces
  - C++ templates
  - C++ Smart Pointers
- Further details on the Tcl class interface
  - Proxy classes
  - Memory management
- Input and output parameters
- Exception handling
- Typemaps
  - What is a typemap?
  - Tcl typemaps
  - Typemap variables
  - Converting a Tcl list to a char **
  - Returning values in arguments
  - Useful functions
  - Standard typemaps
  - Pointer handling
- Turning a SWIG module into a Tcl Package.
- Building new kinds of Tcl interfaces (in Tcl)
  - Proxy classes
- Tcl/Tk Stubs


**Caution: This chapter is under repair!**

This chapter discusses SWIG's support of Tcl. Since SWIG 4.1.0, Tcl 8.4 or a later release is required. Prior to that earlier Tcl 8.x releases were also supported. Tcl 9.0 is supported since SWIG 4.2.1.

## 37.1 Preliminaries

To build a Tcl module, run SWIG using the `-tcl` or `-tcl8` option :

```
$ swig -tcl example.i
```

If building a C++ extension, add the `-c++` option:

```
$ swig -c++ -tcl example.i
```

This creates a file `example_wrap.c` or `example_wrap.cxx` that contains all of the code needed to build a Tcl extension module. To finish building the module, you need to compile this file and link it with the rest of your program.

### 37.1.1 Getting the right header files

In order to compile the wrapper code, the compiler needs the `tcl.h` header file. This file is usually contained in the directory

```
/usr/local/include
```

Be aware that some Tcl versions install this header file with a version number attached to it. If this is the case, you should probably make a symbolic link so that `tcl.h` points to the correct header file.

### 37.1.2 Compiling a dynamic module

The preferred approach to building an extension module is to compile it into a shared object file or DLL. Assuming you have code you need to link to in a file called `example.c`, you will need to compile your program using commands like this (shown for Linux):

```
$ swig -tcl example.i
$ gcc -fPIC -c example.c
$ gcc -fPIC -c example_wrap.c -I/usr/local/include
$ gcc -shared example.o example_wrap.o -o example.so
```

The exact commands for doing this vary from platform to platform. SWIG tries to guess the right options when it is installed. Therefore, you may want to start with one of the examples in the `SWIG/Examples/tcl` directory. If that doesn't work, you will need to read the man-pages for your compiler and linker to get the right set of options. You might also check the SWIG Wiki for additional information.

When linking the module, the name of the output file has to match the name of the module. If the name of your SWIG module is "`example` ", the name of the corresponding object file should be "`example.so` ". The name of the module is specified using the`%module` directive or the `-module` command line option.

### 37.1.3 Static linking

An alternative approach to dynamic linking is to rebuild the Tcl interpreter with your extension module added to it. In the past, this approach was sometimes necessary due to limitations in dynamic loading support on certain machines. However, the situation has improved greatly over the last few years and you should not consider this approach unless there is really no other option.

The usual procedure for adding a new module to Tcl involves writing a special function `Tcl_AppInit()` and using it to initialize the interpreter and your module. With SWIG, the `tclsh.i` and `wish.i` library files can be used to rebuild the`tclsh` and `wish` interpreters respectively. For example:

```
%module example

%inline %{
extern int fact(int);
extern int mod(int, int);
extern double My_variable;
%}

%include "tclsh.i"       // Include code for rebuilding tclsh
```

The `tclsh.i` library file includes supporting code that contains everything needed to rebuild tclsh. To rebuild the interpreter, you simply do something like this:

```
$ swig -tcl example.i
$ gcc example.c example_wrap.c \
        -Xlinker -export-dynamic \
        -DHAVE_CONFIG_H -I/usr/local/include/ \
        -L/usr/local/lib -ltcl -lm -ldl \
        -o mytclsh
```

You will need to supply the same libraries that were used to build Tcl the first time. This may include system libraries such as `-lsocket`, `-lnsl`, and `-lpthread`. If this actually works, the new version of Tcl should be identical to the default version except that your extension module will be a built-in part of the interpreter.

**Comment:** In practice, you should probably try to avoid static linking if possible. Some programmers may be inclined to use static linking in the interest of getting better performance. However, the performance gained by static linking tends to be rather minimal in most situations (and quite frankly not worth the extra hassle in the opinion of this author).

### 37.1.4 Using your module

To use your module, simply use the Tcl `load` command. If all goes well, you will be able to this:

```
$ tclsh
% load ./example.so
% fact 4
24
%
```

A common error received by first-time users is the following:

```
% load ./example.so
couldn't find procedure Example_Init
%
```

This error is almost always caused when the name of the shared object file doesn't match the name of the module supplied using the SWIG `%module` directive. Double-check the interface to make sure the module name and the shared object file match. Another possible cause of this error is forgetting to link the SWIG-generated wrapper code with the rest of your application when creating the extension module.

Another common error is something similar to the following:

```
% load ./example.so
couldn't load file "./example.so": ./example.so: undefined symbol: fact
%
```

This error usually indicates that you forgot to include some object files or libraries in the linking of the shared library file. Make sure you compile both the SWIG wrapper file and your original program into a shared library file. Make sure you pass all of the required libraries to the linker.

Sometimes unresolved symbols occur because a wrapper has been created for a function that doesn't actually exist in a library. This usually occurs when a header file includes a declaration for a function that was never actually implemented or it was removed from a library without updating the header file. To fix this, you can either edit the SWIG input file to remove the offending declaration or you can use the `%ignore` directive to ignore the declaration.

Finally, suppose that your extension module is linked with another library like this:

```
$ gcc -shared example.o example_wrap.o -L/home/beazley/projects/lib -lfoo \
      -o example.so
```

If the `foo` library is compiled as a shared library, you might get the following problem when you try to use your module:

```
% load ./example.so
couldn't load file "./example.so": libfoo.so: cannot open shared object file:
No such file or directory
%
```

This error is generated because the dynamic linker can't locate the `libfoo.so` library. When shared libraries are loaded, the system normally only checks a few standard locations such as `/usr/lib` and `/usr/local/lib`. To fix this problem, there are several things you can do. First, you can recompile your extension module with extra path information. For example, on Linux you can do this:

```
$ gcc -shared example.o example_wrap.o -L/home/beazley/projects/lib -lfoo \
      -Xlinker -rpath /home/beazley/projects/lib \
      -o example.so
```

Alternatively, you can set the `LD_LIBRARY_PATH` environment variable to include the directory with your shared libraries. If setting `LD_LIBRARY_PATH`, be aware that setting this variable can introduce a noticeable performance impact on all other applications that you run. To set it only for Tcl, you might want to do this instead:

```
$ env LD_LIBRARY_PATH=/home/beazley/projects/lib tclsh
```

Finally, you can use a command such as `ldconfig` to add additional search paths to the default system configuration (this requires root access and you will need to read the man pages).

### 37.1.5 Compilation of C++ extensions

Compilation of C++ extensions has traditionally been a tricky problem. Since the Tcl interpreter is written in C, you need to take steps to make sure C++ is properly initialized and that modules are compiled correctly.

On most machines, C++ extension modules should be linked using the C++ compiler. For example:

```
% swig -c++ -tcl example.i
% g++ -fPIC -c example.cxx
% g++ -fPIC -c example_wrap.cxx -I/usr/local/include
% g++ -shared example.o example_wrap.o -o example.so
```

In addition to this, you may need to include additional library files to make it work. For example, if you are using the Sun C++ compiler on Solaris, you often need to add an extra library `-lCrun` like this:

```
% swig -c++ -tcl example.i
% CC -KPIC -c example.cxx
% CC -KPIC -c example_wrap.cxx -I/usr/local/include
% CC -G example.o example_wrap.o -L/opt/SUNWspro/lib -o example.so -lCrun
```

Of course, the extra libraries to use are completely non-portable---you will probably need to do some experimentation.

Sometimes people have suggested that it is necessary to relink the Tcl interpreter using the C++ compiler to make C++ extension modules work. In the experience of this author, this has never actually appeared to be necessary. Relinking the interpreter with C++ really only includes the special run-time libraries described above---as long as you link your extension modules with these libraries, it should not be necessary to rebuild Tcl.

If you aren't entirely sure about the linking of a C++ extension, you might look at an existing C++ program. On many Unix machines, the `ldd` command will list library dependencies. This should give you some clues about what you might have to include when you link your extension module. For example:

```
$ ldd swig
        libstdc++-libc6.1-1.so.2 => /usr/lib/libstdc++-libc6.1-1.so.2 (0x40019000)
        libm.so.6 => /lib/libm.so.6 (0x4005b000)
        libc.so.6 => /lib/libc.so.6 (0x40077000)
        /lib/ld-linux.so.2 => /lib/ld-linux.so.2 (0x40000000)
$
```

As a final complication, a major weakness of C++ is that it does not define any sort of standard for binary linking of libraries. This means that C++ code compiled by different compilers will not link together properly as libraries nor is the memory layout of classes and data structures implemented in any kind of portable manner. In a monolithic C++ program, this problem may be unnoticed. However, in Tcl, it is possible for different extension modules to be compiled with different C++ compilers. As long as these modules are self-contained, this probably won't matter. However, if these modules start sharing data, you will need to take steps to avoid segmentation faults and other erratic program behavior. If working with lots of software components, you might want to investigate using a more formal standard such as COM.

### 37.1.6 Compiling for 64-bit platforms

On platforms that support 64-bit applications (Solaris, Irix, etc.), special care is required when building extension modules. On these machines, 64-bit applications are compiled and linked using a different set of compiler/linker options. In addition, it is not generally possible to mix 32-bit and 64-bit code together in the same application.

To utilize 64-bits, the Tcl executable will need to be recompiled as a 64-bit application. In addition, all libraries, wrapper code, and every other part of your application will need to be compiled for 64-bits. If you plan to use other third-party extension modules, they will also have to be recompiled as 64-bit extensions.

If you are wrapping commercial software for which you have no source code, you will be forced to use the same linking standard as used by that software. This may prevent the use of 64-bit extensions. It may also introduce problems on platforms that support more than one linking standard (e.g., -o32 and -n32 on Irix).

### 37.1.7 Setting a package prefix

To avoid namespace problems, you can instruct SWIG to append a package prefix to all of your functions and variables. This is done using the -prefix option as follows :

```
swig -tcl -prefix Foo example.i
```

If you have a function "`bar`" in the SWIG file, the prefix option will append the prefix to the name when creating a command and call it "`Foo_bar`".

### 37.1.8 Using namespaces

Alternatively, you can have SWIG install your module into a Tcl namespace by specifying the `-namespace` option :

```
swig -tcl -namespace example.i
```

By default, the name of the namespace will be the same as the module name, but you can override it using the `-prefix` option.

When the `-namespace` option is used, objects in the module are always accessed with the namespace name such as `Foo::bar`.

## 37.2 Building Tcl/Tk Extensions under Windows 95/NT

Building a SWIG extension to Tcl/Tk under Windows 95/NT is roughly similar to the process used with Unix. Normally, you will want to produce a DLL that can be loaded into tclsh or wish. This section covers the process of using SWIG with Microsoft Visual C++. although the procedure may be similar with other compilers.

### 37.2.1 Running SWIG from Developer Studio

If you are developing your application within Microsoft developer studio, SWIG can be invoked as a custom build option. The process roughly follows these steps :

- Open up a new workspace and use the AppWizard to select a DLL project.
- Add both the SWIG interface file (the .i file), any supporting C files, and the name of the wrapper file that will be created by SWIG (ie. `example_wrap.c`). Note : If using C++, choose a different suffix for the wrapper file such as `example_wrap.cxx`. Don't worry if the wrapper file doesn't exist yet--Developer studio will keep a reference to it around.
- Select the SWIG interface file and go to the settings menu. Under settings, select the "Custom Build" option.
- Enter "SWIG" in the description field.
- Enter "`swig -tcl -o $(ProjDir)\$(InputName)_wrap.c $(InputPath)` " in the "Build command(s) field"
- Enter "`$(ProjDir)\$(InputName)_wrap.c`" in the "Output files(s) field".
- Next, select the settings for the entire project and go to "C++:Preprocessor". Add the include directories for your Tcl installation under "Additional include directories".
- Finally, select the settings for the entire project and go to "Link Options". Add the Tcl library file to your link libraries. For example "`tcl80.lib`". Also, set the name of the output file to match the name of your Tcl module (ie. example.dll).
- Build your project.

Now, assuming all went well, SWIG will be automatically invoked when you build your project. Any changes made to the interface file will result in SWIG being automatically invoked to produce a new version of the wrapper file. To run your new Tcl extension, simply run `tclsh` or `wish` and use the `load` command. For example :

```
MSDOS > tclsh80
% load example.dll
% fact 4
24
%
```

### 37.2.2 Using NMAKE

Alternatively, SWIG extensions can be built by writing a Makefile for NMAKE. To do this, make sure the environment variables for MSVC++ are available and the MSVC++ tools are in your path. Now, just write a short Makefile like this :

```
# Makefile for building various SWIG generated extensions

SRCS          = example.c
IFILE         = example
INTERFACE     = $(IFILE).i
WRAPFILE      = $(IFILE)_wrap.c

# Location of the Visual C++ tools (32 bit assumed)

TOOLS         = c:\msdev
TARGET        = example.dll
CC            = $(TOOLS)\bin\cl.exe
LINK          = $(TOOLS)\bin\link.exe
INCLUDE32     = -I$(TOOLS)\include
MACHINE       = IX86

# C Library needed to build a DLL

DLLIBC        = msvcrt.lib oldnames.lib

# Windows libraries that are apparently needed
WINLIB        = kernel32.lib advapi32.lib user32.lib gdi32.lib comdlg32.lib
winspool.lib

# Libraries common to all DLLs
LIBS          = $(DLLIBC) $(WINLIB)

# Linker options
LOPT      = -debug:full -debugtype:cv /NODEFAULTLIB /RELEASE /NOLOGO /
MACHINE:$(MACHINE) -entry:_DllMainCRTStartup@12 -dll

# C compiler flags

CFLAGS    = /Z7 /Od /c /nologo
TCL_INCLUDES  = -Id:\tcl8.0a2\generic -Id:\tcl8.0a2\win
TCLLIB        = d:\tcl8.0a2\win\tcl80.lib

tcl:
        ..\..\swig -tcl -o $(WRAPFILE) $(INTERFACE)
        $(CC) $(CFLAGS) $(TCL_INCLUDES) $(SRCS) $(WRAPFILE)
        set LIB=$(TOOLS)\lib
        $(LINK) $(LOPT) -out:example.dll $(LIBS) $(TCLLIB) example.obj example_wrap.obj
```

To build the extension, run NMAKE (you may need to run vcvars32 first). This is a pretty minimal Makefile, but hopefully it's enough to get you started. With a little practice, you'll be making lots of Tcl extensions.

## 37.3 A tour of basic C/C++ wrapping

By default, SWIG tries to build a very natural Tcl interface to your C/C++ code. Functions are wrapped as functions, classes are wrapped in an interface that mimics the style of Tk widgets and [incr Tcl] classes. This section briefly covers the essential aspects of this wrapping.

### 37.3.1 Modules

The SWIG `%module` directive specifies the name of the Tcl module. If you specify `%module example`', then everything is compiled into an extension module `example.so`. When choosing a module name, make sure you don't use the same name as a built-in Tcl command.

One pitfall to watch out for is module names involving numbers. If you specify a module name like `%module md5`, you'll find that the load command no longer seems to work:

```
% load ./md5.so
couldn't find procedure Md_Init
```

To fix this, supply an extra argument to `load` like this:

```
% load ./md5.so md5
```

### 37.3.2 Functions

Global functions are wrapped as new Tcl built-in commands. For example,

```
%module example
int fact(int n);
```

creates a built-in function `fact` that works exactly like you think it does:

```
% load ./example.so
% fact 4
24
% set x [fact 6]
%
```

### 37.3.3 Global variables

C/C++ global variables are wrapped by Tcl global variables. For example:

```
// SWIG interface file with global variables
%module example
...
%inline %{
extern double density;
%}
...
```

Now look at the Tcl interface:

```
% puts $density          # Output value of C global variable
1.0
% set density 0.95       # Change value
```

If you make an error in variable assignment, you will get an error message. For example:

```
% set density "hello"
can't set "density": Type error. expected a double.
%
```

If a variable is declared as `const`, it is wrapped as a read-only variable. Attempts to modify its value will result in an error.

To make ordinary variables read-only, you can use the `%immutable` directive. For example:

```
%{
extern char *path;
%}
%immutable;
extern char *path;
%mutable;
```

The `%immutable` directive stays in effect until it is explicitly disabled or cleared using `%mutable`. See the Creating read-only variables section for further details.

If you just want to make a specific variable immutable, supply a declaration name. For example:

```
%{
extern char *path;
%}
%immutable path;
...
extern char *path;      // Read-only (due to %immutable)
```

### 37.3.4 Constants and enums

C/C++ constants are installed as global Tcl variables containing the appropriate value. To create a constant, use `#define`, `enum` , or the `%constant` directive. For example:

```
#define PI 3.14159
#define VERSION "1.0"

enum Beverage { ALE, LAGER, STOUT, PILSNER };

%constant int FOO = 42;
%constant const char *path = "/usr/local";
```

For enums, make sure that the definition of the enumeration actually appears in a header file or in the wrapper file somehow---if you just stick an enum in a SWIG interface without also telling the C compiler about it, the wrapper code won't compile.

Note: declarations declared as `const` are wrapped as read-only variables and will be accessed using the `cvar` object described in the previous section. They are not wrapped as constants. For further discussion about this, see the SWIG Basics chapter.

Constants are not guaranteed to remain constant in Tcl---the value of the constant could be accidentally reassigned. You will just have to be careful.

A peculiarity of installing constants as variables is that it is necessary to use the Tcl `global` statement to access constants in procedure bodies. For example:

```
proc blah {} {
  global FOO
  bar $FOO
}
```

### 37.3.5 Pointers

C/C++ pointers are fully supported by SWIG. Furthermore, SWIG has no problem working with incomplete type information. Here is a rather simple interface:

```
%module example

FILE *fopen(const char *filename, const char *mode);
int fputs(const char *, FILE *);
int fclose(FILE *);
```

When wrapped, you will be able to use the functions in a natural way from Tcl. For example:

```
% load ./example.so
% set f [fopen junk w]
% fputs "Hello World\n" $f
% fclose $f
```

If this makes you uneasy, rest assured that there is no deep magic involved. Underneath the covers, pointers to C/C++ objects are simply represented as opaque values--normally an encoded character string like this:

```
% puts $f
_c0671108_p_FILE
%
```

This pointer value can be freely passed around to different C functions that expect to receive an object of type `FILE *`. The only thing you can't do is dereference the pointer from Tcl.

The NULL pointer is represented by the string `NULL`.

As much as you might be inclined to modify a pointer value directly from Tcl, don't. The hexadecimal encoding is not necessarily the same as the logical memory address of the underlying object. Instead it is the raw byte encoding of the pointer value. The encoding will vary depending on the native byte-ordering of the platform (i.e., big-endian vs. little-endian). Similarly, don't try to manually cast a pointer to a new type by simply replacing the type-string. This may not work like you expect and it is particularly dangerous when casting C++ objects. If you need to cast a pointer or change its value, consider writing some helper functions instead. For example:

```
%inline %{
/* C-style cast */
Bar *FooToBar(Foo *f) {
  return (Bar *) f;
}

/* C++-style cast */
Foo *BarToFoo(Bar *b) {
  return dynamic_cast<Foo*>(b);
}

Foo *IncrFoo(Foo *f, int i) {
  return f+i;
}
%}
```

Also, if working with C++, you should always try to use the new C++ style casts. For example, in the above code, the C-style cast may return a bogus result whereas as the C++-style cast will return `None` if the conversion can't be performed.

### 37.3.6 Structures

If you wrap a C structure, it is wrapped by a Tcl interface that somewhat resembles a Tk widget. This provides a very natural interface. For example,

```
struct Vector {
  double x, y, z;
};
```

is used as follows:

```
% Vector v
% v configure -x 3.5 -y 7.2
% puts "[v cget -x] [v cget -y] [v cget -z]"
3.5 7.2 0.0
%
```

Similar access is provided for unions and the data members of C++ classes.

In the above example, `v` is a name that's used for the object. However, underneath the covers, there's a pointer to a raw C structure. This can be obtained by looking at the `-this` attribute. For example:

```
% puts [v cget -this]
_88e31408_p_Vector
```

Further details about the relationship between the Tcl and the underlying C structure are covered a little later.

`const` members of a structure are read-only. Data members can also be forced to be read-only using the `%immutable` directive. For example:

```
struct Foo {
  ...
  %immutable;
  int x;        /* Read-only members */
  char *name;
  %mutable;
  ...
};
```

When `char *` members of a structure are wrapped, the contents are assumed to be dynamically allocated using `malloc` or `new` (depending on whether or not SWIG is run with the `-c++` option). When the structure member is set, the old contents will be released and a new value created. If this is not the behavior you want, you will have to use a typemap (described later).

If a structure contains arrays, access to those arrays is managed through pointers. For example, consider this:

```
struct Bar {
  int  x[16];
};
```

If accessed in Tcl, you will see behavior like this:

```
% Bar b
% puts [b cget -x]
_801861a4_p_int
%
```

This pointer can be passed around to functions that expect to receive an `int *` (just like C). You can also set the value of an array member using another pointer. For example:

```
% Bar c
% c configure -x [b cget -x]   # Copy contents of b.x to c.x
```

For array assignment, SWIG copies the entire contents of the array starting with the data pointed to by `b.x`. In this example, 16 integers would be copied. Like C, SWIG makes no assumptions about bounds checking---if you pass a bad pointer, you may get a segmentation fault or access violation.

When a member of a structure is itself a structure, it is handled as a pointer. For example, suppose you have two structures like this:

```
struct Foo {
  int a;
};

struct Bar {
  Foo f;
};
```

Now, suppose that you access the `f` attribute of `Bar` like this:

```
% Bar b
% set x [b cget -f]
```

In this case, `x` is a pointer that points to the `Foo` that is inside `b`. This is the same value as generated by this C code:

```
Bar b;
Foo *x = &b->f;       /* Points inside b */
```

However, one peculiarity of accessing a substructure like this is that the returned value does work quite like you might expect. For example:

```
% Bar b
% set x [b cget -f]
% x cget -a
invalid command name "x"
```

This is because the returned value was not created in a normal way from the interpreter (`x` is not a command object). To make it function normally, just evaluate the variable like this:

```
% Bar b
% set x [b cget -f]
% $x cget -a
0
%
```

In this example, `x` points inside the original structure. This means that modifications work just like you would expect. For example:

```
% Bar b
% set x [b cget -f]
% $x configure -a 3            # Modifies contents of f (inside b)
% [b cget -f] -configure -a 3  # Same thing
```

In many of these structure examples, a simple name like "v" or "b" has been given to wrapped structures. If necessary, this name can be passed to functions that expect to receive an object. For example, if you have a function like this,

```
void blah(Foo *f);
```

you can call the function in Tcl as follows:

```
% Foo x            # Create a Foo object
% blah x           # Pass the object to a function
```

It is also possible to call the function using the raw pointer value. For instance:

```
% blah [x cget -this]   # Pass object to a function
```

It is also possible to create and use objects using variables. For example:

```
% set b [Bar]            # Create a Bar
% $b cget -f             # Member access
% puts $b
_108fea88_p_Bar
%
```

Finally, to destroy objects created from Tcl, you can either let the object name go out of scope or you can explicitly delete the object as shown below. Objects won't get automatically destroyed when the Tcl program exits, so if it's important that the C++ destructor is called for a class you'll need to make sure that you explicitly do this for objects of that class before program exit.

For example:

```
% Foo f                # Create object f
% rename f ""
```

or

```
% Foo f                # Create object f
% f -delete
```

Note: Tcl only destroys the underlying object if it has ownership. See the memory management section that appears shortly.

### 37.3.7 C++ classes

C++ classes are wrapped as an extension of structure wrapping. For example, if you have this class,

```
class List {
public:
  List();
  ~List();
  int  search(char *item);
  void insert(char *item);
  void remove(char *item);
  char *get(int n);
  int  length;
};
```

you can use it in Tcl like this:

```
% List x
% x insert Ale
% x insert Stout
% x insert Lager
% x get 1
Stout
% puts [x cget -length]
3
%
```

Class data members are accessed in the same manner as C structures.

Static class members are accessed as global functions or variables. To illustrate, suppose you have a class like this:

```
class Spam {
public:
  static void foo();
  static int bar;
};
```

In Tcl, the static member is accessed as follows:

```
% Spam_foo        # Spam::foo()
% puts $Spam_bar  # Spam::bar
```

### 37.3.8 C++ inheritance

SWIG is fully aware of issues related to C++ inheritance. Therefore, if you have classes like this

```
class Foo {
...
};

class Bar : public Foo {
...
```

```
};
```

An object of type `Bar` can be used where a `Foo` is expected. For example, if you have this function:

```
void spam(Foo *f);
```

then the function `spam()` accepts a `Foo *` or a pointer to any class derived from `Foo`. For instance:

```
% Foo f      # Create a Foo
% Bar b      # Create a Bar
% spam f     # OK
% spam b     # OK
```

It is safe to use multiple inheritance with SWIG.

### 37.3.9 Pointers, references, values, and arrays

In C++, there are many different ways a function might receive and manipulate objects. For example:

```
void spam1(Foo *x);      // Pass by pointer
void spam2(Foo &x);      // Pass by reference
void spam3(Foo x);       // Pass by value
void spam4(Foo x[]);     // Array of objects
```

In Tcl, there is no detailed distinction like this. Because of this, SWIG unifies all of these types together in the wrapper code. For instance, if you actually had the above functions, it is perfectly legal to do this:

```
% Foo f             # Create a Foo
% spam1 f           # Ok. Pointer
% spam2 f           # Ok. Reference
% spam3 f           # Ok. Value.
% spam4 f           # Ok. Array (1 element)
```

Similar behavior occurs for return values. For example, if you had functions like this,

```
Foo *spam5();
Foo &spam6();
Foo  spam7();
```

then all three functions will return a pointer to some `Foo` object. Since the third function (spam7) returns a value, newly allocated memory is used to hold the result and a pointer is returned (Tcl will release this memory when the return value is garbage collected).

### 37.3.10 C++ overloaded functions

C++ overloaded functions, methods, and constructors are mostly supported by SWIG. For example, if you have two functions like this:

```
void foo(int);
void foo(char *c);
```

You can use them in Tcl in a straightforward manner:

```
% foo 3            # foo(int)
% foo Hello        # foo(char *c)
```

Similarly, if you have a class like this,

```
class Foo {
public:
  Foo();
  Foo(const Foo &);
  ...
};
```

you can write Tcl code like this:

```
% Foo f               # Create a Foo
% Foo g f             # Copy f
```

Overloading support is not quite as flexible as in C++. Sometimes there are methods that SWIG can't disambiguate. For example:

```
void spam(int);
void spam(short);
```

or

```
void foo(Bar *b);
void foo(Bar &b);
```

If declarations such as these appear, you will get a warning message like this:

```
example.i:12: Warning 509: Overloaded method spam(short) effectively ignored,
```

```
example.i:11: Warning 509: as it is shadowed by spam(int).
```

To fix this, you either need to ignore or rename one of the methods. For example:

```
%rename(spam_short) spam(short);
...
void spam(int);
void spam(short);   // Accessed as spam_short
```

or

```
%ignore spam(short);
...
void spam(int);
void spam(short);   // Ignored
```

SWIG resolves overloaded functions and methods using a disambiguation scheme that ranks and sorts declarations according to a set of type-precedence rules. The order in which declarations appear in the input does not matter except in situations where ambiguity arises--in this case, the first declaration takes precedence.

Please refer to the "SWIG and C++" chapter for more information about overloading.

### 37.3.11 C++ operators

Certain C++ overloaded operators can be handled automatically by SWIG. For example, consider a class like this:

```
class Complex {
private:
  double rpart, ipart;
public:
  Complex(double r = 0, double i = 0) : rpart(r), ipart(i) { }
  Complex(const Complex &c) : rpart(c.rpart), ipart(c.ipart) { }
  Complex &operator=(const Complex &c);
  Complex operator+(const Complex &c) const;
  Complex operator-(const Complex &c) const;
  Complex operator*(const Complex &c) const;
  Complex operator-() const;

  double re() const { return rpart; }
  double im() const { return ipart; }
};
```

When wrapped, it works like this:

```
% Complex c 3 4
% Complex d 7 8
% set e [c + d]
% $e re
10.0
% $e im
12.0
```

It should be stressed that operators in SWIG have no relationship to operators in Tcl. In fact, the only thing that's happening here is that an operator like `operator +` has been renamed to a method + . Therefore, the statement `[c + d]` is really just invoking the + method on `c`. When more than operator is defined (with different arguments), the standard method overloading facilities are used. Here is a rather odd looking example:

```
% Complex c 3 4
% Complex d 7 8
% set e [c - d]       # operator-(const Complex &)
% puts "[$e re] [$e im]"
10.0 12.0
% set f [c -]         # operator-()
% puts "[$f re] [$f im]"
-3.0 -4.0
%
```

One restriction with operator overloading support is that SWIG is not able to fully handle operators that aren't defined as part of the class. For example, if you had code like this

```
class Complex {
...
friend Complex operator+(double, const Complex &c);
...
};
```

then SWIG doesn't know what to do with the friend function--in fact, it simply ignores it and issues a warning. You can still wrap the operator, but you may have to encapsulate it in a special function. For example:

```
%rename(Complex_add_dc) operator+(double, const Complex &);
...
Complex operator+(double, const Complex &c);
```

There are ways to make this operator appear as part of the class using the `%extend` directive. Keep reading.

### 37.3.12 C++ namespaces

SWIG is aware of C++ namespaces, but namespace names do not appear in the module nor do namespaces result in a module that is broken up into submodules or packages. For example, if you have a file like this,

```
%module example
```

```
namespace foo {
  int fact(int n);
  struct Vector {
    double x, y, z;
  };
};
```

it works in Tcl as follows:

```
% load ./example.so
% fact 3
6
% Vector v
% v configure -x 3.4
```

If your program has more than one namespace, name conflicts (if any) can be resolved using `%rename` For example:

```
%rename(Bar_spam) Bar::spam;

namespace Foo {
  int spam();
}

namespace Bar {
  int spam();
}
```

If you have more than one namespace and your want to keep their symbols separate, consider wrapping them as separate SWIG modules. For example, make the module name the same as the namespace and create extension modules for each namespace separately. If your program utilizes thousands of small deeply nested namespaces each with identical symbol names, well, then you get what you deserve.

### 37.3.13 C++ templates

C++ templates don't present a huge problem for SWIG. However, in order to create wrappers, you have to tell SWIG to create wrappers for a particular template instantiation. To do this, you use the `%template` directive. For example:

```
%module example
%{
#include "pair.h"
%}

template<class T1, class T2>
struct pair {
  typedef T1 first_type;
  typedef T2 second_type;
  T1 first;
  T2 second;
  pair();
  pair(const T1&, const T2&);
  ~pair();
};

%template(pairii) pair<int, int>;
```

In Tcl:

```
% pairii p 3 4
% p cget -first
3
% p cget -second
4
```

Obviously, there is more to template wrapping than shown in this example. More details can be found in the SWIG and C++ chapter. Some more complicated examples will appear later.

### 37.3.14 C++ Smart Pointers

In certain C++ programs, it is common to use classes that have been wrapped by so-called "smart pointers." Generally, this involves the use of a template class that implements `operator->()` like this:

```
template<class T> class SmartPtr {
  ...
  T *operator->();
  ...
}
```

Then, if you have a class like this,

```
class Foo {
public:
  int x;
  int bar();
};
```

A smart pointer would be used in C++ as follows:

```
SmartPtr<Foo> p = CreateFoo();   // Created somehow (not shown)
...
p->x = 3;                        // Foo::x
int y = p->bar();                // Foo::bar
```

To wrap this in Tcl, simply tell SWIG about the `SmartPtr` class and the low-level `Foo` object. Make sure you instantiate `SmartPtr` using `%template` if necessary. For example:

```
%module example
...
%template(SmartPtrFoo) SmartPtr<Foo>;
...
```

Now, in Tcl, everything should just "work":

```
% set p [CreateFoo]                 # Create a smart-pointer somehow
% $p configure -x 3                 # Foo::x
% $p bar                            # Foo::bar
```

If you ever need to access the underlying pointer returned by `operator->()` itself, simply use the `__deref__()` method. For example:

```
% set f [$p __deref__]    # Returns underlying Foo *
```

## 37.4 Further details on the Tcl class interface

In the previous section, a high-level view of Tcl wrapping was presented. A key component of this wrapping is that structures and classes are wrapped by Tcl class-like objects. This provides a very natural Tcl interface and allows SWIG to support a number of advanced features such as operator overloading. However, a number of low-level details were omitted. This section provides a brief overview of how the proxy classes work.

### 37.4.1 Proxy classes

In the "SWIG basics" and "SWIG and C++" chapters, details of low-level structure and class wrapping are described. To summarize those chapters, if you have a class like this

```
class Foo {
public:
  int x;
  int spam(int);
  ...
```

then SWIG transforms it into a set of low-level procedural wrappers. For example:

```
Foo *new_Foo() {
  return new Foo();
}
void delete_Foo(Foo *f) {
  delete f;
}
int Foo_x_get(Foo *f) {
  return f->x;
}
void Foo_x_set(Foo *f, int value) {
  f->x = value;
}
int Foo_spam(Foo *f, int arg1) {
  return f->spam(arg1);
}
```

These wrappers are actually found in the Tcl extension module. For example, you can certainly do this:

```
% load ./example.so
% set f [new_Foo]
% Foo_x_get $f
0
% Foo_spam $f 3
1
%
```

However, in addition to this, the classname `Foo` is used as an object constructor function. This allows objects to be encapsulated objects that look a lot like Tk widgets as shown in the last section.

### 37.4.2 Memory management

Associated with each wrapped object, is an ownership flag `thisown` The value of this flag determines who is responsible for deleting the underlying C++ object. If set to 1, the Tcl interpreter destroys the C++ object when the proxy class is garbage collected. If set to 0 (or if the attribute is missing), then the destruction of the proxy class has no effect on the C++ object.

When an object is created by a constructor or returned by value, Tcl automatically takes ownership of the result. For example:

```
class Foo {
public:
  Foo();
  Foo bar();
};
```

In Tcl:

```
% Foo f
% f cget -thisown
1
% set g [f bar]
% $g cget -thisown
1
```

On the other hand, when pointers are returned to Tcl, there is often no way to know where they came from. Therefore, the ownership is set to zero. For example:

```
class Foo {
public:
  ...
  Foo *spam();
  ...
};
```

```
% Foo f
% set s [f spam]
% $s cget -thisown
0
%
```

This behavior is especially important for classes that act as containers. For example, if a method returns a pointer to an object that is contained inside another object, you definitely don't want Tcl to assume ownership and destroy it!

Related to containers, ownership issues can arise whenever an object is assigned to a member or global variable. For example, consider this interface:

```
%module example

struct Foo {
  int  value;
  Foo  *next;
};

Foo *head = 0;
```

When wrapped in Tcl, careful observation will reveal that ownership changes whenever an object is assigned to a global variable. For example:

```
% Foo f
% f cget -thisown
1
% set head f
% f cget -thisown
0
```

In this case, C is now holding a reference to the object---you probably don't want Tcl to destroy it. Similarly, this occurs for members. For example:

```
% Foo f
% Foo g
% f cget -thisown
1
% g cget -thisown
1
% f configure -next g
% g cget -thisown
0
%
```

For the most part, memory management issues remain hidden. However, there are occasionally situations where you might have to manually change the ownership of an object. For instance, consider code like this:

```
class Node {
  Object *value;
public:
  void set_value(Object *v) { value = v; }
  ...
};
```

Now, consider the following Tcl code:

```
% Object v                # Create an object
% Node n                  # Create a node
% n setvalue v            # Set value
% v cget -thisown
1
%
```

In this case, the object `n` is holding a reference to `v` internally. However, SWIG has no way to know that this has occurred. Therefore, Tcl still thinks that it has ownership of the object. Should the proxy object be destroyed, then the C++ destructor will be invoked and `n` will be holding a stale-pointer. If you're lucky, you will only get a segmentation fault.

To work around this, it is always possible to flip the ownership flag. For example,

```
% v -disown             # Give ownership to C/C++
% v -acquire            # Acquire ownership
```

It is also possible to deal with situations like this using typemaps--an advanced topic discussed later.

## 37.5 Input and output parameters

A common problem in some C programs is handling parameters passed as simple pointers. For example:

```
void add(int x, int y, int *result) {
  *result = x + y;
```

```
}
```

or perhaps

```
int sub(int *x, int *y) {
  return *x+*y;
}
```

The easiest way to handle these situations is to use the `typemaps.i` file. For example:

```
%module example
%include "typemaps.i"

void add(int, int, int *OUTPUT);
int  sub(int *INPUT, int *INPUT);
```

In Tcl, this allows you to pass simple values instead of pointer. For example:

```
set a [add 3 4]
puts $a
7
```

Notice how the `INPUT` parameters allow integer values to be passed instead of pointers and how the `OUTPUT` parameter creates a return result.

If you don't want to use the names `INPUT` or `OUTPUT` , use the `%apply` directive. For example:

```
%module example
%include "typemaps.i"

%apply int *OUTPUT { int *result };
%apply int *INPUT  { int *x, int *y};

void add(int x, int y, int *result);
int  sub(int *x, int *y);
```

If a function mutates one of its parameters like this,

```
void negate(int *x) {
  *x = -(*x);
}
```

you can use `INOUT` like this:

```
%include "typemaps.i"
...
void negate(int *INOUT);
```

In Tcl, a mutated parameter shows up as a return value. For example:

```
set a [negate 3]
puts $a
-3
```

The most common use of these special typemap rules is to handle functions that return more than one value. For example, sometimes a function returns a result as well as a special error code:

```
/* send message, return number of bytes sent, along with success code */
int send_message(char *text, int *success);
```

To wrap such a function, simply use the `OUTPUT` rule above. For example:

```
%module example
%include "typemaps.i"
%apply int *OUTPUT { int *success };
...
int send_message(char *text, int *success);
```

When used in Tcl, the function will return multiple values as a list.

```
set r [send_message "Hello World"]
set bytes [lindex $r 0]
set success [lindex $r 1]
```

Another common use of multiple return values are in query functions. For example:

```
void get_dimensions(Matrix *m, int *rows, int *columns);
```

To wrap this, you might use the following:

```
%module example
%include "typemaps.i"
```

```
%apply int *OUTPUT { int *rows, int *columns };
...
void get_dimensions(Matrix *m, int *rows, *columns);
```

Now, in Perl:

```
set dim [get_dimensions $m]
set r  [lindex $dim 0]
set c  [lindex $dim 1]
```

## 37.6 Exception handling

The `%exception` directive can be used to create a user-definable exception handler in charge of converting exceptions in your C/C++ program into Tcl exceptions. The chapter on customization features contains more details, but suppose you extended the array example into a C++ class like the following :

```
class RangeError {};   // Used for an exception

class DoubleArray {
  private:
    int n;
    double *ptr;
  public:
    // Create a new array of fixed size
    DoubleArray(int size) {
      ptr = new double[size];
      n = size;
    }
    // Destroy an array
    ~DoubleArray() {
      delete ptr;
    }
    // Return the length of the array
    int   length() {
      return n;
    }

    // Get an item from the array and perform bounds checking.
    double getitem(int i) {
      if ((i >= 0) && (i < n))
        return ptr[i];
      else
        throw RangeError();
    }

    // Set an item in the array and perform bounds checking.
    void setitem(int i, double val) {
      if ((i >= 0) && (i < n))
        ptr[i] = val;
      else {
        throw RangeError();
      }
    }
  };
```

The functions associated with this class can throw a C++ range exception for an out-of-bounds array access. We can catch this in our Tcl extension by specifying the following in an interface file :

```
%exception {
  try {
    $action                // Gets substituted by actual function call
  }
  catch (RangeError) {
    Tcl_SetResult(interp, (char *)"Array index out-of-bounds", TCL_STATIC);
    return TCL_ERROR;
  }
}
```

As shown, the exception handling code will be added to every wrapper function. Since this is somewhat inefficient. You might consider refining the exception handler to only apply to specific methods like this:

```
%exception getitem {
  try {
    $action
  }
  catch (RangeError) {
    Tcl_SetResult(interp, (char *)"Array index out-of-bounds", TCL_STATIC);
    return TCL_ERROR;
  }
}

%exception setitem {
  try {
    $action
  }
  catch (RangeError) {
    Tcl_SetResult(interp, (char *)"Array index out-of-bounds", TCL_STATIC);
    return TCL_ERROR;
  }
}
```

In this case, the exception handler is only attached to methods and functions named `getitem` and `setitem`.

If you had a lot of different methods, you can avoid extra typing by using a macro. For example:

```
%define RANGE_ERROR
{
  try {
    $action
  }
  catch (RangeError) {
    Tcl_SetResult(interp, (char *)"Array index out-of-bounds", TCL_STATIC);
    return TCL_ERROR;
  }
}
%enddef

%exception getitem RANGE_ERROR;
%exception setitem RANGE_ERROR;
```

Since SWIG's exception handling is user-definable, you are not limited to C++ exception handling. See the chapter on " Customization Features" for more examples.

## 37.7 Typemaps

This section describes how you can modify SWIG's default wrapping behavior for various C/C++ datatypes using the `%typemap` directive. This is an advanced topic that assumes familiarity with the Tcl C API as well as the material in the "Typemaps " chapter.

Before proceeding, it should be stressed that typemaps are not a required part of using SWIG---the default wrapping behavior is enough in most cases. Typemaps are only used if you want to change some aspect of the primitive C-Tcl interface.

### 37.7.1 What is a typemap?

A typemap is nothing more than a code generation rule that is attached to a specific C datatype. For example, to convert integers from Tcl to C, you might define a typemap like this:

```
%module example

%typemap(in) int {
  if (Tcl_GetIntFromObj(interp, $input, &$1) == TCL_ERROR)
    return TCL_ERROR;
  printf("Received an integer : %d\n", $1);
}
%inline %{
extern int fact(int n);
%}
```

Typemaps are always associated with some specific aspect of code generation. In this case, the "in" method refers to the conversion of input arguments to C/C++. The datatype `int` is the datatype to which the typemap will be applied. The supplied C code is used to convert values. In this code a number of special variable prefaced by a `$` are used. The `$1` variable is placeholder for a local variable of type `int`. The `$input` variable is the input object of type `Tcl_Obj *`.

When this example is compiled into a Tcl module, it operates as follows:

```
% load ./example.so
% fact 6
Received an integer : 6
720
```

In this example, the typemap is applied to all occurrences of the `int` datatype. You can refine this by supplying an optional parameter name. For example:

```
%module example

%typemap(in) int n {
  if (Tcl_GetIntFromObj(interp, $input, &$1) == TCL_ERROR)
    return TCL_ERROR;
  printf("n = %d\n", $1);
}
%inline %{
extern int fact(int n);
%}
```

In this case, the typemap code is only attached to arguments that exactly match `int n`.

The application of a typemap to specific datatypes and argument names involves more than simple text-matching--typemaps are fully integrated into the SWIG type-system. When you define a typemap for `int`, that typemap applies to `int` and qualified variations such as `const int`. In addition, the typemap system follows `typedef` declarations. For example:

```
%typemap(in) int n {
  if (Tcl_GetIntFromObj(interp, $input, &$1) == TCL_ERROR)
    return TCL_ERROR;
  printf("n = %d\n", $1);
}
%inline %{
typedef int Integer;
extern int fact(Integer n);    // Above typemap is applied
%}
```

However, the matching of `typedef` only occurs in one direction. If you defined a typemap for `Integer`, it is not applied to arguments of type `int`.

Typemaps can also be defined for groups of consecutive arguments. For example:

```
%typemap(in) (char *str, int len) {
    $1 = Tcl_GetStringFromObj($input, &$2);
};

int count(char c, char *str, int len);
```

When a multi-argument typemap is defined, the arguments are always handled as a single Tcl object. This allows the function to be used like this (notice how the length parameter is omitted):

```
% count e "Hello World"
1
```

### 37.7.2 Tcl typemaps

The previous section illustrated an "in" typemap for converting Tcl objects to C. A variety of different typemap methods are defined by the Tcl module. For example, to convert a C integer back into a Tcl object, you might define an "out" typemap like this:

```
%typemap(out) int {
  Tcl_SetObjResult(interp, Tcl_NewIntObj($1));
}
```

The following list details all of the typemap methods that can be used by the Tcl module:

`%typemap(in)`

Converts Tcl objects to input function arguments

`%typemap(out)`

Converts return value of a C function to a Tcl object

`%typemap(varin)`

Assigns a C global variable from a Tcl object

`%typemap(varout)`

Returns a C global variable as a Tcl object

`%typemap(freearg)`

Cleans up a function argument (if necessary)

`%typemap(argout)`

Output argument processing

`%typemap(ret)`

Cleanup of function return values

`%typemap(consttab)`

Creation of Tcl constants (constant table)

`%typemap(constcode)`

Creation of Tcl constants (init function)

`%typemap(memberin)`

Setting of structure/class member data

`%typemap(globalin)`

Setting of C global variables

`%typemap(check)`

Checks function input values.

`%typemap(default)`

Set a default value for an argument (making it optional).

`%typemap(arginit)`

Initialize an argument to a value before any conversions occur.

Examples of these methods will appear shortly.

### 37.7.3 Typemap variables

Within typemap code, a number of special variables prefaced with a `$` may appear. A full list of variables can be found in the "Typemaps" chapter. This is a list of the most common variables:

`$1`

A C local variable corresponding to the actual type specified in the `%typemap` directive. For input values, this is a C local variable that's supposed to hold an argument value. For output values, this is the raw result that's supposed to be returned to Tcl.

`$input`

A `Tcl_Obj *` holding a raw Tcl object with an argument or variable value.

`$result`

A `Tcl_Obj *` that holds the result to be returned to Tcl.

`$1_name`

The parameter name that was matched.

`$1_type`

The actual C datatype matched by the typemap.

`$1_ltype`

An assignable version of the datatype matched by the typemap (a type that can appear on the left-hand-side of a C assignment operation). This type is stripped of qualifiers and may be an altered version of `$1_type`. All arguments and local variables in wrapper functions are declared using this type so that their values can be properly assigned.

`$symname`

The Tcl name of the wrapper function being created.

### 37.7.4 Converting a Tcl list to a char **

A common problem in many C programs is the processing of command line arguments, which are usually passed in an array of NULL terminated strings. The following SWIG interface file allows a Tcl list to be used as a `char **` object.

```
%module argv

// This tells SWIG to treat char ** as a special case
%typemap(in) char ** {
  Tcl_Obj **listobjv;
  int       nitems;
  int       i;
  if (Tcl_ListObjGetElements(interp, $input, &nitems, &listobjv) == TCL_ERROR) {
    return TCL_ERROR;
  }
  $1 = (char **) malloc((nitems+1)*sizeof(char *));
  for (i = 0; i < nitems; i++) {
    $1[i] = Tcl_GetString(listobjv[i]);
  }
  $1[i] = 0;
}

// This gives SWIG some cleanup code that will get called after the function call
%typemap(freearg) char ** {
  free($1);
}

// Now a test functions
%inline %{
  int print_args(char **argv) {
    int i = 0;
    while (argv[i]) {
      printf("argv[%d] = %s\n", i, argv[i]);
      i++;
    }
    return i;
  }
%}
%include "tclsh.i"
```

In Tcl:

```
% print_args {John Guido Larry}
argv[0] = John
argv[1] = Guido
argv[2] = Larry
3
```

### 37.7.5 Returning values in arguments

The "argout" typemap can be used to return a value originating from a function argument. For example :

```
// A typemap defining how to return an argument by appending it to the result
%typemap(argout) double *outvalue {
  Tcl_Obj *o = Tcl_NewDoubleObj($1);
  Tcl_ListObjAppendElement(interp, $result, o);
}

// A typemap telling SWIG to ignore an argument for input
// However, we still need to pass a pointer to the C function
%typemap(in, numinputs=0) double *outvalue (double temp) {
  $1 = &temp;
}

// Now a function returning two values
int mypow(double a, double b, double *outvalue) {
  if ((a < 0) || (b < 0)) return -1;
  *outvalue = pow(a, b);
  return 0;
};
```

When wrapped, SWIG matches the `argout` typemap to the "`double *outvalue`" argument. The numinputs=0 specification tells SWIG to simply ignore this argument when generating wrapper code. As a result, a Tcl function using these typemaps will work like this :

```
% mypow 2 3     # Returns two values, a status value and the result
0 8
%
```

### 37.7.6 Useful functions

The following tables provide some functions that may be useful in writing Tcl typemaps.

**Integers**

```
Tcl_Obj   *Tcl_NewIntObj(int Value);
void       Tcl_SetIntObj(Tcl_Obj *obj, int Value);
int        Tcl_GetIntFromObj(Tcl_Interp *, Tcl_Obj *obj, int *ip);
```

**Floating Point**

```
Tcl_Obj  *Tcl_NewDoubleObj(double Value);
void      Tcl_SetDoubleObj(Tcl_Obj *obj, double value);
int       Tcl_GetDoubleFromObj(Tcl_Interp *, Tcl_Obj *o, double *dp);
```

**Strings**

```
Tcl_Obj  *Tcl_NewStringObj(char *str, int len);
char     *Tcl_GetStringFromObj(Tcl_Obj *obj, int *len);
void      Tcl_AppendToObj(Tcl_Obj *obj, char *str, int len);
```

**Lists**

```
Tcl_Obj  *Tcl_NewListObj(int objc, Tcl_Obj *objv);
int       Tcl_ListObjAppendList(Tcl_Interp *, Tcl_Obj *listPtr, Tcl_Obj *elemListPtr);
int       Tcl_ListObjAppendElement(Tcl_Interp *, Tcl_Obj *listPtr, Tcl_Obj *element);
int       Tcl_ListObjGetElements(Tcl_Interp *, Tcl_Obj *listPtr, int *objcPtr,
                                 Tcl_Obj ***objvPtr);
int       Tcl_ListObjLength(Tcl_Interp *, Tcl_Obj *listPtr, int *intPtr);
int       Tcl_ListObjIndex(Tcl_Interp *, Tcl_Obj *listPtr, int index,
                           Tcl_Obj_Obj **objptr);
int       Tcl_ListObjReplace(Tcl_Interp *, Tcl_Obj *listPtr, int first, int count,
                             int objc, Tcl_Obj *objv);
```

**Objects**

```
Tcl_Obj *Tcl_DuplicateObj(Tcl_Obj *obj);
void     Tcl_IncrRefCount(Tcl_Obj *obj);
void     Tcl_DecrRefCount(Tcl_Obj *obj);
int      Tcl_IsShared(Tcl_Obj *obj);
```

### 37.7.7 Standard typemaps

The following typemaps show how to convert a few common kinds of objects between Tcl and C (and to give a better idea of how typemaps work)

**Integer conversion**

```
%typemap(in) int, short, long {
  int temp;
  if (Tcl_GetIntFromObj(interp, $input, &temp) == TCL_ERROR)
    return TCL_ERROR;
  $1 = ($1_ltype) temp;
}
```

```
%typemap(out) int, short, long {
  Tcl_SetIntObj($result, (int) $1);
}
```

**Floating point conversion**

```
%typemap(in) float, double {
  double temp;
  if (Tcl_GetDoubleFromObj(interp, $input, &temp) == TCL_ERROR)
    return TCL_ERROR;
  $1 = ($1_ltype) temp;
}
```

```
%typemap(out) float, double {
  Tcl_SetDoubleObj($result, $1);
}
```

**String Conversion**

```
%typemap(in) char * {
  int len;
  $1 = Tcl_GetStringFromObj(interp, &len);
}
```

```
%typemap(out, noblock=1, fragment="SWIG_FromCharPtr") char *, const char * {
  Tcl_SetObjResult(interp, SWIG_FromCharPtr((const char *)$1));
}
```

### 37.7.8 Pointer handling

SWIG pointers are mapped into Tcl strings containing the hexadecimal value and type. The following functions can be used to create and read pointer values.

`int SWIG_ConvertPtr(Tcl_Obj *obj, void **ptr, swig_type_info *ty, int flags)`

Converts a Tcl object `obj` to a C pointer. The result of the conversion is placed into the pointer located at `ptr`. `ty` is a SWIG type descriptor structure. `flags` is used to handle error checking and other aspects of conversion. It is currently reserved for future expansion. Returns 0 on success and -1 on error.

`Tcl_Obj *SWIG_NewPointerObj(void *ptr, swig_type_info *ty, int flags)`

Creates a new Tcl pointer object. `ptr` is the pointer to convert, `ty` is the SWIG type descriptor structure that describes the type, and `own` is a flag reserved for future expansion.

Both of these functions require the use of a special SWIG type-descriptor structure. This structure contains information about the mangled name of the datatype, type-equivalence information, as well as information about converting pointer values under C++ inheritance. For a type of `Foo *`, the type descriptor structure is usually accessed as follows:

```
Foo *f;
if (!SWIG_IsOK(SWIG_ConvertPtr($input, (void **) &f, SWIGTYPE_p_Foo, 0))) {
  SWIG_exception_fail(SWIG_TypeError, "in method '$symname', expecting type Foo");
}

Tcl_Obj *;
obj = SWIG_NewPointerObj(f, SWIGTYPE_p_Foo, 0);
```

In a typemap, the type descriptor should always be accessed using the special typemap variable `$1_descriptor`. For example:

```
%typemap(in) Foo * {
  if (!SWIG_IsOK(SWIG_ConvertPtr($input, (void **) &$1, $1_descriptor, 0))) {
    SWIG_exception_fail(SWIG_TypeError, "in method '$symname', expecting type Foo");
  }
}
```

If necessary, the descriptor for any type can be obtained using the `$descriptor()` macro in a typemap. For example:

```
%typemap(in) Foo * {
  if (!SWIG_IsOK(SWIG_ConvertPtr($input, (void **) &$1, $descriptor(Foo *), 0))) {
    SWIG_exception_fail(SWIG_TypeError, "in method '$symname', expecting type Foo");
  }
}
```

## 37.8 Turning a SWIG module into a Tcl Package.

SWIG generates all of the code necessary to create a Tcl extension package. To set the package version use the `-pkgversion` option. For example:

```
% swig -tcl -pkgversion 2.3 example.i
```

After building the SWIG generated module, you need to execute the `pkg_mkIndex` command inside tclsh. For example :

```
unix > tclsh
% pkg_mkIndex . example.so
% exit
```

This creates a file "`pkgIndex.tcl`" with information about the package. To use your package, you now need to move it to its own subdirectory which has the same name as the package. For example :

```
./example/
          pkgIndex.tcl           # The file created by pkg_mkIndex
          example.so             # The SWIG generated module
```

Finally, assuming that you're not entirely confused at this point, make sure that the example subdirectory is visible from the directories contained in either the `tcl_library` or `auto_path` variables. At this point you're ready to use the package as follows :

```
unix > tclsh
% package require example
% fact 4
24
%
```

If you're working with an example in the current directory and this doesn't work, do this instead :

```
unix > tclsh
% lappend auto_path .
% package require example
% fact 4
24
```

As a final note, most SWIG examples do not yet use the `package` commands. For simple extensions it may be easier just to use the `load` command instead.

## 37.9 Building new kinds of Tcl interfaces (in Tcl)

One of the most interesting aspects of Tcl and SWIG is that you can create entirely new kinds of Tcl interfaces in Tcl using the low-level SWIG accessor functions. For example, suppose you had a library of helper functions to access arrays :

```
/* File : array.i */
%module array

%inline %{
double *new_double(int size) {
  return (double *) malloc(size*sizeof(double));
}
void delete_double(double *a) {
  free(a);
}
double get_double(double *a, int index) {
  return a[index];
}
void set_double(double *a, int index, double val) {
  a[index] = val;
}
int *new_int(int size) {
  return (int *) malloc(size*sizeof(int));
}
void delete_int(int *a) {
  free(a);
```

```
}
int get_int(int *a, int index) {
  return a[index];
}
int set_int(int *a, int index, int val) {
  a[index] = val;
}
%}
```

While these could be called directly, we could also write a Tcl script like this :

```
proc Array {type size} {
    set ptr [new_$type $size]
    set code {
        set method [lindex $args 0]
        set parms [concat $ptr [lrange $args 1 end]]
        switch $method {
            get {return [eval "get_$type $parms"]}
            set {return [eval "set_$type $parms"]}
            delete {eval "delete_$type $ptr; rename $ptr {}"}
        }
    }
    # Create a procedure
    uplevel "proc $ptr args {set ptr $ptr; set type $type;$code}"
    return $ptr
}
```

Our script allows easy array access as follows :

```
set a [Array double 100]                    ;# Create a double [100]
for {set i 0} {$i < 100} {incr i 1} {       ;# Clear the array
        $a set $i 0.0
}
$a set 3 3.1455                             ;# Set an individual element
set b [$a get 10]                           ;# Retrieve an element

set ia [Array int 50]                       ;# Create an int[50]
for {set i 0} {$i < 50} {incr i 1} {        ;# Clear it
        $ia set $i 0
}
$ia set 3 7                                 ;# Set an individual element
set ib [$ia get 10]                         ;# Get an individual element

$a delete                                   ;# Destroy a
$ia delete                                  ;# Destroy ia
```

The cool thing about this approach is that it makes a common interface for two different types of arrays. In fact, if we were to add more C datatypes to our wrapper file, the Tcl code would work with those as well--without modification. If an unsupported datatype was requested, the Tcl code would simply return with an error so there is very little danger of blowing something up (although it is easily accomplished with an out of bounds array access).

### 37.9.1 Proxy classes

A similar approach can be applied to proxy classes (also known as shadow classes). The following example is provided by Erik Bierwagen and Paul Saxe. To use it, run SWIG with the `-noobject` option (which disables the builtin object oriented interface). When running Tcl, simply source this file. Now, objects can be used in a more or less natural fashion.

```
# swig_c++.tcl
# Provides a simple object oriented interface using
# SWIG's low level interface.
#

proc new {objectType handle_r args} {
    # Creates a new SWIG object of the given type,
    # returning a handle in the variable "handle_r".
    #
    # Also creates a procedure for the object and a trace on
    # the handle variable that deletes the object when the
    # handle variable is overwritten or unset
    upvar $handle_r handle
    #
    # Create the new object
    #
    eval set handle \[new_$objectType $args\]
    #
    # Set up the object procedure
    #
    proc $handle {cmd args} "eval ${objectType}_\$cmd $handle \$args"
    #
    # And the trace ...
    #
    uplevel trace variable $handle_r uw "{deleteObject $objectType $handle}"
    #
    # Return the handle so that 'new' can be used as an argument to a procedure
    #
    return $handle
}

proc deleteObject {objectType handle name element op} {
    #
    # Check that the object handle has a reasonable form
    #
    if {![regexp {_[0-9a-f]*_(.+)_p} $handle]} {
        error "deleteObject: not a valid object handle: $handle"
    }
    #
    # Remove the object procedure
```

```
    #
    catch {rename $handle {}}
    #
    # Delete the object
    #
    delete_$objectType $handle
}

proc delete {handle_r} {
    #
    # A synonym for unset that is more familiar to C++ programmers
    #
    uplevel unset $handle_r
}
```

To use this file, we simply source it and execute commands such as "new" and "delete" to manipulate objects. For example :

```
// list.i
%module List
%{
#include "list.h"
%}

// Very simple C++ example

class List {
public:
  List();  // Create a new list
  ~List(); // Destroy a list
  int  search(char *value);
  void insert(char *);  // Insert a new item into the list
  void remove(char *);  // Remove item from list
  char *get(int n);     // Get the nth item in the list
  int  length;          // The current length of the list
static void print(List *l);  // Print out the contents of the list
};
```

Now a Tcl script using the interface...

```
load ./list.so list       ; # Load the module
source swig_c++.tcl       ; # Source the object file

new List l
$l insert Dave
$l insert John
$l insert Guido
$l remove Dave
puts $l length_get

delete l
```

The cool thing about this example is that it works with any C++ object wrapped by SWIG and requires no special compilation. Proof that a short, but clever Tcl script can be combined with SWIG to do many interesting things.

### 37.10 Tcl/Tk Stubs

For background information about the Tcl Stubs feature, see https://www.tcl.tk/doc/howto/stubs.html.

As of SWIG 1.3.10, the generated C/C++ wrapper will use the Tcl Stubs feature if compiled with `-DUSE_TCL_STUBS`.

As of SWIG 1.3.40, the generated C/C++ wrapper will use the Tk Stubs feature if compiled with `-DUSE_TK_STUBS`.

By default SWIG sets the minimum Tcl version to support to the 8.4 as that's the minimum Tcl version we aim to support (since SWIG 4.1.0; before this SWIG set it to 8.1, which was the first Tcl version with the stubs mechanism). This minimum version is passed to `Tcl_InitStubs()` and `Tk_InitStubs()`. You can override with a specific version using `-DSWIG_TCL_STUBS_VERSION="8.5"` or set it to the Tcl version being compiled with using `-DSWIG_TCL_STUBS_VERSION=TCL_VERSION`.

---

# 38 SWIG and C as the target language


- Overview
  - Known C++ Shortcomings in Generated C API
- Preliminaries
  - Running SWIG
  - Command line options
  - Compiling a dynamic module
  - Using the generated module
- Basic C wrapping
  - Functions
  - Variables
- Basic C++ wrapping
  - Enums
  - Classes
- Backend Developer Documentation
- Typemaps
  - C Typemaps, a Code Generation Walkthrough
    - The Interface
    - The Wrapper
    - The Proxy
- Exception handling
- C++ Wrappers
  - Additional customization possibilities
  - Exception handling


This chapter describes SWIG's support for creating ISO C wrappers. This module has a special purpose and thus is different from most other modules.

**NOTE:** this module is still under development.

## 38.1 Overview

SWIG is normally used to provide access to C or C++ libraries from target languages such as scripting languages or languages running on a virtual machine. SWIG performs analysis of the input C/C++ library header files from which it generates further code. For most target languages this code consists of two layers; namely an intermediary C code layer and a set of language specific proxy classes and functions on top of the C code layer. We could also think of C as just another target language supported by SWIG. The aim then is to generate a pure ISO C interface to the input C or C++ library and hence the C target language module.

With wrapper interfaces generated by SWIG, it is easy to use the functionality of C++ libraries inside application code written in C. This module may also be useful to generate custom APIs for a library, to suit particular needs, e.g. to supply function calls with error checking or to implement a "design by contract".

Flattening C++ language constructs into a set of C-style functions obviously comes with many limitations and inconveniences, but this module is actually also capable of generating C++ wrappers defined completely inline using the C functions, thus wrapping the original C++ library API in another, similar C++ API. Contrary to the natural initial reaction, this is far from being completely pointless, as wrapping C++ API in this way avoids all problems due to C++ ABI issues, e.g. it is now possible to use the original C++ API using a different C++ compiler, or a different version of the same compiler, or even the same compiler, but with different compilation options affecting the ABI. The C++ wrapper API is not identical to the original one, but strives to be as close to it as possible.

### 38.1.1 Known C++ Shortcomings in Generated C API

- Enums with a context like class or namespace are broken
- Global variables are not supported
- Qualifiers are stripped
- Vararg functions are not supported.

## 38.2 Preliminaries

### 38.2.1 Running SWIG

Consider the following simple example. Suppose we have an interface file like:

```
/* File: example.i */
%module test
%{
#include "stuff.h"
%}
int fact(int n);
```

To build a C module (C as the target language), run SWIG using the `-c` option :

```
$ swig -c example.i
```

The above assumes C as the input language. If the input language is C++ add the `-c++` option:

```
$ swig -c++ -c example.i
```

Note that `-c` is the option specifying the **target** language and `-c++` controls what the **input** language is.

This will generate an `example_wrap.c` file or, in the latter case, `example_wrap.cxx` file, along with `example_wrap.h` (the same extension is used in both C and C++ cases for the last one). The names of the files are derived from the name of the input file by default, but can be changed using the `-o` and `-oh` options common to all language modules.

The `xxx_wrap.c` file contains the wrapper functions, which perform the main functionality of SWIG: each of the wrappers translates the input arguments from C to C++, makes calls to the original functions and marshals C++ output back to C data. The `xxx_wrap.h` header file contains the declarations of these functions as well as global variables.

### 38.2.2 Command line options

The following table list the additional command line options available for the C module. They can also be seen by using:

```
$ swig -c -help
```

| C specific options | |
|---|---|
| -namespace <nspace> | Generate wrappers with the prefix based on the provided namespace, e.g. if the option value is `outer::inner`, the prefix `outer_inner_` will be used. Notice that this is different from using SWIG `nspace` feature, as it applies the prefix to all the symbols, regardless of the namespace they were actually declared in. Notably, this provides a way to export instantiations of templates defined in the `std` namespace, such as `std::vector`, using a custom prefix rather than `std_` . |
| -nocxx | Don't generate C++ wrappers, even when the `-c++` option is used. See C++ Wrappers section for more details. |
| -noexcept | generate wrappers with no support for exception handling; see Exceptions chapter for more details |

### 38.2.3 Compiling a dynamic module

The next step is to build a dynamically loadable module, which we can link to our application. For example, to do this using the `gcc` compiler (Linux, MinGW, etc.):

```
$ swig -c example.i
$ gcc -fPIC -c example_wrap.c
$ gcc -shared example_wrap.o -o libexample.so
```

Or, for C++ input:

```
$ swig -c++ -c example.i
$ g++ -fPIC -c example_wrap.cxx
$ g++ -shared example_wrap.o -o libexample.so
```

Now the shared library module is ready to use. Note that the name of the generated module is important: is should be prefixed with `lib` on Unix, and have the specific extension, like `.dll` for Windows or `.so` for Unix systems.

### 38.2.4 Using the generated module

The simplest way to use the generated shared module is to link it to the application code during the compilation stage. The process is usually similar to this:

```
$ gcc runme.c -L. -lexample -o runme
```

This will compile the application code (`runme.c`) and link it against the generated shared module. Following the `-L` option is the path to the directory containing the shared module. The output executable is ready to use. The last thing to do is to supply to the operating system the information of location of our module. This is system dependant, for instance Unix systems look for shared modules in certain directories, like `/usr/lib`, and additionally we can set the environment variable `LD_LIBRARY_PATH` (Unix) or `PATH` (Windows) for other directories.

## 38.3 Basic C wrapping

Wrapping C functions and variables is obviously performed in a straightforward way. There is no need to perform type conversions, and all language constructs can be preserved in their original form. However, SWIG allows you to enhance the code with some additional elements, for instance using a `check` typemap or `%extend` directive.

It is also possible to output arbitrary additional code into the generated header by using the `%insert` directive with `cheader` section, e.g.

```
%insert("cheader") %{
#include "another.h"
%}
```

### 38.3.1 Functions

For each C function declared in the interface file a wrapper function with a prefix, required to make its name different from the original one, is created. The prefix for the global functions is `module_`, i.e. the name of the SWIG module followed by underscore, by default. If `-namespace` option is used, the prefix corresponding to the given fixed namespace is used instead. If `nspace` feature is used, the prefix corresponding to the namespace in which the function is defined is used -- note that, unlike with `-namespace` option, this prefix can be different for different functions. The wrapper function performs a call to the original function, and returns its result.

For example, for function declaration in the module `mymath`:

```
int gcd(int x, int y);
```

The output is simply:

```
int mymath_gcd(int arg1, int arg2) {
  int result;
  result = gcd(arg1,arg2);
  return result;
}
```

Now one might think, what's the use of creating such functions in C? The answer is, you can apply special rules to the generated code. Take for example constraint checking. You can write a "check" typemap in your interface file:

```
%typemap(check) int POSITIVE {
  if ($1 <= 0)
    fprintf(stderr, "Expected positive value for parameter $1 in $name.\n");
}

int gcd(int POSITIVE, int POSITIVE);
```

And now the generated result looks like:

```
int _wrap_gcd(int arg1, int arg2) {
  int result;
  {
    if (arg1 <= 0)
      fprintf(stderr, "Expected positive value for parameter arg1 in gcd.\n");
  }
  {
    if (arg2 <= 0)
      fprintf(stderr, "Expected positive value for parameter arg2 in gcd.\n");
  }
  result = gcd(arg1,arg2);
  return result;
}
```

This time calling `gcd` with negative value argument will trigger an error message. This can save you time writing all the constraint checking code by hand.

### 38.3.2 Variables

Wrapping variables comes also without any special issues. All global variables are directly accessible from application code. There is a difference in the semantics of `struct` definition in C and C++. When handling C `struct`, SWIG simply rewrites its declaration. In C++ `struct` is handled as class declaration.

You can still apply some of the SWIG features when handling structs, e.g. `%extend` directive. Suppose, you have a C struct declaration:

```
typedef struct {
  int x;
  char *str;
} my_struct;
```

You can redefine it to have an additional fields, like:

```
%extend my_struct {
  double d;
};
```

In application code:

```
struct my_struct ms;
ms.x = 123;
ms.d = 123.123;
```

## 38.4 Basic C++ wrapping

The main reason of having the C module in SWIG is to be able to access C++ from C. In this chapter we will take a look at the rules of wrapping elements of the C++ language.

By default, SWIG attempts to build a natural C interface to your C/C++ code.

| C++ Type | SWIG C Translation |
| --- | --- |
| Class `Example` | Empty structure `Example` |
| Public, mutable member variable `Foo Example::foo` | `Example_foo_get(Example *e);`<br>`Example_foo_set(Example *e, Foo *f);` |
| Public, immutable member variable `Foo Example::bar` | `Example_foo_get(Example *e);` |

This section briefly covers the essential aspects of this wrapping.

### 38.4.1 Enums

C enums and unscoped C++ enums are simply copied to the generated code and both the enum itself and its elements keep the same name as in the original code unless `-namespace` option is used or `nspace` feature is enabled, in which case the prefix corresponding to the specified namespace is used.

For scoped C++11 enums, the enum name itself is used as an additional prefix.

### 38.4.2 Classes

Consider the following example. We have a C++ class, and want to use it from C code.

```
class Circle {
public:
  double radius;

  Circle(double r) : radius(r) { };
  double area(void);
};
```

What we need to do is to create an object of the class, manipulate it, and finally, destroy it. SWIG generates C functions for this purpose each time a class declaration is encountered in the interface file.

The first two generated functions are used to create and destroy instances of class `Circle`. Such instances are represented on the C side as pointers to special structs, called `SwigObj`. They are all "renamed" (via typedef) to the original class names, so that you can use the object instances on the C side using pointers like:

```
Circle *circle;
```

The generated functions make calls to class' constructors and destructors, respectively. They also do all the necessary things required by the SWIG object management system in C.

```
Circle * Circle_new(double r);
void Circle_delete(Circle * self);
```

The class `Circle` has a public variable called `radius` . SWIG generates a pair of setters and getters for each such variable:

```
void Circle_radius_set(Circle * self, double radius);
double Circle_radius_get(Circle * self);
```

For each public method, an appropriate function is generated:

```
double Circle_area(Circle * self);
```

You can see that in order to use the generated object we need to provide a pointer to the object instance (struct `Circle` in this case) as the first function argument. In fact, this struct is basically wrapping pointer to the "real" C++ object.

Our application code could look like this:

```
  Circle *c = Circle_new(1.5);
  printf("radius: %f\narea: %f\n", Circle_radius_get(c), Circle_area(c));
  Circle_delete(c);
```

After running this we'll get:

```
radius: 1.500000
area: 7.068583
```

## 38.5 Backend Developer Documentation

## 38.6 Typemaps

| Typemap | Used for |
| --- | --- |

| | |
|---|---|
| ctype | Provides types used for the C API and<br>Typecasts wrapper functions return values in proxy functions<br>`MyClass *MyClass_new(void) {`<br>`  return (MyClass *)MyClass_new();`<br>`}` |
| in | Mapping of wrapper functions parameters to local C++ variables<br>`SwigObj* MyClass_do(SwigObj *carg1) {`<br>`  SomeCPPClass *arg1 = 0;`<br>`  if (carg1)`<br>`  arg1 = (SomeCPPClass*)carg1->obj`<br>`  else`<br>`  arg1 = 0;`<br>`}` |
| out | Assigns wrapped function's return value to a dedicated return variable, packaging it into SwigObj if necessary |
| cppouttype | Type of the result variable used for the return value if the wrapped function is a C++ function |
| cxxintype | Defines the type for the parameters of C++ wrapper functions corresponding to this type. By default is the same as `ctype`, but may sometimes be different to make the functions more convenient to use. For example, `ctype` for `std::string` is `const char*`, but `cxxintype` typemap for it is `std::string const&`, i.e. even though the C++ string passed as a raw pointer via C API, the C++ wrapper still accepts a C++ string. If this typemap is defined, `cxxin` should normally be defined as well. If it is not defined, `ctype` is used. |
| cxxouttype | Similar to `cxxintype`, but is used for the function return values and together with `cxxout` typemap. Also defaults to `ctype` if not defined. |
| cxxin | Defines how to transform `cxxintype` value to `ctype` |
| cxxout | Defines how to transform `ctype` value returned by a function to `cxxouttype` |
| cxxcode | May contain arbitrary code that will be injected in the declaration of the C++ wrapper class corresponding to the given type. Ignored for non-class types. The special variable `$cxxclassname` is replaced with the name of the class inside this typemap expansion and `$cclassptrname` is replaced with the name of the pointer type used to represent the class in C wrapper functions. |

### 38.6.1 C Typemaps, a Code Generation Walkthrough

To get a better idea of which typemap is used for which generated code, have a look at the following 'walk through'.

Let's assume we have the following C++ interface file, we'd like to generate code for:

#### 38.6.1.1 The Interface

```
%module example

%inline
%{
  class SomeClass{};
  template <typename T> class SomeTemplateClass{};
  SomeClass someFunction(SomeTemplateClass<int> &someParameter, int simpleInt);
%}

%template (SomeIntTemplateClass) SomeTemplateClass<int>;
```

What we would like to generate as a C interface of this function would be something like this:

```
// wrapper header file
typedef struct SwigObj_SomeClass SomeClass;

SomeClass * SomeClass_new();

void SomeClass_delete(SomeClass * carg1);

SomeClass* someFunction(SomeIntTemplateClass* carg1, int carg2);


typedef struct SwigObj_SomeIntTemplateClass SomeIntTemplateClass;

SomeIntTemplateClass * SomeIntTemplateClass_new();

void SomeIntTemplateClass_delete(SomeIntTemplateClass * carg1);
```

#### 38.6.1.2 The Wrapper

We'll examine the generation of the wrapper function first.

```
SWIGEXPORTC SwigObj * module_someFunction(SwigObj * carg1, int carg2) {
  SomeClass * cppresult;
  SomeTemplateClass< int > *arg1 = 0 ;
  int arg2 ;
  SwigObj * result;

  {
    if (carg1)
    arg1 = (SomeTemplateClass< int > *) carg1->obj;
    else
    arg1 = (SomeTemplateClass< int > *) 0;
  }
  arg2 = (int) carg2;
  {
    const SomeClass &_result_ref =  someFunction(*arg1,arg2);cppresult = (SomeClass*) &_result_ref;
  }
  {
    result = SWIG_create_object(cppresult, SWIG_STR(SomeClass));
  }
  return result;
}
```

It might be helpful to think of the way function calls are generated as a composition of building blocks. A typical wrapper will be composited with these [optional] blocks:

1. Prototype
2. C return value variable
3. Local variables equal to the called C++ function's parameters
4. [C++ return value variable]
5. Assignment (extraction) of wrapper parameters to local parameter copies
6. [Contract (e.g. constraints) checking]
7. C++ function call
8. [Exception handling]
9. [Assignment to C++ return value]
10. Assignment to C return value

Let's go through it step by step and start with the wrapper prototype

```
ctype                       ctype           ctype
---------                   ---------       ---
SwigObj * module_someFunction(SwigObj * carg1, int carg2);
```

As first unit of the wrapper code, a variable to hold the return value of the function is emitted to the wrapper's body

```
ctype
---------
SwigObj * result;
```

Now for each of the C++ function's arguments, a local variable with the very same type is emitted to the wrapper's body.

```
SomeTemplateClass< int > *arg1 = 0 ;
int arg2 ;
```

If it's a C++ function that is wrapped (in this case it is), another variable is emitted for the 'original' return value of the C++ function. At this point, we simply 'inject' behavior if it's a C++ function that is wrapped (in this case it obviously is).

```
cppouttype
-----------
SomeClass * cppresult;
```

Next, the values of the input parameters are assigned to the local variables using the 'in' typemap.

```
{
  if (carg1)
  arg1 = (SomeTemplateClass< int > *) carg1->obj;
  else
  arg1 = (SomeTemplateClass< int > *) 0;
}
arg2 = (int) carg2;
```

A reasonable question would be: "Why aren't the parameters assigned in the declaration of their local counterparts?" As seen above, for complex types pointers have to be verified before extracting and casting the actual data pointer from the provided SwigObj pointer. This could easily become messy if it was done in the same line with the local variable declaration.

At this point we are ready to call the C++ function with our parameters.

```
{
  const SomeClass &_result_ref =  someFunction(*arg1,arg2);cppresult = (SomeClass*) &_result_ref;
}
```

Subsequently, the return value is assigned to the dedicated return value variable using the 'out' typemap

```
{
  result = SWIG_create_object(cppresult, SWIG_STR(SomeClass));
}
```

Finally, the return value variable is returned.

```
return result;
```

Note that typemaps may use `$null` special variable which will be replaced with either `0` or nothing, depending on whether the function has a non-void return value or not.

#### 38.6.1.3 The Proxy

Compared to the wrapper code generation, the header code is very simple. Basically it contains just the declarations corresponding to the definitions above.

```
// wrapper header file
typedef struct SwigObj_SomeClass SomeClass;

SomeClass * SomeClass_new();

void SomeClass_delete(SomeClass * carg1);

SomeClass* someFunction(SomeIntTemplateClass* carg1, int carg2);


typedef struct SwigObj_SomeIntTemplateClass SomeIntTemplateClass;

SomeIntTemplateClass * SomeIntTemplateClass_new();

void SomeIntTemplateClass_delete(SomeIntTemplateClass * carg1);
```

## 38.7 Exception handling

Any call to a C++ function may throw an exception, which cannot be caught by C code. Instead, the special `SWIG_CException_get_pending()` function must be called to check for this. If it returns a non-null pointer, `SWIG_CException_msg_get()` can be called to retrieve the error message associated with the exception. Finally, `SWIG_CException_reset_pending()` must be called to free the exception object and reset the current pending exception. Note that exception handling is much simpler when using C++, rather than C, wrappers, see sections 36.6.2.

## 38.8 C++ Wrappers

When `-c++` command line option is used (and `-nocxx` one is not), the header file generated by SWIG will also contain the declarations of C++ wrapper functions and classes mirroring the original API. All C++ wrappers are fully inline, i.e. don't need to be compiled separately, and are always defined inside the namespace (or nested namespaces) specified by `-namespace` command-line option or the namespace with the same name as the SWIG module name if this option is not specified.

C++ wrappers try to provide a similar API to the original C++ API being wrapped, notably any class `Foo` in the original API appears as a class with the same name in the wrappers namespace, and has the same, or similar, public methods. A class `Bar` deriving from `Foo` also derives from it in the wrappers and so on. There are some differences with the original API, however. Some of them are due to fundamental limitations of the approach used, e.g.:

- Only template instantiations are present in the wrappers, not the templates themselves.

Other ones are due to things that could be supported but haven't been implemented yet:

- Only single, not multiple, inheritance is currently supported.
- Only enums using `int` (or smaller type) as underlying type are supported.

### 38.8.1 Additional customization possibilities

Generated C++ code can be customized by inserting custom code in the following sections:

- `cxxheader` for including additional headers and other declarations in the global scope.
- `cxxcode` for additional code to appear after the declarations of all wrapper classes, inside the module-specific namespace.

The following features are taken into account when generating C++ wrappers:

- `cxxignore` May be set to skip generation of C++ wrappers for the given function or class, while still generating C wrappers for them.

### 38.8.2 Exception handling

Exception handling in C++ is more natural, as the exceptions are re-thrown when using C++ wrappers and so can be caught, as objects of the special `SWIG_CException` type, using the usual `try/catch` statement. The objects of `SWIG_CException` class have `code()` and `msg()` methods, with the latter returning the error message associated with the exception.

If necessary, a custom exception type may be used instead of `SWIG_CException`. To do this, a custom implementation of`swig_check()` function, called to check for the pending exception and throw the corresponding C++ exception if necessary, must be provided and `SWIG_swig_check_DEFINED` preprocessor symbol must be defined to prevent the default implementation of this function from being compiled:

```
%insert(cxxheader) %{
#ifndef SWIG_swig_check_DEFINED
#define SWIG_swig_check_DEFINED 1

#include <stdexcept>

class Exception : public std::runtime_error {
public:
    explicit Exception(const char* msg) : std::runtime_error{msg} {}
};

inline void swig_check() {
  if (auto* swig_ex = SWIG_CException_get_pending()) {
    Exception const e{SWIG_CException_msg_get(swig_ex)};
    SWIG_CException_reset_pending();
    throw e;
  }
}

template <typename T> T swig_check(T x) {
  swig_check();
  return x;
}

#endif // SWIG_swig_check_DEFINED
%}
```

# 39 SWIG and MzScheme/Racket


- Creating native structures
- Simple example
- External documentation


This section contains information on SWIG's support of Racket, formally known as MzScheme. SWIG works with Racket versions < 8 (Racket 8 switched to be based on a different scheme interpreter and SWIG hasn't been updated for this change).

## 39.1 Creating native structures

Example interface file:

```
/* define a macro for the struct creation */
%define handle_ptr(TYPE, NAME)
%typemap(argout) TYPE *NAME{
  Scheme_Object *o = SWIG_NewStructFromPtr($1, $*1_mangle);
  SWIG_APPEND_VALUE(o);
}

%typemap(in, numinputs=0) TYPE *NAME (TYPE temp) {
  $1 = &temp;
}
```

```
%enddef

/* setup the typemaps for the pointer to an output parameter cntrs */
handle_ptr(struct diag_cntrs, cntrs);
```

Then in scheme, you can use regular struct access procedures like

```
       ; suppose a function created a struct foo as
       ; (define foo (make-diag-cntrs (#x1 #x2 #x3) (make-inspector))
       ; Then you can do
       (format "0x~x" (diag-cntrs-field1 foo))
       (format "0x~x" (diag-cntrs-field2 foo))
       ;etc...
```

## 39.2 Simple example

A few examples are available in the Examples/mzscheme directory. The code and log of a session using SWIG below should help getting started.

C header file:

```
// example.h
int fact(int n);
```

C source code:

```
// File: example.c
#include "example.h"

int fact(int n) {
  if (n < 0) { /* This should probably return an error, but this is simpler */
    return 0;
  }
  if (n == 0) {
    return 1;
  }
  else {
    /* testing for overflow would be a good idea here */
    return n * fact(n-1);
  }
}
```

SWIG interface file:

```
/* File: example.i */
%module example

%{
#include "example.h"
%}

int fact(int n);
```

The session below using the above files is on an OS X machine, but the points to be made are more general. On OS X, libtool is the tool which creates libraries, which are named .dylib, rather than .so on other unixes, or .dll on Windows.

```
% swig -mzscheme -declaremodule example.i
% gcc -c -m32 -o example.o example.c # force 32-bit object file (mzscheme is 32-bit only)
% libtool -dynamic -o libexample.dylib example.o # make it into a library
% ls # what've we got so far?
example.c example.o
example.h example_wrap.c
example.i libexample.dylib*
% mzc --cgc --cc example_wrap.c # compile the wrapping code
% LDFLAGS="-L. -lexample" mzc --ld example_wrap.dylib example_wrap.o # ...and link it
% mzscheme -e '(path->string (build-path "compiled" "native" (system-library-subpath)))'
"compiled/native/i386-macosx/3m"
% mkdir -p compiled/native/i386-macosx/3m # move the extension library to a magic place
% mv example_wrap.dylib compiled/native/i386-macosx/3m/example_ss.dylib
% mzscheme
Welcome to MzScheme v4.2.4 [3m], Copyright (c) 2004-2010 PLT Scheme Inc.
> (require "example.ss")
> (fact 5)
120
> ^D
% echo 'It works!'
```

Some points of interest:

- This is on a 64-bit machine, so we have to include the -m32 option when building the object file
- If you want to declare a scheme module (and you probably do), it's important that you include the -declaremodule option to swig (if you miss this out, it'll appear to work, but fail later).
- Use mzc to compile and then link the wrapped code. You'll probably need to adjust the link flags to refer to the library you're wrapping (you can either do this with an LDFLAGS declaration, as here, or with multiple ++ldf options to mzc).
- Create the directory with path (build-path "compiled" "native" (system-library-subpath)) and move the freshly-generated .dylib to there, changing its name to module-name_ss.dylib. After that, you can REQUIRE the new module with (require "module-name.ss").
- The above requests mzc to create an extension using the CGC garbage-collector. The alternative -- the 3m collector -- has generally better performance, but work is still required for SWIG to emit code which is compatible with it.

## 39.3 External documentation

See the C API for more description of using the mechanism for adding extensions. The main documentation is here.

Tip: mzc's --vv option is very useful for debugging the inevitable library problems you'll encounter.

# 40 SWIG and OCaml


- Preliminaries
  - Running SWIG
  - Compiling the code
  - The camlp4 module
  - Using your module
  - Compilation problems and compiling with C++
- The low-level Ocaml/C interface
  - The generated module
  - Enums
    - Enum typing in Ocaml
  - Arrays
    - Simple types of bounded arrays
    - Complex and unbounded arrays
    - Using an object
    - Example typemap for a function taking float * and int
  - C++ Classes
    - STL vector and string Example
    - C++ Class Example
    - Compiling the example
    - Sample Session
  - Director Classes
    - Director Introduction
    - Overriding Methods in Ocaml
    - Director Usage Example
    - Creating director objects
    - Typemaps for directors, directorin, directorout, directorargout
    - directorin typemap
    - directorout typemap
    - directorargout typemap
  - Exceptions
- Documentation Features
  - Module docstring


This chapter describes SWIG's support of Ocaml.

Ocaml was the second compiled, typed language to be added to SWIG. Ocaml has a sophisticated type system, compile-time checking which eliminates several classes of common programming errors, and good native performance. While all of this is wonderful, there are well-written C and C++ libraries that Ocaml users will want to take advantage of as part of their arsenal (such as SSL and gdbm), as well as their own mature C and C++ code. SWIG allows this code to be used in a natural, type-safe way with Ocaml, by providing the necessary, but repetitive glue code which creates and uses Ocaml values to communicate with C and C++ code. In addition, SWIG also produces the needed Ocaml source that binds variants, functions, classes, etc.

If you're not familiar with the Objective Caml language, you can visit The Ocaml Website.

## 40.1 Preliminaries

SWIG is known to be compatible with OCaml 4.13.1 and above - older versions are not regularly tested. Given the choice, you should use the latest stable release. The SWIG Ocaml module has been tested on Linux (x86, PPC, Sparc) and Cygwin on Windows. The best way to determine whether your system will work is to compile the examples and test-suite which come with SWIG. You can do this by running `make check` from the SWIG root directory after installing SWIG. The Ocaml module has been tested using the system's dynamic linking (the usual -lxxx against libxxx.so, as well as with Gerd Stolpmann's Dl package. The ocaml_dynamic and ocaml_dynamic_cpp targets in the file Examples/Makefile illustrate how to compile and link SWIG modules that will be loaded dynamically. This has only been tested on Linux so far.

### 40.1.1 Running SWIG

The basics of getting a SWIG Ocaml module up and running can be seen from one of SWIG's example Makefiles, but is also described here. To build an Ocaml module, run SWIG using the `-ocaml` option.

```
%swig -ocaml example.i
```

This will produce 3 files. The file `example_wrap.c` contains all of the C code needed to build an Ocaml module. To build the module, you will compile the file `example_wrap.c` with `ocamlc` or `ocamlopt` to create the needed .o file. You will need to compile the resulting .ml and .mli files as well, and do the final link with -custom (not needed for native link).

### 40.1.2 Compiling the code

The OCaml SWIG module now requires you to compile a module (`Swig` ) separately. In addition to aggregating common SWIG functionality, the Swig module contains the data structure that represents C/C++ values. This allows easier data sharing between modules if two or more are combined, because the type of each SWIG'ed module's c_obj is derived from Swig.c_obj_t. This also allows SWIG to acquire new conversions painlessly, as well as giving the user more freedom with respect to custom typing. Use `ocamlc` or `ocamlopt` to compile your SWIG interface like:

```
% swig -ocaml -co swig.mli ; swig -ocaml -co swig.ml
% ocamlc -c swig.mli ; ocamlc -c swig.ml
% ocamlc -c -ccopt "-I/usr/include/foo" example_wrap.c
% ocamlc -c example.mli
% ocamlc -c example.ml
```

`ocamlc` is aware of .c files and knows how to handle them. Unfortunately, it does not know about .cxx, .cc, or .cpp files, so when SWIG is invoked in C++ mode, you must:

```
% cp example_wrap.cxx example_wrap.cxx.c
% ocamlc -c ... -ccopt -xc++ example_wrap.cxx.c
% ...
```

### 40.1.3 The camlp4 module

The camlp4 module (swigp4.ml -> swigp4.cmo) contains a simple rewriter which makes C++ code blend more seamlessly with objective caml code. Its use is optional, but encouraged. The source file is included in the Lib/ocaml directory of the SWIG source distribution. You can checkout this file with `"swig -ocaml -co swigp4.ml"`. You should compile the file with `"ocamlc -I `camlp4 -where` -pp 'camlp4o pa_extend.cmo q_MLast.cmo' -c swigp4.ml"`

The basic principle of the module is to recognize certain non-caml expressions and convert them for use with C++ code as interfaced by SWIG. The camlp4 module is written to work with

generated SWIG interfaces, and probably isn't great to use with anything else.

Here are the main rewriting rules:

| Input | Rewritten to |
|---|---|
| f'( ... ) as in<br>atoi'("0") or<br>_exit'(0) | f(C_list [ ... ]) as in<br>atoi (C_list [ C_string "0" ]) or<br>_exit (C_list [ C_int 0 ]) |
| object -> method ( ... ) | (invoke object) "method" (C_list [ ... ]) |
| object *'binop* argument as in<br>a '+= b | (invoke object) "+=" argument as in<br>(invoke a) "+=" b |
| **Note that because camlp4 always recognizes << and >>, they are replaced by lsl and lsr in operator names.** | |
| *'unop* object as in<br>'! a | (invoke a) "!" C_void |
| **Smart pointer access like this**<br>object '-> method ( args ) | (invoke (invoke object "->" C_void)) |
| **Invoke syntax**<br>object . '( ... ) | (invoke object) "()" (C_list [ ... ]) |
| **Array syntax**<br>object '[ 10 ] | (invoke object) "[]" (C_int 10) |
| **Assignment syntax**<br>let a = '10 and b = '"foo" and c = '1.0 and d = 'true | let a = C_int 10 and b = C_string "foo" and c = C_double 1.0 and d = C_bool true |
| **Cast syntax**<br>let a = _atoi '("2") as int<br>let b = (getenv "PATH") to string<br>This works for int, string, float, bool | let a = get_int (_atoi (C_string "2"))<br>let b = C_string (getenv "PATH") |

### 40.1.4 Using your module

You can test-drive your module by building a toplevel ocaml interpreter. Consult the ocaml manual for details.

When linking any ocaml bytecode with your module, use the -custom option to build your functions into the primitive list. This option is not needed when you build native code.

### 40.1.5 Compilation problems and compiling with C++

Ocaml's C extension API unfortunately defines a `value` typedef - this overly-generic name can collide with the C or C++ API being wrapped if it defines a variable, function, member or type also called `value`. Prior to SWIG 4.2.1, SWIG attempted to work around this but the hack used itself caused problems so we've removed it. The problem is not one SWIG created, nor one SWIG can really solve.

As mentioned earlier, C++ files need special handling to be compiled with `ocamlc`. Other than that, C code that uses `class` as a non-keyword, and C code that is too liberal with pointer types may not compile under the C++ compiler. Most code meant to be compiled as C++ will not have problems.

## 40.2 The low-level Ocaml/C interface

In order to provide access to overloaded functions, and provide sensible outputs from them, all C entities are represented as members of the c_obj type:

In the code as seen by the typemap writer, there is a value, swig_result, that always contains the current return data. It is a list, and must be appended with the caml_list_append function, or with functions and macros provided by objective caml.

```
type c_obj =
    C_void
  | C_bool of bool
  | C_char of char
  | C_uchar of char
  | C_short of int
  | C_ushort of int
  | C_int of int
  | C_uint of int32
  | C_int32 of int32
  | C_int64 of int64
  | C_float of float
  | C_double of float
  | C_ptr of int64 * int64
  | C_array of c_obj array
  | C_list of c_obj list
  | C_obj of (string -> c_obj -> c_obj)
  | C_string of string
  | C_enum of c_enum_t
```

A few functions exist which generate and return these:

- caml_ptr_val receives a c_obj and returns a void *. This should be used for all pointer purposes.
- caml_long_val receives a c_obj and returns a long. This should be used for most integral purposes.
- caml_val_ptr receives a void * and returns a c_obj.
- caml_val_bool receives a C int and returns a c_obj representing its bool value.
- caml_val_(u)?(char|short|int|long|float|double) receives an appropriate C value and returns a c_obj representing it.
- caml_val_string receives a char * and returns a string value.
- caml_val_string_len receives a char * and a length and returns a string value.
- caml_val_obj receives a void * and an object type and returns a C_obj, which contains a closure giving method access.

Because of this style, a typemap can return any kind of value it wants from a function. This enables out typemaps and inout typemaps to work well. The one thing to remember about outputting values is that you must append them to the return list with swig_result = caml_list_append(swig_result, v).

This function will return a new list that has your element appended. Upon return to caml space, the fnhelper function beautifies the result. A list containing a single item degrades to only that item (i.e. [ C_int 3 ] -> C_int 3), and a list containing more than one item is wrapped in C_list (i.e. [ C_char 'a' ; C_char 'b' ] -> C_list [ C_char 'a' ; C_char 'b' ]). This is in order to make return values easier to handle when functions have only one return value, such as constructors, and operators. In addition, string, pointer, and object values are interchangeable with respect to caml_ptr_val, so you can allocate memory as caml strings and still use the resulting pointers for C purposes, even using them to construct simple objects on. Note, though, that foreign C++ code does not respect the garbage collector, although the SWIG interface does.

The wild card type that you can use in lots of different ways is C_obj. It allows you to wrap any type of thing you like as an object using the same mechanism that the ocaml module does. When evaluated in caml_ptr_val, the returned value is the result of a call to the object's "&" operator, taken as a pointer.

You should only construct values using objective caml, or using the functions caml_val_* functions provided as static functions to a SWIG ocaml module, as well as the caml_list_* functions. These functions provide everything a typemap needs to produce values. In addition, value items pass through directly, but you must make your own type signature for a function that uses value in this way.

### 40.2.1 The generated module

The SWIG `%module` directive specifies the name of the Ocaml module to be generated. If you specified `%module example', then your Ocaml code will be accessible in the module Example. The module name is always capitalized as is the ocaml convention. Note that you must not use any Ocaml keyword to name your module. Remember that the keywords are not the same as the C++ ones.

You can introduce extra code into the output wherever you like with SWIG. These are the places you can introduce code:

| | |
|---|---|
| "header" | This code is inserted near the beginning of the C wrapper file, before any function definitions. |
| "wrapper" | This code is inserted in the function definition section. |
| "runtime" | This code is inserted near the end of the C wrapper file. |
| "mli" | This code is inserted into the caml interface file. Special signatures should be inserted here. |
| "ml" | This code is inserted in the caml code defining the interface to your C code. Special caml code, as well as any initialization which should run when the module is loaded may be inserted here. |
| "classtemplate" | The "classtemplate" place is special because it describes the output SWIG will generate for class definitions. |

### 40.2.2 Enums

SWIG will wrap enumerations as polymorphic variants in the output Ocaml code, as above in C_enum. In order to support all C++-style uses of enums, the function int_to_enum and enum_to_int are provided for ocaml code to produce and consume these values as integers. Other than that, correct uses of enums will not have a problem. Since enum labels may overlap between enums, the enum_to_int and int_to_enum functions take an enum type label as an argument. Example:

```
%module enum_test
%{
enum c_enum_type { a = 1, b, c = 4, d = 8 };
%}
enum c_enum_type { a = 1, b, c = 4, d = 8 };
```

The output mli contains:

```
type c_enum_type = [
   `unknown
| `c_enum_type
]
type c_enum_tag = [
  `int of int
| `a
| `b
| `c
| `d
]
val int_to_enum c_enum_type -> int -> c_obj
val enum_to_int c_enum_type -> c_obj -> c_obj
```

So it's possible to do this:

```
bash-2.05a$ ocamlmktop -custom enum_test_wrap.o enum_test.cmo -o enum_test_top
bash-2.05a$ ./enum_test_top
        Objective Caml version 3.04

# open Enum_test ;;
# let x = C_enum `a ;;
val x : Enum_test.c_obj = C_enum `a
# enum_to_int `c_enum_type x ;;
- : Enum_test.c_obj = C_int 1
# int_to_enum `c_enum_type 4 ;;
- : Enum_test.c_obj = C_enum `c
```

#### 40.2.2.1 Enum typing in Ocaml

The ocaml SWIG module now has support for loading and using multiple SWIG modules at the same time. This enhances modularity, but presents problems when used with a language which assumes that each module's types are complete at compile time. In order to achieve total soundness enum types are now isolated per-module. The type issue matters when values are shared between functions imported from different modules. You must convert values to master values using the swig_val function before sharing them with another module.

### 40.2.3 Arrays

#### 40.2.3.1 Simple types of bounded arrays

SWIG has support for array types, but you generally will need to provide a typemap to handle them. You can currently roll your own, or expand some of the macros provided (but not included by default) with the SWIG distribution.

By including "carray.i", you will get access to some macros that help you create typemaps for array types fairly easily.

`%make_simple_array_typemap` is the easiest way to get access to arrays of simple types with known bounds in your code, but this only works for arrays whose bounds are completely specified.

#### 40.2.3.2 Complex and unbounded arrays

Unfortunately, unbounded arrays and pointers can't be handled in a completely general way by SWIG, because the end-condition of such an array can't be predicted. In some cases, it will be by consent (e.g. an array of four or more chars), sometimes by explicit length (char *buffer, int len), and sometimes by sentinel value (0, -1, etc.). SWIG can't predict which of these methods will be used in the array, so you have to specify it for yourself in the form of a typemap.

#### 40.2.3.3 Using an object

It's possible to use C++ to your advantage by creating a simple object that provides access to your array. This may be more desirable in some cases, since the object can provide bounds checking, etc., that prevents crashes.

Consider writing an object when the ending condition of your array is complex, such as using a required sentinel, etc.

#### 40.2.3.4 Example typemap for a function taking float * and int

This is a simple example `in` typemap for an array of float, where the length of the array is specified as an extra parameter. Other such typemaps will work similarly. In the example, the function printfloats is called with a float array, and specified length. The actual length reported in the len argument is the length of the array passed from ocaml, making passing an array into this type of function convenient.

**tarray.i**

```
%module tarray
%{
#include <stdio.h>

void printfloats( float *tab, int len ) {
  int i;

  for( i = 0; i < len; i++ ) {
    printf( "%f ", tab[i] );
  }

  printf( "\n" );
}
%}

%typemap(in) (float *tab, int len) {
  int i;
  /* $*1_type */
  $2 = caml_array_len($input);
  $1 = ($*1_type *)malloc( $2 * sizeof( float ) );
  for( i = 0; i < $2; i++ ) {
    $1[i] = caml_double_val(caml_array_nth($input, i));
  }
}

void printfloats( float *tab, int len );
```

**Sample Run**

```
# open Tarray ;;
# _printfloats (C_array [| C_double 1.0 ; C_double 3.0 ; C_double 5.6666 |]) ;;
1.000000 3.000000 5.666600
- : Tarray.c_obj = C_void
```

### 40.2.4 C++ Classes

C++ classes, along with structs and unions are represented by C_obj (string -> c_obj -> c_obj) wrapped closures. These objects contain a method list, and a type, which allow them to be used like C++ objects. When passed into typemaps that use pointers, they degrade to pointers through their "&" method. Every method an object has is represented as a string in the object's method table, and each method table exists in memory only once. In addition to any other operators an object might have, certain builtin ones are provided by SWIG: (all of these take no arguments (C_void))

| | |
|---|---|
| "~" | Delete this object |
| "&" | Return an ordinary C_ptr value representing this object's address |
| "sizeof" | If enabled with ("sizeof"="1") on the module node, return the object's size in char. |
| ":methods" | Returns a list of strings containing the names of the methods this object contains |
| ":classof" | Returns the name of the class this object belongs to. |
| ":parents" | Returns a list of all direct parent classes which have been wrapped by SWIG. |
| "::[parent-class]" | Returns a view of the object as the indicated parent class. This is mainly used internally by the SWIG module, but may be useful to client programs. |
| "[member-variable]" | Each member variable is wrapped as a method with an optional parameter. Called with one argument, the member variable is set to the value of the argument. With zero arguments, the value is returned. |

Note that this string belongs to the wrapper object, and not the underlying pointer, so using create_[x]_from_ptr alters the returned value for the same object.

#### 40.2.4.1 STL vector and string Example

Standard typemaps are now provided for STL vector and string. More are in the works. STL strings are passed just like normal strings, and returned as strings. STL string references don't mutate the original string, (which might be surprising), because Ocaml strings are mutable but have fixed length. Instead, use multiple returns, as in the argout_ref example.

**example.i**

```
%module example
%{
#include "example.h"
%}

%include <stl.i>

namespace std {
  %template(StringVector) std::vector < string >;
};

%include "example.h"
```

*This example is in Examples/ocaml/stl*

Since there's a makefile in that directory, the example is easy to build.

Here's a sample transcript of an interactive session using a string vector after making a toplevel (make toplevel). This example uses the camlp4 module.

```
bash-2.05a$ ./runme_top
        Objective Caml version 3.06

        Camlp4 Parsing version 3.06

# open Swig ;;
# open Example ;;
# let x = new_StringVector '() ;;
val x : Example.c_obj = C_obj <fun>
# x -> ":methods" () ;;
- : Example.c_obj =
C_list
 [C_string "nop"; C_string "size"; C_string "empty"; C_string "clear";
  C_string "push_back"; C_string "[]"; C_string "="; C_string "set";
  C_string "~"; C_string "&"; C_string ":parents"; C_string ":classof";
```

```
  C_string ":methods"]
# x -> push_back ("foo") ;;
- : Example.c_obj = C_void
# x -> push_back ("bar") ;;
- : Example.c_obj = C_void
# x -> push_back ("baz") ;;
- : Example.c_obj = C_void
# x '[1] ;;
- : Example.c_obj = C_string "bar"
# x -> set (1, "spam") ;;
- : Example.c_obj = C_void
# x '[1] ;;
- : Example.c_obj = C_string "spam"
# for i = 0 to (x -> size() as int) - 1 do
    print_endline ((x '[i to int]) as string)
  done ;;
foo
bar
baz
- : unit = ()
#
```

#### 40.2.4.2 C++ Class Example

Here's a simple example using Trolltech's Qt Library:

**qt.i**

```
%module qt
%{
#include <qapplication.h>
#include <qpushbutton.h>
%}
class QApplication {
public:
  QApplication( int argc, char **argv );
  void exec();
};

class QPushButton {
public:
  QPushButton( char *str, QWidget *w );
  void resize( int x, int y );
  void show();
};
```

#### 40.2.4.3 Compiling the example

```
$ QTPATH=/your/qt/path
$ for file in swig.mli swig.ml swigp4.ml ; do swig -ocaml -co $file ; done
$ ocamlc -c swig.mli ; ocamlc -c swig.ml
$ ocamlc -I `camlp4 -where` -pp "camlp4o pa_extend.cmo q_MLast.cmo" -c swigp4.ml
$ swig -ocaml -c++ -o qt_wrap.c qt.i
$ ocamlc -c -ccopt -xc++ -ccopt -g -g -ccopt -I$QTPATH/include qt_wrap.c
$ ocamlc -c qt.mli
$ ocamlc -c qt.ml
$ ocamlmktop -custom swig.cmo -I `camlp4 -where` \
  camlp4o.cma swigp4.cmo qt_wrap.o qt.cmo -o qt_top -cclib \
  -L$QTPATH/lib -cclib -lqt
```

#### 40.2.4.4 Sample Session

```
bash-2.05a$ ./qt_top
        Objective Caml version 3.06

        Camlp4 Parsing version 3.06

# open Swig ;;
# open Qt ;;
# let a = new_QApplication '(0, 0) ;;
val a : Qt.c_obj = C_obj <fun>
# let hello = new_QPushButton '("hi", 0) ;;
val hello : Qt.c_obj = C_obj <fun>
# hello -> resize (100, 30) ;;
- : Qt.c_obj = C_void
# hello -> show () ;;
- : Qt.c_obj = C_void
# a -> exec () ;;
```

Assuming you have a working installation of QT, you will see a window containing the string "hi" in a button.

### 40.2.5 Director Classes

#### 40.2.5.1 Director Introduction

Director classes are classes which allow Ocaml code to override the public methods of a C++ object. This facility allows the user to use C++ libraries that require a derived class to provide application specific functionality in the context of an application or utility framework.

You can turn on director classes by using an optional module argument like this:

```
%module(directors="1")

...
```

```
// Turn on the director class for a specific class like this:
%feature("director")
class foo {
  ...
};
```

#### 40.2.5.2 Overriding Methods in Ocaml

Because the Ocaml language module treats C++ method calls as calls to a certain function, all you need to do is to define the function that will handle the method calls in terms of the public methods of the object, and any other relevant information. The function `new_derived_object` uses a stub class to call your methods in place of the ones provided by the underlying implementation. The object you receive is the underlying object, so you are free to call any methods you want from within your derived method. Note that calls to the underlying object do not invoke Ocaml code. You need to handle that yourself.

`new_derived_object` receives your function, the function that creates the underlying object, and any constructor arguments, and provides an object that you can use in any usual way. When C++ code calls one of the object's methods, the object invokes the Ocaml function as if it had been invoked from Ocaml, allowing any method definitions to override the C++ ones.

In this example, I'll examine the objective caml code involved in providing an overloaded class. This example is contained in Examples/ocaml/shapes.

#### 40.2.5.3 Director Usage Example

**runme.ml**

```
open Swig
open Example

...

let triangle_class pts ob meth args =
  match meth with
      "cover" ->
        (match args with
             C_list [ x_arg ; y_arg ] ->
             let xa = x_arg as float
             and ya = y_arg as float in
               (point_in_triangle pts xa ya) to bool
           | _ -> raise (Failure "cover needs two double arguments."))
    | _ -> (invoke ob) meth args ;;

...

let triangle =
  new_derived_object
    new_shape
    (triangle_class ((0.0, 0.0), (0.5, 1.0), (1.0, 0.6)))
    '() ;;

let _ = _draw_shape_coverage '(triangle, 60, 20) ;;
```

This is the meat of what you need to do. The actual "class" definition containing the overloaded method is defined in the function triangle_class. This is a lot like the class definitions emitted by SWIG, if you look at example.ml, which is generated when SWIG consumes example.i. Basically, you are given the arguments as a c_obj and the method name as a string, and you must intercept the method you are interested in and provide whatever return value you need. Bear in mind that the underlying C++ code needs the right return type, or an exception will be thrown. This exception will generally be Failure, or NotObject. You must call other ocaml methods that you rely on yourself. Due to the way directors are implemented, method calls on your object from with ocaml code will always invoke C++ methods even if they are overridden in ocaml.

In the example, the draw_shape_coverage function plots the indicated number of points as either covered (x) or uncovered ( ) between 0 and 1 on the X and Y axes. Your shape implementation can provide any coverage map it likes, as long as it responds to the "cover" method call with a boolean return (the underlying method returns bool). This might allow a tricky shape implementation, such as a boolean combination, to be expressed in a more effortless style in ocaml, while leaving the "engine" part of the program in C++.

#### 40.2.5.4 Creating director objects

The definition of the actual object triangle can be described this way:

```
let triangle =
  new_derived_object
    new_shape
    (triangle_class ((0.0, 0.0), (0.5, 1.0), (1.0, 0.0)))
    '()
```

The first argument to `new_derived_object`, new_shape is the method which returns a shape instance. This function will be invoked with the third argument will be appended to the argument list [ C_void ]. In the example, the actual argument list is sent as (C_list [ C_void ; C_void ]). The augmented constructor for a director class needs the first argument to determine whether it is being constructed as a derived object, or as an object of the indicated type only (in this case `shape`). The Second argument is a closure that will be added to the final C_obj.

The actual object passed to the self parameter of the director object will be a C_director_core, containing a c_obj option ref and a c_obj. The c_obj provided is the same object that will be returned from new_derived object, that is, the object exposing the overridden methods. The other part is an option ref that will have its value extracted before becoming the `ob` parameter of your class closure. This ref will contain `None` if the C++ object underlying is ever destroyed, and will consequently trigger an exception when any method is called on the object after that point (the actual raise is from an inner function used by new_derived_object, and throws NotObject). This prevents a deleted C++ object from causing a core dump, as long as the object is destroyed properly.

#### 40.2.5.5 Typemaps for directors, directorin, directorout, directorargout

Special typemaps exist for use with directors, the `directorin, directorout, directorargout` are used in place of`in, out, argout` typemaps, except that their direction is reversed. They provide for you to provide argout values, as well as a function return value in the same way you provide function arguments, and to receive arguments the same way you normally receive function returns.

#### 40.2.5.6 directorin typemap

The `directorin` typemap is used when you will receive arguments from a call made by C++ code to you, therefore, values will be translated from C++ to ocaml. You must provide some valid C_obj value. This is the value your ocaml code receives when you are called. In general, a simple `directorin` typemap can use the same body as a simple `out` typemap.

#### 40.2.5.7 directorout typemap

The `directorout` typemap is used when you will send an argument from your code back to the C++ caller. That is; directorout specifies a function return conversion. You can usually use the same body as an `in` typemap for the same type, except when there are special requirements for object ownership, etc.

#### 40.2.5.8 directorargout typemap

C++ allows function arguments which are by pointer (*) and by reference (&) to receive a value from the called function, as well as sending one there. Sometimes, this is the main purpose of the argument given. `directorargout` typemaps allow your caml code to emulate this by specifying additional return values to be put into the output parameters. The SWIG ocaml module is a bit loose in order to make code easier to write. In this case, your return to the caller must be a list containing the normal function return first, followed by any argout values in order. These argout values will be taken from the list and assigned to the values to be returned to C++ through directorargout typemaps. In the event that you don't specify all of the necessary values, integral values will read zero, and struct or object returns have undefined results.

### 40.2.6 Exceptions

If an error occurs in a C or C++ function, you may want to convert that error into an OCaml exception. To do this, you can use the `%exception` directive. The `%exception` directive simply lets you rewrite part of the generated wrapper code to include an error check. It is detailed in full in the Exception handling with %exception section.

In C, a function often indicates an error by returning a status code (e.g. a negative number or a NULL pointer). Here is a simple example of how you might handle that:

```
%exception malloc {
  $action
  if (result == NULL) {
    caml_failwith("Not enough memory");
  }
}
void *malloc(size_t nbytes);
```

In OCaml:

```
# let a = _malloc (C_int 20000000000);;
Exception: Failure "Not enough memory".
#
```

If a library provides some kind of general error handling framework, you can also use that. For example:

```
%exception {
  $action
  if (err_occurred()) {
    caml_failwith(err_message());
  }
}
```

If no declaration name is given to `%exception`, it is applied to all wrapper functions. `$action` is a SWIG special variable and is replaced by the C/C++ function call being wrapped.

C++ exceptions are also easy to handle. We can catch a C++ exception and rethrow it as an OCaml exception like this:

```
%exception getitem {
  try {
    $action
  } catch (std::out_of_range &e) {
    caml_failwith(e.what());
  }
}

class FooClass {
  public:
    int getitem(int index);      // Exception handling added
    ...
};
```

The language-independent `exception.i` library file can also be used to raise exceptions. See the SWIG Library chapter.

## 40.3 Documentation Features

The features described in this section can be used to generate documentation comments (colloquially referred to as "docstrings") that can be read by OCamldoc.

### 40.3.1 Module docstring

The first documentation comment of an `mli` file is the comment associated with the entire module. SWIG supports this by setting an option of the `%module` directive. For example:

```
%module(docstring="This is the example module's docstring") example
```

When you have more than just a line or so, you can retain the readability of the `%module` directive by using a macro. For example:

```
%define DOCSTRING
"The `XmlResource` class allows program resources defining menus,
controls on a panel, etc. to be loaded from an XML file."
%enddef

%module(docstring=DOCSTRING) xrc
```

# 41 Extending SWIG to support new languages


- Introduction
- Prerequisites
- The Big Picture
- Execution Model
  - Preprocessing
  - Parsing
  - Parse Trees
  - Attribute namespaces
  - Symbol Tables
  - The %feature directive
  - Code Generation

- SWIG and XML
- Primitive Data Structures
  - Strings
  - Hashes
  - Lists
  - Common operations
  - Iterating over Lists and Hashes
  - I/O
- Navigating and manipulating parse trees
- Working with attributes
- Type system
  - String encoding of types
  - Type construction
  - Type tests
  - Typedef and inheritance
  - Lvalues
  - Output functions
- Parameters
- Writing a Language Module
  - Execution model
  - Starting out
  - Command line options
  - Configuration and preprocessing
  - Entry point to code generation
  - Module I/O and wrapper skeleton
  - Low-level code generators
  - Configuration files
  - Runtime support
  - Standard library files
  - User examples
  - Test driven development and the test-suite
    - Running the test-suite
  - Documentation
  - Coding style guidelines
  - Target language status
    - Supported status
    - Experimental status
    - Deprecated status
  - Prerequisites for adding a new language module to the SWIG distribution
- Debugging Options
- Guide to parse tree nodes
- Further Development Information


## 41.1 Introduction

This chapter primarily describes SWIG's internal organization and the process by which new target languages can be developed. It also provides details for debugging and extending existing target languages and functionality.

First, a brief word of warning---SWIG is continually evolving. The information in this chapter is mostly up to date, but changes are ongoing. Expect a few inconsistencies. Also, this chapter is not meant to be a hand-holding tutorial. As a starting point, you should probably look at one of SWIG's existing modules.

## 41.2 Prerequisites

In order to extend SWIG, it is useful to have the following background:

- An understanding of the C API for the target language.
- A good grasp of the C++ type system.
- An understanding of typemaps and some of SWIG's advanced features.
- Some familiarity with writing C++ (language modules are currently written in C++).

Since SWIG is essentially a specialized C++ compiler, it may be useful to have some prior experience with compiler design (perhaps even a compilers course) to better understand certain parts of the system. A number of books will also be useful. For example, "The C Programming Language" by Kernighan and Ritchie (a.k.a, "K&R") and the C++ standard, "ISO/IEC 14882 Programming Languages - C++" will be of great use.

Also, it is useful to keep in mind that SWIG primarily operates as an extension of the C++ *type* system. At first glance, this might not be obvious, but almost all SWIG directives as well as the low-level generation of wrapper code are driven by C++ datatypes.

## 41.3 The Big Picture

SWIG is a special purpose compiler that parses C++ declarations to generate wrapper code. To make this conversion possible, SWIG makes three fundamental extensions to the C++ language:

- **Typemaps**. Typemaps are used to define the conversion/marshalling behavior of specific C++ datatypes. All type conversion in SWIG is based on typemaps. Furthermore, the association of typemaps to datatypes utilizes an advanced pattern matching mechanism that is fully integrated with the C++ type system.
- **Declaration Annotation**. To customize wrapper code generation, most declarations can be annotated with special features. For example, you can make a variable read-only, you can ignore a declaration, you can rename a member function, you can add exception handling, and so forth. Virtually all of these customizations are built on top of a low-level declaration annotator that can attach arbitrary attributes to any declaration. Code generation modules can look for these attributes to guide the wrapping process.
- **Class extension**. SWIG allows classes and structures to be extended with new methods and attributes (the `%extend` directive). This has the effect of altering the API in the target language and can be used to generate OO interfaces to C libraries.

It is important to emphasize that virtually all SWIG features reduce to one of these three fundamental concepts. The type system and pattern matching rules also play a critical role in making the system work. For example, both typemaps and declaration annotation are based on pattern matching and interact heavily with the underlying type system.

## 41.4 Execution Model

When you run SWIG on an interface, processing is handled in stages by a series of system components:

- An integrated C preprocessor reads a collection of configuration files and the specified interface file into memory. The preprocessor performs the usual functions including macro expansion and file inclusion. However, the preprocessor also performs some transformations of the interface. For instance, `#define` statements are sometimes transformed into `%constant` declarations. In addition, information related to file/line number tracking is inserted.
- A C/C++ parser reads the preprocessed input and generates a full parse tree of all of the SWIG directives and C declarations found. The parser is responsible for many aspects of the system including renaming, declaration annotation, and template expansion. However, the parser does not produce any output nor does it interact with the target language module as it runs. SWIG is not a one-pass compiler.
- A type-checking pass is made. This adjusts all of the C++ typenames to properly handle namespaces, typedefs, nested classes, and other issues related to type scoping.
- A semantic pass is made on the parse tree to collect information related to properties of the C++ interface. For example, this pass would determine whether or not a class allows a default constructor.
- A code generation pass is made using a specific target language module. This phase is responsible for generating the actual wrapper code. All of SWIG's user-defined modules are invoked during this latter stage of compilation.

The next few sections briefly describe some of these stages.

### 41.4.1 Preprocessing

The preprocessor plays a critical role in the SWIG implementation. This is because a lot of SWIG's processing and internal configuration is managed not by code written in C, but by configuration files in the SWIG library. In fact, when you run SWIG, parsing starts with a small interface file like this (note: this explains the cryptic error messages that new users sometimes get when SWIG is misconfigured or installed incorrectly):

```
%include "swig.swg"             // Global SWIG configuration
%include "langconfig.swg"       // Language specific configuration
%include "yourinterface.i"      // Your interface file
```

The `swig.swg` file contains global configuration information. In addition, this file defines many of SWIG's standard directives as macros. For instance, part of of `swig.swg` looks like this:

```
...
/* Code insertion directives such as %wrapper %{ ... %} */

#define %begin       %insert("begin")
#define %runtime     %insert("runtime")
#define %header      %insert("header")
#define %wrapper     %insert("wrapper")
#define %init        %insert("init")

/* Access control directives */

#define %immutable   %feature("immutable", "1")
#define %mutable     %feature("immutable")

/* Directives for callback functions */

#define %callback(x) %feature("callback") `x`;
#define %nocallback  %feature("callback");

/* %ignore directive */

#define %ignore         %rename($ignore)
#define %ignorewarn(x)  %rename("$ignore:" x)
...
```

The fact that most of the standard SWIG directives are macros is intended to simplify the implementation of the internals. For instance, rather than having to support dozens of special directives, it is easier to have a few basic primitives such as `%feature` or `%insert`.

The *`langconfig.swg`* file is supplied by the target language. This file contains language-specific configuration information. More often than not, this file provides run-time wrapper support code (e.g., the type-checker) as well as a collection of typemaps that define the default wrapping behavior. Note: the name of this file depends on the target language and is usually something like `python.swg` or `perl5.swg`.

As a debugging aid, the text that SWIG feeds to its C++ parser can be obtained by running `swig -E interface.i`. This option, like the same option that regular C/C++ compilers support, generates the preprocessed output and is useful for looking at how macros have been expanded as well as everything else that goes into the low-level construction of the wrapper code. Also, like a regular C/C++ compiler, the preprocessed output can be generated in one invocation and then fed back in with a second invocation. This is the approach that the CCache tool uses as part of its strategy to speed up repeated builds with the same inputs.

### 41.4.2 Parsing

The current C++ parser handles a subset of C++. Most incompatibilities with C are due to subtle aspects of how SWIG parses declarations. Specifically, SWIG expects all C/C++ declarations to follow this general form:

```
storage type declarator initializer;
```

*`storage`* is a keyword such as `extern`, `static`, `typedef`, or `virtual`. *`type`* is a primitive datatype such as `int` or `void`. *`type`* may be optionally qualified with a qualifier such as `const` or `volatile`. *`declarator`* is a name with additional type-construction modifiers attached to it (pointers, arrays, references, functions, etc.). Examples of declarators include `*x`, `**x`, `x[20]`, and `(*x)(int, double)`. The *`initializer`* may be a value assigned using= or body of code enclosed in braces `{ ... }`.

This declaration format covers most common C++ declarations. However, the C++ standard is somewhat more flexible in the placement of the parts. For example, it is technically legal, although uncommon to write something like `int typedef const a` in your program. SWIG simply doesn't bother to deal with this case.

The other significant difference between C++ and SWIG is in the treatment of typenames. In C++, if you have a declaration like this,

```
int blah(Foo *x, Bar *y);
```

it won't parse correctly unless `Foo` and `Bar` have been previously defined as types either using a `class` definition or a `typedef`. The reasons for this are subtle, but this treatment of typenames is normally integrated at the level of the C tokenizer---when a typename appears, a different token is returned to the parser instead of an identifier.

SWIG does not operate in this manner--any legal identifier can be used as a type name. The reason for this is primarily motivated by the use of SWIG with partially defined data. Specifically, SWIG is supposed to be easy to use on interfaces with missing type information.

Because of the different treatment of typenames, the most serious limitation of the SWIG parser is that it can't process type declarations where an extra (and unnecessary) grouping operator is used. For example:

```
int (x);         /* A variable x */
int (y)(int);    /* A function y */
```

The placing of extra parentheses in type declarations like this is already recognized by the C++ community as a potential source of strange programming errors. For example, Scott Meyers "Effective STL" discusses this problem in a section on avoiding C++'s "most vexing parse."

The parser is also unable to handle declarations with no return type or bare argument names. For example, in an old C program, you might see things like this:

```
foo(a, b) {
...
}
```

In this case, the return type as well as the types of the arguments are taken by the C compiler to be an `int`. However, SWIG interprets the above code as an abstract declarator for a function returning a `foo` and taking types `a` and `b` as arguments).

### 41.4.3 Parse Trees

The SWIG parser produces a complete parse tree of the input file before any wrapper code is actually generated. Each item in the tree is known as a "Node". Each node is identified by a symbolic tag. Furthermore, a node may have an arbitrary number of children. The parse tree structure and tag names of an interface can be displayed using `swig -debug-tags`. For example:

```
$ swig -c++ -python -debug-tags example.i
 . top (example.i:1)
 . top . include (example.i:1)
 . top . include . typemap (/r0/beazley/Projects/lib/swig1.3/swig.swg:71)
 . top . include . typemap . typemapitem (/r0/beazley/Projects/lib/swig1.3/swig.swg:71)
 . top . include . typemap (/r0/beazley/Projects/lib/swig1.3/swig.swg:83)
 . top . include . typemap . typemapitem (/r0/beazley/Projects/lib/swig1.3/swig.swg:83)
 . top . include (example.i:4)
 . top . include . insert (/r0/beazley/Projects/lib/swig1.3/python/python.swg:7)
 . top . include . insert (/r0/beazley/Projects/lib/swig1.3/python/python.swg:8)
 . top . include . typemap (/r0/beazley/Projects/lib/swig1.3/python/python.swg:19)
...
 . top . include (example.i:6)
 . top . include . module (example.i:2)
 . top . include . insert (example.i:6)
 . top . include . include (example.i:9)
 . top . include . include . class (example.h:3)
 . top . include . include . class . access (example.h:4)
 . top . include . include . class . constructor (example.h:7)
 . top . include . include . class . destructor (example.h:10)
 . top . include . include . class . cdecl (example.h:11)
 . top . include . include . class . cdecl (example.h:11)
 . top . include . include . class . cdecl (example.h:12)
 . top . include . include . class . cdecl (example.h:13)
 . top . include . include . class . cdecl (example.h:14)
 . top . include . include . class . cdecl (example.h:15)
 . top . include . include . class (example.h:18)
 . top . include . include . class . access (example.h:19)
 . top . include . include . class . cdecl (example.h:20)
 . top . include . include . class . access (example.h:21)
 . top . include . include . class . constructor (example.h:22)
 . top . include . include . class . cdecl (example.h:23)
 . top . include . include . class . cdecl (example.h:24)
 . top . include . include . class (example.h:27)
 . top . include . include . class . access (example.h:28)
 . top . include . include . class . cdecl (example.h:29)
 . top . include . include . class . access (example.h:30)
 . top . include . include . class . constructor (example.h:31)
 . top . include . include . class . cdecl (example.h:32)
 . top . include . include . class . cdecl (example.h:33)
```

Even for the most simple interface, the parse tree structure is larger than you might expect. For example, in the above output, a substantial number of nodes are actually generated by the `python.swg` configuration file which defines typemaps and other directives. The contents of the user-supplied input file don't appear until the end of the output.

The contents of each parse tree node consist of a collection of attribute/value pairs. Internally, the nodes are simply represented by hash tables. A display of the entire parse-tree structure can be obtained using `swig -debug-top <n>`, where `n` is the stage being processed. There are a number of other parse tree display options, for example, `swig -debug-module <n>` will avoid displaying system parse information and only display the parse tree pertaining to the user's module at stage `n` of processing. Adding the `-debug-quiet` option is recommended as it removes some noise which is not usually needed, that is, the display of many linked list pointers and symbol table pointers. See Debugging Options for a full list.

```
$ swig -c++ -python -debug-module 1 -debug-quiet example.i
debug-module stage 1
+++ module ----------------------------------------
| name         - "example"
|
+++ insert ----------------------------------------
| code         - "\n#include \"example.h\"\n"
|
+++ include ---------------------------------------
| name         - "example.h"

     +++ class -----------------------------------------
     | abstracts    - 0x7f4f15182930
     | allows_typedef - "1"
     | kind         - "class"
     | name         - "Shape"
     | sym:name     - "Shape"

          +++ access ----------------------------------------
          | kind         - "public"
          |
          +++ constructor -----------------------------------------
          | access       - "public"
          | code         - "{\n    nshapes++;\n  }"
          | decl         - "f()."
          | feature:new  - "1"
          | ismember     - "1"
          | name         - "Shape"
          | sym:name     - "Shape"
          |
          +++ destructor ------------------------------------
          | access       - "public"
          | code         - "{\n    nshapes--;\n  }"
          | decl         - "f()."
          | ismember     - "1"
          | name         - "~Shape"
          | storage      - "virtual"
          | sym:name     - "~Shape"
          |
          +++ cdecl -----------------------------------------
          | access       - "public"
          | decl         - ""
          | ismember     - "1"
          | kind         - "variable"
```

```
          | name         - "x"
          | sym:name     - "x"
          | type         - "double"
          |
          +++ cdecl --------------------------------------
          | access       - "public"
          | decl         - ""
          | ismember     - "1"
          | kind         - "variable"
          | name         - "y"
          | sym:name     - "y"
          | type         - "double"
          |
          +++ cdecl --------------------------------------
          | access       - "public"
          | decl         - "f(double,double)."
          | ismember     - "1"
          | kind         - "function"
          | name         - "move"
          | parms        - 'double dx,double dy'
          | sym:name     - "move"
          | type         - "void"
          |
          +++ cdecl --------------------------------------
          | abstract     - "1"
          | access       - "public"
          | decl         - "f()."
          | ismember     - "1"
          | kind         - "function"
          | name         - "area"
          | storage      - "virtual"
          | sym:name     - "area"
          | type         - "double"
          | value        - "0"
          | valuetype    - "int"
          |
          +++ cdecl --------------------------------------
          | abstract     - "1"
          | access       - "public"
          | decl         - "f()."
          | ismember     - "1"
          | kind         - "function"
          | name         - "perimeter"
          | storage      - "virtual"
          | sym:name     - "perimeter"
          | type         - "double"
          | value        - "0"
          | valuetype    - "int"
          |
          +++ cdecl --------------------------------------
          | access       - "public"
          | decl         - ""
          | ismember     - "1"
          | kind         - "variable"
          | name         - "nshapes"
          | storage      - "static"
          | sym:name     - "nshapes"
          | type         - "int"
          |
     +++ class ----------------------------------------
     | allows_typedef - "1"
     | baselist     - 0x7f4f15182ad0
     | kind         - "class"
     | name         - "Circle"
     | privatebaselist - 0x7f4f15182b10
     | protectedbaselist - 0x7f4f15182af0
     | sym:name     - "Circle"

          +++ access ---------------------------------------
          | kind         - "private"
          |
          +++ cdecl --------------------------------------
          | access       - "private"
          | decl         - ""
          | ismember     - "1"
          | kind         - "variable"
          | name         - "radius"
          | type         - "double"
          |
          +++ access ---------------------------------------
          | kind         - "public"
          |
          +++ constructor ----------------------------------
          | access       - "public"
          | code         - "{ }"
          | decl         - "f(double)."
          | feature:new  - "1"
          | ismember     - "1"
          | name         - "Circle"
          | parms        - 'double r'
          | sym:name     - "Circle"
          |
          +++ cdecl --------------------------------------
          | access       - "public"
          | decl         - "f()."
          | ismember     - "1"
          | kind         - "function"
          | name         - "area"
          | storage      - "virtual"
          | sym:name     - "area"
          | type         - "double"
          |
```

```
          +++ cdecl --------------------------------------
          | access       - "public"
          | decl         - "f()."
          | ismember    - "1"
          | kind         - "function"
          | name         - "perimeter"
          | storage      - "virtual"
          | sym:name     - "perimeter"
          | type         - "double"
          |
     +++ class ----------------------------------------
     | allows_typedef - "1"
     | baselist     - 0x7f4f15183830
     | kind         - "class"
     | name         - "Square"
     | privatebaselist - 0x7f4f15183870
     | protectedbaselist - 0x7f4f15183850
     | sym:name     - "Square"

          +++ access -------------------------------------
          | kind         - "private"
          |
          +++ cdecl --------------------------------------
          | access       - "private"
          | decl         - ""
          | ismember     - "1"
          | kind         - "variable"
          | name         - "width"
          | type         - "double"
          |
          +++ access -------------------------------------
          | kind         - "public"
          |
          +++ constructor --------------------------------
          | access       - "public"
          | code         - "{ }"
          | decl         - "f(double)."
          | feature:new  - "1"
          | ismember     - "1"
          | name         - "Square"
          | parms        - 'double w'
          | sym:name     - "Square"
          |
          +++ cdecl --------------------------------------
          | access       - "public"
          | decl         - "f()."
          | ismember     - "1"
          | kind         - "function"
          | name         - "area"
          | storage      - "virtual"
          | sym:name     - "area"
          | type         - "double"
          |
          +++ cdecl --------------------------------------
          | access       - "public"
          | decl         - "f()."
          | ismember     - "1"
          | kind         - "function"
          | name         - "perimeter"
          | storage      - "virtual"
          | sym:name     - "perimeter"
          | type         - "double"
```

### 41.4.4 Attribute namespaces

Attributes of parse tree nodes are often prepended with a namespace qualifier. For example, the attributes `sym:name` and `sym:symtab` are attributes related to symbol table management and are prefixed with `sym:`. As a general rule, only those attributes which are directly related to the raw declaration appear without a prefix (type, name, declarator, etc.).

Target language modules may add additional attributes to nodes to assist the generation of wrapper code. The convention for doing this is to place these attributes in a namespace that matches the name of the target language. For example, `python:foo` or `perl:foo`.

### 41.4.5 Symbol Tables

During parsing, all symbols are managed in the space of the target language. The `sym:name` attribute of each node contains the symbol name selected by the parser. Normally, `sym:name` and `name` are the same. However, the `%rename` directive can be used to change the value of `sym:name`. You can see the effect of `%rename` by trying it on a simple interface and dumping the parse tree. For example:

```
%rename(foo_i) foo(int);
%rename(foo_d) foo(double);

void foo(int);
void foo(double);
void foo(Bar *b);
```

There are various `debug-` options that can be useful for debugging and analysing the parse tree. For example, the `debug-top <n>` or `debug-module <n>` options will dump the entire/top of the parse tree or the module subtree at one of the four `n` stages of processing. The parse tree can be viewed after the final stage of processing by running SWIG:

```
$ swig -debug-top 1 -debug-quiet example.i
...
          +++ cdecl --------------------------------------
          | decl         - "f(int)."
          | name         - "foo"
          | parms        - int
          | sym:name     - "foo_i"
          | type         - "void"
          |
          +++ cdecl --------------------------------------
          | decl         - "f(double)."
```

```
           | name         - "foo"
           | parms        - double
           | sym:name     - "foo_d"
           | type         - "void"
           |
           +++ cdecl --------------------------------------
           | decl         - "f(p.Bar)."
           | name         - "foo"
           | parms        - Bar *
           | sym:name     - "foo"
           | type         - "void"
```

All symbol-related conflicts and complaints about overloading are based on `sym:name` values. For instance, the following example uses `%rename` in reverse to generate a name clash.

```
%rename(foo) foo_i(int);
%rename(foo) foo_d(double);

void foo_i(int);
void foo_d(double);
void foo(Bar *b);
```

When you run SWIG on this you now get:

```
$ ./swig example.i
example.i:6. Overloaded declaration ignored.  foo_d(double )
example.i:5. Previous declaration is foo_i(int )
example.i:7. Overloaded declaration ignored.  foo(Bar *)
example.i:5. Previous declaration is foo_i(int )
```

### 41.4.6 The %feature directive

A number of SWIG directives such as `%exception` are implemented using the low-level `%feature` directive. For example:

```
%feature("except") getitem(int) {
  try {
     $action
  } catch (badindex) {
     ...
  }
}

...
class Foo {
public:
  Object *getitem(int index) throws(badindex);
  ...
};
```

The behavior of `%feature` is very easy to describe--it simply attaches a new attribute to any parse tree node that matches the given prototype. When a feature is added, it shows up as an attribute in the `feature:` namespace. You can see this when running with the `-debug-top 4 -debug-quiet` option. For example:

```
 +++ cdecl --------------------------------------
 | decl         - "f(int).p."
 | feature:except - "{\n    try {\n       $action\n    } catc..."
 | name         - "getitem"
 | parms        - int
 | sym:name     - "getitem"
 | type         - "Object"
 |
```

Feature names are completely arbitrary and a target language module can be programmed to respond to any feature name that it wants to recognize. The data stored in a feature attribute is usually just a raw unparsed string. For example, the exception code above is simply stored without any modifications.

### 41.4.7 Code Generation

Language modules work by defining handler functions that know how to respond to different types of parse-tree nodes. These handlers simply look at the attributes of each node in order to produce low-level code.

In reality, the generation of code is somewhat more subtle than simply invoking handler functions. This is because parse-tree nodes might be transformed. For example, suppose you are wrapping a class like this:

```
class Foo {
public:
  virtual int *bar(int x);
};
```

When the parser constructs a node for the member `bar`, it creates a raw "cdecl" node with the following attributes:

```
nodeType    : cdecl
name        : bar
type        : int
decl        : f(int).p
parms       : int x
storage     : virtual
sym:name    : bar
```

To produce wrapper code, this "cdecl" node undergoes a number of transformations. First, the node is recognized as a function declaration. This adjusts some of the type information--specifically, the declarator is joined with the base datatype to produce this:

```
nodeType    : cdecl
name        : bar
type        : p.int        <-- Notice change in return type
decl        : f(int).p
parms       : int x
storage     : virtual
sym:name    : bar
```

Next, the context of the node indicates that the node is really a member function. This produces a transformation to a low-level accessor function like this:

```
nodeType    : cdecl
name        : bar
type        : int.p
decl        : f(int).p
parms       : Foo *self, int x            <-- Added parameter
storage     : virtual
wrap:action : result = (arg1)->bar(arg2)  <-- Action code added
sym:name    : Foo_bar                     <-- Symbol name changed
```

In this transformation, notice how an additional parameter was added to the parameter list and how the symbol name of the node has suddenly changed into an accessor using the naming scheme described in the "SWIG Basics" chapter. A small fragment of "action" code has also been generated--notice how the `wrap:action` attribute defines the access to the underlying method. The data in this transformed node is then used to generate a wrapper.

Language modules work by registering handler functions for dealing with various types of nodes at different stages of transformation. This is done by inheriting from a special `Language` class and defining a collection of virtual methods. For example, the Python module defines a class as follows:

```
class PYTHON : public Language {
protected:
public :
  virtual void main(int, char *argv[]);
  virtual int  top(Node *);
  virtual int  functionWrapper(Node *);
  virtual int  constantWrapper(Node *);
  virtual int  variableWrapper(Node *);
  virtual int  nativeWrapper(Node *);
  virtual int  membervariableHandler(Node *);
  virtual int  memberconstantHandler(Node *);
  virtual int  memberfunctionHandler(Node *);
  virtual int  constructorHandler(Node *);
  virtual int  destructorHandler(Node *);
  virtual int  classHandler(Node *);
  virtual int  classforwardDeclaration(Node *);
  virtual int  insertDirective(Node *);
  virtual int  importDirective(Node *);
};
```

The role of these functions is described shortly.

### 41.4.8 SWIG and XML

Much of SWIG's current parser design was originally motivated by interest in using XML to represent SWIG parse trees. Although XML is not currently used in any direct manner, the parse tree structure, use of node tags, attributes, and attribute namespaces are all influenced by aspects of XML parsing. Therefore, in trying to understand SWIG's internal data structures, it may be useful to keep XML in the back of your mind as a model.

In addition to the options beginning `-debug-` for dumping out SWIG's parse tree in a simple text format, the parse tree can also be dumped as XML. There are three options below, where `file` specifies the name of a file for generating the resulting XML into:

1. `-xmlout <file>`

   Use this XML option in addition to any target language options. The parse tree is dumped after the final stage of processing, that is, after the target language has finished processing. It is useful for seeing the parse tree at stage 4 after any target language processing. The parse tree dumped is very similar in content to that generated by `-debug-top 4`.

2. `-xml -o <file>`

   Use this XML option without specifying a target language. The parse tree that is dumped is the same as if there was a no target language option. It is useful for seeing the parse tree at stage 3, which is the same as stage 4, as there is no target language processing. The parse tree dumped is very similar in content to that generated by `-debug-top 3`.

3. `-xml -xmllite -o <file>`

   Same as above, except the addition of `-xmllite` reduces the output by skipping some type information, that is, the typescope and typetab nodes.

The parse tree generated using `-xmlout` is much bigger than that using `-xml` as the target languages process numerous extra SWIG system files and also add to the parse tree quite considerably in stage 4.

The XML is a dump of SWIG's internal parse tree and as such it is subject to change at any time as and when SWIG's implementation changes.

## 41.5 Primitive Data Structures

Most of SWIG is constructed using three basic data structures: strings, hashes, and lists. These data structures are dynamic in same way as similar structures found in many scripting languages. For instance, you can have containers (lists and hash tables) of mixed types and certain operations are polymorphic.

This section briefly describes the basic structures so that later sections of this chapter make more sense.

When describing the low-level API, the following type name conventions are used:

- `String`. A string object.
- `Hash`. A hash object.
- `List`. A list object.
- `String_or_char`. A string object or a `char *`.
- `Object_or_char`. An object or a `char *`.
- `Object`. Any object (string, hash, list, etc.)

In most cases, other typenames in the source are aliases for one of these primitive types. Specifically:

```
typedef String SwigType;
typedef Hash   Parm;
```

```
typedef Hash   ParmList;
typedef Hash   Node;
typedef Hash   Symtab;
typedef Hash   Typetab;
```

### 41.5.1 Strings

**String *NewString(const String_or_char *val)**

Creates a new string with initial value val. val may be a char * or another String object. If you want to create an empty string, use "" for val.

**String *NewStringf(const char *fmt, ...)**

Creates a new string whose initial value is set according to a C printf style format string in fmt. Additional arguments follow depending on fmt.

**String *Copy(String *s)**

Make a copy of the string s.

**void Delete(String *s)**

Deletes s.

**int Len(const String_or_char *s)**

Returns the length of the string.

**char *Char(const String_or_char *s)**

Returns a pointer to the first character in a string.

**void Append(String *s, const String_or_char *t)**

Appends t to the end of string s.

**void Insert(String *s, int pos, const String_or_char *t)**

Inserts t into s at position pos. The contents of s are shifted accordingly. The special value DOH_END can be used for pos to indicate insertion at the end of the string (appending).

**int Strcmp(const String_or_char *s, const String_or_char *t)**

Compare strings s and t. Same as the C strcmp() function.

**int Strncmp(const String_or_char *s, const String_or_char *t, int len)**

Compare the first len characters of strings s and t. Same as the C strncmp() function.

**char *Strstr(const String_or_char *s, const String_or_char *pat)**

Returns a pointer to the first occurrence of pat in s. Same as the C strstr() function.

**char *Strchr(const String_or_char *s, char ch)**

Returns a pointer to the first occurrence of character ch in s. Same as the C strchr() function.

**void Chop(String *s)**

Chops trailing whitespace off the end of s .

**int Replace(String *s, const String_or_char *pat, const String_or_char *rep, int flags)**

Replaces the pattern pat with rep in string s . flags is a combination of the following flags:

```
DOH_REPLACE_ANY       - Replace all occurrences
DOH_REPLACE_ID        - Valid C identifiers only
DOH_REPLACE_NOQUOTE   - Don't replace in quoted strings
DOH_REPLACE_FIRST     - Replace first occurrence only.
```

Returns the number of replacements made (if any).

At most one of DOH_REPLACE_ANY and DOH_REPLACE_FIRST should be specified. DOH_REPLACE_ANY is the default if neither is specified.

### 41.5.2 Hashes

**Hash *NewHash()**

Creates a new empty hash table.

**Hash *Copy(Hash *h)**

Make a shallow copy of the hash h.

**void Delete(Hash *h)**

Deletes h.

**int Len(Hash *h)**

Returns the number of items in h.

**Object *Getattr(Hash *h, const String_or_char *key)**

Gets an object from h. key may be a string or a simple char * string. Returns NULL if not found.

**int Setattr(Hash *h, const String_or_char *key, const Object_or_char *val)**

Stores val in h. key may be a string or a simple char *. If val is not a standard object (String, Hash, or List) it is assumed to be a char * in which case it is used to construct a String that is stored in the hash. If val is NULL, the object is deleted. Increases the reference count of val. Returns 1 if this operation replaced an existing hash entry, 0 otherwise.

**int Delattr(Hash *h, const String_or_char *key)**

Deletes the hash item referenced by key. Decreases the reference count on the corresponding object (if any). Returns 1 if an object was removed, 0 otherwise.

**List *Keys(Hash *h)**

Returns the list of hash table keys.

**List *SortedKeys(Hash *h, int (*cmp) (const DOH *, const DOH *))**

Returns the list of sorted hash table keys.

### 41.5.3 Lists

**List *NewList()**

Creates a new empty list.

**List *Copy(List *x)**

Make a shallow copy of the List `x`.

**void Delete(List *x)**

Deletes `x`.

**int Len(List *x)**

Returns the number of items in `x`.

**Object *Getitem(List *x, int n)**

Returns an object from `x` with index `n` . If `n` is beyond the end of the list, the last item is returned. If `n` is negative, the first item is returned.

**int *Setitem(List *x, int n, const Object_or_char *val)**

Stores `val` in `x`. If `val` is not a standard object (String, Hash, or List) it is assumed to be a `char *` in which case it is used to construct a`String` that is stored in the list. `n` must be in range. Otherwise, an assertion will be raised.

**int *Delitem(List *x, int n)**

Deletes item `n` from the list, shifting items down if necessary. To delete the last item in the list, use the special value `DOH_END` for `n`.

**void Append(List *x, const Object_or_char *t)**

Appends `t` to the end of `x`. If `t` is not a standard object, it is assumed to be a`char *` and is used to create a String object.

**void Insert(String *s, int pos, const Object_or_char *t)**

Inserts `t` into `s` at position `pos`. The contents of `s` are shifted accordingly. The special value `DOH_END` can be used for `pos` to indicate insertion at the end of the list (appending). If `t` is not a standard object, it is assumed to be a `char *` and is used to create a String object.

### 41.5.4 Common operations

The following operations are applicable to all datatypes.

**Object *Copy(Object *x)**

Make a copy of the object `x`.

**void Delete(Object *x)**

Deletes `x`.

**void Setfile(Object *x, String_or_char *f)**

Sets the filename associated with `x`. Used to track objects and report errors.

**String *Getfile(Object *x)**

Gets the filename associated with `x`.

**void Setline(Object *x, int n)**

Sets the line number associated with `x`. Used to track objects and report errors.

**int Getline(Object *x)**

Gets the line number associated with `x`.

### 41.5.5 Iterating over Lists and Hashes

To iterate over the elements of a list or a hash table, the following functions are used:

**Iterator First(Object *x)**

Returns an iterator object that points to the first item in a list or hash table. The `item` attribute of the Iterator object is a pointer to the item. For hash tables, the `key` attribute of the Iterator object additionally points to the corresponding Hash table key. The `item` and `key` attributes are NULL if the object contains no items or if there are no more items.

**Iterator Next(Iterator i)**

Returns an iterator that points to the next item in a list or hash table. Here are two examples of iteration:

```
List *l = (some list);
Iterator i;

for (i = First(l); i.item; i = Next(i)) {
  Printf(stdout, "%s\n", i.item);
}

Hash *h = (some hash);
Iterator j;

for (j = First(j); j.item; j= Next(j)) {
  Printf(stdout, "%s : %s\n", j.key, j.item);
}
```

### 41.5.6 I/O

Special I/O functions are used for all internal I/O. These operations work on C `FILE *` objects, String objects, and special `File` objects (which are merely a wrapper around `FILE *`).

**`int Printf(String_or_FILE *f, const char *fmt, ...)`**

Formatted I/O. Same as the C `fprintf()` function except that output can also be directed to a string object. Note: the `%s` format specifier works with both strings and `char *`. All other format operators have the same meaning.

**`int Printv(String_or_FILE *f, String_or_char *arg1, ..., NULL)`**

Prints a variable number of strings arguments to the output. The last argument to this function must be NULL. The other arguments can either be `char *` or string objects.

**`int Putc(int ch, String_or_FILE *f)`**

Same as the C `fputc()` function.

**`int Write(String_or_FILE *f, void *buf, int len)`**

Same as the C `write()` function.

**`int Read(String_or_FILE *f, void *buf, int maxlen)`**

Same as the C `read()` function.

**`int Getc(String_or_FILE *f)`**

Same as the C `fgetc()` function.

**`int Ungetc(int ch, String_or_FILE *f)`**

Same as the C `ungetc()` function.

**`int Seek(String_or_FILE *f, int offset, int whence)`**

Same as the C `seek()` function. `offset` is the number of bytes. `whence` is one of `SEEK_SET`, `SEEK_CUR`, or `SEEK_END`..

**`long Tell(String_or_FILE *f)`**

Same as the C `tell()` function.

**`File *NewFile(const char *filename, const char *mode, List *newfiles)`**

Create a File object using the `fopen()` library call. This file differs from `FILE *` in that it can be placed in the standard SWIG containers (lists, hashes, etc.). The `filename` is added to the `newfiles` list if `newfiles` is non-zero and the file was created successfully.

**`File *NewFileFromFile(FILE *f)`**

Create a File object wrapper around an existing `FILE *` object.

There's no explicit function to close a file, just call `Delete(f)` - this decreases the reference count, and the file will be closed when the reference count reaches zero.

The use of the above I/O functions and strings play a critical role in SWIG. It is common to see small code fragments of code generated using code like this:

```
/* Print into a string */
String *s = NewString("");
Printf(s, "Hello\n");
for (i = 0; i < 10; i++) {
  Printf(s, "%d\n", i);
}
...
/* Print string into a file */
Printf(f, "%s\n", s);
```

Similarly, the preprocessor and parser all operate on string-files.

## 41.6 Navigating and manipulating parse trees

Parse trees are built as collections of hash tables. Each node is a hash table in which arbitrary attributes can be stored. Certain attributes in the hash table provide links to other parse tree nodes. The following macros can be used to move around the parse tree.

**`String *nodeType(Node *n)`**

Returns the node type tag as a string. The returned string indicates the type of parse tree node.

**`Node *nextSibling(Node *n)`**

Returns the next node in the parse tree. For example, the next C declaration.

**`Node *previousSibling(Node *n)`**

Returns the previous node in the parse tree. For example, the previous C declaration.

**`Node *firstChild(Node *n)`**

Returns the first child node. For example, if `n` was a C++ class node, this would return the node for the first class member.

**`Node *lastChild(Node *n)`**

Returns the last child node. You might use this if you wanted to append a new node to the children of a class.

**`Node *parentNode(Node *n)`**

Returns the parent of node `n`. Use this to move up the pass tree.

The following macros can be used to change all of the above attributes. Normally, these functions are only used by the parser. Changing them without knowing what you are doing is likely to be dangerous.

**`void set_nodeType(Node *n, const String_or_char)`**

Change the node type. tree node.

**`void set_nextSibling(Node *n, Node *s)`**

Set the next sibling.

**void set_previousSibling(Node *n, Node *s)**

Set the previous sibling.

**void set_firstChild(Node *n, Node *c)**

Set the first child node.

**void set_lastChild(Node *n, Node *c)**

Set the last child node.

**void set_parentNode(Node *n, Node *p)**

Set the parent node.

The following utility functions are used to alter the parse tree (at your own risk)

**void appendChild(Node *parent, Node *child)**

Append a child to `parent`. The appended node becomes the last child.

**void deleteNode(Node *node)**

Deletes a node from the parse tree. Deletion reconnects siblings and properly updates the parent so that sibling nodes are unaffected.

## 41.7 Working with attributes

Since parse tree nodes are just hash tables, attributes are accessed using the `Getattr()`, `Setattr()`, and `Delattr()` operations. For example:

```
int functionHandler(Node *n) {
  String *name    = Getattr(n, "name");
  String *symname = Getattr(n, "sym:name");
  SwigType *type  = Getattr(n, "type");
  ...
}
```

New attributes can be freely attached to a node as needed. However, when new attributes are attached during code generation, they should be prepended with a namespace prefix. For example:

```
...
Setattr(n, "python:docstring", doc);     /* Store docstring */
...
```

A quick way to check the value of an attribute is to use the `checkAttribute()` function like this:

```
if (checkAttribute(n, "storage", "virtual")) {
  /* n is virtual */
  ...
}
```

Changing the values of existing attributes is allowed and is sometimes done to implement node transformations. However, if a function/method modifies a node, it is required to restore modified attributes to their original values. To simplify the task of saving/restoring attributes, the following functions are used:

**int Swig_save(const char *ns, Node *n, const char *name1, const char *name2, ..., NIL)**

Saves a copy of attributes `name1`, `name2`, etc. from node `n`. Copies of the attributes are actually resaved in the node in a different namespace which is set by the `ns` argument. For example, if you call `Swig_save("foo", n, "type", NIL)`, then the "type" attribute will be copied and saved as "foo:type". The namespace name itself is stored in the "view" attribute of the node. If necessary, this can be examined to find out where previous values of attributes might have been saved.

**int Swig_restore(Node *n)**

Restores the attributes saved by the previous call to `Swig_save()`. Those attributes that were supplied to `Swig_save()` will be restored to their original values.

The `Swig_save()` and `Swig_restore()` functions must always be used as a pair. That is, every call to `Swig_save()` must have a matching call to `Swig_restore()`. Calls can be nested if necessary. Here is an example that shows how the functions might be used:

```
int variableHandler(Node *n) {
  Swig_save("variableHandler", n, "type", "sym:name", NIL);
  String *symname = Getattr(n, "sym:name");
  SwigType *type  = Getattr(n, "type");
  ...
  Append(symname, "_global");        // Change symbol name
  SwigType_add_pointer(type);        // Add pointer
  ...
  generate wrappers
  ...
  Swig_restore(n);                  // Restore original values
  return SWIG_OK;
}
```

**int Swig_require(const char *ns, Node *n, const char *name1, const char *name2, ..., NIL)**

This is an enhanced version of `Swig_save()` that adds error checking. If an attribute name is not present in `n` , a failed assertion results and SWIG terminates with a fatal error. Optionally, if an attribute name is specified as "*_name_", a copy of the attribute is saved as with `Swig_save()`. If an attribute is specified as "?_name_", the attribute is optional. `Swig_restore()` must always be called after using this function.

## 41.8 Type system

SWIG implements the complete C++ type system including typedef, inheritance, pointers, references, and pointers to members. A detailed discussion of type theory is impossible here. However, let's cover the highlights.

### 41.8.1 String encoding of types

All types in SWIG consist of a base datatype and a collection of type operators that are applied to the base. A base datatype is almost always some kind of primitive type such as `int` or `double` . The operators consist of things like pointers, references, arrays, and so forth. Internally, types are represented as strings that are constructed in a very precise manner. Here are some examples:

```
C datatype                    SWIG encoding (strings)
----------------------------  --------------------------
int                           "int"
int *                         "p.int"
const int *                   "p.q(const).int"
int (*x)(int, double)         "p.f(int, double).int"
int [20][30]                  "a(20).a(30).int"
int (F::*)(int)               "m(F).f(int).int"
vector<int> *                 "p.vector<(int)>"
```

Reading the SWIG encoding is often easier than figuring out the C code---just read it from left to right. For a type of "p.f(int, double).int" is a "pointer to a function(int, double) that returns int".

The following operator encodings are used in type strings:

```
Operator             Meaning
-------------------  -------------------------------
p.                   Pointer to
a(n).                Array of dimension n
r.                   C++ reference
m(class).            Member pointer to class
f(args).             Function.
q(qlist).            Qualifiers
```

In addition, type names may be parameterized by templates. This is represented by enclosing the template parameters in `<( ... )>`. Variable length arguments are represented by the special base type of `v(...)`.

If you want to experiment with type encodings, the raw type strings can be inserted into an interface file using backticks `` wherever a type is expected. For instance, here is an extremely perverted example:

```
`p.a(10).p.f(int, p.f(int).int)` foo(int, int (*x)(int));
```

This corresponds to the immediately obvious C declaration:

```
(*(*foo(int, int (*)(int)))[10])(int, int (*)(int));
```

Aside from the potential use of this declaration on a C programming quiz, it motivates the use of the special SWIG encoding of types. The SWIG encoding is much easier to work with because types can be easily examined, modified, and constructed using simple string operations (comparison, substrings, concatenation, etc.). For example, in the parser, a declaration like this

```
int *a[30];
```

is processed in a few pieces. In this case, you have the base type "`int`" and the declarator of type "`a(30).p.`". To make the final type, the two parts are just joined together using string concatenation.

### 41.8.2 Type construction

The following functions are used to construct types. You should use these functions instead of trying to build the type strings yourself.

**`void SwigType_add_pointer(SwigType *ty)`**

Adds a pointer to `ty`.

**`void SwigType_del_pointer(SwigType *ty)`**

Removes a single pointer from `ty`.

**`void SwigType_add_reference(SwigType *ty)`**

Adds a reference to `ty`.

**`void SwigType_add_array(SwigType *ty, const String_or_char *size)`**

Adds an array with dimension `dim` to `ty` .

**`void SwigType_del_array(SwigType *ty)`**

Removes a single array dimension from `ty`.

**`int SwigType_array_ndim(SwigType *ty)`**

Returns number of array dimensions of `ty`.

**`String* SwigType_array_getdim(SwigType *ty, int n)`**

Returns `n`th array dimension of `ty` .

**`void SwigType_array_setdim(SwigType *ty, int n, const String_or_char *rep)`**

Sets `n`th array dimensions of `ty` to `rep`.

**`void SwigType_add_qualifier(SwigType *ty, const String_or_char *q)`**

Adds a type qualifier `q` to `ty`. `q` is typically `"const"` or `"volatile"`.

**`void SwigType_add_memberpointer(SwigType *ty, const String_or_char *cls)`**

Adds a pointer to a member of class `cls` to `ty`.

**`void SwigType_add_function(SwigType *ty, ParmList *p)`**

Adds a function to `ty`. `p` is a linked-list of parameter nodes as generated by the parser. See the section on parameter lists for details about the representation.

**`void SwigType_add_template(SwigType *ty, ParmList *p)`**

Adds a template to `ty`. `p` is a linked-list of parameter nodes as generated by the parser. See the section on parameter lists for details about the representation.

**`SwigType *SwigType_pop(SwigType *ty)`**

Removes the last type constructor from `ty` and returns it. `ty` is modified.

**`void SwigType_push(SwigType *ty, SwigType *op)`**

Pushes the type operators in `op` onto type `ty`. The opposite of `SwigType_pop()`.

**`SwigType *SwigType_pop_arrays(SwigType *ty)`**

Removes all leading array operators from `ty` and returns them. `ty` is modified. For example, if `ty` is `"a(20).a(10).p.int"`, then this function would return `"a(20).a(10)."` and modify `ty` so that it has the value `"p.int"`.

**`SwigType *SwigType_pop_function(SwigType *ty)`**

Removes a function operator from `ty` including any qualification. `ty` is modified. For example, if `ty` is `"f(int).int"`, then this function would return `"f(int)."` and modify `ty` so that it has the value `"int"` .

**`SwigType *SwigType_base(SwigType *ty)`**

Returns the base type of a type. For example, if `ty` is `"p.a(20).int"`, this function would return `"int"` . `ty` is unmodified.

**`SwigType *SwigType_prefix(SwigType *ty)`**

Returns the prefix of a type. For example, if `ty` is `"p.a(20).int"`, this function would return `"p.a(20)."`. `ty` is unmodified.

### 41.8.3 Type tests

The following functions can be used to test properties of a datatype.

**`int SwigType_ispointer(SwigType *ty)`**

Checks if `ty` is a standard pointer.

**`int SwigType_ismemberpointer(SwigType *ty)`**

Checks if `ty` is a member pointer.

**`int SwigType_isreference(SwigType *ty)`**

Checks if `ty` is a C++ reference.

**`int SwigType_isarray(SwigType *ty)`**

Checks if `ty` is an array.

**`int SwigType_isfunction(SwigType *ty)`**

Checks if `ty` is a function.

**`int SwigType_isqualifier(SwigType *ty)`**

Checks if `ty` is a qualifier.

**`int SwigType_issimple(SwigType *ty)`**

Checks if `ty` is a simple type. No operators applied.

**`int SwigType_isconst(SwigType *ty)`**

Checks if `ty` is a const type.

**`int SwigType_isvarargs(SwigType *ty)`**

Checks if `ty` is a varargs type.

**`int SwigType_istemplate(SwigType *ty)`**

Checks if `ty` is a templatized type.

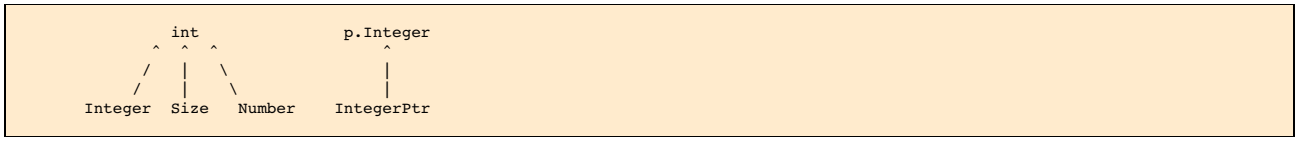
### 41.8.4 Typedef and inheritance

The behavior of `typedef` declaration is to introduce a type alias. For instance, `typedef int Integer` makes the identifier `Integer` an alias for `int`. The treatment of typedef in SWIG is somewhat complicated due to the pattern matching rules that get applied in typemaps and the fact that SWIG prefers to generate wrapper code that closely matches the input to simplify debugging (a user will see the typedef names used in their program instead of the low-level primitive C datatypes).

To handle `typedef`, SWIG builds a collection of trees containing typedef relations. For example,

```
typedef int Integer;
typedef Integer *IntegerPtr;
typedef int Number;
typedef int Size;
```

produces two trees like this:




To resolve a single typedef relationship, the following function is used:

**`SwigType *SwigType_typedef_resolve(SwigType *ty)`**

Checks if `ty` can be reduced to a new type via typedef. If so, returns the new type. If not, returns NULL.

Typedefs are only resolved in simple typenames that appear in a type. For example, the type base name and in function parameters. When resolving types, the process starts in the leaf nodes and moves up the tree towards the root. Here are a few examples that show how it works:

```
Original type            After typedef_resolve()
------------------------ -----------------------
Integer                  int
a(30).Integer            int
p.IntegerPtr             p.p.Integer
p.p.Integer              p.p.int
```

For complicated types, the process can be quite involved. Here is the reduction of a function pointer:

```
p.f(Integer, p.IntegerPtr, Size).Integer         : Start
p.f(Integer, p.IntegerPtr, Size).int
p.f(int, p.IntegerPtr, Size).int
p.f(int, p.p.Integer, Size).int
p.f(int, p.p.int, Size).int
p.f(int, p.p.int, int).int                       : End
```

Two types are equivalent if their full type reductions are the same. The following function will fully reduce a datatype:

**`SwigType *SwigType_typedef_resolve_all(SwigType *ty)`**

Fully reduces `ty` according to typedef rules. Resulting datatype will consist only of primitive typenames.

### 41.8.5 Lvalues

When generating wrapper code, it is necessary to emit datatypes that can be used on the left-hand side of an assignment operator (an lvalue). However, not all C datatypes can be used in this way---especially arrays and const-qualified types. To generate a type that can be used as an lvalue, use the following function:

**`SwigType *SwigType_ltype(SwigType *ty)`**

Converts type `ty` to a type that can be used as an lvalue in assignment. The resulting type is stripped of qualifiers and arrays are converted to a pointers.

The creation of lvalues is fully aware of typedef and other aspects of the type system. Therefore, the creation of an lvalue may result in unexpected results. Here are a few examples:

```
typedef double Matrix4[4][4];
Matrix4 x;    // type = 'Matrix4', ltype='p.a(4).double'

typedef const char * Literal;
Literal y;    // type = 'Literal', ltype='p.char'
```

### 41.8.6 Output functions

The following functions produce strings that are suitable for output.

**`String *SwigType_str(SwigType *ty, const String_or_char *id = 0)`**

Generates a C string for a datatype. `id` is an optional declarator. For example, if `ty` is "p.f(int).int" and `id` is "foo", then this function produces "`int (*foo)(int)`". This function is used to convert string-encoded types back into a form that is valid C syntax.

**`String *SwigType_lstr(SwigType *ty, const String_or_char *id = 0)`**

This is the same as `SwigType_str()` except that the result is generated from the type's lvalue (as generated from SwigType_ltype).

**`String *SwigType_lcaststr(SwigType *ty, const String_or_char *id = 0)`**

Generates a casting operation that converts from type `ty` to its lvalue. `id` is an optional name to include in the cast. For example, if `ty` is "`q(const).p.char`" and `id` is "`foo`", this function produces the string "`(char *) foo`".

**`String *SwigType_rcaststr(SwigType *ty, const String_or_char *id = 0)`**

Generates a casting operation that converts from a type's lvalue to a type equivalent to `ty`. `id` is an optional name to include in the cast. For example, if `ty` is "`q(const).p.char`" and `id` is "`foo`", this function produces the string "`(const char *) foo`".

**`String *SwigType_manglestr(SwigType *ty)`**

Generates a mangled string encoding of type `ty` . The mangled string only contains characters that are part of a valid C identifier. The resulting string is used in various parts of SWIG, but is most commonly associated with type-descriptor objects that appear in wrappers (e.g., `SWIGTYPE_p_double`).

## 41.9 Parameters

Several type-related functions involve parameter lists. These include functions and templates. Parameter list are represented as a list of nodes with the following attributes:

```
"type"        -  Parameter type  (required)
"name"        -  Parameter name  (optional)
"value"       -  Initializer     (optional)
```

Typically parameters are denoted in the source by using a typename of `Parm *` or `ParmList *`. To walk a parameter list, simply use code like this:

```
Parm *parms;
Parm *p;
for (p = parms; p; p = nextSibling(p)) {
  SwigType *type  = Getattr(p, "type");
  String   *name  = Getattr(p, "name");
  String   *value = Getattr(p, "value");
  ...
}
```

Note: this code is exactly the same as what you would use to walk parse tree nodes.

An empty list of parameters is denoted by a NULL pointer.

Since parameter lists are fairly common, the following utility functions are provided to manipulate them:

**`Parm *CopyParm(Parm *p);`**

Copies a single parameter.

**`ParmList *CopyParmList(ParmList *p);`**

Copies an entire list of parameters.

**`int ParmList_len(ParmList *p);`**

Returns the number of parameters in a parameter list.

**`String *ParmList_str(ParmList *p);`**

Converts a parameter list into a C string. For example, produces a string like "`(int *p, int n, double x);`".

**`String *ParmList_protostr(ParmList *p);`**

The same as `ParmList_str()` except that parameter names are not included. Used to emit prototypes.

**`int ParmList_numrequired(ParmList *p);`**

Returns the number of required (non-optional) arguments in `p`.

## 41.10 Writing a Language Module

One of the easiest routes to supporting a new language module is to copy an already supported language module implementation and modify it. Be sure to choose a language that is similar in nature to the new language. All language modules follow a similar structure and this section briefly outlines the steps needed to create a bare-bones language module from scratch. Since the code is relatively easy to read, this section describes the creation of a minimal Python module. You should be able to extrapolate this to other languages.

### 41.10.1 Execution model

Code generation modules are defined by inheriting from the `Language` class, currently defined in the`Source/Modules` directory of SWIG. Starting from the parsing of command line options, all aspects of code generation are controlled by different methods of the `Language` that must be defined by your module.

### 41.10.2 Starting out

To define a new language module, first create a minimal implementation using this example as a guide:

```
#include "swigmod.h"

class PYTHON : public Language {
public:

  virtual void main(int argc, char *argv[]) {
    printf("I'm the Python module.\n");
  }

  virtual int top(Node *n) {
    printf("Generating code.\n");
    return SWIG_OK;
  }

};

extern "C" Language *
swig_python(void) {
  return new PYTHON();
}
```

The "swigmod.h" header file contains, among other things, the declaration of the `Language` base class and so you should include it at the top of your language module's source file. Similarly, the "swigconfig.h" header file contains some other useful definitions that you may need. Note that you should *not* include any header files that are installed with the target language. That is to say, the implementation of the SWIG Python module shouldn't have any dependencies on the Python header files. The wrapper code generated by SWIG will almost always depend on some language-specific C/C++ header files, but SWIG itself does not.

Give your language class a reasonable name, usually the same as the target language. By convention, these class names are all uppercase (e.g. "PYTHON" for the Python language module) but this is not a requirement. This class will ultimately consist of a number of overrides of the virtual functions declared in the `Language` base class, in addition to any language-specific member functions and data you need. For now, just use the dummy implementations shown above.

The language module ends with a factory function, `swig_python()` , that simply returns a new instance of the language class. As shown, it should be declared with the `extern "C"` storage qualifier so that it can be called from C code. It should also return a pointer to the base class (`Language`) so that only the interface (and not the implementation) of your language module is exposed to the rest of SWIG.

Save the code for your language module in a file named " `python.cxx`" and place this file in the`Source/Modules` directory of the SWIG distribution. To ensure that your module is compiled into SWIG along with the other language modules, modify the file `Source/Makefile.am` to include the additional source files. In addition, modify the file `Source/Modules/swigmain.cxx` with an additional command line option that activates the module. Read the source---it's straightforward.

Next, at the top level of the SWIG distribution, re-run the `autogen.sh` script to regenerate the various build files:

```
$ ./autogen.sh
```

Next re-run `configure` to regenerate all of the Makefiles:

```
$ ./configure
```

Finally, rebuild SWIG with your module added:

```
$ make
```

Once it finishes compiling, try running SWIG with the command-line option that activates your module. For example, `swig -python foo.i` . The messages from your new module should appear.

### 41.10.3 Command line options

When SWIG starts, the command line options are passed to your language module. This occurs before any other processing occurs (preprocessing, parsing, etc.). To capture the command line options, simply use code similar to this:

```
void Language::main(int argc, char *argv[]) {
```

```
  for (int i = 1; i < argc; i++) {
    if (argv[i]) {
      if (strcmp(argv[i], "-interface") == 0) {
        if (argv[i+1]) {
          interface = NewString(argv[i+1]);
          Swig_mark_arg(i);
          Swig_mark_arg(i+1);
          i++;
        } else {
          Swig_arg_error();
        }
      } else if (strcmp(argv[i], "-globals") == 0) {
        if (argv[i+1]) {
          global_name = NewString(argv[i+1]);
          Swig_mark_arg(i);
          Swig_mark_arg(i+1);
          i++;
        } else {
          Swig_arg_error();
        }
      } else if ((strcmp(argv[i], "-proxy") == 0)) {
        proxy_flag = 1;
        Swig_mark_arg(i);
      } else if (strcmp(argv[i], "-keyword") == 0) {
        use_kw = 1;
        Swig_mark_arg(i);
      } else if (strcmp(argv[i], "-help") == 0) {
        fputs(usage, stderr);
      }
      ...
    }
  }
}
```

The exact set of options depends on what you want to do in your module. Generally, you would use the options to change code generation modes or to print diagnostic information.

If a module recognizes an option, it should always call `Swig_mark_arg()` to mark the option as valid. If you forget to do this, SWIG will terminate with an unrecognized command line option error.

### 41.10.4 Configuration and preprocessing

In addition to looking at command line options, the `main()` method is responsible for some initial configuration of the SWIG library and preprocessor. To do this, insert some code like this:

```
void main(int argc, char *argv[]) {
  ... command line options ...

  /* Set language-specific subdirectory in SWIG library */
  SWIG_library_directory("python");

  /* Set language-specific preprocessing symbol */
  Preprocessor_define("SWIGPYTHON 1", 0);

  /* Set language-specific configuration file */
  SWIG_config_file("python.swg");

  /* Set typemap language (historical) */
  SWIG_typemap_lang("python");
}
```

The above code does several things--it registers the name of the language module with the core, it supplies some preprocessor macro definitions for use in input files (so that they can determine the target language), and it registers a start-up file. In this case, the file `python.swg` will be parsed before any part of the user-supplied input file.

Before proceeding any further, create a directory for your module in the SWIG library (The `Lib` directory). Now, create a configuration file in the directory. For example, `python.swg`.

Just to review, your language module should now consist of two files-- an implementation file `python.cxx` and a configuration file `python.swg`.

### 41.10.5 Entry point to code generation

SWIG is a multi-pass compiler. Once the `main()` method has been invoked, the language module does not execute again until preprocessing, parsing, and a variety of semantic analysis passes have been performed. When the core is ready to start generating wrappers, it invokes the `top()` method of your language class. The argument to `top` is a single parse tree node that corresponds to the top of the entire parse tree.

To get the code generation process started, the `top()` procedure needs to do several things:

- Initialize the wrapper code output.
- Set the module name.
- Emit common initialization code.
- Emit code for all of the child nodes.
- Finalize the wrapper module and cleanup.

An outline of `top()` might be as follows:

```
int Python::top(Node *n) {

  /* Get the module name */
  String *module = Getattr(n, "name");

  /* Get the output file name */
  String *outfile = Getattr(n, "outfile");

  /* Initialize I/O (see next section) */
  ...

  /* Output module initialization code */
  ...

  /* Emit code for children */
  Language::top(n);
```

```
  ...
  /* Cleanup files */
  ...

  return SWIG_OK;
}
```

### 41.10.6 Module I/O and wrapper skeleton

Within SWIG wrappers, there are five main sections. These are (in order)

- begin: This section is a placeholder for users to put code at the beginning of the C/C++ wrapper file.
- runtime: This section has most of the common SWIG runtime code.
- header: This section holds declarations and inclusions from the .i file.
- wrapper: This section holds all the wrapper code.
- init: This section holds the module initialisation function (the entry point for the interpreter).

Different parts of the SWIG code will fill different sections, then upon completion of the wrappering all the sections will be saved to the wrapper file.

To perform this will require several additions to the code in various places, such as:

```
class PYTHON : public Language {
protected:
  /* General DOH objects used for holding the strings */
  File *f_begin;
  File *f_runtime;
  File *f_header;
  File *f_wrappers;
  File *f_init;

public:
  ...

};

int Python::top(Node *n) {

  ...

  /* Initialize I/O */
  f_begin = NewFile(outfile, "w", SWIG_output_files());
  if (!f_begin) {
    FileErrorDisplay(outfile);
    Exit(EXIT_FAILURE);
  }
  f_runtime = NewString("");
  f_init = NewString("");
  f_header = NewString("");
  f_wrappers = NewString("");

  /* Register file targets with the SWIG file handler */
  Swig_register_filebyname("begin", f_begin);
  Swig_register_filebyname("header", f_header);
  Swig_register_filebyname("wrapper", f_wrappers);
  Swig_register_filebyname("runtime", f_runtime);
  Swig_register_filebyname("init", f_init);

  /* Output module initialization code */
  Swig_banner(f_begin);
  ...

  /* Emit code for children */
  Language::top(n);

  ...
  /* Write all to the file */
  Dump(f_runtime, f_begin);
  Dump(f_header, f_begin);
  Dump(f_wrappers, f_begin);
  Wrapper_pretty_print(f_init, f_begin);

  /* Cleanup files */
  Delete(f_runtime);
  Delete(f_header);
  Delete(f_wrappers);
  Delete(f_init);
  Delete(f_begin);

  return SWIG_OK;
}
```

Using this to process a file will generate a wrapper file, however the wrapper will only consist of the common SWIG code as well as any inline code which was written in the .i file. It does not contain any wrappers for any of the functions or classes.

The code to generate the wrappers are the various member functions, which currently have not been touched. We will look at `functionWrapper()` as this is the most commonly used function. In fact many of the other wrapper routines will call this to do their work.

A simple modification to write some basic details to the wrapper looks like this:

```
int Python::functionWrapper(Node *n) {
  /* Get some useful attributes of this function */
  String   *name   = Getattr(n, "sym:name");
  SwigType *type   = Getattr(n, "type");
  ParmList *parms  = Getattr(n, "parms");
  String   *parmstr= ParmList_str_defaultargs(parms); // to string
  String   *func   = SwigType_str(type, NewStringf("%s(%s)", name, parmstr));
  String   *action = Getattr(n, "wrap:action");

  Printf(f_wrappers, "functionWrapper   : %s\n", func);
```

```
  Printf(f_wrappers, "           action : %s\n", action);
  return SWIG_OK;
}
```

This will now produce some useful information within your wrapper file.

```
functionWrapper   : void delete_Shape(Shape *self)
           action : delete arg1;

functionWrapper   : void Shape_x_set(Shape *self, double x)
           action : if (arg1) (arg1)->x = arg2;

functionWrapper   : double Shape_x_get(Shape *self)
           action : result = (double) ((arg1)->x);

functionWrapper   : void Shape_y_set(Shape *self, double y)
           action : if (arg1) (arg1)->y = arg2;
...
```

### 41.10.7 Low-level code generators

As ingenious as SWIG is, and despite all its capabilities and the power of its parser, the Low-level code generation takes a lot of work to write properly. Mainly because every language insists on its own manner of interfacing to C/C++. To write the code generators you will need a good understanding of how to manually write an interface to your chosen language, so make sure you have your documentation handy.

At this point it is also probably a good idea to take a very simple file (just one function), and try letting SWIG generate wrappers for many different languages. Take a look at all of the wrappers generated, and decide which one looks closest to the language you are trying to wrap. This may help you to decide which code to look at.

In general most language wrappers look a little like this:

```
/* wrapper for TYPE3 some_function(TYPE1, TYPE2); */
RETURN_TYPE _wrap_some_function(ARGS){
  TYPE1 arg1;
  TYPE2 arg2;
  TYPE3 result;

  if(ARG1 is not of TYPE1) goto fail;
  arg1=(convert ARG1);
  if(ARG2 is not of TYPE2) goto fail;
  arg2=(convert ARG2);

  result=some_function(arg1, arg2);

  convert 'result' to whatever the language wants;

  do any tidy up;

  return ALL_OK;

  fail:
  do any tidy up;
  return ERROR;
}
```

Yes, it is rather vague and not very clear. But each language works differently so this will have to do for now.

Tackling this problem will be done in two stages:

- The skeleton: the function wrapper, and call, but without the conversion
- The conversion: converting the arguments to-from what the language wants

The first step will be done in the code, the second will be done in typemaps.

Our first step will be to write the code for `functionWrapper()`. What is shown below is **NOT** the solution, merely a step in the right direction. There are a lot of issues to address.

- Variable length and default parameters
- Typechecking and number of argument checks
- Overloaded functions
- Inout and Output only arguments

```
virtual int functionWrapper(Node *n) {
  /* get useful attributes */
  String   *name   = Getattr(n, "sym:name");
  SwigType *type   = Getattr(n, "type");
  ParmList *parms  = Getattr(n, "parms");
  ...

  /* create the wrapper object */
  Wrapper *wrapper = NewWrapper();

  /* create the wrapper function's name */
  String *wname = Swig_name_wrapper(iname);

  /* deal with overloading */
  ....

  /* write the wrapper function definition */
  Printv(wrapper->def, "RETURN_TYPE ", wname, "(ARGS) {", NIL);

  /* if any additional local variables are needed, add them now */
  ...

  /* write the list of locals/arguments required */
  emit_args(type, parms, wrapper);

  /* check arguments */
  ...
```

```
  /* write typemaps(in) */
  ....

  /* write constraints */
  ....

  /* Emit the function call */
  emit_action(n, wrapper);

  /* return value if necessary  */
  ....

  /* write typemaps(out) */
  ....

  /* add cleanup code */
  ....

  /* Close the function(ok) */
  Printv(wrapper->code, "return ALL_OK;\n", NIL);

  /* add the failure cleanup code */
  ...

  /* Close the function(error) */
  Printv(wrapper->code, "return ERROR;\n", "}\n", NIL);

  /* final substitutions if applicable */
  ...

  /* Dump the function out */
  Wrapper_print(wrapper, f_wrappers);

  /* tidy up */
  Delete(wname);
  DelWrapper(wrapper);

  return SWIG_OK;
}
```

Executing this code will produce wrappers which have our basic skeleton but without the typemaps, there is still work to do.

### 41.10.8 Configuration files

At the time of this writing, SWIG supports nearly twenty languages, which means that for continued sanity in maintaining the configuration files, the language modules need to follow some conventions. These are outlined here along with the admission that, yes it is ok to violate these conventions in minor ways, as long as you know where to apply the proper kludge to keep the overall system regular and running. Engineering is the art of compromise, see...

Much of the maintenance regularity depends on choosing a suitable nickname for your language module (and then using it in a controlled way). Nicknames should be all lower case letters with an optional numeric suffix (no underscores, no dashes, no spaces). Some examples are: `foo`, `bar`, `qux99`.

The numeric suffix variant, as in the last example, is somewhat tricky to work with because sometimes people expect to refer to the language without this number but sometimes that number is extremely relevant (especially when it corresponds to language implementation versions with incompatible interfaces). New language modules that unavoidably require a numeric suffix in their nickname should include that number in all uses, or be prepared to kludge.

The nickname is used in four places:

| **usage** | **transform** |
|---|---|
| "skip" tag | (none) |
| Examples/ subdir name | (none) |
| Examples/test-suite/ subdir name | (none) |

As you can see, most usages are direct.

**configure.ac**
This file is processed by

autoconf to generate the `configure` script. This is where you need to add shell script fragments and autoconf macros to detect the presence of whatever development support your language module requires, typically directories where headers and libraries can be found, and/or utility programs useful for integrating the generated wrapper code.

Use the `AC_ARG_WITH`, `AC_MSG_CHECKING`, `AC_SUBST` macros and so forth (see other languages for examples). Avoid using the `[` and `]` character in shell script fragments. The variable names passed to `AC_SUBST` should begin with the nickname, entirely upcased.

At the end of the new section is the place to put the aforementioned nickname kludges (should they be needed). See Perl5 for examples of what to do. [If this is still unclear after you've read the code, ping me and I'll expand on this further. --ttn]

**Makefile.in**

Some of the variables AC_SUBSTituted are essential to the support of your language module. Fashion these into a shell script "test" clause and assign that to a skip tag using "-z" and "-o":

```
skip-qux99 = [ -z "@QUX99INCLUDE@" -o -z "@QUX99LIBS" ]
```

This means if those vars should ever be empty, qux99 support should be considered absent and so it would be a good idea to skip actions that might rely on it.

Here is where you may also define an alias (but then you'll need to kludge --- don't do this):

```
skip-qux = $(skip-qux99)
```

Lastly, you need to modify each of `check-aliveness`, `check-examples`, `check-test-suite` and `lib-languages` (var). Use the nickname for these, not the alias. Note that you can do this even before you have any tests or examples set up; the Makefile rules do some sanity checking and skip around these kinds of problems.

**Examples/Makefile.in**
Nothing special here; see comments at the top of this file and look to the existing languages for examples.
**Examples/qux99/check.list**
Do `cp ../python/check.list .` and modify to taste. One subdir per line.
**Lib/qux99/extra-install.list**
If you add your language to the top-level Makefile.in var `lib-languages`, then `make install` will install all `*.i` and `*.swg` files from the language-specific subdirectory of`Lib`. Use (optional) file `extra-install.list` in that directory to name additional files to install (see ruby for example).
**Source/Modules/Makefile.am**
Add appropriate files to this Automake file. That's it!

When you have modified these files, please make sure that the new language module is completely ignored if it is not installed and detected on a box, that is, `make check-examples` and `make check-test-suite` politely displays the ignoring language message.

### 41.10.9 Runtime support

Discuss the kinds of functions typically needed for SWIG runtime support (e.g. `SWIG_ConvertPtr()` and `SWIG_NewPointerObj()`) and the names of the SWIG files that implement those functions.

### 41.10.10 Standard library files

The standard library files that most languages supply keeps growing as SWIG matures. The following are the minimum that are usually supported:

- typemaps.i
- std_string.i
- std_vector.i
- stl.i

Please copy these and modify for any new language.

### 41.10.11 User examples

Each of the language modules provides one or more examples. These examples are used to demonstrate different features of the language module to SWIG end-users, but you'll find that they're useful during development and testing of your language module as well. You can use examples from the existing SWIG language modules for inspiration.

Each example is self-contained and consists of (at least) a `Makefile`, a SWIG interface file for the example module, and a 'runme' script that demonstrates the functionality for that module. All of these files are stored in the same subdirectory under the `Examples/[lang]` directory. There are two classic examples which should be the first to convert to a new language module. These are the "simple" C example and the "class" C++ example. These can be found, for example for Python, in `Examples/python/simple` and `Examples/python/class`.

By default, all of the examples are built and run when the user types `make check`. To ensure that your examples are automatically run during this process, see the section on configuration files.

### 41.10.12 Test driven development and the test-suite

A test driven development approach is central to the improvement and development of SWIG. Most modifications to SWIG are accompanied by additional regression tests and checking all tests to ensure that no regressions have been introduced.

The regression testing is carried out by the SWIG *test-suite*. The test-suite consists of numerous testcase interface files in the `Examples/test-suite` directory as well as target language specific runtime tests in the `Examples/test-suite/[lang]` directory. When a testcase is run, it will execute the following steps for each testcase:

1. Execute SWIG passing it the testcase interface file.
2. Compile the resulting generated C/C++ code with either the C or C++ compiler into object files.
3. Link the object files into a dynamic library (dll/shared object).
4. Compile any generated and any runtime test target language code with the target language compiler, if the target language supports compilation. This step thus does not apply to the interpreted languages.
5. Execute a runtime test if one exists.

For example, the *ret_by_value* testcase consists of two components. The first component is the `Examples/test-suite/ret_by_value.i` interface file. The name of the SWIG module **must** always be the name of the testcase, so the`ret_by_value.i` interface file thus begins with:

```
%module ret_by_value
```

The testcase code will then follow the module declaration, usually within a `%inline %{ ... %}` section for the majority of the tests.

The second component is the optional runtime tests. Any runtime tests are named using the following convention: `[testcase]_runme.[ext]`, where `[testcase]` is the testcase name and `[ext]` is the normal extension for the target language file. In this case, the Java and Python target languages implement a runtime test, so their files are respectively, `Examples/test-suite/java/ret_by_value_runme.java` and `Examples/test-suite/python/ret_by_value_runme.py`.

The goal of the test-suite is to test as much as possible in a **silent** manner. This way any SWIG or compiler errors or warnings are easily visible. Should there be any warnings, changes must be made to either fix them (preferably) or suppress them. Compilation or runtime errors result in a testcase failure and will be immediately visible. It is therefore essential that the runtime tests are written in a manner that displays nothing to stdout/stderr on success but error/exception out with an error message on stderr on failure.

#### 41.10.12.1 Running the test-suite

In order for the test-suite to work for a particular target language, the language must be correctly detected and configured during the configure stage so that the correct Makefiles are generated. Most development occurs on Linux, so usually it is a matter of installing the development packages for the target language and simply configuring as outlined earlier.

If when running the test-suite commands that follow, you get a message that the test was skipped, it indicates that the configure stage is missing information in order to compile and run everything for that language.

The test-suite can be run in a number of ways. The first group of commands are for running multiple testcases in one run and should be executed in the top level directory. To run the entire test-suite (can take a long time):

```
make -k check-test-suite
```

To run the test-suite just for target language [lang], replace [lang] with one of csharp, java, perl5, python, ruby, tcl etc:

```
make check-[lang]-test-suite
```

Note that if a runtime test is available, a message "(with run test)" is displayed when run. For example:

```
$ make check-python-test-suite
checking python test-suite
checking python testcase argcargvtest (with run test)
checking python testcase python_autodoc
checking python testcase python_append (with run test)
checking python testcase callback (with run test)
```

The files generated on a previous run can be deleted using the clean targets, either the whole test-suite or for a particular language:

```
make clean-test-suite
make clean-[lang]-test-suite
```

The test-suite can be run in a *partialcheck* mode where just SWIG is executed, that is, the compile, link and running of the testcases is not performed. Note that the partialcheck does not require the target language to be correctly configured and detected and unlike the other test-suite make targets, is never skipped. Once again, either all the languages can be executed or just a chosen language:

```
make partialcheck-test-suite
make partialcheck-[lang]-test-suite
```

If your computer has more than one CPU, you are strongly advised to use parallel make to speed up the execution speed. This can be done with any of the make targets that execute more than one testcase. For example, a dual core processor can efficiently use 2 parallel jobs:

```
make -j2 check-test-suite
make -j2 check-python-test-suite
make -j2 partialcheck-java-test-suite
```

The second group of commands are for running individual testcases and should be executed in the appropriate target language directory, `Examples/test-suite/[lang]`. Testcases can contain either C or C++ code and when one is written, a decision must be made as to which of these input languages is to be used. Replace `[testcase]` in the commands below with the name of the testcase.

For a C language testcase, add the testcase under the C_TEST_CASES list in `Examples/test-suite/common.mk` and execute individually as:

```
make -s [testcase].ctest
```

For a C++ language testcase, add the testcase under the CPP_TEST_CASES list in `Examples/test-suite/common.mk` and execute individually as:

```
make -s [testcase].cpptest
```

A third category of tests are C++ language testcases testing multiple modules (the %import directive). These require more than one shared library (dll/shared object) to be built and so are separated out from the normal C++ testcases. Add the testcase under the MULTI_CPP_TEST_CASES list in `Examples/test-suite/common.mk` and execute individually as:

```
make -s [testcase].multicpptest
```

To delete the generated files, execute:

```
make -s [testcase].clean
```

If you would like to see the exact commands being executed, drop the -s option:

```
make [testcase].ctest
make [testcase].cpptest
make [testcase].multicpptest
```

Some real examples of each:

```
make -s ret_by_value.clean
make -s ret_by_value.ctest
make -s bools.cpptest
make -s imports.multicpptest
```

Advanced usage of the test-suite facilitates running tools on some of the five stages. The make variables `SWIGTOOL` and `RUNTOOL` are used to specify a tool to respectively, invoke SWIG and the execution of the runtime test. You are advised to view the `Examples/test-suite/common.mk` file for details but for a short summary, the classic usage is to use Valgrind for memory checking. For example, checking for memory leaks when running the runtime test in the target language interpreter:

```
make ret_by_value.ctest RUNTOOL="valgrind --leak-check=full"
```

This will probably make more sense if you look at the output of the above as it will show the exact commands being executed. SWIG can be analyzed for bad memory by first rebuilding swig with just the `-g` option. Also define DOH_DEBUG_MEMORY_POOLS, see section. SWIG can then be invoked via valgrind using:

```
make ret_by_value.ctest SWIGTOOL="valgrind --tool=memcheck --trace-children=yes"
```

A debugger can also be invoked easily on an individual test, for example gdb:

```
make ret_by_value.ctest RUNTOOL="gdb --args"
```

SWIG reads the `SWIG_FEATURES` environment variable to obtain options in addition to those passed on the command line. This is particularly useful as the entire test-suite or a particular testcase can be run customized by using additional arguments, for example the -O optimization flag can be added in, as shown below for the bash shell:

```
env SWIG_FEATURES=-O make check-python-test-suite
```

The syntax for setting environment variables varies from one shell to the next, but it also works as shown in the example below, where some typemap debugging is added in:

```
make ret_by_value.ctest SWIG_FEATURES="-debug-tmsearch"
```

There is also a special 'errors' test-suite which is a set of regression tests checking SWIG warning and error messages. It can be run in the same way as the other language test-suites, replacing [lang] with errors, such as `make check-errors-test-suite`. The test cases used and the way it works is described in `Examples/test-suite/errors/Makefile.in`.

### 41.10.13 Documentation

Don't forget to write end-user documentation for your language module. Currently, each language module has a dedicated chapter You shouldn't rehash things that are already covered in sufficient detail in the SWIG Basics and SWIG and C++ chapters. There is no fixed format for *what*, exactly, you should document about your language module, but you'll obviously want to

cover issues that are unique to your language.

Some topics that you'll want to be sure to address include:

- Command line options unique to your language module.
- Non-obvious mappings between C/C++ and target language concepts. For example, if your target language provides a single floating point type, it should be no big surprise to find that C/C++ `float` and `double` types are mapped to it. On the other hand, if your target language doesn't provide support for "classes" or something similar, you'd want to discuss how C++ classes are handled.
- How to compile the SWIG-generated wrapper code into shared libraries that can actually be used. For some languages, there are well-defined procedures for doing this, but for others it's an ad hoc process. Provide as much detail as appropriate, and links to other resources if available.

### 41.10.14 Coding style guidelines

The coding guidelines for the C/C++ source code are pretty much K&R C style. The style can be inferred from the existing code base and is largely dictated by the `indent` code beautifier tool set to K&R style. The code can formatted using the make targets in the Source directory. Below is an example of how to format the emit.cxx file:

```
$ cd Source
$ make beautify-file INDENTFILE=Modules/emit.cxx
```

Of particular note is indentation is set to 2 spaces and a tab is used instead of 8 spaces. The generated C/C++ code should also follow this style as close as possible. However, tabs should be avoided as unlike the SWIG developers, users will never have consistent tab settings.

### 41.10.15 Target language status

Target languages are given a status of either 'Supported', 'Experimental' or 'Deprecated' depending on their maturity as broadly outlined in the Target language introduction. This section provides more details on how this status is given.

#### 41.10.15.1 Supported status

A target language is given the 'Supported' status when

- It is in a mature, well functioning state.
- It has its own comprehensive chapter in the documentation. The level of documentation should be comprehensive and match the standard of the other mature modules. Python and Java are good references.
- It passes all of the main SWIG test-suite. The main test-suite is defined by the tests in the C_TEST_CASES, CPP_TEST_CASES and MULTI_CPP_TEST_CASES lists in Examples/test-suite/common.mk. All the newer C++ standard tests need to work and are grouped together, such as CPP11_TEST_CASES for C++11. These more 'modern' C++ standards are only tested though if the compiler is detected as supporting the given standard.
- The test-suite must also include at least twenty wide-ranging runtime tests. The most mature languages have a few hundred runtime tests. Note that porting runtime tests from another language module is a quick and easy way to achieve this.
- It supports the vast majority of SWIG features. Some more advanced features, such as, directors, full nested class support and target language namespaces (nspace) may be unimplemented. A few support libraries may be missing, for example, a small number of STL libraries.
- It provides strong backwards compatibility between releases. Each point release must aim to be fully backwards compatible. A point release version is the 3rd version digit, so each of the x.y.* versions should be backwards compatible. Backwards compatibility breakages can occur in a new major or minor version if absolutely necessary and if documented. A major or minor version is the first or second digit in the three digit version.
- Fixing unintended regressions in the Supported languages will be given higher priority over the Experimental languages by the core SWIG developers.
- Examples must be available and run successfully.
- The examples and test-suite must be fully functioning on the Github Actions Continuous Integration platform.

#### 41.10.15.2 Experimental status

A target language is given the 'Experimental' status when

- It is of sub-standard quality, failing to meet the above 'Supported' status.
- It is somewhere between the mid to mature stage of development.
- It is in need of help to finish development.

Some minimum requirements and notes about languages with the 'Experimental' status:

- Will at least implement basic functionality - support wrapping C functions and simple C++ classes and templates.
- Have its own documentation chapter containing a reasonable level of detail. The documentation must provide enough basic functionality for a user to get started.
- Have fully functional examples of basic functionality (the simple and class examples).
- The test-suite must be implemented and include a few runtime tests for both C and C++ test cases.
- Failing tests must be put into one of the FAILING_CPP_TESTS or FAILING_C_TESTS lists in the test-suite. This will ensure the test-suite can be superficially made to pass by ignoring failing tests. The number of tests in these lists should be no greater than half of the number of tests in the full test-suite.
- The examples and test-suite must also be fully functioning on the Github Actions Continuous Integration platform. However, experimental languages will be flagged as 'continue-on-error'. This means that pull requests and normal development commits will not break the entire Github Actions build should an experimental language fail.
- Any new failed tests will be fixed on a 'best effort' basis by core developers with no promises made.
- If a language module has an official maintainer, then the maintainer will be requested to focus on fixing test-suite regressions and commit to migrating the module to become a 'Supported' module.
- If a module does not have an official maintainer, then, as maintenance will be on a 'best efforts' basis by the core maintainers, no guarantees will be provided from one release to the next and regressions may creep in.
- Experimental target languages will have a (suppressible) warning explaining the Experimental sub-standard status and encourage users to help improve it.
- No backwards compatibility is guaranteed as the module is effectively 'in development'. If a language module has an official maintainer, then a backwards compatibility guarantee may be provided at the maintainer's discretion and should be documented as such.

#### 41.10.15.3 Deprecated status

Unfortunately target languages that once met 'Experimental' or 'Supported' status can become non-functional and simply bit rot due to neglect or due to the language's C/C++ API evolving and changing over time. If there is no language maintainer and the maintenance burden becomes too much for the core SWIG developers to keep the test-suite working with easily available versions of the target language, the language is put into the 'Deprecated' status.

Changing the status to 'Deprecated' is an unfortunate step not done lightly and only if recent versions cannot successfully run the test-suite and examples on the Github Actions Continuous Integration platform. As the target language would once have had a working set of examples and test-suite, this usually only occurs when the more modern operating system distributions available on Github can no longer run any distribution supplied version of the target language.

Changing status to 'Deprecated' flags it for removal from SWIG in a subsequent release (usually one SWIG release). This step becomes the final plea for help from the community who use the target language. The language will need updating by an interested community member to meet the requirements of at least 'Experimental' status in order to prevent removal. If you are a user of a 'Deprecated' target language and would like to keep it available in future releases, please contact the SWIG developers for details of how you can help.

### 41.10.16 Prerequisites for adding a new language module to the SWIG distribution

New target language modules can be included in SWIG and contributions are encouraged for popular languages. In order to be considered for inclusion, a language must at a minimum fit the 'Experimental' status described above.

Below are some practical steps that should help meet these requirements.

1. The "simple" example needs to be working to demonstrate basic C code wrappers. Port the example from another language, such as from `Examples/python/simple`.
2. The "class" example needs to be working to demonstrate basic C++ code wrappers. Port the example from another language, such as from `Examples/python/class`.
3. Modify `configure.ac`, `Makefile.in` and `Examples/Makefile.in` to run these examples. Please make sure that if the new language is not installed properly on a box, `make -k check` should still work by skipping the tests and examples for the new language module.
4. Copying an existing language module and adapting the source for it is likely to be the most efficient approach to fully developing a new module as a number of corner cases are covered in the existing implementations. The most advanced scripting languages are Python and Ruby. The most advanced compiled target languages are Java and C#.

5. Get the test-suite running for the new language (`make check-[lang]-test-suite`). While the test-suite tests many corner cases, we'd expect the majority of it to work without much effort once the generated code is compiling correctly for basic functionality as most of the corner cases are covered in the SWIG core. Aim to first get one C and one C++ runtime test running in the test-suite. Adding further runtime tests should be a lot easier afterwards by porting existing runtime tests from another language module.
6. The structure and contents of the html documentation chapter can be copied and adapted from one of the other language modules.
7. Source code can be formatted correctly using the info in the coding style guidelines section.
8. When ready, post a patch on Github, join the swig-devel mailing list and email the SWIG developers with a demonstration of commitment to maintaining the language module, certainly in the short term and ideally long term.

Once accepted into the official Git repository, development efforts should concentrate on getting the entire test-suite to work in order to migrate the language module to the 'Supported' status. Runtime tests should be added for existing testcases and new test cases can be added should there be an area not already covered by the existing tests.

## 41.11 Debugging Options

There are various command line options which can aid debugging a SWIG interface as well as debugging the development of a language module. These are as follows:

```
-debug-classes    - Display information about the classes found in the interface
-debug-module <n> - Display module parse tree at stages 1-4, <n> is a csv list of stages
-debug-symtabs    - Display symbol tables information
-debug-symbols    - Display target language symbols in the symbol tables
-debug-csymbols   - Display C symbols in the symbol tables
-debug-lsymbols   - Display target language layer symbols
-debug-quiet      - Display less parse tree node debug info when using other -debug options
-debug-tags       - Display information about the tags found in the interface
-debug-template   - Display information for debugging templates
-debug-top <n>    - Display entire parse tree at stages 1-4, <n> is a csv list of stages
-debug-typedef    - Display information about the types and typedefs in the interface
-debug-typemap    - Display information for debugging typemaps
-debug-tmsearch   - Display typemap search debugging information
-debug-tmused     - Display typemaps used debugging information
```

The complete list of command line options for SWIG are available by running `swig -help`.

## 41.12 Guide to parse tree nodes

This section describes the different parse tree nodes and their attributes.

**cdecl**

Describes general C declarations including variables, functions, and typedefs. A declaration is parsed as "storage T D" where storage is a storage class, T is a base type, and D is a declarator.

```
"name"          - Declarator name
"type"          - Base type T
"decl"          - Declarator type (abstract)
"storage"       - Storage class (static, extern, typedef, etc.)
"parms"         - Function parameters (if a function)
"code"          - Function body code (if supplied)
"value"         - Default value (if supplied)
```

**constructor**

C++ constructor declaration.

```
"name"          - Name of constructor
"parms"         - Parameters
"decl"          - Declarator (function with parameters)
"code"          - Function body code (if any)
"feature:new"   - Set to indicate return of new object.
```

**destructor**

C++ destructor declaration.

```
"name"          - Name of destructor
"code"          - Function body code (if any)
"storage"       - Storage class (set if virtual)
"value"         - Default value (set if pure virtual).
```

**access**

C++ access change.

```
"kind"          - public, protected, private
```

**constant**

Constant created by %constant or #define.

```
"name"          - Name of constant.
"type"          - Base type.
"value"         - Value.
"storage"       - Set to %constant
"feature:immutable" - Set to indicate read-only
```

**class**

C++ class definition or C structure definition.

```
"name"          - Name of the class.
"kind"          - Class kind ("struct", "union", "class")
"symtab"        - Enclosing symbol table.
```

```
"tdname"        - Typedef name. Use for typedef struct { ... } A.
"abstract"      - Set if class has pure virtual methods.
"baselist"      - List of base class names.
"storage"       - Storage class (if any)
"unnamed"       - Set if class is unnamed.
```

**enum**

Enumeration.

```
"name"          - Name of the enum (if supplied).
"storage"       - Storage class (if any)
"tdname"        - Typedef name (typedef enum { ... } name).
"unnamed"       - Set if enum is unnamed.
```

**enumitem**

Enumeration value.

```
"name"          - Name of the enum value.
"type"          - Type (integer or char)
"value"         - Enum value (if given)
"feature:immutable" - Set to indicate read-only
```

**namespace**

C++ namespace.

```
"name"          - Name of the namespace.
"symtab"        - Symbol table for enclosed scope.
"unnamed"       - Set if unnamed namespace
"alias"         - Alias name. Set for namespace A = B;
```

**using**

C++ using directive.

```
"name"          - Name of the object being referred to.
"uname"         - Qualified name actually given to using.
"node"          - Node being referenced.
"namespace"     - Namespace name being reference (using namespace name)
```

**classforward**

A forward C++ class declaration.

```
"name"          - Name of the class.
"kind"          - Class kind ("union", "struct", "class")
```

**insert**

Code insertion directive. For example, %{ ... %} or %insert(section).

```
"code"          - Inserted code
"section"       - Section name ("header", "wrapper", etc.)
```

**top**

Top of the parse tree.

```
"module"        - Module name
```

**extend**

%extend directive.

```
"name"          - Module name
"symtab"        - Symbol table of enclosed scope.
```

**apply**

%apply pattern { patternlist }.

```
"pattern"       - Source pattern.
"symtab"        - Symbol table of enclosed scope.
```

**clear**

%clear patternlist;

```
"firstChild"    - Patterns to clear
```

**include**

%include directive.

```
"name"         - Filename
"firstChild"   - Children
```

**import**

%import directive.

```
"name"         - Filename
"firstChild"   - Children
```

**module**

%module directive.

```
"name"         - Name of the module
```

**typemap**

%typemap directive.

```
"method"       - Typemap method name.
"code"         - Typemap code.
"kwargs"       - Keyword arguments (if any)
"firstChild"   - Typemap patterns
```

**typemapcopy**

%typemap directive with copy.

```
"method"       - Typemap method name.
"pattern"      - Typemap source pattern.
"firstChild"   - Typemap patterns
```

**typemapitem**

%typemap pattern. Used with %apply, %clear, %typemap.

```
"pattern"      - Typemap pattern (a parameter list)
"parms"        - Typemap parameters.
```

**types**

%types directive.

```
"parms"        - List of parameter types.
"convcode"     - Code which replaces the default casting / conversion code
```

**extern**

extern "X" { ... } declaration.

```
"name"       - Name "C", "Fortran", etc.
```

## 41.13 Further Development Information

There is further documentation available on the internals of SWIG, API documentation and debugging information. This is shipped with SWIG in the `Doc/Devel` directory.